



前言

本书书名为《Java 特种兵》，又名《Java 老 A》，目的很简单，希望作为读者朋友的你，有一天能够成为一个单兵作战能力极度强悍的 Java 程序员。众所周知，Java 程序员多如牛毛，我希望学习本书的读者有志者事竟成、出类拔萃，练就一身好本领，成为 Java 界的特种兵。



冰冻三尺，非一日之寒。本书不是什么神功秘籍，所以它无法让你在短时间内从一个小鸟变成一个牛人，更无法让你从牛人成为大师，这本书是希望读者朋友能够在纠结的时候想起还可依赖它。因为本书不仅仅在讲解一些知识，同时也包含了作者以及作者同事的一些工作经历，以及工作经历中的那些痛苦与纠结，我们希望在这本书的陪伴下，你在纠结之时选择的不是放弃自我，而是坚持到胜利的那一刻。

作者本人也不是什么牛人，更不是绝顶高手，只能说我们经历的某些事情也许值得去分享，这些内容不仅仅包含一些知识，同时也包含了很多思想、方法和面对问题的态度，注重功底和修为的提升。如果有一天你发现这本书带给你更多的是方法，从而让你建立起自己的许多思路，能够快速学习和成长，那么本书的一个重要目的就达到了；如果有一天你发现再去看某些“牛人秘籍”会比以前轻松和愉悦，那么也许这本书有点小功劳。

本书不是什么代码大全，也不是专门讲解某些指定的知识板块的书籍（例如 JVM），并不会针对某些专门的知识板块去做过细的说明，也不会去说明详细的 API，更不会全面地说明它们的源码，这是没有意义的。通过本书，希望你能从内心认识到“知识无涯”，我们总会遇到自己没有见过的问题，需要学会的就是如何面对这些问题，即如何去思考和定位，如何去学习和成长。

在本书中，可能会以各种“换位思考”的方式来探讨一些技术内容，从技术本身的角度来讲，侧重于讲解技术之中蕴藏的各种奥秘以及探索奥秘的一些思路，而不是讲解一些技术的使用，希望读者在这个过程中开始有意识地去理解知识应有深浅之分，并同时真正掌握 Java 基础，提升自我的功底，同时还能找到提升功底的方法。

本书中许多对话和探讨的模式，适合像小说一样来静心品味，我相信如果你能静心，那么不论是初学者还是有一定功底的人，都能从本书中受益。作者本人在刚进入大学的时候，是一个连计算机是什么都不知道的人，也曾经有过为了通过 C 语言考试而死记硬背代码的经历，后来通过不断努力，才有能力写出这本书。因此，本书会结合当时胖哥从一个计算机界的“文盲”开始成长的过程中所拥有的一些“可爱”的思想经历，如果你有类似的思想经历，希望这本书能够帮助你，也同时希望你理解到：只要你愿意，你就一定行的！

本书有的内容风格会偏于娱乐，这些内容所描述的技术肯定不会那么严谨，由于不严谨，可能会让不同的读者产生仁者见仁、智者见智的情况。对于某些有着同样工作经历的人看来，也许会有着发自内心的共鸣，对于初学者可以是一个大概的形象理解。虽然书中的许多解释并不是技术点上的准确解释，但我不希望初学者“小伙伴们”仅仅注重娱乐本

身的话题，而要想想这种娱乐的例子所带给自己的一些感性认识。

本书会谈谈到一些成长性的内容，因此我们可以先大概定位一下一些不同层次的人会怎样思考，怎样做事情，让大家有个形象上的认识。



许多初学者，就像是无头苍蝇，四处乱撞，什么都想学，学了就丢，而且喜欢钻牛角尖，甚至没有独立思考的习惯，遇到问题总是先问，再解决问题。他们经常会问：“为什么我的东西在这好用，在那不好用？”因为你根本不知道问题的原因，当然不知道这么多的为什么了，任何问题都需要定位分析，再解决，再总结。面对问题要静下心来通过自己对基础的认识来分析可能的原因，然后逐步缩小范围，再定位问题。如果抛出一个错误就能知道原因，那么直接设计一个答题的机器就可以了，而不需要任何人为的介入。

学到一定程度的人，懂得思考，懂得换位思考，能解决大多实际问题，他们不仅仅能开始自己定位问题，而且开始具有“前瞻性”，思考一些问题未来可能存在的潜在情况，也开始逐步关注一些内在原理。不过，可能是艺高人胆大，这些人有可能会犯一些不该犯的错误，他们通常是一些十分优秀的程序设计者，很多时候可能是为了展现自己的技术能力，会将一些不值得去做的事情不断细化和优化，或者去做一些别人做过的事情，如果因为这样丢掉主营业务，则通常说它是过度优化、过度设计、重复制造。作者本身也有同样的亲身经历，而矛盾在于是否真的是过度设计，在不同的场景下会有不同的看法和评价，因此这个阶段的同学，通常会有一个纠结期。在这个阶段的人开始讲究内外双修，也同样是一个积土成山、积水成渊的过程，因为有一定的功底，学习很多知识都很快，在知识面上会有很大的突破，在知识点上也会逐步加深，可能在工作中不论是业务还是技术上都有了很多的选择，被认为是工作能力很强的人。一些很“牛”的开发人员就会开始意识到很多代码如何去编写会更优雅、更加高效，用更简单的代码搞定更多的问题，但还未上升到系统级别和架构级别。

可能更高级别的人开始具有很强的全局观，让系统全局更加具有可维护性和伸缩性，系统性能良好，稳定性十足，能搞定各类架构、部署、规范化等相关的问题，在某些领域上也是说话很有分量的人物。他们可能会花很多时间去写通用模型和设计，将难题简单化，解决大家解决不了的难题。通常会尽量让系统的维护变成一条配置或是几条简单代码，但是可以顶得上一群人写一堆代码，这就是特种兵程序员的“快”、“准”、“狠”、“稳”。

这里的各种级别仅仅是作者“自言自语”，并不代表什么，要做到没有级别在心中才能真正让自己快速成长。另外，看了这些级别并不代表今天我是初学者，明天将不好的习惯改掉后就“脱胎换骨”了，也并不代表自己在某件小事情上有了大师的风范就代表自己是大师了。这些东西都是经历一些事情后才会形成一些自然意识，我们只是有意识地去学习他们为什么会这样做事情，为什么年轻人比老员工有活力，但通常来讲“姜还是老的辣”。



□ 适合对象

- (1) 有一定 Java 基础，并希望能在 Java 技术上有所成长的人。

本书学习有一定门槛，如果是初学者，则可以先以书中的一些例子为引导，希望了解为什么的时候，再来看其中的解释。

(2) 对于能静心看书的初学者，可以迭代着看本书。

虽然有一定门槛，不过初学者只要能静心地迭代着读几次后，你或许也能体会出里面有很多思想、方法、解决问题的手段、看问题的侧面，甚至面对问题的心态，那么也可以受益良多。

(3) 本书是一个载体，一个推崇自我修为、内外双修的载体，修行看个人，所以适合于想要以自我提升为主要目的的读者。

(4) 工作一段时间，对知识和发展的方向很迷茫，甚至对某些观念也比较迷茫，但是又渴望去解决这些问题，渴望自己成长，渴望自己能找到道路的人。

□ 不适合对象

(1) Java 方面的高手和牛人，因为这本书并不能助你从牛人变成大师。

(2) 做客户端（如 Android）或嵌入式的 Java 程序员，因为书中大多内容是以服务器端 Java 来讲解的，极少考虑客户端的问题。

(3) 如果你对自己或周围的一切很绝望，看到的所有东西都是邪恶、虚伪的，也没有想过要让自己进步。

(4) 希望这本书成为技术字典的同学不适合读，因为这本书不是代码大全，也不是知识点精准解释的汇聚，更不是 API 的列表。

(5) 希望系统性学习某些专业知识的同学不适合看本书，因为本书讲解的是一种“碎片化”学习方式，或者说是以一些小例子为基础，进一步处理相关问题的方式。

如何阅读本书



作为书籍的作者，希望读者能够快乐地学到知识，每天空余时间看看而不是一种压力，面对技术拥有独自思考问题的能力，而并非掌握某种技术本身，通过轻松地接触很多知识并且能有所体会，不断地去总结、抽象，能得出自己的分析问题、解决问题的方法，磨炼出超越普通程序员的功底，最终成为 Java 老 A。

本书其实是一本 Java 的野书、杂谈，会尽量用通俗易懂的方式来讲解一些复杂的问题和技术。不过，这毕竟不是一本娱乐的书，某些内容胖哥还是会说得比较严谨。书中除了讲解一些“功底类”的技术，大部分技术不会讲得太深，大多只是抛砖引玉给你一个思路，当你需要去接触更深的技术的时候，在这本书中可能会有一个较为形象的例子供你参考，也许就大概知道了如何去理解。若功底够好，就会发现万变不离其宗，一切都可以归于基础，而且大多复杂的思路都源自于生活的灵感。

如果是初学者，希望你不要期望每天看太多内容，而是希望你看了后尽量去思考，不用完全看懂，每天能够有一个心得，当你在实践中遇到某些问题开始拥有灵感时，本书或许会印证这一切。

本书提到的技术仅仅是一个引导、一种探讨、一种思路，在阐述一件事情好与不好的时候，通常会以多种角度去看待问题的不同侧面，再来说明为什么会有这样的经验，同时也可能会探讨这些经验是否能够受用终身。因为技术的时代在不断革命，现在的技术十分

泛滥，但都殊途同归，我们要跟上时代又要归纳总结，也许很多时候我们没有精力去学习所有的知识，但有精力让自己提升学习能力从而来学习更多的东西，也有精力来探索工作领域内的知识奥秘，而前提是应当有探索奥秘的习惯以及足够的功底。

本书的目的在于个人成长，极少谈到关于团队合作与规范化等方面的知识，当然作为一个单兵作战能力极强的人，应当有这方面的意识和思维，即使本书中没有提及，大家也应当去掌握。同时也请读者朋友注意本书所提倡的单兵作战能力，并不是期望大家去做一个技术上的孤独者，而是为了更好地去工作。



- 本书将采用 Hotspot VM 1.6 作为讲解的例子(可通过命令“`java -version`”得到 JVM 版本)，而测试的例子如果是在具体的 OS 环境下测试，则会单独说明。书中也会提到 JDK 1.7 的内容，相应的代码如果要运行，也要选择 JDK 1.7。
- 本书中“部分例子”，必须采用-server 模式来运行和模拟，尤其是第 5 章并发编程中的许多例子，在例子中通常会有专门的备注和说明，请读者注意参看。
- 篇幅所限，本书中的例子大多只是片段，以说明实际的问题为主，需要完整的例子，请参看配书光盘中的相关 demo 信息，书中的例子都有与之对应的实际代码，除十分复杂的代码外，从 demo 中获取的代码进行相关的配置后，均可直接在对应平台上按照指定结果运行。
- 本书的代码分源码和代码两种，源码为第三方源码，可以到官方网站下载参看，书中会介绍源码的版本和思路，并贴出部分关键源码的功能和设计思路。
- 本书从第 2 篇开始，每一篇都会有“致读者言”，目的是让读者更好地了解这部分内容应该从何种角度去理解，它会说些什么，会给你带来什么，你为什么需要这些。
- 网络上大家都称作者为“小胖哥”或“胖哥”，所以在本书中，也会以这两个称呼作为作者本人的代名词。



本书分上、下册，总共 5 篇，上册两篇内容分别是 Java 功底篇和源码篇，强调个人修为的提升，也是本书主旨所在，希望能帮助各位读者朋友提升“功力”；下册中，将基于上册的内容融入设计、实现的细节。

上册

第 1 篇 Java 功底篇

建议所有本书的读者朋友都读一下本篇的内容，所谓“练武不练功，到老一场空”，你若想要成为一个老 A 级程序员，功底的提升是十分重要的。通过对第 1 篇的学习，希望读者能知道如何验证自己的功底，如何提升自己的功底，而能提升多少，完全要靠你自己的属性哦！

第 1 章从简单的角度来验证功底，通过一些简单的例子来说明我们应当如何去掌握 Java

的基础，同时也包含一个“老 A 级程序员”除功底外还需要有面对逆境的心态。

第 2 章和第 3 章分别介绍关于计算机的工作原理和 Java 虚拟机的基础知识。胖哥认为一个优秀的 Java 程序员，应当知道自己的程序在计算机内部是如何运行的，更需要知道虚拟机是如何运行的，效率对比怎样，此好比“习武之人，需要知道何为武学”。

第 4 章讲解 Java 通信，在了解了运行基本原理后，你应当了解很多网络交互已经被 Java 的框架包装得不再像通信程序了，就像是“被装修后的房子，你不再知道房屋本身是用什么建造的”。但如果你是老 A，就应当知道这些，这样才能知道遇到各种怪异的问题时如何去解决，你会发现：“一切源自基础，偶然问题隐藏必然”。

第 5 章讲解 Java 并发，如果读者朋友确实遇不到，则可以跳过，但要成为 Java 的老 A，这应该算是基础知识，老 A 应该知道 Java 并发是怎么样。除了客户端程序及单片机等，Java 并发程序处理其实无处不在，就好比“一个城市的交通，很多的车辆需要通信，应当如何调度来提高流量，避免交通事故”。

第 6 章讲解数据库知识，大部分程序员都应该使用过数据库、文件，通过学习第 4 章通信方面的内容，应该会有一些概念，但是面对存储，你是否想要了解一些内在，了解一个程序员所需要知道的内在，这样可能会使得你的工作更加得心应手。说了这么多，你是否发觉学习 Java 为何要了解如此多的东西？是的，你需要知道“多个门派的武学，面对不同的思想碰撞来切磋，才会有更多的体会”。作为一个老 A，才会拥有单兵作战能力极强的作战素质。

第 2 篇 源码篇

达到一定程度的程序员一般会开始对实现本身感兴趣，想了解为什么，了解别人是如何设计和实现的。

起初在面对框架的时候，可能会去猜测或测试它是如何实现的，但逐渐发现猜测和测试并不那么靠谱，因为版本和环境影响因素太大，我们开始想要知道一些内在，看源码成为一个必然。源码面前没有秘密可言，看过源码，你会对问题定位和编写代码拥有自信，开始对技术本身有量化的认识。

本篇并不是网罗天下源码，也不会对某种源码做全面讲解，而是通过阅读源码来说明我们应当如何阅读源码。

在本篇开始部分会先介绍为何要阅读源码，讲解哪些源码，如何阅读源码，请有疑问的小伙伴们关注一下哦。

第 7 章讲解源码基础，说明 Java 常见的框架基础知识，主要包括：反射、AOP、ORM、Annotation 和配置文件的原理。在这里不仅仅会阐述技术本身，同时也会联系一些实际框架的可能实现方法。老 A 一定要：知其然，知其所以然。

第 8 章和第 9 章分别讲解 JDBC、Spring 的源码，这里不会说明这些三方包的所有源码和关键点。但通过几种不同类型的框架源码，希望读者能体会源码之中的“思维方式、设计、架构”，以及了解到不同源码的区别所在。虽然区别很多，但如果你有一些“百川纳海、源自同门”的感受，那么小伙伴们就成长了很多。

第 10 章是对第 2 篇的知识总结，在阅读源码后不是读完就完了，而是应当有所体会。通过对源码基础知识的初步了解和对一些源码的阅读，我们逐步有能力在接触一个自己从未见到的框架时，可以大概知道它的大致内在，并且大多数情况下可以轻松搞定它，即使有一定难度也会从根本的角度去看待这些技术问题，进一步快速地通过看它的代码

了解内在，而不会被浮于表面的使用所迷惑。这就好比虽然魔术师的表演眼花缭乱，但是在他们同行眼中都是知根见底的，因为我们也是专业的，所以不应当被一些东西所迷惑。

下册

第 1 篇 设计篇

老 A 虽然不像军师那样：“运筹帷幄之中，决胜千里之外”，但也同样需要有全局的部署思想，对临场应变以及撤退方案和路径要有全局把控，能“在万军之中取上将首级”，使用最低牺牲得到最高的成果。

在设计篇中，胖哥希望给你带来的不仅仅是现有老 A 的技术，也包含了他们的许多思想。既然是思想，自然是结合场景最佳，它需要融入老 A 的智慧，不限于表达的形式，因为这些形式只是一种思想传承的载体。

在第 1 章中，胖哥会用平时工作中的例子说明一些常用的设计模式，但并不会将 Java 所有的设计模式全部详细讲解，因为这不是本书的主旨，而且那样并没有多大意义。在实际的应用中，场景是非常重要的，变化远远大于这些设计模式本身所提供的“招式”。模式本身提供一种建议性的设计方法，就好比武学中的“招式是死的，人是活的”，不同年龄段修炼同样的武学，与不同的对手较量，不论是招式的顺序还是临场的动作都会有很大的变化。如果在实战对决中一再用套路来较量，就会被套路牵引着走，也会被对手打乱节奏，而套路本身给我们提供了一种处理场景的方法，灵活应用才能用以实战，这样才能达到“无招胜有招”的境界。

第 2 章和第 3 章介绍两个实例的设计，它们是两种完全不同类型的程序，在这个过程中尝试应用设计模式中的一些知识和思路，也就是需要知道如何利用招式来对敌。但大家不要仅仅局限于这两章所给出的例子，因为实际场景可以千变万化，针对自己所遇到的场景，需要去揣摩和思考。我们的例子通常不会写到实现部分，到设计部分基本结束，希望你能理解到“思想和方法应当源于生活和自然”。

第 2 篇 实现篇

在原计划中胖哥并没有打算写关于“实现”方面的内容，不过胖哥发现在实际工作中，小白“童鞋”们所面对的可能最多是实现，于是增加了这部分内容。但是在实现篇并不是拿一个业务实例来写代码，谈什么呢？

第 4 章谈谈项目中各种工作人的交互、开发模式、心态、技术上的思维方式、一些优化思路。第 5 章简单谈谈 UI 的一些用途、繁杂的点在那里。第 6 章谈谈实际工作中经常会遇到的“坑”，这些坑不容易被发现，但是我们经常容易掉进去，希望大家知道的不仅仅是这些坑，而是通过这些坑了解到，众多技术都是一把“双刃剑，有好处必有坏处”，只有最佳的场景选择最合适的技术，学会如何去量化与选择，同时需要了解到现在的工作对个人素质的要求越来越高。

第 3 篇 扩展篇之论道

扩展篇为杂谈，也是论道篇，读者可以先看第 5 篇的内容，本篇是对知识面和思想的一个扩展。一个老 A，应当知道业界的一些技术，所谓：“知己知彼，百战百胜”，接触任何一种问题，都要有相应的解决方案。

第 7 章和第 8 章以走马观花的方式来介绍一些集群知识、分布式知识（但并不意味着实际的技术和走马观花那样简单）。作为老 A 程序员，胖哥认为需要知道这些，这样才能更好地做好一些事情，或者说有机会去做更有挑战的事情。也许你今天用不到这些，但当你用到时，便能更加得心应手。

第 9 章讲解技术量化与权衡的一些事情，会谈一些时间管理、团队合作、过度优化、预知问题、不同项目的实现思路等。这里阐述的中心思想是，我们应当在技术追求的道路，考虑各种成本与价值的问题，从个人做起，再看看团队协作。

第 10 章是完结章，主要是探讨一些心态以及励志，有兴趣的小伙伴们可以看看，没有兴趣的无须关注本章。现代老 A，除素质过硬外，更要拥有智慧，智慧包含了丰富的知识面，同时也包含了对于世界的理解，静心做好当下，有机会去做好创新，在芸芸众生之中脱颖而出。

下册即将出版，敬请期待！



写代码是逻辑的，代码写出来就会有漏洞和 Bug，写书更多是感性的，问题自然会更多，作为刚开始写书的我，会尽我所能将书中的错误尽量减少，但不能减少书中内容的含金量。

本书以灌输思想为导向，更多的是希望读者通过自身独自思考和总结的能力，在一定基础之上进行进一步提升。当然，若读者对本书有任何意见，或者对书中某些问题百思不得其解，欢迎提出宝贵的意见和建议发送到 xieyu_zy@126.com，少量信息也可以发我个人微博 <http://weibo.com/xieyu2013>，我会尽快回复，将有效的勘误结果公布在博客 <http://blog.csdn.net/u010345637> 中，并会在修订版本中尽快修改。



胖哥第一次写书，而且这本书大部分内容是从键盘上敲出来的，所以深感写书也是不容易的事情，尤其是在炎热的夏天，几次想要放弃，不过最终能够坚持下来，我内心之中想要感谢家人、朋友、同事、与我一起玩 Java 的小伙伴们的支持和鼓励，感谢老婆背后默默的付出，再谢谢 2013 年刚出生的女儿，你的每一个依依哇哇的古怪表情都让爸爸有了动力与灵感，最后谢谢电子工业出版社给予我出版本书的机会，特别感谢出版社孙老师对我的指导与鼓励，以及葛娜编辑在这个过程中所付出的努力。

第 1 篇 Java 功底篇

第 1 篇为开篇序，讲什么？讲基础。

基础有“神马”好听的呢？或许你看看胖哥讲的基础，会有其他的理解呢？

基础深不深啊？我怕看不懂啊？我们先说感性认识，再说理性认识，大家快乐看书，只是小伙伴们不要硬生生地想将本书例子中的每个细节与实际技术中的每个细节对号入座，那就是钻牛角尖哦。

现在，就来看看第 1 章“扎马：看看功底如何”吧。如果你很清楚接下来的内容，那么就当作是与胖哥的一场面谈吧；如果还有些不太明白，那么就说明自己还有些基础需要补充。

第 1 章也是本书内容的引导部分，大家可以感性地认识下一些十分基础的内容会有不为人知的一面，而这些东西你可以把它当成兴趣，也可能在你工作中的某些时候提供一种灵感。

第 1 章

扎马：看看功底如何

本章主要内容

- String 的例子，见证下我们的功底
- 一此简单算法你会如何理解
- 简单数字游戏玩一玩
- 功底概述
- 原生态类型其他例子
- 集合类的那些破事儿
- 常见的工具包
- 面对技术，我们纠结的那些事儿
- 老 A 是在逆境中迎难而上者

1.1 String 的例子，见证下我们的功底

哇塞，第 1 节就开始讲代码例子，受不了啦，胖哥，你坏死了！所有的书第 1 节都是写这个领域有什么东西的。

哈哈，小胖哥天生就是个逆天之人哦，希望你能先实践有了感性认识后，再进行理论了解内在。

下面的代码改编于网络牛人的一段程序，先看代码清单 1-1。

代码清单 1-1 一段 String 的比较程序

```
private static void test1() {  
    String a = "a" + "b" + 1;  
    String b = "ab1";  
    System.out.println(a == b);  
}
```

胖哥，你是不是考我智商呀？我们平时对比两个对象不是用 equals() 吗？老师告诉我：两个字符串用等号是匹配不了的，结果应该是 false 吧。那么结果到底是多少呢？

运行结果：

true

🤖 什么？竟然是 true？为什么是 true？这是要逆天吗？这小段程序彻底颠覆了我的经验和老师教我的真理！“我和我的小伙伴们惊呆了……”

胖哥告诉你这不能怪老师，老师带进门，修行靠个人！

也许有朋友做出了 true 或猜出了 true，那么可否知道原因呢？如果你知道，那么本节跳过，无须再看；如果还不知道，就听听胖哥给你说说他所理解的原因。

要理解这个问题，你需要了解些什么？

- ◎ 关于“==”是做什么的？
- ◎ equals 呢？
- ◎ a 和 b 在内存中是什么样的？
- ◎ 编译时优化方案。

下面的内容会很多，现在我们可以站起来简单运动一下，端一杯咖啡，慢慢解读下面的内容。

►► 1.1.1 关于“==”

首先要知道“==”用于匹配内存单元上的内容，其实就是一个数字，计算机内部也只有数字，而在 Java 语言中，当“==”匹配的时候，其实就是对比两个内存单元的内容是否一样。

如果是原始类型 byte、boolean、short、char、int、long、float、double，就是直接比较它们的值。这个大家应该都清楚，这里不再详谈。

如果是引用（Reference），比较的就是引用的值，“引用的值”可以被认为是对象的逻辑地址。如果两个引用发生“==”操作，就是比较相应的两个对象的地址值是否一样。换一句话说，如果两个引用所保存的对象是同一个对象，则返回 true，否则返回 false（如果引用指向的是 null，其实这这也是一个 JVM 赋予给它的某个指定的值）。

理解寓意：大家各自拿到了一个公司的 offer，现在我们看看哪些人拿到的 offer 是同一个公司的。

►► 1.1.2 关于“equals()”

equals()方法，首先是在 Object 类中被定义的，它的定义中就是使用“==”方式来匹配的（这一点大家可以参看 Object 类的源码）。也就是说，如果不去重写 equals()方法，并且对应的类其父类列表中没有重写过 equals()方法，那么默认>equals()操作就是对比对象的地址。

equals()方法之所以存在，是希望子类去重写这个方法，实现对比值的功能，类似的，String 就自己实现了 equals()方法。为什么要自己去实现呢？因为两个对象只要根据具体业务的关键属性值来对比，确定它们是否是“一致的或相似的”，返回 true|false 即可。

迷惑：equals()不就是对比值的吗？为何说相似？

答曰：一日偶遇怪侠“蜗牛大师”一枚，赐予神剑于数人，猎人将其用于打猎；农夫将它用于劈柴；将军用它保家卫国；侠客用它行侠仗义、惩奸除恶，等等。

例如：在对比一些工程的图纸尺寸的时候，由于尺寸都会存在细节误差，可以认为宽度和高度比较接近，就可以返回 true，而不一定非要精确匹配。另外，图纸纸张的属性除了长度、宽度外，还有如名称、存放位置、卷号等属性，但是我们可能只需要对比它的长度与宽度，在这个范围内其余的属性不会考虑。也就是说，两个对象的值是否相同是自己的业务决定的，而不是 Java 语言来决定的。

感悟：变通，让标准变为价值，给你一种思想和标准，你可以有不同的使用，不能死

扣定理，我们要解决问题！

迷惑：equals()重写后，一般会重写 hashCode()方法吗？

要说明这个问题，我们先要补充一些概念。

Java 中的 hashCode 是什么——hashCode()方法提供了对象的 hashCode 值，它与 equals()一样在 Object 类中提供，不过它是一个 native（本地）方法，它的返回值默认与 System.identityHashCode(object)一致。在通常情况下，这个值是对象头部的一部分二进制位组成的数字，这个数字具有一定的标识对象的意义存在，但绝不等价于地址。

hashCode 的作用——它为了产生一个可以标识对象的数字，不论如何复杂的一个对象都可以用一个数字来标识。为什么需要用一个数字来标识对象呢？因为想将对象用在算法中，如果不这样，许多算法还得自己去组装数字，因为算法的基础是建立在数字基础之上的。那么对象如何用在算法中呢？

例如，在 HashMap、HashSet 等类似的集合类中，如果用某个对象本身作为 Key，也就是要基于这个对象实现 Hash 的写入和查找，那么对象本身如何能实现这个呢？就是基于这样一个数字来完成的，只有数字才能真正完成计算和对比操作。

hashCode 只能说是标识对象，因此在 Hash 算法中可以将对象相对离散开，这样就可以在查找数据的时候根据这个 key 快速地缩小数据的范围。但不能说 hashCode 值一定是唯一的，所以在 Hash 算法中定位到具体的链表后，需要进一步循环链表，然后通过 equals()来对比 Key 的值是否是一样的。这时 hashCode()与 equals()似乎就成为“天生一对”。换句话说，一个是为了算法快速定位数据而存在的，一个是为了对比真实值而存在的。

与 equals()类似，hashCode()方法也可以重写，重写后的方法将会决定它在 Hash 相关数据结构中的分布情况，所以这个返回值最好是能够将对象相对离散的数据。如果发生一个极端的情况，即 hashCode()始终返回一个值，那么它们将存在于 HashMap 的同一个链表中，将会比链表查询本身还要慢。

在 JDK 1.7 中，Hash 相关的集合类对使用 String 作为 Key 的情况，不再使用 hashCode 方式，而是有了一个 hash32 属性，其余的类型保持不变。

换个思路，hashCode()与 equals()并不是必须强制在一起，如果不需要用到这样的算法，也未必要重写对应的方法，完全由你自己决定，没有语法上强制的规约。

寓意：此好比，宝剑是否需要剑鞘？宝贝是否需要宝箱？雄性是否必须需要雌性？地球是否需要月亮？

而并非是，树是否需要土壤？生命是否需要食物？鱼儿是否必须需要水？世界是否需要阳光？

感悟：一切在于场景与需求，十分需要，但也可以在某些情况下放弃。

有人说：对比两个对象是否一致，可以先对比 hashCode 值再对比 equals()。

这似乎听上去挺有道理的，但是胖哥本人可并不这么认为！为什么呢？胖哥认为 hashCode 值根本不是为了对比两个对象是否一致而存在的，可以说它与两个对象是否一致“一点关系都没有”。

假如你希望对比两个对象是否是同一个对象，则完全可以直接用“==”来对比，而不需要用 hashCode()，因为直接对比地址值说明两个对象是否为同一个对象才是最靠谱的。另外，默认的 hashCode()方法还会发起 native 调用，并且两个对象都会分别发起 native 调

用（native 调用的开销也是不小的）。

假如不是对比地址，而是对比值，自然就需要对象中的某些属性来对比。拿 String 类型的对象来讲，如果调用某个 String 对象的 hashCode() 方法，它至少会在第 1 次调用这个方法时遍历所有 char[] 数组相关元素来计算 hashCode 值（这里之所以说至少，是因为这个 String 如果并发调用 hashCode() 方法，则可能被遍历多次），遍历过程中还需要外加一些运算。如果对比的两个对象分别获取 hashCode，自然两个 String 对象都会分别遍历一次 char[] 数组。

即使 hashCode 一样了，也同样证明不了这两个 String 是一样的（这个 hashCode 是根据 char[] 数组的值算出来的，不同的 String 完全可以算出一样的值），还得再次循环两个 String 中所有的字符来对比一次才能证明两个对象是一样的。其实遍历了两个 char[] 数组已经是最坏的情况了，equals() 还未必会这样做（在后文的图 1-2 中会详细说明）。

换一个角度来讲，如果是两个其他的自定义类型的对象（不是 String 类型的对象）之间判定出来 hashCode 不一样，也不能说它们的值不一样（有可能 equals() 匹配的是一个综合值，与 hashCode 一点关系都没有），还是要进行 equals()，这样绕来绕去往往是把简单问题复杂化了。

equals() 内部要怎么做就去怎么做嘛，想要优化，完全可以像 JDK 自带的许多类那样，先对比一些简单的属性值，再对比复杂的属性值，或者先对比业务上最快能区分对象的值，再对比其他的值，或者先对比地址、长度等处理方式，将那些不匹配的情况尽快排出。

有人说重写后的 hashCode() 内部操作确实比 equals() 简单许多倍，其实这个动作判定也是可以放在 equals() 方法的第 1 步中来的完成的，无须外部程序关注。

补充：String 的 equals() 方法中默认就要先对比传入的对象与当前的 this 是不是同一个对象。在 HashMap 等集合类的查找过程中，也不是单纯的 equals()，也会先对比两个对象是不是同一个对象。

好累，休息休息！左三圈、右三圈，再来看看胖哥为你做解读！

a 和 b 的内存情况是什么样的？

回到“代码清单 1-1”的例子中，其中的等号说明 a 和 b 是指向同一块内存空间的，就像两个人拿到同一个公司的 Offer 一样，他们像什么呢？见图 1-1，“死冤家，又在一起了！”

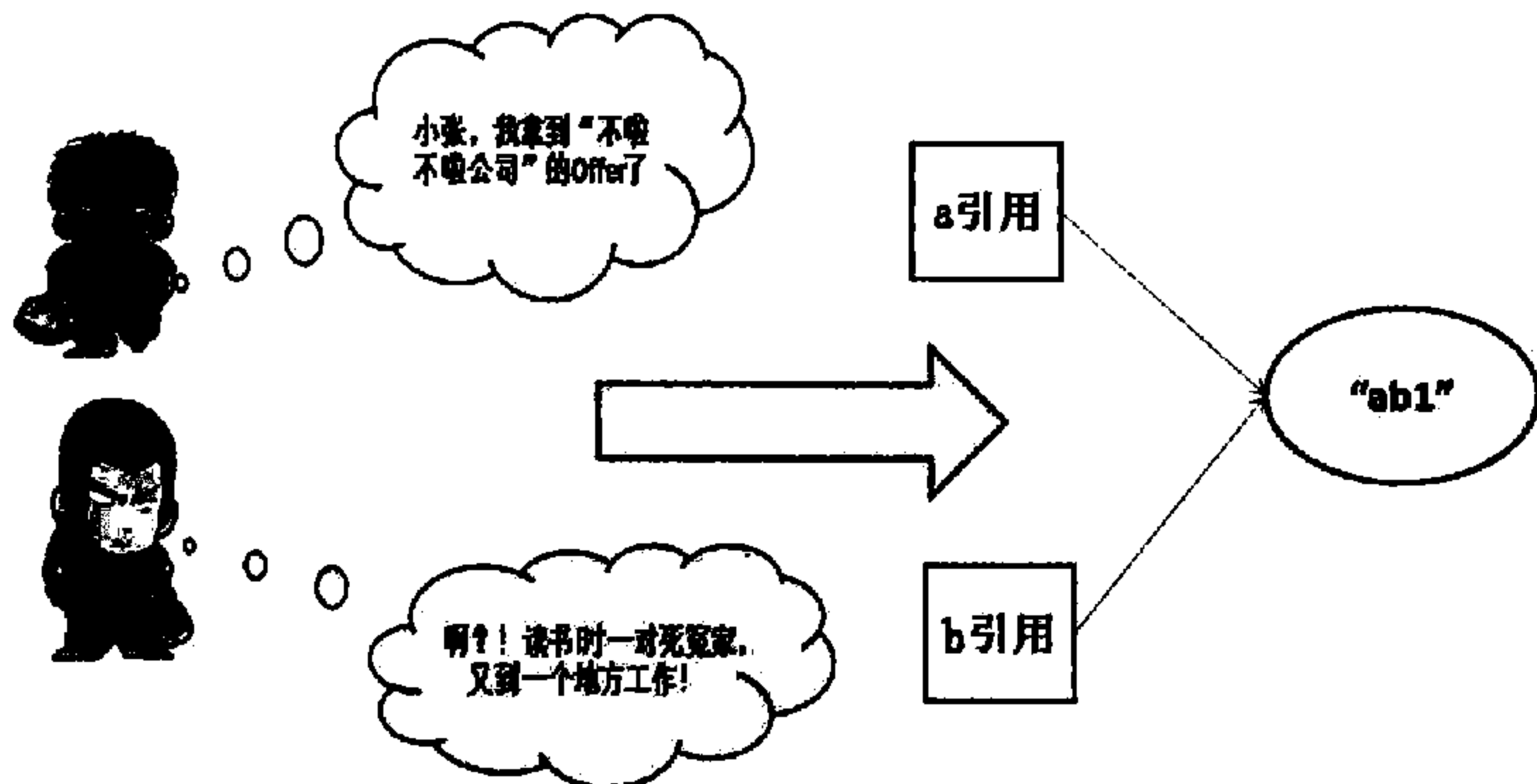


图 1-1 两个冤家又拿到同一个公司的 Offer

为什么 a、b 两个引用都引用到同一块空间了呢？请看 1.1.3 节的内容解释，不过在这一节中我们先感性认识下 JVM 的一些“东东”，在第 3 章中会有更详细的介绍。小伙伴们不要着急，我们一步步来学习。

►► 1.1.3 编译时优化方案

a 引用是直接赋值的，b 引用是通过“+”赋值的，a 和 b 两个引用为什么会指向同一个内存单元？这就是 JVM 的“编译时优化”。如此神奇！小伙伴们惊呆了吧！

当编译器在编译代码：String a = "a" + "b" + 1; 时，会将其编译为：String a = "ab1";。

为何？因为都是“常量”，编译器认为这 3 个常量叠加会得到固定的值，无须运行时再进行计算，所以就会这样优化。

疑惑：编译器为何要做此优化？

寓意：“小胖”说我的报销单写好了并盖章了，“小明”说我的也 OK 了，那么就合并一起邮寄报销单吧。“小锐”说我的快写好了，不过还没盖章，那你写好后再说吧。

寓意：为提升整体工作效率和节约资源，能提前做的事情就提前做。我们自己设计一种平台或语言的时候是否会考虑这些呢？

补充：编译器类似的优化还有许多（在后文中会有介绍），例如，当程序中出现 int i = 3 * 4 + 120 时，并不是在实际运行时再计算 i 的值，而是在编译时直接变成了 i = 132。

容易出错：JVM 只会优化它可以帮你优化的部分，它并不是对所有的内容都可以优化。例如，就拿上面叠加字符串的例子来说，如果几个字符串叠加中出现了“变量”，即在编译时，还不确定具体的值是多少，那么 JVM 是不会去做这样的编译时合并的。而 JVM 具体会做什么样的优化，不做什么样的优化，需要我们不断去学习，才能把工作做得更好。

同理证明的道理：String 的“+”操作并不一定比 StringBuilder.append() 慢，如果是编译时合并就会更快，因为在运行时是直接获取的，根本不需要再去运算。同理，千万不要坚定地认为什么方式快、什么方式慢，一定要讲究场景。而为什么在很多例子中 StringBuilder.append() 比 String 的“+”操作快呢？在后文中，胖哥会继续介绍原因。

►► 1.1.4 补充一个例子

胖哥，我开始有点兴趣了，不过感觉在 String 方面还没过瘾！

OK，胖哥就再补充一个例子。

代码清单 1-2 String 测试 1

```
private static String getA() {return "a";}
public static void test2() {
    String a = "a";
    final String c = "a";

    String b = a + "b";
    String d = c + "b";
    String e = getA() + "b";

    String compare = "ab";
    System.out.println(b == compare);
    System.out.println(d == compare);
    System.out.println(e == compare);
}
```

这段代码是说编译优化，看看输出是什么？


```
false  
true  
false
```

看到这个结果，如果你没有“抓狂”，一种可能是你真的懂了，还有一种可能就是你真的不懂这段代码在说什么。在这里，胖哥就给大家阐述一下这个输出的基本原因。

第1个输出 false。

“b”与“compare”对比，根据代码清单 1-2 中的解释，compare 是一个常量，那么 b 为什么不是呢？因为 `b = a + "b"`，a 并不是一个常量，虽然 a 作为一个局部变量，它也指向一个常量，但是其引用上并未“强制约束”是不可以被改变的。虽然知道它在这段代码中是不会改变的，但运行时任何事情都会发生，尤其是在“字节码增强”技术面前，当代码发生切入后，就可能发生改变。所以编译器是不会做这样优化的，所以此时在进行“+”运算时会被编译为下面类似的结果：

```
StringBuilder temp = new StringBuilder();  
temp.append(a).append("b");  
String b = temp.toString();
```

注：这个编译结果以及编译时合并的优化，并非胖哥凭空捏造的，在后文中探讨 javap 命令时，这些内容将得到实际的印证。

第2个输出 true。

与第1个输出 false 做了一个鲜明对比，区别在于对叠加的变量 c 有一个 final 修饰符。从定义上强制约束了 c 是不允许被改变的，由于 final 不可变，所以编译器自然认为结果是不可变的。

final 还有更多的特性用于并发编程中，我们将在第5章中再次与它亲密接触。

第3个输出 false。

它的叠加内容来源于一个方法，虽然方法内返回一个常量的引用，但是编译器并不会去看方法内部做了什么，因为这样的优化会使编译器困惑，编译器可能需要递归才能知道到底返回什么，而递归的深度是不可预测的，递归过后它也并不确保一定返回某一个指定的常量。另外，即使返回的是一个常量，但是它是对常量的引用实现一份拷贝返回的，这份拷贝并不是 final 的。

也许在 JIT 的优化中会实现动态 inline()，但是编译器是肯定不会去做这个动作的。

疑惑：我怎么知道编译器有没有做这样的优化？

答曰：懂得站在他人角度和场景看待事物的不同侧面，加以推导，结合别人的意见和建议，就有机会知道真相。

寓意：如果我是语言的作者，我会如何设计？再结合提供验证、本质、文档，共同引导我们了解真相。

理解方式：编译器优化一定是在编译阶段能确定优化后不会影响整体功能，类似于 final 引用，这个引用只能被赋值一次，但是它无法确定赋值的内容是什么。只有在编译阶段能确定这个 final 引用赋值的内容，编译器才有可能进行编译时优化（请不要和运行时的操作扯到一起，那样你可能理解不清楚），而在编译阶段能确定的内容只能来自于常量池中，例如 int、long、String 等常量，也就是不包含 new String()、new Integer()这样的操作，因为这是运行时决定的，也不包含方法的返回值。因为运行时它可能返回不同的值，带着这个基

本思想，对于编译时的优化理解就基本不会出错了。

➤➤ 1.1.5 跟 String 较上劲了

看到这里你是否觉得自己对天天使用的 String 还不是特别了解，而胖哥之所以不断谈到 String 的一些细节，不仅是要说 String 本身有什么样的特征，在 Java 语言中还有许许多多的“类”值得我们去研究。

如果你觉得还没过瘾，和 String 较劲上了，那么关于 String 我们再来举个例子，然后胖哥做个小小总结，这里的故事就结局了。

代码清单 1-3 String 测试 2

```
public static void test3() {
    String a = "a";
    String b = a + "b";
    String c = "ab";
    String d = new String(b);
    println(b == c);
    println(c == d);
    println(c == d.intern());
    println(b.intern() == d.intern());
}
```

这段代码与上一个例子类似，只是增加了 intern()的调用。同样的，我们可以先看看输出是什么。

```
false
false
true
true
```

从输出上看规律，胖哥相信有不少小伙伴们应该能猜到是 intern()方法在起作用。是的，没错，就是它。

HotSpot VM 7 及以前的版本中都是有 Perm Gen(永久代)这个板块的(第 3 章中详述)，在前面例子中所谈到的 String 引用所指向的对象，它们存储的空间正是这个板块中的一个“常量池”区域，它对于同一个值的字符串保证全局唯一。

如何保证全局唯一呢？当调用 intern()方法时，JVM 会在这个常量池中通过 equals()方法查找是否存在等值的 String，如果存在，则直接返回常量池中这个 String 对象的地址；若没有找到，则会创建等值的字符串（即等值的 char[]数组字符串，但是 char[]是新开辟的一份拷贝空间），然后再返回这个新创建空间的地址。只要是同样的字符串，当调用 intern()方法时，都会得到常量池中对应 String 的引用，所以两个字符串通过 intern()操作后用等号是可以匹配的。

回到代码清单 1-3 的例子中，字符串"ab"本身就在常量池中，当发生 new String(b)操作时，仅仅是进行了 char[]数组的拷贝，创建了一个 String 实例。当这个新创建的实例发生 intern()操作时，在常量池中是能找到这个对象的，其余的内容，依此类推，可以得到为什么会这样的答案。

此时，能否有所感悟，没有的话没关系，这是第 1 节，胖哥带你一起感悟：

- ◎ String 到底还有多少方法我没见过？
- ◎ intern()有何用途？有什么坑？它是否会在常量池中被注销？
- ◎ 是否开始觉得这些东西复杂，又像是回到最简单的道理上来了？

◎ 是否认为自己的功底需要细化？

1.1.6 intern()/equals()

String 可能有很多的方法还没见过，在研究清楚 String 之前，还得看不少其他的代码。String 并没有我们想的那么简单，而且它在我们的工作中无处不在，所以需要去学习它。

胖哥在本节中也只是给大家讲个大概，String 中的许多奥秘，会在本书后面的章节中不时出现，大家留意哦。

如果仔细看了对 intern() 的文档描述，就应该知道 intern() 本身的效率并不高，它虽然在常量池中，但它也需要通过 equals() 方法去对比，在常量池中找是否存在相同的字符串才能返回结果。显然，这个过程中对比的通常不止一个字符串，而是多个字符串，效率自然要更低一些。另外，它需要“保证唯一”，所以需要有锁的介入，效率自然再打折扣。因此，直接使用它循环对比得出的效率通常会比循环 equals() 的效率低。但这并不是说对比地址比 equals() 要慢一些，它是输在了对比地址之前需要先找到地址上。

它的效率低是相对 equals() 来讲的，如果要细化认识这两者的效率并应用在实践中，那么我们要先看看 String.equals() 方法的实现细节，请看图 1-2。

JDK 1.6 的 equals()

```
public boolean equals(Object anObject) {
    if (this == anObject) {
        return true;
    }
    if (anObject instanceof String) {
        String anotherString = (String)anObject;
        int n = count;
        if (n == anotherString.count) {
            char v1[] = value;
            char v2[] = anotherString.value;
            int i = offset;
            int j = anotherString.offset;
            while (n-- != 0) {
                if (v1[i++] != v2[j++])
                    return false;
            }
            return true;
        }
    }
    return false;
}
```

JDK 1.7 的 equals()

```
public boolean equals(Object anObject) {
    if (this == anObject) {
        return true;
    }
    if (anObject instanceof String) {
        String anotherString = (String) anObject;
        int n = value.length;
        if (n == anotherString.value.length) {
            char v1[] = value;
            char v2[] = anotherString.value;
            int i = 0;
            while (n-- != 0) {
                if (v1[i] != v2[i])
                    return false;
                i++;
            }
            return true;
        }
    }
    return false;
}
```

图 1-2 String.equals()源码

不论是 JDK 1.6 还是 1.7，String.equals() 的代码逻辑大致归纳如下。

(1) 判定传入的对象和当前对象是否为同一个对象，如果是就直接返回 true（就像在代码清单 1-1 中一样，两个人拿到同一个公司的 offer）。

(2) 判定传入对象的类型是否为 String，若不是则返回 false（如果是 null 也不会成立）。

(3) 判定传入的 String 与当前 String 的长度是否一致，若不一致就返回 false。

(4) 循环对比两个字符串的 char[] 数组，逐个对比字符是否一致，若存在不一致的情况，则直接返回 false。

(5) 循环结束都没有找到不匹配的，所以最后返回 true。

由此，我们至少得出两个最基本的结论。

(1) 要匹配上，就要遍历两个数组，逐个下标匹配，这是最坏的情况。没想到吧，匹配上了还是最坏的情况。那么对于大字符串来讲，这件事情是悲剧的。

(2) 字符串首先匹配了长度，如果长度不一致就返回 false。这对于我们来讲也是乐观的事情，如果真的遇上大字符串匹配问题，也许就有其他的技巧性方法哦。

好了，我们回过头来看看 `intern()` 比 `equals()` 还慢的问题。这个理论结果，可以用循环多次 `equals()`，然后循环多次 `intern()`，最后做 “==” 匹配来测试，得到的直接结论是：使用 `equals()` 效率会更高一些。一些曾经认为 `intern()` 对比地址效率高的“技术控”同学们，有点傻眼了。

而 `intern()` 也并非无用武之地，它也可以用在一些类似于“枚举”的功能中，只是它是通过字符串来表达而已（其实枚举在底层的实现也是字符串）。

例如，在某种设计中会涉及很多数据类型（如 `int`、`double`、`float` 等），此时需要管理这些数据类型，因此通常是以字符串方式来存储的。当需要检索数据时，通过字典会得到它们的类型，然后根据不同的类型做不同的数据转换。这个动作最简单的方法可能就是用一个类型列表 `for` 循环 `equals()` 匹配传入的类型参数，匹配到对应的类型后就跳转到对应的方法中。如果类型有上百种之多，自然 `equals()` 的次数会非常多，而 `equals()` 的效率很多时候其实不好说，或许有些时候可以换个思路来做。

在加载数据字典的类型时就直接 `intern()` 到内存中（因为这些类型是固定的），在真正匹配数据类型的时候就是“常量比常量”，对比地址，这样比较就快速多了，因为它不会使用 `equals()`，也不是匹配的时候再去调用 `intern()`。

举个例子：假如记录的是数据库的数据类型。这个中间平台在执行某些操作时需要检测每个表的元数据的每个列的类型，在很多时候这些元数据可能会被事先加载到内存中（前提是元数据不是特别多），并且以 `intern()` 方式直接注入到常量池中，那么在后面逐个列的类型匹配过程中就不用 `equals()` 了，而是直接用 “==”，因为这些内容已经在常量池中了。

在这样的情况下，不同的表和属性如果类型是相同的，那么它们所包含的字符串对象也会是同一个，同时也可以在一定程度上节约空间。

这个方法比枚举要“土”一些，但是它也是一种方案，我们完全可以用枚举来实现，而枚举内在的实现也是类似的方式，只是它是“虚拟机”级别自带的而已。

`intern()` 注入到常量池中的对象和普通的对象一样占用着相应的内存空间，唯一的区别是它存储的位置是在 Perm Gen（永久代）的常量池中。永久代会注销吗？会的，它也会被注销，在 FULL GC 的时候，如果没有任何引用指向它则会被注销。关于永久代的一些故事，我们还是放到第 3 章来讨论细节吧。



注意 本章探讨的 `intern()` 仅仅针对 JDK 1.6 环境，如果将环境更换为 JDK 1.7，String pool 将不会存在 Perm Gen 当中，而是存在堆当中，部分代码的测试结果将有可能更加诡异，请大家悉知。（在 JDK 1.7 环境下运行本章给出的例子，结果会有少数不一致，胖哥在本书光盘相应的测试类中增加了 `testForJDK17()` 方法，大家可以单独看看，在 JDK 1.7 下可能还会有意想不到的效果。）

1.1.7 StringBuilder.append()与 String “+” 的 PK

在很多的书籍和博客中，会经常看到某些小伙伴们用一个 for 循环几千万次，来证明 StringBuilder.append()操作要比 String “+” 操作的效率高出许多倍。其实胖哥并不这么认为，为何呢？因为胖哥知道这是怎么发生的，下面就跟着胖哥的思路来看看吧。

胖哥认为：如果一事物失去了价值，它就没有存在的必要，所以它存在必有价值。

在前面的例子中，String 通过 “+” 拼接的时候，如果拼接的对象是常量，则会在被编译器编译时合并优化，这个合并操作是在编译阶段就完成的，不需要运行时再去分配一个空间，自然效率是最高的，StringBuilder.append()也不可能做到。如果是运行时拼接呢？在后面内容的学习中，我们会发现一个 Java 字符串进行 “+” 操作时，在编译前后大概是如下这样的。

原始代码为：

```
String a = "a";
String b = "b";
String c = a + b + "f";
```

编译器会将它编译为：

```
String a = "a";
String b = "b";
StringBuilder temp = new StringBuilder();
temp.append(a).append(b).append("f");
String c = temp.toString();
```

注：为了说明实际问题，胖哥才用 Java 代码来说明字符串拼接中 “+” 的真正道理，在实际的场景中这是 class 字节码中的内容，而这个 temp 是胖哥虚拟出来的。

如果将 String 的 “+” 操作放在循环中，那么自然在循环体内部就会创建出来无穷多的 StringBuilder 对象，并且执行 append()后再调用 toString()来生成一个新的 String 对象。这些临时对象将会占用大量的内存空间，导致频繁的 GC。我们用图 1-3 来描述循环叠加字符串编译前后的代码对比。

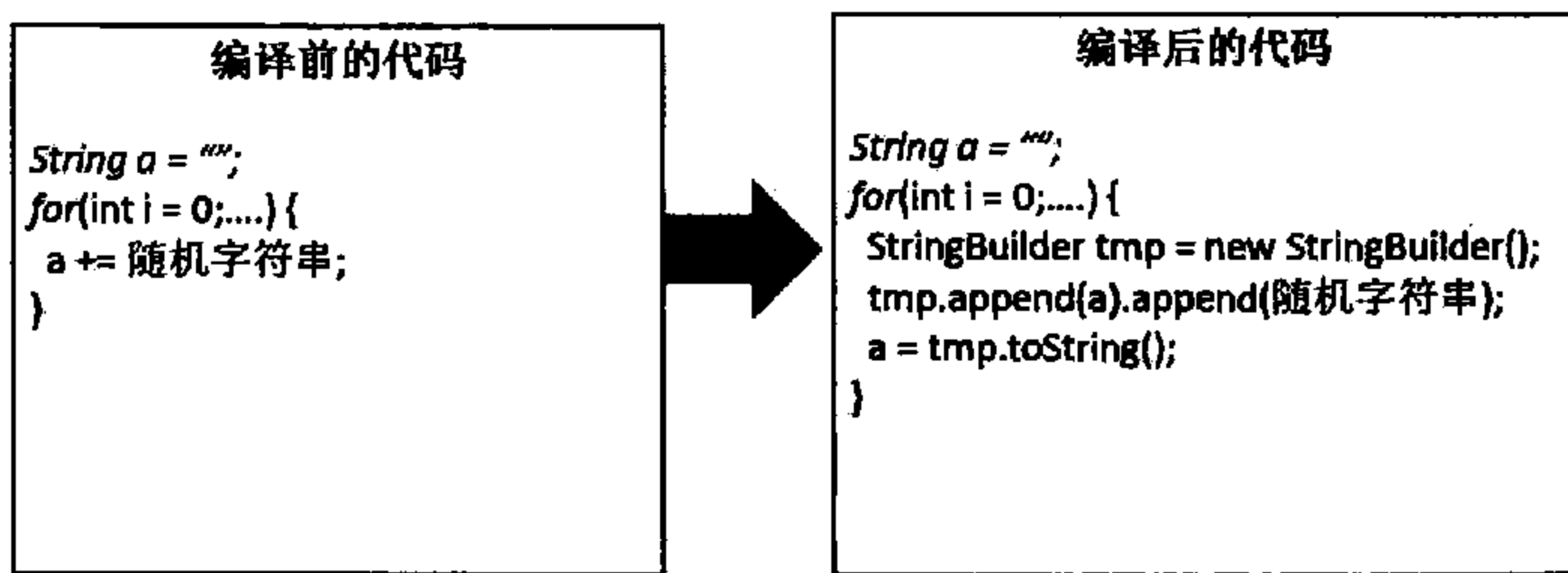


图 1-3 循环叠加字符串编译前后的代码对比

在循环过程中，a 引用所指向的字符串会越来越大，这就意味着垃圾空间也会越来越大。当这个 a 所指向的字符串达到一定程度后必然会进入 Old 区域，若它所占用的空间达到 Old 空间的 1/4，需要再次分配空间的时候，就有可能发生 OOM，而最多达到 1/4 的时候就肯定会发生 OOM。为何是 1/4？我们得先看看 StringBuilder 做了些什么。

上面的代码在循环内部会先分配一个 StringBuilder 对象，这个对象内部会分配 16 个长度的 char[] 数组，当发生 append()操作时，如果空间够用，就继续向后添加元素；若空间不

够用，则会尝试扩展空间。扩展空间的规则是：使用当前“StringBuilder 的 count 值 + 传入字符串的长度”作为新的 char[]数组的参考值，这个参考值将会与 StringBuilder 的 char[]数组总长度的 2 倍来取最大值，也就是最少会扩展为原来的 2 倍。

count 值并不是 char[]数组的总长度，而是当前 StringBuilder 中有效元素的个数，或者说是通过 StringBuilder.length()方法得到的值。char[]数组的总长度可以通过 capacity()方法获取到。

关于 StringBuilder.append()功能的说明，我们来看看它的源码实现就更加清楚了，如图 1-4 所示。

```
public StringBuilder append(String str) {
    super.append(str);
    return this;
}

public AbstractStringBuilder append(String str) {
    if (str == null) str = "null";
    int len = str.length();
    if (len == 0) return this;
    int newCount = count + len;
    if (newCount > value.length)
        expandCapacity(newCount);
    str.getChars(0, len, value, count);
    count = newCount;
    return this;
}

void expandCapacity(int minimumCapacity) {
    int newCapacity = (value.length + 1) * 2;
    if (newCapacity < 0) {
        newCapacity = Integer.MAX_VALUE;
    } else if (minimumCapacity > newCapacity) {
        newCapacity = minimumCapacity;
    }
    value = Arrays.copyOf(value, newCapacity);
}
```

图 1-4 StringBuilder.append()的内存扩展

回到前面提到的“循环叠加字符串”的例子中，每次扩容会最少扩容 2 倍，而在每次扩容前，原来的对象还需要暂时保留，那么至少在某个时间点需要 3 倍的内存空间，自然 JVM 的 Young 空间的 Survivor 区域很快装不下，要让它进入 Old 区域。

当 a 引用的对象空间达到了 Old 区域的 1/4 大小时，同样会先分配一个 StringBuilder 对象，初始化的长度也是 16 个长度的 char[]数组。这个 StringBuilder 首先会进行 append(a)操作，在这次 append 操作时发现空间不够大，需要分配一个更大的空间来存放数据，但是这个字符串本身还不能被释放掉。

此时，字符串已经很大，那么肯定它不会只有 16 个长度，根据上面对 append()源码的描述，新分配的空间应当是 a 引用当前所引用的字符串的长度（因为当前的 count 值应当是 0，而同时也可以发现，其实 StringBuilder 内部已经没有空闲区了），所以 Old 区域最少占用了 1/4 + 1/4 = 1/2 的大小。

还没有完，还需要 append(随机字符串)来操作，此时 StringBuilder 是没有空闲区域的，那么自然空间是不够用的，又会涉及扩容的操作（只要随机字符串有 1 个字符），扩容最少是原来的 2 倍，就需要 1/4 * 2 = 1/2 的空间，那么剩余的一半 Old 空间就又被用掉了。

分配一个 2 倍大小的空间，需要将数据先拷贝过来，原来的空间才能释放掉。但是这个时候还没分配出来，原来的空间自然释放不掉，那么 $Old * 1/4$ 大小的 String 空间再一次叠加的时候就有可能将整个内存“撑死”。

假如 `append()` 的随机字符串是空字符串“”，就不会发生扩容，这样是不是就不会发生 OOM 了呢？这还没有完哦，最后还需要 `toString()` 方式来创建一个新的 String 对象，这个过程会开辟一个同样大小的 `char[]` 数组将内容拷贝过去，也就是说，如果拼接前字符串所占用的空间已经达到了 Old 区域的 $1/3$ 大小，就肯定会发生 OOM 了。如果你非要想想，这个时候的空间是不是就差“几个字节”就不会发生 OOM 呢？但是要考虑到下一轮循环是 100% 会发生 OOM 的。

不过，大家不要因为后面还有一次 `toString()` 需要分配空间，就认为前一种情况在内存空间 $1/5$ 的时候就可以导致 OOM，要知道前面在发生扩容的时候，如果扩容成功，原来的数组就可以被当成垃圾释放掉了（这是在 `StringBuilder` 内部替换 `char[]` 的时候发生的）。也就是说，原来的数组生命周期不会到 `toString()` 这一步。

这里的例子是假设 JVM 所有的内存给一个 String 来使用，在实际的场景中肯定不是这样的，肯定会有其他的开销，因此内存溢出的概率会更高。

再回过头来看看，用 `StringBuilder` 来写测试代码时是怎么写的。

```
StringBuilder builder = new StringBuilder();
for(...) {
    builder.append(随机字符串);
}
```

这段代码中的 `Stringbuilder` 对象只有一个，虽然内部也只是先分配了 16 个长度的 `char[]` 数组，但是在扩容的时候始终是 2 倍扩容。更加重要的是，它每次扩容后都会有一半的空间是空闲的，这一半的空间正好是扩容前数组的大小。

由于多出来的一半空间，通常不会在每次 `append()` 的时候发生扩容，而且对象空间越大，越不容易发生扩容。更不会发生的是，每次操作都会申请一个大的 `StringBuilder` 对象并将它很快当成垃圾，并且在操作过程中还对这个很大的 `StringBuilder` 做扩容操作。

这种写法，最坏的情况是它所占用的内存空间接近 Old 区域 $1/3$ 的时候发生扩容会导致 OOM（这里和 String 的拼接有点区别，如果遇到类似的差几个字节则导致 OOM，此时 String 的拼接在下一轮循环时必然会发生 OOM，但是 `StringBuilder` 扩容后会经历很长的一个过程不再发生扩容）。

另外，在 `StringBuilder.append()` 中间过程中产生的垃圾内存大多数都是小块的内存，它所产生的垃圾就是拼接的对象以及扩容时原来的空间（下一次再发生 String 的“+”操作时，前一次“+”操作的结果就成了垃圾内存，自然垃圾会越来越多，而且内在扩容还会产生更多的垃圾），在这种场景下，通常使用 `StringBuilder` 来拼接字符串效率会高出许多倍。

所以，首先确认一点，不是 String 的“+”操作本身慢，而是大循环中大量的内存使用使得它的内存开销变大，导致了系统频繁 GC（在很多时候程序慢都是因为 GC 多造成的），而且是更多的 FULL GC，效率才会急剧下降。

但是回过头来想一想，在实际的工作场景中很少会遇到 for 循环几百万次去叠加字符串的情况，虽然许多不同的线程叠加字符串的次数加在一起可能会有几百万次，但是它们本身就会开辟出不同的空间（使用 `StringBuilder` 也是每个线程单独使用自己的空间，如果

是共享的就存在并发问题)。总而言之,如果是少量的小字符串叠加,那么采用 `append()` 带来的效率提升其实并不会太明显;但是如果遇到大量的字符串叠加或大字符串叠加的时候(这些字符串不是常量字符串),那么使用 `append()` 来拼接字符串效率的确会更高一些。

在 JVM 中,提倡的重点是让这个“线程内所使用的内存”尽快结束,以便让 JVM 认为它是垃圾,在 Young 空间就尽量释放掉,尽量不要让它进入 Old 区域。一个很重要的因素是代码是否跑得够快,其次是分配的空间要足够小(这些内容将在第 3 章中探讨)。

`StringBuilder` 的使用也是有“坑”的。上面已经看到 `StringBuilder` 内部也可能会创建新的数组,并不断将原来的数组数据拷贝到新的数组中,虽然不会像“+”那样每次操作都会出现拷贝,但是同样会出现很多的内存碎片。

如果你要优化到极致,则需要深知业务细节,申请的空间尽量确保有很少的拷贝和碎片,甚至于没有,例如:`new StringBuilder(1024)`。如果你是一个高手,对代码有洁癖,那么扩展 `StringBuilder` 实现的方法也是同样可以的。但是如果你不是很清楚具体的业务,则最好直接用默认的方式,其实它很多时候未必真的那么节约内存。为什么呢?

如果拼接的字符串不足 1024 个或差距很大,那么肯定是浪费空间的。换个角度,如果是 `append(a).append(b)` 操作, `a` 的长度是 2, `b` 的长度是 1023,扩容是必然的,只是扩容前需要释放掉的 `char[]` 数组长度是 1024 (2KB 以上的空间),而且扩容后的长度是 2048 (分配 4KB 以上的空间)。如果使用默认的 `new StringBuilder()`,也会发生扩容,扩容前需要释放掉的 `char[]` 数组空间长度是 16 (抛开头部的 32 字节),而新分配的 `char[]` 数组长度是 1025 而不是 2048。

这也再次说明世事无绝对,一定要看场景,没有什么写法是绝对好的,使用这种方式,通常是希望在字符串拼接的过程中,将中间扩容降低到最少。

在类似的问题上,也有人问过胖哥:多个字符串拼接时,有的字符串很大,有的字符串很小,是先 `append` 大字符串还是小字符串呢?胖哥也不能给出明确的答案,只能说有一些不同的情况。

先添加小字符串,再添加大字符串,在什么时候比较好用呢?小字符串不是特别多,大字符串就一两个。在这种情况下,如果先 `append` 大字符串,在扩容的时候,可能不会进行 2 倍扩容,而是选择当前的 `count` 值+大字符串的长度来扩容,而且扩容后是没有剩余空间的,此时再来 `append` 小字符串,也会发生 2 倍扩容,自然浪费的空间会很多。如果先 `append` 小字符串,可能就扩展到几十个字符或几百个字符,几次扩容就可以搞定,而且扩容过程中的垃圾内存都是很小的。由于板块小扩容的过程也是迅速的,后面 `append` 大字符串该有的扩容开销依然有,只是通常不会选择 2 倍扩容而已。

如果要添加许多小字符串(例如上千个小字符串),大字符串也不是太多,此时如果先 `append` 大字符串,在发生第 1 个小字符串 `append` 时,就会进行 2 倍扩容,那么就会有一半的剩余空间,这个剩余空间可以容纳非常多的小字符串,自然扩容开销就要小很多。反过来,如果先 `append` 许许多多的小字符串,由于初始化只有 16 个长度的 `char[]` 数组,那么上千个小字符串叠加起来可能会发生 10 次左右的扩容,而扩容的空间也会越变越大。

关于 String 的“+”的补充说明

上面提到 String 的“+”会创建 `StringBuilder` 对象,然后再操作,那么粒度是多大呢?并不是“+”相关的两个 String 为粒度,而是一行代码为粒度。什么叫作一行代码为粒度呢?例如:


```
String str = a + b + c + d + e;
```

这就是一行，它只会申请一个 `StringBuilder` 执行多次 `append()` 操作，然后将其赋值给 `str` 引用，如果换成了两条代码就会申请多个。

但是这个“一行”并不是指 Java 代码中的一行，因为我们可以把 Java 代码中的很多行一层层嵌套到一条代码中，其实实际运行的代码还是多行，而是指可以连续在一起的代码。

关于内存拷贝和碎片，在本书 3.5 节中会有非常详尽的解释和说明。

本节胖哥用了十分简单的 3 个代码例子来说明有许多基础内容是值得我们去研究的。不知你是否能体会到其中的变化都是使用许多简单的基础知识加以变通的结果，这种变化是十分多的，但“万变不离其宗”。

本节从某种意义上印证了功底的重要性，即明白基本是明白内在的一个基础，上层都是由此演变而来的。

同样的，大家可以自己去研究 `String` 的其他方法，例如 `startsWith()`、`endsWith()` 是如何实现的；如果代码中出现了反反复复这样的操作，是否有解决思路。比如说要对每个请求的后缀进行 `endsWith()` 处理，自然就需要与许多的后缀循环进行匹配，而每个匹配都可能会去对比里面的许多字符，那么也许可以用其他的方法来处理。

本节胖哥是否对 `String` 做了一个诠释呢？没有，肯定没有，也不会！

本节所提到的内容，仅仅是想让你客观认识一下自己是否需要补充一些基础知识，希望你以此为例，学会看内在、看源码，活学活用。如果你认为自己“需要补充功底”了，胖哥的目的就达到了，此书分享过去的我的那些事，献给还在成长的你。

本节的故事到这里就结束了，胖哥不知道你是否满意，如果满意则祝愿你做一个美梦，以更好的心情和心态学习下一节的内容。

1.2 一些简单算法，你会如何理解

终于迎来第二次聚会的机会，本节内容会轻松许多，也许一盏茶的工夫就可以听完这个小故事。

注：其实本节并不是讨论算法，例子也会很简单，如果你对算法很熟悉，请跳过此节。

想要从一堆数据中找出一个 `max`、`min`。

想要从 100 万个数字中找出最大的 10 个数字。

你的想法是什么？你会如何找？先排序，再找，或者摸不到头脑。

胖哥的一些方法也许会帮到你：“想想学校里面排队、找人是怎么做的”。

假如一个学校有几千人，你要找一个人，胖哥给你提供几种方法，你选哪一种？

◎ 方法 1：几千个人我逐个去问，你是不是我要找的人，如果回答是，那么就是我要找的；如果不是，则继续问下一个。

◎ 方法 2：我知道学生的年级，然后在年级中每个班级按照方法 1 继续做同样的事情。

◎ 方法 3：我知道具体的班级，然后在班级里面像“方法 1”一样逐个问。

◎ 方法 4：我知道姓名，通过姓名得到有几个班级同名的人，这几个人我逐个问。

◎ 方法 5：我知道班级和姓名，也许就 1 个或 2 个学生同名，然后逐个问。

◎ 方法 6：我知道“学号”，直接找到人。

上面 6 种方法属于思维上的方法学，在不同的场景下使用不同的手段，但你会毫不犹豫地：第 1 种方法肯定是最傻的方法。没错，但是我们的程序往往就是这样写的，一个长长的 for 循环加匹配，就是逐个问的道理，对于人数不多的情况它其实也是实用的，但人数多了自然会有问题。后面几种方法都是知道某些前提内容后，然后通过不同的索引手段找到了人。好了，下面我们回到正题，探讨几种计算方法的场景。

1.2.1 从一堆数据中找 max 和 min

在一堆数据中找到一个最大的数据，如果它是无序的，就不存在“班级、年级”的维度概念，也就是说，没有数据分组的概念，在这种场景下不得不去做一次数组遍历，和学校的学生不按照班级站队的道理一样。但是需要做一次排序吗？当然不需要，因为我们只要找出最大值和最小值就可以了，排序的代价太大了。如果要求最大值，则可以先给一个临时变量赋值为 Integer.MIN_VALUE 或数组中的第 1 个元素值，然后循环数组的每个下标，只要比当前的这个值大，就给它赋予最新的值。这个时间复杂度是循环一次就可以搞定的，自然就不需要排序那么大的时间复杂度了。而排序会是什么样的呢？

在学校排队的时候，学生有自己的主见，每人都有大脑（CPU），当“校长”（号令者）下达命令后，每个学生自己知道去找位置，可以将自己的大致位置找好，细节的可以再由班主任来排序，但计算机中的 CPU 是有限的，不能这样干，这种方式的时间复杂度减少不了，因此排序的代价是巨大的，除非有一天计算机的内存不通过 CPU 或很少通过 CPU 就可以自己排序。

寓意：“所有的算法来自生活，而且没有生活那么复杂。”

进一步来看这个问题，100 个人进行羽毛球比赛，最终需要挑选出水平最高的那一个人，最少会比赛 99 场。用这种方式就像擂台比武一样，倒下的人下台，胜利者继续在台上，在实际生活中，水平高的可能会累死，但在计算机中不会。

比赛的次数 99 次是最少的，有没有其他的方法呢？或许可以让两两之间进行较量，高手再两两之间继续较量，最后决出胜负，但是比赛的次数我们发现还是 $N-1$ 次（例如 8 个人，首先 4 场胜出 4 人，然后 2 场胜出 2 人，最后 1 场加起来就是 7 场）。这种方式有一个重要的特征——在两两比赛的时候，只要有足够多的场地，就可以一起比赛，那么总体时间就节约下来了，在计算机中就是“多线程”技术。前者的设计方法由于简单，无法实现这种并行，但是它足够简单，只要 for 循环里面需要处理的内容足够简单，效率依然是很高的。

寓意：即使是多线程技术，在生活中也能找到活生生的例子。

将这个问题再做一点“发散性”扩展：在一堆杂乱无章的数据中，需要求出某个数据从大到小的“排名”情况，你会将所有的数据全部重排序一次吗？当然不会，因为完全重排序通常是 n^2 的时间复杂度。

不过，聪明的同学会给自己找一个理由：完全重排序后的数据是可以反复使用的，如果排序导致时间开销过多，则可以定时做完全排序，外部的请求直接从排序结果中获取即可。这样我们需要忍受数据上的一点小延迟，因为这个排名可能随时在变化，由于是定时排序，所以变化后的数据并不会立即更新到排序后的结果中，或许在没有面临大数据的时候，换个思路会更快捷——在很多时候（只要数据量不是特别大），只需要找出比这个数据大的数据个数就自然知道排名了。这种思维其实是相通的，将上面的思路放在这里使用就是一种“变通”。

►► 1.2.2 从 100 万个数字中找最大的 10 个数字

通过 1.2.1 节的介绍，胖哥认为你应该可以领悟到一些什么——既然在寻找最大数字的时候是保留一个最大值，那么在寻找 10 个最大数字的时候，就可以记录 10 个数字的最小值，或者将这 10 个数字排序也是很轻松的。

循环大数组的每个元素，如果当前数字大于这 10 个数字的最小值，就剔除最小值，将当前数字写进去。如果 10 个数字是有序的，那么可以快速定位数字要写入的位置；如果是无序的，则只需要重新找出最小值即可。如果数字比较随机，那么随着不断的叠加，这个最小值也会越变越大，这 10 个数字需要再次插入的概率就会变小很多，即使是在最坏的情况下，每个数字都要插入到 10 个数字中剔除最小的，也最多是 10×100 万的运算量，而不是 $100 \text{ 万} \times 100 \text{ 万}$ 的运算量。

►► 1.2.3 关于排序，实际场景很重要

胖哥算法学得不太好，在胖哥思维中的很多排序算法，没有什么排序算法是特别好的，或者在最坏的情况下，它们都半斤八两。在前面的例子中，一个学校要排队，每个学生都有自己的大脑（CPU），计算机要整体排序应如何排呢？

实际的场景很重要，这里假如有“杂乱无章”的数据（200 万个数据），两层 for 循环得到一个 n^2 的复杂度（ $200 \text{ 万} \times 200 \text{ 万} = 4 \text{ 万亿}$ ）。胖哥不想用，我们开始分析业务场景。如果我们发现这些数据中有 95% 以上是相对均匀地分布在 1~200 万之间的，而且重复数据极少，那么在这种场景下会如何排序？还会用两层 for 循环去排序吗？

胖哥的第一感觉就是它是有技巧的，胖哥希望先分堆再排序。由于数据较为均匀，所以将它们分成 2002 份甚至于更多，其中 2000 份为 1~200 万之间（每个堆都是一个连续的区间，例如 [1-1000]、[1001-2000]、[2001-3000]...），自然每个区间的数据范围是一个小堆，而且小堆之间的数据是有序的，“<1”的数据和“>200 万”的数据各自再分一个小堆，2002 个堆之间的数据是完全有序的，因此只要它们各自内部有序，就是全局有序。这个过程需要扫描 100 万数据一次，然后在 2002 个堆中找到自己的位置，由于它本身有序，所以采用二分查找算法是可以快速找到位置的（每次匹配范围会缩小 1 倍，在 2002 个范围中寻找，每个数字最坏匹配 11 次（因为 2 的 11 次方是 2048），总体上 $11 \times 100 \text{ 万} = 1100 \text{ 万}$ ）。

此时平均每个堆大概是 1000 条数据（200 万数据放在 2002 个堆中），那么每个堆自己排序的最坏时间复杂度将是 $1000 \times 1000 = 100 \text{ 万}$ ，有 2002 个堆，因此是 $100 \text{ 万} \times 2002 = 20.02 \text{ 亿}$ ，加上分堆的开销是 20.135 亿，计算次数依然很高，但是可以看到，已经比 1 万亿这个数字降低了 500 倍。这就是根据业务场景来做逻辑优化的魅力，读者可以在这个基础上继续深度挖掘业务细节。

回顾：是否觉得这也是生活中的一些场景——先排好班级，每个班级之间就不用排序了，而不是全校上千人一起来排队？它采用的是我们熟悉的分块排序，只是唯一的区别是，分块排序完成后就不需要再进行板块之间的排序了。这再次说明，我们实践中运用的还是“基础的算法”，在实际场景中需要变化，“变化后的招数才是最有攻击力的招数”。

内容扩展：当拆分成多个块以后，板块内部的排序是隔离的，因此各个板块是可以并行排序的，而且无须加锁，如果面对的是超大数据，还可以利用多线程来降低处理时间。将这个问题细化，就是分布式计算的基本原理了。

场景扩展：其实排序还有很多实际场景，远远不止这些，而且有些的确是很复杂的算

法，不过无论多么复杂，总能在实际的生活中找到逻辑背景。在这种前提下，只要深度挖掘业务背景，就可以在基础算法之上进行灵活扩展。例如，有很多个已经排序后的分块，块内都是有序的，从第 1 个块到最后一个块的特征是后一个块的最小值大于前一个块的最小值，你会如何排序呢？

由于算法不是本书的重点，这里胖哥就简单说明一下。题目说明了每个块内部是有序的，且后一个块的最小值大于前一个块的最小值，第一想法自然是用后一个块的最小值，找出前一个块小于等于当前值的列表。由于本身是有序的，所以这样排序下来肯定是有序的，且剩下的数据肯定比当前取出的数据要大。

但是有个问题，剩下的数据就无法保证后一个块的最小值大于前一个块的最小值了，但是我们觉得这样“很爽”，还想要这样的情况继续发生。

在取出第 1 个块的数据后（不论取出几条数据），记录下现在的最小值，并认为是所有块的最小值。进一步，下一个块取完后，得到最小值，如果它仍然大于前一个块的最小值，就用链表方式写在前一个块的后面；若小于前一个块的最小值，就放在前一个块的前面。我们也可以用一个单独的数组来存放每个块当前的最小值（这个长度与块个数一致，因此不会占用太多的空间），数组本身是有序的，相继递推的时候还可以用“二分查找算法快速定位”板块的位置。这样取完一次数据后，就得到了一个新的链表或者在分配的数组中记录了新的板块顺序。总之，现在是一个崭新的顺序，那么在进一步取数的过程中我们又可以轻松完成了，这也是一种分块排序的特殊场景。

说了这么多，胖哥还是在讲这句话：实际场景千变万化，切不可死记硬背。

►► 1.2.4 数据库是怎么找数据的

按照 1.2.3 节中的例子，将一些数据划分为板块，但是由于数据太多，就会造成板块太多或板块内部的数据太多，此时就会有管理板块的板块。数据库通常承载的数据量都上亿（单表大小），甚至十亿以上，它是怎么做的呢？

其实数据库也离不开这些原理，大多数数据库的索引都会用换汤不换药的“B+树”（当然也有使用 Hash 来做索引的），此思想源自对平衡二叉树的一种扩展，而同时也来自基本算法——二分查找算法，只是在场景中也会有些变化，数据库中的分块、索引、排序等概念在这些理论中体现得非常明显。

例如，SQL 操作表与表之间的 join 的时候，如果是已经排序好的两个列表，join 就无须完全的两层 for 循环来操作，那是 $N \times N$ 的时间复杂度，此时有点类似于 1.2.3 节中的扩展例子，介绍如下。

假如 A 表与 B 表 join，到 B 表中查找 A 表的一个数据 100，它从 B 表最小的数据开始找，找到了许多小于 100 的数据，不过都排除在外，不论是否找到 100 这个数据，当去 B 表查找 A 表的下一个数据 101 的时候，就不会再从最小值开始找了，而是从 100 开始，因为数据库知道双方是有序的。随着这个规律的推移，扫描的数据范围会越来越小，而不是像两层 for 循环那样，每一个外部的 for 循环都需要在内存中从第 1 个下标开始扫描。

通常，如果 SQL 没有问题，join 表的个数往往不会太多，我们可以进一步做简单化优化。

►► 1.2.5 Hash 算法的形象概念

其实 Hash 有很多种，有简单 Hash，还有一致性 Hash 等，这里我们就谈谈简单 Hash 算法。关于 Hash 算法，经常会在教材中看到：Hash 桶，快速定位，先计算再定位，并且

号称复杂度为 $O(1)$ 。

胖哥有自己的看法，这样的概念容易让人迷惑 Hash 到底是“神马”东西，而容易认为 $O(1)$ 就是没有时间开销的，Hash 就是绝对定位的，Hash 算法就是无限快等思维都会出现。但实际场景也许并不是这样的。

胖哥不想搞得那么复杂，胖哥认为 Hash 算法有点像图 1-5 所示的那样（这里给大家一个简单的感性认识，在后文相关的专业内容介绍中，胖哥会再次讲到 Hash）。

简单地看，就是如果知道队名，就能知道在哪个范围，然后在范围内查找名称。在前面的一些介绍中已经有所体现，在找班级的时候就是这样找的，只是各自有自己的排序方法——这里按照“队伍”来分布，而前面的例子是按照“班级”来分布的。

在 Java 语言里 Hash 算法通常是按照 hashCode 来分布的，前面提到过，在不同的场景下这个 hashCode 会计算出不同的值——可用对象头部的一个标识符，可用某些对象的属性通过一定计算得到，同一个 hashCode 肯定在同一个分组上，这个分组也就是专业说法上的“桶”，“桶”内部的数据通常是无序的，通常内部用一个链表存放（有的小伙伴会问链表不是有序的吗？其实这里所谓的无序是指链表上存放的数据并没有要求外部写入的数据以先进先出或后进先出的顺序存放，外部程序也不能依赖于遍历的顺序）。

在查找数据时，首先根据 hashCode 找到桶，然后在这个桶内部需要逐个使用 equals() 方法来对比内部的值，而在极度冲突的情况下，许多数据将被放在同一个桶中（通常叫作“热点”），导致一个桶的数据链表会非常长，这样一来，分布在该桶上的查找、遍历等操作都会变得很慢。因此，千万不要因为 Hash 算法的复杂度是 $O(1)$ 就认为它天下无敌了，其实这只是一个相对量级别的说法，实际场景中的开销我们要用数字来说话。

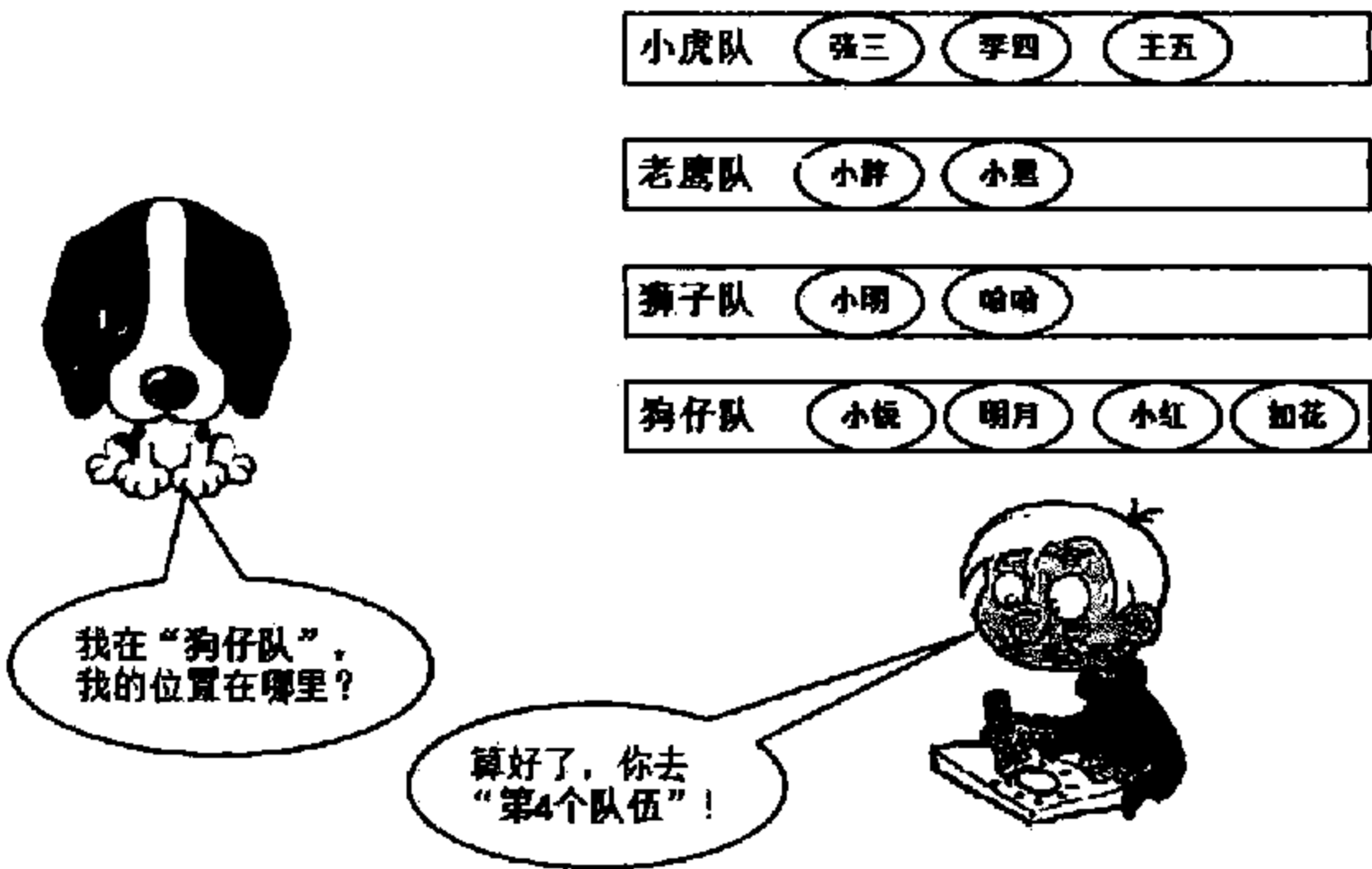


图 1-5 狗狗要找队伍和位置

反过来说，如果要使用 Hash 来做大量数据的查找，那么需要设计合适多的 Hash 桶。另外，在设计 hashCode、hashCode 与桶的下标转换的时候要有足够的离散能力，否则会做“赔了夫人又折兵”的事情。

有人曾经用一种固定 Hash 的方式来存储网络上的 URL，喜欢钻空子的小伙伴利用了这种方式，形成一种网络攻击手段。他们在知道算法的基础上，通过模拟同样 hashCode 的不同 URL，使得某些“桶”内部的数据变得特别多，每一个分布到这个“桶”的请求都会在这个“桶”内部查找，最坏的情况就是一个也没有找到，那么就需要遍历整个链表，这样就可能使得系统性能极度下降。

看到这里，你若有所领悟，胖哥就心满意足了。虽然本节知识会相对枯燥些，但仍然在阐述一些和我们工作密切相关的基础思想。接下来我们再来看看基础知识“运算符”。

1.3 简单数字游戏玩一玩

数字游戏？没错，就是玩数字游戏！

Java 怎么玩？马上见证下！

玩数字有什么用途呢？我们不是虚拟数据给别人看，而是通过玩数字转换，让我们更了解计算机的数字运算，也许数字运算可以有一些神奇的地方，有些变态的问题也不是我们想的那么简单。

这里不讲基本的“四则运算”，胖哥会讲一些运算符，然后再讲讲“大数字”是如何处理的。

1.3.1 变量 A、B 交换有几种方式

胖哥认为有 3 种方法来实现变量交换，其中一种最简单的方法就是定义一个变量 C 作为中间量来实现，代码例子如下：

```
int C = A;
A = B;
B = C;
```

有人开始不想用“定义一个变量 C 作为中间量”来实现，而是尝试用“数据叠加后再减回来”的方法：

```
A = A + B;
B = A - B;
A = A - B;
```

这样也可以实现，A 首先变成 A 与 B 的和，然后减掉 B 自然就得到 A 的值再赋值给 B，此时 B 存储了 A 原先的值，再用 A 减掉 B 就得到原来 B 的值了。不过，这个方法有一个问题，就是 A+B 这个操作有可能会越界，越界后就会出现问題。

既然可能越界，聪明的小伙伴们又想到了另一个办法来解决这个问题，那就是“异或”运算：

```
A = A ^ B;
B = A ^ B;
A = A ^ B;
```

异或运算是按照二进制位进行“异或”的，对于 A、B 两个数字，如果某个二进制位上的 A、B 都是 0 或都是 1，则返回 0；如果一个是 0，一个是 1，则返回 1，这样可以得到一个新的数字。这有什么用呢？当这个数字再与 A 发生“异或”的时候，就能得到 B（这个大家可以自行反推）。这样的技巧其实是二进制位上的一个加法但不进位的做法（由于只有 0、1，所以 1+1=2 后还是为 0），但是这个运算是最低级的 CPU 位运算，所以它的效率是极高的，而且不会越界。

“异或”运算在计算机系统中大量存在，它的一个很大的优势就是可以按照反向的规则还原数据本身。

1.3.2 将无序数据 Hash 到指定的板块

假如有一个长度为 5000 的数组，每个数组下标就像一个 Hash 桶那样存放一个链表，

此时有一个数据 A 需要写入，随机吗？肯定不行，因为我们还要查询数据。那么将它放在 Hash 桶里面可以吗？似乎可以，下面一起来探讨一下。

我们希望写入的数据能按照某种规则读取出来，那么数据应当与数组的下标产生某种关系，这样写入和读取只要按照相同的规则就可以达到目的。此时假设一个“非负数”要写入，或许可以这样做，将这个数据%5000 就可以得到下标了，这样在数字不断变化时，得到的下标也会不断变化，数据自然会相对均匀分布到 5000 个数组中（当然有特殊情况，这里不做探讨）。如果是负数，则需要采用取绝对值 `Math.abs(A) % 5000` 来得到下标。

还有没有其他的方法呢？单纯要将数据写入到数组中，可以使用 `A & 4999` 来完成，这个操作得到的数据将会永远 ≤ 4999 ，因为它自身二进制位为 0 的部分 & 操作后都会被清除为 0。理论上可以实现我们的要求，而且这种位操作也是最低级的系统运算，效率也是极高的。

只是这样的方法会有一个问题：数字 4999 是一个例子，此时我们用数字 11 来说明会更好一些，自然是要用 10 来做 & 操作，10 的二进制位是“1010”，那么意味着 0001、0100、0101 这些数字将永远无法获取到。宏观上讲，如果数组长度是 11，那么 1、8、9 这几个下标永远也没数据。

►► 1.3.3 大量判定“是|否”的操作

如果发现一个业务系统中的许多操作都是在判定是、否，很多时候大家为了节约空间不会为每一个是、否的值用一个单独 int 来存储，因为那样比较浪费空间。而一个 int 是 4 个字节，所以它可以存放 32bit 二进制位，一个二进制位就可以代表一个“是|否”。

这样的设计是没有问题的，而且很多小伙伴们都知道这样的设计，但是胖哥发现有些可爱的同学在设计过后，拿出来判定对应的位置是与否的时候，是先用 `Integer.toBinaryString(int)` 转换为二进制字符串，然后从这个字符串中取出对应位置的 char，再判定这个 char 是'0'还是'1'的操作。

这种操作显然有点过了，而且操作的时间复杂度很高，或许我们可以学一学 Java 提供的一些类中是如何处理的。例如“`java.lang.reflect.Modifier`”，它对类型的修饰符有各种各样的判定（例如，字节码中存储 `public final static` 这样的一长串内容只用一个数字来表达），那么它是如何做到的呢？请听胖哥慢慢道来：Java 将这些符号进行了固定的规格编码，就像商品类型的名称与编号一样（这个编码会在生成 class 文件时存在，JVM 运行时也会使用）。

◎ 是否为 public，使用十六进制数据（0x00000001）表示。

◎ 是否为 static，使用十六进制数据（0x00000008）表示。十六进制的 8 代表二进制的最后 3 位是 100。

◎ 是否为 final，使用十六进制数据（0x00000010）表示。十六进制的 10 代表二进制最后 4 位为 1000。

在 `Modifier` 类中大家可以找到 `private`、`protected`、`synchronized`、`volatile`、`transient`、`native`、`interface`、`abstract`、`strict` 等各种类型的表达数字。

Java 程序在读取字节码时，读取到一个数字，该数字只需要与对应的编码进行“&”（按位求与）操作，根据结果是否为 0 即可得到答案。设计有点小技巧吧！

此时有的小伙伴就会说：“很多时候，需要判定它是不是 `public final static` 的，这么多

类型要一个个判定好麻烦。”胖哥告诉你,这种方式其实还可以打组合拳。组合拳如何打呢?用“或”运算,具体操作就是:

```
final static int PUBLIC_FINAL_STATIC = PUBLIC | FINAL | STATIC;
```

这个值将会事先保存在程序中,由于这三项内容的二进制并不冲突,所以做或运算就代表三者都成立的意思。当传入参数后,只需要与这个值做“按位求与”就能得到结果了。不过,现在“按位求与”的结果不是与 0 比较,而是与 PUBLIC_FINAL_STATIC 比较是否等值(因为需要每个二进制位都要对应上,才能证明它的 3 个特征都成立)。所有的二进制位都必须匹配上,即使一个不匹配也不行,示例如下:

```
public static boolean isPublicFinalStatic(int mod) {
    return (mod & PUBLIC_FINAL_STATIC) == PUBLIC_FINAL_STATIC;
}
```

1.3.4 简单的数据转换

数据转换有两种,其中一种是进制转换,另一种是数字与 byte[] 的等值转换。我们先来看看进制转换。

- ◎ 将十进制数据转换为“二进制的字符串”,使用 Integer.toString()、Long.toString() 来操作。大家注意了,这里说的是二进制的字符串,数字本身就是以二进制形式存储的,在 UI 上要看到这个数字的二进制位,所以才转换成字符串来显示。
- ◎ 将十进制数据转换为“十六进制的字符串”,使用 Integer.toHexString() 来操作。类似的,在 float、double、long 中也有相应的方法存在。
- ◎ 将其他进制的数据转换为十进制数据,使用 Integer.parseInt(String, int)、Integer.valueOf(String, int) 来完成,其中第二个参数传入的就是字符串数据。例如: Integer.valueOf("10", 16) 返回的值就是 16, Integer.parseInt("10", 16) 返回的值是 2,这样就可以实现任意进制之间的转换了。long 也提供了类似的方法。

数字与 byte[] 之间的转换是什么意思呢?大家都知道,在 Java 语言中, int 是由 4 个字节(byte)组成的。在网络上发送数据的时候,都是通过 byte 流来处理的,所以会发送 4 个 byte 的内容,4 个 byte 会由高到低顺序排列发送,接收方反向解析。在 Java 中可以基于 DataOutputStream、DataInputStream 的 writeInt(int) 和 readInt() 来得到正确的数据,代码如图 1-6 所示。

DataOutputStream.writeInt(int) 方法

```
public final void writeInt(int v) throws IOException {
    out.write((v >>> 24) & 0xFF);
    out.write((v >>> 16) & 0xFF);
    out.write((v >>> 8) & 0xFF);
    out.write((v >>> 0) & 0xFF);
    incCount(4);
}
```

DataInputStream.readInt() 方法

```
public final int readInt() throws IOException {
    int ch1 = in.read();
    int ch2 = in.read();
    int ch3 = in.read();
    int ch4 = in.read();
    if ((ch1 | ch2 | ch3 | ch4) < 0)
        throw new EOFException();
    return ((ch1 << 24) + (ch2 << 16) + (ch3 << 8) + (ch4 << 0));
}
```

图 1-6 DataXXStream 对于数字的处理

如果使用了通道相关的技术, ByteBuffer 则由相关的 API 来实现。这是原始的实现方式,如果其他语言提供的 API 希望将这些 byte 位交换位置(例如要求低位先发送),那么你要自己来处理了,而处理方式就可以参看 DataXXStream 的处理方式。

在 DataXXStream 的类中, 不仅仅有对 int 类型的处理, 还有对 boolean、byte、unsigned byte、short、unsigned short、char、int、long、float、unsigned float、double、UTF 的处理, 而且还有一个 readFull() 方法, 这个方法要求读满一个缓冲区后才返回数据, 它会在内部区循环处理。

在 ByteBuffer 的实现类中也提供了各种数据类型的 put、get 操作, 内部也是通过 byte 转换来完成的。

在 java.io.Bits、java.nio.Bits 类中也有大量的数字与 byte[] 的转换操作 (这两个类我们的程序无法直接使用, 但是可以看源码), 大家可以参考它们的代码来实现自己的特定需求。

1.3.5 数字太大, long 都存放不下

long 采用 8 个字节来存放数字, 它连时间的毫秒值、微秒值甚至纳秒值都可以存放下, 它存放不下的数字很少见, 但是它毕竟只有 8 个字节, 如果真的遇到了某些变态的设计怎么办呢? 此时需要使用 BigDecimal 来操作, 它不是以固定长度的字节来存储数据, 而是以对象的方式来管理位的。我们用一个例子来看看使用 BigDecimal 对一个大数字操作包含几个步骤。

- ① 首先将大数字加上 10, 输出结果。
- ② 然后将结果的 byte 位取出来以二进制形式展现。
- ③ 最后根据二进制字符串反转为数字对象。

代码清单 1-4 BigDecimal 测试

```
import java.math.BigDecimal;
import java.math.BigInteger;

public class BigNumberTest {

    private static String lPad(String now ,
                                int expectLength ,
                                char paddingChar) {
        if(now == null || now.length() >= expectLength) {
            return now;
        }
        StringBuilder buf = new StringBuilder(expectLength);
        for(int i = 0 , paddingLength = expectLength - now.length();
            i < paddingLength ; i++) {
            buf.append(paddingChar);
        }
        return buf.append(now).toString();
    }

    public static void main(String []args) {
        //这个数字 long 是放不下的
        BigDecimal bigDecimal
            = new BigDecimal("1233243243243243243243243243243243241432423432");
        System.out.println("数字的原始值是: " + bigDecimal);

        //bigDecimal = bigDecimal.add(BigDecimal.TEN);
        //System.out.println("添加 10 以后: " + bigDecimal);

        //二进制数字
        byte[] bytes = bigDecimal.toBigInteger().toByteArray();
        for(byte b : bytes) {
```

```
String bitString = lPad(Integer.toBinaryString(b & 0xff) ,
                        8 , '0');
System.out.println(bitString);
}
//还原结果
BigInteger bigInteger = new BigInteger(bytes);
System.out.println("还原结果为: " + bigInteger);
}
}
```

在这段代码中，有一个 substring()、lPad()操作。这是因为 Integer.toBinaryString(b)传入的虽然是 byte，但是会转型为 int，如果 byte 的第 1 个 bit 位是 1，则代表是负数，那么转换为 int 的高 24 位将会填充 1。其实我们不需要这 24 位，所以用了 substring()。如果是正数，那么输出的字符串会将前面的 0 去掉，为了在显示上使用 8 位二进制对齐方式，所以在代码中用了 lPad()。我们再来看看输出结果。

```
数字的原始值是: 1233243243243243243243243243243243243241432423432
添加 10 以后: 1233243243243243243243243243243243243241432423442
00110111
01001100
11110000
00101001
11111111
10001010
00010101
00001101
00011100
01111111
00101001
00000001
01000111
01000110
10110011
01111000
01100100
00011100
00001000
还原结果为: 1233243243243243243243243243243243243241432423442
```

数字能转换为二进制数据，又能还原成数字，理论上应该没有问题，大家也可以用一些比较小的数字做测试来印证这个结论（例如 2、15、16、1024 等）。不过，在印证的过程中，有的同学发现高位出现整个字节的 8 位全是 0 的情况，例如 128，输出的结果是两个字节的二进制字符串：0000000011111111。大家很疑惑：既然高 8 位全是 0，为何还要单独拿一个 byte 来存放呢？有一点要注意了，因为 128 是正数，而一个字节中的最高位变成了 1，如果直接用 11111111 来表达就表示它是一个负数（具体值是-1），所以需要多一个字节来表达自己是正数。

long 的存储能力到底有多大？

许多同学设计一些 PK 列的时候认为数字存储不下，用字符串来存放数字（当然不排除某种编码规则），但是用数字往往计算快速得多，而且空间也少很多。为了了解 long 的存储能力有多大，我们先看看 int 的存储能力有多大。Integer.MAX_VALUE 的值为“2147483647”，也就是 2G-1 的大小。如何得来？自然是 4 字节对应 32 位，然后去掉负数和 0 得来的。这个值换算成十进制数据就是 21 亿，理论上很多表达不到那么大，或者说绝大部分表不会有这么多 ID，但是若某些跳跃和历史数据超过这个数字怎

么办？那么就用 long 吧，它的宽度是 int 的 2 倍，即 $4G \times 4G / 2 - 1$ （为什么？自己想想）。十六进制数据 `0x7fffffffffffffffL`，换算为十进制数据是“9223372036854775807”，这个数字看不出到底有多大，我们可以和时间进行比较。

$\text{Long.MAX_VALUE} / (3651 \times 24 \times 60 \times 60 \times 1000) = 292471208$

$\text{Long.MAX_VALUE} / (3651 \times 24 \times 60 \times 60 \times 1000 \times 1000) = 292471$

$\text{Long.MAX_VALUE} / (3651 \times 24 \times 60 \times 60 \times 1000 \times 1000000) = 292$

如果存放毫秒值，则可以存放 2.92 亿年，微妙值可以存放 29.2 万年，纳秒值可以存放 292 年的数据。换句话说，一个表的 ID，即使加上跳跃每秒有 $10E9$ 个 ID 自动增长，也可以增加 292 年的数据不重复，胖哥认为这种并发在现在的计算机时代还不存在。小伙伴们，通过这种量化后，自然应该理解某些设计应该如何做了吧。

数字游戏玩到这里就结束了，在本书的后文中，胖哥还会提到一些和数字相关的小玩意，例如 `i++` 与 `++i` 的问题，或许胖哥的说法与在教材中看到的解释不一样哦。



1.4 功底概述

本节是本章的“道”第一次总结，胖哥会尽量简单说明。

1.4.1 什么是功底

古人有句话：“心有灵犀一点通”，形容相互之间的交流十分默契，而融洽的基础在于彼此十分了解对方。在专业领域上要做到这一点，就是要深深地了解技术以及它的内在。

武侠中有一种武学叫“九阳神功”，也有一种武学叫“易筋经”等，它们之所以很神奇，是因为它们都有一种不变的特征，就是学习了这些武功后，再学习任何武学都很快，而且可以比别人学得更好。同时，这些“武学秘籍”也有一个共同的思想，就是以内功修为为主，而不是以招式为主。

胖哥写的书自然不敢与这些“神”一样的“秘籍”相提并论，胖哥只是借此告诉大家内在修为将会决定你在技术这条路上能走多远。

胖哥无法帮助你成为“张无忌”一样的绝世高手，但是一个真正的老 A，也同样需要有很深厚的内功修养；否则，当新的事物出现时，你会跟不上潮流，“out”了，正所谓：练武不练功，到老一场空。

在功底方面，我们还需要做到知其然并知其所以然，形成一种由知识引导思维，由思维引导答案，由结果印证理论，不断迭代的过程。在这个过程中，对问题的认识会越来越清晰，自然在见解上也会越来越有道理，同时也会对自己所在的领域充满自信。

1.4.2 功底有何用途

第 1 节讲解了 String 的故事，第 2 节讲解了关于一些算法的故事，第 3 节我们开始玩数字游戏，是否觉得自己用了很久的 Java，好多东西还不知道？当然，如果你认为胖哥说得过于小儿科，那么就跳过去吧。

在前面的这些知识中，很多技术本质源于生活，很多意想不到的事情发生了，这一切在本书中才刚刚开始，或许后续章节中的一些内容会更加让你感觉出乎意料。这些原本认为是真理的内容，被彻底颠覆，在实际的场景中也许就在不经意之间，你会留下一个很难

琢磨的 Bug，或者根本不会认为是 Bug 的 Bug，然后让别人来解决这个 Bug。正所谓：我们不怕犯错，怕的是不知道什么是错，更怕的是我们一直坚信正确的事竟然是错的。

很多时候，印证了一个通过测试得到的经验并不那么靠谱，胖哥的例子也许已经令你感受到测试场景可能会决定许多不同的结果。在不知道本质的时候，仅仅通过某些特定的测试场景是不能作为真理来指导开发的，最多只能是指导同样场景下的开发。测试通常可以帮助我们理解本质，以及在知道本质后它是一个印证的过程。

知道了内在，就像知道了 String 的编译优化、String 的常量池、String 的内存结构，知道了常规的算法，知道了生活中的对应方法，我们发现它并不是那么难，关键是你是否愿意去看它的本质。其实看本质对于许多程序员来讲是一件拥有快感的事情，因为生活的现实世界在自己的工作中找到了灵感，但是它又不像人性那么复杂，它比生活更加简单。

只是我们过于看中技术本身，过分仰望技术本身，“容易陷入深渊，而难以自拔”，甚至有人喜欢钻牛角尖，对于这样的同学来讲，胖哥只能说“苦海无涯”，你需放下才能找到新的方向，需解脱和放下才能超越自我。

总的来讲，胖哥认为：只有不断颠覆自我，找出本质，才能将一些问题连根拔起。而不是过分相信自己曾经做过的实验和别人说过的所谓真理，这些都仅仅作为参考而已。

►► 1.4.3 如何磨练功底

功底并非天高，任何人只要愿意，能静心，就都行！

大多数初学者，对能做出一个例子，或做非常多的例子引以为傲，其实这并不是坏事，只是学习后一定要“落实于根本，回顾与总结”，切勿让“猴子掰玉米”的故事发生在你的身上。换句话说，浮躁的心态是“猴急”，什么都想要去学习，结果学了就丢了，没有自己知识的凝固，没有自己的总结和笔记。

首先，要能静心，静心才能落实，立足于当下，要知道学海无涯、浩瀚无边，并且社会在发展，知识在进步，我们的确需要不断学习，但是要从当下做起。

当我们静下心来，就可以开始总结了，总结啥呢？感想吗？也可以，不过老这么总结就不像 IT 从业者了，而像一个感情丰富的编辑。我们刚开始做总结有点像记笔记，就是将老师所讲，或自己所学记录下来，按照条目一条条地陈列。但是这不算总结，只能算是笔记，这些内容在许多的书籍和网站上可以找到更好的内容，而总结应当包含自己的理解和感受，应当能阐述学习和理解过程中的痛苦。

在技术领域你可以总结思路，总结所学所用，总结所遇到的问题，总结为什么，总结解决问题的手段和方法，分析问题和定位问题的思路，总结在解决问题时所用的知识，总结技术点上是否还有相关的方案，以及对比相关知识的优缺点。

在业务上总结自己最近做过些什么，那些是大事还是小事，自己所做的事情和几个月前比提高没有，效率上提升没有，时间安排如何，如果加班太多能否不加班，自己的工作效率是否可以进一步提升，以及如何提升，通过技术完成某些自动化还是基于时间管理来更合理地安排自己，未来几个月准备或希望去做什么样的事情，是否有能力去参与更重要的职责。



图 1-7 “一休”同学已静下心来，在思考

当然，我们不是每天都要去做总结，那样会很浪费时间。

即使每天去做总结，也是一个小总结而已，如果我们非常忙，那就没必要这样做。但是当我们看到一篇比较好的文章时，可以先初步看看内容，把它记录下来，写到总结中表示想要去学习的内容，等到闲暇之余去看看，再进一步来总结或许会更好。

本节我们用一个简单 String 等值对，就能说出一大堆基础知识，也能说出类似的方法，相应的算法也能联想出来，你不觉得这样的由小的点入手引导相关知识的方法也是一种不错的学习方法吗？

工作与学习本身并不矛盾，工作本身就是一种学习，只是它更偏重于实践，基于工作驱动学习一个新东西会更快一些，不过大多在使用层面。当遇上了稀奇古怪的问题时，就想要去看看内在的原理了，此时去看看源码，你会更加有动力。这样的碎片化学习方式，其实也是一个积土成山的过程，达到一定程度自然是“量变发生质变”，那个时候再去看看“牛人的秘籍”，就是一种交流，一种知识梳理的过程。

也许刚开始我们无法做到很深入的细化与知识联想，尤其是某些相关技术点需要很大的知识面才能把它讲清楚，技术需要理论，理论又需要技术，这是一个死循环，我们便开始产生了“鸡与蛋”的纠结问题，许多人在长期的纠结中选择了原地踏步，也就是相当于放弃了。其实我们需要的还是立足于当下业务，尽自己所能去挖掘，通过业务细化驱动技术的发展，尽我们所能广泛联系知识，整个学习过程是一个迭代的过程，功底是逐步提升的，不要纠结于死循环的问题。

如果你希望自己从事技术这个领域，就要学会相信自己，拥有一定的自信，有信心不是说自负，而是有信心去面对困难。当你有一定成就的时候，不可狂傲，要知道天外有天，人外有人，世界上没有最高，只有更高，即使是今天的最高，明天也会有更高出现。而我们的功底将支撑自己的这座山峰，我们更加愿意去追求自己的极限，追求自己理想的巅峰。

1.5 功底补充

看完 1.4 节，发现胖哥废话很多，貌似没啥干货了！

为了不让大家认为功底只有 String 那么一点点东西，胖哥就再增加对原生态类型、集合类的说明，这两方面的内容相信所有的 Java 开发者都必然会用到。

►► 1.5.1 原生态类型

原生态类型是“神马”？

原生态类型就是 Java 中不属于对象的那 5% 部分。

那到底是哪些东西呢？

包含：boolean、byte、short、char、int、float、long、double 这 8 种常见的数据类型(Primitive)。

好麻烦，为啥会用它们呢？用对象不可以吗？

计算机中的运算基础都来源于简单数字，包括 Java，即使是包装后的对象(Wrapper)，在真正计算的时候也是通过内在的数字来完成的，Java 失去了它们，就好比鱼儿失去了水一样，失去了生命力。

它与包装后的对象有什么区别呢？

包装后的对象会按照对象的规则存储在堆中（例如，int 所对应的就是 Integer 类的对

象), 而“线程栈”上只存储引用地址。对象自然会占用相对较大的空间存放在堆中, 在原生类型中, “栈”上直接保存了它们的值, 而不是引用 (Reference)。

下面看个例子。

代码清单 1-5 一个 Integer 的简单测试

```
public static void main(String []args) {
    Integer a = 1;
    Integer b = 1;
    Integer c = 200;
    Integer d = 200;
    System.out.println(a == b);
    System.out.println(c == d);
}
```

输出结果:

```
true
false
```



注意 这个结果在较低版本的 JDK 当中不会出现。

现在胖哥来解释一下这个结果。

在编译阶段, 若将原始类型 `int` 赋值给 `Integer` 类型, 就会将原始类型自动编译为 `Integer.valueOf(int)`; 如果将 `Integer` 类型赋值给 `int` 类型, 则会自动转换调用 `intValue()` 方法, 如果 `Integer` 对象为空, 这时会在自动拆箱的时候抛空指针 (这个自动转换可以通过后文中介绍 `javap` 命令的方法来证明)。

这些赋值操作可能不是那么明显, 例如一些集合类的写入、一些对比操作, 这就需要我们知道什么时候会自动拆装箱。换句话说, 它简化了代码, 但是并不是让我们一无所知。

即使是这样, 两个结果也应该一样, 要么都是 `true`, 要么都是 `false`, 但为何不一样呢? 这算是一个 Java API 的坑, 如果我们不知道这些坑, 稍微不留神就会掉进去。知道了装箱是 `Integer.valueOf(int)` 方法, 那么就来看看 `Integer.valueOf(int)` 方法的源码, 如图 1-8 所示。

```
public static Integer valueOf(int i) {
    if(i >= -128 && i <= IntegerCache.high)
        return IntegerCache.cache[i + 128];
    else
        return new Integer(i);
}
```

图 1-8 `Integer.valueOf(int)` 方法的源码截图

根据代码可以看出, 当传入 `i` 的值在 `[-128, IntegerCache.high]` 区间的时候, 会直接读取 `IntegerCache.cache` 这个数组中的值。

在代码中为什么使用 `i + 128` 作为数组的下标呢?
因为数组下标是从 0 开始的, 而表示的数字范围是从 -128 开始的, 加上 128 正好对上了。

继续跟踪源码可以得到，在默认情况下 `IntegerCache.high` 是 127。也就是说，如果传入的 `int` 值是 -128~127 之间的数字，那么通过 `Integer.valueOf(int)` 得到的对象是被 `cache` 的，自然的，对于同一个数字 `cache` 的对象是同一块内存地址，所以第 1 个输出结果是 `true`。第 2 个输出已经不在这个范围，因此会重新 `new Integer(int)`（创建一个新对象），所以得到的结果是 `false`。

我们可以通过设置 JVM 启动参数 `-Djava.lang.Integer.IntegerCache.high=200` 来间接设置 `IntegerCache.high` 值，也可以通过设置参数 `-XX:AutoBoxCacheMax` 来达到目的（这个不用查官方资料，看看源码以及源码周边的注释就懂了）。如果要将这个值变得更大来满足自己的需求，则可以在启动参数中增加该值（缩小也是一样的道理）。

这好像是做好事，将我们的数据 `cache` 起来，更加节约空间了，但是有的同学开始认为 `Integer` 可以用“==”匹配了，因为大家自己“测试”的时候发现 1、2、3、4 等数据都是没有问题的，但是程序发布后出现了诡异的问题，而这个最不容易被认为是问题的地方却真的成了问题。而 `Java API` 中没有明确地说明这一点，但开发人员不会将官方文档都学习一遍再来做开发吧，所以我们说它是“坑”。

有人问：真正的场景中会这样吗？

胖哥认为：肯定会，而且你遇不到的场景并不代表不会发生，今天遇不到的事情并不代表明天不会发生。例如，在某些工程设计中，某些状态值有特殊的意义，如果它们是非连续排布的，那么不在 -128~127 范围内的可能性是肯定存在的。

这个例子很简单，我们学到的应该不止这些，因为坑无处不在，我们要学会看源码和本质；否则，即使是 `Java` 本身提供的 `API` 出现了“坑”，也会让我们“防不胜防”，在技术面前变得十分“可怜”。

我们可以说这个 `API` 写得不好，没有说明详细的使用情况，但是一个老 A 不应当被“武器”所玩弄，而是要驾驭武器，老 A 即使拿一把普通步枪也同样能战胜拿着“狙击步枪”的普通士兵，因为他们除了拥有极强的战斗素质外，还深知武器的脾气与秉性，这是人与武器之间的驾驭和被驾驭关系。

也许自动拆装箱还有另一个很大的“坑”，就是如果大家不知道自动拆装箱是怎么完成的，可能就会有更多的问题发生，在程序中传递参数可能一会用 `Integer` 类型，一会用 `int` 类型，自然的就一会在做拆箱操作，一会在做装箱操作，这貌似没有太大的问题，但每次装箱的时候都有可能会创建一个对象（因为很多时候数字不一定在 `cache` 范围内，较低版本的 `JDK` 是没有 `cache` 的）。另外，这种装箱操作是很隐藏的。例如，我们想要用一个 `int` 类型的数字来作为 `HashMap` 的 `Key`，那么在 `put()` 操作的时候就会自动发生装箱操作（因为 `Key` 被认为是 `Object` 的，`HashMap` 需要获取这个对象的 `hashCode()` 方法来做离散规则，所以它会自动转型为 `Integer`）。同样的，如果想将许多基本类型的数据放在 `List` 里面，在 `add()` 操作的时候也会自动发生装箱操作。此时，如果数据取出来后变成了基本类型，再用这个基本类型放入另一个集合类，就又会发生装箱操作，在这个过程中就会隐藏地浪费大量的空间，而自己却什么也不知道。

关于对象空间的大小，请参看第 3 章的内容。

□ 横向扩展

通过对 `Integer` 的一些了解，想到了 `Boolean`、`Byte`、`Short`、`Long`、`Float`、`Double`，它们是否有同样的情况，胖哥不想写重复的东西，直接给出结果，大家可以自己去看看代码，

看看这些类型中的 `valueOf()` 方法是如何操作的，或者说自动装箱是如何操作的。

- ◎ Boolean 的两个值 `true` 和 `false` 都是 `cache` 在内存中的，无须做任何改造，自己 `new Boolean` 是另外一块空间。
- ◎ Byte 的 256 个值全部 `cache` 在内存中，和自己 `new Byte` 操作得到的对象不是一块空间。
- ◎ Short、Long 两种类型的 `cache` 范围为 -128~127，无法调整最大尺寸，即没有设置，代码中完全写死，如果要扩展，需自己来做。
- ◎ Float、Double 没有 `cache`，要在实际场景中 `cache` 需自己操作，例如，在做图纸尺寸时可以将每种常见尺寸记录在内存中。

□ 思维发散扩展

如果上面的操作变成 `Integer` 与 `int` 类型比较会是什么样的结果呢？如果是两个 `Integer` 数据做 “>”、“>=”、“<”、“<=” 比较，做 `switch case` 操作，会得到什么结果？反射的时候是否有特殊性？

这个结果大家可以去论证，且测试结果可以就认为是当前虚拟机的设计规范。下面胖哥直接给出结果。

- ◎ 当 `Integer` 与 `int` 类型进行比较的时候，会将 `Integer` 转化为 `int` 类型来比较（也就是通过调用 `intValue()` 方法返回数字），直接比较数字，在这种情况下是不会出现例子中的问题的。
- ◎ `Integer` 做 “>”、“>=”、“<”、“<=” 比较的时候，`Integer` 会自动拆箱，就是比较它们的数字值。
- ◎ `switch case` 为选择语句，匹配的时候不会用 `equals()`，而是直接用 “==”。而在 `switch case` 语句中，语法层面 `case` 部分是不能写 `Integer` 对象的，只能是普通的数字，如果是普通的数字就会将传入的 `Integer` 自动拆箱，所以它也不会出现例子中的情况。

在 JDK 1.7 中，支持对 `String` 对象的 `switch case` 操作，这其实是语法糖，在编译后的字节码中依然是 `if else` 语句，并且是通过 `equals()` 实现的。

- ◎ 在反射当中，对于 `Integer` 属性不能使用 `field.setInt()` 和 `field.getInt()` 操作。在本书的 `src/chapter01/AutoBoxReflect.java` 中用例子来说明。

➤➤ 1.5.2 集合类

如果读了上一节后你有所体悟，那么胖哥认为你可以跳过此节，因为此节知识为上一节的一个平行扩展，我们不重视知识点本身，而在于让大家了解到许多秘密。

集合类非常多，从早期的 `java.util` 的普通集合类，到现在增加的 `java.util.concurrent` 包下面的许多并发集合类，其实我们有些时候只是知道它们是很好的东西，但在遇到某些问题的时候是否会想到是它们造成的（就像 `String` 一样），它们的使用技巧有哪些？它们的设计思想是什么？

本节不讨论并发包，就简单说说集合类的故事。

疑惑：集合类中包含了 `List`、`Map`、`Set` 几大类基本接口，而我们最常用、最简单的集合类是什么呢？

答曰：`ArrayList`、`HashMap`。

那么，当你用 ArrayList 的时候是否想起了 LinkedList、Vector；当你用 HashMap 的时候是否想起了 TreeMap、HashSet、HashTable；当你要排序的时候是否想起了 SortedSet 等。

它们有何区别？在什么情况下使用？

在讨论 String 后面的部分内容中，我们提到了 StringBuilder，它内在的数组的实现有大量的拷贝，这在集合类的内存拷贝方面的体现更加明显，并且占用空间更大。

占用更大空间的原因是集合类都是存储对象的引用的，在 32bit 及 64bit 压缩模式下，一个引用会占用 4 个字节，在 64bit 非压缩模式下会占用 8 个字节，而 StringBuilder 只是存储 char 字符的数组，每个位只占用 2 个字节。

此时以 ArrayList 为例，我们看看它的 add(E e)方法源码，如图 1-9 所示。

```
public boolean add(E e) {
    ensureCapacity(size + 1);
    elementData[size++] = e;
    return true;
}

public void ensureCapacity(int minCapacity) {
    modCount++;
    int oldCapacity = elementData.length;
    if (minCapacity > oldCapacity) {
        Object oldData[] = elementData;
        int newCapacity = (oldCapacity * 3) / 2 + 1;
        if (newCapacity < minCapacity)
            newCapacity = minCapacity;
        // minCapacity is usually close to size, so this is a win:
        elementData = Arrays.copyOf(elementData, newCapacity);
    }
}
```

图 1-9 ArrayList 的 add(E e)方法源码截图

通过源码我们发现，如果空间不够，会通过 Arrays.copyOf 创建一个新的内存空间，新空间的大小最小为原始空间的 3/2 倍+1，并将原始空间的内容拷贝进去。

这里所提到的新空间的大小为原始空间的 3/2 倍+1，是最小的，在 add(E)方法中不会发生，而在 addAll()方法中会发生。addAll()允许同时写入多个数据，如果写入的数据较多，每次按照 1.5 倍数扩容，可能发生多次扩容，这样就会有冗余的垃圾空间产生，addAll()操作就会对比写入的量与 1.5 倍的大小，谁大就用谁，这个道理在 StringBuilder 中我们就知道，因此文中提到的是“最小”。

我们知道，ArrayList 是基于“数组”来实现的（本书 3.5 节会详细介绍内存结构），如果遇到 remove()操作，add(int index)指定的位置写入操作，我们有没有虑过 ArrayList 内部其实会移动相关的数据，而且随着数组越长，移动的数据会越多。如果要替换一个数据，我们会不会先 remove 再 add 一个数据，或者通过 set(int index, E e)将对应下标的数据替换掉。

基于数组的 ArrayList 是非常适合于基于下标访问的，这是它擅长的地方（又回到基本的数据结构与算法了）。下面胖哥给出几个简单扩展，希望大家去思考。

◎ 在经常做修改操作的列表中，或者在数组通过下标检索并不是那么多的情况下，你是否考虑过使用 LinkedList 呢？因为 ArrayList 通常始终有些数组元素是空着的。

◎ 在知道 List 长度范围的情况下，你是否在实例化 ArrayList 的时候带上长度？例如

`new ArrayList(128);` 这样就降低了内存碎片和内存拷贝的次数。

◎ 当 List 太大的时候是否考虑过对它做分段处理, 而不要一次加载到内存中? 其实很多 OOM 都会在集合类中找到问题。

◎ 常见的框架中用了什么集合类? 在什么情况下也会出现问题?

大家熟知的 HashMap 浪费空间更加严重, 它的代码里面有一个 0.75 因子, 当写入 HashMap 的数据个数 (不是说所使用的数组下标个数, 而是所有元素个数, 也就是说, 包含了同一个下标的链表中的所有元素个数) 达到数组长度的 0.75 后, 数组会自动扩展 1 倍, 并且还需要做一个 rehash 操作, 其实这个时候也许很多桶上的节点都是空的。

胖哥不想扯太多的集合类出来, 把读者“读晕”, 大家在理解这两个基本的集合类基础上, 再去看其他的集合类也许会简单一点。胖哥只想让你知道这些东西是可选择的, 在什么时候去选择, 如何去选择完全要看你自己的功底, 不论是做基础程序、做功底还是去做优化, 都需要深知它的细节, 才能做到心中有数。

1.6 常见的目录与工具包

很多做 Java 开发的同学们, 在达到一定程度后, 开始“身手不凡”, 成为大侠, 在了解了底层后, 开始自己写东西。这个阶段容易纠结的就是重复制造, 在了解了底层后我们需要提升知识面, 知道哪些是别人提供的, 哪些是需要我们自己写的。

Java 的三方包无穷无尽, 无法一一列举这些工具包, 但是我们应当知道有许多别人写好的工具包可以直接拿来使用, 以及如何来寻找这些工具包, 基础知识可以帮助我们快速学习新的工具包, 以及掌握它的本质。同时, 这里的内容也算是为“源码篇”做一个简单铺垫。

首先, 要知道 Java 本身提供的一些源码放在哪里。一般来讲, 安装完 JDK 后, 在 JDK 的根目录下会有一个叫“src.zip”的压缩包, 解压后是一个目录, 其中包含了常见的 Java 源码, 但是以 sun 开头的文章是找不到的, 你可以通过一些反编译手段得到, 也可以在 OpenJDK 上找到一些源码, 主要目的是看懂意思和知道它的坑。

有工具后, 从哪里开始看起? 从我们用到的 API 看起, 当你需要用到某个 API 的时候, 就可以查看是否有相关的 API, 简单记录下来。看看是否有更好的方式, 要有个大致笔记。例如, 对于 List 的简单使用, 小胖哥问几个小问题。

◎ 将 ArrayList 转换为 LinkedList 用什么方法? 各自的好坏?

◎ 将集合类、数组做一次浅拷贝用什么方法? 用 for 循环还是别的方法?

◎ 对 List、数组做排序用什么方法?

其实在对集合类、数组操作上, 有一些 Java 本身提供的工具类 (静态工具方法), 分别位于 `java.util.Collections`、`java.util.Arrays` 类中, 它们拥有非常多的对集合类和数组的操作动作, 足以满足绝大部分需求。下面通过两个场景给大家一点感性认识。

场景 1: 通过常量构造一个 ArrayList 返回。

最直接的写法是:

```
List<String>list = new ArrayList<String>();
list.add("a");
list.add("b");
list.add("c");
```


用工具就可以这样写：

```
List<String> list = Arrays.asList("a" , "b" , "c");
```

或许你认为就是语法糖而已，而 Java 本身要的就是简单，我们希望将更多的精力花在实际创造上，这样的代码可能会在系统中反反复复出现，其实很多时候是枯燥无味的。

场景 2：中文拼音排序。

代码片段 1-6 一个简单的中文拼音排序

```
public class SampleChineseSort {

    @SuppressWarnings("rawtypes")
    private final static Comparator CHINA_COMPARE
        = Collator.getInstance(java.util.Locale.CHINA);

    public static void main(String []args) {
        sortArray();
        sortList();
    }

    private static void sortList() {
        List<String>list = Arrays.asList("张三", "李四", "王五");
        Collections.sort(list , CHINA_COMPARE);
        for(String str : list) {
            System.out.println(str);
        }
    }

    private static void sortArray() {
        String[] arr = {"张三", "李四", "王五"};
        Arrays.sort(arr, CHINA_COMPARE);
        for(String str : arr) {
            System.out.println(str);
        }
    }
}
```

关于排序不仅仅如此，有些时候我们还需要用对象排序，对象怎么排序呢？当然是基于对象内部某些自定义的属性来排序了。而不论用什么来做 Java 排序，都需要使用数字来排序，也就是要有大小关系。Java 提供了 Comparable 和 Comparator 两种接口（两种接口是分开用的），Comparable 接口需要让列表中的对象来实现接口中的方法 compareTo(E)，返回正数表示当前对象比传入对象大，当前对象会排序靠后；返回 0 表示等值，返回负数表示当前对象比传入对象小，排序靠前。也就是你告诉 Java 对象之间的大小关系（而谁大谁小是你自己决定的），Java 就会从小到大排列；如果想要反向排序，那么就将返回值“取反”；如果想要按照多个字段排序，在这个方法内部也是可以完成的。

大家可能发现这样的方式不是太灵活，因为一个对象实现接口后，这个方法就固定了。也就是说，它的排序算法已经固定了，如果它的排序算法不是固定的，是可以动态调整的，那么就用 Comparator 接口来扩展，它独立于被排序的对象单独存在，当需要排序的时候，以参数的形式传递，上面例子中的“CHINA_COMPARE”就是一个 Comparator 实例。你也可以自己实现一个自定义对象的排序方式来满足特定对象的要求，如果同样的对象一会想这样排序，一会想那样排序，就只需要使用不同的 Comparator 实例即可。

这是一种基本的封装思路，Java 就是希望让开发者关注更少的事情，它帮你去完成排序。这种思路也衍生到 SortedSet 中的排序，也在 Timer、ScheduleThreadPool 调度任务中使用（确切说是内部包装的，在 PriorityQueue 中使用，5.6 节会详细阐述）。关于这些内容，可以参考胖哥的个人博客：<http://blog.csdn.net/xieyuooo/article/details/8611198>。

例子举不完，Java 本身提供的工具非常多，而且每个版本都会有新的工具包出现。除此之外，还有许多的二方包、三方包都提供了大量的工具类，这里就不再逐个举例了。Java 是希望将一些复杂的逻辑细节封装在工具中，让业务代码尽量干净、整洁，容易维护。反之，如果业务代码中出现了大量的排序、字符串处理、日期处理、类型转换、数组或集合类循环拷贝与组装、文件处理、网络 IO 等片段，我们就会感觉代码“不干净”，这样的代码会在许多地方出现，这也意味着如果需要修改则要到许多地方去完成，而很多时候我们不知道到哪里去修改。这些代码会让我们的思路不断在技术和业务之间切换，没法专心做好业务细节，更没法深度挖掘业务。技术本身是在业务驱动下才能发挥作用的，而人在业务驱动下去学习技术是能得到最佳实践的。

例如，Apache 就提供了许多有用的三方包给我们使用，很多人都应该熟知 StringUtils 这个工具包吧，类似的还有很多，如 upload、连接池、log4j、字符集处理等都可以算是工具包，这些工具包提供了大量 API，大多数情况下无须自己去实现处理细节，因为它们的正确性和性能是经过很多公司验证的，如果有问题大家都会知道。所以，我们在注重功底的基础上也需要大量的学习，铺开知识面，才能有选择，但“深知内在细节是我们量化选择的条件”。

当然，不是所有的工具包都能完全满足我们的需求，而往往“最烦人”的就是这个工具包满足一部分需求，那个工具包满足一部分需求，而将这些工具包组合起来使用可能比自己写一个还麻烦，翻译为基础代码还会有一大堆的冗余，此时就可以考虑扩展了。

每个场景都会有一些变化，开源的工具仅仅是提供一些通用的处理，同时提供一种 Java 封装思想，许多代码也值得我们去参考。因此当遇到个性化问题的时候，并非别人没提供自己就不写了，个性化包装就是一个优秀程序员需要去承担的责任，所以老 A 程序员除了拥有非凡的功力外，也同时需要深知所涉及领域的业务和个性化，这样才能针对业务做出一个“很爽”的架构体系。

胖哥说话好像在“绕圈”，怎么又绕回功底来了？其实胖哥此处再谈功底，是要说明在这个基础上根据实际的场景应当“因地制宜”才能“事半功倍”，同时要以大量的知识面为基础，充分利用现有的资源。

1.7 面对技术，我们纠结的那些事儿

人生需要面对很多纠结，我们都是在纠结中磨练自我意志的。

纠结容易让人浮躁，容易让人犯错；但是纠结同样会让人成长，纠结是黎明前的黑暗。

学会将纠结化作成长的力量，在逆境中能生存的强者才是真正的“老 A 级程序员”。

1.7.1 为什么我这里好用，哪里不好用

“为什么我这里好用？哪里不好用？”

小鬼，你是不是经常说这句话？如果是，那你“中标”了。

你怎么知道我经常这么说呀？

这句话要么是老鸟说，要么是小鸟说，同一句话，老鸟说的意思和小鸟的得意思完全不一样哦！

有啥不一样呢？

小鸟通常是碰到问题就说这句话，而老鸟通常是解决问题后才说这句话；小鸟说这句话是纠结，继续跟自己过不去；老鸟说这句话是为了搞懂本质和原理，整合事实与理论，证明自己的结论，总结自己的知识。

小鸟还是不懂这是为什么，好可怜。

小鸟啊小鸟，你想想，你还不会飞，还没有走出你的窝，怎么能看清楚这个世界，这个世界上万物生灵的成长？你提出的为什么只是虚幻，没碰过的。老鸟已经在天空中翱翔数年，它们拥有自己的生存与自然界经验，深知自然界的生命循环与弱肉强食的道理，知道什么是好与坏，知道什么是美与丑，知道什么是天敌与猎物。你若想要了解这个世界，“努力锻炼自己，拥有一双可以翱翔的翅膀，去感悟这个世界吧”。

可能从一只小鸟成长起来会经历沉默，然而不在沉默中爆发，就会在沉默中灭亡。当你成长为老 A 的时候，你应该知道的是在解决问题后才会提出相关性问题的，而不是在解决问题前，所以真正的一个老 A、一只老鸟，可以说这样的话。

反过来讲，要提出问题，也需要有点水平才能真正切入主题将问题说清楚，有的小鸟喜欢占主动权，问问题的时候想要牵着老鸟的鼻子走，老鸟会问小鸟想知道的问题细节，但小鸟继续说自己想说的，结果是老鸟不知道小鸟要什么，也搞不清楚问题是什么。老鸟逼于无奈而且自己还有事情，最后就装着什么也不知道，小鸟就经常会说老鸟有什么用啊，什么问题都搞不定，当自己发现低级错误的时候，还可能得意地想：“不就是一个低级错误吗，老鸟竟然解决不了。”

无论如何我们离不开的是实际场景，要根据实际的工作背景来解决问题，这样才能达到较好的效果，同时在一些代码 bug 的修复上，可能还会分析实际场景中代码修改的相关影响。在运维层面可能是保留现场后直接重启，再通过 copy 的现场做模拟；在一些长远问题上，可能我们需要慢慢来分析。

这就是胖哥讲的老鸟与小鸟的故事，如果你是小鸟，你懂了吗？

➤➤ 1.7.2 你的程序不好用，你会不会用，环境有问题

“你的程序不好用？你会不会用？”

能说出这句话也许你是一只老鸟了，但是不一定是一个老 A。

这是一个对自己充满自信的程序员，在面对别人提出质疑时的第一反应，可能作为老鸟，你会说自己不是这样的人，那么你真是一朵“奇葩”，或者你真的升华到了“大师”级别，修身养性的境界非同一般。在众多老鸟之中，我们都有一股“程序员的野性”，这种野性通常来自于自己研究了许多东西，以及在相关领域待的时间很长所赋予自己的强烈自信感。

学海浩瀚无边，任何人（即使是大师）也有犯错的时候，所谓“老虎也有打盹的时候”，尤其是在工期要求很紧的时候，“一流高手”和“二流高手”体现的水平不会有多大的区别，因为他们都是“老鸟级”的人物，代码编写速度和问题解决能力都是非常强的，而更高境界上的区别，在这种情况下是体现不出来的。

在面对别人提出质疑的时候,第一反应其实是看你的内心世界是否足够强大。别人质疑你的第一句话,有可能是顶住你心窝的话,有可能是将你苦思冥想的方案瞬间彻底否决的话,有可能是将你很长时间的辛苦付出瞬间说得毫无价值的话,有可能是将你的成果瞬间打入无底深渊的话,也许是对你和大家都认可的事情简单敷衍的话。这些话相信比质疑你的代码有 Bug 会更加重一点吧,这个时候你会如何想呢?

或许一颗强大的内心需要去磨练,小胖哥也在不断磨练中成长自我。也许你会觉得很累,因为这是在挑战自己的个性,改变自我是最痛苦的事情。也许这个时候我们可以看看别人是如何修身养性的,别人能做到,我们同样可以做到。

代码有问题,我们就看原因,不论是不是自己的问题都要告诉真正的原因。有问题没关系,我们不怕犯错误,而是怕错了不改,反反复复犯同样的错误,可能还是同样低级的错误。

如果反反复复有人提出同样的问题,而绝大多数情况下都知道不是自己的问题,那么就将它以文档的方式呈现出来,因为代码是我们写的,使用者通常不知道所有的细节。

如果没有任何规约,就意味着可以随便“乱搞”,事实上逻辑的东西暂时是不可能做到这一点的(就算是写代码也得遵循基本的语法规则,用数据库也得遵循它的基本规约而不能乱删除东西,用接口需要遵循接口规约),“乱搞”的结果就是东西越多、问题就越多,做一件事情来解决问题,但是带来了更多的问题。

这个“乱搞”的底线是什么?完全是看系统的设计思路,也就是软件的伸缩性,你需要让用户知道不能去做或不建议去做的事情,我们可能会提供,但是不完全保证它的正确性或安全性,为什么不保证也许某些用户也想要知道。

回过头来,对于自己来讲最重要是冷静面对事情的真相,客观面对我们所面临的事情,加上对业务本身的熟悉程度、技术功底的配合、冷静思考、量化的判断、全局性影响的分析,逐步突破层层技术和业务难关,相信我们可以很“酷”地解决绝大部分的问题。解决问题的手法也会多种多样,而并非等闲之辈。

➤➤ 1.7.3 经验是否能当饭吃

通过对前面几节内容的介绍,胖哥就是想让你知道,也许某些经过试验论证或曾经解决问题得到的结论不是很靠谱,或者说并不是“永远靠谱”的。

前面也提到了,其实在技术的发展过程中,我们都是站在前人的肩膀上做进一步的事情,它们虽然以“平台、插件、思想”的方式提供给我们作为基础,但是通常也都是代码,可以先不说“只要是代码,肯定就会存在 Bug”,但是软件程序在不断更新换代的过程中肯定会存在各种逻辑的复杂性来满足更多的需求,就像一个系统做复杂了的道理一样。

随着实际代码的复杂性增强,就会产生各种各样的运行逻辑,此时如果 API 写得过分简单,那么许多程序员可能就不知道这个 API 下面的许多细节,也没有那么多精力来关注每个细节,因此就可能会导致很多不可预见的问题。在某种场景下我们可能将问题解决了,但是也许并不是因为看到了问题的本质,而有可能是因为某种巧合走到了一条正确的执行路径上。

除了上面的例子,曾经有不少人在“乱码”上经常出现怪异的问题,小胖哥也曾经为此纠结过很久,很多人在遇到乱码后都会像小鸟一样说:“为什么我这里好用,另外一个地

方却不好用呢？”这类问题在网上通常有很多方法，大多要求你将数据库、应用服务器、接口、环境变量、页面等输入/输出的字符串全部统一。其实这种方法只能解决某些局部问题，并未看到问题的根本。

为什么说只能解决某些局部问题呢？

因为在某些场景下，系统会连接不同的数据库、缓存、分布式存储、应用方、中间件等，它们都有可能会提供不同的字符集输入输出，我们也无法控制别人提供的服务必须用什么字符集。换句话说，你或许在某种程度上可以保证自己的应用系统使用什么字符集来避开许多问题，但是当面对复杂的分布式环境时，你不得不去知道字符集的根本所在。只有搞清楚这类问题的本质，才能在遇到类似的问题时去定位分析。

关于乱码更多的介绍：

乱码问题不是本节讨论重点，关于乱码方面的问题在本书后文中第2.6节、第17.2节中有更多的介绍，同时会在第4章的通信里面提到一些相关的基础知识。

再换位思考一下，软件在不断革命的过程中，内在的逻辑在不断变化，也变得更加复杂，它们提供的一些参数可能发生改变，即使是同样参数的API的内在实现也有可能以前大不相同。此时，随着技术的发展一些经验开始变得“不太靠谱”，包括我们曾经记录下来的经验值、默认值，甚至是官方标准，它们都可能在在一个小的版本变化中变得不靠谱。

因此，我们需要新的学习，是否会觉得很累呢？我们在不断学习别人提供的东西，在不断记忆别人的东西，前一样东西还没掌握清楚，新的又来了，肯定很累。但这需要方法，也需要功底，当你懂得它的根本与发展的方向和模式，基本就知道它要做什么了，也基本知道会怎么做了，自己也可以实现，只是没必要而已。一些开源软件中常见的参数不会太多，而更多的参数当你知道它的根本后，通过某些命令和文档查阅是非常快速的，我们学习的代价不应该太大。

回过头来总结一下，我们应当“站在前人的肩膀上看问题，不仅仅是要沿用前人写好的东西，更需要看到他们所遇到的那些坑”。在理论指导的基础上，比前人看得更高、更远，社会和技术才可能向前发展和进步，新一代程序员不必都经历上一代程序员所经历的同样的坑，那样只是延续，而没有任何发展。新一代程序员即使要经历坑，也是经历一些更新的坑，去挖掘更新的东西，而不是走老路。

小胖哥说了这么多话有什么用呢？就是想要告诉读者朋友，经验可以用以借鉴和参考，但经验不能用来当饭吃，同时别人告诉你的是他的经验，也同样是一种参考和借鉴，我们需要学会去看问题本质的方法与习惯。也许你不太认同小胖哥的观点，可能你和小胖哥看待问题的角度不一样，胖哥也很希望你有不同的自我见解，但也相信我们会有一些共同的认识。

接下来我们用轻松的方式来论道，谈论老A的那些事，老A在面对逆境时是如何面对的。

1.8 老A是在逆境中迎难而上者

小胖哥虽然不是一个“传道者”，但是喜欢小小论道，因为在人生的道路上，很多时候你我都会面临许多纠结的事情，而这个时候我们的态度会决定命运，而道就是道理和方法。

- ◎ 作为“程序员”，每天被要求修改代码是否会烦躁不堪，甚至觉得失去工作的价值？当有人说你的代码有很多 Bug 时，你会像小鸟或老鸟一样反应吗？
- ◎ 作为“架构师”，成天被程序员说这里（那里）不好用、性能不好、伸缩性差、太重等，而程序员可能并没有理解框架的真正意义，你是否会纠结？你应该如何去面对和解决？
- ◎ 作为“经理”，要被客户骂，又无处发泄，还要听下面人的抱怨，你有一颗强大的内心世界吗？

有很多事情只可意会不可言传，对于大多数事情都需要你我修炼内心世界。大家可能都看过一些特种兵的电影或故事，老 A 们通常会面对常规兵种没有遇到过的困难和绝境，但他们总能找到绝处逢生的道路，这就是老 A 的本色，老 A 除了要练好基本功外，拥有面对逆境的心态和战胜困难的决心是十分重要的。

我们在解决问题时，不仅要学会面对成功，而且要学会面对失败与困难，这样才会进步，如果有人说他没有失败过，只能说他不愿意去承认自己的过去而已。我们都会在成功、失败、困难的过程中不断交替轮询，对于许多深入问题的看法，也不断在模糊与明了中轮询（就像图 1-10 所示的太极图那样）。在这个过程中只要愿意去坚持和思考，都会有自己对人生的理解和丰富的经验，并且能跟上社会的潮流，经历从量变到质变的过程，以达到境界的提升。



图 1-10 太极图

在逆境中敢于迎难而上的老 A 们，除了拥有信心战胜困难以外，也拥有极强的功底和快速学习的能力。提到学习能力，在不同的博客、网站、论文、通信群里面你能看到不少人提出的学习方法，而且每个人总结出来的学习方法都会有些不一样的地方，即使同样的学习方法，你也未必全部理解它们的意思。胖哥绞尽脑汁写出下文，希望你能看懂真正意义，能得到一些帮助。

每个人在不同的阶段应当有不同的学习方法，不能一概而论。提到学习方法，还是一句老话：多看、多练、多思考、多练习、多总结而对于这句老话，胖哥的理解如下。

□ 多看

看什么？看别人如何写代码，如何分析和解决问题，用到了什么技术，如何面对工作的压力，如何为人处事，如何协调资源等。

怎么看？周围都是你的同事，同事中肯定有高手。网上有你的朋友，朋友中肯定有佼佼者。你会用到许多开源框架，其中肯定包含了许多经典的代码和思想。

□ 多练

多练，但是很多人找不到方向，应该练什么？这是问题的关键，也是初学者陷入迷茫的一个重要原因（很多已经有一定工作经验的人也会陷入这种迷茫）。

有些时候看到别人写的程序，自己什么都想练习，什么都想学习。其实要知道，无论别人写出多么优秀的程序，干出多么大的事情，这并非一朝一夕之事。

需要做的是立足当下，找到自己的进步点，而不要好高骛远。什么是立足当下呢？

立足当下就是找到自己的兴趣，而非别人的兴趣；找到自己的业务挖掘点，而并非别人的业务（并不是说不要去关注别人所做的事情，而是一种经验的交流，不要老觉得别人碗

里面的饭要香一些)。

兴趣建立在你对所解决的问题的价值是否理解上，兴趣可以给你带来不错的收入，兴趣是你可以处理一些周围人处理不了的问题，兴趣是你擅长的点是否能够得到发挥等，其实兴趣是自己挖掘的。

初学者一定要多练习，即使是练习代码，也是很好的，达到一定程度再思考都可以。即使是水平很高的人，也会多练习，只是练习的代码不一样罢了。练习可以让我们对问题有更多的感性认识，许多问题在练习后自然会逐步变得清晰起来。在练习的基础上，才会进一步有理性认识。

□ 多思考、多练习

经过多练习，很多人会走偏方向，进入一个“技术控”的死角里，因为“对技术的控制”会成为一种乐趣。“技术控”如果“深陷泥潭、难以自拔”，最终可能是被技术控制，会被技术牵着鼻子走。

这个阶段我们需要更多的思考——这种技术有什么缺点？我们用它能解决什么核心问题？相关的技术有什么？它们有什么区别？

当我们深入地去学习一样东西的时候，会自然而然地需要去学习另外一样东西，这就是知识之间的串联，碎片化的学习并非完全碎片化，随着学习不断深入，知识自然会串联在一起。

在练习、思考的基础上还要如何深入？各类网络博客、官方文档、源代码、书籍都是积累的关键要素，当资料很多的时候要学会挑关键点。

什么是关键点？我们关注的点就是关键点。其实关键点就是你最想要知道的一个点，这种关键点对不同层次的人会有不同的划分概念，它可以细到一条代码怎么写，一个二进制位在机器内部如何处理，也可以粗到总体架构是怎么回事，这完全和你的工作相关，同时也与你对工作的理解程度相关。这是一个积累的过程，知识和业务体系越“通”，想要找到关键点就越容易，所以学习、思考、时间是一套不断迭代的“组合拳”。

思维上我们要去放纵扩展、体系化扩展（但不要钻牛角尖），在扩展的过程中会迫使自己去学习相关的许多知识，而且这些知识会刻骨铭心，随着知识不断地串联化，也会逐步形成体系。如果有一天你发自内心地认为“一切源于基础思想和生活方法”，这时你再看高手们的书，或许就是一种知识的梳理过程和经验的交流。

□ 多总结

一个人不论学习多么努力，多么会思考，但如果他的知识没有沉淀，终究会丢掉一些东西。

何时沉淀？沉淀什么？这是我们不断摸索的东西，胖哥也认为没有定论，如果你认为它对你未来的发展是有帮助的，或者说你认为它可以帮助你提高水平或扩展知识面，那么你就将它沉淀下来。

每天我们有了一个小沉淀，每周就会有点小总结，每月就有一些感悟，每季度就会发现几个月前的自己是那么的“幼稚”，那么自己就真的成长了。经过1年、2年的坚持，你可能会比别人知道得更多，也深入了很多。

也许有人说无论你怎么沉淀都没有用，始终在原地踏步，几年后还是一个样。那么你就要想想：在总结的过程中是否考虑过方法有没有问题？或许自己始终在沉淀那些很简单、

很容易的东西，仅仅是为了“记录”而缺乏自己的见解；或许在总结的过程中没有去考虑如何做得比以前更好。当然如何总结是个人的自由，胖哥无法干预，每一种总结都有它的价值所在，即使是“面”上的扩展总结，只要是没有曲解基本的道理，仍可以“引导他人”成长。

什么时候需要去沉淀知识呢？当你解决了问题时，当你学习到问题的本质时，当你看到了以前看不到的东西时，当你觉得值得去总结时，长期坚持总结是厚积薄发的基础，这是亘古不变的道理。从量变到质变取决于在总结的过程中你能否从某个点去突破和深入，也许就差那么一点点儿，当发生质变后在对待问题的态度和方法上会有一个更加广阔的空间，这就是我们通常所说的“茅塞顿开”。

沉淀什么？我们不仅要总结知识点，也要总结方法、手段、心态，以及身边可以深度挖掘的业务。

接下来的几章将会真正介绍 Java 的一些基础知识。



第2章

Java 程序员要知道 计算机工作原理

本章主要内容

- Java 程序员需要知道计算机工作原理吗？
- 关于 CPU
- 内存
- 关于磁盘
- 关于 Cache
- 关于网络与数据库
- 总结与思考：系统模型的选择

Java 程序员要想成为特种兵，基本功力尤为重要，计算机人要懂何为计算机就好比“习武之人要知道何为武学”一样，本章将总结计算机的基本工作原理，引导你从计算机本身的角度来考虑问题。

2.1 Java 程序员需要知道计算机工作原理吗

Java 程序员需要知道计算机工作原理吗？

如果在国内问 Java 程序员这个问题，大部分程序员会说：NO！

但胖哥认为：Java 程序员也应该知道计算机是如何运行的，计算机本身大致是如何完成工作的，这样才能更好地量化我们的代码开销。

作为 Java 程序员，也许并不需要知道太多的细节，但是对必要的工作流程和体系结构、每个工作模块的组成、性能量级，应当有一个基本的概念。因为作为老 A 级程序员，应当具备这样的素养。

当你需要对系统做硬件评估时，当你需要对数据库和磁盘做评估时，当你需要对网络做评估时，当你的 Java 代码出现了更低抽象层次的问题时，当你需要做一个多线程设计时，当你遇到各种乱码时……此时计算机本身的一些机制显得比较重要。

也许有些时候可以从网络中找到一些方法解决问题，可是你知道问题的原因吗？如果

再次遇到类似的问题场景，你还能够应付吗？计算机的基本原理让我们学会在解决问题后通过计算机本身来理解原因。下面是一个简单的解决实际问题的例子。

一天“sleeping”同学遇到了乱码，一直在纠结系统中的乱码该怎么处理，他在网上找到了一系列答案——将应用系统、数据库、前端 UI、接口的字符集保持一致，他做到了，他成功地解决了这个问题。“sleeping”同学总结了他解决乱码的经验，认为乱码就是那样解决的，只要按照步骤和套路就没有问题了。

可是有一天他发现这个问题再度出现，场景却不一样了，数据库来自于第三方，中间还有各种接口，接口的发布是对外通用的，通常不会因为某个系统的要求而修改字符集的输入输出规范，因此乱码怪异的情况让他再次陷入了纠结中。

当时胖哥和他一起学习计算机网络输入、输出的字符编码、解码的过程，与他一起看从对应的 Web 容器、第三方框架、应用代码本身转码的过程，了解乱码的产生过程，最终定位是“某个环节的解码与编码产生了不一致的情况”。

后来，“sleeping”同学提到“别人说自动转码会有识别率的问题”。关于这个问题，胖哥也给了解释，最终可以确定的是“说这种话的同学还没理解字符集解析的本质所在”。

“sleeping”同学发现这个问题并没有自己想的那么顺利，需要知道内在才可以逐步定位，搞懂问题的本身，针对实际场景进行实际分析才是最好的。反过来讲，这也体现出人的价值，如果所有的东西都规律化，提出的问题都有一个统一的方法可以解决，那么人的智慧就不存在了。

关于乱码，本质上讲它产生的原因只有一个，但场景会有许多，在后文中会提到乱码的问题。

随着时代的不断发展，技术的不断革命，计算机本身的运行性能也在不断改变，不过计算机的总体体系架构的变化一直不是特别大，基础原理将引导我们从整体上去认识计算机本身，它的重要性远远大于参数本身，参数可以随时从网站上查到最新的资料，得到参数后，这些参数的“数量级”在 1~2 年一般不会发生大的变化。

后文中也不会太深入地去讲计算机的内核，因为我们毕竟不是内核专家，但是要知道有这些东西，以及它们的大致原理，胖哥可以带你从一个 Java 程序员的角度去考虑如何面对计算机本身的一些内在。

下面我们从 CPU、内存、磁盘等各个角度来阐述计算机的一些基本原理，通过缓存来理解优化设计，讲解系统的 I/O 与数据库交互的联系。

2.2 CPU 的那些事儿

➤➤ 2.2.1 从 CPU 联系到 Java

我们首先应当清楚，每个进程或线程发出操作请求后，最后会由 CPU 来分配时间片处理，处理时将操作数传递给 CPU，CPU 计算将其回写到“本地变量”中，这个本地变量通常存在于程序所谓的“栈”中，如果多次对这些本地变量进行操作，为了提升运算效率，它们有可能会被 Cache 到 CPU 的缓存中。CPU 有寄存器、一级缓存、二级缓存，有的也有三级缓存，CPU 为啥要有这么多组件呢？其实就是一个“就近原则”。

打个比方，去某个地方办“几件”事情，不会办完一件事情回家一趟，再坐车去办下一件事情，因为时间开销太多了。如果发现办完一件事情后就在旁边可以办下一件事情，

则可以直接继续办理；如果发现有点累，则可以在外面小坐一会儿，但是不用回家。

CPU 也可以一次做几件事情，所以它会有一个小 Cache 区域，将这些内容暂时 Cache 住，这样距离处理器越近，操作的延迟就越小，整体效率自然就越快。

一般来讲，一级缓存与 CPU 的延迟一般在 2~3ns 之间，二级缓存通常为 10~15ns，三级缓存为 20~30ns，而内存通常会在 50ns 以上甚至更高。

缓存是有道理的，就近原则可以达到某种场景下的高效。这与工作中的一些优化方法很类似，在系统的设计中也可以参考计算机本身的这些基本的思想模型。此时回想一下我们所做的工作、所写的代码，如果将思想模型用到设计中或者某些业务领域中，是否有用？胖哥相信肯定是有用的，因为它的思想本身就来自于现实的抽象，而我们现在是将它回归到现实中。

在多核的 Cache 中，对于某些数据 Cache 后，数据在“写入”和“读取”的时候必须满足一些规范，通常叫作“缓存一致性协议”。通过这种规范来实现架构，用以满足多个 CPU 对同一个变量修改时，相互之间都是知道的。不过，它并不保证所有的数据都是这样的，因为这样做的开销是巨大的，所以在某些时候是允许不一致情况发生的。

我们首先提出问题：我们所编写的程序是如何和 CPU 进行交互的？它是否会被 Cache 住？是否存在并发问题？在可能的情况下，如何利用 CPU 来提高程序运行效率？……带着一系列问题和思考，然后通过部分 Java 基础代码 demo 展开。

相信所有的 Java 程序员在编写 Java 程序时，大部分操作都是申请对象和操作对象，而对象实例的空间大部分存储在 Java 的堆（Heap）当中，而 Java 的“栈”当中存储什么呢？

Java 的“栈”更多的是通过 Java 与 OS 一起管理的一块区域，写过 C/C++ 程序的人应该知道，如果“不是”通过 malloc、realloc、new 等关键字申请的内存空间，那么作用域结束时空间自然释放，因为这部分的回收操作是由 OS 来管理和控制的。

在 Java 程序中“栈空间”也是类似的，当程序的局部变量中使用基本类型时，它直接在“栈”上申请了一些空间，而当使用引用来引用对象时，这些引用的空间也位于“栈”上。

确切地说，在编译阶段，Java 就可以决定方法的“本地变量”（LocalVariable）的个数，因此在方法调用的时候，就可以直接分配一个本地变量的区域。

这个空间是基于 slot 来分配的，每个 slot 占用 32bit，就算是 boolean 也会占用这么宽，当然 long、double 会占用 2 个 slot。这些 slot 可以被复用，也就是说，在方法体内部，如果某个局部变量是在循环或判定语句内部声明的，那么在退出这个区域后，对应的 slot 是可以被释放给在它之后声明的局部变量使用的。

在程序运行过程中，会通过 Java 的虚指令（后文中会介绍）来完成对 Java 虚拟机中的对象和数据做一些操作。虚指令只是 Java 的指令，而不是最终指令，有它才会跨平台，它最终会在对应 OS 的虚拟机上被翻译为汇编指令来完成实际硬件的运行操作。

为了先感性认识下虚指令是什么，下面我们来看一个比较常见的例子。

代码清单 2-1 一个简单的 Java 加法程序

```
Public static void main(String []args) {  
    int a = 1;  
    int b = 2;  
    int c = a + b;  
}
```

这里胖哥带你用 `javap` 命令来看看运行后的指令是什么（关于字节码的一些细节，将在第 3 章中介绍）。

我们运行的命令是“`javap -verbose`”（这是一种输出比较全的方式），结果分两个部分来看：一个部分是常量池描述信息，一个部分是字节码的 `body` 部分。首先来看常量池部分的内容。

```
C:\java_A>javap -verbose chapter02.JavaSampleAppend
Compiled from "JavaSampleAppend.java"
public class chapter2.JavaSampleAppend extends java.lang.Object
  SourceFile: "JavaSampleAppend.java"
  minor version: 0
  major version: 50
  Constant pool:
    const #1 = Method      #3.#12;      // java/lang/Object."<init>":()V
    const #2 = class       #13;          // chapter2/JavaSampleAppend
    const #3 = class       #14;          // java/lang/Object
    const #4 = Asciz        <init>;
    const #5 = Asciz        ()V;
    const #6 = Asciz        Code;
    const #7 = Asciz        LineNumberTable;
    const #8 = Asciz        main;
    const #9 = Asciz        ([Ljava/lang/String;)V;
    const #10 = Asciz       SourceFile;
    const #11 = Asciz       JavaSampleAppend.java;
    const #12 = NameAndType #4:#5; // "<init>":()V
    const #13 = Asciz       chapter2/JavaSampleAppend;
    const #14 = Asciz       java/lang/Object;
    line 9: 8
  }
```

常量池描述信息是在编译时就确定的。所谓常量池就是要记录一些常量，这些常量通常包含：类名、方法名、属性名、类型、修饰符、字符串常量，记录它们的入口位置（符号#带上一个数字，可以理解为一个入口标志位）、一些对象的常量值。

初步看，常量池还只是一些单纯的“列表”，和程序运行没有多大的关系，在实际运行过程中需要组合成有效的运算指令，它们在 `body` 内部。

```
{
public chapter2.JavaSampleAppend();
  Code:
    Stack=1, Locals=1, Args_size=1
    0:   aload_0
    1:   invokespecial   #1; //Method java/lang/Object."<init>":()V
    4:   return
  LineNumberTable:
    line 3: 0

public static void main(java.lang.String[]);
  Code:
    Stack=2, Locals=4, Args_size=1
    0:   iconst_1
    1:   istore_1
    2:   iconst_2
    3:   istore_2
    4:   aload_1
    5:   aload_2
    6:   iadd
    7:   istore_3
    8:   return
}
```



```
LineNumberTable:
  line 6: 0
  line 7: 2
  line 8: 4
  line 9: 8
  LocalVariableTable:
    Start Length Slot Name Signature
    0      9      0  args    [Ljava/lang/String;
    2      7      1   a      I
    4      5      2   b      I
    8      1      3   c      I
}
```

指令不是本章讲解的重点，不过我们是第一次亲密接触虚指令，所以胖哥对这段代码稍作分析。

- ◎ `iconst_1`，将 `int` 类型的值 `1` 推送到栈顶。
 - ◎ `istore_1`，将栈顶的元素弹出，赋值给第二个 `slot` 的本地变量。
- 这两个虚指令相当于 `int a = 1;`。

`iconst` 相关的指令包括：`iconst_m1`、`iconst_0-5`，对应到虚指令的范围是 `[0x03~0x08]`，表示 `[-1~5]` 之间的数字常量加载到栈顶，如果不是这个范围的数字，就 `bipush` 指令。

`istore_1` 是赋值给 `slot` 为起始位置的本地变量，`istore_0` 才是赋值给第一个 `slot` 起始位置，那么第 `0` 个本地变量是什么呢？是 `main` 方法传入的 `String[]` 参数，同样的，如果是非静态方法，`this` 将作为任何方法的第 `1` 个本地变量。

我们应该在输出的字节码加粗部分看出了一些内容，几个变量所对应的 `slot` 编号也输出了，这里的每个局部变量都单独占用一个 `slot`，如果局部变量中有 `double`、`long` 类型，`slot` 的个数就会变多。

这里的 `LocalVariableTable` 列表是本地变量的列表，某些开发工具（例如 `Eclipse`）默认就有这些信息，通过 `javac` 命令编译后的 `Class` 文件默认是没有这些输出信息的，换句话说，在默认情况下本地变量没有名称的概念。

`javac -g:vars` 可以输出相应的信息，如果使用 `javac -g:vars,lines`，则可增加输出 `LineNumberTable`，这个参数将会用于 `debug`（调试）中 `Class` 与源文件的行号对应关系。如果使用 `javap -help`，则可看到在 `-g` 命令后面还可以跟一个 `source`，即 `javap -g:source`。其实在默认情况下都是带上参数的，只有使用 `-g:none` 时才会什么也不带，但 `-g:lines` 是自动带上的。`-g:source` 做什么呢？通过字节码发现 `-g:none` 时常量池中的“`SourceFile`”以及“文件名”不存在了，在实际的应用中，通过这个类去做断点调试时是做不了的，连断点都没办法打上（测试方法是，编译完的 `Class` 打包，然后源文件也打包，在 `Eclipse` 中绑定后，可以跟踪到源文件，但是无法给它打断点）。

`LocalVariableTable` 列表中每一行代表一个本地变量，每一列的解释如下：

- ◎ `Start`，代表本地变量在虚指令作用域的起始位置（如第 `1` 个本地变量 `args` 是从 `0` 开始的）。
- ◎ `Length`，代表本地变量在虚指令列表中作用域的长度（如第 `1` 个本地变量 `args` 是 `9` 条指令的作用域）。
- ◎ `Slot`，代表本地变量的 `slot` 起始位置编号（这里是顺序排布的，但是如果出现 `long`、`double` 就会跳跃）。
- ◎ `Name`，代表本地变量的名称，也就是本地定义的名称。

◎ Signature, 代表本地变量的类型 (如第 1 个 args 是 String 类型, 其他几个都是 int 类型)。

接下来, 同样是将 int 类型的值 2 推至栈顶, 并保存到第 3 个 slot 的本地变量中。

iload_1、iload_2 是将两个本地变量的值推至栈顶, 然后执行一个 iadd 操作计算出叠加后的结果放在栈顶, istore_3 将结果数据从栈顶 pop 出来, 保存到第 4 个 slot 所对应的本地变量中。

这里的“栈顶”是一个后进先出的 Stack, 可以具体由与 CPU 交互的操作数组成, 但是它并不等价于本地变量, 它可以进行多个操作后, 写回到指定的本地变量中, 或者只是用于读操作, 就不需要写回到本地变量中了。

这些指令是 JVM 的虚指令, 它们可能会重排序某些原来代码的顺序 (反编译程序也是根据这些指令反转计算回来的, 但是反编译程序也会有问题, 在后文中会提到)。虚指令在翻译为 CPU 的指令运行阶段可能被再次重排序, 也就是运行的顺序可能并不是我们想的那样, 在第 5 章关于 Java 并发的章节中会更加详细地阐述这种问题。

本节, 我们只需要知道 JVM 能发出指令请求, 由 OS 去完成具体工作, JVM 自身无法做计算工作就行。JVM 内部对象的操作, 例如字符串叠加, 表面上也只是调用了一些数组拷贝, JVM 内部数组拷贝等过程也是通过类似的手段来实现的。也就是说, JVM 只关注自己的事情并做好, 将其他的职责交给别人来完成。

一群老 A 要干掉敌人的指挥部, 每个老 A 也需要坚守自己的职责, 以发挥最大效应。在社会工作中各司其职, 计算机本身也是这样, CPU 就是用来做计算的, 内存只是用作运行数据时的一个存储 (带电下), CPU 调度线程和进程的指令请求, CPU 提供数据的计算功能。

➤➤ 2.2.2 多核

多核 CPU 就像有多个计算中心同时做事情, 或者说一个人力气再大也会有极限, 也有做不到的事情, 那么就多个帮手一起来做吧。

多个 CPU 要发挥最大效应就是不让某些人偷懒, 自然就需要一些算法来协调和管理, 让请求负载均衡。比方说, 这个指令由哪个 CPU 来处理呢? 同一份数据被多个 CPU 处理, 并对其进行修改后, 如何让其他的 CPU 知道呢? 这中间产生各种各样的问题, 算法也层出不穷, 为了解决一个问题而将问题变得更加复杂, 但是为了发展又不得不去做。在工作中也同样会遇到系统不断变得复杂的过程, 这是社会需求的能动, 不得不去做, 即使是计算机部件也是同样的命运。

当发起一个计算请求的时候, 例如一个中断, 这么多的 CPU 会干什么呢? 这就要从任务的模型开始说起。起初, 操作系统可能是不断地定时扫描各个部件看是否有指令来, 有的话就处理, 但是显然这种模式会很慢, 因为“知道的时候往往都晚了”, 这就是所谓的“不及时”或者叫“非实时”。

后来, CPU 有了“中断”模型, 通过中断来完成调用, 貌似实时性很强, 但是某些部件发起中断频率非常高 (例如鼠标移动, 就在不断地发送指令)。此时对中断会有一个缓冲区, 由于 CPU 的处理速度非常快, 可以在一瞬间处理大批量的请求, 所以这种缓冲区是不错的设计思路 (在许多设计中都会用到)。

一系列任务可能要做各种各样的事情，在通常情况下，CPU 的计算速度非常快，很快就可以完成指令。但是操作系统不希望 CPU 为一个“等待指令”或者是长期执行的任务使得自己“陷入困境”，比如一些 I/O 等待（网络 I/O 和磁盘 I/O），它中途基本都不参与，而是以事件注册的方式来实现回调，对于某些执行时间长的任务，CPU 可能会分配一些时间片来处理其他的任务。

当 CPU 不断去切换任务处理的时候，又会有一种新的现象出现，那就是“上下文切换”，在后文中会介绍。

当多个 CPU 在同一台计算机中出现的时候，会出现什么情况呢？是大家一起去抢指令？还是由一个 CPU 来分配指令？抑或是某些 CPU 专门管理某一块区域的指令？

CPU 是一个大家都要用的资源，那么就必须要有一个规范，而如何来规范呢？这没有明确的说法，只是大家的实现方法不同、架构不同，也许在不同的测试场景下效果会不同，调优的方法也有变化。这些会体现在计算机兼容性上，CPU、内存、硬盘、主板等，在熟悉了具体厂商的实现机制后，相互配合兼容性就会做得更好。

现在来说“大家一起去抢指令”，这种方式是不多见的，只是在某些系统的设计中有多进程模式，让多个进程监听同一个端口，当这个端口得到信号时，可能多个进程同时被唤醒的现象还是存在的，我们通常称之为“惊群”。

而“由一个 CPU 来分配指令”，这种方式也存在，那这样会不会让这个 CPU 很忙？也就是这个 CPU 会不会出现相对“飙”高的现象，尤其是有大量请求的时候？没错，它也有缺陷，只是通常情况下问题不会太大，而且它至少实现了调度，在有些时候可以划分下“领土”，使资源隔离是比较“靠谱”的。

那么我们按照板块来划分，每个板块有一个或多个 CPU 来管理对应的内存区域，是不是就合理了？也未必。首先，数据还需要和其他的部件通信，其他的部件资源是不是也是隔离的？要做到完全的物理划分是十分复杂的，通常意义上的物理划分只是相对底层的指令量、数据量的协调而已；其次，板块划分完以后，就完全有可能出现热点问题，如果遇到热点问题我们应当如何处理呢？这个时候我们就要想其他的方法了。

那到底哪种方式好呢？

其实没有好坏之分，只有场景的选择，例如最后一种方式，就很适合一些各自为政的系统分配，也就是它们之间交互少，或者根本没有交互。我们也可以考虑根据某种算法的一种指令分配规则，这些都是在已有技术的基础上，根据实际场景演变、变通出来的，而这所有的思路都接近于生活方式。

这里就不再详细探讨 CPU 的细节了，从某种意义上讲暂时用不上，基于这些基础的理论，如果某一天你真的用上了，那么就根据具体的 CPU 参数规格去分析系统吧。

►► 2.2.3 Cache line

上一节中提到 CPU 的时候，我们说到 Cache，就是将数据 Cache 在 CPU 的三级缓存中，而 Cache line 是什么呢？

看名字猜猜，是 Cache 行，没错，那为什么要 Cache 一行？什么叫作一行？一行有多长？写 Java 程序和它有什么关系？

下面我们就一起来探讨吧。

前面提到“大家办事就近原则”，也就是一次可以办多件事情，或一件事情的多个步骤

可以一次办完，相信这一点已经毋庸置疑。而 Cache line 是从另一个侧面来看问题，它好比是在办事情时需要携带许多材料，有些时候因为缺少材料要反复回家拿。例如，我们去办证大厅办理证件，每种证件都会有很多个步骤、很多份材料，这些我们都可以预先准备好，一次搞定，而不用因为缺少材料回家拿再去办理。

这样说，你是否明白？那 Cache line 实际上是讲什么的呢？

它将“连续的一段内存区域”进行 Cache，不是每次就 Cache 一个内存单元，而是一系列内存单元。在计算机中，通常以连续 64 字节为基本单位进行 Cache 操作，未来可能会更大。

这样，在操作相关连续数据的时候，就不用再次从内存中读取了。这好比是一车货物需要很多工人“一包一包”地搬运上车，然后整车拉到目的地，而不需要“一包一包”地拉到目的地。

那它与我们的 Java 程序到底有何关系呢？

下面胖哥举个例子，也许这个例子有点“囧”，不过你可以看出一些区别。

循环遍历二维数组 `int[][]a = new int[5][10]`。要遍历这个二维数组，通常用两层循环来遍历，分别遍历两个维度，有两种方法：一种方法是外层循环第一维，内层循环第二维；另一种方法正好相反，也就是说，两层循环交换顺序，但是效率孰高孰低呢？结论是第一种方法的效率高（当然，一个简单的 main 方法是无法验证结果的，只有多线程并发请求时才会了解到整体的开销情况）。

这些细小的差距，不仅仅体现在 CPU 上面，在第 3 章中胖哥还会和你说说：

```
int a[][] = new int[2][100];
int a[][] = new int[100][2];
```

它们两者在内存开销上的巨大区别。

这个结论和 Cache line 有什么关系呢？因为 Java 数组在内存中分配是先分配第一维（或者说 Java 没有真正意义上的二维数组），然后再分配多个第二维子数组，也就是说，`a[0][x]` 和 `a[0][x]` 是位于两个不同数组上的，空间也自然不会在一起。

单个数组的内存空间是连续的，当获取 `a[0][0]` 时，Cache line 操作通常会将 `a[0][0]` 相关的一些数组元素（`a[0][1]`、`a[0][2]`、`a[0][3]`、`a[0][4]`、`a[0][5]`、`a[0][6]`、`a[0][7]`）全部都 Cache 到 CPU 的缓存中，当使用第一种遍历方式来遍历时，这些连续的数据都只需要 Cache 一次，就可以遍历完成。反过来，如果外层循环是第二维，内层循环是第一维，下标情况就不是这样的，访问完 `a[0][0]` 后不是访问 `a[0][1]`，而是访问 `a[1][0]`，这两块区域并非连续的，所以不会一次被 Cache，需要重新访问内存才行。

简单的变化也有内在巨大的差别，虽然一切来源于基础和生活，但是变化是无穷的，这些变化可能在实际工作中具有指导性。

感悟：Cache line 就是这样的一个东西——一次拿一批信息去处理。

有人说这就是平时所说的“懒人做重担，十担做一担”。其寓意是：一次做不到非要一次做，最后可能花的时间和精力很多，而且可能会把事情搞得一团糟，还不如简单地一次次做。

任何事情都要讲究“度”，切不可一概而论，这里的“批量”是指有能力做到每次拿一批信息和拿一个单元的数据时间开销其实基本是一样的，“一天有能力可以做完的事情，没

有必要做两天”。

疑惑：如果 CPU Cache 时是按照连续内存空间进行的，那么对内存进行了修改，回写时是不是按照行进行的呢？

答曰：不是，Cache line 的目的是为了快速访问，它对内存单元做出的修改有明确的定义，在回写时，也会找到对应的内存空间。

➤➤ 2.2.4 缓存一致性协议

上一节我们了解到了 Cache line，以及 CPU 的 Cache 基本作用，此时我们来了解一下缓存一致性协议，它是什么呢？当有来自于内存中的同一份数据 Cache 在多个 CPU 中，且要求这些数据的读写一致时，多个 CPU 之间就需要遵循缓存共享的一致性原则。

其实你完全可以理解为很多人都拿到了一份设计报告的模板作为初稿，每个人都拿回去进行修改，但是要求：任何人做出的修改都要让其他人知道你对哪些内容做了什么修改，以便于大家可以在你修改的基础上做进一步的修改。这就是我们说的一致性，如图 2-1 所示。

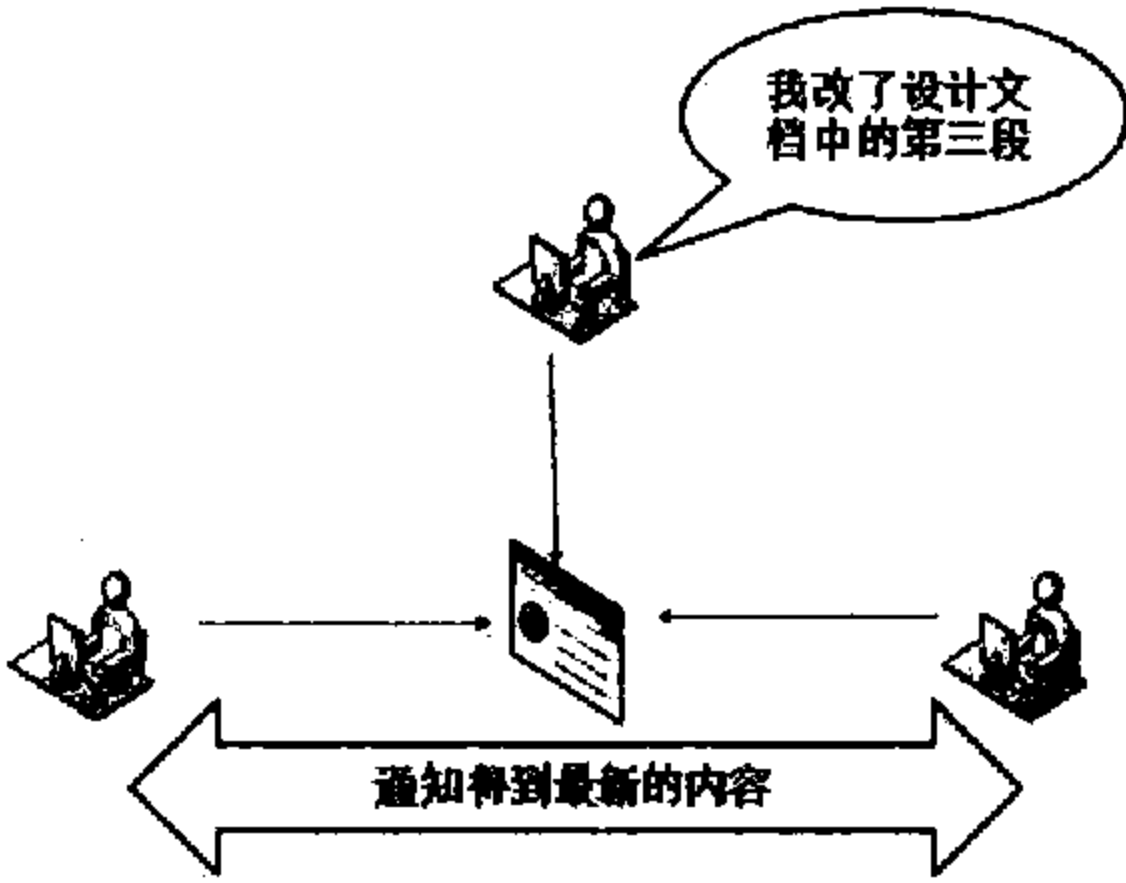


图 2-1 大家一起来修改一份设计报告

这种模型，在 CPU 上的实现就对应于多个 CPU 得到同一个内存单元都有自己的一份拷贝，当某个 CPU 修改它们的共享数据时，需要通知另一个 CPU 已经修改。

内存单元有修改 (Modified)、独占 (Exclusive)、共享 (Shared)、失效 (Invalid) 多种状态，多个 CPU 通过总线相互连接，每个 CPU 的 Cache 处理器除了要去响应本身所对应的 CPU 的读写操作外，还需要监听总线上其他 CPU 的读写操作，通过监听对自己的 Cache 做相应的处理，形成了一种虚共享，这个协议叫作 MESI 协议。这个协议比较复杂，一两句话也说不清楚，胖哥认为 Java 程序员大概知道它有几个基本规则即可（一个 CPU 加载一块内存是专用的；当另一个 CPU 也加载了同一块内存时，那么此时两个 CPU 持有的信息将处于共享状态。两者的区别是，CPU 加载数据前是否监听到“读”广播；当发生写操作时会先尝试发起写广播（此时若有读，则会有一个优先问题），这个广播则会改掉数据，并且相应的其他 CPU 监听到数据被修改后，则会将自己的相应拷贝设置为失效状态）。

一个数据修改了，它需要告诉其他的 CPU 这份数据被修改，现代 CPU 中 Intel 的微架构技术是 QPI 来完成的，其他的 CPU 会根据其 CPU 的架构不同而有不同的设计，不过 CPU 之间的交互是有一个时间差的，通常在 20ns~40ns 级别，频繁的交互会导致系统性能下降，而且 CPU 的使用量会飙高。其实没干多少活，或者叫作“瞎忙”，换句话说，一个公司人多了，大家的工作都很混乱，都还要交互开会，大家都在忙，但是貌似没做多少事情。

那么，有什么样的一些场景呢？例如，一些人设计一个数组来表示一组状态，数组中

多个元素下标分别表示一种“状态”，每个线程独自享有自己的状态。这个设计表面上看好像已经脱离了锁，让它更加具有性能扩展性，然而并非如我们所想，看下面这个例子。

我们定义一个拥有 volatile 变量的对象数组。

代码清单 2-2 一个 volatile 对象数组

```
class VolatileInteger {
    volatile int number;
}
VolatileInteger[] values = new VolatileInteger[10];
for(int i = 0 ; i < 10 ; i++) {
    values[i] = new VolatileInteger();
}
```

此时如果有多个 CPU 使用这个数组中的元素，即便每个“线程”只是操作其对应的某个指定的数组下标，那么数组中的多个元素由于 Cache line 也可能被不同的 CPU Cache 到，每个 CPU 可能只修改到某个元素，但会有大量的交互存在，导致出现大量的 QPI 现象。

疑惑：Java 的数组对象，数组中仅仅包含引用，对象应当有单独的空间，而不包含内容的怎么会有连续的空间呢？

答曰：Java 的堆内存分配时，会相对较为连续地去分配，这样在一些问题的处理上会比较方便。上面的代码是使用 for 循环来分配空间的，分配过程很快，如果没有并发，其实是相对较为连续的；若发生了 GC，同一个对象相关的引用合并后，它们的内存空间也会相对较为连续，所以 Cache line 的概率是极高的。

总结：以此为例只是为了说明有 QPI 存在，让大家知道它大概是怎么回事，并非让大家将代码完成得尽善尽美，因为那样往往会造成过度优化或影响整体代码结构的设计，倡导的是知其内在和在实际场景中做评估、优化时提供参考。

作为 Java 程序员，不用纠结于太多的十分深入的内核细节，但是要知道它是存在的，它会对程序产生什么样的影响，以及如何利用好它的原理。

➤➤ 2.2.5 上下文切换

“上下文切换”(Context switch)中的“上下文”，也就是内容，简单来说就是内容切换。

疑惑：为何有内容切换？何时进行内容切换？切换的内容是什么？Java 系统是如何来做这些事情的？

答曰：说到上下文切换，下面举个小例子。

一天，“猩猩”去办证大厅办理一些证件，需要很多个步骤（这里假设有 10 个步骤），“猩猩”办到第 7 个步骤时，发现“少带了点材料”，需要回家去取，然后再回来“补办”。在他回家去拿材料还没回来的这段时间内，给他办理材料的人不会眼睛直勾勾地瞪着一个地方不动等他回来，这个人会先帮别人办理，自然要重新去看别人的材料。当“猩猩”回来的时候，可能办理材料的是另一个人了，不论是谁给办理都得重新看看这些材料，从还未办完的步骤继续向下办理，最后“猩猩”经历了一波三折，终于将事情办好了。

线程已经执行了一部分内容，需要记录下它的内容和状态，中途由于调度算法（例如分时间片、调用让步或休眠等）或阻塞等原因，导致还未完全结束的线程离开 CPU 的调度，此时需要保留现场，以便于在重新调用时找到它上一次执行到哪里了。最为常见的例子是在 I/O 方面，例如一次普通的网络 API 调用、一次数据库访问、一次磁盘请求（写一条没有缓冲区的日志），此时线程会陷入 Blocking 状态，以至于发生所谓的“上下文切换”。

大多数书籍中会提到进程比线程有更多的开销，进程独立向 OS 申请资源，而线程是共享进程的资源，其实就状态来讲，它们的区别并不是特别大，各有好处和坏处。

目前 CPU 调度的最基本单位是线程，Java 也是基于多线程模式的，而非多进程模式，因为资源共享，在 Java 中一个线程可以让一个静态的数据变大，将一个 JVM 进程直挂死掉，这种情况在多进程模式中不会发生。

通常在实际运行中会有代码段和数据段，内容切换时要保存这些运行中的上下文信息，需要使用的时候还要加载回来，以便于知道从哪个位置继续开始执行，以及执行中的本地信息。

类似于上面提到的日志写操作，现在很多开源的日志工具（如 log4j）都采用异步模式去实现日志写操作，而程序通常不直接参与这个写操作，那么大部分线程就可以直接完成处理。它的实现方式是，日志写操作只是将日志写入一个消息队列中，由单独的线程来完成写操作，如果用阻塞的方式就会阻塞当前执行业务操作的这个线程，这样也同时降低了许多 I/O 的竞争，将许多随机写操作变成了顺序写操作，从而可以提升平均性能。

这种处理方式，类似于在银行排队时，很多人都需要等待，假如有一个代理人员，则可以将任务交给他，由他来代理完成。这个代理人员可以帮很多人做事情，而且可以批量做。

容易出错的地方：这个处理人员何时处理，以及是否确保一定能处理，这个人对于许多问题处理的顺序也很难说，在技术上要根据实际的框架而定。

2.2.6 并发与征用

只要是服务器端程序，迟早会遇到并发。当并发时，就会存在对各种资源的征用，包括对各个部件的征用，例如 CPU。

我写的 Web 程序也算是服务器端程序吧，为什么没有遇到过并发呢？

那是因为 Web 容器帮助处理好了线程的本地变量分配，我们几乎不用关注并发，但并不代表没有并发存在，当程序去访问一些共享的内存区域时，并发问题同样是要考虑的，而这些共享区域并不仅仅局限于自己编写的代码，许多框架也提供了这些共享区域（例如连接池），当多个线程去征用这些数据的时候，都是有开销的。如果在某些特定的场景下需要自己来写这样的代码，你会思考这些问题吗？

当程序中出现“加锁”时（例如 Java 的 `synchronized`），说明这块区域是一块“临界区”，临界区的范围取决于加锁的对象是谁，以及跨越的代码段。如果多个线程尝试进入临界区，那么不论是单核的还是多核的系统（即使有 100 个 CPU），也只允许一个线程进入这个区域，其余的通常会被放入一个等待队列中，进入 Blocking 状态。

在程序中如果你认为多个线程同时访问的一段代码有问题，那么可以尝试作为临界区；如果没有问题，那么就可以并行执行。

通过上面内容的介绍，发现征用就像“独木桥”只允许一个人在上面玩，前一个人下桥后，后一个人才能上桥，如果这个独木桥很长，那么系统就会被逐步串行化，不论有多少个 CPU 也会被串行化，极端情况就是从接收到返回全部被锁住，就算有 100 个 CPU，其性能也是单核的性能甚至更低（因为征用也会有许多的开销）。

征用带来更多的是同步的开销，它会发出许多指令要求在所有 CPU 处理中不允许其

他的线程进入临界区，而且需要将等待的线程放入阻塞队列，当前一个线程退出临界区以后还需要激活被阻塞的线程。

这时我们希望这个独木桥变短一些，变成一截截的独木桥，这就有点像异步了，它将粒度锁定在很短的距离内（例如，当前一个人上桥 10 米后，后一个人就可以上桥了，而不需要等到前一个人过桥），在锁的这个大范围内，可以将一个大锁分解为多个小锁。好像还忽略了一个问题：如果允许一个人上桥，那么在等待上桥的许多人中，是谁上桥呢？此时要么有一个全局的管理者（要过去的人需要拿到管理者的绿卡，而管理者怎么选就不一定了），要么就是大家去争抢上桥的机会。如果将桥分解成多截，那么每一截都有可能存在这样的征用情况，这个开销也是不能忽略的。

在实际的业务场景中锁粒度是有许多优化方法的，它不仅仅局限于“独木桥”的问题上，而是有机会将锁粒度划分得更细，在这个基础上还会产生许多的乐观机制。关于这些问题的细节，将在本书的第 5 章中介绍。

征用 CPU 的访问也不仅仅体现在锁上面，CPU 本身数量是有限的，即使没有锁，一个 CPU 也不会同时处理多个线程的任务，当有多个线程同时要求一个 CPU 来处理时，CPU 会对这些任务进行队列化，CPU 会基于时间片、优先级、任务大小等进行分别调度，这样就会产生“上下文切换”的问题。

反过来说，很多小伙伴在配置 Web 应用的线程池时，往往将“线程数”配置得相当高，系统扛不住压力，就疯狂地增加线程数，结果发现效果可能比增加之前更差了。这是为什么呢？在真正配置这些线程池时，通常“合适”就行。什么样才算是合适呢？根据实际场景而定，并非一个绝对值，在 CPU 密集型系统中理想状态下的线程数是 CPU 数±1，目的是让 CPU 转起来。我们希望看到的不是仅仅简单地让 CPU 的使用量升高，而是让它真正在做许多事情。反过来讲，如果给系统配置了非常多的线程，在调度时自然会有大量的上下文切换，表面上 CPU 的使用在一路上升，但是很多时候是在进行切换中的加载与保存现场的处理，这种开销如果过大，也是不可忽视的。

这个参数的值并不是绝对的，尤其是在 I/O 密集型系统中，本身就需要大量的线程，并不在乎上下文切换的开销。

大多数系统会有一些 I/O 也有部分 CPU 时间，那么它们到底应该如何配置线程数呢？我们做一个简单的分析。

（1）系统 CPU 密集度

一般系统分为计算密集型和 I/O 密集型两种。所谓计算密集型就是指系统大部分时间是在做程序正常的计算任务，例如数字运算、赋值、分配内存、内存拷贝、循环、查找、排序等，这些处理都需要 CPU 来完成，所以也叫 CPU 密集型。

I/O 密集型是指系统大部分时间是在做 I/O 交互，而这个时间线程不会占用 CPU 来处理（但是通常会在系统中记录下这个线程正在等待 I/O，以便 I/O 数据返回时，系统可以将其激活，被 CPU 再次调度）。换句话说，在这个时间范围内，可以由其他的线程来使用 CPU，因此就可以多配置一些线程。

寓意：两个或多个出租车司机租用了同一辆出租车，其中一个出租车司机累了，为了让出租车继续利用起来，就让另一个出租车司机继续开这辆车，以达到出租车的最大利用率（虽然“交接班”时需要单独浪费一些时间，但是可以有效利用这辆出租车）。一种极端情况就是，如果一个出租车司机永远不会累，那么想最佳地利用出租车就要永远不发生“交

接班”。

(2) 关键程序的各项时间比例

其一，普通常规的操作，理论上我们认为这些操作都需要 CPU 调度来处理（不论是否需要将数据从主内存加载到栈或者加载到 CPU 的 Cache 中，总之需要 CPU 来处理），在最理想的情况下，大牛们建议将线程数设置为 CPU 数±1，因为这样设置后，理论上是没有线程上下文切换的，或者说发生上下文切换的概率会小很多（排除系统本身及其他的进程也需要使用 CPU）。

其二，I/O 操作（包括网络 I/O、磁盘 I/O、键盘 I/O、屏幕输出 I/O），很多人会说：如果不做系统之间的调用就不会用到网络 I/O 吧。其实不然，数据库、缓存操作也是网络 I/O。其实在我们的工作中 I/O 无处不在，就算 `System.out.println` 也是一种 I/O。在这里提到的都是阻塞 I/O（在非阻塞 I/O 中，有一个十分短暂的与 Kernel 交互的停顿，而上下文信息很多是通过程序来保存的，因此它的好与坏也是针对实际场景说的）。前面提到，在等待 I/O 相应的时间范围内，允许其他线程访问 CPU，否则此时被挂起的就不是线程了，而是 CPU（计算机的零部件中最慢的就是 I/O 操作，因此只要发生 I/O，它通常占用的时间都会比较长，当然这只是相对的长，相对于内存的访问时间要长许多）。

例如，系统中一段比较“关键”的程序访问总共花费了 120ms，I/O 操作占用了 100ms，那么意味着在这 100ms 时间内 CPU 是可以被其他线程访问的。此时，这个程序在单核系统中的线程数理论上可以设置为 6，在多核系统中自然是乘以 CPU 的个数，一般会在这数字上下浮动。

此时得到的仅仅是理论数字，可能存在很多的实际情况未考虑到，所以需要通过测试来得到更准确的结果，一般关键程序决定大致的配置数量。

选择测试的程序要具有足够的代表性，如果这段程序并不是系统中的关键程序，那么结果是没有什么意义的。选择测试的 CPU 与实际运行的系统 CPU 厂商及型号是否一致，也是重要的参考因素。选择测试的系统配置会不会受到其他因素的影响（如：网速、JVM 的 GC 等）？

其三，“锁”，也就是临界区的范围，前面提到了它有粒度，在这个区域内不论有多少个 CPU，也无法同时进入。因此这部分不好计算，要根据关键程序本身来计算，如果在锁内部发生了 I/O 操作，或有大量的循环、递归处理，那么就必须等待这些操作处理完成后才能有下一个线程进入处理。如果这段程序不是关键程序，那么有些时候我们可以忽略它，但如果是关键程序（也就是被频繁访问的程序），在系统真正并发时，就会导致很多的线程阻塞在这个位置。此时计算线程数，首先要看这个锁的对象是不是静态对象或 Class，如果是，则可以认为是一个 JVM 进程全局锁，那么无论配置多少个线程效果都是一样的（线程多的时候其余的线程还得等待与分配，效率可能会更差），并且在这个时候不能乘以 CPU 的个数，因为锁是全局的，多核也无法并行处理临界区的内容，和 CPU 个数是无关的。如果对锁的粒度、范围做了优化，也许可以根据分段规则、细粒度规则，乘以这个粒度级别（例如：`ConcurrentHashMap` 内部默认分解为 16 个 Segment，数据都是先查找 Segment，再在内部加锁，因此在理想情况下，锁粒度可以降低 16 倍，那么自然的应该允许 16 个并行。当然并不排除由于热点问题，导致某些 Segment 上的请求更多，而某些 Segment 上的没有征用。最坏的情况就是所有的请求分布到了同一个 Segment 上）。在划分锁粒度后，这个值自然不那么靠谱了，因为最理想情况与最坏情况和实际情况十分密切。这个值只能做出一

个预估值，在实际场景中要先看具体的处理方法再进行估算，这是最靠谱的（所谓的公式也只是告知底层原理的另一种方法，但是实际情况往往是千变万化的，很多时候我们甚至希望系统有一些自适应能力，根据实际情况自动来做一些调节）。

（3）JVM 本身的调节

不论 CPU 跑得多么快，如果 JVM “hold 不住”这个节奏，在不断地做 GC 操作，或者因为其他因素跑得“慢死”，那么怎么配置线程池，系统的性能也不可能上来。

很多时候，我们不希望程序在磁盘（Swap）上跑，除非是 SSD 硬盘可以考虑。有些时候，宁愿系统宕机，也不愿意让程序跑得很慢，因为磁盘上跑的程序在不断发生 I/O 操作，我们认为程序很正常，但是外部运行却奇慢无比，而又找不到原因。

可以根据 JVM 运行日志中，平均做 Young GC 的时间间隔（通过 Young GC 与运行时长对比），以及系统统计的 QPS（每秒的请求数），来估算每个请求大致占用的内存大小，虽然不是十分的准，但是有一定的参考价值。

怎么计算？

Eden 空间的大小我们是知道的，通常一个请求分配的空间都在 Eden 区域，Eden 区域满的时候就会发生 Young GC。所以有了 Young GC 的时间间隔，就可以知道多长时间会填满 Eden 区域，例如 3s，进一步通过 QPS 数量*3 得到多少个请求可以填满 Eden 区域，那么自然的可以计算出平均每个请求的内存空间大小。

当然，这也是估算值，并不是那么准，很多时候会有许多不可预见的情况发生，例如 Young GC 的条件并不一定等价于 Eden 区域满，其他的某些条件也会导致出现这种情况，只是我们认为如果发生的概率不高，就可以做一个估算。

通过这些介绍，或许你开始对程序运行的评估有了一些初步的了解，也许你第一遍看这些偏于概念化的内容时很模糊，但是在知识逐步细化的过程中就会逐渐清晰起来。



2.3 内存

内存有什么好说的？所有的程序员都知道它就是跑程序的地方。

没错，它的确就是跑程序的地方，正因为如此，它才是最重要的地方！如果程序员连自己的领地都 hold 不住，那么怎么 hold 住这块领地上的各种建设呢？下面胖哥从 Java 程序员的角度说说内存的一些基本概念，如：地址位数、逻辑地址与物理地址、内核区域与进程分配、JVM 内存和其他进程内存的一些区别、内存与 CPU、内存与磁盘等。

了解了这些基本的概念后，在实际工作中遇到一些相对偏深的技术说明时，也许好理解一些，在遇到某些问题时，头脑中也会有个概念。

内存存在程序中一直处于主导地位，它是磁盘存储与 CPU 之间的一座“缓冲区桥梁”，它与 CPU 通信的速度是磁盘的千倍以上，它几乎没有 IOPS 的概念，带宽也是几十 G 每秒。从容量上讲，它拥有比 CPU 的缓存大出几百倍、几千倍的空间来存储程序和数据，现代的 PC 上大多也已经是 8G、16G 或者更高的配置在运行系统，内存的大小在一些场景下可以决定计算机的整体能力。

前文中提到，CPU 的 Cache 比起访问内存效率要高，主要体现在延迟上，因为内存距离 CPU 有一段距离。但通常系统都有大量的程序和数据需要计算，CPU 的 Cache 大小虽然

也在不断变大，但目前来讲都是以 MB 为单位，目前最大的三级缓存大小也不过几十 MB（一级缓存和二级缓存相对较小），如果缓存变得过大，又会像内存那样涉及各种复杂的算法来管理了。

使用磁盘虽然可以存储比内存更多的数据，但是磁盘需要经过主板的很多步骤才能到达 CPU（通常会经过南桥、北桥）。另外，机械硬盘的 IOPS 本身就很低，即使 SSD 硬盘，安装在原先的 SATA 接口上，由于带宽的限制也很难达到理想中的高效。

内存的容量都是以 GB 为单位的，带宽也非常高，大量的程序运行时都依赖于内存来存储，由 OS 来管理和调度。

接下来简单理解下 OS 对内存的一些管理。

在很多资料上经常会看到“逻辑地址”、“虚拟地址”、“线性地址”和“物理地址”。这么多地址，看得人都要疯掉了。我们是 Java 程序员，或许可以心情放轻松一点，来简单理解这些细节概念。

以 32 位模式下的 OS 为例：最基础的是物理地址，系统会为每个内存单元编写一些物理地址（它是地址总线上的对应关系，每个地址位的 0/1 都可以表达出不同的地址），物理地址可以认为是唯一的，它由内存本身的电路来控制，我们不去考虑它的太多细节，只需要知道它是统一编址就可以了。

所有的“程序中”使用的地址都是虚拟地址（在段式管理中也叫逻辑地址），这些地址在不同的进程之间是可以重复的，所以才叫虚拟地址（就像每个班级中学生的学号都可以从 1 开始那样）。此时小伙伴们会想：通过虚拟地址如何找到物理地址呢？

在程序中为什么要使用虚拟地址呢？用过 C 代码的同学都知道，在编译后的指令中，许多调用的地址在编译阶段就得确定下来，许多方法入口和变量位置在编译时确定了虚拟地址，而真正运行时是要由 OS 来分配实际的地址给程序的。

另外，使用虚拟地址后，地址是可以被复用的，程序并不关注其他的进程是否会用同一个地址（这一点由 OS 分配时来确保），而只关注自己的进程可以用这个地址。

如果没有使用分页机制，那么每个程序会有一个“段”的概念，也就是为进程分配的一段内存区，它的起始位置+逻辑地址（此时逻辑地址就像偏移量一样）得到线性地址，而此时由于每个程序段是单独的一块连续区域，因此线性地址就是物理地址。

若一个进程访问的不是“本进程的内存”，那么就会出现错误，在 Windows 上写 C 程序遇到这种情况会提示“内存不能 read”等错误，进程可能会被 crash 掉。

现代 CPU 启动了分页模型，以支撑较大的内存，分页模型会将内存区域划分成较小的页，物理上大多将其划分为 4KB/页，要管理这些页就需要一些管理者。下面来看看大概的情况。

系统会为每个进程分配页目录，这个页目录也是一个页（大小也是 4KB），这些页是由 Kernel 来管理的（因此 Kernel 需要单独的区域，程序所使用地址并不是从 0 开始的），当通过一个虚拟地址访问内存时，会寻找对应进程的页目录的地址，加载到 CR3 寄存器中。

找到页目录后做什么呢？页目录是 4KB 大小，在 32 位系统中，每 4 字节可以存放一个地址，可以存放 0~1023 共 1024 个地址，因此传入的逻辑地址只需要 10 位就可以定位到一个页目录中一个保存地址的单元（32 位中的高 10 位），这个单元存储的地址就是“页表”的地址。

页表本身也是一个页，大小也是 4KB（有 1024 个页表，所以页表总大小为 4MB），每

个也存储了 1024 个地址，因此逻辑地址中的中间 10 个二进制位就可以标志出页表中的对应单元（每 4 字节为一个标志地址的单元）。页表中的这个单元也许不一定是物理页，可能还需要经过一层转换后才能得到物理页。也就是说，使用 20 个 bit 就可以定位到具体的页地址，在 32 位系统中，最后 12 位就可以在页内通过偏移地址找到对应的字节（这种思想其实在 JVM 的 8 字节对齐中也有类似的用法，按照 8 字节为基本单位的内存管理，地址的最后 3 个字节始终是 0，若利用好这 3 个字节就可以虚拟出 $8 \times 4\text{GB} = 32\text{GB}$ 的空间）。从这里可以看出，这样的寻址方式对于每个进程来讲都有机会得到 $1024 \times 1024 \times 4\text{KB} = 4\text{GB}$ 的地址空间。如图 2-1 所示为页目录、页表和页的大致关系示意图。

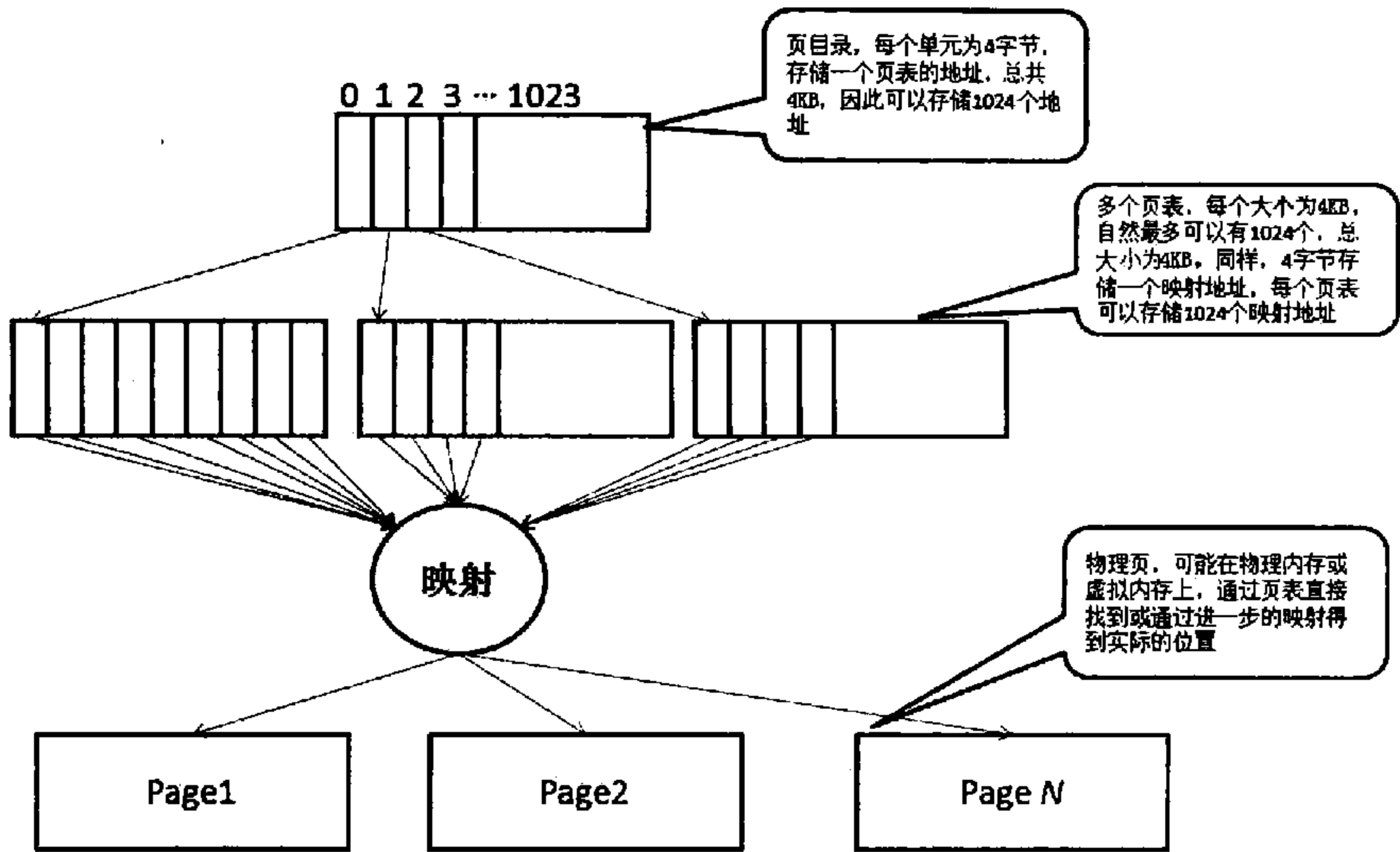


图 2-1 页目录、页表和页的大致关系示意图

反过来，根据虚拟地址，页目录可保存 1024 个单元，因此逻辑地址的前面 10 位的范围是 0000000000~1111111111。同样的，通过页目录找到页表后，每个页表也是 1024 个单元，因此中间 10 位的范围也是 0000000000~1111111111。因此，前面 20 位的地址都是连续的，而后面 12 位是内部偏移量，也是连续的，这样的地址也叫“线性地址”（通常程序要分配一个连续地址，也就是一个线性地址）。

线性地址最终要映射的物理位置，不仅仅是日常所说的物理内存本身，还可以是网卡设备、磁盘配置的 Swap 空间等，也就是当物理内存不够用时，只要地址空间是够用的，那么就可以用其他的设备来代替，只是效率方面需要进一步评估。如果一个进程所需要的空间连地址空间也不够用，那就悲剧了。

在 64 位系统中，面对大内存（例如 500GB），页目录也会变得十分大，此时某些解决方案是增加层次，也就是在页管理上增加层次或采用大页内存来降低页数量，那么访问自然会变慢一点。所以，64 位系统的程序未必就一定比 32 位跑得好。

OS 通常会对“内核区”和“用户区”进行划分，“内核区”通常为操作系统本身所使用，它会管理各种进程、线程、各类设备的状态，管理它们之间的关系对应、程序调度、分配资源、标记资源使用以及资源释放工作。

当一个进程需要申请“一次”分配内存时，操作系统会分配一块连续的虚拟地址内存给它。当然，进程可以多次向操作系统申请内存空间，这些空间最终和进程所对应的页表映射起来，所以当进程退出后，这些资源都会被释放。

在 Java 语言中，主要是看 Heap 区域，当系统参数设置为-Xms、-Xmx 时，JVM 通常是申请一个连续的虚拟地址，也就是 Java 申请一块大内存，通过 OS 预先分配的实际物理内存空间是-Xms 的大小，但是 OS 未必会立即分配一个-Xmx 大小的空间给 JVM 使用，许多空间也是到真正使用时才分配的（大家可以在 64 位系统中做个小实验，启动一个 Java 程序时，-Xmx 设置为比物理内存大几十倍也没问题，但是-Xms 不行）。

线性地址分配好以后，自己来虚拟分配内存空间，对象分配内存时由 JVM 与 OS 交互来完成，这个分配可能会比 C 语言在分配内存上更加简单。但也正因为如此，JVM 所申请的内存空间需要由自己来释放，因为这些地址相对 OS 来讲都已经给 JVM 了，它并不知道什么时候会释放这部分内存空间。

进程向 OS 申请的内存空间首先会从 Free 状态到 Reserved 状态，这种状态仅仅是占用了一个坑，但是还没有真正用这个内存空间，而真正用的内存页的状态为 Committed，这通过对许多操作系统的监控也能看到。

而 JVM 这块大内存要求逻辑地址是连续的，所以会比较苛刻，通常在 32 位系统中，由于 OS 会被占用一部分内存空间，通常 JVM 的内存空间不会被设置得太大，当设置太大时，会提示无法分配内存的错误。

在 32 位系统中，1.5GB 的 Heap 区域是比较合适的，因为 JVM 除了这部分空间外还需要使用很多空间，必须为这些开销预留空间（第 3 章中会介绍）。在 64 位系统中，这个空间几乎不会受到限制，不过这时 JVM 也必须换成 64 位模式的。

2.4 磁盘

说到内存与 I/O，自然就要说到内存与磁盘，而提到磁盘时通常是说它“太慢了”。

为什么这么说？磁盘自存在以来，一直拖着计算机的后腿，发展停滞不前，小的创新没有带来质的飞跃，直到 SSD 硬盘问世。

SSD 硬盘存在的灵感来自于“磁带”（磁带是最便宜、密度最大的顺序存储设备），也就是“机械刻录”和“机械读取”，它发展这么多年来，一直以：提高转速、增加盘片来提升单盘速度，通过尺寸大小、阵列、冗余等方法来提高容量和稳定性，即使是高端存储也离不开这些原理（只是做了更多的冗余、更多的缓存设备、更多的策略与算法来提高性能和保证数据不丢失）。但是由于磁盘本身的磁头原理所限制，磁盘本身的效率一直不高。

市面上的机械硬盘大多以 SATA、SAS 为主，SATA 盘十分便宜，PC 上用的很多都是这种盘（现在某些 IT 公司为了让开发者的机器跑得更好，原装的 PC 也不再使用 SATA 盘了），一些测试服务器也可以使用它，因为这样会比较省钱（当然不是性能测试机，尤其是和存储相关的性能服务器）。

SATA 盘的 IOPS 低的可能只有 60 左右。SAS 盘相对 SATA 盘要贵一些，自然也会更快一些，现在很多数据库的线上服务器会使用这种硬盘，它们的 IOPS 通常能达到 120 左右（一些厂商可能会提供 180 的极限指标，但是测试方法就不好说了）。

直到最近几年,随着 SSD 硬盘的问世,IOPS 的概念逐渐变成了“笑话”,许多优化算法也瞬间被秒杀。SSD 硬盘采用了很多内存的设计原理,而非机械硬盘的转盘思想,可以实现多通道读/写操作,IOPS 开始有了“飞一般的感觉”。SSD 硬盘本身的访问性能可以说已经十分高了,不过由于接口大多还是 SATA,在传统机型的配置上 SSD 硬盘的性能一直达不到它本身的极限,但也已经远远超越了机械硬盘几十倍甚至上百倍(对于 SSD 硬盘,已经有 PCIE Express 的标准正在制订,未来硬盘可能会和内存一样接近 CPU,那又将是计算机的一次革命,所以技术永远在变化,跟得上时代才是最重要的)。

目前 SSD 硬盘的价格偏高,还未完全普及使用在服务器应用中,一些应用服务器也暂时不需要那么高的 IOPS。不过,SSD 硬盘在不断发展中,逐步体现了它的优势,同时由于技术突破和批量生产,它的价格也在逐步降低。

- ◎ IOPS: 磁盘每秒最多可以完成的 I/O 次数。这个 I/O 大家不要理解为一次做大量数据处理的读/写 I/O,一般是指小 I/O,很大的数据读/写也通常会被分解为多次小的 I/O 读/写来操作。
- ◎ 随机读/写: 这是一个相对的概念,磁盘本身在每次发生读/写操作的过程中,需要寻找位置再进行读/写操作。由于频繁地发生磁盘小 I/O 请求(例如 4KB),所以就频繁地寻找位置,就像是随机读写。
- ◎ 顺序读/写: 是相对随机读/写来说的,每次读/写的数据会比较大(例如 1MB,对于过大的数据块读写,也会拆分为多个小 I/O 请求)。简单来说,如果找到了位置,那么处理 1MB 和处理 4KB 的数据时间是差不多的。

接下来我们一起来了解下机械磁盘定位数据的基本原理。

1. 机械硬盘基本原理

硬盘,用于存储数据,存储空间十分庞大,比内存大得多,而且它在断电的时候不会丢失数据,所以存储在硬盘上的数据又叫持久化数据。

加载程序也需要从硬盘中读取,它是在进程启动时被加载的(有些语言也会按需加载),但程序所占用的空间本身不会太多,这部分问题不大。在程序中做得最多的是对数据的读/写操作,数据读/写就是 I/O,机械硬盘与内存通信,一般许多硬件架构中的信息还会经过“南桥”(South Bridge)再到“北桥”(North Bridge),传送过程中的通信带宽被大大降低。

不过,对于机械硬盘来讲,效率低的根本原因不是带宽,而是 IOPS,也就是机械硬盘“本身的读/写效率”。那么机械硬盘到底如何读/写数据呢?

首先,它有一个“摆头”,就像“百年前的留声机”的摆头,摆头在位于半切面的位置摆动,在盘片上进行“寻道”操作(一块盘上有多个磁道),找到数据所在的磁道后,通过硬盘本身的不断“转动”转到数据实际的起始位置,才开始真正读取数据,而读取数据之前的时间,都可以被认为是延迟时间。硬盘查找数据基本示意图如图 2-2 所示。

而摆头能否快速找到磁道,一般情况下和尺寸有关系,而磁道内寻址和转速有关系,所以硬盘厂商一般都会提供这些参数。例如 15000 转/分的磁盘,磁盘旋转一圈的时间是: $1000\text{ms} \times 60\text{s} / 15000 = 4\text{ms}$ 。一般来讲,厂商通常会按照“半转”的旋转时间作为“平均寻址延迟时间”,所以这种硬盘的寻址延迟时间大概是 2ms,加上寻道时间 3~4ms,也就是 5~6ms 时间(大概是这个量级,不同厂商会有些区别,不过大家应该看得出来,这个延迟是 ms 级别的,而内存是 ns 级别的,两者根本不在一个数量级上)。

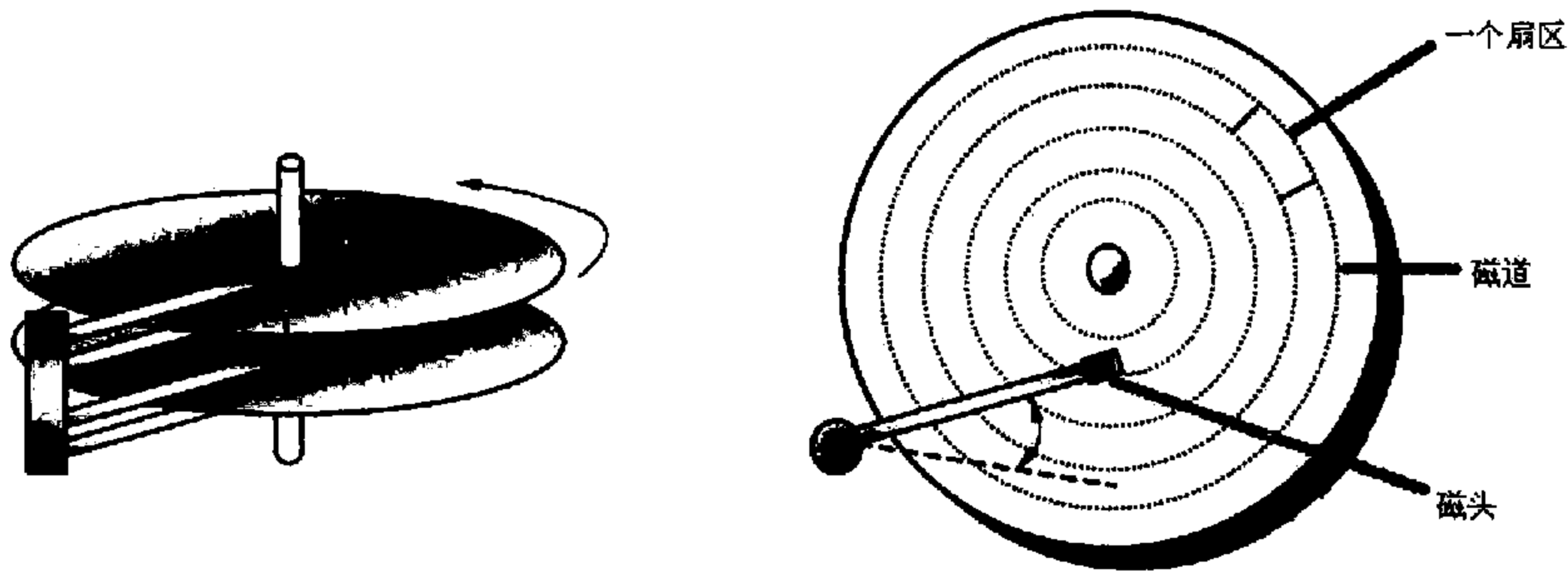


图 2-2 硬盘查找数据基本示意图

当硬盘找到数据后，前面提到加载 4KB 和加载 1MB 数据的时间差别并不大，所以往往许多小的 I/O 就是 KB 级别的，会不断地随机读/写硬盘，不断地寻道和寻址，那么自然在每次 I/O 时都会有这样的延迟。因此，在随机 I/O 操作较多的系统中，IOPS 成为其重要的量化标准。

除此之外，一些硬盘厂商在同一块硬盘上使用了“多个盘片”，这是用来干什么的呢？有的小伙伴们可能会认为这是用来提高 IOPS 的，事实真的是这样吗？

磁头摆动依然还是串行的，虽然在每个盘片上都有磁头，但是它只是为了提高读/写的效率。可以将数据分布写到多个盘片上，例如 3 个盘片，在写入数据 1、2、3 的时候，就可能将 2 这个数据写到不同的盘片上，读的时候会被重新组合起来，那么在磁头定位到数据后，读/写的效率可以提升数倍，但摆头定位磁道及磁盘本身转动到数据位置的时间依然不会减少。

因此可以发现，这样的方式对于顺序读/写的操作，可以加快读取的速度，而对于随机读/写似乎作用不大（因为大量的随机读/写将导致大量的 I/O 请求，而每次请求都会存在摆头定位磁道及磁盘旋转到指定位置的延迟）。

硬盘这么慢，能支持全世界的“商业数据”及“互联网数据”这么多年，靠的就是阵列、存储厂商提供的各类大型存储设备。阵列就是大家常说的“RAID”，它是一张卡，相当于一层硬盘的代理。有很多种 RAID 方式，现在做得最多的就是 RAID 5 和 RAID 10。

RAID 5 可以实现硬盘负载均衡，并且可以在坏掉任意一块硬盘的情况下，将内容进行恢复，但是不能坏掉两块或更多的硬盘（如果有 3 块硬盘做 RAID 5，那么比一块硬盘的读/写性能理论上提升 1 倍，其中任意一块坏掉，都可以通过另外两块找回数据）。

RAID 10 可以实现一半硬盘的负载均衡效率，每一块硬盘都有一块冗余的硬盘和它存储一样的数据（读操作访问其中任意一块都可以）。在这种情况下，只要不是相邻备份的两块硬盘“同时”坏掉，就不会丢失任何数据。

我们是搞 Java 的程序员，所以对于硬盘的太多细节就没有必要知道了，而胖哥介绍这些内容是为后面程序使用服务的。

2. SSD 的一些应用扩展

SSD 是最近几年问世的，它算是传统硬盘的一个终结者，也许这句话对于很多人来说并不是那么好听，但这确实是事实、趋势。

SSD 问世后, 瞬间秒杀许多公司多年来的技术储备, 尤其是许多大型存储厂商瞬间感觉到了压力, 因为它的 IOPS 能力超越传统机械硬盘上百倍甚至上千倍。现在在很多服务器上拖后腿的是一些接口带宽和架构本身的限制, 因此大家也在极力去改变现状。

很多要求高 IOPS 的数据库服务器 (在数据库的联机事务处理系统上, 大多为随机读/写操作, 当然一些商业数据库也会采用方法来尽量进行顺序读/写) 已经开始大量采用 SSD 硬盘, 而传统 PC 也开始有很多开发者使用 SSD 硬盘用于内存的扩展, 以弥补程序设计“内存不够用”的不足。

最近几年, 服务器市场的 SSD 硬盘由 300GB、600GB、1TB、2TB 逐步增大, PC 市场也从 8GB、16GB、32GB、128GB、256GB、512GB 不断增大, 价格也在每年以 20% 多的速度下降, 越来越多的人青睐于 SSD 带来的效率提升体验, 它在带来商机的同时, 也让很多苦苦钻研传统硬盘架构以及高端存储商家感到了生存压力, 他们也在寻求新的生存空间, 如果不做出改变, 再大的公司也有可能走入没落。

总之, SSD 硬盘的问世, 改变了很多硬盘厂商、存储底层程序设计者、DBA 的理论和经验, 乃至颠覆了很多应用程序设计人员对于存储认识的初衷, 一个创新的想法带来的改变是巨大的, 它拥有无穷魅力而且又变得十分残酷, 可以颠覆你原本认为是真理的事物, 所以社会的进步不仅仅需要辛勤劳动, 也同时需要创新理念来改变这个世界。

3. 在 Java 程序中使用硬盘吗

通常程序和磁盘直接打交道的就是写日志较多, 通过网络 I/O 让数据库访问磁盘也间接地算是一种硬盘操作。有一些工程类的软件, 它们专门用于处理文件类, 也会使用到磁盘。因此, 在程序中其实有很多时候会直接或间接地使用到硬盘的。

当然, 数据库一般由一个 DBA 来关注它的实现, 而开发者需要懂得你写的 SQL 大概对 I/O 有多大的影响, 因为系统的瓶颈会在这里。将这个思路再放大一些, 在设计上或许采用相对批量、本地化操作来提升整体效率, 这就是本书第 6 章要介绍的一些基本知识。

而对于硬盘本身, 拿日志来讲, 许多日志写工具, 会将日志替换为 Buffer 的 append 方式, 它将日志写入一个缓冲区中, 由专门的程序来实现日志写操作, 或者写的时候发现缓冲区已满, 则本次写入磁盘。这样一来, 一次写入多条数据, 每次写入量就开始变大, 也就是将随机 I/O 逐渐变成顺序 I/O 操作 (将多次硬盘寻道、寻址“尽量”变成一次)。

所谓“尽量”当然就是“不确保”, 一个原因是磁盘上可能有磁盘碎片会导致重新寻址, 所以碎片清理也很重要。

另外是 Buffer 多大的问题, 过大的 Buffer 一次写入磁盘时也并非是一次性写入的, 而且过大的 Buffer 会占用 JVM 的内存空间, 长期占有或过大就会占用 Old 区域的空间, 使得它更容易做 FULL GC。因此, 在有些问题的考虑上, 我们也不能过分强求, 关键要找到契合点。

2.5 缓存

2.5.1 缓存的相对性

小伙伴们在网上、书籍中到处可以看到 IT 行业的一个技术术语: 缓存, 这个术语满天飞。

CPU 上有缓存, 系统架构上有缓存, 存储上有缓存, 分布式上有缓存, 数据库上有缓存, 应用上有缓存, ORM 框架上有缓存……缓存是什么?

胖哥也无法给你非常准确的答案, 但胖哥的解答会给你提供许多思路, 胖哥认为“缓存是相对的概念”, 在计算机中, 只要“就近”就可以被认为是“缓存”。

“缓存”根据用途可能会有不同的架构和原理, 但是目的都是为了提高性能和效率, 而方法就是“就近原则”。

所谓“就近”是指需要使用的数据所在的位置可以靠得更近一点, 这样“更加亲密一些”, 自然相互之间的沟通延迟就更小了。

为了说明缓存, 先来提出几个“无聊”的问题: 具体的缓存实例是什么样的? 缓存的用途和场景有哪些? 缓存不适合用在哪些地方?

□ 缓存的简单实例, 引导下我们的思路。

对话 1:

一休说: 小蜗牛, 在代码中把这些数据缓存到内存中吧。

小蜗牛说: 是的, 不怎么修改的数据放在内存中访问会快很多的。

——将磁盘上的数据放在内存中, 能得到快速的访问, 这个我们可以认为是缓存。

对话 2:

Sleeping 说: 我们的网站其他省的朋友访问太慢, 主要是网络延迟引起的, 网站中大部分流量来自 JS、CSS、图片, 希望在全国很多地方缓存这些信息, 这样全国各地的朋友可以就近访问, 那么效率自然就提升了。

豆豆说: 是啊, 这些信息占用了我们网站 80% 以上的流量, 这样做后外地朋友访问网站的速度肯定会快很多的。

——这些信息相对于“使用者”靠得更近, 也叫缓存。

对话 3:

仙仙小说: 这个浏览器为什么比那个浏览器访问那个网站要快很多呢?

胖哥说: 除了浏览器本身对域名解析、交互、网站输出的内容解析以外, 浏览器会缓存很多内容在本地, 具体要看浏览器是如何处理的, 而处理得好与坏就是浏览器性能的关键因素。

——我们上网的时候, 根据浏览器设置会适当将服务器上的静态资源缓存到本地的磁盘上, 这也称为缓存。

对话 4:

小张说: 某些访问的数据出现热点怎么办?

小李说: 将热点数据缓存在好的服务器上、分布式缓存或 SSD 硬盘上, 在访问时自动路由到这上面。

——由于材料不同, 效率也完全不同, 有时候我们也简单地说这是缓存。



注意 将数据写到磁盘上, 写入前可以 Buffer 住一段数据, 大小也一般可以控制, 但这通常不会认为是 Cache, 只认为它是一个小的数据 Buffer 作为缓冲。

其实缓存无处不在：数据库的 LRU 队列及 Keep 在内存中的存储过程也可以认为是缓存；使用 ORM 框架中类似的 Hibernate 和 iBatis 也有许多缓存机制；CPU 的三级缓存可以认为是缓存；操作系统的 oscache 也可以认为是缓存；在高端存储中为了提高磁盘访问效率也会使用缓存；在负载均衡设备中也经常会缓存部分静态页信息，用于直接输出，这也可以认为是缓存；将写少读多的数据放在类似的 memcached 中，这也是一个专门的分布式缓存。

因此，说到缓存，并不一定非要联想到缓存设备，也不一定要联想到 CPU 的缓存。总之，不要将想法硬绑定到某种定义上，胖哥认为它是一个概念，一个就近图快的概念，它以不同的形式来解决不同的问题。它可以被持久化，也可以不被持久化，在实际的场景中只要访问的利用率能达到自己想要的最佳效果即可，而缓存本身是相对的概念。

2.5.2 缓存的用途和场景

如果自己设计系统，如何掌握缓存的应用场景？胖哥可不是拍脑袋，即使拍脑袋也要有点道理，而是希望总结它是否真的适合使用缓存。

胖哥偶尔会看到一些朋友神奇的问题，例如：“胖哥啊，我们这里有海量数据，并发！”。

胖哥“严阵以待”看看具体的场景。

接着收到咨询者传来的消息是：“我们有好几十万数据，每天有好几千次访问，太多了，我怕系统处理不过来，胖哥你帮我看看系统架构该怎么做，要不要使用什么架构、分布式、大数据……”

胖哥差点昏过去，其实这类系统无所谓用什么，代码没什么大问题就好，除非以后访问量会上涨几个数量级。计算机的时代不再是原始社会，可谓是“天高地大”，这个数据和访问量至少要高出两个数量级以上，才会考虑一些高可用的问题，除非系统设计本身就存在很多问题。

从胖哥本人的看法来讲，首先要确定系统的缓存用途，例如要给应用系统构建缓存，做一些数据的缓存。在集群环境下，肯定优先选择分布式缓存，因为如果本地缓存的话，集群中各台服务器之间的缓存同步会成为一个问题。

“分布式缓存”是一个共享的服务中心，一般不需要程序过于关心，程序员只需要通过 API 去访问它就可以了。这个服务中心可以是单机，也可以是集群，总之：提供数据的独立缓存服务。一般来说，要读取数据先到缓存中读取，如果读取不到再到数据库中读取，并放在缓存中，如果数据被修改了，那么缓存中对应的数据将失效。

分布式缓存多种多样，我们在这里只谈它的基本原理，具体的大家可以参考各个厂商的手册。

这个看似简单的操作原理，其实在网络中实现起来可能会比较费劲，主要是因为要保持一致性。另外，读者朋友是否发现了一个问题：如果发起访问 10 次，有 5 次都在缓存中未命中，那么可能这 5 次操作既要访问缓存又要访问数据库；如果缓存中大量的写操作要与数据库同步，那么此时既要写缓存又要写入数据，这样有可能比直接访问数据库更慢。

缓存用于“读多写少”的场景，数据中存在大量的读操作、少量的写操作，这是一个比例，通常读的比例比写的比例要大几倍、十多倍甚至几十倍，这时就会利用缓存的优势来降低数据库的压力（很多人都喜欢将用户登录后的基本信息放到缓存中，其实也就是一

个用户在登录时发生一次写操作)。

很多时候缓存用于“非强制一致性”(在分布式环境下,由于网络延迟,也很少有架构会完成绝对一致性),即使是现在的数据库,利用缓存的优势也是通过非绝对一致性完成的(数据库会通过日志、冗余等方法来完成丢失数据的回补工作)。宕机后有可能丢失少量数据是存在的,有些缓存是完全用内存实现的,自然更加严重,但通常我们认为宕机的概率很低,再加上这种机制可以带来巨大的性能优势,在某些场景下去做是值得的。

通常缓存的访问效率很高,可以简单做一个对比:对于普通的 MySQL 数据库服务器,不论采用哪一种存储引擎,单机的 QPS 达到几千就很高了;但是对于缓存就完全不一样,可以达到几万甚至十万以上,主要原因是缓存基于 K-V 结构,它剥离了数据库原本存在的许多复杂的一致性概念,而且大部分情况下是在内存中完成的,通常它就处理简单的存取操作。

胖哥这里说的数据库是采用 PC-SERVER 的数据库机器,没有使用高端存储或 SSD 硬盘,一个原因是如果用得起高端存储没必要用 MySQL,另一个是高端存储内部本身有缓存。

总而言之,如果缓存的命中率可以达到 95%以上,那么系统肯定算是比较成功地利用了缓存;反过来,如果缓存命中率很低,那么可以考虑放弃用缓存。

换个角度讲,缓存让复杂性开始变强,让一致性不是那么容易保证,因此它对可用性的要求肯定没有数据库高,它存储的数据通常可以允许丢失一点点。

在一个传统的应用系统中,访问量不大,非要用缓存,这没有错,但要看应用它是否足够简单。其实在绝大部分的传统应用中,数据库就足以支撑这些信息,只是很多时候大家自然地认为必须要用缓存。

2.6 关于网络与数据库

➤➤ 2.6.1 Java 基本 I/O

I/O 的意思是输入/输出 (Input/Output), 只要是与内存程序通信的就可以将它理解为 I/O。

疑惑: 怎么理解 CPU 呢? CPU 不是也与内存通信吗?

答曰: CPU 相对内存来讲级别更高, 内存中的数据最终由 OS 调度给 CPU 来处理, 也就是其他的设备要处理的数据通常会先放入内存中, 然后再得到处理。而内存中处理好的数据反馈到某个设备上, 这才算输入/输出。CPU 是真正处理的实体, 不算 I/O 设备。

感悟: 原来内存也是个跑腿的, 可以说是一个“高级跑腿”的, CPU 才是老大, 内存是 CPU 的“大护法”, 所有人要见 CPU, 一般首先要通过内存这一关。

疑惑: 经常见到的 I/O 有哪些?

答曰: 比如, 向硬盘中写入数据, 或者从硬盘中读取数据, 这就是 I/O; 和远程的程序通信也是一种 I/O; 向控制台输出数据还是一种 I/O。

感悟: 似乎理解了一点点, 好像输入/输出是针对内存程序来讲的。

补充: 没错, 就是这样, 相对于程序内存, 向其他的程序或设备写数据, 称为“输出”, 从其他的程序或设备读取数据, 称为“输入”。

感悟: 内存虽然是个跑腿的, 但依然是程序运行的中心, 输入/输出只要考虑“它就是我的家”, 谁进家门就是输入, 谁出家门就是输出, 就这么简单。

在 Java 世界中,可以说没有 I/O 就没有什么生产价值,最为重要的是在网络 I/O 层面,网络 I/O 的存在,使得系统之间的通信成为可能,同时使得服务器与客户端进行通信成为可能。

浏览器访问 Web 应用服务器,这就是一种 I/O (因为浏览器本身就是一个客户端,只是它是满足于通用协议的客户端); Web 应用系统访问数据库,和数据库服务器之间的通信是一种网络 I/O 操作; 数据库访问磁盘上的数据是一种磁盘 I/O 操作; 数据库“主备”之间需要备份是一种网络 I/O 操作; 某个单独的服务系统为其他系统提供服务 (例如缓存), 两个系统之间访问者和服务者的通信是一种网络 I/O 操作; 负载均衡设备与各个应用系统之间的通信也是一种网络 I/O 操作……

由此可见, I/O 操作在我们的工作中无处不在。

►► 2.6.2 Java 的网络基本原则

疑惑: 计算机 I/O 交互在网络传输过程中到底是什么样子的?

答曰: I/O 传送数据内在都是基于字节 (byte) 的,当然也可以说是二进制数据,不论你的程序是通过字符还是对象等方式,都会经过中间层处理转换为字节后发送到 I/O 端进行交互,或者从 I/O 端读取二进制数据转换为字符或对象。

疑惑: 为什么不可以用字符传输啊? Java 中的 I/O 不是有字符流吗?

答曰: 每种语言表达字符的方式不一样,而在计算机内部表达字符是用数字的。换句话说,每种语言表达字符未必会用同一种数字,但数据交互只有数字的概念存在,如果用这种方式发送数字,就会发生接收方可能无法正确解析内容的情况。

将字符转换为字节时,通常会指定一种编码,这种编码有国际规范,没有语言界限,而接收方会用相同的编码方式进行解码反转为字符,这样就能得到正确的字符。

Java 之所以有字符流是因为它在程序中做过包装,真正在做网络交互时需要换算为字节来进行。

疑惑: 要传输一串中文字符,网络中是什么样子的?

答曰: 在计算机设计之初,计算机中原本只有二进制 byte,为了让人识别信息,才有了“字符”的概念,字符其实是计算机中使用数字表达的,我们看到的只是屏幕上最终的输出,作为程序员应当理解计算机内在是数字,也就是一连串的二进制数代表一个字符。计算机内部运行的其实只有数字,在网络传输过程中也是这样,都只有二进制数,没有“字符”的概念。

对于任意一个字符 (也包括汉字),使用什么数字来表达,有很多种方式,这就是“字符集”的概念,全世界有很多种字符,以 ISO-8859-1 为基础,最初只能表示 256 个字符,后来发展出各种各样的字符集,在中文界的代表为 GB2312、GBK、GB18030,目前支持中文字量最大的是 GB18030,而 UTF 系列的通常是全球码,理论上支持世界上所有的字符,例如 UTF-8 就是大家熟知的字符集,Unicode 也是大家熟知的,对应到 UTF-16。

简单来讲,每一种字符集都会对它所包含的字符进行相应的数字编码,因此字符集是字符与字节转换过程中的公式或映射表 (但这并不代表每一种字符集对同一个字符的编码是一样的数字,也并不代表一种字符集的字符编码数字在另一种字符集中肯定能找到,因此就可能会在 I/O 交互过程中出现所谓的乱码问题)。

一般在 Java 程序运行中, char 代表字符,双字节编码,能表示几乎所有的常用字符。

在进行 I/O 操作以前，需要将字符转换为字节，此时就要视具体情况来选择字符集进行编码了。

例如，一个汉字通过 UTF-8 编码，它一般被编码为 3 个字节，一个英文字母被编码为 1 个字节；相反，接收方在接收到字节串时，会将字节流解码为字符，将英文字母的 1 个字节转换为对应的字符，而将 3 个字节转换为对应的汉字（它们区分的条件是 UTF-8 编码中的前缀字节通常代表语言），接收方需要知道编码的字符集才能正确地解码。简单来说，发送方用什么编码，接收方就要用什么来解码，如果发送方使用 UTF-8 编码，而接收方使用 GBK 来解码，那么肯定是无法得到正确结果的，而且一旦转错，就无法转回来，但有的字符集是可以转回来的，如 ISO8859-1。

为何 GBK 和 UTF-8 转错后无法转回来？

17.2 节会详细地阐述一些关于字符集的细节，本节知道有字符集的概念即可。

在网络传输中只有字节，没有字符的概念，如果程序中没有使用字节来传输，那么 Java 就会包装一层，你可以自己设置字符集，若没有设置，则一般使用与“环境变量”相同的字符集来编码，这个字符集可以在程序中使用 `Charset.defaultCharset()` 来获取到，只是默认的字符集会被环境所驾驭。

疑惑：接收方有没有办法自动知道发送方使用的字符集是什么？

答曰：在真正编写代码的时候，接收方要知道正确的编码字符集有两种方式：一是双方约定好编码，然后再进行交互；二是在数据的头部包装字符集（大家不用过分注重头部的意思，其实它就是数据发送中标识数据元信息的前缀部分）。例如浏览器和服务器的交互非常多，它们就可以使用头部的 `contentType` 参数作为传输数据类型、字符集的标识，从浏览器向服务器提交数据可以包装在头部（当然也可以不包装，通常是约定一种字符集，服务器就用这种字符集来解析），服务器通常在没有用户手工设置字符集时采用这种字符集来解析。如果服务器输出时告诉浏览器这一次传送的数据是什么字符集，浏览器就能得到正确的编码，只要点对点保证即可。

因此，不要认为在一个页面的渲染过程中，必须要让页面本身、JS 文件、CSS 文件、Ajax 交互、数据库、应用启动参数都使用相同的字符集，那是无奈的做法，也是只知其一不知其二的做法。

思考：接收方如何知道字节流结束了呢？

一般是双方约定一种结束标记，可以是一些字节串（相对靠谱），也可以是一些可识别的字符串（不靠谱，但很简单），或者在传输数据的头部指定位置标记 `body` 部分的字节长度。

接收消息结束后，只能代表本次交互结束了（即接收方认为本次交互已经拿到了一个完整的数据），并不能代表两者之间不再交互，所以双方是否继续保持连接，保持多长时间的连接，完全由自己来决定。

那“输入/输出”的概念在这里怎么体现呢？

对于发送方来讲，就是输出数据，可以是客户端，也可以是服务器端，只要是输出数据给对方就是“输出”（Output）。

接收方接收数据就是“输入”（Input），也同样不分服务器端和客户端，因为它们一旦建立连接就是相互的，只是在某一个时间片里其中一个输入，另一个是输出。因此，“输入/输出”的概念都是相对自身的程序来讲的，不是一个整体的网络概念，更不是说服务器

端只有“输出”的概念。

应该说 Java 的所有 I/O 都会经历一个 Socket 或 Channel 的过程，内在的基本道理差别不大，在输出数据时，这些数据一般都会先经过 JVM，然后到 OS 管理的一块内核态区域，再由 Kernel 去完成输出的交互，输出到网络或磁盘也类似这样。

输入的过程正好相反，只是在输入过程中通常会让程序发生阻塞，也就是程序在发生 read() 操作时，在 I/O 尚未响应之前（确切地说是 Kernel 未准备好数据之前），线程会被阻塞（Blocking）。

疑惑：为何程序会经历那么多的拷贝才能做 I/O？能否少拷贝几次？

答曰：因为 JVM 的内存是自己管理的，是虚拟出来的，数据都在 JVM 中，要输出数据其实最终要靠 OS 和网络，理论上说必要的拷贝是必然的。而能否少做 I/O，这在某些场景下的确可以做到（例如可以用 DirectBuffer 来完成），更具体的内容我们将会在第 4 章中介绍。

疑惑：程序是否可以不同步、不阻塞，当有返回值时，再做处理？

答曰：由于同步、阻塞中存在许多资源闲置情况，所以系统和语言设计者必然会考虑如何处理这些问题。在 Java 的 I/O 模型中，包含了对异步、阻塞的一些处理，上面提到的 I/O 又叫传统 I/O，或者叫 BIO（Blocking I/O，阻塞 I/O），而非阻塞 I/O 是在 JDK 1.4 时提出的 NIO（New I/O），在 JDK 1.7 中增加了“异步 IO”，也叫 NIO 2.0，又叫 AIO（Asynchronous I/O）。关于它们的细节，将会在第 4 章中详细介绍。

疑惑：Java 框架 Webservice、EJB、MQ、JDBC、HttpClient、FTP，包括类似于终端 LED 显示屏、浏览器与服务器的交互等通信方式，都是基于 Socket 或 Channel 的方式吗？

答曰：基本可以认为底子都是这样的，大多只是协议上的区别，它们会包装出不同的传递方法。例如在 RMI 中，在 Java 对象序列化时，除了传递数据外，还需要传递对象的结构，对象的结构更像 Class 文件，它们有自己独特的字节格式约定，这些细节不需要应用程序开发人员去思考，已经由 RMI 封装。Java 这样做的目的之一是想将网络的交互变得和 JVM 进程类似，使用面向对象来处理数据。

很多同学在学习这些框架时往往有一种“狐假虎威”的心态，认为用了这些东西就很牛（这些是 Java 通信、分布式的基础，会用它们并不代表很牛），而且某些同学可能会认为这些框架中包含了很多不是 Java 本身提供的机制，而是框架自己来实现的特征。胖哥认为这是一种可怕的现象，因为大家沉迷于浮于表象的东西，而忽略了它的本质特征，自然在遇到问题时很难驾驭。

总的来讲，只要是基于 Java 开发的软件或工具，在基本功能和语法层面都要基于 Java 语言本身的机制，框架都是在 Java 语言本身提供的机制基础上做了很多包装的。

如果要扩展底层的机制，则需要利用 JNI 来扩展（也需要 JVM 支持这种扩展）。例如，SAP 公司就有它单独的一些 JNI 对底层做一些扩展调用，Tomcat 中的 APR 通信协议也是自己的 JNI 扩展，由于 JNI 需要更底层的 C 语言来编写，它需要考虑各种操作系统的情况，因此这样的扩展将导致系统对本地的依赖性增强，并且可能会由于 JNI 代码在某种 OS 上引发 Bug 导致整个系统崩溃。

一切源于对基本原理的理解，你是否曾经想过自己所学都源自计算机最基本的原理，因此胖哥认为：

技术能力：用过不代表你的技术牛，没用过不代表没有能力用或没有能力处理问

题。胖哥曾经为别人处理过许多问题是自己从来没接触过的，使用者一个月没搞定，胖哥 10 分钟搞定了，为何？

其实有些时候不需要知道太多的使用细节，便知道它是如何实现的，因为你的技术功底和解决问题的能力在那里。

这并不代表你知道了胖哥所讲的这些内容，瞬间就能达到这个能力，胖哥希望你从现在开始逐渐找到自己提升功底的方法。技术是相通的，一切在于基本原理的变化，变化源于现实需求的抽象，才会演变出各种各样的技术，但是它们都是基于 Java 的，解决问题的思路和方法都是类似的，出现的问题类型和点也是类似的。

疑惑：在 Java 的 Socket 中主要关注哪些参数呢？

答曰：在网络通信中，参数非常多，不过得知道一些比较重点的参数，例如是否能连通、心跳、对方是否已经关闭程序等。

对应的手段通常是连接超时、读取数据超时，它们在 Socket 中都提供了参数，在 Socket 原始的连接方法中，有一个方法是：

```
connect(SocketAddress endpoint, int timeout)
```

当设置了 timeout 参数后，就可以达到连接超时的效果，而通过调用方法 setSoTimeout(int timeout)，可以设置每次读取超时。

如果不设置这些超时参数，那么当发起网络连接时，网络是通的，但是协议不通（指对应的端口未打开，如果网络不通，则通常直接抛错：No route to host: connect），则可能很长时间（通常是 180s）才会报出错误：connect timed out；而发生 read() 操作若未设置超时，则将会永久等下去，直到进程关闭或被 OS 的 Socket 自动超时所触发，所以超时对于程序来讲是十分重要的。

Socket 是通信的基础，胖哥在这里再补充几个 API。

◎ setReuseAddress(true|false)，该参数设置为 true，允许将多个 Socket 绑定到同一个端口上，通过 getReuseAdree() 方法获取当前值即可，但是在发生 Socket.bind() 之前必须先设置才会生效。这个参数很少会去考虑使用。

◎ setTcpNoDelay (true|false)，该参数默认为 false，会启用 Nagle 算法。启用这种算法是为了避免网络阻塞：当 Socket 交互双方存在大量的小数据交互时，例如 1 个字节的网络交互，还会携带 IPv4 中 20 个字节的包头（IPv6 中为 40 个字节），还不包括数据链路层对网络包的进一步封装。在这样的情况大量发生时，就会导致网络负载增加几十倍。Nagle 算法为了解决这一问题，将较小的包延迟发送（最长 200ms），若 200ms 内有多次小数据发送，则会合并在一起发送，若数据达到一定的范围，也会立即发送。这虽然解决了网络堵塞的问题，但是同时给我们带来的问题是 Socket 双方的许多小包交互最少会延迟 200ms 以上，尤其是一些状态交换，因此发现这类简单的小包通信，也需要 200ms 以上的时间。将该参数设置为 true，即可瞬间提升性能，在网络较好的情况下应该为 1~2ms。同样，该参数打开后可能会导致网络堵塞，所以对于许多小数据的发送可以在本地做 Buffer，一次性发送既可以得到速度上的提升，又可以减少网络负载。

◎ setSoLinger(true|false,int linger)，该参数决定 Socket 关闭时是否尝试继续发送 Kernel 缓冲区中还未发送出去的数据，若设置为 true，则由第 2 个 int 类型的参

数决定发送 Kernel 缓冲区中还未发送的内容最长的等待时间, 单位为秒, 通过 `getSoLinger()` 可以获取到设置的值。

- ◎ `setSendBufferSize(int)`, 设置发送缓冲区的大小, 默认值为 8192 字节, 通过 `getSendBufferSize()` 得到当前值, 一般保持默认就 OK。
- ◎ `setReceiveBufferSize(int)`, 设置接收数据的缓冲区大小, 默认值为 8192 字节, 通过 `getReceiveBufferSize()` 来获取。
- ◎ `setKeepAlive(true|false)`, 它和前端的 `keepAlive` 是有区别的, 它的原理是每隔一段时间 (很长的一段时间, 例如 2 小时) 会将数据包发送到对方, 如果对方响应, 则认为连接依然存活; 如果未响应, 则在十多分钟后再发送一个数据包; 如果对方还未响应, 则再过十多分钟继续发送一个数据包; 如果对方还是未响应, 则会将客户端的 Socket 关闭, 在某些情况下设置该参数后可以避免客户端的一些垃圾 Socket 永远无法关闭的状况。该参数的默认值为 `false`, 可以通过 `getKeepAlive()` 获取到当前值。
- ◎ `setOOBInline(true|false)`, 这个参数默认为 `false`, 若开启, 则允许通过 Socket 的方法 `sendUrgentData(int)` 发送, 这个 API 是直接发送, 不会经过缓冲区。入口参数为 `int` 类型, 不过只有最低 8 位被发送, 这些数据如果同时发送到接收方的缓冲区中, 则将与普通的数据混在一起。换句话说, 如果通过 `OutputStream` 发送一段文本再发送几个数字, 那么接收方读取时会认为是一串数据, 它不会经过缓冲区而直接发送, 所以有可能代码是写在 `OutputStream` 发送数据之后, 但是接收方会先接收到这部分数据 (除非先进行了 `flush` 操作), 因为对方与本地是通过同一个套接字在通信, 不会因为 API 不同而区分处理。

关闭有些异常情况, 下面列举几种常见的情况。

情况 1: 如果服务方 (这里指提供数据的一方) 已经将对应的 Socket 关闭, 而这里是正常关闭 Socket 操作, 请求方再发起多次 `writeLine()` 操作时, 通常会抛出异常: `java.net.SocketException: Software caused connection abort: socket write error`。这是一类十分常见的错误, 当客户端关闭时, 服务器端尝试读取客户端的一行数据也会报出这样的错误。如果此时发起 `readLine()` 操作, 则通常读取到的是 `null`。

情况 2: 如果服务方的进程直接退出, 请求方使用了 `BufferedReader` 或 `BufferWriter`, 请求方发起 `readLine()`、`writeLine()` 操作时则会出现 `java.net.SocketException` 异常, 异常的消息通常有两种: 请求方发起 `writeLine()` 操作得到的结果是 “Connection reset by peer: socket write error”, 而发起 `readLine()` 操作得到的结果是 “Connection reset”。

情况 3: 当通过 `read()` 方法读取一个字节时, 如果对方已经关闭, 则会返回 -1, 如果通过一些 Java 提供的流包装处理类中类似于 `readByte()` 的方法时 (如: `DataInputStream`), 则会抛出 `java.io.EOFException` 异常。

情况 4: 如果 `BufferedReader/BufferedWriter` 自己本身已经关闭了, 但还在做 `read`、`write` 操作, 就会从内部的一个 `ensureOpen()` 方法中抛出 `java.io.IOException` 异常, 异常的消息为 “Stream closed”。

抛出的异常还有很多, 某些异常是 JVM 级别本身检测抛出的, 所以就与具体的实现类相关, 如果大家看到一种没有见过的异常, 则要学会去看源代码, 这样就会知道异常是在哪里抛出的, 是在什么情况下抛出的。

本来不想大篇幅贴代码，不过写到这里，还是需要给出一段十分简单的 Socket 交互程序让还未接触过 Socket 的同学有点感性认识。在这段代码中，胖哥首先做了一些简单的包装，使得操作相对比较简单一点。

代码清单 2-3 简单的 Socket 交互程序——包装 Socket 传送类

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.net.Socket;

public class SocketWrapper {

    private Socket socket;

    private InputStream inputStream;

    private BufferedReader inputReader;

    private BufferedWriter outputWriter;

    public SocketWrapper(Socket socket) throws IOException {
        this.socket = socket;
        this.inputStream = socket.getInputStream();
        this.inputReader = new BufferedReader(
            new InputStreamReader(inputStream, "GBK"));
        this.outputWriter = new BufferedWriter(
            new OutputStreamWriter(
                socket.getOutputStream(), "GBK"));
    }

    public String readLine() throws IOException {
        return inputReader.readLine();
    }

    public void writeLine(String line) throws IOException {
        outputStream.write(line + '\n');
        outputWriter.flush(); // 由于是 Buffer, 所以需要 flush
    }

    public void close() {
        try {
            this.socket.close();
        } catch (Exception e) {
            /*this body code need you write*/
        }
    }
}
```

包装后，也许代码要稍微好写一点点，至少不用每段代码都去写“流”，这个流在代码中既是输入流，又是输出流，提供一个这样的对象，通过对象 read、write 就可以了。接下来需要写一个 Server 端程序。

代码清单 2-4 简单的 Socket 服务器端构建程序

```
import java.io.IOException;
import java.net.ServerSocket;

public class SocketServer {
```

```

    public static void main(String []args) throws IOException {
        ServerSocket serSocket = new ServerSocket(8888);
        System.out.println("端口已经打开为 8888, 开始准备接收数据");
        SocketWrapper socket = null;
        try {
            socket = new SocketWrapper(serSocket.accept());
            String line = socket.readLine();
            while(!"bye".equals(line)) {
                System.out.println("客户端传来数据: " + line);
                socket.writeLine("我接收到你的数据: " + line);
                line = socket.readLine();
            }
            socket.writeLine("close");
        }finally {
            if(socket != null)
                socket.close();
        }
    }
}

```

这个 SockerServer 建立了一个 8888 端口来对外提供服务, 接下来就写一个程序来调用服务器端的 8888 端口。

代码清单 2-5 客户端程序

```

import java.io.IOException;
import java.net.Socket;
import java.util.Scanner;

public class SocketClient {

    public static void main(String []args) throws IOException {
        Scanner scanner = new Scanner(System.in);
        SocketWrapper socket
            = new SocketWrapper(new Socket("localhost" , 8888));
        try {
            System.out.println("已经连接上服务器端, 现在可以输入数据开始通信了");
            String sendMsg = scanner.nextLine();
            socket.writeLine(sendMsg); //发送消息
            String recivedMsg = socket.readLine();
            while(!"close".equals(recivedMsg)) {
                System.out.println("===【服务器返回】===>" + recivedMsg);
                sendMsg = scanner.nextLine();
                socket.writeLine(sendMsg); //发送消息
                recivedMsg = socket.readLine();
            }
            /*
            胖哥注释掉这部分代码, 如果放开就会报错, 错误原因自然明了
            socket.writeLine("aaaa");
            socket.writeLine("aaaa");
            socket.writeLine("aaaa");
            */
            System.out.println("我是客户端, 结束了!");
        }finally {
            if(socket != null)
                socket.close();
        }
    }
}

```

接下来运行和调用, 最好打开两个窗口分别模拟服务器端和客户端, 而 SocketWrapper 只要放到客户端和服务端都能 import 到的地方就可以了。

这段程序十分简单, 客户端发送消息, 服务器端接收到后反馈给客户端, 当客户端发

送“bye”字符串时，服务器端向客户端发送“close”表示自己即将关闭，客户端若再继续操作，就可能会报错。

►► 2.6.3 Java 与数据库的交互

就目前来讲，大部分 Java 与数据库通信的 API 还是基于 Java 的 Socket，并且都是 BIO 模型，不论是 RDBMS 还是 NOSQL 都是这样。

Java 与数据库通信在 Socket 基础上会建立许多不同的协议，JDBC 是其中一种，以它为例，数据库服务器端此时就类似于 SocketServer 提供 Socket 服务，内在提供配套的线程池来对应连接进行处理。Java 本身通过 JDBC 的驱动程序与数据库端传送相应的指令来得到对应的结果（这个驱动程序内部就完成了通过 Socket 的协议包装，本书的第 8 章中会谈到在 Oracle JDBC 和 MySQL JDBC 源码中如何与服务器端建立通信）。

这个过程看似简单，其实中间会有十分复杂的过程，客户端需要根据服务器端对应协议的要求来传送数据，这中间不再是我们所看到的 SQL 这样简单，所以需要数据库厂商将它封装为驱动。

数据库服务器端的处理方式也很复杂，不同的数据库处理线程与资源分配的关系也是不同的，也有一些数据库是以进程为单位进行处理的，例如 Oracle 就是这样，并且会分配一个 5MB 左右的用户区域，那么自然的 Oracle 建立一个新连接的开销要大许多，但是用起来要顺很多；而 MySQL 并不是这样的。

JDBC 通常有 Java 语言标准接口，由第三方或数据库厂商来提供驱动程序的实现，也就是说，驱动程序就是实现接口的，主要组成的接口有 Driver、Connection、DataSource、Statement、ResultSet 等。

- ◎ Driver 用于注册到 DriverManager 中并提供对外的统一创建 Connection 的接口（此时的 DriverManager 就像静态工厂，其实胖哥认为它只是一个代销商，最多算是一个加工厂）。
- ◎ Connection 就是连接的对象，确切地说是包装了连接操作的对象。
- ◎ DataSource 是对 Connection 操作的相关管理 API，它的实现部分一般是由第三方提供的。
- ◎ Statement 提供通用的语句级别对象处理，其子类接口为 PreparedStatement。
- ◎ ResultSet 就是返回值结果集的一个接口，包含对结果集操作的规范 API 的定义，例如 next()方法、getString()方法等。

在上述提到的这些抽象过后的对象中，操作数据库时最终都是通过与服务器端建立 Socket 来通信的。以 MySQL 为例，连接超时参数在 MySQL 的连接参数中变成了 connectTimeout，每次网络操作发生读操作时使用的响应超时参数都为 socketTimeout。这些内容只需要大家稍微看一下 MySQL JDBC 的驱动程序就可以得到结论。

在 JDBC 中的 Statement 层面还有一个 queryTimeout 参数，也有类似的功能，只是这个参数更加偏上层，它不仅仅针对单次网络 I/O 的读取超时，而且是一个 SQL 的执行超时，很多时候一个 SQL 的执行会发生多次 I/O。对于 JDBC 来讲，协议包装后会有很多参数，远远不止本节所提到的这些，例如可以设置 maxRows、fetchSize、批处理（batch）、预编译机制等，它们都可能对系统造成影响，甚至导致 JVM 出现 OOM。

看到这里，大家可能对 JDBC 的实现也有一些兴趣，在第 8 章的 JDBC 源码中将看到其真正的实现。

2.7 总结

在本章的第 1 节中，胖哥用了很大的篇幅来说明为何要知道计算机的基本原理。在第 2 节开始接触到 CPU，紧接着是内存、磁盘的基本原理，对缓存的相对性概念做了一个简单说明，后来又提到了 Java 网络的基本概念。不知道读者朋友是否在这些内容中找到了一些系统选型的基本理论，而需要更进一步根据场景、基础功底来量化细节。

本章所提到的这些内容，只是让大家有个初步的认识，如果要在技术领域长期发展，则需要知道一些系统基本知识。其实它和我们的工作息息相关，也许你今天用不到，但是你总有一天会用到，这些架构的思想也不仅仅局限于它本身，可以放大到工作的各种场景中。

对于许多东西可以等用到时再去学，但有些东西并不是一两天就能学明白的，它体现了你对计算机本身的理解，而不是在用一种工具去查询一些 API，查询一种报错的方法，它指引的是计算机架构的思想，以及从系统角度分析问题的思路。

当某一天你的上司需要你去做一件从没做过的事情，你的把握有多少呢？我们说一个设计师或一个架构师能坐到这个位置，并不是说他经过了很多年后突然某一天由一个很普通的程序员一下子就变成了“牛人”，这是不可能的，也是不现实的。他同样是在不断的在工作中积累自己的方方面面，很早就开始培养这方面的意识。有些时候，对于做技术研究的人来讲，技术还没出来便可以预知肯定会有相关的技术出现。

IT 技术非常多，我们不可能用到所有的技术，但是需要知道有这种技术，知道它是用来做什么的，它大概会给我们提供什么。当某一天真正需要的时候，如果你的功底足够强，这一切都不是问题。

计算机基础是胖哥所认为的功底之一，修道之人需要知道何为道，修佛之人需要知道何为佛，否则就是假道、假佛。在后面几章的内容中，会正式介绍 Java 的基础功底，或许这些计算机基础知识可以帮助你理解一些问题。

第3章

JVM, Java 程序员的 OS

本章主要内容

- 学习 Java 虚拟机的好处
- 跨平台与字节码基本原理
- 虚拟机的板块
- 常见的虚拟机回收算法
- 浅析 Java 对象的内存结构
- 常见的 OOM 现象
- 常见的 Java 工具

本章中将会出现很多代码片段和插图，这是为了更好地说明问题，以及让大家通过代码做测试来得到一些感性认识。因为篇幅所限不会贴出所有的代码，但会在相应的位置标识出来，这些代码都可以在本书的光盘中找到。

3.1 学习 Java 虚拟机对我们有什么好处

学习 JVM 为了什么？

- ◎ 很多人是为了自己的职业，他们专注于 JVM 的改造，在努力地改变 JVM 本身。
- ◎ 有人是为了兴趣爱好，觉得这样“很牛”。
- ◎ 有人是遇到了很多问题不得不去面对，而去学习 JVM。
- ◎ 有人通过 JVM 的学习发现自己的代码还有许多可改造的地方，于是更加喜欢去学习 JVM。

你为了什么而学习 JVM？也许想想这个问题（不一定要想清楚），再来看 JVM 会更加顺畅一些。

有人说 JVM 很难学？

确实是。如果你说好学，那么胖哥认为你要么是真牛人，要么是学得还不够懂。

JVM 成为现代 Java 程序员（尤其是服务器端 Java 程序员）进阶的基本素养之一，因为它有梯度，所以才能用来衡量人的能力，如果大家都在陈列式的功能列表上学习，那么也就没有高低之分了。JVM 是一块“不好啃的骨头”，但是它也是“越啃越有味道”的骨头，在进入一定阶段后，许多人会发现它除了给自己带来一些愉悦外，也同

时提升了自己解决问题的专业素养。

对于系统程序员学习这一块是比较轻松的，对于计算机基础扎实的人学习也不是难事，它会探讨许多 Java 内在的一些运行机制，让我们感受程序到底会如何来运行。在本章，胖哥会尽量用简单的方式来说明这些内容，而大家需要注重的是细节，以及迭代式地看这些内容，这样你以后再去一些大牛们编写的 JVM 书籍或资料会轻松一些。

JVM 的中文名为“Java”虚拟机，胖哥认为，作为一个比较好的 Java 程序员，Java 虚拟机是需要懂的。这也是一个 Java 程序员进阶的必然过程，因为胖哥认为这算是 Java 程序员真正的“内功”（当然不一定是胖哥写的 JVM 知识，在许多地方都会找到更为专业的 JVM 知识）。

在这里，胖哥就自己的理解，与大家聊聊在什么情况下也许会用到 JVM。

场景 1：某一天发生“Out of memory”问题，团队的“高手”不在，你会如何去处理？是等待高手降临，还是认为自己该做点什么？（要知道，这种情况总是突如其来地出现）也许你会说：“该死，怎么让我遇到这种事情？”

我们可以换个角度来想想，这是上帝赐予的一个机会，一个解决问题的机会，没有问题出现我们也许就没有机会或很少有机会去实践，也许你饱读诗书，却不知何用。此时，如果你懂 Java 虚拟机，正好碰上了问题，你就可以解决问题。

场景 2：当你工作一段时间后，老大觉得你不错，准备提拔你，老大问你一些关于系统服务器架构的问题，比如需要投入多少服务器成本，虚拟机分配多大，如何分配，为何要这样分配，希望你来解答（此时翻资料可来不及哦）。你平时对 Java 虚拟机知识的积累，现在就是派上用场的时候，所谓厚积薄发就是这个道理。

场景 3：某一天你成为了团队的核心编码人员，需要给大家讲解如何写代码更好，还需要告诉大家哪种代码性能更好，难道你从网上去找些别人说好的代码吗？如果听众中坐着一位空降牛人你会害怕说错吗？如果坐着一位真牛人你会怕出丑吗？在讲解的过程中，如果有听众问你一些问题，或者对你的观点产生质疑，你能否有充分的理由来解释，或者最终将思路达成一致？

学习 JVM，会让我们更加深刻地了解 Java 本身的内存设计，会为我们进行编码、解决问题、思考问题、系统量化、技术选择奠定基础。

胖哥只是希望大家能通过本章学习到一些常见的 JVM 原理以及 JVM 本身的方法，再去学习更深的 JVM 知识或许容易一些，需要大家继续努力。

3.2 跨平台与字节码基本原理

3.2.1 javap 命令工具

第 1 章中我们就提到了有些地方需要用 javap 命令工具来看编译后的指令是什么，2.2.1 节中胖哥使用了一个简单的程序让大家感受 javap 命令工具是什么，这里再次谈到 javap 命令工具，这一次我们可以对 javap 命令工具说得稍微清楚一点。为此，胖哥会单独写几段简单的程序给大家说说 javap 命令工具的结果怎么看。

胖哥为什么要给简单的程序呢？为啥不直接来个复杂的程序呢？

javap 命令工具输出的内容是繁杂的，即使是一段小程序，输出后结果也比原始代码要复杂很多。我们要学的其实并不是看指令就能完全反转为 Java 代码，把自己当成一个“反编译工具”，而是通过这种方式来认知比 Java 更低一个抽象层次的逻辑，或许有许多问题直接用 Java 代码不好解释，但是看到虚指令后就一切明了了。

在本节，胖哥分别演示 String 的小代码和几段数字处理小程序（延续第 1 章的数字游戏）。

首先来回顾下代码清单 1-1 的例子，如图 3-1 所示。

当时我们提到这个结果是 true，并且解释了它是在编译时被优化的，现在就用 javap 命令工具来论证这个结论吧。

```
private static void test1() {
    String a = "a" + "b" + 1;
    String b = "ab1";
    System.out.println(a == b);
}
```

图 3-1 代码清单 1-1 的截图

```
D:\java_A>javac-g:vars,lineschapter01/StringTest.java
D:\java_A>javap -verbose chapter01.StringTest
public class chapter01.StringTest extends java.lang.Object
  minor version: 0
  major version: 50
  Constant pool:
const #1 = Method      #6.#21; // java/lang/Object."<init>":()V
const #2 = String      #22;    // ab1
const #3 = Field       #23.#24; // java/lang/System.out:Ljava/io/PrintStream;
const #4 = Method      #25.#26; // java/io/PrintStream.println:(Z)V
const #5 = class       #27;    // chapter01/StringTest
const #6 = class       #28;    // java/lang/Object
const #7 = Asciz       <init>;
const #8 = Asciz       ()V;
const #9 = Asciz       Code;
const #10 = Asciz      LineNumberTable;
const #11 = Asciz      LocalVariableTable;
const #12 = Asciz      this;
const #13 = Asciz      Lchapter01/StringTest;;
const #14 = Asciz      test1;
const #15 = Asciz      a;
const #16 = Asciz      Ljava/lang/String;;
const #17 = Asciz      b;
const #18 = Asciz      StackMapTable;
const #19 = class     #29;    // java/lang/String
const #20 = class     #30;    // java/io/PrintStream
const #21 = NameAndType #7:#8; // "<init>":()V
const #22 = Asciz      ab1;
const #23 = class     #31;    // java/lang/System
const #24 = NameAndType #32:#33; // out:Ljava/io/PrintStream;
const #25 = class     #30;    // java/io/PrintStream
const #26 = NameAndType #34:#35; // println:(Z)V
const #27 = Asciz      chapter01/StringTest;
const #28 = Asciz      java/lang/Object;
const #29 = Asciz      java/lang/String;
const #30 = Asciz      java/io/PrintStream;
const #31 = Asciz      java/lang/System;
const #32 = Asciz      out;
const #33 = Asciz      Ljava/io/PrintStream;;
const #34 = Asciz      println;
const #35 = Asciz      (Z)V;

{
public chapter01.StringTest();
  Code:
    Stack=1, Locals=1, Args_size=1
```

```
0:  aload_0
1:  invokespecial  #1; //Method java/lang/Object."<init>":()V
4:  return
LineNumberTable:
  line 4: 0

LocalVariableTable:
  Start Length Slot Name Signature
  0      5      0  this  Lchapter01/StringTest;

public static void test1();
Code:
  Stack=3, Locals=2, Args_size=0
0:  ldc      #2; //String abl
2:  astore_0
3:  ldc      #2; //String abl
5:  astore_1
6:  getstatic      #3; //Field java/lang/System.out:Ljava/io/PrintStream;
9:  aload_0
10: aload_1
11: if_acmpne      18
14: iconst_1
15: goto 19
18: iconst_0
19: invokevirtual  #4; //Method java/io/PrintStream.println:(Z)V
22: return
LineNumberTable:
  line 7: 0
  line 8: 3
  line 9: 6
  line 10: 22
LocalVariableTable:
  Start Length Slot Name Signature
  3      20      0   a   Ljava/lang/String;
  6      17      1   b   Ljava/lang/String;
StackMapTable: number_of_entries = 2
  frame_type = 255 /* full_frame */
    offset_delta = 18
    locals = [ class java/lang/String, class java/lang/String ]
    stack = [ class java/io/PrintStream ]
  frame_type = 255 /* full_frame */
    offset_delta = 0
    locals = [ class java/lang/String, class java/lang/String ]
    stack = [ class java/io/PrintStream, int ]
}
```

首先我们看比较靠前的一个部分——“常量池”(Constant pool)，每一项都以“const # 数字”开头，这个数字是顺序递增的，通常把它叫作常量池的入口位置，当程序中需要使用到常量池的时候，就会在程序的对应位置记录下入口位置的标识符（在字节码文件中，就像一个列表一样，只是列表中的每一项存放的内容和长度是不一样的而已）。

根据入口位置找某些常量内容，常量内容会分为很多种。在每个常量池项最前面的 1 个字节，用于标识常量池的类型（我们看到的 Method、String 等都是经过映射转换后得到的，字节码中本身只会有 1 个字节来存放）。

找到类型后，接下来就是内容，内容可以直接存放在常量池的入口中，也可能由其他的一个或多个常量池域组合而成。下面胖哥给大家讲几个例子。

例子 1:

```
const #1 = Method      #6.#21; // java/lang/Object."<init>":()V
```


入口位置#1, 简称入口#1, 代表一个方法入口, 方法入口由入口#6 和入口#21 组成, 中间用了一个“.”。

```
const #6 = class      #28;    // java/lang/Object
const #21 = NameAndType #7:#8; // "<init>":()V
```

入口#6 为一个 class, class 是一种引用, 所以它引用了入口#28 的常量池。

入口#21 代表一个表示名称和类型 (NameAndType), 分别由入口#7 和入口#8 组成。

```
const #7 = Asciz      <init>;
const #8 = Asciz      ()V;
const #28 = Asciz     java/lang/Object;
```

入口#7 是一个常量池内容, <init>;代表构造方法。

入口#8 是一个真正的常量, 值为()V, 代表没有入口参数, 返回值为 void, 将入口#7 和入口#8 反推到入口#21, 就代表名称为构造方法的名称, 入口参数个数为 0, 返回值为 void。

入口#28 是一个常量, 它的值是“java/lang/Object;”, 但这只是一个字符串值, 反推到入口#6, 要求这个字符串代表一个类, 那么代表的类是 java.lang.Object。

综合起来就是: java.lang.Object 类的构造方法, 入口参数个数为 0, 返回值为 void, 其实这在 const #1 后面的备注中已经标识出来了 (这在字节码中本身不存在, 只是 javap 工具帮助合并的)。

例子 2:

```
const #2 = String      #22;    // abl
```

代表将会有有一个 String 类型的引用入口, 而引用的是入口#22 的内容。

```
const #22 = Asciz      abl;
```

代表常量池中会存放内容 abl。

综合起来就是: 一个 String 对象的常量, 存放的值是 abl。

例子 3:

```
const #3 = Field       #23.#24; // java/lang/System.out:Ljava/io/PrintStream;
const #4 = Method      #25.#26; // java/io/PrintStream.println:(Z)V
```

入口#3 代表一个属性, 这个属性引用了入口#23 的类、入口#24 的具体属性。

入口#4 代表一个方法, 引用了入口#25 的类、入口#26 的具体方法。

```
const #23 = class      #31;    // java/lang/System
const #24 = NameAndType #32:#33; // out:Ljava/io/PrintStream;
const #25 = class      #30;    // java/io/PrintStream
const #26 = NameAndType #34:#35; // println:(Z)V
```

入口#23 代表一个类 (class), 它也是一个引用, 引用了入口#31 的常量。

入口#24 代表一个名称和类型 (NameAndType), 分别对应入口#32:#33。

入口#25 代表一个类的引用, 具体引用到入口#30。

入口#26 与入口#24 类似, 也是一个返回值+引用类型, 对应入口#34:#35。

```
const #30 = Asciz      java/io/PrintStream;
const #31 = Asciz      java/lang/System;
const #32 = Asciz      out;
const #33 = Asciz      Ljava/io/PrintStream;;
const #34 = Asciz      println;
const #35 = Asciz      (Z)V;
```

入口#30 对应常量池的值为: java/io/PrintStream;, 反推到入口#25, 代表类 java.

lang.PrintStream。

入口#31 对应常量池的值为: java/lang/System;，反推到入口#23，代表类 java.lang.System。

入口#32 对应常量池的值为: out;，反推到入口#24，而入口#24 要求名称和类型，这里返回的显然是名称。

入口#33 对应常量池的值为: Ljava/io/PrintStream;，反推到入口#24，这里得到了类型，也就是 out 的类型是 java.io.PrintStream。

入口#34 对应常量池的值为: println;，反推到入口#26，代表名称为 println。

入口#35 对应常量池的值为: (Z)V;，反推到入口#26，代表入口参数为 Z（代表 boolean 类型），返回值类型是 V（代表 void）。

综合来讲，要执行的操作就是：

入口#3 是获取到 java/lang/System 类的属性 out，out 的类型是 Ljava/io/PrintStream。

入口#4 是调用 java/io/PrintStream 类的 println 方法，方法的返回值类型是 void，入口类型是 boolean。

小伙伴们应该发现这个常量池仅仅是操作的陈列，还没有真正地开始执行任务，那么自然就要看第 2 部分的内容，通过指令将这些内容组合起来。从输出的结果来看，这些指令是按照方法分开的（其实前面应当还有属性列表）。首先看第一个方法。

```
public chapter01.StringTest();
Code:
  Stack=1, Locals=1, Args_size=1
  0:  aload_0
  1:  invokespecial  #1; //Method java/lang/Object."<init>":()V
  4:  return
LineNumberTable:
  line 4: 0

LocalVariableTable:
  Start  Length  Slot  Name  Signature
  0       5      0    this  Lchapter01/StringTest;
```

这是一个构造方法，程序中没有写构造方法，但是 Java 会帮我们生成一个，说明这个动作是在编译时完成的。虽然是构造方法，但是它足够简单，所以我们先从它开始介绍，请看胖哥的解释。

```
Stack=1, Locals=1, Args_size=1
```

这一行是所有方法都会有的，其中 Stack 代表栈顶的单位大小（每个大小为一个 slot，每个 slot 是 4 个字节的宽度），当需要使用一个数据时，它首先会被放入栈顶，使用完后会写回到本地变量或主存中。这里的栈的宽度是 1，其实是代表有一个 this 将会被使用。

Locals 是本地变量的 slot 个数，但是并不代表与 Stack 宽度一致，本地变量在这个方法生命周期内，局部变量最多的时候，需要多大的宽度来存放数据（double、long 会占用两个 slot）。

Args_size 代表的是入口参数的个数，不再是 slot 的个数，即使传入一个 long，也只会记录 1。

```
0:  aload_0
```

第 1 个 0 代表虚指令中的行号（后面会用到，确切地说，应该是方法的 body 部分第几个字节），每个方法从 0 开始顺序递增，但是可以跳跃，跳跃的原因在于一些指令还会接操

作的内容, 这些操作的内容可能来自常量池, 也可以标识是第几个 slot 的本地变量, 因此需要占用一定的空间。

`aload_0` 指令是将“第 1 个”slot 所在的本地变量推到栈顶, 并且这个本地变量是引用类型的, 相关的指令有: `aload_[0-3]` (范围是: `0x2a~0x2d`)。如果超过 4 个, 则会使用“`aload` + 本地变量的 slot 位置”来完成 (此时会多占用 1 个字节来存放), 而 `aload_[0-3]` 是通过具体的几个指令直接完成的。

许多地方会解释为第 1 个引用类型的本地变量, 但胖哥认为这句话有问题, 并不是第 1 个引用变量, 普通变量如果在它之前, 它也不是第 1 个了, 此时本身就是第 1 个本地变量, 更确切地说, 是第一个 slot 所在位置的本地变量。

```
1: invokespecial #1; //Method java/lang/Object."<init>":()V
```

指令中的第 2 个行号, 执行 `invokespecial` 指令, 当发生构造方法调用、父类的构造方法调用、非静态的 `private` 方法调用时会使用该指令, 这里需要从常量池中获取一个方法, 这个地方会占用 2 个字节的宽度, 加上指令本身就是 3 个字节, 因此下一个行号是 4。

```
4: return
```

最后一行是一个 `return`, 我们虽然没有写 `return`, 但是在 JVM 中会自动在编译时加上。

LocalVariableTable:

Start	Length	Slot	Name	Signature
0	5	0	this	Lchapter01/StringTest;

代表本地变量的列表, 本地变量的作用域起始位置为 0, 作用域宽度为 5 (0~4), slot 的起始位置也是 0, 名称为 `this`, 类型为 `chapter01.StringTest`。

看了这个构造方法后, 如果你理解了, 再来看 `test1` 方法或许会轻松一点。下面胖哥对于细节就不再一一讲述了, 只是在指令后面写上注释。

```
public static void test1();
```

Code:

```
Stack=3, Locals=2, Args size=0
```

//Stack=3 代表本地栈的 slot 个数为 3, 两个 String 需要 load, System 的 out 也会占用一个, 当发生 //对比生成 boolean 的时候, 会将两个 String 的引用从栈顶 pop 出来, 所以栈最多是 3 个 slot

//Locals 为 2, 因为只有两个 String

//如果是非静态方法, 则本地变量会自动增加 this

//Args_size 为 0 代表这个方法没有任何入口参数

```
0: ldc #2; //String abl
```

//指令的 body 部分第 0 个字节为 `ldc` 指令, 从常量池入口 #2 中取出内容推到栈顶

//这里的 String 也是引用, 但是它是常量, 所以使用 `ldc` 指令, 而不是 `aload` 指令

```
2: astore 0
```

//将栈顶的引用值写入第 1 个 slot 所在的本地变量中

//它与 `aload` 指令正好相反, 对应 `astore_[0-3]` (范围是 `0x4b~0x4e`)

//更多的本地引用变量写入, 则使用 `astore` + 引用变量的 slot 位置

```
3: ldc #2; //String abl
```

//与第 0 行的操作一致, 引用常量池入口 #2 来获得

```
5: astore 1
```

//类似第 2 行, 将栈顶的值赋值给第 2 个 slot 位置的本地引用变量

```
6: getstatic #3; //Field java/lang/System.out:Ljava/io/PrintStream;
```

//获取静态域, 放入栈顶, 引用常量池入口 #3 来获得

//此时的静态区域是 System 类中的 out 对象

```
9: aload 0
```

//将第 1 个 slot 所在位置的本地引用变量加载到栈顶

```
10: aload 1
```

//将第 2 个 slot 所在位置的本地引用变量加载到栈顶

```
11: if_acmpne 18
```

```
14: iconst 1
```

```

15: goto    19
18: iconst 0
//判定两个栈顶的引用是否一致(引用值也就是地址),对比处理的结束位置是18行
//在 if_acmpne 操作之前会先将两个操作数从栈顶 pop 出来,因此栈顶最多是3位
//如果一致,则将常量值1写入栈顶,对应到 boolean 值 true,并跳转到19行
//如果不一致,则将常量值0写入栈顶,对应到 boolean 值 false
19: invokevirtual #4; //Method java/io/PrintStream.println:(Z)V
//执行 out 对象的 println 方法,方法的入口参数是 boolean 类型,返回值是 void
//从常量池入口#4 获得方法的内容实体
//此时会将栈顶的元素当成入口参数,栈顶的0或1则会转换为 boolean 值 true、false
22: return
LineNumberTable:
line 7: 0
line 8: 3
line 9: 6
line 10: 22
//对应源文件行号,左边是字节码的位置(也可以叫作行号),右边是源文件中的实际文本行号
//javac 编译默认有这个内容,但是如果-g:none 则不会有,调试就会有问题
LocalVariableTable:
Start Length Slot Name Signature
   3     20     0    a    Ljava/lang/String;
   6     17     1    b    Ljava/lang/String;
//本地变量列表,javac 中需要使用-g:vars 才会生成,使用一些工具会自动生成,若没有,则调试
//时在断点中看到的变量是没有名称的
//第一个本地变量的作用区域从第3个字节的位置开始,作用区域范围为20个字节,所在 slot 的位置
//是第0个位置,名称为a,类型为 java.lang.String
//第二个本地变量用类似的方式也可以得到结果

```

在这里,还有一些内容没有细化,例如 StackMapTable 的内容,在研究清楚现有的内容后,可以自己去深入和细化,这部分内容包含的知识非常多,关于指令部分,大家可以参考官方文档来学习:

<http://docs.oracle.com/javase/specs/jvms/se7/html/jvms-6.html>

<http://docs.oracle.com/javase/specs/jvms/se7/html/jvms-7.html>

我们回过头来看为何会输出 true 就很简单了,第一个变量 a,代码中编写的是 "a" + "b" + 1 的操作,但是在常量池中却找不到这3个值,而且指令中也看不到对它们的操作,指令中只看到了对字符串 "ab1" 的操作,因此在编译阶段 JVM 就将它们合并了。

这样貌似是去找一些钻牛角尖的问题,其实不然,这是帮我们从根本上去了解一些细节,或者说是相对抽象层次较低的细节。当然平时你可能用不上,但是当真的有一天遇到一些诡异的问题时,就用得上了。为此,胖哥举个例子。

第1章我们玩了数字游戏,现在我们来看看一个简单的数字操作的指令细节是什么。请先看如下代码。

代码清单 3-1 简单的数字叠加程序

```

public static void test() {
    int a = 1, b = 1, c = 1, d = 1;

    a++;
    ++b;

    c = c++;
    d = ++d;

    System.out.println(a + "\t" + b + "\t" + c + "\t" + d);
}

```

输出结果如下:

```
2    2    1    2
```


此时胖哥估计有的小伙伴惊呆了，为何会有一个 1 呢？

其余的几个结果 2 很好解释，但是 1 这个结果怎么解释呢？

教科书通常告诉我们：`i++`是先做操作再自增，而`++i`是先自增再做操作。好的，我们按照这种思路来理解`c = c++`；这句代码，如果是先做操作，那么这里只有赋值操作，就是 `c` 赋值给 `c`；再自增，显然自增后应该是 2，但是输出结果为 1，解释不通。难道是先自增再赋值？如果是这样，结果也应该是 2 才对。

这到底是怎么回事呢？有的小伙伴可能会说这是钻牛角尖的问题。胖哥也是这么认为的，这样的问题或许结果并不重要，重要的是它可以让我们了解到一个简单的自增操作不止一个步骤来完成，让我们真正拥有一种去探索知识内在的兴趣。

教科书上的说法仅仅是为了方便大家理解而给出的一种通用说法，每一种语言在实现它的时候都有自己的实现方式，我们是 Java 程序员，自然需要知道 Java 程序是怎么处理它的。

看看指令就知道了，就用它来输出指令看看指令里面到底做了什么（篇幅所限，这里不再看常量池，只说指令，而且只说关键部分）。

```
public static void test();
```

Code:

Stack=3, Locals=4, Args size=0

```
0:  iconst_1//将 int 类型的常量值 1 推送到栈顶
1:  istore_0//将栈顶抛出的数据赋值给第 1 个 slot 所在的 int 类型的本地变量中
2:  iconst_1//与第 0 行一致
3:  istore_1//将栈顶抛出的数据赋值给第 2 个 slot 所在的 int 类型的本地变量中
4:  iconst_1//与第 0 行一致
5:  istore_2//将栈顶抛出的数据赋值给第 3 个 slot 所在的 int 类型的本地变量中
6:  iconst_1//与第 0 行一致
7:  istore_3//将栈顶抛出的数据赋值给第 4 个 slot 所在的 int 类型的本地变量中
8:  iinc    0, 1//将第 1 个 slot 所在的 int 类型的本地变量自加 1
11: iinc    1, 1//将第 2 个 slot 所在的 int 类型的本地变量自加 1
14: iload_2//将第 3 个 slot 所在的 int 类型的本地变量放入栈顶
15: iinc    2, 1//将第 3 个 slot 所在的 int 类型的本地变量加 1
18: istore_2//将栈顶抛出的数据写入到第 3 个 slot 所在的 int 类型的本地变量
19: iinc    3, 1//将第 4 个 slot 位置所在的 int 类型的本地变量自增 1
22: iload_3//将第 4 个 slot 位置所在的 int 类型的本地变量加载到栈顶
23: istore_3//将栈顶数据抛出，写入到第 4 个 slot 所在的 int 类型的本地变量中
```

LocalVariableTable:

Start Length Slot Name Signature

2	70	0	a	I//本地变量 a, int 类型, 作用域从第 2 行开始, 作用域范围是 70 行
4	68	1	b	I//本地变量 b, int 类型, 作用域从第 4 行开始, 作用域范围是 68 行
6	66	2	c	I//本地变量 c, int 类型, 作用域从第 6 行开始, 作用域范围是 66 行
8	64	3	d	I//本地变量 d, int 类型, 作用域从第 8 行开始, 作用域范围是 64 行

第 2 章中就提到，slot 其实是 4 个字节的长度，可以理解为字长的意思，任何一个数据至少会分配一个 slot，即使一个 byte 也是这样；long、double 会占用 2 个 slot。

现在来根据指令反向分析输出结果的道理。首先发现第 8 行、第 11 行都做了 `iinc` 操作，都是对本地变量做叠加操作，即分别对前面两个本地变量（a、b）做叠加操作，后续没有其他的动作。换句话说，当一个本地变量发生 `i++` 或 `++i` 操作的时候，如果这句代码发生在单行上面，即不会用于其他计算操作，那么它们最终的指令都是 `iinc`，也就是 `i++` 也会被改为 `++i` 操作。

进一步来看第 3 个本地变量 `c` 的操作。首先通过 `iload_2` 指令将其拷贝到栈顶，然后发生 `iinc` 操作（即自增操作），再通过 `istore_2` 指令将栈顶的数据赋值给这个本地变量，因此，

你可以认为它就像做了一个这样的操作：

```
int tmp = c; c++; c = tmp;
```

只是这个 tmp 并不是真实存在的本地变量，而是栈顶的一份数据拷贝，这份拷贝的数据其实是其他操作，而自己叠加数据并不参与其他计算，这才是 Java 实现 i++ 的真实道理。

对比 d 的操作，可以看到 d 先进行了 iinc 操作，然后再做 iload、istore 两个动作用于赋值，所以 d 是会被叠加的，只是最后两个动作是多余的而已。

我们画个图来看看，这样或许会清楚一点。

在进入方法前，JVM 分配的栈大概是如图 3-2 所示的样子（这个部分不包含指令及指令中指向的常量池位置）。

进入方法后最少分配的操作数栈、本地变量区域，每个小方格代表一个 slot

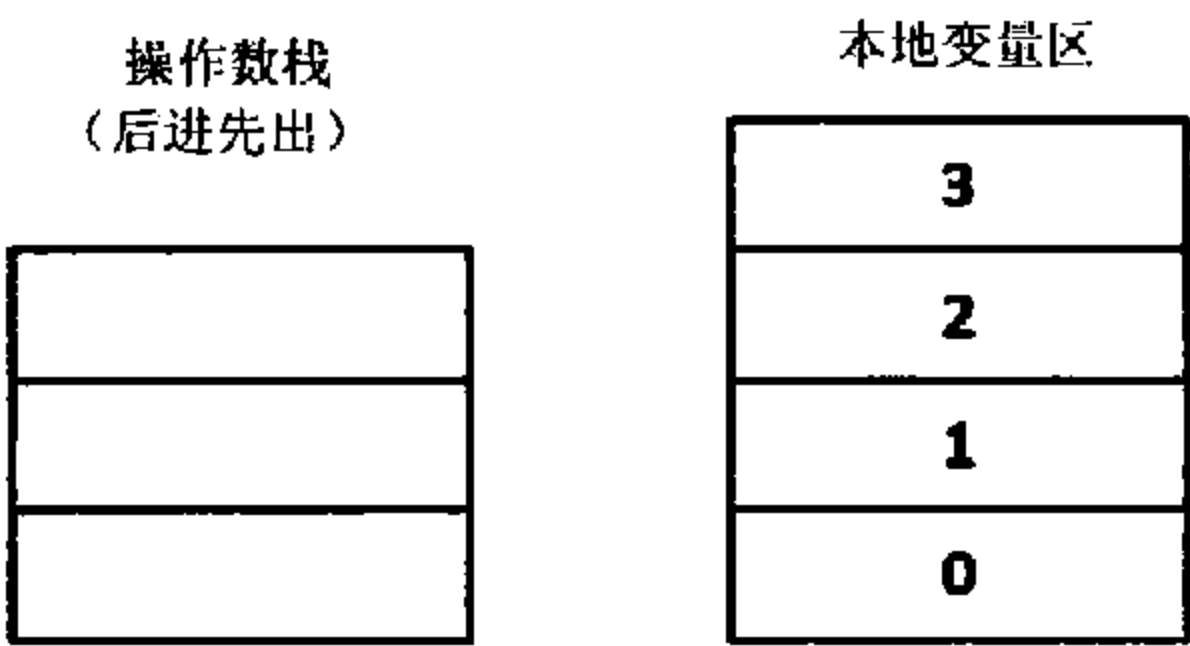


图 3-2 进入一个方法，线程栈空间最少会分配这样的两块区域

一个 Java 方法分派时不仅仅分配这部分空间，至少还需要一个 frame 的数据区。职责包含：负责指向 Class 的常量池，以便于得到指令；会记录一些内容帮助方法返回到正确的来源位置；设置 PC 寄存器得到要执行的指令；记录异常表，控制异常的处理权。

当 iconst_1 发生操作的时候，结构就发生改变（见图 3-3）。

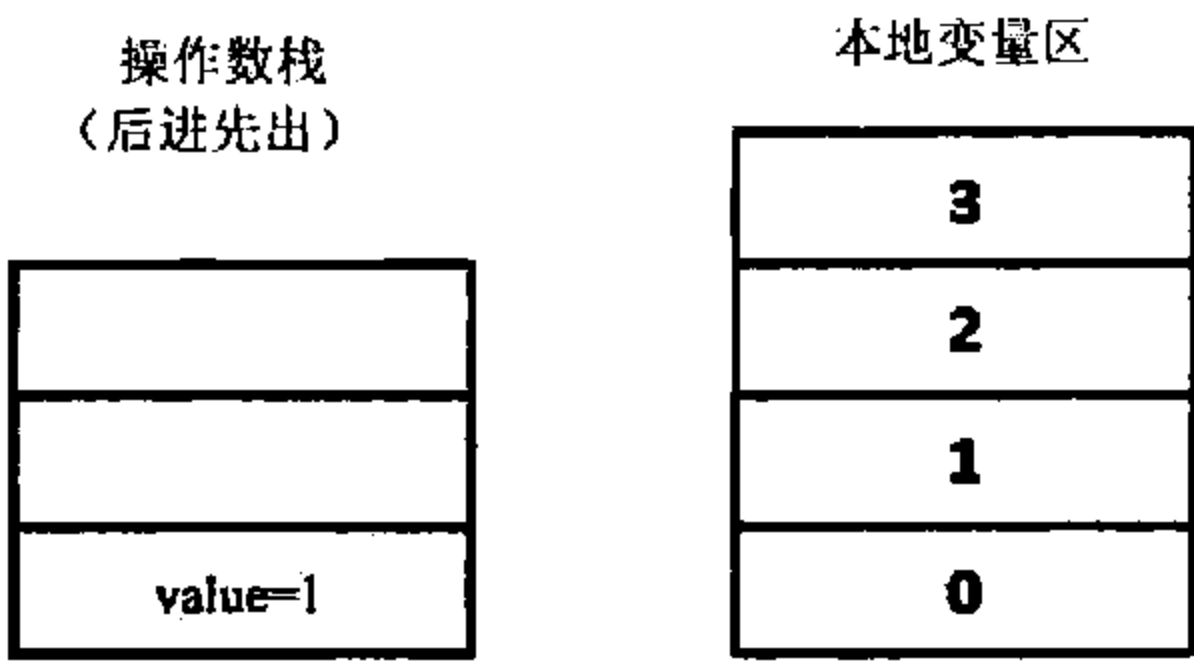


图 3-3 将常量值 1 写入栈顶

当 istore_0 发生操作的时候，将栈顶数据抛出，赋值给变量 a，此时的结构如图 3-4 所示。

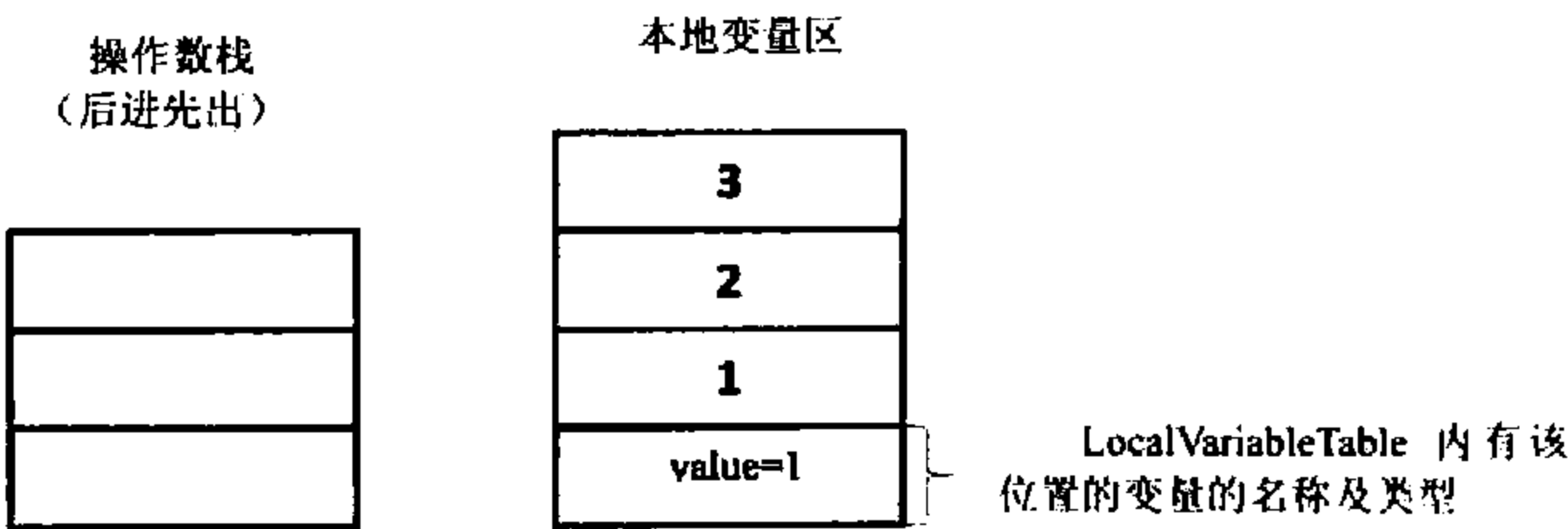


图 3-4 将栈顶数据抛出赋值给第 1 个 slot 所在的本地变量，名称为 a

依此类推，到第 7 行，4 个本地变量都会发生这样的赋值操作，最终结果如图 3-5 所示。

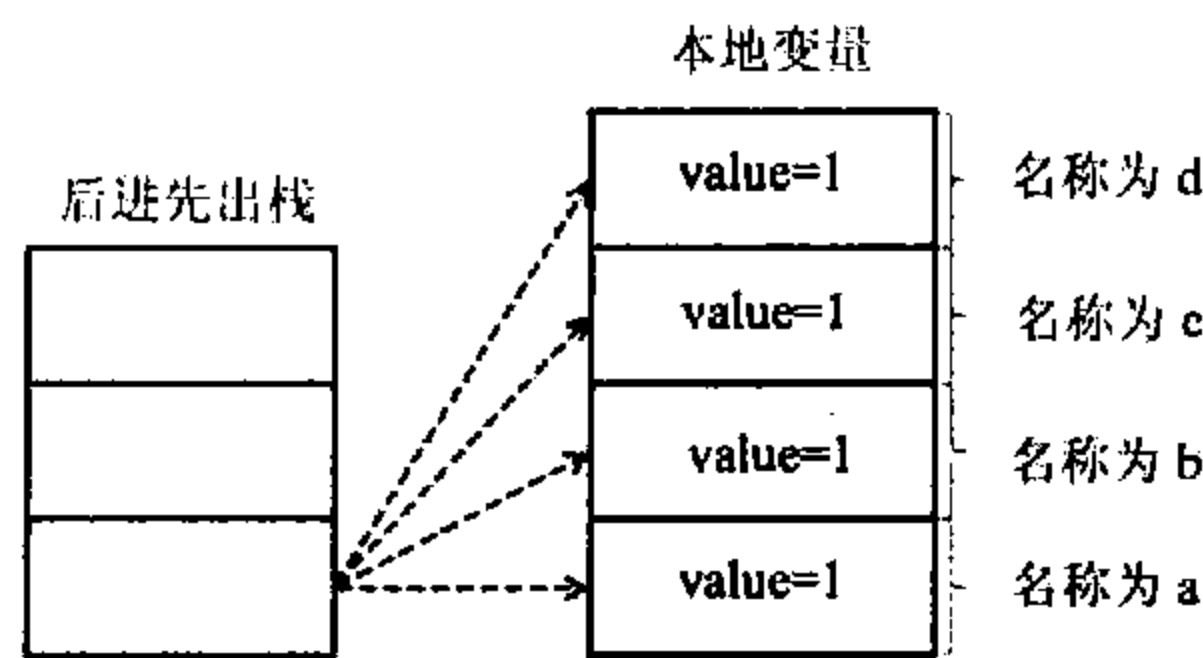


图 3-5 通过栈顶数据赋值后的结果，栈顶只用了一个 slot

iinc 指令我们没有必要详细讲解（实现的细节也可以是利用一个栈顶数据来 store、叠加 1、load），总之 a、b 两个变量都变成了 2。当进一步做 c = c++;操作的时候，首先发生的第 1 个操作是将数据拷贝到栈顶，然后将本地变量改为 2，再从栈顶拷贝回来，如图 3-6 所示。

有的小伙伴会问：后进先出栈明明只用一个 slot，为什么会有 3 个呢？

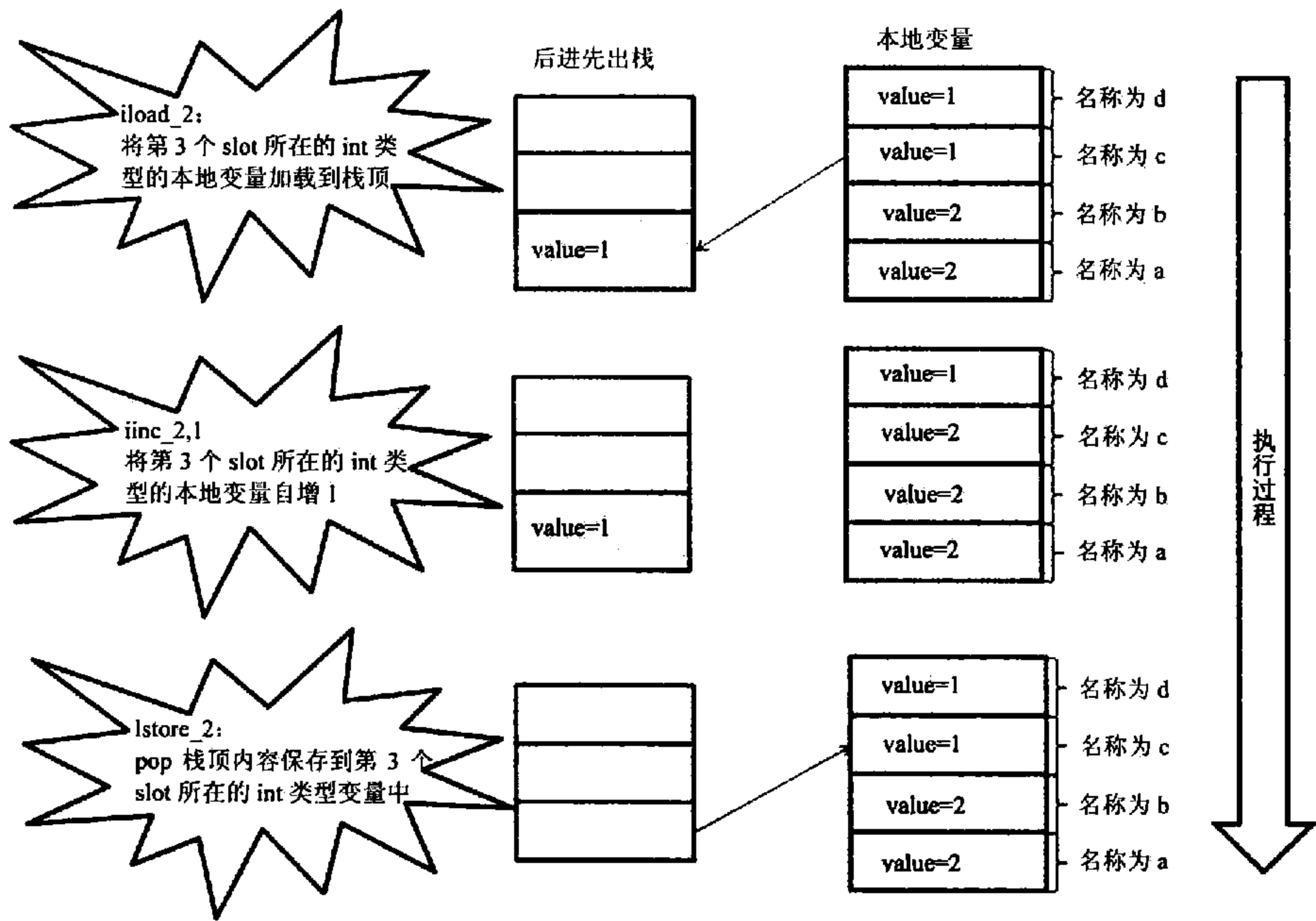


图 3-6 c=c++;操作的程序运行过程

能提出这个问题说明你懂得思考，其实刚开始我们只是输出了一些简单的操作指令，后面还有一条代码 System.out.println(a + "\t" + b + "\t" + c + "\t" + d);的相关指令没有输出。虽然这就一行代码，但是指令可多了（代码短，并不代表指令短，也就是不能代表跑得快），一起来看看接下来的一些指令。

```
24: getstatic    #2; //Field java/lang/System.out:Ljava/io/PrintStream;
27: new          #3; //class java/lang/StringBuilder
30: dup
31: invokespecial #4; //Method java/lang/StringBuilder."<init>":()V
```

```
34: iload_0
35: invokevirtual #5; //Method java/lang/StringBuilder.append:(I)Ljava/lang/StringBuilder;
38: ldc #6; //String \t
40: invokevirtual #7; //Method java/lang/StringBuilder.append:(Ljava/lang/String;)Ljava/lang/StringBuilder;
43: iload_1
44: invokevirtual #5; //Method java/lang/StringBuilder.append:(I)Ljava/lang/StringBuilder;
47: ldc #6; //String \t
49: invokevirtual #7; //Method java/lang/StringBuilder.append:(Ljava/lang/String;)Ljava/lang/StringBuilder;
52: iload_2
53: invokevirtual #5; //Method java/lang/StringBuilder.append:(I)Ljava/lang/StringBuilder;
56: ldc #6; //String \t
58: invokevirtual #7; //Method java/lang/StringBuilder.append:(Ljava/lang/String;)Ljava/lang/StringBuilder;
61: iload_3
62: invokevirtual #5; //Method java/lang/StringBuilder.append:(I)Ljava/lang/StringBuilder;
65: invokevirtual #8; //Method java/lang/StringBuilder.toString:()Ljava/lang/String;
68: invokevirtual #9; //Method java/io/PrintStream.println:(Ljava/lang/String;)V
71: return
```

其实大部分指令都是 `invokevirtual` 指令，关键看它在操作什么。大家看看后面的注释应该就懂了（刚开始看不懂虚指令，可以直接看注释）。现在我们想要知道本地栈要使用 3 个 slot 是怎么来的。

首先，`getstatic` 指令要将 `System` 类的 `out` 静态属性获取出来放入栈顶（因为没有局部变量存放，只能放在栈顶）；然后，通过 `new` 指令创建一个对象（注意，此时仅仅是分配对象的空间，尚未开始初始化对象），这个对象通过常量池入口#3 获得一个 `StringBuilder` 类型（这是证明第 1 章中提到的字符串拼接的结论）。此时栈的样子应当如图 3-7 所示。

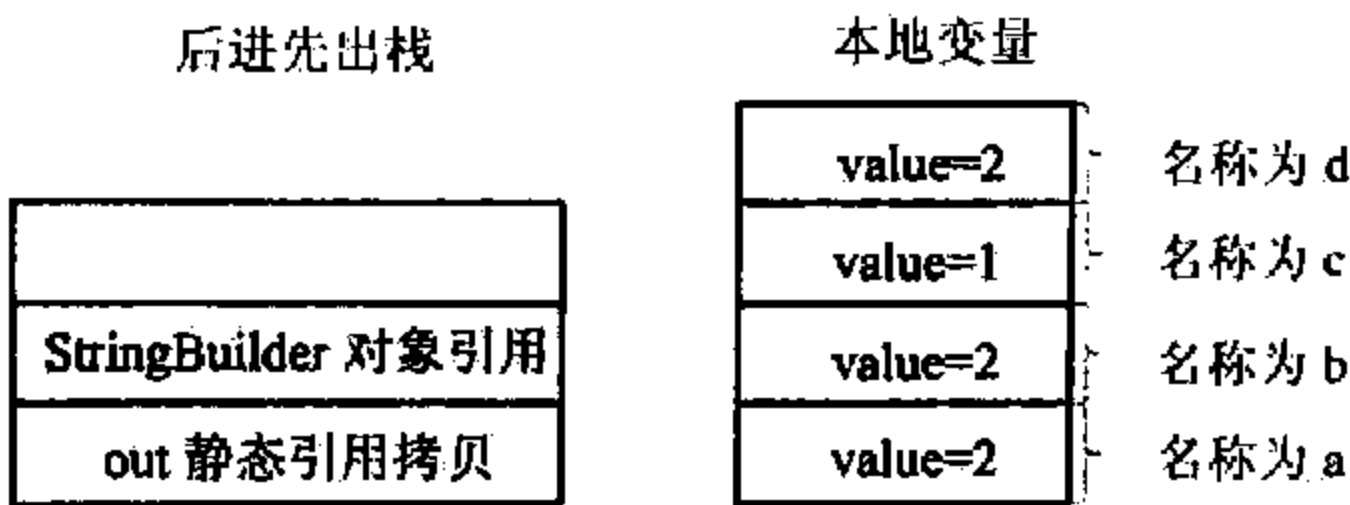


图 3-7 执行前两条指令后，栈空间的情况

此时发生的是 `dup` 命令，它会拷贝一份栈顶的内容，并写入栈顶。为什么要这样做呢？因为后续的 `invokespecial` 操作将会把栈顶数据抛出执行，执行 `StringBuilder` 的构造方法（小伙伴又会问：不是已经创建了吗？为什么还有构造方法？其实刚才的 `new` 仅仅分配了空间，还没有对内容进行初始化，一个简单的创建对象操作其实需要多条指令来完成）。因此，此时的栈就变成了如图 3-8 所示的情况。

至此，小伙伴们应该清楚 `Stack` 为 3 的情况了吧！大家可以将 `System.out.println(a + "\t" + b + "\t" + c + "\t" + d)`；这条代码去掉，或者自己用一个 `StringBuilder` 来拼接看看结果是什么。

接下来的指令动作是将栈顶数据抛出，执行 `StringBuilder` 对象的构造方法对其进行初始化（两个引用其实引用的是同一个对象），此时栈的使用情况回到与图 3-7 所示一样的情

况，区别在于现在的 `StringBuilder` 对象已经执行完构造方法（但是并不代表所有属性都初始化完成，在第 5 章会提到重排序的问题）。

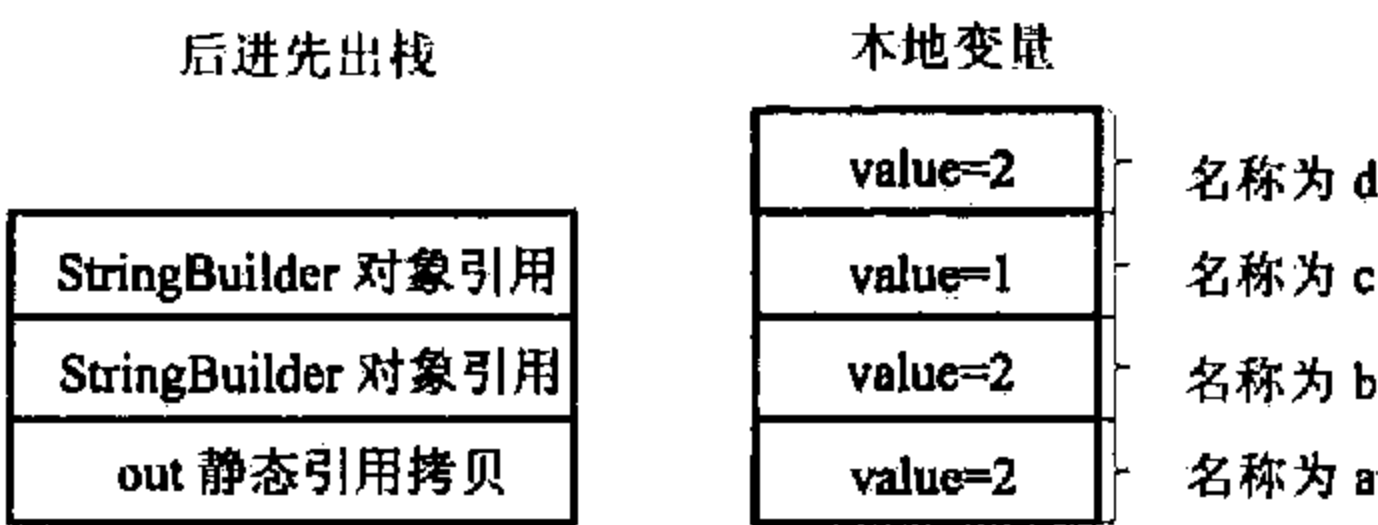


图 3-8 执行 dup 指令后的情况

紧接着，将本地变量、常量（“`\t`”）逐个 `iload` 或 `aload` 到栈顶，然后调用 `invokevirtual` 指令调用 `StringBuilder` 类的 `append` 方法，虽然它也会 `pop` 出来执行操作，但这个方法会有一个 `StringBuilder` 返回值，由于下一个动作是基于这个返回值来操作的，所以这个返回值将会再次被赋值到栈顶。因此，它执行前无须再拷贝了。如果这个 `StringBuilder` 是一个自定义的本地变量，那么也无须再一次执行 `iload` 操作了。

- ◎ 大家可以在代码清单 3-1 上做几个小改动，进一步分析；
- ◎ 将拼接过程换成一个 `StringBuilder`，看看 `Stack` 的数量有没有变化。
- ◎ 换成一个 `StringBuilder` 在一行代码中 `append` 多个变量，与分成多行分别 `append`，在指令上是否有区别（这里 `append` 的内容有 7 个，你完全可以拆分 2、3 个出来看看）。
- ◎ 添加一个自定义对象，自定义对象中有一个 `void` 返回值的方法，也像 `append` 那样反复调用，看看它是不是需要每次 `iload`，而 `StringBuilder` 不需要。

大家可以继续扩展，例如 `(i++) + (++i) + (i++)` 等，或许看看指令就清楚内在的执行顺序了。关于 JVM 的指令有 200 多个，我们要一一看完不容易，可以先看自己想看的指令，或者自己写几个简单程序看看指令。

这些指令还是 `javap` 命令工具告诉我们的，`javap` 命令本身也将字节码翻译成了文字，与反编译工具相比，它更加接近于字节码的结构（反编译工具就是基于这种指令反向计算出程序代码的），但是它还不是真正的字节码。下一节将对字节码本身进行介绍，然后介绍 `javap` 命令工具是如何解析这个字节码得到内容的。

3.2.2 Java 字节码结构

小伙伴们都知道，Java 是一种跨平台的语言，它为何跨平台呢？
哦，老师告诉我，是因为 Java 的 Class 文件是统一的，是基于字节码的。
嗯，没错，不过字节码是什么呢？
咦，老师说字节码就是以 `byte` 为单位存储的文件。

嗯，这也没错，但是和跨平台结合起来考虑，它就是描述程序要运行的虚指令的集合，而这个虚指令的集合与任何平台无关，Java 虚拟机认识它，Java 虚拟机将它翻译为对应的 OS 指令就好了！

上一节，我们看到了虚指令，其实是 `javap` 命令帮我们翻译过来的显示文字，在 Java 程序编译为 Class 文件后，它内部的结构是什么样子的？这一节，就请跟着胖哥一起来窥探一些小奥秘吧。

首先，编译一个 Java 源文件，一般我们是通过 `javac` 命令来将它编译为 Class 文件的，

javac 命令本身是一个“引导器”，它引导编译器程序的运行。编译器本身也是一个 Java 程序，当运行 javac 命令时所引导的 Java 类是“com.sun.tools.javac.main.JavaCompiler”，这个类会完成 Java 源文件的解析（Parser）、注解处理（Annotation process）、属性标注、检查、泛型处理、一些语法糖转换，最终生成 Class 文件。

如果大家想要动态编译一些 Java 源码，则可以使用 ToolProvider.getSystemJava Compiler()来得到一个编译器，得到的是 javax.tools.Compiler 类的一个对象，通过它可以创建 CompilationTask 任务来进行编译，编译任务运行后就可以得到编译后的 Java 字节码（换句话说，是一个 byte[]数组），接下来就交给 ClassLoader 了（本节后文中还会有更详细的介绍）。

首先来认识一下 Java 字节码文件的主体结构。胖哥根据自己的理解，先给出一个字节码文件（Class 文件）的主体结构（见图 3-9），然后再逐步细化一些板块。

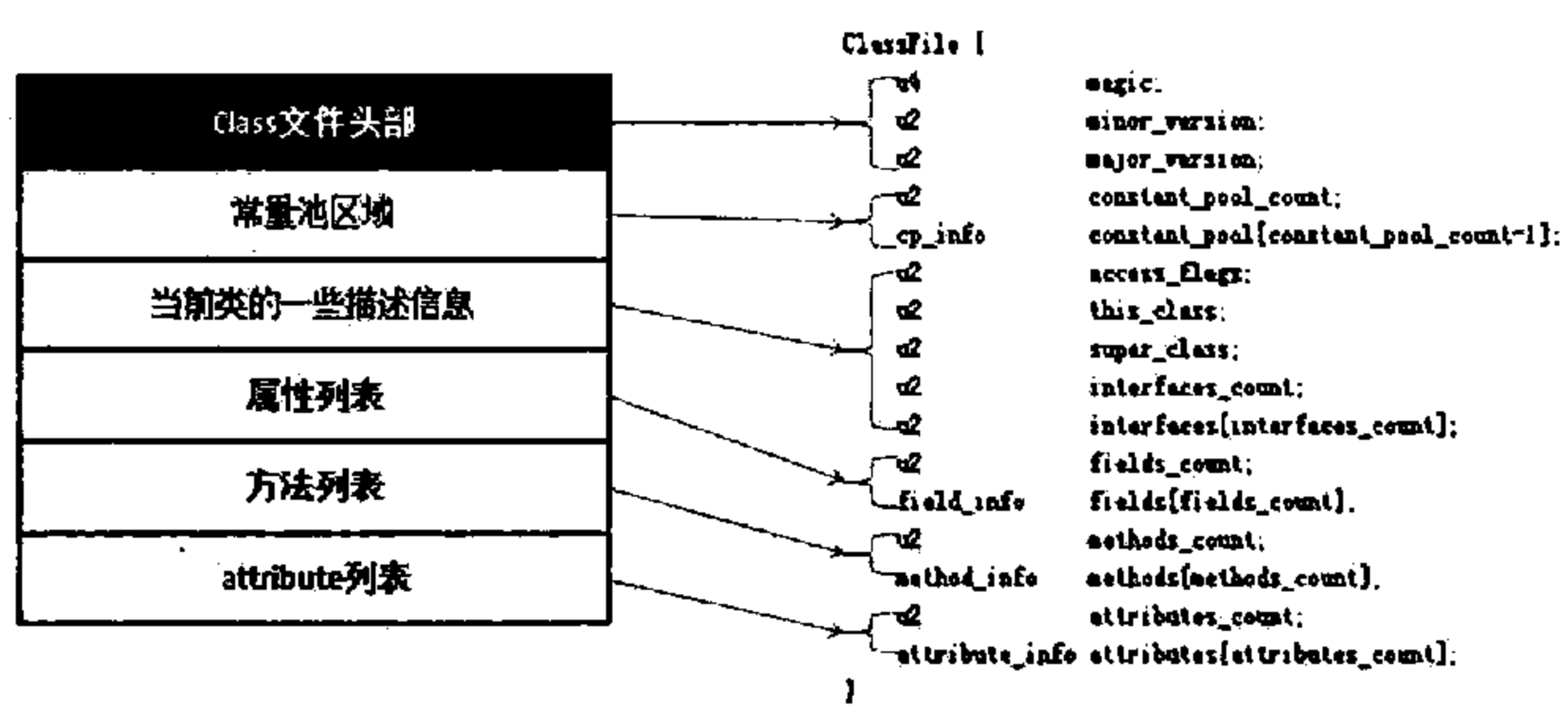


图 3-9 Java 字节码文件的主体结构

请参看官方文档：<http://docs.oracle.com/javase/specs/jvms/se7/html/jvms-4.html#jvms-4.1>。下面进行解释。

首先，文件头部包含 4 个字节的头部验证码，这 4 个字节的十六进制表示分别是：0xCA、0xFE、0xBA、0xBE。这是固定的，如果文件头部不是这 4 个字节，通常 JVM 不认。

接下来是 Class 的 minor version 和 major version，分别占用 2 个字节大小的空间，这两项信息代表 Class 是用什么版本的编译器编译的，当 JRE（Java运行时环境）的版本低于这个版本时，它就会报错：

```
Unsupported major.minor version 50
```

这个错误内容后面有一个数字：50，它就是 major version 和 minorversion 的组合，如果 minor version 是 0，那么有些时候只显示 major version。无论如何，字节码中始终会存储这两项信息。

平时看到的 JDK 1.6、JDK 1.7 等，它们的版本对应关系如表 3-1 所示。

表 3-1 版本对应关系

运行时环境版本	major.minor	描 述
1.1	45.3	比较原始的版本
1.2	46.0	提供 strictfp 标识符支持

续表

运行时环境版本	major.minor	描 述
1.3	47.0	
1.4	48.0	
5 (1.5)	49.0	新的 attribubtes 支持许多强大的语法规则
6 (1.6)	50.0	StackMaps 支持
7 (1.7)	51.0	Invokedynamic 支持

归结起来，Class 文件头部如图 3-10 所示。

回到图 3-9 中，看第 2 个部分——常量池。

在前文中反复提到过常量池，javap 命令也输出了内容，下面我们一起来看看字节码中是如何存储这些内容的。

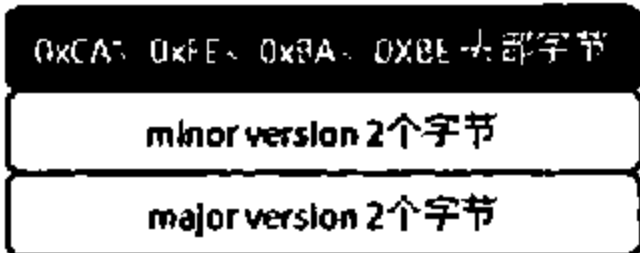


图 3-10 Class 文件头部

这部分的开始有 2 个字节来表示常量池入口位置的最大下标（其实不是长度，虽然官方给出的是长度），它的下标是从 1 开始的（从 javap 命令输出的结果中可以看到，都是从入口#1 开始的）。

现在来看看常量池中的每一项。每一项会有 1 个字节来标志它的类型，然后是常量池的内容。常量池中每一项的结构及类型列表如图 3-11 所示。

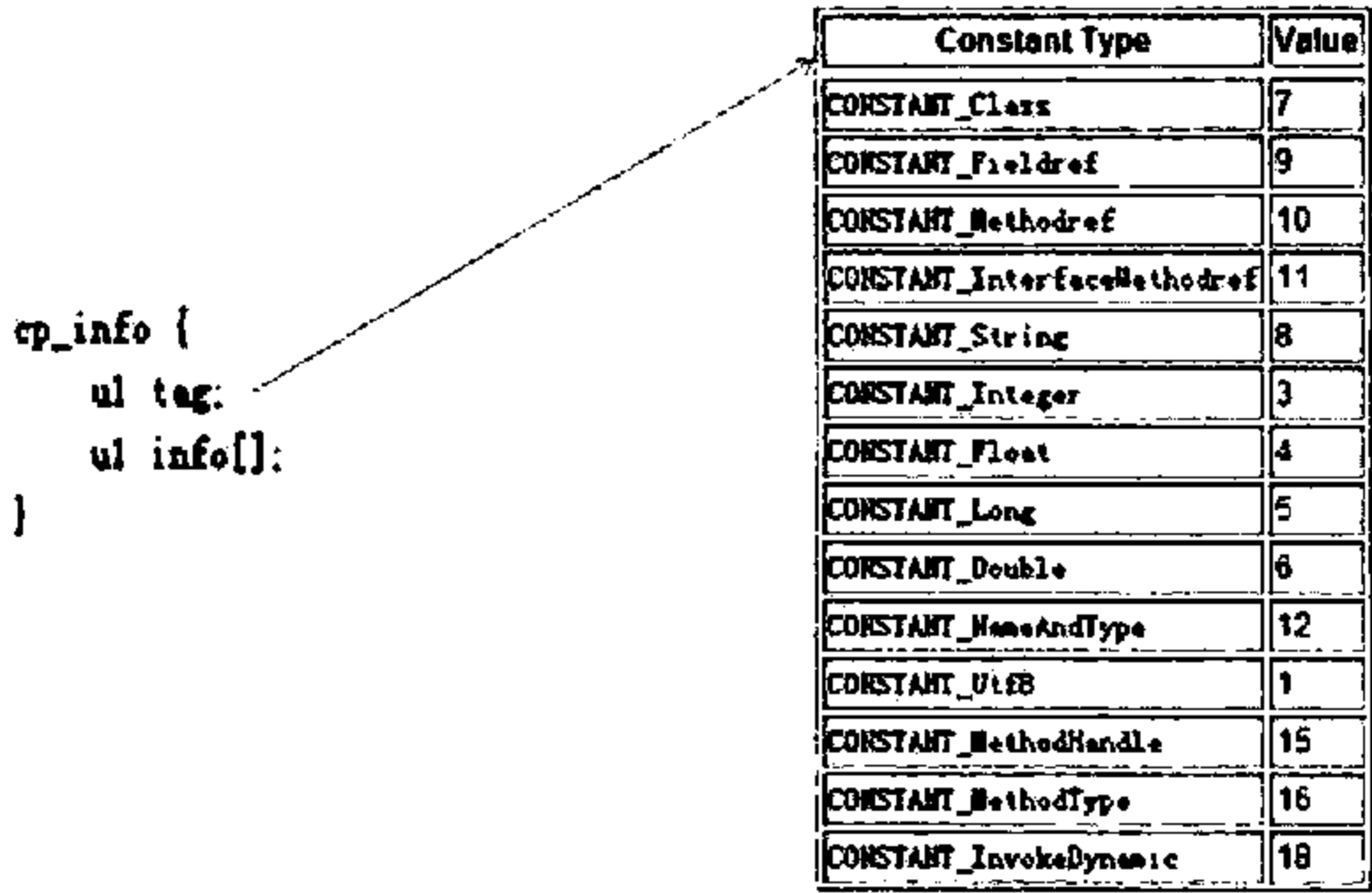


图 3-11 常量池中每一项的结构及类型列表

请参看官方文档：<http://docs.oracle.com/javase/specs/jvms/se7/html/jvms-4.html#jvms-4.4>。

每一种类型将会对应着不同宽度的内容，例如 Class 紧接着的是 2 个字节的内容，这 2 个字节代表 Class 的名称在常量池中的位置（通过前一节的介绍大家应该知道，这个 Class 会记录另一个常量池位置，另一个常量池位置才会真正记录 Class 的名称，而这里该项仅仅代表返回的这个名称应当转换为一个 Class）；Fieldref、Methodref、InterfaceMethodref 会用 2 个字节代表 Class 在常量池中的位置，再用 2 个字节代表 NameAndType 信息，对于属性自然是属性的名称和类型，对于方法自然是方法的名称，以及方法的入口参数和返回值类型。

综合起来，常量池的结构如图 3-12 所示。

继续来看类的基本描述信息，它会包含什么内容呢？

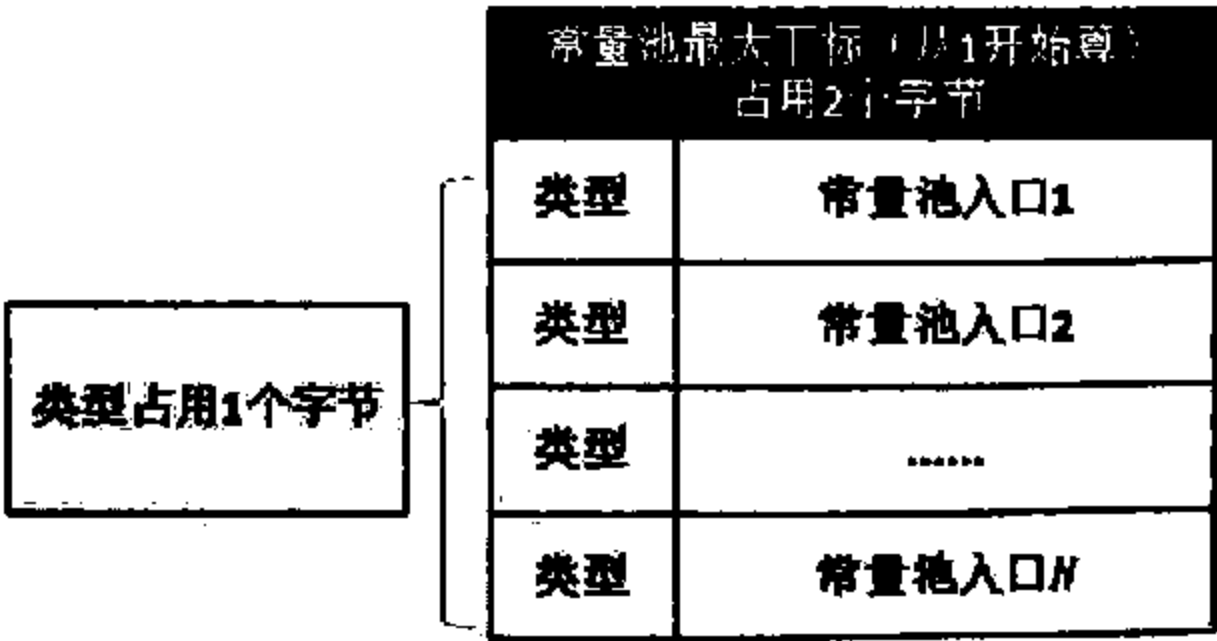


图 3-12 常量池的结构

首先，它会用 2 个字节来标识类的 Class 修饰符，这个修饰符代表的是这个类的访问方式（access_flags），如图 3-13 所示。

Flag Name	Value	Interpretation
ACC_PUBLIC	0x0001	Declared public; may be accessed from outside its package.
ACC_FINAL	0x0010	Declared final; no subclasses allowed.
ACC_SUPER	0x0020	Treat superclass methods specially when invoked by the <i>invokespecial</i> instruction.
ACC_INTERFACE	0x0200	Is an interface, not a class.
ACC_ABSTRACT	0x0400	Declared abstract; must not be instantiated.
ACC_SYNTHETIC	0x1000	Declared synthetic; not present in the source code.
ACC_ANNOTATION	0x2000	Declared as an annotation type.
ACC_ENUM	0x4000	Declared as an enum type.

图 3-13 access_flags 列表

看起来觉得麻烦，仔细看看其实不然，它就是用 2 个字节表达了类的许多属性。例如，低位的第 1 个二进制位代表是否为 public，是否为 final 的二进制数最后 3 位是 110（十六进制形式的 0x0010），依此类推。

小伙伴们有没有体会到设计的技巧，一堆修饰符，需要占用较大空间的内容，这里只需要 2 个字节就搞定了！这种大量“是”、“否”的设计，就可以利用 1、0 来标识，正好代表一个二进制位，可以组合，也可以单独拆分。

紧接着会有 2 个字节来标识类名在常量池中的位置。然后再用 2 个字节来标识父类名称在常量池中的位置（如果该类没有写父类，那么就是 Object 类了，所以这 2 个字节始终存在）。

如果自己想要写程序来解析修饰符就用“java.lang.reflect.Modifier”，它提供了相关的方法来判定，这样写代码更加直观。例如，要判定修饰符是否为 public，就调用 Modifier.isPublic(int)，它内部是如何实现的呢？其实你应该猜到了，看看源码截图（见图 3-14），验证下你的猜测。

```
/**
 * The <code>int</code> value representing the <code>public</code>
 * modifier.
 */
public static final int PUBLIC          = 0x00000001;

public static boolean isPublic(int mod) {
    return (mod & PUBLIC) != 0;
}
```

图 3-14 如何判定修饰符是否为 public

如果在运行的程序中，想要判定一个 Class 或某个属性、方法的 public 情况，就需要获取到这个标识符。程序中是有方法的：

```
int XXX.class.getModifiers()
int XXX.class.getDeclaredField(...).getModifiers()
int XXX.class.getDeclaredMethod(...).getModifiers()
```

现在来看看接口。在 Java 语言中，任意一个类最多只能有一个父类，但是可以有多个接口，也可以没有接口，所以要表达接口，需要先表达数量，再逐个表达每一个接口。因此解析接口时也肯定会先读取接口的数量，再根据数量循环读取每一个接口的信息。

Java 字节码中也是用 2 个字节来表达接口数量的，然后循环获取接口，在每一个接口的描述中，标识了它们在常量池中的入口位置，通过这个入口便可以找到所有的接口列表了。

综合起来，类的基本描述信息结构如图 3-15 所示。

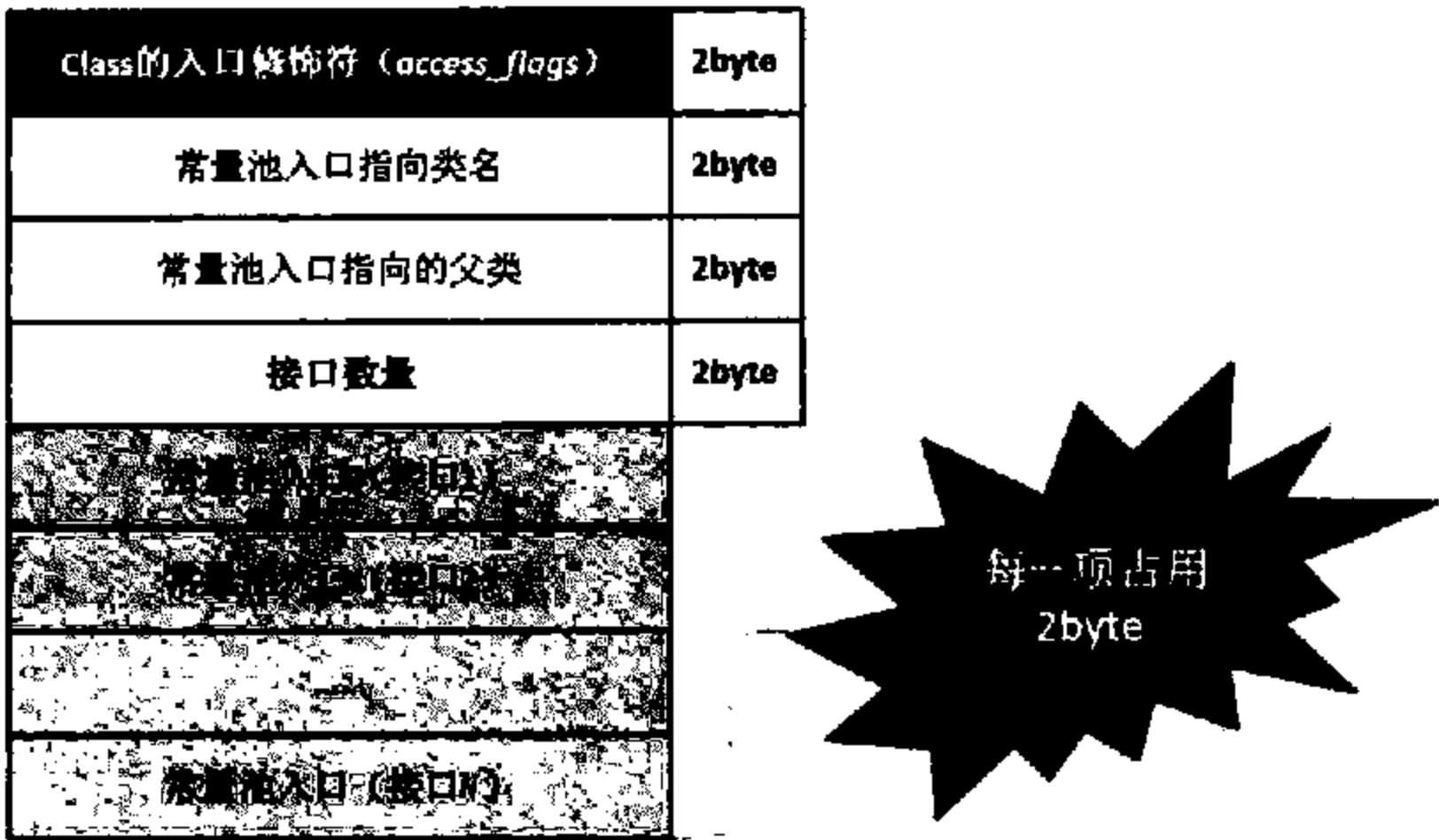


图 3-15 类的基本描述信息结构

接下来我们该说属性（field）了。类似地，用 2 个字节来代表长度是多少，也就是代表有多少个 field。

根据这个长度，就可以开始循环了，对于属性的内部结构，官方的描述如图 3-16 所示。

```
field_info {
    u2          access_flags;
    u2          name_index;
    u2          descriptor_index;
    u2          attributes_count;
    attribute_info attributes[attributes_count];
}
```

图 3-16 属性的内部结构

它与 Class 一样，用 2 个字节来代表 field 的“access_flags”，与之不同的是 field 还会增加一些 transient、volatile、static 的判定，如图 3-17 所示。

Flag Name	Value	Interpretation
ACC_PUBLIC	0x0001	Declared public; may be accessed from outside its package.
ACC_PRIVATE	0x0002	Declared private; usable only within the defining class.
ACC_PROTECTED	0x0004	Declared protected; may be accessed within subclasses.
ACC_STATIC	0x0008	Declared static.
ACC_FINAL	0x0010	Declared final; never directly assigned to after object construction (JLS §17.5).
ACC_VOLATILE	0x0040	Declared volatile; cannot be cached.
ACC_TRANSIENT	0x0080	Declared transient; not written or read by a persistent object manager.
ACC_SYNTHETIC	0x1000	Declared synthetic; not present in the source code.
ACC_ENUM	0x4000	Declared as an element of an enum.

图 3-17 属性的 access_flags 列表

紧接着的 2 个字节用于获取常量池的入口，对应到“属性名称”（name_index）；然后就是属性类型在常量池中的入口位置。

另外，还有一些附加属性（attribute 列表），它通常标识 Signature、Annotation、Deprecated 等信息，这部分内容细说就会非常多，大家可以自己去进一步细化。

属性列表貌似不难，大概是如图 3-18 所示的一种结构。

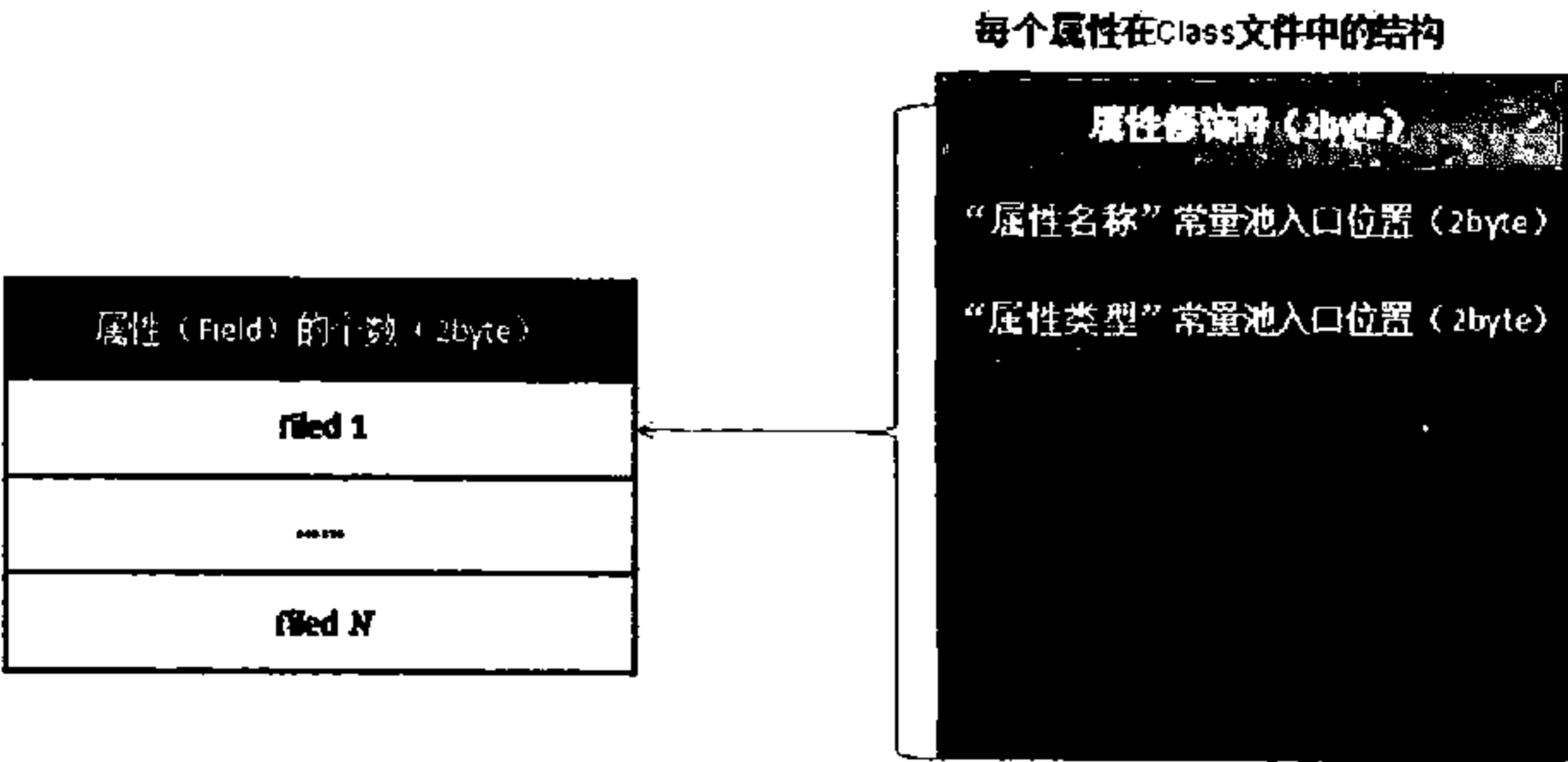


图 3-18 属性的基本结构定义

读取完属性后，接下来就是 method 了，这部分内容相对比较复杂，主要是结构一层套一层容易将人“套晕”。

同样的，需要 2 个字节来表示 method 的数量，依照这个数量循环取 method 的列表信息。对于每个 method 都有 access_flags、方法名，不过方法名具有一些特殊性，就是它们有可能是“构造方法”或“static 匿名块”（都被归结到 method 列表中）。这里我们不去细说它们，只需要知道它们都有共同的特征就是方法名即可。对于每一个方法，官方给出的内部结构如图 3-19 所示。

```
method_info {
    u2          access_flags;
    u2          name_index;
    u2          descriptor_index;
    u2          attributes_count;
    attribute_info attributes[attributes_count];
}
```

图 3-19 方法的内部结构

下面开始细说方法。

方法肯定有入口参数和返回值，这里方法使用了 2 个字节来保存常量池区域的入口位置 (descriptor_index)，为了节约空间，常量池本身会将平时 Java 代码中的类型进行转换来存放。

例如“(II)V”的一个常量池表，就代表入口参数为 2 个 int，而返回值为 void，对于 Java 常量的基本类型，它在字节码中的规范定义如表 3-2 所示。

表 3-2 Java 常量的基本类型在字节码中的规范定义

类型字符	对应类型	说 明
B	byte	代表字节，但不是 Byte 的意思
Z	boolean	代表布尔值，同样不是 Boolean 值
S	short	
C	char	
I	int	
J	long	
F	float	

续表

类型字符	对应类型	说 明
D	Double	
V	void	没有返回值
LClassName;	Reference	引用类，例如 String 被标识为：Ljava/lang/String;
[	Array Reference	数组的引用，例如 double[][] 被标识为：[[D;String[] 被标识为：[Ljava.lang.String;



注意 如果是 Integer，则不会像 int 那样被标识为 I，它会以对象的形式存在，对象类型在这里都会以一个字符 “L” 开头，以一个英文分号 “;” 结尾，因此 Integer 会被标识为：Ljava/lang/Integer;。由于每一种类型都有自己的特征，所以多种类型连续组合在一起也可以被识别出来，因此类型就这样串联起来了。

接下来解析方法的内部信息。

方法的 body 部分是用 attribute 来标识的，首先用 2 个字节代表 attribute 的个数，然后遍历每一个 attribute 的内容。每一个 attribute 的内部结构如图 3-20 所示。

图中，u2 代表用 2 个字节标识名称所在的常量池位置。
这里的名称是用来标识方法里面不同的 body 类型的。
方法里面还有不同的 body 类型吗？

```
attribute_info {
    u2 attribute_name_index;
    u4 attribute_length;
    u1 info[attribute_length];
}
```

图 3-20 attribute 的内部结构

是的，有指令 Code 列表、Exceptions 标识抛出的异常、StackMapTable 操作数栈的位置、LineNumberTable 记录行号对应表、LocalVariableTable 记录本地变量列表。属性、类、方法都有 attribute，具体细节大家可以参考官方文档：<http://docs.oracle.com/javase/specs/jvms/se7/html/jvms-4.html#jvms-4.7>。

如果 attribute_name 获取到的值是 “Code”，则代表是代码的指令列表，那么它的内部结构如图 3-21 所示。

```
Code_attribute {
    u2 attribute_name_index;
    u4 attribute_length;
    u2 max_stack;
    u2 max_locals;
    u4 code_length;
    u1 code[code_length];
    u2 exception_table_length;
    {
        u2 start_pc;
        u2 end_pc;
        u2 handler_pc;
        u2 catch_type;
    } exception_table[exception_table_length];
    u2 attributes_count;
    attribute_info attributes[attributes_count];
}
```

图 3-21 一个方法的 Code attribute 的内部结构

这里的 `max_stack`、`max_locals` 就是 `javap` 指令中的 `Stacks`、`Locals` 两个值的内容，分别用了 2 个字节来表达。

紧接着的是用 4 个字节代表 `Code` 的长度，然后用一个 `byte` 数组来表达 `Code` 的列表（数组的长度正好是前面用 4 个字节表达的长度）。如果要解析这个 `Code`，需要先逐个字节进行遍历（这里遍历到的第几个字节正好对应到 `javap` 指令中的行号，所以前面胖哥说其实这个是 `Code` 的字节顺序）。

在循环遍历过程中，首先读取 1 个字节，这个字节代表 1 个指令，对应到指令列表中（可参考文档：<http://docs.oracle.com/javase/specs/jvms/se7/html/jvms-7.html>）。

得到指令后，就要根据指令是否需要接操作数来决定读取多少个字节，例如 `invokespecial` 指令就需要 2 个字节来标识常量池中的入口位置，`iconst_0` 无须读取任何操作数，细节请参考文档：<http://docs.oracle.com/javase/specs/jvms/se7/html/jvms-6.html>。

`Code` 部分到这里并没有结束，它还需要读取 `Exception` 列表（`try`、`catch`）。首先会读取 2 个字节代表 `Exception` 的个数，然后循环提取 `Exception`，每个 `Exception` 的表达由 `from`、`to`、`target`、`typeDescribe` 几项信息来表达，分别代表异常的 `try` 开始位置、`try` 结束位置、异常跳转的目标、异常的类型（在常量池中）。

接下来还有 `attribute` 列表，为什么有 `attribute` 列表呢？这是字节码不爽的地方，如果以 `Code` 开头，那么 `LineNumberTable`、`StackMapTable`、`StackMapTable` 以及包含的异常信息也会包含进来。

这并不代表的 `body` 部分只有这样一种结构，有的编译器是先编译生成 `Exception` 信息（向外抛出的异常信息），然后再声明其他的，所以如果自己要写代码解析 `Class` 文件，这里就不能将代码完全写死。

在方法结束后，通常还会有 `Class` 的 `attribute` 信息，例如 `SourceFile`、`InnerClasses` 等信息可以存放在这里。

方法部分的解析是复杂的，毕竟它要解析这么多的代码逻辑，但是大家应该清晰地认识到编译的结果是一种逻辑结构，而且是方便被计算机认识的逻辑结构。这种逻辑结构对于 `Java` 语言来讲是通用的，由于它不是真正的运行指令，所以叫它虚指令。

`Java` 虚拟机也会为每个 `OS` 平台编写对应的 `JRE` 运行时环境，与 `OS` 动态链接，将这些虚指令编码翻译为对应操作系统的汇编指令信息，即可在对应的 `OS` 上调用执行了。

疑惑：为何要有常量池？将内容直接放在字节码中不就行了吗？

答曰：既然叫常量池，那么就是一个信息共享池，其实在真正的程序中，会反反复复出现很多重复的内容拷贝，将它们用常量池保存起来，在描述程序的过程中用很短的一个入口地址就可以表示了，而不需要使用很长的字符串来表示。这样做，一是可以节约空间；二是 `Class` 文件的字节码结构将变得更加规整。

疑惑：看了字节码结构后，想想它除了跨平台，对我还有什么用途？

答曰：从两个角度来看，其一，让我们了解到什么是真正的字节码，`Java` 程序编译为 `Class` 字节码文件后到底是什么样子的。了解了字节码的结构，这对于我们去了解 `Class` 的内存结构、字节码增强技术、反编译技术都是一个很大的帮助；其二，如果想要设计一种语言或一些类似于解析类的程序，这种字节码结构就是一种可以借鉴的方法。

►► 3.2.3 Class 字节码的加载

上一节, 我们一起了解了 Class 文件的内部结构是什么样的, 也大概清楚了程序如何识别 Java 字节码中的内容, 那么你是不是想知道 Class 的一些加载过程?

提到 Class 的加载, 就不得不提到 ClassLoader, 因为 Java 加载类就是靠它加载的, 它负责读取字节码的字节流进行加载。

ClassLoader 的继承关系是从 BootstrapClassLoader 开始的, 也是由它最先加载类, 然后是 ExtClassLoader, 接下来是 AppClassLoader (应用程序默认的), 最后是用户自己的 ClassLoader (通常在容器下编写代码, 都是由容器自定义创建的 ClassLoader 创建的类)。下面简单介绍一下它们的用途。

(1) BootstrapClassLoader 主要用于加载一些 Java 自带的核心类 (例如 java.lang.*), 通常这些核心类的 Class 被签名, 不能被替换掉, 它是由 JVM 内核实现的, 在 Hotspot VM 中是用 C++ 实现的, 有了它加载最核心的内容, 才会有后面的 ClassLoader 的存在。

(2) ExtClassLoader 是加载在 jre/lib/ext/目录下的 jar 包, 用户也可以自己将 jar 包放到这个目录下, 通过这个 ClassLoader 来加载。

(3) AppClassLoader 也是用户可见的 ClassLoader, 它加载的是 classpath 下面的内容, 也就是和 classpath 相关的类, 在默认情况下都由它来加载。

(4) 用户自定义的 ClassLoader 要加载的内容可能不在系统的 classpath 范围内 (当然也可以在这个范围内), 甚至不是 Class 文件或 jar 文件, 也就是加载方式可以完全由自己来定义。

例如, 想要将自己写的一段 Java 代码动态加载到 JVM 进程中, 就可以通过自定义的 ClassLoader 来实现 (当然自定义的 ClassLoader 还有其他的用途, 比如可以加载一些远程的 jar 包)。在 3.2.4 节介绍字节码增强的时候也会说到这个问题。

用户自定义的 ClassLoader 一般继承于 URLClassLoader, 也可以继承于 ClassLoader 或 SecureClassLoader, 它们之间本身也是继承关系, 根据实际情况重写不同的方法即可。

进程启动时, 通常会加载一些 JVM 的核心库, 它并不会加载项目中所有的 Class (因为这些 Class 还是会占用很多空间的, 例如项目中可能有好几百 MB 甚至更大的 jar 包, 它们都不是系统启动时加载的)。除了加载一些 JVM 的核心库外, 通常还会加载引导方法 (main) 所在的类, 以及 main 所在类会使用到的类。

Class 本身也是一种对象, 是用于描述普通 Java 对象的格式的一种对象, 既然是对象, 那么它的加载也需要一个过程。

第一步: 读取文件。读取文件首先得找到文件, 也就是找到 .class 文件, 它将以“类全名+ClassLoader 名称”作为唯一标识, 加载于方法区内部。

在加载过程中, 首先申请父 ClassLoader 来加载, 也就是尽量用父类的 ClassLoader 来加载; 如果父 ClassLoader 无法加载这个类 (例如 classpath 下没找到), 那么就由子类来加载; 如果最终都没找到对应的类, 则会抛出异常 ClassNotFoundException。

第二步: 链接。这个动作内部就是要对加载的字节码进行解析、校验, 看是否符合字节码的规范结构, 如果不符合就会抛错 ClassNotFoundException (注意: 这里不是 ClassNotFoundException, 两者区别很大)。这一步还会为 Class 对象分配内存 (注意: 第一步仅仅是找到文件并读取文件, 即使分配了内存, 也只是一个 byte[] 数组, 这一步才会为 Class 对象分配内存, 两者完全不一样)。这一步可选的是将常量池中的符号引用解析为直接引用。

第三步：初始化。会调用这个 Class 对象自身的构造函数，对静态变量、static 块进行赋值（通过 javap 指令可以发现，其实许多静态变量的赋值会在编译时放在 static 块中完成，后面还会提到一些细节）。

所有的类在使用前都必须被加载和初始化，初始化过程由<clinit>方法来确保它是线程安全的（包括 static 块也必须执行完，才能被使用），如果有多个线程同时尝试访问该类，则必须等待 static 块执行完成，否则都将被阻塞。

虽然<clinit>方法可以保证线程安全，但是不会保证加载过程是绝对 OK 的（即使 Class 文件本身没问题）。例如，当在 static 块中引用自身类的一个实例时，JVM 在加载这个类时会调用这个 static 块，因此就会去引用相应的实例，而 JVM 又会发现这个类还未加载，但是目前正在加载这个类的过程中，因此就会出现类无法加载的问题。这类问题很多时候隐藏得比较深，也就是这部分调用是通过某些调用间接调用的。下面是一段代码示例（光盘：src/chapter03/load/LoadClassErrorDemo.java）。

代码清单 3-2 代码不当导致 Class 加载失败

```
abstract class A {
    public A() {
        list();
    }

    public void list() {
        test();
    }

    abstract void test();
}

class B extends A {
    private final static B instance = new B();

    public static B getInstance() {return instance;}

    public void test() {
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        instance.test2();
    }

    public void test2() {}
}

public class LoadClassErrorDemo {
    public static void main(String []args) {
        new Thread() {
            public void run() {
                try {
                    Thread.sleep(300);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                B.getInstance().test2();
            }
        }.start();

        new Thread() {
```



```
public void run() {
    try {
        Thread.sleep(400);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    B.getInstance().test2();
}
}.start();
}
```

程序启动不到 2s 的时间, 两个线程就会抛出错误信息:

```
java.lang.NoClassDefFoundError: Could not initialize class chapter03.load.B
```

看到这个错误, 说明代码有问题, JVM 通常是无法关心的, 它也不会去关心, 所以我们自己写代码就要小心了。

为了证明多个线程同时去访问类时, 线程是安全的, 胖哥又写了一段代码, 见本书光盘: src/chapter03/load/LoadClassBlockingDemo.java, 运行代码会发现多个线程去访问类时, 都会被阻塞在类加载上 (第 1 个线程会先 sleep)。

Class 的加载会阻塞多个线程, 因此会使用到的 Class 最好被预先加载到内存中 (可以叫它预热), 否则用的时候再去加载就可能会发生阻塞。

另外, 在 static 块中写的代码越复杂, 处理时间就越久, 需要访问这个类的所有线程将被阻塞得越久, 如果在 static 块中出现了死循环, 那么相关的线程将一直阻塞在这里; 如果是在系统启动时出现了类似的情况, 则可能系统一直都在启动中。

换句话说, static 块是一个好东西, 它可以让我们做一些线程安全的扩展, 但是它并不是适合于编写大量复杂业务的地方, 尤其是大量的 I/O 操作逻辑。

现在, 回到类加载的话题上。

我们知道了, 程序访问一个类之前, 必须先加载类。预先加载如果我们自己来做就会很麻烦, 通常在实际场景中, 程序是一层嵌套一层的, 很多类都会在初始化的时候被加载。例如, Tomcat 启动时除了会加载自己的类以外, 还会加载各个 deploy 的一些类, 各个 deploy 下面可能会有 Listener、Filter、Servlet 等信息, 而在这个过程中还会间接加载一些三方框架, 比如 Spring, 这些框架又会间接加载许多业务程序的类, 这样就串联起来了。

如果想看看一段程序到底加载了什么类, 则可以通过在 Java 程序运行时增加启动参数: -XX:+TraceClassLoading 来实现。大家可以在一个大工程里面自己写一个简单的 main 方法, 这个 main 方法只具有类似于 HelloWorld 的简单输出, 看看会不会将整个工程类都加载进来, 或者只是加载某些相关的类。

疑惑: 用到类的时候再加载, 是不是会很慢?

答曰: 如果是在容器下写代码, 大部分时候我们不用关心, 大部分类都会被直接或间接加载。另外, 在同一个 ClassLoader 中的一个类原则上只会被加载一次。

ClassLoader 的一些知识补充

(1) 同一个“进程”内任何一个 ClassLoader 加载的 Class 都可以由任意一段程序直接或间接地访问到。在常见的 Web 容器 (如 Tomcat) 中加载不同的 deploy 会进行相互隔离, 它使用了不同的 ClassLoader 来加载不同的 deploy, 也就是并非真正意义上的隔离。要想跨 ClassLoader 访问一些信息是比较麻烦的, 不过也并非不可能, 例如使

用 JMX 就可以帮我们做到。

(2) 在 Class 加载中, 对于同名的类 (也就是 package 路径以及类名都是一样的), 在同一个 ClassLoader 中只会被加载一次 (除非被卸载或被重定义)。

(3) JVM 做 FULL GC 的时候, 允许对 Class 执行 unload 操作, 但是苛刻的条件是相应的 ClassLoader 下所有的 Class 都没有实例的引用, 否则这个 ClassLoader 及它下面所有的 Class 都释放不掉。跟踪类的卸载, 可以通过添加启动参数: `-XX:+TraceClassUnloading` 达到目的 (在后文中有演示)。

(4) 如果希望任意一段代码中所创建的对象是通过自定义的 ClassLoader 创建的, 那么需要这段代码的调用者所在类是通过相应的 ClassLoader 加载的, 因此调用者通常需要指定 ClassLoader 去实例化才能达到这样的效果 (见第 6 条)。其次, 自定义的 ClassLoader 所加载的调用者的类一般不放在默认的 classpath 下面, 否则将被系统 ClassLoader 所加载。如果确实遇到了这样的情况, 那么这个 ClassLoader 在创建的时候, 强制将 `parentClassLoader` 设置为空也可达到目的, 例如: `new URLClassLoader(urls, null);`。

(5) 查看一个类本身是使用哪一个 ClassLoader 来加载的很简单, 例如有一个 User 类, 使用 `User.class.getClassLoader()`。

(6) 当使用 `Class.forName()` 的时候, 有一个重载方法可以带上指定的 ClassLoader, 如果没有带上 ClassLoader, 则默认与“调用者当前类”加载的 ClassLoader 一致。

(7) ClassLoader 自身也是一个 Java 类, 因此它本身也需要加载, 加载的顺序是先由 JVM 内在级别的 ClassLoader 开始的。换句话说, ClassLoader 本身是一类更为特殊的 Class, 在没有 ClassLoader 时谁来加载它呢? 只能由 JVM 内核来加载。

(8) 当通过一个指定的 ClassLoader 调用方法 `loadClass("类名")` 加载一个类的时候, 这个 ClassLoader 会首先判定自身是否加载该类, 如果未加载, 就到 `parentClassLoader` 找是否加载该类, 依此类推。如果最终没有找到, 则会从顶层 ClassLoader 开始去尝试加载该类 (如果父类无法加载, 子类再尝试加载), 如果在这个 ClassLoader 所能找到的路径下, 没找到这个类, 则会抛出异常 `ClassNotFoundException`。

(9) 加载类的时候, 如果类文件出现了问题 (字节码结构不符合规范, 可以自己篡改一个 Class 文件); 类中引用了 native 方法, 但是缺少对应的动态链接库 (.dll、.so 文件); 由于代码问题直接或间接出现对当前类的使用, 导致加载失败, 此时会报错 `NoClassDefFoundError` (例子见代码清单 3-2)。

(10) 当用到反射时, 反射的代码通常会将类的操作实体做一份拷贝给我们使用。如果频繁大量地使用反射, 为了提升效率, 可以将相关的 Class、method、field 等信息保存起来, 便于复用。

(11) ClassLoader 内部有一个 `defineClass` 方法 (方法是 protected 类型, 继承才能调用到), 通过传入 `byte[]` 数组就可以动态加载 Class 类 (在下一节讲解字节码增强时会说明)。

(12) Class 加载的时候, 如果父类并没有被加载到相关的 ClassLoader 中, 则会先加载父类, 并且会先执行父类的 static 块 (这个顺序似乎不是那么重要, 因为程序中必须要等待相应的 Class 加载完后才能使用对象)。

在 ClassLoader 加载 Class 的过程中, 是按照前面介绍的字节码结构开始解析 Class 字

节码文件的内部结构的。相对 Java 源文件, 它更加结构化一些, 因此也叫中间代码。

在 JVM 加载的过程中还会产生很多的中间代码, 会做一些优化后转换为目标代码开始运行, 运行时会有 JIT 的介入, 它会做动态内联及代码的进一步优化并将它尽量 Cache 住。作为 Java 程序员, 应该知道它的运行时优化不止一层, 也不是立即被优化到最佳的, 而是随着运行次数的增加, 会被 JIT 不断去优化。这样, 自己在写代码的时候, 就要考虑代码是否可能被 JIT 优化。

例子 1:

```
String a() {  
    return b() == null ? "" : b();  
}
```

说明: b()方法调用后, 如果返回不为空, 则会再一次调用 b()方法。这表面上看没有什么问题, 如果这个 b()方法内部很简单也无所谓, 而且在 JIT 动态 inline()后就会合并在一起(在一个数组的大量 for 循环中, 许多人说将 size()先获取出来会快一些, 其实当 JIT 动态 inline()后效率是一样的)。但是如果 b()方法内部存在大量的计算, 甚至是数据库、磁盘、接口的访问, 在这个代码中, 隐藏就访问了两次 b()方法, 而有一次访问显然是多余的, 增加了系统额外开销。

例子 2:

```
Map map = ....  
....  
if (map.containsKey()) {  
    Object b = map.get(xxx);  
    ....  
}
```

说明: 其实很多朋友会这样写代码, 因为这样的代码很干净, 不过这样写代码其实会进行两次 Map 的查找, Map 的查找本身也是需要时间开销的(在第 1 章已经提到, 这个开销是和具体的 Map 结构相关的)。这段代码, 一对性能会有点影响, 毕竟进行了两次查找, 如果要改, 自然是用 get 操作, 看数据是不是空来判定是否存在。Map 结构内部也是这样实现的。

JIT 优化会将优化后的本地代码存放在 CodeCache 中, 这个空间默认的大小为 48MB, 可以使用参数-XX:ReservedCodeCacheSize=64m 来修改这个空间的大小, 参数-XX:+Use CodeCache Flushing 可以对 CodeCache 进行清理。另外, 参数-XX:CICompilerCount 的设置相对较大可以在一定程度上提升 JIT 编译的速度(默认为 2), 在高并发系统中适当设置可以有效地提升性能。

当需要使用自定义的 ClassLoader 来加载某些 Class 时, 而这些 Class 通常会使用当前 JVM 进程中由其他 ClassLoader 已经加载的一些类, 那么在创建自定义的 ClassLoader 时, 可以调用指定的 URLClassLoader.getURLs()方法获取到相应的 classpath 列表, 然后通过 File.pathSeparator 来获取到当前 OS 的 classpath 的分隔符(在 Windows 上是英文分号“;”, 在 Linux 上是英文冒号“:”)。这样就可以重新拼接出一个新的 classpath, 当需要编译某个动态类的时候, 就可以带上这些 classpath 作为参数的 option, 被动态编译的 Java 文件内部所 import 的相关依赖就可以被识别到了。除此之外, 自定义的 ClassLoader 也能识别到 parentClassLoader 的所有类, 若自定义的 ClassLoader 在实例化时未指定 parentClassLoader, 也会默认为调用者的类所对应的 ClassLoader, 此时可能会产生一些加载冲突。为了避免冲突, 通常是在自定义的 ClassLoader 实例化时将 parentClassLoader 参数传递 null 值来达到目的。

再来看看和 Class 相关的一些实用方法。

◎ 同一个应用中, 同名的类出现在不同的 jar 包中, 导致加载出现了问题, 但是自己

不知道该来自哪个 jar 包时,则可以使用 `XXX.getClass().getResource("").getPath()`, 获得这个 Class 的来源路径是哪个 jar 包。

- ◎ 当需要找到某个 ClassLoader 下的 Resource 列表时,就可以使用 ClassLoader 提供的 `getResource(String)` 方法 (Spring 的扫描器就是通过这种方式实现的,具体请参看第 9 章),传入的参数如果是 package 名称,则会找到 jar 包内部;如果是空字符串,则不会找到 jar 包内部。值得注意的是,如果 jar 包中还有 jar 包,那么通过这个方法是获取不到的。

疑惑: 同名的类不是只能被加载 1 次吗?

答曰: 一个 ClassLoader 对同一个类只能加载 1 次,不同的 ClassLoader 分别加载同一个类。

疑惑: 除了 ClassLoader 加载多个 Class 外,我想直接修改这个 Class 可以吗?

答曰: 可以,“字节码增强”可以改变这一切。

疑惑: 加载这么多 Class,如果 Class 不需要了,GC 能回收吗?

答曰: 可以,在后文中会论证这个结论。

疑惑: 平时写代码时类的加载存在上面描述的问题,一定会抛出错误吗?

答曰: 不一定,本书光盘中的 `LoadClassErrorDataDemo.java` 文件演示了由于代码使用不当,最终导致数据的结果不对。

类中的 static 块、static 变量、final static 变量、普通块、对象的构造方法执行顺序是什么?

本书光盘中的 `LoadObjectStepDemo.java` 文件演示了 static 块、普通块、构造方法的调用顺序。大家用 `javap` 命令查看指令,便可以发现 static 静态变量的赋值操作会被放在 static 块中来执行,如果被赋值的是 final static 普通变量 (如 int 或常量池中的 String,即直接常量赋值,不是 `new String()`),则会直接赋值;如果是对象,则依然编译到 static 块中执行,由 `<clinit>` 保证线程安全。普通块 (非 static 块) 将被编译到相应类的构造方法自定义语句的前面,不过是在调用 `super()` 操作的后面。

3.2.4 字节码增强

本节将介绍有哪些常见的字节码增强技术,以及字节码增强技术的实现方式,了解 AOP 实现的原理。

有些时候想要用业务中的“某些代码”去执行一些公共的动作,例如写日志、事务管理等。大家都知道,每个地方都写这样的代码,那么重复代码就会很多,而且不好维护。那么通过设计模式的方式,或许模板方法可以帮助我们解决一些问题 (父类将公共的操作写好,对于业务上的动作父类用抽象方法表示,具体实现部分交给子类去完成)。就像一些请求分发框架,要求每个 Action 都要继承框架提供好的 `BaseAction`,然后实现 `execute` 方法。不过,这样的方式缺少灵活性,每个动作都必须写一个类来完成,对于交互性很强的系统来讲,就需要写太多的类。

于是我们想要尝试用修饰的方式,就是将具体的实例类放入修饰类中。外部程序访问任意一个方法都会先经过修饰类的同名方法,然后再调用对应的实体来完成具体的操作,修饰类可以在调用具体的方法前后做一些操作。不过,新的问题出现了,相关的每个方法都要去写修饰,那还不如直接写到原来的类当中了 (当然,这里并非说修饰是没用的,它也有许多存在的价值,当它的修饰得到巨大的通用性时,使得子类和调用者可乐享其成就

是很好的使用方式)。

遇到大量的方法需要相同的修饰方式时,我们期望有一种动态的方式来代理操作,于是动态代理机制出现了(在后文中有介绍)。当然,实现方式不止这一种,还可以直接修改类的字节码,或者生成某个类的子类。

这样一来,作为代码的管理者则不会再担心每个地方都写同样的调用,担心如果漏掉了会出问题,担心写的方式 API 发生改变,担心某一天不需要这些内容的时候该如何处理。

下面再举一个与字节码增强相关的例子。

某一天,系统出现了 OOM,通过工具分析出来是某个类的对象占用了很大的空间,如果知道这个类在什么地方被写入,则很容易解决问题;但是如果这个对象的写入被许多程序访问,那么就要知道什么代码会存在大量的写入,就要追溯到调用这个类的地方在哪里。它们可能是二方包、三方包的类,怎么找?难道用工程的全文匹配?那样也只能找到自己的业务代码调用的地方,而不能找到更加深入的反射、二方包、三方包的调用。或许这个时候 AOP 可以帮助完成这件事情。

AOP 技术其实是字节码增强技术,JVM 提供的动态代理追根究底也是字节码增强技术。

先简单聊聊动态代理的基本使用。动态代理是 Java 默认提供的 AOP 技术,也是早期的 Spring AOP 框架所选用的技术(早期的 Spring 代理类必须用接口获取返回值)。既然是代理,自然就需要一个代理原始类的实体存在,Java 提供了“`java.lang.reflect.InvocationHandler`”接口,我们需要实现这个接口来实现它的 `invoke` 方法。

所有类的实例化都通过 `java.lang.reflect.Proxy.newProxyInstance()` 来返回(但是要通过接口获取返回值),在这个动作的内部会动态创建一个类(这个类会实现所有传入实例的类的所有接口),通过这个类创建一个 Proxy 的实体,返回的实例也是动态 Proxy 创建的类的实例(传入的实例将作为这个实体内部的一个 `target` 属性存在)。

这样一来,在执行方法的时候,就会先调用自定义实现 `InvocationHandler` 接口的 `invoke` 方法,所以实现了 AOP(第 7 章中还会继续介绍更多的细节,并有实例代码可以参考)。

这种技术可以实现 AOP,但是有几个缺陷:其一,它必须基于接口来实现,也就是代理的类一定要有接口,而且只能调用接口对应的方法,实例的其他方法无法访问;其二,它实现的方法就好比一扇大门,进去了就可以随便调用,方法之间的调用不会再进行 AOP,它只在外部调用这些方法时,才会调用代理的 `invoke()` 方法。

除此之外,字节码增强技术有两种实现机制:一种是通过创建原始类的一个子类,也就是动态创建的这个类继承原来的类,现在的 SpringAOP 也正是通过这样的方式来实现的,它们创建的子类通常以“原始类的名称”为前缀,再加上一个奇怪的后缀,以避免类名重复;另一种是很暴力的,即直接修改原先的 `Class` 字节码,在许多类的跟踪过程中会使用到此技术。

实现字节码增强要通过如下两个步骤:

❶ 在内存中获取到原始的字节码,然后通过一些“开源届”写好的 API 来修改它的 `byte[]` 数组,得到一个新的 `byte[]` 数组。

❷ 将这个新的 `byte[]` 数组写到 `PermGen` 区域,也就是加载它或替换原来的 `Class` 字节码(字节码增强还可以在进程外调用完成,后文中也会提到)。

先谈谈第 1 个步骤中提到的开源技术。现在使用比较多的字节码修改的 API 大概有 ASM、javassist、BCEL、SERP、CGLib(基于 ASM)等基础技术,本节将通过 ASM、javassist

来介绍一些字节码增强技术，在 3.6.2 节中将通过 CGLib 来动态创建子类模拟 PermGen 内存溢出。这些开源组建的 jar 包，胖哥已经放到了本书所附带的光盘中，请大家直接使用即可。

获取到新的字节数组后，就要想办法来加载它。其中一种方式是基于 Java 的 Instrument API 来实现；另一种方式是通过指定的 ClassLoader 来完成。它们两者之间的区别，后文中会提到。

现在提出一个 AOP 的需求：想要在某些方法执行前，输出点内容。

此时要采用的技术是使用 ASM 来改变字节码，然后通过 ClassLoader 来加载改变后的字节码信息，当系统再一次通过这个 ClassLoader 来获取类的实例时，就能得到修改后的 Class 对应的实例了。

首先，得有“切入”的目标，因此我们自己来定义一个需要被“切”的类，这个类的定义如下。

代码清单 3-3 一个简单的用 ASM 测试的类的定义

```
public class ForASMTTestClass {

    private String name;

    private String value;

    public void display1() {
        System.out.println(name);
        System.out.println(value);
    }

    public void display2() {}
}
```

这段代码够简单吧，胖哥就喜欢用简单代码来给大家说明问题。

接下来使用 ASM，代码片段如下（类名是“ASMClassModifyAdpter”，在本书光盘的 src/chapter03/asm/ASMClassModifyAdpter.java 文件中）。

代码清单 3-4 主要代码用于访问方法

```
public MethodVisitor visitMethod(final int access,
    final String methodName,
    final String desc,
    final String signature,
    final String[] exceptions) {
    if("display2".equals(methodName)) {
        return null; //屏蔽了这个 display2 方法
    }
    if("display1".equals(methodName)) {
        MethodVisitor methodVisitor
            = cv.visitMethod(access, methodName,
                desc, signature, exceptions);
        methodVisitor.visitCode();
        //下面几条代码是对 name 属性赋值，相当于增加代码：name = "我是 name"
        //加载 this 到栈顶（此时本地变量其实只有一个 this）
        methodVisitor.visitVarInsn(OpCodes.ALOAD, 0);
        //ldc 指令，从常量池中取出值加载到栈顶
        //这个代码会隐藏修改常量池
        methodVisitor.visitLdcInsn("我是 name");

        //putfield 指令，修改 ForASMTTestClass 的 name 属性
        methodVisitor.visitFieldInsn(OpCodes.PUTFIELD,
            "chapter03/asm/ForASMTTestClass",
```



```

        "name" , "Ljava/lang/String;");

//同样是对属性赋值, 相当于增加代码: value = "我是 value"
//加载 this 到栈顶
methodVisitor.visitVarInsn(Opcodes.ALOAD , 0);
//ldc 指令, 从常量池中值加载到栈顶
methodVisitor.visitLdcInsn("我是 value");
//putfield 指令, 将栈顶的值赋值给 ForASMTTestClass 类的 value 属性
methodVisitor.visitFieldInsn(Opcodes.PUTFIELD ,
                             "chapter03/asm/ForASMTTestClass" ,
                             "value" , "Ljava/lang/String;");

//再添加一个属性获取出来并打印 (也就是打印会多一次)
//getstatic 指令, 获取 System 类的 out 属性
methodVisitor.visitFieldInsn(Opcodes.GETSTATIC,
                             "java/lang/System",
                             "out" ,
                             "Ljava/io/PrintStream;");

//加载 this 到栈顶
methodVisitor.visitVarInsn(Opcodes.ALOAD, 0);
//通过 getfield 指令将 ForASMTTestClass 类的 name 属性加载到栈顶
methodVisitor.visitFieldInsn(Opcodes.GETFIELD,
                             "chapter03/asm/ForASMTTestClass",
                             "name", "Ljava/lang/String;");

//调用 out 的 println 方法
methodVisitor.visitMethodInsn(Opcodes.INVOKEVIRTUAL,
                             "java/io/PrintStream",
                             "println",
                             "(Ljava/lang/String;)V");

methodVisitor.visitEnd();
return methodVisitor;//返回 visitor
}else { //其余方法不做任何处理直接返回
    return cv.visitMethod(access, methodName,
                          desc, signature, exceptions);
}
}
}

```

这样字节码就增强了? 是的!

不过感觉好复杂!

确实是。不过, 如果你看过前面两个小节, 那么理解起来并不困难, 就是在写虚指令操作。

现在, 通过这段代码就可以得到一个修改方法后的类的字节码信息, 但是还需要进一步写到 Perm Generation 中, 写入后就正式成为一个真正的 Class 类了。

按照我们的预期, 如果通过这个修改后的 Class 类来创建一个实例, 并通过这个实例来调用 display2()方法, 就会出现报错提示: 没有这个方法, 当调用 display1()方法时就会先打印上面想打印的内容。

为了得到这个结论, 我们自己需要实现一个 ClassLoader 来加载修改后的 byte[]数组, 以得到一个 Class 对象 (现在只有一个 byte[]数组, 还得进一步转换)。为何要自己实现一个 ClassLoader 呢? 前面提到过在 ClassLoader 中通过 byte[]数组加载类的方法是 protected 类型的, 外部程序无法直接访问到这个方法 (用反射也可以), 我们所选择的继承方式是简单的, 其实不用写什么代码。

代码清单 3-5 准备好 ClassLoader 是关键

```
public class DynamicClassLoader extends URLClassLoader {
```

```

    public DynamicClassLoader(URLClassLoader parentClassLoader) {
        super(parentClassLoader.getURLs(), parentClassLoader);
    }

    public Class<?> defineClassByByteArray(String className, byte[] bytes) {
        return this.defineClass(className, bytes, 0, bytes.length);
    }
}

```

大家是不是想起来 ClassLoader 部分的知识了？注意：这个 defineClass 对于同名的类如果调用两次以上，就会报错。

有基本测试类了，有 ASM 字节码增强的代码了，也有自定义的 ClassLoader 了。“黄瓜白菜全齐了，接下来就要将“菜切好下锅了”。

代码清单 3-6 测试字节码增强代码从这里开始

```

public class ASMTTestMain {

    Private final static DynamicClassLoader TEST_CLASS_LOADER
        = new DynamicClassLoader(
            (URLClassLoader) ASMTTestMain.class.getClassLoader()
        );

    public static void main(String []args)
        throws Exception {
        //修改前的 Class
        Class <?>beforeASMClass
            = TEST_CLASS_LOADER.loadClass("chapter3.asm.ForASMTTestClass");
        //修改 Class
        TEST_CLASS_LOADER.defineClassByByteArray(
            "chapter3.asm.ForASMTTestClass", asmChangeClassCall());
        //修改后的 Class
        Class <?>afterASMClass
            = TEST_CLASS_LOADER.loadClass("chapter3.asm.ForASMTTestClass");

        Object beforeObject = beforeASMClass.newInstance();
        Object afterObject = afterASMClass.newInstance();
        beforeASMClass.getMethod("display1").invoke(beforeObject);
        afterASMClass.getMethod("display1").invoke(afterObject);
    }

    private static byte[] asmChangeClassCall() throws IOException {
        ClassReader classReader
            = new ClassReader("chapter3.asm.ForASMTTestClass");
        ClassWriter classWriter = new ClassWriter(ClassWriter.COMPUTE_MAXS);
        ASMClassModifyAdpter modifyAdpter
            = new ASMClassModifyAdpter(classWriter);
        classReader.accept(modifyAdpter, ClassReader.SKIP_DEBUG);
        //这里输出的字节码，代码已被注释，若输出到文件，则可以用 javap 命令来查看
        //byte []bytes = classWriter.toByteArray();
        //new FileOutputStream("d:/ForASMTTestClass.class").write(bytes);
        //return bytes;
        return classWriter.toByteArray();
    }
}

```

输出结果如下：

```

null
null
我是 name
我是 name
我是 value

```


看到这个结果是不是有点蒙，什么玩意啊？

大家可以再回头来看看代码，两个实例分别是用修改前的 Class 和修改后的 Class 创建的，都调用了 `display1()` 方法，在这个方法中使用了两个输出语句分别输出属性 `name` 和 `value` 的值。首先是修改前的 Class 创建的对象，输出了两个 `null`，因为没有被赋值。

接下来输出的 3 行是什么呢？在字节码增强的地方，首先对 `name` 和 `value` 分别赋值为：“我是 `name`”、“我是 `value`”，然后将 `name` 取出来并打印，也就是相当于在 `display1()` 方法的第 1 条代码前增加了 3 条代码：

```
this.name = "我是 name";
this.value = "我是 value";
System.out.println(name);
```

这样加上原本方法的 `body` 中两条输出 `name`、`value` 的代码，就输出了结果中的后面 3 行。

如果还没明白，可以将本书光盘中的代码拷贝到自己的工程中运行，读一读，改一改，就清楚了。

大家可以自己改改代码，尝试下两个 Class 的实例分别调用 `display2` 方法，被修改后的 Class 对应的实例在调用时会报错：`java.lang.NoSuchMethodException`。

ASM 是一种修改字节码本身的 API，它实现的抽象层次是很低的，几乎接近于指令级别，类似上面增加的都是基于指令和操作数的，跟我们前面提到的字节码本身的结构很像。关于 ASM 更多的操作细节，请参考互联网上的相关介绍。

再来想想 Spring 的 AOP，如果它用字节码增强来实现会怎么做？或者说，如果你要用字节码增强来实现，你会如何做？

我们有可能会自己定义一个 `ClassLoader`，然后所有相关的类都用这个 `ClassLoader` 来加载，而不是默认的 `ClassLoader`，通过 ASM 字节码增强实现之，当用户代码调用 `getBean` 方法时，返回的就是对应字节码修改后的 Class 所产生的对象。这是一种实现方式，行得通。

但是细心的你可能已经发现了一些现实的问题，ASM 操作虽然和字节码很像，简化了操作二进制的麻烦（将二进制指令转换为字符、常量，内在的许多依赖结构变化由 ASM 封装），但是也同样需要记 200 多个指令，还有指令后接的东西，太烦琐了。

于是我们就在想：给一小段 Java 代码，它能动态载入，我想切入到哪里都行，这样就“爽歪歪”了。下面我们就介绍用 `javassist` 的 API 来实现字节码增强，代码有点像下面这样。

代码清单 3-7 javassist 操作的 API 要简单多了

```
CtClass ctClass
= ClassPool.getDefault().get("xxx.xxx.xxx");
CtMethod ctMethod = cc.getDeclaredMethod("test");
ctMethod.insertBefore("System.out.println(\"打印我吧，求求你了！\")");
ctMethod.insertAfter(
    "String a = \"XXX 来了，快跑！\" +
    \"System.out.println(\"我勒个去！\")");
);
byte[] bytes = cc.toBytecode();
```

就这么简单的方式字节码就被增强了，调用程序得到返回的 `byte[]` 后，就可以像前文中提到的通过 `ClassLoader` 来加载类了。需要注意的是，`javassist` 对同一个类只能重新定义一次，它会记录这个类已经被修改，再次字节码增强时，会报错：`class is frozen and pruned`。

喜欢思考问题的你，又开始琢磨了：很多时候我们的代码其实是 new 出来的，不一定是从类似 Spring 的容器里面获取的，就像线上发布的程序，我们想随时写点代码跟踪一下，某些类不一定是从 Spring 里面获取的，那就没法子了。另外，这种 ClassLoader 加载后，如果对同一个类再做一次 define 操作，就会报错，但是想要再次修改还有方法吗？

有的，还不止一种哦！

其一，在加载 Class 前增加一个 ClassFileTransformer，通过 ClassFileTransformer 来修改字节码。

其二，我们可以通过 Instrumentation API 中的 redefineClasses(ClassDefinition)方法来重定义 Class 类，但是有些细节需要知道，否则无法运行起来（我们会在下文中说明）。

我们要知道 Instrumentation API 程序无法直接使用，需要通过 Agent 的方式来启动它（或通过动态加载 Agent 的方式），它有点像在启动 main 方法前由 JVM 内部去调用，所以用这段代码时，首先需要将操作 Instrumentation 的代码打包为一个 jar 包，例如 agent.jar，在运行实际调用代码部分时，在启动参数中需要增加 -javaagent:agent.jar 来运行。

接下来胖哥用两种方式来修改字节码信息，其中第 1 种方式是类加载时去修改字节码信息；第 2 种方式是运行时修改字节码信息。

1. 类加载时修改字节码信息

首先我们要构造一个 Transformer，它需要实现接口 ClassFileTransformer，并实现其中的 transform 方法，代码如下。

代码清单 3-8 我们构造自己的一个 Transformer

```
public class TestTransformer implements ClassFileTransformer {

    @Override
    public byte[] transform(ClassLoader loader, String className,
                           Class<?> classBeingRedefined,
                           ProtectionDomain protectionDomain,
                           byte[] classfileBuffer) throws IllegalClassFormatException {
        //输出加载的类名，同样看到了类的加载
        System.out.println("load class:" + className);
        //只有指定的类才加载
        if("chapter03/asm/ForASMTTestClass".equals(className)) {
            try {
                CtClass ctClass
                    = ClassPool.getDefault().get(
                        className.replace('/', '.'));
                CtMethod ctMethod = ctClass.getDeclaredMethod("display1");
                ctMethod.insertBefore(
                    "name=\"我是name! 这次用javassist了哦!\";" +
                    "value=\"我是value!\";" +
                    "System.out.println(\"我是加进去的哦: \" + name);"
                );
                ctMethod.insertAfter("System.out.println(value);");
                return ctClass.toBytecode();
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
        return classfileBuffer;
    }
}
```



```

        return classfileBuffer;
    }
}

```

在这段代码中, 如果传入的类是 `chapter03/asm/ForASMTestClass`, 则会将 `display1()` 方法切入。但是这段代码还仅仅是切入代码的定义部分, 接下来要做的是让 JVM 在加载类的时候调用它才能达到效果。

此时需要将这个 Transformer 注册到 Instrumentation 中, 那么自然就需要先写一个 Agent 程序, 代码如下。

代码清单 3-9 将 Instrumentation 载入内存

```

public class InstForTransformer {

    /*编译为 Agent 后, 系统启动执行 main 方法前会调用它*/
    public static void premain(String agentArgs,
                               Instrumentation instP) {
        instP.addTransformer(new TestTransformer());
    }
}

```

如果注册成功, 那么在 JVM 加载用户的 Class 时, 就会调用对应的 TestTransformer 类的 transform 方法来获得修改后的字节码, 达到修改字节码的目的。就像一个应用上的过滤器一样, 所以这个概念也是相对的, 就算是虚拟机层面也是这样。

Agent 的代码并不复杂, 这里就一条代码, 接下来要将这一小段程序的 Class 文件打一个 jar 包 (仅仅是这一小段程序)。你可以就在这个工程里面打包 (打包是指定文件), 也可以将 Class 文件单独拷贝出去打包, 如果要拷贝出去, 就要在对应的地方建立 package 目录。当前的 package 路径是: `chapter03.inst.transformer`, 在 `chapter03` 目录下使用命令:

```
jar cvf transformer.jar *
```

打包完成后, 事情还没完, 我们还要做点事情才行!

一些复杂的功能自然需要一些复杂的操作, 而且这类操作都是一次性的, 通常项目建立后, 就不会有太多这样的操作了。

打开这个 jar 包, 进入 META-INF 文件夹, 里面有个文件叫 “MANIFEST.MF”, 是一个文本文件, 在文件中增加一行:

```
Premain-Class: chapter03.inst.transformer.InstForTransformer
```

它用于标识这个 jar 包中的哪一个类是 Agent 的类, 这个类的 premain 方法将被 JVM 调用。第 2 个参数就是 Instrumentation 的实例, 用来做接下来的操作。

需要特别注意: 冒号后面要有个空格, 这个不能少, 否则会报错:

```

Error opening zip file or JAR manifest missing : /tm/agent/transformer.jar
Error occurred during initialization of VM
agent library failed to init: instrument

```

接下来将这个 `transformer.jar` 放到一个目录下, 例如 `/tmp/agent/transformer.jar`。在 Windows 上操作也是类似的, 例如 `d:/tmp/agent/transformer.jar` (注意: 这里的目录标识符都用 “/” 符合统一标准, 在任何操作系统上都通用)。

与上面的例子一样, “各样菜都准备好了, 要准备下锅了, 这次下锅有两种方式: 一种是清蒸, 一种是爆炒, 不过不论怎么样, 调料需要先准备好, 就是怎么调用下一个被 “虐” 的类: `ForASMTestClass`, 简单的两条代码搞定调料”。

代码清单 3-10 下点小调料, 美味就出来了

```
public class TestformerTestMain {

    public static void main(String []args) {
        ForASMTestClass testClass = new ForASMTestClass();
        testClass.display1();
    }
}
```

这段代码没有任何主动调用字节码增强的部分, 程序本身的代码中 new 出来的对象, 竟然可以被修改? 带着这个疑问, 我们就来运行一下吧, 运行的方式请看接下来的介绍。

使用命令行或 IDE 工具来运行这段代码都可以, 用命令行怎么运行呢? 首先, 将命令的当前路径切换到即将运行的 Class 对应的最外层的 package 上, 或者将该路径设置到 classpath 中。然后将 javassist.jar 加入到环境变量中 (可以直接在当前命令行加上, 无须修改用户的配置文件)。最后, 根据 transformer.jar 的位置来运行, 运行结果如图 3-22 所示。

```
[xieyu000@java_A ~]$ java -javaagent:/tmp/agent/transformer.jar chapter03.inst.transformer.TestformerTestMain
load class:sun/launcher/LauncherHelper
load class:java/lang/Enum
load class:chapter03/inst/transformer/TestformerTestMain
load class:java/lang/void
load class:chapter03/asm/ForASMTestClass
我是加进去的哦, 哈哈: 我是name! 这次用javassist了哦!
我是name! 这次用javassist了哦!
我是value!
我是value!
load class:java/lang/Shutdown
load class:java/lang/Shutdown$Lock
```

图 3-22 运行结果, 在 Windows 上改改路径就可以了



注意 如果运行中抛出了这样的错误 (有些机器可以直接运行):

```
java.lang.UnsupportedOperationException: adding retransformable transformers
is not supported in this environment
```

那么就在 Agent 的 jar 包中增加如下一行来解决问题:

```
Can-Retransform-Classes: true
```

现在回过头来看看输出结果为什么是那样的。关于 load class: 的几条信息, 大家应该看得出来是类加载的信息, 就像跟踪了类的加载一样。

通过 javassist 的 insertBefore 插入的代码, 对 name 和 value 两个参数赋值, 并且输出了 name 以及一段“恶搞的前缀”, 然后用 insertAfter 添加输出 value 的代码, 加上原有的代码输出 name 和 value, 实际运行时的代码就变成了:

```
name = "我是name! 这次是用javassist了哦! "//添加代码前给name赋值
value = "我是value";//给value赋值
System.out.println("我是加进去的哦, 哈哈: " + name);//输出点恶搞信息
System.out.println(name);
System.out.println(value);
System.out.println(value);//添加到最后的代码
```

反过来, 如果没有添加这段代码, 输出的结果肯定是两个 null 值, 那么结论就很清楚了, 这个切入的动作可以让加载的类使用前就被改变。



注意 在 Instrumentation API 中,不仅可以用于做字节码增强,而且还有许多的功能。例如,可以通过调用其他的 `getObjectSize(Object o)` 方法来测量一个对象的精确大小。不过,对象中如果有引用指向其他的对象,那么这个方法返回的仅仅是计算引用的大小,进一步深入需要自己写代码来递归完成。

现在用工具来运行。这里用 Eclipse 来运行,在运行前不要直接使用 `run`,而是在“Run Configuration”→“Arguments”→“VM arguments”中增加上面提到的 `javaagent` 设置,这样就可以在 Eclipse 中运行了。设置如图 3-23 所示。

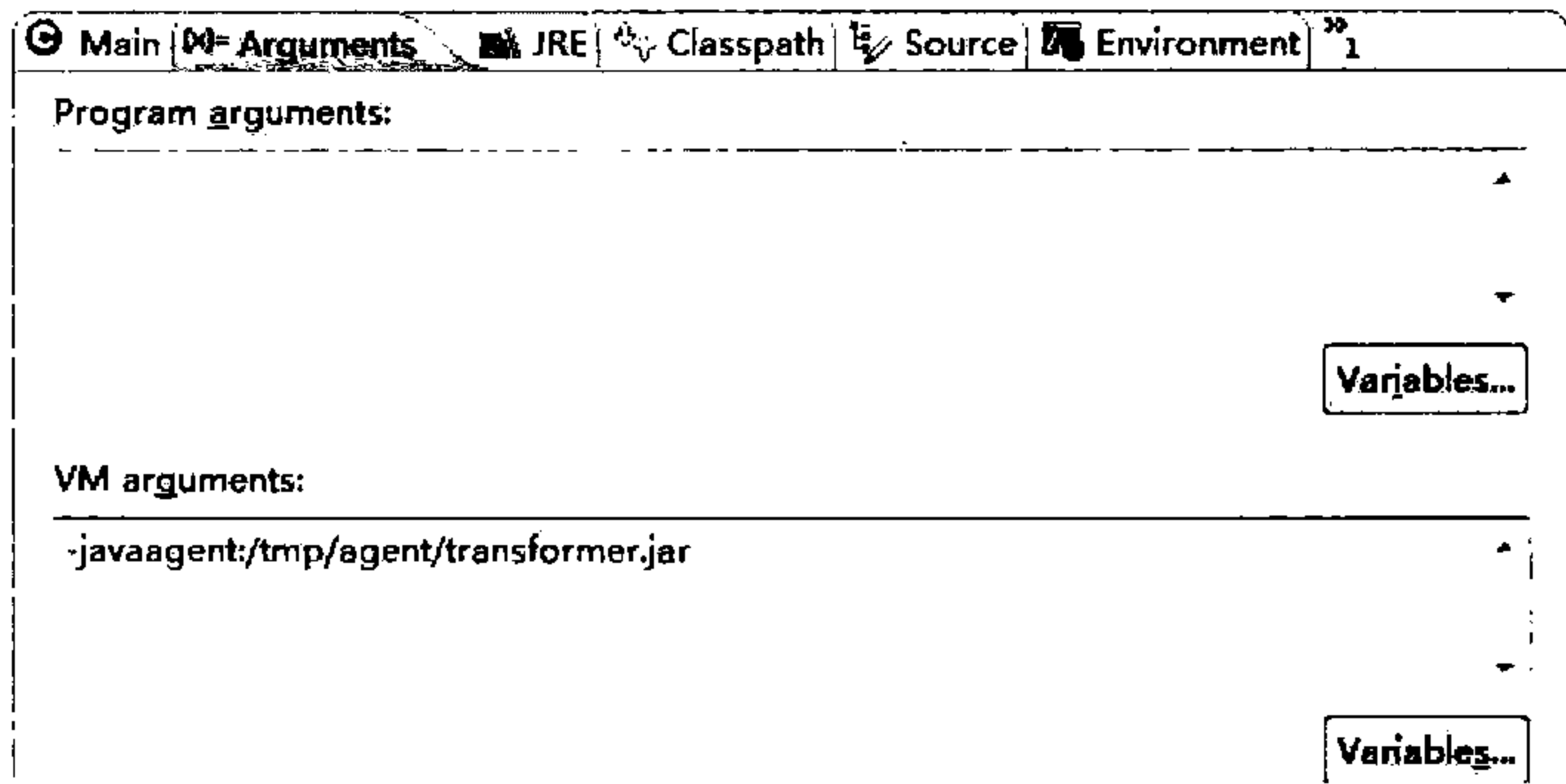


图 3-23 在 Eclipse 的“Run Configuration”中设置 javaagent

这样就完成了程序在被加载前字节码被修改。大家也许会欣喜,竟然还有这种工具;也许会郁闷,写得好好的 Java 程序竟然很容易被改掉。不论如何,这只是一些机制,我们合理地控制好一些安全措施就好,但关键是你要知道有这些内容。

在调用 `addTransformer` 的时候,还有另一种方式可以使用。

```
instP.addTransformer(new TestTransformer() , true);
```

这样设置后,可以在程序运行中调用 `retransformClasses(Class)` 方法来实现对字节码的修改,也就是运行中随时改掉一个 `Class` 的字节码,和这个类相关的实例会立即生效,这个功能与即将介绍的 `redefine` 的功能类似。

2. 运行时修改字节码信息

这部分内容由于篇幅限制,不会像上面那样进行详细说明,在本书光盘中也有代码实例。

它的操作和上一个例子类似,也是利用 `Instrumentation` 来完成的,所以都有 `Agent`。下面说下区别。

在系统加载的时候,和上面一样,都要持有 `Instrumentation` 的实例,以方便进一步操作,但是这个测试中不再有前面提到的自定义的 `Transformer`,因为这个例子不是在 `Transformer` 中去修改类,而是直接 `redefine` 这个 `Class` 的内容,所以提供一个方法来重定义这个 `Class` 就可以了。

代码清单 3-11 redefine Class 的 Agent 代码部分

```

public class InstForRedefineClass {

    private static Instrumentation inst;

    public static void premain(String agentArgs,
                               Instrumentation instP) {
        inst = instP;
    }

    public static void redefineClass(Class<?> theClass,
                                    byte[] theClassFile)
        throws ClassNotFoundException, UnmodifiableClassException {
        inst.redefineClasses(
            new ClassDefinition(theClass, theClassFile));
    }
}

```

将修改后的字节码通过 `redefineClass()` 调用传入时，它就不需要像 `ClassLoader` 那样每个 `ClassLoader` 只能加载一个同名的类，而是直接修改字节码本身 `Class` 的内容，所以是立即生效的（即使是在这个字节码被修改前创建的实例，如果再次调用对应的方法，也会运行修改后的代码）。这是与 `ClassLoader` 去加载一个类本质上的区别。

它与上面一段程序的区别是，在 Agent 的 jar 包下的 `META-INF/MANIFEST.MF` 中，除了所需要的 `Premain-Class` 外，还需要增加一行：

```
Can-Redefine-Classes: true
```

这一行就表示这个 Agent 环境是否运行重定义 `Class`。这一行添加好以后，运行方式就和上一个例子一样，通过 `-javaagent:/tmp/agent/redefineclass.jar` 作为运行时的前缀部分来运行，即可得到想要的效果。

在这里我们提到的仅仅是进程内部，而且必须在启动时增加 Agent 命令，在后文中还会提到通过进程外部链接动态加载 Agent 程序的例子。

这一节内容很多，但是是 JVM 的一段基础内容，本书光盘中有一些例子，多练习下就熟悉了。

胖哥希望大家不要成为“技术控”，也不要滥用“字节码增强”技术，它的缺陷不言而喻，在实际项目中一定要本着如何提升价值而服务来使用技术，因为技术只是一种手段。

3.3 从虚拟机的板块开始

3.3.1 Hotspot VM 板块划分

很多人看到板块划分一下子就像打了“鸡血”一样来劲！

但是一看到板块细节，就逐渐地没劲了！

为啥呢？咋一看，好爽；仔细一看，好多东西不懂。

就像给你一个世界地图，拿到手好爽，拿到手上越看内容越多，越看越烦人！

不管你是否烦，我们还是先看看 JVM 的世界，如图 3-24 所示是 JVM 内存中的板块图概要。

其实对于 Java 程序员来讲，一般主要关心的是 `Heap` 区，其他的区域当遇到的时候再去关心也不晚。本节还会简单说说永久代和栈空间。

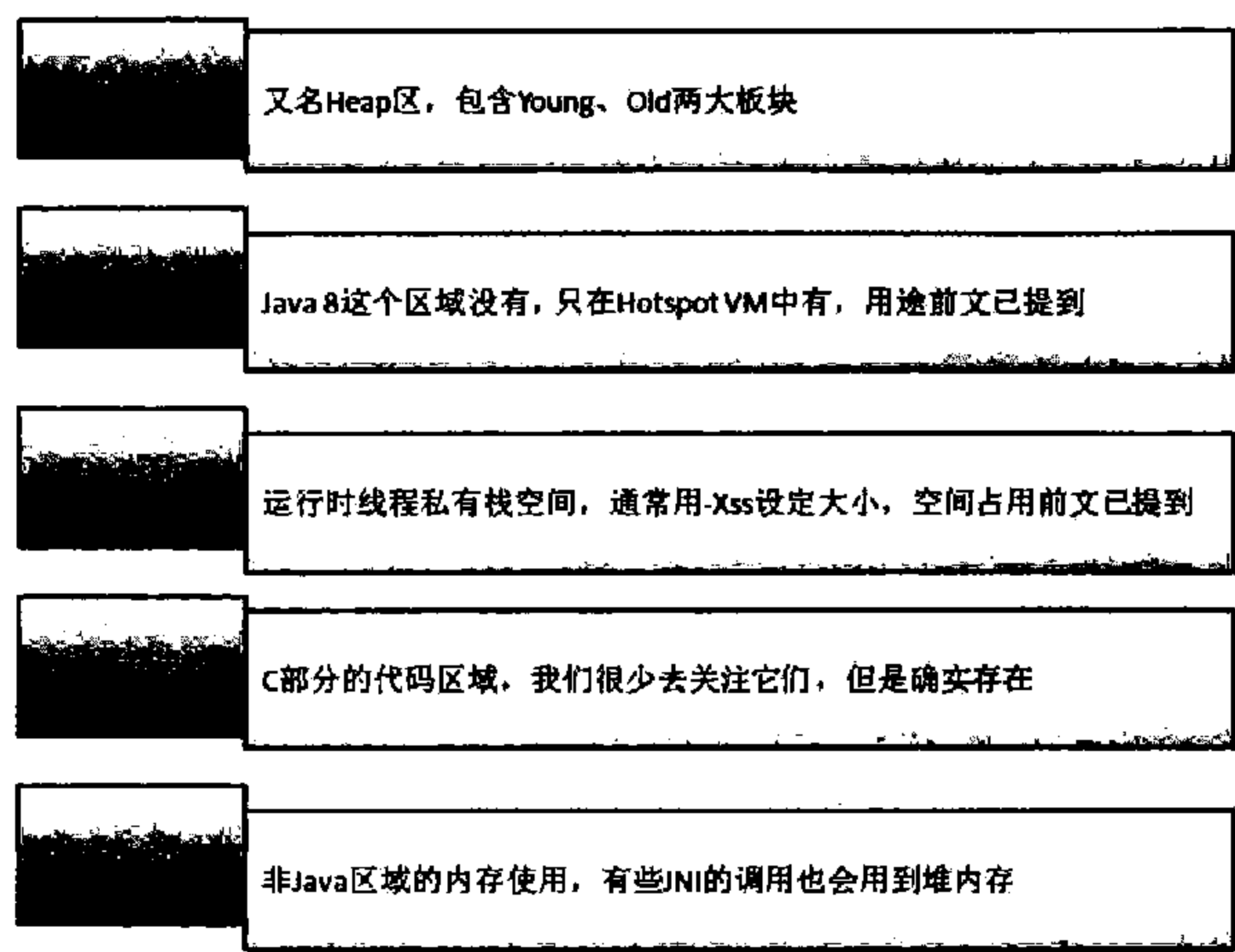


图 3-24 JVM 内存中的板块图概要（仅供参考）

Heap 就是我们经常提到的“堆”，它主要包含两大板块：Young 和 Old，两个板块在分配内存的方法上和 Heap 类似，但是回收方式区别很大。

也有人将永久代(Perm)归属到 Heap 区，只是大家的看法和侧重点有所不同而已，没有必要过分纠结，只需要知道现在的 HotSpot VM 中 Perm 区域还是单独的一块就行了。但请注意：以后 Hotspot VM 是打算去掉 Perm 这个块空间的，其他厂商的 JVM 也没有这个块空间。

通常来讲，Young 空间会小很多，具体比例是多少一般要看应用场景和配置。Young（年轻代）和 Old（老年代）的划分，如图 3-25 所示。



图 3-25 Young 和 Old 的划分

看到图 3-25，大家可能会有所误解，第一反应就是认为 Young 一定比 Old 小许多，未必然也（许多参考值 Young 大概是 Old 的 1/4~1/3，但是仅仅是常规的参考默认值）。在一些特定的应用场景下，Young 可以很大，甚至有可能大于 Old 的情况，至于实际场景分析的方法，就要结合一些内在的算法和原理了。接下来我们一起来细化这些知识吧。

有一些小伙伴开始对很多“默认值”感兴趣，其实胖哥希望大家不要去看那些默认值，也许看了会把自己“搞糊涂”。例如 Young 空间，有一个 NewSize 默认值，和 NewRatio 默认值，那么 Young 空间的默认值到底是多大呢？用这两个数字对比会发现差别是天壤之别！看默认值就是看看大概的比重和数量级，不要去强记数字，也没有什么默认值。

胖哥认为在 JVM 中，上一个版本的默认值并不一定是下一个版本的默认值，不同的 GC 方式有不同的整套默认值，操作系统不同、JVM 位数不同，默认值都会有所不同。另外，参数表是非常多的，JDK 1.6 较高的 update 版本中已经有接近 600 个参数设置项，如此多的参数设置项，就算是 JVM 的作者恐怕也未必完全记得住，只是他能很快找到原理和解释。

而我们需要知道什么呢？我们需要知道的是找到默认值的方法，掌握默认值的大概量级，在不同的版本下哪些是常用的默认参数，哪些是必须设置的参数，哪些是可以选择的、尽量不要去碰的设置。

看默认值的方法通常有两种：一种是通过 `jinfo` 来获取在 3.7.5 节中会有详细的描述；一种是通过 `-XX:+PrintFlagsFinal` 来获取结果。其实还有 `-XX:+PrintCommandLineFlags`，可以打印出真正的参数启动命令，也就是说，有些参数你没设置，但是系统启动时会设置。

`-XX:+PrintFlagsFinal` 参数是从 JDK 1.6 update 21 开始支持的。

使用 `-XX:+PrintFlagsInitial` 输出真正的默认参数列表，只不过这个默认值就像是程序中写好的默认值，并不是系统启动时设置的默认值（千万别被这句话“绕晕”，它的意思是指：在写代码的时候可能会对一个属性赋予一个默认值，当程序真正运行的时候，可能会根据不同的情况动态赋予不同的值）。

下面我们来做一个实验，看看在 64bit 模式下，内存多大的时候是自动指针压缩的，胖哥在用 JVM 相对较低的版本时发现是以 32GB 为分割点的，现在突然变了（见图 3-26）。

```
[xieyu000@java_A ~]$ java -version
java version "1.6.0_31"
Java(TM) SE Runtime Environment (build 1.6.0_31-b04)
Java HotSpot(TM) 64-Bit Server VM (build 20.6-b01, mixed mode)
[xieyu000@java_A ~]$ java -Xmx31g -XX:+PrintFlagsFinal|grep Oops
    bool UseCompressedOops                := true                {lp64_product}
[xieyu000@java_A ~]$ java -Xmx32g -XX:+PrintFlagsFinal|grep Oops
    bool UseCompressedOops                = false                {lp64_product}
```

图 3-26 一个查看默认值的例子

看到这个例子，某些同学又开始记了：JVM 的压缩指针默认是在 32GB 的位置开启的。其实没这个必要，我们只需要大概知道在这个位置就可以了。

回到刚才谈到的堆话题中，一般 Young 空间都用 `-Xmn` 来设置，虽然还有其他的设置方法，但暂时不要去想太多，想太多容易乱。

JVM 中的参数设置令人不爽的一点就是同一类型搞这么多参数，而且都有默认值，那么 JVM 真正用的是什么呢？例如 `-XX:NewSize`、`-XX:MaxNewSize`，设置了 `-Xmn` 后相当于将这两个值设置为一样大，也有用参数 `-XX:NewRatio` 来设置的。大家可不要混用哦，混用的结果很多时候难以预料，虽然在某种情况下可能得到了一个测试结果，但如果你没有真正了解 JVM 的内核源码，是不可能知道所有细节的，即测试结果不能当成任何场景下的一个结论，只能作为一种参考。

Old 区域有多大呢？它就是 Heap 空间剪掉 Young 空间后的内存区域。Heap 总大小使用 `-Xms` 来设置它的初始值（这部分不包含 PermGen 的大小），使用 `-Xmx` 来设置它的上限，在服务器端我们通常将这两个值设置为一样大，因为大小的不断调整只是为了给某些客户端程序节省开销，但是对大内存 JVM 来说就是在“瞎折腾”。

虽然是“瞎折腾”，但是为了通用 Java 还是有两个参数来控制，当 GC 后剩余空间的百分比小于 `-XX:MinHeapFreeRatio` 时，JVM 的堆内存开始变大（一般输出为 40，代表 40%）；反之，当 GC 后空闲内存大于 `-XX:MaxHeapFreeRatio` 时（一般为 70，代表 70%），JVM 的堆内存开始变小。

接下来,就来谈谈 Young 空间的管理。在专门谈 JVM 的书籍中,大家都将看到会说一大堆“历史”或“故事”,其实这种方式很好,因为要知道一件事情的真相,就一定要知道它的“前世与今生”。

胖哥在本书中就不想搞那么复杂了,先来谈谈一些常见情况,变化其实在于细节,从技术角度看应当都是“换汤不换药”。

好了,我们直接看 Young 空间的内在布局情况。

Young 空间被分为 2 个部分、3 个板块,即“1 个 Eden 区 + 2 个 Survivor 区”。

Eden 区域干什么用?使用 new 或 newInstance()等方式创建的对象,默认都是先将对象的空间放到 Eden 区域的(除非这个对象太大了 Eden 区域放不下,或者设定了一个对象阈值-XX:PretenureSizeThreshold,这样对象在分配的时候就会直接进入 Old 区域)。

2 个 Survivor 区通常简称为 S0、S1,理论上可以认为 S0 和 S1 是一样大的,就像“孪生兄弟”一样,当某些动态参数设置后,这两个区域的大小可能会发生一些改变。

再来看看这几个区域是如何配合工作的。

在不断创建对象的过程中,Eden 区域会满,满的时候就会开始做 Young GC(也称为 Minor GC),而 Young 空间的第 1 次 GC 就是找出“Eden 区域活着的对象”,将这些活着的对象向 S0 或 S1 其中一个区域存放。

假设第 1 次选择了 S0,它会逐步将活着的对象拷贝到 S0 区域,但是如果 S0 空间放满了,剩下的活着的对象就只能放在 Old 区域了。接下来的动作是直接将 Eden 区域清空,此时大家应当注意到 S1 区域是空的。

第 2 次 Eden 区域满的时候,就将“Eden 中活着的对象 + S0 中活着的对象”迁移到 S1,迁移方式也是一样的(同样的,如果 S1 放不下,对象就会放到 Old 区域),只是这一次对象来源增加了 S0 区域的对象,最后被清空的区域是 Eden 区域和 S0 区域。

第 3 次、第 4 次操作依此类推,细心的你在纸上画一画这个交换过程,就会发现 S0 和 S1 始终有一个是空的。

反反复复多次没有被淘汰的对象也会放入 Old 区域(也是由一个参数来决定的,但是这个参数的默认值并不一定是许多地方提到的 15,后文中有介绍)。

下面我们对 Young 空间的划分,以及简单的交换过程,做一个简单的示意图,如图 3-27 所示。

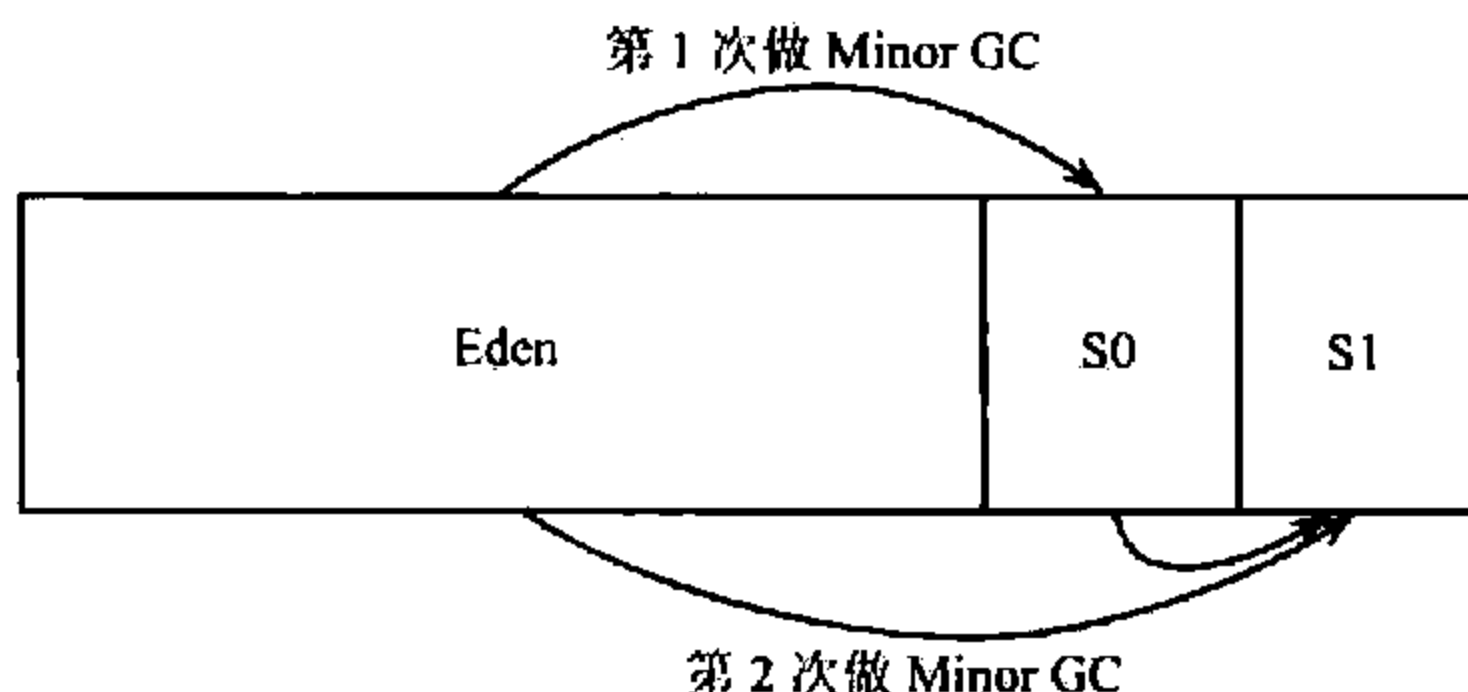


图 3-27 Yong 空间的划分以及 Minor GC 的简单示意图

疑惑: 什么是活着的对象?

答曰: 从根引用开始找对象,对象内部的属性可能也是引用,只要能级联到的都被认为是活着的对象(包括 C Heap 区域的对象空间)。

疑惑: 什么是根?

答曰：“本地变量引用”、“操作数栈引用”、“PC 寄存器”、“本地方法栈引用”、“静态引用”等这些就是根。（可见，所谓的“根”其实不止一个，或许我们把它叫作入口更合适一些）关于本地变量和操作数栈，前文中已经介绍过。PC 寄存器中可能包含了从栈顶抛出的引用，这些引用正在被使用，只是它们并不是本地变量（就像前文中提到的字符串拼接最终用于输出一样，中间结果没有本地变量，可能在栈顶被抛出，但是在生命周期内不会被 GC）。本地方法在一些内部调用中，在生命周期内依然是生效的（例如 `new Thread().start()` 这个操作，虽然 `Thread` 本身没有 Java 的引用指向它，但是只要线程没有结束，这个对象就会一直存在）。

这几部分内容通常是线程私有的，所以都把它们归结到运行时栈中。静态引用本身是具有全局生命周期的，它如果不是 `final` 的，则其引用的对象可以发生改变，但是静态属性本身是全局的生命周期，它当前所引用的对象及间接引用的对象也都被认为是活着的对象，因此它也是入口之一，如图 3-28 所示为活着的对象示意图。

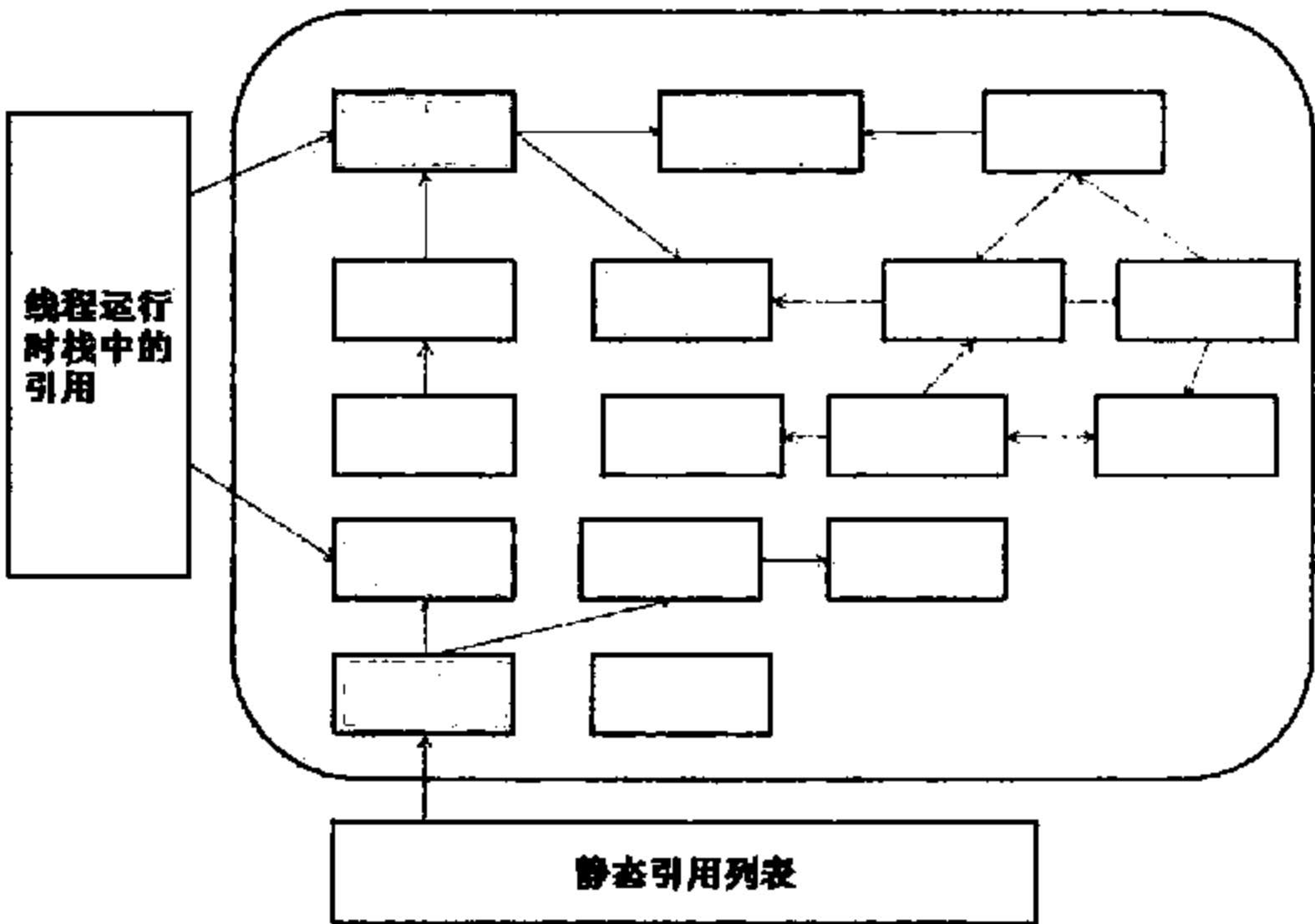


图 3-28 查找活着的对象示意图

疑惑：S0、S1 这两块区域大不大，进入 Old 区域有什么问题？

答曰：一般来讲，S0 和 S1 这两块区域是不大的，从 JVM 的发展来讲，它从古至今经历了很多种内存拷贝的方法，到了现在的“分代”（不久的将来还有 G1），分代并不是“拍脑袋”随便想出来的一种方式，它的设计是有经验积累基础的。

JVM 发现大多数的 Java 应用，如果代码没有问题，90% 以上的对象在创建后都会被马上注销掉。例如，客户端请求服务器端输出一段列表，当服务器端输出完毕后，中间的 List、Map、自定义对象都已经没有用了，包括容器、框架内在创建的对象也都没有用了，如果没有自己需要 Cache 的内容，活下来的对象就是少数的。

好了，有了这个前提和基础，我们可以得到如下一些结论。

- ◎ 即使 Young 空间很大，但是活下来的对象是少数的，那么寻找活着的对象就会很快。
- ◎ 由于活下来的对象不多，所以存放活下来的对象的空间并不需要太大，因此就产生了 Survivor 区域。也正因为如此，所以它不会太大（它到底有多大，后文中有介绍）。
- ◎ 如果程序跑得很慢，对象的存活周期就会变长，被当成垃圾的概率就会变小，当 Survivor 区域存放不下时，就会进入 Old 区域，所以有些时候不仅仅是参数的问题，程序跑得快慢也和 GC 有点关系。
- ◎ 内存大块，或者很多内存用静态区引用，就很可能一直存活，进入 Old 区域的概率

就非常大了。

疑惑: 进入 Old 区域有什么坏处呢?

答曰: Old 区域又称为老年代, 老年代自然和年轻代不同, 年轻代我们可以认为存活下来的对象很少, 而老年代的对象则很多, 认为它“足够老”, 或者在 Minor GC 时, S0 或 S1 空间不够时对象才会进去 Old 区域 (不论以何种方式进入 Old 区域, 它们得到的待遇是平等的, 有可能这个对象并不是真正的足够老才进入 Old 区域的)。

换句话说, 这块区域中的对象一般都被认为是长命的, 因为通常认为只有长命的才会进入这个“社区”, 或者说“不容易死”能更好理解些 (在实际情况中能否真正做到这点, 其实要看代码以及参数配置)。

自然的, 此时要找活着的对象就要难多了, 因为通常大部分对象是活着的, JVM 可以设置到 10GB 甚至更大, 此时如果做一次 Full GC 找活着的对象, 那么对于性能可想而知。如果系统频繁地发生 FULL GC, 而这些活着的对象在这么短的时间内被注销的概率很低, 那么 FULL GC 的时间越长, 就会有越多的请求得不到响应或响应很慢, 它们所持有的内存就无法释放掉, 就会有越多的对象进入 Old 区域, 这就会在一个“点”上产生一种连锁效应, 因此我们写代码时可以多考虑这方面的问题。

再来看另一个问题。在发生 FULL GC 时, 由于需要对 Old 区域进行清理动作, Old 区域通常会很大, 而且 JVM 会认为在 Old 区域的对象很多都应该是活着的, 它并不会像 Young 空间那样 95% 的对象会在发生 GC 时变成垃圾, 所以 JVM 在设计上不会再给 Old 区域分配一块像 Survivor 区一样的区域来存放活着的节点。那么 Old 区域的对象清除操作就不能像 Eden 区域那样。所以它需要在自己的空间内扫描遍历, 然后将对应的垃圾对象清除掉。这个时间开销自然是比较大的, 虽然可以并行, 但并行之中又带来了许多冲突的复杂性, 因此 FULL GC 一直都不是我们想看到的 GC, 我们甚至希望它永远不要发生。

FULL GC 通常会做压缩碎片操作。什么是碎片呢? 上面已经提到 Old 区域需要遍历内存, 清除对象, 由于有这样一个清除动作, 所以就会导致内存中产生许多碎片, 如果碎片很多, 自然就会导致分配空间出现一些麻烦, 因此需要整理碎片。

同样的, 如果活着的对象越多, 那么整理碎片就越麻烦, 有可能就是为了几个字节的碎片而导致几百 MB 内存对象的移动 (JVM 不会因为这里有几个字节的碎片, 而寻找一个几个字节的对象放在这个位置, 它采用的方式是顺序移动的)。为了处理这些问题, JVM 采用了一些特定的手段, 但就目前来讲, 还不够理想。

值得说明的是, 现在已经不再通过 JVM 大量缓存业务数据来实现性能提升, 一些特定的场景除外。另外, FULL GC 也不仅仅回收 Old 区域, 而是全局的, 包含 JVM 的 Perm 区域和 DirectBuffer 等 (在后文中会提到)。

疑惑: S0 和 S1 一般多大? 靠什么参数来设置? 有什么变化?

答曰: 一般来说很小, 我们只需大概知道它们与 Young 空间相比“差不多相差 1 个数量级”的比例。设置的参数一般有两个: `-XX:SurvivorRatio=8` 和 `-XX:InitialSurvivorRatio=8`。这两个参数很诡异, 很多人会把它们搞混。

第 1 个参数其实是 Eden/Survivor 的区域比重 (注意是一个 Survivor 区的大小)。如果将其设置为 8, 则说明 Eden 区是一个 Survivor 区的 8 倍, 换句话说, S0 或 S1 空间是整个 Young 空间的 1/10, 剩余的 80% 由 Eden 区来使用。

第 2 个参数是 Young/S0 的比值结果。例如, 当其设置为 8 时, 则代表 S0 或 S1 占用

Young 空间的 12.5% 的大小, Eden 区占用剩余的 75% 的大小。

这两个参数之一设定后就可以设定两个 Survivor 区域的大小了, 不过, 两个 Survivor 区还会受到 `-XX:+UseAdaptiveSizePolicy` 参数的影响, 它会动态调整这两个区域的大小。这个参数在 JDK 1.6 以后默认是开启的, 即默认就会动态调整。

这个参数开启后, 在监控工具 (如 Visual VM 监控) 中会看到每个区域都有当前值和最大值信息。通过监控工具发现 S0 和 S1 有时候可能不一样大, 有可能其中一个不见了, 有可能变得非常大, 这有点类似 `-Xms` 和 `-Xmx` 的概念。JVM 其实是在某些必要的情况下尽量存放更多的垃圾, 不让它们进入 Old 区域, 但是可能在某些情况下会好心做了坏事, 在线上系统中很多时候还是会关掉这个参数。

疑惑: 一个对象每次 Minor GC 时, 活着的对象都会在 S0 和 S1 区域转移, 若存放不下则会转移到 Old 区域, 如果一个对象经历 Minor GC 达到一定次数后也会进入 Old 区域, 那么到底需要多少次 Minor GC 后对象才会进入 Old 区域呢?

答曰: 一般用 `-XX:MaxTenuringThreshold=15` 这个参数来设置。当一个对象被第 1 次 GC 后, 计数器会在此对象头部相应位置标记 1。至于到底多少次, 其实不用太纠结, 这个参数也没有必要去修改它。

在 CMS GC 下这个默认值会变成 4, 通过 `-XX:PrintFlagsFinal` 打印出来即可看到结果, 所以不需要强记这些数字。

疑惑: 在 Minor GC 的过程中, Young 空间活着的对象如果被 Old 区域的某个对象所引用, 会不会有特殊的处理?

答曰: JVM 通常会记录下这些信息, 也就是 JVM 会做一些特殊的处理。除此之外, JVM 其实还有许多的特殊处理, 不过这些会涉及版本、厂商的各种细节, 甚至会涉及很多内核源码。这些内容在本书中就不多提了, 有兴趣的小伙伴们可以自己去深入。

3.3.2 “对象存放位置”小总结

- ◎ 当实例化一个对象时, 如果对象不是特别大或没有达到参数限制, 那么都会在 Eden 区分配空间来存放这个对象。
- ◎ 当第 1 次发生 Minor GC 时, 会在 S0 或 S1 中选择一个区域将 Eden 区中活着的对象写进去 (这里假设第一次使用的是 S0), 如果 S0 区域存放不下, 则会放入 Old 区域, 然后清空 Eden 区域。
- ◎ 当第 2 次发生 Minor GC 时, 就会将 “Eden 区中活着的对象+S0 区中活着的对象” 拷贝到 S1 空间中, 如果 S1 区域存放不下, 则会进入 Old 区域, 最后将 Eden 和 S0 两个区域清空。
- ◎ 第 3 次就将 Eden + S1 区中活着的对象拷贝到 S0, 将 Eden 和 S1 区清空, 依此类推。关于写代码和调用参数说明如下:
- ◎ 通过前面的说明, 大家要认定一点, 那就是 FULL GC 通常是我们不愿意看到的情况, 胖哥见过许多非常牛的系统, 在比较高并发的压力下, 很多天都不会做一次 FULL GC。
- ◎ 新创建的对象, 认为大多数是 “短命鬼”, 写代码的时候要如何适应 JVM 的 GC 方式? 最直接的方法就是程序跑得快, 局部内容尽量少; 反之是跑得慢, 大量使用 JVM

来缓存许多业务数据。

- ◎ 每次拷贝时, 如果 S0 或 S1 区域放不下, 就会进入 Old 区域, 那么如果写代码时有偏大的一些内存在使用, 就可能导致类似的问题。这些对象不一定要有上百 MB、几个 GB 的大小才叫大, 在多线程系统中, 每个请求所对应的线程如果多几 KB 或几 MB 的空间, 整体上很快就会多出很大的空间, 以至于这些对象很快进入 Old 区域, 并将 Old 区域填满, 从而导致 FULL GC。这些对象本身在业务层面可能只有较短的生命周期, 但有可能会由于配置不当导致大量进入 Old 区域, 在没有发生 FULL GC 前它们不会被释放掉, 因此 Survivor 区域的大小在一定程度上决定了 Old 区域增长的速度, 同时也间接决定了 FULL GC 的频率。
- ◎ Old 区域满的时候就会开始做 Full GC, 在 Minor GC 时, Survivor 区域存放不下, 对象进入 Old 区域的过程, 通常叫作“晋升”, 在晋升时, JVM 会检测到 Old 区域是否有足够的空间来存放这个对象, 如果不足, 就会先做一次 Full GC 再做晋升。
- ◎ 如果通过 intern() 调用一个 String 对象, 那么这个对象会在 Perm 区域的常量池中分配空间, 当然它会通过 equals 保证值是唯一的。但是如果调用大量不同 String 对象的 intern() 方法, 那么这些对象将会在 Perm 区域形成一份拷贝, 逐步导致 Perm 区域满, 这块区域满的时候, 也会导致 FULL GC。在发生 Full GC 的时候, 如果没有引用指向 Perm 区域中的 String 常量, 那么它同样可以被回收, 但是如果有引用指向它, 就无法被回收, 最终可能会导致 OOM。
- ◎ 如前文中提到的 Class 本身也是对象, 存放的位置也在 Perm 区域中, 如果动态加载大量的 Class, 而无法卸载的话, 那么 Perm 区域也会满, 自然也会导致 FULL GC 甚至 OOM (如果相关的 ClassLoader 下面的任意一个 Class 还有对象是活着的, 那么它就无法被卸载)。
- ◎ 在通过 I/O 输出数据的时候, 这些数据通常还会经历一个系统的内核区, JVM 内存与 OS 内核需要一次数据的拷贝过程, 但在某些情况下也可以直接通过 DirectBuffer 来完成。不过, 需要注意的是这块空间虽然没有在 JVM 的堆内存中, 但是同样是占用内存空间的, 如果这块空间满了, 它并不会直接导致 FULL GC, 但只有 FULL GC 才能释放这块空间。在这个问题上一直是矛盾的, 在后文中还会详细讲解这个问题。
- ◎ 当在程序中进行方法调用时, JVM 都会为该方法分配栈帧 (frame), 这个空间是线程的私有栈, 它会存放程序运行过程中的局部变量信息、后进先出栈等内容。在程序中进行方法调用时进行分配, 如果程序中不断进行方法的深度调用, 就会不断地消耗栈空间, 只有相应的方法完成后 (发生指令上的 return 操作), 相应的栈空间才会被释放。如果程序中出现直接或间接的“死递归”, 就会出现 StackOverflowError。

3.3.3 关于永久代

前文中提到的 Class 加载, 其实是被加载到永久代 (PermGen)。Class 也会占用空间, 而且并不小, Class 本身算是永久代的最大用户, 不过永久代是否还会存储其他的内容呢?

当我们调用 String 对象的 intern() 方法时, 它会在永久代中一块所谓的“常量池区域”中查找是否有相同值的常量 (使用 equals() 方法进行匹配), 如果没有找到, 就在永久代中生成一个具有相同值的 String 对象并返回这个 String 对象的引用; 如果找到了, 则直接返

回所找到的 String 对象的引用地址。

当程序中不断地使用不同的字符串去操作 `intern()`，且这些字符串都被直接或间接地引用时，常量池迟早会被撑爆的。

考虑：静态 (static) 数据放在哪里？

静态数据存储的位置其实是在方法区内部，也就是 Class 所存在的位置内部，在 Class 初始化时，这块区域就被分配了（其实 Class 本身也是一个对象）。

胖哥认为，大家不必十分纠结于它到底放在哪里，我们只需要知道它在 Perm 区域中且要占用一定大小的空间就可以了。值得重视的是，如果是静态引用，它的生命周期将是所在类加载到所在类卸载（包括进程退出）；如果是普通类型（例如 `int`、`long` 等基本变量类型），那么占用的是和这些类型等价宽度的空间大小；如果是引用，则存放引用的大小。不过，它所引用的对象，将具有较长的生命周期（如果是 `final` 类型，则它引用的对象将与其“同寿”；如果不是 `final` 类型，则从对象创建开始一直存活，直到这个静态引用的值发生改变，且这个对象不再有其他引用指向它为止。换句话说，一旦静态引用来引用对象，通常对象是长命的，间接地包括对象内部包装的对象也是长命的，经常做全局变动的就没有必要用它。这一点切切注意哦）。

胖哥认为，永久代的设计初衷是最初 Java 认为这块区域应该不会被回收或几乎不会去参与回收，就像我们通常认为 Class 结构一旦加载就不会被改变一样，而且这样更加适合于 JIT 动态优化（因为不变了，所以动态优化就更简单，且可以做到更细）。

但是随着 Java 功能的增强，永久代的功能并不像以前那样十分透明，例如，“字节码增强”就可以改变 Class 内部结构，使得永久代的空间可能受到程序的控制；再例如，上面提到的 String 对象的 `intern()` 方法可以拷贝 String 对象到常量池 (StringTable) 中，而常量池也存在于永久代在，它同样也使得永久代的空间存在不定性因素，就使得这块区域也不得不在某些时候去做 GC，那么它的管理就有点像 Heap 区域，这块区域就逐渐地失去单独划分的必要性，在未来的 Hotspot VM 设计中，这块区域可能就不见了。

反过来说，大家不要滥用 `intern()` 和字节码增强技术，即使要用也是在可控条件下，例如 Spring 的 AOP 在系统启动时就决定了子类是什么样子，创建可控数量的子类，而不是在业务代码中大量使用。

写代码时，如果考虑到“永久代是固定的”来设计代码，代码中自然就不会滥用上面提到的那些操作（注意：滥用和不用是两回事），相应的也就符合 Java 对永久代的一些期望了。

递推性考虑：在配置 `-XX:PermSize` 和 `-XX:MaxPermSize` 参数时，永久代够用就好，不必太大。如何确定到底多大算合适？我们可以通过运行时工具看看永久代的使用情况，若发现永久代的使用接近满的程度，就稍微调大一点点，以避免不要的问题；如果发现永久代的使用量不到 50%，那么就调小一点点，以空出更大的内存空间给其他区域使用。

板块划分就说到这里，有些 C Heap 区域的板块（主要包括：DirectBuffer、JNI、编译器、GC 自己的使用），我们不容易用到，因此很少提及，在后文中还会有部分介绍。

接下来介绍虚拟机回收算法，从配置参数上让大家了解 JVM 的一些道理。

3.4 常见的虚拟机回收算法

为了方便理解，先看一个图（见图 3-29），一个描述 JDK 1.6 所包含的 GC 方式图。

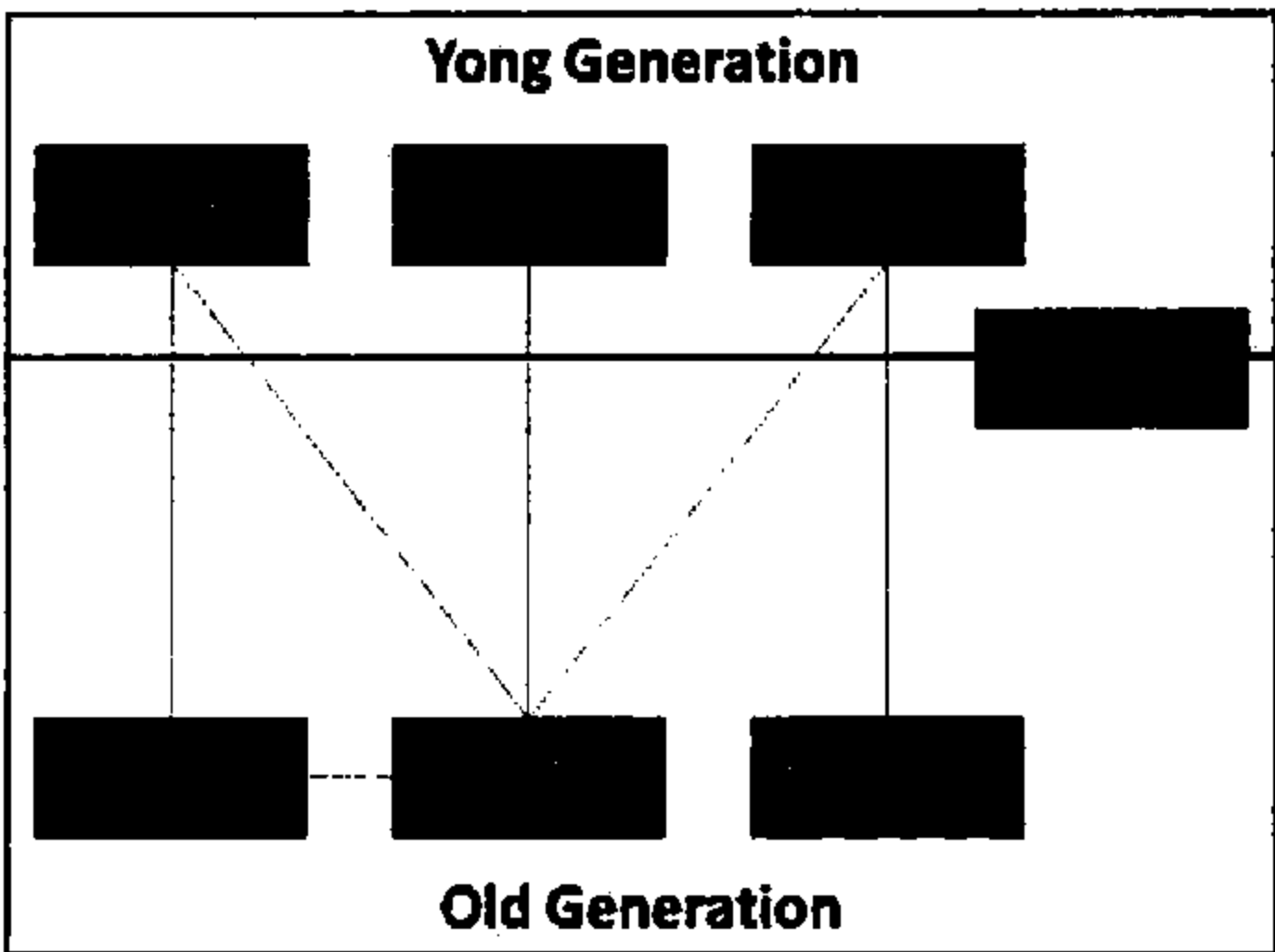


图 3-29 JDK 1.6 所包含的 GC 方式

这个图显示了 JDK 1.6 有哪些 GC 方式，上半部分为 Young 空间可以使用的 GC 方式，下半部分为 Old 空间可以使用的 GC 方式，相互之间的连线代表它们的兼容性，如果能通过一条线直接到达（间接到达不算），则说明它们在配置过程中是兼容的。

例如：如果设置了参数-XX:UseParallelGC,此时就会对 Young 空间启用 Parallel Scavenge 方式，对 Old 空间启用 MSC 方式；如果设置了参数-XX:UseParallelOldGC, Young 空间将保持不变，Old 空间将采用 Parallel Old 方式。这些知识我们在下面的内容中逐步介绍。

要介绍这些 GC 方式，还是先从最原始的 GC 开始讲起吧！

串行 GC 就是最原始的，并行、并发都是在实际场景中由驱动衍生出来的。本节将逐步讲解多种 GC 方式，以及与之配合的一些参数项。

➤➤3.4.1 串行 GC

串行 GC 就是在 GC 时由单个线程来完成，一般在“Client 模式”下这是默认的，或者自己运行时增加：-XX:+UseSerialGC，也就会采用串行 GC 了。

Client 模式是 32 位操作系统的默认值,甚至有些 JVM 中只有 Client 或只有 Server,具体的需要看 JDK 安装目录下的子目录 jre/bin/下面是否存在一个 server 目录和 client 目录,若只存在一个目录,就说明仅仅支持其中一种;如果两个目录都存在,那么在启动时使用 java -client 或 java -server 进行选择。

一般来讲，串行 GC 在单机程序或客户代码中使用，对于小内存程序（官方推荐是小于 100MB），且 CPU 个数很有限，用串行 GC 其实是明智的选择。为何？因为串行 GC 没有太多复杂的算法逻辑结构，它就是一个线程做 GC，不用考虑任何征用相关的问题，排除很多“锁”本身带来的开销，同时，算法也变得更加简单。反过来说，小内存也没有必要用并行 GC，并行 GC 中间产生的资源开销可能会更大，效率更低。另外，串行 GC 出现在较早的年代，那个时候 Java 面对的内存可能还不算大（其实现在 Java 也不容易解决大内存问题，只是这个大小是相对的概念，现在所讲的大内存通常最少是几十 GB 甚至几百 GB）。

上面提到的 Old 区域，在串行 GC 模式下也叫作 Tenured，因为串行 GC 对 Old 区域回收的算法名就叫 Tenured。

对于服务器端程序，尤其是并发量较大或处理大数据的服务器端程序，通常会用较大

的内存来承载用户的访问。面对较大的内存，如果仅仅针对 Young 空间的回收也许用串行 GC 就可以满足需求（因为 Young 空间活着的都是少数），但如果发生 FULL GC，那么串行回收的效率是远远不够的。因此希望将回收操作并行、并发起来，面对大内存这样操作，对于多线程的分配开销通常是值得的。

另外，线上服务器通常有很多很好的 CPU，如果采用并行、并发来操作，那么 CPU 就可以充分地利用起来，于是，JVM 经理开始了并行 GC 的研发。它是怎么回事呢？请看下一节。

3.4.2 ParallelGC 与 ParallelOldGC

如果系统启动默认设置了 `-server` 参数，那么就默认使用 `-XX:+UseParallelGC` 参数，在 64 位模式下的 JVM 和 OS 中，默认也会使用 `-server` 参数。还有一个 `-XX:+UseParallelOldGC` 参数（在 JDK 1.6 后出现的参数），都是并行 GC，到底哪里有什么区别呢？

它与串行搜集器（Collector）其实是类似的，最重要的区别在于它是用多线程来处理的。接下来大家可以在启动 Java 进程时，通过增加参数 `-XX:+PrintGCDetails` 来看看它们对 Young 空间、Old 空间、Perm 空间分别采用了什么方法来做 GC（注意：对于 `-XX:+PrintGCDetails`，已经输出了很完整的 GC 信息，没必要带上 `-verbose:gc`；如果对输出时间有日期格式要求，则可以使用 `-XX:+PrintGCDateStamps` 来输出日期格式）。例如，使用 `-XX:+UseParallelGC` 参数，当看到 FULL GC 时你会看到一条类似的输出：

```
[Full GC [PSYoungGen: 3456K->3456K(6848K)] [PSOldGen: 20479K->20479K(20480K)]
23935K->23935K(27328K) [PSPermGen: 2949K->2949K(21248K)], 0.0541652 secs] [Times:
user=0.05 sys=0.00, real=0.05 secs]
```

其中 PSYoungGen 用于回收 Young 空间，PSOldGen 用于回收 Old 空间，PSPermGen 用于回收 Perm 空间。其实 PSYoungGen、PSOldGen、PSPermGen 是 3 种不同的算法，为了进一步来说明这个问题，我们再用 `-XX:+UseParallelOldGC` 来看看结果（注意：使用该参数后不需要再设置 `-XX:+UseParallelGC`）：

```
[Full GC [PSYoungGen: 3456K->1909K(6848K)] [ParOldGen: 20479K->20479K(20480K)]
23935K->22388K(27328K) [PSPermGen: 2955K->2955K(21248K)], 0.0929884 secs] [Times:
user=0.16 sys=0.00, real=0.09 secs]
```

可以看到，它和 ParallelGC 唯一的区别是在回收 Old 区域上面，设置上述参数后，在回收 Old 区域时使用了 ParOldGen，不再是 PSOldGen 算法，那么它们有何区别与联系呢？这些 PS 开头的算法又到底是什么意思呢？为了说明这个问题，我们先看 Oracle 官方提供的一段说明（参考 <http://www.oracle.com/technetwork/java/javase/gc-tuning-6-140523.html> 文中介绍 Parallel Collector 部分的内容）：

The parallel collector is enabled with the command line option `-XX:+UseParallelGC`. By default, only minor collections are executed in parallel; major collections are performed with a single thread. However, parallel compaction can be enabled with the option `-XX:+UseParallelOldGC`.

PSYoungGen 是针对 Young 空间来讲的，也就是“并行清除”的意思（Parallel Scavenge），它讲究“吞吐量”，尽量增加平均每段时间内对外提供服务的时间。但是并不会像并发 GC（CMS）那样考虑用户访问的尽量不暂停，也许它会暂停一会儿，但是它对于整体时间来讲，服务时间更多。

在设置了-XX:+UseParallelGC 参数的情况下, 可以通过-XX:MaxGCPauseMillis 参数来设置最多的暂停时间, 也可以通过-XX:GCTimeRatio 参数来设置 GC 时间的比例。

大家不要将-XX:MaxGCPauseMillis 设置得太小, 这个时间是牺牲性能的。例如, 一个水缸可以装 100 桶水, 通过许多渠道向里面放水, 等水缸满的时候, 要做点事情:

- (1) 取出 1 桶水后干点别的事情, 1 分钟后回来再做 1 次。
 - (2) 将水缸中 100 桶水清空后再去做别的事情。
 - (3) 每次取 5~10 桶水, 每次取完可以持续更久但又不用等待太久。
- 就是这样的道理, 至于如何选择, 那就是自己在实际场景中的选择了。

不过, 根据上面的官方介绍, 启用-XX:+UseParallelGC 参数仅仅针对 Minor GC, Major GC 依然使用一个线程去做 (虽然它显示为 PSOldGen, 但这可能是程序的 Bug), 它对 Old 区域的回收采用了 MSC (Mark Sweep Compact) 方式, 它会对活着的对象标记完成后, 对没有活着的对象做清除操作 (包括 Old 区域、Perm 区域、C Heap 区域中的内容), 然后进行压缩。

当启用-XX:+UseParallelOldGC 参数后, 对 Old 区域采用 ParOldGen 算法, 它将对 Old 区域采用 Parallel Compacting (并行压缩) (这个过程开销是巨大的), 而且是部分压缩。什么意思呢?

JVM 发现一种规律: 按照前面介绍的年轻代的概念, 生存越久的对象才会进入 Old 区域, 但是 Old 区域的对象也未必代表“真的够长命”。Old 区域每次压缩时通常是将相邻的对象合并在一起, 那么经过多次压缩后, 真的长命的对象通常会被排布到 Old 区域的“底部”, 而其余部分在 FULL GC 后通常是比较稀疏的, 比较稀疏的地方不太影响空间的分配, 因此就有了这样一个部分压缩的概念。

胖哥认为, JVM 想要解决的问题是: 某些对象是否真的够长命?

在前面提到了对象进入 Old 区域的条件, 有些是由于 S0 或 S1 装不下才会进入 Old 区域, 有些是内存占用空间达到一个值时会进入 Old 区域, 也就是说, 它们未必真的是具有生命力。在这个时候, 对 Old 区域也可以简单做一些划分。

此时, 我们认为往 Old 区域放对象就像是往一个筐子里面放物品一样, 较早进入 Old 区域的对象, 简单地认为在筐子的下面, 而近期晋升进去的对象, 在筐子的上面。部分压缩, 就可以认为对筐子“底部”的空间很少去关心, 因为在通常情况下, 认为进入 Old 区域后, “最近”放进去的对象也是最容易被 GC 掉的 (也有了点分代的概念, 但是并不是所有的程序都是这样的, 只是一种规律性)。

回到上面看到的 GC 输出, 以及 Oracle 官方文档。

当我们设置-XX:+UseParallelOld 参数时, 对于 Young 空间也已经使用了 PSYoungGen 算法, 也就是在做 Minor GC 时进行了并行搜集, 在做 Major GC 时对 Old 区域进行了 Parallel Compacting。因此, 在设置了-XX:UseParallelOldGC 参数后, 就无须再设置-XX:UseParallelGC 参数了。

下面补充一个很多大牛们提到的 JVM 并行 GC 的一个小“悲观策略”。

它是怎么悲观的呢? 就拿我们提到的“筐子”来讲吧, 每次放进筐子里面的东西“有大也有小”, 但在并行 GC 中, 如果有对象晋升到 Old 空间, 则会记录每次晋升对象的大小, 也会在晋升后计算 Old 区域剩余的空间大小, 如果这次 Minor GC 顺利完成后, 即使 Old

区域还可能会剩余空间,但是如果 JVM 发现 Old 区域剩余的空间已经不能承载“每次晋升对象的平均大小”时,它会再做一次 FULL GC 尝试释放内存,换句话说,就是这个时候根本还没有满,但是做了 FULL GC。

把这个概念放大一些,如果每次晋升的内容都是几百 MB 的大小,也许 Old 区域还剩下几百 MB 就开始做 FULL GC 了,它提前做,因为它认为下一次晋升时“多半”要做 FULL GC,这是一种“悲观”的做法,所以大家叫它“悲观策略”。

在本书后续章节中,会多次引用到“悲观”、“乐观”的概念,因为在服务器端程序中,这两个概念的细化是很重要的。

下面有一些基本的选择经验可以借鉴。

如果 JVM 内存在 32 位模式下,通常 Heap 区域配置到 1.5GB 左右,此时 `-XX:+UseParallelOldGC` 是一个很好的选择,同时可以自己设定并行的线程数 `-XX:ParallelGCThreads` (在默认情况下,与处理器个数一致: On a machine with N processors the parallel collector uses N garbage collector threads; however, this number can be adjusted with a command line option)。

由于 GC 过程大多为 CPU 操作,所以线程数不宜超过 CPU 个数(第 2 章有详细说明)。另外,过多的线程会使得 GC 过程的锁征用变得更大,即使 CPU 足够多,也未必要设置那么多的线程。

在测试中,大家可能会遇到一些怪异的情况,例如你可能不论用 SerialGC、ParallelGC 还是 ParallelOldGC 效率都会差不多,而且也会拿出测试用例来说明自己的观点,其实中间有很多测试数据的不定性因素,并不能完全说明哪种 GC 完全不好。

大家习惯用大量存活的对象来测试,此时根据我们对 JVM 的理解,对于大量存活的节点来讲,不论是标记、回收还是压缩等动作,不论是什么算法都会存在许多问题,尤其是在 GC 后,对象之间的空隙很小时(其实存活的节点越多,对象的空隙自然越小),要移动这些对象的难度自然很高,这对于写代码的你来讲,自己想一想就应该能明白胖哥所讲的意思了。

本书中提到的 JVM 知识只是冰山小角,许多细节的情况需要大家在实际工作中去挖掘和总结,在方法上也要理论与实际相互结合。例如上面提到的悲观情况,如果在可控的情况下,可以尝试调整 Survivor 区域的大小,使其可以放得下相对较多的对象,这样尽量避免一些对象直接蹦到了 Old 区域。

总结: 通过本节了解到,在并行 GC 下最少有两种情况会导致 Full GC。

◎ Old 区域满的时候(确切地说,是要晋升的对象大于 Old 区域剩余的空间)。

◎ Old 区域的剩余空间已经小于平均晋升空间的大小的时候。

另外,还有一些情况可能会导致 FULL GC。

◎ Perm 区域满的时候也会导致 FULL GC (后文中会有例子)。

◎ 系统中使用 `System.gc()` 的时候也会默认导致 FULL GC,这样的 FULL GC 在日志中可以清晰地看到关键字“(system)”。设置 `-XX:+DisableExplicitGC` 参数后,代码中的 `System.gc()` 就不再生效了(这些代码可能不仅仅存在于业务代码中,还存在于远程方法调用的第三方包中,例如 EJB)。但是关闭“显式 GC”后有些时候又会有新的问题,例如 `DirectBuffer` 满的时候(后文详细介绍)。

◎ dump 内存的时候,例如:通过 jmap 等工具来 dump 内存的时候,也同样会先发生

一次 FULL GC 再开始 dump。

3.4.3 CMS GC 与未来的 G1

CMS GC 又称为并发 GC, CMS 的英文全称为: Concurrent Mark Sweep, 是并发标记、清除的意思。它希望在用户使用过程中暂停尽量较小, 但是绝非没有暂停。

对于 1GB、1.5GB、2GB 的 JVM 内存, 如果代码没有问题, 那么 GC 的开销时间并不多 (当然不会和 C 语言比较微秒、纳秒的效率); 如果采用了并行搜集, 那么每次 GC 基本在毫秒级可以处理完, 这对于传统的 Web 应用来讲基本足够, 因为用户感知是在秒级别的, 除非遇到存在大量活着的而不该活着的对象情况, 这种情况不论用哪种 GC 都很慢。

当有一天遇到内存达到 10GB、100GB 的时候, 就遇到了麻烦, GC 的暂停时间将会十分的恐怖, 可能会是 10s、20s、100s 以上等。那么 Java 就与大内存无缘了吗?

我们有一种“土办法”, 但是很管用, 就是将一个大内存的机器划分为多个虚拟机或者启动多个 JVM 实例来实现大内存的使用, 两种方法都可以达到目的, 实现了成本节约, 而且效果不错。

多实例方式的效率通常高于虚拟机, 但是多实例部署时会存在大量端口的修改, 比较麻烦, 而资源隔离性, 虚拟机会更好一些, 所以在应用部署上通常选择虚拟机划分方式。

但这种“土办法”对于一些单个实例需要大内存计算的应用背景不适合, 土办法变得真的有点土了。CMS 的存在就是为大内存做了一个伟大的铺垫, CMS 是一种并发 GC, 适合管理更大的内存。它是如何做到的呢? 要了解这一点, 就要先来了解一下 CMS GC 操作的步骤, 下面分别讲解 CMS GC 七个步骤的基本动作。

① CMS-initial-mark, 单线程处理环节, 并且会完全暂停 JVM, 找到所有的 root 根, 将 root 根所引用的对象标记在一个 BitMap 中。此时业务恢复正常运行, GC 进入下一个步骤。

② CMS-concurrent-mark, 根据第 1 个步骤对记录在 BitMap 中的信息进行并发标记对象。此时 JVM 内所运行的程序不会暂停。

③ CMS-concurrent-preclean, 处理 Young 与 Old 之间的引用, 尽量将可能的引用梳理出来, 节约 remark 的时间。

④ CMS-concurrent-abortable-preclean。

⑤ CMS-remark, JVM 全暂停的第 2 个阶段, 由于前面几个步骤中存在业务的并发, 所以会导致一些“脏”数据。remark 是来解决这个问题的, 因此这个阶段也会全部暂停, 但是它不是再用 root 开始遍历, 而是根据一个“卡表”, 换句话说, 卡表中有我们想要的“脏”块信息。

⑥ CMS-concurrent-sweep, 不停机, 单线程操作, 按照内存的地址顺序遍历 Old、Perm 区域, 根据 BitMap 中的记录, 对垃圾内存进行清除操作。

⑦ CMS-concurrent-reset。

既然中间是并发的, 那么在标记过程中用户也是可以操作的, 因此内存结构很可能会发生改变, 如果这些 GC Root 引用断开了怎么办? 有了新的引用怎么办?

因为在程序运行过程中, 很多局部变量脱离了作用域或被重新赋值, 此时 CMS-remark

的作用就是进行这部分数据的回补，将中间产生变化的内容进行一次重新标记，然后开始进一步做并发的清除和 reset 工作。

从上面介绍来看，CMS GC 貌似一种不错的 GC。其实，这种 GC 算法看似简单，其实设计起来十分复杂（大家可以想一想自己写代码来实现类似的算法会有多少细节需要考虑），Hotspot VM 是花了数年时间才成就了 CMS，又通过了好几年的时间才发布正式版，并且到目前为止依然存在很多的缺陷。

使用 CMS GC，一般是使用 `-XX:+UseConcMarkSweepGC` 来启用 CMS。启用该参数后，它将默认对 Young 空间启用算法：ParNew，相当于设置了 `-XX:+UseParNewGC` 参数，它们两个是一套组合拳（也就是 `-XX:+UseParNewGC` 参数不用设置）。

另外，还有几个和 CMS 相关的参数大家需要知道，下面我们来聊聊。

首先来介绍两个十分重要的参数：`-XX:+UseCMSInitiatingOccupancyOnly` 和 `-XX:CMSInitiatingOccupancyFraction=70`。第 1 个参数是告诉虚拟机仅仅使用设置的比例，而不要去做自动调整；第 2 个参数是指当 Old 区域的空间上升到 70% 的时候开始启用 CMS GC，默认值是 92%（The default value of this initiating occupancy threshold is approximately 92%, but the value is subject to change from release to release）。很多时候真正到 92% 的时候，由于一边做 GC 程序一边在运行业务，那么晋升到 Old 对象会由于太快或过大导致 Old 区域没有足够的空间来存放它们，此时可能会出现 promotion failure、current mode failure 的现象。

CMS GC 为了减少暂停时间是不做压缩的，但是它同时带来的另一个问题是，在不断做 CMS GC 的过程中肯定会产生“内存碎片”。此时使用 `-XX:CMSFullGCsBeforeCompaction` 来设定一个数字（默认值为 0），它代表多少次不做压缩 FULL GC 后做一次压缩（这里的 FULL GC 并不是全暂停的 MSC，其功能类似于 CMS 的功能，但是内部实现是另一套分支结构，当发生压缩 FULL GC 时就是全暂停的），启用 FULL GC 配合参数 `-XX+UseCMSCompactAtFullCollection` 来对碎片做整理。

在 CMS GC 中如果要自己设置线程数就不再设置 `-XX:ParallelGCThreads` 参数了，而是设置 `-XX:ParallelCMSThreads` 参数。通常来讲，也不会去设置这个参数，默认是与 CPU 个数等值，无须设置它。

在 CMS 中回收 Perm 区域的 Class，设置 `-XX:+CMSClassUnloadingEnabled` 和 `-XX:+CMSPermGenSweepingEnabled` 参数。在 JDK 1.6 以后设置 `-XX:+CMSClassUnloadingEnabled` 参数即可。通过参数 `-XX:CMSInitiatingPermOccupancyFraction` 可控制 Perm 区域的百分比达到多少的时候开始启用 CMS 对它回收，对于 Perm 区域依然是设置够用就好，但是这里的“够用”是指在参数 `CMSInitiatingPermOccupancyFraction` 所设置的比例以下的大小（否则 CMS GC 会被它不断触发），具体多少需要根据自己的应用场景来决定。

`-XX:+CMSScavengeBeforeRemark` 参数开启在 CMS GC 发生 remark 之前会先做一次 Minor GC，也就是先对 Young 空间进行回收，这个时间也是暂停的，但是它可以减少 remark 带来的暂停时间（因为 remark 标记的是最近修改的内容，最近变化的内容往往在 Young 空间内部）。

当系统中有大于 98% 的时间在做 GC，并且只有小于 2% 的内存恢复时，此时会抛出 OOM 错误，通常是内存空间不够用。关于时间上的限制，也可以启动 `-XX:-UseGCOverheadLimit` 参数。

还有一些增量 GC 参数。增量 GC 是什么呢？我们以前文中提到的水缸提水的问题为

例, 水缸可以装 100 桶水, 极端一点假设你有 100 个桶, 而且用 100 个人来分别使用这 100 个桶, 胖哥认为可以一次将水缸里的水取走。但如果你只有 10 个桶, 2 个人, 一次性做完或许要很久, 但是做的过程中不允许注入水, 就可能会导致很长时间无法注水。或许我们可以先做一部分, 例如先取 10 桶水后, 水缸就有 10 桶水的空闲空间, 水就可以继续注入, 这 10 桶水的剩余空间可以使用一段时间, 那么就可以一会再来做取水 (“桶”的概念与人的个数, 我们简单理解就是 CPU 个数)。这样的做法, 每次取水的多少以及注入水的速度两个因素决定了取水的频率, 如果频率太高, 还不如一次多取一些水, 所以好坏是自己决定的。启用增量 GC 的参数在 JDK 1.6 以后只需要使用 `-XX:+CMSIncrementalMode` 即可, 在 JDK 1.5 中还需要增加参数:

```
-XX:+CMSIncrementalPacing
-XX:+CMSIncrementalDutyCycleMin=0
-XX:+CMSIncrementalDuty=10
```

这几个参数配合使用, 前面给出的官方文档中有详细介绍。

在 CMS GC 中, 如果不想禁用 `System.gc()`, 而是想当发生系统显式 GC 调用时启用并发 GC (默认是启用 FULL GC), 就使用 `-XX:+ExplicitGCInvokesConcurrent` 参数。启动该参数后, 显式 GC 将启用 CMS GC 的方式完成, 那么在这种情况下显式 GC 还希望启用回收 Class 对象的功能该如何操作呢? `-XX:ExplicitGCInvokesConcurrentAndUnloadsClasses` 参数启用后将达到这样的目的。

一旦 CMS GC 失败或者 promotion fail 导致 FULL GC, 就会转入全暂停的 FULL GC。CMS GC 只会与 Serial GC 兼容, 也就是说, 当 CMS GC 失败时就会转入串行 GC (MSC), 而 CMS GC 通常是配合大内存使用的, 一个大内存转入串行 GC, 这是我们很不愿意看到的事情!

因此, 对我们的系统设计提出了考验, 这些 GC 的“坑”无处不在, 程序员的技术能力在其中发挥很大的作用。从 JVM 的角度来讲, CMS GC 也并非我们心目中的神, 即使是神, 我们也期望有神中之神存在, 或许 G1 GC 可以成为这个角色哦。

G1 (Garbage-first) GC 可能是目前统治大内存的神 (在 IBM 的 J9 上已经实现), 在 Hotspot VM 上只算是一个“雏形”, 或者说是一个试验品, 喜欢吃螃蟹的小伙伴们就来尝尝吧。早在 JDK 6u14 就开始有 G1 了, 但是一直效果达不到理想的情况。

Java 堆将被划分为多个相对较小且大小相同的内存板块 (有点类似于我们手工划分为多个虚拟机或多个进程, 但是在同一个进程中, 板块之间是存在许多关系的, 所以要复杂许多), 这些板块我们叫作 region。它也有分代, 在 region 里面分代, 在运行过程中会维护一个优先级列表, 这个优先级列表其实是按照垃圾多少来排序的, 这样垃圾最多的 region 就是最前面的一个板块, 每次就回收这个垃圾最多的板块。

有了这个基本思想, 下面从理论上来分析几个优势 (不一定是 G1 要实现的目标哦)。

- ◎ 内存被划分为多个 region 后, 每个 region 的管理就变成了一个小内存, 至少是相对较小的内存, 不论是查找还是遍历, 小内存都会更快一些。
- ◎ 垃圾多的始终会被优先处理, 如果垃圾多, 那么活着的就少, 由于活着的少, 所以找活着的就快, 这个暂停时间也将是最短的。
- ◎ 理论上还可以做到: 只有访问这个板块的线程将被暂停。
- ◎ 有些初始化加载的对象, 可能在某个或某些集中的 region 中, 这些对象几乎不会成为垃圾多的 region, 那么也几乎不会排序在前, GC 也永远不会去扫描这些可能不

需要回收的“老不死”的 region。

- ◎ 理论上, 如果这个 region 一直不死, 那么有些其他区域中很难死掉的对象也可以放进来, 就像有多代 GC 一样, 这样很多“老不死”的就在一起了。

这里谈到的优势, 目前还只是胖哥自己在自娱自乐, 到了真正的 G1 成熟的那一刻, 我们再来讨论它的细节吧。

另外, 即使 G1 实现很牛的 GC 算法, 也无法解决代码上的问题。换句话说, 如果写代码时就有大量的数据放在内存里面, 那么不论是多么牛的 GC、多么牛的服务器也同样干不掉, 同样会宕机。

文中虽然提到了许多参数设置, 但是无法列举所有的参数, 以及它们的应用场景。除了通过前文中提到的方法来查看这些参数外, 也需要大家多去参考官方文档。例如, 现代 VM 常见的参数可以参考以下地址的内容: <http://www.oracle.com/technetwork/java/javase/tech/vmoptions-jsp-140102.html>。

➤➤ 3.4.4 简单总结

本节首先提到了串行 GC 用在 CPU 数量少、内存小的应用中, 例如本地开发、手机客户端, 串行 GC 是不错的选择, 在 Client 模式下是模式启用的。

在普通的服务器应用中使用 1~2GB 的 JVM 节点较为常见, 推荐启用 -XX:+UseParallelOldGC 参数, 设置它过后就不需要再设置 -XX:+UseParallelGC 参数了。

最后, 谈到了 CMS GC 是大内存时代的一个铺垫, 以及胖哥对于未来的 G1 GC 的一个自娱自乐的幻想。

通过上面介绍我们可以发现, 不论是何种 GC 方式, 都希望在业务代码中尽量多地使用“小而美”的内存。

- ◎ “小”, 是指程序中所申请的对象不是那么大, 没有特殊的用途, 不去申请一个特别大的内存空间。例如一次性分配 10MB 或更大的空间, 即使它可能马上变成垃圾, 这些空间也很可能会在变成垃圾以前最先进入 Old 区域, 甚至直接进入 Old 区域。
- ◎ “美”, 我们希望它像昙花一样美丽, 它不在乎永久, 只在乎曾经拥有, 它给这个世界带来了美, 干净利落的编码, 就是美的象征。当我们的代码跑得“风快”时, 持有的对象越早变成垃圾, 它其实就越加珍贵、越加美。

如果在写 Java 代码时, 注重很多“小而美”的思路, 胖哥相信代码很多时候是没有问题的。

- ◎ “小而美”, 对这个词汇的误解: 它并不是说所有的代码都要将不用的对象设置为 null 等才叫小, 那样的代码其实很不整洁, 只有在必要的情况下才会偶尔那样去做。另外, 某些 Cache 也是必然要做的, 例如需要反复读取、使用的信息, 这些信息的创建是需要付出巨大代价的 (例如创建数据库连接, 就没有必要为了保证“小而美”每次去创建一个连接, 因为在连接池的管理策略下, 一是可以节约创建的开销, 二是可以统一控制访问并行度)。

举一个不是小而美的例子, 大家也许就会有点感性认识了, 不要认为什么东西用起来简单就随意滥用它。胖哥见过太多人在业务设计中为了保存用户操作信息, 将大量的操作数据放入 Session 中, 但是又无法得知什么时候该从 Session 中注销。其实这就是一种让数据变得“大而丑”的过程。这种现象在 Java 中也是一种“内存泄漏”, 不过它并非 C 语言

中的内存泄漏，只是隐藏着一个没有断开的引用而已，在这种情况下不论用什么配置，也终有一天会遇到宕机的问题（除非这个应用系统的访问量一直很小，而且这也只能证明这个应用访问真的很小，所以就不具备探讨性）。面对这样的问题，可能你会想出一些类似集群的方案来缓解，或者使用定时重启等土办法来治标不治本地解决问题。不过，使用这些土办法来应对也能达到一定的效果，但是没有解决根本问题。如果员工这样干了，不懂技术的老板可能还会很开心，但是如果此员工见势态发展已经不可控制而“跑路”了，那么这个项目就可能面临灾难。

其实“小而美”带来的效果也不仅仅体现在内存上，它也体现在很多方法设计上，在面向对象的设计中，也会注重小而美的思想，让代码变得十分简单，层次清晰。有人会说这样的代码方法分派太多，性能很低，其实真正小而美的代码，JIT 才会去做动态 inline，大方法 JIT 很多时候是会放弃优化的。所以不要在乎这些细节，代码是程序员的一种艺术，内存的使用也是程序员的一种艺术，编程给程序员带来的是一种职业艺术享受。

▶▶ 3.4.5 小小补充

曾经有小伙伴问胖哥，JVM 到底在 GC 的时候是如何暂停所有的线程执行的呢？

胖哥认为这是 JVM 内在的一种机制，我们自己在写代码的时候可以想出许多方法来暂停线程的执行（例如 sleep、wait 等方式），而 JVM 更加偏重底层，它要做到这一点会更加容易，更何况我们所有的程序都运行于 JVM 的管制之下。为了满足大家的好奇，胖哥在这里做一个简单解释。

要实现完全暂停（Stop The World），就是要将 JVM 中的线程都停止下来。要了解这个问题，我们需要先了解“安全点”（Safepoint）的概念。JVM 编译为本地代码后（汇编指令），会生成相应的 OopMap，这个对象可以帮助 JVM 快速完成 GC Root 的枚举，但 JVM 并不会为每条汇编指令都生成 OopMap，因为那样需要大量的空间，并且也没有必要，它只会在特定的引用变化位置进行记录，在这个位置也会插入指定的汇编指令。这些位置我们就叫它“安全点”。换句话说，指令只要运行到安全点位置，就是可以被暂停的。

曾经有人希望在发生 GC 时直接强制先暂停所有的线程，如果发现某些线程还没有运行到安全点再让它运行到安全点，这种方式又叫抢先式中断（Preemptive Suspension），不过它的难度很高，写程序的你应该明白。所以现在大多采用一种简单的方式，就是让每个线程主动到安全点位置停止，因此把它叫主动式中断（Voluntary Suspension）。具体的方法是当发生完全暂停时，则会将虚拟机的内存页“0x160100”设置为不可读状态，每个线程执行到安全点时则会运行插入在这个点上的一条汇编指令，这条指令是 test 0x160100 这个内存区域，若该内存不可读，线程就会自动暂停下来，那么运行中的线程到达安全点后就相继暂停下来，其实这样的方式与 Java 代码中的锁是类似的。

有的小伙伴会想：如果一直运行不到安全点怎么办？其实不必担心，因为安全的位置也是很快的，几个汇编指令的执行是非常快速的。

不过，上面的方式只能暂停正在运行指令的线程，或者说处于 RUNNING 状态的线程，如果一些线程此时正处于睡眠、等待、阻塞等状态，它们被唤醒时 GC 还在进行中，就会出现问题的。为了解决这个问题，JVM 又提出一个“安全区域”（Safe Region）的概念，在这个区域内也是一些代码，但是一些不会导致引用变化的代码（具体由 JVM 来控制），当出现上述状态时，线程就会进入这个区域，如果线程需要离开这个区域，则需要检查 JVM

本身是否在做 GC 或 CMS GC 的标记阶段，若 JVM 的相应标记说明是安全的才能离开，继续执行。

这个安全区域有点类似于：街上有许多“恐怖分子正在进行恐怖袭击”，警察安排群众到一些“安全区域”等待，在没有等到警察将“恐怖分子”一网打尽或赶走的时候，这些群众不能离开这个安全区域。

3.5 浅析 Java 对象的内存结构

>> 3.5.1 原始类型与对象的自动拆装箱

我们常见的几种基本类型有：boolean、byte、char、short、int、long、float、double，除了 char 以外，其他类型都有一个装箱后的类型，我们称之为 Wrapper。最常见的 Integer 就是对 int 的包装类，在 JDK 1.5 以后它们可以相互转换。例如，int 的自动装箱使用 Integer.valueOf()来实现，它在编译阶段就完成了这样一个简单的转换（在第 1 章说明很详细）。现在我们看下面两段代码。

代码段 1:

```
int size = 100 * 1024 * 1024;
int []values = new int[size];
for(int i = 0 ; i < size ; i++) {
    values[i] = i;
}
```

代码段 2:

```
Integer values = new Integer[size];
for(int i = 0 ; i < size ; i++) {
    values[i] = i;
}
```

这两段代码有什么区别吗？

不用问，肯定有区别！

也许大家认为这两段代码的开销是一样多的，其实不然。

第 2 段代码有个自动装箱的过程，也就是 for 循环中这条代码会变成 values[i] = Integer.valueOf(i);。虽然会有-128~127 的缓存，但是对于上限是 100×1024×1024 来讲，Cache 的占比几乎可以忽略不计。

第 1 段代码仅仅有一个数组，数组中每个元素就存放 int 的值；而第 2 段代码的数组中存放的是 Integer 对象的引用，第 2 段代码中的数组除了引用空间的大小，还要算上所有 Integer 对象的内存占用才算真正的内存占用。第 2 段代码所使用的内存示意图如图 3-30 所示。

根据图 3-30 中所描述的，第 2 段代码的数组的大小应当等于数组本身占用空间加上 size * Integer 对象占用的空间，那么一个 Integer 对象占用多大的内存呢？下面我们一起来看看对象内存结构。

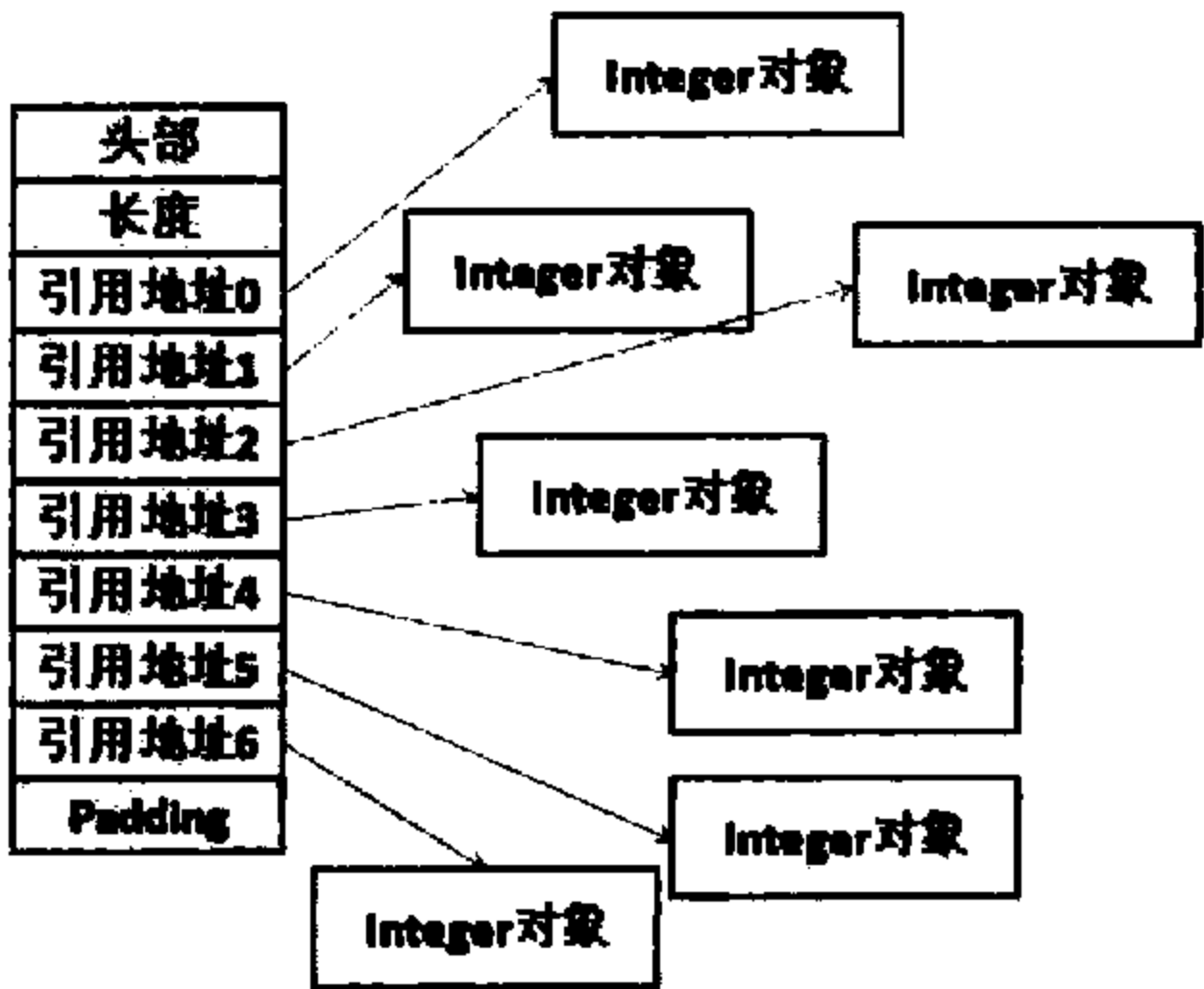


图 3-30 第 2 段代码所使用的内存示意图

3.5.2 对象内存结构

一个对象本身的内在结构需要一种描述方式, 这个描述信息是以字节码的方式存储在方法区当中的, 当申请分配对象时, 每一个对象实体是否还需要对象的描述信息呢? 如果需要, 每一个对象的空间会增加多少? 如果有上万个对象, 又会有多少额外的空间? 为了更细地了解这个问题, 我们先来分析以下两个技术点。

◎ Class 本身就是一个对象, 都以 KB 为单位, 如果 `new Integer()` 为了表示一个数字就占用 KB 级别的内存能接受吗? 肯定不能! JVM 肯定也不能傻到那种程度。因为一个 `int` 才 4 字节, 用几千个字节去表达 4 字节的内容, 就像花巨额资金买到一个不值钱的东西一样, 相信干了这样的事情后“晚上会睡不着”。

◎ 再回想一下前面提到的字节码增强技术, 如果对象的结构发生改变, 是不是要让每个对象都感知到, 如果大家都有一份 Class 的拷贝, 要修改这些拷贝是多么痛苦的事情。

但为了表示对象的属性、方法等信息, 不得不需要这个结构描述。于是, Hotspot VM 使用对象头部的一个指针指向 Class 区域的方式来找到对象的 Class 描述, 以及内部的方法、属性入口。不过, 大家需要注意的是, 对象头部不止存储这些内容, 它通常还会存放: 是否加锁、GC 标志位、Monitor GC 次数、对象默认的 hashCode (`System.identityHashCode(o)` 可获取对象的这个值) 等信息。或许许多小伙伴会认为这些信息似乎也需要很大的空间来存储, 其实不然, 就像 Class 存储方法、属性的标识符那样, 0、1 用一个二进制位就可以表达是与否, 它们也是类似的, 例如存放锁标记可能只需要 1~2 个二进制位就可以表达, 所以也并不需要太大的空间 (这部分空间通常叫作 Mark Word)。

在 32 位系统下, 存放 Class 指针的空间大小是 4 字节, Mark Word 空间大小也是 4 字节, 因此就是 8 字节的头部, 如果是数组还需要增加 4 字节来表示数组的长度。

在 64 位系统及 64 位 JVM 下, 开启指针压缩 (参数是 `-XX:+UseCompressedOOPS`, 现在很多 JVM 在小于 32GB 的情况下是默认打开的), 那么头部存放 Class 指针的空间大小还是 4 字节, 而 Mark Word 区域会变大, 变成 8 字节, 也就是头部最少为 12 字节。

若未开启压缩, 那么保存 Class 指针的空间大小也会变成 8 字节, 那么对象头部会变成 16 字节。另外, 在 64 位模式下, 若未开启压缩, 引用也会变成 8 字节。

开启指针压缩是用算法开销带来内存节约, 简单的理解是后面将要提到的 Java 对象都是以 8 字节对齐的, 也就是以 8 字节为内存访问的基本单元, 那么在地址处理上, 就有 3 个位是空闲的, 这 3 个位可以用来虚拟, 利用 32 位的地址指针原本最多只能寻址 4GB, 但是加上 3 个位的 8 种内部运算, 就可以变化出 32GB 的寻址。

Java 对象将以 8 字节对齐在内存中, 也就是对象占用的空间不是 8 字节的倍数, 将会被补齐为 8 字节的倍数, 好处自然是在对象分配和查找的过程中不用考虑过多的偏移量问题。

按照上面的理解, 我们可以将一个对象看成如图 3-31 所示的这样 (这个图在某些情况下是有问题的)。

下面我们来估算一些常见对象占用的空间大小。

以 32 位系统为例, 当使用 `new Object()` 时 (其实这个对象里面什么属性都没有), JVM 将会分配 8 字节的空间, 128 个 Object 对象将占用 1KB 的空间。

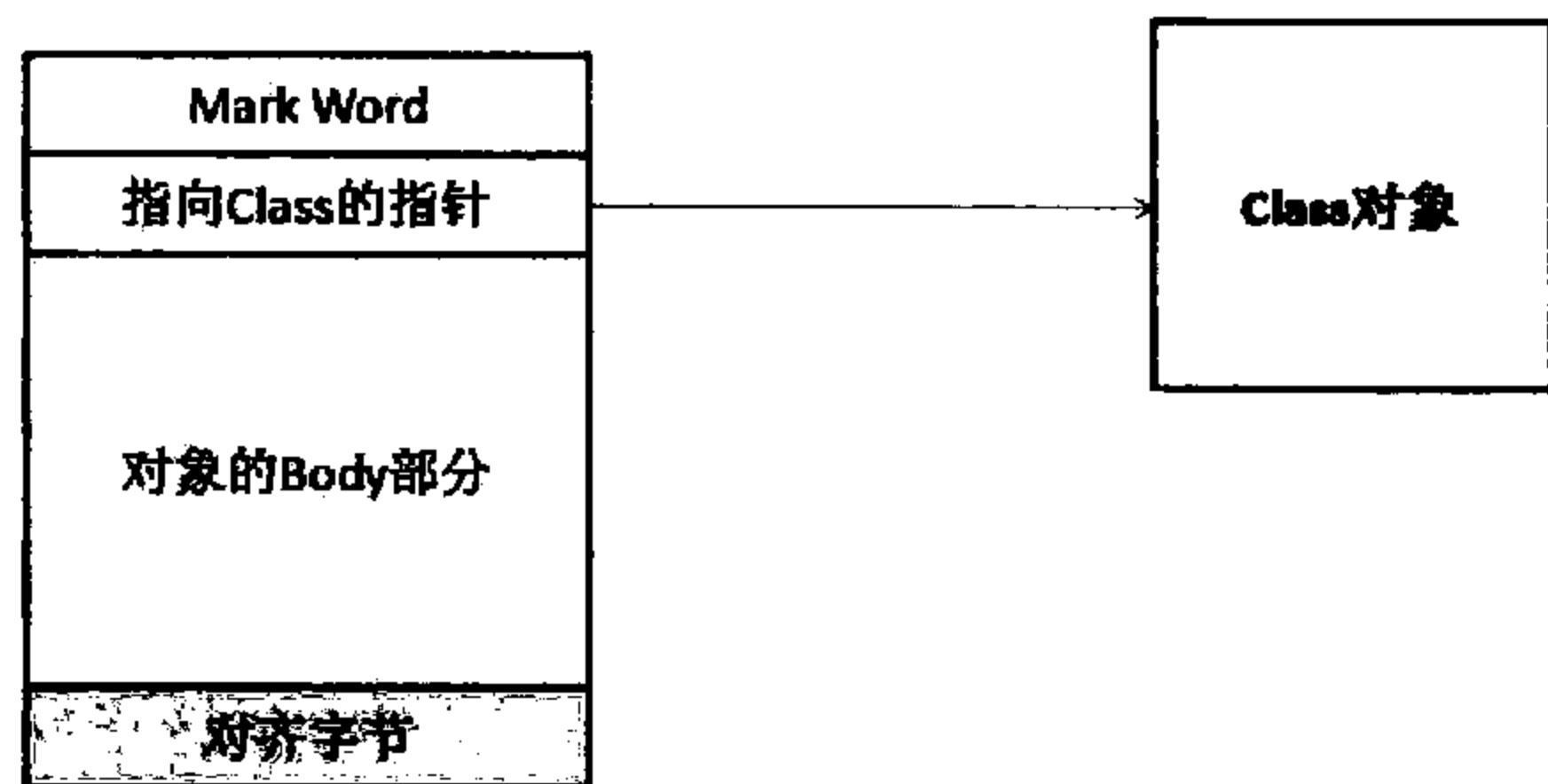


图 3-31 一个简单的对象模型，数组要加上 length

如果是 `new Integer()`，那么对象里面将包含一个 `int` 值，占用 4 字节，对象此时是 12 字节，按照对象 8 字节对齐的说法，它将对齐到 16 字节。

再回头看看 3.5.1 节的第 2 段代码。

在 32 位系统下，第 2 段代码会多出（数组长度×16 字节）的开销，其中的 `size` 为 $1024 \times 1024 \times 1024 = 100\text{MB}$ ，所以会有 1600MB 的多余开销。这是个恐怖的数字。

原始的数组也会有开销：

- ◎ 8 字节头部。
- ◎ 4 字节描述数组长度。
- ◎ $100\text{MB} \times 4 = 400\text{MB}$ 。
- ◎ padding 区域 4 字节。

也就是（400MB + 16 字节）的空间，忽略数组的 16 字节，占用 400MB 的空间。第 1 段代码也会占用这么大的空间，第 2 段代码占用 $1600\text{MB} + 400\text{MB} = 2\text{GB}$ 空间（抛开 0~127 的 `Integer` 是被缓存的），差距是接近 5 倍。

在 64 位模式下，它们的差距会更大。当然，这里用了一个极端的例子来说明这个问题，目的在于让大家了解到普通对象占用的空间除了本身的 `Body` 还有许多空间，而且每个对象都会有这样的空间，单个对象也许看不出区别，但是一旦对象多起来，其实占用的空间还是很大的。而类似的 C 语言中是没有的，这也许是同行们“笑话 Java”的原因之一吧。Java 关注的并不是这些内容，只要在代码中尽量注重“小而美”，在 Java 领域是足以应付的，因为这个例子就是一个极端的“大而丑”。

前面我们说的都是一些简单对象，到底对象内部的属性是怎么排布的？

（1）没有继承的对象属性排布（Object 没有属性，所以不用关心）

在默认情况下，Hotspot VM 会按照一个顺序排布对象的内部属性，这个顺序是：`long/double->int/float->short/char->byte/boolean->Reference`（与对象本身的属性顺序无关）。由此可以发现，对于普通类型，Java 首先按照宽度由大到小的顺序排列，这样处理的好处就是可以让属性排布的顺序很小。不过，对象的引用不是最小的，放在最后可能是为了 Java 语言的统一处理。下面举几个例子（例子说明都是在 32 位系统下）。

例子 1：

```
class A {
    byte b1;
}
```

在 32 位系统下，除了 8 字节头部，`byte` 会单独占用 1 字节，但是 padding 对齐后，会

占用 16 字节。

例子 2:

```
class A {
    byte b1 , b2 , b3 , b4 , b5 , b6 , b7 , b8;
}
```

添加了 8 个属性, 其实还是 16 字节, 此时的利用率最高 (如果加 1 个 int, 则相当于 4 个 int)。

例子 3:

```
class A {
    byte b1 , b2 , b3 , b4 , b5 , b6 , b7 , b8 , b9;
}
```

此时变成 24 字节的宽度, 实际的 Body 部分只有 17 字节, 是对齐到 24 字节的。

例子 4:

```
class A {
    int i;
    String str;
}
```

此时对象空间的占用也是 16 字节, 因为 int 占用了 4 字节, 而 String 的引用也占用了 4 字节 (这里由于 String 没有赋值, 所以本身的大小没有计算)。

例子 5:

```
class A {
    int i;
    byte b;
    String str;
}
```

按照 8 字节对齐原则, 对象大小应当是 24 字节。这个结果没错, 但是图 3-31 中的对齐部分在该例子中就有些不太正确了。那 24 字节的结果是错误的吗? 不是, 结果依然是 24 字节, 只不过对齐 (padding) 的位置会有所改变, 请看下面的解释。

在 Hotspot VM 中, 对象排布时, 空隙是在 4 字节基础上的 (在 32 位和 64 位压缩模式下), 在例子 5 中, int 后面跟了一个 byte, 空隙只剩下 3 字节, 对象引用需要 4 字节来存放, 因此 byte 和对象引用之间就会有 3 字节对齐, 对象引用排布后, 最后会有 4 字节对齐, 因此结果上依然是 7 字节对齐。此时对象的结构示意图应该如图 3-32 所示。

这是 Hotspot VM 对象属性默认的排布方式, 通过设置 `-XX:FieldsAllocationStyle=0` (默认值是 1) 参数, 引用会排布到最前面去, 对象的排布方式将会变成: Reference->long/double->int/float->short/char->byte/boolean。

最后请大家注意: 静态属性所占用的空间通常不计算到对象本身的空间上, 因为它的引用是在方法区, 所以在计算 Body 里面到底有多少东西时, 要抛开这些静态元素。

(2) 有继承的对象属性排布

在 Java 对象中, 大部分都有继承关系, 所以继承的排布我们不得不考虑。在 Hotspot VM 中, 类的继承并不会使得在创建对象时会创建多个, 也就是说, 它们只会有一个对象头部, 但是在内部结构中, 父类的属性依然要被分配到相应的对象中, 这样才能在程序中通过父类访问它的属性。

父类的属性不能和子类混用, 它们必须单独排布在一个地方, 可以认为它们就是从上到下的一个顺序。以两重继承为例, 对象继承属性排布规则如图 3-33 所示。

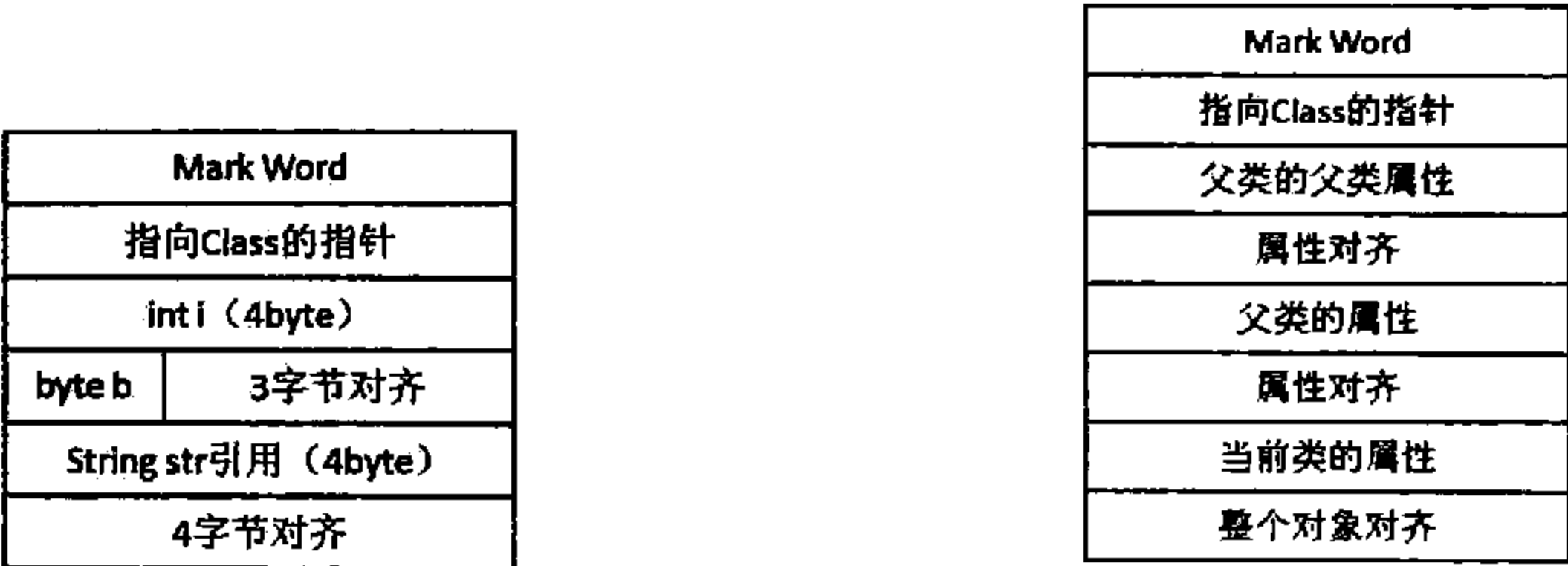


图 3-32 对象的结构示意图

图 3-33 对象继承属性排布规则

这里的对齐有两种：一是整个对象的 8 字节对齐；二是父类到子类的属性对齐。这个属性对齐就像前面提到的 byte 后面有 3 字节的空隙一样，在 32 位及 64 位压缩模式下，会按照 4 字节对齐。请看下面的例子。

```
class A {byte b;}
class B extends A {byte b;}
class C extends B {byte b;}
```

如果按照 3 字节合并为基本原则，此时对象应该是 16 字节，但是实际的对象却有 24 字节，这是怎么计算出来的？请看图 3-34。

结合前面的例子，依此类推，大家可以用其他属性来计算换算对象的大小。

这个对象的大小和排布是不是“忽悠”我们的？如何来证明这些结论是正确的呢？

除了看一些文档和博客外，就是测试对象内存的大小了。

曾经在网上看到过通过调用 System.gc()前后的内存使用来测量内存大小的，这也许是一种“没办法的办法”。其实在 JVM 运行中，即使将线程 sleep，也同样会有 JVM 内部的一些运行机制在占用空间（即使是输出语句也会占用空间），所以那样的方法可能不是那么“靠谱”。在 3.5.7 节中也会告诉大家如何获取一个对象计算内部属性在对象内部的位置。

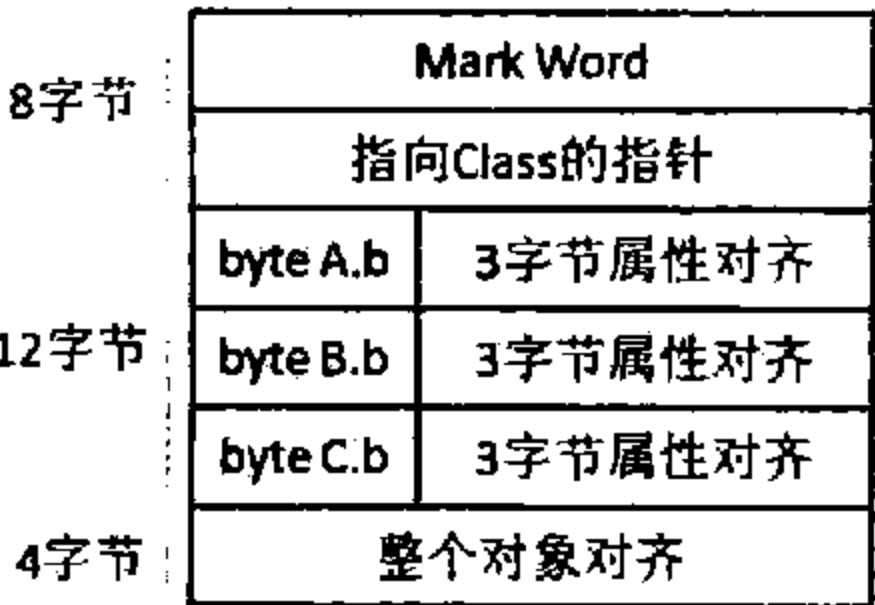


图 3-34 对象结构图

3.5.3 对象嵌套

什么是对象嵌套？

就胖哥的理解，对象嵌套是对象内部有引用指向了其他的对象，并且是一层嵌套一层，可能会有很多层的情况，而且这种情况在 Java 系统中是普遍存在的。

顺便再谈一个 GC 的问题：在早期的 JVM 垃圾回收中使用引用计数法回收时，对于对象出现了循环嵌套的情况是毫无办法的，也是 Java 的一个难解的结。不过现在好了，GC 只需要找活着的，就没有这个问题了。

对象嵌套使得统计一个对象占用了多大的空间变得更难，就像上面讨论的数组一样，数组本身并没有占用那么大的空间，但是在计算这个数组实际占用的空间时，就需要计算数组上引用所包含的对象空间。

对象嵌套其实在 Java 世界中无处不在，嵌套层次越深，中间浪费的内存就越多。例如前文中所提到的 Integer 数组，其实真正有用的数据是 Integer 对象内部的 int 值，数组只是

一个引用的载体而已，也就是说，这个数组的空间是多出来的，不是真正的数据。

再深入一点，使用二维数组、三维数组时，又会如何？

Java 中并不存在真正的二维数组、三维数组，它只有一维数组，二维数组是由多个一维数组构成的。这简单来看似乎没啥问题，但是仔细一想问题多多。

例如，在二维数组的第 1 维上存储的是一个数组引用，确切地说，存储的是所有第 2 维数组的引用。换句话说，第 2 维数组将是多个，最多是第 1 维数组的元素个数。每个第 2 维数组上的每个元素存储每个真正对象的引用，其实这中间所有的数组都没有存储真实的数据，存储的都是引用，这就是我们所说的“层数越多，浪费越多”。

如果是三维数组、四维数组将会有更多的浪费，大家可以试想一下，如果这些数组上包含的数据很少，那么将是一种什么样的情况。为了形象地表示出来，我们将二维数组简单地理解为如图 3-35 所示的内容。

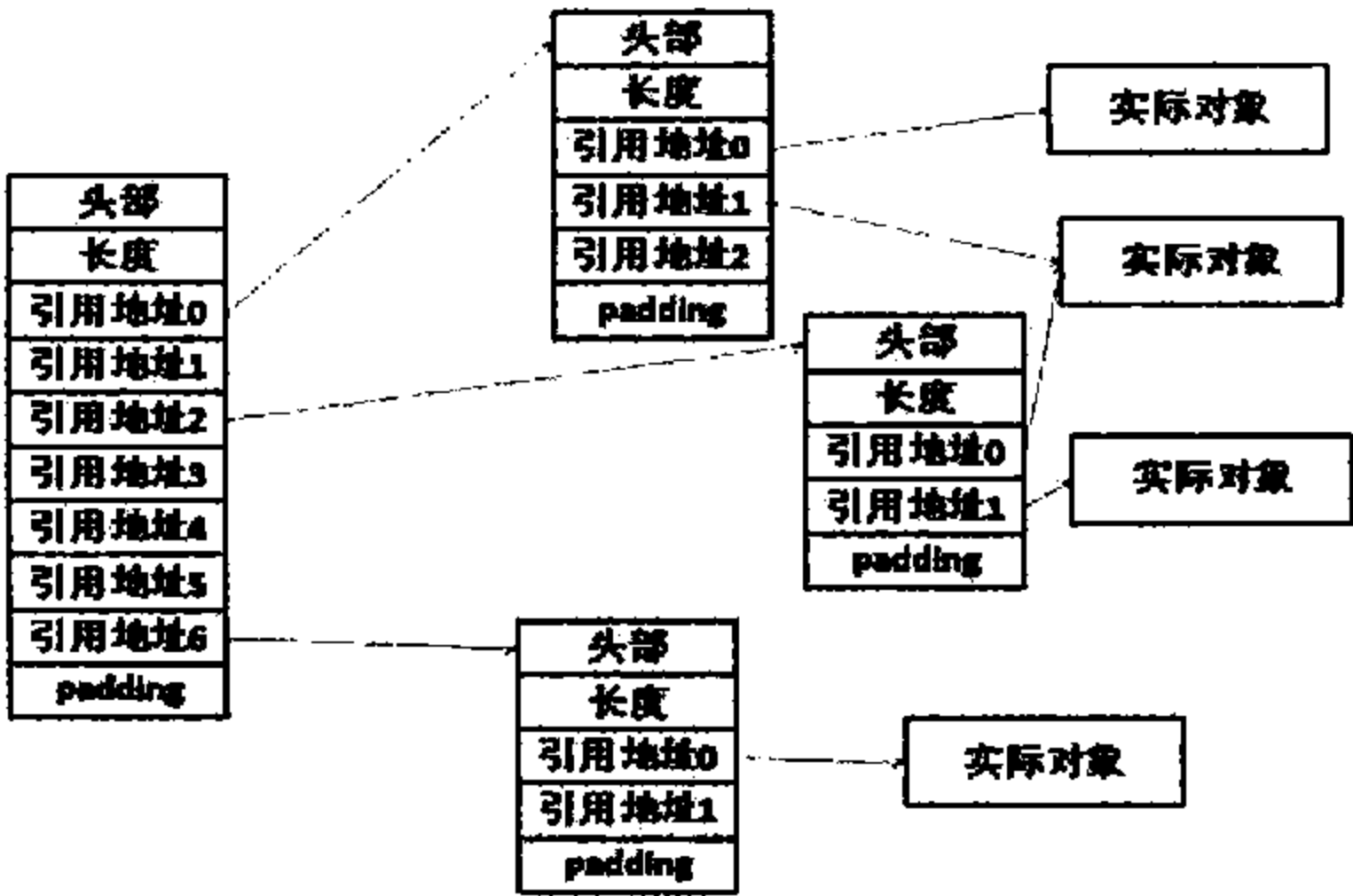


图 3-35 对二维数组的简单理解

这里面有很多引用地址是空的，实际对象只有几个，不同的数组引用还指向了同一个对象，此时大家可以计算一下这样的对象“实际数据”在对象空间中所占用的百分比，简直是低得可怜。

合理利用对象是关键，但同时请大家不要走极端，因为代码的可读性是第一重要的，只是在我们很明确地知道可以不浪费内存的情况下，就没必要去浪费内存。

类似这样的空间浪费还很多，不仅仅体现在数组中，在集合类中，尤其是集合类嵌套集合类，浪费更胜于数组。同时，自定义对象内部可以包装数组、集合类，换句话说，这些基础类型的空间浪费其实是在给自定义对象提供基础。

3.5.4 常见类型 & 集合类的内存结构

我们上面提到了 Integer，本节再说明一下第 1 章中就提到的 String，它的内部结构如何？按照上面计算对象内存的方式，来计算 String 可能会占用的空间大小。当我们使用 new String()时，Java 会执行如下代码（见图 3-36）。

JDK1.6及更早的版本

```
public String() {
    this.offset = 0;
    this.count = 0;
    this.value = new char[0];
}
```

JDK1.7以上的版本

```
public String() {
    this.value = new char[0];
}
```

图 3-36 创建一个空的 String 执行的代码

可以看出，内部申请了一个 char[] 数组，虽然长度为 0，但是同样占用数组的空间。下面再来看看它所包含的非静态属性都有哪些（见图 3-37）。

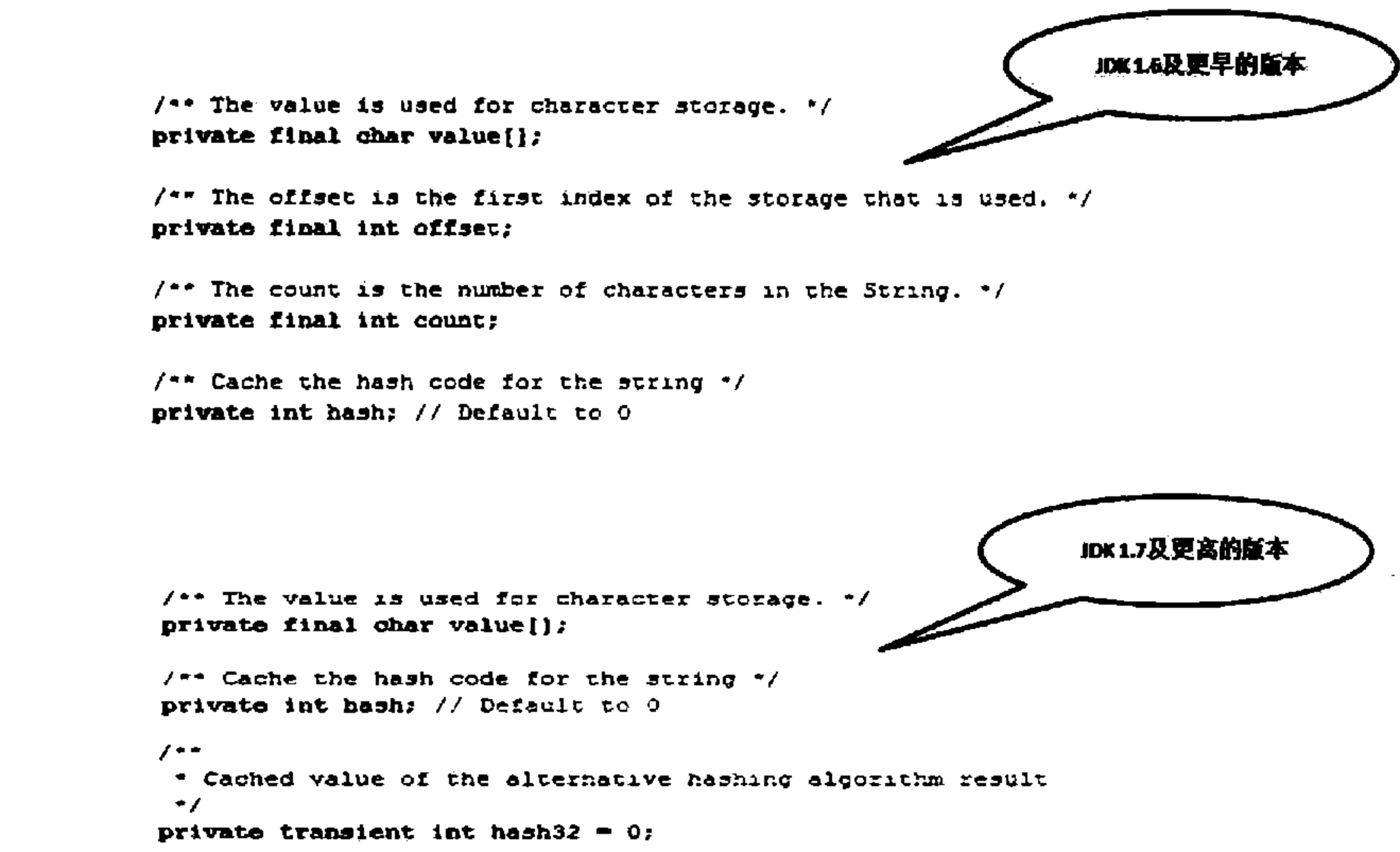


图 3-37 String 的属性

在 JDK 1.6 及以前的版本中，在 32bit 环境中，对象空间大致如下：

String 对象头部 8 个字节+3 个 int 类型的变量占用 12 字节+
char[] 数组的引用占用 4 字节= 24 字节

value[] 数组的宽度为 8(头部)+4(length)+4(padding)=16 字节（没有 Body），加上 String 本身的大小 24 字节，共 40 字节。换句话说，一个长度为 0 的字符串，也会占用 40 字节的空间。如果是在 JDK 1.7 中，则会少一个 int 类型的属性，但是由于 padding 的原因，在 32bit 环境中，该例子的计算结果不会变化（大家可以自行演算或测试）。

如果是在 64 位系统中（开启指针压缩），String 本身将占用的空间为：

12 字节（头部）+ 12 字节（3 个 int 变量）+ 4 字节（char[] 引用）+
4 字节（padding）= 32 字节的空间

而 String 对象中的 char[] 数组本身也将占用一定的空间：

12 字节（头部）+ 4 字节（数组长度）= 16 字节

加上 String 本身的 32 字节为 48 字节，即一个长度为 0 的字符串在这样的情况下会占用 48 字节的空间。不过，在 JDK 1.7 中，String 对象中的属性 int 变量会少一个，相应的 padding 也会自动去掉，因此计算结果还是 40 字节。

在 JDK 1.7 中，String 的属性以及代码都有很大的变化，不再有 offset、count，但是多了一个 hash32 属性，它在 Hash 相关算法的数据结构中大量使用，其内部通过 sun.misc.Hashing.stringHash32 使用 hash32，可参看资料：

http://java-performance.info/changes-to-string-java-1-7-0_06

接下来看看“可爱”的集合类。为啥说它可爱呢？因为大家都喜欢它。它屏蔽了像 C 语言那样通过不断改变指针和分配内存来操作数据结构，将它们包装成

了简单的接口信息,其里面也是对象嵌套对象。大家应该都会经常用到 ArrayList、HashMap 这样的集合类,它们的内部结构是什样子呢?以 HashMap 为例来看看(见图 3-38)。

```
/**
 * The table, resized as necessary. Length MUST Always be a power of two.
 */
transient Entry[] table;

/**
 * The number of key-value mappings contained in this map.
 */
transient int size;

/**
 * The next size value at which to resize (capacity * load factor).
 * @serial
 */
int threshold;

/**
 * The load factor for the hash table.
 *
 * @serial
 */
final float loadFactor;

/**
 * The number of times this HashMap has been structurally modified
 * Structural modifications are those that change the number of mappings in
 * the HashMap or otherwise modify its internal structure (e.g.,
 * rehash). This field is used to make iterators on Collection-views of
 * the HashMap fail-fast. (See ConcurrentModificationException).
 */
transient volatile int modCount;
```

图 3-38 HashMap 中占用空间的属性

这里面除了包含 3 个 int 属性、1 个 float 属性外,还包含 1 个 Entry[] 数组。每一个 Entry 里面有 4 个属性,分别是 key、value、next、hash,其中 key、value、next 为引用值,hash 为 int 类型。也就是说,在 32bit 环境中,一个 Entry 的 4 个引用就会占用 16 字节,加上头部最少 24 字节。

key 和 value 的值是需要单独计算空间的,因为它们都是对象,Entry 中的属性只是保存了它们的引用。而 next 是指向下一个 Entry 的,需要递归才能算出实际的空间。所以,一个 Entry 对象的空间最少 = 24 字节 + key 对象的大小 + value 对象的大小。

再回到图中的代码,代码中的属性 Entry[] 数组中的任意一个下标的 Entry 并不是一个简单的 Entry 对象,它会跟随一个 Entry 链表,所以都要一起计算才能得到结果。

一个 HashMap 中到底有多少 Entry? 每一个数组下标所对应的链表到底有多长? 这需要实际问题实际分析。总之,一个对象嵌套在 HashMap 中的嵌套层次是: HashMap 对象->Entry 数组->Entry 链表->Entry 对象->找到 value 引用以及对应的对象值。这中间其实只有 value 才是你的数据,更确切地说,只有 value 对象中的某些属性才是真正需要的值。

浪费了对象空间,这就是以前很多人不推荐层层嵌套对象的原因,尤其是不要嵌套集合类太多层次。看到一个 HashMap 就这样,如果嵌套两层、三层集合类会是什么样子呢?

ArrayList 会稍微好一些,它类似于 String,将对象用一个数组来存储,String 存储的是

char[]数组，而 ArrayList 存储的是对象引用。到目前为止，不论是任何基于数组结构的集合类，都逃不出内存碎片和复制拷贝的问题，当我们看了 3.5.5 节后，就会发现本节介绍的空间浪费还只是一个开始而已。

3.5.5 程序中内存拷贝和垃圾

第 1 章说到了 StringBuilder，也提到了许多场景，其中提到了比较关键的概念是“单行”中 String 的“+”将变成 StringBuilder 的一次或拼接，最后 toString()。这中间产生了大量的碎片和垃圾，碎片和垃圾导致了频繁的 GC，这也是许多朋友用 StringBuilder 和 String 单纯叠加后性能存在天壤之别的真正原因，可见内存碎片不容忽视。

对于 Java 语言，上一节中已经提到它对空间是十分浪费的，对于第 1 章中分析 String 的叠加过程，由于对象拷贝，可能对象只有 Old 区域的 1/4 时，内存就在“嗷嗷叫”了。作为 Java 程序员成天与内存打交道来讲，心理真是“一把汗、一把泪”啊！

第 1 章详细分析过 StringBuilder 内部其实存在大量的内存拷贝问题（只是它不会像 String 那样疯子般地产生大片的垃圾），同样的，对于数组类的集合类，内部也有大量的数组拷贝，在可控的情况下也可以在代码中设置初始化大小，尽量避免内存浪费。

如果大家在使用集合类时，经常进行删除、增加等操作，则可以考虑使用链表或 Tree 结构（例如 LinkedList、TreeMap），这样会更好一些。其实数组最大的好处在于通过下标访问最快（也叫随机访问）。

大家不要走极端，千万不要将自己系统中的 ArrayList 全部换成 LinkedList，我们要根据实际场景来落实具体的方法，“切忌”听到说什么好，就一个劲地使用它，它同样有不好的地方，一定要搞懂为什么，这样才能真正驾驭技术，而不是被技术所驾驭。

3.5.6 如何计算对象大小

本节胖哥带着大家一起测试 Java 内存对象的大小。关于一些朋友使用 System.gc()来测量内存，就不再做评价了，这里直接进入正题。

在 3.2.4 节中，我们一起探讨“字节码增强”时，使用到了 Instrumentation API，本节我们将再次使用 Instrumentation API 来测量 Java 对象内存的大小。

这个 API 这么强大？

没错，请看下面的代码清单。

代码清单 3-12 用 Instrument API 来测量对象的大小

```
public class NormalObjectSizeOf {

    private static Instrumentation inst;

    public static void premain(String agentArgs,
                               Instrumentation instP) {
        inst = instP;
    }

    public static long sizeof(Object object) {
        return inst.getObjectSize(object);
    }
}
```

该代码是用于测量对象的内存占用情况的，接下来我们可以在任何一个地方写一段调

用代码用于测试某个对象内存的大小，例如 `NormalObjectSizeOf.size(new String())` 用来测试一个长度为 0 的字符串所占用的空间，在本书 `src/chapter03/inst/sizeof` 目录下的 `NormalSizeOfTestmain.java` 中有相关的测试类。

在使用方式上，可以像 3.2.4 节中所介绍的那样，使用 agent 将这个类打包成 jar 包，假如是 `java_A_sizeof.jar`，在 jar 包所在的 `META-INF/MANIFEST.MF` 文件中追加 `Premain-Class: chapter3.inst.Normal ObjectSizeOf`，然后通过如下命令来运行：

```
java-javaagent:/tmp/agent/java_A_sizeof.jar chapter03.inst.sizeof.NormalSizeOfTestMain
```

这个计算结果是精确的，即使是父类的属性也会被叠加。运行后，你会发现这个地方算出来的结果可能和胖哥描述的对象计算结果有区别，为何呢？因为 `Instrumentation API` 中计算的对象内存大小，只是当前对象，如果内部有引用属性，它只会计算引用的大小，而不会进一步计算引用对象的大小。

如果要计算该如何办呢？这就需要自己去层层遍历属性以达到目的。

遍历的方式一种是递归，另一种是通过一个缓冲队列来记录。本书中的 `src/chapter 03/inst/sizeof/DeepObjectSizeOf.java` 就是选择后者实现的，由于代码比较多，胖哥就不列到这里来占大量篇幅了。

胖哥，对象的内在布局我们又怎样来判定呢？

其实方法很简单，分为两个步骤。

❶ 拿到 `Unsafe` 实例。这个好象直接拿不到，但是可以通过反射来拿到。

```
private static Unsafe getUnsafe() throws IllegalAccessException {
    Field filed = Unsafe.class.getDeclaredFields()[0];
    filed.setAccessible(true);
    Unsafe unsafe = (Unsafe)filed.get(null);
    return unsafe;
}
```

❷ 通过这个 `Unsafe` 实例 `objectFieldOffset(Field)` 就可以得到属性在对象内部的偏移量了。大家可以设置 `-XX:FieldsAllocationStyle=0|1` 参数来看看结果会不会发生变化，再对照前文中提到的对象布局方式看看是不是一致的。

➤➤ 3.5.7 轻松玩一玩 `int[2][100]` PK `int[100][2]`

从空间的角度看，`int[2][100]`和 `int[100][2]`有区别吗？貌似没啥区别。

如果你是这样回答的，则说明你没看过本节前面的内容或没看懂前面的内容。二维数组其实是由多个一维数组组成的，据此来算算它们的空间实际占用情况（在 32 位系统下）。

◎ 如果是 `new int[2][100]`，第 1 维的数组长度是 2，那么第 1 维数组的内存占用是：

8 字节	对象头部
4 字节	数组长度描述
2×4=8 字节	第 1 维的引用宽度
4 字节	padding
24 字节	合计

最多有两个子数组（我们只考虑子数组都存在的情况），每个子数组有 100 个 `int` 元素，那么每个子数组的空间占用是：

8 字节	对象头部
4 字节	数组长度描述

100×4=400 字节	每个元素为一个 int 宽度
4 字节	padding
416 字节	合计

两个这样的数组，再加上第 1 维的宽度，总共是 2×416+24 = 856 字节。

◎ 如果是 new int[100][2]，第 1 维的数组长度是 100，那么第 1 维的内存占用是：

8 字节	对象头部
4 字节	数组长度描述
100×4=400 字节	第 1 维的引用宽度
4 字节	padding
416 字节	合计

最多有 100 个子数组（本例只考虑 100 个子数组都存在的情况），此时每个子数组的大小如下：

8 字节	对象头部
4 字节	数组长度描述
2×4=8 字节	两个 int 的宽度
4 字节	padding
24 字节	合计

按照 100 个这样的子数组计算是 2400 字节，加上第 1 维的 416 字节，共 2816 字节的占用空间，而前者是 856 字节的占用空间，两者相差接近 3 倍。

这个例子并不是要讲解一些实际情况，仅仅为了让大家理解在对象空间分配时，稍加变化，分配的空间大小是完全不一样的。或许我们可以通过代码手段，在一些细节上节约一点点内存，那么总体来讲就会节约很大的内存空间了。这些都来源于对许多细节的理解，使得代码设计可以做得更好。本节仅仅是告诉大家一种思路而已。

3.6 常见的 OOM 现象

OOM 的实际场景其实是千变万化的，本节带着大家一起看看那些常见的 OOM。不过，解决方法除了使用许多工具外，还需要原理上的支撑。

>> 3.6.1 HeapSize OOM

最常见的 OOM 莫过于 HeapSize 的 OOM，什么时候会出现这样的 OOM 呢？用一段代码来试一试吧！

代码清单 3-13 一个简单的 Heap 区域的 OOM

```
public class HeapOOM {
    public static void main(String []args) {
        List<String>list = new ArrayList<String>();
        while(true) {
            list.add("内存溢出呀，内存溢出呀！");
        }
    }
}
```

相信运行不久，就会报错，并看到这样的关键字：

```
java.lang.OutOfMemoryError: Java heap space
```


这就是 Java 堆空间的溢出,也就是说,Old 区域剩余的内存,已经无法满足将要晋升到 Old 区域的对象大小,此时就会报出这样的错误。

遇到这样的情况,在代码层面处理方式和手段是什么?

首先看代码是否有问题,如果代码有问题就一定要修改代码。代码清单 3-13 模拟的就是代码中只做 add,但是没有 remove 的操作,这就是间接的“内存泄漏”(内存泄漏有些时候不会像这个例子一样简单,或许是很多对象有较长的生命周期,或许是在偶然的情况下,程序向一个全局的区域增加一条数据导致隐藏的数据膨胀,或许是一个死循环写入数据导致的)。

如果代码没有什么大的问题,但由于并发导致内存还不能被 GC 掉,或者说很多对象还有引用,那就说明对象所在的生命周期范围内的代码部分还没有执行结束。那么优化的方式自然会联想到:代码提速,代码运行速度一旦上来,相同的对象生存时间或许会更短,会更快地被当成垃圾,因此它是一个不错的方法。但有些代码提速并不是对每段代码都是有效的,有些时候我们还得想想其他的办法。在运行的代码中,如果能够找到一块比较大的内存区域,而且发现这块内存区域在后续的生命周期中再也没有什么用途,但后续的生命周期可能还会较长(例如还会做一些 I/O 操作),此时 `object = null` 是可以来辅助 GC 的。但我们并不倡导大部分代码都这样写,因为当方法脱离作用域后,相应的局部引用会自动被注销掉,只有它所引用的对象后续有较长的生命周期,且对象相对占用空间较大时(例如达到 KB 级别的对象),才是有必要的;否则我们的代码会出现很多“不太干净”的地方。

当代码无法优化,程序跑得很快后,还是会出现 OOM,那就考虑去修改参数配置吧。此时最需要改的就是堆的大小,需要将堆的空间设置得足够大,这样它才可以有足够的空间去承载更多的单位时间内的并发所使用到的内存区域。

多大才算是大呢?虽然我们希望它能做到无限大,那就永远没有 GC 了,但是没有这样的内存。反过来,当很大的内存发生 FULL GC 时,这个开销是非常“恐怖”的。

还有一种内存泄漏的情况,它更不容易被发现,那就是内存可能在某些使用的情况下增加几十字节的空间没释放,刚开始时发现可以被 GC 掉,但是每次 GC 掉总会上涨那么一点点,很老的对象可能也能被 GC 掉,但是会比较慢(例如在 Session 中存放数据,Session 会话失效就会注销掉,但会话要注销需要很长的时间)。

当程序运行几天后,发现系统超级慢,系统不断在做 FULL GC,且每次做 FULL GC 的时间非常长。为何?因为绝大部分内存对象都是活着的,所以在 GC 过程中标记活着的对象所需要的时间也很长。相应的,在 GC 过程中的压缩环节由于存在大量存活的对象,所以空隙变得更小,那么压缩的时间也会变长。这个时候,系统所表现出来的状态是不断在做 FULL GC,每次 GC 完后释放一点点内存,然后一下子就又满了,不断反复,当次数达到一定量,并且平均 FULL GC 时间达到一定比例时,就会报错:

```
java.lang.OutOfMemoryError:GC over head limit exceeded
```

这种现象很难模拟,但是通常在发生这种现象前,系统会变得奇慢无比(值得注意的是,系统奇慢无比的原因不止这一种)。为此,胖哥简单模拟一段不断做 FULL GC,但是又一直不抛出 OOM 的代码给大家看看。

代码清单 3-14 模拟一段不断做 FULL GC 的代码

```
public class GCOverHead {  
    /*这里先占用掉 Old 区域超过 14MB 的空间*/
```

```

public final static byte[] DEFAULT_BYTES = new byte[12 * 1024 * 1024];

public static void main(String []args) {
    List<byte[]> temp = new ArrayList<byte[]>();
    while(true) {
        temp.add(new byte[1024 * 1024]);
        if(temp.size() > 3) {
            temp.clear();
        }
    }
}

```

这段代码的命令如下:

```
$java -XX:+PrintGCDetails -Xmn5m -Xms20m -Xmx20m chapter03.oom.GCOverHead
```



注意 -XX:+UseGCOverheadLimit 处于启用状态才会报错, 不过该参数默认是启动的, 若未启动可以强制加上该参数。

这是一类十分经典的错误, 出现这种错误一般是这类问题发生了, 而这类问题通常是一些隐藏的 Bug 或配置导致的。

胖哥曾经遇到过 Tomcat 的 Session 导致的这类问题, 主要原因是程序中将 Session 信息保存到一个全局的 CurrentHashMap 中, 由于外部有大量的 HttpClient 访问创建的临时 Session, 这些 HttpClient 程序在访问时并没有保存系统为之分配的 sessionKey 和相关的 Cookie 信息, 导致每次请求都被认为以前没有创建过 Session, 所以会不断地创建 Session (Session 的创建发生在 request.getSession() 被调用的时候, 但这个操作不仅仅在自己所编写的代码中, 也可以在框架的间接调用中, 因此很多时候它是被隐藏创建的)。这些没有用的 Session 里面并没有多少内容, 就几十字节, 所以一般看不出来有问题, 但是这是一个积土成山的过程。

OOM 并不一定会导致宕机!

前面我们提到了多种 OOM 的情况, 大多是指系统的一个静态引用指向了一个只增不减的对象 (例如一个 HashMap), 有些时候 OOM 并不一定是从这里出来的。

胖哥也曾经遇到过一个 OOM 案例, 是网络上一位仁兄写的一段 JSON 解析代码, 被团队内的一位同学所使用, 网络上这位仁兄写的代码有点意思, 它在解析 JSON 字符串时是通过最简单的字符串截取来完成的, 当然这不算是问题的重点, 重点是在截取过程中遇到特殊字符 (双引号) 时的处理存在一点问题。

什么样的问题呢? 在程序中发现如果遇到了双引号, 则会将下标回退 1 个字符, 看前面一个字符是否为 “\”, 如果是这个符号, 则认为这个双引号是一个普通字符; 如果不是, 则认为是内部一个 JSON 串的结束字符。描述的思想很清晰, 细节在于这位仁兄判定如果是 JSON 串的结束字符, 就向后跳跃 1 个字符 (因为在前面将下标回退了 1 位), 而判定这个字符是普通字符时, 没有跳跃 1 个字符, 导致循环中下一次遍历时又回到了双引号的位置, 这样就导致了死循环。在这个循环过程中, 会将每个字符写入到一个 StringBuilder 中, 死循环后就不断地向这个 StringBuilder 中写入双引号, 最终导致 OOM。

这种 OOM 很特殊, 这个 `StringBuilder` 是一个局部变量, 导致 OOM 后会抛出一个 `OutOfMemoryError`, 在方法外部只要使用 `try {...} catch(Throwable e) {...}` 就能捕获到这个 `Error`。

捕获到 `Error` 有何用途呢?

通常捕获到 `Error`, 是不让这个 `Error` 抛到容器顶层来处理, 这样可以让进程继续存在。此时 OOM 错误是因为一个局部变量导致的, 当抛出的错误在方法外部捕获时, 局部变量的作用域其实已经脱离, 如果发生一次 FULL GC, 内存就可以释放掉, 系统是可以照常运行的。不过这样会导致的问题是, 即使出现这样的 OOM, 也不容易发现, 因为系统一切运行正常, 只有这段代码引起的错误被大家知道才会去跟进。

同样的, 这样的情况发生了 OOM, 用 `jmap` 将内存 dump 出来或内存溢出时 dump 出来看到的内存情况可能很正常, 因为在 dump 时通常会先做一次 FULL GC, 这个时候若局部变量脱离作用区域就没法捕获它。

3.6.2 PermGen OOM

现在来看看“悲催”的永久代的内存溢出情况。前面提到了永久代存放的东西有 `Class` 和一些常量, 这里用两种方式来模拟。首先用第 1 章提到的 `String` 中的 `intern()` 来模拟一段 Perm 区域的内存溢出情况, 代码如下。

代码清单 3-15 用 `String.intern()` 让 Perm 区域内存溢出

```
public class StringInternPermGenOOM {  
  
    public static void main(String []args) {  
        int i = 0;  
        List<String> list = new ArrayList<String>();  
        while(true) {  
            list.add(  
                ("快点死掉吧, 快点死掉吧!!!!!!!!!!!!!!!!!!!!!!"+i++).intern());  
        }  
    }  
}
```

为了快速出来效果, 需要加上 `-XX:PermSize10m -XX:MaxPermSize10m` 参数 (最好不要用工具来执行), 在命令行下运行, 在胖哥的机器上 1~2s 就出来结果了, 如图 3-39 所示。

```
$java -XX:PermSize=10m -XX:MaxPermSize=10m chapter3.oom.StringInternPermGenOOM  
Exception in thread "main" java.lang.OutOfMemoryError: PermGen space  
    at java.lang.String.intern(Native Method)  
    at chapter3.oom.StringInternPermGenOOM.main(StringInternPermGenOOM.java:12)
```

图 3-39 用 `String.intern()` 让 Perm 区域内存溢出

大家可以修改代码, 将代码中的 `ArrayList` 去掉, 不断地 `intern()` 发现它不会发生 OOM, 但是不是就没有问题了呢? 再加上 `-XX:+PrintGCDetails` 参数, 会发现它不断在做 FULL GC。

这说明什么呢? 这说明 PermGen 区域一旦满了, 就会做 FULL GC, 并且 FULL GC 会回收常量池里面的内容, 这与前面提出的导致 FULL GC 的几个原因相对应。

`String` 常量对象会在常量池中以 Hash 方式存储和访问, 这个 Hash 表默认大小为 1009, `String` 常量池里面也包含了类名、方法名、属性名等信息, 如果 `String` 常量很多, 则可以适当修改 `-XX:StringTableSize` 参数来增加 Hash 元素的个数, 减少 Hash 冲突的

概率。(在第 1 章中介绍了 Hash 冲突的问题)。

另外, 在第 1 章中已经提到, String 常量在 JDK 1.7 以上就不再存放在 PermGen 中了, 所以要注意本书的许多结论是基于 JDK 1.6 的。

再来看看如何通过 Class 的加载, 使得 PermGen 区域内存溢出, 这里胖哥会用到 CGLib 框架。

代码清单 3-16 用 CGLib 不断创建子类让 Perm 区域内存溢出

```
public class ClassPermGenOOM {

    public static Object createProxy(Class <?>targetClass) {
        Enhancer enhancer = new Enhancer();
        enhancer.setSuperclass(targetClass);
        enhancer.setUseCache(false);
        enhancer.setCallback(new MethodInterceptor() {
            public Object intercept(Object object,
                                   Method method, Object[] args,
                                   MethodProxy methodProxy)
                throws Throwable {
                return methodProxy.invokeSuper(object, args);
            }
        });
        return enhancer.create();
    }

    public static void main(String []args) {
        while(true) {
            createProxy(ClassPermGenOOM.class);
        }
    }
}
```

同样将 Perm 区域设置得足够小时, 就很快报出异常, 而导致异常的原因就是 Perm 区域内存溢出, 如图 3-40 所示。

```
$java -XX:+TraceClassUnloading -XX:PermSize=10m -XX:MaxPermSize=10 chapter3.oop.ClassPermGenOOM
[Unloading class sun.reflect.GeneratedConstructorAccessor1]
Exception in thread "main" net.sf.cglib.core.CodeGenerationException: java.lang.reflect.InvocationTargetException-->null
    at net.sf.cglib.core.AbstractClassGenerator.create(AbstractClassGenerator.java:237)
    at net.sf.cglib.proxy.Enhancer.createHelper(Enhancer.java:377)
    at net.sf.cglib.proxy.Enhancer.create(Enhancer.java:285)
    at chapter3.oop.ClassPermGenOOM.createProxy(ClassPermGenOOM.java:28)
    at chapter3.oop.ClassPermGenOOM.main(ClassPermGenOOM.java:33)
Caused by: java.lang.reflect.InvocationTargetException
    at sun.reflect.GeneratedMethodAccessor1.invoke(Unknown Source)
    at sun.reflect.DelegatingMethodAccessorImpl.invoke(Unknown Source)
    at java.lang.reflect.Method.invoke(Unknown Source)
    at net.sf.cglib.core.ReflectUtils.defineClass(ReflectUtils.java:384)
    at net.sf.cglib.core.AbstractClassGenerator.create(AbstractClassGenerator.java:219)
    ... 4 more
Caused by: java.lang.OutOfMemoryError: PermGen space
    at java.lang.ClassLoader.defineClass1(Native Method)
    at java.lang.ClassLoader.defineClassCond(Unknown Source)
    at java.lang.ClassLoader.defineClass(Unknown Source)
    ... 9 more
```

图 3-40 Class 导致永久代内存溢出

若想要看看 Class 太多时, 有没有卸载过 Class, 那么上面的运行代码中增加启动参数-XX:+TraceClassUnloading, 发现没有任何 Class 被卸载的日志, 最后直接内存溢出。这并不代表 Perm 中的 Class 是不会被卸载的, 但是为何没卸载, 内存溢出呢?

回想一下本章前文中提到 Class 的地方, 它卸载的条件非常苛刻, 这个 Class 所对应的 ClassLoader 下所有的 Class 都没有活着的对象引用才会被卸载。

为了验证这个“说法”, 我们刻意在每个 Class 动态创建时, 增加一个自定义的

ClassLoader, 为此将代码做了如下调整。

代码清单 3-17 不断创建子类, 修改代码, 也未必内存溢出

```
public class ClassPermGenFullGC {

    private static URLClassLoader parentClassLoader;

    static {
        parentClassLoader
            = (URLClassLoader)ClassPermGenFullGC.class.getClassLoader();
    }

    public static Object createProxy(Class <?>targetClass) {
        Enhancer enhancer = new Enhancer();
        enhancer.setSuperclass(targetClass);
        enhancer.setClassLoader(
            new URLClassLoader(parentClassLoader.getURLs(),
                               parentClassLoader));
        enhancer.setUseCache(false);
        enhancer.setCallback(new MethodInterceptor() {
            public Object intercept(Object object,
                                   Method method, Object[] args,
                                   MethodProxy methodProxy) throws Throwable {
                return methodProxy.invokeSuper(object, args);
            }
        });
        return enhancer.create();
    }

    public static void main(String []args) {
        while(true) {
            createProxy(ClassPermGenFullGC.class);
        }
    }
}
```

每次 CGLib 动态创建时, 都重新给它设置一个 ClassLoader, 这样运行代码就不会出现 OOM 了, 这似乎印证了我们的结论。再增加启动参数-XX:+TraceClassUnloading, 会发现大量的 Class 被卸载, 如图 3-41 所示。

```
$java -XX:+TraceClassUnloading -XX:PermSize=10m -XX:MaxPermSize=10 oom.ClassPermGenOOM
[Unloading class oom.ClassPermGenOOM$$EnhancerByCGLIB$$586cdbba]
[Unloading class oom.ClassPermGenOOM$$EnhancerByCGLIB$$586cdbba$$FastClassByCGLIB$$68e9df0f]
[Unloading class oom.ClassPermGenOOM$$EnhancerByCGLIB$$586cdbba]
[Unloading class $java.lang.Object$$FastClassByCGLIB$$3f697993]
[Unloading class oom.ClassPermGenOOM$$EnhancerByCGLIB$$586cdbba]
[Unloading class oom.ClassPermGenOOM$$EnhancerByCGLIB$$586cdbba]
[Unloading class $java.lang.Object$$FastClassByCGLIB$$3f697993]
[Unloading class oom.ClassPermGenOOM$$EnhancerByCGLIB$$586cdbba]
[Unloading class $java.lang.Object$$FastClassByCGLIB$$3f697993]
[Unloading class oom.ClassPermGenOOM$$EnhancerByCGLIB$$586cdbba]
[Unloading class oom.ClassPermGenOOM$$EnhancerByCGLIB$$586cdbba$$FastClassByCGLIB$$68e9df0f]
[Unloading class oom.ClassPermGenOOM$$EnhancerByCGLIB$$586cdbba$$FastClassByCGLIB$$68e9df0f]
[Unloading class oom.ClassPermGenOOM$$EnhancerByCGLIB$$586cdbba]
[Unloading class oom.ClassPermGenOOM$$EnhancerByCGLIB$$586cdbba$$FastClassByCGLIB$$68e9df0f]
[Unloading class $java.lang.Object$$FastClassByCGLIB$$3f697993]
[Unloading class oom.ClassPermGenOOM$$EnhancerByCGLIB$$586cdbba]
[Unloading class oom.ClassPermGenOOM$$EnhancerByCGLIB$$586cdbba$$FastClassByCGLIB$$68e9df0f]
[Unloading class oom.ClassPermGenOOM$$EnhancerByCGLIB$$586cdbba$$FastClassByCGLIB$$68e9df0f]
[Unloading class oom.ClassPermGenOOM$$EnhancerByCGLIB$$586cdbba$$FastClassByCGLIB$$68e9df0f]
[Unloading class $java.lang.Object$$FastClassByCGLIB$$3f697993]
[Unloading class $java.lang.Object$$FastClassByCGLIB$$3f697993]
[Unloading class oom.ClassPermGenOOM$$EnhancerByCGLIB$$586cdbba$$FastClassByCGLIB$$68e9df0f]
```

图 3-41 每次创建一个 ClassLoader, 这些 Class 就可以被卸载了

看来我们的结论是正确的, 这也证明这些 Class 的释放并不是那么容易的。如果我们需要在应用系统中动态加载一些类, 或者动态编译一段 Java 代码进行加载, 那么最好的方

式是有单独的 ClassLoader, 因为当 Class 被替换时 (例如当代码发生变化时), 原来的 Class 就可以被当成垃圾释放掉了。

►► 3.6.3 DirectBuffer OOM

DirectBuffer 这块区域是什么区域呢?

Java 的普通 I/O 采用输入/输出流的方式来实现, 输入流 (InputStream) 都会经历从终端到直接内存再到 JVM 的过程; 输出流 (OutputStream) 是反过来的, 这中间其实有多次 Kernel 与 JVM 之间的内存拷贝, 有些时候为了提高速度, 就会想办法利用直接内存 (当然在一些场景下未必性能就是最好的)。

在 Java 里面有一块区域叫作 DirectBuffer, 这块区域不是 Java 的 Heap, 而是 C Heap 的一部分。不过, 它也有大小限制, 通常在 FULL GC 时会回收, 我们也可以写代码来回收分配的空间。如果这块区域使用不当也是一个大坑, 后文中会谈到具体的一些坑。

下面先写一段让这块区域 OOM 的代码。

代码清单 3-18 让 DirectBuffer 内存溢出

```
import java.nio.ByteBuffer;

public class ByteBufferOOM {

    //-XX:MaxDirectMemorySize=256m
    public static void main(String []args) {
        ByteBuffer.allocateDirect(257 * 1024 * 1024);
    }
}
```

由于不同的 JDK 版本和位数 OOM 的值不一样, 在测试中, 胖哥设定了一个启动参数 -XX:MaxDirectMemorySize=256m。通过实验可以发现, 申请的空间只要比这个空间多一点, 就会报出错误: “java.lang.OutOfMemoryError: Direct buffer memory”, 如果只是申请 256MB, 则不会报错 (C Heap 区域的对象尺寸是十分精确的)。

大家会发现一件很有意思的事情, 就是如果不设置 MaxDirectMemorySize 参数, 该参数的默认值会等价于 Xmx 的值减去 1 个 Survivor 区域的值。

例如, 设置启动参数 -Xmx=300m -Xmn10m, 根据前文提到的比例, 一个 Survivor 默认是 1m, 此时会发现申请 299MB 的 DirectBuffer 空间是没有问题的, 申请 300MB 就出现了 OOM 现象, 这个值可以任意替换, 结果也都是这样。

在启动参数 -XX:MaxDirectMemorySize=256m 不变的情况下, 再进行另一种测试, 将代码修改为两行:

```
ByteBuffer.allocateDirect(256 * 1024 * 1024);
ByteBuffer.allocateDirect(1024 * 1024);
```

也就是先申请 256MB, 再申请 1MB, 此时会发现 JVM 不会出现 OOM 现象。

为何? 难道内存被 GC 掉了? 为了证明这个结论, 开启 -XX:PrintGCDetails 参数来看看, 果然发生了一次 FULL GC, 不过在 FULL GC 的日志中看不到任何和 DirectBuffer 相关的信息。

```
$java -XX:MaxDirectMemorySize=256m -XX:+PrintGCDetails chapter03.oom.ByteBufferOOM
[GC [PSYoungGen: 323K->224K(18880K)] 323K->224K(62080K), 0.0030508 secs] [Times:
```



```

user=0.00 sys=0.00, real=0.00 secs]
[Full GC(System) [PSYoungGen: 224K->0K(18880K)] [PSOldGen: 0K->163K(43200K)]
224K->163K(62080K) [PSPermGen: 2982K->2982K(21248K)], 0.0092478 secs] [Times:
user=0.00 sys=0.00, real=0.01 secs]

```

这次 Full GC 后面仅跟着一个(system)关键字,它代表这一次 Full GC 是由 System.gc()发起的。上面的代码中没有任何这样的操作,难道是空间满了由 Java 代码做的?不可能,如果是那样的话,不会出现这个关键字。

于是,跟踪源代码,在分配内存时,间接调用了 java.nio.Bits 类的 reserveMemory 方法,发现方法体中果然有调用 System.gc(),如图 3-42 所示。

```

static void reserveMemory(long size, int cap) {
    synchronized (Bits.class) {
        if (!memoryLimitSet && VM.isBooted()) {
            maxMemory = VM.maxDirectMemory();
            memoryLimitSet = true;
        }
        // -XX:MaxDirectMemorySize limits the total capacity rather than the
        // actual memory usage, which will differ when buffers are page
        // aligned.
        if (cap <= maxMemory - totalCapacity) {
            reservedMemory += size;
            totalCapacity += cap;
            count++;
            return;
        }
    }
}

System.gc();
try {
    Thread.sleep(100);
} catch (InterruptedException x) {
}

```

图 3-42 申请 DirectByteBuffer 空间内部的代码

这里的代码证明了我们的结论,不过这件事情是不是就完了呢?

我们可以再做进一步测试,如果将上述测试代码中的两次申请的内存再设置小一点,加起来小于 256MB,就不会导致 DirectByteBuffer 空间不够用,或许此时的结论是不会发生 FULL GC。这一点也论证了官方对于显式 GC 的一个结论:在调用 System.gc()时未必立即去做 GC,也未必会去做 GC。

如果使用了-XX:+DisableExplicitGC 参数,那么系统就不会在 System.gc()动作发生时去做 FULL GC 动作(即使空间不足也不会去做)。例如,在下面的例子中,即使使用了修改后的代码,结果也将 OOM。

```

$java -XX:MaxDirectMemorySize=256m -XX:+DisableExplicitGC -XX:+PrintGCDetails
chapter03.oom.ByteBufferOOM
Exception in thread "main" java.lang.OutOfMemoryError: Direct buffer memory
at java.nio.Bits.reserveMemory(Unknown Source)

```

我们发现-XX:+DisableExplicitGC 参数是一个“坑”,无论是否设置都是一个“坑”,为什么呢?不禁用显式 GC,可能会导致许多地方反复被调用 System.gc(),系统出现不必要的频繁 FULL GC;但禁用显式 GC 后,发现 DirectBuffer 满后不会触发 FULL GC,而是直接 OOM,所以在某些系统的设计上一定要考虑清楚再设置参数,并不是说该参数是否一定要禁用。

我们再将测试代码改成下面这样,增加一个引用保存第 1 次申请的 256MB 的内存:

```
ByteBuffer byteBuffer = ByteBuffer.allocateDirect(256 * 1024 * 1024);
ByteBuffer.allocateDirect(1024 * 1024);
```

此时一旦运行，就直接 OOM 了，这说明什么呢？DirectBuffer 的 GC 规则与堆对象的回收规则一样，只有垃圾对象才会被回收，而判定是否为垃圾对象依然是根据引用树中的存活节点来判定。

难道 DirectBuffer 就真的只有等它用满的时候才能回收吗？

其实还可以通过下面的代码来回收：

```
((DirectBuffer)byteBuffer).cleaner().clean();
```

这个 DirectBuffer 是一个接口，直接类 DirectByteBuffer 不是 public 类型的，通常使用接口访问它，它实现了 ByteBuffer 和 DirectBuffer 接口，因此可以通过强制类型转换为 DirectBuffer 接口来访问相应的方法。以前有很多朋友是通过反射来获取 cleaner 对象再做 clean()操作的。

在使用完内存后，调用一次这样的代码，相应的内存会立即释放，无须等到 GC 才被释放。另外，在上面出现 OOM 的代码中，即使手工将 GC 关闭，在两行代码的中间，插入该行代码，也不会出现 OOM，而且不会出现 FULL GC，这个大家可以自行测试。

突然发现 Java 也有一块空间可以玩转自如，自己申请，自己释放。但是小伙伴们切记滥用这些东西哦，在本书的后文中还会有相关的一些介绍。

3.6.4 StackOverflowError

在前文中提到，在程序运行过程中，方法分派时会分配 frame 来存放本地变量、后进先出栈、PC 寄存器等信息，如果方法再调用方法，在不断分派的过程中它也会占用十分多的空间。Java 为了控制一个线程栈空间的无休止增长（例如，在某种情况下导致了死递归），就会设立一个私有栈空间大小，通常用 -Xss 来设置，它的默认值会随着版本、操作系统、位数不同而发生一些改变，大家记得数量级是 256KB~1MB 就可以了，没必要去纠结到底是多少，具体的在系统上看就可以了。

线程私有栈所占用的不再是堆内存，通常叫它 Native Memory。若使用的空间超过 -Xss 这个限制，就会出现 StackOverflowError。

为了更好地理解，下面简单写段模拟死递归的代码，让它抛出这样的错误。

代码清单 3-19 模拟一个栈溢出的现象

```
public class StackOverFlowTest {

    public static void main(String []args) {
        new StackOverFlowTest().testStackOver();
    }

    public void testStackOver() {
        testStackOver();
    }
}
```

运行，就会报出错误：

```
java.lang.StackOverflowError
```

这段代码很简单，是一个自身调用自身的过程，也就是死递归的一个简单例子，现实中的代码会被隐藏得很深（但道理相同）

当遇到某些业务场景的时候,我们不得不用递归去解决问题,需要尽量控制好递归层数。例如,可以在参数上携带调用的次数,将它们都包装在对象中或直接作为参数传入,每次访问时,将这个参数叠加 1,若达到一个限额,那么就不再递归了;也可以使用异常或状态标注出来,为线程运行提供一种可控的处理方式,而不是不受控制地递归。

死递归并不等价于死循环,死循环是类似于 `while(true)` 的操作,或者因为某种原因跳不出循环,但是它的栈空间使用是不会递增的;而死递归由于需要记录退回的路径,就必须记住递推过程中的方法调用过程,以及每个方法运行过程中的本地变量,这个我们称为上下文信息,随着内容的增加,就会占用很多的内存空间,JVM 是为了控制它的无限制增长,才做了安全检测的处理。

也有人设置 `-XX:ThreadStackSize` 参数,设置这个参数也是没问题的,但是这个参数与 `-Xss` 放在一起就会有冲突,不过不像其他的某些参数冲突后会导致 JVM 跑不起来。对于这类问题,胖哥本人的建议是:如果不是自己玩虚拟机源码,不要去跟参数冲突过不去,即使是测试结果也未必是完全靠谱的。类似的还有设置 `-ss`,在 IBM 的 JVM 中设置 `-Xoss`,如果这些混在一起,可能我们就彻底晕了,其实我们只需要知道大多数情况下都用什么参数来设置即可,能满足自己的需求就好。本节中用于测试的都是设置 `-Xss`,例如 `-Xss=512k`。

发生 `StackOverflowError` 通常都是程序有问题,因为 JVM 对栈帧的大小设置已经很大了,通常程序用不了多少。问题通常出在程序的死递归上面,简单的递归可以尝试通过一些递归深度去控制,而很多时候程序的递归往往比想象的复杂许多倍,有些看起来不是递归的程序最终发生了递归。

◎ 现在来说两个例子。

◎ 某些复杂的设计使得对象之间来回进行方法调用,把开发的同学搞得很糊涂,或者每个组件都由单独的一个程序员来编写,相互之间调用对于每个编写代码的程序员来讲都是单向的,而大家都认为得到了组件的复用,就很容易产生这类问题。如果在开发阶段发生了这类问题,我们可以及时地找到问题所在,并将其解决,我们担心的是这种循环嵌套是在某种条件因素成立的时候才会发生,而开发阶段忽略了这种用例(例如出现某种异常,而且一旦出现异常就会不断出现),这样就可能会出现本地的程序是好用的,发布到线上运行后会偶然发生 `StackOverflowError` 的现象。

◎ “子类”调用“父类”的方法(为了复用父类的方法),“父类”再直接或间接地调用“子类”的方法(为了达到多态的效果),此时中间可能会经历多个方法,但是由于某种条件因素成立,最终导致了一个环形,进而形成了死递归。

◎ 三方框架中的代码问题。在开源界有个最不好的地方,就是开源是不用承担商业责任的,所以开源框架有问题是正常的,即使是我们认为不可能出现问题的三方框架也有可能出现问题。胖哥就在不久前发现 MySQL JDBC 的一个问题导致了 `StackOverflowError`,在本书下册中我们将详细介绍其中的细节。

定位 `StackOverflowError` 问题其实很简单,输出的线程栈信息明确地说明了调用路径。但写程序的人不能因为问题好定位就不重视这类问题,更不要认为 Java 什么事情都帮我们做好了,有许多事情是机器无法完成的,需要我们来思考,这样就可以避免许多不必要的麻烦。

3.6.5 其他的一些内存溢出现象

下面再介绍一些常见的 JVM 使用不当导致的问题。

(1) unable to create new native thread

在前一节提到栈本身是占用空间的，只是它占用的不是 Heap 区域的空间，而是 Native Memory。同时也提到了通过 -Xss 来设置每个私有栈空间的大小，也就是对一个线程来讲它的私有空间的最大限度，如果反复申请了大量的线程并让它们处于运行状态，那么这些线程所占用的 Native Memory 空间就会很多。

如果物理内存不够用，或者操作系统限制了单个进程使用的最大内存，那么在大量线程分配时就有可能抛出错误：“unable to create new native thread”。

这个错误表示无法分配本地内存，我们要模拟这个错误发生的方法说简单也简单，说复杂也复杂，说简单就是大家可以在一个死循环中创建线程并启动，每一个线程内部申请一些变量，但是线程本身并不会退出，同时要确保操作系统对进程有内存限制；否则可能会将整个机器变得十分卡，也模拟不出来。

(2) request {} byte for {} out of swap

看到这个信息，基本是地址空间开始不够用了。什么是地址空间？我们在第 2 章中有所阐述，它并不一定是物理地址，还应该有 Swap、显卡、网卡这些区域，只要是进程可能被使用到的位置都叫地址空间。

理论上，只要这些空间够用就不会产生这样的错误，Java 堆是初始分配的，如果那个时候 OS 的地址空间不够用，JVM 是无法启动的，Java 还会在本地内存区域继续分配，对于进程来讲，在 32 位下默认都可以得到 4GB 范围的地址空间，但是如果没有这么多的空间来映射，就自然会报错。此时可以考虑换成 64 位系统，并且配合大内存，64 位的机器上不会报出这样的错误。

(3) IOException: Too many open files

这个错误和 OOM 无关，但是通常也算是系统级别的问题，因为发生这类问题代表系统的某些设计存在问题或者某些使用已经达到了极限。该错误提示打开了太多的文件，但是并不一定是文件，有可能是本地的 Socket 打开太多，而没有被关闭，换句话说，是有太多没有关闭的套接字导致的问题。

其实这只是一些常见的内存问题，还有可能存在 JNI 的调用问题，Swap 与内存频繁交互导致系统假死等现象，JVM 的内核 Bug 导致进程直接 Crash 掉（此时还得看 Crash 日志的描述才能到官方找到 Bug，以便于做出相应版本的升级）。总之，实际场景是千奇百怪的，大家的思路不要局限于这个范畴。胖哥在这里仅仅是抛砖引玉，希望大家可以更快找到方向。

遇到 OOM 时，先大致分析原因，加上 -XX:HeapDumpOnOutOfMemoryError 参数，设定参数后，JVM 通常会在 OOM 时 dump 出一份内存的二进制文件，这份二进制文件可以用 MAT (Memory Analyzer Tool) 工具来分析（3.7.8 节会在实际案例中阐述如何使用该用具），如果要指定 dump 的目录，就使用“-XX:HeapDumpPath=目录”来指定。

3.7 常见的 Java 工具

有很多东西虽然我们知道内在，甚至可以自己写代码去探索内在，但是当面对问题的时候我们不可能马上考虑到所有的细节，此时如果有一些自动化的工具帮助我们，或许可

以让问题解决更加顺畅。

查问题、跟踪问题是在认识 Java 的基础上, 使用工具不仅可以让问题更好地解决, 也可以让我们反向了解到 Java 的一些原理。

在这里, 将用较少的篇幅来说明每个工具是用来做什么的, 让大家有个感性认识。换句话说, 胖哥不会花大量的篇幅来说明工具的“每个按钮”该如何来使用。另外, 这里也不会讲解 JVM 所有的工具, 更加不会提到 Hotspot VM 外的 JVM 工具。

不过, 为了说明一些实际价值, 胖哥会在讲解类似 MAT 诊断的实际问题中加入一个曾经帮助一位网友解决 OOM 的案例, 或许小伙伴们会有趣一点。

3.7.1 jps

Java 最简单的一种命令工具, 就是 jps 命令。在 Linux 上大家可能用过类似 `ps -ef|grep java` 的命令来看 Java 进程, jps 仅仅过滤出 Java 本身的进程以及运行的引导类 (也就是引导的 main 方法所在的类)。

```
$sudo jps -l
15068 org.jboss.Main
17576 sun.tools.jps.Jps
```

我们看到有两个 Java 进程, 其中一个是 JBoss 的进程, 启用的引导类是 org.jboss.Main; 另一个是 jps 本身的进程, 也是一个 Java 进程。这样看 Java 进程就清晰多了, 而且还可以看到启动主方法 (main) 所在的类名。

jps 命令其实也是基于 Java 实现的, 它的实现很简单, 有点像下面的代码 (如果在 Eclipse 中使用以下代码, 记得将 tools.jar 单独引入 Eclipse 项目, 因为 Eclipse 不会默认引入这个 jar 包)。

代码清单 3-20 jps 的实现模拟

```
public class JPSTest {

    public static void main(String []args) {
        List<VirtualMachineDescriptor> machineDescriptors
            = VirtualMachine.list();
        for(VirtualMachineDescriptor machineDescriptor : machineDescriptors) {
            System.out.println(machineDescriptor.id() +
                               "\t" + machineDescriptor.displayName());
        }
    }
}
```

该代码的运行结果与 jps 命令运行后的结果类似。其实代码中拿到虚拟机描述对象 machine Descriptor 后, 可以通过调用 VirtualMachine.attach(machineDescriptor)来连接对应的虚拟机, Java 的很多工具都是以类似的方式实现的。

3.7.2 jstat

jstat 命令一般用来定位问题, 是比较快速地用于初步定位的一种方式, 使用它可以看到 JVM 中每个区域的情况, 根据不同的参数, 可以看到每个区域的使用情况和比例。

例如 `jstat -gcutil <pid> 1000 2` 命令, 其中 <pid>代表进程号, 1000 代表每 1000ms 采集一次数据, 也就是每秒采集一次, 最后一个参数 2 代表总共采集两次数据, 大家可以根据自己的需要修改参数。

```
S0    S1    E    O    P    YGC    YGCT    FGC    FGCT    GCT
```

0.00	31.67	3.12	26.03	74.81	287	2.315	0	0.000	2.315
0.00	31.67	18.09	26.03	74.81	287	2.315	0	0.000	2.315

这里的 S0、S1 就代表前面提到的两个 Survivor, E 代表 Eden, O 代表 Old, P 代表 Perm, 数字代表占用区域的百分比。

YGC、FGC 分别代表 Minor GC 和 FULL GC 的次数, YGCT、FGCT 分别代表两种 GC 所花的总时间, GCT 代表所有 GC 的总时间。

jstat 后面是不是只接参数-gcutil 呢? 当然不是, 它后面所接的参数还有很多。

例如, 如果板块的大小是可以动态调整的, 则要看其当前值、最大值、最小值, 可以通过-gccapacity 来获取。类似的参数还有: -gccause、-gc、-gcnew、-gcnewcapacity、-class、-compiler、-printcompilation 等, 大家可以自行测试及查阅资料。

>> 3.7.3 jmap

jmap 后面经常接如下 3 个参数。

- ◎ jmap -heap <pid>, 用于查看。一些 OS 上的 JVM 不支持这个参数。
- ◎ jmap -histo <pid>, 用于输出活着的对象数和大小, 也是一个统计结果, 以文本形式输出, 可以将它重定向到一个文件中, 然后进行查看。
- ◎ jmap -dump:format=b,file=heap.bin <pid>, 它导出的文件和前面提到 OOM 时使用的参数-XX: HeapDumpOnOutOfMemoryError 导出的文件一样, 都是导出 JVM 内存的信息并形成二进制文件, 都使用 MAT 工具来分析, 这个命令可以随时导出文件。

jmap 的一个“坑”是当使用它来 dump 内存时, 系统会被暂停 (因为要保证拷贝中内存没有变化)。它会先做一次 FULL GC 再 dump 内存, 比 GC 的时间更长, 活着的节点越多, 时间越长 (这一点大家可以在一个大内存中系统刚启动时 dump 一次, 系统内存快满的时候 dump 一次, 分别看看时间)。另一个问题就是如果 dump 出一个超大 JVM 内存的文件, 在普通的机器上根本没办法跑起来, 因为 MAT 工具也是 Java 虚拟机, 它需要载入这些内容才能展现出来。

>> 3.7.4 jstack

jstack 命令主要用于输出线程信息, 而我们主要是看线程状态, 在 jstack 中是可以得到遍历的。通常使用 jstack -l <pid>命令, 如果未输出线程信息, 则可以尝试使用-F 参数来强制输出。我们看一下常见的线程状态。

```
"DestroyJavaVM" prio=10 tid=0x00002aaabc053000 nid=0x3add waiting on condition
[0x0000000000000000]
  java.lang.Thread.State: RUNNABLE
  Locked ownable synchronizers:
    - None
```

上面这个线程的名字是“DestroyJavaVM” (如果在程序中 new Thread(名称), 名称将会显示在这里, 所以线程名还是很有用的), 线程的优先级是 10 (也是 Java 线程的最高优先级, Java 线程的优先级分 1~10 个级别, 10 为最高级别, 但并不等价于数字越大优先级就一定越高, 因为在不同的 OS 中会有不同的线程优先级定义, JVM 会在具体的 OS 上与之映射, 这些内容在第 5 章会详细说明), tid 是该线程在 Heap 区域的 Thread 对象的地址, nid 是线程的 ID, 线程此时处于 RUNNABLE 状态。

```
"ScannerThread" daemon prio=10 tid=0x00002aaabc1ed800 nid=0x3b13 in Object.wait()
[0x000000000040a98000]
```



```
java.lang.Thread.State: WAITING (on object monitor)
  at java.lang.Object.wait(Native Method)
    - waiting on <0x0000000073002f0a0> (a java.lang.Object)
  at java.lang.Object.wait(Object.java:485)
  at org.jboss.deployment.scanner.AbstractDeploymentScanner$ScannerThread.run
(AbstractDeploymentScanner.java:216)
    - locked <0x0000000073002f0a0> (a java.lang.Object)
  Locked ownable synchronizers:
    - None
```

这个线程的名字是“ScannerThread”。

daemon 代表这是一个后台线程, 程序运行在 AbstractDeploymentScanner\$ScannerThread 类的 run() 方法中, 调用了一个对象的 wait() 方法使得线程处于 WAITING 状态 (这个对象的类型是 Object, 地址是 0x0000000073002f0a0)。

补充: 请见第一行中的 Object.wait() [0x0000000040a98000], 这个是 last_java_sp, 它是 JVM 在线程运行时分配的 frame 的 sp 指针地址。

```
"GC Daemon" daemon prio=10 tid=0x00002aaabc2f7000 nid=0x3b17 in Object.wait()
[0x0000000042b9c000]
  java.lang.Thread.State: TIMED_WAITING (on object monitor)
  at java.lang.Object.wait(Native Method)
  at sun.misc.GC$Daemon.run(GC.java:100)
    - locked <0x0000000073002cd18> (a sun.misc.GC$LatencyLock)
  Locked ownable synchronizers:
    - None
```

这个线程竟然是 GC 的后台线程。

它首先锁住了一个对象, 对象的类型是 sun.misc.GC\$LatencyLock, 在基于这个锁的基础上调用了 wait() 方法, 只是在方法中增加了时间限制 (如 wait(1000) 代表最多 1s)。这样一来, 线程的状态就是 TIME_WAITING, 此时不论是其他线程调用相应对象的 notify(), 还是设置的时间到了, 该线程都会被唤醒。

以上这些线程状态都是认为比较正常的状态, 如果大量的线程长期处于 BLOCKING 状态, 甚至可能会有 DEAD LOCK 类似的关键字出现, 这样的系统是有问题的。在第 5 章中将介绍 Java 并发的一些知识, 并详细阐述一些关于线程状态的知识。

3.7.5 jinfo

在 Java 进程运行的过程中, 我们可以通过 jmap -heap 命令来查看内存使用情况的基本信息。但一些更为详细的参数情况获取不到, 例如输出以 -XX: 开头的参数。如果想查看某些运行时参数甚至去做一些修改, 则可以使用 jinfo。

面对这样的测试, 我们没有必要单独拿出一个应用系统来做, 可以随便写个程序, 使用类似于 Eclipse 的工具来运行, 在这条代码上加断点即可 (主要是让进程不结束), 也可以使用 Thread.sleep() 来让程序不结束, 还可以用 System.in.read() 让程序在没有输入时一直阻塞。无论如何, 想要让程序运行时间足够长。

不过悲剧的是, 胖哥测试时发现, 在 Windows 64 位模式下运行 64 位的 JVM, jinfo 是不支持的, 会抛出错误: Unable to attach to 64-bit process (这个位数不是指运行 jinfo 的 JVM, 而是运行 jinfo 要连接的目标 JVM 的位数)。重装操作系统是不可能的, 解决这个问题也是有办法的, 在 Windows 64 位模式下运行 32 位的 JVM 就可以达到目的, 或者使用类似于 Eclipse 的工具将 JRE 指定为一个 32 位的 JVM 也可以达到目的。

现在来测试吧。假如用的是“断点”, 程序运行起来后就进入了断点, 不去管它了, 然

后通过 jps 查看 JVM 进程号是多少, 胖哥的机器上看到的进程 ID 是 7392。接下来, 启动一个命令窗口查看一个参数、修改一个参数, 然后放开断点看下结果, 如图 3-43 所示。

```
$jinfo -flag PermSize 7392
-XX:PermSize=12582912

$jinfo -flag PrintGCDetails 7392
-XX:-PrintGCDetails

$jinfo -flag +PrintGCDetails 7392

$jinfo -flag PrintGCDetails 7392
-XX:+PrintGCDetails
```

图 3-43 jinfo 的一个简单例子

此时测试还没完哦!

我们需要将程序中的“断点”放开, 让程序跑完 (如果使用的是 Thread.sleep(), 则需要等待时间达到; 如果使用的是 System.in.read(), 早需要在命令行按任意按键), 待程序结束时, 发现输出了 Heap 的信息, 如图 3-44 所示。



注意 如果在运行过程中不使用 jinfo 修改参数, 而是直接让程序结束, 这些信息是不会输出的。换句话说, 通过 jinfo 动态修改这个参数生效了。

Heap

```
def new generation    total 4928K, used 538K [0x04570000, 0x04ac0000, 0x09ac0000)
  eden space 4416K,   12% used [0x04570000, 0x045f6900, 0x049c0000)
  from space 512K,    0% used [0x049c0000, 0x049c0000, 0x04a40000)
  to   space 512K,    0% used [0x04a40000, 0x04a40000, 0x04ac0000)
tenured generation    total 10944K, used 0K [0x09ac0000, 0x0a570000, 0x14570000)
  the space 10944K,   0% used [0x09ac0000, 0x09ac0000, 0x09ac0200, 0x0a570000)
compacting perm gen   total 12288K, used 2107K [0x14570000, 0x15170000, 0x18570000)
  the space 12288K,  17% used [0x14570000, 0x1477ecf0, 0x1477ee00, 0x15170000)
No shared spaces configured.
```

图 3-44 最后程序会输出 GC

在 JVM 提供的几百个参数中, jinfo 可以动态修改其中的很多参数, 但是很多参数也是无法修改的。例如一些内存尺寸的参数是无法修改的 (但可以通过 jinfo 来查看这些参数值, 在图 3-43 中就可以看到 PermSize 的值 (PermGen 的大小), 只是不能修改这个参数)。

许多关于“是否启用”类的参数是可以修改的, 比如“-XX:+参数”和“-XX:-参数”。上面的例子中就是通过设置-XX:-PrintGCDetails 参数来达到输出 GC 信息的目的, 并且在设置后程序运行就生效了。

一般来讲, 对于运行时信息只能看, 不能改, 启动参数只有那么几个, 很多参数是被启动参数引导出来的参数。只有在某些参数未设置的情况下, 但又需要临时加上时可能会用到 (例如排查问题)。

3.7.6 JConsole

JConsole 是一个图形化工具, 这让很多小伙伴感觉兴奋, 因为图形化工具让许多问题变得更为直观、方便, 这也是在不断提升用户体验的过程中必然的产物。

不过，图形化工具是用基本的底层 API 来编写的软件程序。换句话说，有些时候需要更细的功能，可能图形化工具未必能满足。另外，有些 OS 并不支持图形化，在这样的 OS 上运行的 JVM 程序，如果要实现图形化的监控，则至少要有一个终端的 OS 必须是支持图形化的，然后通过某种方式连接上才能看到图形化的 UI。虽然 JVM 可以提供 JConsole 的远程访问，但很多时候连接网络都是隔离的。

本节将简单介绍 JConsole 的使用方式。在具有图形化的 OS 命令行中键入（例如 Windows 的 MS-DOS）“jconsole”，或者到 JDK 的 bin 目录下找到 JSonsole 文件，双击运行，启动后会出现如图 3-45 所示的“新建连接”界面。

可以选择一个“本地进程”，本地进程列表中的结果与前面介绍使用“jps -l”得到的结果其实是一样的。选择一个进程后，单击“连接”按钮，即可开始监控，如图 3-46 所示（远程进程在后面会有介绍，这里可以先大致看看里面有哪些内容）。

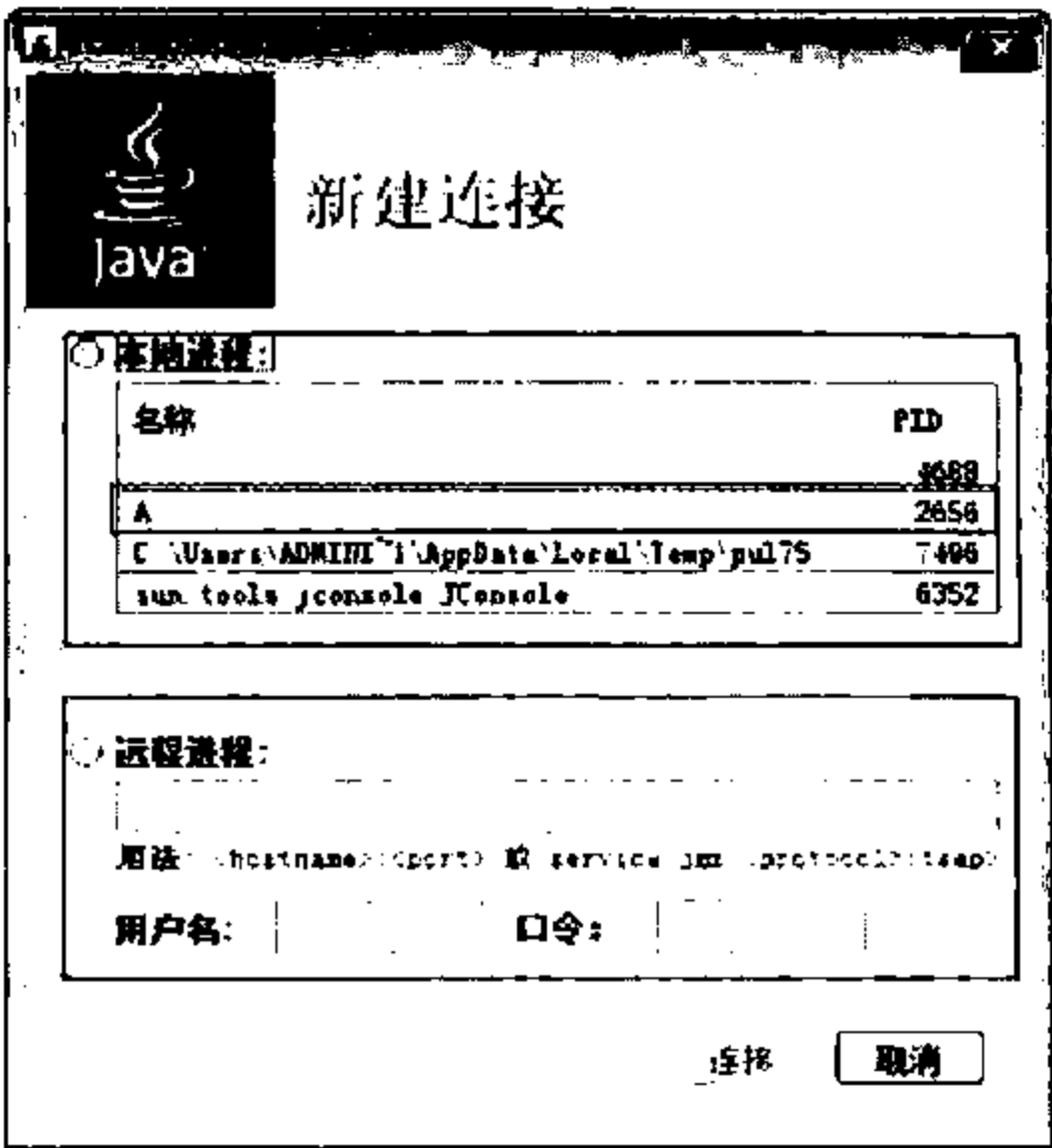


图 3-45 JConsole 的“新建连接”界面

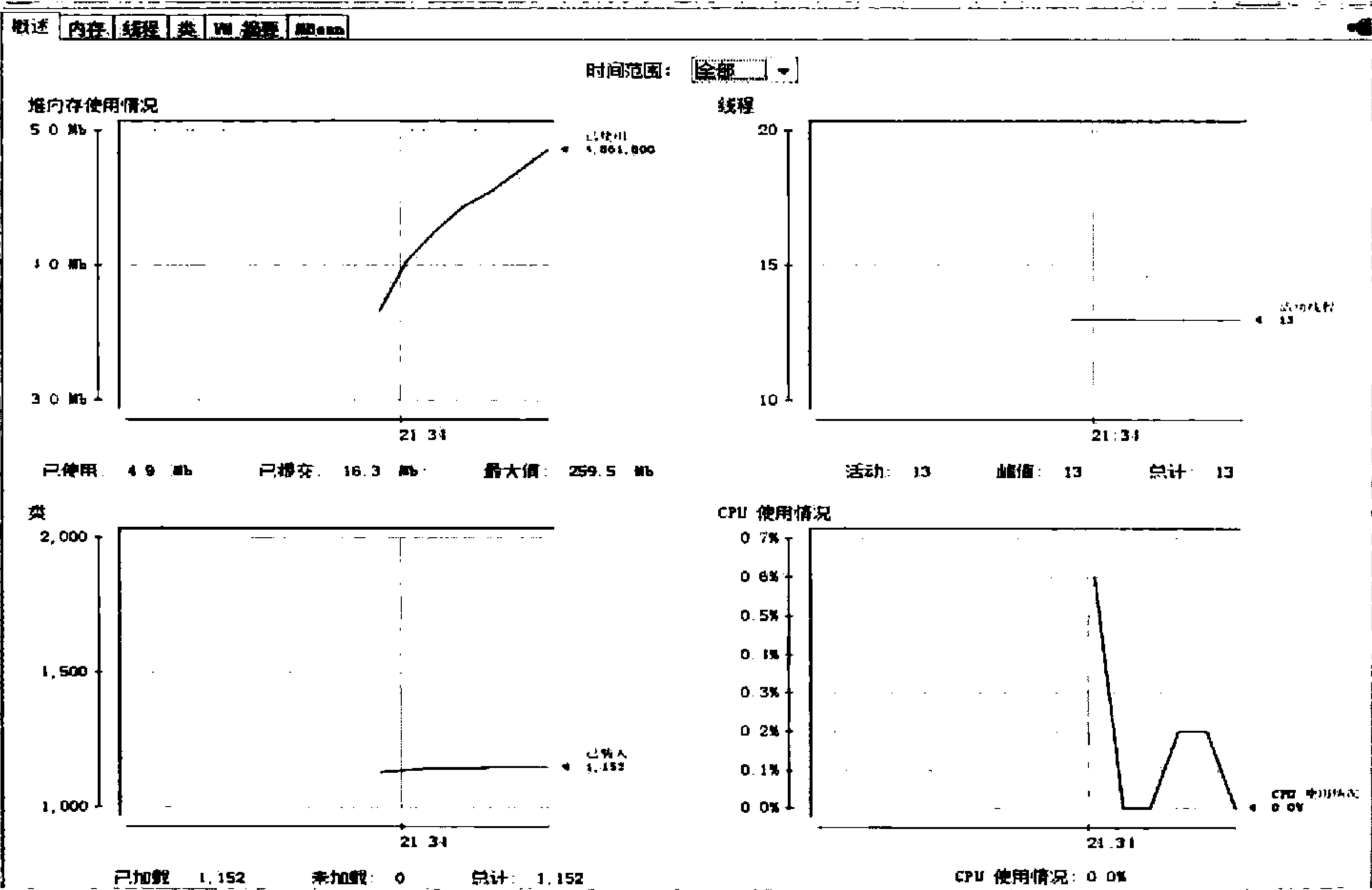


图 3-46 JConsole 的主界面

这里有概述、内存、线程、类、VM 摘要、MBean 的信息，“概述”中的内容就是对后面几个板块的大致情况展示，后面的各个板块也会有更为详细的图形展现。“VM 摘要”自然是 VM 的基本信息了，包括内存基本信息、OS 基本信息、处理器信息、GC 信息、线程信息、VM 参数、加载的 classpath 路径、环境变量中的 Path 路径、核心引导类的路径。

再回过头来看看图 3-45 中的远程进程，这个部分需要输入 hostname:port（当然输入

ip:port 是一样的道理), 还需要输入用户名和密码, 这里的用户名和密码到底是什么呢? 从什么地方可以获取到呢?

可以试想一下, 如果没有经过授权的一台机器就可以直接访问到我们的机器, 这是一件多么恐怖的事情, 所以我们要做的事情就是设置授权, 通常一旦涉及这种操作, 就会稍微有点复杂, 其实再麻烦的事情也是由简单的步骤构成的, 大家只需要记住下面 3 步就好了。

① 设置用户名和密码。

JConsole 是基于 JMX 的, 因此需要配置远程 JMX 的用户名和密码。如何配置呢? (在光盘的 src/chapter03/jmx 目录下有 JMX 的简单使用例子, 大家可以将例子作为 JMX 的入门方式, 再进一步去深入学习, 部分代码需要在 JDK 1.7 上运行。)

在 \$JAVA_HOME/jre/lib/management/ 目录下有个文件 “jmxremote.password.template” (这里的 \$JAVA_HOME 是指 Linux 下 JAVA_HOME 环境变量, 在 Windows 下改为 %JAVA_HOME%), 这个文件是一个配置 password 的模板文件, 此时在同一层目录下将这个文件拷贝一份, 并将名称改为 “jmxremote.password”, 即去掉 .template 后缀, 然后打开文件, 最后两行为:

```
# monitorRole QED
# controlRole R&D
```

这是两个 Java 默认的 jmx 用户及其所对应的密码! 这两行默认是被注释掉的, 将注释放开, 用户名及密码则会生效。

这两个用户的密码分别是 QED 和 R&D, 这是两个默认密码, 可以在这个文件中修改。想要增加用户怎么办?

在同一层目录下有一个文件 “jmxremote.access”, 它里面已经有两个账号, 其中一个为 readonly; 另一个为 readwrite, 可以在这里加上你自己想设置的用户名信息, 不过密码还是要到 jmxremote.password 文件中添加。

② 在启动 JVM 进程的启动参数上需要增加以下内容。

```
-Dcom.sun.management.jmxremote
-Dcom.sun.management.jmxremote.port=9000
-Dcom.sun.management.jmxremote.ssl=false
```

这里的端口配置为 9000, 大家可以自行修改。参数也不一定放在 JVM 启动参数中, 也可以在 \$JAVA_HOME/jre/lib/management/ 下的 management.properties 文件中设置。内部的设置都是注释掉的, 其也代表了默认值。

这样设置后, 服务器端的参数和配置就准备完成了。需要注意的是, 如果服务器端的进程已经在运行, 则需要重启一次, 才会生效。

③ 客户端启动 JConsole, 输入用户名和密码, 以及远程主机的 ip:port (当然, 确保网络畅通是必然的, 按照例子中的启动参数, 端口自然是 9000), 单击 “连接” 按钮就可以连接上远程的 JVM 进程进行监控了。值得注意的是, 用户名和密码都要区分大小写。

这样操作通常会在启动进程时遇到问题: 必须限制口令文件读取访问 \$JAVA_HOME/jre/lib/management/jmxremote.password。

这提示什么呢? 提示文件必须限定为只读文件。其实这个问题也好理解, 就是密码确实不能随便改。

将文件设置为只读后, 还是有可能继续报错, 就是 JVM 进程有可能使用了 root 这类账号来启动 (Windows 上使用了 Administrators 账号来运行), 主要原因是这个账号的权限太大。

如果是在 Linux 下使用了 root 账号,则可以在修改完用户密码后将 jmxremote.password 文件的属性设置为 000,也就是 `chmod 000 jmxremote.password`,这样启动就没问题了。

如果是在 Windows 下使用了 Administrator 账号,则有两种方式修改,其中一种是命令行方式;另一种是图形操作方式。命令行方式进行如下操作:

```
D:\jdk1.6.0\jre\lib\management>cacls jmxremote.password /P Administrator:R
是否确定 (Y/N)?Y
```

通过这两个步骤就可以限制 Administrator 账号对这个文件的修改权限了,当然你作为管理员是有权限将其修改回来的。

对于图形操作方式,在 Windows 中右键单击 jmxremote.password 文件,选择“属性”,然后选择“安全”,如图 3-47 所示。

选择 Administrator 账户后,单击“编辑”按钮,只保留“读取”相关的权限即可。这样可以达到和命令操作同样的效果,上面描述的问题就解决了。

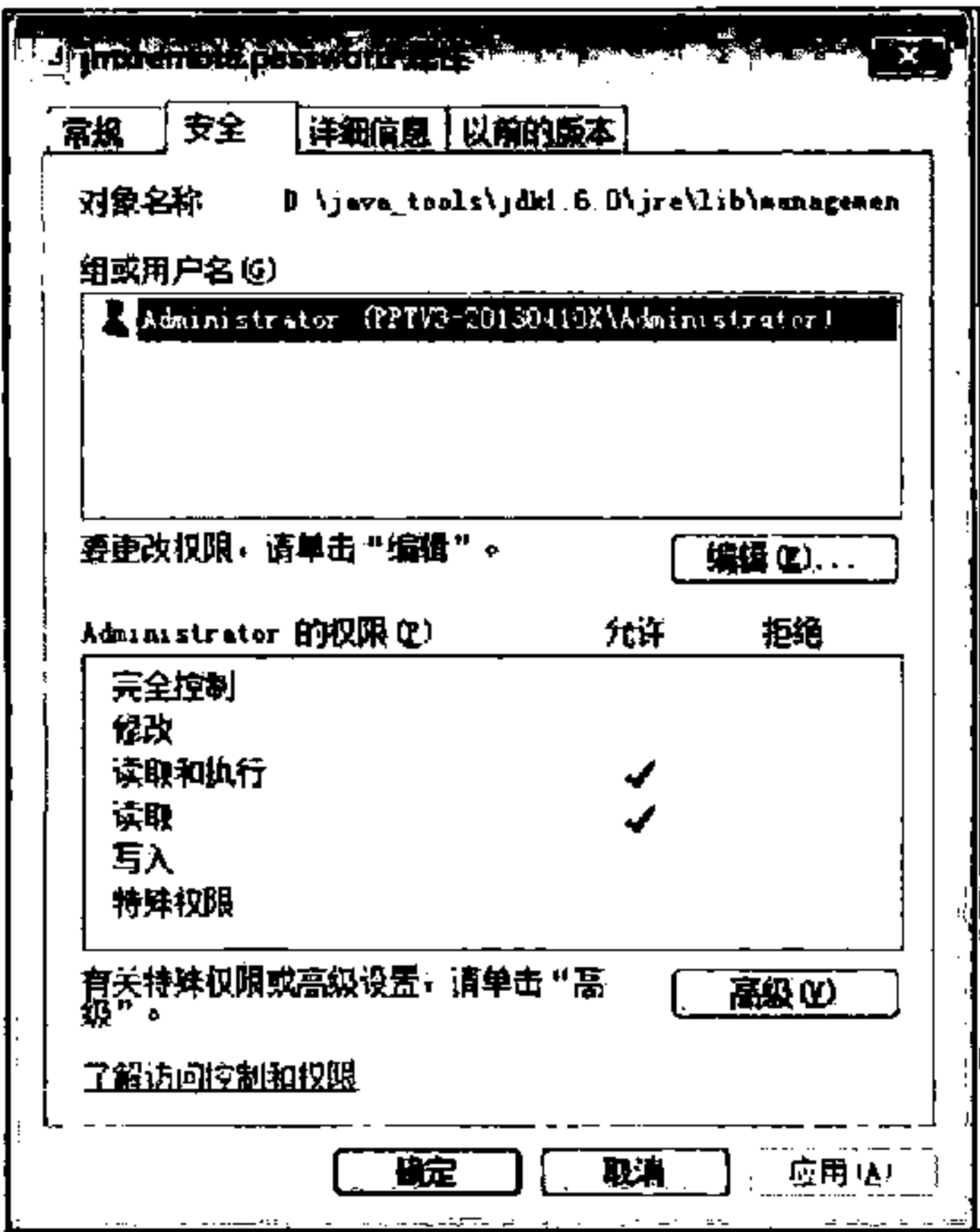


图 3-47 在 Windows 中设置 Administrator 的只读权限

疑惑: 远程连接不想用用户名和密码,行不行?
答曰: 可以的,只要你认为它是安全的就行,通常在公司这都不是安全的(因为世界上有一种攻击方式叫肉机,即使在内网也有可能被攻击),如果是自己玩玩则完全可以。
在前面的设置中有一个参数是: `-Dcom.sun.management.jmxremote.authenticate=true`,这个参数默认值就是 `true`,所以胖哥没去设置它,也就是默认要做用户名和密码的校验,当不想做用户名和密码的校验时,将这个参数值设置为 `false` 即可。
不过,当设置这个参数时,用户名和密码文件不再生效,从客户端连接时,只需要通过 `ip:port` 就可以连接,根本不用输入用户名和密码,所以一定要注意安全问题。

在上面提到的方式中,是要将 jmxremote.password 文件拷贝到指定的目录,将这个文件放在其他目录下,也是可以运行的,在启动参数中增加 `-Dcom.sun.management.jmxremote.password.file=`来修改 password 文件的路径,也可以在 `$JAVA_HOME/jre/lib/management/`目录下修改 `management.properties` 文件,相关的参数修改会全局生效。

在 JConsole 工具中,例如对每个板块的监控还不是那么理想,功能像是“东一块、西

一块”拼出来的。下一节将介绍另一个图形化工具，也是目前在 Hotspot VM 上使用非常广泛的一个工具，那就是 Visual VM。

3.7.7 Visual VM

大家看到，很多工具零零散散，我们迫切地希望有一个可以集成非常多的工具平台，Visual VM 就是一个非常不错的选择，以前这个工具是第三方提供的，目前 JDK 中会自带这个工具（在高版本的 JVM 的 bin 目录下就有 jvisualvm 文件）。也可以到 <http://visualvm.java.net/> 来下载，其中包含了它的使用说明。

要启动 Visual VM，也需要 OS 支持图形化界面，在 Linux 上就直接运行对应的 ./visualvm（在 Windows 下运行 visualvm.exe 即可），某些版本的 Visual VM 启动时还需要指定 jdkhome 信息，用来指定 JDK 的目录。打开 Visual VM，界面中的左侧目录树如图 3-48 所示。

从图 3-48 中发现，本地的 JVM 进程 Visual VM 会自动检测出来。单击一个本地进程，即可看到该进程的监控信息。不过，此时发现默认的监控信息并不多，基本功能有点像 JConsole，并没有包含更细的内容。我们希望它可以有更加强大的功能，此时需要安装插件，这是 Visual VM 的一个优势。

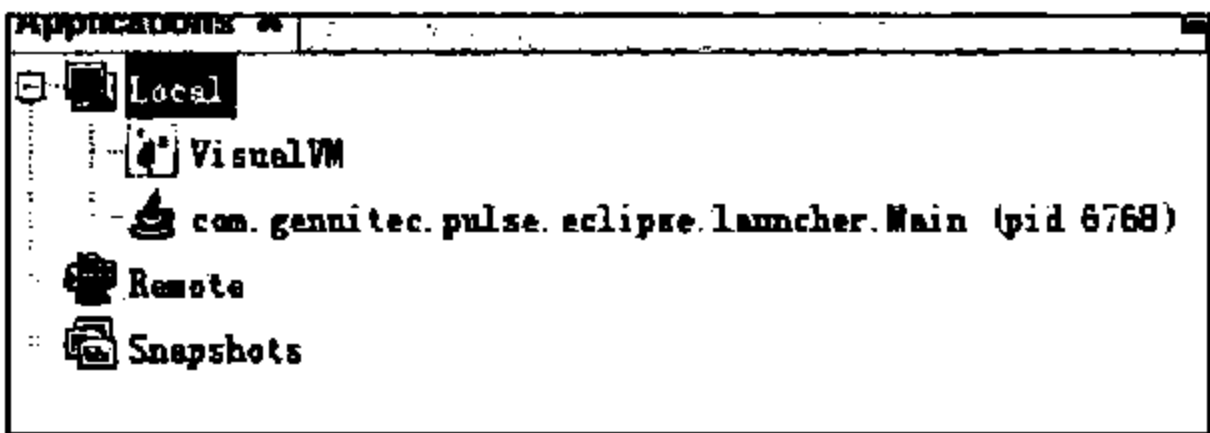


图 3-48 Visual VM 左侧目录树

Visual VM 自带了安装插件功能，只需要选择即可一键安装。单击菜单 “Tools” -> “Plugins”，可以打开安装插件窗口，如图 3-49 所示。

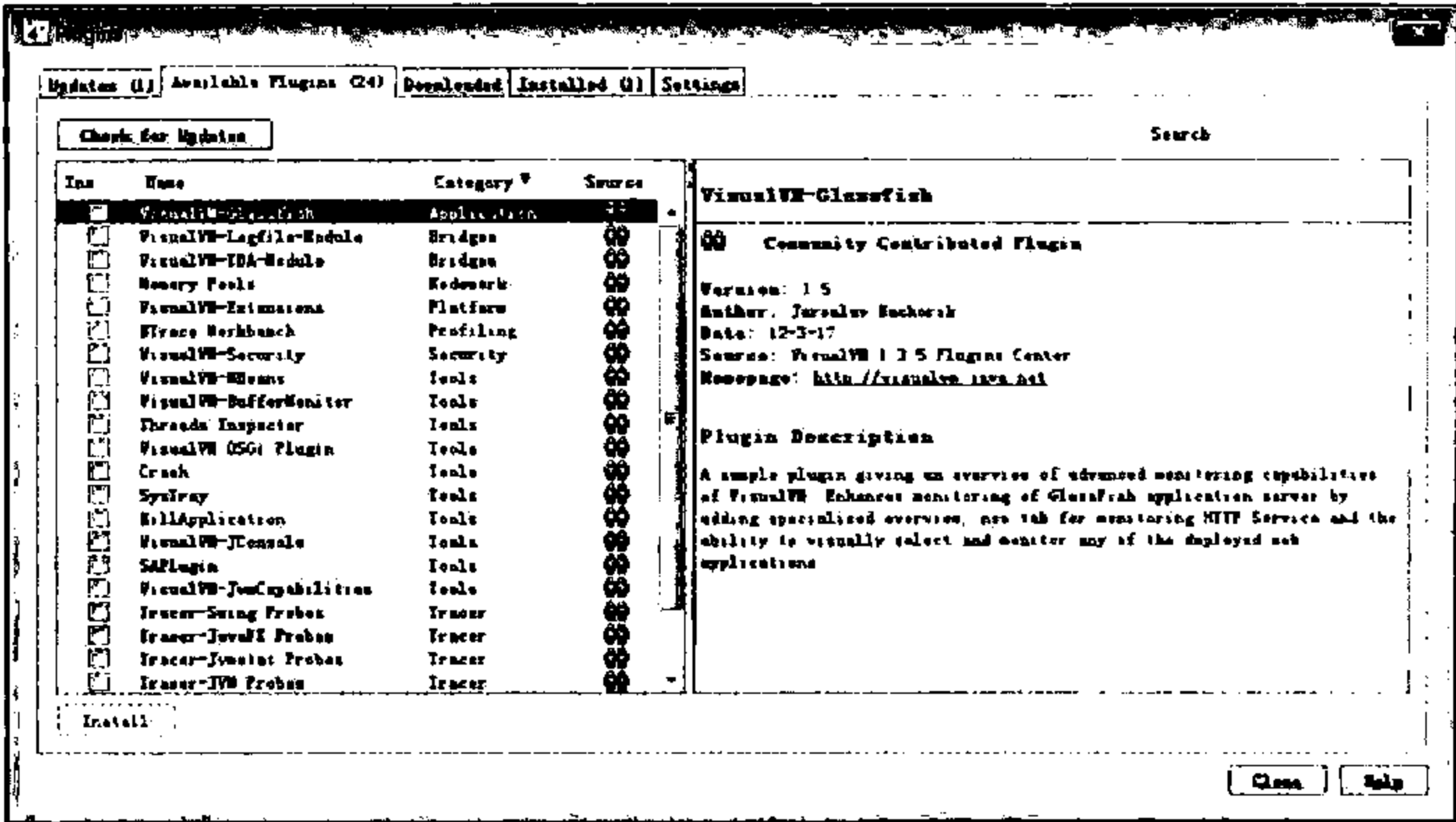


图 3-49 Visual VM 的安装插件窗口

我们通常只需要关心 “Available Plugins” 和 “Installed” 选项卡。“Available Plugins” 是指可以安装哪些插件，“Installed” 是指已经安装了哪些插件。有的同学运气不好，在 “Available Plugins” 中一个插件也找不到，那有可能是因为 Visual VM 默认提供的插件下载路径已经过时，也可能你正好“悲剧”地遇上了那一天 Plugins 服务器宕机了，导致无法找到插件列表。不过，插件地址通常不止一个，在 “Settings” 选项卡中可以修改这项设置，

如图 3-50 所示。

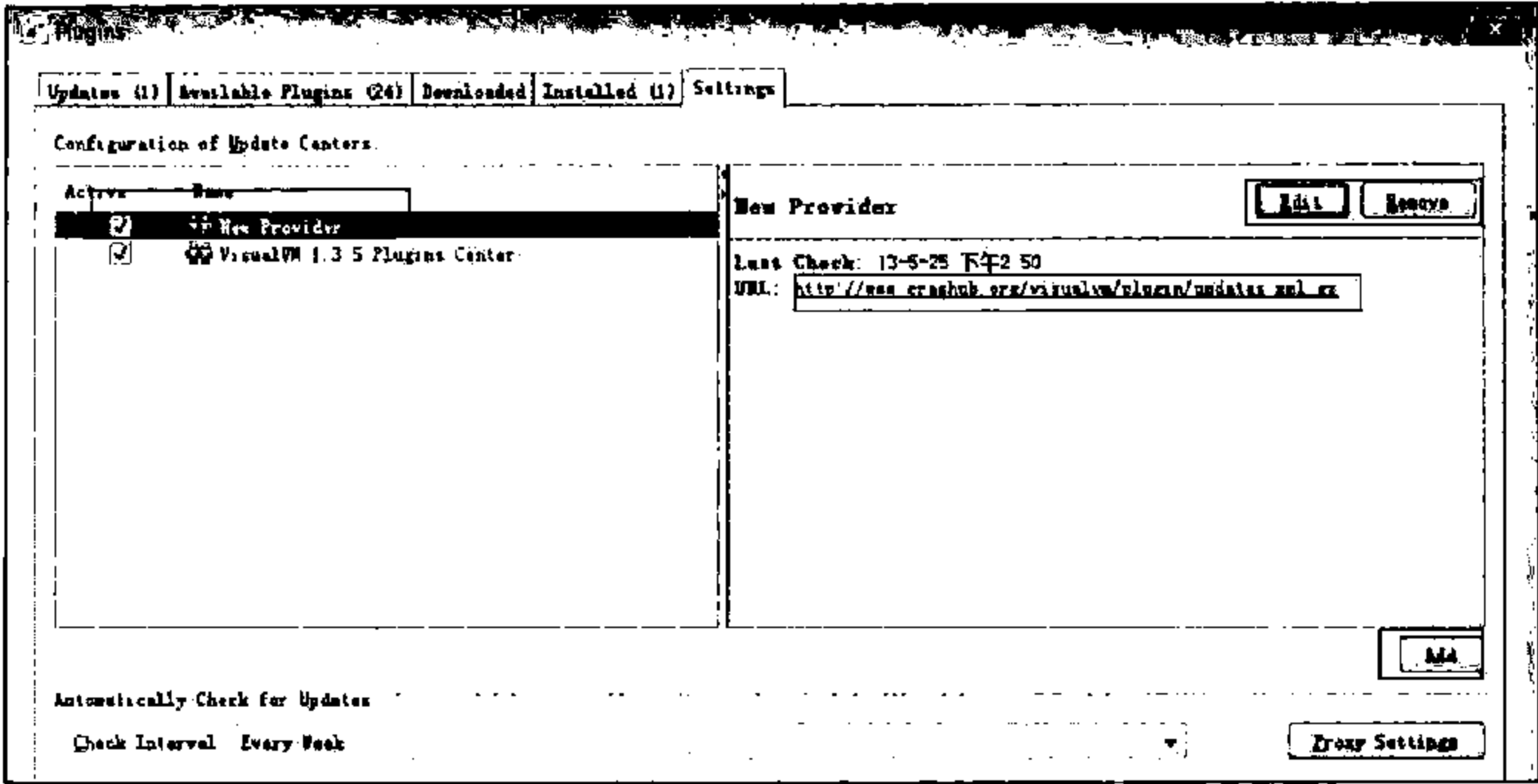


图 3-50 Visual VM 中如果没有插件就自己加地址

在有插件的情况下，选中对应的插件，然后单击“Install”按钮即可安装。当然，也可以自己下载一个插件，安装到 Visual VM 的指定目录中。在这里安装了一个 Visual GC 插件，此时来看看 JVM 板块，是不是感觉非常的清晰，如图 3-51 所示。

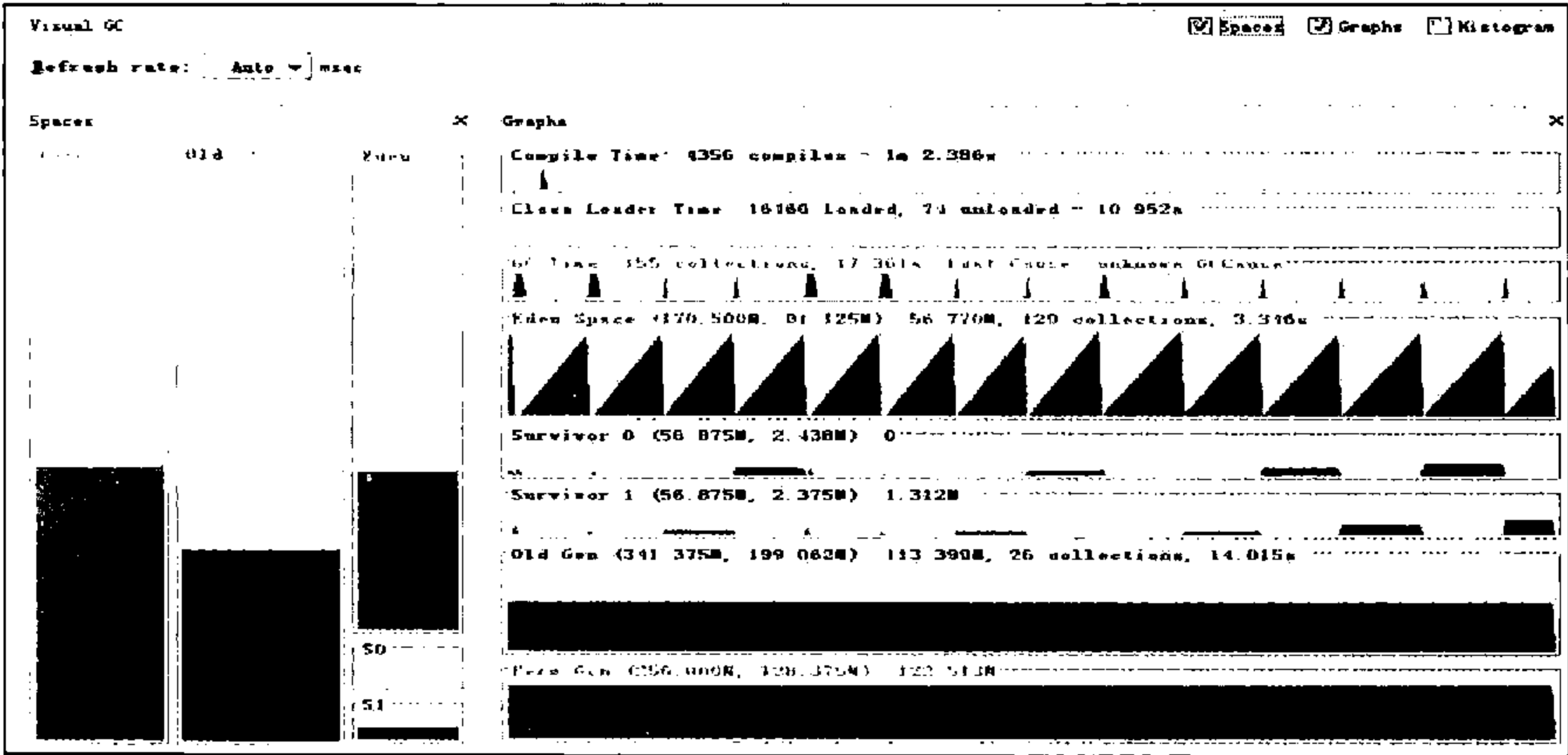


图 3-51 Visual GC 插件对 JVM 的内存监控

从图 3-51 来看，又一次印证了前面介绍到的 JVM 版本划分，每个板块的展现方式很明显。

同样的，Visual VM 也可以设置远程监控，但是它的设置并不像 JConsole 那么麻烦，它需要一个对 policy 文件的说明（如下面的例子），在对应的服务器上有一个 jstatd 的 RMI 代理即可（就是在对应的服务器上开启一个小的 JVM 进程，允许外部访问，通过它代理访问本地机器的 JVM 进程）。

```
grant codebase "file:${java.home}/../lib/tools.jar" {
    permission java.security.AllPermission;
};
```

文件编写好以后，可以放在任意目录下，例如为文件取名“visualvm.remote.policy”，然后放在/ux/security/目录下。接下来我们需要运行 jstatd 工具，运行的命令是：

```
jstatd -J-Djava.security.policy=/ux/security/visualvm.remote.policy -p 9000
```

如果运行该命令没有报错，那么就在对应的服务器上开启了 9000 端口，可以接受来自远程的 Visual VM 工具的访问了。

Visual VM 在本机上如果不做虚拟机则模拟不了远程，Visual VM 会自动检测是不是本机，而 JConsole 可以在本地模拟远程。

接下来回到图 3-48 中，启动 Visual VM 后，左侧目录树中还有一个“Remote”，右键单击“Remote”，选择“Add Remote Host”，会弹出一个选项窗口，在上面输入 Host name 后，单击“Advanced Settings”按钮，可以设定端口号、刷新频率。默认的端口号为 1099，这里需要将端口号改为 9000，然后根据自己的需要修改刷新频率。

这里的显示名（Display name）方便自己管理和维护，例如可以取名为“胖哥牛逼服务器 1 号”，如图 3-52 所示。

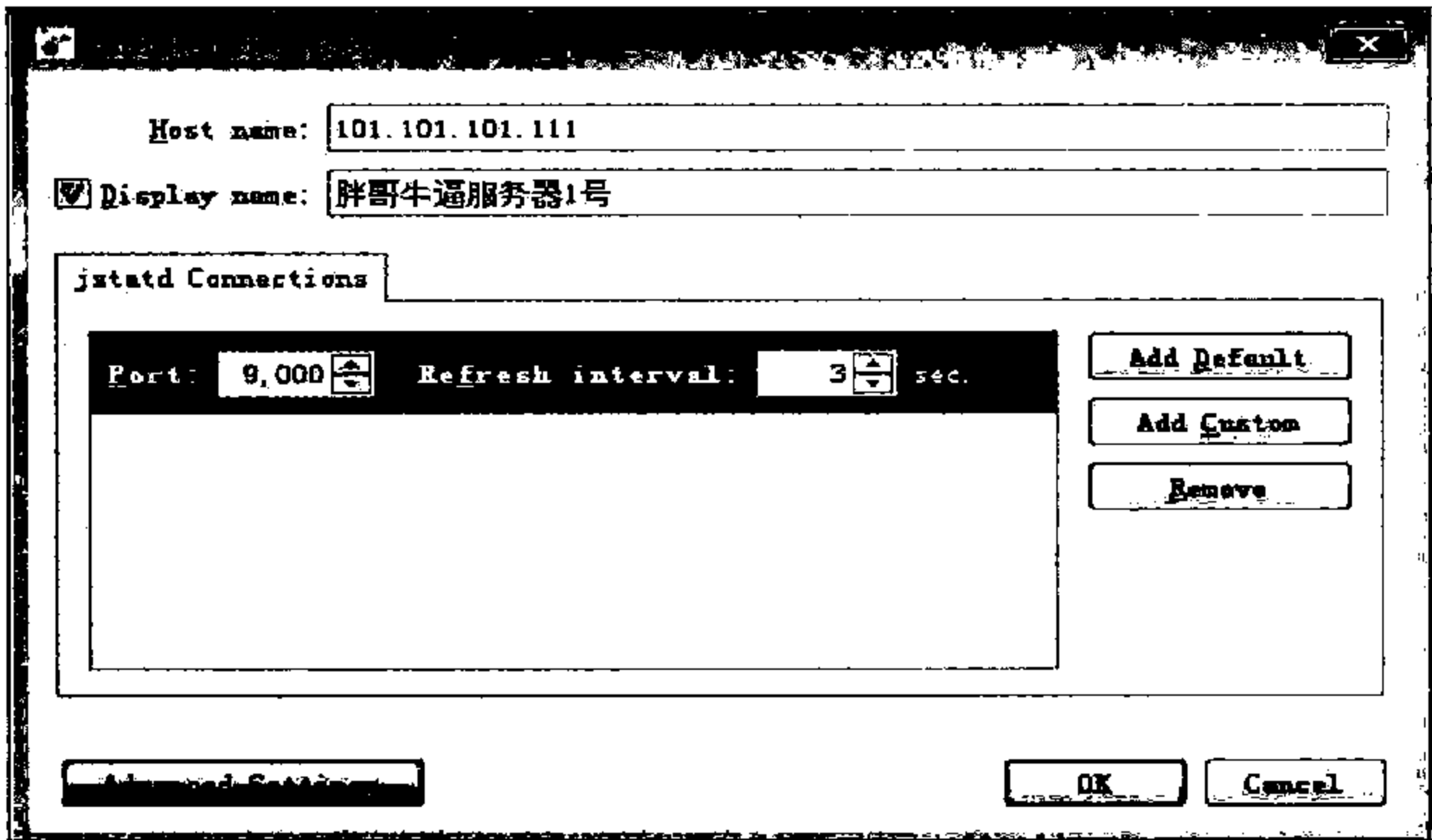


图 3-52 Visual VM 中配置一个远程服务器

配置好以后，在网络畅通的情况下，就可以连接上对应的服务器了，连接上后，会在左侧目录树中展现每个实例下的 JVM 节点，此时展现出来的也可能是多个 JVM 进程。单击每个 JVM 节点，就可以像本地进程那样监控对应的 JVM 进程状况了，如图 3-53 所示。

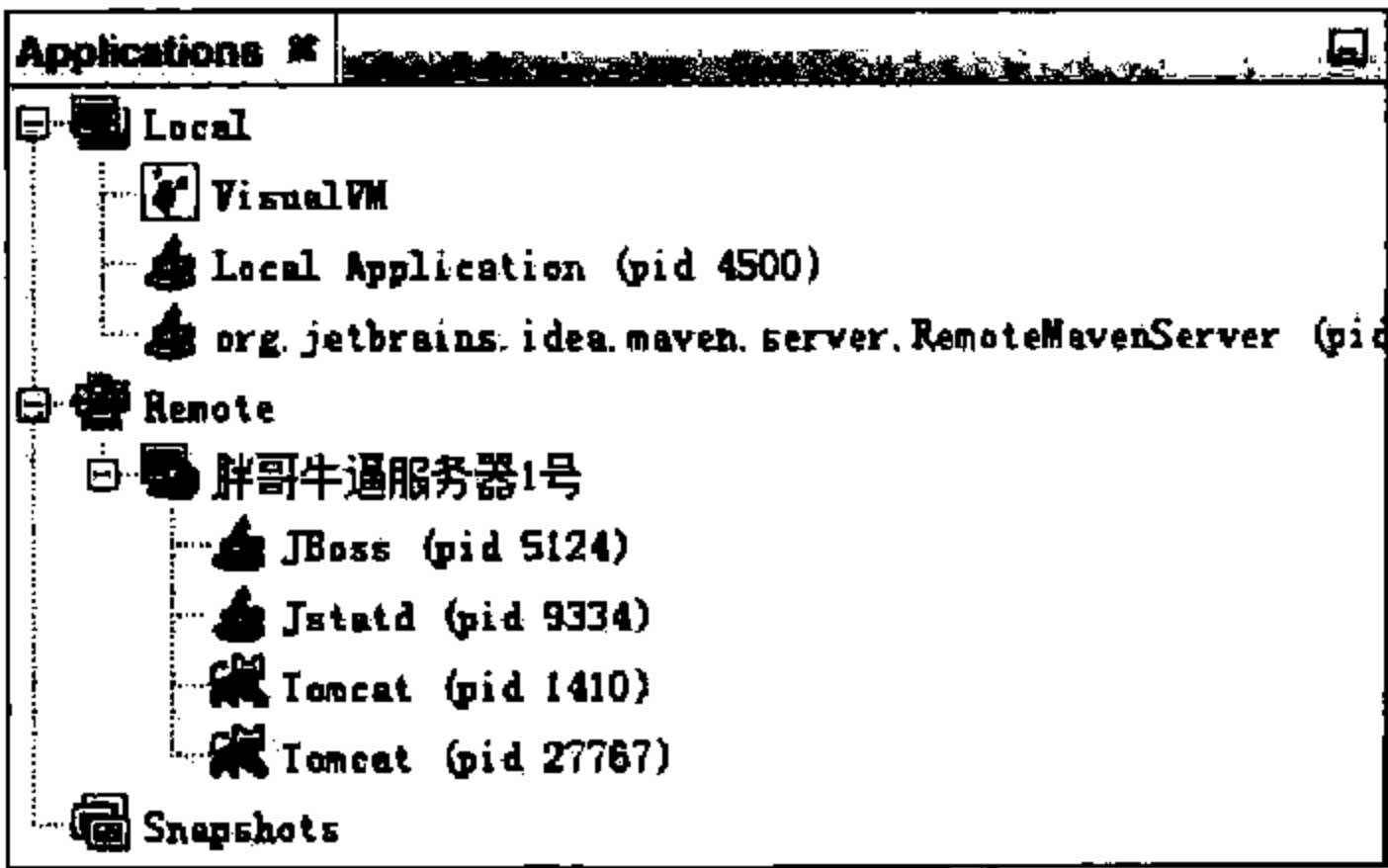


图 3-53 远程监控 JVM 进程状况

3.7.8 MAT (Memory Analyzer Tool)

MAT 这个工具在前文中多次提到，它是 Java 的内存分析工具，它分析的是 JVM 内存的 dump 文件，此文件可以是通过 jmap -dump:format=b,file=xxx.bin <pid>将内存 dump 出来的文件，也可以是通过设置-XX:+HeapDumpOnOutOfMemoryError 参数，在 JVM 出现

OOM 时自动输出的文件,还可以是不太常用的两个参数-XX:+HeapDumpBeforeFullGC 和 -XX:+HeapDumpAfterFullGC 让系统在 FULL GC 之前或之后输出的内存的 dump 文件,或者是在 Visual VM 中直接对监控的 JVM 进程单击右键,选择“Heap Dump”,将 JVM 的内存 dump 出来的文件。

内存的 dump 文件是一个二进制类型的文件,我们无法读取出来,不过它内在本身的一些结构提供了相应的数据规范,因此许多工具可以将文件解析为可读的数据格式, MAT 工具就是可以解析这种文件的工具。MAT 工具需要大家到网上去下载,地址为 <http://www.eclipse.org/mat/downloads.php>,根据 OS 不同选择不同的版本即可。

接下来,胖哥给大家举个例子,通过这个例子来说明 MAT 的使用。

一天“TEND”同学找到胖哥,说自己的系统 OOM 了,是什么原因导致的?是不是内存泄漏啊?

胖哥无奈地回答他:胖哥不是答题器,不会出现一个 OOM 就知道原因的,要根据实际情况来决定。于是开始细化问题了,“TEND”同学自己会用 Visual VM,他描述了通过 Visual VM 工具观察到的内存增长过程。根据输出的 OOM 信息是 Heap 区域内存溢出,Visual VM 上显示的结果是 Old 满了且不断在做 GC,每次 GC 完成后没有多少内存被释放掉,不一会内存又满了。胖哥初步判定是内存泄漏的基本表象,至于细节还得再看。

于是,胖哥希望这位同学提供一份 dump 文件,从这份 dump 文件来看,使用量才 30MB,根本不可能内存溢出。胖哥当时也蒙了,这怎么可能(除非是局部变量脱离作用域的情况),最后搞了半天发现 dump 出来的是刚刚重启后不久的 JVM 内存,我们带着纠结只好等待下一次 OOM。

果然,随着系统的运行 OOM 再一次出现了,不过这一次这位同学才发现:HeapDumpOnOutOfMemoryError 参数处于关闭状态。

胖哥当时建议他用 jmap 来 dump 内存,不过一连串的问题就会出现,jmap 会导致系统暂停,无法服务,而“TEND”同学的服务器没做集群。胖哥跟他说:首先要知道,宕机本身也是无法服务的,重启也需要花时间,而且永远也解决不了问题,既然这样,为何不先做 dump 后再重启呢?

“TEND”同学终于带着害怕的心理将文件 dump 出来了,终于有点希望了,但是又遇到了新的问题。

dump 文件出来后,要将文件传送出来,但是这位同学说他们的服务器网速也是“上个世纪的网速”,奇慢无比,dump 文件太大,传递不知道几天几夜。

胖哥也只好用远程工具来看了。dump 文件有 1.2GB,也不算太大,不过相对这位同学的整个 JVM 进程来讲,这算是比较大的。

“TEND”同学问胖哥为何是 1.2GB?既然 OOM 了为什么不是设置的 JVM 最大值 1.5GB 呢?

因为在 1.5GB 的内存中有一部分是 Young 空间的,在这位同学的 JVM 配置中,去掉 Young 空间,Old 区域是 1.05GB 左右,此时 Old 区域已经装满对象,Young 空间还存在一些活着的对象,所以是 1.2GB 左右。明白了这个数字是怎么来的,接下来就来开始做 dump 分析吧。

MAT 工具也是用 Java 写的,因此它也运行在 Java 虚拟机上,二进制文件是存放内存数据的,要将其加载,内存也得够用才行。在默认情况下, MAT 工具是 1024MB 的内存空

间，无法打开 1.2GB 的 dump 文件，因此需要在启动文件 “MemoryAnalyzer.ini” 中修改内存设置。文件终于打开了，打开后看到第一幕就大致明白了，如图 3-54 所示。

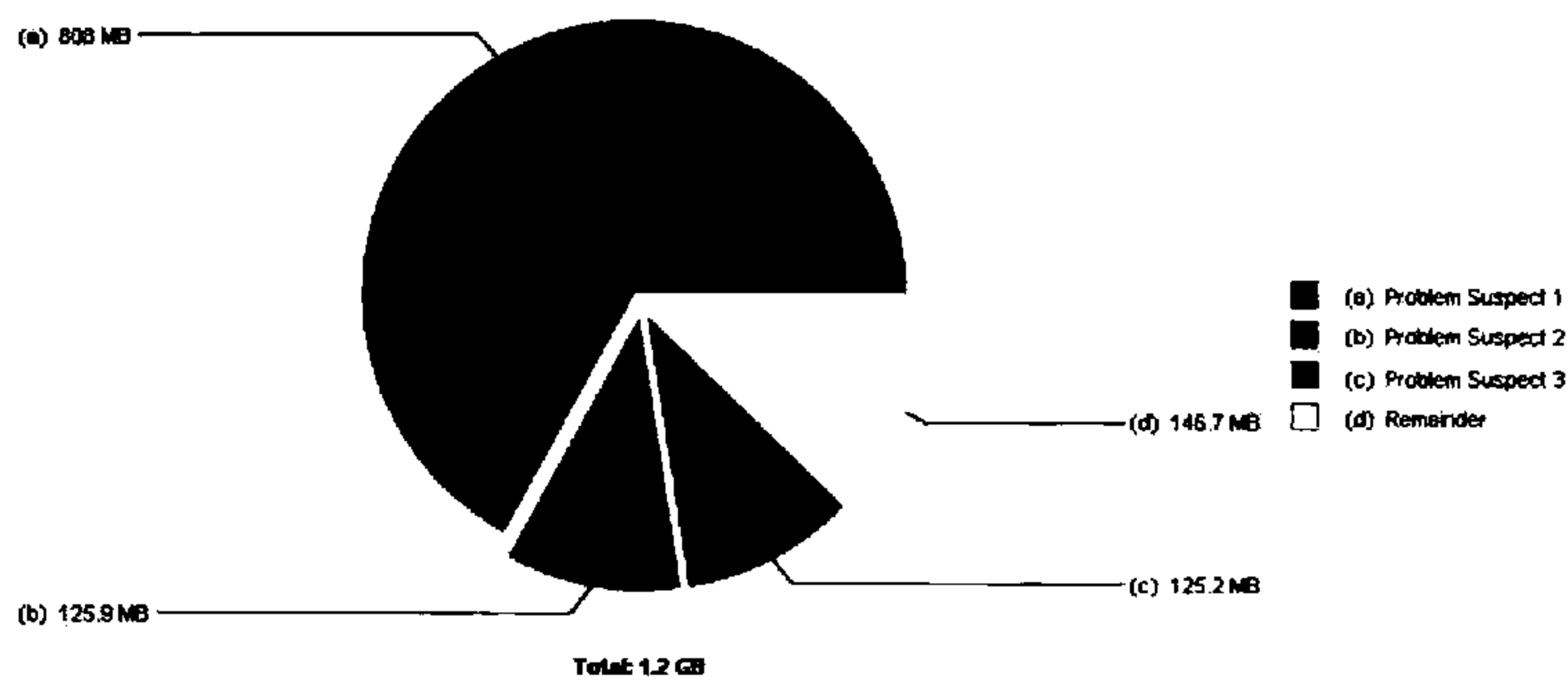


图 3-54 打开 dump 文件后一块大内存让我们震惊了

看到了这有一块十分大的内存区域没有被 GC 掉，对于问题的判定开始充满了希望。根据前面所学到的一些基础对象结构来理解，这类对象通常应该是类似于集合类或数组一样的对象，才有可能会占用十分大的内存区域。再看看下面一些 Suspect 的介绍，如图 3-55 所示。

▼ • Problem Suspect 1

One instance of "com.jrads.ip.IPSeeker" loaded by "sun.misc.Launcher\$AppClassLoader @ 0x69f2320" occupies 847,213,744 (67.01%) bytes. The memory is accumulated in one instance of "java.util.Hashtable\$Entry[]" loaded by "<system class loader>".

Keywords
sun.misc.Launcher\$AppClassLoader @ 0x69f2320
java.util.Hashtable\$Entry[]
com.jrads.ip.IPSeeker

[Details »](#)

图 3-55 Suspect 的介绍

这里的问题说明 IPSeeker 占用的 67%的空间内部有个 HashTable，就是它了，现在来跟踪它，怎么跟踪呢？

可以在图 3-54 中最大的那块内存上单击左键-> “List objects” -> “with outgoing references”，列举出对应的对象及它所引用的对象。对应的操作还有 “with incoming references”，表示指向该对象的对象。也可以单击工具栏中的 “🔍” 按钮来展现对象的引用树关系，最大的对象会在最前面列举出来。将 “TEND” 同学的 dump 文件打开后，如图 3-56 所示。

打开 “Heap Dump View” 窗口后也会得到同样的结果，这个结果印证了前面对于 IPSeeker 很大的猜测。接下来就要看看代码 IPSeeker 到底做了什么？为何会这么大？

“TEND” 同学给了调用 IPSeeker 的代码，其实很简单，而且也可以看得出来 IPSeeker 是一个单例，大致如下：

```
private IPSeeker seeker = IPSeeker.getInstance();
public RegioInfo getFromIp(String name) {
    seeker.getCountry(name)
}
```


com.jrads.ip.IPSeeker @ 0x5e0ebff0	56	847,213,744	67.01%
java.util.Hashtable @ 0x6e0ac28	40	847,213,360	67.01%
java.util.Hashtable\$Entry[6291455] @ 0x2c843780	25,165,820	847,213,320	67.01%
java.util.Hashtable\$Entry @ 0x48de1840	24	1,864	0.00%
java.util.Hashtable\$Entry @ 0x47f7e948	24	1,624	0.00%
java.util.Hashtable\$Entry @ 0x43fba900	24	1,616	0.00%
java.util.Hashtable\$Entry @ 0x47d476c8	24	1,616	0.00%
java.util.Hashtable\$Entry @ 0x399f2540	24	1,536	0.00%
java.util.Hashtable\$Entry @ 0x432aac60	24	1,464	0.00%
java.util.Hashtable\$Entry @ 0x44416e80	24	1,464	0.00%
java.util.Hashtable\$Entry @ 0x47423f20	24	1,464	0.00%
java.util.Hashtable\$Entry @ 0x4164c310	24	1,456	0.00%
java.util.Hashtable\$Entry @ 0x453b0d68	24	1,456	0.00%
java.util.Hashtable\$Entry @ 0x458ff380	24	1,448	0.00%
java.util.Hashtable\$Entry @ 0x3f3f0858	24	1,440	0.00%
java.util.Hashtable\$Entry @ 0x42896178	24	1,440	0.00%
java.util.Hashtable\$Entry @ 0x42b2b7c8	24	1,440	0.00%
java.util.Hashtable\$Entry @ 0x45588550	24	1,440	0.00%
java.util.Hashtable\$Entry @ 0x31e55a48	24	1,432	0.00%
java.util.Hashtable\$Entry @ 0x43696d68	24	1,432	0.00%
java.util.Hashtable\$Entry @ 0x4389aac8	24	1,432	0.00%
java.util.Hashtable\$Entry @ 0x449ed4a0	24	1,432	0.00%
java.util.Hashtable\$Entry @ 0x499149d8	24	1,432	0.00%
java.util.Hashtable\$Entry @ 0x34fd1078	24	1,424	0.00%
java.util.Hashtable\$Entry @ 0x413b25f8	24	1,424	0.00%
java.util.Hashtable\$Entry @ 0x42a04060	24	1,424	0.00%
java.util.Hashtable\$Entry @ 0x450ca5f0	24	1,424	0.00%
java.util.Hashtable\$Entry @ 0x45111ae0	24	1,424	0.00%
Total: 25 of 2,998,455 entries			

图 3-56 发现有 2 998 455 个 Entry，罪魁祸首就是它了

这是一段十分简单的代码，看上去没有问题，只好去看看 IPSeeker 的源码了（下面的这段源码与实际的有点区别，它使用的不是 Hashtable 而是 HashMap，但总体意思相同）。打开源码，如图 3-57 所示。

注：IPSeeker 不会被回收

```
// 单一模式实例
private static IPSeeker instance = new IPSeeker();

private Map<String, IPLocation> ipCache;

private IPSeeker() {
    ipCache = new HashMap<String, IPLocation>();
}

public String getCountry(String ip) {
    return getCountry(Util.getIpByteArrayFromString(ip));
}

public String getCountry(byte[] ip) {
    // 检查ip地址文件是否正常
    if(ipFile == null)
        return bad_ip_file;
    // 保存ip, 转换ip字节数组为字符串形式
    String ipStr = Util.getIpStringFromBytes(ip);
    // 先检查cache中是否已经包含有这个ip的结果, 没有再搜索文件
    if(ipCache.containsKey(ipStr)) {
        IPLocation ipLoc = ipCache.get(ipStr);
        return ipLoc.country;
    } else {
        IPLocation ipLoc = getIPLocation(ip);
        ipCache.put(ipStr, ipLoc.getCopy());
        return ipLoc.country;
    }
}
```

在这里被定义为HashMap

最终在这里写入了Cache

图 3-57 IPSeeker 的源码

代码的意思很简单，首先是看 Hash 表中是否存在这个 IP，如果不存在则放入 Hash 表中。似乎这是很常见的错误，事情也越来越明了，不过可以冷静下来先做分析。

首先可以百分百确定的一点是，IPSeeker 这个引用是不会被回收的，因为是 static 类型，因此它内部的 HashMap 也不会被回收（除非被替换，但在代码中并没有看到）。getCountry()

方法最终是通过 Map 来 Cache 数据的，当无限多的 IP 写入时就会导致这个 Map 非常大，最终将 JVM 撑爆。就像这个例子中的一样，Map 中的 Entry 个数接近 300 万个。

也许这种问题不能怪代码的作者，他可能是好心做了坏事，他希望通过本地 Cache 数据来提升性能，但是没想到有如此多的 IP 需要去 Cache。所以使用者也应该小心了，这也再一次告知使用别人框架的同学，不要认为所有的东西都是“拿来主义”，一定要知道代码中可能存在潜藏着的场景应用，知道了内在才能将它用得更好。



注意 如果通过 HashMap 来 Cache 数据，在多线程的系统中是不靠谱的！

它不仅存在并发可见性问题，而且可能存在由于内在的判定修改动作导致的 RaceCondition 问题，容易出现丢数据现象，甚至在极端情况下可能会导致死循环。另外，在多线程对这类非线程安全的集合类同时做写时可能还会发生 java.util.Concurrent ModificationException（这个错大家可能在单线程中循环删除元素时也见到过，可以使用迭代器解决，但这并不是同一个问题，多线程中将更加复杂一些。翻看源码查看这个异常抛出的因素，便知道多线程并发为何会导致这样的问题了）。

关于多线程并发操作的问题，在第 5 章的并发编程中大家会看到更多。

我们应该如何去修改呢？这个问题必须处理，否则后患无穷。

胖哥的建议是要么换 API，要么修改源码或 Override 相关方法。不论使用哪一种方式，目的都是要让内存的使用尽量变得“小而美”，而不是我们看到的“大而丑”。

首先考虑：要是不做 Cache，每次计算需要花多长时间？系统是否可以接受？若不能接受能否换种方式，将它存在文件、数据库、分布式缓存中可否？或者能否在内存中使用 LRU 替换规则？只要系统可以接受这种方式，而且相对结果的代价不是很高，那么就是值得的。

现代的 JVM 本身并不是一个适合存储大量数据的地方，将大量数据 Cache 在内存中其实是很古老的认识。我们不需要在这些细节上纠结，现在可选的技术手段非常多，要根据场景来选择。

最后，胖哥建议这位同学给老板说说，一定要将网络做好，将集群做起来，不要每次影响用户使用时重启，那样会让用户十分没有安全感。

胖哥的这个例子也许很简单，也很明显，你通过这个例子的解决过程是否学会了如何使用 MAT，但应该不仅仅如此吧。这个例子经历了很多恶心的环境，经历了一些思维斗争和困难排除，一起分析了原因，看了调用三方包的代码，看了三方包的部分源码，最终印证原因与猜测是一致的，在这个过程中，MAT 是中间利用的一个工具，一个辅助我们解决问题的工具，而不是取代我们去解决问题的工具。作为老 A 一定要学会驾驭一些好的武器，或者说在你知道有一个好武器的情况下，有能力很快地去驾驭它。

►► 3.7.9 BTrace

当 JVM 中发生宕机时，如果我们已经发现了占用空间较大对象的类型，但是很难定位这个大对象到底是哪段代码所创建或写入的数据，第一想法是去翻阅项目中所有的代码。如果翻阅代码的时候发现了问题自然是好，但如果翻阅代码时并没有发现任何问题，那就有可能是二方包、三方包调用过的，我们是不是要把这些二方包、三方包的代码都要翻读

得十分清楚呢？那太费劲了，在定位问题时我们只想知道哪段代码中调用过出现问题的类中的某个方法或写入过数据，将调用过这个类的相应方法的“调用栈路径”打印出来即可，BTrace 可以帮助我们做到这一点，它是不是很神奇呢？或许不是，还记得胖哥在 3.2.4 节中谈到的字节码增强技术吗？它就是采用类似的技术来实现的。怎么实现呢？它只需要在调用的目标方法中插入一段代码，输出调用栈即可。举个例子：在系统中发现一个 HashMap 占用非常大，经过细节定位发现是 HashMap 中的数据个数非常多，这个 HashMap 是一个 static 类型的对象，因此可以断定的是 HashMap 具有全局的生命周期，并且可以确定有代码不断向这个 HashMap 写入新的数据，但是无法知道这个 HashMap 到底是哪一段代码调用的（因为它被全局共享，许多代码都会向这个 HashMap 写入数据），此时如果能够在 HashMap 的 put 方法上加上脚本，搜集调用来源，并进行统计，就可以帮助我们快速定位问题。

BTrace 就是使用字节码增强技术来实现代码的跟踪，它是直接修改字节码的，而不是创建子类。另外，BTrace 无须 Agent 的支持，而是基于 Java 的 Attach API，Attach API 在 tools.jar 包中的 com.sun.tools.attach 下面。

在讲解 JPS 时就提到过，如果获取了一个进程 VirtualMachine 对象，这个对象就负责与对应的 JVM 进程连接，它可以通过 loadAgent 方法来加载 Agent。

这个 Agent 的加载方式和上面直接使用 java -javaagent:xxx.jar 有两个区别。

- ◎ 此时加载 Instrumentation 不能用 premain()方法，而是用 agentmain()方法，入口参数保持不变。
- ◎ 在 META-INF/MANIFEST.MF 文件中不再是添加 Premain-Class 而是 Agent-Class，本书中对于动态 Attach 也写了一个 demo，用于加载自己的 Agent 的 jar 包，随书光盘中的 src/chapter03/attach/AttachLoadAgentMain.java 是客户端的 Attach 类（大家需要自己修改相应的代码，尤其是需要修改连接的进程名称，另外需要修改 jar 包的名称）。另外，胖哥在同一层目录下编写了一个 AttachRedefineClassMain.java 类，这部分代码相当于被 Attach 的目标进程代码，可以单独运行，还有一个 AttachAgentRedefineClass.java 用于作为 Agent 的类，大家可以将这个类所生成的 Class 打成 jar 包，作为 Agent 传送。如果大家能将这个代码跑过去，通过进程调用的方式实现字节码增强的基本原理就不难以理解了，那么理解 BTrace 的原理相信不是难事。

大家可以到 <https://kenai.com/projects/btrace/downloads/directory/releases/> 下载 Btrace 文件，通常包含了 Linux 的 shell 文件和 Windows 的 bat 文件。

BTrace 网上的例子非常多，这里用一个简单的例子来介绍 BTrace 跟踪一段代码。胖哥写一个简单的 main 方法，main 方法申请了一个实例，然后调用了另一个方法，BTrace 跟踪这个被调用的方法，负责输出它的相关 Class、Method、属性，以及一些传入的参数和返回值等信息。

首先应该构造一段被跟踪的代码，通常会写个死循环来调用程序模拟；否则可能跟踪程序还未连接上进程，被跟踪的程序就结束了（此时用断点的方式不是那么靠谱，因为我们现在要看方法的调用情况）。下面看看被调用的代码片段。

代码清单 3-21 即将被 BTrace 跟踪的代码片段

```
public class TestHello {
```

```

    private String name = "testHello";

    private final static int THREE_M_SIZE = 3 * 1024 * 1024;

    public static void main(String []args)
        throws InterruptedException {
        System.out.println("=====>");
        TestHello testHello = new TestHello();
        int times = 1;
        String a = "xieyuooo_";
        Map<String , String> map = new HashMap<String , String>();
        map.put("fuck" , "fuckAAA");
        while(true) {
            testHello.test(times++ , a + times , map);
            Thread.sleep(1000);
        }

        public boolean test(int times , String name ,
            Map<String , String>map)
            throws InterruptedException {
            System.out.println("=====>");
            Thread.sleep(2000);
            return true;
        }
    }
}

```

接下来编写 BTrace 跟踪脚本（注意：这个文件是 Java 文件，是被 Btrace 动态编译的，动态编译在本地就要完成，但这个类中相应的依赖包在目标进程和客户端进程中都得有。由于它单独存在，所以将它称为“脚本”）。

代码清单 3-22 使用 BTrace 来跟踪上面的代码

```

import com.sun.btrace.annotations.*;
import static com.sun.btrace.BTraceUtils.*;

import java.lang.reflect.Field;
import java.util.Map;

@BTrace
public class BTraceTest {

    @TLS
    static long startTime;

    @OnMethod(
        clazz="chapter3.tools.btrace.TestHello",
        method="test"
    )
    public static void traceExecute() {
        startTime = timeMillis();
    }

    @OnMethod(
        clazz="chapter3.tools.btrace.TestHello",
        method="test",
        location=@Location(Kind.RETURN)
    )
    public static void traceExecute(@Return boolean result ,
        @Self chapter3.tools.btrace.TestHello testHello,
        @ProbeClassName String className,
        @ProbeMethodName String methodName,

```



```

        int time ,
        String name ,
        Map <String , String> map) {
    println("=====>");
    println(strcat("call class = " , className));
    println(strcat("call method = " , methodName));

    Field fieldName = field(className , "name");
    Field fieldFreeMSize = field(className , "THREE_M_SIZE");

    println(strcat("property name = " ,
                    str(get(fieldName , testHello))));
    println(strcat("static property THREE_M_SIZE = " ,
                    str(get(fieldFreeMSize))));

    println(strcat("map.toString() = " , str(map)));
    println(strcat("map.get() = " , get(map,"fuck")));
    println(strcat(strcat("times: ",str(time)),
                    strcat(" , name:", name)));
    jstack();
    println(strcat("delay:" , str(timeMillis() - startTime)));
    println(strcat("return value is:",str(result)));
}
}

```

此时将这个文件保存为“BTraceTest.java”，就像 Java 源文件一样（不过通常没有 Package，在本书的 demo 中，胖哥将这个类放在单独的 script 目录下，表示它独立于 Java 的脚本来运行）。

这里有许多 Annotation，是不是有点晕，其实看惯了就好，就像第一天用 Java 本身提供的 Annotation 一样。关于 Annotation 的原理将在本书的 7.2.4 节中详细介绍。

Btrace 主要提供两个东西：一个是 Annotation；另一个是静态方法。

Annotation 位于 com.sun.btrace.annotations 下面，静态方法位于 BTraceUtils 中，通过这个 Utils 操作切入是允许的，但是其他的方法不行，可能是出于一些安全因素的考虑。

在 BTrace 中最常见的 Annotation 有两个，即 @BTrace 和 @OnMethod，它们分别代表 BTrace 脚本，当某个指定的方法被调用时会被调用。在 BTrace 中还有很多其他的 Annotation，如 @OnTimer、@OnExit、@OnError、@OnEvent、@OnLowMemory、@OnProbe 等，以及一些方法参数注解，如 @Self、@Retrun、@ProbeClassName、@ProbeMethodName 等，大家可以慢慢品味，这里就不细说了。

现在来说说这段脚本的一些小细节。

上面的代码片段中对属性 startTime 使用了 @TLS 注解，这说明这个参数将使用 ThreadLocal 处理（接下来调用的方法都是静态的，属性也是静态的，使用 ThreadLocal 自然是为了线程安全。ThreadLocal 的原理将在 5.2.6 节中介绍）。

代码中有两个 @OnMethod 注解，很奇怪是吧！这两个注解的唯一区别是在第 2 个注解中包含了 location = @Location(Kind.RETURN)，表示在发生“return 前”会调用这个方法，第 1 个注解默认是在方法执行前执行，所以在理论上会在方法执行前输出 startTime，执行后将当前时间减掉 startTime 得到该方法的执行时长。

代码中的 printf()、strcat()、str()、timeMillis() 方法都是 BTraceUtils 里面的静态方法，类似的方法还有很多，例如可以通过 jstack() 输出当前方法的调用栈路径，此时想要跟踪某个方法被谁调用过，则可以使用这种方式来达到目的。比如前面提到的 HashMap 如果不是

应用本身导致的，而是三方包，就可以跟踪 HashMap 的 put()和 putAll()方法，看看它们被谁调用过，并且看谁调用得最多。

BTraceUtils 里面的方法是可以直接使用的，但是大家别认为任何代码都可以往里面加，BTrace 在运行时对于要编译的脚本会进行检测，并不是想干什么就干什么的。例如在上面传入的参数中，如果对传入的 Map 直接通过 map.get()操作是会报错的，必须通过 BTraceUtils 中的静态方法 get(Map, String)来获取对应 Key 的值，包括反射中获取 Field 的方法也不能直接使用，而是要通过 BTraceUtils 的静态方法 field(String, String)来获取，分别传入类名和属性名，依此类推。

疑惑：这不是和 AOP 一样吗？有何区别吗？

答曰：在实现机制方面有共同之处，但有一个很大的区别是，BTrace 在运行时可以随时切入进去，而无须应用重启。

接下来胖哥用上面的代码来跟踪程序（大家使用的时候记得将要访问的类和方法名改掉）。胖哥同样通过 jps 来查看，需要跟踪的进程号是 26180，使用命令后的结果如图 3-58 所示。

```
[xieyuooo@linux001 bin]$ sudo ./btrace -classpath /home/xieyuooo/javaTest 26180 /home/xieyuooo/btrace-script/BTraceTest.java
=====>
call class = chapter3.tools.btrace.TestHello
call method = test
property name = testHello
static property THREE_M_SIZE = 3145728
map.toString() = {fuck=fuckAAA}
map.get() = fuckAAA
times:148, name:xieyuooo_149
chapter3.tools.btrace.TestHello.test(TestHello.java:29)
chapter3.tools.btrace.TestHello.main(TestHello.java:21)
delay:2003
return value is:true
=====>
call class = chapter3.tools.btrace.TestHello
call method = test
property name = testHello
static property THREE_M_SIZE = 3145728
map.toString() = {fuck=fuckAAA}
map.get() = fuckAAA
times:149, name:xieyuooo_150
chapter3.tools.btrace.TestHello.test(TestHello.java:29)
chapter3.tools.btrace.TestHello.main(TestHello.java:21)
delay:2004
return value is:true
```

图 3-58 一段简单的 BTrace 脚本跟踪代码后的输出结果

上面的程序每 1s 调用一次 test()方法，每次调用 test()方法会延迟 2s 后继续调用。图 3-58 所示的结果就是循环两次的结果，胖哥认为这个结果对照代码和前面的脚本是很容易看懂的。



注意 小伙伴们看到这里运行时命令加上了一个 -classpath 很困惑，其实是因为实际代码放在了 /home/xieyuooo/javaTest 目录下，但是当前目录可能不是这个目录，使用该方式就可以访问到程序。这和我们设置 classpath 后，就能访问到不同路径下的 jar 包的道理一样。

关于更多 BTrace 的知识，大家可以在遇到实际问题时去网上查阅，本书光盘的 doc 目录下有一个概要文档可以参考，胖哥在本节希望通过这个例子起到抛砖引玉的作用。

既然 BTrace 是基于 Attach API、ASM API、Instrumentation API 来完成的，那么自己写是不是也可以实现呢？

没错！而且 BTrace 肯定也不是唯一的，从上面介绍可以发现，写一段 BTrace 脚本其实还是挺麻烦的。它其实是一种工具，虽然在很长一段时期内号称 Java 工具中的神，但它始终是一种工具，使用工具就肯定有缺陷，而且工具的目标应当是操作简单，如果它有缺陷、不够简单，业界的牛人们就要挑战它或完善它，这些朋友也能拿出许多自己的产品，说明其相对于 BTrace 的优势，这样才能使得开源界不断发展。

3.7.10 HSDB

Java 还提供了一个 HSDB 工具，它类似于 Visual VM 集成了很多工具，只是它在命令行下使用，这样可以使问题在不方便使用图形化工具时得到解决。

由于工具是在命令行下操作的，第 1 次看到可能会觉得有点抽象，不过“一回生、二回熟”，不要怕它，迟早可以驾驭它。

接下来我们启动一个 HSDB 进程看看是什么样子的，它的启动方式是运行命令：

```
java -classpath .:$JAVA_HOME/lib/sa-jdi.jar sun.jvm.hotspot.CLHSDB
```

如果是 Windows，则需要将 \$JAVA_HOME 改为 %JAVA_HOME%。

运行后会进入 HSDB 交互界面，此时输入 help 命令，可以查看自己可以进行的操作，如图 3-59 所示。

```
$java -classpath .:$JAVA_HOME/lib/sa-jdi.jar sun.jvm.hotspot.CLHSDB
hsdb> help
Available commands:
  assert true | false
  attach pid | exec core
  detach
  dumpcodecache
  echo [ true | false ]
  examine [ address/count ] | [ address,address]
  field [ type [ name fieldtype isStatic offset address ] ]
  findpc address
  flags [ flag ]
  help [ command ]
  history
  inspect expression
  jhisto
  jstack [-v]
  livenmethods
  pmap
  print expression
  printas type expression
  printstatics [ type ]
  pstack [-v]
  quit
  reattach
  revptrs address
  scanops start end [ type ]
  search [ heap | perm | rawheap | codecache | threads ] value
  source filename
  symbol name
  thread { -a | id }
  threads
  tokenize ...
  type [ type [ name super isOop isInteger isUnsigned size ] ]
  universe
  verbose true | false
  versioncheck [ true | false ]
  where { -a | id }
```

图 3-59 进入 HSDB 时看到有这些命令

大家可以通过查阅相关资料来查看每个命令的作用，例如 jstack 用于输出栈信息，就像单独运行 jstack 命令一样；attach <pid>是连接到对应进程上的命令等。

下面写一小段代码，通过 HSDB 工具来查看代码中的一些对象属性、被谁引用，以及每个属性的内容是什么。

代码清单 3-23 一段简单的用于 HSDB 测试的代码

```
class Node {
    String a = "aaaaa";
    String b = "bbbbbb";
}

public class TestHsdb {

    private Node node1;

    private Node node2;

    public static void main(String[] args) throws IOException {
        TestHsdb testHsdb = new TestHsdb();
        testHsdb.node1 = new Node();
        testHsdb.node2 = new Node();
        System.in.read();
    }
}
```

首先将该程序运行起来,然后再启动上面的 HSDB 程序,开始进一步跟踪,通过 universe 命令查看 JVM 每个区的“虚拟地址”范围,通过 scanscope 命令查看一个指定的 Class 类型有哪些内存单元地址。这里要跟踪的就是 Node 类型,如图 3-60 所示(由于刚分配的内存肯定在 Eden 中,所以我们检索空间的范围是 Eden 区域)。

```
$jps
19924 Jps
19799 TestHsdb
$java -classpath .:$JAVA_HOME/lib/sa-jdi.jar sun.jvm.hotspot.CLHSDB
hsdb> attach 19799
Attaching to process 19799, please wait...
hsdb> universe
Heap Parameters:
ParallelScavengeHeap [ PSYoungGen [ eden = [0x00000007ac600000, 0x00000007ac740f68, 0x00000007b04b0000] , from = [0x00000007b0f20000, 0x00000007b0f20000, 0x00000007b1990000] , to = [0x00000007b04b0000, 0x00000007b04b0000, 0x00000007b0f20000] ] PSOldGen [ [0x0000000705200000, 0x0000000705200000, 0x000000070f940000] ] PSPermGen [ [0x0000000700000000, 0x000000070027e778, 0x00000007014c0000] ] ] hsdb> scanscope 0x00000007ac600000 0x00000007ac740f68 chapter3.tools.Node
0x00000007ac6347d0 chapter3.tools.Node
0x00000007ac6347e8 chapter3.tools.Node
```

图 3-60 我们发现有两个内存地址使用了 Node 类

接下来通过 revptrs 命令查看谁引用了对应的 Node 对象,再通过 inspect 命令查看对象中有哪些属性,以及它们的内存地址,如图 3-61 所示。

```
hsdb> revptrs 0x00000007ac6347d0
Oop for chapter3/tools/TestHsdb @ 0x00000007ac633438
hsdb> inspect 0x00000007ac633438
instance of Oop for chapter3/tools/TestHsdb @ 0x00000007ac633438 @ 0x00000007ac633438 (size = 24)
<<Reverse pointers>>:
_mark: 1
node1: Oop for chapter3/tools/Node @ 0x00000007ac6347d0 Oop for chapter3/tools/Node @ 0x00000007ac6347d0
node2: Oop for chapter3/tools/Node @ 0x00000007ac6347e8 Oop for chapter3/tools/Node @ 0x00000007ac6347e8
```

图 3-61 查看对象被谁引用,以及引用的对象中有哪些元素

继续跟踪 Node 看里面包含什么内容,以及 Node 中的每个元素又包含哪些内容,直到 String 的数组中包含的元素为止,如图 3-62 所示。

关于 HSDB 工具还有很多功能,大家可以参阅更多的资料来进行操作。

>> 3.7.11 工具总结

在本书中胖哥无法对所有的工具进行诠释,因为很多工具胖哥也没有用过,甚至没有听过,大家要学会在原理基础上来使用工具,用工具反向深刻理解原理,就能以不变应万变。


```

hsdb> inspect 0x00000007ac6347d0
instance of Oop for chapter3/tools/Node @ 0x00000007ac6347d0 @ 0x00000007ac6347d0 (size = 24)
<<Reverse pointers>>:
_mark: 1
a: "aaaaa" @ 0x000000070027e6f8 Oop for java/lang/String @ 0x000000070027e6f8
b: "bbbbbb" @ 0x000000070027e738 Oop for java/lang/String @ 0x000000070027e738
hsdb> inspect 0x000000070027e6f8
instance of "aaaaa" @ 0x000000070027e6f8 @ 0x000000070027e6f8 (size = 32)
<<Reverse pointers>>:
_mark: 1
value: [C @ 0x000000070027e718 Oop for [C @ 0x000000070027e718
offset: 0
count: 5
hash: 0
hsdb> inspect 0x000000070027e718
instance of [C @ 0x000000070027e718 @ 0x000000070027e718 (size = 32)
<<Reverse pointers>>:
_mark: 1
0: 'a'
1: 'a'
2: 'a'
3: 'a'
4: 'a'

```

图 3-62 继续查看对象中每个元素的细节

即使在本章中谈到的工具，也没有对每个参数进行详细讲解，更没有为每个参数单独去写一个 demo。因为胖哥希望本章的内容尽量简单，但即使这样，本章的内容依旧很庞大，如果你有耐心看完本章，并结合实践多看几次，那么相信未来你看真正的牛人秘籍时会轻松许多。

在古代，所谓“宝剑赠英雄”，宝剑（工具）可以认为是古代武林高手的神兵利器；在现代，工具可以认为是老 A 的狙击步枪，悄无声息地干掉对方的司令官。那么我们用 Java 的这些“宝贝”能干什么？这是我们需要思考的问题，一个想要成为老 A 级 Java 程序员需要思考的问题。

工具可以配合使用，但是一定要明确每个工具的特长和含义，我们还未达到“无招胜有招”的境界，就要先知道“什么招克制什么招”，也就是我们遇到问题时要学会见招拆招，熟练了，或许就可以达到所谓的“挥洒自如”。

3.8 总结

3.8.1 写代码

本章胖哥一直在强调一个词汇“小而美”，写代码大部分情况下注重“小而美”的概念是毋庸置疑的，代码中申请的对象越少，代码跑得越快，就像昙花一样美丽瞬间，但是它的魅力是最珍贵的。

经过多年的发展，Java 如今的速度可以说脱胎换骨，它不仅仅在企业级内部应用，也在高并发的互联网领域、分布式计算、NoSQL 等领域得到了验证。

可能很多 C、C++ 的朋友此时开始较劲了，它们可能会拿纳秒、微秒来跟我们 PK，以至于一招击败 Java。

好吧，胖哥或许可以说句阿 Q 点的话：在 Java 领域不在乎那么小的性能区别，一个原因是 Java 的大部分应用场景并不在乎微秒级别或更细的时间差，另一个更重要的

原因是现在并不是一个单机称王的时代，而是在社会需要太多的“快速发展和学习方法”、“清晰的架构思路”、“快速设计实现来满足变态场景”、“系统易维护性”。也许在这个时代，Java 更适合做这样的事情，类似于超大数据的分布式计算平台仍然以 MapReduce 为最火热的发展方向，同样的 HBase 也同样能接受高并发的挑战，这些都是以 Java 为基础的。

回到“小而美”的概念，它也并非适合所有的场景，这也许就是所谓的世事无绝对吧！

很多时候我们要考虑自己的东西是否真的要缓存，例如 Class 不可能每次用的时候都去加载一次吧，它应当在加载后被保存在 JVM 内存中。因为这些数据几乎不会被修改，而且占用空间是可控的，放在内存中也无所谓。一些从 I/O 读取运算的数据，也必须考虑一些缓冲区（此时更明确的概念应该叫 Buffer，而不是 Cache），操作时不能读取一条数据访问一次 I/O，那样 I/O 将成为瓶颈，我们可能读取一部分数据在内存中作为 Buffer 随时使用。希望大家能把握好一个“度”，切忌走极端认为小而美的操作就是将原先所有的批处理代码改为逐条处理，这肯定是不合理的，具体的度需要在实际场景中根据官方数据和测试数据一起来决定。

这个“度”，很多同学可能还没有感性认识，那么听胖哥讲个小故事或许会有感性认识了。

小胖哥在小的时候，坐在妈妈旁边看妈妈做事情，妈妈累了，对小胖哥说：儿子，给我拿两颗瓜子过来，乖。

小胖哥很听妈妈的话，细细地数好“两颗瓜子”递给妈妈。

妈妈笑着对儿子说：傻儿子，妈妈叫你拿两颗瓜子你真的就拿两颗瓜子过来呀，其实妈妈是叫你多拿点瓜子过来哦。

第二天，小胖哥的爸爸对小胖哥说：儿子，给我拿两颗瓜子过来。

小胖哥这回知道不是两颗了，把家里的瓜子全部拿过来了，爸爸很奇怪地问儿子：我不是说给我两颗就行了吗？儿子傻傻地看着爸爸说：妈妈说拿两颗的意思是多拿点。

这是小胖哥小时候犯的“囧”事之一。大家知道这个意思了吗？少一些，不是说少到极限，大一些也并非大到极限。这就像在很多对话中，根据场景的最佳时刻理解“他、她、它”的意思，那么你就要知道“他、她、它”的思想和内在。在代码中一个极端的例子是，一个 int[] 数组存储 10 个变量，我们没必要定义 10 个 int[] 变量，而且那也节约不了空间。

什么样的内存使用是我们必须要去关注的呢？

假如 GC 是身体的免疫系统，每天将身体的废物排出体外，那么不受身体控制的东西是什么？淋巴，也许最后是癌症，它会扩散，会长大，会破坏我们的身体，免疫系统对它无效。这就是我们必须要去掉的那颗“毒瘤”，大而丑的毒瘤，否则后患无穷，即使有再好的身体，也会被“病魔缠身”。

一些类似“禽流感”这样的传染病来了，令我们防不胜防，对于程序员最重要的是要提升“理论功底和利用高科技的仪器”才能快速应对这些病魔，找到“解药”；否则我们都是不断解决一个问题，同时带来更多更严重的问题。

当引入一种新技术时，并没有经过严格的测试，并不知道里面有没有 Bug，一旦运行起来出现问题，自己本身没病，而是被外界感染，该如何应对？要找到问题的本质，功底、工具也许会帮助你。

“小而美”依然适用于大部分代码，我们用 8 个“当”句来给大家简单总结小而美，但

是胖哥不希望大家背诵概念,这也不是概念,就是一些小总结,也许你有所感悟和体会,那么就将它记下来。如果你还没有感觉,则只需要隐约记着胖哥在本书上讲过这个内容就行,当有所感悟的时候翻开看看,也许会有所“共鸣”。

- ◎ 当程序可控时,申请更少的内存可以做完的事情,尽量不要申请多余的内存,但是大家不要因为很小的细节,而让整个代码毁容。
- ◎ 当遇到大数据处理时,要尝试用“小而美”的分段来处理(千万不要一顿吃掉一周的饭菜,那会撑死的),这种分段方式,不仅仅在自己的业务代码中需要去控制,在一些三方包的 API 中可能存在许多隐藏的批处理,同样可能导致内存使用的暴涨,这一点大家要特别注意。
- ◎ 对于大批量的小 I/O 操作,可以尝试用批处理的方式来完成,这样效率会提高许多,代码也会更加整洁。
- ◎ 当遇到局部代码段中已经申请许多占用空间的对象,这些对象在其“作用域内”的后续代码中还需要执行相对较长时间,并且这块内存的确在后面就不会再使用了时,那么可以手工将一些较大对象的引用设置为 null,这样一来,GC 很快就认为它是垃圾。通常,在这种情况下将对象的引用设置为 null 才会对 JVM 有好处;代码跑得很快时,对象可以快速脱离作用域,在这样的场景下将引用设置为 null 是没有必要的,那样只会让代码很难看。
- ◎ 当程序中确实需要不少内存在程序中跑时,要想尽办法缩小它的作用域,尤其是不要把它放在“寿与天齐”的对象(通常指 static,这种引用关系不论是直接还是间接的都会有问题),以及“长命百岁”的对象中(例如 Session,而且将 Session 的过期时间设置得很大),除非你自己知道何时可以手工来释放这些对象,以确保这段代码肯定会在使用完成后释放相应的内存。另外,在某些场景下也可以通过其他的办法让程序跑得更快(例如访问数据库的 SQL 优化或访问磁盘的 Buffer 读取方式等)。这几种方法都有相对场景,并非绝对,大家要想清楚再考虑如何使用。
- ◎ 当在实际的应用中确定某些内存确实经常用到,不会改变,而且加载到内存的代价相对比较大时(例如一些基础配置信息),在进程启动时就要想办法将数据填充好,避免和业务代码中的内存使用抢占 Survivor 空间。一般来说,这块空间不会太大。
- ◎ 当知道了某些数据内存中有 Cache 时,我们要尽量去用 Cache 的数据,而不要再自己 new 了;否则 Cache 就没有任何意义了,成为一个“白拿钱不干活的家伙”。例如 Integer、自动拆装箱,就没有必要自己执行 new Integer(1)这样的操作,因为这又多申请了内存。
- ◎ 当遇到一些反复叠加的操作时(例如字符串叠加、集合类添加数据),尝试使用一个外部的引用变量或对象来操作(当然这些操作是可以被封装起来的),在程序可控的情况下可以给予一个足够大的缓冲区来做叠加,以避免在叠加过程中不断产生垃圾。这个概念貌似与前面提到的作用域要小的概念是冲突的,其实一点都不冲突,这个操作中间的对象,无论怎么操作它的作用域都不会减少,因为始终有个最大的变量需要提供最终值,将一个 Buffer 放在外面,是整个叠加过程产生的碎片变得非常少。

3.8.2 心理上战胜虚拟机带来的恐惧

大多数人在第一次面对虚拟机问题时，都会出现“怕”的心理，认为它很高深莫测，觉得自己太菜，搞不定这玩意。胖哥要说的是，你越是对它“仰望”，你就越难掌握它，就像每个程序员调试第一个程序那样，在没写以前可能会存在各种恐惧，但是写了如何呢？能够坚持到现在，就说明我们是有机会战胜它的。

还有的同学是太不怕了，觉得虚拟机这玩意就是小儿科。胖哥不得不说这玩意对于一些真正的顶尖级高手来讲，确实是小儿科，但是对于对 JVM 还不太熟悉的人来讲，切记不要不知天高地厚，如果真的觉得自己已经什么都知道了，则可以去尝试与一些牛人交谈，或许你会有意外的收获。

胖哥希望你既然选择阅读本书，就拿出一点点霸气，“她”就是你追求的对象，征服它，驾驭它！当我们跨过了“怕”的心理障碍，战胜了自我后，很多事情就会变得顺畅，遇到问题需要有点紧张，但是紧张过度会使你想到的答案只有“重启”。

面对 JVM 的问题，我们就是医生，要给它看病，而不是给它下猛药，那样有失“职业道德”。我们希望自己成为 JVM 的好医生，这需要的是功底、探知问题的方法，要逐步缩小可能性。

当我们对这个专业足够成熟时，在遇到宕机情况时该先做什么、再做什么心里自然开始有谱了。例如尽量保持系统对外服务，在这个前提下尽量拿到现场的拷贝来做分析，也就是要稳定病情，分析病情，缩小病情范围。换句话说，我们虽然知根知底，但是场景千变万化，我们也会针对复杂的病情做一些尝试，可是不能无限限制地枚举尝试，而是在进行分析的前提下有准备地去做一些尝试，最终定位问题。

你有耐心看到这里，胖哥恭喜你已经突破了 Java 老 A 的一个重要关口，如果还没完全看懂没关系，希望你这样一些知识，当你在实践中遇到一些问题时结合知识来思考，就会很快将知识串联起来。

第4章

Java 通信，交互就需要通信

本章主要内容

- 通信概述
- 通信调度方式
- Java I/O 与内存的那些事
- Tomcat 中对 I/O 的请求处理

相对上一章的 JVM 魔鬼训练营，本章会容易一些，但是依然有许多不好理解的问题，而 Java 通信是 Java 程序员都应当掌握的，因为 Java 设计的初衷之一就是为了网络，网络的基础就来自于通信。

本章将更详细地介绍一些 I/O 知识，胖哥希望大家能知其然，知其所以然。

4.1 通信概述

▶▶ 4.1.1 Java 通信的基本过程

在 2.6.2 节我们简单介绍过网络交互的基本过程，当时是从系统角度简单认识一下 I/O 与网络，本章将从总体的原理上去了解一些内容。

请先看图 4-1，这个图是胖哥所理解的，两个进程之间通过网络传递一个字符串(String)的大概过程。

在图 4-1 中可以看出，Java 进程并不真正参与网络通信，而只是将数据交给 Kenel 去完成。

通常使用 Java 编写 I/O 程序时，都会先学习两组接口，其中一组接口是 InputStream；另一组接口是 OutputStream。这可能不是大家最喜欢用的接口，因为它们操作的是字节，很多小伙伴都喜欢看得见摸得着的字符操作，也就是用 Writer、Reader 来操作，但我们去看源码时会发现这两个 API 是通过对 InputStream、OutputStream 的一层封装来完成的。

在真正发生 I/O 的时候，一定要转换为字节来操作，所谓的字节流只是一个 Java 层的假象而已。在计算机中其实也只有数字存在，字符仅仅是一种字符集下由一串数字表达出

来的，最终由屏幕输出给我们看的東西，所以有时候会说：“你看到的未必是真实的”，作为学习计算机的同学应当懂得这一点。

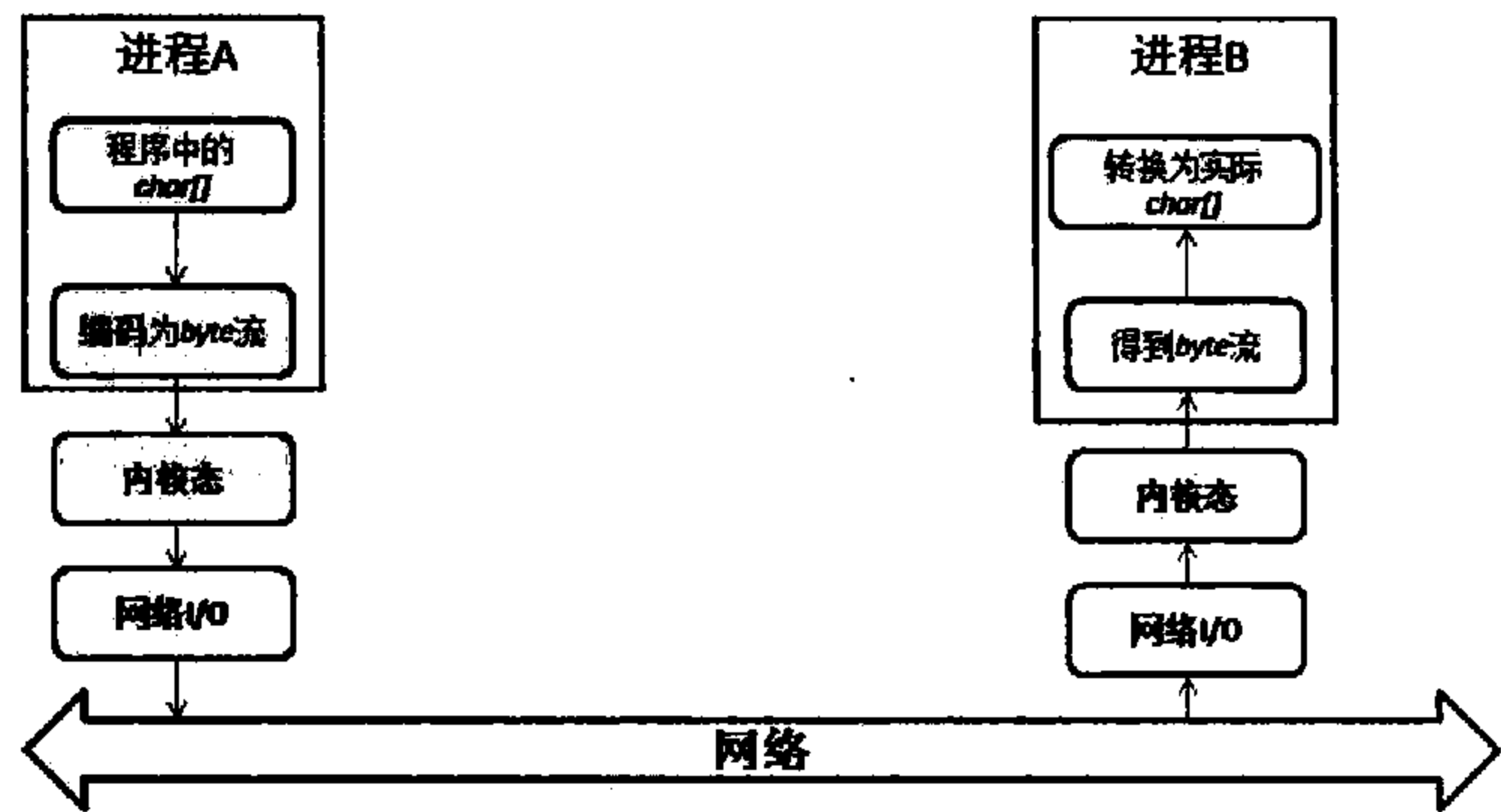


图 4-1 一个简单的 Java 网络通信概要图

计算机中的 char 也是由数字组成的，Java 的 char 是 UTF-16 编码，每个字符占用 2 个字节的宽度，假如网络中直接传送 char，那么 Java 程序可能就需要传送 UTF-16 编码相对应的数字，但是对方的程序所使用的运行时字符未必采用这样的编码，因此传送中需要约定字符集是什么，发送方用什么方式来编码，接收方就用什么方式来解码，这样就可以得到正确的字符了。

例如发送 2 个汉字，用 UTF-8 编码发送后会占用 6 个字节，对方若知道传来的数据是 UTF-8 编码，便知道这 6 个字节代表什么汉字，就自然能正确得到 char 字符了。但是如果对方用 GBK 来编码，则可能会认为有 3 个字符，若按照每 2 个字节计算一个字符，得到的 3 个字符自然不是需要传递的 2 个汉字字符。假如此时意识到自己的编码错误了，通过得到的 3 个字符的字符串调用 `getBytes("GBK")` 还可以还原 6 个字节，然后通过这 6 个字节再用 `new String(byte[], "UTF-8")` 得到实际的两个汉字。真的是这样吗？要知道这是偶然的，不是必然的，因为 UTF-8 转换出来的 6 个字节，当按照每 2 个字节组成编码时，这个编码未必在 GBK 的编码范围内，若不在 GBK 的编码范围内，就可能会用一个“？”或其他字符来代表，由于“？”本身也是一个字符，当再次调用 `getBytes("GBK")` 时得到的对应字节就是“？”对应的字节，而不是原来字符的字节，有可能都不再是 6 个字节了。换句话说，这样的情况是永远无法转换回来的。

上面的例子很特殊，是 2 个汉字的 UTF-8 编码对应 6 个字节，如果是 2 个汉字的 GBK 编码对应 4 个字节，用 UTF-8 来解析，或者 3 个汉字的 UTF-8 编码对应 9 个字节，用 GBK 编码来解析，出现的问题将会更复杂，有很多字符集还带有头部输出，所以所谓的“字符流”在网络中并不是那么靠谱的，我们需要了解通信协议和字节流与字符流的转换过程（字符集并不是系统的，也不是字符或字节本身的描述，而是在系统的一段程序中有字符与字节的转换才有字符集的概念）。

通常，在 I/O 交互过程中，会在协议的头部中告诉对方自己用的是什麼编码，对方就会按照这种格式来解码。例如在 HTTP 协议交互中，通常浏览器会以 `contentType` 的方式告诉服务器端本次数据传送的字符集是什么，服务器端输出数据时，也会用这种方式告诉浏览器，大家可以在浏览器的 debug 下面看到相应的头部信息。

上文中提到的 UTF-8 编码采用了 3 个字节表达一个汉字, 不过 UTF-8 编码是变长的, 或者说它表示其他文字未必使用 3 个字节, 请看下面的解释。

首先它将英文字母和常用字符使用 1 个字节来表示, 1 个字节 8 个二进制位, 可以表达 256 个不同的字符, 它不会将这 256 个数字用完, 尤其是负数部分不会使用。这些负数怎么使用呢? 它可以作为 2 个以上字节编码的字符的前缀字节, 这样编码解析通过前缀字节的范围就能判定它是否为单字节编码了。至于具体数字是多少, 什么语言用几个字节来表达, 我们目前不用关心 (有兴趣的同学可以自行研究), 大家只需要知道 UTF-8 只是针对汉字使用 3 个字节来存放, 其他的语言则有可能大于或小于 3 个字节就行了。

这是网络交换的最基本道理, 不过对于网络通信, 为了表达两者之间的某种约定, 就得有一种协议存在。请看下一节内容, 我们来介绍协议。

►► 4.1.2 Java 通信的协议包装

什么是协议? 协议就是双方之间的“暗号”, 有点类似于“心有灵犀一点通”。

如何对应到网络协议呢?

在数据传送的过程中会在前面包含一些除数据本身以外的其他内容 (因为在数据之前, 所以称它为头部), 它通常用来描述连接操作、数据格式相关的内容。例如, HTTP 网络协议编码和解码就是在协议头部中通常出现的 content-type 项, 另外类似于 user-agent、referer 等内容都可能包装在协议的头部, 那么头部就代表了要传送什么给对方, 对方就知道如何来解析。

在使用 Java 的 Socket 时, 它是一种基于 TCP/IP 协议的通信, TCP/IP 本身也有自己的底层协议包装, 不过这仅仅是底层的通信交换规则, 但是没有包含自己想要告诉对方的一些头部信息 (例如想要告诉对方内容的长度、格式)。

于是, 教科书上所讲的例子通常是发送一个字符串“bye”, 对方接收到后就知道结束了。显然这种方式仅仅是让大家了解通信编程, 在实践中还会存在许多问题, 在真正的网络编程中, 会比这个复杂许多。例如, 发送字符串“bye”也只能代表一个普通的字符串 (在实际的场景中, 需要自己通过 Socket 来约定协议规范)。接下来看 4.1.3 节的内容, 将介绍自定义协议的传送包装和拆分整个过程。

如果大家还没了解协议是什么, 胖哥给大家做一点点扩展: 协议在计算机中无处不在, 即使是两个计算机部件之间传递数据也需要协议, 只是更偏重于底层。在网络中较低层次的数据传递, 发送方会在规定的数据包位置告诉对方自己的地址、数据宽度等信息。在我们使用的 JSON、XML 中其实也是人为约定的一种协议, 因为接收方就是按照 JSON 或 XML 格式来解析并编写程序的。如果还没看懂, 我们可以想一想生活中两个人约地方见面会约好时间、地点、联系方式等, 这就是两个人之间的协议, 如果协议没有搞好自己心里没底, 或者给对方一个错误的地址, 自己却去了另一个地方。

►► 4.1.3 编写自定义通信协议

要发送和解析细节, 就要先约定协议, 此时我们来想想如果要约定一个通信协议, 都需要做哪些事情。

首先, 我们可以用 1 个字节来代表命令的类型, 就像标识一种协议类型一样, 类型表

示它允许传递的字符串、文件、二进制文件等，总之传递的数据类型不同，第一个字节的数字就不同。如果是字符串，就再用 1 个字节来代表传送的字符集，不过如果双方约定统一使用一种字符集来交互，那么就不用传送了（也可以将字符集、传送数据的类型合并到 1 个字节中，因为 3 种数据传递的类型用 2 个 bit 就可以表达（因为 2 个 bit 可以表达 4 个不同的数据），1 个字节有 8 个 bit，数据传递类型使用 2 个 bit 后，另外 6 个 bit 可以表示 64 种字符集。

然后，用 4 个字节来表示内容的长度（4 个字节足以表示 2G-1 的大小，通常一次数据传递不会有这么大，不过确有相应需求也可以换用 8 个字节来表示）。

或许你还不放心，希望发送一个结束符来表示一次数据发送已经结束，其实没多大必要，因为前面已经发送了长度，那么接收方自然知道接收多长的数据，发送结束符其实是多此一举。

胖哥在本节中也会用类似的方式来实现一个简单的程序，让大家有点感性认识。为了看懂程序，下面先详细描述一下要实现的几个基本功能。

- ◎ 在服务器端简单分配多线程处理，每个客户端的请求对应服务器端的一个线程，每个线程都有一个名字，以便于对应到客户端传送过来的数据，并且能打印出它们要请求什么类型的数据。
- ◎ 实现基本的文字传送，在第 2 章中介绍过此功能，此时还需要另外包装协议，协议类型为“1”，然后发送字符串的编码长度（用 4 个字节表示），以及实际的内容。

例子：客户端命令为：“sendMsg 你好啊！”，代表发送 4 个字符“你好啊！”。



注意 由于双方约定都用某种字符集，所以字符集就不再像前文所讲那样传送了。

- ◎ 实现客户端向服务器端传送指定的文本文件，协议类型为“2”，需要传送文件的“字符集、文件名、内容”三大部分，此时对于字符集使用 1 个字节单独表示；文件名长度用 2 个字节表示，文件名也通过双方约定的字符集编码；内容使用 8 个字节代表长度（以保证文件超过 2GB 时可以继续接收），后面全部是文件内容字节流。服务器端接收到文件后，将文件的前面 1000 个左右的字符输出打印在控制台上，并将文件保存在一个指定的目录下，若发现目录下有同名的文件，则向客户端输出错误指令“-1”。

例子：“sendFile d:/abc.txt charset=utf8”代表要传送盘符 d: 下面的 abc.txt 文件。

- ◎ 客户端向服务器端发送非文本文件，协议类型为“3”，类似于 Word、Excel、PDF、视频等文件的传送。表示文件的方式和上面一样，如果发送非文本文件，则接收方不输出文件的内容，其余的同上。

例子：“sendBFile d:/bbb.doc”代表要传送客户端盘符 d: 下面的 bbb.doc 文件。

- ◎ 客户端向服务器端请求下载文件，协议类型为“4”，下载的文件也是上传的，指定一个文件名，由服务器端处理，若服务器端没有找到文件，则向客户端输出没有找到文件的指令“-2”。

客户端程序需要指定下载的文件存放在本地的哪个目录下，否则下载后的文件默认

会放在 JVM 的系统参数 `user.dir` 所对应的目录下。另外，指定的目录下有同名的文件，则会报错。

例子：“`getFile abc.txt c:/download/`”代表要将文件 `abc.txt` 下载到 `c:/download/` 目录下。

接下来我们就用程序来实现这个交互过程。先将 `Socket` 做个简单包装，然后再用，否则使用时会觉得费劲。

包装后的部分代码如图 4-2 所示（由于篇幅所限，这里只贴出一些有代表性的代码，完整的代码请参看本书光盘中 `src/chapter04/socket` 下面的代码）。

为何要用 `DataInputStream`、`DataOutputStream` 呢？

因为我们想要发送一些数据格式，而不需要自己来组装格式。例如想发送或接收 `int`、`long`、`short` 类型的数据等，如果使用普通的 `InputStream` 和 `OutputStream`，发送时就需要自己将 `int` 转换为 4 个字节，发送或接收后组装起来，接收时需要将接收到的 4 个字节转换为 `int`，挺麻烦的。`DataInputStream` 和 `DataOutputStream` 在这个基础上做了数据类型的封装（我们自己就不用再去封装了）。

在这里，将 `Socket` 的输入/输出流包装在内部，这样对外就提供一些简单的读、写 API，而不需要让外部知道太多的流、文件的细节。

```
private DataInputStream inputStream;

private DataOutputStream outputStream;

public SocketWrapper(Socket socket) throws IOException {
    this.socket = socket;
    processStreams();
}

public SocketWrapper(String host, int port) throws IOException {
    this.socket = new Socket();
    this.socket.connect(new InetSocketAddress(host, port), 1500);
    this.socket.setKeepAlive(true);
    this.socket.setTcpNoDelay(true);
    processStreams();
}

private void processStreams() throws IOException {
    inputStream = new DataInputStream(socket.getInputStream());
    outputStream = new DataOutputStream(socket.getOutputStream());
}
```

图 4-2 对 `Socket` 进行简单的包装处理

另外，胖哥包装了对文件的发送处理代码，外面只需要提供文件的路径，其余的由内部包装对文件的发送处理动作，内部操作中还会有对大文件的特殊处理，如图 4-3 所示。

这段代码通过 `FileInputStream` 读取本地文件的内容，并将得到的内容通过 `this.write(byte, length)` 将字节数组发送出去。

有的小伙伴可能会问：代码中为什么要分两种情况来读取与发送数据呢？

两种情况使用一个 `PAGE_SIZE` 来区分，每次通过 `FileInputStream` 获取的数据首先被加载到 JVM 中（当然中间还会经历一个 OS 内核区的过程），将过多的数据一次加载到 JVM 中会导致出现很多不同的问题。`PAGE_SIZE` 在这个过程中起到的作用是设定一个阈值，当小于这个阈值时，就一次将整个文件的字节读取出来发送到 Kernel，由 Kernel 去调度发送；当大于这个阈值时，就分批发送，每一批发送的就是 `PAGE_SIZE` 的大小。

```

public void writeFile(String path) throws IOException {
    File file = new File(path);
    FileInputStream fileInputStream = new FileInputStream(file);
    try {
        long fileLength = file.length();
        if(fileLength > PAGE_SIZE) {
            byte []bytes = new byte[PAGE_SIZE];
            int allLength = 0;
            int length = fileInputStream.read(bytes);
            while(length > 0) { //超过一定大小文件，分段传送
                allLength += length;
                this.write(bytes, length);
                length = fileInputStream.read(bytes);
                print(".");
            }
            println("实际发送文件长度为: " + allLength);
        }else {
            byte []bytes = new byte[(int)fileLength];
            fileInputStream.read(bytes);
            this.write(bytes);
        }
    }finally {
        closeStream(fileInputStream);
    }
}

```

图 4-3 将指定文件发送到远程的代码片段

为何不用 `BufferedInputStream` 和 `BufferedOutputStream` 等来达到缓冲的作用？

其实在这里用也可以，这两个所谓的缓冲区是 Java 堆中的一个 `byte[]` 数组，默认是 8192 个字节，每次通过写操作时，如果没有发起 `flush`、`close` 操作且 `byte[]` 数组没有满，则不会将数据真正发送出去，而真正发送时依然是将 Java 区域的一个 `byte[]` 数组调用 `write` 方法交给 Kernel。换句话说，如果有可控大小的 `byte[]` 数组，则也可以选择不用 Buffer 来控制。

Socket 使用的是 `SocketInputStream` 和 `SocketOutputStream`，父类是 `FileInputStream` 和 `FileOutputStream`，`FileInputStream` 和 `FileOutputStream` 没有实现 Buffer 的功能，发送数据是直接基于默认的 `OutputStream` 的 `flush()` 方法的，它是空方法，不会做任何动作，因此在使用 `SocketOutputStream`、`FileOutputStream` 时，不需要调用 `flush()` 方法（只是在内核中有一些 buffer 机制）。

在实际的网络通信中，在某些容器或第三方框架中，通过修饰、继承方式实现了对 `BufferOutputStream` 的间接使用，或者自定义一种 Buffer 机制，因此我们还不能仅仅简单地看拿到的类是什么类型，就决定它是否使用了 Buffer。

继续向下看，为了构建服务器端，实现多线程。在这里，我们不用“线程池”，我们就简单地做一个线程的包装，包装的目的在于需要对线程做一些扩展处理。简单地抽象出来一个 Worker 类，如图 4-4 所示。

这里的构造方法，让 Worker 获得 `SocketWrapper` 类型对象的引用，并传入一个名称，以保证对应的名称与客户端是一套组合。

在 `run()` 方法中，是一个死循环，但是有 `break` 的条件，为何？

从代码中可以看出，当程序出现某些异常时，会跳出循环，而且这些异常一般是程序不知道如何处理的异常或网络通信异常。某些自定义异常我们是可以自己处理的，例如 `SaveExistsFileException`、`DownloadNotExistsFileException` 这些异常都是胖哥定义好的异常类型，分别表示“保存已经存在的文件异常”、“下载不存在的文件异常”。


```

public Worker(SocketWrapper socketWrapper, String name) {
    this.socketWrapper = socketWrapper;
    this.name = name;
    logInfo("我是线程: " + name + ", 开始启动接收客户端传来数据.....");
    this.start(); // 启动
}

public void run() {
    while(true) {
        try {
            if(this.isInterrupted()) break;
            byte type = socketWrapper.readByte();
            switch(type) {
                case SEND_MESSAGE : processMessage(); break;
                case SEND_FILE : processFile(); break;
                case SEND_B_FILE : uploadFileContent(null); break;
                case GET_FILE : downloadFile(); break;
                default :
            }
        } catch(SaveExistsFileException e) {
            logInfo(e.getMessage());
        } catch(DownloadNotExistsFileException e) {
            logInfo(e.getMessage());
        } catch(EOFException e) {
            logInfo("客户端关闭socket, 线程: " + name + " 结束执行.");
            break; // 对方socket已经断开
        } catch(SocketException e) {
            logInfo("Socket异常: " + e.getMessage() + ", 线程: " + name + " 结束执行.");
            break; // socket异常
        } catch(Exception e) {
            e.printStackTrace();
            logInfo("线程: " + name + " 结束执行.");
            break;
        }
    }
}

```

图 4-4 构建线程的 Worker 的包装

提到 I/O 交互, 我们就又顺便扯到了异常, 此时小伙伴会问胖哥: 自定义异常有何用途? 这里就有了一个比较好的场景来展开解释了。

同学们可以想想, 平时使用到的异常有那些?

空指针、数据下标越界、SQL 异常、通信异常、交互异常、没找到类的异常等, 以及一些 Error, 看到异常后是否就知道了大概意思。换句话说, 我们不仅仅是通过简单的 message 来定位一些内容, 而是通过异常的“类型”规范化了它的大致范围。

异常类的定义可以没有任何的自定义信息, 但是它是一种类型的标志, 当然你也可以包装自己的一些对象, 以便于在程序中进行某些通用处理或特殊处理。

异常本身也可以理解为一些错误编号, 一些被规范后的错误编号。在 Java 世界里, 希望用面向对象的思想来处理问题, 而不是用过程式的判定语句来对不同的错误编号进行处理。

当我们去做设计时, 是否需要这样做呢? 也许在某些通用设计中, 会使得系统更加麻烦, 而且变得不是那么通用了。应当客观看待自己的项目或产品, 针对在现在或未来几年中可能的一些扩展, 去做一些设计, 而不是觉得某些东西好就非要用。这个思想不仅仅适用于自定义异常, 在其他的技术选择方面也依然适用。

有人问: 抛出异常是不是对性能影响很大?

第一, 是有一些影响, 若是有非常大的影响, 只能说异常使用不恰当导致的, 例如每个请求都抛出异常, 甚至通过异常来控制正常的流程, 那样肯定是不行的。换句话说, 我们不会因为一个系统变态地抛出一大堆异常导致性能下降而提出不要使用异常, 而是说明发现了频繁地抛出异常算是性能优化的一个突破点。其实, 在通常情况下在这里不用考虑太多, 需要考虑的是如何将系统设计得更清晰, 我们重点要解决的是“毒瘤”。

第二，抛出异常的影响及创建对象的巨大性能影响是很多年前对于 Java 的认识，许多朋友还一直沿用这些认识在思考问题，现代的 Java 虚拟机做了很多优化，这些性能开销几乎不用去做过多的考虑，只要别滥用、故意去导致一大堆的垃圾对象创建和异常就可以了，Java 的存在就是希望你的思维从这中间跳出来，更多地去思考系统的设计和架构。

在 Java 运行的过程中确实有很多开销，我们在关注的同时也能提升自己的能力，但是重点并不是在这里，因为系统中只有在适当的情况下才会抛出异常，而不是每个地方都要抛出异常。现代服务器的飞速发展，作为程序员更多的是需要写出干净漂亮的代码，构建优雅的架构和提升可读性。

继续说图 4-4 中的代码，每一次循环，先读取 1 个字节，然后用 switch case 来确定处理方式，而这个字节正好是需求中提到的传送类型，这些类型包括：发送普通消息字符串、发送文本文件、发送二进制文件、下载文件，分别对应到这里的 4 个方法来处理。

这里的每一个 case 的值，我们没有使用 1、2、3、4 数字来代表，都是静态常量，相信大家一看便懂意思。如果不是这样，这段代码时间长了可能没有人看得懂是什么意思，或许有文档来支持，不过在国内的开发过程中，大家觉得这样的方式“靠谱”吗？（这里也可以通过枚举来实现，胖哥为了使例子简单就使用了静态常量来实现，通常我们还是推荐使用枚举的，细节我们在后面提到一些设计思想时会说到。）

在实现方面，代码通过判定逻辑处理后，用了一个方法来处理，这似乎在面向过程的语言中是十分清晰的。胖哥为何没有用面向对象方式来处理呢？

胖哥认为：如果每种情况的处理都十分简单，那么这样处理也无所谓了，因为这种写法同样具有不错的可读性。知道了每种情况，应该调用不同的方法来处理，只是当我们根据业务的细节去扩展方法的内容时，逐步发现这些方法之间有许多内容是可以共享的，也有不共享的，有些是需要相互嵌套调用的，这个类逐步就会变得臃肿，本身是一个很干净的类，就是做逻辑中转处理，但此时你会发现有很多逻辑方法、业务方法都在里面，很多时候我们可能找不到程序的入口在哪里。

面向对象处理方式，通常会将这种处理分解到不同的类，例如 Message、File、二进制 File、下载文件会有单独的对象去处理，它们可能会有一些公共处理放在父类里面，不同的逻辑单元由子类来重写，比如处理普通的文本文件和处理二进制文件有相似之处。

本书中的相应 Server 端代码胖哥认为没必要，因此读者朋友不要认为胖哥都这样写了，我也这样写。为此胖哥在构建 Client 端代码时，就使用了更加接近于面向对象的方式，希望大家能发现两者的设计思路是有区别的（不过，这并不代表客户端程序就该这样设计，服务器端程序就该那样设计，胖哥只是希望大家能体会两者之间的区别，在本书下册的设计篇中会提到更多的设计理念）。

在这个例子中将静态常量、静态方法都放到了 Commons 的一个类中，其他的类是通过 import static 来引用的，在代码中如果出现 print、println 就是控制台输出，logInfo 方法也并非 log4j 的，而是简单地仿照日志格式在控制台的输出。这些设计在实际的场景中也许不是那么合理，在这里胖哥也只是针对特定问题而写的一个 demo 而已。

接下来，服务器端还有两个问题我们需要解决。

◎ 有了线程的 Worker 类后，要考虑的就是这些 Worker 应该在什么时候被创建。

最简单的处理就是在接收到一个 Socket 连接时被创建，一个 Socket 对应到一个客户端

(注意：这里并没有考虑线程池以及异步 I/O 的概念)。

◎ 如果涉及上传文件、下载文件，自然的在服务器端就需要有地方存储文件，目录应当是通用的，这最好使用类似于配置的方式来实现。在这个 demo 中，胖哥并没有使用配置文件，而是在 Commons 中写个静态常量来说明文件的基本路径。

此时，设计 SocketServer 的简单处理，如图 4-5 所示。

```
public class SocketServerMain {

    private final static List<Worker> workers = new ArrayList<Worker>();

    public static void main(String []args) throws IOException {
        initPath();
        ServerSocket serverSocket = new ServerSocket(8888);
        logInfo("端口已经打开为8888，开始准备接收数据.....");
        try {
            int index = 1;
            while(true) {
                SocketWrapper socketWrapper = new SocketWrapper(serverSocket.accept());
                workers.add(new Worker(socketWrapper, "socket_thread_" + index++));
            }
        } finally {
            serverSocket.close();
            interruptWorkers();
        }
    }

    private static void interruptWorkers() {
        for(Worker worker : workers) {
            worker.interrupt();
        }
    }

    private static void initPath() {
        File file = new File(SERVER_SAVE_BASE_PATH);
        if(!File.exists()) {
            boolean success = file.mkdirs();
            if(!success)
                throw new RuntimeException("无法创建目录：" + SERVER_SAVE_BASE_PATH);
        }
    }
}
```

图 4-5 Socket 服务器端的代码概要

首先初始化了保存文件的路径，如果路径不存在，则通过 `mkdirs()` 方法创建（这个方法可以同时创建多层目录，对应的方法 `mkdir()` 只能创建一层目录）。

文件路径初始化以后开启 8888 端口接收请求，当接收到一个请求的 Socket 连接后，创建一个 Worker 并放入 List 中，最后 finally 关闭服务器端 Socket 并将所有的 Worker 调用一次 `interrupt()` 方法的操作（从图 4-4 中的代码可以看到，在 `run()` 方法中，每次循环前都会通过 `isInterrupt()` 来判定 `interrupt()` 的状态）。

现在我们运行这段代码，来启动服务器端，如图 4-6 所示，当你看到这样的输出信息时，就代表服务器端已经运行起来了。

```
$java chapter04.socket.server.SocketServerMain
2013-11-17 20:25:44 [] INFO chapter04.socket.server.SocketServerMain - 端口已经打开为8888，开始准备接收数据.....
```

图 4-6 看到这样的输出就代表服务器端正常地运行起来了

接下来就需要建立客户端的程序。

客户端程序不用再去建立多线程，我们只需要做一个类似于简单的命令行交互 UI 就可以了，但是客户端需要有对输入命令的解析过程，将输入的数据解析为结构化数据，然后才能转换为与服务器端通信的数据格式，这似乎很麻烦，其实不然，前文中所规定的几种命令类型都是比较简单的。

为了说明前面提到的对象，所以我们这里将要进行多个对象的设计。为了实现规范的调用，会设计一个接口，这个接口刚开始建立的时候 API 肯定不是那么完善，不过我们不用担心这些，可以先考虑核心业务逻辑所需要的 API。

抽象的接口都是由 Client 端将命令和内容发送到 Server 端的，我们简单地将接口名称定义为：Sendable。

有了接口，现在需要来定义接口所需要的方法用以外部调用。

要发送数据，不可避免地要发送命令的内容，我们定义一个方法 `sendContent` 用于发送内容，传入 `SocketWrapper`，在发送中有可能会产生一些 `IOException` 需要处理，所以在方法的描述中需要有 `throws IOException` 的描述，以便于让外部知道有这个异常可能会被抛出，且需要处理。

不论哪一种数据的传送都会先发送操作的类型，再发送内容信息，这样就可以将发送操作类型独立出来做，而每个对象内部只关心数据发送，而不是类型发送（那么发送类型的代码就可以抽象出来了）。在这里增加了一个接口方法返回每个类自己的操作类型，以便于外部统一发送类型（也可以通过中间增加抽象类描述方法调用过程来完成，只是这里胖哥在后面还增加了一层 `Processor` 的处理，由 `Processor` 来发起这个调用）。Client 端的接口抽象如下：

代码清单 4-1 抽象一个发送消息的接口

```
import java.io.IOException;
import chapter04.socket.SocketWrapper;

public interface Sendable {

    public byte getSendType();

    public void sendContent(SocketWrapper socketWrapper)
        throws IOException;
}
```

也许在实际应用中接口还不够，但是目前的接口要足够清晰，也就是实现业务需求，要知道外部可以调用的 API 有哪些，以便于我们进一步设计和迭代，所以现在不必过分注重细节。

接下来考虑输入的每一行命令，都需要解析，解析以“行”为基本单位，因此胖哥设计了一个所谓的 `LineProcessor` 的类型，专门用于解析命令，将它转换为实际的指令信息（细心的同学应当发现这个过程还需要校验指令的有效性）。

每当输入一行命令后，就会创建一个 `LineProcessor` 对象，它负责解析输入的这行命令，判定命令有效性后，生成不同的命令处理类，并将输入的有效信息解析为多个 `token`，以便于对应的处理类直接使用这些 `token`。

这里所说的 `token`，是指字符串按照一定的分隔符分隔开后，每个逻辑单元字符串，在这个例子中分隔符仅仅是空格或 `\t`。如输入 `sendMsg aaa`，被解析为“`sendMsg`”和“`aaa`”两个 `token`。

分析了这么多，貌似很神奇，其实蛮简单的，来看看这个 `LineProcessor`，代码如下：

代码清单 4-2 `LineProcessor` 的源码

```
public class LineProcessor {

    private String []tokens;

    private Sendable sendable;
```



```

public LineProcessor(String line) throws Exception {
    line = preLine(line).trim();
    if(line.trim().length() == 0) { //没有任何操作
        throw new NoOptionException();
    }
    tokens = line.trim().split("\\s+");
    String firstToken = tokens[0];
    Class <?>clazz = findSendableClassByOrder(firstToken);
    try {
        sendable = (Sendable)clazz.getConstructor(String[].class)
            .newInstance(new Object[] {tokens});
    } catch (InvocationTargetException e) {
        throw (Exception)e.getCause();
    }
}

public void sendContentBySocket(SocketWrapper socketWrapper)
    throws IOException {
    if(sendable != null && sendable.getSendType() > 0) {
        socketWrapper.write(sendable.getSendType()); //发送类型
        sendable.sendContent(socketWrapper);
    }
}

private String preLine(String line) {
    if(line == null) return "";
    if(line.startsWith(">")) return line.substring(1);
    return line;
}
}

```

在构造方法中，取出了 token 列表，将取出的 token 列表中的第 1 个 token 赋值给局部变量 firstToken，这个 token 标志了发送的类型，通过这个字节使用 findSendableClassByOrder 方法找到对应的处理类，然后通过获取其构造方法来实现实例化的过程。

sendContentBySocket 方法负责传送命令类型，以及调用对应类型的处理类来进行进一步的处理。

代码读到这里，很多小伙伴会想想 findSendableClassByOrder 做了什么。这十分类似于从页面上传递不同的 URL 或参数会调用到后台不同的类或方法。在这里，胖哥事先将 4 种类定义好与命令映射到一个 Map 中，如图 4-7 所示（通常这种映射关系配置是写到配置文件中的，系统启动时将它们加载到内存中）。

```

private final static Map<String , Class<? extends Sendable>> ORDER_CLASS_MAP
    = new HashMap<String , Class<? extends Sendable>>() {
    private static final long serialVersionUID = 3431099761909680054L;
    {
        put(SEND_MESSAGE_STR.toLowerCase() , MessageSender.class);
        put(SEND_FILE_STR.toLowerCase() , FileSender.class);
        put(SEND_B_FILE_STR.toLowerCase() , BFileSender.class);
        put(GET_FILE_STR.toLowerCase() , GetFileSender.class);
    }
};

```

图 4-7 实现定义好的类的对应关系

通过这个已经存在的 Map 进一步提供了 findSendableClassByOrder 方法来根据输入的类型获取处理类，并在 findSendableClassByOrder 方法中添加了默认处理方式，也就是当命令不是这 4 种类型时也需要一个类来处理，代码片段如图 4-8 所示。

这样外部始终能得到一个处理类的对象，不用关注空指针的问题。

```

public static Class<? extends Sendable> findSendableClassByOrder(String order) {
    Class<? extends Sendable>clazz = ORDER_CLASS_MAP.get(order.toLowerCase());
    return clazz == null ? DefaultSender.class : clazz;
}

```

图 4-8 findSendableClassByOrder 方法代码

回到代码清单 4-2 中, 在获取到类创建实例时, 做了一个异常的 cache, 在代码中捕获到 InvocationTargetException 异常后, 将 e.getCause() 抛出, 原因在于此异常被反射包装了一层, 这里直接看到的异常并不是真正的异常, 真正的异常需要通过 getCause() 来获取 (从这里我们还学到了从层层嵌套的异常如何找到内层的异常)。

接下来就要实现几个 Sender 类了。在这里胖哥不会贴出所有的代码, 大家可以参考光盘中 src/chapter04/socket/client/sender/目录下的内容。以最简单的 sendMsg 为例, 将其展开说明, 其代码如下:

代码清单 4-3 MessageSender

```

public class MessageSender implements Sendable {

    private String message; //普通的 message 消息

    private byte []messageBytes; //消息发送时使用

    private int length = 0;

    public MessageSender(String []tokens)
        throws UnsupportedOperationException {
        if(tokens.length>= 2) {
            message = tokens[1];
            this.messageBytes = message.getBytes(DEFAULT_MESSAGE_CHARSET);
            this.length = messageBytes.length;
        }else {
            throw new RuntimeException("请在 sendMsg 后面添加内容。");
        }
    }

    @Override
    public void sendContent(SocketWrapper socketWrapper)
        throws IOException {
        println("我此时向服务器端发送消息: " + message);
        socketWrapper.write(length);
        socketWrapper.write(messageBytes);
        println("发送消息完毕。");
    }

    @Override
    public byte getSendType() {
        return SEND_MESSAGE;
    }
}

```

在这里, 构造方法解析出要发送内容的长度以及字节数组后, 将内容放在对象的引用中, 在调用 sendContent(SocketWrapper)时, 就是简单的发送长度和字节数组两个动作。

其余的处理类都是差不多的处理, 在掌握代码骨架的情况下, 剩下的工作就是添砖加瓦了。

有人还会问: 胖哥, 现在还没看到 SocketClient 在哪里? 其实不要着急, 因为解析做好了, 交互这部分代码并不难, 或者说我们设计好了 LineProcessor 后, 代码中就仅仅是循环调用的逻辑了, 如图 4-9 所示。

客户端发送了这么多内容，服务器端也会输出相应的内容，如图 4-11 所示。

```
2013-06-20 23:47:44 [I] INFO chapter4.socket.server.SocketServer - 端口已经打开为8888, 开始准备接受数据.....
2013-06-20 23:48:52 [I] INFO chapter4.socket.server.Worker - 我是线程: socket_thread_2, 开始后动接收客户端传来数据.....
2013-06-20 23:49:02 [I] INFO chapter4.socket.server.Worker - 线程: socket_thread_2 接受到来自客户端传来message信息: 你好!
2013-06-20 23:49:12 [I] INFO chapter4.socket.server.Worker - 线程: socket_thread_2 接受到来自客户端传来message信息: 第二条消息来啦!
2013-06-20 23:49:31 [I] INFO chapter4.socket.server.Worker - 线程: socket_thread_2接受来源客户端发送文件, 字符集为: gbk
2013-06-20 23:49:31 [I] INFO chapter4.socket.server.Worker - 线程: socket_thread_2接受来源客户端发送文件, 来源文件名为: aaa.txt
2013-06-20 23:49:31 [I] INFO chapter4.socket.server.Worker - 线程: socket_thread_2接受来源客户端发送文件, 文件长度为: 37
中文
\中文中文中文中文
中文中文中文

文件接收完毕并保存, 向客户端发送确认信息, 实际接受内容长度为: 37
2013-06-20 23:49:55 [I] INFO chapter4.socket.server.Worker - 线程: socket_thread_2接受来源客户端发送文件, 来源文件名为: Hadoop权威.PDF
2013-06-20 23:49:55 [I] INFO chapter4.socket.server.Worker - 线程: socket_thread_2接受来源客户端发送文件, 文件长度为: 62422510
.....
2013-06-20 23:49:56 [I] INFO chapter4.socket.server.Worker -
文件接收完毕并保存, 向客户端发送确认信息, 实际接受内容长度为: 62422510
2013-06-20 23:50:20 [I] INFO chapter4.socket.server.Worker - 线程: socket_thread_2接受客户端下载文件, 下载文件名为: Hadoop权威.PDF
实际发送文件长度为: 62422510
2013-06-20 23:50:33 [I] INFO chapter4.socket.server.Worker - 线程: socket_thread_2接受客户端下载文件, 下载文件名为: xxxx.abc
2013-06-20 23:50:33 [I] INFO chapter4.socket.server.Worker - 无法下载文件: d:/javaA/upload/xxxx.abc, 因为服务器端不存在这个文件....
2013-06-20 23:50:41 [I] INFO chapter4.socket.server.Worker - 客户端关闭socket, 线程: socket_thread_2 结束执行.
```

图 4-11 服务器端的输出



注意 这里输出的内容并不是 log4j 输出的，而仅仅是使用 System.out 来模拟 log4j 的输出格式达到类似输出的目的，真正的 log4j 输出有多种选项和形式，并有助于提升日志输出性能。本节内容的重点不在此，因此仅仅做简单模拟。

下面再看看客户端与服务器端的一些简单的输出对应。

服务器端启动后，输出了第 1 条日志信息，代表成功启动，说明开放的端口。

这时有一个客户端连接到服务器端后，服务器端显示创建了一个名称为“socket_thread_2”的线程来处理，由于将线程的名称与客户端对应，因此它也标志了一个单独的客户端来源。

当客户端输入了“sendmsg 你好!”命令后，服务器端输出了相应的第 2 条日志信息，说明已经收到了对应的消息。客户端发送后续的信息时服务器端也相继输出。

此时传递一个文本文件 aaa.txt，设定了字符集为 GBK（注意：字符集需要自己知道，只有设置了正确的字符集才能正确输出），服务器端首先接收到字符集，然后接收到文件名并将文件名输出（文件名是统一规约的字符集传送的），再接收到文件的字节长度，这里接收到的长度是 37，说明文件总长度是 37 字节。进一步输出文件内容（转换为字符串后的文件内容），此时会在服务器相应的路径下产生一个同名的文件，内容也与客户端传送的内容一致。

以同样的方式传递二进制 PDF 文件，服务器端输出了文件名以及文件的大小，也许文件较大，需要一点时间，在传送过程中会不断显示“.....”以表示还在进行中。

接下来是下载文件服务器端输出了下载的文件名，然后输出给客户端，此时又测试服务器端文件不存在的情况，服务器端也会输出相应的结果，服务器端最后一行数据是指客户端调用了 exit 命令退出，被服务器端捕获到，服务器端说明线程可以结束了。

这是一段客户端与服务器端通信的简单程序的模拟，实现了一种简单的客户端向服务器端索要各类资源的基本功能。在实际场景中，这样的代码能否使用呢？不能，这段代码看似简单，却依然存在很多的问题。

问题 1：这里的网络通信是没有用户校验的，谁都可以连接上。如果要进行校验，应当如何做？

很多 Java 开发人员都想到：JDBC 与数据库通信需要用户名和密码信息，Web 应用中很多系统后台需要用户名和密码的验证。同样的道理，如果要进行验证应当如何考虑呢？

胖哥在这里没有写出代码的具体实现，只给大家提供一些方法作为参考。例如，在 Web 登录时要求输入用户名和密码，传送到服务器端后进行验证，验证好后会分配一个唯一的 ID，这个 ID 默认就是 `jsessionid`，不过你也可以不依赖这个 ID，而是用自己的方法来生成另一个唯一的 ID，这个 ID 会写入到浏览器的 Cookie 中，并在服务器的某个地方也保存一份与用户登录信息相关的映射，在访问该站点时任何请求都会携带相关的 Cookie 信息，那么服务器端接收到这个唯一值后，便可以通过服务器端所保存的那一份映射关系来找到登录用户的基本信息。

问题 2：没有做数据加密，用户名、密码也没做加密，这肯定是问题。

加密在一定程度上是必须要做的，如果没有加密就可以被某些网络抓包工具截取到数据的明文信息，自然信息就会泄露。至于加密的细节和方法有很多，有的小伙伴说用 Base64 吧，其实胖哥认为这个加密算法基本等于明文，不要在实际程序中使用，原因是这个算法太通用了，任何语言都有通用的解析类，也不需要任何密匙就可以解密。

如果要求不是那么高，通常用一些类似于 AES、Blowfish 等带有密钥的算法来加密就行。也可以在服务器端只存储密文，使用原来密码配合一段字段自定义的长串组合而成并采用 MD5 加密后直接存储，匹配密码也是匹配密文是否一致。也可以根据需要对开源的加密算法进行部分改造，这样解密会更难一些（例如，客户端与服务器端进行握手，服务器端根据客户端信息计算出来一个密钥，然后交给客户端，但这仅仅在第一次握手发生时，以后双方传递数据时不再传递密钥，那么网络中抓包工具抓到密钥的概率就会非常低，因为在大量的网络交互过程中抓包工具并不是每个包都能抓到的）。

这里提到的在 MD5 前添加自定义规则的内容，复杂性由自己决定。许多小伙伴通常喜欢把密码设置得太简单，我们把它叫“弱密码”，此时便有一群“暴力分子”会通过计算机手段事先将网络中的许多常用词汇加密为 MD5 值后存储起来，就可以映射出真正的值，这叫“解密”。

问题 3：在上面的 Client 代码中，在上传文件时，为何发送完文件名后，要先读取服务器端返回的状态再进行传送呢？

在需求中描述到：如果服务器端已经存在这个文件，那么服务器端就不会去读取客户端传来的数据了，也就是 Socket 传来的数据不会读取，如果此时客户端向服务器端传来数据，而且数据很大（例如传递 1GB 或更大的文件），那么就会堆积在内存中，这时可能会导致 OOM 现象，虽然程序会在结束后关闭流，但是在关闭前就可能出现大量数据的堆积。

问题 4：胖哥的代码中也使用了 `System.out`，输出类似于 `log4j` 格式的内容，是不是 `log4j` 的实现与 `System.out` 一样呢？

不完全是！本节为了不引入三方依赖包，因此直接用 `System.out` 来实现。不过，为了避免大家认为这样实现是正确的，胖哥简单地讲解一下 `log4j` 的内在实现机制。

`log4j` 可以输出到屏幕、文件、远程某个地方、异步输出等，都是为了满足某些特定的需求。就屏幕输出来讲，它的实现是通过 `System.out` 或 `System.err` 包装后实现的，默认是 `System.out`，当然可以使用 `target` 来指定，在输出时并没有任何优化，而是直接通过输出流

将内容进行相应字符集编码后进行输出。换句话说，在屏幕上的输出，log4j 的性能与 System.out 一致，虽然这不是我们想看到的，因为这个对象是 JVM 进程全局唯一的，在输出时还会使用 synchronized 让这个动作串行化。

不过，屏幕输出通常在程序员本地调试代码时使用，大多数服务器端运行的程序不会采用这样的方式，而是将日志保存在文件中或其他的地方。这个时候 log4j 会在配置中提供一些 Buffer 机制，例如采用文件输出日志，当启用参数 BufferedIO 时，则会在默认的文件输出上套用一层 BufferedWriter，而这个 Buffer 的默认大小为 8192，不过是 8192 个字符而不是字节。若需要修改这个 Buffer 的大小，则可以设置参数 bufferSize。当启用 Buffer 机制后，就不会在每次调用打印日志方法时写文件了，而是会先写入到内存中，当内存的 Buffer 不够时才会将其一次性写入磁盘中，通过这样的方式就提升了系统的性能。

►► 4.1.4 Java 的 I/O 流是不是很难学

在很多地方，大家都会听到老师说 Java 的“流”很难教，学生也难学。

胖哥不否认这一点，因为它的层层嵌套，它提供的一大堆类的处理，让人确实觉得很不好接受。当然，教科书上不得不这样写，包括所有的代码大全、技术大全通常都会这样写，也通常会画一张类的关系图给大家看，这张图对于已经比较懂的人来讲，就是知识的完善和总结；但对于还没有太多接触的人来讲，就感觉是一个很抽象的东西。如果你觉得自己是后者，那么下面的内容也许可以带你找到一些方法（但是它不是神功能让你速成，方法只是让我们找到一些思路，具体还得靠自己努力）。

首先，我们要了解 I/O 的基本概念就是“输入/输出”。

◎ 输入就是从其他终端设备向内存或者程序中输入数据，对应 I/O 中的 I，也就是 Input。例如从磁盘中读取数据，从网络中读取数据，从键盘上读取数据到内存或者程序中（有些时候为了适配某些接口，还会在内存中直接通过字符串模拟一个输入流），它们都统一叫作 Input。

◎ 输出正好相反，对应 I/O 中的 O，也就是 Output，例如从内存写入到磁盘，从内存写入到网络，从内存写入到显示器等设备上，都把它们叫作 Output。

不过，有些 API 很诡异，它要向你传递一个文件，却让你提供一个输出流，这样它可以通过这个输出流写入到一个地方去，而我们唯一可以做的就是指定一个地方让它写进来。

对于输入/输出，最终会使用字节来处理，所以大家首先需要知道的是 java.io.InputStream、java.io.OutputStream，它们都是通过字节来与终端进行输入/输出的。

java.io.InputStream、java.io.OutputStream 是两组接口，它们只是定义了输入/输出最基本的 API，具体从什么终端读写数据，如何读写数据，将由它们的子类来实现。

例如 FileInputStream、FileOutputStream 实现了上述两组接口，看名称便知道实现了对文件的输入/输出操作。

FileReader 与 FileWriter 又是什么东西呢？

教科书上通常会告诉你这是字符流，是操作字符的输入/输出，有的小伙伴便认为在网络传送过程中可以用字符，而且不会出现任何问题，其实许多问题往往出现在这里。

Reader/Writer 内在的实现是通过 sun.nio.cs.StreamDecoder、sun.nio.cs.StreamEncoder 来对字符集进行处理，最终还是通过字节来完成发送和接收的。而当程序中没有设定字符集时，将会采用一些与环境变量相关的字符集来默认转码，此时就可能会出现：“程序在这个

地方没有乱码, 在另一个地方出现乱码”的情况。因此, Reader 和 Writer 就是字节转换为字符的桥梁, 在实例化时最好自己设定字符集, 不要让它自适应; 否则, 我们将被技术所驾驭。例如:

```
newInputStreamReader(inputStream, "GBK");  
new OutputStreamWriter(outputStream, "GBK")
```

这样得到的一个 Reader 将通过 GBK 编码将 byte[] 转换为字符形式返回, 此时读取出来的都是 char[] 的信息或 String 的信息。同样的, Writer 也将写入的字符转换为 GBK 的字节来传送。

因此 Reader 和 Writer 本身也归属到基本的输入/输出上, 它是在 Java 语言层面帮我们做了一个字符集的转换器而已。此时你应当知道了这个根源, 那么阅读接下来的内容相信会轻松许多。

关于流的实际用法中还会涉及许多套用关系, 例如在上述的 InputStreamReader 上可以继续嵌套 BufferedReader, 这样层层嵌套搞得我们头很大, 这也许是难教与难学的一个关键地方之一, 胖哥接下来会专门说说嵌套的问题。

上面我们提到了 FileInputStream 和 FileOutputStream, 现在来简单横向扩展一下知识面 (注意: 接下来的内容中许多地方提到的 XXXStream 都是 InputStream、OutputStream 的简称, 例如 FilterXXXStream 就是 FilterInputStream 与 FilterOutputStream 两者的简称)。

- ◎ 简单的网络通信会利用到 SocketInputStream 和 SocketOutputStream, 它们分别继承于 FileInputStream 和 FileOutputStream, 不过 SocketInputStream 和 SocketOutputStream 本身的缓冲区是在内核中完成的。
- ◎ 当期望直接通过 API 处理流当中不同的数据类型时 (例如通过输入流读取一个 int 数据或通过输出流发送一个 long 数据), 就要使用 DataInputStream 和 DataOutputStream 来进行嵌套。除此之外, DataInputStream 还有一个 readFully(byte[]) 功能, 这个功能会尝试将传入的 byte[] 数组读满才返回 (也可以传入 offset、count), 在没有读满的情况下对方关闭连接则会抛出 EOFException。它们同时还提供了 readUTF、writeUTF 操作, skipBytes 可以跳跃读的位置等强大的 API。
- ◎ 当需要对数据做压缩和解压缩时, 就需要使用到 ZipInputStream、ZipOutputStream、GZipInputStream、GzipOutputStream, 读写 jar 包可以使用 JarXXXStream。这些类在三方框架中还有一些功能强大的扩展。不过大家要注意的是, 这些压缩流不仅仅可以套装在文件的读写流上, 只要是任何输入/输出流都可以套装, 例如套装在网络输入/输出流上可以节约网络通信量, 也可以套装在一个 ByteArrayXXXStream 上得到压缩后的 byte[] 数组信息。
- ◎ 当希望通过内存的某些数据得到一个输入/输出流, 以满足某些 API 的输入要求时, 就可能会用到 ByteArrayXXXStream。例如, 通过一个 String 得到一个输入流, 则可以使用:

```
InputStream in = new ByteArrayInputStream("字符串".getBytes("GBK"));
```

- ◎ 当需要对自定义对象做输入/输出时, 就需要使用到 ObjectInputStream 和 ObjectOutputStream。
- ◎ 而类似的 ServletInputStream、ServletOutputStream 是 Web 层对原本 Socket 的流做

了很多层的包装等。

- ◎ 当需要对输入/输出的数据做 Buffer 时, 则可能会用到 `BufferedInputStream` 和 `BufferedOutputStream`。
- ◎ 当需要输出流与输入流进行对接时 (例如输出的内容, 在相应的一个输入流中可以得到结果, 反之也是成立的, 就看对接的方式), 则可以使用 `PipedInputStream` 和 `PipedOutputStream`。
- ◎ 通常流是顺序向前读操作, 如果希望有回退功能, 那么套一层 `PushbackInputStream`, 就有一个 `unread` 方法来达到目的。
- ◎ 希望使用 `print` 相关方法时, 就用 `PrintStream` 来实现, 其实 `System.out` 和 `System.err` 就是它来实现的。如果希望系统中发生 `System.out` 和 `System.err` 调用时重定向到文件中, 则可以使用 `System.setOut(PrintStream)` 和 `System.setErr(PrintStream)`。同样的, 如果系统中的 `System.in` 键盘输入流希望修改为其他的输入流, 则可以使用 `System.setIn(InputStream)`。

还有许多流, 我们没法一一列举, 其实不论是什么流, 都是 `Stream`, 归根结底就是输入/输出, 通过字节来完成, 外部都是功能的修饰和包装而已。基于对根本的理解, 这些修饰和包装注意常见的几种, 在实战中与三方框架加以灵活应变, 就可以铺开自己的知识面。

这里的各种“流”都提供了独立的一类功能, 如果希望提供多种功能应该如何去做呢? 聪明的同学便联想到了“嵌套”, 其实除了嵌套外, 继承也有类似的作用, 我们就从这个小问题开始说起。

当两个流的功能十分类似时, 那么用继承来完成, 实现的代价是很低的。例如 `SocketXXXStream` 是基于 `FileXXXStream` 来实现的, 因为 `SocketXXXStream` 有很多处理功能都和 `FileXXXStream` 类似, 可以共享这些方法, 少数方法重写一下即可达到目的。

“嵌套”是大家经常看到的 `Stream` 一层包装一层 (这种嵌套在某种意义上可以理解为修饰), 不论如何嵌套, 其返回的对象始终适配顶层接口。我们发现的规律是, `Stream` 相关的类都适配接口 `InputStream` 和 `OutputStream`, 大多数情况下嵌套都是允许的, 因此这种流可以多层嵌套。

许多地方可能会用管道来比喻这种一层套一层的情况, 从原理上讲, 它就是将一个 `Stream` 的对象引用设置为另一个 `Stream` 的属性, 当发起调用时, 会逐层向下调用, 每一层可以做自己功能的修饰, 因此它可能会具有多个 `Stream` 的特征, 达到多种 `Stream` 的功能合并。

举个简单例子, 当读取文件时, 可以用 `FileInputStream`, 不过依然可以用 `BufferedInputStream` 在外面套一层, 那么它便具有了 Buffer 的功能来读取文件。

上述具有各种功能的流大多是 `FilterXXXStream` 的子类, 不过 `FilterXXXStream` 这个类几乎什么也没干, 或者说它只是做了十分简单的嵌套封装, 它的构造方法提供对应的 `InputStream` 或 `OutputStream` 的参数入口, 所有的 `Stream` 都能传递进去, 它将传入的 `Stream` 对象的引用保存在自己的一个属性中来实现嵌套。`FilterXXXStream` 本身是最简单的流嵌套方式, 通过源码会发现它仅仅做了一个引用的保存, 所有的功能都是通过间接调用构造方法中传入的 `Stream` 来完成的。

Java 为什么要这样做呢?

这是一个中间过滤层, 它将接口的方法实现了一层通用的代理处理, 也就是默认与原来对象的功能保持一致, 许多具体功能修饰可以基于这个类进一步通过继承的方式来完成, 这样一来, 子类就不用重写所有的方法了, 只要重写那些需要修改为具体特征的方法就行。例如, `BufferedOutputStream` 会重写 `write` 相关的方法, 以便将数据先写到一个数组中, 数组满时才会发送数据; 它也会重写 `flush()` 方法, 以便强制将数组的数据发送出去, 但它并不需要重写 `close()` 等方法。其他的子类也是类似的, 根据需要实现某些特定的特征去重写某个方法。

例如 `DataXXXStream` 就是继承于 `FilterXXXStream` 来实现的, 因为它有大量的方法直接沿用 `FilterXXXStream`, 不需要自己来编写, 这也就是 `FilterXXXStream` 类起到的一个被共享的作用, 我们通常很少去使用这个类, 但是它被继承后可以很轻松地扩展。

用什么思路来快速掌握层层继承与嵌套呢?

方法其实并不难, 还是本着输入/输出的基本概念, 嵌套、继承的基本思想, 掌握常见的输入/输出类, 在实际应用中通过“顺藤摸瓜”的方法就可以找到自己想要的东西。

现在回过头来谈谈大家非常喜欢使用的 `Reader/Writer`。

在通常情况下, 只有处理文本类数据时才会用到 `Reader/Writer`, 其代表是: `InputStreamReader` 和 `OutputStreamWriter`, 它是在 `Stream` 基础上增加了一个转换器, 不用转换器, 自己通过 `byte[]` 数组也能转换为字符串。不过, 通过转换器的方式可以使得程序更像面向对象的程序, 因此它的存在也是有很多实际价值的。

大家喜欢叫它字符流, 是因为它的读写方式是以字符为单位的, 而操作字符的方法与字节流操作字符的方法十分类似, 不过大家要清楚的是内读写都是字节。

举个例子: 字节流读取 1 个字节就是 1 个字节, 而字符流要读取 1 个字符, 则可能要读取多个字节, 它会根据设置的字符集不同, 取不同长度的字节转换为对应的字符, 或者将字符转换为不同长度的字节发送 (这一点十分重要, 大家一定要清楚, 否则永远不清楚字符流是怎么回事, 那么自然无法搞清楚许多奇怪现象的原因)。

字符流也提供许多功能, 我们来随便列举几个。

字符流与字节流类似, 也有 `Buffer` 机制来提升性能, 其产物为 `BufferedReader` 和 `BufferedWriter`, 同样的它也是一个数组实现, 不过此时为一个 `char[]` 数组, 而不是 `byte[]` 数组, 默认长度也是 8192 (但是占用的空间肯定要多 1 倍)。

如果需要对“读写的行”进行标记处理 (即记录行号), 则 Java 又扩展了 `LineNumberReader` 和 `LineNumberWriter`。

由于某些接口需要参数中传入 `Reader/Writer`, 如果程序中的数据是 `String`、`char` 等其他类型的数据, 则可以通过 `StringReader` 和 `StringWriter` 来满足需要。

与 `Stream` 类似, `Reader/Writer` 也提供了 `FilterReader/FilterWriter` 给子类来继承扩展使用。

如果需要使用到 `Print` 相关方法, 则可以使用 `PrintWriter` 来实现。

讲了这么多, 大家应该了解到 Java 语言已经为我们写好了非常多、非常棒的 API, 可以满足非常多的需求。如果需要在实际场景中扩展, 则也不难。

下面总结一下这一节的内容。

◎ 按照 I/O 基本原则分为输入流和输出流, 以当前程序为目标, 输入程序为 `Input`, 输出程序为 `Output`, 最终以字节为处理的基本单位。

- ◎ 按照明显的 API 划分为字节流和字符流，我们要以字节流为基础来掌握字符流，而不要因为字符流简单就忽略了字节流，那样不利于理解，容易让自己的思维受到限制，学得“浑浑噩噩”。
- ◎ 嵌套都是为了修饰方法为新的功能，让功能组合起来，让自己所使用的流的内在功能变得拥有多个流的功能。
- ◎ Stream 的顶层接口都是 InputStream 和 OutputStream，因此在嵌套后，外部的类通常也适配原来的输入/输出接口。
- ◎ 如果要将字节流转换为字符流，使用 InputStreamReader 和 OutputStreamWriter 作为中间桥梁，就适配到 Reader/Writer 上了，不过大家要注意字符集的问题。

疑惑：胖哥你说如此嵌套，该如何关闭流呀？

答曰：在前面提到过，嵌套就是将一个 Stream 的引用放在了当前 Stream 对象的一个属性中。当前 Stream 对象发起 close() 操作时，会间接调用内部的 Stream.close() 方法来完成实际的动作，它自身可能会保存一个状态或做一些 Buffer 处理。

以 FilterOutputStream 为例(很多的 OutputStream 都是继承它来编写的)，当它发生 close() 操作时，它就会先调用一次内部的 flush()，然后再调用被引用对象的 close() 方法，它防止的是类似 BufferedOutputStream 还存在未输出的数据，代码如图 4-12 所示。

```
public void flush() throws IOException {
    out.flush();
}
public void close() throws IOException {
    try {
        flush();
    } catch (IOException ignored) {}
    out.close();
}
```

图 4-12 FilterOutputStream 的 close()/flush() 方法

推测：给人的感觉好像是无论怎么关闭都无所谓，因为关闭最外层，也最终会关闭最内层，所以关闭哪一层都可以，真的是这样吗？

答曰：通过我们的理论分析，关闭外层和关闭内层感觉上是一样的，但是有些小细节需要大家注意，主要有以下两点。

- ◎ 拿 BufferedOutputStream 来讲，它嵌套了另一个 OutputStream，如果程序中也有这个被嵌套的 OutputStream 的引用，直接调用它的 close() 方法，那么这个 BufferedOutputStream 里面就有可能会有部分数据未输出，从而导致部分数据丢失。

许多朋友会问：我的程序已经写入文件了，可是文件中为什么一直没有数据呢？而且我的流已经关闭了？

- ◎ 关闭了 BufferedOutputStream 是不是就完全没问题了呢？未必！

因为嵌套可以一层嵌套一层，也可以对同一个 Stream 使用多个 Buffer 来嵌套。这就类似于在同一个下水道口中有多个蓄水池。蓄水池通常在满的时候会放掉里面的水，也可以主动放掉里面的水，前面也提到在 close() 时会先放掉所有的水，然后将下水道也关闭。如果多个蓄水池对应到一个下水道，一个蓄水池在做 close() 时也把下水道 close() 了(逐层关闭)，那么另一个蓄水池的水就成了废水，换句话说，另一个蓄水池中的水还没有放入下水道。

也就是说，虽然设计上允许我们将流来回嵌套、多层嵌套，但是我们不能在程序中滥用它，滥用它不仅使得代码的可读性降低，也使得系统容易出现不可预见的问题。

许多同学的第一反应还会认为多个 Buffer 是不是性能会高一些，其实要考虑到下水道的最大出口就那么大，Buffer 默认的 8192 大小是有一些 I/O 的经验数据作为参考的，通常不用去修改它，在特殊情况下也可以自己调整 Buffer 的大小来改善性能。

小伙伴看到这里应该有点懂了，不过发现这个流的使用依然是比较烦人的，尤其是做一些简单文件的操作如此麻烦不堪，其实技术只要不太好用就会不断革命，现在有非常多现成的东西让这些操作变得十分简单。

以前面提到的文本文件读取为例，通常经过多层嵌套后才能得到一个 `BufferedReader`，还得在中间的转换层设置字符集。JDK 1.7 提供了一个 `Files` 类，它内部包含了大量的文件操作，例如文件读写、拷贝等都不需要关心太多的细节。例如想用某种指定的字符集读取一个文本文件，那么通过下面这种方式就可以获得 `BufferedReader`。

```
Files.newBufferedReader(Paths.get("/tmp/xxx.txt"), Charset.forName("GBK"));
```

4.2 Java I/O 与内存的那些事

4.2.1 常规 I/O 操作的运作过程

在前文中提到过 Java 的基本 I/O 的简单模型，在 Java 程序普通的输入/输出过程中，都会经历从 Kernel 到 JVM 的内存拷贝。

例如要做一次文件拷贝，一般情况下会开启一个输入流将文件读取到内存，然后开启一个输出流将内存中的数据输出到另一个文件中。它经历的动作并不是直接从磁盘到磁盘，也不是从磁盘直接到 JVM 内存，而是会经过一个内核区域。在输入数据时会先进入 Kernel 区域，再拷贝到 JVM，输出时也会先由 JVM 拷贝到 Kernel 区域，再输出到终端。

许多技术爱好者就在思考：虽然内存的拷贝是十分快速的，但是对于反复大量的这样拷贝，开销也许并不小。因此，这群技术爱好者有一些想法：跳过 JVM 拷进、拷出的动作，或许在某些场景下能够派上用场。

4.2.2 DirectBuffer 的使用

`DirectBuffer` 是一个接口，实现类通常是 `DirectByteBuffer`，通过 `ByteBuffer.allocateDirect(int)` 来获取，此时获取到的是一个 `ByteBuffer` 接口类型的对象，需要强制类型转换为 `DirectBuffer` 接口，才可以得到和它相关的 API 调用。关于 `DirectByteBuffer` 的接口和父类关系如图 4-13 所示。

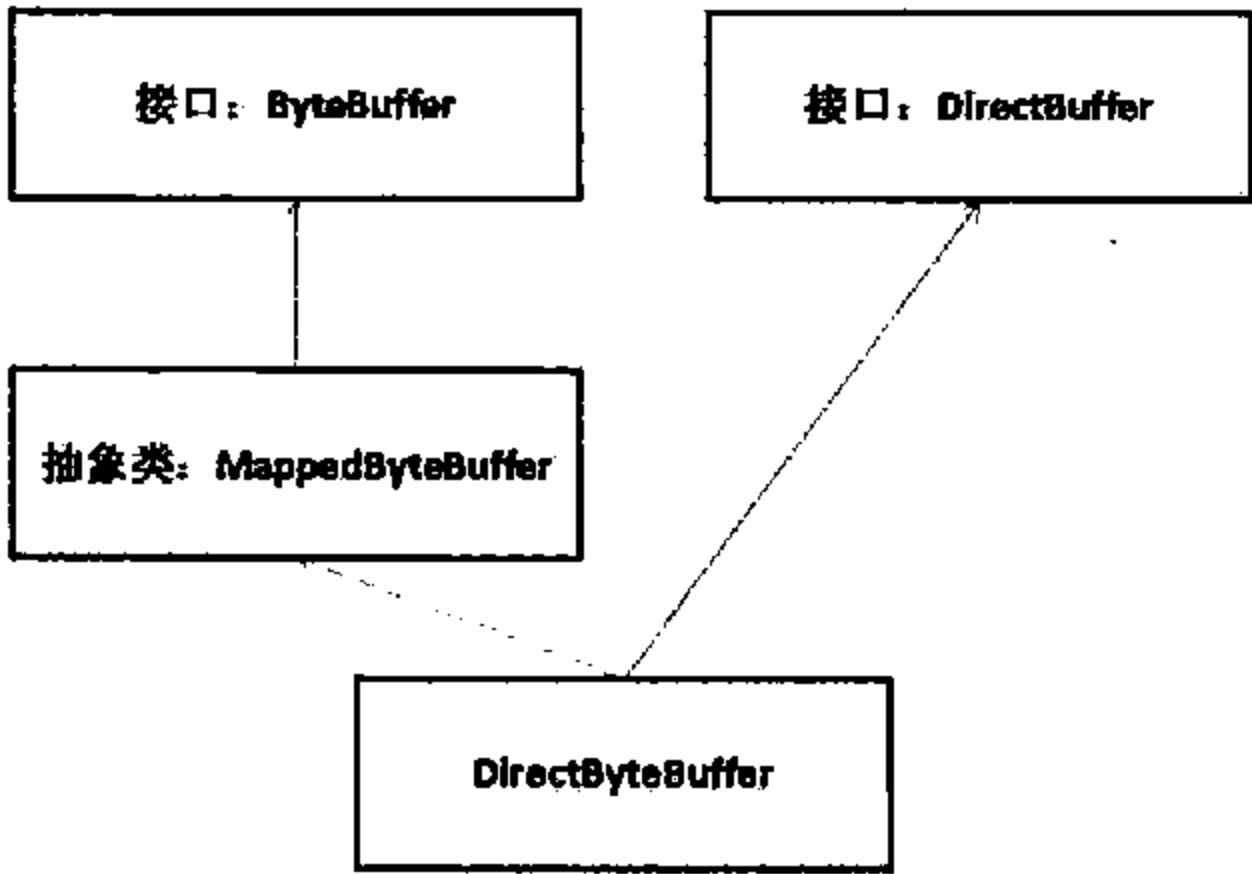


图 4-13 DirectByteBuffer 的接口和父类关系

在第 3 章中我们介绍过 DirectByteBuffer 区域模拟 OOM 的情况，也在那里阅读过它的一些源码。通过 OOM 的一些模拟和分析，不难发现它已经与-XX:+DisableExplicitGC 结下了“千年的梁子”。同时也发现它在分配内存时启用了 JVM 全局锁，还会调用 System.gc()，内部操作时还调用了 sleep(100)。通过这些大家应当发现它分配空间时所产生的开销是很大的，以至于我们很多时候得到的测试结论与所想的不太一样。DirectBuffer 应该还是有存在价值的，下面说个拷贝文件的例子。

胖哥首先写一段用普通的 FileInputStream 和 FileOutputStream 拷贝文件的代码来做测试（记得将其他占用系统资源的进程关闭，避免干扰）。

代码清单 4-4 用不同的 Stream 来拷贝文件

```
public static void copyFile(String srcFileName, String dstFileName)
    throws IOException {
    File srcFile = new File(srcFileName);
    File dstFile = new File(dstFileName);
    if (!srcFile.exists())
        throw new RuntimeException("src file not exists!");
    if (dstFile.exists())
        throw new RuntimeException("dst file exists!");
    DataInputStream inputStream = null;
    FileOutputStream outputStream = null;
    try {
        inputStream = new DataInputStream(new FileInputStream(srcFile));
        outputStream = new FileOutputStream(dstFile);
        int fileAvailable = inputStream.available();
        if (fileAvailable <= COPY_FILE_SIZE) {
            byte[] bytes = new byte[fileAvailable];
            inputStream.readFully(bytes);
            outputStream.write(bytes);
        } else {
            byte[] bytes = new byte[COPY_FILE_SIZE];
            int len = inputStream.read(bytes);
            while (len > 0) {
                outputStream.write(bytes, 0, len);
                len = inputStream.read(bytes);
            }
        }
    } finally {
        closeStream(inputStream, outputStream);
    }
}
```

代码位于光盘的 src/chapter04/file/FileUtils 中。接下来拷贝几个文件做实验，进行多次测试，每次拷贝完后，待磁盘稳定了再做下一次拷贝，测试结果如表 4-1 所示。

表 4-1 测试结果（一）

次 数	拷贝文件大小（MB）	拷贝时间范围（s）
1	60.96	0.19~0.21
2	145.6	0.45~0.46
3	252.5	0.80~0.82
4	526	1.564~1.707

也许在你的机器上测试数据会不太一样，这是因为大家所使用的机器、磁盘情况以及 JVM 内存的设置都有所区别（胖哥机器使用的是 SSD 硬盘），这个没关系，我们先得到一些测试数据后，再来分析具体情况。

胖哥在测试中唯一开启了-XX:+DisableExplicitGC 参数, 虽然它与 DirectByteBuffer 有“梁子”, 但这里仅仅是探讨它本身的使用, 我们需要先抛开 GC 以及其他方面的开销, 否则效果会很奇怪, 这也是一直在强调的测试并不代表一切的原因。

接下来胖哥会用到 DirectByteBuffer 来拷贝文件, 用同样的文件大小, 同样测试多次, 看看结果会如何。

代码清单 4-5 使用 DirectByteBuffer 拷贝文件

```
public static void copyFileByByteBuffer(String srcFileName ,
                                         String dstFileName ,
                                         ByteBuffer byteBuffer ,
                                         boolean cleanDirectBuffer)
                                         throws IOException {
    File srcFile = new File(srcFileName);
    File dstFile = new File(dstFileName);
    if(!srcFile.exists())
        throw new RuntimeException("src file not exists!");
    if(dstFile.exists())
        throw new RuntimeException("dst file exists!");
    FileChannel inFileChannel = null;
    FileChannel outFileChannel = null;
    FileInputStream in = new FileInputStream(srcFile);
    FileOutputStream out = new FileOutputStream(dstFile);
    try {
        inFileChannel = in.getChannel();
        outFileChannel = out.getChannel();
        int size = inFileChannel.read(byteBuffer);
        while(size > 0) {
            byteBuffer.flip();
            outFileChannel.write(byteBuffer);
            byteBuffer.clear();
            size = inFileChannel.read(byteBuffer);
        }
    } finally {
        closeStream(in , out);
        byteBuffer.clear();
        if(byteBuffer.isDirect() && cleanDirectBuffer) {
            ((DirectBuffer)byteBuffer).cleaner().clean();
        }
    }
}
```

代码中传入的参数为 ByteBuffer 类型, 可以传入 HeapByteBuffer 或 DirectByteBuffer 等, 只要继承了抽象类 ByteBuffer 即可。

此时测试的目标是 DirectByteBuffer, 所以在测试代码中传入的都是 DirectByteBuffer 类型。

此时写段代码来调用, 在测试代码中写一段创建 DirectBuffer 的代码, 然后将这个 ByteBuffer 传递进去, 反复地使用, 最后 clean 回收掉 (但在实际场景中可以不回收, 可以继续复用, 原因自然是前面提到的 DirectByteBuffer 分配的开销是巨大的), 代码如图 4-14 所示。

```
ByteBuffer byteBuffer = ByteBuffer.allocateDirect(1024 * 1024);
long start = System.nanoTime();
FileUtils.copyFileByByteBuffer("/home/xieyuooo/BIG.mkv", "/home/xieyuooo/BIG_Stream.mkv", byteBuffer, true);
System.out.println(System.nanoTime() - start);
```

图 4-14 这里调用拷贝文件

胖哥希望你同样设置-XX:+DisableExplicitGC 参数，以保证不受“显式 GC”的影响（因为在默认情况下 System.gc()导致 FULL GC），测试结果如表 4-2 所示。

表 4-2 测试结果（二）

次 数	拷贝文件大小（MB）	拷贝时间（s）
1	60.96	0.09~0.11
2	145.6	0.20~0.23
3	252.5	0.34~0.38
4	526	0.838~0.860

从上面几组不同大小的文件测试中发现 DirectBuffer 还是有一定优势的，那是否就是绝对优势呢？

其实这些数据仅仅作为一个参考而已，不要将上面的测试数据当成权威测试结果，因为你的机器可能和胖哥的机器有着很大的差别。

在上述测试中，在不断增加文件大小的过程中会发现，通过 DirectBuffer 拷贝出来的结果逐步和普通的 Stream 的效率接近，如果使用 ByteBuffer.allocate(int) 创建一个 HeapByteBuffer，那么得到的结果会更为接近。

胖哥是这样理解的：其实抽象层次较低的拷贝未必就是最快的，因为无论是否拷贝都会在与 I/O 直接交互的位置产生瓶颈，也就是最终磁盘到 OS 的页面交换速度本身就是有瓶颈的，当达到一定程度时，数据可能都会堵在 Kernel 中没有写入到终端，这里如果有大量的数据，就有可能会导致一些局部不稳定的情况发生。因此，我们不可认为是直接内存就滥用它来做一些操作。

接下来胖哥再用另一种方式来拷贝文件，这种拷贝文件的方式就是利用内存映射，也就是这块区域是直接映射到磁盘上的。在 Java 中可以通过 FileChannel.map()在磁盘对应的位置直接分配一块空间出来，就像虚拟内存一样，它负责返回一个 MappedByteBuffer 类型的对象，其实本质上也是 DirectByteBuffer，只是它是直接映射到磁盘上的，但是并不代表没有缓冲区，只是它由 OS 来控制而已。此时若对这个 ByteBuffer 写入任何数据，就直接写入到磁盘了。很显然这中间省去了拷贝的过程，那么在理论上效率是很高的，除非瓶颈在磁盘的 I/O 层面。这段代码如下：

代码清单 4-6 通过 FileChannel.map 直接映射到磁盘来拷贝文件

```
public static void copyFileByMappedByteBuffer(String srcFileName ,
                                              String dstFileName) throws IOException {
    FileChannel inFileChannel
        = new RandomAccessFile(srcFileName , "r").getChannel();
    FileChannel outFileChannel
        = new RandomAccessFile(dstFileName , "rw").getChannel();
    try {
        long fileSize = inFileChannel.size();
        long position = 0;
        while(position < fileSize) {
            long copyFileSize
                = Math.min((fileSize - position), COPY_FILE_SIZE);
            MappedByteBuffer mappedByteBuffer
                = outFileChannel.map(MapMode.READ_WRITE,
                                   position , copyFileSize);
            inFileChannel.read(mappedByteBuffer);
            position += mappedByteBuffer.position();
        }
    }
```



```
    }finally {  
        closeStream(outFileChannel);  
    }  
}
```

在整个过程中就只有 `inFileChannel` 发生了 `read(mappedByteBuffer)` 操作，没看到发生任何 `write` 操作。这个原因就不难解释了，因为上面已经提到，这个数据的 `Buffer` 本身就是在磁盘上虚拟出来的，所以这就相当于 I/O 对 I/O 的直接传送。

拷贝文件的方式还有很多，例如：

- ◎ 输入流的 `Channel` 调用 `transferTo(long position, long count, WritableByteChannel channel);`
- ◎ 输出流的 `Channel` 调用 `transferFrom(ReadableByteChannel channel, long position, long count);`
- ◎ 在光盘代码的 `FileUtils` 里面也有对应方法：`copyFileByChannel(String, String)`。

在 JDK 1.7 以后，大家直接使用 `Files.copy()` 方法，类似这样就简单多了。JDK 1.7 的 `Files` 还提供了大量的文件操作类，例如递归遍历目录等功能都是以前版本没有的。

4.2.3 关于 Buffer

上一节提到了 `DirectByteBuffer`，那么这里就“趁热打铁”，将 Java 中的 `Buffer` 概念提出来。在讲解 `Buffer` 内容以前，先来聊聊一个经常提到的问题：

`Buffer` 是什么东西？它与 `Cache` 有何区别？

这个问题从概念的角度不容易说清楚，不过从程序的角度也许会好理解一点，`Cache` 是将一些数据放在就近的地方，最方便拿到；而 `Buffer` 通常是与 I/O 相关的，它是与 I/O 交互过程中的一个缓冲区，这些缓冲区被反复清空和写入，换句话说，这是一块不大的区域，但是可以被反复使用，利用它可以在交互过程中达到一次交互一批信息，而不是 1 个字节。关于这个问题我们来看个生活中的例子。

- ◎ 一个程序员在做项目时，每做完一个小功能点便去让经理分配下一个任务，这样一来经理就忙死了，他不断地被中断，甚至不知道这个程序员到底做了多少事情，只知道每个人貌似都做了很多事情。
- ◎ 另一个程序员做项目时，当被分配一个大方向后，半年都不与经理沟通，经理也不知道他的项目做得如何，甚至觉得他什么也没做，但是经理貌似很闲。
- ◎ 其实我们可以建立一周的工作 `Buffer` 机制，长期计划的内容会逐步进入一周的工作 `Buffer` 中，一周内完成后，再将长期计划输入，让上级掌握最新动态，经理也不用太忙。

在 Java 的世界里，其实 `BufferedInputStream`、`BufferedOutputStream`、`BufferedReader`、`BufferedWriter` 等就是 `Buffer` 使用的代表，它们使用 Java 的一个数组来完成，在将对应的流修饰后便使得输入/输出过程中自动产生 `Buffer` 功能。

在 JDK 1.4 以后，开始有了单独的 `Buffer` 功能，也就是它作为对象可以单独存在来提供各种功能。其顶层的抽象类是 `java.nio.Buffer`，它的子类便是各种 `Buffer` 的实现，如图 4-15 所示。

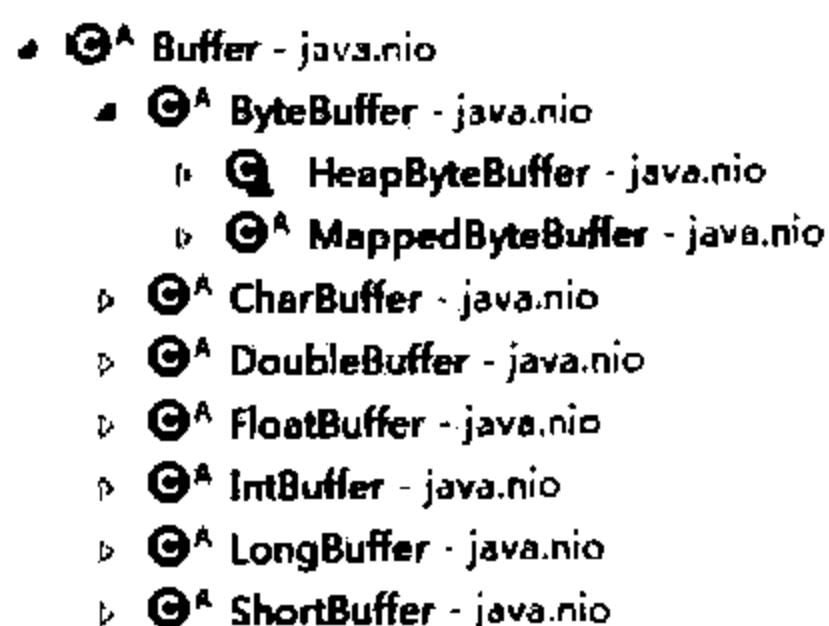


图 4-15 `Buffer` 的直接子类

图 4-15 所示的这种关系很清楚地说明, Buffer 的第一层是按照数据类型来划分的, 换句话说, 它针对每种类型都提供了 Buffer 的处理机制。其中十分常用的是 ByteBuffer, 它是抽象层次最低的, 功能非常完善, 因此看懂它的一些内容, 再去看其他的 Buffer 就很容易了。

从图 4-15 中可以发现, ByteBuffer 分为 HeapByteBuffer 和 DirectByteBuffer 两种, 看名字就应该知道, 一个是在 Java Heap 中分配空间, 一个是在 C Heap 中分配空间 (另外, 通过 Channel.map(MapMode, long, long)得到的也是 DirectByteBuffer 类型)。

接下来看看 ByteBuffer 的常见操作方法有哪些。

- ◎ ByteBuffer.allocate(int) 用于分配空间, 在 JVM 堆中分配。
- ◎ ByteBuffer.allocateDirect(int) 用于分配空间, 在 C Heap 中分配。
- ◎ FileChannel 或 SocketChannel 等的 read(byteBuffer)方法用于从通道中读取数据写入到 ByteBuffer 中 (其实这部分数据应当在 Kernel 中, 尽量将 Buffer 读满, 如果读不满, 则可以通过 ByteBuffer 的 position()获取到读取的位置), 如果这个 ByteBuffer 是映射到一个文件上的, 那么在读取内容到 Buffer 时就直接写到文件中了。
- ◎ 使用 put()、putInt()、putLong()、putChar()、putShort()、putFloat()、putDouble()方法, 向 Buffer 中手工写入各种类型的数据, 默认添加到 Buffer 的尾部, 也可以在指定的位置覆盖数据。
- ◎ get()、getInt()、getLong()、getChar()、getShort()、getFloat()、getDouble()方法与 put 方法类似, 请大家自行测试。不过, 大家在测试前需要设置 position 一次, 否则默认的 position 是在读取写入的位置。
- ◎ wrap(byte[]) 根据 byte[]数组内容创建一个 ByteBuffer 对象并返回, 也同时包含另一个重载方法用于传入 offset 和 length。
- ◎ isDirect() 用于判断是否为直接内存, 也就是判定是否为 C Heap 的 Buffer。比如在前面的代码中, 想要自己释放 DirectBuffer 就这样使用, 如图 4-16 所示。

```
if(buffer.isDirect()) {
    ((DirectBuffer)buffer).cleaner().clean();
}
```

图 4-16 判定如果为直接内存则清空, 实际情况可以不这样做

- ◎ capacity()用于返回 Buffer 的总容量大小, 是图 4-15 中所有数据类型的 Buffer, 但如果不是 Byte 类型的 Buffer, 这个大小就不是 byte 的数量, 而是具体数据类型的元素个数。例如, CharBuffer 就是指字符的数量。
- ◎ limit()返回容量的限制, 也可以通过 limit(int)方法设置, 它说明这个 Buffer 能装多少内容, 它的大小区间为[0 - capacity], position 为数据当前的位置, 因此必须小于 limit 的限制, 这两个参数配合会产生在编写代码过程中所想要得到的一些效果。例如, 当我们向 Buffer 中写入一些数据后, 但是没有将 Buffer 写满 (其实我们不知道是否将 Buffer 写满), 在某个条件成立的情况下希望将 Buffer 的现有数据转移到另一个地方去做处理, 此时将 limit 移动到 position 的位置, position 移动到 0 位置, 便知道现有数据的范围, 很容易读取 Buffer 现有的数据了, 一般我们通过 flip()就可以达到目的。
- ◎ flip()将 limit 设置为当前 position 的位置, 将 position 设置为 0, 将 mark 设置为-1,

这个动作只是修改属性，不会修改数据本身。这个动作的目的通常是为了读取 Buffer 中[0-position]当中所有的数据，或者说 Buffer 当中现有的所有数据。

- ◎ clear() 用于清空缓冲区，它不会清零，仅仅将 position 设置为 0，limit 设置为与 capacity 同样的大小，mark 设置为-1。也就是数据还在，则可以继续从这里面读取数据。有些时候大家分不清楚 clear()与 flip()之间的区别，通过上述解释应该得出的结论是，区别主要是在 limit 的设置上，而导致的结果是 flip()将 limit 放在了数据有效位置的末尾，以便于读取所有的有效数据；而 clear()放在了总容量最后的位置，以便于重新写入数据。
- ◎ remaining() 用于查看剩余空间的大小，也就是 limit-position 的值。
- ◎ hasRemaining() 用于返回是否还有剩余空间。
- ◎ mark()+reset() 巧妙的用法：mark()将当前的 position 记录到一个 mark 值中，发生 reset()时，会将 mark 的值赋值给 position，也就是记录下一个位置，做了一系列操作后，将指针恢复到这个 mark 的位置做其他的操作。

►► 4.2.4 FileChannel 的加锁

FileChannel 对象可以通过 FileInputStream、FileOutputStream、RandomAccessFile 的 getChannel()方法获取到。通过 FileInputStream、FileOutputStream 获取到的 FileChannel 分别可以进行读写操作，通过 RandomAccessFile 获取到的 FileChannel 可以自行设置读写功能。

FileChannel 拥有加锁的能力，可以对整个文件加锁，也可以对某部分字节码加锁。

通过 channel.lock()、channel.tryLock() 对文件进行加锁，在并发时第一个方法如果没有获取到锁则将阻塞，直到获取到锁；第二个方法若未获取到锁则返回 null，若获取到则返回 FileLock 对象；在需要关闭锁的时候，调用其 release 方法即可。

通过 channel.lock(long start, long size, boolean shared)、channel.tryLock(long start, long size, boolean shared)对文件某些位置进行加锁。

文件加锁并不是那么容易控制，虽然有此功能，但是应用不当可能会出现很多问题，因为这种文件加锁和具体的 OS 有些关系。例如在一些系统上，关闭任意一个通道就会释放这个文件上所有的锁。

关于 Channel 在后文中还会提到。

4.3 通信调度方式

►► 4.3.1 同步与异步

为了说明同步和异步，下面举个例子。

在家里用全自动洗衣机洗衣服，将衣服放进去，洗衣机大概需要 40 分钟洗好衣服，我们会不会在洗衣机边上看着洗衣机发傻，等待 40 分钟呢？

通常我们不会这样做，我们通常会利用这段时间可以去看看电影、看看书、洗澡、做饭等，至于什么时候把洗衣机里的衣服取出来有很多种方式。异步方式与这个例子有许多共同特征，当一个动作需要相对较长时间的等待，也就是大部分时间都消耗在这里了，那么可以利用这段时间去做点别的事情，使得资源得到更加有效的利用。

但是如果相对时间并不长，也许就没有必要利用这段时间去做其他的事情了（例如，在写代码的过程中去泡一杯茶，这个动作的时间可能也就几十秒，在泡茶的过程中没必要再回去写点代码）。如果一些小事情不断被打断，就可能什么事情都做不好，什么事情也做不完。换句话说，当我们去做另一件事情和回来做刚才未完成的事情时，在行为、思维上都需要一个“保存档案”和“载入档案”的过程。

例如，程序写到一半时，被老板拉出去开会几分钟，回来写程序的思路接不上了，得想一会才能接上思路。其实计算机也是类似的，这些开销也是很大的，我们不要一味地认为异步就是最强大的，需要综合考虑这样做是否真正值得。

扩展例子：同样的，如果在工作中需要别人配合完成任务，别人不在时（不知道什么时候回来），那么我们可以在这个时间范围内去做点别的事情，而不是等待。同样的，在自己闲的时候可以找点事情做，而不是默默地等待老板分配任务。

异步也并非完全友好，例如前文中提到的洗衣机在洗衣服的过程中如果漏水了，或者出现了其他的状况你并不会立即知道，最终有可能会综合处理任务的效率更低，所以同步、异步需要就场景而论。

➤➤ 4.3.2 阻塞与非阻塞

“阻塞”与“非阻塞”概念很容易和“同步”、“异步”混淆，胖哥会尽量用一些例子来说明它们之间的区别。

阻塞有一个很明显的特征是 **Blocking**，有了这个特征才能叫作阻塞，在 Java 程序中的线程通常处于 **Blocking** 状态，同步并不是这样的（不是指多线程同步的 **Synchronized**），同步通常是指步骤需要一步步来完成，就像常规的代码一条条地执行一样（但异步可以在没有执行完当前这行代码之前，就执行下一行代码，就像许多 JS 代码、UI 控件、后台启动线程、将任务放入线程池等动作）。

相对于阻塞来讲，同步的程序线程应当是处于 **Running** 状态的，下面就来探讨一下线程在 **Running** 状态与 **Blocking** 状态有什么区别。

线程处于 **Blocking** 状态就差不多可以看成是睡眠的，也就是说它被动什么也没法做，只有一直等待一个信号的产生才能唤醒，这个信号可能是基于时间的，也可能是基于某种事件的，总之不是自己能控制的。不过大家需要注意的是，在 **BIO** 当中程序发起 **read()** 操作时，线程阻塞是由 JVM 底层配合 OS 来完成的，此时通过 Java 获取到线程的状态依然是 **Runnable** 状态，但它确实已经被阻塞了。

处于 **Running** 状态下的线程是活跃的，在这种情况下可以去做许多其他的事情，可以不断地去检测一些事情做好没有。

在服务器上传统的 I/O 在发起读请求后会被挂起，但是在许多系统中接收 I/O 返回结果的时间比实际运行时间还要长，甚至长很多，换句话说，线程睡眠的时间比干活的时间还多得多。基于这个现实的结果，人们就思考其实有很多线程在这个过程中是不需要被挂起的，线程是服务器资源，大家不希望让它大部分时间在睡眠，希望它去干活。

线程去干活了，I/O 返回了怎么办？此时可以由一个单独的线程来监视 I/O 返回的状况（由于这样的动作十分简单，就是一次 **System Call**，获取一些状态而已，所以一个线程就足够了），如果发现某 I/O 的数据已经准备好，可以进行某种操作了，则会尝试用某种方式来处理（将这些请求当成任务交给线程池或分配一个线程来处理，当然，如果返回结果的处

理也十分简单，那么就用这一个线程来处理也是可以的)。

在 Java NIO 中使用 `SocketChannel` 来替代原有的 `Socket`，它就是非阻塞模式的 I/O，在 JDK 1.4 中就已经存在这样一种特征，它通过选择器来进行系统调用，每次调用都可以获得事件处理列表，然后进行处理，这似乎已经不错了。

不过这样的处理，中间有一个将数据从内核区 (Kernel) 拷贝到进程的过程是同步的，而且这个动作是需要程序自己来处理的 (换句话说，这个时间是占用本身的线程时间的)，于是，我们希望有一种送货上门的机制，我们叫它异步模型，在后文中会逐步介绍。

下面先来看看 Linux 的几种调度模型，或许我们可以从中找到一些共同特征让知识连贯起来。

►► 4.3.3 Linux OS 调度 IO 模型

本书是介绍 Java 的，似乎不该提到过多的 OS 的 I/O 机制，不过这里提起只是为了让我们有更多的感性认识。

我们不用知道太多的细节，但是得知道为何要说 I/O 模型呢？技术在飞速发展，为何有不同种类的 I/O 模型？在实现同一种功能的情况下，为何有如此多的实现手段？为何不能用一种东西来一统天下？

对此，胖哥有自己的见解。不论是否对错，不论遇到多少困难，胖哥始终认为这些内容都可以在生活中找到模型，找到例子，也始终相信一些东西存在必有价值。

例如在前面用“洗衣机”例子来感性认识同步与异步。也许这种方式在逻辑层面并不那么严谨，但是至少让我们知道了它大体就是这个思路。至于同步、异步与阻塞、非阻塞的区别，我们抓住了可操作性一个关键因素来展开，其他的都是表现特性而已，也就是一旦抓住了一种模型、方法、机制，抓住中间的关键主干，其余的顺藤摸瓜就能搞清楚。这是最基本的区别，胖哥认为这不算技术问题，而是方法问题。

本节我们要介绍的是：`select`、`poll`、`epoll` (Linux 本身的 I/O 模型不止这几种)。

`select` 模型是由一个数组管理的，也就是每当注册一个事件时，就会占用一个数组的位置。由于是使用数组来存储事件的注册，所以就有宽度限制，在 32 位机器上限制为 1024，在 64 位机器上限制为 2048。

每次系统请求时都会线性遍历整个数组看是否有可处理的事件，若没有则睡眠，直到超时或被事件触发唤醒后重新遍历，性能自然不怎么样。

`poll` 和 `select` 很类似，最大的区别在于它不是用数组实现的，而是使用链表实现，因此它就没有 `select` 最大数量的限制了。

`epoll` 基于事件回调机制，即回调时直接通知进程，而无须使用某种方式来查看状态。它通过 `mmap` 映射内存来实现，不用做内存拷贝。

胖哥为何说这些呢？

这是 OS 级别的一些机制，其实我们的程序中或多或少会直接或间接地利用 OS 的这些机制来提升系统的整体性能，JVM 平台可能没有完全沿用这些可复用的 I/O 模型，但是可参考这些常见的 I/O 模型，因为基础思想都是一致的。

►► 4.3.4 Java 中的 BIO、NIO

BIO 模型，也就是 `Blocking I/O` (阻塞 I/O)，这是最古老的 Java 通信机制，本章前面的代码都是用 BIO 来完成的 (即使前面部分代码使用了 `Channel`，也是采用类似于阻塞 I/O

的方式来写代码的)。

在这种模型中,应用程序首先会通过 System Call 发送请求给内核 (Kernel),然后由内核去进行网络通信。应用程序如果发起的系统调用是读操作,在内核准备好数据以前,这个线程将会被挂起,一直等待下去,直到有返回的数据在内核中准备好位置,或者在设置 SoTimeout 后超时被唤醒。

假如抛开超时机制在网络等待机制中的作用,发起 read()操作等待远程返回数据的过程由以下两个阶段组成。

阶段 1: 等待 I/O 返回数据,这取决于 I/O 请求的目标返回数据的速度。例如在网络 I/O 请求中,在远程返回数据前也需要经过一些处理才返回数据,远程输出数据后将取决于网络本身的速度及数据本身的大小。

阶段 2: 返回的数据首先被填充到内核 (Kernel) 的缓冲区里,然后再从内核区将数据向进程内部拷贝 (copy data from kernel to user)。这个过程完成后,应用程序才会继续向下执行。

根据上面的结论,在 BIO 过程中,程序与内核交互的过程如图 4-17 所示。

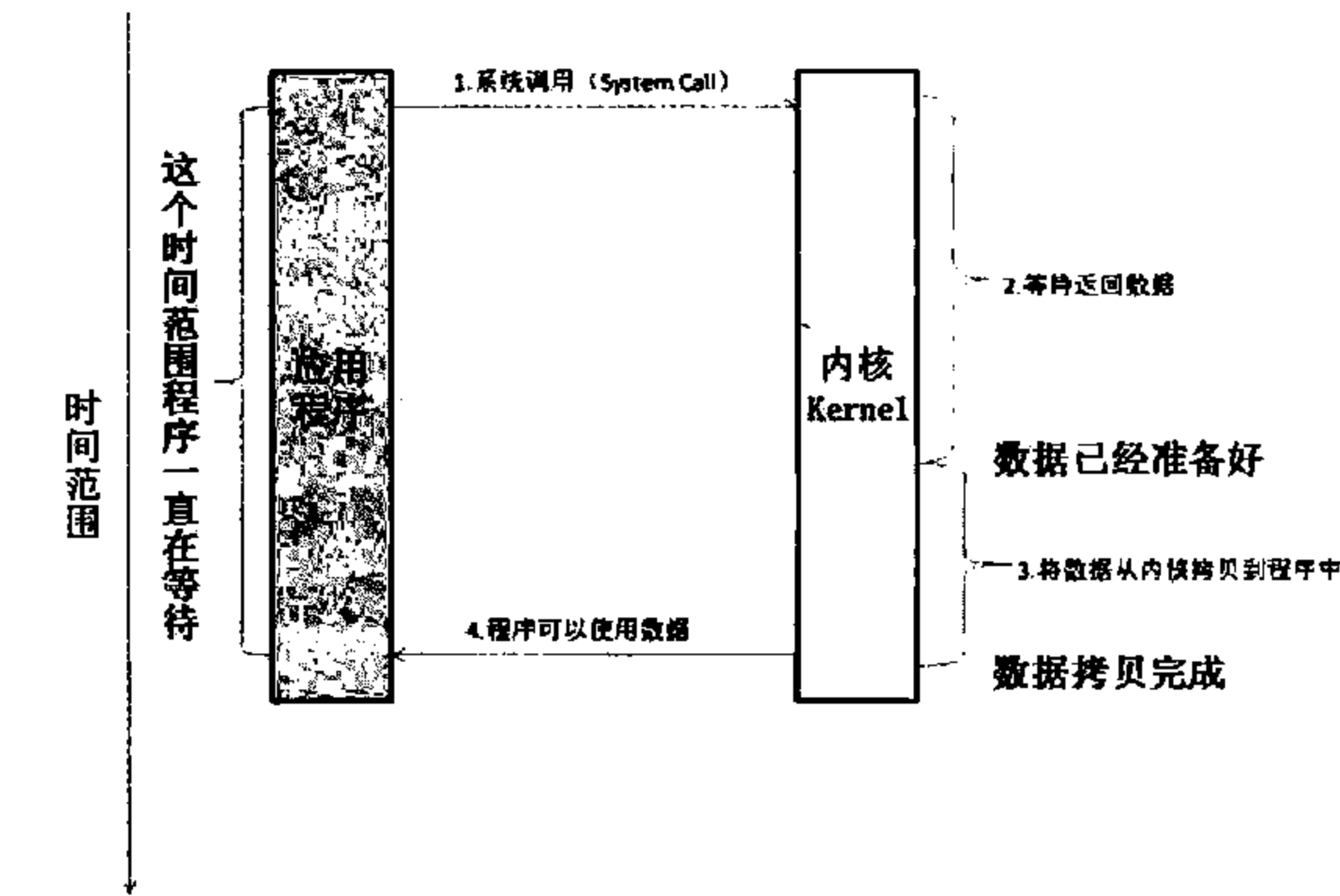


图 4-17 BIO 中程序等待的状态,注意步骤编号和时间轴

非阻塞 I/O (Nonblocking I/O) 是什么样的呢? 它的交互示意图如图 4-18 所示。

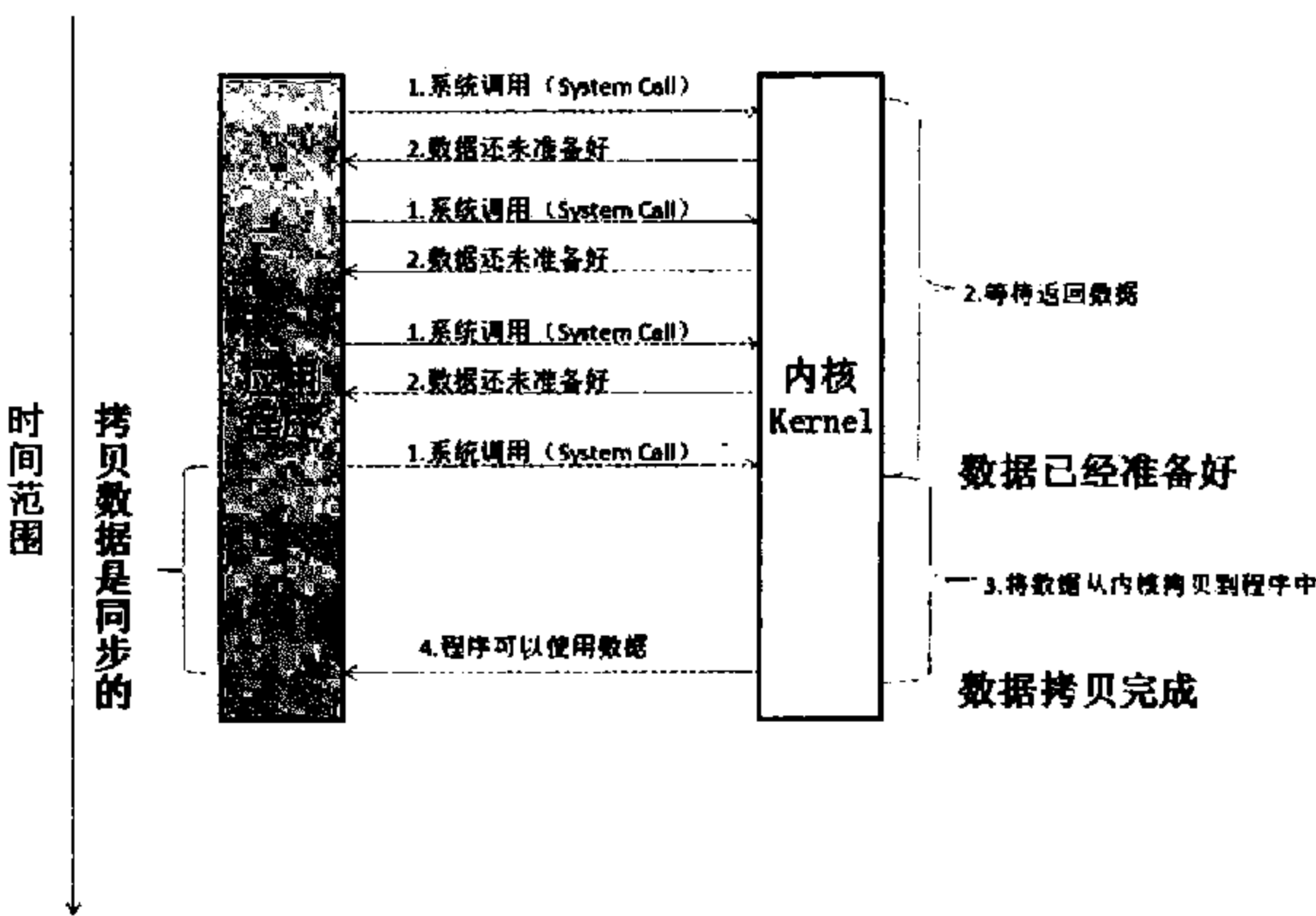


图 4-18 非阻塞 I/O 的交互示意图

非阻塞 I/O 和阻塞 I/O 有个很明显的区别是：发起第一次 System Call 请求后，线程并没有被阻塞。但它并没有去做别的事情，而是在不断发起 System Call 请求。

这样操作似乎在空耗 CPU，给人的感觉是还不如一直挂起等待返回，那样至少会让出 CPU 资源。接下来我们来探讨一下非阻塞 I/O 可能的使用方式。

非阻塞 I/O 最大的特征是发起系统请求后，就无须等待了，可以去做其他的事情，每次 System Call 其实只是看看数据准备好没有，通常它的时间是很短暂的，这样简单的动作完全可以只用一个线程来完成对很多事件的监听。换句话说，其他的线程可以去干别的事情，只需要某个线程定期来做下检测即可，在设计上注意这个检测频率，就可以达到效率高且节约资源的目的。

在 Java 的 NIO 中使用到这种方式，在套接字上提供 Selector 选择机制，当它发起 select() 时会阻塞等待至少一个准备好的事件返回，然后通过 selectedKeys 获取到事件 Set 列表，其中每一项为一个 SelectionKey 对象，这个 SelectionKey 与每一个具体的通道相关，接下来的许多操作也将与这个 SelectionKey 密切相关，在后文中会介绍。

Selector 常规操作：

◎ selectNow(), 不会阻塞，如果没有任何准备好的事件，则返回 0。使用该方法不断发起系统调用是最接近图 4-18 所示模型的，因为它与 select()相比最大的特征就是，它本身就是非阻塞的。但并不代表这是最好的使用方式，这一点大家要注意。

◎ select(long), 可以设置阻塞的超时时间，同样的，如果没有任何准备好的事件，则返回 0。

◎ weakUp(), 当发生 select()阻塞后，可以调用该方法唤醒 Selector，不过这肯定是由另一个线程来调用的。

还有一种 I/O 模型叫 I/O Multiplexing，这种模型更接近于 Java NIO 中 Selector 使用 select()方法所实现的机制，也就是最少等待一个准备好的事件才返回一种模型，它的模型如图 4-19 所示。

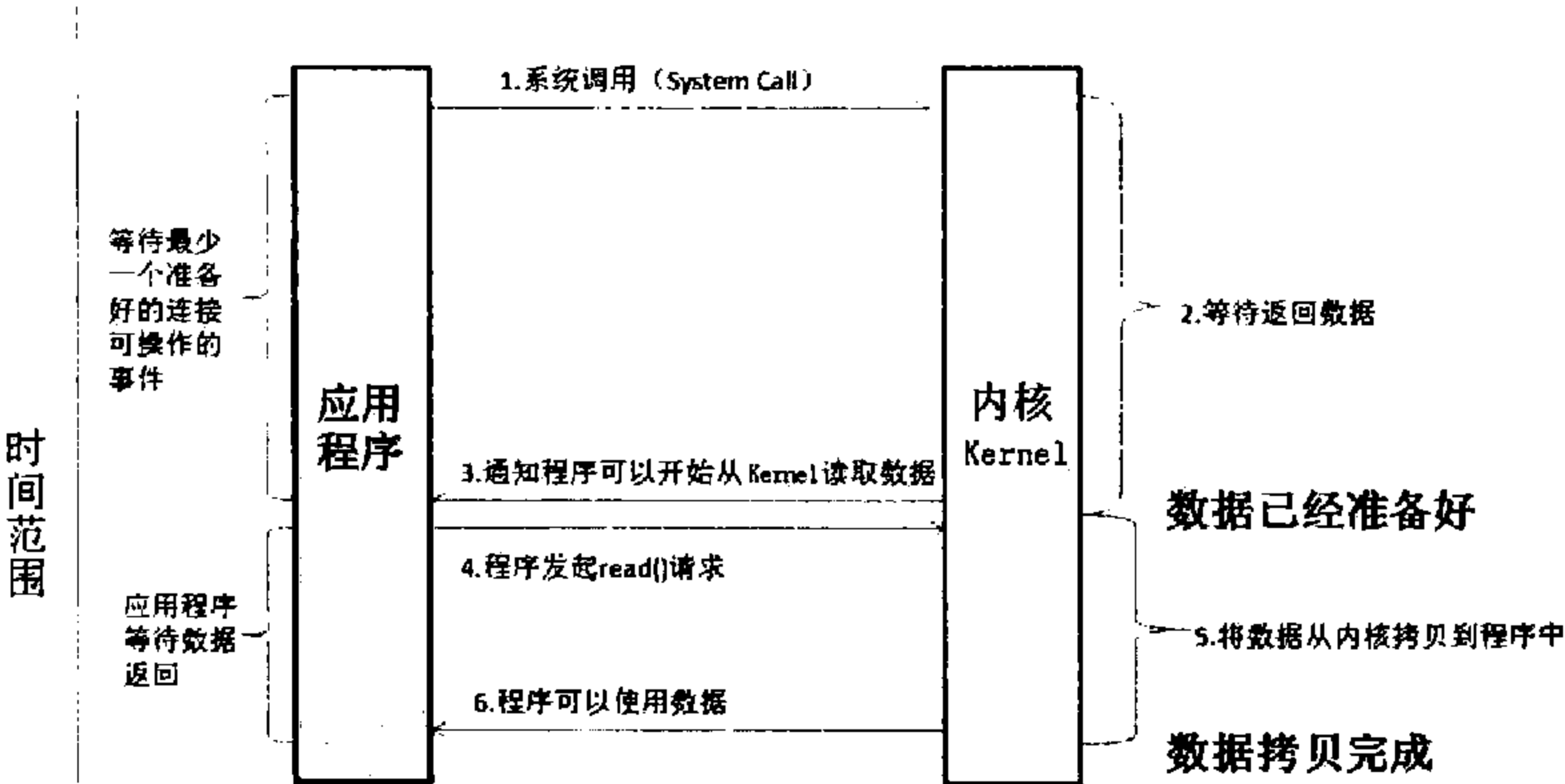


图 4-19 I/O Multiplexing 模型

如果一个系统要承载很多相对长连接的并发请求（例如很多个终端要下载文件或看视频），但由于在 BIO 处理过程中会发生线程阻塞，所以在每一个 I/O 上都需要一个线程在等待数据返回。从线程的角度来讲，需要处理完一个请求再处理下一个请求，而每个请求的时间可能都会比较长，这个时候就会有許多新的请求可能得不到响应。

如果一个系统与许多终端随时保持连接，这些终端可能随时会发送一些请求到服务器端来让服务器端处理，服务器端也可能向客户端推送数据，那么双方可能都需要相应的套接字来监听是否有发来的数据，自然的监听者不知道什么时候会有数据会发送过来，在 BIO 中等待数据时线程被挂起，也就是线程可能睡眠了。

此时聪明的小伙伴开始思考：是否有方法可以解决这种问题？

一种想法就是给予足够多的线程，因为当线程在等待 I/O 时是会释放 CPU 资源的，所以可以在这个时间范围内设置足够多的线程来达到资源的充分利用。这种方式确实可以达到目的，尤其是现代的计算机内存资源都是十分充足的，对于大部分都是短连接请求，使用这种方式是一种简单、方便的选择。

长连接则不一样，终端可能非常多（终端未必来自于用户，也可能来自于其他服务器），它们之间会进行频繁的交互，服务器可能会给外部开许多的连接，这些连接由于是打开的，在 BIO 中需要相应的线程来读取相应的请求（因为请求随时可以发过来，所以就需要在 Socket 上发起等待事件，但是短连接通常新的请求过来在很短的时间范围内是同一个 Socket，超过一定时间，则会创建新的 Socket 来处理）。

JDK 从 1.4 版本开始支持 NIO (New IO)，套接字当中的 `SocketChannel` 及 `ServerSocketChannel` 是可以实现非阻塞 I/O 的，`FileChannel` 不能实现非阻塞 I/O。所以，接下来主要探讨的是套接字的非阻塞 I/O。

下面用一段简单的代码来模拟 NIO 的服务器端，用 50 个线程来模拟 50 个客户端同时请求服务器端下载文件。每个客户端收到服务器端返回的一部分数据时，就进行简单打印，表示这个客户端得到了数据并一直还在与服务器端通信。服务器端使用一个线程来处理所有的请求，结果发现所有的客户端都有响应，并不是处理完一个客户端请求后再处理下一个请求。

传统的 BIO 通过某种方式保存多个 Socket，将文件切分为小板块也可以达到同样的效果。不过，这个例子有一些细节就是，客户端如果还没有将数据接收和处理，服务器端不会再一次得到 `write` 的状态事件，就不会向某个客户端发送数据。换句话说，客户端发起真正“读”请求时，服务器端才会返给它一部分数据。

该例子写得比较简单，在这个例子中没有考虑许多网络的异常情况，但是大家在真正实现时，这是需要考虑的。

本书中所要阐述的重点是服务器端程序，所以在接下来的例子中对于客户端调用没有使用 NIO 模型，而是使用传统的 Socket 加上模拟 50 个线程同时并行访问服务器端即可达到我们的目的。

要下载文件，自然少不了读取文件的处理部分，读取文件本来用什么都是可以的，不过最后在 NIO 中需要发送的数据类型是 `ByteBuffer`，如果使用 `FileInputStream`，那么读取出来的 `byte[]` 需要 `put` 到一个 `ByteBuffer` 中，或者直接通过 `wrap` 方法转换为一个 `ByteBuffer`；若使用 `FileChannel`，因为它本身就是使用 `ByteBuffer` 来处理的，所以在下面的例子中我们

使用 `FileChannel` 来处理。

下面定义一个读取文件的处理器, 提供几个简单的 API, 让读取外部调用文件变得简单, 容易操作 (Java NIO 的例子在 `src/chapter04/nio/server/` 目录下)。

代码清单 4-7 文件读取部分的代码

```
public class DownloadFileProcessor implements Closeable {

    private final static String FILE_PTH = "/javaA/upload/Hadoop.PDF";

    private FileChannel fileChannel;

    protected ByteBuffer fileByteBuffer = ByteBuffer.allocate(8192);

    public DownloadFileProcessor() throws FileNotFoundException {
        fileChannel = new FileInputStream(FILE_PTH).getChannel();
    }

    public int read() throws IOException {
        fileByteBuffer.clear();
        int count = fileChannel.read(fileByteBuffer);
        fileByteBuffer.flip();
        return count;
    }

    @Override
    public void close() throws IOException {
        fileChannel.close();
    }

    public ByteBuffer getFileByteBuffer() {
        return fileByteBuffer;
    }
}
```

大家可以根据自己的实际情况修改下载文件的路径 (用于下载的文件不要太小, 不然许多效果看不出来)。

前面提到, Java NIO 通常会创建一个 `java.nio.channels.Selector` 选择器用来获取 `SelectionKey`, 然后根据该 `SelectKey` 的事件类型进行处理, 通信方式不再是传统的 `Socket`, 而是 `Channel` 方式。为此, 我们来看看这个 `Selector` 的创建是怎么回事, 让大家有一个感性认识。

代码清单 4-8 `Selector` 的创建与调用

```
public class NIOServer {

    public final static int BYTE_BUFFER_SIZE = 8192;

    public final static ByteBuffer CLIENT_BYTE_BUFFER
        = ByteBuffer.allocate(BYTE_BUFFER_SIZE);

    public final static int DEFAULT_PORT = 8888;

    public static void main(String []args) throws IOException {
        Selector selector = createSelector();
        logInfo("服务器端已经打开端口: " + DEFAULT_PORT);
        try {
            while (true) {
                selector.select();
                Set<SelectionKey> selectionKeySet = selector.selectedKeys();
                for (SelectionKey selectionKey : selectionKeySet) {
                    SelectionKeyProcessor selectionKeyProcessor

```

```

        = new SelectionKeyProcessor(selectionKey, selector);
        selectionKeyProcessor.processKey();
    }
    selectionKeySet.clear();
}
}finally {
    selector.close();
}
}

private static Selector createSelector() throws IOException {
    ServerSocketChannel server = ServerSocketChannel.open();
    Selector selector = Selector.open();
    server.socket().bind(new InetSocketAddress(DEFAULT_PORT));
    server.configureBlocking(false);
    server.register(selector, SelectionKey.OP_ACCEPT);
    return selector;
}
}

```

通过上面这段代码就十分轻松地得到了绑定在端口 8888 上的 Selector 对象, 并且它也像 ServerSocket 那样开始监听响应的端口, 只是这里监听的不仅仅是新的连接请求, 而是将其放大为一种事件, 连接请求仅仅是作为一种事件而存在, 如接收新请求、可读、可写事件。

一直在说 Channel, 到底 Java NIO 中有几种 Channel 呢?

- ◎ FileChannel, 文件读写的通道, 但是它并不支持 Selector 的非阻塞模式。
- ◎ SocketChannel, 网络通信中 TCP 的通道。
- ◎ ServerSocketChannel, 监听网络中新建的 TCP 连接, 通过它调用 accept() 可创建 SocketChannel 与客户端正式连接, 就像 ServerSocket 创建 Socket 一样。
- ◎ DatagramChannel, 从 UDP 中读写网络包数据。

所有事件都是基于 SelectionKey 来处理的, 所以首先需要知道它的事件类型, 判定其类型主要有 isConnectable()、isAcceptable()、isReadable()、isWritable()、isValid() 方法。其中 isValid() 方法主要用于检测有效性, 这里就不再多说了, 下面看其他几个方法。

- ◎ isConnectable(), 通常在客户端使用, 需要先注册事件: SelectionKey.OP_CONNECT 后才会触发该事件成立。服务器端通常不需要调用这个方法。客户端 NIO 的写法通常有以下几条代码。

```

Selector selector = Selector.open();
SocketChannel client = SocketChannel.open();
client.configureBlocking(false);
client.register(selector, SelectionKey.OP_CONNECT);
client.connect(ADDRESS);

.....
for(;;) {
    selector.select();
    for (SelectionKey key : selector.selectedKeys()) {
        if (key.isConnectable()) {
            //需要进一步调用通道的 isConnectionPending() 方法判定才能表示连接已经正常
            //可在确认连接正常后开始注册读、写事件
        } else if (key.isReadable()) {
            //读操作, 可以再一次注册读事件, 或结束
        } else if (key.isWritable()) {
            //写操作, 可以再一次注册写事件
        }
    }
}

```


isConnectable 这个词有点忽悠人，有点感觉是“是否可以连接”的意思，来看看代码注释中的说明。

```
Tests whether this key's channel has either finished, or failed to
finish, its socket-connection operation.
```

其意思基本可以理解为：测试通道的 Socket 连接操作，是否已经正常地完成。这里并没有说是否连接上，只是说通道准备好了，可以从 SelectionKey 中获取通道了。

获取到通道后，判定通道是否已经初始化好，这是通过通道的方法 channel.isConnectedPending()来完成的，然后再开始连接操作，使用 finishConnect()方法。

这几个方法似乎不是那么好理解，或许可以将它们理解为在家里上网前需要先准备好路由器、电源设备，就像准备好通道那样。而路由器这些设备启动需要一点时间，这个过程可以理解为初始化，初始化完成后才能开始拨号上网。

◎ isAcceptable()，在服务器端使用，是否准备好创建一个 Channel 用于接受一个新的连接，该判定条件基于代码清单 4-8 中的 ServerSocketChannel 已经注册事件：SelectionKey.OP_ACCEPT，当有一个新的连接请求到来时成立。

这个事件成立后，通过 SelectionKey.channel()获取到的就是 ServerSocketChannel 对象，此时要通过该对象的 accept()方法来创建一个与客户端的连接对象 (SocketChannel)，并将这个 SocketChannel 的读或写事件注册到 Selector 上(在我们的例子中是先发起一个读操作，然后再进行写操作，因此这里是发起读事件的注册)，处理动作如代码清单 4-9 所示。

代码清单 4-9 条件 isAcceptable()成立后可建立通道

```
private void processAccept() throws IOException {
    //这里获取到 Server 打开的通道
    ServerSocketChannel server
        = (ServerSocketChannel) selectionKey.channel();
    SocketChannel channel = server.accept(); //接受一个请求的 Channel 通道
    channel.configureBlocking(false); //通道配置为非阻塞模式
    channel.register(selector, SelectionKey.OP_READ); //通道注册读取事件
}
```

◎ isReadable()，表示是否可以发生读操作。该方法成立的条件是先注册事件：SelectionKey.OP_READ (可以在代码清单 4-9 中看到已经注册了该事件)，并且有发送数据方发送的数据到达；此时通过 SocketChannel 的 read()方法就可以从内核缓冲区中将数据拷贝到一个指定的 ByteBuffer 中。

◎ isWritable()，表示是否可以发生写操作。该方法成立的条件是先注册事件：SelectionKey.OP_WRITE 后，如果连接时正常就会被触发，不过如果接收数据方一直没有 read()数据，或使用 NIO 后没有注册事件 SelectionKey.OP_READ，那么发送方的 isWritable()方法就不会再成立。

为了模拟这种现象，可以让客户端不接收任何数据，然后使用 System.in.read()让客户端进程一直保持，服务器端如果每次发送 8KB 大小的数据，大概发送 3 次后 isWritable()方法就不再成立 (如果发送 4KB 大概是 6 次，2KB 大概是 12 次，偏差一般不超过 1 次)，发送数据的总大小大概等价于服务器端的 SocketBuffer + 2 × 客户端 SocketBuffer 大小。这个结论大家可以通过已有的代码测试得到 (设置 SocketChannel 的 Buffer 使用 channel.socket().setSendBufferSize()来完成)。

回过头来看看程序如何编写。

对于每个 `SelectionKey`，单独设置了一个 `SelectionKeyProcessor` 对象，主要是包装对 `SelectionKey` 的各种事件的处理分发，对外简单调用 `processKey()` 方法进行处理。在这个操作中首先要判断事件类型，以使用不同的方法来处理，如图 4-20 所示。

代码中 `catch(IOException e)` 不能少，一定要注意：这也是容易掉进去的“坑”，例如在双方建立连接的情况下客户端由于某种原因与服务器端断开，服务器端还在继续写操作，就会抛出 `IOException` 异常，在这里应当通过 `closeStreams(Closeable...closeable)` 方法来关闭相应的通道。反过来，如果不捕获 `IOException` 异常，就不会关闭通道，事件也将会继续被触发，而且可能会无限制地被触发，无限制地报错，服务器此时的开销一下子就会“飞”上去，日志文件也会很快被写满。另外，在抛出异常时，`DownloadFileProcessor` 也和 `Channel` 一起执行了 `close()`，这个 `DownloadFileProcessor` 的 `close()` 方法用来关闭内部需要下载文件的输入流。

```
public void processKey() throws IOException {
    try {
        if (selectionKey.isAcceptable()) { //接收
            processAccept();
        } else if (selectionKey.isReadable()) { //读取
            processRead();
        } else if (selectionKey.isWritable()) { //写
            processWrite();
        }
    } catch (IOException e) { //IO异常需要关闭，否则会导致无限制地报错，因为通道没结束，就会一直被检测
        SelectableChannel channel = selectionKey.channel();
        if (channel instanceof SocketChannel) {
            DownloadFileProcessor downloadFileProcessor = (DownloadFileProcessor) selectionKey.attachment();
            closeStreams(downloadFileProcessor, channel);
            logInfo("下载由于IO异常，结束.....");
        }
    }
}
```

图 4-20 事件判断逻辑处理

继续向下看，每种不同的事件到底要进行什么操作，前文有许多文字描述，也给出了 `processAccept()` 方法的内容，不过很枯燥，现在来看看比较全的代码，如图 4-21 所示（具体请参看本书光盘中的 `src/chapter04/nio/server/SelectionKeyProcessor`）。

```
private void processAccept() throws IOException {
    ServerSocketChannel server = (ServerSocketChannel) selectionKey.channel(); //这里获取到Server打开的通道
    SocketChannel channel = server.accept(); //接受一个请求的Channel通道
    channel.configureBlocking(false); //将接受请求的通道配置为非阻塞模式
    channel.register(selector, SelectionKey.OP_READ); //将这个通道注册一个读取事件
}

private void processRead() throws IOException {
    SocketChannel channel = (SocketChannel) selectionKey.channel(); //此时取出的通道就是在processAccept()方法中所注册的通道
    channel.read(NIOserver.CLIENT_BYTE_BUFFER);
    //以下转换字符集过程也可以自己使用转换器: CharsetDecoder未完成
    byte[] bytes = new byte[NIOserver.CLIENT_BYTE_BUFFER.position()];
    NIOserver.CLIENT_BYTE_BUFFER.flip(); //将指针重新偏移，才能得到准确的数据
    NIOserver.CLIENT_BYTE_BUFFER.get(bytes); //读取到Byte中
    NIOserver.CLIENT_BYTE_BUFFER.clear();
    logInfo("客户端消息>>" + new String(bytes, DEFAULT_MESSAGE_CHARSET)); //转换输出字符集

    SelectionKey writeSelectionKey = channel.register(selector, SelectionKey.OP_WRITE); //注册一个写的选择器
    writeSelectionKey.attach(new DownloadFileProcessor()); //这里可以绑定一个对象到selectionKey当中，触发相应事件的时候，可以将其取出
}

private void processWrite() throws IOException {
    SocketChannel channel = (SocketChannel) selectionKey.channel();
    DownloadFileProcessor downloadFileProcessor = (DownloadFileProcessor) selectionKey.attachment(); //获取注册WRITE事件的对象
    int count = downloadFileProcessor.read();
    if (count <= 0) {
        closeStreams(downloadFileProcessor, channel);
        logInfo("下载结束.....");
    } else {
        channel.write(downloadFileProcessor.getFileByteBuffer());
    }
}
```

图 4-21 接受连接、读数据处理、写数据处理

在上述代码中, DownloadFileProcessor 中的 close()方法实现了 Closeable 接口。为何?

Closeable 接口说明其内部有一个 close()方法, 不论是流还是自定义对象, 它们都有一个通用的 close()方法(换句话说, close 做什么可以由自己的代码来决定), 这样便于程序通用化, 在此基础上封装关闭方法并使用动态参数 closeStreams(Closeable... closeable)就可以让很多流的关闭动作变得很简单。

在 JDK 1.7 以后, 在 try-with-resource 语法中, 就要求关闭的对象都实现 Closeable 接口, 这样 JDK 可以帮助我们来完成关闭动作(不过这只是 JDK 1.7 的语法糖而已, 内在也是通过 finally 实现的)。

接下来看看客户端代码情况。前面提到, 客户端没有使用 NIO, 换句话说, 服务器端用 NIO, 并不代表客户端也要用 NIO 来完成。

因为客户端未必就是客户的终端, 它有可能是某台服务器, 多台服务器之间需要大量的交互, 即使是客户端也需要考虑一些资源的利用率问题, 因为每台服务器也可能连接许多其他的服务器, 其本身也可能为其他服务器提供服务。

在前文中提到客户端使用 NIO 方式的简易代码, 不过本例子中客户端只是使用了传统的 Socket, 因此 NIO 客户端在这里不再重述, 现在来看看实际用于测试的代码, 如图 4-22 所示。

```
for(int i = 0 ; i < 50 ; i ++){
    final String num = String.valueOf(i);
    new Thread(String.valueOf(i)) {
        public void run() {
            SocketWrapper socketWrapper = null;
            FileOutputStream stream = null;
            try {
                stream = new FileOutputStream("c:/temp/aaa" + num + ".pdf");
                socketWrapper = new SocketWrapper("localhost" , 8888);

                println("已经连接上服务器端.....");
                socketWrapper.write("我是线程: " + num , DEFAULT_MESSAGE_CHARSET);
                byte []buffer = new byte[8192];
                int len = socketWrapper.read(buffer);
                while(len > 0) {
                    println("我是线程: " + num + " 我接收到数据, 长度为: " + len);
                    stream.write(buffer , 0 , len);
                    len = socketWrapper.read(buffer);
                }
                //System.in.read();
            }catch(Exception e) {
                e.printStackTrace();
            }finally {
                FileUtils.closeStream(stream , socketWrapper);
            }
        }
    }.start();
}
```

图 4-22 客户端主要代码截图

接下来我们来看看结果。

首先看看服务器端的一些输出情况, 如图 4-23 所示。

如果客户端异常关闭, 服务器端则会输出如图 4-24 所示的结果(大家可以通过按“Ctrl+C”键的方式强制终止客户端程序的运行来查看服务器端的效果)。

当客户端正确接收数据完成后, 服务器端的输出情况如图 4-25 所示。


```
我是线程: 26 我接受到数据, 长度为: 8192
我是线程: 26 我接受到数据, 长度为: 8192
我是线程: 32 我接受到数据, 长度为: 8192
我是线程: 32 我接受到数据, 长度为: 8192
我是线程: 38 我接受到数据, 长度为: 8192
我是线程: 38 我接受到数据, 长度为: 8192
我是线程: 4 我接受到数据, 长度为: 8192
我是线程: 4 我接受到数据, 长度为: 8192
我是线程: 42 我接受到数据, 长度为: 8192
我是线程: 42 我接受到数据, 长度为: 8192
我是线程: 4 我接受到数据, 长度为: 8192
我是线程: 4 我接受到数据, 长度为: 8192
我是线程: 38 我接受到数据, 长度为: 8192
我是线程: 38 我接受到数据, 长度为: 8192
我是线程: 32 我接受到数据, 长度为: 8192
我是线程: 32 我接受到数据, 长度为: 8192
我是线程: 26 我接受到数据, 长度为: 8192
我是线程: 26 我接受到数据, 长度为: 8192
我是线程: 18 我接受到数据, 长度为: 8192
我是线程: 39 我接受到数据, 长度为: 8192
我是线程: 18 我接受到数据, 长度为: 8192
我是线程: 33 我接受到数据, 长度为: 8192
我是线程: 33 我接受到数据, 长度为: 8192
我是线程: 18 我接受到数据, 长度为: 8192
我是线程: 39 我接受到数据, 长度为: 8192
我是线程: 39 我接受到数据, 长度为: 8192
我是线程: 26 我接受到数据, 长度为: 8192
我是线程: 32 我接受到数据, 长度为: 8192
我是线程: 26 我接受到数据, 长度为: 8192
我是线程: 26 我接受到数据, 长度为: 8192
我是线程: 26 我接受到数据, 长度为: 8192
我是线程: 38 我接受到数据, 长度为: 8192
我是线程: 4 我接受到数据, 长度为: 8192
我是线程: 4 我接受到数据, 长度为: 8192
我是线程: 42 我接受到数据, 长度为: 8192
我是线程: 42 我接受到数据, 长度为: 8192
我是线程: 4 我接受到数据, 长度为: 8192
我是线程: 4 我接受到数据, 长度为: 8192
我是线程: 38 我接受到数据, 长度为: 8192
```

图 4-26 客户端输出情况

客户端下载文件后，会在本地保存，如图 4-27 所示。

 aaa0.pdf	2013/11/24 16:51	Adobe Acrobat ...	60,960 KB
 aaa1.pdf	2013/11/24 16:51	Adobe Acrobat ...	60,960 KB
 aaa2.pdf	2013/11/24 16:51	Adobe Acrobat ...	60,960 KB
 aaa3.pdf	2013/11/24 16:51	Adobe Acrobat ...	60,960 KB
 aaa4.pdf	2013/11/24 16:51	Adobe Acrobat ...	60,960 KB
 aaa5.pdf	2013/11/24 16:51	Adobe Acrobat ...	60,960 KB
 aaa6.pdf	2013/11/24 16:51	Adobe Acrobat ...	60,960 KB
 aaa7.pdf	2013/11/24 16:51	Adobe Acrobat ...	60,960 KB
 aaa8.pdf	2013/11/24 16:51	Adobe Acrobat ...	60,960 KB
 aaa9.pdf	2013/11/24 16:51	Adobe Acrobat ...	60,960 KB
 aaa10.pdf	2013/11/24 16:51	Adobe Acrobat ...	60,960 KB
 aaa11.pdf	2013/11/24 16:51	Adobe Acrobat ...	60,960 KB
 aaa12.pdf	2013/11/24 16:51	Adobe Acrobat ...	60,960 KB
 aaa13.pdf	2013/11/24 16:51	Adobe Acrobat ...	60,960 KB
 aaa14.pdf	2013/11/24 16:51	Adobe Acrobat ...	60,960 KB
 aaa15.pdf	2013/11/24 16:51	Adobe Acrobat ...	60,960 KB

图 4-27 下载后的文件列表

疑惑：使用 NIO 有何好处？我们在何时会选择 NIO 呢？

答曰：如果系统在运行中有大量时间在做 I/O，而且这些 I/O 通常需要一个相对较长时间的连接交互，那么若程序设计得不够好，线程通常会被无端地浪费。NIO 是非阻塞的，它基于事件注册在 Kernel 中，只有真正事件触发时，才会真正让线程去处理 I/O 的返回值，理论上，线程的利用率提高了。由此应当清楚，它的场景通常是 I/O 密集型的，也就是一个请求中有大量反复的 I/O 操作，尤其是基于长连接的频繁交互。

另外，它的应用场景应当是业务相对简单的服务，或者说是一些无状态的服务模型，若业务模型过于复杂，则会使得代码在事件处理过程中保持上下文信息，若上下文信息十分复杂，则会导致代码十分繁杂。

疑惑：非阻塞 I/O 在资源利用率上还有什么缺陷？

答曰：虽然非阻塞 I/O 有很多功能，但是仍然缺陷多多，这里来列举几个。

缺陷 1：它需要一个线程不断去检测状态，每次检测状态信息时都会从 Kernel 中将准备好的事件拷贝到进程中，每次注册事件时又需要将其从进程拷贝到 Kernel 中，这中间的多次内存拷贝的开销很大。

缺陷 2：在非阻塞 I/O 中，在 Kernel 准备好数据后，就交给线程来处理，线程此时来处理的过程需要通过 read 操作将数据从 Kernel 拷贝到程序的 ByteBuffer 中，也许内存拷贝时间很短，但是懒惰的我们总是希望这部分工作交给非进程的资源来完成。

缺陷 3：Selector 不支持 FileChannel 的非阻塞化。JDK 从 1.7 版本开始支持 AIO 模型，或许在那里换了一些 I/O 方案上的一些思路，下面请看 4.3.5 节的内容。

关于 Channel 的补充：

(1) java.nio.channels.Pipe 可以从该类的实例中分别通过 sink()、source() 方法获取 Pipe.SinkChannel 和 Pipe.SourceChannel，分别对应生产者和消费者，通道交互时会用到，有点类似于 PipeInputStream 和 PipeOutputStream。

(2) Channel 如果要 read 和 write 多个 buffer，则可传入 buffer 数组。它的用途是：当知道要读写的数据中有几个部分，每个部分有多少个字节时，可以提前预定宽度，然后进行填充。

4.3.5 Java AIO

JDK 1.7 引入了 NIO 2.0，也就是 AIO（本书大部分是讲 JDK 1.6 的知识，本节只是简单引导），又叫作异步 IO（Asynchronous I/O）。

前文中提到的 NIO 1.0 最少需要一个线程去获取事件信息，而且有许多拷贝，我们更加希望有一种“送货上门”的模型。为了更加形象地认识这个问题，我们来联想一下生活中的事情。

抛开一些计算机的概念，用物流送货的动作来举例。

- ◎ BIO 模型，你需要到物流的中转站去等货且不能离开中转站，如果货没到，其他的事情就别想做。
- ◎ NIO 模型，每天去检测一下货物是否到了，这个动作十分简单，可以让每个小区派 1 个人去看看有没有这个小区的货物，如果有就带回来，或者这个人通知你去物流中转站拿货。
- ◎ AIO 模型，货到的时候送货上门，换句话说，去拿货的路途虽然不长，但是由别人帮你承担了。

由此，胖哥对 AIO 的理解如图 4-28 所示。

可以发现在异步 I/O 模型中，I/O 的事情已经交给内核来处理，线程都在做业务相关的事情，几乎都不需要关注 I/O 了，那么我们的代码该如何写呢？要用异步 IO 来写代码自然会比普通的代码复杂很多，复杂在哪里？请看下面的解释。

首先程序无须等待 I/O 返回值，那么产生的问题就是：如何根据返回值做进一步的处理呢？（如果 JDBC 是异步的，客户端请求一个分页数据，程序通过 JDBC 查询数据库后，数据库尚未返回数据，但是程序会继续向下运行，那么此时就不得不将客户端的许多信息放在一个上下文中，其中包含了客户端的连接信息，以及程序运行中的许多中间结果，以

便于 JDBC 返回数据时回调方法，可以继续运行和计算。试想一下：如果多次进行数据库操作，那么程序将会非常难以读懂，所以东西是好的，要看什么时候用。)

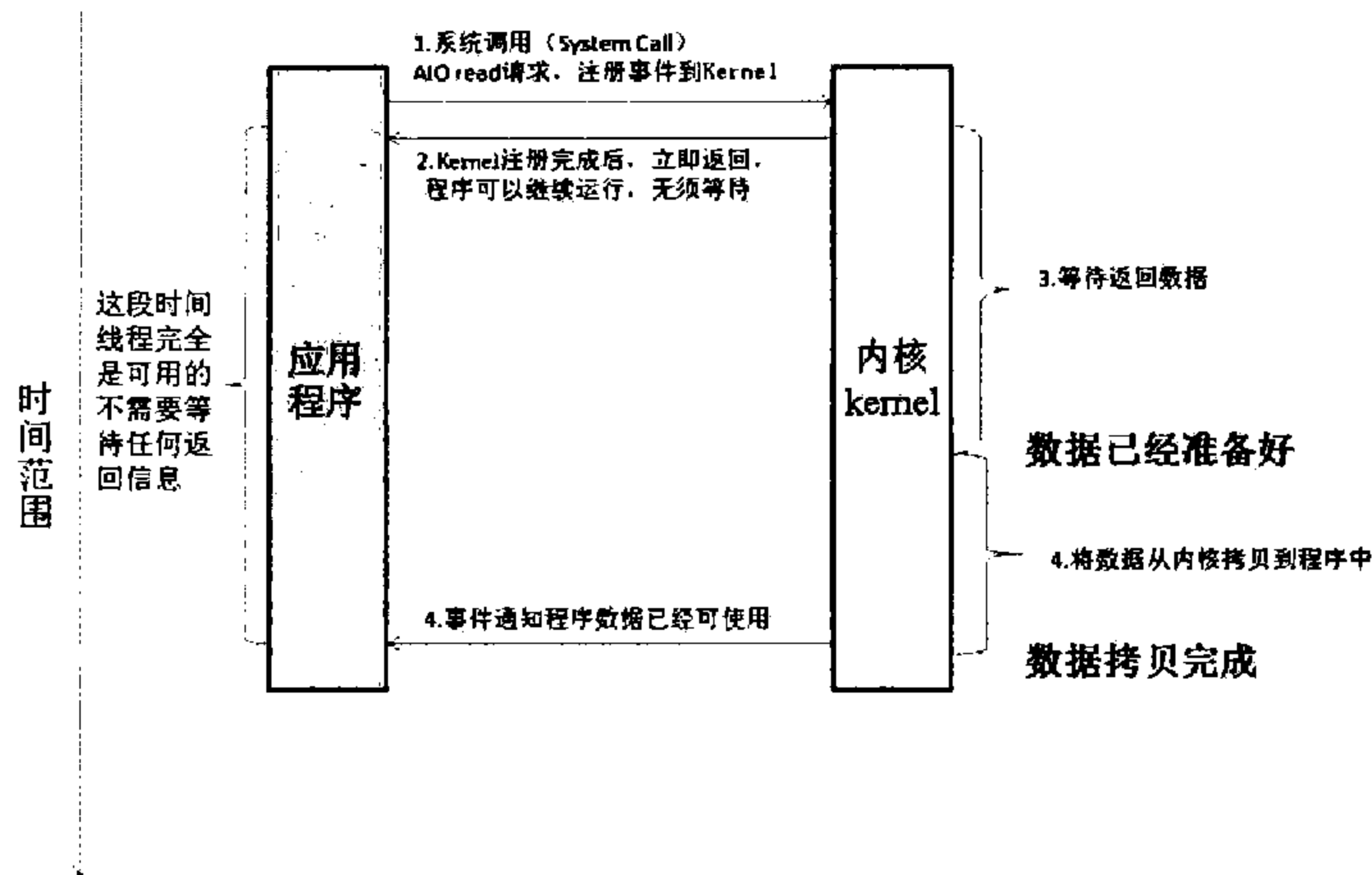


图 4-28 异步 I/O 模型的概要图，程序注册好事件后就什么也不用管了

AIO 相对 NIO，在某种意义上讲更加提高了资源利用率，但这仅仅是相对进程本身而言的，对于整个服务器而言，还得看实际的情况，因为进程不想做的事情交给内核去做了。异步 I/O 应当更加适合运用在 I/O 密集型的系统中，BIO 也并非没有价值，它使得交互简化，而且在许多情况下本身没有必要使用异步和非阻塞，胖哥始终相信存在即有价值的道理。

对于 Java 的 AIO 也有很多的 Channel，统一接口来自于 AsynchronousChannel 接口，其衍生出的 Channel 如图 4-29 所示。

- ① AsynchronousChannel - java.nio.channels
 - Ⓜ AsynchronousFileChannel - java.nio.channels
 - Ⓜ AsynchronousFileChannelImpl - sun.nio.ch
 - Ⓜ WindowsAsynchronousFileChannelImpl - sun.nio.ch
 - Ⓜ AsynchronousServerSocketChannel - java.nio.channels
 - Ⓜ AsynchronousServerSocketChannelImpl - sun.nio.ch
 - Ⓜ WindowsAsynchronousServerSocketChannelImpl - sun.nio.ch
 - ① AsynchronousByteChannel - java.nio.channels
 - Ⓜ AsynchronousSocketChannel - java.nio.channels
 - Ⓜ AsynchronousSocketChannelImpl - sun.nio.ch
 - Ⓜ WindowsAsynchronousSocketChannelImpl - sun.nio.ch

图 4-29 异步 I/O 的 Channel

从图 4-29 中可以看出，它主要分为 3 个部分：一是文件的 Channel；二是 SocketServer 端用于打开端口提供服务的 Channel；三是 Socket 连接的 Channel。

比较稀奇的是我们发现：Java 的 AIO 对文件（File）也支持，在这里就写一段用 AIO 对文件进行读写操作的代码。

在写代码前我们要考虑一个问题：I/O 已经异步化了，Java 是如何让程序继续运行下去的？

Java 在这个模型中提供了两种机制来帮助我们写代码。

- ◎ 一种机制就是基于“回调”，我们只需要编写实体类实现接口 `CompletionHandler`，然后将这个实体传递给相应的 API 后，就会在事件触发时自动回调。
- ◎ 另一种机制是返回一个 `Future`，这个 `Future` 中有一个方法叫 `isDone()`，若该方法成立，那么数据就准备好了，或者通过 `get()` 方法等待数据完成。很显然，`isDone()` 方法的方式有点类似于 NIO 中的一个线程判定事件方式，而 `get()` 方法有点类似于阻塞 I/O。换句话说，它们虽然是 AIO 模型，但是貌似没有达到某些目的。在下面的例子中用一种基于回调的方式来完成。

胖哥在代码中写了一个叫作 `FileReadCompletion` 的类来实现 `CompletionHandler` 接口。在例子中还会对文件进行写操作，写文件操作依然使用同步的 `FileChannel` 来完成，就不去做异步化了，如果两个异步套在一起将会导致程序非常复杂，难以看懂。

另外，在程序中会看到一个 `System.in.read()` 调用，它和功能本身没有关系，它的存在是因为事件回调将交给 Kernel 来完成，程序中并不会马上启动一个线程来处理，如果仅仅是一个 `main` 方法来模拟这段程序，则可能看不到任何效果程序就结束了。因此增加一个 `System.in.read()`，可以使得程序在没有输入时不会结束。主要代码如下：

代码清单 4-10 通过 AIO 拷贝文件操作

```
public static void main(String []args) throws IOException {
    AsynchronousFileChannel readChannel
        = AsynchronousFileChannel.open(Paths.get("d:/BBB"));
    FileChannel writeChannel
        = new FileOutputStream("d:/BBB2").getChannel();
    ByteBuffer byteBuffer = ByteBuffer.allocate(1024);
    FileReadCompletion completion
        = new FileReadCompletion(byteBuffer, readChannel);
    readChannel.read(byteBuffer, 0L, writeChannel, completion);
    System.in.read(); //让程序暂停在这里，否则会直接退出
}
```

在代码中没有关闭 Channel，因为这是异步程序，当发生 `read()` 调用后，立即走下一步，如果使用 `finally` 将 Channel 关闭，就可能导致文件中的内容还没读出来就被关闭了。

胖哥的 `FileReadCompletion` 代码是怎么写的呢？为何能读整个文件呢？难道是里面回调又读取过？没错！下面来看看 `FileReadCompletion` 的一些实现。

该类需要实现 `CompletionHandler` 接口的两个方法：`failed(Throwable e, V attachment)` 和 `completed(V result, A attachment)`，它们分别对应出错和完成的回调。入口参数基本都是泛型，在实现类的声明中就可以定义 `attachment` 的类型为 `FileChannel`，由此就可以在每次 `completed()` 回调时都得到文件的 Channel。

在 `AsynchronousFileChannel` 的 `read()` 方法中传入的 `CompletionHandler` 为一个 `FileReadCompletion`，其泛型定义中的 `result` 必须为一个 `Integer` 类型（这是 `AsynchronousFileChannel` 类的 `read()` 方法对于回调对象的泛型要求），通过这个 `Integer` 值将得到读取的内容长度（如果大于 0，则表示有数据）。

`CompletionHandler` 泛型的第二个参数也将作为 `completed()` 方法的第二个入口参数，它以 `attachment` 的扩展参数的方式存在，我们可以不用它（`AsynchronousFileChannel()` 的 `read()` 方法的第三个参数），那么回调 `completed()` 时，对应的参数值是 `null`。在例子中是将写入文件的 `FileChannel` 作为参数传递的，这并不代表在实际场景中是合理的，仅仅是为了给大家展现一下可以这样传递。

下面看看 `completed()` 方法是怎么写的，其实很简单，代码片段如下：

代码清单 4-11 `FileReadCompletion` 的 `completed()` 方法的内容

```
@Override
public void completed(Integer result, FileChannel writeChannel) {
    if(result > 0) {
        byteBuffer.flip();
        try {
            int writeLength = writeChannel.write(byteBuffer, nowPosition);
            nowPosition += writeLength;
        } catch (IOException e) {
            /*可以加入自己的处理哦*/
        }
        byteBuffer.clear();
        readFileChannel.read(byteBuffer, nowPosition,
                             writeChannel, this);
    } else {
        closeStreams(readFileChannel, writeChannel);
    }
}
```

具体的大家可以参考光盘中 `src/chapter04/aio/file/` 目录下的代码，也可以进行测试。

使用 AIO 读取并不代表读取更快，我们不能用 AIO 读取一个大文件与用 BIO 来对比速度，那样没有任何意义。

因为 AIO 的目的在于在 I/O 过程中程序去做别的事情，也就是在并发时，更少的资源可以做更多的事情，并不是看在 I/O 过程中谁跑得更快，它也做不到这一点，因为能做到 I/O 更快只有提高磁盘或网络本身的速度，而这些 I/O 模型只是调度 I/O 的机制而已。

疑惑：程序中会不断注册事件、回调方法，这不是一种递归调用呢？如果是递归调用，程序会不会出现 `StackOverflow` 呢？

答曰：递归调用是在同一个线程中完成的，也就是同一个线程的“栈”一直被占用和延长。异步 I/O 并不是这样的，程序是将事件交给内核与 JVM 本身去管理，业务线程就开始去做其他的工作了。

唯一沾边的是上下文信息在注册时一起带入保存到一个对象中，以便于回调时继续使用，回调时会启动其他的线程来处理，而并非当前的线程，所以它不是递归调用，也不会导致 `StackOverflow` 的问题。

这里所讲的上下文信息的保存并不是由内核来完成的。内核所保存的仅仅是在内核调度时，当一个线程阻塞后，脱离 CPU 调度需要保存的上下文信息，也就是线程的运行栈空间依然保持，这里本身运行的线程会结束，私有栈空间已经被释放，回调时的处理线程通常是随机的。

大家也可以发现这个过程的开销其实也是不小的！所以没有必要测试速度，而我们也应该理解它用在哪里。

既然提到了线程，那么回调时在没有任何线程池的基础上自然会生成一个新的线程来处理，如果想要通过线程池来处理回调应该如何做呢？

在所有的 `AsynchronousFileChannel` 调用 `open` 操作的重载方法中，第三个参数可以传递 `ExecutorService` 入口参数，写入这个参数后，会通过这个线程池来调度回调操作任务（线程池通常在频繁调度中为了控制资源平衡而存在，在 5.6 节中将会详细阐述其内在）。

对于 `AsynchronousServerSocketChannel`，有一个 `AsynchronousChannelGroup`，它可以实

现对回调任务通过线程池来调度。AsynchronousChannelGroup 通过 withFixedThreadPool、withCachedThreadPool 和 withThreadPool 三种方式来创建线程池。启动一个服务器端，代码如图 4-30 所示。

```
public class AsynchronousServerTest {

    public static void main(String []args) throws IOException, InterruptedException, ExecutionException {
        AsynchronousChannelGroup group = AsynchronousChannelGroup.withCachedThreadPool(Executors.newFixedThreadPool(10), 2);
        AsynchronousServerSocketChannel serverChannel = AsynchronousServerSocketChannel.open(group);
        serverChannel.bind(new InetSocketAddress("localhost", 8888), 128);
        Future<AsynchronousSocketChannel> future = serverChannel.accept();
        //serverChannel.accept(attachment, handler)//可传入参数和回调的CompletionHandler达到和Future一样的目的
        process(future.get());
    }

    private static void process(AsynchronousSocketChannel asynchronousSocketChannel) {
        /*可以做一些自己的操作*/
    }
}
```

图 4-30 异步 I/O 模型启动服务器端，被注释的部分为回调方式

在 bind(SocketAddress, int)中，int 参数是做什么的？

int 类型的参数代表最多的等待队列的数量，虽然使用了异步 I/O，但是也不能确保同一个瞬间所有的请求被处理，因此有一个缓冲区是很正常的。

好奇的小伙伴或许会问：Java 的 AIO 是基于 epoll 实现的吗？

胖哥只能说可以通过 epoll 来实现，但未必必须要通过这样的方式来实现，这一层实现更多的是在 JVM 内部与 Kernel 的关系上，如果使用 epoll 实现，那么通知自然可以直接基于内核所提供的 I/O 机制来完成，而 JVM 承担的职责是将“送货上门”的任务拿来按照指定的代码来执行，而且 Java 通常只会在同一个端口上绑定一个进程，因此 epoll 的回调也不会导致经典的“惊群”问题。

如果不用 epoll 也是可以变相实现的，既然有 JVM 这一层，自然就有一层管理平台，任何一层管理都是抽象出来的模型，假如内核中只有类似 epoll 的功能，那么 JVM 可以承担起任务分派的职责。简单来说，JVM 会不断地从内核拿到事件列表，如果有准备好的事件，则将任务放入线程池队列中；若没有线程池，则直接创建一个线程来处理也是可以的。

总之，如果大家有兴趣可以自己去看更多的源码，但通常与环境有关系，如果大家有一天希望自己去模拟一些这样的模型，便会有许多的思路。

4.4 Tomcat 中对 I/O 的请求处理

前面提到了很多 I/O 模型，在本节中我们将简单梳理下浏览器与 Web 服务器之间的一些 I/O 交互并将其与前面的代码关联。Web 容器有很多种，我们无法一一列举，本节中将以使用较多的 Tomcat 服务器为例来说明。

关于 Tomcat 的下载以及文档，请大家参看：<http://tomcat.apache.org/>。接下来会提到一些官方参数，它们将以 Tomcat 6 的参数为准，具体的请参看：<http://tomcat.apache.org/tomcat-6.0-doc/config/http.html>。大家如果想要关心 Tomcat 7 的文档，则将路径中的 6.0 改为 7.0 即可找到。

4.4.1 Tomcat 的配置&一个请求的响应

打开 <http://tomcat.apache.org/tomcat-6.0-doc/config/http.html>，可以先看看左侧的目录，不难发现有一系列配置体系，如：Server、Service、Executor、HTTP 等。

这是做什么用的呢? 为何会这样列举呢?

这是 Tomcat 的一个由外层到内层的访问机制, 用过 Tomcat 的小伙伴都应该配置过 Server.xml。现在我们就从这个熟悉的 XML 文件开始吧。

此时打开 Tomcat 的 conf 目录下的配置文件 Server.xml, 即可看到最外一层是 Server, 接着的各层配置与官方的配置目录类似。其实这个层次结构也基本上是与源码对应的, 为此, 我们就看看 Server.xml 的层次结构模板, 如图 4-31 所示。

```
<?xml version='1.0' encoding='utf-8'?>
<Server port="8005" shutdown="SHUTDOWN">
  <Listener className="..." />
  <GlobalNamingResources>

    <Resource name="..." />

  </GlobalNamingResources>
  <Service name="Catalina">
    <Connector port="8080" protocol="HTTP/1.1"
      connectionTimeout="2000"
      useBodyEncodingForURI="true"
      URIEncoding="UTF-8"
      redirectPort="8443" />
    <Connector port="8009" protocol="AJP/1.3" redirectPort="8443" />

    <Engine name="Catalina" defaultHost="localhost">
      <Host name="localhost" appBase="webapps"
        unpackWARs="true" autoDeploy="true">

        <Context path="..." />
        <Valve className="..." />
      </Host>
    </Engine>
  </Service>
</Server>
```

图 4-31 Server.xml 配置的层次结构模板

Tomcat 启动时也是启动 Java 的 main 方法, 它在哪里呢?

在 Tomcat 的 lib 目录下有一个 jar 包 catalina.jar, 引导方法 (main) 就在这个 jar 包的 org.apache.catalina.startup.Bootstrap 类当中, 使用 jps 这类工具就能在 JVM 进程中看到 (在启动脚本中就可以看到 MAIN_CLASS 的设置以及 catalina.jar)。

如果跟踪源码, 我们会发现, 系统启动时会相继启动 Server, Tomcat 中的类是 org.apache.catalina.core.StandardServer, 然后设置 Listener 等信息, 并相继调用其 initialize() 方法, 该方法用于启动 Service, 对应于 Tomcat 源码中的 org.apache.catalina.core.StandardService, 在 StandardService 的初始化方法中, 就开始初始化 Connector、Engine 等相关信息。

配置文件中有许多配置项, 从何处开始看起呢? 我们可以从最常见的开始看起来引导思路。什么是最常见的呢?

通常最关注的是当一个请求发送到 Web 容器时是如何处理的, Tomcat 中的组件 Connector 在配置文件中配置是最多、最常用的, 对应于 Tomcat 源码中的 org.apache.catalina.connector.Connector, 如图 4-32 所示。

```
<Connector port="8080" protocol="HTTP/1.1"
  connectionTimeout="2000"
  URIEncoding="UTF-8"
  redirectPort="8443" />

<Connector port="8009" protocol="AJP/1.3" redirectPort="8443" />
```

图 4-32 Connector 配置代码

配置 Connector 时通常会出现 protocol="HTTP/1.1"、protocol="AJP/1.3"等，它们表示 Tomcat 会使用哪一种协议来处理请求，对应到代码中就是用哪一套程序来处理 I/O 请求。下面来看看这部分源码是怎么写的，位于 Connector 类中的 setProtocol(String)方法是这样的，如图 4-33 所示。

```
public void setProtocol(String protocol) {  
  
    if (AprLifecycleListener.isAprAvailable()) {  
        if ("HTTP/1.1".equals(protocol)) {  
            setProtocolHandlerClassName  
                ("org.apache.coyote.http11.Http11AprProtocol");  
        } else if ("AJP/1.3".equals(protocol)) {  
            setProtocolHandlerClassName  
                ("org.apache.coyote ajp.AjpAprProtocol");  
        } else if (protocol != null) {  
            setProtocolHandlerClassName(protocol);  
        } else {  
            setProtocolHandlerClassName  
                ("org.apache.coyote.http11.Http11AprProtocol");  
        }  
    } else {  
        if ("HTTP/1.1".equals(protocol)) {  
            setProtocolHandlerClassName  
                ("org.apache.coyote.http11.Http11Protocol");  
        } else if ("AJP/1.3".equals(protocol)) {  
            setProtocolHandlerClassName  
                ("org.apache.jk.server.JkCoyoteHandler");  
        } else if (protocol != null) {  
            setProtocolHandlerClassName(protocol);  
        }  
    }  
}
```

图 4-33 Tomcat 设置协议的代码，注意加框部分

在这段代码中根据配置文件中输入的名称来设置类名，对代码简单解释如下。

Tomcat 首先通过 isAprAvailable()判定 APR 的本地库是否能加载，它是 Tomcat 一个的 JNI 库，这个库通常在 bin 目录下，在 Windows 操作系统下是一中 DLL 文件，在 Linux 系统中是一个 SO 文件，源码类是 org.apache.tomcat.jni.Library。其实 Tomcat 没有加载对应 OS 的 APR 照样可以运行（在 OS 和位数不匹配时可能会在日志里面抛出错误）。换句话说，Tomcat 本身是不区分操作系统以及位数的，只有它的 JNI 操作才会区分 OS 和位数，但是即使不匹配，Tomcat 也可以照常运行。默认加载的类关系如表 4-3 所示。

表 4-3 默认加载的类关系

名 称	APR 成功加载	接受请求的处理类
HTTP/1.1	Y	org.apache.coyote.http11.Http11AprProtocol
	N	org.apache.coyote.http11.Http11Protocol
AJP/1.3	Y	org.apache.coyote.ajp.AjpAprProtocol
	N	org.apache.jk.server.JkCoyoteHandler
其他设置（非空）	不区分	输入值直接作为协议类名
空（未设置）	Y	org.apache.coyote.http11.Http11AprProtocol
	N	不做任何处理

Tomcat 不是支持 NIO 吗？在哪里？

通过表 4-3 可以发现，如果设置的 protocol 不为空，而且并不是默认的 HTTP/1.1、AJP/1.3，则将直接输入的参数作为协议类名（可以放一个自己写的协议处理类进去，记得

要实现接口 `ProtocolHandler`、`MBeanRegistration` 做相应的处理)。Tomcat 支持 NIO 的类为：`org.apache.coyote.http11.Http11NioProtocol`。

现在我们至少知道了 3 种协议的处理类，里面分别有一个“`XXXEndpoint`”属性，这个是请求处理的关键部分。现在来看看这些 `Endpoint` 与协议类的对应关系，如表 4-4 所示。

表 4-4 Endpoint 与协议的对应关系

序 号	协 议 类	对应的 Endpoint
1	<code>org.apache.coyote.http11.Http11AprProtocol</code>	<code>AprEndpoint</code>
2	<code>org.apache.coyote.http11.Http11Protocol</code>	<code>JioEndpoint</code>
3	<code>org.apache.coyote.http11.Http11NioProtocol</code>	<code>NioEndpoint</code>

这些 `Endpoint` 都位于 `org.apache.tomcat.util.net` 下面，接下来以最常见的 `JioEndpoint` 为例，看看它接受请求的地方在哪里？

在 `Http11Protocol` 类调用 `init()`初始化方法时会创建 `socketFactory`，并将其设置到 `JioEndpoint` 中，再调用 `JioEndpoint` 的 `init()`方法，代码截图如图 4-34 所示。

```
public void init() throws Exception {
    endpoint.setName(getName());
    endpoint.setHandler(cHandler);

    // Verify the validity of the configured socket factory
    try {
        if (isSSLEnabled()) {
            sslImplementation =
                SSLImplementation.getInstance(sslImplementationName);
            socketFactory = sslImplementation.getServerSocketFactory();
            endpoint.setServerSocketFactory(socketFactory);
        } else if (socketFactoryName != null) {
            socketFactory = (ServerSocketFactory) Class.forName(socketFactoryName).newInstance();
            endpoint.setServerSocketFactory(socketFactory);
        }
    } catch (Exception ex) {
        log.error(sm.getString("http11protocol.socketfactory.initerror"),
            ex);
        throw ex;
    }

    if (socketFactory!=null) {
        Iterator<String> attE = attributes.keySet().iterator();
        while( attE.hasNext() ) {
            String key = attE.next();
            Object v=attributes.get(key);
            socketFactory.setAttribute(key, v);
        }
    }

    try {
        endpoint.init();
    } catch (Exception ex) {
```

图 4-34 Http11Protocol 的 init()方法中调用 Endpoint 的初始化方法

在这个 `JioEndpoint` 的 `init()`方法中设置了 `acceptorThreadCount`，用于标识接受请求的线程数（也就是可以有多个线程同时接受请求，默认是 1。一般不用修改这个参数，如果要修改，参数名也是 `acceptorThreadCount`），然后开始创建 `SocketServer` 对象，源码如图 4-35 所示。

相关内容初始化好后，系统开始启动 `start()`方法（这是前面提到的系统启动相关的串联操作）。同样经过一系列的组件调用后会调用到 `Http11Protocol` 类的 `start()`方法，它会相继调用 `JioEndpoint` 的 `start()`方法，`start()`方法也会先判定是否初始化，若未初始化，则会先调用初始化方法，如图 4-36 所示。

```

public void init()
    throws Exception {

    if (initialized)
        return;

    // Initialize thread count defaults for acceptor
    if (acceptorThreadCount == 0) {
        acceptorThreadCount = 1;
    }
    if (serverSocketFactory == null) {
        serverSocketFactory = ServerSocketFactory.getDefault();
    }
    if (serverSocket == null) {
        try {
            if (address == null) {
                serverSocket = serverSocketFactory.createSocket(port, backlog);
            } else {
                serverSocket = serverSocketFactory.createSocket(port, backlog, address);
            }
        }
    }
}

```

图 4-35 JioEndpoint 类的初始化方法

```

public void start()
    throws Exception {
    // Initialize socket if not done before
    if (!initialized) {
        init();
    }
    if (!running) {
        running = true;
        paused = false;

        // Create worker collection
        if (executor == null) {
            workers = new WorkerStack(maxThreads);
        }

        // Start acceptor threads
        for (int i = 0; i < acceptorThreadCount; i++) {
            Thread acceptorThread = new Thread(new Acceptor(), getName() + "-Acceptor-" + i);
            acceptorThread.setPriority(threadPriority);
            acceptorThread.setDaemon(daemon);
            acceptorThread.start();
        }
    }
}

```

图 4-36 JioEndpoint 的 start()方法

这段代码做两件事情：

- ◎ 第一件事情是在没有设置 Executor 时创建一个线程组来处理请求，线程组的容量由 maxThreads 决定（它仅仅是一个数组管理线程，并不是 Java 本身的 ThreadGroup）。Executor 的单独配置例子如下：

```

<Executor name="tomcatThreadPool" namePrefix="testThreadPool"
    maxThreads="150" minSpareThreads="4"/>

```

然后在 Connector 上配置 executor="testThreadPool"即可。

- ◎ 第二件事情是将接受请求的线程启动起来。前文中提到，启动的线程仅仅是用于接受请求使用。这些接受请求的线程启动后的动作是什么呢？那得看启动线程时传入的任务 Acceptor，它作为 Thread 的一个参数传入后，将被设置到 target 属性中，最终被 Thread 调用执行（第 5 章会详细介绍）。

再来看看这个 Acceptor 的 run()方法，如图 4-37 所示。


```

public void run() {

    // Loop until we receive a shutdown command
    while (running) {

        // Loop if endpoint is paused
        while (paused) {
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                // Ignore
            }
        }

        // Accept the next incoming connection from the server socket
        try {
            Socket socket = serverSocketFactory.acceptSocket(serverSocket);
            serverSocketFactory.initSocket(socket);
            // Hand this socket off to an appropriate processor
            if (!processSocket(socket)) {
                // Close socket right away
                try {
                    socket.close();
                } catch (IOException e) {
                    // Ignore
                }
            }
        } catch (IOException x) {
            if (running) log.error(sm.getString("endpoint.accept.fail"), x);
        } catch (Throwable t) {
            log.error(sm.getString("endpoint.accept.fail"), t);
        }
    }
}

```

图 4-37 Acceptor 接受请求并处理请求的调用

在代码中通过 `serverSocketFactory` 对象的 `acceptSocket()` 方法间接调用 `ServerSocket` 的 `accept()` 方法，得到一个 `Socket` 连接，也就是当一个连接请求到来时，就走到这里来了（还记得前面的 `Socket` 交互代码吗？Web 应用程序也是这样写的，只是它们帮助我们做了更多层面的包装，让我们尽量少关注交互的细节）。

`processSocket(socket)` 方法就开始对 `Socket` 内容进行处理了，其内部对处理线程开始分配。此时有的小伙伴会思考：配置单独的线程池与配置了 `maxThreads` 在内部处理方式上到底有什么样的区别？正好看到这里，谜底马上揭晓，代码截图如图 4-38 所示。

```

protected boolean processSocket(Socket socket) {
    try {
        if (executor == null) {
            getWorkerThread().assign(socket);
        } else {
            executor.execute(new SocketProcessor(socket));
        }
    } catch (Throwable t) {
    }
}

```

图 4-38 分配请求的两条路径

这两条执行路径前者是没有配置线程池的（`executor`），后者配置了线程池。如果配置了线程池，则创建一个 `SocketProcessor`（同样是基于 `Runnable` 接口的任务），通过 `execute(Runnable)` 方法将其提交给线程池来调度即可。

如果是通过 `getWorkThread()` 来获取一个处理线程，则会通过以下两段代码来处理，如图 4-39 所示。

在图 4-39 中可以看到，如果没有配置 `executor`，仅仅是根据 `maxThreads` 参数创建一个线程组，那么跟踪源码就可以发现线程组的元素都是空的，这个 `workers` 的 `size()` 在刚开始时也应当是 0，那么在什么时候才会发生 `pop` 操作呢？请看图 4-40。

```

protected Worker getWorkerThread() {
    // Allocate a new worker thread
    synchronized (workers) {
        Worker workerThread;
        while ((workerThread = createWorkerThread()) == null) {
            try {
                workers.wait();
            } catch (InterruptedException e) {
                // Ignore
            }
        }
        return workerThread;
    }
}

protected Worker createWorkerThread() {

    synchronized (workers) {
        if (workers.size() > 0) {
            curThreadsBusy++;
            return workers.pop();
        }
        if ((maxThreads > 0) && (curThreads < maxThreads)) {
            curThreadsBusy++;
            if (curThreadsBusy == maxThreads) {
                log.info(sm.getString("endpoint.info.maxThreads",
                    Integer.toString(maxThreads), address,
                    Integer.toString(port)));
            }
            return (newWorkerThread());
        } else {
            if (maxThreads < 0) {
                curThreadsBusy++;
                return (newWorkerThread());
            } else {
                return (null);
            }
        }
    }
}

```

图 4-39 该代码片段是没有配置线程池的处理

```

public void run() {

    // Process requests until we receive a shutdown signal
    while (running) {

        // Wait for the next socket to be assigned
        Socket socket = await();
        if (socket == null)
            continue;

        // Process the request from this socket
        if (!setSocketOptions(socket) || !handler.process(socket)) {
            // Close socket
            try {
                socket.close();
            } catch (IOException e) {
            }
        }

        // Finish up this request
        socket = null;
        recycleWorkerThread(this);
    }
}

```

图 4-40 线程运行一个任务后调用 recycleWorkerThread 来回收

回收的动作如图 4-41 所示。

总的来讲，其工作流程为：如果没有配置线程池，则会使用线程组来处理访问请求，线程组内部如果有闲置的线程，则会分配一个线程来处理，若没有则创建一个线程来处理。任务结束后（但是线程不会结束），会将当前线程放入 workers 数组当中。


```
protected void recycleWorkerThread(Worker workerThread) {
    synchronized (workers) {
        workers.push(workerThread);
        curThreadsBusy--;
        workers.notify();
    }
}
```

图 4-41 回收的动作

上面我们讲解了配置文件，以及创建 Socket、创建线程池的源码，同时大家应该理解配置参数与源码的对应关系，有些内部的处理要比想象的复杂许多。

对应的 Web 容器如果仅仅说到这里，许多小伙伴会感到意犹未尽，那么下面我们再来说说 request 等信息是在什么时候生成的。

➤➤ 4.4.2 Request、Response 对象生成

接着看上面的代码，分配的线程类是 JIoEndpoint 类其中的一个内部类 Worker，它作为一个线程运行起来后是如何处理请求的，自然是要看它的 run()方法的主要处理逻辑了，在图 4-40 中已经给出。

现在我们需要回头再看看图 4-40 中的代码，其内部调用了 setSocketOptions()方法来设置 Socket 的相关参数，如图 4-42 所示。

```
protected boolean setSocketOptions(Socket socket) {
    // Process the connection
    int step = 1;
    try {

        // 1: Set socket options: timeout, linger, etc
        if (soLinger >= 0) {
            socket.setSoLinger(true, soLinger);
        }
        if (tcpNoDelay) {
            socket.setTcpNoDelay(tcpNoDelay);
        }
        if (soTimeout > 0) {
            socket.setSoTimeout(soTimeout);
        }
    }
}
```

图 4-42 设置 Socket 的相关参数

这些参数是否似曾相识，请参看 2.6.2 节中介绍 Socket 参数的部分。如果基础很扎实，看这些内容是很轻松的。

回到图 4-40 中，设置完 Socket 参数后就会交给一个 Handler 接口来处理，调用其 process(Socket)方法，对于目前讨论的协议类型 (Http11Protocol)，这个 Handler 接口的实现类是 Http11Protocol 的内部类 Http11ConnectionHandler，它的 process()方法的主要内容如图 4-43 所示。

这里会试图从回收的列表中获取一个 Http11Processor 对象，如果没有获取到，则会创建一个新的 Http11Processor 对象。

这个 Http11Processor 对象是 ActionHook 接口的实现类的对象，所以根据代码会调用到 Http11Processor 类的 action()方法。这个方法的 body 部分很长，不过幸运的是，我们发现当其内部判定入口参数为 ACTION_START 后就只是做了一个状态位的设置，所以这个步骤我们基本可以认为就是设置启动状态。

```
public boolean process(Socket socket) {
    Http11Processor processor = recycledProcessors.poll();
    try {

        if (processor == null) {
            processor = createProcessor();
        }

        if (processor instanceof ActionHook) {
            ((ActionHook) processor).action(ActionCode.ACTION_START, null);
        }

        if (proto.isSSLEnabled() && (proto.sslImplementation != null)) {
            processor.setSSLSupport
                (proto.sslImplementation.getSSLSupport(socket));
        } else {
            processor.setSSLSupport(null);
        }

        processor.process(socket);
    }
    return false;
}
```

图 4-43 将 Socket 交给一个 processor 来处理

在图 4-43 中还出现了对 SSL 的处理，不过它不是本书要讲解的内容，在这里我们只需要关注请求的对象是如何生成的，下面我们继续跟踪代码，看一看代码中创建 Processor 对象时到底做了什么，请看图 4-44。

```
protected Http11Processor createProcessor() {
    Http11Processor processor =
        new Http11Processor(proto.maxHttpReaderSize, proto.endpoint);
    processor.setAdapter(proto.adapter);
    processor.setKeepAliveRequests(proto.maxKeepAliveRequests);
    processor.setKeepAliveTimeout(proto.keepAliveTimeout);
    processor.setTimeout(proto.timeout);
    processor.setDisableUploadTimeout(proto.disableUploadTimeout);
    processor.setCompressionMinSize(proto.compressionMinSize);
    processor.setCompression(proto.compression);
    processor.setNoCompressionUserAgents(proto.noCompressionUserAgents);
    processor.setCompressableMimeTypes(proto.compressableMimeTypes);
    processor.setRestrictedUserAgents(proto.restrictedUserAgents);
    processor.setSocketBuffer(proto.socketBuffer);
    processor.setMaxSavePostSize(proto.maxSavePostSize);
    processor.setServer(proto.server);
    register(processor);
    return processor;
}
```

```
public Http11Processor(int headerBufferSize, JioEndpoint endpoint) {
    this.endpoint = endpoint;

    request = new Request();
    inputBuffer = new InternalInputBuffer(request, headerBufferSize);
    request.setInputBuffer(inputBuffer);

    response = new Response();
    response.setHook(this);
    outputBuffer = new InternalOutputBuffer(response, headerBufferSize);
    response.setOutputBuffer(outputBuffer);
    request.setResponse(response);

    initializeFilters();

    // Cause loading of HexUtils
    int foo = HexUtils.DEC(0);
}
```

图 4-44 Request、Response 在这里

看到这里的 Request、Response，小伙伴们似乎开始有点兴奋不已，它们到底是不是我们平时所使用的 Request 和 Response 对象呢？大家千万不要被自己的眼睛欺骗哦！

通过跟踪你会发现，这个 Request 类为：org.apache.coyote.Request 类，它并没有实现接口 HttpServletRequest，这说明它肯定是假货，那么这个 Request 是做什么的？它主要用于内在的输入/输出流的绑定，以及对许多头部相关信息的 Byte 字节长度进行绑定，提供给外面使用。

现在回头再来看调用 Http11Processor 类的 process(Socket)方法。打开一看，“哎呀妈呀”这方面也太长了吧！我们只要看其中的几个关键点就行（见图 4-45）。

```
inputBuffer.setInputStream(socket.getInputStream());
outputBuffer.setOutputStream(socket.getOutputStream());
```

图 4-45 process(Socket)方法会首先保存输入/输出流

上面这段代码，看名字就猜到在操作过程中自然可以 Buffer 数据。回到图 4-44 中，看右边部分，可以发现这两个 Buffer 对象将被分别绑定到 Request、Response 所在的对象上。

接着看 process(Socket)方法的代码片段，如图 4-46 所示。

```

keepAlive = true;
int keepAliveLeft = maxKeepAliveRequests;
int soTimeout = endpoint.getSoTimeout();
int curThreads = endpoint.getCurrentThreadsBusy();
int maxThreads = endpoint.getMaxThreads();
if (curThreads > 0 && maxThreads > 0) {
    if ((curThreads*100)/maxThreads > 75) {
        keepAliveLeft = 1;
    }
}
try {
    socket.setSoTimeout(soTimeout);
} catch (Throwable t) {
    log.debug(sm.getString("http11processor.socket.timeout"), t);
    error = true;
}
boolean keptAlive = false;
while (started && !error && keepAlive && !endpoint.isPaused()) {
    try {
        if (keptAlive) {
            if (keepAliveTimeout > 0) {
                socket.setSoTimeout(keepAliveTimeout);
            }
            else if (soTimeout > 0) {
                socket.setSoTimeout(soTimeout);
            }
        }
        inputBuffer.parseRequestLine();
        request.setStartTime(System.currentTimeMillis());
        keptAlive = true;
        if (disableUploadTimeout) {
            socket.setSoTimeout(soTimeout);
        } else {
            socket.setSoTimeout(timeout);
        }
        inputBuffer.parseHeaders();
    } catch (IOException e) {
        异常处理省略...
    }
    部分默认参数设置代码省略...
    if (maxKeepAliveRequests > 0 && --keepAliveLeft == 0)
        keepAlive = false;
    后续处理代码省略...
}

```

图 4-46 process 内部对参数的一些处理

这段代码似乎很难读懂，下面解释一下参数与代码的关系。

keepAlive 默认为 true，第一个连接请求到来时，如果容器还在运行，那么程序中的 while 判定第 1 次循环肯定是会进去的。不过 keptAlive 这个局部变量初始化时为 false，第 1 次运行结束才会为 true，根据代码的判定逻辑，第 1 次进入循环不会与 keepAliveTimeout 相关，循环第 2 次才会优先使用 keepAliveTimeout 来作为 Socket 的参数设置。

代码中 keepAliveLeft 是从 maxKeepAliveRequests 中获取的（该参数可以自己设置，通过阅读源码可以发现其默认值为 100），它表示 keepAlive 一个连接最多保持多少个请求。换句话说，同一个客户端的多个连续请求，可以用同一个连接来完成，这样效率更高。

不过，在接下来的处理中，如果发现 curThreads 使用量达到了 maxThreads 的 75%，则会将 keepAliveLeft 的值设置为 1（注意：只设置一次，在进入循环后就不再重新设置了）。换句话说，如果系统认为线程使用量比较大，则会自动放弃 keepAlive。在后续的代码中，

每一次循环都会将 `keepAliveLeft` 减 1，结果等于 0 时，则将 `keepAlive` 参数设置为 `false`，也就是同一个连接每个请求到来时会减 1。前面判定线程使用量达到 75% 时将参数设置为 1，如果第 1 次循环将 `keepAlive` 设置为 `false`，那么就不会进入下一次循环了。

这段代码的逻辑理清清楚后，便知道系统什么时候启用 `keepAlive` 参数，以及该参数与线程数之间的某些关联关系了。在处理完这些基本的连接信息后，开始进入业务层，如图 4-47 所示。

```
if (!error) {
    try {
        rp.setStage(org.apache.coyote.Constants.STAGE_SERVICE);
        adapter.service(request, response);
    }
}
```

图 4-47 开始进入业务层的一些处理

其中的 `adapter` 对应到源码中的 `org.apache.catalina.connector.CoyoteAdapter`，它是业务逻辑处理的关键部分，其内部的 `service` 方法也很长，下面以代码片段的方式来看一些关键代码。首先来看一段关键代码里面也有创建 `Request` 和 `Response` 的过程，如图 4-48 所示。

```
// Create objects
request = (Request) connector.createRequest();
request.setCoyoteRequest(req);
response = (Response) connector.createResponse();
response.setCoyoteResponse(res);
```

图 4-48 `CoyoteAdapter` 的 `service` 方法中也创建了 `Request`、`Response`

小伙伴们注意了，在这个方法内部还包含了对 Comet 的处理，有一个很明显的特征是，如果启用 Comet，则不会调用 `Response` 的 `finishResponse()` 方法。

其中的 `createRequest` 也是返回 `Request` 类型，那为什么还要把传入的 `Request req` 写到这个创建的 `Request` 里面呢？

这个 `Request` 对象所在类的 `package` 路径是 `org.apache.catalina.connector.Request`，相应的 `Response` 也是类似的。它将前文中介绍的已经创建的 `org.apache.coyote.Request` 对象作为自身的一个属性包装起来。

现在这个 `org.apache.catalina.connector.Request` 确实实现了接口 `HttpServletRequest`，但是我们用的是不是这个对象呢？

不是！

我们继续看下面的代码。

```
Connector.getContainer().getPipeline().getFirst().invoke(request, response);
```

这里会调用到 `org.apache.catalina.core.StandardEngineValve` 类的 `invoke()` 方法，不要理解为动态调用，它就是一个普通的方法而已，然后会依次调用下面的类。

```
org.apache.catalina.valves.ErrorReportValve
org.apache.catalina.core.StandardHostValve
org.apache.catalina.authenticator.AuthenticatorBase
org.apache.catalina.core.StandardContextValve
org.apache.catalina.core.StandardWrapperValve
```

一系列的 `invoke` 方法处理，它们都有自己的职责，本书不对每个步骤细节进行详述了，现在看一下最后一个 `StandardWrapperValve` 的 `invoke` 方法中的一些关键代码，如图 4-49 所示。

大家应该注意到图 4-49 中的代码调用是和 Filter 相关的，这里用了一个 request.getRequest()传入 Filter 的 doFilter()里面，自然这个 getRequest()方法里面的返回对象才是程序中所使用的 Request 对象，如图 4-50 所示。

```
if ((servlet != null) && (filterChain != null)) {
    // Swallow output if needed
    if (context.getSwallowOutput()) {
        try {
            SystemLogHandler.startCapture();
            if (comet) {
                filterChain.doFilterEvent(request.getEvent());
                request.setComet(true);
            } else {
                filterChain.doFilter(request.getRequest(),
                                     response.getResponse());
            }
        } finally {
            String log = SystemLogHandler.stopCapture();
            if (log != null && log.length() > 0) {
                context.getLogger().info(log);
            }
        }
    } else {
        if (comet) {
            request.setComet(true);
            filterChain.doFilterEvent(request.getEvent());
        } else {
            filterChain.doFilter
                (request.getRequest(), response.getResponse());
        }
    }
}
request.removeAttribute(Globals.JSP_FILE_ATTR);
```

图 4-49 StandardWrapperValve 的 invoke 方法的关键代码

```
public HttpServletRequest getRequest() {
    if (facade == null) {
        facade = new RequestFacade(this);
    }
    return (facade);
}
```

图 4-50 创建一个业务程序中使用的 Request 对象

该方法把自身包装到了一个 RequestFacade 中，同样实现了接口 HttpServletRequest，对应类的全名为：org.apache.catalina.connector.RequestFacade。在业务程序中如果没有经过框架再包装或者在 Filter 里被替换为相应的对象，那么用到的 Request 就是这个对象。

这里另一个容易迷惑的地方是，代码中出现的 Filter 还不是程序中自己写的 Filter，而是 ApplicationFilterChain，通过调用它的 doFilter 方法间接地调用 internalDoFilter 方法，在这个方法内部实现了对过滤器的调用和业务方法的分派。

现在一条线基本串通了，下面来看看这些 Request 对象的包装关系，如图 4-51 所示。

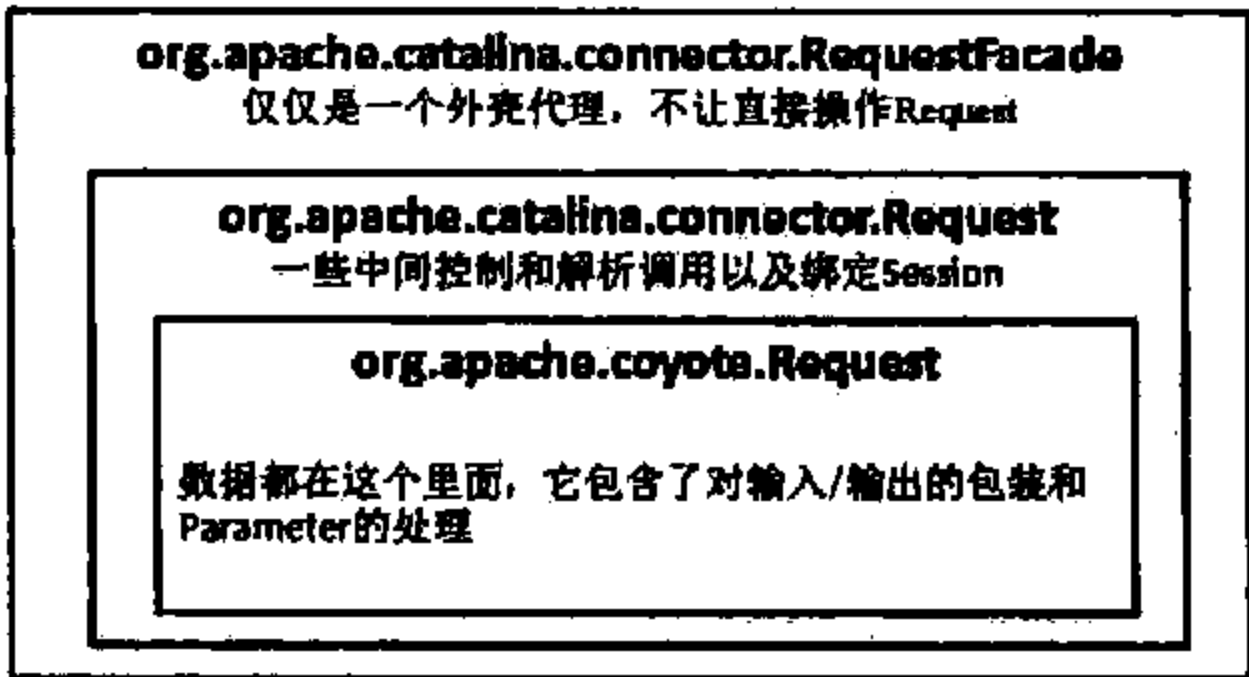


图 4-51 Request 对象的包装关系

在后面的章节中，会逐步细化图 4-51，目前还只是一个主体框架，并且是从 Request 角度来看问题的，当我们遇到实际问题时，就需要看这些层次结构的一些细节了。

特别提醒：这里并没有对 Tomcat 的源码进行诠释，而仅仅是针对接受请求后相关对象生成过程进行了跟踪和理解，具体问题尚未提到，例如 Tomcat 对 Comet 的支持、资源回收、启动 ClassLoader、字符集解码、JNDI、密码加密等。

4.4.3 拉与推

最后来简单谈谈拉与推的关系。为了说明这个问题，我们来思考一下生活中的买卖关系。刚开始做商品销售的同学，由于自己没有群众基础，所以采用的方式通常是“拉客户”，因为他有职业需求。就像通过浏览器请求服务器一样，因为你需要它为你处理一些内容或返回一些内容。

推呢？当某位同学手里面拥有大量的用户群体后，希望将他的最新优惠、利益等推送给他的客户，让客户与自己得到双赢。

如果站在客户端的角度，则可以认为总是在给服务器发内容，即使提交也是由服务器端处理好以后反馈的，这样的服务器实现都是以“拉”为目的而存在的。“推”的目的“很暴力”，只要你打开软件，就把数据给你，通常不关注是否反馈，就像一些“群发短信”一样，不想收还非要发出来。

“推送”数据的方式如果利用得不错，则是一种不错的做法，现在许多网站技术的实现效果类似于这样，服务器端某些状态发生改变后便可以很快地在客户端反映出来，这样让客户端的体验很好。

推送的实现方式通常有如下几种。

一种方式是每个客户端每隔很短的时间就发起一次服务器端请求获取最新的信息。假如每 5 秒发起一次请求，也许这个请求在后台的操作十分简单，处理速度非常快，但是这个请求并不知道消息是否已经发生改变，而是闷着脑袋发送消息，服务器端也会对每个请求做出应答（即使反馈不要再发请求，也会应答）。若每个请求都需要去检索数据库或某些缓存，那么此时服务器端自然会承受很大的压力。另外，若基于浏览器/服务器模式，浏览器开启的多个选项卡、子页面都可能与服务器端每 5 秒进行一次通信，那么此时的请求会成倍上升。

但是胖哥认为使用这个办法与服务器端通信并非完全不可以，因为通常服务器端的操作是十分简单的，如果客户端发起的请求是可控的（例如手机客户端 Android 有类似的方式，可以在软件级别进行控制），并且思路清晰，能把控边界，那么很多时候就是可行的方法，因为计算机内有无限多这样的线程在做类似的事。

另一种方式就是真正的推送，使用推送方式理论上是最有效的，也就是当信息发生改变时触发被改变的内容通知每个终端，也许会直接将数据交给终端，也许只是一个状态改变通知，然后进一步由客户端自己决定如何处理。

要完成推送，通常客户端与服务器端的连接是长连接，理论上长连接是不断开的一种连接，在 Web 层面最简单的方式是不断开 Response 输出流（前文中提到，Comet 启用后就不会调用 finishResponse() 的操作就是这个意思），不断开的 Response 输出内部处理要复杂一些，幸运的是，现在有很多开源框架已经支持，例如 pushlet、Cometd、DWR、Atmosphere，也包含 Tomcat 6 通过 CometProcessor 对 Comet 的支持。当建立长连接后，服务器端对消息

的处理可以拥有主动性，消息也可以由服务器端准备好后放入内存中，不用针对每个请求去分配消息所需要的内存（如果是基于客户端软件的应用，那么这个问题十分简单，例如手机客户端软件）。

长连接进来后就涉及连接与线程的解耦合问题，因为这类请求不再是一个对应一个线程了，因为一个线程可能会操作很多个连接，所以前文中的非阻塞 I/O 的引入是必然的。

知道了这些，或许我们在看待相关的问题上会更加清晰。在 IT 技术行业，想要一直深入下去，并不会像单纯的工作一样一成不变，你始终会遇到新的问题、没解决过的问题，我们需要知道一些基本思想，知道技术变更的方向。换句话说，不知道基本思想，那么就像无头的苍蝇一样在乱转；不知道变革方向，那么可能很快就 out。在技术变化的时代两者都很重要，往往相互磨合，相互提升。

对于基础不好的同学，可以迭代式看本章内容。另外，大家要多实践，胖哥给的一些代码可以多改改参数进行测试，一些没有写全的代码可以自己动手完成。

本章内容是 Java 基础中的基础，否则许多涉及网络的知识将会难以理解。

下一章是 Java 并发编程的一些基础知识，是本书的另一个大章节，大家在阅读之前要准备好精神粮食哦！

第5章

Java 并发，你会遇到吗

本章主要内容

- 基础介绍
- 并发可见性
- 原子性
- 并发编程的一些集合类
- 并发编程的一些工具类
- 线程池与调度池
- 其他并发编程知识
- 并发编程应当注意什么

本章不是对 Java 并发编程的诠释，关于更多并发编程的知识，大家可以参看站点 <http://tutorials.jenkov.com/java-concurrency/index.html> 上的文章、中文站点 <http://ifeve.com/> 上的国内著名的“大侠”写的文章，以及国外的并发编程的译文。

5.1 基础介绍

➤➤ 5.1.1 线程基础

本节内容介绍给那些还没接触过 Java 线程的朋友，希望能有个感性认识。

Java 线程，英文名是 Thread，所有的 Java 程序的运行都是在进程中分配线程来处理的。如果是一个 main 方法，则由一个主线程来处理，如果不创建自定义线程，那么这个程序就是单线程的。如果是 Web 应用程序，那么就由 Web 容器分配线程来处理（在 4.4.1 节中介绍了在 Tomcat 源码中是如何分配线程的）。

也许在使用 main 方法写代码时我们感觉不到多线程的存在，在 Web 程序中也感觉不到多线程和自己编写程序有什么关系，但是当遇到一些由于 Java 并发导致的古怪的问题时，当需要自己用多线程来编写程序或者控制多个线程访问共享资源时，就会用到相应的知识。

掌握知识的目标是驾驭知识，要驾驭知识的前提是了解知识，认识它的内在！

在 Java 代码中，单独创建线程，都要使用类 `java.lang.Thread`，通常可以通过继承并扩展 Thread 原本的 `run()` 方法，也可以创建一个 Thread，将一个 Runnable 任务实体作为参数传入，这是通过线程来执行任务的过程，但并不能说实现了 Runnable 接口就是一个线程。

Runnable 接口，顾名思义，就是“可以被执行”的意思。在 Java 语言中还有一些类似的接

口，例如 `Closeable`，它只能代表“可以被关闭”，`Closeable` 接口的描述中提供了一个 `close()` 方法要求子类实现。类似的，`Runnable` 接口提供了一个要求在实现类中去实现的 `run()` 方法，换句话说，`Runnable` 接口只是说明了外部程序都可以通过其实例化对象调用到 `run()` 方法，因此我们通常把它叫作“任务”（切忌将任务和时间挂在一起，基于时间的任务只是一类特殊的任务而已）。

从另一个角度来看，一个线程的启动是需要通过 `Thread.start()` 方法来完成的，这个方法会调用本地方法（JNI）来实现一个真正意义上的线程，或者说只有 `start()` 成功调用后由 OS 分配线程资源，才能叫作线程，而在 JVM 中分配的 `Thread` 对象只是与之对应的外壳。

`Runnable` 既然不是线程，那么有何用途？

前面提到，可以把 `Runnable` 看成一个“任务”，如果它仅仅与 `Thread` 配合使用，即在创建线程的时候将 `Runnable` 的实例化对象作为一个参数传入，那么它将被设置到 `Thread` 所在的对象中一个名为“target”的属性上，`Thread` 默认的 `run()` 方法是调用这个 target 的 `run()` 方法来完成的，这样 `Runnable` 的概念就与线程隔离了——它本身是任务；线程可以执行任务；否则 `Thread` 需要通过子类去实现 `run()` 方法来描述任务的内容。

在后文中会提到线程池中的每个 `Thread` 可以尝试获取多个 `Runnable` 任务，每次获取过来后调用其 `run()` 方法，这样就更加明显地说明 `Thread` 和 `Runnable` 不是一个概念。

区分了这个概念后，下面用一段简单代码来模拟一个线程的创建和启动。请看代码清单 5-1，在这段代码中 `new Thread() {...}` 在 Java 堆中创建了一个简单的 Java 对象，当通过这个对象调用其 `start()` 方法后就启动了一个线程。不过大家需要注意的是，在这段代码中胖哥将两条代码合并为一条来完成，不过在内在的执行上依然会是两条代码来完成。

代码清单 5-1 一个简单的 `Thread` 的执行

```
public static void main(String []args) {
    new Thread() {
        public void run() {
            System.out.println("我是被创建的线程，我执行了...");
        }
    }.start();
    System.out.println("main process end...");
}
```

为了简单起见，这段程序使用了一个匿名子类，重写了 `Thread` 的 `run()` 方法，与单独写一个继承于 `Thread` 的类在功能上是一致的。

这段程序，只是让初学者了解到线程的存在。

如果是顺序执行的程序，则应当先输出“我是被创建的线程，我执行了...”，然后再输出“main process end...”（因为代码顺序是这样的），但是大家通过测试结果会发现不一定，而且一般是先输出“main process end...”，这是因为 `run()` 方法被另一个线程调用了，`main()` 方法启动线程后就直接向下执行，不过启动线程还需要做一些内核调用的处理，最后才会由 C 区域的方法回调 Java 中的 `run()` 方法，此时 `main` 线程可能已经输出了内容。

为了进一步验证，大家在 `main()` 方法和 `run()` 方法内部分别输出当前线程 ID 或 NAME，即可发现执行的线程是完全不同的，如：`Thread.currentThread().getName()`。

此代码验证了两个结果：

◎ 通过 `Thread` 的 `start()` 方法启动了另一个线程来处理任务。

◎ 线程的 `run()` 方法调用并不是线程在调用 `start()` 方法时被同步调用的，而是需要一个很短暂的延迟。

线程到底是什么东西？它与进程有何区别呢？

通常将线程理解为轻量级进程，它和进程有个非常大的区别是，多个线程是共享一个进程资源的，对于 OS 的许多资源的分配和管理（例如内存）通常是进程级别的，线程只是 OS 调度的最小单位，线程相对进程更加轻量一些，它的上下文信息会更少，它的创建与销毁会更加简单，线程因为某种原因挂起后不会导致整个进程被挂起，一个进程中又可以分配许多的线程，所以线程是许多应用系统中大家所喜欢的东西。

但是并非多线程就没有问题，它有个很大的问题就是，由于某个线程占用过多的资源会导致整个进程“宕”机，由于资源共享，所以线程之间会相互影响，但是多进程通常不会有这个问题（它们共享服务器资源，相互影响的级别在服务器资源上，而不是在进程内部）。

选择多线程还是多进程要根据实际情况来定，类似于 Nginx 这类负载均衡软件就采用多进程模型，因为它的异步 I/O 对于高并发来讲，已经足以解决进程或线程资源不足的情况，而且比多线程模型处理得更好，因为它是 I/O 密集型的。但是应用程序如果是计算密集型的，或者涉及大量的业务逻辑处理，则并不适合这样做，换句话说，最终还得根据实际场景来定。

前文中提到，`new Thread()`操作并非完成了线程的创建，只有当调用 `start()`方法时才会真正在系统中存在一个线程。在 OS 处理线程上也有多种方式，至于线程是哪种方式，对于我们来讲并不是那么重要，我们只需要知道存在一个单独的线程可以被调度即可。

我们回想一下第 3 章提到的一些内容：当大量分配线程后，可能会报错“unable to create new native thread”，说明线程使用的是堆外的内存空间，也再次说明 Thread 本身所对应的实例仅仅是 JVM 内的一个普通 Java 对象，是一个线程操作的外壳，而不是真正的线程。

补充知识：通过 Thread 的实例对象调用 `start()`方法到底是怎么启动线程的？下面对其实现方式做一些简单的补充。

- ◎ 基于 Kernel Thread (KLT) 的映射来实现：KLT 是内核线程，内核线程由 OS 直接完成调度切换，它相对应用程序的线程来讲只是一个接口，外部程序会使用一种轻量级进程 (Light Weight Process, LWP) 来与 KLT 进行一对一的接口调用。也就是说，进程内部会尝试利用 OS 的内核线程去参与实际的调度，而自己使用 API 调用作为中间桥梁与自己的程序进行交互。
- ◎ 基于用户线程 (User Thread, UT) 的实现：这种方式是考虑是否可以没有中间这一层映射，自己的线程直接由 CPU 来调度，或许理论上效率会更高。不过这样实现时，用户进程所需要关注的抽象层次会更低一些，跳过 OS 更加接近 CPU，即自己要去做许多 OS 做的事情，自然的 OS 的调度算法、创建、销毁、上下文切换、挂起等都要自己来搞定（因为 CPU 只做计算）。这样做显然很麻烦，许多人曾经尝试过，后来放弃了。
- ◎ 混合实现方式：它的设计理念是希望保留 Kernel 线程原有架构，又想使用用户线程，轻量级进程依然与 Kernel 线程一一对应保持不变，唯一变化的就是轻量级进程不再与进程直接挂钩，而是与用户线程挂钩，用户线程并不一定必须与轻量级进程一一对应，而是多对多，就像在使用一个轻量级进程列表一样，这样增加了一层来解除轻量级进程与原进程之间的耦合，可能会使得调度更为灵活。

在以前的 JDK 版本中，尝试使用 UT 的方式来实现，但后来放弃了，采用了与 Kernel 线程对应的方式，至于一些细节，与具体的平台有很大的关系，JVM 会适当考虑具体平台的因素去实现，在 JVM 规范中也没规定过必须如何去实现，所以对于程序员来讲，只需要知道在 `new Thread()`，调用 `start()`方法后，理论上就有一个可以被 OS 调度的线程了。

►► 5.1.2 多线程

在上一节的代码中，自己创建了一个线程，main()方法本身也有一个线程，虽然有主次之分，但是已经是多线程了。

写多线程程序无非就是加线程数量，让多个线程可以并行地去做一些事情。大家可以根据代码清单 5-1 增加线程来模拟，本书就不再给出代码了。

大家在代码清单 5-1 的基础上，多创建几个 Thread 就得到多线程的结果了，例如可以让多个线程输出某些结果，通过输出会发现它们会交替输出，而不是一个线程输出结束后，下一个线程紧跟着再输出结果。

►► 5.1.3 线程状态

谈线程，就必然要谈状态，为何？

对线程的每个操作，都可能会使线程处于不同的工作机制下，在不同的工作机制下某些动作可能会对它产生不同的影响，而不同的工作机制就是用状态来标志的，所以我们一定要了解它的状态；否则在编写多线程程序时，就会出现奇怪的问题。在本小节中，胖哥会逐个描述线程中的状态，说明导致此线程状态可能的原因，以及在某种状态下可以做的事情。

我们不仅要关注线程本身的状态，而且要养成一种关注状态变化的习惯，甚至于在自己做多线程设计时尝试用一些状态控制某些东西。因为在多线程的知识体系中，关于状态的信息，远远不止线程本身的状态这样一些信息（当然它是最基础的），在后文中介绍的许多 Java 的并发模型中，都会存在各种各样的状态转换，如果没有养成习惯去抓住这个重点，我们将很难看懂代码。

要获取状态可以通过线程（Thread）的 getState() 来获取状态的值。例如，获取当前线程的状态就可以使用 Thread.currentThread().getState() 来获取。该方法返回的类型是一个枚举类型，是 Thread 内部的一个枚举，全称为“java.lang.Thread.State”，这个枚举中定义的类型列表就是 Java 语言这个级别对应的线程状态列表，包含了 NEW、RUNNABLE、BLOCKED、WAITING、TIMED_WAITING、TERMINATED 这些值。现在对照源码中的注释，以及胖哥自己的理解来说明它们的意思。

(1) NEW 状态

```
/**
 * Thread state for a thread which has not yet started.
 */
NEW,
```

意思是这个线程没有被 start() 启动，或者说还根本不是一个真正意义上的线程，从本质上讲这只是创建了一个 Java 外壳，还没有真正的线程来运行。

再一次提醒大家注意：调用了 start() 并不代表状态就立即改变，中间还有一些步骤，如果在这个启动的过程中有另一个线程来获取它的状态，其实是不确定的，要看那些中间步骤是否已经完成了。

(2) RUNNABLE 状态

```
/**
 * Thread state for a runnable thread. A thread in the runnable
 * state is executing in the Java virtual machine but it may
 * be waiting for other resources from the operating system
 * such as processor.
 */
RUNNABLE,
```

当处于 NEW 状态的线程发生 start() 结束后线程将变成 RUNNABLE 状态。程序正在运行中的线程就肯定处于 RUNNABLE 状态，上面提到用 Thread.currentThread().getState() 来获取当前线程的状态，只会得到“RUNNABLE”，而不会得到其他的值，因为要得到结果就必然处于运行中。所以，获取状态都是获取其他线程的状态，而不是自己的状态。

RUNNABLE 状态也可以理解为存活着正在尝试征用 CPU 的线程（有可能这个瞬间并没有占用 CPU，但是它可能正在发送指令等待系统调度）。由于在真正的系统中，并不是开启一个线程后，CPU 就只为这一个线程服务，它必须使用许多调度算法来达到某种平衡，不过这个时候线程依然处于 RUNNABLE 状态。

举个例子：当某个运行中的线程发生了 yield() 操作时，其实看到的线程状态也是 RUNNABLE，只是它有一个细节的内部变化，就是做一个简单的让步。既然谈到让步，我们就来简单说说什么叫作让步。

胖哥认为这是一种“高素质”的做法，它自己可能在做大量 CPU 计算，认为自己会在相对较长的时间内占用资源，如果调度算法存在问题，就会一直占用 CPU，所以在适当的时候做下让步，让别人也来使用下 CPU 资源。

在生活中就好比一群人排队到取款机上取款，有些人可能喜欢查了取、取了查、查了再取、取了再查，也许中间还有许多思考的过程（或许在计算），也许还有许多从包里拿出和放入的动作，或许再打个电话，再整理下衣服。这在这些人心中是正常的，因为他们认为现在是属于自己的私人空间，但却忽略了后面还有很多人在等待的因素，可能有人在等待太久以后就放弃了（就像放弃 CPU 调度一样）。而高素质的人会觉得自己占用的时间太长了，会“不好意思”，主动意识到耽误了别人太多的时间，自己会出来让别人先处理，等别人处理好以后自己再进去。

对应到代码中，比如某些任务可能会在一个比较集中的时间在后台启动，有可能反复执行，有可能执行时间相对较长。在资源有限的情况下，这样的系统有可能和其他的系统部署在一台服务器上，甚至于一个进程上，自然会相互抢占资源，在某些必要的情况下，可以使用该方式做出一点让步，让双方的资源得到平衡。

RUNNABLE 状态可以由其他的许多状态通过某些操作后进入该状态，处于 RUNNABLE 状态的线程本身也可以执行许多操作转换为其他的状态，比如执行 synchronized、sleep()、wait() 等操作。在接下来的状态介绍中，还会提到许多和 RUNNABLE 相关的状态转换关系。

不过，就 Java 本身层面的 RUNNABLE 状态来讲，并不代表它一定处于运行中的状态，例如在 BIO 中，线程正阻塞在网络等待时，看到的状态依然是 RUNNABLE 状态，而在底层线程已经被阻塞，这也是 Java 内一些状态不协调的问题所在。所以我们不仅仅要看状态本身，还得了解更多的计算机与 Java 之间的关系才能在面对问题时更加接近本质。

(3) BLOCKED 状态

```
/**
 * Thread state for a thread blocked waiting for a monitor lock.
 * A thread in the blocked state is waiting for a monitor lock
 * to enter a synchronized block/method or
 * reenter a synchronized block/method after calling
 * {@link Object#wait() Object.wait}.
 */
BLOCKED,
```

BLOCKED 称为阻塞状态，或者说线程已经被挂起，它“睡着”了，原因通常是它在等待一个“锁”，当某个 `synchronized` 正好有线程在使用时，一个线程尝试进入这个临界区，就会被阻塞，直到另一个线程走完临界区或发生了相应锁对象的 `wait()` 操作后，它才有机会去争夺进入临界区的权利。

细节补充：`synchronized` 会有各种粒度的问题，这里的临界区是指多个线程尝试进入同一块资源区域，这个区域在 Java 代码中的体现方式通常是基于某个对象锁代码片段。关于它的一些细节，将在后文中详细介绍。

争取到锁的权利后才会从 BLOCKED 状态恢复到 RUNNABLE 状态，如果在征用锁的过程中没有抢到，那么它就又要回到休息室去等待了。

在实际的工作中，BLOCKED 状态也并非显式地存在于 `synchronized` 上，可能会是一种嵌套隐藏的方式，例如使用了某种三方控件、集合类。

一旦线程处于阻塞状态，线程就像真的什么也不做一样，在 Java 层面始终无法唤醒它。许多人说现在用 `interrupt()` 方法来唤醒它，小伙伴们可以进行小测试，一点用处都没有，因为 `interrupt()` 只是在里面做一个标记而已，不会真正唤醒处于阻塞状态的线程。

所以，在程序中出现 `synchronized` 时通常会考虑它的粒度问题，更要考虑它是否可能会被死锁的问题。

(4) WAITING 状态

```
/**
 * Thread state for a waiting thread.
 * A thread is in the waiting state due to calling one of the
 * following methods:
 * <ul>
 * <li>{@link Object#wait() Object.wait} with no timeout</li>
 * <li>{@link #join() Thread.join} with no timeout</li>
 * <li>{@link LockSupport#park() LockSupport.park}</li>
 * </ul>
 *
 * <p>A thread in the waiting state is waiting for another thread to
 * perform a particular action.
 *
 * For example, a thread that has called <tt>Object.wait()</tt>
 * on an object is waiting for another thread to call
 * <tt>Object.notify()</tt> or <tt>Object.notifyAll()</tt> on
 * that object. A thread that has called <tt>Thread.join()</tt>
 * is waiting for a specified thread to terminate.
 */
WAITING,
```

这种状态通常是指一个线程拥有对象锁后进入到相应的代码区域后，调用相应的“锁对象”的 `wait()` 方法操作后产生的一种结果。变相的实现还有 `LockSupport.park()`、

`LockSupport.parkNanos()`、`LockSupport.parkUntil()`、`Thread.join()`等，它们也是在等待另一个对象事件的发生，也就是描述了等待的意思。

上面提到的 BLOCKEN 状态也是等待的意思，它们有什么关系与区别呢？

其实 BLOCKEN 是虚拟机认为程序还不能进入某个区域，因为同时进去就会有问题，这是一块临界区。发生 `wait()` 操作的先决条件是要进入临界区，也就是线程已经拿到了“门票”，自己可能进去做了一些事情，但此时通过判定某些业务上的参数（由具体业务决定），发现还有一些其他配合的资源没有准备充分，那么自己就等等再做其他的事情。

理解起来是不是很麻烦？其实有一个非常典型的案例就是通过 `wait()` 和 `notify()` 来完成生产者消费者模型，当生产者生产过快，发现仓库满了，即消费者还没有把东西拿走（空位资源还没准备好）时，生产者就等待有空位再做事情，消费者拿走东西时会发出“有空位了”的消息，那么生产者就又开始工作了。反过来也是一样，当消费者消费过快发现没有存货时，消费者也会等存货到来，生产者生产出内容后发出“有存货了”的消息，消费者就又来抢东西了。

这种通过制衡方式的协调工作机制，在工作中用得很多，它稍加变化就能产生巨大的价值，现代的 Java 语言很牛，已经将这些复杂的细节包装成了对象，对外提供了很好用的 API，这些 API 为我们提供的仅仅是简单的任务内容输入，具体的调度细节由 Java 来完成。

在这种状态下，如果发生了对该线程的 `interrupt()` 是有用的，处于该状态的线程内部会抛出一个 `InterruptedException` 异常，这个异常应当在 `run()` 方法里面捕获，使得 `run()` 方法正常地执行完成。当然在 `run()` 方法内部捕获异常后，还可以让线程继续运行，这完全是根据具体的应用场景来决定的。

在这种状态下，如果某线程对该锁对象做了 `notify()` 动作，那么将从等待池中唤醒一个线程重新恢复到 `RUNNABLE` 状态。除 `notify()` 方法外，还有一个 `notifyAll()` 方法，前者是唤醒一个处于 `WAITING` 状态的线程，而后者是唤醒所有的线程。

`Object.wait()` 是否需要死等呢？不是，除中断外，它还有两个重构方法：

- ◎ `Object.wait(int timeout)`，传入的 `timeout` 参数是超时的毫秒值，超过这个值后会自动唤醒，继续做下面的操作（不会抛出 `InterruptedException` 异常，但是并不意味着我们不去捕获，因为不排除其他线程会对它做 `interrupt()` 动作）。
- ◎ `Object.wait(int timeout, int nanos)`，这是一个更精确的超时设置，理论上可以精确到纳秒，这个纳秒值可接受的范围是 0~999999（因为 1000000ns 等于 1ms）。

同样的，`LockSupport.park()`、`LockSupport.parkNanos()`、`LockSupport.parkUntil()`、`Thread.join()` 这些方法都会有类似的重构方法来设置超时，达到类似的目的，不过此时的状态不再是 `WAITING`，而是 `TIMED_WAITING`。

通常写代码的人肯定不想让程序死掉，但是又希望通过这些等待、通知的方式来实现某些平衡，这样就不得不去尝试采用“超时+重试+失败告知”等方式来达到目的。

(5) `TIMED_WAITING` 状态

```
/**
 * Thread state for a waiting thread with a specified waiting time.
 * A thread is in the timed waiting state due to calling one of
 * the following methods with a specified positive waiting time:
 * <ul>
 *   <li>{@link #sleep Thread.sleep}</li>
 *   <li>{@link Object#wait(long) Object.wait} with timeout</li>
```



```

* <li>{@link #join(long) Thread.join} with timeout</li>
* <li>{@link LockSupport#parkNanos LockSupport.parkNanos}</li>
* <li>{@link LockSupport#parkUntil LockSupport.parkUntil}</li>
* </ul>
*/
TIMED_WAITING,

```

相信使用过线程的小伙伴们都应该使用过 `Thread.sleep()`，前文中已经提到了通过其他的方式也可以进入这种 `TIME_WAITING` 状态。或许可以这样理解：当调用 `Thread.sleep()` 方法时，相当于使用某个时间资源作为锁对象，进而达到等待的目的，当时间达到时触发线程回到工作状态。

(6) TERMINATED 状态

线程结束了就处于这种状态，换句话说，`run()`方法走完了，线程就处于这种状态。其实这只是 Java 语言级别的一种状态，在操作系统内部可能已经注销了相应的线程，或者将它复用给其他需要使用线程的请求，而在 Java 语言级别只是通过 Java 代码看到的线程状态而已。

下面再来探讨一些问题。

为什么 `wait()`和 `notify()`必须要使用 `synchronized`？

如果不用就会报错 `IllegalMonitorStateException`，常见的写法如下：

```

synchronized(object) {
    object.wait(); //object.notify();
}
synchronized(this) {
    this.wait();
}
synchronized fun() {
    this.wait(); //this.notify();
}

```

首先要明确，`wait()`和 `notify()`的实现基础是基于对象存在的。那为什么要基于对象存在呢？

解释：既然要等，就要考虑等什么，这里等待的就是一个对象发出的信号，所以要基于对象而存在。

不用对象也可以实现，比如 `suspend()/resume()`就不需要，但是它们是反面教材，表面上简单，但是处处都是问题，在 5.1.4 节中会介绍。

理解基于对象的这个道理后，目前认为它调用的方式只能是 `Object.wait()`方法，这样才能和对象挂钩。但这些东西还与问题“`wait()/notify()`为什么必须要使用 `synchronized`”没有半点关系，或者说与对象扯上关系，为什么非要用锁呢？

我们还得继续探讨：既然是基于对象的，因此它不得不用一个数据结构来存放这些等待的线程，而且这个数据结构应当是与该对象绑定的（通过查看 C++代码，发现该数据结构为一个双向链表），此时在这个对象上可能同时有多个线程调用 `wait()/notify()`方法。

在向这个对象所对应的双向链表中写入、删除数据时，依然存在并发的问題，理论上也需要一个锁来控制。在 JVM 内核源码中并没有发现任何自己用锁来控制写入的动作，只是通过检查当前线程是否为对象的 `OWNER` 来判定是否要抛出相应的异常。由此可见它希望该动作由 Java 程序这个抽象层次来控制，它为什么不想去自己控制锁呢？

因为有些时候更低抽象层次的锁未必是好事，因为这样的请求对于外部可能是反复循环地去征用，或者这些代码还可能在其他地方复用，也许将它粗粒度化会更好一些，而且这样的代码写在 Java 程序中本身也会更加清晰，更加容易看到相互之间的关系。

在这个问题上，胖哥的解释就到此结束了，其中包含了许多个人的理解，有兴趣的朋友，可以去查阅资料细化这个问题的根源。

`interrupt()`操作在线程处于 `BLOCKEN` 状态时没用，在其他状态下都有效吗？

`interrupt()`操作对线程处于 `RUNNING` 状态时也没用，或者说只对处于 `WAITING` 和 `TIME_WAITING` 状态的线程有用，让它们产生实质性的异常抛出。

在通常情况下，如果线程处于运行中状态，也不会让它中断，如果中断是成立的，则可能会导致正常的业务运行出现问题。另外，如果不想用强制手段，就得为每条代码的运行设立检查，但是这个动作很麻烦，JVM 不愿意做这件事情，它做 `interrupt()`仅仅是打一个标记，此时程序中通过 `isInterrupt()`方法能够判定是否被发起过中断操作，如果被中断了，那么如何处理程序就是设计上的事情了。

举个例子，如果代码运行是一个死循环，那么在循环中可以这样做：

```
while(true) {
    if(Thread.currentThread().isInterrupt()) {
        //可以做类似的 break、return，抛出 InterruptedException 达到某种目的，这完全由自己决定
        //如抛出异常，通常包装一层 try catch 异常处理，进一步做处理，如退出 run 方法或什么也不做
    }
}
```

许多小伙伴认为这太麻烦了，为什么不可以自动呢？

小伙伴们可以通过一些生活的沟通方式来理解一下：当你发现门外面有人呼叫你时，你自己是否搭理他是你的事情，胖哥认为这是一种有“爱”的沟通方式，反之是暴力地破门而入，把你强制“抓”出去的方式。

在 JDK 1.6 及以后的版本中，可以使用线程的 `interrupted()`方法来判定线程是否已经被调用过中断方法，表面上的效果与 `isInterrupted()`方法的结果一样，不过这个方法是一个静态方法，直接通过 `Thread.interrupted()`调用判定的就是当前线程。除此之外，更大的区别在于这个方法调用后将会重新将中断状态设置为 `false`，这样方便于循环利用线程，而不是中断后状态就始终为 `true`，就无法将状态修改回来了。类似的，判定线程的相关方法还有 `isAlive()`、`isDaemon()`，分别用来判定线程是否还活着，以及是否为后台线程。

➤➤ 5.1.4 反面教材 `suspend()`、`resume()`、`stop()`

虽然是反面教材，但是胖哥认为反面教材往往体现在自己写代码时容易犯错的地方。只有看清楚这些反面教材，自己写代码时才会去多考虑一些细节性的问题。

`suspend()`、`resume()`、`stop()`这些 API 虽然 Java 一直保留着，但在代码中使用时会发现 JVM 已经不推荐使用了，它们都被加上了 `@Deprecated` 注解，表示它们已经过时了，保留只是为了兼容而已。

关于 `suspend()`/`resume()`这两个方法类似于 `wait()`/`notify()`，但是它们不是等待和唤醒线程。通过对它们的实验会发现，`suspend()`后的线程处于 `RUNNING` 状态，而不是 `WAITING` 状态，但是线程本身在这里已经挂起了，线程本身的状态就开始对不上号了。

如果是在 `synchronized` 区域内部发生 `suspend()`操作，那么它并不会像发生 `wait()` 那样把锁释放出来，因为它自己还在运行中。而当发生 `resume()`时，程序正常结束了，其实如果代码正常走过 `synchronized` 区域，锁也会释放的。但是很多资料上讲解的是没有释放资源，这是怎么回事呢？下面我们就写个反面教材的例子。

代码清单 5-2 反面例子

```
public class SuspendAndResume {
```



```
private final static Object object = new Object();

static class ThreadA extends Thread {

    public void run() {
        synchronized(object) {
            System.out.println("start...");
            Thread.currentThread().suspend();
            System.out.println("thread end...");
        }
    }

    public static void main(String []args) throws InterruptedException {
        ThreadA t1 = new ThreadA();
        ThreadA t2 = new ThreadA();
        t1.start();
        t2.start();
        Thread.sleep(100);
        System.out.println(t1.getState());
        System.out.println(t2.getState());
        t1.resume();
        t2.resume();
    }
}
```

输出结果如下：

```
start...
RUNNABLE
BLOCKED
thread end...
start...
```

代码中启动了两个子线程，这两个子线程几乎是同时启动的，main 方法所在的线程延迟 100ms，目的是为了两个子线程都进入运行的区域，至少其中一个发生了 suspend() 操作。

输出时，首先会输出一个“start...”，刚开始也只会输出一个“start...”，因为这是由 synchronized 来保证的，此时第一个进入 synchronized 区域的线程调用了 suspend() 方法，此时它停止执行了。

然后输出的两个状态是在 main 方法中打印出来的（因为一个线程在 synchronized 区域外部等待，另一个线程调用了 suspend() 方法而被挂起），这里输出的状态一个是 BLOCKED 状态，另一个是 RUNNABLE（多次测试后结果相同），说明有一个线程被阻塞了，阻塞线程自然在 synchronized 区域外面等待进入，而一个线程肯定是已经进入 synchronized 区域的线程，并在调用 suspend() 方法时挂起，但是我们看到的状态是 RUNNABLE。

如果去掉 synchronized 动作，将会输出两个 RUNNABLE，但是两个线程都在 suspend() 方法时停止执行了，这说明什么呢？suspend()/resume() 并不需要 synchronized 的支持，因此不需要基于对象。

接下来输出“thread end...”，说明有一个线程正常结束了，也说明 resume() 操作确实生效了，在它输出后，紧接着会输出一个“start...”，说明另一个线程进入了 synchronized 区域，但是神奇的事情发生了，另一个线程也被主线程调用过 resume() 方法，但实际情况是这个线程在这里卡住了，没有释放掉，为何？

因为在这个例子中，main 方法所在的线程对第 2 个进入 synchronized 区域的线程做的 resume() 操作很可能发生在它未进入 synchronized 区域之前，也自然发生在它调用 suspend()

操作之前，在线程没有调用 `suspend()` 方法之前调用 `resume()` 是无效的，也不会使得线程在其后面调用 `suspend()` 方法直接被唤醒。当该线程被挂起时，相应持有的锁就释放不掉了（因为它的操作与锁无关），而外部认为已经将这个线程释放掉了，因为外部看到的状态是 `RUNNING`，而且已经调用过 `resume()` 方法了，由于这些信息的不一致就导致了各种资源无法释放的问题。

总的来说，问题应当出在线程状态对外看到的是 `RUNNING` 状态，外部程序并不知道这个线程挂起了需要去做 `resume()` 操作（如果有状态判定还可以做检测）。另外，它并不是基于对象来完成这个动作的，因此 `suspend()` 和 `wait()` 相关的顺序性很难保证。所以 `suspend()/resume()` 不推荐使用了。

反过来想，这也更加说明了 `wait()` 和 `notify()` 为什么要基于对象来做数据结构，因为它要控制生产者和消费者之间的关系，它需要一个临界区来控制它们之间的平衡。它不是随意地在线程上做操作来控制资源的，而是由资源反过来控制线程状态的。当然 `wait()/notify()` 并非不会导致死锁，只是它们的死锁通常是程序设计不当导致的，并且在通常情况下是可以通过优化解决的。

关于 `stop()`，胖哥认为它和 `interrupt()` 最大的区别如下：

`interrupt()` 是相对友爱的行为，它不是破门而入，而 `stop()` 却是这样的，当你发起对某个线程的 `stop()` 操作时，如果这个线程处于 `RUNNING` 状态，`stop()` 将会导致这个线程直接抛出一个 `java.lang.ThreadDeath` 的 `Error`。这似乎没有问题，那么我们就来探讨一下是否会有问题。

假如线程是一个死循环，被外部容器所复用，在业务代码中会通过多个步骤的计算将某些值赋予线程内的某些属性或更大作用域的属性，这些属性可能是多个，当发起 `stop()` 时程序可能会进入 `try {} catch(Throwable e)` 区域，但是前面执行的计算和赋值只做了一半，而且做到那里没法找回来，这样就可能会导致业务程序中上下文数据不一致的情况发生。

➤➤ 5.1.5 调度优先级

线程的优先级就是对优先权的 `level` 设置，就像 VIP 专区，为何要设立 VIP 呢？因为资源有限才会存在特权，给予更多所以享有特权。

计算机也是这样的，CPU 资源是有限的，那么在某些情况下，我们希望先保证某些 VIP 先被执行。任务没有高低贵贱之分，但是有重要性、紧急性之分，因此会设立线程的优先级，让 OS 根据不同的优先级进行调度，这样在算法策略上就不再是一视同仁“吃大锅饭”了，可以使得调度更加灵活，达到局部优化的目的。

线程调度的优先级，每个 OS 有着不同的实现，而 Java 虚拟机为了兼容各种 OS 平台设定了 1~10 个优先级（理论上数字越大，优先级越高），但这并不代表每个 OS 也有 10 个优先级，某些 OS 可能只有 3 个或 5 个优先级。因此，JVM 会在相应的平台上根据实际情况设定 1~10 这 10 个数字与 OS 的线程优先级做一个映射关系，总体保持顺序化。通过这一点大家应该清楚，Java 中连续的两个数字所表示的优先级在实际场景中可能是同一个优先级。

作为程序员使用优先级时，又不想脱离 Java 语言本身的限制，通常将优先级设置为“普通”、“最大”、“最小”（如图 5-1 所示，其定义在 `Thread` 类中），通常不会设置一些细节的数字，那样设置可能根本达不到目的。


```
/**
 * The minimum priority that a thread can have.
 */
public final static int MIN_PRIORITY = 1;

/**
 * The default priority that is assigned to a thread.
 */
public final static int NORM_PRIORITY = 5;

/**
 * The maximum priority that a thread can have.
 */
public final static int MAX_PRIORITY = 10;
```

图 5-1 线程优先级代码截图

创建一个线程时，默认的优先级是 `Thread.NORM_PRIORITY`，值为 5。在程序中可以为指定线程设定优先级，通过 `setPriority(int)` 方法来完成，调用这个方法时传入上面描述的几种值，就基本可以达到调度优先的目的。

在 JVM 中还有一种特殊的后台线程，通过对线程调用 `setDaemon(boolean)` 标志是否为后台线程，它通常优先级极低，也就是通常不会跟别人抢 CPU，但是它可能在某些时候提升自己的优先级来做一些事情。例如 JVM 的 GC 线程就是后台线程，它很多时候不去和业务争用 CPU，而是在资源忙时会被提升优先级来做事情。

这类线程貌似与普通线程没有区别，因为普通线程也可以做到这一点。但是后台线程有一个十分重要的特征是，如果 JVM 进程中活着的线程只剩下后台线程，那么意味着就要结束整个进程。

大家可以做一个实验来证明这个结论。在一个线程中做死循环，`main` 方法启动这个线程后就结束了，此时整个进程不会退出。如果将线程设置为后台线程（`setDaemon(boolean)`），当 `main` 方法结束后，进程会立即结束。本书光盘中的 `src/chapter05/base/ThreadDaemonTest.java` 是一个简单的测试例子，大家只需要将代码中的 `setDaemon(true)` 操作注释掉或启用就会得到不同的结果。

►► 5.1.6 线程合并（Join）

许多同学刚开始学 Java 多线程时可能不会关注 `Join` 这个动作，因为不知道它是用来做什么的，而当需要用到类似的场景时却有可能会说 Java 没有提供这种功能。为此，胖哥就先说它的一些应用场景，再说怎么用吧。

当我们将一个大任务划分为多个小任务，多个小任务由多个线程去完成时，显然它们完成的先后顺序不可能完全一致。在程序中希望各个线程执行完成后，将它们的计算结果最终合并在一起，换句话说，要等待多个线程将子任务执行完成后，才能进行合并结果的操作。

这时就可以选择使用 `Join` 了，`Join` 可以帮助我们轻松地搞定这个问题，否则就需要用一个循环去不断判定每个线程的状态。

在实际生活中，就像把任务分解给多个人去完成其中的各个板块，但老板需要等待这些人全部都完成后才认为这个阶段的任务结束了，也许每个人的板块内部和别人还有相互的接口依赖，如果对方接口没有写好，自己的这部分也不算完全完成，就会发生类似于合并的动作（到底要将任务细化到什么粒度，完全看实际场景和自己对问题的理解）。下面用一段简单的代码来说明 `Join` 的使用。

代码清单 5-3 Join 的例子

```

public class ThreadJoinTest {

    static class Computer extends Thread {
        private int start;
        private int end;
        private int result;
        private int []array;

        public Computer(int []array , int start , int end) {
            this.array = array;
            this.start = start;
            this.end = end;
        }

        public void run() {
            for(int i = start; i < end ; i++) {
                result += array[i];
                if(result < 0) result &= Integer.MAX_VALUE;
            }
        }

        public int getResult() {
            return result;
        }
    }

    private final static int COUNTER = 10000000;

    public static void main(String []args) throws InterruptedException {
        int []array = new int[COUNTER];
        Random random = new Random();
        for(int i = 0 ; i < COUNTER ; i++) {
            array[i] = Math.abs(random.nextInt());
        }
        long start = System.currentTimeMillis();
        Computer c1 = new Computer(array , 0 , COUNTER);
        Computer c2 = new Computer(array , COUNTER / 2 , COUNTER);
        c1.start();
        c2.start();
        c1.join();
        c2.join();
        System.out.println(System.currentTimeMillis() - start);
        //System.out.println(c1.getResult());
        System.out.println((c1.getResult() + c2.getResult())
        & Integer.MAX_VALUE);
    }
}

```

这个例子或许不太好，只是 1000 万个随机数叠加，为了防止 CPU 计算过快，在计算中增加一些判定操作，最后再将计算完的两个值输出，也输出运算时间。如果在有多个 CPU 的机器上做测试，就会发现数据量大时，多个线程计算具有优势，但是这个优势非常小，而且在数据量较小的情况下，单线程会更快一些。为何单线程可能会更快呢？

最主要的原因是线程在分配时就有开销（每个线程的分配过程本身就需要执行很多条底层代码，这些代码的执行相当于很多条 CPU 叠加运算的指令），Join 操作过程还有其他的各种开销。

如果尝试将每个线程叠加后做一些其他的操作，例如 I/O 读写、字符串处理等操作，多线程的优势一下子就出来了，因为这样总体计算下来后，线程的创建时间是可以被忽略

的，所以我们在考量系统的综合性能时不能就一个点或某种测试就轻易得出一个最终结论，一定要考虑更多的变动因素。

要模拟单线程做许多相对时间较长的操作，也不一定非要用文件读写、字符串处理等操作，这样设计测试比较麻烦，由于已经知道了关键点在于运行时间与线程创建时间的比重，所以可以让每个线程循环时休眠一个随机的毫秒值，这个时间其实不需要太长，例如 10ms、20ms、30ms 就可以模拟出效果了。

但这并不代表多线程就一定能提升效率，首先要检测 CPU 是不是多核，如果不是，那么使用多线程带来更多的是上下文切换的开销，多线程操作的共享对象还会有锁瓶颈，否则就是非线程安全的。

综合考量各种开销因素、时间、空间，最后利用大量的场景测试来证明推理是有指导性的，如果只是一味地为了用多线程而使用多线程，则往往很多事情可能会适得其反。

Join 只是语法层面的线程合并，其实它更像是当前线程处于 BLOCKEN 状态时去等待其他线程结束的事件，而且是逐个去 Join。换句话说，Join 的顺序并不一定是线程真正结束的顺序，要保证线程结束的顺序性，它还无法实现，即使在本例中它也不是唯一的实现方式，本章后面会提到许多基于并发编程工具的方式来实现会更加理想，管理也会更加体系化，能适应更多的业务场景需求。

►► 5.1.7 线程补充小知识

本小节的内容是一些小例子，简单地讲解线程栈的获取，以及 UncaughtExceptionHandler 的简单使用，大家只需要对照本书光盘中的例子来运行，以及本书的相应讲解，就会清楚这些小例子的用途和意义。

(1) 线程栈的获取

在前文中多次提到过栈，尤其在第 3 章中介绍 BTrace 时，通过 BTraceUtils 的 jstack() 方法就可以输出调用栈信息。由此我们知道了代码切入是怎么回事，但是线程栈如何获取呢？其实很简单，请看下面的例子。

代码清单 5-4 获取线程栈的简单例子

```
public class ThreadStackTest {

    public static void main(String []args) {
        printStack(getStackByThread());
        printStack(getStackByException());
    }

    private static void printStack(StackTraceElement []stacks) {
        for(StackTraceElement stack : stacks) {
            System.out.println(stack);
        }
        System.out.println("\n");
    }

    private static StackTraceElement[] getStackByThread() {
        return Thread.currentThread().getStackTrace();
    }

    private static StackTraceElement[] getStackByException() {
```

```

        return new Exception().getStackTrace();
    }
}

```

这样就通过两种方式输出线程栈了！

这么简单？

不信，我们就看看输出结果：

```

java.lang.Thread.getStackTrace(Thread.java:1568)
chapter05.base.ThreadStackTest.getStackByThread(ThreadStackTest.java:23)
chapter05.base.ThreadStackTest.main(ThreadStackTest.java:11)

chapter05.base.ThreadStackTest.getStackByException(ThreadStackTest.java:27)
chapter05.base.ThreadStackTest.main(ThreadStackTest.java:12)

```

这和异常信息很像，只是没有异常类型而已。没错，在例子中大家也应当看到有通过异常来获取线程栈的方式。对于该例子，大家可以方法套用方法，进行多层套用后看看输出结果会是什么样子的。

获取到的这个线程栈是一个数组，数组的顺序就是调用代码的来源路径，数组中的每个元素是一个 `java.lang.StackTraceElement` 类型的对象，它内部包含了相应的 class、方法、文件名、行号信息，我们可以通过这些信息来追踪代码、监控、定位异常、控制调用来源等。

对于调用来源的类，可以通过 `sun.reflect.Reflection` 的 `getCallerClass(int)` 来获取，在 JDK 1.7 以后 API 有少量变化。

(2) UncaughtExceptionHandler 的简单使用

这是 Java 本身提供的一种对 `run()` 方法没有捕获到的异常、错误的一次补救，在这里可以吃点后悔药。通常我们不依赖这种方式，因为这是线程级别的，业务代码中通常不会关心这个层次，即使要关心也是在框架当中，通常我们希望在内层就将该异常处理掉，走到这个位置也意味着线程已经脱离了 `run()` 方法，会立即结束，不能再被线程所复用了。

不过，从学习 Java 的角度来讲，也需要知道 Java 确实提供了这样一种机制，请看下面的例子。

代码清单 5-5 UncaughtExceptionHandler 的测试

```

class TestExceptionHandler implements UncaughtExceptionHandler {
    @Override
    public void uncaughtException(Thread t, Throwable e) {
        System.out.printf("线程出现异常：");
        e.printStackTrace();
    }
}

public class ExceptionHandlerTest {

    public static void main(String []args) {
        Thread t = new Thread() {
            public void run() {
                Integer.parseInt("ABC");
            }
        };
        t.setUncaughtExceptionHandler(new TestExceptionHandler());
        t.start();
    }
}

```


代码中模拟了一个数字转换的异常抛出，在 `run()` 方法中并没有捕获此异常，最终会进入自定义的 `TestExceptionHandler` 中来处理（也可以直接 `throw new Error()` 抛出，得到的结果也是类似的）。

5.2 线程安全

接下来的内容都将基于多核处理器，因为单核处理器不会出现将要谈到的可见性问题，不过并不代表单核 CPU 上多个线程就没有一致性问题，因为 CPU 有时间片原则，还会有其他的一些问题，例如重排序。

本节的内容比较偏重于理论化，看过的同学应该知道这部分内容并不容易看懂。不过要学习并发编程就必然会涉及这方面的内容，所以我们一定要努力去学习，胖哥会尽量通过自己的理解，用一些相对简单的方式让大家得到一些感性认识，进一步的深入就得靠大家自己多去参看相关的资料了。

5.2.1 并发内存模型概述

前文中提到，为了提升性能 CPU 会 Cache 许多数据。但在多核 CPU 下，每个 CPU 都有自己独立的 Cache，每个 CPU 都可以修改同一个单元数据，因此它们会有自己的一套“缓存一致性协议”，这个一致性是从 CPU 的角度认为某些来自主存的数据的读取和修改需要一定的一致性保障。

在 JVM 运行时，每个线程的私有栈在使用共享数据时，会先将共享数据拷贝到栈顶进行运算，这份数据其实是一个副本，因此也同样存在多个线程修改同一个内存单元的一致性问题。JVM 自己完成了一套内存模型（Java Memory Model, JMM）的规范，这套模型将基于不同的平台去做特定的优化，JMM 的基本管理策略是满足于一种通用的规范，所以这件事情是比较麻烦的。这种模型在 JDK 1.2 就有了，直到 JDK 1.5 (JSR-133) 及以后的版本中，JVM 的内存管理才开始逐步成熟起来。

要了解并发的問題，就要知道并发的問題到底是什么，下面给出一段简单的代码来做个测试，以便对并发的問題有一个初步的了解。

代码清单 5-5 初识线程不安全的代码

```
public class NoVisiabilityTest {  
  
    private static class ReadThread extends Thread {  
  
        private boolean ready;  
  
        private int number;  
  
        public void run() {  
            while(!ready) {  
                number++;  
            }  
            System.out.println(ready);  
        }  
  
        public void readyOn() {  
            this.ready = true;  
        }  
    }  
}
```

```

    public static void main(String []args) throws InterruptedException {
        ReadThread readThread = new ReadThread();
        readThread.start();
        Thread.sleep(200);
        readThread.readyOn();
        System.out.println(readThread.ready);
    }
}

```

这段代码请确保运行在 Server 模式下，运行的结果可能会与我们所预想的不一样，不过某些 JDK 本身不支持 Server 模式，则达不到本例子所想要模拟的结果。

由于服务器端程序绝大部分都是在 Server 模式下，所以本例也并非故意制造变态环境，因此编写服务器端程序的同学应当关注这样的问题。

按照常理来讲，这段代码会在运行一段时间后自动退出，因为 readyOn()方法会将对象的 ready 参数设置为 true，那么 while(!ready)会失败，理论上就会跳出循环，结束子线程的运行。但是在 Server 模式下运行时，这段代码会走入死循环，主线程输出了结果，而子线程一直都不结束。

在同一个对象内部，一个线程将该对象的属性修改后，另一个线程看不到该属性被修改后的结果，或者说未必能马上看到结果，这就是并发模型中的“可见性”问题。因为普通变量的修改并不需要立即写回到主存，而线程读取时也不需要每一次都从主存中去读取，因此就可能产生这样的现象。此时若在循环中增加一个 Thread.yield();操作就可以让线程让步 CPU，进而很可能到主存中读到最新的 ready 值，达到退出循环的目的。也可以在循环中加一条 System.out 语句，或者将 ready 变量增加一个 volatile 修饰符也可以达到退出的目的。

我们研究这段代码的目的不仅仅是为了解决一个死循环的问题，而且是要告诉大家代码运行过程中确实存在可见性问题。或许前面提到的几种方法可以达到目的，但我们可能并不是太清楚是怎么解决的，或者说在什么情况下用这种方式来解决。

为了解决这个问题，早期做法都是加悲观锁，随着计算机需求的发展，大牛们对技术内在做了许多研究，他们发现这种方式的开销很大，我们希望有更细粒度的控制方式来提升性能。

从宏观上我们将 JVM 内存划分为堆和栈两个部分，堆内存就是 Java 对象所使用的区域，在 JMM 的定义中通常把这块空间叫作“Main Memory”（主存）。栈空间内部应当包含局部变量、操作数栈、当前方法的常量池指针、当前方法的返回地址等信息，这块空间在我们看来是最接近 CPU 运算的，也是每个线程私有的空间，当调用某方法开始时将该私有栈分配空间，在方法内部再调用方法还会继续使用相应的栈空间，方法返回时回收相应的栈空间（不论是否抛出异常）。这块空间通常叫作“Working Memory”（工作内存）。

如果栈内部所包含的“局部变量”是引用（Reference），则仅仅是引用值在栈中，而且会占用一个引用本身的大小，具体的对象还是在堆当中的，即对象本身的大小与栈空间的使用无关。

工作内存与主存之间会采用 read/write 的方式进行通信，而当工作内存中的数据需要计算时，它会发生 load/store 操作，load 操作通常是将本地变量推至栈顶，用来给 CPU 调度运算，而 store 就是将栈顶的数据写入本地变量。

通过 `javap` 命令输出指令列表，从中可以看到各种 `XXload` 指令（`0x15~0x35`），就是将不同类型的信息推至栈顶，指令除了可以标识数据类型外，还可以标识宽度；相应的 `XXstore` 指令（`0x36~0x56`）就是将不同类型的栈顶数据存入本地变量。

局部变量（本地变量）在使用时通常不存在一致性问题，因为它的定义本身就归属于线程运行时，生命周期由相应的代码块决定。所以在一致性问题，我们关注的是多线程对主存的一些数据读写操作。

关于线程换个角度来理解，如果将 JVM 当成一个操作系统，则可以将多线程本身理解为多个 CPU（理论上多个线程可以并行一起运行），站在抽象层次上看，多核处理缓存一致性就和这些思想有许多相通之处了。

Java 想要自己来实现一套一致性协议，需要有一些基础规则，下面列举几个。

（1）JMM 中一些普通变量的操作指令

◎ Load 操作发生在 `read` 之后（两个之间可以有其他的指令）。

◎ 普通变量的修改未必会立即发生 Store 操作，但发生 Store 操作，就会发生 `write` 操作。

有了这个基本规则后，我们似乎就不太关注 `write/read` 了，因为 Load 发生之前肯定有 `read`，而 Store 操作之后肯定有 `write`，因此我们将读/写的 4 个步骤理解为简单的 2 个步骤。

这个 JMM 的基本规约似乎与多核处理器上的缓存一致性还没有太大关系，因为在代码运行过程中完全有可能一个线程读、一个线程写，此时就可能出现许多奇怪的现象，比如程序中读到了老的数据，或者很久甚至永远都读不到最新的数据。把这类问题叫作“可见性”问题。

在现实中，我们通常会遇到如下版本管理问题。

例子 1：你认为自己从 SVN 上拿到了最新代码去修改，结果你在拿代码时，别人提交了最新版本，在现实中或许没有多大关系，因为版本管理最终可以一致，但是在计算机中或许不行，因为会导致许多意想不到的后果（例如，可能会导致算错数据）。

例子 2：拿到了最新版本的代码，一直在修改，一直没有从 SVN 更新最新的代码，如果在这个过程中别人提交了代码，代码就不是最新的了。

在现实中想要解决这个问题，就需要大家睁大眼睛随时看着 SVN 这个公共的区域是否发生改变，包括对每一个文件的修改，或者大家将代码就直接写在 SVN 上，在同一台服务器上调试代码，显然这十分影响工作效率，而且也不现实。计算机也是这样，在绝大部分情况下，各自的变量没有冲突，那么就无须关注对方是否和自己操作同一个内存单元，这样一来计算机的性能就会很高，只是在某些特殊场景下，我们不得不有选择性地使用另一种方式来保障一致性。

这个时候喜欢思考的小伙伴们有了美妙的联想，希望每次使用某个文件前都自动从 SVN 上提取最新的，如果有人修改了文件，则自动立即上传，上传中如果有人正在下载，则必须等待上传者完成。当然这只是一种假设，在计算机中或许能达到这个粒度，我们就有机会进一步做事情了。

这种最细的粒度支持，也就是对 Load、Store 的各种顺序控制，`load`、`store` 两两组合为 4 种情况：`LoadLoad`、`StoreStore`、`LoadStore`、`StoreLoad`，它们以一种指令屏障的方式来控制顺序，有些系统可能不支持某些指令的顺序化，不过绝大部分系统都支持 `StoreLoad`。

(2) StoreLoad 的意思

我们可以简单地认为让 Store 先于 Load 发生。例如两个在某个瞬间同时修改和读取主存中的一个共享变量，此时的读取操作将发生在修改之后。有了这样一种特征，就实现了最细粒度的锁，也是最轻量级的锁（在这里所提到的轻量的概念与 JVM 本身对悲观锁优化中所引导出的轻量级锁的概念不是同一个）。

不过，这样的方式仅仅能保证读的一瞬间确保线程读取到最新的数据，因此要进一步做到读取、修改、写入的动作是一致的，就将其升级为原子性。要达到原子性的效果，可以通过可见性、CAS 自旋来完成，也可以通过 synchronized 来完成。

5.2.2 一些并发问题描述

前文中的一个简单例子提到了可见性问题，这个问题比较好模拟，但是还有一些非常不好模拟的并发问题，而且总是存在于一些细节上，本节我们就来描述一些并发问题。

(1) 指令重排序

指令重排序可能是 Java 编译器在编译 Java 代码时对虚指令进行重排序，也可以是 CPU 对目标指令进行重排序，它们的目的当然都是为了高效（换句话说，我们自己写的代码对于计算机的解析和运行来讲顺序未必是最高效的）。

重排序在提升性能的同时，也给我们带来了许多麻烦，而这些麻烦通常带来的问题十分诡异。为了说明这样的问题，请大家先看图 5-2。

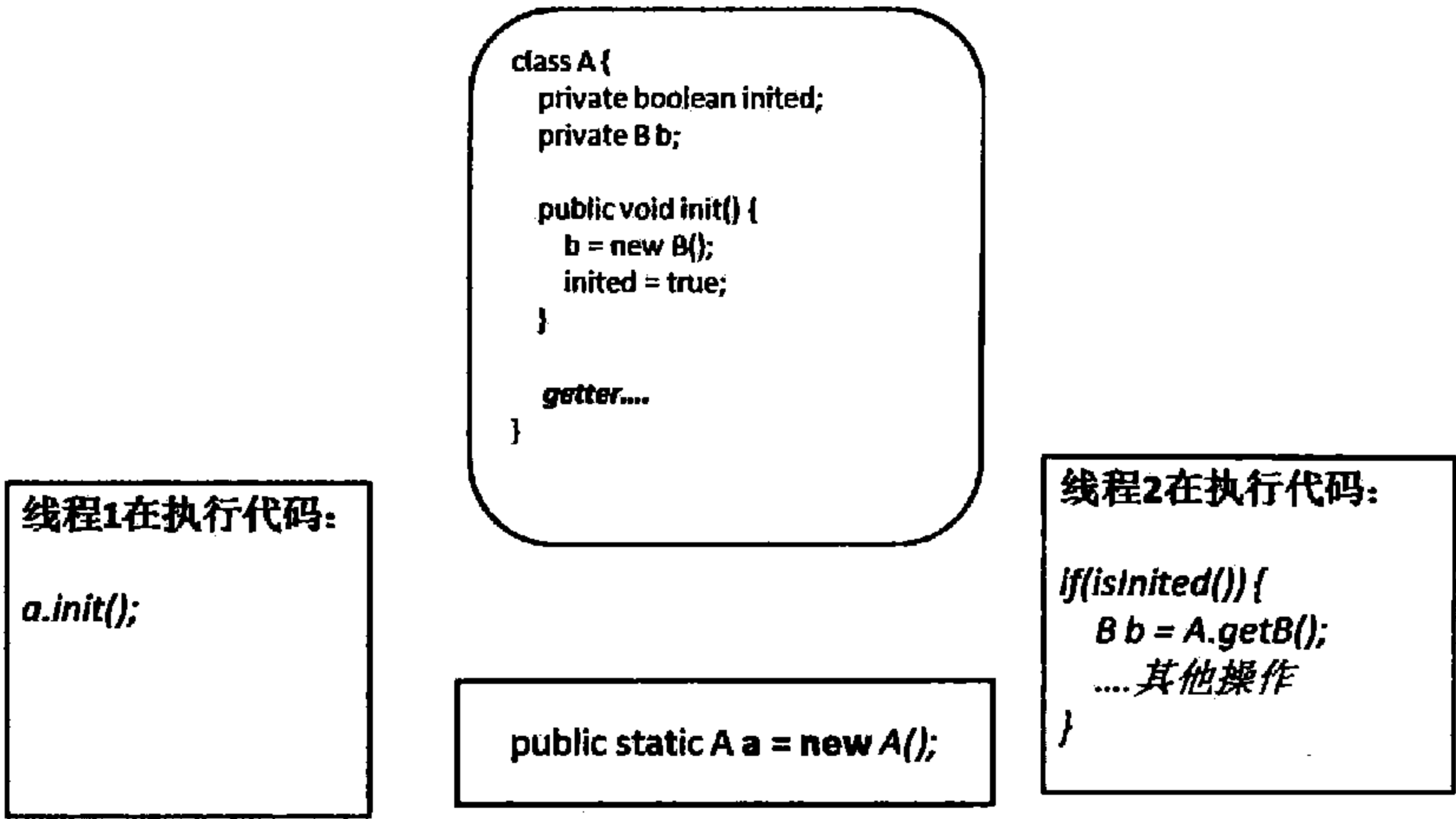


图 5-2 两个线程操作共享变量

根据图 5-2 描述的代码及调用关系，按照常理线程 1 执行 init()方法将先创建一个 B 对象赋值给属性 b，然后再将初始化参数 initd 设置为 true（先抛开可见性问题），其他的线程会先通过 isInit()方法判定对象内的属性是否已经被初始化好，然后再操作对象本身，这样对象的操作应当是安全的。

如果这里发生指令重排序，就未必安全了！假如 initd=true 先于其他属性被赋值前被执行，那么此时线程 2 通过 isInit()方法得到返回值为 true，但有可能对象内部的某些属性根本还没有初始化完成，从而导致许多不可预见的问题。



注意 (1) 重排序并不意味着程序中的所有代码是杂乱无序的，只是重排序那些没有相互依赖的代码，便于某些优化，我们可以认为在单线程中执行代码结果永远不会变 (as-if-serial)。

(2) 用 Java 写代码时本身不注重性能，但是并不意味着不注重较低的抽象层次，因为较低的抽象层次的优化可能影响到 Java 中大量频繁的简单操作的性能，这种时间的提升虽然很小，但有可能对 Java 整体的运行性能是有影响的。

(2) 4 字节赋值问题

在 JVM 中允许对一个非 volatile 的 64 位 (8 字节) 变量赋值时，分解为两个 32 位 (4 字节) 来完成，但并不是必须要一次性完成 (从 Java 角度来理解，在虚指令中对变量的操作都以 slot 为单位，每个 slot 就是 4 字节)。

问题出来了，如果变量是 long、double 类型的数据，在赋值某个 32 位后，正好被另一个线程所读取，那么它读取出来的数据就可能是不可预见的结果。

(3) 数据失效

正常的在 JavaBean 中提供大量的 set、get 是没有问题的，如果这样的对象提供给多线程使用就不一定了，用前面提到的可见性的道理来讲，当一个线程调用 set 后，其他的线程未必能看得到。

这似乎没什么大不了的，那么胖哥就说个有点小影响的例子。

假如某个赋值是一个应用平台的系统配置参数，它在内存中有一份拷贝，这份配置参数将会影响程序对某些金额的计算方法或业务流程，这时偶然发生了另一个线程看不到的情况，导致虽然有最新的配置，但是这个线程还在走老路，那么结果自然是错误的，完全由可能带来经济损失。

(4) 非安全的发布

为了说明问题，请先看图 5-3。

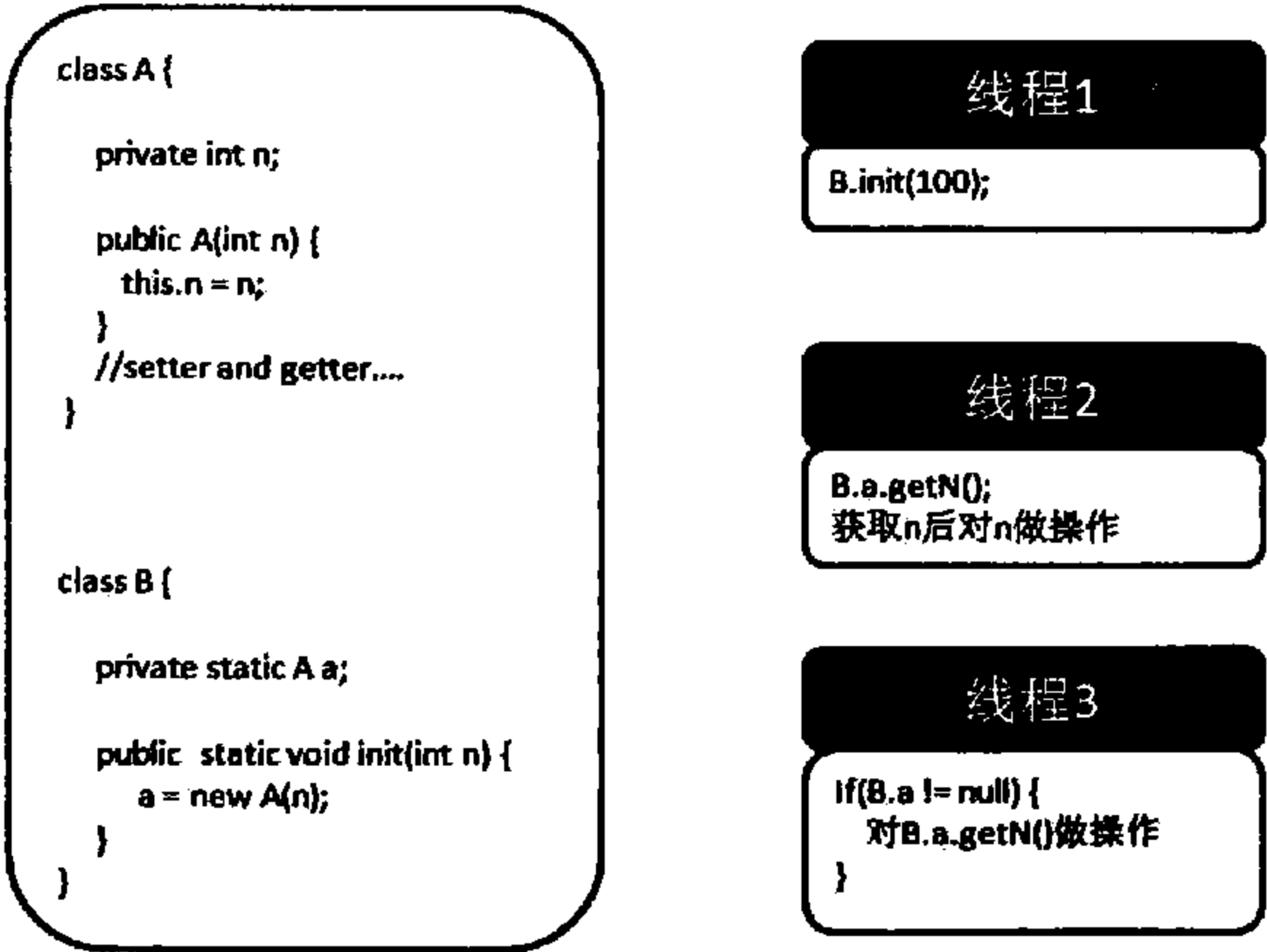


图 5-3 对象发布逃逸

这里的引用 `a` 可以直接被外部使用, 它希望有一个线程将其初始化后再让外部看到, 但在未初始化好 `a` 以前, 线程 2 可能对 `a` 直接进行使用了, 那么自然是空指针。更加可怕的是另一种现象, 就是此时 `a` 引用指向的是一个还未初始化完成的 `A` 对象, 这个对象空间可能被创建了, 但是它的内部属性的初始化还需要一个过程 (例子中只有 1 个简单属性, 在实际的程序中可能会有很多复杂属性), 这是由于在 JMM 中并没有规定普通属性的赋值必须发生在构造方法的 `return` 语句之前。换句话说, `A` 对象的构造方法内的属性赋值可以在 `return` 发生之后, 而当 `A` 对象的构造方法发生 `return` 后, `B` 类中的静态属性 `a` 就可以被赋值了, 这样就会导致线程 3 在判定 `B` 类中的属性 `a` 不为空的情况下, 使用这个对象来做操作, 而这个对象依然有可能没有被初始化好。这种问题算是发布的一种“逃逸”问题, 换句话说, 就是程序访问了不该访问的内容, 访问了可能还没准备好的内容。

并发问题的例子远不止这些, 下面再做一点点补充。

例如, 在内存中用数组或集合类来 `Cache` 一些数据, 提供给应用服务器的多个线程访问使用, 那么我们应当如何提供 API 给它们使用呢?

假如 API 返回整个集合类或数组, 那么集合类或数组中的元素将有可能被多线程并发修改, 这种修改自然就可能会存在并发的问題; 如果返回集合类或数组中其中一个下标的元素理论上没有问题, 但集合类或数组中所存放的元素也是对象, 对象中包含了许多可变的属性, 则该对象依然可能会被多线程并发地修改。

因此, 为了避免并发问题, 我们需要考虑一些场景, 例如可以使用一份数据拷贝返回, 或者返回不允许修改的代理 API (`Collections.unmodifiable` 相关 API)。返回的数据拷贝, 可以是一个集合类或数组的一个元素或者整个集合类或数组, 而拷贝的深度需要根据业务来控制, 对于复杂的包装对象, 要保证绝对的线程安全性是十分麻烦的。如果要返回不允许修改的数据结构的代理操作类, 则可以使用 `Collections` 提供的 `unmodifiable` 相关的静态方法, 例如 `Collections.unmodifiableList(List)` 将返回不可修改的 `List` 对象, 同样的, 该 API 限制仅仅局限于 `List` 本身不能被修改, 若 `List` 内部的元素是数组、集合类、包含许多非 `final` 属性的对象, 则依然可以在获取元素后修改相应的内容。

下面再举个通过子类“逃逸”的例子。

从图 5-3 中可以看到一种通过“逃逸”的例子。很多时候场景千变万化, 例如在子类中提供了自定义的构造方法, 在构造方法中将 `this` 引用交给另一个线程所访问或作为任务的属性提交给线程池, 它们可以通过 `this` 访问这个对象的一些方法或属性, 而此时对象可能还没有初始化完成相应的属性。

描述了这么多问题, 或许我们不会这样写代码, 或许有的牛人会说“根本就不该这样写代码”, 不过反过来讲, 其实我们是在就问题而探讨问题, 或许真正的程序会隐藏得更深, 但道理基本是一致的。换句话说, 我们要理解内在机制, 当某一天遇到类似的问题时, 可以有能力去分析和解决。

接下来, 我们就介绍一些 Java 面对各种并发问题提供的语法支持, 或者称作 JVM 层面上的功能支持。

►► 5.2.3 volatile

变量被 `volatile` 关键字修饰后, 貌似就会避开前面提到的部分问题。在前面我们用一个 SVN 上传/下载的理想例子来说明, 要做到完美就得牺牲一些性能, 虽然 `volatile` 它被誉

为“最轻量级的锁”，但它依旧比普通变量的访问慢一些。之所以说它轻，是因为它只是在读这个瞬间要求一个简单的顺序，而不是一个变量上的原子读写，或者在一段代码上的同步。

在 JDK 1.5 以前，volatile 变量并没有完全实现轻量级的锁，不过 JDK 1.5 对可见性做了更为严格的定义，因此我们可以开始放心使用。不过也因此 volatile 修饰的变量的性能可能会有所下降。JDK 也同样对 synchronized 做了各种轻量级、偏向的优化，简单来讲是 Java 的作者们开始意识到一些 Java 锁的场景，认为在很多场景下锁的开销是不需要完全使用悲观锁的，因此做了很多改造和优化，不过从理论上讲，其开销应该不会比 volatile 小（因为它最少会保证一个原子变量的读写一致性，而 volatile 不需要）。另外，synchronized 通常是基于代码段的，开销变小仅仅是加锁的过程开销，而并非锁所包含的代码区域也会加快运行速度，相关的代码区域依然被串行访问，因此不要期望于系统的锁优化为我们解决所有的问题，很多优化原则依然需要根据我们对于锁粒度本身的理解，以及结合相关的业务背景来综合分析，才能得出答案。

volatile 变量也会像普通变量那样从主存中拷贝到各个线程中去操作，区别在于它要求实现 StoreLoad 指令屏障（当然 JVM 也未必一定要用这种指令来实现，前文中提到的 4 种指令屏障只要对应的处理器支持，也可以采用具体的平台来优化），由此我们可以简单地认为 volatile 的第一个作用就是：保证多线程中的共享变量是始终可见的（但这并不保证 volatile 引用对象内部的属性是完全可见的）。

在 JSR-133 中对 volatile 语义的定义进行了增强（主要是为了真正实现更简单的锁），要求在对 volatile 变量进行读/写操作时，其前后的指令在某些情况下不允许进行重排序（不论是编译时重排序还是处理器重排序，都不允许）。对于这种限制分为如下几种情况。

（1）如果是一条对 volatile 变量进行赋值操作的代码，那么在该代码前面的任何代码不能与这个赋值操作交换顺序。在图 5-2 中，一个 initd 变量如果使用 volatile 修饰，那么就能够达到目的，因为它能确保前面属性的赋值在 initd=true 发生之前。

我们需要注意两点：

◎ 如果这个操作后有普通变量的读写操作，则是可以与它交换顺序的。

◎ 在这个动作之前的指令相互之间还是可以重排序的，只是不能排序到该动作后面。

它就像一个向前的隔板，隔板前面的多个动作依然可以被重排序，隔板一边的普通变量可以进入另一边，以便于做更多的优化。

（2）如果是一条读取 volatile 变量的代码，则正好相反，相当于隔板翻了一个面，在它后面的操作动作不允许与它交换顺序，之后的多个动作依然可以重排序，在它之前的普通变量的操作动作也可以与它交换顺序。

（3）普通变量的读写操作相互之间是可以重排序的，只要不影响它们之间的逻辑语义顺序就可以重排序。但是如果普通变量的读/写操作遇上了 volatile 变量的操作，就需要遵循前两个基本原则。

（4）如果两个 volatile 变量的读/写操作都在一段代码中，则依然遵循前两个基本原则，此时无论两者之间读/写顺序如何，都肯定不会重排序。

这里反复强调的顺序，大家可能会有所疑惑，由于它的重要性，也同时帮助大家理解，下面举几个真实代码的例子。

- ◎ 一个变量被赋值后，经过某些读写操作，再赋值给另一个变量，这个顺序是会不会被打乱的。
- ◎ 程序中 try、catch、finally 代码肯定不会交换顺序来执行，比如：try 部分的代码还没执行完，也没有抛出异常，就执行 finally 的代码，这种情况是不会发生的，否则就全部乱套了。我们可以用前面提到的“始终保持与单线程中的最终计算结果是相同的”这句话来理解这个逻辑顺序问题。

综上所述，volatile 修饰变量的第 2 个作用是：防止相关性代码的重排序，从指令级别达到了轻量级锁的目的。除此之外，volatile 还有一个重要的作用是解决前面提到的 4 字节赋值问题，对于 volatile 修饰的变量，必须一次性赋值。

volatile 内在到底是怎么回事？或许我们可以输出某些平台上的汇编指令来看看它到底与普通变量的操作有何特殊性。

输出汇编指令？这在 Java 领域很少听到，不过确实可以做到，只是有点麻烦。首先要做的事情就是下载插件，由于与平台相关，所以插件也与平台相关。打开地址 <https://kenai.com/projects/base-hsdis/downloads>，可以下载到很多种操作的插件，Windows 32bit 版本在 <http://hllvm.group.iteye.com/> 上可以找到。

将插件下载好以后，放在哪里？在 Linux JVM 中放在 \$JAVA_HOME/jre/lib/amd64/server 及 client 两个目录下，并记得执行 chmod +x 相应的命令；在 Windows JVM 中放在 %JAVA_HOME%/jre/bin/server 及 client 两个目录中，若只存在一个目录，则只放一个。

接下来，我们需要编写一段简单的程序，用于检测不同变量的赋值。

代码清单 5-6 测试输出汇编指令

```
public class AssemTest {
    int a, b;
    volatile int c, d;

    public static void main(String[] args) throws Exception {
        new AssemTest().test();
    }

    public void test() {
        a = 1;
        b = 2;
        c = a;
        d = b;
        b = 3;
        c = 2;
    }
}
```

该代码通过 javac 编译为 Class 文件，下面使用 java 命令携带相应的参数来输出汇编指令。

```
java -XX:+UnlockDiagnosticVMOptions -XX:+PrintAssembly -Xcomp -XX:CompileCommand=compileonly,*AssemTest.test AssemTest
```

在 Linux 系统上输出结果如图 5-4 所示。

从图 5-4 所示的汇编指令中可以看出，汇编指令中出现了 lock addl \$0x0 操作，它是通过某个较低抽象层次的锁实现了相应的屏障。


```
$ javac AssemTest.java
$ java -XX:+UnlockDiagnosticVMOptions -XX:+PrintAssembly -Xcomp -XX:CompileCommand=compileonly,*Assem
test.test AssemTest
Java HotSpot(TM) 64-Bit Server VM warning: PrintAssembly is enabled. turning on DebugManSafePoints to gain additional output
CompilerOptions: compileonly *AssemTest.test
Loaded disassembler from hdis-and64.so
Decoding compiled method 0x00002aaaab104f90:
Code:
[Disassembling for arch='i386:x86-64']
[Entry Point]
[Constants]
# [method] 'test' (JVM) in 'AssemTest'
# [sp=0x20] [sp of caller]
0x00002aaaab105000: mov     0x8(%rsi),%r10d
0x00002aaaab105004: cmp     %r10,%rax
0x00002aaaab105007: jne     0x00002aaaab0dc820 ; (runtime_call)
0x00002aaaab10500d: xchg    %rax,%rax
[Verified Entry Point]
0x00002aaaab105009: push    %rbp
0x00002aaaab10500f: sub     $0x10,%rsp
0x00002aaaab1050f6: nop
                                ;synchronization entry
                                ; - AssemTest::test@1 (line 16)
0x00002aaaab1050f6: movl    $0x1,0xc(%rsi) ;putfield a
                                ; - AssemTest::test@2 (line 16)
0x00002aaaab1050fd: movl    $0x2,0x10(%rsi)
0x00002aaaab105104: movl    $0x1,0x14(%rsi) ;putfield c
                                ; - AssemTest::test@15 (line 16)
0x00002aaaab10510b: mov     0x10(%rsi),%r11d
0x00002aaaab10510f: mov     %r11d,0x18(%rsi)
0x00002aaaab105113: movl    $0x3,0x10(%rsi)
0x00002aaaab10511a: movl    $0x2,0x14(%rsi)
0x00002aaaab105121: lock add $0x0,(%rsp) ;putfield d
                                ; - AssemTest::test@23 (line 19)
0x00002aaaab105120: add     $0x10,%rsp
0x00002aaaab10512a: pop     %rbp
0x00002aaaab10512b: test    %eax,-0xc0131(%rsp) ; 0x00002aaaad37000
                                ; (poll_return)
0x00002aaaab105131: retq
0x00002aaaab105132: hlt
0x00002aaaab105133: hlt
0x00002aaaab105134: hlt
0x00002aaaab105135: hlt
0x00002aaaab105136: hlt
0x00002aaaab105137: hlt
0x00002aaaab105138: hlt
0x00002aaaab105139: hlt
0x00002aaaab10513a: hlt
0x00002aaaab10513b: hlt
0x00002aaaab10513c: hlt
0x00002aaaab10513d: hlt
0x00002aaaab10513e: hlt
0x00002aaaab10513f: hlt
[ExceptionHandler]
[Stub Code]
0x00002aaaab105140: jmpq    0x00002aaaab102160 ; (no_reloc)
[Deopt Handler Code]
0x00002aaaab105145: callq   0x00002aaaab10514a
0x00002aaaab10514a: subq    $0x5,(%rsp)
0x00002aaaab10514f: jmpq    0x00002aaaab0dd7c0 ; (runtime_call)
0x00002aaaab105154: add     %al,(%rax)
0x00002aaaab105156: add     %al,(%rax)
javaTest1$
```

图 5-4 输出 Java 在对应平台上的汇编指令

>> 5.2.4 final

在 JMM 中要求 final 域（属性）的初始化动作必须在构造方法 return 之前完成。换言之，一个对象创建以及将其赋值给一个引用是两个动作，对象创建还需要经历分配空间和属性初始化的过程，普通的属性初始化允许发生在构造方法 return 之后（指令重排序）。

似乎这个问题变得很可怕，因为在 Java 程序中得到的对象竟然有可能还没有执行完构造方法内的属性赋值，但在大部分情况下，对象的使用都是在线程内部定义的，在单线程中是绝对可靠的，或者说在单线程中要求使用对象引用时，该对象已经被初始化好。但如果在此过程中有另一个线程通过这个未初始化好的对象引用读取相应的属性，那么就可能读取到的并不是真正想要的值。在 Java 中 final 可以保证这一点，所以它可以避免这种类型的逃逸问题。

但是它并不能完全解决所有的逃逸问题，而只是确保在构造方法 return 以前是会被初始化的，无法确保不与其他指令进行重排序，比如下面的代码：

```
private static TestObject testObject = null;
final int a;
public 构造方法() {
    a = 100;
    testObject = this; //这个地方可能和 a=100 发生指令重排序
}
public static void read() {
    if(testObject != null) {
        //对变量 testObject.a 做操作
    }
}
```

如果有另一个线程调用静态方法 `read()`，则可能得到 `testObject` 非空值，而此时有可能 `a=100` 这个动作还未执行（因为它可以与 `testObject = this` 进行重排序），那么操作的数据就将是错误的。

进一步探讨：如果 `final` 所修饰的不是普通变量，而是数组、对象，那么它能保证自己本身的初始化在其外部对象的构造方法返回之前，但是它本身作为对象，对内部的属性是无法保证的。如果是某些具有标志性的属性，则需要根据实际情况做进一步处理，才可以达到线程安全的目的。

经过 JSR-133 对 `final` 进行语义增强后，我们就可以比较放心地使用 `final` 语法了。但是我们想看看构造方法还没做完，变量是什么样子呢？普通变量和 `final` 变量到底又有什么区别呢？下面我们就写一段和并发编程没多大关系的代码来跑一跑看看。

代码清单 5-7 构造方法未结束，看看属性是什么样子

```
public class FinalConstructorTest {

    static abstract class A {

        public A() {
            display();
        }

        public abstract void display();
    }

    static class B extends A {

        private int INT = 100;

        private final int FINAL_INT = 100;

        private final Integer FINAL_INTEGER = 100;

        private String STR1 = "abc";

        private final String FINAL_STR1 = "abc";

        private final String FINAL_STR2 = new String("abc");

        private final List<String> FINAL_LIST = new ArrayList<String>();

        public B() {
            super();
            System.out.println("abc");
        }

        public void display() {
            System.out.println(INT);
            System.out.println(FINAL_INT);
            System.out.println(FINAL_INTEGER);
            System.out.println(STR1);
            System.out.println(FINAL_STR1);
            System.out.println(FINAL_STR2);
            System.out.println(FINAL_LIST);
        }
    }

    public static void main(String []args) {
```



```

        new B();
    }
}

```

在这段代码中，我们跳开了构造方法返回之前对 `final` 的初始化动作，而是在构造方法内部去输出这些 `final` 属性。这段代码的输出结果可能会让我们意想不到，大家可以自行测试，如果在测试过程中使用断点跟踪去查看这些数据的值，则可能在断点中看到的值与实际输出的值还会有所区别，因为断点也是通过另一个线程去看对象的属性值的，看到的对象可能正好是没有初始化好的对象。

这样看来，`volatile` 和 `final` 在程序中必不可少吗？当然不是！

如果每个属性都使用这样的修饰符，那么系统就没有必要设置这样的修饰符了！其实它们是有一定的性能开销的！我们关注的是代码是否真的有并发问题，如果数据本身就是某些只读数据，或者这些 Java 对象本身就是线程所私有的局部变量或类似于 `ThreadLocal` 的变量，那么就没有必要使用这些修饰符了。

提到 `final`，我们再补充一个相关的话题（该话题与并发编程无关）。当在方法中使用匿名内部类时，匿名内部类的方法要直接使用外部方法中的局部变量，这个局部变量必须用 `final` 来声明才可以被使用。很多人会问这到底是为什么？伪代码如下：

```

public void test() {
    final int a = 100; // 这个 a 必须定义为 final，才能被匿名子类直接使用
    new A() {
        public void display() {
            System.out.println(a);
        }
    }
    其他操作
}

```

这其实是对一个语法的疑问，本身没什么好解释的，但是如果非要解释，或许我们可以从这个角度来理解：JVM 本身也是一种软件平台，它设计了一种语法规则，自然也有它的局限性和初衷。在编译时，这个地方会自动生成一个匿名内部类，而本地变量 `a` 的作用域是在方法 `test()` 中，它如何能用到另一个内部类中呢？

其中一种方式是参数传递；另一种方式就是作为一个属性存在于匿名内部类对象中。不论哪一种方式都会在这个匿名内部类对象中有一份数据拷贝（如果本地变量是引用，那么拷贝将是引用的值），不过这似乎与外部定义本地变量时是否定义为 `final` 没有关系。

JVM 在设计时并不知道我们的代码会怎么写，或者说它不明确在所创建的匿名内部类中到底会做什么，例如在代码中完全可以在内部创建一个线程来使用这个变量，或者创建一个任务提交给线程池来使用这个变量。如果是这样，匿名内部类的运行将与该方法本身的运行处于两个线程中，当外部方法可能已经结束时，那么相应的局部变量的作用域已经结束，自动会被回收，要保证匿名内部类一直可以使用该变量，就只能用拷贝的方法，这似乎还是与 `final` 没有多大关系。

我们现在回过头来回答：为何外部方法必须使用 `final` 定义这个这个变量？我们将上述结论反过来想，如果这个属性不是 `final` 修饰的，在匿名内部类中使用“同名”的变量操作，并且可以对它做任意修改，自然外部也应当能感知到。但是事实上不能感知到，因为这是一份数据拷贝，就像传递的参数一样，不是原来的数据。胖哥认为这种语法的设计手段是为了避免误解，语法上强制约束它为 `final` 修饰。

如果这里的 `a` 是一个引用，那么就只会拷贝引用值，而不是整个对象的内容，在内部修改引用所指向的对象不会影响外部，外部的 `final` 也无法约束其改变，但如果改变其对象内部的属性，只要外部还会使用这个对象，那么就会受到影响。

5.2.5 栈封闭

栈封闭算是一种概念，也就是线程操作的数据都是私有的，不会与其他的线程共享数据。简单来说，如果每个线程所访问的 JVM 区域是隔离的，那么这个系统就像一个单线程系统一样简单了，我们把它叫作栈封闭。这样说是不是有点抽象，下面讲实际的例子。

通常在做 Web 开发时不需要自己去关注多线程的各种内在，因为容器给我们做好了，从前端请求就给业务处理分配好了数据，我们无须关注那些并发的过程，Web 容器会自动给我们提供私有的 `Request`、`Response` 对象的处理，因为它不会被其他的线程所占用，所以可以放心使用，它是线程绝对安全的（当使用 `Session` 或 `ServletContext` 时，它也在内部给你封装好了并发的控制）。

但这并不意味着程序永远不关注多线程与异步的问题，当需要并发去访问某些共享缓存数据时，当需要去操作共享文件数据时，当自定义多线程去并发做一些任务时，都必然会用到这些基本的知识体系来作为支撑，否则代码出现某些诡异的问题还不知道怎么回事。

比如在一个项目中，使用了 Spring 注入的 DAO 层，大家都应该知道 Spring 生成的对象默认是“单例”的，也就是一个类只会生成一个对应的实例。在这个 DAO 的许多方法中，使用 `StringBuilder` 进行 SQL 拼接，在 DAO 里面定义了 `StringBuilder`，认为这个变量可以提供给许多的 DAO 层的方法共同使用，以节约空间，每个相应的方法都是对它做一个或多个 `append` 操作后，通过 `toString()` 获取结果 `String` 对象，最后再把这个 `StringBuilder` 清空。

先抛开并发本身的问题，这样做也根本节约不了什么空间，因为这个对象将拥有与这个 DAO 一样永久的生命周期占用内存，由于 DAO 是“单例”的，所以相当于永久的生命周期。我们节约空间的方式通常是希望它短命，在 Young 空间就干掉它，而不是让它共享。

继续看有什么并发的問題。虽然这个 `StringBuilder` 不是 `static` 修饰的，但由于它所在的这个 DAO 对象实例是“单例”的，由 Spring 控制生成，所以它几乎等价于全局对象。它至少会在这个类里面的所有方法被访问时共享，就算这个类里面只有一个方法也会有并发问题（因为同一个方法是可以被多个线程同时访问的，为何？因为它是代码段，程序运行时只需要从这里获取到指令列表即可，或者反过来理解，如果所有的代码都不能并行访问，那么多线程程序就完全被串行化了）。

如果数据区域发生共享就有问题了，多个线程可能在同时改一个数据段，这样没有任何安全策略的数据段，最终结果会是什么样子，谁也不清楚。本例中提到的 `StringBuilder` 就是一个共享的数据区域，假如有两个线程在 `append()`，然后一个线程 `toString()` 得到的结果将有可能是两个线程共同写入的数据，它在操作完成后可能还会将数据清空，那么另一个线程就可能拿到一个空字符串甚至于更加诡异的结果。这样的程序很明显是有问题的。

如果改成 `StringBuffer`，是否可行？

答曰：`StringBuffer` 是同步的，但是并不代表它在业务上是绝对安全的，认为它安全是因为它在每一次做 `append()` 类似操作时都会加上 `synchronized` 的操作，但是在实际的程序中是可以对 `StringBuffer` 进行多次 `append()` 操作的，在这些 `append()` 操作之间可能还会有其他

的代码步骤，StringBuffer 可以保证每次 append()操作是线程安全的，但它无法保证多线程访问时进行多次 append()也能得到理想的结果。

难道我们还要在外层加一个锁来控制？

如果是这样的话，新的问题就出现了，这个类所有的方法访问到这里都是串行的，如果所有的 DAO 层都是这样的情况，抛开锁本身的开销，此时系统就像单线程系统一样在运行，外部的并发访问到来时，系统将奇慢无比。如果访问过大，就会堆积大量的线程阻塞，以及线程所持有的上下文无法释放，而且会越堆积越多，后果可想而知。

锁的开销是巨大的，它对于并发编程中的性能是十分重要的，于是许多大牛开始对无锁化的问题有了追求，或者说尽量靠近无锁化。在大多数情况下我们希望事情是乐观的，希望使用尽量细粒度化的锁机制，不过对于大量循环调用锁的情况会反过来使用粗粒度化的锁机制，因为加锁的开销本身也是巨大的。

关于栈封闭，除了使用局部变量外，还有一种方式就是使用 ThreadLocal，ThreadLocal 使用一种变通的方式来达到栈封闭的目的，具体的请参看下一小节的内容。

➤ 5.2.6 ThreadLocal

虽然 ThreadLocal 与并发问题相关，但是许多程序员仅仅将它作为一种用于“方便传参”的工具，胖哥认为这也许并不是 ThreadLocal 设计的目的，它本身是为线程安全和某些特定场景的问题而设计的。

ThreadLocal 是什么呢！

每个 ThreadLocal 可以放一个线程级别的变量，但是它本身可以被多个线程共享使用，而且又可以达到线程安全的目的，且绝对线程安全。

例如：

```
public final static ThreadLocal<String> RESOURCE = new ThreadLocal<String>();
```

RESOURCE 代表一个可以存放 String 类型的 ThreadLocal 对象，此时任何一个线程可以并发访问这个变量，对它进行写入、读取操作，都是线程安全的。比如一个线程通过 RESOURCE.set("aaaa");将数据写入 ThreadLocal 中，在任何一个地方，都可以通过 RESOURCE.get();将值获取出来。

但是它也并不完美，有许多缺陷，就像大家依赖于它来做参数传递一样，接下来我们就来分析它的一些不好的地方。

为什么有些时候会将 ThreadLocal 作为方便传递参数的方式呢？例如当许多方法相互调用时，最初的设计可能没有想太多，有多少个参数就传递多少个变量，那么整个参数传递的过程就是零散的。进一步思考：若 A 方法调用 B 方法传递了 8 个参数，B 方法接下来调用 C 方法->D 方法->E 方法->F 方法等只需要 5 个参数，此时在设计 API 时就涉及 5 个参数的入口，这些方法在业务发展的过程中被许多地方所复用。

某一天，我们发现 F 方法需要加一个参数，这个参数在 A 方法的入口参数中有，此时，如果要改中间方法牵涉面会很大，而且不知道修改后会不会有 Bug。作为程序员的我们可能会随性一想，ThreadLocal 反正是全局的，就放这里吧，确实好解决。

但是此时你会发现系统中这种方式有点像在贴补丁，越贴越多，我们必须要求调用相关的代码都使用 ThreadLocal 传递这个参数，有可能会搞得乱七八糟的。换句话说，并不是不让用，而是我们要明确它的入口和出口是可控的。

诡异的 ThreadLocal 最难琢磨的是“作用域”，尤其是在代码设计之初很乱的情况下，如果再增加许多 ThreadLocal，系统就会逐渐变成神龙见首不见尾的情况。有了这样一个省事的东西，可能许多小伙伴更加不在意设计，因为大家都认为这些问题都可以通过变化的手段来解决。胖哥认为这是一种恶性循环。

对于这类业务场景，应当提前有所准备，需要粗粒度化业务模型，即使要用 ThreadLocal，也不是加一个参数就加一个 ThreadLocal 变量。例如，我们可以设计几种对象来封装入口参数，在接口设计时入口参数都以对象为基础。

也许一个类无法表达所有的参数意思，而且那样容易导致强耦合。

通常我们按照业务模型分解为几大类型对象作为它们的参数包装，并且将按照对象属性共享情况进行抽象，在继承关系的每一个层次各自扩展相应的参数，或者说加参数就在对象中加，共享参数就在父类中定义，这样的参数就逐步规范化了。

我们回到正题，探讨一下 ThreadLocal 到底是用来做什么的？为此我们探讨下文中的几个话题。

(1) 应用场景及使用方式

为了说明 ThreadLocal 的应用场景，我们来看一个框架的例子。Spring 的事务管理器通过 AOP 切入业务代码，在进入业务代码前，会根据对应的事务管理器提取出相应的事务对象，假如事务管理器是 DataSourceTransactionManager，就会从 DataSource 中获取一个连接对象，通过一定的包装后将其保存在 ThreadLocal 中。并且 Spring 也将 DataSource 进行了包装，重写了其中的 getConnection() 方法，或者说该方法的返回将由 Spring 来控制，这样 Spring 就能让线程内多次获取到的 Connection 对象是同一个。

为什么要放在 ThreadLocal 里面呢？因为 Spring 在 AOP 后并不能向应用程序传递参数，应用程序的每个业务代码是事先定义好的，Spring 并不会要求在业务代码的入口参数中必须编写 Connection 的入口参数。此时 Spring 选择了 ThreadLocal，通过它保证连接对象始终在线程内部，任何时候都能拿到，此时 Spring 非常清楚什么时候回收这个连接，也就是非常清楚什么时候从 ThreadLocal 中删除这个元素（在 9.2 节中会详细讲解）。

从 Spring 事务管理器的设计上可以看出，Spring 利用 ThreadLocal 得到了一个很完美的设计思路，同时它在设计时也十分清楚 ThreadLocal 中元素应该在什么时候删除。由此，我们简单地认为 ThreadLocal 尽量使用在一个全局的设计上，而不是一种打补丁的间接方法。

了解了基本应用场景后，接下来看一个例子。定义一个类用于存放静态的 ThreadLocal 对象，通过多个线程并行地对 ThreadLocal 对象进行 set、get 操作，并将值进行打印，来看看每个线程自己设置进去的值和取出来的值是否是一样的。代码如下：

代码清单 5-8 简单的 ThreadLocal 例子

```
public class ThreadLocalTest {

    static class ResourceClass {

        public final static ThreadLocal<String> RESOURCE_1 =
            new ThreadLocal<String>();

        public final static ThreadLocal<String> RESOURCE_2 =
            new ThreadLocal<String>();

    }

}
```



```

static class A {

    public void setOne(String value) {
        ResourceClass.RESOURCE_1.set(value);
    }

    public void setTwo(String value) {
        ResourceClass.RESOURCE_2.set(value);
    }
}

static class B {
    public void display() {
        System.out.println(ResourceClass.RESOURCE_1.get()
            + ":" + ResourceClass.RESOURCE_2.get());
    }
}

public static void main(String []args) {
    final A a = new A();
    final B b = new B();
    for(int i = 0 ; i < 15 ; i ++) {
        final String resouce1 = "线程-" + i;
        final String resouce2 = " value = (" + i + ")";
        new Thread() {
            public void run() {
                try {
                    a.setOne(resouce1);
                    a.setTwo(resouce2);
                    b.display();
                }finally {
                    ResourceClass.RESOURCE_1.remove();
                    ResourceClass.RESOURCE_2.remove();
                }
            }
        }.start();
    }
}

```

关于这段代码，我们先说几点。

- ◎ 定义了两个 ThreadLocal 变量，最终的目的就是要看最后两个值是否能对应上，这样才有机会证明 ThreadLocal 所保存的数据可能是线程私有的。
- ◎ 使用两个内部类只是为了使测试简单，方便大家直观理解，大家也可以将这个例子的代码拆分到多个类中，得到的结果是相同的。
- ◎ 测试代码更像是为了方便传递参数，因为它确实传递参数很方便，但这仅仅是为了测试。
- ◎ 在 finally 里面有 remove()操作，是为了清空数据而使用的。为何要清空数据，在后文中会继续介绍细节。

测试结果如下：

```

线程-6: value = (6)
线程-9: value = (9)
线程-0: value = (0)
线程-10: value = (10)
线程-12: value = (12)
线程-14: value = (14)
线程-11: value = (11)
线程-3: value = (3)

```

```

线程-5: value = (5)
线程-13: value = (13)
线程-2: value = (2)
线程-4: value = (4)
线程-8: value = (8)
线程-7: value = (7)
线程-1: value = (1)

```

大家可以看到输出的线程顺序并非最初定义线程的顺序，理论上可以说明多线程应当是并发执行的，但是依然可以保持每个线程里面的值是对应的，说明这些值已经达到了线程私有的目的。

不是说共享变量无法做到线程私有吗？它又是如何做到线程私有的呢？这就需要我们知道一点点原理上的东西，否则用起来也没那么放心，请看下面的介绍。

(2) ThreadLocal 内在原理

从前面的操作可以发现，ThreadLocal 最常见的操作就是 set、get、remove 三个动作，下面来看看这三个动作到底做了什么事情。首先看 set 操作，源码片段如图 5-5 所示。

```

public void set(T value) {
    Thread t = Thread.currentThread();
    ThreadLocalMap map = getMap(t);
    if (map != null)
        map.set(this, value);
    else
        createMap(t, value);
}

```

图 5-5 ThreadLocal.set 源码片段

图 5-5 中的第一条代码取出了当前线程 t，然后调用 getMap(t) 方法时传入了当前线程，换句话说，该方法返回的 ThreadLocalMap 和当前线程有点关系，我们先记录下来。进一步判定如果这个 map 不为空，那么设置到 Map 中的 Key 就是 this，值就是外部传入的参数。这个 this 是什么呢？就是定义的 ThreadLocal 对象。

代码中有两条路径需要追踪，分别是 getMap(Thread) 和 createMap(Thread, T)。首先来看看 getMap(t) 操作，如图 5-6 所示。

```

ThreadLocalMap getMap(Thread t) {
    return t.threadLocals;
}

```

图 5-6 getMap(Thread) 操作

在这里，我们看到 ThreadLocalMap 其实就是线程里面的一个属性，它在 Thread 类中的定义是：

```
ThreadLocal.ThreadLocalMap threadLocals = null;
```

这种方法很容易让人混淆，因为这个 ThreadLocalMap 是 ThreadLocal 里面的内部类，放在了 Thread 类里面作为一个属性而存在，ThreadLocal 本身成为这个 Map 里面存放的 Key，用户输入的值是 Value。太乱了，理不清楚了，画个图来看看（见图 5-7）。

简单来讲，就是这个 Map 对象在 Thread 里面作为私有的变量而存在，所以是线程安全的。ThreadLocal 通过 Thread.currentThread() 获取当前的线程就能得到这个 Map 对象，同时将自身作为 Key 发起写入和读取，由于将自身作为 Key，所以一个 ThreadLocal 对象就能存放一个线程中对应的 Java 对象，通过 get 也自然能找到这个对象。

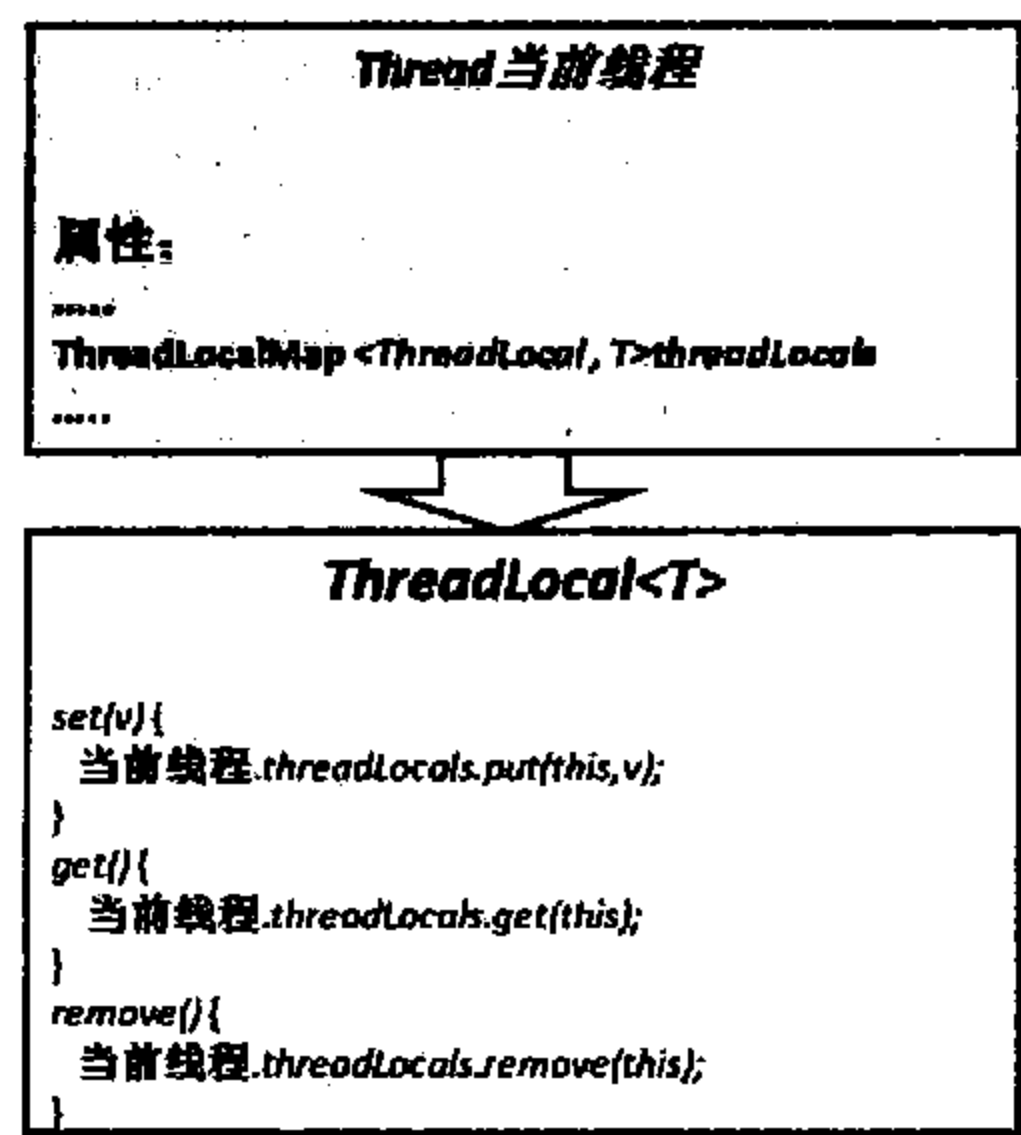


图 5-7 Thread 与 ThreadLocal 的伪代码关联关系

如果还没有理解，则可以将思维放宽一点。当定义变量 `String a` 时，这个“a”其实只是一个名称（在第 3 章中已经说到了常量池），虚拟机需要通过符号表来找到相应的信息，而这种方式正好就像一种 K-V 结构，底层的处理方式也确实很接近这样，这里的处理方式是显式地使用 Map 来存放数据，这也是一种实现手段的变通。

现在有了思路，继续回到上面的话题，为了验证前面的推断和理解，来看看 `createMap` 方法的细节，如图 5-8 所示。

```
void createMap(Thread t, T firstValue) {
    t.threadLocals = new ThreadLocalMap(this, firstValue);
}
```

图 5-8 createMap 操作

这段代码是执行一个创建新的 Map 的操作，并且将第一个值作为这个 Map 的初始化值，由于这个 Map 是线程私有的，不可能有另一个线程同时也在对它做 `put` 操作，因此这里的赋值和初始化是绝对线程安全的，也同时保证了每一个外部写入的值都将写入到 Map 对象中。

最后来看看 `get()`、`remove()` 代码，或许看到这里就可以认定我们的理论是正确的，如图 5-9 所示。

```
public T get() {
    Thread t = Thread.currentThread();
    ThreadLocalMap map = getMap(t);
    if (map != null) {
        ThreadLocalMap.Entry e = map.getEntry(this);
        if (e != null)
            return (T)e.value;
    }
    return setInitialValue();
}

public void remove() {
    ThreadLocalMap m = getMap(Thread.currentThread());
    if (m != null)
        m.remove(this);
}
```

图 5-9 get()/remove()方法的代码片段

给我们的感觉是，这样实现是一种技巧，而不是一种技术。
其实是技巧还是技术完全是从某种角度来看的，或者说是从某种抽象层次来看的，如

果这段代码在 C++ 中实现，难道就叫技术，不是技巧了吗？当然不是！胖哥认为技术依然是建立在思想和方法基础上的，只是看实现的抽象层次在什么级别。就像在本书中多个地方探讨的一些基础原理一样，我们探讨了它的思想，其实它的实现也是基于某种技巧和手段的，只是对程序封装后就变成了某种语法和 API，因此胖哥认为，一旦学会使用技巧思考问题，就学会了通过技巧去看待技术本身。我们应当通过这种设计，学会一种变通和发散的思维，学会理解各种各样的场景，这样便可以积累许多真正的财富，这些财富不是通过某些工具的使用或测试就可以获得的。

ThreadLocal 的这种设计很完美吗？

不是很完美，它依然有许多坑，在这里对它容易误导程序员当成传参工具就不再多提了，下面我们来看看它的使用不当会导致什么技术上的问题。

(3) ThreadLocal 的坑

通过上面的分析，我们可以认识到 ThreadLocal 其实是与线程绑定的一个变量，如此就会出现一个问题：如果没有将 ThreadLocal 内的变量删除 (remove) 或替换，它的生命周期将会与线程共存。因此，ThreadLocal 的一个很大的“坑”就是当使用不当时，导致使用者不知道它的作用域范围。

大家可能认为线程结束后 ThreadLocal 应该就回收了，如果线程真的注销了确实是这样的，但是事实有可能并非如此，例如在线程池中对线程管理都是采用线程复用的方法 (Web 容器通常也会采用线程池)，在线程池中线程很难结束甚至于永远不会结束，这将意味着线程持续的时间将不可预测，甚至与 JVM 的生命周期一致。那么相应的 ThreadLocal 变量的生命周期也将不可预测。

也许系统中定义少量几个 ThreadLocal 变量也无所谓，因为每次 set 数据时是用 ThreadLocal 本身作为 Key 的，相同的 Key 肯定会替换原来的数据，原来的数据就可以被释放了，理论上不会导致什么问题。但世事无绝对，如果 ThreadLocal 中直接或间接包装了集合类或复杂对象，每次在同一个 ThreadLocal 中取出对象后，再对内容做操作，那么内部的集合类和复杂对象所占用的空间可能会开始膨胀。

抛开代码本身的问题，举一个极端的例子。如果不想定义太多的 ThreadLocal 变量，就用一个 HashMap 来存放，这貌似没什么问题。由于 ThreadLocal 在程序的任何一个地方都可以用得到，在某些设计不当的代码中很难知道这个 HashMap 写入的源头，在代码中为了保险起见，通常会先检查这个 HashMap 是否存在，若不存在，则创建一个 HashMap 写进去；若存在，通常也不会替换掉，因为代码编写者通常会“害怕”因为这种替换会丢掉一些来自“其他地方写入 HashMap 的数据”，从而导致许多不可预见的问题。

在这样的情况下，HashMap 第一次放入 ThreadLocal 中也许就一直不会被释放，而这个 HashMap 中可能开始存放许多 Key-Value 信息，如果业务上存放的 Key 值在不断变化 (例如，将业务的 ID 作为 Key)，那么这个 HashMap 就开始不断变长，并且很可能在每个线程中都有一个这样的 HashMap，逐渐地形成了间接的内存泄漏。曾经有很多人吃过这个亏，而且吃亏的时候发现这样的代码可能不是在自己的业务系统中，而是出现在某些二方包、三方包中 (开源并不保证没有问题)。

要处理这种问题很复杂，不过首先要保证自己编写的代码是没问题的，要保证没问题不是说我们不去用 ThreadLocal，甚至不去学习它，因为它肯定有其应用价值。在使用时要明白 ThreadLocal 最难以捉摸的是“不知道哪里是源头” (通常是代码设计不当导致的)，只

有知道了源头才能控制结束的部分，或者说我们从设计的角度要让 `ThreadLocal` 的 `set`、`remove` 有始有终，通常在外部调用的代码中使用 `finally` 来 `remove` 数据，只要我们仔细思考和抽象是可以达到这个目的的。有些是二方包、三方包的问题，对于这些问题我们需要学会的是找到问题的根源后解决，关于二方包、三方包的运行跟踪，可参看第 3.7.9 节介绍的 `BTrace` 工具。

补充：在任何异步程序中（包括异步 I/O、非阻塞 I/O），`ThreadLocal` 的参数传递是不靠谱的，因为线程将请求发送后，就不再等待远程返回结果继续向下执行了，真正的返回结果得到后，处理的线程可能是另一个。

5.3 原子性与锁

在上一节中，讲解了包括可见性、重排序、屏障、栈封闭等许多线程安全的基本概念，而这只是一个开始，我们接下来要讲解原子性和锁的基本概念，它相对可见性实现了读取、修改、写入几个动作的一致性操作，相当于完成一个步骤。很显然，原子性和锁应当在可见性的基础上实现。本节我们将探讨锁与原子性的那些稀奇古怪的事情。

既然谈到了锁的问题，那么我们就从 Java 语言中大家十分熟悉的 `synchronized` 开始说起。

➤➤ 5.3.1 `synchronized`

`synchronized` 关键字是 Java 一直支持的锁，经历多个版本的发展和优化，它的内在实现也发生了巨大的改变。不过不论如何，它始终是一把锁，始终要保证临界区的访问控制。下面就来简单探讨临界区的概念。

`synchronized` 是针对一个临界区的，所谓临界区就是指访问这个地方最多只能有一个线程在里面（就像一支钢笔可以被多人共用，但是它在一段时间内最多可以被一个人使用）。`synchronized` 能做到这一点，它通过在对象上加锁后进入临界区来达到临界区串行访问的目的，而对象本身的作用域将决定锁的粒度。什么意思？请看下面的几种场景。

（1）普通方法前面加 `synchronized`

```
synchronized public void test() { ... }
```

该操作等价于在方法体前后包装了一个 `synchronized(this)`，或者说给当前类所在的对象加上了锁标记。与它互斥的情况通常有以下 3 种（也就是相互之间是串行的）：

- ◎ 在该类的所有非静态方法中发生 `synchronized(this)`。
- ◎ 在该类的所有非静态方法前面加上了 `synchronized` 关键字。
- ◎ 在其他类中得到该对象的引用，并对该对象进行 `synchronized` 操作。

（2）静态方法前面加 `synchronized`

```
synchronized public static void test() { ... }
```

这个锁操作等价于锁住了当前类的 `class` 对象，比如这个类叫作 A，那么相当于执行了 `synchronized(A.class)` 操作。与它互斥的场景就十分明显了：

- ◎ 代码中任意一个地方发生了 `synchronized(A.class)`。
- ◎ 在该类的所有静态方法前面增加了 `synchronized` 关键字。

我们需要注意的是，锁住了类，并不代表锁住了类所在的对象，类本身也是一种对象，

它与类的实例是完全不同的两个对象，在加锁时不是相互依赖的，换句话说，对类进行加锁并不与前一个案例所描述的加锁互斥。锁住了“子类”或“子类的对象”，与锁住“父类”或“父类的对象”是不相关的，它们彼此独立。

`synchronized` 主要就是这两种方式，其余的操作方式都是在这个基础上衍生出来的。例如大家经常说的“代码块加锁”：

```
synchronized(object) {
    //代码内容
}
```

锁住的其实并不是代码块，而是 `object` 这个对象，因此在其他代码中也发生 `synchronized(object)` 时就会发生互斥。其实 `synchronized(this)` 也是用这样的方式衍生出来的，只是在方法前面增加 `synchronized` 与之有一个区别，就是它的粒度并不是在方法级别，而是在某个指定的代码块级别。

我们为什么会研究这些？因为我们如果不知道锁住的是什么，就不清楚锁住了多大范围的内容，自然就不知道是否已经锁住了想要得到互斥效果的内容，同时也不知道如何去优化锁的使用。

因此 Java 中的 `synchronized` 就真正能做到临界区的效果（临界区的范畴并不取决于方法，而是取决于对象的范畴），在临界区内多个线程的操作绝对是串行的，这一点 Java 是肯定可以保证的。不过线程安全远远不止如此，如果它正在操作的对象是共享的，而一段没有加锁的程序也可以访问，那么依然不是线程安全的。

`synchronized` 造成的开销也是很大的，虽然 JVM 已经对 `synchronized` 做了许多优化，但这仅仅是技术上的，如果无法掌握好它的粒度控制（锁粒度将在 5.7.1 节中进行探讨），就会导致频繁的锁征用，进入悲观锁状态。这些内容也是研究锁粒度的技术和思想基础。

➤➤ 5.3.2 什么是乐观锁

什么是悲观锁？什么是乐观锁？这两个问题其实算是同一个问题，在网上和许多书籍中也有解释，胖哥在这里给大家提供一个另类的解释。

大家先抛开锁的概念，仅仅就乐观与悲观举一个小例子（不要将例子中的内容与锁强制扯上关系）。

我们去一个自己没有去过的地方办事，通常我们不敢将自己的贵重物品放在眼睛看不到的地方，甚至会时不时检查一下，因为不知道这里是否安全或者知道肯定不安全，所以认为可能会有问题。也许这里根本没有问题，但是我们同样会认为可能有问题，这或许就叫作悲观。

大家对自己所在的工作环境很放心，中午不会背上自己的所有办公用品和电脑下楼去吃饭，因为信任，所以大家很乐观。

看到这个例子，应该明白乐观就是指我们乐观地认为大多数情况下不会有问题。乐观地认为的基础是什么？历史经验告诉我们结果确实是这样的。反过来，悲观就是指没有历史经验告诉我们，或者说历史经验证明了另一个反面。

回到锁的话题，当发生“锁”操作时，如果锁是悲观的，那就意味着必须发生锁操作，不论多个线程会不会发生冲突都必须加锁。悲观锁将会产生非常多的指令开销来用以同步，甚至由于锁粒度的使用不当导致不应该锁住的代码被锁住了（锁粒度请参看 5.7.1 节），

许多历史证明，当系统合理划分，并将锁粒度细化到一定程度后，即使并发量很大，同一个临界区的征用概率也不会太大，也许在并发不高时 10000 次经过临界区也不会有一次冲突，达到一定访问量后也许会有一点点征用冲突，或许随着访问频率上升冲突的概率会逐渐增加，不过也会发现它在整体上冲突的比例依然很低，例如 10 次当中有 1 次冲突，这样的情况也同样存在于绝大部分系统中，因此提出了新的思想，那就是乐观锁的概念。

现代的 CPU 基本都会提供一种 CAS 指令机制（例如，x86 系统中是用 `cmpxchg` 指令来实现的），这种机制就是先获取 Old 值，然后计算出 New 值，基于可见性的变量检查是否被修改，若没有修改，则回写到主存表示成功（这个检查回写的过程就是 CAS）。这个步骤大概几条指令就可以完成，如果多个线程发生征用，则只有一个线程能成功，它与悲观锁相比，绝大部分情况下没有那么大开销，这就是乐观锁存在的意义，而外表上最大的特征就是它通常不会将线程挂起。

这种 CAS 机制在 Java 中应用到两个抽象层次：一是应用到 JVM 级别本身的 `synchronized` 锁里面；二是应用到基于 Java 语言本身这个抽象层次上面。前者主要是对 `synchronized` 本身做一些内在的优化，不过依然存在许多内在的复杂的状态处理；而后者提供一种宏观上的锁机制，在某些时候可以达到更为细粒度的原子处理。

有关 JVM 中内在的锁优化原则，近几个 JVM 版本有许多提升策略，这部分内容将在 5.3.7 节中阐述。在 Java 语言这个级别的 CAS 应用十分广泛，尤其是基于它来做自旋，在 JDK 1.6 以上版本中许多类都开始采用新的锁方式，在 JDK 1.7 以上版本中 Java 类几乎全部换成了这种方式。在接下来的内容中，我们将对照源代码来了解 Java 语言级别的 CAS 应用（例如 `Atomic`、`Lock` 以及各类工具和线程池）。

►► 5.3.3 并发与锁

或许有的小伙伴现在还没理解并发，还没理解为何会有锁。在这里胖哥给出一个小例子，或许可以引导一下思路。

胖哥认为如果大家想要理解并发，可以想想城市交通，有各种公路、地铁，为了建设城市需要花费大量的人力和物力（换句话说，并发的问题并不是那么容易处理的，尤其是达到一定数量级时）。

再进一步想一想，在城市各种交通十分畅通的情况下，某一天城市某个地方搞促销活动，全城中的大部分人蜂拥而至，甚至其他城市的人都涌向这里，问题就出现了，其中一个问题是到达这里的公交车、私家车全部堵上，地铁也全部很挤；另一个问题是这个区域是有限的，大家体会到的只有热闹，没有宽松的环境（并发也是这样，大家都蜂拥而至同一个地方）。于是我们考虑这样的活动是否能分散到城市的多个角落，有各种档次、品味的区域，这样可以缓解问题。

当某种促销活动超级给力时（例如一辆汽车一块钱），可能会引来大批观众。如果遵循先到先得的原则，大家会去抢机会，这就会导致大量的系统并发写征用（除非改变系统策略，例如：结果早已经内定、报名后统一随机抽取、几秒内的请求随机抽奖，那并发的压力就小很多了）。第一个抢到的人就开始付款，这并没有问题。如果是悲观锁，就会导致后面的人并不知道这一辆车已经被卖出，从而继续排队，后面的线程都会逐一去尝试追问：“是不是被抢走了？”因为蜂拥而至的人非常多，所以需要系统保持大量的会话以及它们的上下文信息，就会有越来越多的系统资源被占用而没有被释放，导致系统变慢，而且系

统越慢导致单个业务的处理速度就拖延越久，导致系统堵住的人就越多，如此就产生一个问题——达到临界点后的连锁反应，这样显然不是我们想要的结果。

我们希望的结果是第一个人抢到后，在办理手续时，后面所有的人便知道车已经被卖出，就不用再等待了，在程序上可能直接告知活动结束。或者通过某种方式来实现第一个人抢到资源后，大家不用逐一去问资源是否被抢走，而大家可以在一个大屏幕上看到没有机会了，所有等待的人就一起离开了，那么在系统中就等价于所有的线程都释放了上下文资源。

这就是锁与并发的那些事，在生活中你能理解这个道理，接下来就要看在计算机中如何去实现这些场景。

5.3.4 Atomic

由于 synchronized 用法的粒度始终是在代码级别，稍有不慎就可能成为悲观锁。另外，在许多运算操作中，许多时候只是希望对一个原子变量进行叠加，或者通过某种原子变量是否修改成功来判定线程是否得到了锁。如果有了这种轻量级的处理方式，可控方式完全可以由自己来设计。

Atomic 为我们提供了一系列 Java 原子变量的操作方法，在大多数情况下，我们不用直接基于 CAS 机制来做二次封装，除非自己想要实现对锁的控制粒度（例如，希望自旋的次数可以受到某种自定义算法的控制，如果超过一定的指标，则会抛错、返回 false、挂起等），可以进一步来做自定义扩展。

在本小节中，将谈到 Java 提供的一系列 Atomic 变量，以及它们的使用方式、基本原理。

(1) Atomic 为应用程序提供了哪些原子操作

Java 提供的 Atomic 原子操作类都在 java.util.concurrent.atomic 包下，如图 5-10 所示。

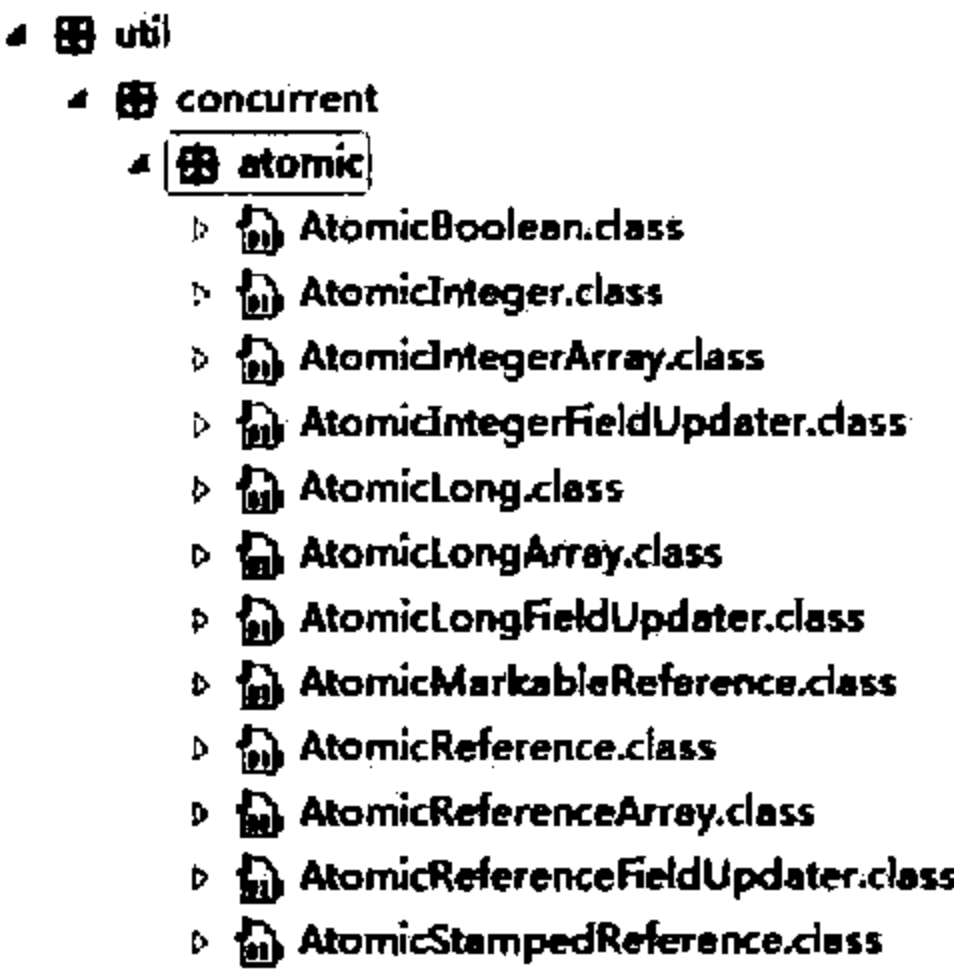


图 5-10 Atomic 原子操作类列表

下面胖哥把它们简单地分分类，说说它们都是用来做什么的。

胖哥认为，Atomic 原子变量可以分为 4 种类型。

- ◎ 基本变量操作：分别对 Boolean、Integer、Long 进行操作，对应的类名为 AtomicBoolean、AtomicInteger 和 AtomicLong。
- ◎ 引用（Reference）操作：也就是对象引用的操作，可以做到原子级别，当多个线程对同一个引用发生修改时，只会有一个成功。对于这种操作还存在一个 ABA 问题的处理，所以会有多个类处理引用的原子操作，如 AtomicReference、AtomicMarkableReference、AtomicStampedReference。

◎ 数组 (Array) 操作：这个操作并不是操作数组的对象，而是数组中的每一个元素，针对每一个元素的读写操作是线程安全的，Java 提供了 `AtomicIntegerArray`、`AtomicLongArray`、`AtomicReferenceArray`，分别对 `Integer`、`Long`、`Reference` 类型的数组元素进行原子读写。

◎ Updater：Java 提供了一种 Updater 机制，可以在原有的类定义 `volatile` 变量的基础上，实现一种原子性的管理，而不需要将变量本身定义为 `Atomic` 的，这样可以在不破坏原有程序逻辑的基础上实现一致性的读写。Java 提供的 Updater 包括 `AtomicIntegerFieldUpdater`、`AtomicLongFieldUpdater` 和 `AtomicReferenceFieldUpdater`。

(2) Atomic 使用方式

图 5-10 中显示了 12 个 `Atomic` 类，胖哥不会将这 12 个类的使用逐个写出来，因为那样没有意义。在这里胖哥会演示 4 种类型的操作，以及 ABA 问题及解决方法。

首先演示原子变量的操作，以 `Integer` 的操作为例来做一个模拟计数器的试验。

在做试验以前，大家要理解的是，当多个线程同时对普通的 `Integer` 变量做递增时，最终结果通常会小于理想中的值。例如初始化值为 1，有 10 个线程并发让这个数字不断做递增 1 的操作，每个线程做 100 次，理论结果是 1001，但是最终输出的结果可能会小于 1001，即使使用 `volatile` 来修饰它也依然可能是同样的结果，原因自然是前面提到的原子性问题。接下来这个例子会分别输出两个值，在程序中会发现 `AtomicInteger` 每一次输出的都是 1001，但是 `volatile` 变量有可能不是（前提是必须在多核的计算机上运行这段代码）。

代码清单 5-9 `AtomicInteger` 的例子

```
public class AtomicIntegerTest {

    /**
     * 常见的方法列表
     * @see AtomicInteger#get()           直接返回值
     * @see AtomicInteger#getAndAdd(int)  增加指定的数据，返回变化前的数据
     * @see AtomicInteger#getAndDecrement() 减少 1，返回减少前的数据
     * @see AtomicInteger#getAndIncrement() 增加 1，返回增加前的数据
     * @see AtomicInteger#getAndSet(int)   设置指定的数据，返回设置前的数据
     *
     * @see AtomicInteger#addAndGet(int)   增加指定的数据，返回增加后的数据
     * @see AtomicInteger#decrementAndGet() 减少 1，返回减少后的值
     * @see AtomicInteger#incrementAndGet() 增加 1，返回增加后的值
     * @see AtomicInteger#lazySet(int)     仅仅当 get 时才会 set
     *
     * @see AtomicInteger#compareAndSet(int, int) 尝试新增后对比，若增加成功，则
     * 返回 true；否则返回 false
     */
    public final static AtomicInteger TEST_INTEGER = new AtomicInteger(1);

    private static int index = 1;

    public static void main(String []args) throws InterruptedException {
        final CountDownLatch startCountDown = new CountDownLatch(1);
        final Thread []threads = new Thread[10];
        for(int i = 0 ; i < 10 ; i++) {
            threads[i] = new Thread() {
                public void run() {
                    try {
                        startCountDown.await();
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
            };
        }
    }
}
```

```

        for(int j = 0 ; j < 100 ; j++) {
            index++;
            TEST_INTEGER.incrementAndGet();
        }
    };
    threads[i].start();
}
Thread.sleep(1000);
startCountDown.countDown();
for(Thread t : threads) {
    t.join();
}
System.out.println("Atomic 最终运行结果: " + TEST_INTEGER.get());
System.out.println("volatile 最终运行结果: " + index);
}
}

```

大家可以直接用这段代码来做测试，多测试几次或许就会发现有什么不同了。

细心的同学可能发现胖哥在注释中写了 `AtomicInteger` 的许多操作选项，也描述了它们的特征（大家可以自行编写代码测试）。还有的同学发现这里使用到了 `CountDownLatch` 工具，它主要用来保证这些线程几乎同时开始有机会去征用 CPU，在后文中还会有相关介绍。继续看看数组（Array）的演示。

代码清单 5-10 数组元素的下标操作

```

public class AtomicIntegerArrayTest {

    /**
     * 常见的方法列表
     * @see AtomicIntegerArray#addAndGet(int, int)
     * 执行加法，第一个参数为数组的下标，第二个参数为增加的数量，返回增加后的结果
     * @see AtomicIntegerArray#compareAndSet(int, int, int)
     * 对比修改，参数 1: 数组下标; 参数 2: 原始值; 参数 3: 修改目标值，成功返回 true，否则返回 false
     * @see AtomicIntegerArray#decrementAndGet(int)
     * 参数为数组下标，将数组对应下标的数字减少 1，返回减少后的数据
     * @see AtomicIntegerArray#incrementAndGet(int)
     * 参数为数组下标，将数组对应下标的数字增加 1，返回增加后的数据
     * @see AtomicIntegerArray#getAndAdd(int, int)
     * 和 addAndGet 类似，区别是返回值是变化前的数据
     * @see AtomicIntegerArray#getAndDecrement(int)
     * 和 decrementAndGet 类似，区别是返回变化前的数据
     * @see AtomicIntegerArray#getAndIncrement(int)
     * 和 incrementAndGet 类似，区别是返回变化前的数据
     * @see AtomicIntegerArray#getAndSet(int, int)
     * 将对应下标的数字设置为指定值，第二个参数为设置的值，返回值是变化前的数据
     */
    private final static AtomicIntegerArray ATOMIC_INTEGER_ARRAY
        = new AtomicIntegerArray(10);

    public static void main(String []args) throws InterruptedException {
        Thread []threads = new Thread[100];
        for(int i = 0 ; i < 100 ; i++) {
            final int index = i % 10;
            final int threadNum = i;
            threads[i] = new Thread() {
                public void run() {
                    int result = ATOMIC_INTEGER_ARRAY.addAndGet(index,
index + 1);

                    System.out.println("线程编号为: " + threadNum
+ " , 对应的原始值为: " + (index + 1)
+ " , 增加后的结果为: " + result);
                }
            };
        }
    }
}

```



```

        }
        };
        threads[i].start();
    }
    for(Thread thread : threads) {
        thread.join();
    }
    System.out.println("=====>\n 执行已经完成, 结果列表:");
    for(int i = 0 ; i < ATOMIC_INTEGER_ARRAY.length() ; i++) {
        System.out.println(ATOMIC_INTEGER_ARRAY.get(i));
    }
}
}

```

代码中定义了一个长度为 10 的数组，然后用 100 个线程去修改里面的内容，这 100 个线程编号为 0~99，进一步与 10 取模，得到数组下标，将对应下标的数据进行叠加，这样平均每个下标会有 10 个线程去修改数据。

叠加的方式是每次叠加“下标值+1”的数字，例如下标 0 每次叠加 1，下标 1 每次叠加 2，依此类推。每个下标将会有 10 个线程来叠加，因此最终的输出结果应当是 10、20、30...100。运行上面的代码，应当会印证这样的结论。在输出的结果中，会看到有许多杂乱无章的输出，这从某种角度来讲，多线程确实是在并发地访问和修改下标。

接下来看看引用（Reference）的情况，这个如何去测试呢？

用许多线程同时去修改一个引用，如果发现只有一个线程能修改成功，那么就达到目的了，我们也将知道赋值后的结果就是成功获取到线程锁的那个线程。反过来讲，如果没有这种原子操作，很多线程都可能会赋值，赋值后的结果将不可预测。请看下面的代码。

代码清单 5-11 引用的原子修改

```

public class AtomicReferenceTest {

    /**
     * 相关方法列表
     * @see AtomicReference#compareAndSet(Object, Object)
     * 对比设置值，参数 1：原始值，参数 2：修改目标引用
     * @see AtomicReference#getAndSet(Object)
     * 将引用的目标修改为设置的参数，直到修改成功为止，返回修改前的引用
     */
    public final static AtomicReference<String>ATOMIC_REFERENCE
        = new AtomicReference<String>("abc");

    private final static Random RANDOM_OBJECT = new Random();

    public static void main(String []args) throws InterruptedException {
        final CountDownLatch startCountDownLatch = new CountDownLatch(1);
        Thread []threads = new Thread[20];
        for(int i = 0 ; i < 20 ; i++) {
            final int num = i;
            threads[i] = new Thread() {
                public void run() {
                    String oldValue = ATOMIC_REFERENCE.get();
                    try {
                        startCountDownLatch.await();
                        Thread.sleep(RANDOM_OBJECT.nextInt() & 1000);
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                    if(ATOMIC_REFERENCE.compareAndSet(oldValue ,
                        oldValue + num)) {

```

```

        System.out.println("线程: " + num + ",进行了对象
修改!");
    }
}
};
threads[i].start();
}
Thread.sleep(200);
startCountDownLatch.countDown();
for(Thread thread : threads) {
    thread.join();
}
System.out.println("最终结果为: " + ATOMIC_REFERENCE.get());
}
}

```

通过测试，大家应当得到前面提出的理论结果！这样一来，貌似已经控制好了对引用的修改，不过有些时候未必！

还需要解决一种特殊的 ABA 问题。什么是 ABA 问题呢？

由于某种原因使得多个线程尝试将数据从 A 变成 B，我们只允许业务上有一个提交成功，通过上面的方式可以实现只有一个线程能成功，此时若由于并发问题导致多个重复请求同时到来要求将 A 变成 B，此时仅有一个请求会成功，但有可能部分请求尚未被调度，此时又有另一个线程将 B 变成了 A，那么尚未调度的请求被调度时认为对象还是原先的 A，将其再次改为 B。

这貌似没有问题，因为本身为 A 的时候，就应该修改为 B。在某些场合确实没有问题，但是将问题细化来想想，另一个线程将 B 改为 A 的时间点应该发生在 A 改为 B 之后，按照道理说最终的 A 才是最终值。

下面举一个例子，大家就明白了。

小明有一天去提款机取钱，由于提款机问题重复提交了两次，服务器端为了避免这种问题使用了 CAS 机制，即先将账户的金额取出，减掉后进行回写，回写时除了带上 ID 外，还要带上原来的金额作为条件，如果原来的金额对不上，则不会修改任何数据，这样避免并发时减掉多次金额。

不过在并发时如果两个减金额的请求被先后执行，小明的妈妈给小明的账户内存入了相同的钱，那么在第一次减金额后，金额又恢复了小明取钱之前的余额，那么第 2 条扣款的语句可能依然会执行成功，此时金额就会被减掉 2 次，虽然这是低概率事件，但是在计算机系统的设计中对各种状态的修改其实是经常存在类似场景的。

其实这个问题归结起来是版本问题，为了说明代码中的情况，下面特意给出一段代码来说明。

代码清单 5-12 ABA 问题测试

```

public class AtomicReferenceABATest {

    public final static AtomicReference<String>ATOMIC_REFERENCE
        = new AtomicReference<String>("abc");

    private final static Random RANDOM_OBJECT = new Random();

    public static void main(String []args) throws InterruptedException {
        final CountDownLatch startCountDownLatch = new CountDownLatch(1);
        Thread []threads = new Thread[20];
        for(int i = 0 ; i < 20 ; i++) {

```



```

        final int num = i;
        threads[i] = new Thread() {
            public void run() {
                String oldValue = ATOMIC_REFERENCE.get();
                try {
                    startCountDownLatch.await();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                try {
                    Thread.sleep(RANDOM_OBJECT.nextInt() & 500);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                if(ATOMIC_REFERENCE.compareAndSet(oldValue,
                                                    oldValue + num)) {
                    System.out.println("我是线程: " + num + ",进行对
象修改!");
                }
            }
        };
        threads[i].start();
    }
    Thread.sleep(200);
    startCountDownLatch.countDown();
    new Thread() {
        public void run() {
            try {
                Thread.sleep(RANDOM_OBJECT.nextInt() & 200);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            while(!ATOMIC_REFERENCE.compareAndSet(ATOMIC_REFERENCE.get(),
                                                    "abc")) {
                System.out.println("已经改为原始值!");
            }
        }
    }.start();
}

```

测试的结果类似下面的情况:

我是线程: 19,我获得了锁进行了对象修改!

已经改为原始值!

我是线程: 8,我获得了锁进行了对象修改!

结果证明最少有两个线程修改过这个值，而这两个线程都认为 String 对象的值还是最原始的“abc”。大家都认为基于最原始的值在修改，并且都成功了。这或许在某些场景下并不是我们想看到的结果。

Java 提供了 AtomicStampedReference 来解决这个问题，它基于版本号或者说是一种状态，在修改的过程中不仅仅对比值，也会同时会对比版本号。该类提供了 getReference()、getStamp()方法用以获取对象本身的引用和版本号，而在进行 compareAndSet 操作时需要带上老的版本号与新的版本号。我们将上面的代码稍微修改一下，如下所示。

代码清单 5-13 使用 AtomicStampedReference 处理 ABA 问题

```

public class AtomicStampedReferenceTest {

    /**
     * 常见的方法
     * @see AtomicStampedReference#compareAndSet(Object, Object, int, int)
    */
}

```

```

* 对比修改
* @see AtomicStampedReference#attemptStamp(Object, int)
* 用于绝对判定是否一致, 返回 true|false
* @see AtomicStampedReference#getReference() 返回引用
* @see AtomicStampedReference#getStamp() 返回版本
* @see AtomicStampedReference#get(int[])
* 传入一个数组, 返回数组的 0 号元素为版本号
*/
public final static AtomicStampedReference <String>ATOMIC_REFERENCE
    = new AtomicStampedReference<String>("abc", 0);

private final static Random RANDOM_OBJECT = new Random();

public static void main(String []args) throws InterruptedException {
    final CountDownLatch startCountDownLatch = new CountDownLatch(1);
    Thread []threads = new Thread[20];
    for(int i = 0 ; i < 20 ; i++) {
        final int num = i;
        threads[i] = new Thread() {
            public void run() {
                String oldValue = ATOMIC_REFERENCE.getReference();
                int stamp = ATOMIC_REFERENCE.getStamp();
                try {
                    startCountDownLatch.await();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                try {
                    Thread.sleep(RANDOM_OBJECT.nextInt() & 500);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                if(ATOMIC_REFERENCE.compareAndSet(oldValue ,
                    oldValue + num , stamp , stamp + 1)) {
                    System.out.println("线程: " + num + ", 进行对象修改!");
                }
            }
        };
        threads[i].start();
    }
    Thread.sleep(200);
    startCountDownLatch.countDown();
    new Thread() {
        public void run() {
            try {
                Thread.sleep(RANDOM_OBJECT.nextInt() & 200);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            int stamp = ATOMIC_REFERENCE.getStamp();
            while(!ATOMIC_REFERENCE.compareAndSet(
                ATOMIC_REFERENCE.getReference(),
                "abc" , stamp , stamp + 1)) {
                stamp = ATOMIC_REFERENCE.getStamp();
            }
            System.out.println("已经改为原始值!");
        }
    }.start();
}

```

经过修改后的这段代码, 只有一个线程能将数据从 abc 修改为其他的值, 若另一个线程修改回来, 其他的线程就不会再做修改了。但是这并不意味着以后永远都无法再将 abc

修改为其他的值，这里讲究的是一瞬间重复提交的并发，由于 CPU 调度顺序可能导致值在中间发生两次由 A 到 B 的过程。当该值被再次修改为 A 后，如果业务上看到的是最新版本，那么再将其修改为 B 依然会成功（即并非重复提交类型的并发）。

另外，Java 还提供了一个叫作 `AtomicMarkableReference` 的类，它也可以解决上面提到的问题。它有一个很大的特征是状态只有两种，而不是像 `AtomicStampedReference` 那样使用版本号。例如，如果将一个值从 abc 变化为其他的值后，将这个值标志为已经修改，如果要修改成原先的值，不允许修改这个标志位，只允许修改内容，这样就可以防止 ABA 的问题发生。

最后说说 `Updater`，它是在原有的 `volatile` 变量基础上来进行原子修改操作的。

`Updater` 通过反射找到某个类的属性（也就是 `Updater` 要能访问这个 `Field`），每次 `compareAndSet` 时，要求传入具体的对象，这样说明 `Updater` 将独立于对象本身而存在，可以操作多个对象的同一个 `volatile` 属性。关于 `Updater` 的测试代码，由于篇幅所限，就不在书中贴出来了，在本书光盘的 `/src/chapter05/atomic/updater/` 目录下有相应的测试代码（主要看：`AtomicIntegerFieldUpdaterTest.java`）。

（3）Atomic 基本原理

其实 `Atomic` 的许多操作都来自于一个对比、修改的动作。下面我们以 `AtomicInteger` 的内部实现作为引导进一步展开对这些问题的探讨。在前面的例子代码中，我们使用了 `AtomicInteger` 的 `incrementAndGet` 方法，那么就从它开始吧，源码截图如图 5-11 所示。

```
public final int incrementAndGet() {
    for (;;) {
        int current = get();
        int next = current + 1;
        if (compareAndSet(current, next))
            return next;
    }
}
```

图 5-11 `incrementAndGet` 方法的内部代码

这段代码是一个死循环操作，循环内部首先获取一个“当前值”，然后将“当前值+1”得到下一个 `next` 值，再通过 `compareAndSet` 尝试将当前值修改为 `next` 值（这个动作就是 CAS）。这个方法返回了一个 `boolean` 变量，标识是否修改成功，若修改成功，则返回 `next` 值，自然也会退出死循环；否则就继续循环，直到成功为止（这就是自旋，另外，这个方法返回 `next` 值并不代表所有的自旋都返回 `next` 值，通过 `getAndIncrement` 就可以返回修改前的值）。

继续看看 `compareAndSet` 方法的代码是怎么写的，如图 5-12 所示。

这里调用了 `unsafe` 执行 `compareAndSwapInt` 操作来达到目的，这个 `Unsafe` 类的全名是：`sun.misc.Unsafe`（在第 3 章中已经提到）。与 `compareAndSwapInt` 类似的方法还有 `compareAndSwapObject`、`compareAndSwapLong`，它们对应于基本类型变量的原子操作。其中 `compareAndSwapLong` 要根据本地方法 `VMSupportsCS8()` 来决定 JVM 是否支持 8 字节的 CAS 操作，如果不支持，则会采用锁的方式来实现。

```
public final boolean compareAndSet(int expect, int update) {
    return unsafe.compareAndSwapInt(this, valueOffset, expect, update);
}
```

图 5-12 `compareAndSet` 方法的内部代码

既然提到了 Unsafe 就不妨将话题展开一点,看看 Unsafe 到底是个什么东西。其实从名称就看得出来它是不安全的,所谓不安全就是如果“操作不当”就会变得十分不安全。通过它可以操作一些在 Java 程序中不太方便操作的内容,例如前文中提到的直接内存就是 Unsafe 提供的 API 来完成的,不过这样的 API 在使用时通常是使用 Java 的二次封装,让我们不用考虑太多的问题,如果自己用就有可能产生一些问题。

如果自己真的想要用,怎么获取这个对象的实例呢?

在 AtomicInteger 中,通过 Unsafe.getUnsafe()就能获取到。我们也可以这样获取到吗?不行!因为这个 getUnsafe()方法内部对调用来源方的类做了判定,它要求调用来源方的类的 ClassLoader 是空的,也就是说,这个类是被 VM 本身的 ClassLoader 加载的,不在 Java 这个抽象层次,如图 5-13 所示。

```
public static Unsafe getUnsafe()
{
    Class localClass = Reflection.getCallerClass(2);
    if (localClass.getClassLoader() != null)
        throw new SecurityException("Unsafe");
    return theUnsafe;
}
```

图 5-13 Unsafe 的获取好像不太容易

要获取 Unsafe 变量的引用就不得不用到反射,但是否必须要反射 Unsafe 类来达到目的的呢?胖哥认为不一定,只要是通过间接引用 Unsafe 对象的地方就都可以用(如通过 AtomicInteger 反射得出的结果也一样),在本书光盘的 src/chapter05/atomic/extend/UnsafeTest 中胖哥写了一个简单的 Unsafe 获取方法,希望你能用得上。

进一步会发现 Unsafe 类里面提供了大量的 native 方法调用,就拿 AtomicInteger 来讲,它通过 unsafe.objectFieldOffset(Field)可以获取到属性在对象中的偏移量位置,这个值就是图 5-12 中 valueOffset 参数的值,也是通过进一步调用 Unsafe 的本地方法来完成。

由于通过 objectFieldOffset 方法可以看到属性在对象中的偏移量,因此通过反射遍历所有属性输出偏移量后,便可以清晰地看到对象属性具体的排布方式。

当然,类的静态属性(static)不会出现在对象中,因此用这种方式获取不到,修改成全局的,要实现类似的功能,就用 unsafe.staticFieldOffset(Field)来获取偏移量。

在 Unsafe 类中有大量的本地方法,或许我们可以找到一些原本认为 Java 不会提供的 API,大家可以根据 UnsafeTest 类的代码进一步去做一些方法调用的测试。

回到图 5-12,Java 级别的 CAS 就是通过 Unsafe 的 compareAndSet 相关的本地 API 完成的。总之,它基于可见性,修改前提取、修改后对比来确定是否写回到主存,最终达到原子性。

保证了原子性,是对单个变量来讲的,如果将它进一步扩展,谁能修改谁就进入临界区,那么自然就变成锁了。下一小节中将介绍 JDK 1.6 以后推出的一些 Java 抽象层次的锁机制,或许有许多 API 可以让我们使用,或许还有许多思想供给我们参考改造。

➤➤ 5.3.5 Lock

现在我们不仅仅可以通过 synchronized 来达到锁的目的,Java 还提供了许多纯 Java 语言级别的锁,其接口为 java.util.concurrent.locks.Lock,如图 5-14 所示。

一个是 ReadLock(读锁),一个是 WriteLock(写锁),它们都是 ReentrantReadWriteLock 中的 public static 类型的内部类,很多时候是先定义一个 ReentrantReadWriteLock 对象,然

后通过 `writeLock()`、`readLock()` 分别获取读锁和写锁，我们通常把这个叫作读写锁。

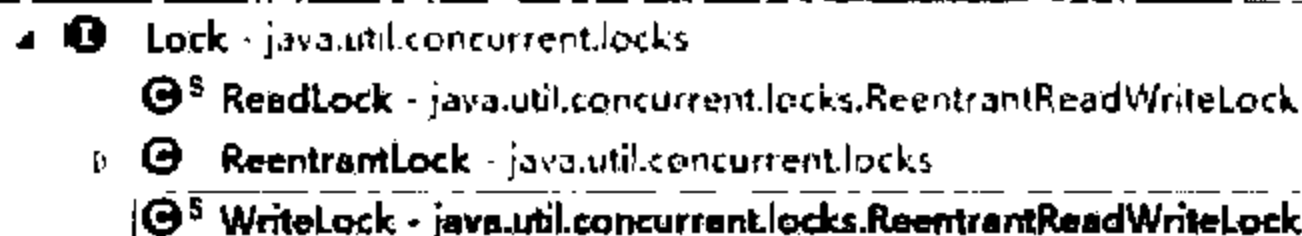


图 5-14 Java 提供的和 Lock 相关的几个类

还有一个是 `ReentrantLock`（排它锁），即是完全互斥的。反过来说读写锁自然也会有一定的排它性，只是允许同时多个读锁存在，但是读与写操作本身也是排它的，否则就会出现各种问题。如果将 `ReentrantLock` 展开，进一步发现 `ConcurrentHashMap.Segment` 内部类也是 `ReentrantLock` 的子类。

不论是哪一种锁，都会提供最基本的两个操作，就是 `lock()` 和 `unlock()` 方法，代码通常是这样写的：

```
lock.lock(); // 这条代码后面最好紧跟着 try 或某些非常简单不会抛异常的代码
try {
    // 业务代码...
} finally {
    lock.unlock(); // 这条代码一定要在 finally 中完成
}
```

通过 Lock 可以实现临界资源的同步，它的使用相对 `synchronized` 来讲语法要稍微复杂一点，不过可以让使用者更清楚地看到锁的粒度是基于这个 lock 对象的，那么要优化粒度自然就在这个锁对象级别。

另外，它是在 Java 语言层面基于 CAS 自旋方式来实现锁的，在并发条件下，其性能可能会比 `synchronized` 更好一些，因为程序一旦发生征用 `synchronized` 就很可能进入悲观锁状态。在 JDK 1.7 以后的版本中，Java 本身提供的许多类当中的锁都从 `synchronized` 修改成了 Lock 方式。

Lock 的实现除了基于 CAS 自旋外，其内在还有一个 AQS 机制，AQS 也同时是并发编程许多组件的基础，例如后文中将要提到的 `CountDownLatch`、`Semaphore` 都是基于 AQS 来完成的，因此我们有必要来研究下它的基本原理。

5.3.6 并发编程核心 AQS 原理

AQS 的全称为 `AbstractQueuedSynchronizer`，这个类也在 `java.util.concurrent.locks` 下。这个类似乎很不容易看懂，因为它仅仅提供了一系列公共的方法，让子类来调用。要理解它的意思，就得从子类入手，如图 5-15 所示。



图 5-15 AQS 的子类实现

这么多类，我们看哪一个？刚刚提到过锁（Lock），我们就从锁开始吧。下面就以 ReentrantLock（排它锁）为例展开讲解如何利用 AQS，然后简单介绍读写锁的要点（读写锁本身的实现十分复杂，要完全说清楚需要大量的篇幅）。

首先来看看 ReentrantLock 的构造方法，它的构造方法有两个，如图 5-16 所示。

```
public ReentrantLock() {
    sync = new NonfairSync();
}

public ReentrantLock(boolean fair) {
    sync = (fair)? new FairSync() : new NonfairSync();
}
```

图 5-16 排它锁的构造方法

很显然，对象中有一个属性 sync，有两种不同的实现类，默认是 NonfairSync 实现类，另一个是 FairSync，不论用哪一个都能实现排它锁，只是内部原理有点区别。这里以 NonfairSync 类为例，它的 lock() 方法是如何实现的呢？见图 5-17。

```
final void lock() {
    if (compareAndSetState(0, 1))
        setExclusiveOwnerThread(Thread.currentThread());
    else
        acquire(1);
}

protected final void setExclusiveOwnerThread(Thread t) {
    exclusiveOwnerThread = t;
}

public final void acquire(int arg) {
    if (!tryAcquire(arg) &&
        acquireQueued(addWaiter(Node.EXCLUSIVE), arg))
        selfInterrupt();
}
```

图 5-17 排它锁的 lock() 方法

lock() 方法先通过 CAS 尝试将状态从 0 修改为 1。若直接修改成功（前提条件自然是锁的状态为 0），则直接将线程的 Owner 修改为当前线程。这是一种理想情况，如果并发粒度设置适当，这也是一种乐观情况。

若上一个动作未成功，则会间接调用 acquire(1) 来继续操作，这个 acquire(int) 方法就在 AbstractQueuedSynchronizer 中。这个方法表面上看起来简单，但实际上比较难以看懂，甚至有可能第一次看这段代码时不知道它要做什么！如果你也遇到了，请看胖哥一步步来分解。

首先看 tryAcquire(arg) 这里的调用（当然传入的参数是 1），在默认的 NonfairSync 实现类中，会这样来实现，如图 5-18 所示。

首先获取这个锁的状态，如果状态为 0，则尝试设置状态为传入的参数（这里的参数值就是 1），若设置成功就代表自己获取到了锁，返回 true。状态从 0 设置到 1 的动作在外部调用前就做过一次，该代码再做只是为了再一次尝试，尽量用最简单的方式获得锁。

如果状态不为 0，则判定当前线程是否为排它锁的 Owner，如果是 Owner，则尝试将状态值增加 acquires（也就是增加 1），如果这个状态值越界，则会抛出异常；若没有越界，则将状态设置进去后返回 true（实现了类似于偏向的功能，可重入，但是无须进一步征用）。


```

protected final boolean tryAcquire(int acquires) {
    return nonfairTryAcquire(acquires);
}
final boolean nonfairTryAcquire(int acquires) {
    final Thread current = Thread.currentThread();
    int c = getState();
    if (c == 0) {
        if (compareAndSetState(0, acquires)) {
            setExclusiveOwnerThread(current);
            return true;
        }
    }
    else if (current == getExclusiveOwnerThread()) {
        int nextc = c + acquires;
        if (nextc < 0) // overflow
            throw new Error("Maximum lock count exceeded");
        setState(nextc);
        return true;
    }
    return false;
}

```

图 5-18 tryAcquire()的代码

如果状态不为 0，且自己不是 Owner，则返回 false。

回到图 5-17 中，对 tryAcquire()的调用判定是将 if(!tryAcquire())作为第 1 个条件的，如果返回 true，则条件就不会成立，自然后面的 acquireQueued 动作就不会再执行了，这样的情况是最理想的。

无论多么乐观，征用是必然存在的，如果征用存在，则 Owner 自然不会是自己，tryAcquire()方法会返回 false，接着就会再调用 acquireQueued(addWaiter(Node.EXCLUSIVE), arg)方法做相关的操作。

这个方法需要从里往外看，这里的 Node.EXCLUSIVE 是节点的类型，看名称应该清楚是排它类型。接着调用 addWaiter()方法来增加一个排它锁类型的节点，addWaiter()方法的代码如图 5-19 所示。

```

private Node addWaiter(Node mode) {
    Node node = new Node(Thread.currentThread(), mode);
    // Try the fast path of enq. backup to full enq on failure
    Node pred = tail;
    if (pred != null) {
        node.prev = pred;
        if (compareAndSetTail(pred, node)) {
            pred.next = node;
            return node;
        }
    }
    enq(node);
    return node;
}

```

图 5-19 addWaiter()的代码

这里创建了一个 Node 对象，将当前线程和 Node.EXCLUSIVE 传入，也就是说，Node 节点理论上包含了这两项信息。代码中的 tail 是 AQS 的一个属性，刚开始时肯定为 null，也就是不会进入第一层 if 判定的区域，而是直接进入 enq(node)。那就继续看看 enq(node)

的代码，如图 5-20 所示。

```
private Node enq(final Node node) {
    for (;;) {
        Node t = tail;
        if (t == null) { // Must initialize
            Node h = new Node(); // Dummy header
            h.next = node;
            node.prev = h;
            if (compareAndSetHead(h)) {
                tail = node;
                return h;
            }
        }
        else {
            node.prev = t;
            if (compareAndSetTail(t, node)) {
                t.next = node;
                return t;
            }
        }
    }
}
```

图 5-20 enq(Node)的代码

看到了 tail 就应该猜到了 AQS 是链表,而且它还应该有一个 head 引用指向链表的头节点, AQS 在初始化时 head、tail 都为 null,在运行时来回移动。此时,我们至少要知道 AQS 是一个基于状态 (state) 的链表管理方式。

这段代码就是对链表的操作,胖哥简单讲解一下这段代码。

首先这是一个死循环,而且本身没有锁,因此可以有多个线程并发访问,假如某个线程进入方法,此时 head、tail 都为 null,自然会进入 if(t == null)所在的代码区域,这部分代码会创建一个名字为“h”的 Node 对象,然后将这个 h 的 next 引用到传入的 node 对象,此时传入的 node 和某个线程创建的 h 对象的关系如图 5-21 所示。

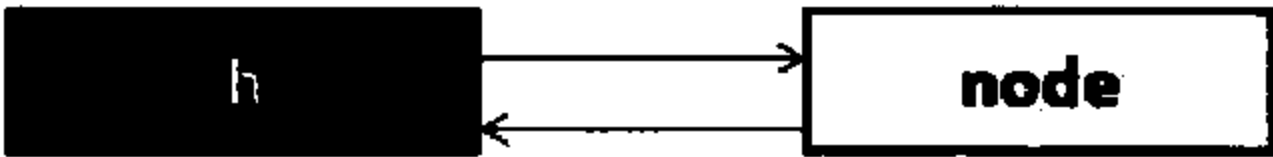


图 5-21 传入的 Node 和 h 对象的关系

我们很理想地认为只有一个线程会出现这种情况,如果有多个线程并发进入这个 if 判定区域,那么可能就会同时存在多个这样的数据结构,在各自形成数据结构后,多个线程都会去执行 compareAndSetHead(h)动作,也就是尝试将临时的 h 节点设置为 head,显然并发时只有一个线程会成功,因此成功的那个线程会执行 tail = node 操作,整个 AQS 链表就如图 5-22 所示。

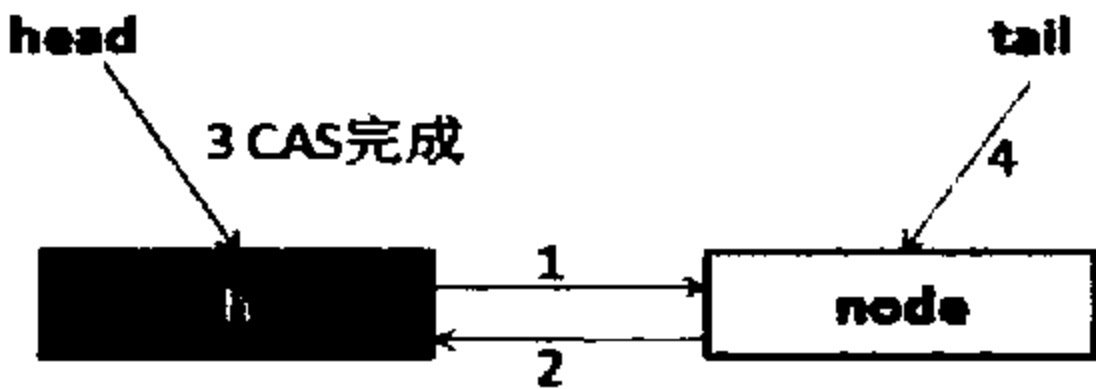


图 5-22 AQS 中被第一个线程成功访问后内部的结构

有一个线程会成功修改 head 和 tail 的值，其他的线程会继续循环，再次循环时就不会进入 if(t == null)的逻辑了，而会进入 else 语句的逻辑中。

在 else 语句所在的逻辑中，第一步是 node.prev = t，这个“t”就是 tail 的临时值，也就是首先让尝试写入的 node 节点的 prev 指针指向原来的结束节点，然后尝试通过 CAS 替换 AQS 中 tail 的内容为当前线程的 Node，无论有多少个线程并发到这里，依然只有一个能成功，成功者执行 t.next = node，即让原先的 tail 节点的 next 引用指向现在的 node，现在的 node 已经成为最新的结束节点，不成功者则会继续循环。

图 5-23 是对上述逻辑的简单图解，请注意图中的 3 个数字标明的步骤。

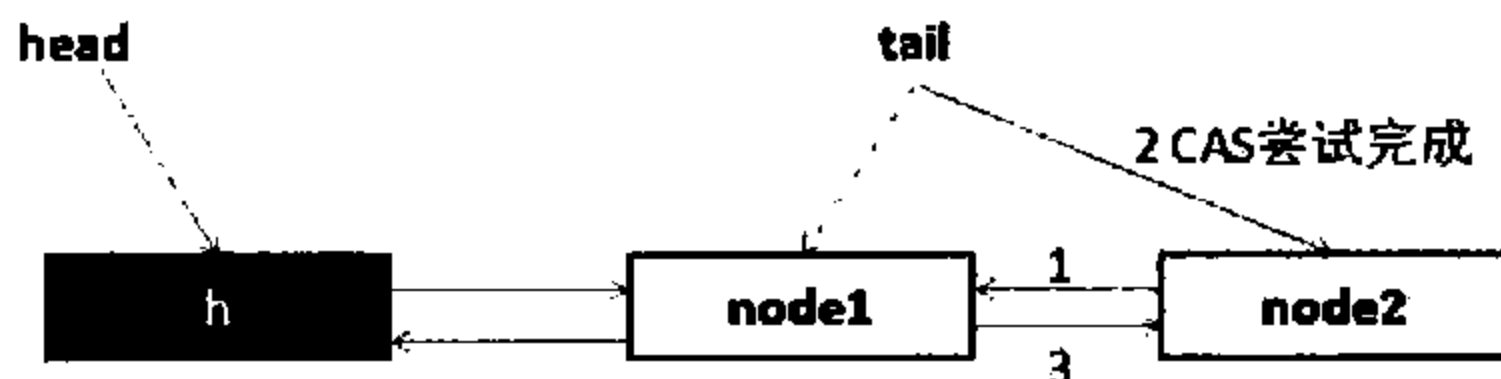


图 5-23 AQS 中再次插入一个节点的步骤

可以插入多个节点，节点都是在链表尾部插入的，而且是线程安全的。

知道了 AQS 的写入是一种双向链表的插入操作，但插入链表节点对锁有何用途呢？我们回退到图 5-19，addWaiter()方法最终返回了要写入的 node 节点，再回退到图 5-17，需要将这个返回的 node 节点作为 acquireQueued()方法的入口参数，并传入另一个参数（依然是 1）。如图 5-24 所示，看看 acquireQueued()方法里面到底做了些什么。

这段代码也是一个死循环，只有判定条件 if(p == head && tryAcquire(arg))成立时才会返回。p 是通过 node.predecessor()得到的，这个方法返回 node 节点的前一个节点，也就是说，只有当前一个节点是 head 时，进一步尝试通过 tryAcquire(arg)来征用才有机会成功。tryAcquire(arg)方法前面介绍过，成立的条件是锁的状态为 0，且通过 CAS 尝试设置状态成功，或者线程的持有者本身是当前线程，才会返回 true。我们现在来详细拆分这部分代码。

如果这个判定条件成立，则首先调用 setHead(Node)的操作，在这个操作内部会将传入的 node 节点作为 AQS 的 head 所指向的节点。线程属性设置为空（因为现在已经获取到锁，不再需要记录这个节点所对应的线程了），再将这个节点的 perv 引用赋值为 null。

```
final boolean acquireQueued(final Node node, int arg) {
    try {
        boolean interrupted = false;
        for (;;) {
            final Node p = node.predecessor();
            if (p == head && tryAcquire(arg)) {
                setHead(node);
                p.next = null; // help GC
                return interrupted;
            }
            if (shouldParkAfterFailedAcquire(p, node) &&
                parkAndCheckInterrupt())
                interrupted = true;
        }
    } catch (RuntimeException ex) {
        cancelAcquire(node);
        throw ex;
    }
}
```

图 5-24 acquireQueued()方法

进一步将前一个节点的 next 引用赋值为 null。在进行了这样的修改后，队列结构就变成了如图 5-25 所示的这种情况，通过这样的方式，就可以让执行完的节点释放内存区域，而不是无限制地增长队列，在这段代码中更像 FIFO 的处理。

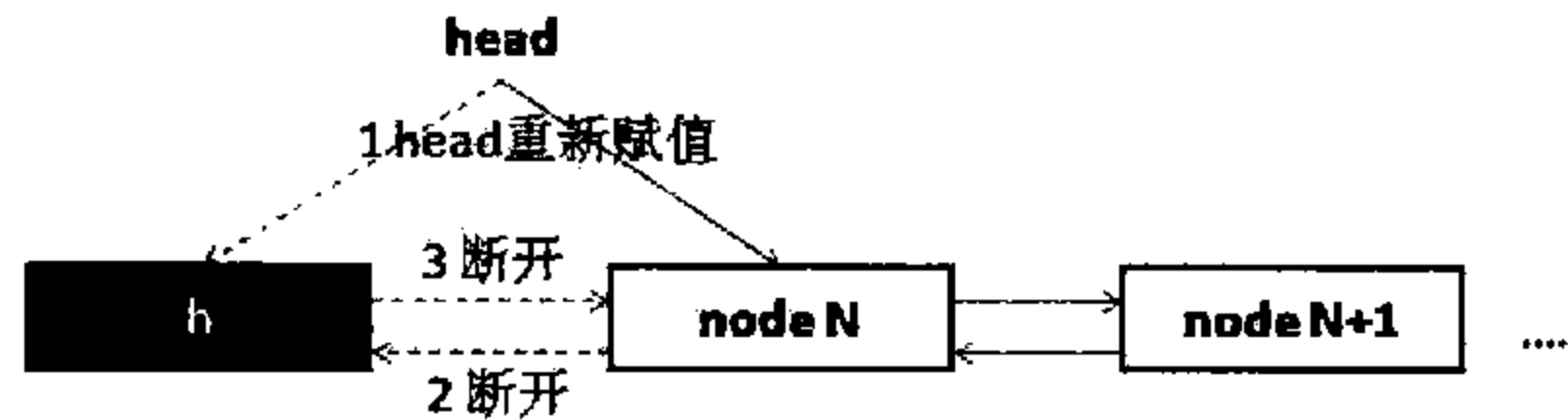


图 5-25 CAS 成功获取锁后，队列的变化

如果这个判定条件不成立，代码则会运行到后续的判定条件，该判定条件首先会判定方法 `shouldParkAfterFailedAcquire(p, node)` 的返回值，在这个方法内部会判定前一个节点的状态是否为 “Node.SIGNAL”，若是则返回 `true`，若不是则返回 `false`，但会再执行一些操作：判定节点的状态值是否大于 0，若大于 0 则认为被 “CANCELLED” 掉了（本书中没有说明状态值，通过查看源码可以发现状态值大于 0 的只可能是被 CANCELLED 的状态），因此会从前一个节点开始逐步向链表前循环找到一个没有被 “CANCELLED” 的节点，然后与这个节点的 `next`、`prev` 的引用相互指向；如果前一个节点的状态值不大于 0，则通过 CAS 尝试将状态修改为 “Node.SIGNAL”，虽然只是一个状态设置，但外部调用是一个死循环，若下一轮循环再一次判定 `shouldParkAfterFailedAcquire(p, node)`，就肯定会发现前一个节点的状态是 `Node.SIGNAL`，从而返回 `true`。

如果 `shouldParkAfterFailedAcquire(p, node)` 返回了 `true`，则会执行 `parkAndCheckInterrupt()` 方法，它通过 `LockSupport.park(this)` 将当前线程挂起到 `WAITING` 状态，它需要等待一个中断、`unpark()` 方法来唤醒它，通过这样一种 FIFO 机制的等待就实现了 Lock 的操作。

相应的，大家可以自己看看 `FairSync` 实现类的 `lock()` 方法，其实区别不大，但有些细节上的区别可能会决定某些特定场景的需求，我们也可以按照这样的思路去实现一个自定义的锁。

如果获取到了锁不释放，那么自然就成了死锁，所以必须要释放，与 `lock()` 相对应的方法就是 `unlock()`，看看它内部是如何释放的。同样从排它锁（`ReentrantLock`）中的 `unlock()` 方法开始，代码如图 5-26 所示。

```
public void unlock() {
    sync.release(1);
}

public final boolean release(int arg) {
    if (tryRelease(arg)) {
        Node h = head;
        if (h != null && h.waitStatus != 0)
            unparkSuccessor(h);
        return true;
    }
    return false;
}
```

图 5-26 `unlock()` 方法间接调用 AQS 的 `release(1)` 来完成

通过 `tryRelease(int)` 方法进行了某种判定，若成立则会将 `head` 传入到 `unparkSuccessor(Node)` 方法中并返回 `true`，否则返回 `false`。首先来看看 `tryRelease(int)` 方法，如图 5-27 所示。


```
protected final boolean tryRelease(int releases) {
    int c = getState() - releases;
    if (Thread.currentThread() != getExclusiveOwnerThread())
        throw new IllegalMonitorStateException();
    boolean free = false;
    if (c == 0) {
        free = true;
        setExclusiveOwnerThread(null);
    }
    setState(c);
    return free;
}
```

图 5-27 tryRelease(int)方法

这个动作可以认为就是一个设置锁状态的操作，而且是将状态值减去传入的参数值（参数值是 1），如果结果状态值为 0，就将排它锁的 Owner 设置为 null，以使得其他的线程有机会执行。

在排它锁中，加锁时状态值会增加 1（当然可以自己修改这个值），在解锁时减去 1，对于同一个锁重入后，该值会被继续叠加，只有 unlock() 的次数与 lock() 的次数对应才会将 Owner 线程设置为空，而且也只有在这种情况下才会返回 true。

这一点大家写代码时要注意，若调用 lock() 两次以上，而只调用一次 unlock()，最终可能无法释放锁，例如在循环体中调用 lock() 方法就要在循环体内调用 unlock() 方法，且需要保证 unlock() 方法一定能够被执行。在本书光盘的 src/chapter05/locks/ 目录下有相应的代码，大家可以自行测试。

在 unparkSuccessor(Node) 方法中，就意味着真正要释放锁了，它传入的是 head 节点（head 节点是已经执行完的节点，在后面阐述这个方法的 body 时都叫 head 节点），内部首先会发生的动作是获取 head 节点的 next 节点，如果获取到的节点不为空，则直接通过 LockSupport.unpark() 方法来释放对应的被挂起的线程，这样将会有有一个节点唤醒后继续进入图 5-24 所示的循环中进一步尝试 tryAcquire() 方法来获取锁，但是也未必能完全获取到，因为此时可能有一些外部的请求正好与之征用，而且还奇迹般地成功了，那么这个线程就有点悲剧了，不过通常乐观地认为不会每一次都悲剧。

再看看共享锁。从前面的排它锁可以看得出来是用一个状态来标志锁的，共享锁也不例外，但是 Java 不希望定义两个状态，所以它与排它锁的第一个区别就是在锁的状态上。它用 int 来标志锁的状态，int 有 4 个字节，用高 16 位标志读锁（共享锁），低 16 位标志写锁（排它锁），高 16 位每次增加 1 相当于增加 65536（通过 $1 \ll 16$ 得到），自然的在这种读写锁中，读锁和写锁的个数都不能超过 65535 个（条件是每次增加 1，如果递增是跳跃的将会更少）。在计算读锁数量时将状态左移 16 位，而计算写锁数量时会与 65535 按位求“与”操作，如图 5-28 所示。

写锁的功能与 ReentrantLock 基本一致，唯一的不同是在 tryAcquire 操作时，判定状态会复杂一点。

读锁也会写入队列，Node 的类型被改为“Node.SHARED”，lock() 时调用的是 AQS 的 acquireShared(int) 方法，进一步调用 tryAcquireShared() 操作里面只需要检测是否有排它锁，就可以尝试通过 CAS 修改锁的状态，如果没有修改成功，则会自旋这个动作。如果在自旋的过程中检测到排它锁竞争成功，那么 tryAcquireShared() 会返回 -1，从而会走与排它锁的 Node 类似的流程，可能也会被 park 住，等待排它锁相应的线程最终调用 unpark() 动作来唤醒。

```

static final int SHARED_SHIFT = 16;
static final int SHARED_UNIT = (1 << SHARED_SHIFT);
static final int MAX_COUNT = (1 << SHARED_SHIFT) - 1;
static final int EXCLUSIVE_MASK = (1 << SHARED_SHIFT) - 1;

/** Returns the number of shared holds represented in count */
static int sharedCount(int c) { return c >>> SHARED_SHIFT; }
/** Returns the number of exclusive holds represented in count */
static int exclusiveCount(int c) { return c & EXCLUSIVE_MASK; }

```

图 5-28 读写锁的数量计算及限制

这就是读写锁，不过共享锁里面也有多种机制，我们只是说了一种而已。在读写锁下运行的程序，读和写操作本身是互斥的，但是可以一起发生多个读操作。这样的锁非常适合应用在“读多写少”的环境下，这样锁征用的粒度会大大降低，同时系统的瓶颈会减少，效率得到总体提升。

在本节中我们除了学习到 AQS 的内在，还应看到 Java 通过一个 AQS 解决了许多问题，AQS 是纯 Java 语言的队列模型，除此之外，还可以利用队列模型来解决许多其他类型的问题，甚至改写模型来满足需求（在本章的 5.6.1 节中将会详细介绍）。

关于 Lock 及 AQS 的一些补充：

(1) Lock 的操作不仅仅局限于 lock()/unlock()，因为线程可能进入 WAITING 状态，这个时候如果没有 unpark() 就没法唤醒它，可以使用 tryLock()、tryLock(long, TimeUnit) 来做一些尝试加锁或超时来满足某些特定场景的需要。例如，有些时候发现尝试加锁无法加上，可以先释放对其他对象已经成功添加的锁，过一会再来尝试，这样在某些场合下可以避免“死锁”。

(2) lockInterruptibly() 允许抛出 InterruptedException 异常，也就是当外部发起了中断操作时，程序内部有可能会抛出这种异常。

(3) newCondition() 操作是返回一个 Condition 对象，Condition 只是一个接口，它要求实现 await()、awaitUninterruptibly()、awaitNanos(long)、await(long, TimeUnit)、awaitUntil(Date)、signal()、signalAll() 方法，AbstractQueuedSynchronizer 中有一个内部类 ConditionObject 实现了这个接口，它也是一个类似于队列的实现，在大多数情况下可以直接使用。

(4) 在 AQS 的每个 Node 中都有自己的状态 (waitStatus)，我们这里归纳一下：

- ◎ SIGNAL，从前文的代码讲解中可以看出，该状态代表自己正在等待前面的线程运行结束，并调用 unpark() 方法才能激活自己，值为 -1。
- ◎ CANCELLED，当征用锁过程抛出异常调用 fullyRelease(Node) 方法或调用 fullyRelease(Node) 方法时，会处于这种状态。值为 1，也是几种状态中唯一一种大于 0 的状态，所以判定状态值大于 0 就基本是 CANCELLED 的意思。
- ◎ CONDITION，线程基于 Condition 对象发生了等待，进入了相应的队列，自然也需要 Condition 对象来激活，值为 -2。
- ◎ PROPAGATE，在读写锁中，读锁最开始没有获取到操作权限，得到后会发起一个 doReleaseShared() 动作，内部也是一个循环，当判定后续的节点状态值为 0 时，尝试通过 CAS 自旋方式将状态修改为此状态，表示节点可以运行。
- ◎ 状态 0，初始化状态，代表正在尝试去获取临界资源的线程所对应的 Node 的状态。

5.3.7 锁的自身优化方法

在 5.3.2 节中，提到了“乐观”的概念，本节将继续探讨一些 JVM 级别的锁，根据“乐观”概念锁自身也做了某些优化。

在前文中提到了通过 CAS 来修改数据，将这种操作做一个 `while(times < 最大次数) {尝试...}` 循环，也就是不断进行 CAS 尝试，便可以达到锁的目的（前一节的 AQS 也是大量采用类似的方式）。JVM 底层的锁也逐步通过这样的思路进行改善，因为乐观，所以认为尝试次数不会太多，如果尝试次数太多就会将它转换为悲观，也就是将线程阻塞，否则这个线程就会在这里空耗 CPU。

胖哥认为，乐观不仅仅局限于此，它还乐观地认为自己有尝试或者说有选择，因为在尝试一定次数后，完全可以选择不再尝试直接离开；而悲观锁是它自己在被动地调度，不论排队几天几夜都要誓死拿到“门票”。

尝试的这个过程通常叫作“自旋”，这种锁在 JDK 1.4.2 就被引入了，开启参数是 `-XX:+UseSpinning`，自旋的默认次数通过参数 `-XX:PreBlockSpin=10` 来设置。在 JDK 1.6 过后，自旋开始向智能化发展，也就是说，它觉得在某些情况下可以自己调节自旋的次数，这个次数的改变通常是由历史的代码请求自旋成功的概率和次数来决定的，它内部有一套自己所认为智能的算法，通常会认为前面通过自旋几次成功过，那么后面的线程就尽量不要阻塞，在一定程度上会增加自旋的次数。

这种方式就有点像我们去车站买火车票，有长长的买票队伍，自己可能会排一会，但是发现排了很久还是离卖票窗口很远，就会认为自己即使排到窗口也没票了，不如回家睡大觉，等明天出新票的时候再来买。

但是如果有许多朋友排队一段时间就拿到了票，或许就是比自己多排了一小会就排到了卖票窗口，那么自己下次排队时，或许会多排一会，因为多努力一点点就可能买到票了。

这是 JVM 的一种改善机制，JVM 还有没有其他的改善呢？有的，我们接下来探讨一下轻量级锁、偏向锁的基本原理。

`synchronized` 会在对象的头部打标记，这个加锁的动作是必须要做的，悲观锁通常还会做许多其他的指令动作，轻量级锁希望通过 CAS 实现（因为它很乐观），它认为通过 CAS 尝试修改对象头部的 `mark` 区域的内容就可以达到目的，由于 `mark` 区域的宽度通常是 4~8 字节，也就是相当于一个 `int` 或 `long` 的宽度，所以这十分适合于 CAS 操作。

轻量级锁通常会做以下 4 个步骤。

（1）在栈当中分配一块空间用来做一份对象头部 `mark word` 的拷贝，在 `mark word` 中将对象锁的二进制位设置为“未锁定”（在 32 位的 JVM 中通常有 2 位用于存储锁标记，未锁定的标记为 01），这个动作是方便等到释放锁的时候将这份数据拷贝到对象头部。

（2）通过 CAS 尝试将头部的二进制位修改为“线程私有栈中对 `mark` 区域拷贝存放的地址”，如果成功，则会将最后 2 位设置为 00，代表已经被轻量级锁锁住了。

（3）如果没有成功，则判定对象头部是否已经指向了当前线程所在的栈当中，如果成立，则代表当前线程已经是拥有者，可以继续执行。

（4）如果不是拥有者，则说明有多个线程在征用，那么此时会将锁升级为悲观锁，线程进入 `BLOCKED` 状态。

在 JDK 1.6 以后默认开启了偏向锁的, 参数为 `-XX:+UseBiasedLocking`。JVM 发现在轻量级锁里多次“重入”与“释放”时, 需要做的判定和拷贝动作还是较多, 而在某些应用程序中, 锁就是被某一个线程一直使用, 为了进一步减小锁的开销, JVM 中出现了偏向锁, 偏向锁希望记录的是一个线程 ID, 它比轻量级锁更加轻量, 当再次重入判定时, 首先判定对象头部的线程 ID 是不是当前线程, 若是则表示当前线程已经是对象锁的 Owner, 无须做其他任何动作。

偏向锁实现大概分为以下几个步骤。

(1) 一个线程去竞争时, 如果没有其他线程征用, 则会尝试 CAS 去修改 mark word 中的一个标记为偏向 (mark word 单独有 1 个 bit 表示是否可偏向, 记录锁的位置依然为 01), 这个 CAS 的动作同时会修改 mark word 部分 bit 以保留线程的 ID 值 (真实的应用场景中线程数是很少的)。

(2) 当线程不断发生重入时, 判定就会非常简单, 只需要判定头部的线程 ID 是不是当前线程, 若是当前线程, 则不用做其他任何 CAS 动作了。

(3) 如果同一个对象存在另一个线程发起了访问请求, 则首先会判定该对象是否已经被锁定。如果已经被锁定, 则会将锁修改为轻量级锁 (00), 也就是锁粒度上升了; 而如果没有锁定, 则会将对象的是否可偏向的位置设置为不可偏向。

自然的, 在后续的操作中, 如果锁定了这个对象, 就会最少使用轻量级锁, 因为轻量级锁的征用上升依然会进入悲观锁。也就是说, 偏向锁只是解决了没有任何锁征用的场景, 当出现锁征用后它便没什么用途了。

除此之外, JVM 还对许多锁做了优化, 类似于 StringBuffer、Vector、HashTable 这些操作, 虽然表面上没有使用 synchronized, 但是它们的每一次读写操作内部都间接地使用了 synchronized, 而这些开销是巨大的。在 JDK 1.4 以前的版本中, 隐藏更深的是 `String a = b + c`; 这样的操作, 会编译为 StringBuffer 的 append() 操作。Java 后来推出了另一个极端组件, 就是完全无锁的 StringBuilder, Vector 对应 ArrayList、LinkedList 等结构, HashTable 对应 HashMap 等结构。

在程序中我们或许不再使用 StringBuffer 来做字符串拼接, 但是 Java 并没有将所有的 StringBuffer 都替换为 StringBuilder, 其实 Java 内部有许多类在 JDK 1.6 版本中还未修改为最新的 StringBuilder, 包括一些三方框架的代码可能也是 StringBuffer。JVM 对原本可能不需要加锁的代码加锁了, 我们没得选择, 不过自己应该如何写代码是值得思考的。

JVM 有时候还总会做一些让人意想不到的优化, 除了前文中提到的动态 inline、指令重排序、判定逻辑合并与取消、字符串的编译时优化等, JVM 还在锁层面做了些类似的事情。当发生简单、反复的某种锁操作时, JVM 会尝试将它的粒度变粗, 也就是在最外面做一层锁, 而不是做反复地进入锁、退出锁的动作, 这样在某种程度上可以提升性能, 但这仅仅局限于简单反复的加锁动作, 例如循环中调用 StringBuffer 的 append() 方法。

5.4 JDK 1.6 并发编程的一些集合类

在 JDK 1.6 以后, Java 除了提供许多并发编程工具和锁以外, 还提供了新版的并发编程的集合类, 旨在寻求锁与无锁之间的某些元素, 在线程安全的情况下, 性能又能得到更好的发挥。

这些集合类也是位于 `java.util.concurrent` 包下，本节我们谈一谈经常使用的集合类，如果遇到并发问题，以前是如何处理的，现在又有什么替代品。

胖哥相信用过 Java 的人，对 `ArrayList`、`LinkedList`、`HashMap`、`TreeMap`、`HashSet`、`TreeSet` 这些集合类是非常熟悉的，因为它们和你的代码同在。在以前，我们可能会用锁来处理对它们的并发读写操作，或者通过 Java 本身提供的集合类来解决。

以 `List` 为例，对于列表结构，如果要控制并发读写操作，以前要么自己加锁处理，要么利用 `Vector` 集合类，但是这会导致一些问题，抛开 `synchronized` 本身的开销，还会导致锁粒度非常大。简单来讲，`Vector` 中有非常多的元素，每次读写操作都会将数组中所有的元素访问都锁住。例如一个线程访问 `Vector` 中下标为 1 的元素，另一个线程访问 `Vector` 中下标为 2 的元素，而且都是读操作，也会产生互斥的现象。

显然我们不希望这样的事情发生，希望通过某些方式来解决此问题。通过前面的了解，我们能想到利用读写锁来解决。Java 本身也提供了一种 `CopyOnWrite` 机制来处理这种问题，最终使得读操作始终能够并发执行，写操作是拷贝一份数组出来修改，修改后的数组为最新版本，将它赋值给内部的数组或链表的引用即可。这相比读写锁中对于读操作的控制粒度更低，因为写操作发生时，读操作依然可以进行。

Java 中提供有两个 `CopyOnWrite` 机制的类，分别是 `CopyOnWriteArrayList` 和 `CopyOnWriteArraySet`，它们分别实现了数组结构的 `List` 和 `Set` 的并发控制访问，利用拷贝的空间来换取锁粒度的降低，同样肯定对“读多写少”的情况非常适合。它们对于并发的写操作依然是排它的，不允许两个写操作同时进行，即使访问的不是同一个下标的元素也不允许并发修改，很多时候不要完全依赖它们来解决并发问题，尤其是当写操作非常多的时候，这样的拷贝会非常频繁。同样的道理，在写操作很多的情况下通常数组也不会太小，数组越大，拷贝出来的空间也会越大，此时就会产生大量的碎片垃圾，如果是那样还不如直接用锁。由此可以看出这些并发编程集合类是有条件的，我们需要理解其基本原理才能很好地利用它们。

同样的对于 `HashMap`，提供了一个叫 `ConcurrentHashMap` 的类（相对 `HashMap` 的使用，其 `Key` 和 `Value` 都不允许为空），它将 `Map` 的锁粒度默认分解为 16 段，每一个段又叫作一个 `Segment`，`Segment` 之间是不存在并发征用的（如果想要修改并行度，则需要调用重载构造方法，第 3 个参数是 `concurrencyLevel`，程序运行时将通过数字 1 不断左移操作，得到一个大于等于 `concurrencyLevel` 的值，使得数字满足 2 的整数次方）。

这样一来，在理论上锁就被打散了，同时内在的拷贝操作也被分解了，每次不用拷贝那么大的内存，读和写也不会产生互斥，写操作只会在同一个 `Segment` 上的请求才会互斥。基于这样的思想可以解决许多锁粒度问题。

JDK 1.6 中的 `get()` 操作，只是在得到 `HashEntry` 后读取 `Value` 时才会发生一次 `lock()` 操作，以保证读取到的 `Value` 值一定是最新的，但它并不会一进入 `get(Object)` 方法就开始加锁。

在 JDK 1.7 中修改为通过 `volatile` 来保证可见性，因此 JDK 1.7 对于读操作没有 `lock()` 的动作。

类似的集合类还有许多，例如 `ConcurrentLinkedDeque`、`ConcurrentLinkedQueue`、`ConcurrentSkipListMap`、`ConcurrentSkipListSet` 等，这里就不做一一介绍了，大家可以自行参看源码实现某些目的。

5.5 常见的并发编程工具

本节胖哥会用 Java 提供的工具类给大家演示一些场景的解决方案。

反过来讲,我们不仅要学会这些工具怎么用,而且应当理解其思想和原理,如果发现 Java 本身的实现不能满足你的应用场景需求,那么可以参考它的思想进行二次修改,以达到目的。

本节提到的工具,并非一种官方说法,只是胖哥认为它们像工具,它们实现了许多需要大量代码去完成的功能,现在利用它们来写代码。你可以将它们理解成一种小框架,在该框架架构下编程。

5.5.1 CountdownLatch

如果想要让多个线程同时去执行某些动作,这时 CountdownLatch 可以帮助我们来完成这样的动作。前面提到过用线程 join() 来实现等待某些线程结束,但是它仅仅能等待结束,这种方式需要逐个去等待结束。而 CountdownLatch 是等待一个信号量,可以为这个信号量设定一个数字,每个达到指定目标的线程会给信号量的值叠加 1,当信号量的值达到设定的数字时,等待信号量的线程就被激活了。



注意 当 10 个线程创建出来并顺序 start(), 并不意味着它们会一起跑,因为 start() 过程需要很多条本地代码运行,都需要 CPU 来调度,当运行到 run() 方法时,相互之间有一点时间差,这个时间差相对计算机运行速度来讲是很大的。

假设默认 10 个人赛跑的游戏分为两组,5 个人一组进行赛跑。当发令枪响时,10 个人可能几乎同时开跑,到达终点时,相互之间肯定有前后关系。编写代码如下:

代码清单 5-14 赛跑游戏

```
public class CountdownLatchTest {

    private final static int GROUP_SIZE = 5;

    private final static Random random = new Random();

    public static void main(String []args) throws InterruptedException {
        processOneGroup("分组 1");
        processOneGroup("分组 2");
    }

    private static void processOneGroup(final String groupName)
        throws InterruptedException {
        //等待所有队员准备就绪
        final CountdownLatch preCountDown = new CountdownLatch(GROUP_SIZE);
        final CountdownLatch startCountDown = new CountdownLatch(1);
        final CountdownLatch endCountDown = new CountdownLatch(GROUP_SIZE);
        System.out.println("====>\n分组: " + groupName + "比赛开始: ");
        for(int i = 0 ; i < GROUP_SIZE ; i++) {
            new Thread(String.valueOf(i)) {
                public void run() {
                    preCountDown.countDown(); //代表准备就绪
                }
            }.start();
        }
        startCountDown.countDown();
        endCountDown.await();
    }
}
```



```

        System.out.println("我是线程组: [" + groupName + "], 第"
            + this.getName() + " 号线程, 我已经准备就绪!");
        try {
            //等待裁判发出开始指令
            start_count_down.countDown();
            startCountDown.await();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        try {
            //运行一个随机时间, 代表赛跑的过程
            Thread.sleep(Math.abs(random.nextInt()) % 1000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("我是线程组: [" + groupName + "], 第"
            + this.getName() + " 号线程, 我已执行完成!");
        endCountDown.countDown();
    }
    }.start();
}
preCountDown.await(); //等待所有队员就位
System.out.println("各就各位, 预备!");
startCountDown.countDown(); //开始赛跑
try {
    endCountDown.await(); //等待多个赛跑者逐个结束
} catch (InterruptedException e) {
    e.printStackTrace();
}
System.out.println(groupName + "比赛结束!");
}
}

```

输出结果如下:

```

=====>
分组 1 比赛开始:
我是线程组: 【分组 1】, 第 0 号线程, 我已经准备就绪!
我是线程组: 【分组 1】, 第 1 号线程, 我已经准备就绪!
我是线程组: 【分组 1】, 第 2 号线程, 我已经准备就绪!
我是线程组: 【分组 1】, 第 3 号线程, 我已经准备就绪!
我是线程组: 【分组 1】, 第 4 号线程, 我已经准备就绪!
各就各位, 预备!
我是线程组: 【分组 1】, 第 0 号线程, 我已执行完成!
我是线程组: 【分组 1】, 第 1 号线程, 我已执行完成!
我是线程组: 【分组 1】, 第 2 号线程, 我已执行完成!
我是线程组: 【分组 1】, 第 4 号线程, 我已执行完成!
我是线程组: 【分组 1】, 第 3 号线程, 我已执行完成!
分组 1 比赛结束!
=====>
分组 2 比赛开始:
我是线程组: 【分组 2】, 第 0 号线程, 我已经准备就绪!
我是线程组: 【分组 2】, 第 1 号线程, 我已经准备就绪!
我是线程组: 【分组 2】, 第 3 号线程, 我已经准备就绪!
我是线程组: 【分组 2】, 第 4 号线程, 我已经准备就绪!
我是线程组: 【分组 2】, 第 2 号线程, 我已经准备就绪!
各就各位, 预备!
我是线程组: 【分组 2】, 第 0 号线程, 我已执行完成!
我是线程组: 【分组 2】, 第 1 号线程, 我已执行完成!
我是线程组: 【分组 2】, 第 3 号线程, 我已执行完成!
我是线程组: 【分组 2】, 第 4 号线程, 我已执行完成!
我是线程组: 【分组 2】, 第 2 号线程, 我已执行完成!
分组 2 比赛结束!

```

这个时候也许有的同学还是没看出什么门道来, 其实大家在这个时候千万不要仅仅关

注于结果好看。代码中比较关键的部分是在指挥官发出“各就各位，预备”信号时，每个线程都“已经准备好了”，当指挥官吹响哨子说：“分组 X 比赛结束！”时，每个线程都已经到达了终点。

如果要更加真实地模拟赛跑比赛，大家可以在线程中增加运行时间的记录，这样每个线程的起跑时间、到达终点时间、总时长都会记录下来。

在这段代码中把 `CountDownLatch` 定义为局部变量，是不是可以公共化？不行，它就局部用，如果 `CountDownLatch` 使用第 2 次就有问题了。对于反复用的一种场景怎么办？请看下一节的内容。

►► 5.5.2 CyclicBarrier

`CyclicBarrier` 是一种什么东西呢？有多个拦截的步骤时会用到它。

我们也说个生活中的小例子：一个导游带着大家玩，每当从一个地方准备去下一个地方的时候，导游总要点名以确保人都在，然后才会做下一个动作，这就相当于每次做下一个动作时，大家都是一起出发的。

这样说貌似和计算机还扯不上多大的关系，再说一个和计算机相关的例子。你的程序需要给几百上千台服务器做操作，这些服务器之间有一定的关系，它们被划分为很多个组，隶属于不同的环境中，操作中要求对其中任意一组服务器的操作可以并行完成，但组与组之间有一个顺序关系，必须先执行完一组再执行下一组，这样就必须等待上一组服务器全部执行完才能执行下一组。可以使用 `CyclicBarrier` 来解决问题。

代码清单 5-15 `CyclicBarrier` 的一个简单例子

```
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CyclicBarrier;

public class CyclicBarrierTest {
    private static final int THREAD_COUNT = 3;

    private final static CyclicBarrier CYCLIC_BARRIER = new CyclicBarrier(
        THREAD_COUNT, new Runnable() {
            public void run() {
                System.out.println("=====>我是导游，本次点名结束，准备走
下一个环节!");
            }
        });

    public static void main(String[] args) throws InterruptedException,
        BrokenBarrierException {
        for (int i = 0; i < THREAD_COUNT; i++) {
            new Thread(String.valueOf(i)) {
                public void run() {
                    try {
                        System.out.println("我是线程: "
                            + this.getName() + " 我们到达旅游地点!");
                        CYCLIC_BARRIER.await();
                        System.out.println("我是线程: "
                            + this.getName() + " 我们开始骑车!");
                        CYCLIC_BARRIER.await();
                        System.out.println("我是线程: "
                            + this.getName() + " 我们开始爬山!");
                        CYCLIC_BARRIER.await();
                        System.out.println("我是线程: "
                            + this.getName() + " 我们回宾馆休息!");
                        CYCLIC_BARRIER.await();
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    } catch (BrokenBarrierException e) {
                        e.printStackTrace();
                    }
                }
            }.start();
        }
    }
}
```



```

        System.out.println("我是线程: "
            + this.getName() + " 我们开始乘车回家!");
        CYCLIC_BARRIER.await();
        System.out.println("我是线程: "
            + this.getName() + " 我们到家了!");
    } catch (InterruptedException e) {
        e.printStackTrace();
    } catch (BrokenBarrierException e) {
        e.printStackTrace();
    }
}

).start();
}
}
}

```

输出结果如下：

```

我是线程: 0 我们到达旅游地点!
我是线程: 2 我们到达旅游地点!
我是线程: 1 我们到达旅游地点!
=====>我是导游，本次点名结束，准备走下一个环节!
我是线程: 1 我们开始骑车!
我是线程: 0 我们开始骑车!
我是线程: 2 我们开始骑车!
=====>我是导游，本次点名结束，准备走下一个环节!
我是线程: 2 我们开始爬山!
我是线程: 1 我们开始爬山!
我是线程: 0 我们开始爬山!
=====>我是导游，本次点名结束，准备走下一个环节!
我是线程: 0 我们回宾馆休息!
我是线程: 2 我们回宾馆休息!
我是线程: 1 我们回宾馆休息!
=====>我是导游，本次点名结束，准备走下一个环节!
我是线程: 1 我们开始乘车回家!
我是线程: 0 我们开始乘车回家!
我是线程: 2 我们开始乘车回家!
=====>我是导游，本次点名结束，准备走下一个环节!
我是线程: 2 我们到家了!
我是线程: 1 我们到家了!
我是线程: 0 我们到家了!

```

在这里，我们需要看到内在的特征是，导游始终在每个人都到达目的地时就开始说话了，而每个人到达目的地就是通过 `CYCLIC_BARRIER.await()` 来告诉导游的，当所有人都调用了这个方法后，导游便知道这个步骤就做完了。换句话说，如果导游还没有说明当前步骤结束，那么所有的线程都将等待。因此，在输出的结果中才会出现每个步骤所有的线程都报到后再继续下一个步骤，而不是每个线程自己做完后直接进入下一个步骤。



注意 在例子代码中，每个步骤仅仅是简单的输出语句，没有执行时间，但这并不影响结论，因为无论怎么测试，最终的结果都是听导游的安排。

大家也可以修改代码，让每个步骤输出前 `sleep` 一个随机时间再看看结果。

`CyclicBarrier`，的内在实现也是一个计数器，这个计数器更像是一种轮训式的计数器，当然最简单的方式是利用叠加的数据与管理对象的总数进行求模，查看其结果是否等于 0 就能实现了。

5.5.3 Semaphore

Semaphor, 看名称就知道是信号量, 其实它也像一个屏障一样, 只是它拦截的点在 Java 语言这个层面。它同样设定一个数字, 当请求的数量达到指定的数字时, 就将请求拦截在门外。如果有线程释放了资源, 则会放一个请求进去。

其实在许多容器中, 都会利用这一点, 通常也会在这个基础上做二次改造。例如, 有些时候我们希望控制应用系统下载大文件的并发度, 因为这些请求对系统的资源开销很大, 我们并不想在 Tomcat 的线程池中减少数量, 这样可能会对其他业务造成许多影响, 所以就自己想自己单独对这个下载操作做流控处理。如果自己用 wait/notify 来实现也是可以的, 只是相对会比较麻烦一点, 我们来看看利用这个信号量是怎么做的。

现在同时运行 20 个线程, 但是拦截器要求最多 5 个线程在运行中, 代码如下:

代码清单 5-16 Semaphore 例子

```
public class SemaphoreTest {

    private final static Semaphore MAX_SEMA_PHOE = new Semaphore(5);

    private final static SimpleDateFormat DEFAULT_DATE_FORMAT
        = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");

    public static String getDateTime() {
        Date datetime = Calendar.getInstance().getTime();
        return DEFAULT_DATE_FORMAT.format(datetime);
    }

    public static void main(String[] args) {
        for (int i = 0; i < 20; i++) {
            final int num = i;
            final Random radom = new Random();
            new Thread() {
                public void run() {
                    boolean acquired = false;
                    try {
                        MAX_SEMA_PHOE.acquire();
                        acquired = true;
                        System.out.println("我是线程: " + num +
                            " 我获得了使用权! " + getDateTime());
                        Thread.sleep(1000 + (radom.nextInt() & 5000));
                        System.out.println("我是线程: " + num +
                            " 我执行完了! " + getDateTime());
                    } catch (Exception e) {
                        e.printStackTrace();
                    } finally {
                        if (acquired) {
                            MAX_SEMA_PHOE.release();
                        }
                    }
                }
            }.start();
        }
    }
}
```

每个线程在获取到执行权时会输出一条语句, 执行完以后也会输出一条语句, 根据这样的代码逻辑, 我们来看看输出结果是什么样的。

```
我是线程: 0 我获得了使用权! 2013-09-24 23:21:35
我是线程: 2 我获得了使用权! 2013-09-24 23:21:35
我是线程: 4 我获得了使用权! 2013-09-24 23:21:35
```



```

我是线程: 6 我获得了使用权! 2013-09-24 23:21:35
我是线程: 8 我获得了使用权! 2013-09-24 23:21:35
我是线程: 2 我执行完了! 2013-09-24 23:21:36
我是线程: 10 我获得了使用权! 2013-09-24 23:21:36
我是线程: 6 我执行完了! 2013-09-24 23:21:36
我是线程: 12 我获得了使用权! 2013-09-24 23:21:36
我是线程: 12 我执行完了! 2013-09-24 23:21:36
我是线程: 14 我获得了使用权! 2013-09-24 23:21:36
我是线程: 14 我执行完了! 2013-09-24 23:21:36
我是线程: 16 我获得了使用权! 2013-09-24 23:21:36
我是线程: 8 我执行完了! 2013-09-24 23:21:36
我是线程: 18 我获得了使用权! 2013-09-24 23:21:36
我是线程: 0 我执行完了! 2013-09-24 23:21:40
我是线程: 1 我获得了使用权! 2013-09-24 23:21:40
我是线程: 4 我执行完了! 2013-09-24 23:21:40
我是线程: 3 我获得了使用权! 2013-09-24 23:21:40
我是线程: 1 我执行完了! 2013-09-24 23:21:40
我是线程: 5 我获得了使用权! 2013-09-24 23:21:40
我是线程: 5 我执行完了! 2013-09-24 23:21:40
我是线程: 7 我获得了使用权! 2013-09-24 23:21:40
我是线程: 18 我执行完了! 2013-09-24 23:21:40
我是线程: 9 我获得了使用权! 2013-09-24 23:21:40
我是线程: 9 我执行完了! 2013-09-24 23:21:40
我是线程: 11 我获得了使用权! 2013-09-24 23:21:40
我是线程: 16 我执行完了! 2013-09-24 23:21:40
我是线程: 13 我获得了使用权! 2013-09-24 23:21:40
我是线程: 10 我执行完了! 2013-09-24 23:21:41
我是线程: 15 我获得了使用权! 2013-09-24 23:21:41
我是线程: 7 我执行完了! 2013-09-24 23:21:41
我是线程: 17 我获得了使用权! 2013-09-24 23:21:41
我是线程: 11 我执行完了! 2013-09-24 23:21:41
我是线程: 19 我获得了使用权! 2013-09-24 23:21:41
我是线程: 13 我执行完了! 2013-09-24 23:21:41
我是线程: 15 我执行完了! 2013-09-24 23:21:45
我是线程: 17 我执行完了! 2013-09-24 23:21:45
我是线程: 3 我执行完了! 2013-09-24 23:21:45
我是线程: 19 我执行完了! 2013-09-24 23:21:45

```

代码中保证让每个线程的执行时间在 1s 以上，并随机叠加 5s 以内的时间，这样就更容易看得出时间上的区别，可以发现前面 5 个线程都是一起做的，紧跟着每个线程执行完，就有另一个线程进入了执行区域。

再回头来想想：使用 wait()/notify() 如何实现呢？胖哥结合这种方式，再利用 CAS 简单实现了一个流控处理中心，代码如下：

代码清单 5-17 使用 wait/notify 实现

```

public class SemaphoreByWaitNotify {

    private final static int QUERY_MAX_LENGTH = 5;

    private final static int THREAD_COUNT = 20;

    private final static AtomicInteger NOW_CALL_COUNT = new AtomicInteger(0);

    private final static Object LOCK_OBJECT = new Object();

    public static void main(String[] args) {
        final Random radom = new Random();
        for (int i = 0; i < THREAD_COUNT; i++) {
            new Thread(String.valueOf(i)) {
                public void run() {
                    tryToLock(); //尝试获取到执行权限
                    System.out.println(this.getName() + "=====我开始做操作

```

了!");

```

        try {
            Thread.sleep(100 + (radom.nextInt() & 3000));
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println(this.getName() + "=====操作结束了!");
        tryToUnLock(); //释放锁
    }
    }.start();
}

private static void tryToUnLock() {
    NOW_CALL_COUNT.getAndDecrement();
    synchronized(LOCK_OBJECT) {
        LOCK_OBJECT.notify();
    }
}

private static void tryToLock() {
    int tryTimes = 0;
    int nowValue = NOW_CALL_COUNT.get();
    while(true) {
        if(nowValue < QUERY_MAX_LENGTH
            && NOW_CALL_COUNT.compareAndSet(nowValue, nowValue + 1)) {
            break; //获取到了锁,跳出循环
        }
        if(tryTimes % 3 == 0) { //尝试3次若不成功,则休眠
            waitForObjectNotify();
        }
        nowValue = NOW_CALL_COUNT.get();
        tryTimes++;
    }
}

private static void waitForObjectNotify() {
    synchronized(LOCK_OBJECT) {
        try {
            LOCK_OBJECT.wait(500); //最多等待500ms后重试
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
}
}

```

虽然代码多了一点点,但也可以封装起来,而且更加灵活一点点。有人会问胖哥:将代码做得这么麻烦,为什么不直接用个大的 `synchronized` 呢?其实应该能看得出来,代码中在细化锁的粒度和进行乐观处理,用 CAS 去尝试几次(代码中是3次),如果不成功,则进入休眠,将锁释放出来,我们乐观地认为更多的线程处于 `RUNNING`、`TIME_WAITING` 状态。

反之,虽然用一个大的 `synchronized` 可以达到某些目的,但其余的线程都会阻塞在外面,将被完全串行化,这个过程将会导致锁优化策略基本失效,采用悲观的重量级锁,那样开销就可能会更大。

➤➤ 5.5.4 其他工具简介

除了 Java 本身提供的工具类外,还有很多开源框架也做了扩展,我们自己也可以再次扩展,因此很多时候学会的不仅仅是如何利用工具,还有它提供的方法与思路。

本小节将扩展说明一些 Java 本身提供的常见并发编程工具类,然后再谈谈它们有什么

用途，或者说为何要这样做。

(1) Executors

该类其实就是一个提供静态方法的空壳，在大多数情况下是用它来创建线程池、调度的。例如：`Executors.newFixedThreadPool(10)`用于创建一个长度为 10 的线程池，但并不意味着创建线程池和调度池就必须用它，没有它也照样可以实现，比如可以用 `new ThreadPoolExecutor()`、`new ScheduledThreadPoolExecutor()`传入指定的参数达到同样的目的。`Executors` 的静态方法也是通过间接调用这样的构造方法来完成的。

既然如此，Java 为什么要多此一举提供这样一个静态方法来完成线程池的创建动作呢？

胖哥认为这是一种提高 API 可读性的方法，因为当我们使用 `ThreadPoolExecutor`、`ScheduledThreadPoolExecutor` 创建时，通常需要传入非常多的参数，不同的参数可能会导致不同的运行结果，有些参数本身也是一个复杂的对象，它的创建也需要自己写代码来构造。由参数细节不同导致结果不一样的方式很容易让人忘记具体参数是什么意思，那么 Java 将常见的几种模型抽象出来，每种模型都会有一个非常简单的入口参数，同时也用自己的特定名称来标识，这样写代码就不再过多关注于太多的参数细节了。

`Executors` 的常见方法介绍与说明如下：

- ◎ `Executors.newFixedThreadPool(int)`，创建一个线程数固定的线程池，假如传入 10，那么线程并行度最大为 10，初始化为 0，第一个任务创建时启动一个线程处理，如果更多任务导致一个线程处理不过来，则创建新的线程，最多为 10 个线程，但是线程数在 `shutdown()`以前只增不减，如果 10 个线程依然处理不过来，那么更多的任务将会被放在阻塞队列中，默认阻塞队列为一个无界队列。
- ◎ `Executors.newFixedThreadPool(int, ThreadFactory)`，操作同上，只是 `ThreadFactory` 是自定义的，它本身是一个接口，需要实现 `new Thread(Runnable r)`方法，负责返回创建后的线程，此时可以自己设置相应的线程参数，例如设置统一的线程名前缀，方便定位跟踪。如果不是自定义的，则默认会采用 `Executors` 里面的内部类 `DefaultThreadFactory` 来处理。
- ◎ `Executors.newCachedThreadPool()`，创建一个线程池，默认对线程并行的数量没有上限，如果出现瞬间并发，那么这些并发的请求可能都会分配一个线程，线程数会瞬间增加，如果有空闲的线程，则会去执行新提交的任务；如果没有任何任务提交，那么这些线程会默认保持 60s 时间，60s 内没有获取到任务的线程一般会被回收。
- ◎ `Executors.newCachedThreadPool(ThreadFactory)`，同上，可以自定义 `ThreadFactory`。
- ◎ `Executors.newSingleThreadExecutor()`，创建一个长度为 1 的线程池，表面上好像没啥用，但它的目的是线程复用，提交的都是任务。另外，可以通过它来模拟一些资源不够用的情况。
- ◎ `Executors.newSingleThreadExecutor(ThreadFactory)`，同上，可以自定义 `ThreadFactory`。
- ◎ `Executors.newScheduledThreadPool(int)`，创建调度池，调度池是基于线程池来完成的，在这个基础上做了进一步的包装，只是提供的功能要求定时调度。与之类似的方法还有 `newScheduledThreadPool(int, ThreadFactory)`、`newSingleThreadScheduledExecutor()`和 `newSingleThreadScheduledExecutor(ThreadFactory)`。

调度池和线程池的细节将在 5.6 节中详细阐述。

(2) Exchanger

Exchanger 表示交换中心，而且是一个线程安全的交换中心，这种交换是相互的。也就是说，如果有非常多的线程在内部做数据交换，那么可以保证的是如果 A 的数据给了 B，B 的数据肯定会给 A，而不是交给其他的线程。

若 A 发起交换，但是没有线程与它交换，此时线程会处于 WAITING 状态（当然同样也可以设定超时），直到有线程与它进行交换为止。本书光盘中/src/chapter05/tools/目录下的 ExchangerTest.java 为该工具的测试类。

(3) CompletionService

前文中使用线程 Join 来判定多个线程是否都已经结束，当想要复用线程时，对于大批量的操作都是以任务方式交给线程池来调度的（并不是每个任务都单独开一个线程），若想看到所有的任务是否都结束了，则可以使用“CompletionService”。

线程的 Join 操作只是对线程单纯的顺序 Join，但是这个顺序并不一定是线程真正结束的顺序，而 CompletionService 可以按照线程结束的顺序给我们返回结果。另外，一个更大的区别在于线程的 Join 操作是针对线程的合并操作，这将意味着为每个任务创建对应的线程，如果任务数很多将导致创建大量的线程来处理，而 CompletionService 是基于线程池的任务，无论任务数量多少，线程的数量总是可控的。

例如，利用多个线程对多台服务器做某种测试操作，每个测试结果返回的时间由子线程得到，我们最终需要按照时间顺序将返回的结果串联为整体日志。用 Join 无法保证多个线程测试结果返回的顺序，如果要求日志的时间顺序，则需要做排序才能达到目的，但是 CompletionService 不需要这样。

CompletionService 利用 LinkedBlockingQueue 来做一个 FIFO 的阻塞队列，重新提供任务的 submit 方法，在 submit 方法中将任务包装为 QueueingFuture 类型，该 QueueingFuture 继承于 FutureTask 类，在 FutureTask 中任务运行完成后会调用一个 done() 方法，QueueingFuture 类重写了 done() 方法将完成的任务的 Future 对象写入到了 LinkedBlockingQueue 中。

因此 CompletionService 中获取到的 Future 对象就是完成的任务，通过 Future 的 get() 获取任务的实体，且该方法肯定不会阻塞，因为只有结束后的任务对应的 Future 才会放到 LinkedBlockingQueue 中，若发生阻塞肯定是阻塞在从队列中通过 take() 方法获取 Future 的过程中（此时队列中没有任何一个元素，即没有任何一个完成的任务）。本书光盘中/src/chapter05/tools/目录下的 CompletionServiceTest.java 是一个简单的例子。

(4) fork/join

fork/join 是从 JDK 1.7 开始才提供的，它是一种分治策略，希望将复杂的问题逐渐分治到多个子任务中去执行，进行逐层分治处理，不断创建子任务，而每个层次的任务都在等分派的子任务结束将结果进行合并。

这个过程看起来写代码很复杂，但是在 fork/join 框架下就很简单了，代码中只需要继承 RecursiveAction 或 RecursiveTask 抽象类，然后实现内部的接口 compute() 即可。如果是实现 RecursiveAction 抽象类的 compute() 方法没有返回结果，则可以自己将值写入到某些内部属性中。与之不同的是，继承 RecursiveTask 抽象类的时候，重写的 compute 方法需要有返回值。

这种分治任务通常是一些十分复杂的问题和任务，单个线程计算在效率上无法满足需求，有些时候我们会自己通过分解任务到多个线程来解决，但如果任务量较大将会创建无

限多的线程。当我们尝试使用固定大小的线程池来解决时，由于父任务在执行中会占用一个线程等待子任务返回结果，线程数饱和后，子任务得不到调度在等待线程资源，则可能会出现“饥饿等待”的问题。

fork/join 通常允许执行的线程数与 CPU 数相同，也解决了“饥饿等待”的问题，但它不一定适用于所有的场景，它中间会有许多等待、切换及多余的计算开销，我们要思考这种做法是否值得，例如当我们使用一些数据的简单叠加计算、求最大值、求最小值代码来做测试时，通常效果不是特别明显，甚至会比单线程更慢，因为数字计算本身就是 CPU 最擅长的，如果在计算过程中增加一个休眠来代表运行过程是复杂的，也许单线程和多线程的区别就出来了)。另外，fork/join 是在多核 CPU 下使用的，单核 CPU 的效果只会越来越差。在本书中，基于 RecursiveTask 胖哥编写了一个简单的求最大值的程序，大家可以修改代码后再测试，但是请注意：需要将 JDK 换到 JDK 1.7 以上的版本中才能运行该程序（类似的，在 JDK 1.7 中还增加了一个 java.util.concurrent.Phaser，它也是并发中很有意思的工具）。

5.6 线程池&调度池

Java 提供了默认的线程池与调度池，那么它们是如何管理的呢？是不是和 OS 内核有关系？本节胖哥将根据自己对源码的阅读给大家一些引导。在了解线程池和调度池以前，希望大家先来了解阻塞队列，因为它与线程池是息息相关的。

»»

5.6.1 阻塞队列模型

在上一节，我们看到了很多地方使用 AQS，其实在 JDK 1.6 以后提供了 7 种队列模型，这为解决不同场景的问题提供了基础。在本小节，将就阻塞队列本身的功能做一个介绍。阻塞队列的接口为 BlockingQueue，它的实现类有很多，如图 5-29 所示。

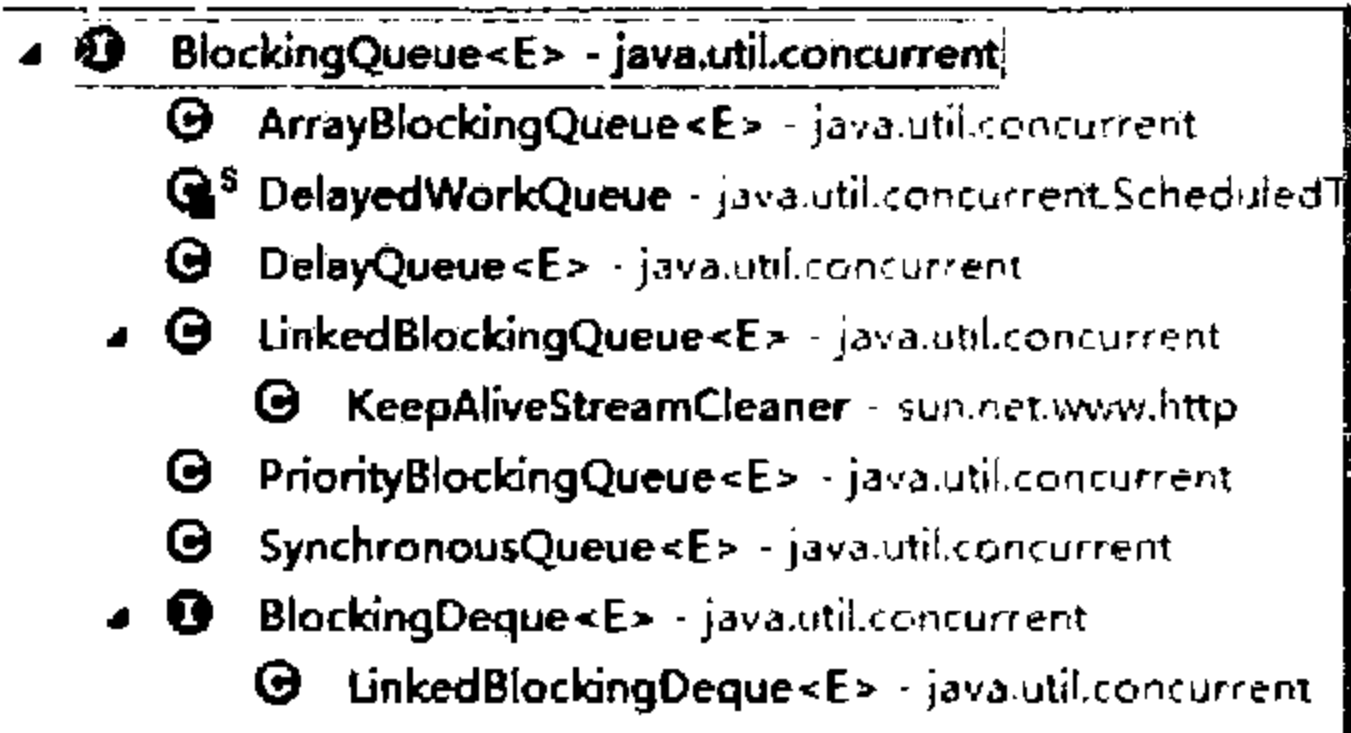


图 5-29 BlockingQueue 的实现类

- ◎ LinkedBlockingQueue，基于链表结构的阻塞队列，可以通过重载的构造方法设定最大值，但默认是 Integer.MAX_VALUE，相当于无界。
- ◎ ArrayBlockingQueue，功能类似于 LinkedBlockingQueue，区别在于它基于数组实现。
- ◎ PriorityBlockingQueue，支持优先级调度，无界阻塞队列，排序基于 compareTo 或 Comparator 来完成。
- ◎ SynchronousQueue，生产者与消费者之间的传递者，通过它可以实现生产者与消费者之间的访问平衡（生产者放入队列的数据在没有消费者获取时会等待，直到有消费者消费数据，这样不会造成生产者过快而导致阻塞队列中存放大量的数据），在前文介绍的 Executors.newCachedThreadPool()源码中就使用了这种队列。

- ◎ DelayQueue, 支持延迟获取元素, 在系统调度中使用较多, 或者应用于某些条件变量达到要求后需要做的事情。
- ◎ LinkedTransferQueue, 提供链表结构的 TransferQueue, 不要与 SynchronousQueue 内部的 TransferQueue 扯在一起哦。
- ◎ LinkedBlockingDeque, 双向阻塞队列, 也就是可以从队列的头部、尾部发生读写操作, 而不仅仅是 FIFO 或 LIFO 操作。

这里的很多队列可以直接拿出来用, 更为细节的概念大家可以参考资料及源码。在下面两个小节中, 我们会逐步阐述 Java 本身的线程池和调度池是如何使用一些阻塞队列的。在这个过程中, 希望大家能学习到与工作相关的东西, 理解 Java 所提供的线程池本身的优势与缺陷, 通过队列来处理工作中的许多问题, 而同时又能在不同的场景选择不同的队列模型。

►► 5.6.2 ThreadPoolExecutor

大家都知道这是 Java 默认提供的线程池, 那么什么叫作线程池呢? 胖哥的理解就是可以控制线程创建、释放, 并通过某种策略尝试复用线程去执行任务的一个管理框架, 从而实现线程资源与任务之间的一种平衡。为此, 你可以简单地认为程序需要执行的任务交给线程池就像生产者生产任务, 而线程池执行任务就像消费者一样, 这就是典型的生产者/消费者模型, 不过其中的资源征用和调配方式存在许多细节, 因此我们就需要了解这些细节, 以便于更好地驾驭它, 甚至改造它。

在 JDK 1.5 以后推出 ThreadPoolExecutor 这个默认的线程池, 是纯 Java 语言实现的, 基本思想是很不错的, 大家可以直接使用或参考它的模型来实现线程池。在本节, 大部分篇幅是讲解一些 ThreadPoolExecutor 基本的原理, 而不是在使用上 (如果还没有使用过线程池, 请大家自行写写代码实现), 使用层面的细节更多会在原理上体现出来。

第 1 次看这部分内容的小伙伴会很痛苦、很纠结, 因为线程池代码对于大部分学 Java 的人来讲不太容易读懂。不过胖哥自己也不是第一次就读懂这些代码的, 甚至于都不是第一次读完后就知道线程池是干什么用的, 胖哥也是不断反反复复阅读代码, 加上自己的工作中使用线程池, 才逐渐感悟到线程池原来是这样的, 现在也经常拿出来翻开, 也有许多新的体会。胖哥想说的是, 这是一个坚持的过程, 并不是因为难而就不去做, 有难度才能体现价值, 胖哥在本节写下了自己的理解和源码的分析, 并从中阐述一些参数在源码中的逻辑, 但是代码需要大家对照着一起看, 自己去坚持, 这样才会有效果。

前面提到, 如果要创建一个线程池, Executors 的许多方法可以实现, 使用较多的是 Executors.newFixedThreadPool(int), 用来获取一个固定数量的线程池, 那么就从它开始介绍吧, 再顺便介绍与 Executors.newCachedThreadPool() 的区别。首先看入口方法, 如图 5-30 所示。

```
public static ExecutorService newFixedThreadPool(int nThreads) {
    return new ThreadPoolExecutor(nThreads, nThreads,
                                  0L, TimeUnit.MILLISECONDS,
                                  new LinkedBlockingQueue<Runnable>());
}
```

图 5-30 创建一个指定长度的线程池

假如传入了 10，则相当于调用代码：

```
new ThreadPoolExecutor(10, 10, 0L, TimeUnit.MILLISECONDS,
    new LinkedBlockingQueue<Runnable>());
```

这里只是构造方法的入参，具体内容还得看 `ThreadPoolExecutor` 的构造方法是如何定义的，见图 5-31。

下面提到的源码截图都是在 JDK 1.6 中得到的，在 JDK 1.7 中许多源码已经被修改，但基本的道理是一致的。

```
public ThreadPoolExecutor(int corePoolSize,
    int maximumPoolSize,
    long keepAliveTime,
    TimeUnit unit,
    BlockingQueue<Runnable> workQueue,
    RejectedExecutionHandler handler) {
    this(corePoolSize, maximumPoolSize, keepAliveTime, unit, workQueue,
        Executors.defaultThreadFactory(), handler);
}
public ThreadPoolExecutor(int corePoolSize,
    int maximumPoolSize,
    long keepAliveTime,
    TimeUnit unit,
    BlockingQueue<Runnable> workQueue,
    ThreadFactory threadFactory,
    RejectedExecutionHandler handler) {
    if (corePoolSize < 0 ||
        maximumPoolSize <= 0 ||
        maximumPoolSize < corePoolSize ||
        keepAliveTime < 0)
        throw new IllegalArgumentException();
    if (workQueue == null || threadFactory == null || handler == null)
        throw new NullPointerException();
    this.corePoolSize = corePoolSize;
    this.maximumPoolSize = maximumPoolSize;
    this.workQueue = workQueue;
    this.keepAliveTime = unit.toNanos(keepAliveTime);
    this.threadFactory = threadFactory;
    this.handler = handler;
}
```

图 5-31 `ThreadPoolExecutor` 的构造方法

从这个构造方法中能看到一些简单的赋值，也能看到其使用了默认的 `ThreadFactory` 来处理。为了更容易地看后面的代码，胖哥根据自己对这部分代码的理解，将构造方法中赋值的参数做一个初步的介绍。

- ◎ `corePoolSize`，线程池的主要线程数量。当有任务请求提交时，如果线程数量（`poolSize`）小于 `corePoolSize` 值，则会直接创建新的线程放入 `workers` 中（该 `workers` 是一个 `HashSet`）；如果线程数量大于这个值，则会尝试放入等待队列 `workQueue`（一个 `BlockingQueue` 队列）中；如果写入等待队列失败，则会进入 `addIfUnderMaximumPoolSize(Runnable)` 方法做 `maximumPoolSize` 的判定和处理。
- ◎ `maximumPoolSize`，进入 `addIfUnderMaximumPoolSize(Runnable)` 方法后，如果发现线程数量小于 `maximumPoolSize` 值，则会创建线程放入 `workers` 中。这是 `CachedThreadPool` 的关键，在固定线程数的线程池中不会出现这种情况。
- ◎ `workQueue`，是一个 `BlockingQueue` 队列，而 `BlockingQueue` 只是一个接口，具体的实现类可以是 5.6.1 节中提到的队列中的其中之一，在调用 `newFixedThreadPool()` 时默认采用 `LinkedBlockingQueue`，调用 `newCachedThreadPool()` 时默认采用 `SynchronousQueue`。
- ◎ `keepAliveTime`，其主要用途是线程池中的线程尝试从阻塞队列 `workQueue` 中获取

数据时会使用到（后文中会提到），传入参数的单位是秒，在线程池的构造方法中会根据入口参数的值和单位自动转换为纳秒值。

- ◎ `threadFactory`, `ThreadFactory` 接口的实现类，默认为 `DefaultThreadFactory`，也可以自定义。
- ◎ `handler`, `RejectedExecutionHandler` 接口，需要实现 `rejectedExecution(Runnable, ThreadPoolExecutor)` 方法。当 `addIfUnderMaximumPoolSize(Runnable)` 失败后，会调用它来处理，也就是当任务无法放到执行队列，也无法放入等待队列，而且线程数已经大于 `maximumPoolSize` 值时，需要做一个丢弃处理，实现类就是丢弃处理的回调。`RejectedExecutionHandler` 默认有 4 个实现类，分别是：
 - `AbortPolicy`，这个方法直接抛出一个 `RejectedExecutionException` 异常。
 - `DiscardPolicy`，这个方法什么也不做，外部不会感知到异常，任务自然被直接丢弃。
 - `DiscardOldestPolicy`，如果线程池没有发生 `shutdown` 操作，则会尝试从阻塞队列（`workQueue`）中取出一个任务，然后通过线程池的 `execute` 方法将任务放进去。这种方法依然有问题——如果有一个任务被取出，则意味着这个任务会被无辜地丢掉。无论是否从队列中取出任务，线程都会尝试执行 `execute` 方法，如果一直得不到空闲的位置，则会一直这样递归调用，线程栈会不断递增，理论上有可能导致 `StackOverflow` 问题，只是我们通常乐观地认为不会那么“倒霉”而已。
 - `CallerRunsPolicy`，如果线程池还没有被 `shutdown`，则直接调用任务的 `run()` 方法，不过并没有开启一个新的线程或交给线程池来调度，这个过程是直接使用生产者的当前线程来运行的。

其实丢弃时不论哪一种处理都不是很理想，我们最希望的是这种事情不要发生，发生了就通常说明生产者太快，或者消费者处理太慢，此时可以尝试提升单个任务的处理速度或增加线程数来解决，若机器的资源利用已经达到极限，且没有条件增加服务器，这个时候自己只好自己想想办法，例如将生产者阻塞，让消费者来不及处理时，生产者也停一下。

看到这些参数的细节介绍，其实不用看代码也大概知道是怎么回事了，不过我们还是看看代码来验证上面所提到的参数意思。

现在来看看线程池的任务添加过程。对线程池 `ThreadPoolExecutor` 的访问，最终会访问其中的 `execute` 方法（可直接访问或间接通过某些方法访问，例如 `invokeAll()` 是间接访问），自然的要从这个方法开始看起，如图 5-32 所示。

```
public void execute(Runnable command) {
    if (command == null)
        throw new NullPointerException();
    if (poolSize >= corePoolSize || !addIfUnderCorePoolSize(command)) {
        if (runState == RUNNING && workQueue.offer(command)) {
            if (runState != RUNNING || poolSize == 0)
                ensureQueuedTaskHandled(command);
        }
        else if (!addIfUnderMaximumPoolSize(command))
            reject(command); // is shutdown or saturated
    }
}
```

图 5-32 `ThreadPoolExecutor` 的 `execute` 方法（JDK 1.6 版本）

这段代码写得很奇怪，首先看最外层的判定：

```
if(poolSize >= corePoolSize || !addIfUnderCorePoolSize(command)) {.....}
```

此逻辑是两个 `if` 条件任意一个成立，就能进入 `body` 部分，也就是当 `poolSize` 达到了

corePoolSize 的大小或者 addIfUnderCorePoolSize(command) 执行返回 false 时会进入 body 部分。不过这样的写法还有一层隐藏的意思，即只有 poolSize < corePoolSize 成立时才会调用 addIfUnderCorePoolSize(command) 方法，反之则不会。换句话说，只有当前线程池中的线程数小于 corePoolSize 的大小时才会调用该方法。通过这一点，应该猜到此方法主要是用来增加线程的，来看看代码截图，如图 5-33 所示。

```
private boolean addIfUnderCorePoolSize(Runnable firstTask) {
    Thread t = null;
    final ReentrantLock mainLock = this.mainLock;
    mainLock.lock();
    try {
        if (poolSize < corePoolSize && runState == RUNNING)
            t = addThread(firstTask);
    } finally {
        mainLock.unlock();
    }
    if (t == null)
        return false;
    t.start();
    return true;
}

private Thread addThread(Runnable firstTask) {
    Worker w = new Worker(firstTask);
    Thread t = threadFactory.newThread(w);
    if (t != null) {
        w.thread = t;
        workers.add(w);
        int nt = ++poolSize;
        if (nt > largestPoolSize)
            largestPoolSize = nt;
    }
    return t;
}
```

图 5-33 addIfUnderCorePoolSize 操作

在 addIfUnderCorePoolSize(command) 里面首先加锁，然后在锁的区域内再一次判定条件：poolSize < corePoolSize，若成立则会执行 addThread(Runnable) 方法来添加线程，并返回线程的引用。代码中名称为“t”，若“t”不为空，则调用 t.start() 启动线程（线程池也是这样启动线程的）。

在调用该方法前已经隐藏地判定过 poolSize < corePoolSize，外部的判定是没有加锁的，为何这里还要判定呢？这样做是否多此一举？

这个看似不是问题的问题或许是一种技巧，在外部不加锁判定的开销是很低的，不过外部判定由于没有加锁无法确保安全，可能会导致出现 Race condition 现象。

或许可以换个角度考虑问题，有时候我们会乐观地认为如果外部判定成立，则进入该方法后在锁下进行判定成立的概率就已经很高了，如果外部判定不成立就不会进入这个方法来加锁处理，在固定长度的线程池中，其实初始化时线程池中没有任何线程，线程会不断地增加，达到一定程度后就不会再进入这个方法进行加锁判定了，这样的开销是值得的。反之，如果没有前一个步骤，那么每一次判定就必须要用锁，或者说没有选择的余地，自然征用的概率就要更高一些。这种手段十分类似于后文中要介绍到的双重锁判定，通过双重锁判定可以大幅度降低锁的征用。

这段代码也说明了另一个问题——如果同时存在多个线程并发进入 addIfUnderCorePoolSize(Runnable) 方法，那么经过该方法内部严格的串行判定，或者说进入该方法的生产者线程也未必都能成功创建线程池的线程，如果线程没有创建成功，则会返回 false。

思考：addThread(Runnable r) 创建的线程到底与传入的任务有何关系？

从代码截图 5-33 中可以看到, 创建线程时会使用 ThreadFactory 的 newThread(Runnable r)方法, 入口参数 Runnable 并不是业务中提交的任务, 而是经过了 ThreadPoolExecutor 的内部类 Worker 包装后的任务。换句话说, 业务中提交给线程池的任务作为 Worker 的一个属性存在。

默认的 ThreadFactory 是 Executors.DefaultThreadFactory, 它主要做的工作就是简单创建一个 Thread, 并将传入的 Runnable 作为 Thread 的入口参数, 这样的 Runnable 参数将作为线程执行的 target 目标, 线程运行时就会调用 target 的 run()方法。

而内部类 Worker 也是一个基于 Runnable 接口的实体类, 在它的构造方法中需要传入一个 Runnable 的实体对象, 这个对象是业务中真正提交的任务, 这个任务作为内部的一个 firstTask 属性存在, 它将被当成该线程的第 1 个任务执行。

下面我们简单归纳一下 ThreadFactory 创建线程、Worker、用户任务之间的关系, 如图 5-34 所示。

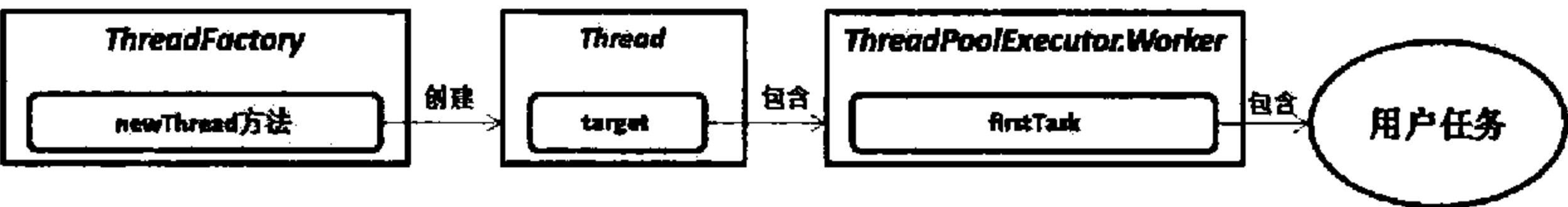


图 5-34 线程池中对象之间的关联关系

线程池中添加第 1 个任务时创建线程的逻辑到这里基本清楚了, 即使不清楚, 也应当知道了线程池内在的包装关系。现在回到图 5-32 中, 如果 poolSize >= corePoolSize 或线程池创建失败, 就会执行以下逻辑:

```
if (runState == RUNNING && workQueue.offer(command)) {
    if (runState != RUNNING || poolSize == 0)
        ensureQueuedTaskHandled(command);
}
else if (!addIfUnderMaximumPoolSize(command))
    reject(command);
```

这个逻辑层次有点深, 还是先看最外层, 状态为 RUNNING 才会执行 workQueue.offer(command)方法, 否则会直接执行 addIfUnderMaximumPoolSize(command)方法。同样 workQueue.offer(command)返回 false, 也会执行 addIfUnderMaximumPoolSize(command)方法。主体逻辑清楚后, 现在就来理清楚一些细节。

RUNNING 是运行时状态, 当线程池调用了 shutdown()方法后才会改变状态。换句话说, 当线程池调用了 shutdown()方法后, 再向线程池中提交任务时, 内部逻辑就完全不是我们想象中的那样了。

再来看看这个 workQueue.offer(command), 在前文中提到, workQueue 是一个阻塞队列, workQueue.offer(command)就是尝试将任务放入阻塞队列中, 在一个有容量限制的阻塞队列中, 如果达到了阻塞队列的上限, 则会返回 false, 否则返回 true。

当使用固定线程容量的线程池 (newFixedThreadPool(int)) 时, 在默认情况下这个阻塞队列是一个没有长度限制的 LinkedBlockingQueue, 所以通常它不会返回 false, 当然这些都是可以通过参数来修改的。

当使用 Cache 线程的线程池时 (newCachedThreadPool()), 默认会使用 SynchronousQueue, 提交任务的生产者如果发现没有任何消费者线程等待消费, 则直接返回 false, 在瞬间并发时, 线程大多处于忙碌状态, 这里返回 false 的概率将会非常高。

当真正返回 false 时，就会执行如图 5-35 所示的代码。

```
private boolean addIfUnderMaximumPoolSize(Runnable firstTask) {
    Thread t = null;
    final ReentrantLock mainLock = this.mainLock;
    mainLock.lock();
    try {
        if (poolSize < maximumPoolSize && runState == RUNNING)
            t = addThread(firstTask);
    } finally {
        mainLock.unlock();
    }
    if (t == null)
        return false;
    t.start();
    return true;
}
```

图 5-35 addIfUnderMaximumPoolSize(Runnable)方法

这个方法似乎与 addIfUnderCorePoolSize()方法很相似，都是判定线程数量（poolSize）后的操作，唯一的区别是判定线程池数量的条件从 corePoolSize 改成了 maximumPoolSize。为什么会出现这样的结果呢？

为了搞清楚这个问题，此时需要回想一下线程池的两大类构造方法。

- ◎ 如果通过 newFixedThreadPool(int)方式创建线程池，corePoolSize 与 maximumPoolSize 是等值的，自然这两个方法可以说是毫无区别，addIfUnderCorePoolSize()判定不成立，再进入到 addIfUnderCorePoolSize()方法也不太可能会创建新线程，大多数情况下会返回 false，返回 false 后就会调用 reject()进行丢弃处理操作。
- ◎ 如果通过 newCachedThreadPool()方式创建线程池，corePoolSize 的值是 0，maximumPoolSize 的默认值是 Integer.MAX_VALUE。

如果保持该默认值，在图 5-32 中最外层的判定：poolSize >= corePoolSize 恒为 true，永远不会调用 addIfUnderCorePoolSize(Runnable)。此时的阻塞队列是 SynchronousQueue，前文中已说明如果没有空闲的线程在等待任务就会立即返回 false。因此，在 Cache 线程池中返回 false 的概率是很高的，尤其是系统遭到瞬间冲击时。

返回 false 后就同样调用 addIfUnderMaximumPoolSize(command)。由于 maximumPoolSize 的值是 Integer.MAX_VALUE，所以当前线程数是永远无法超越的一个值，该方法将永远都返回 true，换句话说，就是到这里的每个请求都会创建线程放入 workers 中（前提是线程池没有做 shutdown()操作，否则都是空谈）。

现在我们应该对线程池有了一点点认识，至少应当知道向线程池提交任务后，生产者线程通常就不用阻塞在那里了，调度过程由线程池来完成。但小伙伴心里应该还有许多问题没有想明白，例如，线程池是不是有一个死循环在扫描提交的任务？自己提交的任务何时会被调度呢？前文中提到使用 ThreadPoolExecutor.Worker 包装自己提交的任务，为何要这样包装？文中多次提到的 SynchronousQueue 与 LinkedBlockingQueue 到底有何区别？

接下来带着这些问题来学习就是答疑阶段了。我们依旧通过看源码的方式来说明一些问题的本质和原因。

首先来了解前文中提到的 ThreadPoolExecutor.Worker 包装用户提交的任务，那它又是如何使用这个任务的呢？

前文中提到，线程运行时调用 ThreadPoolExecutor.Worker 的 run()方法，那么我们自然要从它开始讲起了。请看图 5-36。

```
public void run() {
    try {
        Runnable task = firstTask;
        firstTask = null;
        while (task != null || (task = getTask()) != null) {
            runTask(task);
            task = null;
        }
    } finally {
        workerDone(this);
    }
}
```

图 5-36 Worker 中运行任务的 run()方法 (JDK 1.6)

进入 run()方法后，首先一个本地变量 task 被 firstTask 赋值，这个 firstTask 就是用户提交的任务，赋值结束后，firstTask 就会被赋值为空，因为这个任务一旦被调用就没有用了，可以被 GC 掉。

接下来进入一个 while 循环开始运行，while 循环的条件为 task 不为空，或者通过 getTask()返回值赋值给 task 后依然不为空。由于 task 已经被 firstTask 所赋值，所以通常情况下在第 1 轮循环时就直接调用 runTask(Runnable)来运行了，但并不排除 firstTask 为空的情况，在后文继续分析的过程中就会见到。

runTask(Runnable)方法的细节可以进一步跟踪，内部直接调用了这个 task 的 run()方法，而这里的任务通常就是用户提交的真正任务，自然 run()方法内容也是由用户编写的内容了（除非在线程池外部还有其他组件的包装，例如 CompletionService）。

线程池不会再新建线程来处理任务，而是不断用这个线程去处理多个任务来达到线程复用的目的。

在再一次循环前会先将 task 赋值为 null，因此会尝试从 getTask()方法中获取任务，而这里可以猜到 getTask()应该是从 BlockingQueue 中获取任务的，请看代码，如图 5-37 所示。

```
Runnable getTask() {
    for (;;) {
        try {
            int state = runState;
            if (state > SHUTDOWN)
                return null;
            Runnable r;
            if (state == SHUTDOWN) // Help drain queue
                r = workQueue.poll();
            else if (poolSize > corePoolSize || allowCoreThreadTimeOut)
                r = workQueue.poll(keepAliveTime, TimeUnit.NANOSECONDS);
            else
                r = workQueue.take();
            if (r != null)
                return r;
            if (workerCanExit()) {
                if (runState >= SHUTDOWN) // Wake up others
                    interruptIdleWorkers();
                return null;
            }
            // Else retry
        } catch (InterruptedException ie) {
            // On interruption, re-check runState
        }
    }
}
```

图 5-37 getTask()方法的实现

这段代码也是一个死循环，而且是在循环体内部做了 try catch 操作，简单地看代码逻辑，要跳出循环只有 return 语句能做到。代码逻辑会根据参数和状态分为多种情况，不容易看懂，不用着急，胖哥来帮助你分解。

(1) 循环体内部，首先获取线程池的当前状态，如果线程池的状态大于 SHUTDOWN 值，就直接返回空。在什么情况下线程池的状态会大于 SHUTDOWN 值呢？通常会有两种场景会发生。

其一是当线程池调用了 shutdown() 方法后，线程池的状态会首先被设置为 SHUTDOWN，然后遍历线程池中所有的线程调用一次 interrupt() 方法，如果在休眠中的线程将会被激活，激活后的线程以及调用 shutdown() 方法本身的线程都会去尝试调用 tryTerminate() 方法，该方法将判定如果线程池中所记录的线程数为 0，则将线程池的状态修改为 TERMINATED，这个值为 3，将大于 SHUTDOWN 状态值。

其二是当线程池调用了 shutdownNow() 方法后，首先，会将线程池的状态修改为 STOP，这个状态是大于 SHUTDOWN 值的，接下来它也会通过中断激活线程，只是它来得更暴力一些，连加锁和一些基本判定都没有，直接做中断；其次，在调用 tryTerminate() 方法之前会先清空阻塞队列中所有的元素，这些元素被组装为一个 List 列表作为 shutdownNow() 方法的返回值。换句话说，没有执行的任务在 shutdownNow() 执行后的返回值中可以得到。相应的，在程序某些必要的情况下，可以通过线程池的 isTerminating()、isTerminated()、isStopped()、isShutdown() 来对线程池做一些状态判定。

(2) 进一步看循环体的第二条代码，判定线程状态如果为 SHUTDOWN，则会直接使用 BlockingQueue.poll() 方法获取一个任务，这个方法返回阻塞队列的一个元素，如果阻塞队列为空，则返回 null。如果是通过 newFixedThreadPool(int) 方式创建的线程池，在阻塞队列中有任务时则会返回一个任务，否则直接返回 null。如果通过 newCachedThreadPool() 方式创建线程池使用的是 SynchronousQueue，前文中提到生产者在没有消费者线程资源空闲时等待则不会停留在阻塞队列中，而此时线程尝试通过这个队列去获取元素肯定是无法立即获取到的，这个方法不会阻塞，所以在理论上讲这种情况将永远返回 null。

(3) 接着看下一个条件：poolSize > corePoolSize || allowCoreThreadTimeOut，在固定大小的线程池中肯定不会出现 poolSize > corePoolSize 的情况，所以只会出现在 newCachedThreadPool() 当中，因为它的 corePoolSize 默认值是 0。

参数 allowCoreThreadTimeOut 是可以单独通过 allowsCoreThreadTimeOut(boolean) 来扩展的，或者说该参数与 newCachedThreadPool() 的关系并不是那么密切。当这两个条件任意一个成立时，将会通过 workQueue.poll(keepAliveTime, TimeUnit.NANOSECONDS) 来获取任务，它是介于 poll() 与 take() 之间的一种方法，当阻塞队列中没有可获取的任务时会像 take() 方法那样被挂起，但是当时间达到了 keepAliveTime 毫秒值时就会自动被唤醒，而不会永远地沉睡下去。

keepAliveTime 在这里就起到了作用，通过 newCachedThreadPool() 所创建的线程池默认是 1 分钟超时，如果遇到前面所提到的瞬间冲击，那么线程数量将瞬间快速膨胀，而且这些瞬间膨胀的线程的生命周期最少在 1 分钟以上。当然，这个值也不是只能用在 newCachedThreadPool() 所创建的线程池中，我们可以对线程池使用 setKeepAliveTime(long, TimeUnit) 方法来修改参数。

(4) 对于其他的情况，通常会调用 BlockingQueue.take() 方法，同样会从阻塞队列中获取一个元素，不过这个方法内部会不断尝试，直到获取到元素后才会返回（不过，发生中断后抛出异常也会打破这种情况，例如 shutdown() 时肯定能激活它）。由于 take() 方法不会因为队列中没有任务就返回空值，所以通过 newFixedThreadPool(int) 方式创建的线程池的线

程数量在默认情况下是只增不减的，但它的最大值不会超过 `corePoolSize` 值。通过前面的描述，可以尝试设置两个参数：`allowCoreThreadTimeOut` 和 `keepAliveTime` 来改变内在的代码逻辑，也就是说，通过改变参数后这样的线程池也是可以回收线程的。

如果是 `newCachedThreadPool()` 方式创建的线程池，虽然有超时机制，但是外面包装着一个死循环，其本身会在瞬间冲击时线程数量快速膨胀，如果线程不回收就可能存在问题。从前面的代码截图中发现，只有返回的任务不为空时才会发生 `return` 操作，还有就是只有 `workerCanExit()` 方法成立时才会 `return null`，跳出这个循环。我们来看看 `workerCanExit()` 方法的源码，如图 5-38 所示。

```
private boolean workerCanExit() {
    final ReentrantLock mainLock = this.mainLock;
    mainLock.lock();
    boolean canExit;
    try {
        canExit = runState >= STOP ||
            workQueue.isEmpty() ||
            (allowCoreThreadTimeOut &&
                poolSize > Math.max(1, corePoolSize));
    } finally {
        mainLock.unlock();
    }
    return canExit;
}
```

图 5-38 `workerCanExit()` 方法的源码

这段代码最关键的是 `canExit` 的值，它由 3 个条件组成，并且任意一个成立就返回 `true`，下面分别看看 3 个条件。

第 1 个条件是线程池的状态要大于等于 `STOP` 值，只有线程池调用了 `shutdown()`、`shutdownNow()` 方法才能让该条件成立。前文中已经解释，这里不再重述。

第 2 个条件是阻塞队列为空，如果是固定大小的线程池所使用的 `LinkedBlockingQueue`，则会返回链表的长度，即返回等待任务的个数，如果没有任何等待任务就会成立。在使用 `Cache` 类型的线程池时，使用的阻塞队列是 `SynchronousQueue`，它的这个方法将永远返回 `true`，因为在这种线程池里面生产者永远不会将任务放入一个队列中让消费者来获取。

第 3 个条件是在设置了 `allowCoreThreadTimeOut` 条件的基础上，`poolSize` 值要大于 `corePoolSize` 值，在 `newCachedThreadPool()` 方法中 `corePoolSize` 值默认是 0，所以这时 `poolSize` 值大于 1 即可成立。由于固定大小的线程池 `corePoolSize` 与 `maximumPoolSize` 是等值的，所以在理论情况下这个条件永远不会成立。

当这 3 个条件任意一个成立时，就有机会返回到 `Worker` 的 `run()` 方法中，此时由于返回的值也是 `null`，所以 `Worker` 的 `run()` 方法也会跳出循环，即线程即将结束。在线程真正结束前，还会在 `finally` 内部执行 `workerDone(Worker)` 方法。该方法的代码如图 5-39 所示。

该方法的代码很简单，在加锁的基础上，将线程执行完的任务数叠加到线程池总的完成任务数上（外部可以通过 `getCompletedTaskCount()` 方法获取），将 `Worker` 从 `workers` 队列中删除，将 `poolSize` 值减 1，若减后为 0 则尝试调用 `tryTerminate()`，该方法会尝试关闭线程池，不过在没有调用 `shutdown()`、`shutdownNow()` 方法之前，它是不会主动去关闭线程池的，否则对于 `Cache` 线程池就会有各种各样的问题。

在本小节的最后，我们简单探讨一下 `LinkedBlockingQueue`、`SynchronousQueue` 的几个关键方法的源码。


```
void workerDone(Worker w) {
    final ReentrantLock mainLock = this.mainLock;
    mainLock.lock();
    try {
        completedTaskCount += w.completedTasks;
        workers.remove(w);
        if (--poolSize == 0)
            tryTerminate();
    } finally {
        mainLock.unlock();
    }
}
```

图 5-39 线程退出时所做的动作

以 `LinkedBlockingQueue` 的 `take()` 方法为例，代码如图 5-40 所示。

```
public E take() throws InterruptedException {
    E x;
    int c = -1;
    final AtomicInteger count = this.count;
    final ReentrantLock takeLock = this.takeLock;
    takeLock.lockInterruptibly();
    try {
        while (count.get() == 0) {
            notEmpty.await();
        }
        x = dequeue();
        c = count.getAndDecrement();
        if (c > 1)
            notEmpty.signal();
    } finally {
        takeLock.unlock();
    }
    if (c == capacity)
        signalNotFull();
    return x;
}
```

图 5-40 `take()` 方法阻塞的位置

线程等待的位置就在 `while` 循环中，它在等待一个非空的信号量，在队列为空时将一直等待。当有生产者向阻塞队列中放入任务时（通过 `add()`、`offer()`、`put()` 等方法），如果发现当前的阻塞队列为空，则会调用 `notEmpty` 的 `signal()` 方法来激活一个等待的线程，一个阻塞的线程获取到任务后如果发现阻塞队列的数量在获取任务前是大于 1 的，则说明还有剩下的等待任务，则会再一次调用 `notEmpty` 的 `signal()` 方法来激活下一个等待的线程进一步处理。同样的，大家可以看看 `poll()` 等方法的实现，就能将阻塞队列实现的原理看懂了。

总地来说，对于具有固定大小的线程池初始化是没有任何线程的，也没有不断循环的线程来扫描任务，在任务提交时会产生线程，更确切地说是提交任务没有空闲线程处理时会创建线程，但是总的线程数不会超过 `corePoolSize` 值，在默认情况下这些线程只增不减，但是可以通过修改参数来达到回收的目的。在通常情况下对于固定大小的线程池来讲没有必要这样做，我们通常希望的是只要有请求能尽快处理就可以了，线程只要不做事情，它的开销几乎可以忽略不计。

再以 `SynchronousQueue` 为例，它是生产者/消费者的平衡者，内在的实现可以是队列或栈，根据构造方法的参数是 `true` 或 `false` 来决定，默认为 `false`，使用栈来实现，两者的最大区别在于找到可用资源的获取方式是 `FIFO` 还是 `LIFO`。首先来看看 `take()` 方法，如图 5-41 所示。

```

public E take() throws InterruptedException {
    Object e = transferer.transfer(null, false, 0);
    if (e != null)
        return (E)e;
    Thread.interrupted();
    throw new InterruptedException();
}

```

图 5-41 SynchronousQueue 的 take()方法

代码中的 transferer 就是队列或栈，以默认的实现栈为例，主要代码如下：

```

Object transfer(Object e, boolean timed, long nanos) {
    SNode s = null; // constructed/reused as needed
    int mode = (e == null)? REQUEST : DATA;
    for (;;) {
        SNode h = head;
        if (h == null || h.mode == mode) { // empty or same-mode
            if (timed && nanos <= 0) { // can't wait
                if (h != null && h.isCancelled())
                    casHead(h, h.next); // pop cancelled node
                else
                    return null;
            } else if (casHead(h, s = snode(s, e, h, mode))) {
                SNode m = awaitFulfill(s, timed, nanos);
                if (m == s) { // wait was cancelled
                    clean(s);
                    return null;
                }
                if ((h = head) != null && h.next == s)
                    casHead(h, s.next); // help s's fulfiller
                return mode == REQUEST? m.item : s.item;
            }
        } else if (!isFulfilling(h.mode)) { // try to fulfill
            if (h.isCancelled()) // already cancelled
                casHead(h, h.next); // pop and retry
            else if (casHead(h, s=snode(s, e, h, FULFILLING|mode))) {
                for (;;) { // loop until matched or waiters disappear
                    SNode m = s.next; // m is s's match
                    if (m == null) { // all waiters are gone
                        casHead(s, null); // pop fulfill node
                        s = null; // use new node next time
                        break; // restart main loop
                    }
                    SNode mn = m.next;
                    if (m.tryMatch(s)) {
                        casHead(s, mn); // pop both s and m
                        return (mode == REQUEST)? m.item : s.item;
                    } else // lost match
                        s.casNext(m, mn); // help unlink
                }
            }
        } else { // help a fulfiller
            SNode m = h.next; // m is h's match
            if (m == null) // waiter is gone
                casHead(h, null); // pop fulfilling node
            else {
                SNode mn = m.next;
                if (m.tryMatch(h)) // help match
                    casHead(h, mn); // pop both h and m
                else // lost match
                    h.casNext(m, mn); // help unlink
            }
        }
    }
}

```


这段代码不容易看懂，因为读写操作都是走这个方法，请把思路切换到胖哥这里。

首先代码中根据传入的参数 `e` 是否为空判定 `mode` 为 `REQUEST` 还是 `DATA`，两者分别代表是请求数据还是写入数据，或者说读取数据时传入的参数 `e` 就是空的，写入数据就是要写入参数 `e`。在图 5-41 所示的代码中可以看到传入的 `e` 就是 `null`。

然后是一个死循环，在每次循环中首先取出 `head` 节点赋值给局部变量 `h`，`head` 节点初始化时是 `null`，因此第一次判定 `h == null` 是肯定成立的，因此会进入这个判定逻辑的内部。根据图 5-41 的描述，`take()` 方法的第二个参数将会传入 `false`，所以 `if (timed && nanos <= 0)` 不会成立，从而执行 `casHead(h, s = snode(s, e, h, mode))` 方法。

该方法的执行为两部分来看：首先看 `snode(s,e,h,mode)` 方法的内部实现，它用于尝试创建一个节点并将 `next` 指向传入的 `h`，创建节点的内容是传入的参数 `e`，这个 `e` 为空也是可以的，因为它只是作为一个节点中的属性。

节点创建成功后，则会通过 `casHead()` 方法通过 CAS 尝试将当前节点设置为头节点（`head` 引用）。如果修改 `head` 成功，则会调用 `awaitFulfill()` 方法进入等待状态。

`awaitFulfill()` 方法内部也有许多逻辑，主要是根据传入的参数以及头节点的变化情况做一定次数的尝试，在尝试一定次数后会通过 `LockSupport.park()` 使得线程进入等待状态，直到有中断发生才会被激活。

当另一个请求进入时，会做相同的操作，如果也是 `take()` 请求，则会使得头节点的 `mode` 与它的 `mode` 一样，会进入同样的处理逻辑，此时需要写入数据的动作才能将它们脱离。以 `offer(E)` 为例，看看它会传入什么参数，如图 5-42 所示。

```
public boolean offer(E e) {
    if (e == null) throw new NullPointerException();
    return transferer.transfer(e, true, 0) != null;
}
```

图 5-42 offer()方法会传入的参数

这个方法传入了一个请求写入的数据，第二个参数传入了 `true`，如果此时存在前文中描述的消费者处于等待状态，则 `if (h == null || h.mode == mode)` 不会成立，因此会判定 `!isFulfilling(h.mode)`，这个方法是用以判定 `mode` 的，从前面的代码中可以看到传入的 `mode` 只可能是 `REQUEST` 或 `DATA`，那么 `head` 节点也应当是两者之一，此时需要看看它们的值以及判定方法，如图 5-43 所示。

```
/** Node represents an unfulfilled consumer */
static final int REQUEST = 0;
/** Node represents an unfulfilled producer */
static final int DATA = 1;
/** Node is fulfilling another unfulfilled DATA or REQUEST */
static final int FULFILLING = 2;

/** Return true if m has fulfilling bit set */
static boolean isFulfilling(int m) { return (m & FULFILLING) != 0; }
```

图 5-43 mode 的值以及 `isFulfilling()` 的判定方法

`REQUEST` 节点是 0，`DATA` 节点是 1，而 `FULFILLING` 是 2，此时传入的节点应当是 0 或 1，那么与 2 进行按位“与”操作永远返回 0，因此这个方法肯定在此时会返回 `false`，那么条件 `!isFulfilling(h.mode)` 正好成立，会进入这个逻辑中，在这个逻辑中同样会将当前节点尝试写入到 `head`，唯一的区别在于它所创建的节点的 `mode` 不再是 `REQUEST` 或 `DATA`，而是将这个值与 `FULFILLING` 进行按位“或”操作。如果这个节点通过 `casHead()` 设置成功，

则会尝试取出下一个节点，此时的下一个节点自然是一个处于等待中的消费者线程对应的节点，调用该节点的 `tryMatch(s)` 方法，该方法会将等待节点的 `match` 值设置为当前请求写入数据的节点，并通过 `LockSupport.unpark()` 方法将等待者激活，等待者被激活后发现 `match` 值则会立即返回操作，而同时请求数据的线程会在这个时候将 `head` 设置为等待节点的下一个节点。即使这样，但是由于其他的逻辑也可能同时在做 `tryMatch(s)` 操作，所以依然可能不成功，如果不成功，则会将当前节点的 `next` 指向等待节点的下一个节点，进一步循环。

由于存在并发操作，所以可能会有多个线程同时 `!isFulfilling(h.mode)` 成立，但是只有一个线程会在 `casHead()` 方法上成功，其余的会继续循环，下一轮循环时，如果成功设置头部的线程还没有完成所有的动作，则 `!isFulfilling(h.mode)` 不再成立，如果还有等待的节点，则程序会进入代码中的最后一个 `else` 逻辑。这段代码中的第 1 个逻辑是为了防止前文中提到的并发地做 `tryMatch(s)` 操作使得写入数据节点还没有脱离出来，第 2 个逻辑也是 `tryMatch(s)` 的操作逻辑。

换句话说，请求写入数据的线程都在尝试将等待数据的第一个消费者取出来（具体第一个消费者是谁要看实现的类是栈还是队列，最大的区别也在这里），取出成功则激活消费者，若没有消费者，则写入数据的动作也会立即退出，即生产者不会等待消费者。

这样看起来似乎很绕，而且使用栈在某种意义上似乎显得不是那么公平，但在某些规则下，这些不公平往往会更加合适一些。回到 `Cache` 类型线程池的话题中，上述代码已经充分说明提交任务的生产者如果发现没有空闲的线程资源在等待任务，则不会写入任何数据，进而创建新的线程。同时，`SynchronousQueue` 认为自己永远不会存放数据（其实它也存放了，只是它存放的是消费者线程的等待队列，而不是生产者请求的任务数据），所以它的 `isEmpty()` 方法返回的永远是 `true`。

在本小节中，通过源码层层分析了线程池的内在实现机制，通过参数设置可以实现各种变化，大家也可以在这个基础上进一步扩展来实现自己理想中的线程池。

►► 5.6.3 调度器 `ScheduleThreadPoolExecutor`

`ScheduleThreadPoolExecutor` 是 Java 提供的多线程调度器，可以指定时间点运行某个任务，也可以让某些指定的任务反复根据某些时间规则运行。

有人会问：这个调度器使用 `Timer` 和 `TimerTask` 不就可以实现了吗？确实可以实现，但是通过 `Timer` 和 `TimerTask` 所实现的调度与 `ScheduleThreadPoolExecutor` 实现的调度之间会有一些区别。它们之间的区别类似于 `Thread` 与 `ThreadPoolExecutor` 的区别。换句话说，就是一个 `Timer` 虽然可以调度多个任务（即同一个 `Timer` 对象可以调度多个 `TimerTask` 任务），但是调度多个任务时是完全串行的，这显然在有大量任务调度时资源是不够的，在服务器程序上我们更希望由多线程的调度器来完成，当然你可以变通地认为创建多个 `Timer` 来完成，这样是不是很麻烦？因为任务之间的均衡法则很难控制，与并发编程中的许多高级组件的结合也不太可能。

为了更好地学习 `ScheduleThreadPoolExecutor` 的调度，我们先来看看 `Timer` 和 `TimerTask` 的基本原理，因为多线程也是基于单线程的，在调度层面的许多基本道理是类似的。

(1) `Timer`

`Timer` 与 `TimerTask` 配合完成任务调度，它们都在 `java.util` 包下。`TimerTask` 是一个抽象类，它实现了 `Runnable` 接口，但并没有实现 `run()` 方法，仅仅扩展了和时间相关的处理，

因为它并不知道业务上到底要做什么，所以 `run()` 方法的内容需要我们自己来编写。对 `TimerTask` 的了解先到这里，后文中还将继续介绍它的内在。首先来看看 `Timer` 提供的调度方法，如图 5-44 所示。

```
● schedule(TimerTask task, Date time) : void - Timer  
● schedule(TimerTask task, long delay) : void - Timer  
● schedule(TimerTask task, Date firstTime, long period) : void - Timer  
● schedule(TimerTask task, long delay, long period) : void - Timer  
● scheduleAtFixedRate(TimerTask task, Date firstTime, long period) : void - Timer  
● scheduleAtFixedRate(TimerTask task, long delay, long period) : void - Timer
```

图 5-44 调度方法

这几个调度方法的基本功能介绍如下：

- ◎ `schedule(TimerTask task, Date time)`，按指定时间点 `time` 进行执行，执行一次。
- ◎ `schedule(TimerTask task, long delay)`，当前系统时间延迟 `delay` 毫秒后执行，只执行一次。
- ◎ `schedule(TimerTask task, Date firstTime, long period)`，指定第一次执行时间点，后续每隔 `period` 毫秒再次执行，循环执行任务。
- ◎ `schedule(TimerTask task, long delay, long period)`，当前系统时间延迟 `delay` 毫秒后第一次执行，后续每隔 `period` 毫秒再次执行，并不断基于这个时间片来执行。
- ◎ `scheduleAtFixedRate(TimerTask task, Date firstTime, long period)`，也是指定第一次执行时间后，每隔 `period` 时间片循环执行。不过它与 `schedule` 方法有一些区别，例如时间片都是 5s 一次，`Timer` 本身在调度中可能会延迟到第 8s 才调度这个任务，若通过 `schedule` 方法调度任务时计算下一次任务的执行时间结果将是第 13s，而通过 `scheduleAtFixedRate` 计算出来的依然是第 10s，自然的在优先级排序上会优先调度后者，以便于执行次数尽量是一致的。
- ◎ `scheduleAtFixedRate(TimerTask task, long delay, long period)`，与 `schedule(TimerTask task, long delay, long period)` 方法的区别同上。
- ◎ `cancel()`，清空队列信息，并修改 `newTasksMaybeScheduled` 状态为 `false`，这时 `Timer` 也会结束。
- ◎ `purge()`，对 `TimerTask` 做 `cancel()` 后（注意：不是对 `Timer` 做 `cancel()` 操作），只是修改了 `TimerTask` 的状态，其实这个任务依然在队列里面，该操作会清空队列中被 `cancel()` 掉的 `TimerTask` 任务。

这几个方法的参数解释如下：

- ◎ 参数中传入 `Date` 为指定时间点调度，传入 `delay` 是基于当前系统时间+`delay` 算出指定的时间点调度。
- ◎ 时间片 `period` 若没有传入，则内部默认为 0，最终也合并到一起处理。
- ◎ `scheduleAtFixedRate` 方法与 `schedule` 方法的区别是，`schedule` 方法会将传入的 `period` 取相反数再做操作（会变成一个负数），方法入参都要求 `period` 值必须大于 0，而 `delay` 值不能小于 0。
- ◎ 参数合并后，都调用了内部的一个私有方法：`sched(TimerTask task, long time, long period)`，这里的 `task` 依然是用户提交的任务，`time` 变成了真正准备执行时间的毫秒值，传入的 `period` 值如果是通过 `schedule` 方法调度的则会变成负数，只执行一次的任务 `period` 值为 0。

前文中提到 Timer 是一个单线程的，下面通过 Timer 的源码来深刻认识这一点。它内部定义了一个 TimerThread 类，这个类继承了 Thread，自定义了 run()方法。

进一步跟踪到 TimerThread 中，它包装了一个 TaskQueue 队列，这是一个内部类，它提供了类似于 List 的基本功能，实现任务队列的调度。说到这里，我们总结一下几个对象之间的包装和调用关系，如图 5-45 所示。

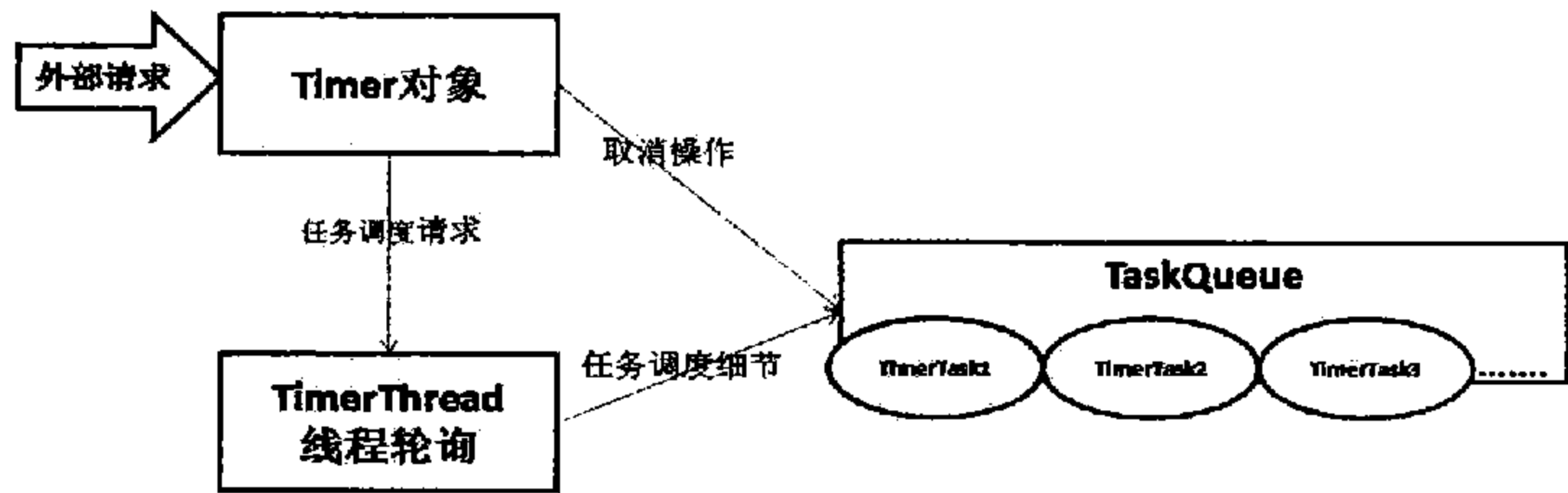


图 5-45 Timer 任务调度示意图

看了图 5-45 后，我们该真正开始看源码了。源码从哪开始看呢？直接对照图 5-44 中的 6 个方法来看源码，这 6 个方法最终都会调用内部的一个私有方法：sched()，sched()方法的源码如图 5-46 所示。

```
private void sched(TimerTask task, long time, long period) {
    if (time < 0)
        throw new IllegalArgumentException("Illegal execution time.");

    synchronized(queue) {
        if (!thread.newTasksMayBeScheduled)
            throw new IllegalStateException("Timer already cancelled.");

        synchronized(task.lock) {
            if (task.state != TimerTask.VIRGIN)
                throw new IllegalStateException(
                    "Task already scheduled or cancelled");
            task.nextExecutionTime = time;
            task.period = period;
            task.state = TimerTask.SCHEDULED;
        }

        queue.add(task);
        if (queue.getMin() == task)
            queue.notify();
    }
}
```

图 5-46 sched()方法的源码

源码中对 queue 队列的操作都是通过 synchronized 加锁的，因此是线程安全的。在赋值过程中 task 有一个 nextExecutionTime 可以理解为任务下一次执行的预计时间，初始化时直接由传入的参数决定，task 的相应参数设置好以后，将其放入到 queue 队列中。

初始化的动作我们看到了，但是有些调度方法还没有想通，例如 TimerThread 是如何被调度的？queue 队列是如何运作的？接下来分别就这两个问题做展开说明。前文中提到 TimerThread 是一个线程，自然从 run()方法开始看，因为那里才是运行的地方。跟踪到 run()方法后发现是间接调用 mainLoop()方法来运行的，mainLoop()方法内部是一个 while(true)死循环，我们来看看循环体内部的操作，如图 5-47 所示。

在循环体中使用 synchronized(queue)是为了保证运行时线程安全，即 Timer 正好在运行时，外部生产者线程提交新的任务给 Timer。

进入同步区后首先对 queue 队列进行判空操作，如果队列为空且 newTasksMayBeScheduled 为 true，则会 queue.wait()，newTasksMayBeScheduled 的初始值确实是 true，因此在 Timer 刚刚被创建时肯定会走到这里。在图 5-46 所示的 sched()源码中可以看到，如果 getMin() 为当前任务，则会执行 queue.notify()操作，也就是说，队列中没有元素时，Timer 会在这里等待，此时第一个任务请求会激活线程开始运行（当然中断也是同样可以激活的）。

```
synchronized(queue) {
    // Wait for queue to become non-empty
    while (queue.isEmpty() && newTasksMayBeScheduled)
        queue.wait();
    if (queue.isEmpty())
        break; // Queue is empty and will forever remain; die

    // Queue nonempty: look at first evt and do the right thing
    long currentTime, executionTime;
    task = queue.getMin();
    synchronized(task.lock) {
        if (task.state == TimerTask.CANCELLED) {
            queue.removeMin();
            continue; // No action required, poll queue again
        }
        currentTime = System.currentTimeMillis();
        executionTime = task.nextExecutionTime();
        if (taskFired = (executionTime <= currentTime)) {
            if (task.period == 0) { // Non-repeating, remove
                queue.removeMin();
                task.state = TimerTask.EXECUTED;
            } else { // Repeating task, reschedule
                queue.rescheduleMin(
                    task.period < 0 ? currentTime - task.period
                        : executionTime + task.period);
            }
        }
    }
    if (!taskFired) // Task hasn't yet fired; wait
        queue.wait(executionTime - currentTime);
}
if (taskFired) // Task fired; run it, holding no locks
    task.run();
```

图 5-47 mainLoop()方法中循环体内部的操作

从源码中可以看到，激活后会再一次进行 queue.isEmpty()判定，这是为什么呢？

当对 Timer 发起一个 cancel()操作时，会清空队列并将 newTasksMayBeScheduled 状态设置为 false，同时也发起 queue.notify()操作，此时 Timer 也会跳出 while 循环结束线程，在这种情况下被激活队列确实为空，所以通过 break 操作跳出循环。

如果是正常的程序，则会立即通过 getMin()获取一个任务（getMin()的细节在后文中有介绍），如果任务的状态为 CANCELED（对该 TimerTask 发起了 cancel()操作），则直接从 queue 队列中删除这个任务，并通过 continue 重新回到循环体的开始部分（换句话说，若对 TimerTask 做 cancel()时没有 purge()清理这些废弃的资源，那么 Timer 在调度时也会自动将它们清除，因为过多的垃圾可能会造成内存泄漏）。

如果任务是正常的，那么接下来 currentTime 被赋值为当前系统时间，executionTime 被赋值为任务计划开始执行时间，通过判定 executionTime <= currentTime 来决定任务是否需要执行，变量 taskFired 将得到匹配结果的 boolean 值。如果任务需要执行，则会判定 period 的值来做不同的处理，会分为如下 3 种情况（该 period 的值对应到前文中的构造方法入参）。

- ◎ 若 period 值为 0，则只修改状态，并通过 queue.removeMin()将其从执行队列中删除。
- ◎ 若 period 值小于 0，则通过 currentTime - period 计算下一次执行时间，负负得正得到的时间依然是向后延迟的时间，在 schedule 方法调用时，将 period 取反一次，这里再取反回来，可见取反的目的并非特殊作用，而仅仅是为了区分 scheduleAtFixedRate。

◎ 若 `period` 值大于 0，则通过 `executionTime + period` 计算下一次执行时间，而 `executionTime` 是上一次计划开始执行时间，不是当前系统时间，这一点就印证了前文中对 `scheduleAtFixedRate`、`schedule` 两个方法对比的结论。

在代码中 `queue.rescheduleMin(long newTime)` 就是为队列中的第一个任务设置最新的下一次执行时间并排序。

前文中提到 `taskFired` 记录了任务是否需要执行的 `boolean` 值，此时判定该变量如果为 `false`，则会发起一次 `queue.wait(executionTime - currentTime)`，也就是 `Timer` 的线程也可能在这里等待。如果在这个等待的时间范围内有一个任务插入，优先级很高，则被排序到第一个元素（这个任务的优先级可能会比现在的第一个任务还要高，那么 `TimerThread` 线程就不能再挂起了），生产者线程通过 `getMin()` 获取到的任务发现就是提交的任务时，就会调用 `queue.notify()` 将 `Timer` 的线程唤醒，所以我们不必担心这样的情况会导致新的任务得不到调度。

在 `synchronized` 块外部，再一次通过 `taskFired` 来判定任务是否需要执行，若为 `true` 则会调用该任务的 `run()` 方法来执行（也就是使用当前线程来执行），放在外部判定是因为任务已经从队列中获取到，就不存在线程安全问题了，任务执行的时间长短是未知的，在这个过程中可能会有新的任务提交到 `Timer` 中，我们自然不能等到这个任务结束后再将任务写入队列。

`Timer` 在面对多个 `TimerTask` 任务时，肯定会按照时间顺序优先执行，这一点在调度的 `getMin()` 当中已经有所体现，但这个排序的关键点不在 `Timer` 或 `TimerThread` 当中，而是在 `TaskQueue` 队列中。

`TaskQueue` 队列提供了许多方法，如 `add(TimerTask)`、`size()`、`getMin()`、`get(int)`、`removeMin()`、`quickRemove(int)`、`rescheduleMin(long newTime)`、`isEmpty()`、`clear()`、`fixUp()`、`fixDown()`、`heapify()` 等，其中有些方法和调度相关。

以 `add(TimerTask)` 方法为例，它用于增加一个任务，这个方法与普通的 `List` 一样将元素写到列表的尾部，多做的一个动作是 `fixUp(int)` 操作，传入的参数是当前写入的任务在 `TaskQueue` 中的下标。这个 `fixUp(int)` 是干什么的？用来排序，来看看代码，如图 5-48 所示。

```
private void fixUp(int k) {
    while (k > 1) {
        int j = k >> 1;
        if (queue[j].nextExecutionTime <= queue[k].nextExecutionTime)
            break;
        TimerTask tmp = queue[j]; queue[j] = queue[k]; queue[k] = tmp;
        k = j;
    }
}
```

图 5-48 `fixUp()` 排序

这个排序很奇怪，它不是一种传统的排序，而是“相对有序”，这样做的目的是可以让大量堆积的任务的排序代价很低，因为它只需要将最新的元素放在一个相对合适的位置即可，每次会与下标的 1/2 的元素的 `nextExecutionTime` 值进行对比（代码中对下标采用了右移位运算，对于正整数它等价于数据的 1/2，且效率更高），判定是否需要交换来达到目的。这个过程会有许多不公平，但是这种调度本身就没有要求绝对的公平，在这种相对公平的情况下，性能也可以得到较大的提升。

在排序过程中对比的是 `nextExecutionTime`，再回过头看 `scheduleAtFixedRate`、`schedule` 方法，两者最终的区别在于计算 `nextExecutionTime` 的值，这个值就正好体现在调度排序上，

那么它们在实际应用中到底有何区别呢？

假如时间片是 5s，schedule 方法会将调度时当前系统时间+5s 作为 nextExecutionTime 的值，假如任务在第 8s 时得到运行，下一次计划执行时间为第 13s，如果第 13s 时未调度，则推迟到第 16s 被调度，计算出的 nextExecutionTime 的值就是第 21s。这似乎没有什么问题，但是这中间被少调度了一次。如果 scheduleAtFixedRate 提交的任务在第 8s 时，下一次执行时间是第 10s，此时距离第 10s 只有 2s 了，那么它的优先级会被自然提升；若它在第 16s 时才被调度，计算出的 nextExecutionTime 的值依然是第 15s，此时减去第 15s 时的当前系统时间得到的值应当是-1，那么它的优先级会得到更大的提升。换句话说，它会尽量通过提升优先级的方式让任务的执行次数是理想中的情况。

Timer 的基本原理就是这样的，它始终是单线程的，所以它可能会因为正在执行一个长时间任务导致其他的任务没有机会执行，即使使用多个 Timer 也无法通过任务资源的共享来解决这个问题，因此我们希望有多线程调度器，请看下一部分内容。

(2) ScheduleThreadPoolExecutor

现在来看 ScheduleThreadPoolExecutor 也许会轻松点，因为它与 Timer 的思路类似，而更多的是利用多线程+任务资源共享的方式来实现服务器端大量任务的调度。同样的，先来看看相关的调度方法，如图 5-49 所示。

```
✧ schedule(Callable<V> callable, long delay, TimeUnit unit) : ScheduledFuture<V>  
✧ schedule(Runnable command, long delay, TimeUnit unit) : ScheduledFuture<?>  
✧ scheduleAtFixedRate(Runnable command, long initialDelay, long period, TimeUnit unit) : ScheduledFuture<?>  
✧ scheduleWithFixedDelay(Runnable command, long initialDelay, long delay, TimeUnit unit) : ScheduledFuture<?>
```

图 5-49 ScheduleThreadPoolExecutor 的调度方法

在通常情况下，访问 ScheduleThreadPoolExecutor 是通过 ScheduledExecutorService 接口来实现的，因为 ScheduleThreadPoolExecutor 是 ThreadPoolExecutor 的子类，ThreadPoolExecutor 包含大量的线程池操作方法，两者的方法套在一起会十分混乱，ScheduleThreadPoolExecutor 接口是专用的，它比较干净，大多是与调度相关的方法，这也是接口存在的一个道理。

ScheduleThreadPoolExecutor 一般使用 Executors 的 newScheduledThreadPool()静态方法来创建，它的 corePoolSize 可以从该静态方法传入，maximumPoolSize 是 Integer.MAX_VALUE(但它不会导致线程数在瞬间冲击时迅速增长)，阻塞队列默认使用 DelayedWorkQueue。

图 5-49 中前两个方法的区别在于传入的第一个参数分别是 Callable 和 Runnable，它们都可以被认为是任务，只是 Callable 要求实现的是 call()方法，它要求有返回值，并允许抛出异常。

不论传入的是 Runnable 还是 Callable，在其方法内部都会被一个叫 ScheduledFutureTask 的私有内部类所包装，这种包装就像 TimerTask 一样，会记录任务的一些执行时间的属性，以及和调度相关的计算。

Runnable 传入 ScheduledFutureTask 构造方法后，会将其传递给父类 FutureTask 的构造方法，在这个构造方法中会通过 Executors.callable(Runnable , T)间接地通过 Executors 内部类 RunnableAdapter 将 Runnable 任务适配到 Callable 接口上（大家也可以调用这个方法来实现转换，经过这样转换得到的 Callable 对象的 result 始终是 null），这样转换的目的是为了方便统一接口输入/输出，代码如下：

```

static final class RunnableAdapter<T> implements Callable<T> {
    final Runnable task;
    final T result;
    RunnableAdapter(Runnable task, T result) {
        this.task = task;
        this.result = result;
    }
    public T call() {
        task.run();
        return result;
    }
}

```

图 5-49 中前两个方法比较明显的是都没有传入 period，通过前面的介绍不难理解它们都是一次性任务。后两个方法有了 period 或 delay，并且可以设置时间的单位，我们通过对 Timer 的学习应该猜到，其中一个规范的时间片 (period)，一个是基于执行时间的延迟。为了证明这些结论，我们现在来看源码，就从 scheduleWithFixedDelay() 方法开始吧，请看图 5-50。

```

public ScheduledFuture<?> scheduleWithFixedDelay(Runnable command,
                                                  long initialDelay,
                                                  long delay,
                                                  TimeUnit unit) {
    if (command == null || unit == null)
        throw new NullPointerException();
    if (delay <= 0)
        throw new IllegalArgumentException();
    RunnableScheduledFuture<?> t = decorateTask(command,
        new ScheduledFutureTask<Boolean>(command,
            null,
            triggerTime(initialDelay, unit),
            unit.toNanos(-delay)));
    delayedExecute(t);
    return t;
}

```

图 5-50 scheduleWithFixedDelay() 方法的代码

在这段代码中可以看到，任务会经过 ScheduledFutureTask 包装，比较重要的是将传入的 delay 参数取反，并且要求 delay 参数必须大于 0，这似乎与 Timer 提交任务的校验是同样的操作，这在某种角度上在向我们的猜测靠近。

紧接着将包装后的任务交给了 delayedExecute(Runnable) 方法，这个方法应当与调度有着很大的关系，我们来看图 5-51。

```

private void delayedExecute(Runnable command) {
    if (isShutdown()) {
        reject(command);
        return;
    }
    // Prestart a thread if necessary. We cannot prestart it
    // running the task because the task (probably) shouldn't be
    // run yet, so thread will just idle until delay elapses.
    if (getPoolSize() < getCorePoolSize())
        prestartCoreThread();

    super.getQueue().add(command);
}

```

图 5-51 delayedExecute(Runnable) 方法的代码

对这段代码简单说明如下：

- ◎ 这段代码首先判定 isShutdown()，如果做了这个操作，则会选择 reject(command) 进行丢弃处理。关于这个处理，在 ThreadPoolExecutor 部分已经详细说明了。
- ◎ 如果 poolSize 小于 corePoolSize，则会调用 prestartCoreThread()，这个方法用于启

动一个新的线程，如图 5-52 所示。

```
public boolean prestartCoreThread() {
    return addIfUnderCorePoolSize(null);
}
```

图 5-52 prestartCoreThread()方法

这个方法是不是很熟悉？没错，前文中讲解线程池时，它就是用来创建新线程的方法，ThreadPoolExecutor 是它的父类，自然可以调用到。在这里调用该方法时传入的参数不是用户的任务，而是一个 null 值，这并不影响什么，而且任务本身就不应该立即被执行，传入 null 只是将所创建的 Worker 对象的 firsttask 属性设置为 null，并不代表线程不会被启动，线程启动后会自动到阻塞队列中去获取数据。

◎ 最后将任务放入 workQueue 队列中，这个阻塞队列自然是前文中提到的 DelayedWorkQueue（在默认情况下是这样的），这个阻塞队列存放了待执行的任务列表，线程可以随时从这个队列中获取被调度的资源来执行。由此可见，任务在什么时候被调度与 DelayedWorkQueue 有着莫大的关联。

回想一下，线程池中的线程会不断地尝试从 workQueue 中获取数据，也就是说，只要阻塞队列中有数据返回，就会被线程立即执行，要达到延迟的目的，就必须让队列中存在数据，但必须达到规定的时间才返回数据。简单地说，这个阻塞队列 DelayedWorkQueue 应该有延迟获取的功能。

通过源码可以发现，DelayedWorkQueue 是 SchedulePoolExecutor 的一个内部类，在 DelayedWorkQueue 内部通过创建 java.util.concurrent.DelayQueue 对象来实现延迟的功能。所以，我们现在需要来研究阻塞队列中 DelayQueue 的实现，首先来看它的属性定义，如图 5-53 所示。

```
public class DelayQueue<E extends Delayed> extends AbstractQueue<E>
    implements BlockingQueue<E> {

    private transient final ReentrantLock lock = new ReentrantLock();
    private transient final Condition available = lock.newCondition();
    private final PriorityQueue<E> q = new PriorityQueue<E>();
```

图 5-53 DelayQueue 的属性定义

其内部包装了一个 PriorityQueue，这也是前文中提到的队列之一，它是一个优先级队列，DelayQueue 也是基于优先级队列来实现的。换言之，优先级保证某种顺序，然后再通过延迟来达到定时的目的，当然优先级队列也可以单独使用在某些地方。

现在开始跟踪代码，从哪开始呢？队列是存放数据的，所以我们习惯性从队列数据的“读”、“写”方法开始入手，下文将以 offer(E e)和 take()两个方法为代表来跟踪实现逻辑。首先来看 offer(E e)方法（用于实现队列数据的写入），如图 5-54 所示。

```
public boolean offer(E e) {
    final ReentrantLock lock = this.lock;
    lock.lock();
    try {
        E first = q.peek();
        q.offer(e);
        if (first == null || e.compareTo(first) < 0)
            available.signalAll();
        return true;
    } finally {
        lock.unlock();
    }
}
```

图 5-54 offer(E e)方法

这个 offer(E e)方法间接地调用了内部的 q 的 offer()方法,通过图 5-53 可以看出,这个 q 就是 PriorityQueue。在调用 q.offer(E)之前会先调用 q.peek()方法取出第一个元素 (peek()仅仅是读取第一个元素,不会将其从队列中删除),通过 compareTo()方法判定当前要写入的元素是否小于 0,如果小于 0,则认为当前写入的任务的优先级比队列中所有任务的优先级都要高,所以可能需要立即执行,或者需要重新计算最高优先级任务的时间,使得它可以得到及时的调度。因此调用了 available.signalAll(),这个操作就像往 Timer 中写入任务后,如果发现 getMin()获取的任务就是当前写入的任务,则会激活 Timer 的线程,道理是一样的。如果大家还没有理解,请看下面的小例子。

如果阻塞队列中最高优先级的任务还需要一段时间执行,例如 2s,当前线程则会调用 wait(2000)方法,此时插入任务后发现该任务的预期执行时间小于当前系统时间,需要立即调度,或者小于 2s (例如 1 分钟),此时不应该让其继续等待直到 2s,而是被立即激活,即使现在不用立即执行,但是也需要重新计算等待的时间。

接下来看看优先级队列 PriorityQueue 的 offer(E)方法的源码,如图 5-55 所示。

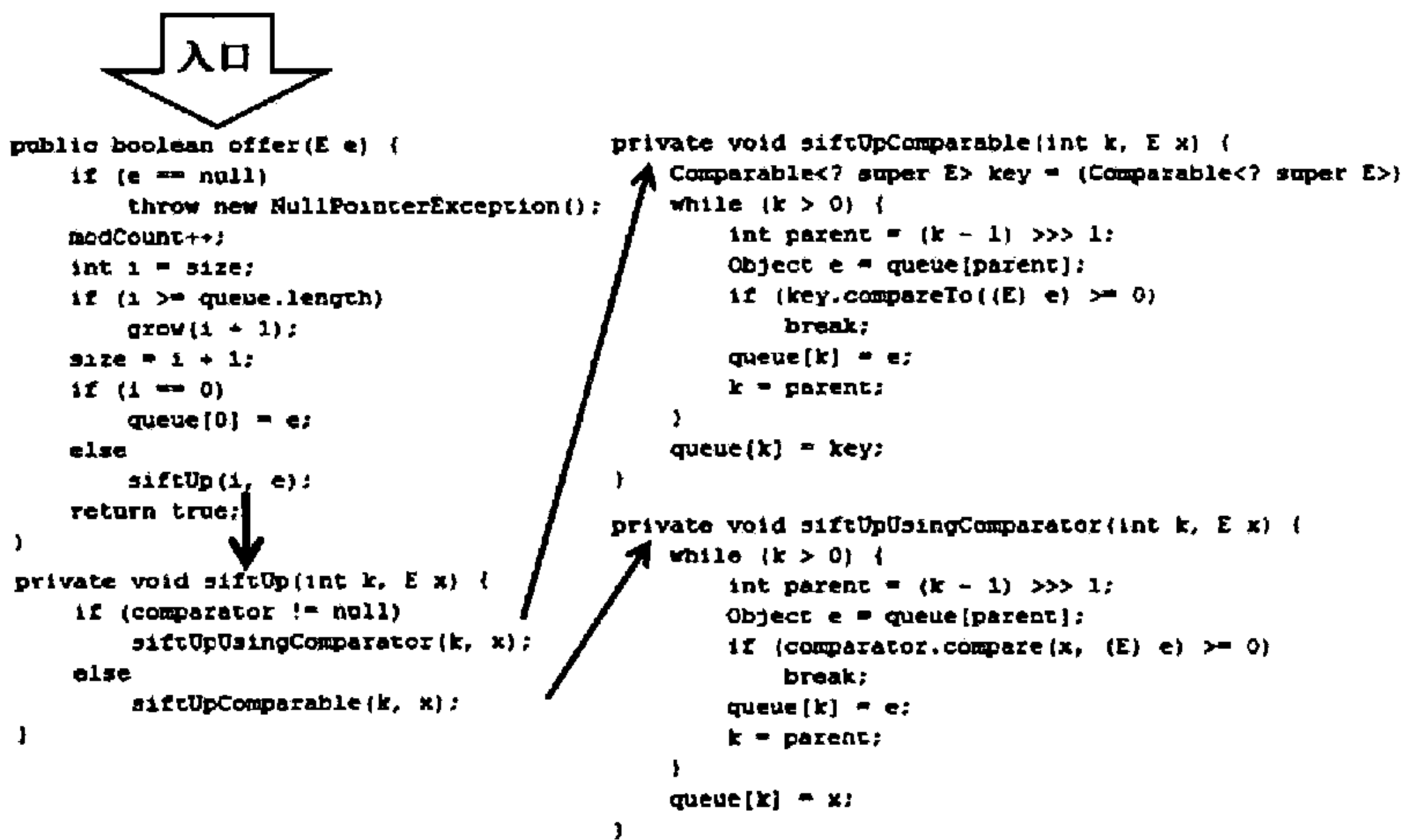


图 5-55 PriorityQueue 的 offer()方法

在这个 offer(E)方法中,除了通过 grow()将数组翻倍操作外,比较重要的就是 siftUp()操作,只要队列中已经存在一个以上的元素该方法就会被调用。siftUp()首先会判定这个优先级队列是否定义过 Comparator,在构造方法中可以设置,不过在本例中是通过 DelayQueue 来创建的,从前面的 DelayQueue 定义中已经看出它不会设置这个参数。因此,在本例中代码肯定会走 siftComparable(int,E),也就是要求排序时对象实现了 Comparable 接口(它是一个通用的优先级队列,它的排序与 Collections.sort()相关的方法类似,我们提供的是对比方法,如果提供的是时间,自然的就会按照时间去对比)。

不论是 Comparator 还是 Comparable 实现,假如已知传入的是时间,被调用的 siftUpUsingComparator 和 siftUpComparable 两个方法的代码是否在前文中见过呢?对,没错,在 Timer 中也是这样排序的,它也是通过不断地右移二进制位来找到合适的位置,也就是说,它并不是完全排序的,也是相对优先的,但同样能在很大程度上提升性能。

知道了 offer(E)进去的数据会被排序,那么任务被调度的先后顺序得到了保证,并非一

种单纯的 FIFO 机制或 LIFO 机制，但是还有一个问题就是：如何保证时间达到后才返回数据，否则会被线程立即执行？那么就要来看看 take()方法的实现了，如图 5-56 所示。

```
public E take() throws InterruptedException {
    final ReentrantLock lock = this.lock;
    lock.lockInterruptibly();
    try {
        for (;;) {
            E first = q.peek();
            if (first == null) {
                available.await();
            } else {
                long delay = first.getDelay(TimeUnit.NANOSECONDS);
                if (delay > 0) {
                    long tl = available.awaitNanos(delay);
                } else {
                    E x = q.poll();
                    assert x != null;
                    if (q.size() != 0)
                        available.signalAll(); // wake up other takers
                    return x;
                }
            }
        }
    } finally {
        lock.unlock();
    }
}
```

图 5-56 DelayQueue 的 take()方法

这个方法也同样取出第一个元素，如果为空则会等待，为空也同时表明队列为空。在图 5-54 中可以看到，在插入任务时，若队列为空或插入的任务的优先级最高，则会调用 available.signalAll()来激活等待的线程。

如果获取到第一个元素不为空，则调用这个元素的 getDelay()方法，如果该方法的返回值大于 0，则继续挂起，但是挂起的时间最多是 delay 的时间；如果不是大于 0 的值，则任务可以被执行，此时通过 poll()方法从优先级队列中提取第一个元素返回，不过此方法肯定会将相应的任务从队列中删除，延迟功能就是通过这样的方式来实现的。

思路似乎是通了，但是这个 getDelay()是从哪里来的呢？我们提交的任务只是一个 Runnable 或 Callable 接口实现，根本没有这个方法。另外，前面提到需要任务实现 Comparable 接口，我们提交的任务似乎并没有这样的东西，它们怎么实现的呢？如果我们没做，很明显只能是包装任务的 ScheduledFutureTask 私有内部类做的。

或许读懂这个类就读懂了整个调度池的最后一个关键环节。首先来了解一下这个类的继承实现关系，如 5-57 图所示。

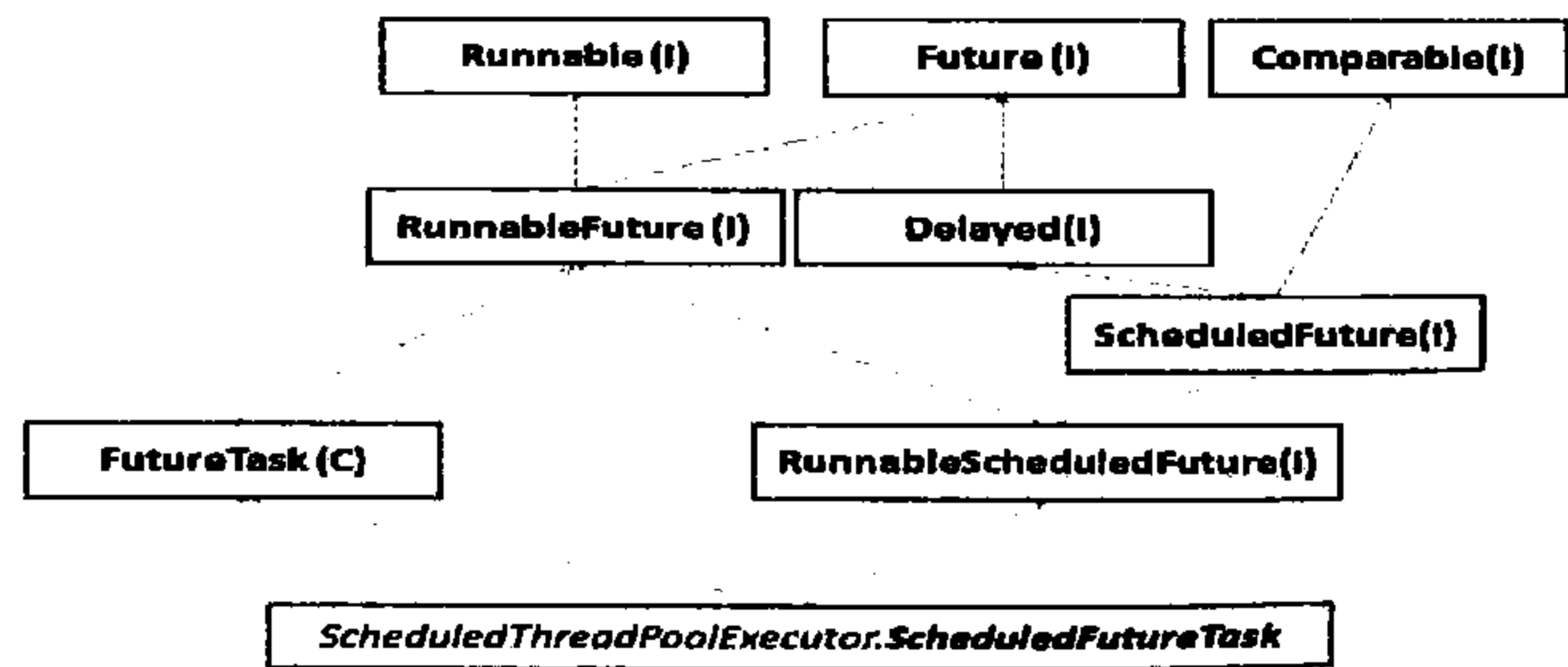


图 5-57 ScheduledFutureTask 类的继承实现关系

在图 5-57 中有许多接口,这些接口体现了 ScheduledFutureTask 本身所具有的功能特征,从这里可以看到,它实现了 Comparable 接口,自然也就实现了其内部的 compareTo()方法,如图 5-58 所示。

```
public int compareTo(Delayed other) {
    if (other == this) // compare zero ONLY if same object
        return 0;
    if (other instanceof ScheduledFutureTask) {
        ScheduledFutureTask<?> x = (ScheduledFutureTask<?>)other;
        long diff = time - x.time;
        if (diff < 0)
            return -1;
        else if (diff > 0)
            return 1;
        else if (sequenceNumber < x.sequenceNumber)
            return -1;
        else
            return 1;
    }
    long d = (getDelay(TimeUnit.NANOSECONDS) -
              other.getDelay(TimeUnit.NANOSECONDS));
    return (d == 0)? 0 : ((d < 0)? -1 : 1);
}
```

图 5-58 任务包装类实现的 compareTo()方法

这个方法已经可以充分说明,其主要是按照预计执行的时间来作为对比基础的,因此简化了外部任务,无须我们自己去实现这个对比,当然它的前提是通用的时间对比,并非所有的对比都可以被抽象。现在再来看看 run()方法的实现部分,如 5-59 图所示。

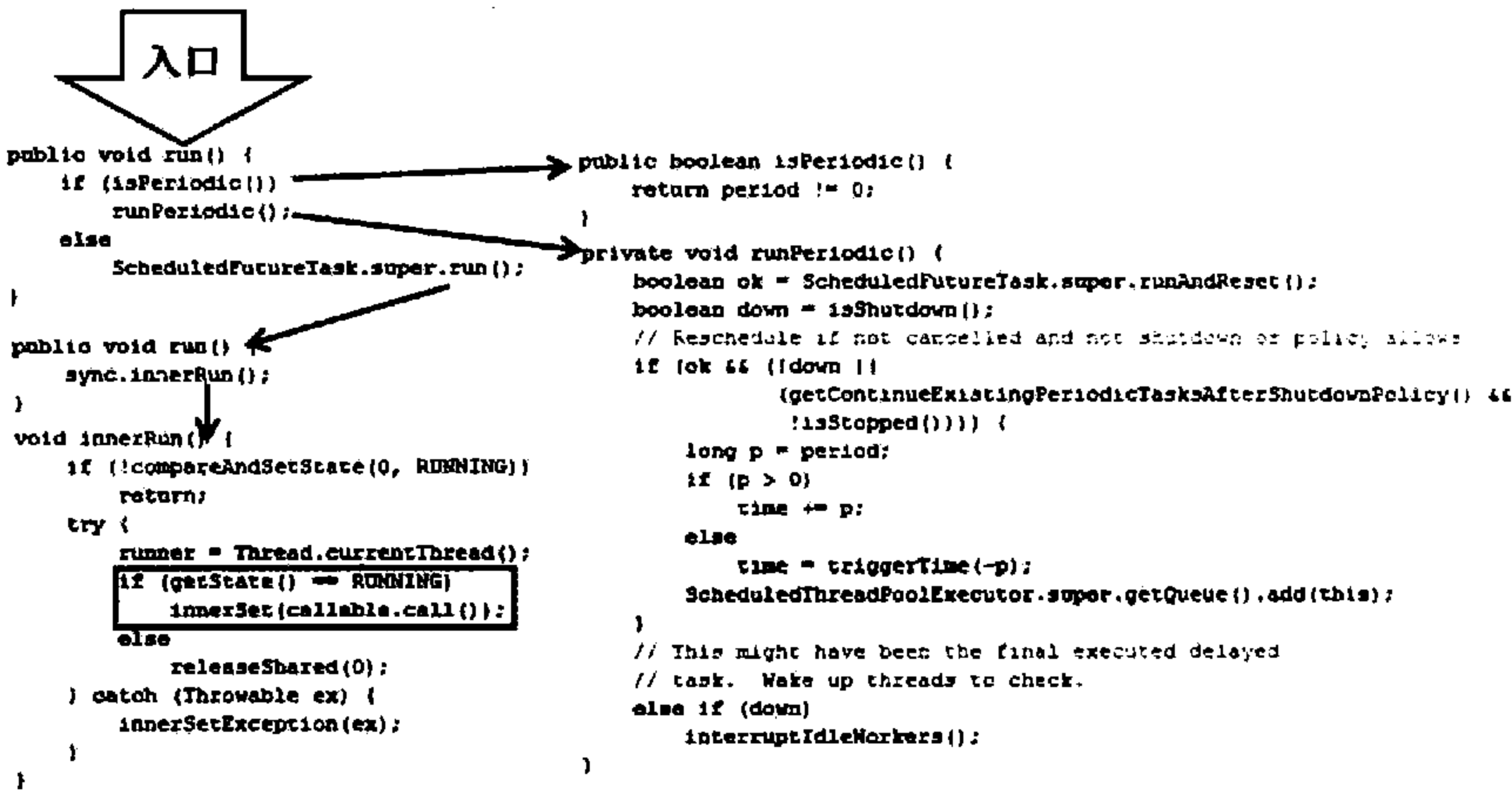


图 5-59 任务包装后的 run()方法的实现部分

方法中首先通过 isPeriodic()判定是否为基于时间片来执行的,如果成立则执行 runPeriodic()方法,否则直接执行 ScheduledFutureTask.super.run()方法。runPeriodic()会首先调用 ScheduledFutureTask.super.runAndReset(),它与直接调用 ScheduledFutureTask.super.run()方法有一个很大的区别是,它会将执行的线程 runner 设置为 null,并将任务的状态设置为 0,那么任务就可以被再次认为还未使用过,就可以再次修改状态被使用,如图 5-60 所示。

接着看图 5-59 中 runPeriodic()方法后面的部分,会算出下一次执行时间,与 Timer 类似,它会根据 period 是否大于 0 做不同的处理,大于 0 时是基于本次计划执行时间叠加 period 实现的,而小于 0 时是以当前系统时间为准的(如果非要说与 Timer 计算时间的区别,这里的当前系统时间是任务执行之后取出来的,而 Timer 是执行之前取出的当前系统时间,

中间有个执行时间的区别，理论上对于整体调度影响不大)，这样大于 0、小于 0 就自然对应到了图 5-49 中的最后两个方法，小于 0 正好是图 5-50 中 `scheduleWithFixedDelay` 对 `period` 取相反数的操作，那么相应的 `scheduleAtFixedRate` 与 `scheduleWithFixedDelay` 的区别就十分明显了。

```
protected boolean runAndReset() {
    return sync.innerRunAndReset();
}

boolean innerRunAndReset() {
    if (!compareAndSetState(0, RUNNING))
        return false;
    try {
        runner = Thread.currentThread();
        if (getState() == RUNNING)
            callable.call(); // don't set result
        runner = null;
        return compareAndSetState(RUNNING, 0);
    } catch (Throwable ex) {
        innerSetException(ex);
        return false;
    }
}
```

图 5-60 `runAndReset()`方法

`runPeriodic()`方法在计算下一次执行时间后将任务再次写入到阻塞队列 `workQueue` 中，在写入的过程中自然又会涉及在队列中查找位置，这样就既可以实现循环调度，又可以实现最新优先级的排序（这里加入队列的方法虽然是 `add()`方法，但内部也是调用 `offer()`来实现的）。

不论是 `ScheduledFutureTask.super.run()`还是 `ScheduledFutureTask.super.runAndReset()`都是调用父类 `FutureTask` 的相应方法来完成的，`FutureTask` 中有一个 `Sync` 属性是基于 AQS 实现的，这个 `Sync` 在这里除了可以通过状态控制是否循环执行任务外，也可以控制同一个任务不会被多个线程同时执行，但是一种十分轻量级的使用。

大家应当对 `ScheduleThreadPoolExecutor` 的原理有所认识了吧，它是对调度任务进行共享化管理，基于 `DelayQueue` 实现了延迟调度及优先级控制，通过 `Executors.callable()`完成了 `Runnable` 到 `Callable` 的适配，通过任务包装实现了延迟计算、对比、控制任务的访问，基于 `ThreadPoolExecutor` 完成了线程池并发地从任务队列中获取可被执行的任务等。

胖哥希望大家在这个基础上不仅仅学会调度池如何使用，同时也要学会它的内在模块化设计思想，学会它的本质，那么在面对问题时就有了底子。

本章内容偏重于理论，对于许多同学来讲比较深，不过如果你能看到这里，后面就相对比较轻松了，我们后面会论证一些概念，以及讲解一些实际场景中的问题。

另外，理论归理论，小伙伴们要有自己的理解和领悟，胖哥的理解始终是胖哥自己的，你可能在未来会遇到不同的问题，但依然可以通过这些思路去解决。

5.7 总结：编写并发程序要注意些什么

5.7.1 锁粒度

锁粒度这个问题，在前文中有许多探讨，本小节专门将它抽出来讲解，说明它很重要，至少胖哥认为它很重要。胖哥认为，如果你一旦踏足于多线程编程，就肯定需要面对这类

问题。不论多么优秀的框架已经包装好了，如果用得不适当，同样会出现各种古怪的问题。举几个简单例子。

- ◎ 前文中提到 `Vector` 或 `CopyOnWriteArrayList` 是线程安全的，只是说它的多线程发生单次读写不会出现问题，并不代表两次读/写操作过程中没有任何其他的线程写入数据。
- ◎ 同样的 `Atomic` 系列的类是实现了线程安全操作，但是并不代表对 `Atomic` 多次写入过程中没有其他的线程写入。
- ◎ 线程池和各种队列很好用，但是如果使用不当也会导致各种各样的怪问题。

这里需要我们进一步去理解“粒度”的问题，换句话说，技术是个好东西，但更加重要的是使用的人，如果使用的人没有理清楚基本的道理，再好的技术在他手上都无用武之地。

胖哥认为你想成为老 A 级程序员这是必然要经历的，即我们在理解原理细节的基础上，再去理解所处理的问题细节点，就能相对较好地掌握粒度。

在讲解 `synchronized` 时就提到过一些粒度的问题，在生活中比较暴力的是一个人进入食堂吃饭要锁住整个食堂的门；类似的，如果进入一个生活小区没必要只让一个人在小区里面，等到他出小区的时候再让下一个人进去。在生活中我们不会犯这样的错误，因为这样做很浪费资源，但是程序中通常会这样做，因为想要程序简单，或者不了解锁到底是什么，它到底锁住了多少东西。

解决这类问题可以对锁操作做一些细化，初步细化的自然是按照读/写分开。此时，我们简单地把读操作比喻成要看房源的人，而写操作是楼房建造者或装修队。如果有人看房可以随便看，可以多个人一起看房子，不过装修队正在做某些装修时，暂时不允许别人进去，而有人看房时也不允许装修队进去装修。这种现象类似于读/写锁，`CopyOnWrite` 也是类似的，并且它允许在写的时候照常发生读操作。

从另一个角度细化锁的操作，是将锁打散，什么意思呢？用前文中提到的小区例子来讲，或许可以打散到楼号、单元号、楼层、门牌号、房间甚至床位等。在 Java 语言中有体现吗？当然有，例如 `ConcurrentHashMap` 默认分解了 16 个 `Segment`，每个 `Segment` 内部去加锁（这个锁机制和读/写锁很类似），而当一个写请求到来时，首先会被分解到不同的 `Segment` 上面去，这样锁的粒度就在理论上降低了 16 倍。

还可以继续细化相互之间的互斥粒度，当需要通过 `synchronized` 或 `Lock` 来锁定一段代码区域时，除了考虑它们锁定的对象是什么以外，还需要考虑是否可以将范围缩小一些。我们举个例子。

前文中提到在类的静态方法前面加 `synchronized`，相当于锁住的是对应类的 `Class` 对象，此时在任何一段代码中使用 `synchronized(XXX.class)` 或该类的任何一个静态方法增加 `synchronized` 修饰后，相互之前都是互斥的，这显然有点痛苦，有些时候想要将不同的方法之间的互斥拆分开，此时可以定义多个对象，然后让不同的方法锁住不同的对象。

同样的 `synchronized` 修饰在普通方法中相当于锁住 `this`，而作用范围是整个方法体，这时如果是在一个大方法中，或许可以考虑范围是否可以缩小一些，其中一种方式是拆分子方法，对某个必要的子方法加锁；另一种方式就是通过锁块来隔离部分代码段。

例如：某个远程终端提供某种服务，接收请求后处理一段时间返回处理结果，但它在 1ms 内最多接收一个请求，发送方需要控制速度，否则可能会丢失数据。

为此，最直接的方式就是在交互的方法上加上 `synchronized`，但是后来发现，整个系统

就像单线程系统一样在这里卡住了，最主要的原因是远程终端处理每个请求时需要花上几分钟、几十分钟，甚至几个小时都有可能，针对这个问题我们就需要考虑怎么拆分锁粒度了。

我们可以尝试将 `synchronized` 的粒度放在发送代码段上，每个发送的代码在发送前延迟 1ms，就可以将其隔离了。

继续细化，如果远程终端非常多，例如有成千上万个终端，即使将代码范围缩小，但是不同的主机之间的发送还是串行的，或许每个终端之间相互不影响，那么还可以按照主机将锁打散，即按照主机级别存放相应的 Lock，在这个级别上做相应的锁（这样会不会使得大量的主机信息存放在内存中导致性能问题呢？其实关键看如何使用，IPv4 结构的地址可以按照 4 字节来存放，IP 数在万级别还不足以造成内存问题，如果遇到十万、百万的机器数，则需要考虑 LRU 机制来控制）。

如果再细化业务，则可以考虑不同的端口实例是否相互不影响，不同的业务类型相互之间是否不影响。

补充：对于每个终端处理较长时间的问题也可以通过异步 I/O 来解决，但后面对主机、业务类型等细化锁粒度的拆分在技术上是不会满足的，换句话说，始终需要根据场景来改造才能达到较好的效果，同时也体现了人的智慧。

结合业务背景，通过思考去完成这样的事情时，许多优化方法就会变化多端，下面再举一个例子来说明问题。在许多单例程序中通常会有这样一段代码：

```
private final static Object object = new Object();
private static A a;
public static A getInstance() {
    synchronized(object) {
        if(a == null) {
            a = new A();
        }
    }
    return a;
}
```

这段代码没有什么问题，肯定要加 `synchronized`，否则会发生 Race Condition 的问题，但是它在并发编程时会使得每一次获取调用 `getInstance()` 方法时都会做一次加锁操作，这个开销是巨大的，如果存在征用则是悲观锁。

这是一个让人纠结的问题，这里不得不加锁，但是只想在运行时加锁一次，大家想到的办法就是“双重锁检查”（double-checked locking），于是修改后的代码如下：

```
private final static Object object = new Object();
private volatile static A a;
public static A getInstance() {
    if(a == null) { //先做一次无锁判定
        synchronized(object) {
            if(a == null) {
                a = new A();
            }
        }
    }
    return a;
}
```

代码修改后有点丑陋，甚至会被许多讲究规范的同学所鄙视，不过在并发编程时它确实是一种很好的办法。这样修改以后在第一次 `a == null` 时，多个线程依然会征用 `object` 对象，`a` 一旦被赋值，就不会进入 `if(a == null)` 的内部代码中，也就是根本不会再进行锁的操作。

作（这种方式在提到线程池多次判定 `poolSize` 与 `corePoolSize` 的大小时有所体现）。

这种方式虽然可以解决问题，但是并不觉得它有多好，很多时候我们还是推荐在初始化时将这部分工作完成，`getInstance()`是直接返回值的一种方式，而不是在这个方法内部再去初始化，代码可以修改成这样：

```
private final static A a = new A();
public static A getInstance() {
    return a;
}
```

其实很多时候代码都可以修改成这样，在前文中提到，这样的对象初始化是在类的 `<clinit>` 方法中完成的，它会确保初始化过程中的线程安全。很多时候我们容易被一些模型所误导，也可能是因为“别人都是这样写的”，当然也不排除某些复杂对象确实不方便构造，这些其实在现代基于容器的开发中都可以实现，例如可以借助于 Spring 的初始化注入，以及在自定义的类中做一些 `static` 匿名块等方法。

如果遇到了这样的情况，又需要用到双重锁判定，那么可以用上面提到的方法，记得加上 `volatile`，以确保 `a` 的赋值可见。为何这样说呢？其实最外层的判定是没有加锁的，如果没有可见性的保证，而对象的创建会有分配空间、初始化、赋值给外部引用等几个动作，在单线程中赋值给外部引用和初始化属性的重排序并不会影响外部的正常使用，但是在多线程中另一个线程可能看到一个属性没有初始化好的对象，所以我们通常会用 `volatile` 来保证可见性，这也是 JDK 1.5 以后对 `volatile` 语义增强后的定义，在 JDK 1.5 以前还得通过一些其他的方式来解决这个问题（如果这个对象属性还是复杂对象，那么它无法保证复杂对象的属性初始化全部完成，这个时候就需要进一步的处理）。

将锁的粒度和范围细化，也未必绝对是好事，例如不会将一个本身需要锁 10 行代码的锁，改为逐行做 10 次加锁。换句话说，我们所倡导的并不是一味地将不能拆分的“锁”进行拆分。

例如，前文中提到的 `StringBuffer` 在连续多次进行字符串拼接时，它就间接地加了多次锁，在我们看来，这个锁的粒度是很细的，但它的范围其实是很大的，而且每个步骤都有进入锁和出锁的动作，所以 JVM 后来对锁本身的优化有一个粗粒度的动作，我们自己写代码时尽量不依赖于 JVM 这种优化机制。换句话说，我们在不是十分了解 JVM 某个版本的实现细节时，无法控制 JVM 是否会对自己编写的代码做这样的优化，而且这种优化是否还会在下一个 JVM 版本中延续。

既然谈到锁的粒度，那么就再向极限靠近点：如何做到无锁状态下的管理？有人提出一种环形队列（`Ringbuffer`），也就是它在逻辑上是首尾相接的，理论上可以通过数组或链表来实现。

假如是数组，写入元素就像一个生产者，它在移动下标时，可以使用一个不断叠加的值与队列的长度求模，得到真正的数组下标来形成环形的目的。消费者也有一个单独的叠加值，使用同样的方式获得数组读取的下标，也是不断地增长并与数组长度求模。此时如果只有一个生产者和一个消费者理论上不用加锁，我们认为是一对绝配，只是对于多个消费者、多个生产者在操作同一个下标时会通过 CAS 自旋方式来达到目的，但是这种锁的粒度就非常小了，或者说在理论上它有机会去实现，在前文提到的多种队列中已经有许多实现的影子。

如果用链表实现通常需要保留一个 head 和 tail 引用，生产者不断制造对象并修改 tail，而消费者不断从 head 头部取出对象。在有多个生产者和多个消费者的情况下，使用链表的实现方式是：多个生产者通过 CAS 尝试将 tail 指向当前节点，若成功则将原先的 tail 节点的 next 指针指向自己。同样的，多个消费者尝试将 head 修改为 head.next 节点，修改成功后原本的 head 节点就可以被取出了。

这种方式表面上看很理想，实现起来却有很多问题，例如刚才提到的消费者比生产者的速度快，或者生产者过快超越消费者达到一周后，此时就不得不介入许多重试机制，在多次重试后太快的一方不得不处于等待状态以节约 CPU 资源，如果处于等待状态，那么无锁化问题就没有得到解决。简单来说，生产者和消费者之间的速度通常很难做到完全的平衡，许多时候解决方案是尽量向锁力度偏低的方向上靠。

不过回过头来想一想，如果我们尽量加大这个数组或链表的长度，这样可以在一定程度上缓解生产者过快的问题，但是它不可能会无限增加，因为那样可能会导致内存不够。在有限的长度下，两者有速度的差距就始终会有碰撞的一天，例如生产者超过消费者一周，在这样的情况下，通常的方式不是让消费者取出一个数据就激活生产者，因为生产者本身就很快，激活后立即又追上了消费者，在不影响业务的情况下，可以让生产者空闲一小会，甚至于帮着消费者做点事情，这样拉开一定距离后，就又需要一定的时间才会追上。

最后我们还可以通过增加慢的一方的线程数来尽量平衡相互之间的速度，在经过实际场景的调节后发生等待的概率开始降低。

5.7.2 死锁

死锁是研究并发编程必须要知道的，死锁的情况多种多样，而胖哥希望大家所认识到的是，所谓死锁都是因为“互掐”导致的，也许是暗中“互掐”，也许是太明显的“互掐”，死锁并不是每一次都能重现，是偶然发生的，所以代码中有这样的问题很多时候不会注意到。换句话说，我们的代码中存在问题，只是可能还没有爆发。本小节我们来了解死锁的一些道理，以避免代码中出现这样的情况。

例如两个人喝咖啡，一个人有水杯，一个人有咖啡，但是谁都不给对方，结果两个人都喝不到咖啡，而在代码中会如何体现呢？

线程 A 执行的代码：

```
synchronized(object1) {  
    ...业务操作  
    synchronized(object2) {  
        ...业务操作  
    }  
}
```

线程 B 执行的代码：

```
synchronized(object2) {  
    ...业务操作  
    synchronized(object1) {  
        ...业务操作  
    }  
}
```

如果线程 A 将 object1 锁住，线程 B 同时将 object2 锁住，此时线程 A 进一步需要锁 object2，线程 B 进一步需要锁 object1，它们相互锁住了，这是一种最基本、最简单的死锁规则，也是理解死锁的基础，其他的死锁场景都可以从中演变而来。虽然胖哥说得简单，

但实际的工作场景会复杂许多。例如：

- ◎ 多个线程乱序征用某些对象锁，当多个线程之间的锁征用依赖形成环形结构时，就可能会引起死锁（上述例子只是简单的 2 个线程对 2 个资源的交叉征用）。
- ◎ 容易忽略对 `this` 加锁，也是对一个对象加锁，在这个过程中去调用其他拥有锁的方法，就可能导致不容易发现的死锁。示例代码如下：

```
class A {
    private B b = new B();
    synchronized void fun() {
        b.fun();
    }
    synchronized void fun2() {}
}
class B {
    private A a = new A();
    synchronized void fun() {
        a.fun2();
    }
}
```

A 的实例当中包含了 B 的实例，B 也包含了 A 的实例，当 A 类中调用 `fun()` 方法时，会间接调用 B 类中的 `fun2()` 方法，这没有什么问题。但如果在这个过程中一个线程访问 A 中的 `fun()` 方法成功，获取到 A 的实例对象（`this`）的锁，存在另一个线程访问 B 的 `fun()` 方法，也同样获取到 B 的实例对象（`this`）的锁，就出现“互掐”现象了。

- ◎ 父类方法调用子类方法，子类直接或间接地再调用父类的方法，在这个过程中可能会发生某个方法级别加锁，也就是更加隐藏地锁住 `this`，某些方法中还可能对外部某个对象进行了锁操作，同样可能产生“互掐”。

上面提到的仅仅是一些简单的死锁例子，还有一些更加隐藏的死锁。例如：下面这段代码就可能会发生死锁。

```
void fun(Object from, Object to) {
    synchronized(from) {
        synchronized(to) {
            ...业务代码
        }
    }
}
```

这样的代码给人的感觉是没有任何问题的，因为加锁的对象是局部变量。但如果 `from` 和 `to` 是从某个共享的数据结构中取出来的，那么这里就有可能会出现问题，我们来举个例子。

假如有两个线程都持有了 A、B 两个对象，其中一个线程传入的参数是 A、B，另一个线程传入的是 B、A，此时就动态地出现了“互掐”情况，这种死锁我们把它叫作“动态死锁”。换言之，虽然从程序看到的是 `from`、`to`，但是根据传入参数的顺序交换就变相导致共享对象的交叉锁。

要解决这个问题，首先要深究数据的来源，有很多来源隐藏得很深（例如传递 2 个 `Integer` 对象，传递的 2 个值是被 `Cache` 在内存中的值，加上无意中使用了自动装箱，问题变得更加不可预测）。通常我们希望这样的对象不是共享的，而是创建新的对象或使用拷贝来达到目的。如果数据必须是从共享的内存区域获取的，该如何处理呢？此时可以在加锁顺序上固定化。如何固定呢？自然是排序，按照某种维度进行排序，例如 `hashCode` 排序、简单字符串排序或某种业务数据排序，通过时排序的结果逐个加锁，使得不论 A、B 传入

的顺序是什么样的，它们加锁的顺序都是一致的。当传入 3 个、4 个或更多的参数时，这类问题会隐藏得更深，处理的方法一样。这是一种解决方案，但是并不代表这样做是好的，很多时候我们还是希望通过细化分析业务背景，进一步通过隔离资源来达到某些目的。

最后再来谈一种经典的“饥饿死锁”情况。

前面提到了 fork/join 工具，如果没有这个工具，使用 fixed 方式的线程池来实现类似的功能，即使用固定大小的线程池去实现，就有可能会出现“饥饿死锁”情况。

例如，在任务分解的过程中，任务会越来越多，当任务非常多的时候需要将子任务放入到 BlockingQueue 中等待线程去执行，固定大小的线程池是有线程数量上限的，放入到阻塞队列中说明线程已经不够用了，“父任务”目前是要等待“子任务”返回结果，它自己的线程资源无法释放出来，“子任务”在等待线程调度，一堆任务得不到线程资源，它们在这里“互掐”上了，由于它们是在这种“饥饿”中死去的，所以通常叫作“饥饿死锁”。

虽然通过 Cached 类型的线程池可以解决，但会创建大量的线程去处理这个问题。

我们可以用代码清单 5-14 中的 CountdownLatch 来说明一个更为形象的例子，将代码中启动线程的方式修改为 fixed 方式，线程数设置得要比任务数少 1 个，此时运行代码后就会发生“饥饿死锁”。因为代码中需要等待所有的“赛跑者”都准备好以后才开始赛跑，有一个任务得不到调度，其他的任务一直占用线程不释放，就会导致“饥饿死锁”。这也是在前文的许多例子中，胖哥没有使用固定大小的线程池的原因之一。

当然，死锁不仅仅是本小节例子中的这样，还有许多场景，例如在 Lock 的使用中，如果没有做 unlock() 操作或没有将 unlock() 操作放在相应的 finally 里面也会导致死锁；如果一个操作中由于某种原因对同一个 Lock 对象发起了多次 lock() 操作，那么 unlock() 的次数应当与它相等，否则也会造成死锁。

所以在使用 Lock 时，有些时候是使用 tryLock()/tryLock(long,TimeUnit) 的方式尝试去加锁的，如果锁不住则退出，以释放已经锁住的内容。处理的方法是，可以根据业务背景抛出异常或者休眠一段时间再去尝试，这也是与使用 synchronized 的最大区别之一，因为这种方式有选择退出的余地。前面提到了降低锁的粒度，如果粒度降低了征用自然就少了，征用少了出现死锁的概率自然就低了。

►► 5.7.3 “坑”很多

Java 并发编程中的“坑”是很多的，在编写程序的过程中稍有不慎就可能会出现稀奇古怪的问题，在前面提到过由于锁的粒度不当可能会使得系统中的征用加大，也提到了死锁，而实际情况不仅仅如此。我们要学会运用基础知识对实际问题进行实际分析，在本小节再介绍几种在并发程序中由于代码使用不当可能会埋下的“坑”。

(1) Slipped Conditions

详细请参看：<http://tutorials.jenkov.com/java-concurrency/slipped-conditions.html>，代码如下：

```
private boolean isLocked = true;
public void lock() {
    synchronized(this) {
        while(isLocked) {
            this.wait();
        }
    }
}
```



```

    synchronized(this) {
        isLocked = true;
    }
}

```

这个例子是一个简单的 Lock 实现，它内部希望用一个 isLocked 属性标识来标志当前是否为锁定状态，多个线程进去征用 lock() 方法时，synchronized 区域控制只有一个线程能成功。获取到锁的线程如果发现 isLocked 为 false，则直接跳出循环，并将 isLocked 修改为 true，其余的线程进入后由于 isLocked 为 true 就会进入 while 循环，并调用 this.wait() 等待。当获取到锁的线程发生 unlock() 操作时，isLocked 将会重新修改为 false，并调用 notify() 唤醒一个处于 WAITING 状态的线程，再一次循环判定 isLocked 是否为 false，这样就实现了串行化。

直接解读起来貌似没有什么问题，不过问题就是出在了这里！

假如多个线程同时征用第一个 synchronized 代码块，此时只有一个线程能顺利进入，并且当 isLocked 为 false 时直接脱离第一个 synchronized 代码块，自然的会有另一个线程会进入第一个 synchronized 区域，脱离 synchronized 代码块的线程还没来得及将 isLocked 设置为 true，此时第二个线程可能就读取到了 isLocked 为 false，同样也跳出了循环。在设计层面我们要求其他的线程必须在做 unlock() 后才能进入，但是现在出现了两个线程同时抢到资源的可能性。第二个线程是偷偷摸摸地溜下来了，导致了一致性存在问题，通常把这种现象叫作 “Slipped Conditions”。

这个问题也是一个锁粒度的问题，也同时说明了锁粒度拆分未必是什么好事，至少不能拆分逻辑紧密相关的东西。同时，这种情况往往在工作中容易被忽略或者被草率处理，所以我们如果从事这方面的工作，就需要更加深入地去认识一些问题的细节，多思考得出自己的见解。

(2) 在并发编程中使用非线程安全的数据结构

许多资料上会介绍，对很多类型的对象的操作是非线程安全的，例如 ArrayList、LinkedList、HashMap、HashSet、StringBuilder。但我们首先要确认的一点是，如果没有写操作则都是线程安全的，只有读/写同时发生时才会出现不安全的情况。

许多同学可能在想这些数据结构到底怎么不安全了？平时也用过这些集合类来做共享数据缓存，但也没发现什么问题呀，难道概念被推翻了？胖哥也曾经经历过这样的事情，后来才发现以前的理解是有问题的。在这里胖哥就从源码上去讲解这些集合类为什么不安全？为什么有并发的问題？

接下来的内容以 HashMap 集合类为例，假如在某个应用中存在多个线程对它进行读/写，通常会导致的问题有：数据错乱、丢失数据、在极端情况下会导致死循环。

为了说明问题，先简单回顾下 HashMap 数据结构。在图 1-5 中提到了 HashMap 存储结构，当时为了让大家了解用了比较简单的“狗狗找队伍”来说明 Hash 的基本原则。在实际存储过程中 HashMap 连接各个队伍的是一个数组，每个数组元素存储了一个队伍的队头，每个队伍其实是一个单向链表，用数据结构的形式来表达，有点类似于图 5-61 所示的这样。

这里的每一个 Entry 就代表了一个数据，包含了 Key、Value、Next 引用、Hash 值，查找数据时，会根据传入对象的 hashCode 运算出一个 0~3 之间的数据，定位到下标，这样就将数据的范围缩小了 4 倍，在链表上用 Next 遍历 Entry，找到的 Entry 取出 Key 值与传入的 Key 值做 equals 对比，成功则认为找到，并返回 Entry 中的 Value 值；否则继续找 Next

节点，若到链表尾部都未找到，则返回 null。

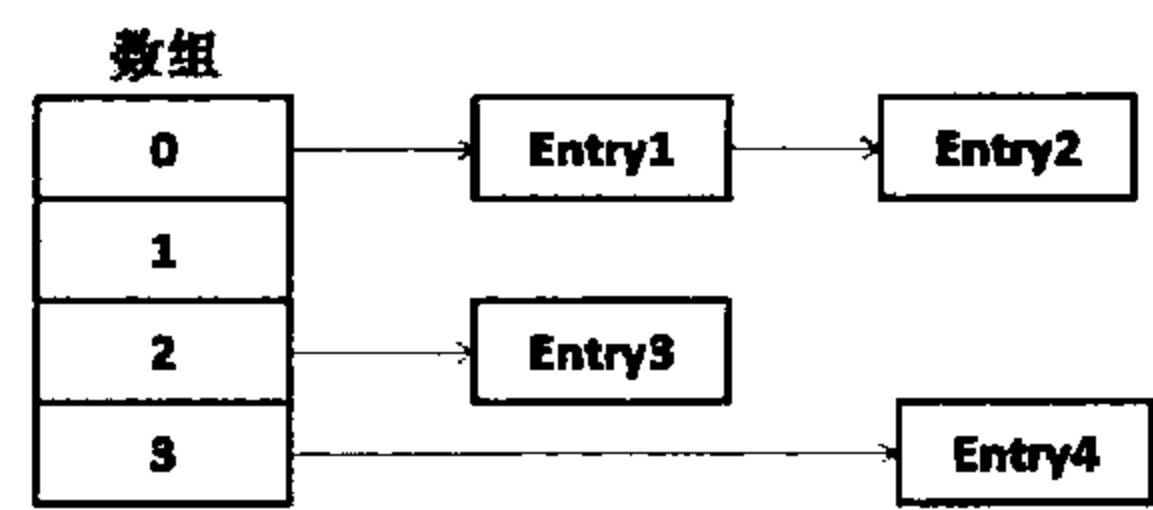


图 5-61 HashMap 数据结构的简化表示方式

同样的，要写入一个元素（put 操作），也会用类似的方式查找 Key 是否存在，若存在同样的 Key，则直接将相应的 Entry 中的 Value 值替换为新值（这不算丢数据），并返回旧值；如果没有找到相同的 Key，则会在对应的链表头部写入该元素。一个节点插入到单向链表头部，如图 5-62 所示。

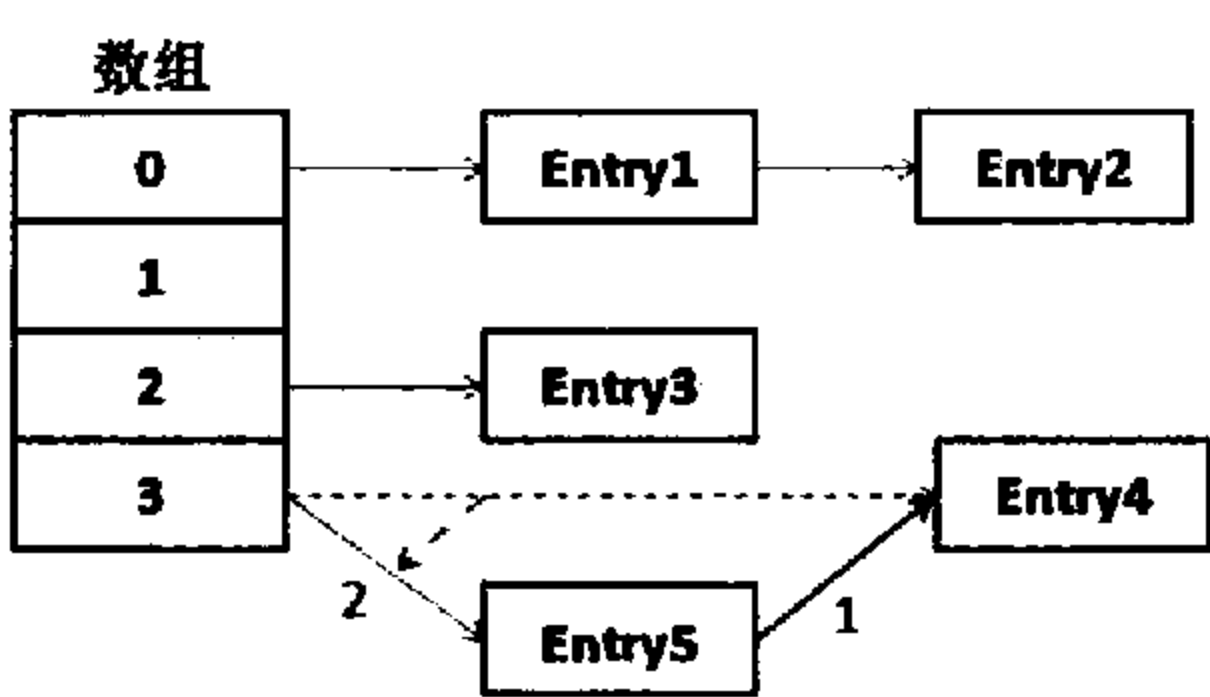


图 5-62 找到数组下标 3，写入链表头部

如果插入节点时 Hash 表的节点总数大于“threshold”，则会 resize 这个数组，也就是数组大小会发生扩容，通常扩容为原数组的 2 倍，在扩容的过程中会导致同样的元素计算出的数组下标发生改变，因此需要遍历原有的 Hash 表的所有节点，即遍历每一个单向链表，每个节点将按照新的 Hash 表来计算出自己的位置，逐个迁移到新的 Hash 表上，我们通常把这个过程叫作 rehash 过程。

了解了基本过程，接下来分析几种并发的情况。

第一种情况：如果有两个线程同时写入数据，而且两个线程写入数据的 Key 是一样的，而这两个值都在 Hash 表中没有找到，此时在写入时有可能会这样，如图 5-63 所示。

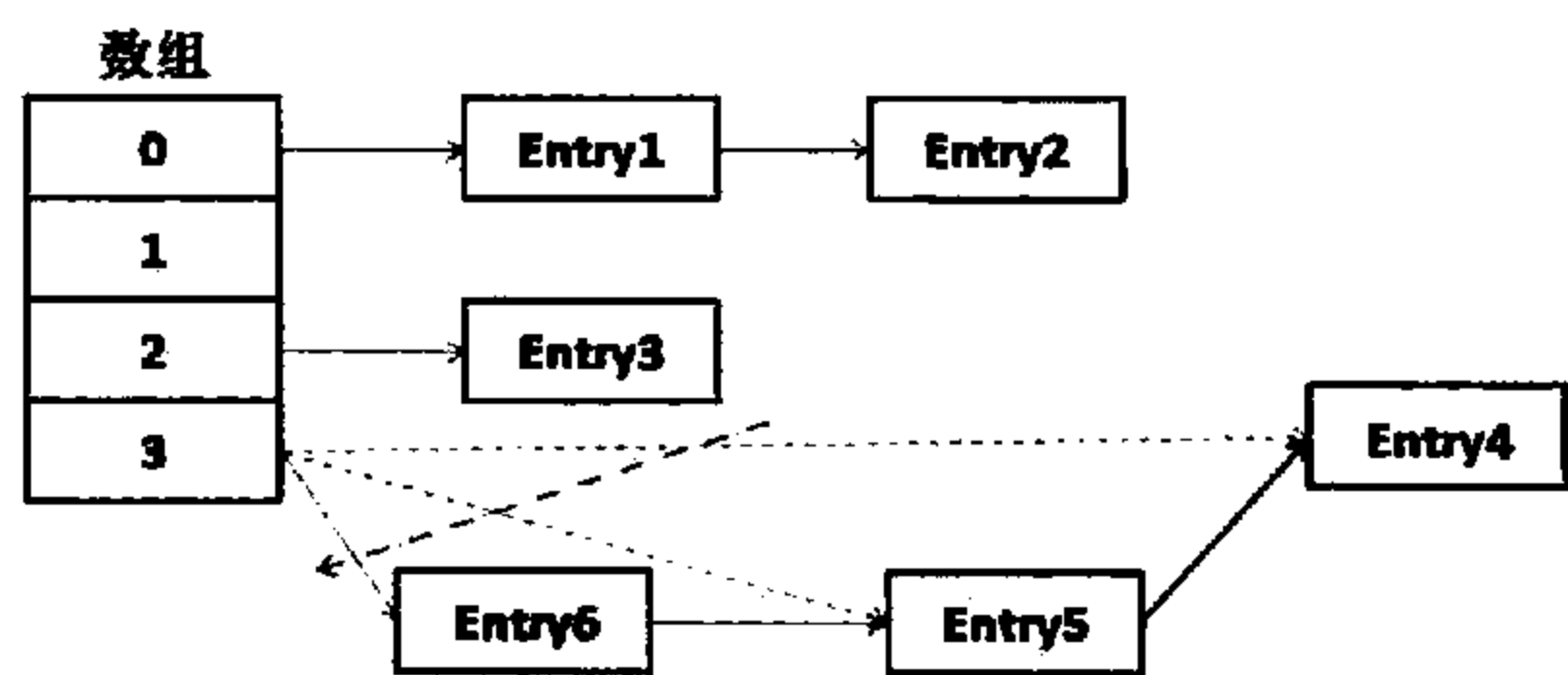


图 5-63 Entry6 和 Entry5 中的值是一样的，Hash 中出现重复 Key

由于两个线程都发现不存在这个 Key，而 Key 又是相同的，所以识别出来的下标是相同的，向同一个数组下标的链表中写入。假如写入时有短暂的先后顺序，则此时会形成图中的重复数据情况，Hash 表要求数据不允许重复，但是现在有可能会重复，所以我们把它叫作“数据错乱”。

第二种情况：当两个线程写入不同的数据时，虽然数据不同，但是根据 Hash 表计算出需要写入同一个单向链表中，并且它们几乎同时写入这个链表中。如图 5-64 所示，Entry5、Entry6 两个节点先将自身的 Next 指向 Entry4，然后再将它们自身赋值给数组下标 3，最终值肯定是其中之一，那么另一个节点就丢掉了，这就是并发时的一种数据丢失情况。

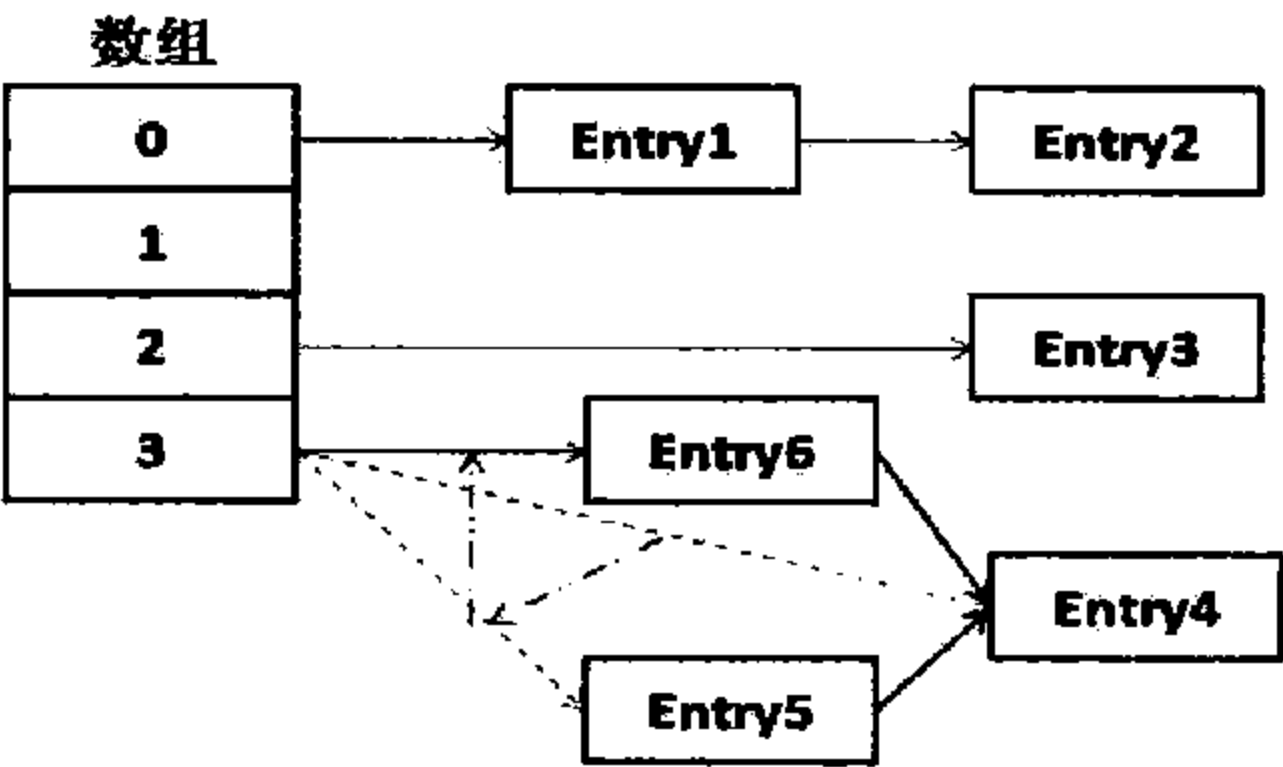


图 5-64 这里的 Entry5 丢掉了

第三种情况：还有一种比较复杂的丢失数据情况，它涉及 rehash 过程，此时简单绘制一个 rehash 的示意图，如图 5-65 所示（为了简单描述模型，会使用一些数字来标志 Hash 表中的 Entry 对象的 Key 值，用数字与数组大小简单求模得到数据应存放的下标，这样会更加形象一些）。

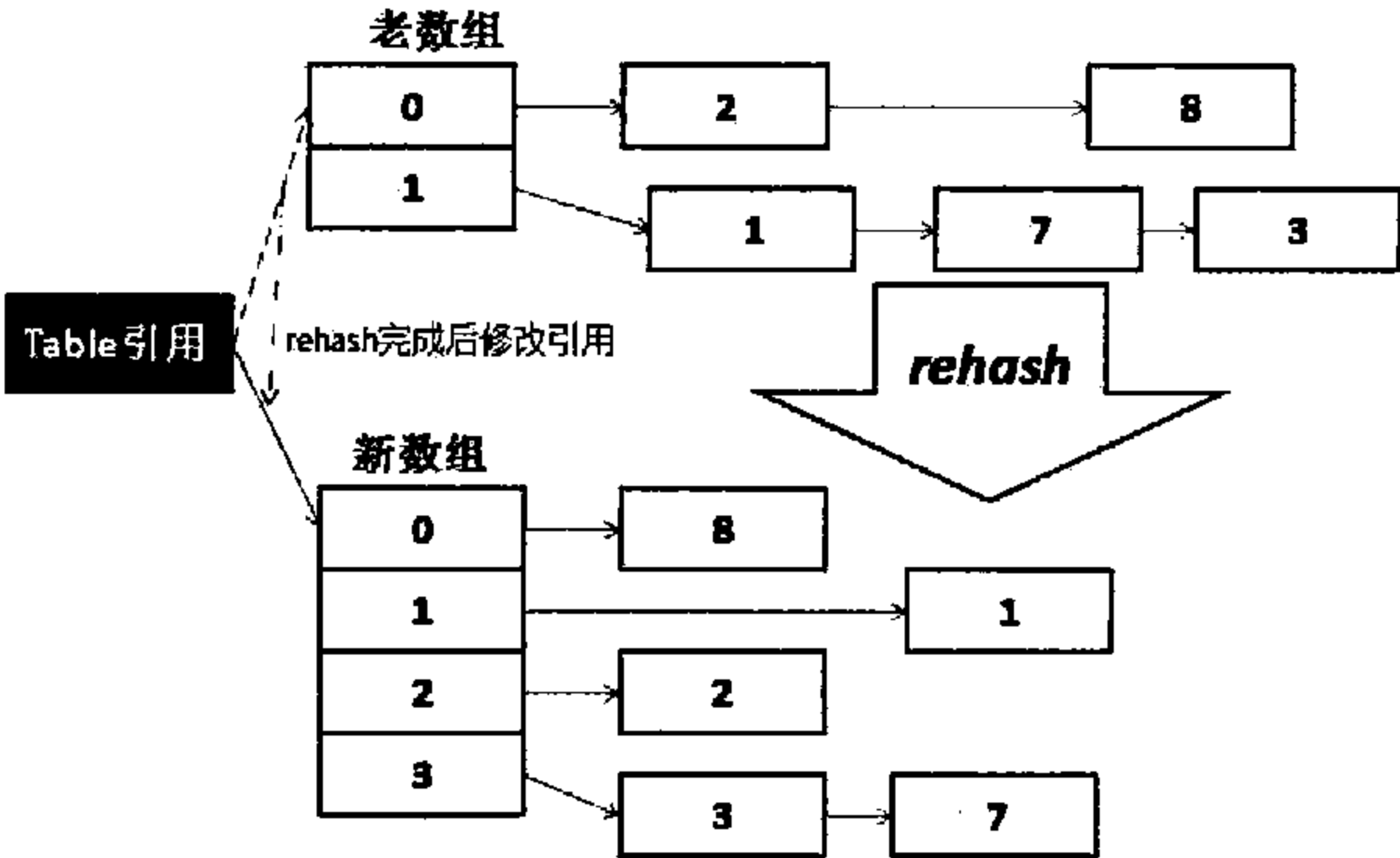


图 5-65 rehash 的简单示意图

rehash 通常发生在写入操作时，先做元素写入，发现元素个数较多，再决定做 rehash。此时假如有 A、B 两个线程先后写入数据，当线程 A 写入一个数据时，发现元素个数较多需要 rehash，创建了一个新的 Hash 表，开始遍历，遍历到图中的“元素 2”时，正好线程 B 将元素插入到了数组下标 0 当中，即在“元素 2”所在的单向链表最前面插入一个数据，此时线程 A 做 rehash 时不知道这里写入了一个元素，将这个元素丢弃掉。

小伙伴们会问：线程 B 也会做 rehash 动作，最终会不会一致呢？下面我们再来详细分析一下。

如果线程 B 写入数据后，线程 A 的 rehash 动作也做完了——即用最新的数组赋值给 Table 引用，在这个瞬间，线程 B 获取到的 Table 也是最新的，它校验长度没有问题（因为新数组已经是 2 倍长度），不会做 rehash 动作，所以它自己也不知道数据丢掉没有。由于获取到的 Table 引用是最新的，即使因为某种原因导致它认为现在需要 rehash，也找不到开始写入的数据了，因为新的 Table 中没有这个数据。

如果线程 B 写入数据后，线程 A 的 rehash 动作还未完成——由于 Table 引用还未修改，看到的依然是老数组，所以发现长度不够，也需要做 rehash，就是基于老数组在做 rehash 了（抛开第三个线程进来搅局，否则就彻底乱了）。在这个过程中线程 A 做完 rehash 后将 Table 引用修改为线程 A 创建的新数组，线程 B 还在对老数组做 rehash，根本不知道线程 A 创建的新 Hash 表是否发生了写入操作。换句话说，线程 B 对老数组做 rehash 这个过程可能会需要一段时间，在这个时间范围内新的写入是对线程 A 创建的新数组进行的，线程 B 做完 rehash 后依然会将 Table 引用修改到新数组上，在这个过程中相互之间并不知道，自然的丢失数据十分严重，而且 rehash 的时间越长，丢失数据越严重。

退一步说，即使数据都不丢失，如果多个线程同时去做 rehash，在 JVM 中就会创建非常多的垃圾内存，这也是我们不想看到的。

第四种情况：死循环是一种极端情况，但是它确实可能发生，这一点在许多网站上都有介绍，尤其是在一些牛人的博客中。在此只是抛砖引玉，引导一下思路，说明出现死循环的原因。为了说明这一点，我们需要来看看 rehash 过程的源代码，如图 5-66 所示。

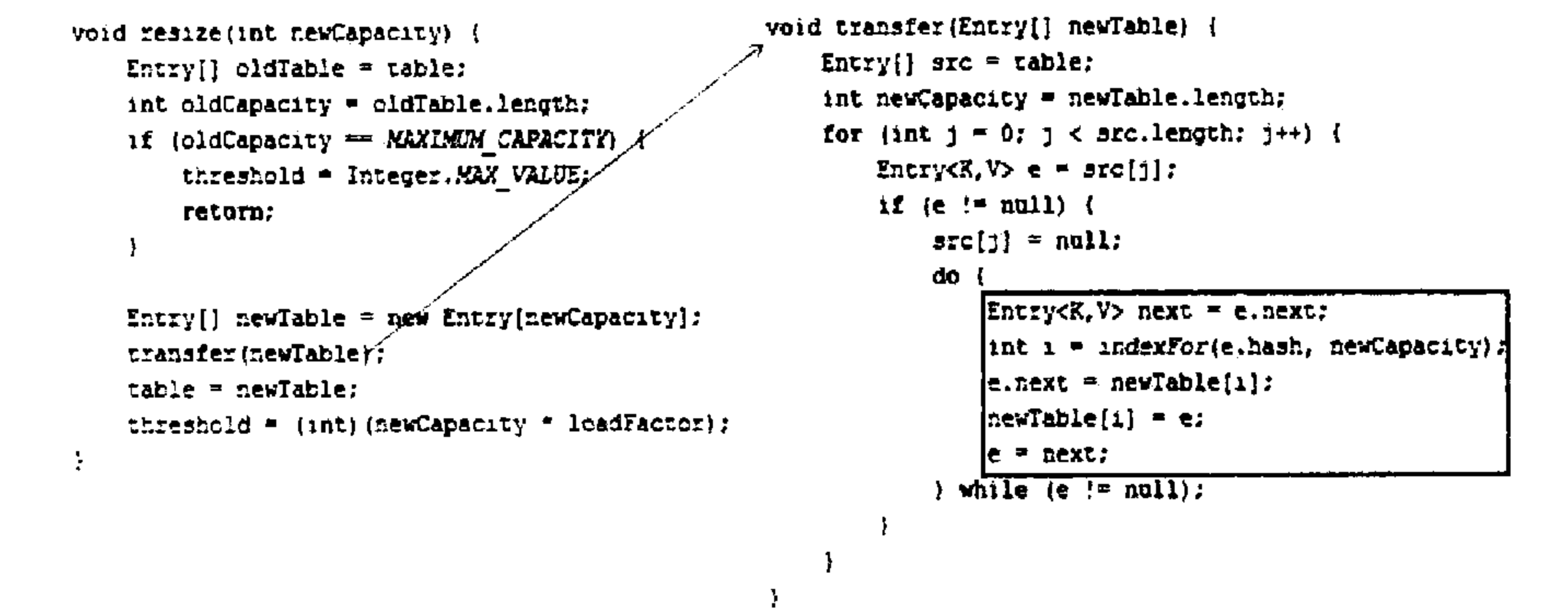


图 5-66 rehash 过程的源码

代码中关于数组扩容、遍历写入、赋值给 table 引用体现得十分明显，不再多说了，这里主要关注被圈出的代码部分。

- 这几条代码很简单，但是为了分析问题需要梳理一下。
- ◎ 首先将当前的 e.next 引用赋值给一个本地变量 (next)，一定要注意这是一份 e.next 的拷贝。
 - ◎ 进一步通过 e.hash 在新的 Hash 表中计算下标，找到下标后，通过后面两条代码，插入到对应下标中链表的头节点上。
 - ◎ 将 next 赋值给当前节点 e，以便于遍历下一个节点。

注意 最初将 e.next 赋值给一个本地变量 (next)，是因为元素插入到新的 Hash 表后，e.next 的值将会被改变为新链表的下一个节点。了解到这一点，就可以来分析什么时候会产生死循环了。

此时假如有两个线程 A、B 分别创建了两个新的 Hash 表，都进入了 transfer 方法，也

就是说，它们都在遍历原先的 Hash 表。在这里做一个假设：两个线程都访问到图 5-65 中的“元素 7”，那么两个线程的本地变量都会保存节点 7 的 Next 引用“节点 3”，如果抛开其他的节点，只关注这个问题，此时的线程及创建的新数组情况如图 5-67 所示。

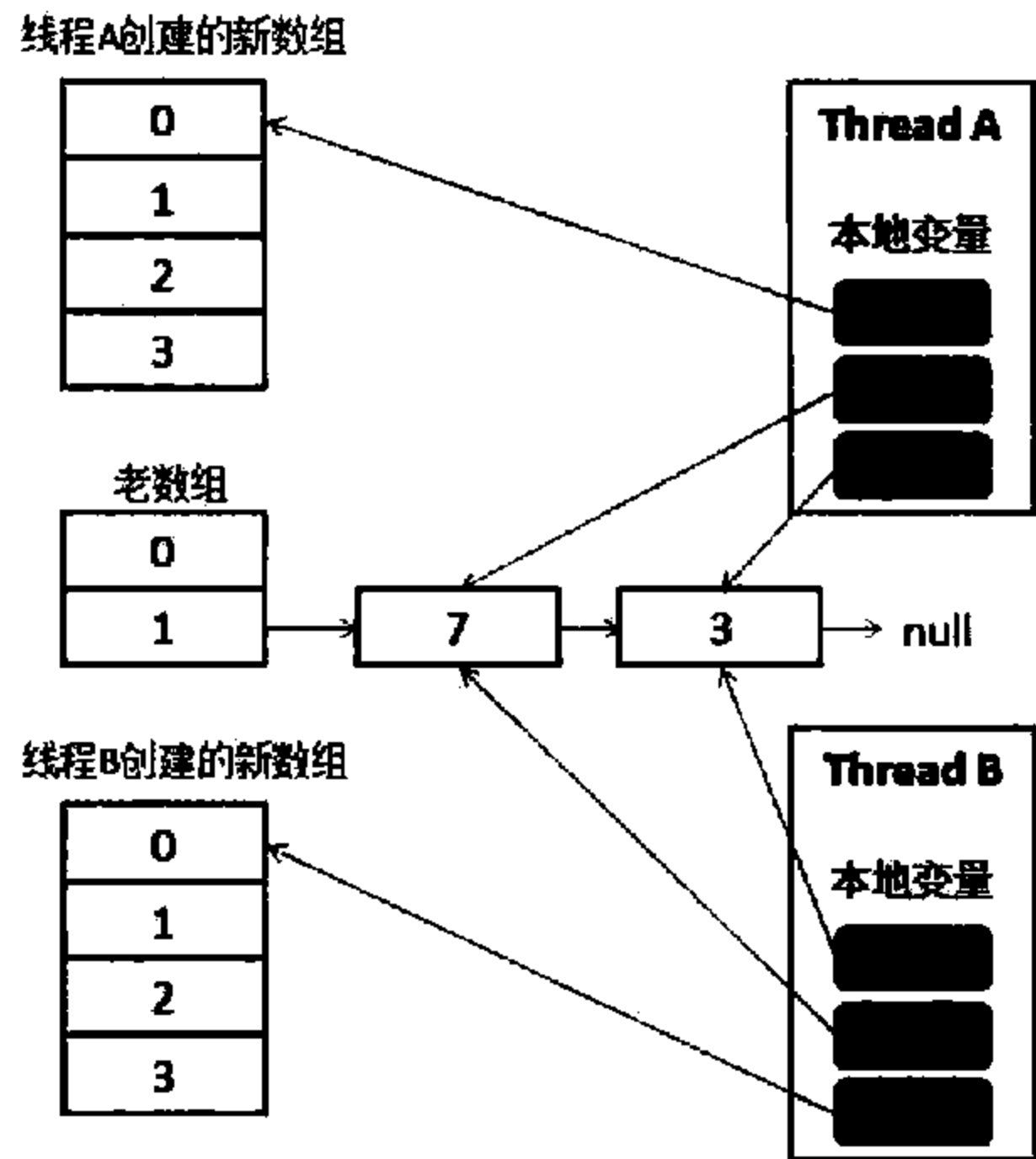


图 5-67 两个线程的本地变量中都保存了节点 7 和节点 3

此时若线程 B 因 CPU 调度的原因被暂停了一下，在这个短暂的时间内线程 A 将节点 7 和节点 3 都移动到新数组上，由于写入到新链表都是写入到链表头部，因此形成的链表结构是节点 3 指向节点 7，那么得到的结果就如图 5-68 所示。

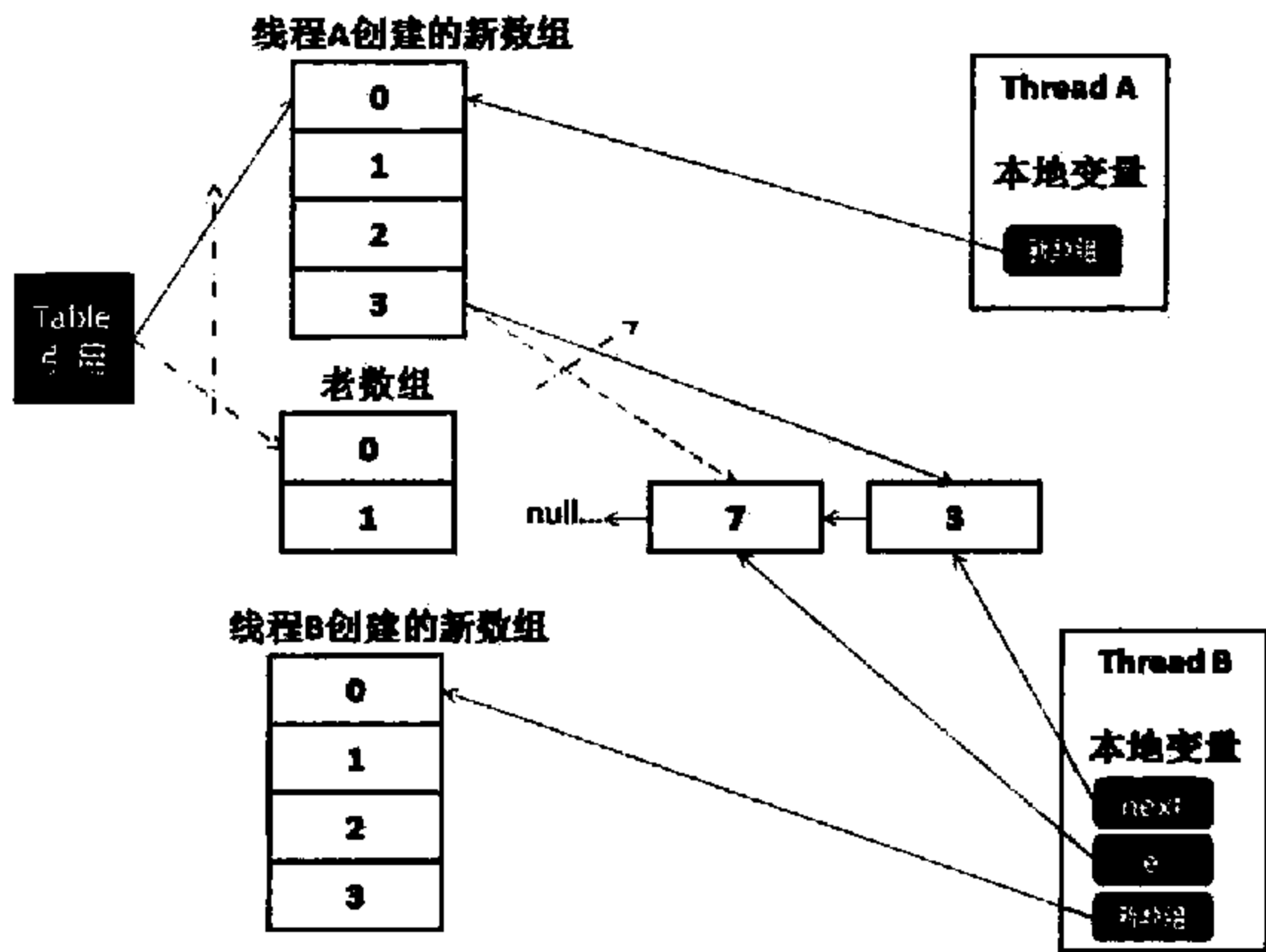


图 5-68 线程 A 移动完了，线程 B 在这里被卡了一下

线程 B 恢复计算后也会先移动节点 7，由于 next 本地变量保存的也是节点 3，所以还会继续遍历节点 3，节点 3 会再一次和节点 7 分配到同一个链表中，并插入到节点 7 前面。

在这之前，节点 3 已经指向了节点 7，根据图 5-66 所示的代码，在本次循环之前节点 7 又将被认为是节点 3 的 next，因此会被保存到 next 本地变量中。此时会将节点 7 再一次插入到节点 3 前面，即节点 7 的 next 指针会指向节点 3，此时节点 7 和节点 3 就相互指向了，如图 5-69 所示。

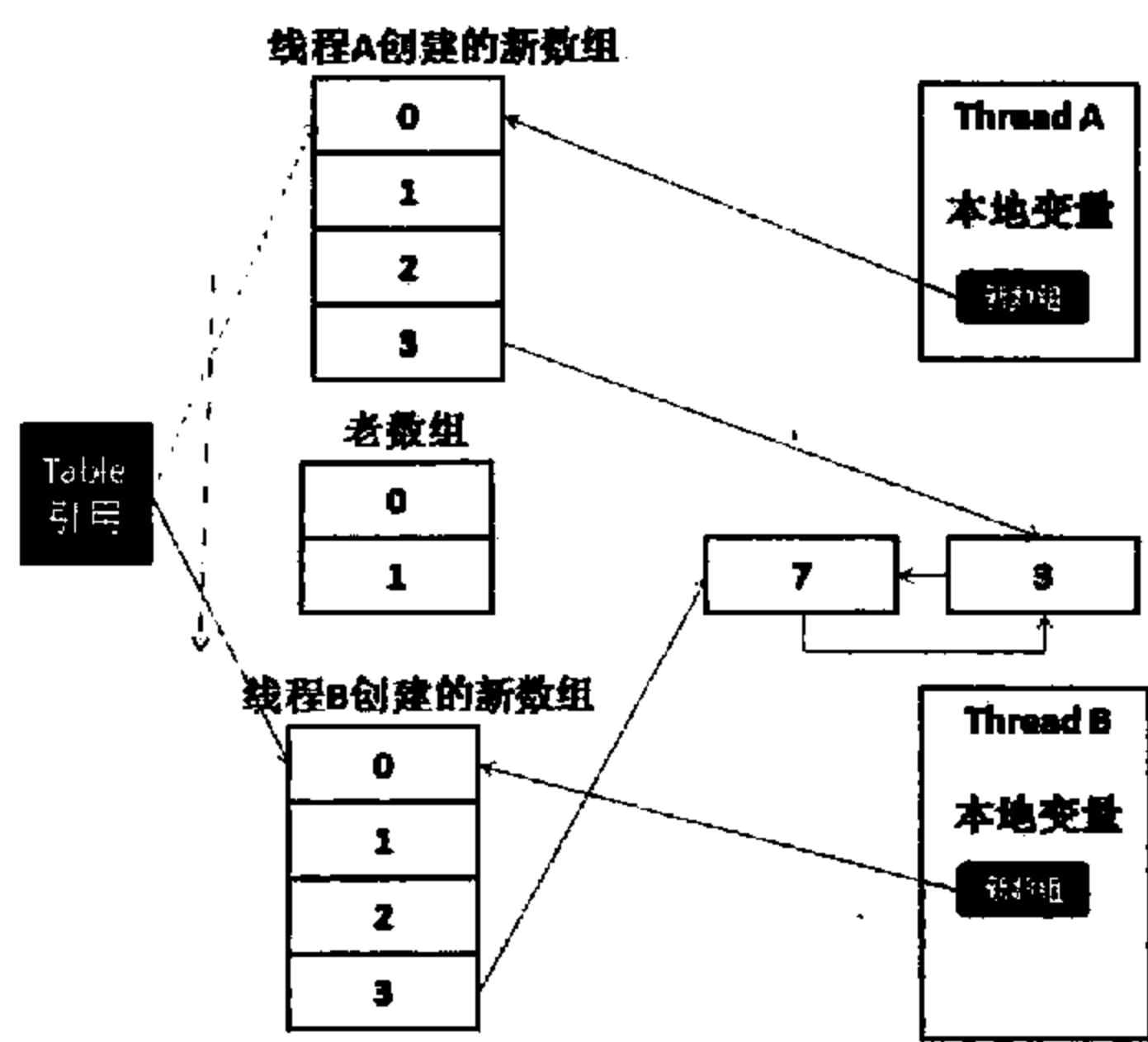


图 5-69 环形结构正式形成

不过在 transfer 方法过程中，在形成环形结构之前取出节点 7 的 e.next 是 null，将它保存在本地变量 next 中，因此 transfer 循环是会退出的，不会继续。但此时两个节点确实相互指向了，自然的如果发生任何查找操作（写入操作也会间接形成查找），在查找过程中访问到了这个链表就会进入死循环，在多线程系统中进来一个线程就多一个死循环的线程，逐渐就会把 CPU 耗死掉。同样的再次 transfer 时，遍历到这个链表时也会形成死循环。

这就是非线程安全的集合类以“共享”的方式提供给多线程访问的一些可能结果，正因为如此才叫非线程安全。

任何一个软件在发展的过程中都会遇到许多坑，而很多时候我们可能会不小心踩坑，但是在不断学习中提升自我才会让自己少踩坑。另外，要从踩坑的过程中学习到相关的原理和知识，而不仅仅是某个坑的处理方法，否则我们会不断地掉到新的坑里面去。

➤➤ 5.7.4 并发效率一定高吗

并发效率不一定高，从前面的介绍中我们知道并发有可能存在大量的征用问题、锁问题，根据本章的一些知识，胖哥做个简单总结。

- ◎ 当只有一个 CPU 时，程序基本都是计算类的操作，用多线程是无法提升效率的，带来更多的的是线程切换的问题。但是如果程序有大量的 BIO 操作，是可以在这样的系统中使用多线程的，因为在 I/O 等待的过程中，CPU 是空闲的，其他的线程依然可以使用 CPU 来处理问题。
- ◎ 当程序是十分简单的运算时（例如在一个大数组中求最大值、平均值），多线程的效果并不明显甚至不如单线程，因为创建线程本身就需要花时间，而这个时间可以计算许多的数字，因为计算数字对于 CPU 来讲完全是小儿科。
- ◎ 当用多线程从一个公用硬盘 I/O 中读取大量的数据时，希望以此方式来提升 I/O 读取的性能，基本上很难（除非是 SSD 硬盘可以考虑），因为硬盘 I/O 本身就是硬件中最慢的设备，并行的效果只会导致更多的征用。在 I/O 层面我们可以根据某些隔离条件使用多线程去处理，但隔离的粒度一定是基于一些场景下的测试数据，例如从远程数据库读取数据，到底开多少个连接并行拖数据不会影响在线业务，而且拖数据的性能是最佳的。

有的小伙伴从 I/O 中获取到数据后，通常还要做许多其他的动作，在这个过程中希望另一个线程来读取 I/O 中的数据，难道用多线程不对吗？

此时需要考虑下，做其他的操作需要多长时间，如果在 100ms 中用 90ms 或更多的时间做 I/O，剩余的时间做其他的事情，那么另一个线程等待了很久终于获取到锁了，该线程还没获取完数据，前一个做完事情的线程又回来请求数据，征用就会无休止地进行下去，其实就是为了 10ms，如果配置了 5 个、10 个线程，征用将会更加恐怖。

当然，如果发现做完 I/O 读操作后，处理数据的时间占比较大，这个时间浪费了不值得，那么可以尝试用生产者/消费者模式来平衡，让一个线程去读，读完后放入一个队列中（放入队列的时间相对 I/O 的时间来讲几乎可以忽略），所以这个线程几乎一直在做 I/O 读操作，这样在低级的磁盘上很多时候还可以实现顺序读/写操作，有效地利用了磁盘的带宽。当生产者读取数据太慢时，消费者等等也无所谓，不会形成征用；当生产者读取数据太快时，可以尝试增加消费者的线程，或者在达到某个高水位线时，需要让生产者等待一下（利用前文中提到的一些阻塞队列就可以实现）。

有的小伙伴可能会说，可以将文件分解为多个片段，多个线程分开读取每个片段，不过这需要磁盘的 IOPS 能力足够强才行，要知道普通的机械磁盘是串行的。另外，需要考虑这个文件是否足够大，需要我们去分段。

5.8 其他的并发编程知识

本章最后这一节为知识的补充，如果能将前面内容看完，那么看这一节将是非常轻松的，我们将工作中用到的一些小东西拿出来聊聊。

▶▶ 5.8.1 ShutdownHook（钩子线程）

当发生了 `System.exit(int)` 时，希望在系统退出前，执行一点任务来做一些资源方面的回收操作，`ShutdownHook` 可以达到这个目的，它利用 `hook` 的思路来实现，有些时候也把它叫作“钩子”。

假如在系统中通过 `Runtime.getRuntime().exec()` 或 `new ProcessBuilder()` 启动了子进程（`Process`），这个子进程一直在运行中，在当前进程退出时，子进程未必会退出，但此时业务上希望将它退出，就可以利用这一点。例如可以这样写：

```
Runtime.getRuntime().addShutdownHook(
    new Thread() {
        public void run() {
            processer.destroy();
        }
    }
);
```

这里需要注意的是，传入参数是通过 `new Thread()` 创建的线程对象，在 Java 进程调用 `exit()` 时，会调用该线程对象的 `start()` 方法将其运行起来，所以不要手工先启动了。另外，这种回调线程就不要设定为死循环程序，否则就无法退出了。

再简单阐述下原理，大家也可以自行分析源码得到答案。

◎ `java.lang` 包下有一个 `Shutdown` 类，提供了一个 `Runnable[] hooks` 数组，数组的长度为 10，也就是最多定义 10 个 `hook`（这与程序写入多少个线程回调没有关系，下面会介绍），提供 `add()` 方法来注册新的 `hook` 对象。

- ◎ 在 `java.lang.ApplicationShutdownHooks` 的 `static` 匿名块中，通过 `add()` 方法注册了一个 `hook`，在 `run()` 方法内部运行内部的静态方法 `runHooks()`，它内部用一个 `IdentityHashMap` 来存放 `hook` 信息，可以通过 `add()` 方法来添加，而程序中定义的回调线程都放在了这里，它自身用 `hook` 的形式存在于 `hooks` 列表当中，如图 5-70 所示。

```
class ApplicationShutdownHooks {
    static {
        Shutdown.add(1 /* shutdown hook invocation order */,
            new Runnable() {
                public void run() {
                    runHooks();
                }
            });
    }

    /* The set of registered hooks */
    private static IdentityHashMap<Thread, Thread> hooks = new IdentityHashMap<Thread, Thread>();
}
```

图 5-70 ApplicationShutdownHooks 的定义部分

- ◎ 当调用 `Runtime.getRuntime().addShutdownHook(Thread)` 方法时，会间接调用 `ApplicationShutdownHooks.add(Thread)` 将线程放到 `IdentityHashMap` 中。
- ◎ 当调用 `System.exit(int)` 方法时，会间接调用 `Shutdown.exit(int)` 方法，再调用 `sequence()->runHooks()` 方法，调用如图 5-71 所示。

```
private static void runHooks() {
    /* We needn't bother acquiring the lock just to read the hooks field,
     * since the hooks can't be modified once shutdown is in progress
     */
    for (Runnable hook : hooks) {
        try {
            if (hook != null) hook.run();
        } catch (Throwable t) {
            if (t instanceof ThreadDeath) {
                ThreadDeath td = (ThreadDeath)t;
                throw td;
            }
        }
    }
}
```

图 5-71 Shutdown.runHooks()方法

- ◎ 循环中的一个 `hook` 对象就是由 `ApplicationShutdownHooks` 的匿名块定义的，因此会调用 `ApplicationShutdownHooks` 类的 `run()` 方法，再调用它的 `runHooks()` 方法，这个 `runHooks()` 方法如图 5-72 所示。

```
static void runHooks() {
    Collection<Thread> threads;
    synchronized(ApplicationShutdownHooks.class) {
        threads = hooks.keySet();
        hooks = null;
    }

    for (Thread hook : threads) {
        hook.start();
    }
    for (Thread hook : threads) {
        try {
            hook.join();
        } catch (InterruptedException x) { }
    }
}
```

图 5-72 ApplicationShutdownHook.runHooks()方法

从这个方法中取出所有的 `Thread`，然后将线程启动起来，最后通过 `join()` 方法等待各个

线程结束的动作。换句话说，在进程关闭前，对多个回调任务的处理方式是每个任务单独有一个线程处理，而不是所有的任务在一个线程中串行处理。

◎ `Runtime.getRuntime().removeShutdownHook` 用于删除一个钩子线程。

5.8.2 Future

Future 是一个接口，它也有许多实现类，用得较多的是 `FutureTask`。Future 接口规约如图 5-73 所示。

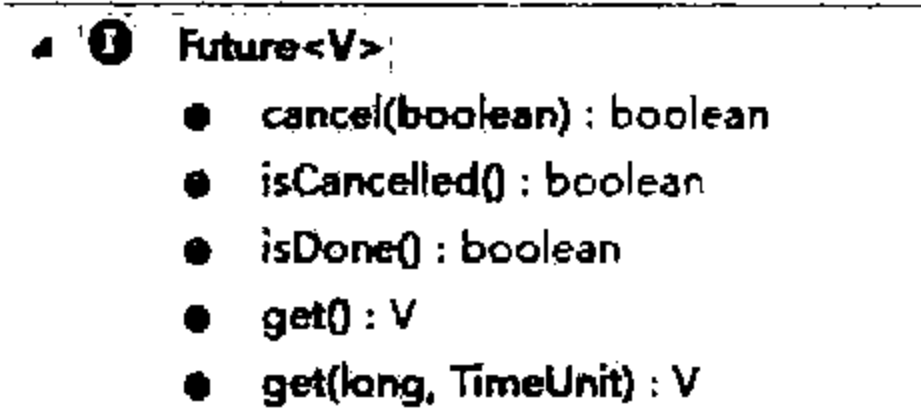


图 5-73 Future 接口规约

以 `FutureTask` 为例，它内部有一个 `sync` 属性，这个属性所在的类是 `AQS` 的子类，即它是基于 `AQS` 来实现的。它有什么功能呢？下面来看看 API 的简单介绍。

- ◎ `get()`用于获取任务结束的返回值，如果是 `Runnable` 则通常返回 `null`，如果是 `Callable` 则返回对应的 `return` 值，若任务尚未结束，则当前线程将会等待直到任务结束才返回。
- ◎ `get(long , TimeUnit)`也是尝试获取任务结束的返回值，只是可以设置一个超时时间，如果任务一直未结束，但是达到超时时间，则会自动放弃等待，如果超时则会抛出 `TimeoutException` 异常。
- ◎ `isDone()`用于判定任务是否结束，该方法不会阻塞，可以自行选择什么时候判定，若该方法返回 `true`，则表示任务已经结束，此时调用 `get()`方法就不会有任何阻塞了。
- ◎ `cancel(boolean)`尝试将任务的状态修改为 `CANCELLED`，此时若有线程调用 `get()`方法，则由于任务尚未结束而等待。该动作若将状态成功修改为 `CANCELLED`，则会释放 `AQS` 的锁，若 `cancel` 操作成功则返回 `true`，失败则返回 `false`。传入的 `boolean` 值如果为 `true`，则判定如果有正在运行的线程 `runner`，则会将其中断，该线程通常不是从 `FutureTask` 中获取数据的线程，而是运行任务的线程。
- ◎ 如果状态已经为 `CANCELLED`，此时调用 `isCanceled()`则会返回 `true`，否则返回 `false`。

本书光盘中的 `src/chapter05/tools/CompletionServiceTest.java` 有一个例子，不过那里的 `Future` 并不会阻塞，因为它是通过一个队列实现的，而只有完成的 `Future` 才会放入到队列中，否则获取不到 `Future`。在第 4 章介绍异步 I/O 时也提到过，可以通过获取 `Future` 来判定 I/O 的返回信息是否已经准备好，但是并没有给出实例。

下面再举一个简单例子，是专门关于 `Future` 的，通过 `Executors` 创建多个任务，返回许多 `Future`，然后每个 `Future` 分别等待结束时间。代码如下：

代码清单 5-18 FutureTask 测试

```

public class FutureTaskTest {
    static class CallImpl implements Callable<String> {
        private String input;
        public CallImpl(String input) {
            this.input = input;
        }
    }
}

```



```

    }
    public String call() {
        try { //延迟一点点
            Random random = new Random();
            Thread.sleep((random.nextInt() + this.input.hashCode())
                        & 10000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        return "输入参数:" + input;
    }
}

public static void main(String []args)
    throws InterruptedException, ExecutionException {
    ExecutorService executorService = Executors.newFixedThreadPool(2);
    @SuppressWarnings("unchecked")
    List<Future<String>> list = Arrays.asList(
        executorService.submit(new CallImpl("t1")),
        executorService.submit(new CallImpl("t2")),
        executorService.submit(new CallImpl("t3"))
    );
    /*List<Future<String>> list
       = executorService.invokeAll(Arrays.asList(
           new CallImpl("t1"),
           new CallImpl("t2"),
           new CallImpl("t3")
       ));*/
    for(Future<String> future : list) {
        String result = future.get(); //如果没返回，则会阻塞
        System.out.println(result + "\t" + System.currentTimeMillis());
    }
    executorService.shutdown();
}
}

```

这里注释掉了一段 `invokeAll()` 的代码，它内部帮我们做了一个 `isDone()` 自旋等待操作，也就是说，它会在内部等待线程结束，这是一种简化的写法。与之对应的是逐个 `submit()` 得到的 `Future` 列表，不过通过 `invokeAll()` 得到的 `Future` 对象调用 `get()` 方法肯定不会阻塞，而逐个 `submit()` 得到的 `Future` 对象可能会阻塞等待任务的结束。

5.8.3 异步并不等价于多线程

本小节介绍异步并不完全等价于多线程，而多线程也并不等价于异步。下面从两个角度来讲解。

对于某些任务可以将其交给一个单独的线程来完成，当前线程做其他的事情，最后以某种方式来通知。例如，将许多任务提交给线程池后，当前线程就无须关注线程池调度的细节了，这对于使用者来讲也是异步的。不过，异步并不一定非要这样来完成，它可以将这个动作交给 `Kernel` 去处理，因为大多数时候内核处理一些底层的等待问题会比应用程序更加直接，即使在没有通过内核做异步的情况下，内核也会与程序进程交互完成许多同步的模型。换句话说，内核始终会做许多事情，在某些场景下希望内核线程做更多的事情（“送佛送到西”），通常在一些上下文无关的 I/O 密集型场景中应用广泛。

反过来，多线程自然不仅仅用于实现异步，它可以实现并行、并发，而异步只是一种设计方式，通常基于某种任务队列来实现，只是这个任务队列的抽象层次可能在程序内部（例如生产者/消费者模型，生产者就是将任务放入队列，消费者就不用关心了，消费者去

处理接下来的事情即可)。

不过，异步并不完美，这样的程序不知道何时返回数据，在代码中不断写回调方法，不断需要保存运行的上下文信息，程序员很多时候会写得头晕脑胀，某些初级程序员甚至会搞不清楚流程，出现问题也相对不好定位，在性能上也未必完胜同步和阻塞，也要看场景（在第 4 章有所探讨），所以异步也有问题，不要认为异步就是最快的。

本章许多知识相对枯燥乏味，但是了解了这些基础后，再去看一些开源框架的代码会相对轻松许多，尤其是看一些开源的调度器代码（例如 Quartz）。同时在高性能的代码设计思路上，你会有许多新的体会。

接下来我们将接触数据库的一些知识，胖哥为何在 Java 书籍中强调数据库呢？请看下一章分解。

第 6 章

好的程序员应当知道 数据库基本原理

本章主要内容

- 程序员为何需要了解数据库原理
- 学会最基本的性能诊断
- 从开发人员角度看数据库原理
- 数风流存储，还看今朝
- 从程序员角度看数据库优化方法

许多同学认为自己是做开发的不用了解数据库，但胖哥认为好的开发人员应当懂数据库的一些常见原理和优化策略，这对于我们的工作有着极大的帮助。换句话说，我们不将这些烦琐的事情交给 DBA 来完成，因为 DBA 有着更为专业的运维目标，而这些事情绝大多数是开发人员需要自己学会的。

数据库经历几代数十年的发展，其内在的各种算法和优化策略可以说已经发挥得淋漓尽致，开发人员不仅仅可以学习如何使用它，同时也可以学习它内在的许多体系架构与设计思想，这些思想都可以借鉴到我们的项目中，让我们的方案做得更好，更有思路。

胖哥不是数据库高手，只是对数据库的基本原理略懂一二，本章或许可以从程序员角度来理解数据库。本章不会特别针对某种数据库去说明操作，而是针对常见的数据库原理来说明如何理解它的性能，以及从程序员角度可以做出的改善。

6.1 开发人员为什么要知道数据库原理

在国内许多开发模式的背景下，在大部分开发人员的意识中认为数据库不是程序员所需要关注的，而胖哥认为面向服务器端开发程序的人员都应该懂一些数据库知识，这样才能把工作做得更好。

相信绝大部分面向服务器端的 Java 开发人员都会接触到数据库，那么开发人员与 DBA 的界限在哪里？有一些做服务器端开发的同学由于在实际应用场景中不会使用到数据库，也许用的是其他的设备与存储，但并不代表就不需要关注数据库，可能在某些意义上需要

更加懂数据库。

从第一代数据库到现在经历了半个世纪以上的发展,它已经成为一种完善的产品,尤其是 RDBMS 一直是主流。数据库之所以如此盛行,是因为它十分重要,几乎在所有的产品中它都有着举足轻重的地位。在商业的信息服务中,通常将数据看作业务系统的灵魂,失去了它就基本失去了一切,其他的技术都是为数据服务的,而数据库是现代数据存储的主流技术之一。

数据库提供了十分强大的功能,抛开功能本身,许多商业数据库也提供越来越接近现实业务的管理模型,它的许多管理思想也同样在工作中可以借鉴。

例如:某些商业数据库会提供各类数据的安全策略、角色划分等。另外,还会按照各种不同类型的对象模型为开发者和 DBA 提供服务,同时提供不同层次问题的解决方案,也提供各种优化策略,并且可以根据自己的需要做许多配置来达到某些应用场景的需求等。这些方面在许多商业数据库中做得非常好,它们本身就是一个不错的软件平台,我们自己做软件也可以参考其长处。

学习数据库不仅仅是学习它简单的 SQL 语法定义,认为它就能进行简单的增、删、改、查操作,即使认为它很简单,也是数据库本身让它变得简单,而不是它的内在很简单。

学习数据库,除了学习语法外,还应当学习 SQL 在数据库系统中会按什么样的步骤去执行,这样才能理解它的真正开销,从而逐渐理解怎么样才能写好 SQL,在什么场景下写好 SQL。

SQL 被称为结构化查询语言,说明它是基于某种数据结构的,它本身是利用二维表的各种变通方式表达出多种数据结构,相对复杂的数据结构本身又更加容易让人理解。在理解结构的基础上,我们开始懂得通过设计让性能的伸缩性更强,懂得一个简单的设计可能比一个费劲的优化要简单许多(就像前面程序中的锁粒度一样)。

不论是数据库自身的架构体系,还是各种各样的优化策略,抑或是自己通过逻辑理解后的设计,都给我们提供了许多思想、建议在里面,而这些内容可能在数据库里面体现得最完善,当你学习的内容不断增加时,很多新奇的东西也大多在这个基础上演变而来,换汤不换药,区别只在于细节。

有人可能会说数据库的知识应该由 DBA 来学习和深入,其实在这个过程中你可能会失去了许多可以学习与提升的机会,错过了学习许多经典的机会。另外,初学者学习数据库时所关注的仅仅是 SQL 和它的基本功能,从一个成熟的软件开发者的角度来看是一类技术、一种产品、一种体系架构、一种设计思想、一种存储解决方案。

作为开发者专注于开发没有问题,不过应当学会去解决一些相关技术的日常问题,而不是一旦遇到一个小问题就交给 DBA 来处理,遇到复杂而较为专业的问题时可以与 DBA 一起探讨,自己同时也可以从中受益。DBA 更专注于数据库的稳定性、高性能、容灾、数据安全、自动化等方面的运维工作,也可能会给出一些分布式层面的解决方案,否则 DBA 会被一些琐碎而繁杂的事情困扰,因为在通常情况下开发人员的人数是 DBA 人数的几倍、十几倍、几十倍甚至上百倍,如果每个人都向 DBA 问一个简单的问题,那么 DBA 每天的工作可想而知,当数据库和业务规模成指数级快速增长时,DBA 的人数随之成指数增长,因此需要开发者更加了解数据库并对数据库拥有更多的权限才有可能解决。

当然，开发者拥有了更高的水平后，还需要一系列自动化工具来帮助开发者完成许多事情，并为开发者提供常见的数据库状态监控和优化建议，也需要为 DBA 提供一系列运维工具以减少工作量，同时降低风险。

在工具层面，会根据公司的场景而定，在服务器少的情况下不需要，在访问量低且数据量不大的情况下并不需要，因为开发软件工具也同样需要成本。另外，我们需要知道的是，工具永远做不到尽头，人始终要提升自己才能将事情做得更好，就像软件一样永远无法替代人，只能提升我们的工作效率。

在国外的许多大型站点里，几乎没有 DBA 或只有为数不多的几个 DBA，而且这些 DBA 还分白天和晚上分开值班，其中一个十分重要的因素是开发人员就像“特种兵”一样能搞定自己绝大部分的问题，而且知道如何去优化自己的系统与代码，而 DBA 并不是一个辅助于系统开发的人，而是一个从 DB 角度提供高可用方案的合作者，DBA 也有他们更为专注的领域。

当然，国内由于许多条件因素，很多时候不能像国外的“特种兵”那样牛，但是我们应该从某些问题上自己提高成长的意识，这样与别人的差距就不至于会越来越大了。

对于本章来讲，胖哥希望大家理解数据库本身，客观并认真地对待它，而不是随意地肆虐它，然后带着无辜的表情问 DBA 这是怎么回事？为什么会出现这样的问题？为什么不可以保证高可用性？

现在越来越多的系统接入云平台、云数据库，这样的平台可能会有一些专门的团队来维护数据库的稳定性，保证数据的可靠性，以相对于商业解决方案廉价许多的成本提供较为可靠的服务。不过云数据库一旦购买后，用户将拥有巨大的控制权，通常需要自己更加懂数据库，更加懂得如何去优化 SQL，减少人为失误等，因为这个时候很少有一个专门的 DBA 来检查我们的 SQL 语句是否足够好。开发人员与 DBA 之间的分界线将会越来越明显，同时开发人员自身能力在这里也会有更多的体现。当然云数据库的发展还有很长的路要走，包括对分库分表的隔离，完善的监控体系不仅仅面向 DBA，更多的问题应当让开发者清楚并需要开发者通过报告来决定如何解决。

总而言之，在现代的技术发展中，对我们的要求不仅仅局限于自己所在圈子的“一亩三分地”，而是要放眼自己的周围所能接触到的东西，而数据库的一些问题既是在业务上我们需要掌握的，也是可以给我们带来财富的。

6.2 从开发人员角度看数据库原理

小伙伴们看了上一节内容是否会想一想：我们是否需要像 DBA 那样擅长 DB 的知识呢？当然不是，相互之间是一种合作关系，但不能在相互之间隔着一堵墙甚至于“隔行”，相互之间是需要交流与沟通的，在交流与沟通过程中相互学习并在许多问题上达成共识。

开发人员到底应该学习哪些方面的数据库知识呢？胖哥认为数据库的基本原理、索引原理、常见的优化手段等是开发人员需要掌握的，这些内容对于开发者来讲应该并不算复杂，而且也花不了太多时间就可以学清楚，在解决大多数日常问题的基础上，自己也得到了提升。在接下来的内容中，胖哥将简单阐述一下自己认为一个开发人员所需要了解的数据库知识。

在本节就一些通用数据库的特征进行说明，这些内容大多是思想和方法上的，在这个基础上，大家如果想要学习一种具体的数据库，则会更加轻松一些，至少看待数据库的角度应该会有所改变。

►► 6.2.1 实例与存储

数据库不仅仅拥有存储 (Store)，还应当有为外界提供数据服务的程序，这个程序对存储的操作细节进行封装，对外提供简单的服务，通常把它叫作实例 (Instance)，有了存储与实例的组合，才称为真正的数据库。

部分数据库的定义是“实例”与“Database”的组合，这个时候 Database 指的就是存储。大家不用过分纠结这些概念，否则就会将自己的思维局限在某一种数据库上，无法跳出这个圈子，有了自己的理解就无须过分关注到底用的是什麼名词。

存储本身可以简单理解为存储设备上的一套文件系统，存储设备可能是一块硬盘、一组磁盘组成的磁盘阵列 (RAID)、一个专门的存储服务设备，通常情况下程序通过 OS 提供的 API 对存储设备进行读/写，当然某些数据库系统也可以跳过 OS 直接操作裸设备。

反过来讲，如果磁盘上存储的数据仅仅是一组文件，而没有像数据库那样提供各种管理功能，就不能算是数据库（因为数据库应当有对数据进行操作的功能）。要满足一个数据库的要求，需要有整套程序对文件的分隔、数据的格式进行统一管理，并在这个基础上提供通用的数据操作 API 才能叫数据库，这就是建立在存储基础上的程序实例，简称实例。

说明实例与存储来组成数据库，是首先要让大家形象了解数据库是什么样一种形象，胖哥认为任何一种数据库都脱离不了这个基本道理。

了解到这里，可以将思维再打开一点——实例与存储既然是分开的，那么它们并不需要一一对应起来，但是如果不是一对一的关系，就会有許多实现上的区别，区别是各个数据库厂商对数据库的理解会有所不同，数据库的体系架构自然会不同。通常来讲，同一个数据库的多套存储，是用于冗余备份容灾的。一套存储对应到多套实例是为了提升整体对外的负载能力，以及达到高可用性。

数据库实例启动后代表数据库已经开始运行，并对外提供服务，数据库使用者需要了解账户与数据库之间的关系，下面简单介绍一下。

- ◎ 部分数据库可能会强化数据库的概念，数据库是一个对外服务的整体，一个数据库的建立包含了存储、实例的建立，一台主机也可以安装多个库。如果库与存储是一对一的关系，那么库名与实例名通常就是一致的，如果是集群则通常会有一个服务名的概念存在。这类数据库除了超级账户外，每个账户登录后都会有自己的默认空间和权限范围，如果需要访问其他账户的信息，则必须得到其他账户或超级账户授权才可以访问到。
- ◎ 有的数据库则会弱化数据库的概念，数据库安装后仅仅是一个空的数据库实例，也会有超级账户，当在实例上创建数据库时，通常会为创建的数据库分配一个目录，此时的数据库概念很弱，它仅仅是存在于实体上的逻辑单位。在这样的数据库上，登录用户不需要与具体的某个库绑定起来，即没有所谓的私有区域，只有授权访问的数据库列表。账户登录时可以指定库名登录，那么登录后就直接进入了某个具体

的逻辑库下进行操作，也可以不指定库名，可以在这些拥有权限的库之间任意切换。

6.2.2 数据库基本原理

本小节胖哥就自己的理解来简单谈谈数据库的基本原理，从开发者的角度看看数据库需要做什么样的事情。前面谈到了存储与实例，实例是程序，用于将物理上的关系转换成为逻辑上的关系，将复杂的问题简单化。下面我们首先谈谈物理与逻辑上的内容。

(1) 块与页

谈到数据库的物理存储，就从最小的数据操作开始。如果我们自己写程序对文件系统进行读/写操作，则可能为了让程序简单，以行为单位进行读取，或者通过随机读/写方式定位后读取一段数据。数据库的实例也是程序，也有类似的处理方式，它要处理数据不可能在磁盘上做，肯定是在程序中做，在程序中做自然要加载到程序空间所在的内存中（这里抛开虚拟内存的情况）。

数据库通常不会以行为单位来加载数据，因为行的粒度不容易控制，也不利于调度管理，例如，很多时候每行数据都是以百字节为基本单位的，甚至更低，如果发生一次 I/O 只是读取一点点数据，那么 I/O 就太频繁了。很多时候我们希望程序读取这一行数据时再读取连续的许多行数据到内存中（由于当前行接下来的几行数据被访问到的概率通常很高，那么这样就可以提升内存命中率），加载到内存后由一个大进程来管理这些板块。

在数据库中通常对数据做物理大小的划分，例如以 4KB、8KB、16KB 等大小为基本单位来存储（而不是以多少行为单位），单位的名称有的数据库叫作块（Block），有的数据库称为页（Page），为了统一，在本书的后文中我们将统称为“块”。

这样一来每个块通常会存储许多行数据，数据库为了记住每个块，通常会给每个块分配一个标识符（这个标识符不一定是唯一的，不过它可能与某些更高抽象层次的标识符组成一个唯一标识符，例如对象编号、表空间编号）。由于块内部存放多条数据，为了标识每条数据，块内部的每条数据通常都有自己的行号。

在不同的应用中，可能会设置不同大小的块，希望它存储相对较多的数据，又便于数据库中细粒度的管理。

物理上的磁盘并不会单独生成一个块，也没有“块”这个概念，“块”本身也是将文件系统中的某个数据范围抽象出来的逻辑概念，由于数据库对存储的输入/输出控制有自己规约好的一套体系，因此数据库实例程序可以识别出“块”。

什么是数据库对存储的输入/输出的规约呢？数据库有数据字典来记录数据库各种各样的信息，它们会记录下每个块在文件系统中的偏移量，相当于记住了它的位置，找到了偏移量就等价于找到了块的起始位置，再从这个起始位置开始读取等同于块大小的区域就成功地读取了这个块所存放的内容。这种文件操作方式类似于 Java 用随机读/写来访问文件。

(2) 多个连续块

一个表要存储许多数据，随着数据的增加肯定不止一个块来存放，也就是需要很多这样的块来存放数据，这些块存放的位置或许会在一定程度上影响性能。

通常存储设备的效率是最低的，数据库性能的巨大障碍之一就是磁盘存储，而磁盘存储效率低主要是低在磁盘的 IOPS 上面，即每秒可以发生的 I/O 操作次数太低（不考虑固态

硬盘), 其实每次 I/O 操作从磁盘上读取连续 1KB 或连续 1MB 左右的数据时间几乎是差不多的。换句话说, 慢的主要原因是一块块只有一个磁头, 对一块硬盘的 I/O 请求都会被串行化, 每个 I/O 请求都需要通过摆头摆动进行磁道定位, 再在该磁道内通过磁盘旋转等待数据的开始位置, 当数据开始位置找到后, 便开始真正读取数据, 读取速度是非常快的。

根据上述理论, 我们希望数据库每次读取的不仅仅是 1KB、2KB、4KB 的数据, 很多时候希望能够读取更多的数据, 此时将多个块的数据连续放在一起组成一组数据, 这样在理想情况下可以成倍提升存储设备的性能。

我们将多个块一起读的操作叫作顺序读, 顺序读是相对随机读的, 也就是每次读取的量会更大一些, 但是在物理上并不存在每次读取 100MB 的数据, 即使应用程序是这样的, 物理上也会拆分为多个小的 I/O 来调度。

将多个块组成一个分组后, 这个分组在一些数据库上叫作 extends, 它被认为在文件系统这个级别是连续存储的, 理论上在磁盘上是相对连续的一块空间, 但不绝对保证是这样的, 就像一个大文件存放在磁盘上时, 也会在磁盘碎片的空隙中寻找位置一样。换句话说, 这种连续也是逻辑的, 但是只要做好相应的碎片管理, 在很多时候就认为它确实是连续的。

我们希望读/写操作都是顺序的, 不过在许多实际场景中往往不是这样的, 由于内存空间的限制, 数据库通常会以类似于 LRU 的算法来管理块, 让某些访问多的块留在内存, 或让访问少的块从内存中释放掉。如果一些连续的数据块加载后, 这些连续块中的部分块没有被访问, 那么这部分块就可能会在 LRU 的调度下写回磁盘, 这些连续加载的顺序块之间存在空隙, 而当再次加载这样的块时, 数据库通常不会再一次连续加载这些块, 而是加载空隙中所没有加载的块, 当连续块之中出现多个空隙时, 通常需要发生多次 I/O 操作来读取这些空隙块, 这样顺序读操作就变成了随机读, 最坏的情况就是在一大堆连续块中空隙块与内存中 Cache 的块交替出现。

有些时候数据块的分布不容易被确定, 但是一些索引的顺序是可以被确定的, 因为索引本身是有序的, 尤其是基于自动增长列这类索引, 离这类数据最近的数据通常是访问最热的, 那么被连续加载的概率是很高的, 如果将它散列, 此时就可能会稀疏分布在许多块上面。

另外, 连续读的粒度并非仅仅受到磁盘的控制, 通常数据库也会有参数来控制一次连续读的块个数, 例如 Oracle 中就是使用参数 `db_file_multiblock_read_count` 来控制的, 其默认值是 16。

在实际的数据库中, 还会有更大的空间概念, 例如某些数据库会用多个 extends 组成一个 Segments, 一个表对象可以划分到多个 Segments 中, 这样来实现分区表独立管理。

(3) 修改表

试想一下, 如果一个表中的每个数据块中都存储了数据, 此时要向一个表中增加一个字段, 而且马上要向这个字段当中写值, 那么会发生什么事呢?

某些数据库会事先在写入每个块时给它们预留少部分空间, 也就是每个块都不会写满, 以便于添加字段时可以用, 但是这并不能确保永远够用 (例如增加一个超大的字段)。如果空间不够用, 就会将一行数据的某个部分放在另一个块当中, 这两个块可能此时相隔很远, 在访问数据库时, 以前只需要一次 I/O 就可以得到的数据, 现在需要多次 I/O 才可以。

遇到这类问题时，虽然可以通过压缩、整理等方式来处理，但这个过程由于需要移动相关的数据，因此会消耗数据库的性能。所以作为程序设计者，千万不要认为对线上数据库加字段是一件很随意的事情。

某些开源数据库没有数据字典的概念，大多数描述信息都是在以表为单位的文件头部完成的，当表结构修改时，通常会创建一个新表，并将原表中所有的数据全部拷贝到新表中，在拷贝过程中为了保证数据的一致性通常会发生“锁表”的情况。对于小表，这样的操作可能会十分畅快，但对于大表这样的操作将会十分恐怖，外界可能长时间无法访问这个表。因此对于大数据的存储，通常会采用所谓的“分库分表”策略。

（4）删除表

删除表操作对于使用者来讲很简单，但是对于数据库却可能需要做一番复杂的处理，因为表中可能会有大量的数据需要清理，一些商业数据库还会考虑表被误删除需要恢复的问题。

删除表时清理数据不可能像 `delete` 操作那样逐条去删除，通常是基于数据库的体系结构去清理的。例如基于页、块去清理，如果数据库表与文件对应，那么清理数据时可直接删除文件。

不过，我们似乎忽略了一个问题，相应的表在内存中可能还有数据，这些信息也应当被清理。某些数据库在内存中基于表名称 Hash 分布数据，这样就可以通过表名称快速定位到该表在内存中的数据，那么清理动作应该会很快。不过某些开源数据库的存储方式并非这样，它在内存中可能是基于一种线性结构来存储的，此时就可能发生问题了。即使在一个数据库中删除一个“空表”，也可能导致系统延迟很大。为何？请看下面的解释。

所有的数据库都存在内存命中率的问题，如果内存命中自然就不用访问效率低上百倍的磁盘了，因此对于数据库来讲，我们通常会给它配置较大的内存来达到很高的命中率，例如 24GB、48GB 甚至更大。这意味着如果要找到一个表在内存中 Cache 的数据有可能需要扫描整个数据区域，在某些时候还不止扫描一次，例如 MySQL 的 InnoDB Buffer 若设置非常大，那么在连续删除表时就可能导致数据库系统性能下降，即使删除整个空表也会导致系统的延迟飙升。

删除某些数据库中的表，为了防止误删除，会在默认的处理删除动作时，只是修改一个名字并将修改后的表名与原来的表名做一个简单的映射，同时也改变其元数据的状态，在默认情况下我们看不见它（就像回收站一样）。但是它依然占用磁盘空间，数据一点不少存在。这样的删除操作自然是瞬间完成的，如果发现删错表了，通过一个简单的语句就可以瞬间恢复。如果想要直接物理删除表或清理回收站，就需要数据库提供特定的操作语法来完成。

（5）一个 SQL 的运行可能会经历的步骤

当一个 SQL 请求访问到数据库时，它通常会经历 SQL 解析的过程，这个解析过程就是将 SQL 语句解析为数据库操作。换句话说，SQL 是人类约定语法后看得懂的信息，因为人类学会了如何解析 SQL 语句，但是传达到机器后，机器也需要认识 SQL 语句，它需要解析 SQL 语句后才知道到底需要做什么，比如 SQL 语句做读/写操作、对哪些表做操作、表与表之间的关系是什么、条件是什么等。

一些数据库为了提供高并发的访问，让数据库运行得更快，会将这个解析过程缓存起来，因为它认为这样的系统通常 SQL 语句复用的概率很高，当同样的 SQL 语句发送过来

时,就会取出已经解析好的结果直接运行,通常将这个过程叫作“软解析”,或者说它是假的解析。软解析在并发系统中十分有效,在这类数据库中其性能可以成指数级提升。反过来讲,在这类数据库中最不希望看到的就是硬 SQL (即参数是通过值拼接出来的,而不是通过预编译出来的),因为所有的 SQL 对于服务器端都被认为是不一样的,SQL 复用的概率很低,由于存储这些 SQL 的空间也是有限的,它同样会利用类似于 LRU 的算法来管理内存区域,在高并发的系统中,可能会由于存在有大量拼接参数的 SQL 导致许多一次性的垃圾 SQL 太多,这样有可能会将一些原本应该留在内存的 SQL 替换出内存。

补充: 软解析机制在 Oracle 数据库上体现得淋漓尽致,但并不代表这种机制在所有的数据库上都能优化整体性能。例如在分布式数据库上,硬解析的成本将被打散到多台服务器上,而 SQL 语句的解析若不需要做各种优化分析和 Cache,其开销通常在微秒级别,这个时间对于整体系统来讲几乎可以忽略。另外,由于数据库表不断拆分,表名称会增多,要达到较高的命中率,就需要更大的内存,这种思路是与分布式的解决思路背道而驰的,并且用如此大的内存去解决一个微秒级别的问题显然不值得。

部分数据库提供了对于某些内存区域的优先级管理机制,可以指定或智能计算出某些数据是加载后几乎永远不会被卸载的;某些数据区域是加载使用后可能会被立即卸载或内存不够时最先被卸载的;某些数据区域则是采用传统的 LRU 管理方法。

从写程序的角度来讲,SQL 解析的开销其实并不是那么恐怖,因为它是纯 CPU 操作,在高并发下会在某些数据库上体现出性能优势。我们做一个假设,如果某软解析程序是自己写的,想继续优化,也许软解析过程可能不仅仅会保存 SQL 语句的解析结果,也可能会保存关联的结构信息(在表结构发生改变以前都是有效的),还可能会保存一些执行计划(在表的数据量没有发生太大变化时执行计划也不会有太大的变化),甚至可能会缓存一些 SQL 返回的平均行数、平均执行时间,在高并发的系统中可能用来做一些智能调度的优化策略。

解析完成后便知道 SQL 到底要做什么了,以一个查询语句为例,在经过解析后便知道了查询表,以及多个表之间的关联条件,为了知道这样的表是否存在,以及所查询的字段是否存在,就必须要先查询数据字典(即使 SQL 语句不是 `select *`,也肯定会查询数据字典来确保字段是存在的)。

数据字典就是用于保存数据库的一些描述信息,通常也是以表的方式存储的。由于一个数据库本身的对象通常不会太多,而且它的更新速度肯定不会像更新数据那么频繁,所以通常将它加载到内存中提供访问。

数据库访问自身的元数据时通常是可以透过抽象层次相对较低的 API 来完成的,但也不排除某些数据库也是用 SQL 完成的。总地来说,上文中所提到的数据字典加载并不是说还要去查询一次数据库后台。

确保表和属性都存在后,数据库就要开始计划如何读取表中的数据并对数据进行处理,单表操作时会考虑是否走索引,多个表操作时会用 JOIN 的方式或 UNION 的方式,排序有排序方法,分组有分组策略,它们始终沿着一条路径在执行数据库,数据库在执行语句前会计算出这条执行路径,胖哥认为这种数据库根据某些参数计算出的执行顺序,就是大家经常提到的“执行计划”。

所以,在很多时候定位一个 SQL 的性能如何,专业的做法是先看执行计划,当执行计划没有问题时可以不用去增加索引,当执行计划出现问题时,就要看执行计划中的那些路

径并不是我们期望的方式（例如应该走索引的却没有走索引），那么此时就要考虑合适的办法来解决，例如通过修改 SQL 逻辑、调整索引或者某些数据库指定的方式来实现。关于执行计划，下面举个简单例子来说明它的重要性。

假如有两个表 table1 和 table2，其中 table2 表较大，有几千万上亿的数据量，table1 表较小，现在有这样的 SQL 语句：

```
select ...  
from table1 a ,table2 b  
where a.rid = b.id and a.type = 1;
```

SQL 语句很简单，代表 table1 表通过 rid 与 table2 表的 id 关联，table1 表通过 type 过滤。

如果我们在具体的数据下得知 table1 表通过 type 过滤后就只有 20~50 条数据，且在相应的字段上有索引，理论上会走索引。虽然 table2 表有几千万上亿的数据，但由于将 table1 表作为驱动表，即使是逐条到 table2 表中查找数据，也会因为走索引不会太慢，因为外层循环最多几十次，对于计算机来讲是小意思，而对于外层循环得到的每一个 rid 到内层的 table2 上查找数据时都会走索引。

而如果将 table2 表作为驱动表，如果两个表反过来 JOIN 时，事情就完全不一样了，外层循环将是几千万上亿次，即使内层全部走索引也会非常慢。

部分数据库也提供一种“HASH JOIN”方式，遇到这样的情况其实也是很恐怖的，在后文中会介绍到这种方式。

可见，要得到一个良好的执行计划是复杂的，因为它需要参考 SQL 的 JOIN 方式、原有表的大小、索引的建立方法来综合计算得到执行计划的结论。

对于小表、访问量低的表，无所谓执行计划的问题；而对于大表、访问量高的表，执行计划就显得尤为重要，所以很多时候为了让数据库有更好的执行计划，会提供一些统计值或告诉数据库应当如何去执行，如果没有这些强大的智能计算能力，就只有让数据库变傻一点，把它当成单纯的存储引擎，自己写代码来完成一些复杂的事情。

当一个 SQL 执行计划制定完成后，就开始按照这个计划进行读/写数据、合并数据、排序数据等操作，这个过程是需要数据库分配内存或磁盘资源来处理的，这种资源分配在不同的数据库上实现机制有所不同，例如在某些数据库上，在用户连接创建时就会分配一个几 MB 的用户空间，用于满足用户绝大部分的数据请求都可以在内存中实现数据的合并与排序操作，同时将不同的用户的资源进行隔离，但如果在计算过程中涉及大量用户数据的操作，这些数据库则会尝试选择一块额外的临时空间来做一些排序、JOIN 的操作，同时需要注意的是这类数据库创建连接的开销会相对大一些。某些数据库在分配连接时仅仅是分配一个套接字，可能会有一个相应的线程来对应，但是不会单独为连接预分配空间，在实际运行过程中，根据连接需要使用的资源再去分配空间。

在数据库实际读取数据时，其内部可能还会经历多个步骤，在某些数据库上可能会使用单线程来提供数据服务，因此它就可能会成为资源瓶颈，可能会像 CPU 那样基于时间片为外部提供数据服务或基于“行”为基本单位对外服务，也可能服务完一个请求的所有数据后再为下一个请求服务。但这种服务毕竟不是 CPU 服务，它没有 CPU 那么快速，它所操作的可能是计算机中最慢的 I/O 部件（当然有可能是直接读取内存的 Cache）。当遇上大量的请求时，如果以太细的粒度为外部许多请求进行轮询服务，那么许多大的任务就可能在并发系统中很难得到返回值，在并发量小时可能体现不出来。如果是以 FIFO 方式来逐

个处理请求的，那么在并发系统中，可能会因为一个大数据的提取请求就将所有的小请求全部堵在门外。似乎两种极端都各有优势和劣势，因此针对这样的数据库的使用，具有很强的场景化要求。

基于“行”为基本单位的服务方式可以改善的点就是粒度的问题，也就是在上面两种极端中都向中间走一点，也许可以让其返回的行并不仅仅是一行，而是多行，这种粒度可以人为控制甚至智能计算出来（例如数据库通过自动搜集 SQL 平均执行时间和 SQL 平均行数，用以指导数据服务行的粒度）。

程序员应当知道的是，在这样的数据库中，之所以不在业务运营的高峰期做大操作，尤其是在主库上做这样的操作，是因为在这种模式下无论怎么操作都会影响正常的运行。

但是在某些数据库中是允许的，因为它有一套体系架构来隔离资源，更为重要的是，在这样的体系架构中可以支撑更大的存储设备，这些存储设备以及它本身的架构提供了这样的基础，但它同时带来的是这个数据库软件的复杂性以及伸缩能力的问题。

(6) 加锁

数据库中有各种各样的锁机制，在规范的数据库定义中也就有各种锁的定义。

当访问某些数据库的表时，不论是读操作还是写操作，都会在数据字典上加锁，以保证在访问此表时它不会被删掉，表结构也不会被修改，我们可以认为这就是共享锁，这把锁并不是放在数据上的，所有的数据操作依然可以并发地进行。

当发生数据的写操作、for update 操作时，就相当于锁定了某些数据，这种锁通常是基于行的，也是排它的，即不允许在这个区域同时发生两个写操作，但是只读操作是允许的。与第 5 章的锁机制对比，这更加像是读/写锁，而且是一种写操作不会影响正常的只读请求操作的读/写锁。

这样就会产生疑问，当发起读操作时，数据库正在进行写操作，那么读操作获取到的数据就可能不是最新的，这对于数据库来讲是正确的结论，因为数据库通常认为你要读取的数据是发起 SQL 查找时的时间点的数据，如果在读取过程中数据发生修改，即使还没有读取到修改数据的行，也不应该将修改后的数据返回给客户端，这在数据库中会存在一个时间戳或版本号的概念来保证对应的数据。

对于某些数据库锁住的行可能并不仅仅是 where 条件过滤掉的行，如果它没有索引，则可能会锁住所有的行，当 where 条件中的部分字段有索引，部分字段没有索引时，锁住的行有些时候仅仅是有索引的条件过滤出来的行，而不是最终过滤出来的行。例如 update table set ... where a=? and b=?，其中条件 a 走了索引，而条件 b 没走索引，此时锁住的行可能会包含所有条件 a 过滤出来的数据，而不会在用条件 b 过滤掉以后再加锁。

部分数据库为了加快加锁的速度，会以“块”为单位加锁，在块的头部会包含这个块内部的行信息，对于每一行来讲会有单独的 1 个 bit 位来标志是否已经被加锁，多行数据就用多个 bit 位来表示，它只要将要加锁的行对应的 bit 位变成 1，与该块中本身的锁 bit 位序列按位求“或”即可将这个块中相应的行一次加锁，这样一来，一个块中多行数据就可以一次性加锁（当然会先按位求“与”得到一个结果，如果该结果大于 0 就表示有冲突的行）。

不过，数据库的锁很多时候也无法完全保证所有的一致性，加锁即使再快也有一个过程，尤其是对一个大表做没有 where 条件的 update 时需要一个相对较长的过程，它通常会

按照块的顺序进行加锁，这样的 SQL 加锁的过程可能会出现一些问题，例如某个表有 10 万个块，一个 SQL 全表加锁可能需要花费 100ms，在这个过程中另一个请求需要将 10 万个块的最后几个块的数据加锁，通常会成功获得锁，这导致的结果是先执行的 SQL 需要等待后执行的 SQL，这从严格意义上来讲是不符合要求的，但是如果强制要求先后顺序，则可能会导致数据库的处理十分复杂甚至串行化。为何说这个先后顺序是很重要的呢？例如银行正在结算利息时，有人正在存钱或取钱，如果计算方法存在疏忽就可能会导致数据错乱，所以在很多时候对大表的批量修改需要有其他的方式来保证一致性。

对于这样的问题，可以在某些数据库中做个实验：找一个十分大的表的最后一个块（在 Oracle 中有一个 rowid 的概念，可以找到这个块所对应的 rowid），一个会话的 SQL 修改这个表中某个属性的所有行，另一个会话的 SQL 就修改最后一个块的某一条数据，第一个会话先执行，但是它需要一点时间，第二个会话在第一个会话执行后马上开始执行，有可能会发生的是第二个会话先占用了锁，第一个会话还要反过来等第二个会话的锁。

按照常例应该先到先得，但是如果要严格限制这种先到先得，就可能会导致大量的访问阻塞，导致每个 SQL 都需要让前一个 SQL 的锁执行完以后再去执行，那么当遇上一个大的 SQL 时就可能会将 SQL 全部阻塞住一段时间。也许将这个耦合放开一点，会让问题变得更加简单，这种先后的瞬间问题，可以由业务层面来保证。如何来保证呢？

以金额修改为例，首先不能在 SQL 语句中指定结果值，如果指定结果值，就可能会出现覆盖问题，后提交的 SQL 因为先执行，而被先提交的 SQL 后执行所覆盖，这是不合理的（有可能取款后，发现余额比以前还多）。我们可以采用的方式是相对金额，可以用类似于 CAS 的方式根据修改数据的版本号来提交要修改的数据行，即在修改时带上读取时的版本号，若版本号一致则修改相对金额（余额判定也可以加入其中），若版本号发生了变化，则不会修改任何行，此时可以用重试或打标记的方式来完成接下来的工作。

数据库的行锁也是技术层面提供的东西，在使用时也同样有粒度的问题，也许掌握好粒度，会让数据库系统运行得更加良好（这与并发编程的道理是一致的）。例如，要利用数据库本身来做一个分布式数据库的全局序列，可能大家会建立一个表来存放序列的名称以及当前值，然后多台服务器都来 update 这个序列表以达到目的，业务上可能会满足，但是这个序列表将会成为瓶颈，因为每个表的写入都需要在这里做 update 操作，自然系统就被它给拖慢了，如果说这样也能达到性能需求，胖哥基本也可以认为不需要做分布式数据库了。在这个全局序列的修改过程中，胖哥还见过有的小伙伴在程序中写一个 synchronized 来访问此表，以保证并发时只有一个请求，其实这样的方式在集群中没有什么作用，而且带来的是让每个节点上的程序逐步地都阻塞在这段代码上。或许可以稍微改改方式，怎么改呢？

首先在修改的 SQL 语句上不是每次叠加 1，而是叠加 100、200 这样的数字，叠加成功的节点便拥有了 100、200 这个范围内的数据所有权，在自己的程序中通过 AtomicLong 这类原子变量循环分配即可。

如果某个节点出现宕机或重启，全局序列会出现断层，断层对于数据库来讲时有发生，是十分正常的现象，如果是单台机器重启或宕机也最多出现数字 100 的断层，而且这也是最坏的情况，对应 long 来讲简直不值一提。

通过这样的方式这个中心点的修改压力会以百倍的速度降低，如果觉得某个行锁压力依然很大，则可以将一个序列递增值加大，也可以将一个序列拆分为多行，多行之间数据

交叉使用（例如两行序列，一行从 0 开始，一行从 100 开始，每行每次递增 200，但是只使用 100 就可以达到隔离的目的），而服务器可能会选择其中一行来操作，如何选择就用其他的策略来控制，这样就将一行上锁分解到多行，这类似于并发编程中的锁分段机制。

为了保证每个节点上对同一个序列只有一份数据，所以在一台机器上做必要的锁是需要的，但是没必要做一个全局的 `synchronized` 操作，至少可以以序列本身为分隔点将它们隔离开，这样就在应用程序端将锁打散了。

程序有乐观锁，乐观锁也同样可以结合到数据库中使用，如果多个服务器节点上的程序都去修改数据库中的同一行数据，就不知道会修改成什么数据，如果要达到一致性，就需要锁住这一行，读取最新的数据来修改（因为这个最新的数据将作为程序的起始值，所以程序要查出来），这样的锁方式就会造成类似于 `synchronized` 的串行处理，应用程序端没有任何选择的余地。

或许可以利用一些 CAS 的思想，在表中增加一个类似版本号的字段，更新前先查询出原先的版本号，修改时的 `where` 条件中不仅仅携带唯一标识符，也携带原来的版本号作为 `where` 条件的补充（当然也可以用旧值来作为 `where` 的辅助条件）。如果有多个请求同时并发发生写操作，那么只会有一个写操作成功，成功者事务处理完毕后，没有成功的请求此时得到的影响行数是小于或等于 0 的一个值（因此它们不会锁住任何行，且相互之间不会互斥），当客户端失败后，具体要再做任何操作由自己来决定，这样来达到一种乐观的目的。

不过，这种实现机制，在某些类型的数据库上需要对索引特殊处理才能达到效果，例如 MySQL 的 InnoDB 存储引擎，如果 `update` 操作的 `where` 条件没有索引就会锁住所有的行。如果在上面的例子中只有唯一标识符有索引，但是版本号没有和它在同一个索引上，那么就会导致并发时达不到乐观锁的目的，因为多个线程也会尝试去锁住这一行记录，其原因是它的锁粒度是跟着索引走的，而不是跟着记录的结果走的。

如果使用 MyISAM 存储引擎问题就会更大，它的锁都是表级别的，即使是一个查询操作的游标未关闭，修改操作也是无法进行的。试想一下：如果一个系统存在频繁的数据导出，是否适合使用这种存储引擎？写操作很多时是否适合使用这样的存储引擎？当然 MyISAM 存储引擎曾经是 MySQL 的标配，自然有它的好处，那就是在大量读操作的情况下非常快。为何？因为它足够简单，它省掉了在一个复杂数据库中需要做的许多事情，包括版本号、事务、行锁等，所以它在某些场景中足够快，而那樣的场景正是你所需要的。正因为它的简单，也才会出现上述场景的各种问题，这个话题依然说明了技术是双刃剑的道理。

回到乐观锁本身的概念上，所谓乐观就是我们乐观地认为在粒度拆分得当的情况下，冲突的概率并不高，且希望在发生冲突时可以被程序所控制，这样就不必再写 `synchronized` 在代码块中全部阻塞了，胖哥认为这样的乐观概念不论在数据库还是集群等方面都是实用的。

为了说明乐观的概念在数据库中的使用，我们举一个极端的例子：当许多人都去“抢”同一个商品时，在数据库层面所有的 SQL 都是尝试根据商品 ID 修改数据，若使用常规的手段去修改数据库的值，当所有请求到达后，会有一个人成功获取该商品，商品的状态信息已经被修改，但由于其他所有请求的商品 ID 也是同一行数据，所以其他所有的请求还需要逐个进行加锁处理判定。当改为乐观锁方式后，当第一个成功购买的请求释放锁以后，后面阻塞的请求将被唤醒，但它们在数据库中无法匹配出任何行，所以后面的请求就不会再去加锁了，且所有的请求不会去串行处理，自然的在数据库端后续的请求会被很快地释

放掉。这种乐观锁的思路适合在互联网的高并发系统中使用。

不过，这种优化策略仅仅是利用数据库本身这个层面，数据库本身后台的许多复杂逻辑操作是不可避免的，通常数据库本身所能够支撑的并发度是有限的，要在这个方面尝试更大的并发度，就要尝试去打开其他的某些耦合，也就是会放弃一些一致性的东西来解决问题。

(7) 提交

在数据库中发起 `commit` 操作时，数据库通常保证数据肯定会存在，但是未必保证在 `commit` 时会把“脏”数据写入到存储设备上。大家可能会比较奇怪数据没有被持久化到存储中如何能保证它肯定存在，如果数据库发生宕机或断电的话，数据就会丢掉才对。

数据库通常在做 `commit` 操作时，会将日志写入到磁盘，这种日志写操作，从程序的角度来讲可以被认为是顺序的，许多高并发的数据写操作需要找到对应的块、行才能写入，而且每次写入一点点数据，这种写入通常是随机写入的。通过这样的方式，在避开数据随机写的情况下，效率通常会很高。日志中的内容足以保证内存中的数据丢失后，还能根据日志将数据恢复回来。

另外，数据库通常会在内存中分配一个日志缓冲区，这个缓冲区也是有大小限制的，即使不做 `commit`，在一段时间后或日志缓冲区达到一定的容量时也会发生存盘的操作。因此，在大多数情况下，发生 `commit` 操作时，并不会因为事务本身的大小导致提交操作的时间长短，如果真的发生这样的情况，则需要考虑是否是其他的因素导致的。

通过日志恢复的基本条件是有一个版本号的概念，它与时间点基本是对应的，如果数据库启动时发现日志的版本号与数据的版本号不一致，则可能会提示错误，然后进一步操作或自动根据日志同步为一致的版本。

记录这种日志信息的方式其实就是记录要做什么操作，有可能并不是原始的 SQL，而是每一行的数据修改，有可能还会记录每行数据修改之前和之后的数据分别是什么，或者是基于块的版本记录。

作为写程序的人，我们可以再将思路大胆地放开一点，如果在高并发系统中，对数据库进行 `commit` 操作也是十分频繁的，那么有可能导致日志写操作也会很频繁，而且每次写入的数据也不多。为了解决这样的问题，我们做一个简单的假设：如果这种日志信息不是在 `commit` 操作的一瞬间写入磁盘的，它还有一个简单的 `sync` 机制，例如 3ms 的 1 次同步，或者说在这 3ms 内所有的 `commit` 请求线程会停一下，在这个过程中所有的 `commit` 操作就可以一次性写入到日志中，这对于高并发的系统来讲或许是十分有效的，而且对于大量的随机读/写用户来讲，在最坏情况下 3ms 的延迟是很容易接受的。

(8) 回滚

上面提到了数据丢了要回补，通常叫作“重做”(REDO)，而数据想要回滚就是要恢复到前面的版本(UNDO)，数据库也通常会有一些机制来支持。

这种机制就是重做的反向机制，知道了将数据修改成什么，自然就知道原来的数据是什么，一些数据库自然是以日志的方式来提供支持的，这类数据库会提供日志的查询功能，以及解析日志的功能，可以让数据库的开发者进一步来做操作。

一些数据库记录了每一个块的相应版本保存到某个空间，只要这个空间还够用，就不会将相应版本的数据清空，当需要恢复到某个时间点的数据时，就在某个区域内查找对应

版本号的数据即可。这类数据库通常也会提供一些基于时间点或版本号的数据查询功能，也就是不用直接使用回滚功能，可以直接查询某个时间点的数据（可以只恢复一部分数据），甚至可以查询出要回滚到这个时间点的数据需要操作的 SQL 语句是什么，回滚的动作通常也是执行这样的语句来完成的，而版本号始终是向前发展的。

写这些 REDO、UNDO 肯定会消耗性能，为了提升性能，某些数据库存储引擎会将这些操作去掉。但它同时失去的将是许多数据库本身所具有的功能，它可能会为了保证数据一致性采用“表”级锁，而非行级锁。

（9）版本号读取

版本号的概念前文中已经多次提到，为了更加明确地认识这个概念，这里举一个极端的例子：当发起一个大表的统计操作时，执行了 10 分钟，在第 5 分钟时，对它所查询的部分数据进行增加、删除、修改操作，这些操作已经成功执行，也成功执行了 commit 动作，并且可以确保这个大表统计操作的 SQL 语句还未扫描到这部分修改的数据，那么最终的统计结果是否会包含这些修改后的数据呢？

在数据库中答案通常是 NO，前文中也提到了这一点，数据库的要求也是这样的，从一个 SQL 语句发起的时间点开始算起，它所查询的数据都应当是这个时间点的数据，在这个时间点后修改的数据是不算的。

如果数据被修改和提交，那么自然就需要从前文中提到的类似于 UNDO 的地方去查找历史版本的数据，因此 UNDO 还有这个作用。但是 UNDO 肯定也有存放空间的限制，当 UNDO 这个空间存放不下时，就会将较早版本的数据清理掉，如果一个 SQL 语句发生的时间很长了，并且系统有较高的写操作，这个空间被清理的概率是很高的，那么 SQL 语句在 UNDO 中找不到相应的数据后就可能会报错。通常为了避免这样的问题，都是将这个空间加到足够大，或者尽量避免在极高并发的系统中使用 OLAP 语句来抽取数据。

数据库要完成这些基本功能，就需要各种各样的机制来支撑，其内在机制在这里只是做一个初步的介绍，数据库的内在机制会比这个复杂许多，读到这里希望大家对数据库有一个初步的概念，同时认识到数据库内部会有许多我们值得学习的地方。

►► 6.2.3 索引基本原理

大家都知道索引通常可以提升查找性能，也会降低写性能。这是为什么呢？如果不清楚为什么，就永远不知道优化的思想是什么，只能凭借猜想和经验，但是经验很快就可能过时了。

所以要专业地了解 SQL 如何优化，通常会先了解索引，而索引到底如何提升性能，就要从原理上来理解。知道了如何提升性能，便知道有些数据库为什么会这样选择，而对于我们在面对一些疑难问题时有一个理论的指导，而不是片面的数据推断（当然不是说测试数据不重要，一定是要由理论推导的测试数据才会更加有意义，否则就是在穷举）。

关于索引，大家都会看到一个 B+树的概念，而大部分数据库的常规索引都是通过 B+树来实现的（有的数据库也支持其他的类型，例如 Hash 结构，但这通常是非主流方法），各种数据库都会有所区别，但是换汤不换药，因此在本节主要讲解 B+树在数据库中到底是什么样子的。

索引也是数据，因此它也会存储起来，它的存储原理和数据类似，例如以数据块或数据页的方式存储，但索引并不会存储整行数据（除非是索引组织表，不过有的数据库对于

有 PK 的表默认就是索引组织表), 而是存储索引所对应的字段内容, 然后通过某种标识符与原表关联起来。这种标识符在某些数据库上体现为数据的物理位置, 而在某些数据库上体现为主键值 (有的数据库可能会将某个主键 PK 作为标识符, 如果表中没有主键, 则会隐藏地建立一个主键)。例如 Oracle 数据库会利用数据所在的表空间编号、文件编号、对象编号、块号、块内行号来标识一条数据的物理所在 (这种所谓的物理位置也是相对而言的, 真正的物理位置还需要存储来定位), 这个物理位置将用一个 rowid 来表示, Oracle 的索引上就会存储某一个索引值所对应的 rowid, 以便于找到索引值以后直接定位到数据的行。

由此可见, 索引通常是可以与数据分开存储的 (索引组织表除外), 因此可以单独为索引分配空间, 使得某些索引与数据的存储和管理相分离, 这样一来, 在某些场景下可以充分地利用磁盘并达到局部优化的目的。

查找数据也分为两个过程, 首先通过索引查找到相应的标识符, 然后通过这个标识符到原表中提取相应的数据 (这个动作叫作“回表”操作)。也正是这个原因, 有些时候查询小表不走索引可能会更快, 因为索引要做最少两次操作, 但这仅仅局限在某些情况下。请大家注意, 如果查询中所需要返回的字段全部都在索引中, 那么就不需要回表了, 也就是说, 操作都在索引上完成。

在正常情况下, 向表的数据块中写入一行数据, 就会向索引块中写入一条数据, 而当修改表中的数据时, 如果修改到索引字段, 索引操作通常是先删除再插入, 删除的方式通常是在老的索引位置打一个作废的标记, 而不是物理删除。这也是为什么在某些数据库中大量删除数据后, 索引占用的空间不会变小的原因。由于存在很多作废的索引没有回收空间, 为了提升性能通常会重编译一次索引, 这个动作相当于重建索引。

由于表的修改很多时候会导致索引的修改, 通常大家说数据修改的效率会下降, 其实这也是根据实际场景而定的 (例如 update 时, 如果没有索引, 在大表中定位行的效率是很低的), 并且这种效率下降并没有想象中那么严重, 绝大部分系统也没有要求那么高的性能。

由于索引是有序的, 所以每一次在插入时也会寻找位置, 如果是自动增长的自然都是在最后插入数据, 如果经常在中间插入数据, 则可能会导致局部数据的移动和分裂。接下来我们详细描述一个数据块写入和扩展的过程。



注意 由于索引通常没有存储整行数据, 所以在在一个索引的数据块内可以存储的行会比在数据块中存放的数据行多很多。

当向表中写入一些数据时, 如果数据很少 (例如只有几条或几十条数据), 此时索引通常只有一个块, 这个块就是叶子块, 也是外部访问索引的入口, 内部也像数据块一样存储实际的数据, 以及数据块的物理位置, 通过存放数据块内数据的位置来定位数据, 如图 6-1 所示。

图 6-1 中显示有两个数据块, 但是只有一个索引块, 实际场景可能并非如此, 但我们的目的是为了说明索引的宽度通常是小于数据本身的宽度的, 那么一个索引块存放的内容会更多一些, 自然的索引叶子块通常会比数据块要少一些, 随着数据的增加, 这种现象会越来越明显 (但这并非绝对说明索引的大小一定小于数据块的大小, 如果在一个表上给大部分列都建立了索引就不一定了, 因为索引中除了存放索引数据外, 还会存放数据块的位置, 在数据增加的过程中, 索引还会有其他的存储开销, 后文中会继续介绍)。

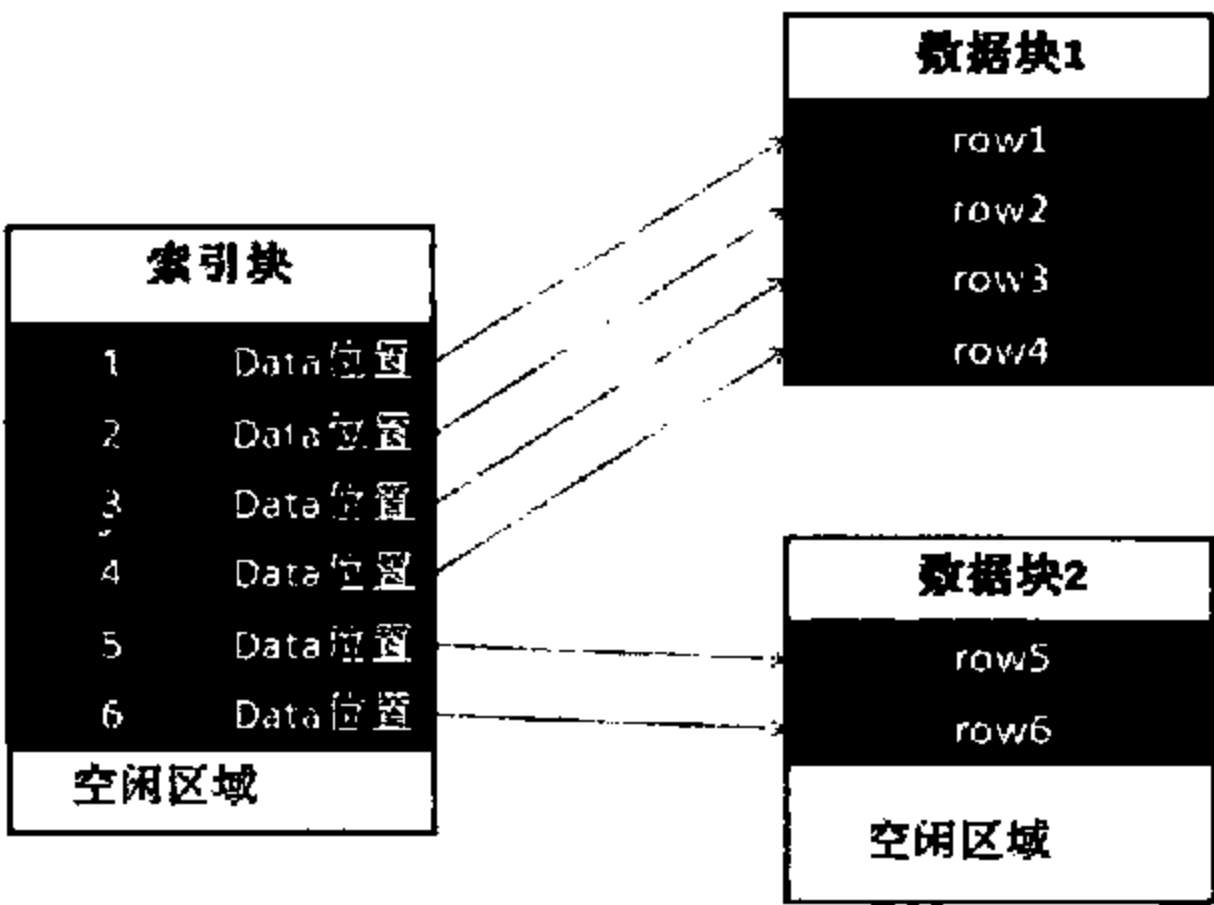


图 6-1 此时只有一个索引块

如果此时只有一个数据块，访问数据时通常不走索引则会更快一些（当然 SQL 语句中请求的字段包含了不在该索引上的字段），因为走索引后还需要做一次“回表”访问，有可能会发生两次 I/O 操作，两次计算。

当数据块越来越多时，索引也同样不可能用一个块存放所有的索引值，它也会用更多的块来存储索引数据，当索引块从一个块变成两个块时（这个过程叫作“分裂”），就需要使用一个管理块来存储两个块数据的范围，形成一个树结构，而分裂的两个块就被称为“叶子块”，两个叶子块之间的数据是顺序的，通过指针相互指向对方，如图 6-2 所示。

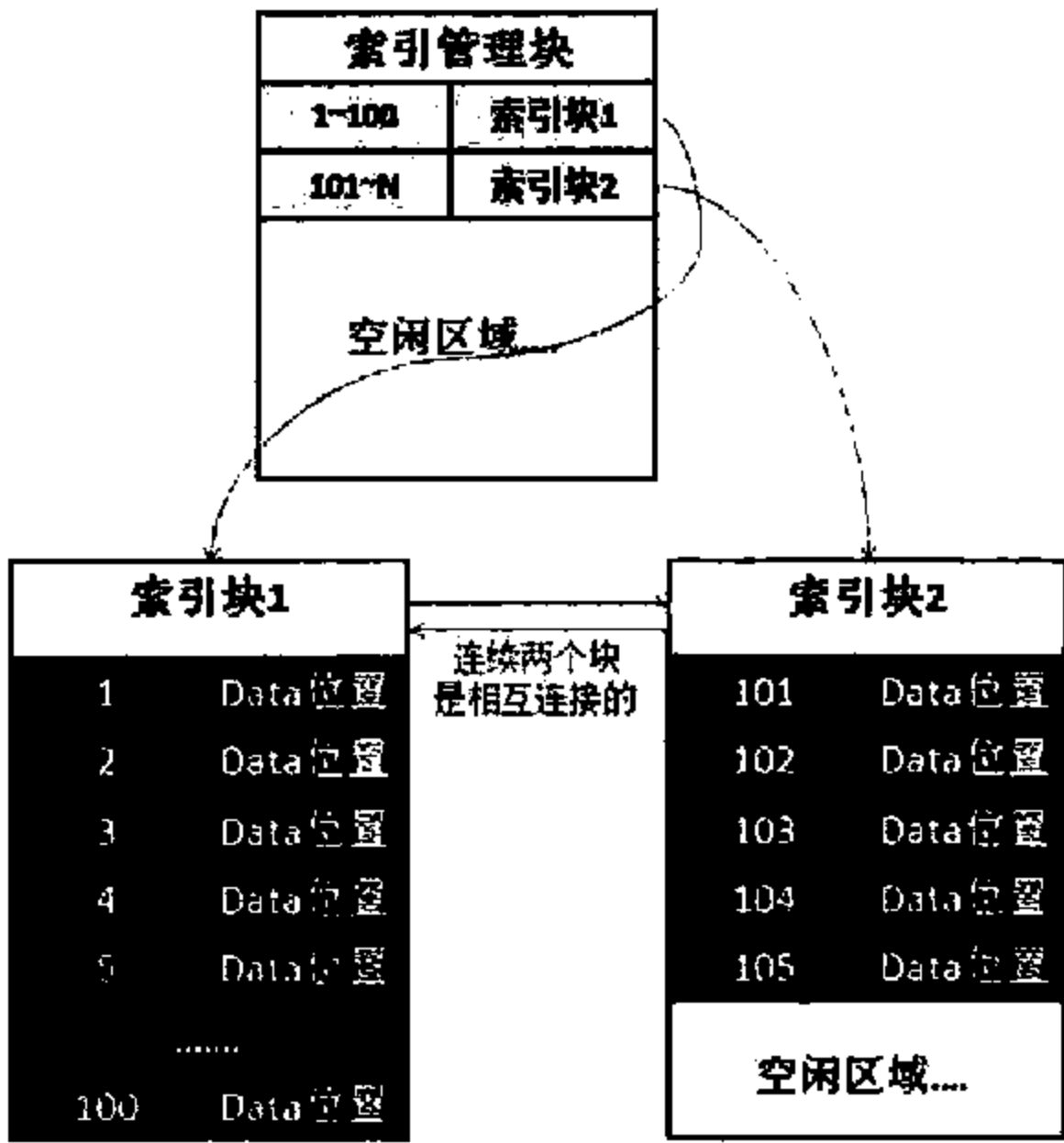


图 6-2 索引块分裂为两个块

管理块也是一个数据块，不过它只需要存储索引值的范围，通常它存储一个“起始值”就可以了，因为后一个块的最小值就可以认为是前一个块的最大值。

在分裂的过程中，如果是类似于自动增长类的索引块，则可以直接分裂出一个空块，在后一个块中写入数据，如果是离散写入的数据，那么为了确保顺序并不会导致频繁的数据移动，通常会将一部分数据拷贝到另一个块中，在这个过程中是有开销的。

由于管理块关注的只是叶子块范围，所以它也可以管理非常多的索引叶子块，通常这个数量会上百或更多，加上叶子块本身可以对应的数据范围，就是几万级别的数据，通常索引只需要两层就可以搞定（除非建立索引的字段太大，导致索引块太多）。换句话说，如果要定位数据，理论上只需要扫描两个索引块就行，由于这样的块占用空间不大，而数据库通常也会配置较大的内存，所以在经过访问后通常都可以被 Cache 在内存中，在适当的

配置下内存命中率会非常高，最终定位数据也许就发生一次物理 I/O 甚至不发生物理 I/O。如果没有索引，则可能会扫描几百甚至几千个数据块才能找到数据，这样数据占用的空间会大很多，放在内存中的概率也低很多，即使全部放在内存中，要找到数据也需要大量的 CPU 匹配计算才能实现。

大家可能还发现图 6-2 中的两个索引块是相互连接的，在索引中通常是叶子块通过类似于双向链表的方式相互指向来达到快速遍历的目的（例如求 count）。

随着数据的增加，索引叶子块越来越多，一个索引管理块已经无法管理下面所有的叶子块了，所以管理块也会分裂，它分裂后就需要一个更高 level 的管理块，更高 level 的管理块每行数据存放的是管理块中的大范围，换句话说，是大范围管理小范围，这样看来管理块之间的数据也是有序的。此时 B+树的结构会变成 3 层，自然对于常规索引的几百万甚至接近于千万的数据也是没有问题的，此时索引可能类似于图 6-3 所示的这样。

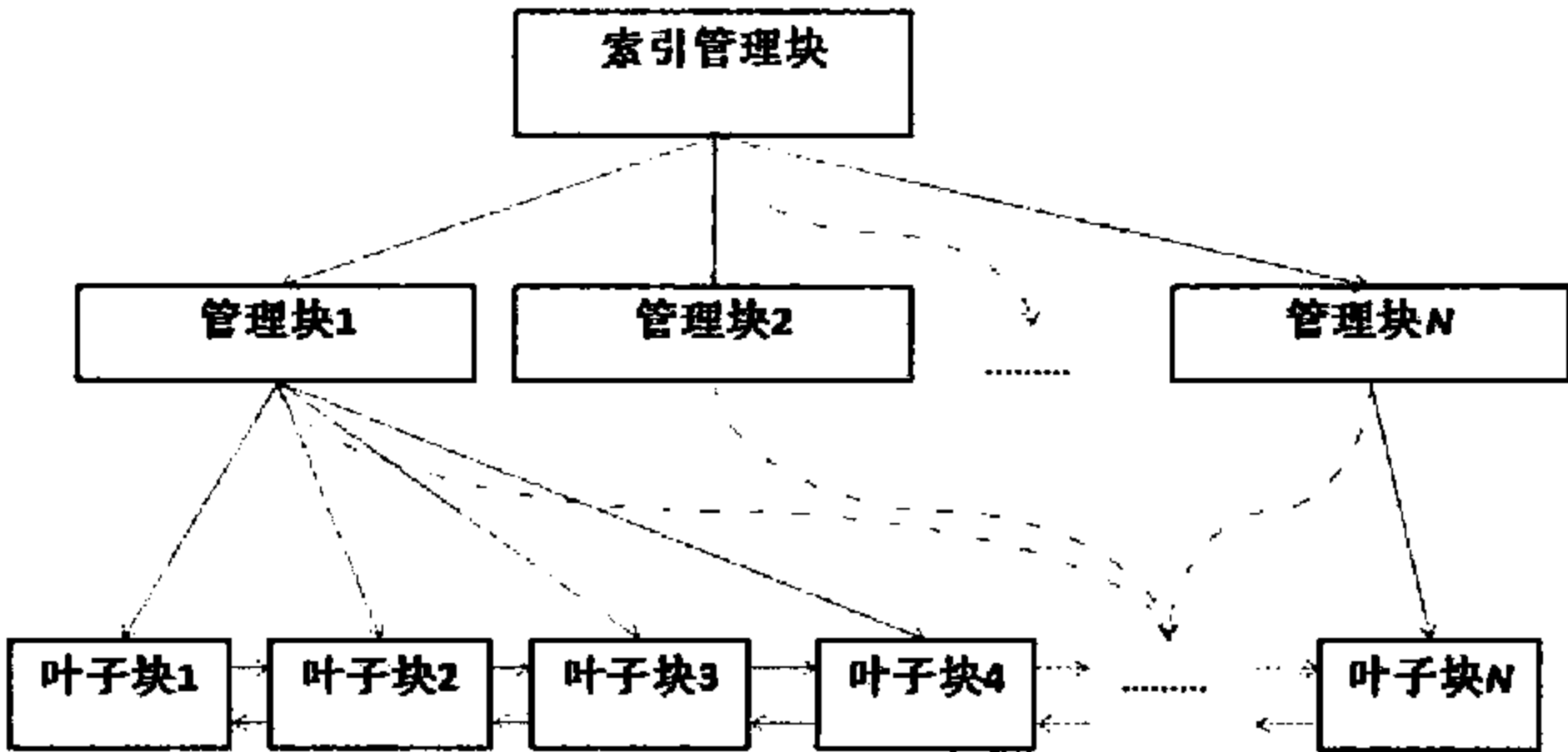


图 6-3 索引树

一个普通的 SQL 查询请求到达数据库时，会根据条件选择是否走索引，如果走索引，那么通常会先从树根开始查找，由于它们都是以块为单位加载到内存中的，所以加载一个索引块在没有 Cache 命中时理论上最多只需要一次物理 I/O。由于索引的数据都是有序的，所以可以通过在块内做二分查找找到数据所属的小范围，在加载小范围的管理块后用同样的方式，就可以得到叶子块，加载叶子块用同样的方式就得到了数据实际的位置，如果检索的数据还需要查询非索引字段，则会“回表”，否则就直接返回索引内部数据。

同样的理论，当索引的层数扩展到 3 层以后，如果该索引访问频繁也会被加载到内存中，至少前两层会很容易地被加载到内存中（因为前两层通常只有几 MB 的空间，在内存够用的情况下，也不排除第 3 层的 Cache 命中率很高的情况）。也就是说，只要配置适当，在大多数情况下前两层索引可以不经过物理 I/O，第 3 层一次物理 I/O 就可以使性能得到极大的提升。反过来讲，如果几百万甚至上千万的数据，每行都有几十个字段，那么这样的数据是不可能被全部加载到内存中的，即使全部加载到内存中，也需要访问几万甚至几十万块来进行大量的匹配计算。

当 SQL 需要做统计操作时（例如 count 操作），如果按照计划，统计操作将通过索引完成，那么执行过程中会先找到索引的第一个叶子块（或许某些数据库会直接记住第一个叶子块的入口），然后从第一个叶子块开始扫描，通过连续的两个叶子索引块之间相互连接所使用的双向链表，可以快速扫描到最后一个叶子块，并进行计算。

因为索引块存储的数据与原表的数据是一致的，所以这样计算的结果肯定是正确的。

一个索引块通常存储的行数会更多一些，所以扫描索引可能发生的 I/O 次数会更少一些，扫描的数据范围也会更小一些，因此对大表做 count 操作走索引会更快。换句话说，当发生某些统计操作时并不一定是主键最快，有可能某些“索引”的值比“主键”的值更小，扫描它自然经历的索引块会更少，效率也就会更高。

在实际场景中 count 的前提都是有 where 条件的，在大多数情况下 where 条件也将决定所走的索引是什么。因此对于 count 的优化，一定要就具体的数据库实现方法而论，而且要看执行计划，并不是通过一两次测试就可以得出结论的，因为测试结果很多时候只是在一些场景下得出的结论，它并不能代表全部，就算代表了全部，也只能说明它在某一种数据库的某一版本下是这样的结论。

由于是双向链表，有些时候为了加快遍历速度，可以并行地从两边向中间遍历。但是不论如何，随着叶子块的增加，这个 count 也会越来越慢，它也存在极限，所以对于十分大的表是不建议做 count 操作的。另外，针对 count 这种类型的操作，某些数据库提供了位图索引，它的 count 效率将是最高的，后文中会提到这一点。

当 SQL 发起 max、min 操作时，如果有 where 条件，则会走正常的索引去找到最大值或最小值；而如果没有 where 条件，就可以直接找到第一个叶子块或最后一个叶子块，因为最大值和最小值肯定在这里。

当遇到了分页时，除了 where 条件通常也会有 order by，然后扫描行号来确定分页的范围，如果使用普通的 SQL 来分页，那么每次经过索引查找到相应的数据范围后，就开始回表扫描数据，扫描数据的过程其实就是在数数以得到行号，随着翻页数量越来越多，需要在回表中扫描的行数也会越来越多（例如 10 万~10 万+20，就必须按照这个顺序扫描到第 10 万条数据再向后扫描 20 条数据），自然的翻页就会越来越慢。

聪明的程序员想到了一些从索引上翻页的方式（因为索引上扫描的块会少很多，而且索引本身是有序的，只需要数数），它的思路大概是这样的：如果数据库提供了物理方式访问，例如 Oracle 的 rowid，此时 SQL 语句的条件照常写索引语句，不过 select 语句不会返回所有需要的列，而是一个行标识符，这样就暂时不会“回表”，在此基础上进行分页过滤，经过过滤后只剩下几十条数据的物理位置时，再来查询原表。

如果没有提供这种物理地址，就通常用主键来代替。换句话说，索引要挂在主键上，或许这种方式有点别扭，聪明的小伙伴们又想出一些新的招数：可以根据条件在索引上查询出分页下限的最大值（例如查询第 10 万到第 10 万+20 条的数据，此时可以先求出前 10 万条数据的最大值），这个最大值其实也是建立在索引上的，所以不会回表，然后通过这个最大值再求出近 20 条数据（将最大值当成一个子查询即可搞定）。

通过实践证明，对于页数非常多的情况，前面 10~20 页和最后 10~20 页被翻阅的次数占据了绝大部分的访问次数，在某些场景下，可以确定是翻前面还是翻后面，那么当翻页翻到靠后的位置时，会尝试将索引 order by 的方式改为 desc 的方式。

使用这样的功能，前提是索引建立得足够合适，让条件和排序在索引上可以完成，而且业务不是特别的复杂，否则无法使用这个功能。

另外，大家会发现这样的索引查找，始终脱离不了一些很傻的逻辑过程，尤其是数数的过程，无论我们想不想让它数，都要数，要脱离这个节奏，就需要采用更多的设计思路，这些将在后文中介绍。

当遇到了复合索引（即多个字段的索引，假如 `column1`、`column2`）时，在 SQL 中只用 `column1` 来查询没有太大的问题，但是当用 `column2` 来查询时可能会有很多问题。由于索引是有序的，但是有序的条件是前缀条件，单纯的后缀本身是无序的，所以它很可能会走全表扫描，如果要走索引，数据也会分散到多个板块中，那么就要跳跃式去找这些叶子块里面的数据（有些时候把它叫作 `skip`），匹配完以后依然要回表，它有可能导致效率比全表扫描更低。

同样的道理用在 `like` 上，如果 `like` 是基于前缀的，自然可以缩小数据的范围，走一种范围查找（`Range`），但是如果基于后缀来查找，很可能就会走全表扫描。

在 `IN` 的检索上，通常会被解析为和 `OR` 类似的方式去完成，`IN` 的检索通常是离散在多个块当中的，在通常情况下每个条件都需要单独去查找，所以在 `IN` 的数量达到一定比例的情况下会自动选择走全表扫描，不过这也和表的大小有一定的关系。

但这并不代表 `IN` 一点用途都没有，也不代表 `IN` 就一定比 `exists` 效率差，具体要看执行计划中的表与表之间的驱动关系，会不会让一些大表走理想中的索引。例如，通常子查询不会返回太多的数据（否则不适合子查询），所以理想中是用子查询的结果作为外部查询的条件，让外部的大表走索引，而当某些复杂的 SQL 操作导致计算机无法识别时，可能是用外部的查询结果驱动，循环访问子查询看是否匹配，这样的结果是很恐怖的。

唯一性索引（`UK`、`PK`）本身与普通索引的结构没有太多的区别，唯一的区别是在进行等值查询、少量的 `IN` 查询时，只需要找到相应的一条数据就不会再查找相关范围的数据了，这样在检索时效率会更高一点，但普通索引并不确保是否有重复的索引值，所以会在找到一条匹配的索引数据后，还要再去寻找相关范围的索引数据。

提到了普通索引的重复，就会想到压缩的概念，如果重复的数据经过压缩，那么一个叶子块存储的索引信息会更多，理论上索引块就会更少，那么整个索引可能会更节约空间，理论上查找的效率会更高。压缩方式自然是让同一个块中的同一个“索引字段的值”只存储一份，但会保存多个数据块地址。复合索引中的压缩，也不一定非要将所有的字段进行压缩，可以按照某个前缀字段压缩，压缩的条件自然是它的重复率很高的属性。不过这个方法说起来简单，实现起来麻烦，内部还会有各种各样复杂的问题存在。

当一个表中的某个状态字段只有几种值时，如果仅仅是为了在这个字段上实现高效的统计，那么此时用位图索引（`BitMap`）是最佳的选择（不过一些数据库不支持位图索引）。位图索引的结构类似于 `B+` 树，不过由于它清楚数据有几种值，所以在存储方式上会先按照状态值分开，然后在每一种值的空间中会单独存储每个实际的数据行是否是这个值，如果是则用 1 表示，如果不是则用 0 表示。在这样的情况下，都可以不用存储数据的物理地址了，因为每一种值都会有所有的行序列，当然如果是也可以存储与这个位图相关的行所在的物理地址。此时由于只需要存储是与否，所以通常只需要用一个 `bit` 位来存放，因此理论上一个叶子块可以存储非常多的 `bit` 位用来表示不同的行。所以用它来统计时是非常快速的，加载到内存中后几乎是纯 `CPU` 的叠加操作。

需要注意的是，位图索引有一个很大的缺陷是它的锁粒度很大，当尝试新增或修改一条数据时，通常与它在同一个位图的数据操作都会被锁住。例如通过状态字段设置了位图索引，假如有 1、2、3 三种状态，此时对某行数据进行修改，它的状态是 1，这个操作将会锁住所有状态为 1 的行；若尝试将状态 1 设置为状态 2，则锁住这两种状态的所有行，这个锁也同样要等待 `commit` 或 `rollback` 后才会释放。所以它并不适合于高并发联机事务处

理系统 (OLTP), 而适合于并发较少且统计运算较多的 OLAP 类系统。

▶▶ 6.2.4 数据库主从基本原理

对于程序员来讲, 可能不需要知道太多数据库备份的知识, 以及操作方法和故障处理方式, 但是可以了解一下备份的基本原理, 因为备份也并不是没有开销, 也并不是可以无限利用, 它同样可能会在一定程度上影响总体的架构以及性能。

前面提到过数据库也是由程序组成的, 只是它是一套写好的软件程序, 因此所有动作都是用代码来完成的, 备份也不例外, 只是不同的备份有不同的方式。

一套主从库中通常会有一个可写的主库 (也有双主互备结构的), 可写的主库中发生数据修改后, 可能会以“推”的方式发送到从库上, 或从库上轮询从主库上“拉”修改的数据。而修改的数据从主库到从库执行的过程有很多种模式, 下面介绍两种。

◎ 第 1 种方式就是逻辑模式, 这种方式是一种容易理解的方式。例如可以基于主库执行的 SQL, 到从库上去执行相同的 SQL, 也可能会基于主库上执行 SQL 时所影响的数据行, 进一步根据这些行的主键生成一系列 SQL 语句到从库上去执行。这种粒度是最细的, 也是最容易识别的, 只是生成的 SQL 会比较多。

针对这样的方式, 我们会发现:

如果以 SQL 方式去执行, 那么从库肯定会有一定的压力, 但这种从库的写通常是单线程来完成的。不论主库有再高的并发压力, 从库只有一个线程在发生写操作, 不断做循环写操作, 在这个过程中可以减少大量的系统资源征用问题。这好比一个单独的系统在抗高并发访问时负载会很高, 但用一个单线程不断循环去做事情时负载往往不高, 而且效率可能还不错, 例如用单线程循环写磁盘数据一小会就可以把磁盘写满, 而且程序本身压力不会太大。不过很自然的是, 主库写得太快的话, 从库有可能会跟不上主库写入的速度, 此时就要考虑用其他方式来解决。

例如并行复制之类的操作, 不过将这个并行打开后, 自然在从库上的并行写也会被打开, 从库的压力会开始上升, 而此时从库如果再提供其他的读访问, 也许效果就没有想象当中那么好了。将延迟、并行等概念结合起来看, 在这类数据库上读/写分离, 或者说在从库上提供读服务, 很多时候都是在读多写少的场景下。

◎ 第 2 种方式通常是物理模式, 它基于修改的数据块来做复制, 它可以提取比从库版本号高的数据块复制到从库。由于以数据块为单位进行操作, 所以通常它不需要执行 SQL, 可以不用经历复杂的 SQL 解析、锁、调度等逻辑, 它的效率自然会高很多, 但同时也会有一些新问题, 这样的从库如果要提供对外访问就相对比较麻烦。

另外, 在主从库方面, 还有更多的专业知识, 例如延迟解决方案, 写入数据与从库拿到数据的时间点, 跨机房、异地容灾解决方案, 以及区域化数据访问解决方案。

▶▶ 6.2.5 我们经常相信的那些经验

从我们第一天学习写 SQL 开始, 就以学懂一种语法规则为兴趣点, 然后我们欣喜于自己学会各种语法规则, 能写出复杂的 SQL 语句, 或许在某个阶段开始感觉够用了。

在面对一些实际的业务背景时, 编写 SQL 的技巧开始给我们许多灵感, 尤其是它的灵活性让同一个需求有多种不同的写法, 大家会庆幸自己懂得许多写法。而逐渐的, 面对越来越复杂的业务背景时, 大家会开始摸索一些写法上的区别, 可能会参阅别人的一些资料, 在某些业

务场景下经过测试得到了一些验证,开始有了自己的看法,这些看法可能和别人一样或不一样。

当场景发生改变时,许多结论并不是我们想象中的那样,也就是说,并不是通过一些列举性的测试得到的结果就可以把它当成一种真理或经验,我们希望通过原理和理论来指导实践的方向,通过原理来定位问题的根本所在,测试是必要的,测试是用来验证理论指导的,测试是用来驱动理论方向的,在理论指导的前提下来测试,得出的结论将会更加靠谱,而此时得出的某些结论通常会这样描述:在使用什么环境、什么字符集、多大的数据、机型配置、内存设置、优化参数、索引状况、读/写状况等条件下的一个测试数据。

类似的,在前文中提到的数据库原理,如果你已经大概理解了,那么应该了解到数据库并不是想象当中那样简单,也并不是想象当中那样可以无止境地放大,它的内部体系架构将决定许多极限,同时它的一系列内部机制将决定某些问题的处理方法。因此,我们要知道任何东西存在即有价值,没有任何场景的优化方案都是“扯淡”的,包括一些 SQL 的写法也与具体的数据库和具体的应用场景有关系。当然别人说的可以参考,但是切忌当作自己的结论,而且再将这种自己没有切身体验的结论传授给别人。

在下一节中,胖哥将利用一些实际的场景,再结合本节所介绍的一些数据库基础知识,来讲解从程序员角度如何看待 SQL 的优化方法。

6.3 从程序员角度看数据库优化方法

➤➤ 6.3.1 不同领域的 SQL 区别

在前文中将“位图索引”与“普通索引”对比时,大家应当感受到不同场景下的索引建立思路会有所区别,而不同领域的 SQL 解决方案也会有许多区别。

在互联网行业中,大家追求更多的是高效,而不想被十分复杂的内在机制所牵制,导致出现许多不可预料的后果,而同时也希望在性能上更加具有水平扩展性,所以希望自己能来控制一些抽象层次,将许多业务逻辑编写到应用程序端。SQL 语句通常是十分简单的,可以说绝大部分的 SQL 语句都是简单的 SQL 语句,此时数据库的作用就像是一个提供了事务管理的存储平台。

而在传统行业中,在大多数情况下数据库没有多大的问题,很多时候希望数据库能够解决更多的业务问题,这样程序的代码会很简单,会将 SQL 写得很复杂,甚至于用存储过程来解决许多问题,这样做会在某些时候让很多看起来相对复杂的问题变得简单,在这样的情况下很多时候也会发现应用程序的 DAO 层基本没做太多的事情。

存储过程通常是被编译好的,它是一段代码在服务器端执行,通常我们只需要传入参数让存储过程执行即可。许多人会说这样快很多,因为它除了编译还节约了网络开销。

胖哥认为这要看实际的数据库场景,以及实际的网络状况,目前许多数据库也会对 SQL 语句做编译和 Cache 动作,而现在的网络更加不会成为什么大问题。

网络主要体现在延迟层面,网络通信条件不好,每次交互的延迟就会很严重,如果交互次数多,自然延迟是成倍上升的(例如跨国访问数据库)。对于延迟很低的网络状况,这个时间是可以忽略的,因为在正常的交互中,循环请求的次数不会太多,如果遇到了循环太多的交互(例如是一个数组循环提交数据),即使是调用存储过程,也需要循环将数据交给存储过程来执行,不过部分数据库的 API 中允许传递数组这类对

象,这样就可以一次性传递一批数据了,例如 Oracle 的 OCI 接口,但是 Bug 非常多,如果要跳过数据库本身的限制,变相的数据传输方式就是将多条数据组装成 XML、JSON 格式进行传送,接收方得到数据后将其进行解析,再进行循环处理。这样的方式类似于 Batch 处理,即一个 SQL 语句对应到一批参数,可一次性传递(部分数据库的 JDBC 处理方式会有所区别)。

存储过程,就是操作存储的过程,也就是对存储计算的过程描述,它通常不是用一个步骤就能达到目的,也通常不止操作一条 SQL 语句,将一个逻辑的数据操作分解为多个清晰的步骤来完成,或者在数据库服务器端对数据进行批量操作,这样数据库处理起来速度会非常快(例如要将一个表里面的部分数据,按照某种数据规则转换后,再将结果分解到其他的几个表中)。不过,对于业务逻辑上的复杂数据处理,用应用代码来编写其实比用存储过程写还会清晰一些。

当然,根据前面的理论,存储过程也可以用在网络状况不算太好、且会反复做操作的地方(否则我们需要在数据库的服务器端编写一些代理程序),当我们选择了利用存储过程后,一些数据库还提供了对存储过程 keep 到内存的操作(因为它本身也是一个数据库对象,如果不用,数据库也不希望它占用空间),这样就可以手动将它加载到内存中,而且通常不会被释放出去。

除此之外,许多数据仓库类的 SQL 还会利用一些特殊的函数,例如 Oracle 有一个叫作“分析函数”的东西(有的说法也叫“窗口函数”),它是一系列十分强大的工具,在编写 SQL 时利用分析函数可以让许多不容易实现的功能变得简单。

举个例子,有一张表包含了一个学校所有学生的每个科目的期末考试成绩,此时要挑选出学校里面每个班级、每个科目的前 3 名学生列表(前 3 名需要考虑的情况包括:并列排名、非并列排名、并列排名出现后的排名跳跃,例如有两个并列第 2 名,那么下一名可以被认为是第 4 名或第 3 名)。这个需求似乎十分简单和平常,但是如果用传统的 SQL 来编写却是十分复杂的,甚至不太可能用一个 SQL 语句得到结果,而一些数据库上提供的分析函数可以十分简单地搞定这个问题。

上面提到的问题,如果是在 Oracle 数据库中,大家可以使用 row_number()、rank()、dense_rank()这几个函数分别看看结果有什么区别,便知道真相了。

6.3.2 执行计划

前面多次提到了执行计划,那么执行计划到底是什么?怎么运行执行计划?又怎么来认识执行计划?下面我们将以 Oracle 数据库为例,概要性说明一些执行的查看方法,让大家有个感性认识(以 Oracle 为例是因为它有足够多种类的执行计划,中间我们也会穿插部分 MySQL 的执行计划)。

首先来看看怎么运行执行计划。

以 Oracle 为例,如果在命令行下要运行执行计划,方法有很多种,比较通用的方法是先执行 explain plan for <sql>语句来运行执行计划,或者说将执行计划分析的结果写到一个地方,然后再执行 select * from table(DBMS_XPLAN.DISPLAY)获取到执行。

另外,也可以使用 SET AUTOTRACE ON EXPLAIN、SET AUTOTRACE ON、ET AUTOTRACE TRACEONLY、SET AUTOTRACE TRACEONLY STAT 等命令来打开执行计划,在打开执行计划后,运行 SQL 就会输出相应的执行计划。不过,在这种方式下,不能

使用动态绑定的参数。

要运行 MySQL 的执行计划则比较简单，直接通过命令 `explain<sql>` 就可以输出执行计划了。

对于各类数据库都会有一些三方的操作工具，通常会提供十分强大的功能（尤其是桌面工具的功能更为强大），也通常包含了执行计划的功能，例如 PL/SQL Developer 对于 Oracle 数据库来讲提供了十分强大的操作功能（在 SQL Window 窗口，按 F5 键就可以运行对应 SQL 的执行计划），而其本身又十分的轻量。

接下来在一个 Oracle 库上建立几个表。
我们要做实验，就需要在对应的数据库上建立几个表，并且建立相应的索引后，再写入一些数据来做测试。胖哥在一个 Oracle 库上建立了一个简单表 `ZZ_XIEYU_TABLE1`，里面存放的数据并不多，就十多万条，但是足以看出一些问题，其表结构和索引如图 6-4 所示。

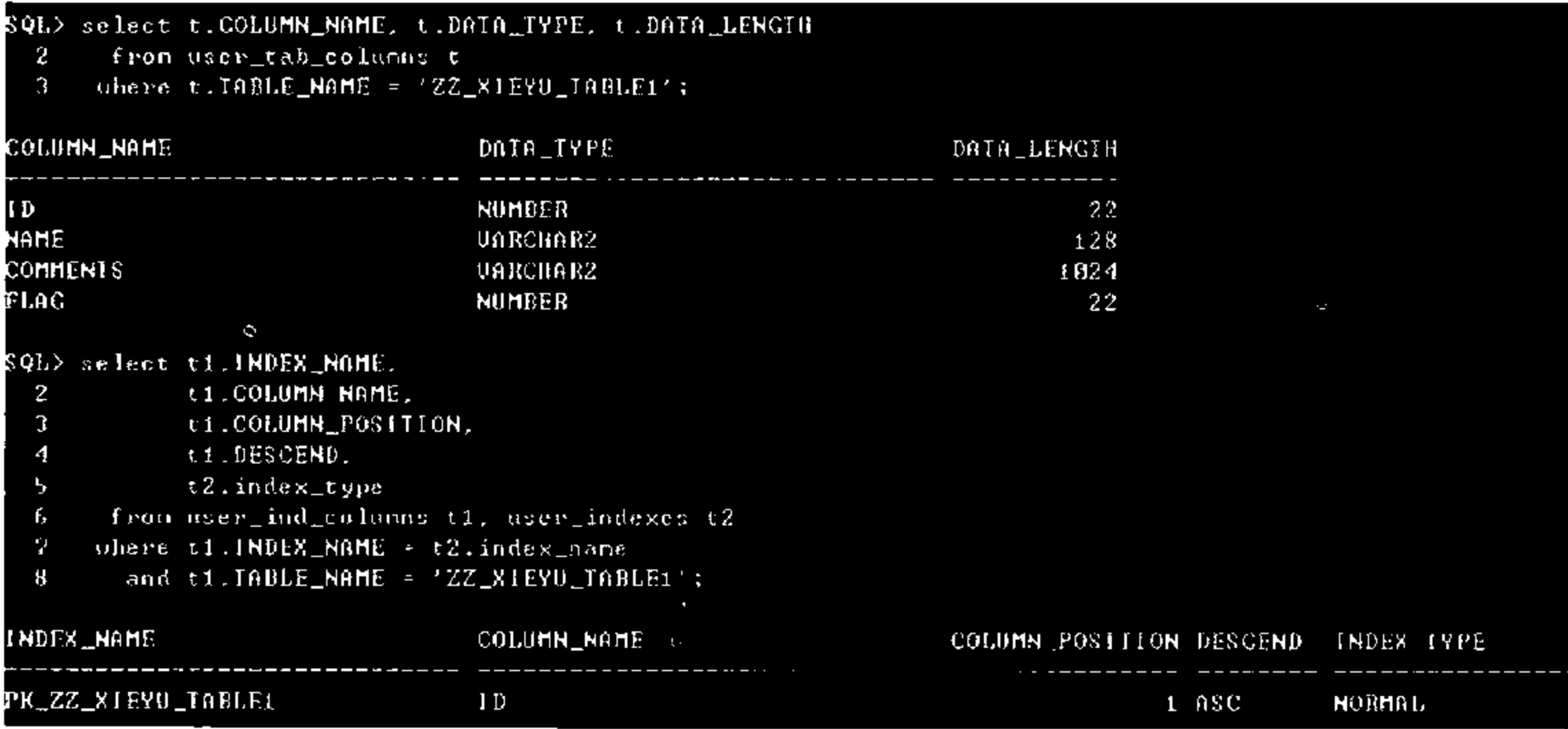


图 6-4 建立表结构

这个表有 4 个字段，但只在 ID 字段上建立了一个索引，下面我们来测试一些场景。由于我们知道在 ID 字段上创建了索引，所以就基于 ID 来做一个查询，但返回的字段是 `flag`，执行计划如图 6-5 所示。

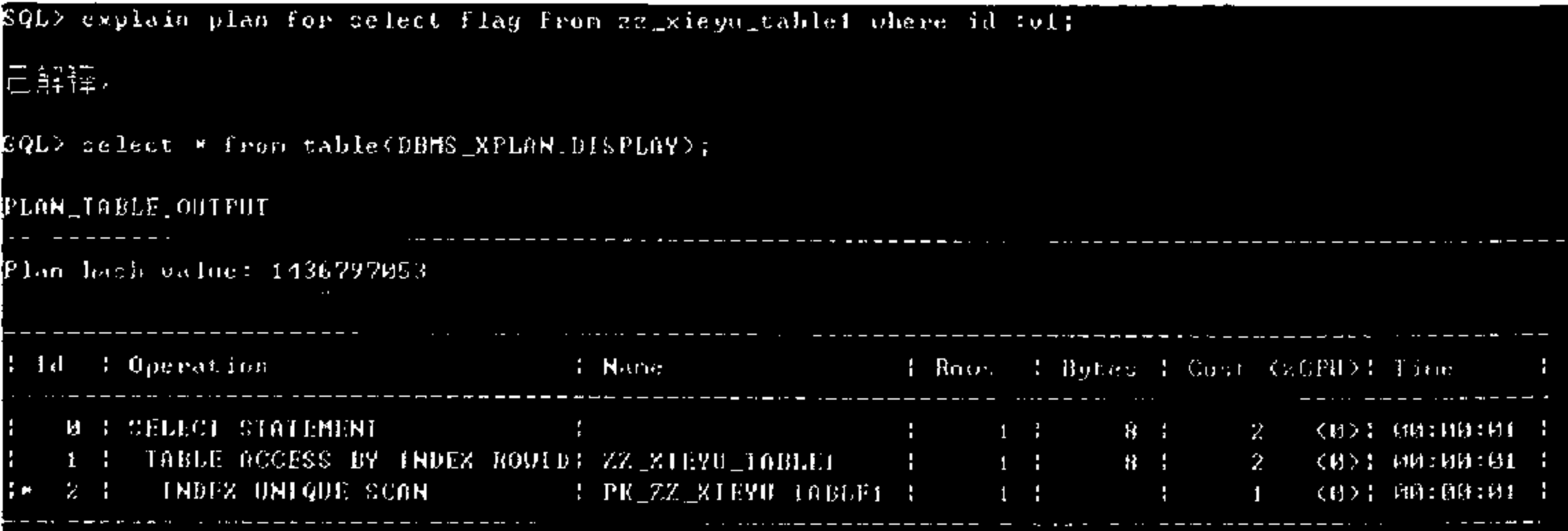


图 6-5 这条语句走了唯一性索引

这个执行计划怎么看？根据缩进从内向外看，首先基于索引 `PK_ZZ_XIEYU_TABLE1` 走了唯一性索引查找数据，然后在表对象 `ZZ_XIEYU_TABLE1` 上做 `TABLE ACCESS BY INDEX ROWID` 操作，也就是基于 `ROWID` 进行“回表”的操作，最后回到 `SELECT` 语句上。大家可以自行测试，将返回的列变成 `ID` 会是什么样的结果呢？中间的回表动作就不见了。如果将返回的列再改成 `*` 呢？这作为思考留给大家去发挥吧。

在每一行中的后面都会涉及扫描数据的行、字节数、开销等信息，这个开销是通过一些复杂的计算方法计算出来的，在一定程度上可以看出总体开销，不过它并不能说明所有的问题，就像有些时候我们不能以 Load 高就说明系统肯定有问题一样。

在 SQL 语句中，:v1 是一个占位符，即这种以 “:” 开头的符号代表一个动态参数（冒号后面的具体信息是什么不重要，只需要知道每个符号都代表一个占位符即可，同名的占位符代表的是同一个参数），但是具体值不定，就像通过 JDBC 编写参数时使用 “?” 作为占位符一样，在 Oracle JDBC 传递 SQL 前会将 “?” 转换为 Oracle 所能识别的占位符，Oracle 服务器端才能做相应的解析处理，而前文中提到将 SQL 语句的编译信息 Cache 在服务器端，也是保存这样的语句。

前文中我们一直再说全表扫描，全表扫描在执行计划下到底是什么样呢？

这个表除了 ID 字段外，其余的字段都没有索引，我们用 flag 作为 where 条件来看看执行计划是什么样的，请看图 6-6。

```
SQL> explain plan for select * from zz_xieyu_table1 where flag=:v1;
已解释。
SQL> select * from table(DBMS_XPLAN.DISPLAY);

PLAN_TABLE_OUTPUT
-----
Plan hash value: 2299008676

-----
| Id | Operation          | Name                | Rows  | Bytes | Cost (CPU)| Time     |
-----
|  0 | SELECT STATEMENT    |                     |      1 |   256 |       77   (2)| 00:00:01 |
|*  1 |  TABLE ACCESS FULL| ZZ_XIEYU_TABLE1    |    131K| 2560K |       77   (2)| 00:00:01 |
-----
```

图 6-6 一个全表扫描的例子

在 Oracle 数据库中，见到 TABLE ACCESS FULL 时，就说明走了全表扫描，对于大表走全表扫描是很恐怖的。在这个地方大家可以发现扫描行、字节数、开销都有了明显的上升，而且这也说明了全表扫描就是要扫描所有的数据行，不论在内存中发生逻辑读还是在磁盘上发生物理读，它始终要逐行匹配。同样的，如果什么条件也不加，肯定也是全表扫描，这个结论大家可以自己去验证。

有人说 count(1)、count(*)、count(主键)、count(索引字段)快，那么有没有说明为什么呢？

现在我们就用 Oracle 来看看到底执行计划是什么样的吧。首先来看看比较常用的 count(*)吧，它的执行计划如图 6-7 所示。

```
SQL> explain plan for select count(*) from zz_xieyu_table1;
已解释。
SQL> select * from table(DBMS_XPLAN.DISPLAY);

PLAN_TABLE_OUTPUT
-----
Plan hash value: 2947001881

-----
| Id | Operation          | Name                | Rows  | Cost (CPU)| Time     |
-----
|  0 | SELECT STATEMENT    |                     |      1 |       42   (0)| 00:00:00 |
|  1 |  SORT AGGREGATE     |                     |      1 |           |          |
|  2 |   INDEX FAST FULL SCAN| PK_ZZ_XIEYU_TABLE1 |    155K|       42   (0)| 00:00:01 |
-----

Note
-----
   dynamic sampling used for this statement (level 2)
```

图 6-7 count(*)的执行计划

在这里大家看到的 INDEX FAST FULL SCAN 就是扫描所有的索引块的过程，扫描的索引自然是名称列所显示的 PK_ZZ_XIEYU_TABLE1，然后通过 SORT AGGREGATE 来做数据合并。在这里可以看到扫描的行比全表扫描的行还要多，这并不能说明它比全表扫描慢，多出的这些行应该是索引块扫描过程中所留下的。根据这里显示的 Cost，可以发现它的开销并不是特别高。

从这一点大家也应当发现，count(*)在经过数据库优化后，如果存在索引，则会通过索引走执行计划（由于这里只有主键，所以我们不能确保一定会走主键，实际情况我们看看执行计划就知道了）。同样的，数据库不会那么傻在 count(*)时非要将所有的列都拿出来计算行数，很多人也认为这样很占用带宽，对于稍微懂点计算机常识的人也知道不应该这样去实现，何况现代的计算机已经飞速发展了这么多年，这些远古时期的问题，就不要再把它当成永恒的真理了。

以同样的方式，大家可以来看看 count(1)、count(主键)的结果是一致的，如图 6-8 和图 6-9 所示。

```
SQL> explain plan for select count(1) from zz_xieyu_table1;
已解释。
SQL> select * from table(DBMS_XPLAN.DISPLAY);

PLAN TABLE OUTPUT
-----
Plan hash value: 2947001881

-----
| Id | Operation                | Name                | Rows  | Cost | (%CPU)| Time     |
-----+-----+-----+-----+-----+-----+-----+
|  0 | SELECT STATEMENT          |                     |      1 |    47 |      0 | 00:00:01 |
|  1 |   SORT AGGREGATE          |                     |      1 |    47 |      0 | 00:00:01 |
|  2 |    INDEX FAST FULL SCAN    | PK_ZZ_XIEYU_TABLE1 | 155K  |    47 |      0 | 00:00:01 |
-----+-----+-----+-----+-----+-----+

Note
-----
- dynamic sampling used for this statement (level=2)
```

图 6-8 count(1)的执行计划

```
SQL> explain plan for select count(id) from zz_xieyu_table1;
已解释。
SQL> select * from table(DBMS_XPLAN.DISPLAY);

PLAN TABLE OUTPUT
-----
Plan hash value: 2947001881

-----
| Id | Operation                | Name                | Rows  | Cost | (%CPU)| Time     |
-----+-----+-----+-----+-----+-----+
|  0 | SELECT STATEMENT          |                     |      1 |    47 |      0 | 00:00:01 |
|  1 |   SORT AGGREGATE          |                     |      1 |    47 |      0 | 00:00:01 |
|  2 |    INDEX FAST FULL SCAN    | PK_ZZ_XIEYU_TABLE1 | 155K  |    47 |      0 | 00:00:01 |
-----+-----+-----+-----+-----+-----+

Note
-----
- dynamic sampling used for this statement (level=2)
```

图 6-9 count（主键）的执行计划

可以说，在这样的场景下，几个 count 的结果是一模一样的，但是胖哥没有办法证明所有的场景它们都是一样的，更加没有办法告诉大家所有的数据库的所有版本中都是这样的结论，胖哥只能告诉大家学会自己去看执行计划，不要永远认为什么是对的，什么是错的，所有的技术都在不断发展，我们要学会自己分析原理和解决问题。

为了进一步说明这一点，我们在该表的 flag 字段上建立一个普通索引，如图 6-10 所示。

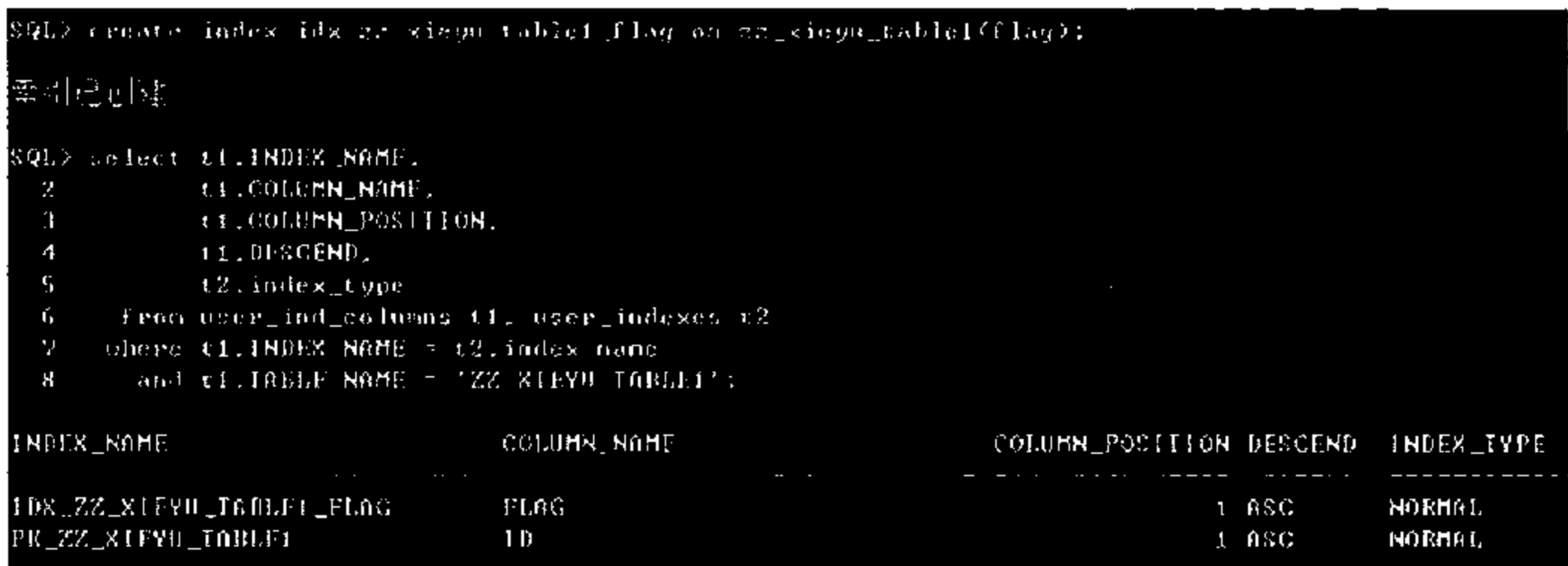


图 6-10 在 flag 字段上创建一个普通索引

胖哥的这个表的 flag 字段就只有两种值，此时如果用 flag 作为 where 条件，大家可能会发现依然会做全表扫描，因为 Oracle 认为通过索引扫描出来的数据占比会很大，而且还要回表操作，不如直接做全表扫描“划算”，因此就直接走全表扫描了。

这个结论大家可以自行测试，得到这样的结论后，就应当理解到，并不是索引就一定能提升性能，它和数据分布有关系，如果某些局部数据有特殊增长的情况，则可能会导致全表扫描，此时需要其他的解决方案来处理，而不是索引。

在这里对 flag 字段做 count 操作，看看结果是什么样的，请看图 6-11。

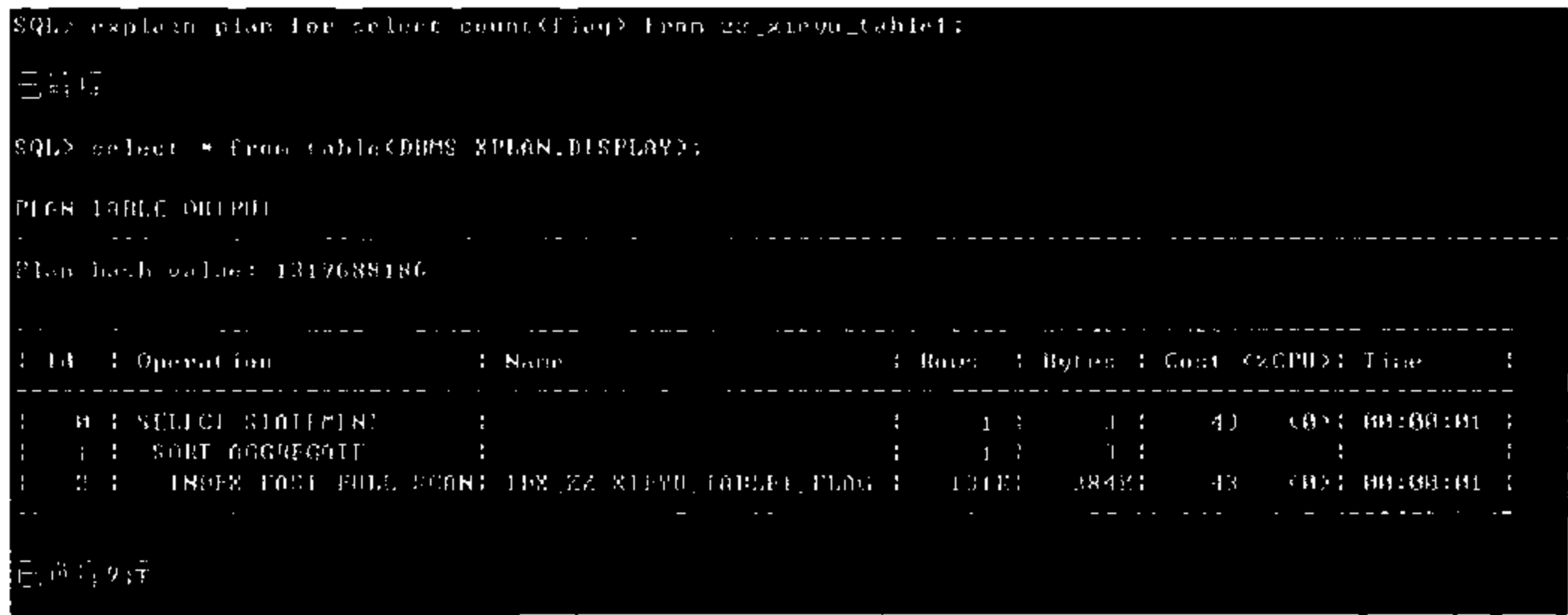


图 6-11 对 flag 字段做 count 操作

同样是 INDEX FAST FULL SCAN，但是这一次走的索引是 IDX_ZZ_XIEYU_TABLE1_FLAG，此时发现扫描的行、字节数、开销都要小一些，难道它的性能比主键还好？没错，主键也未必是最快的。如果仅仅是比速度，或许从十多万的数据还看不出结果，大家可以用更大的数据量来测试，得出的结论应该是只要索引值本身比主键小，做某些操作它的速度就有可能会比主键快。

既然在 flag 字段上创建了索引，我们就顺便再证明一下前面提到的回表操作（此时用普通索引得出的结论或许更具有说服力），如果只查询 flag 字段，理论上它不会回表，请看图 6-12。

这里依然走了 flag 字段的索引，但是没有做任何 TABLE ACCESS BY INDEX ROWID 的动作就直接返回数据了。

同样的，回到 count 的话题，flag 字段只有两种值，这种统计运算是否可以使用前文中提到的位图索引来做呢？这样做的效率如何？

此时将索引修改为位图索引，需要执行的动作是 drop index idx_zz_xieyu_table1_flag，再执行 create bitmap index idx_zz_xieyu_table1_flag on zz_xieyu_table1(flag)，执行后再运行 count 的执行计划，如图 6-13 所示。


```
SQL> explain plan for select flag from zz_xieyu_table1 where flag=01;
已执行.

SQL> select * from table(DBMS_XPLAN.DISPLAY);

PLAN_TABLE_OUTPUT
-----
Plan hash value: 4231329115

-----
| Id | Operation          | Name                | Rows  | Bytes | Cost (CPU%) | Time     |
-----
| 0  | SELECT STATEMENT    |                     |       |       |              |          |
|* 1  | INDEX FAST FULL SCAN| IDX_ZZ_XIEYU_TABLE1_FLAG | 131K  | 384K  | 44 (CJ)     | 00:00:01 |
-----

Predicate Information (identified by operation id):
-----
1 - filter("FLAG"=TO_NUMBER(:01))
```

图 6-12 没有任何回表操作

```
SQL> explain plan for select count(flag) from zz_xieyu_table1;
已执行.

SQL> select * from table(DBMS_XPLAN.DISPLAY);

PLAN_TABLE_OUTPUT
-----
Plan hash value: 118931720

-----
| Id | Operation          | Name                | Rows  | Bytes | Cost (CPU%) | Time     |
-----
| 0  | SELECT STATEMENT    |                     |       |       |              |          |
| 1  | SORT AGGREGATE      |                     |       |       |              |          |
| 2  | BITMAP CONVERSION TO ROWIDS |                     | 131K  | 384K  | 3 (CJ)     | 00:00:01 |
| 3  | BITMAP INDEX FAST FULL SCAN| IDX_ZZ_XIEYU_TABLE1_FLAG |       |       |              |          |
-----
```

图 6-13 位图索引统计

从这个计划来看，这个位图索引的统计操作开销降低了 10 倍以上，这个结论可以用更大的数据量来支撑。不过大家要注意了，并不是所有的字段都适合建立位图索引，它仅仅适合于某些状态位的字段，而且前文中也提到，它并不适合于 OLTP 类系统。

现在来看看范围索引。

如果用主键 ID 直接等值匹配，则肯定会进行唯一性索引查询，如果要做范围查询，则可以让它查询一个 ID 的范围值，例如使用 BETWEEN AND 语句来实现，如图 6-14 所示。

```
SQL> explain plan for select flag from zz_xieyu_table1 where id between :01 and :02;
已执行.

SQL> select * from table(DBMS_XPLAN.DISPLAY);

PLAN_TABLE_OUTPUT
-----
Plan hash value: 3981752342

-----
| Id | Operation          | Name                | Rows  | Bytes | Cost (CPU%) | Time     |
-----
| 0  | SELECT STATEMENT    |                     |       |       |              |          |
| 1  | FILTER              |                     |       |       |              |          |
| 2  | TABLE ACCESS BY INDEX ROWID| ZZ_XIEYU_TABLE1    | 128  | 2624  | 50 (CJ)     | 00:00:01 |
|* 3  | INDEX RANGE SCAN     | PK_ZZ_XIEYU_TABLE1 | 590  |       | 3 (CJ)     | 00:00:01 |
-----

Predicate Information (identified by operation id):
-----
1 - filter("ID">=TO_NUMBER(:01) AND "ID"<=TO_NUMBER(:02))
```

图 6-14 主键在查找范围时也会走范围索引

在 Oracle 数据库中，范围索引的明显标志就是 INDEX RANGE SCAN，这个开销很明显已经大于查找一条数据的开销了。

不过，普通索引即使是等值查询也默认会走 INDEX RANGE SCAN，原因是它的索引值可以重复，胖哥为此增加了一个 ZZ_XIEYU_TABLE2 表，在这个表的 name 字段上创建一个普通索引，此时对 name 字段进行查询时，执行计划如图 6-15 所示。

```
SQL> explain plan for select * from zz_xieyu_table2 where name =02;
已解释:
SQL> select * from table(DBMS_XPLAN.DISPLAY);

PLAN_TABLE_OUTPUT
-----
Plan hash value: 178136204

-----
| Id | Operation | Name | Rows | Bytes | Cost (%CPU)| Time |
-----
| 0 | SELECT STATEMENT | | | | | |
| 1 | TABLE ACCESS BY INDEX ROWID: ZZ_XIEYU_TABLE2 | | 1 | 389 | 2 (<0)| 00:00:01 |
|* 2 | INDEX RANGE SCAN | IDX_ZZ_XIEYU_TABLE2_NAME | 1 | | 1 (<0)| 00:00:01 |
-----

Predicate Information (identified by operation id):
-----
 2 - access("NAME"=02)
```

图 6-15 对一个普通索引字段做 where 条件查询时的执行计划

现在再来说说 IN 的故事。

前文中提到如果 IN 条件是一些固定的值，在值并不多的情况下，则会采用 OR 的方式来执行。现在就来证明这个结论，如图 6-16 所示。

```
SQL> explain plan for select * from zz_xieyu_table1 where id in(01,02,03,04);
已解释:
SQL> select * from table(DBMS_XPLAN.DISPLAY);

PLAN_TABLE_OUTPUT
-----
Plan hash value: 1013867332

-----
| Id | Operation | Name | Rows | Bytes | Cost (%CPU)| Time |
-----
| 0 | SELECT STATEMENT | | 4 | 80 | 6 (<0)| 00:00:01 |
| 1 | INLIST ITERATOR | | | | | |
| 2 | TABLE ACCESS BY INDEX ROWID: ZZ_XIEYU_TABLE1 | | 4 | 80 | 6 (<0)| 00:00:01 |
|* 3 | INDEX UNIQUE SCAN | PK_ZZ_XIEYU_TABLE1 | 4 | | 5 (<0)| 00:00:01 |
-----

Predicate Information (identified by operation id):
-----
 3 - access("ID"=TO_NUMBER(01) OR "ID"=TO_NUMBER(02) OR "ID"=TO_NUMBER(03) OR "ID"=TO_NUMBER(04))
```

图 6-16 Oracle 中的 IN 少量固定的值是通过 OR 去实现

如果此时是 IN 子查询会是什么样的结果呢？就以刚才新创建的 ZZ_XIEYU_TABLE2 表作为子查询的一个条件，在子查询中再通过带索引的 name 字段过滤一次，看看结果会如何，请看图 6-17。

```
SQL> explain plan for select * from zz_xieyu_table1 where id in(select id from ZZ_XIEYU_TABLE2 where name =01);
已解释:
SQL> select * from table(DBMS_XPLAN.DISPLAY);

PLAN_TABLE_OUTPUT
-----
Plan hash value: 70270095

-----
| Id | Operation | Name | Rows | Bytes | Cost (%CPU)| Time |
-----
| 0 | SELECT STATEMENT | | 28 | 1584 | 38 (<0)| 00:00:01 |
| 1 | NESTED LOOPS | | | | | |
| 2 | NESTED LOOPS | | 28 | 1584 | 28 (<0)| 00:00:01 |
| 3 | TABLE ACCESS BY INDEX ROWID: ZZ_XIEYU_TABLE2 | | 28 | 1584 | 8 (<0)| 00:00:01 |
|* 4 | INDEX RANGE SCAN | IDX_ZZ_XIEYU_TABLE2_NAME | 8 | | 1 (<0)| 00:00:01 |
|* 5 | INDEX UNIQUE SCAN | PK_ZZ_XIEYU_TABLE1 | 1 | | 0 (<0)| 00:00:01 |
| 6 | TABLE ACCESS BY INDEX ROWID: ZZ_XIEYU_TABLE1 | | 1 | 20 | 1 (<0)| 00:00:01 |
-----

Predicate Information (identified by operation id):
-----
 4 - access("NAME"=01)
 5 - access("ID"="ID")
```

图 6-17 将一个表的查询作为子查询

这个执行计划会相对复杂一些，依然从缩进由里向外看，最里层是通过 IDX_ZZ_

XIEYU_TABLE2_NAME 索引走了一个 INDEX RANGE SCAN，即走了一个范围索引找到相应的 ROWID，然后通过 TABLE ACCESS FULL BY INDEX ROWID 回表自然是要找 ID 字段。换句话说，即使是主键字段没在索引上，也需要回表来查找。在查找 ID 的过程中同时做的事情是走 PK_ZZ_XIEYU_TABLE1 这个主键索引，并且使用的是 INDEX UNIQUE SCAN 走唯一性索引方式，最后通过这个唯一性索引的 ROWID 回表查询 ZZ_XIEYU_TABLE1 相应的列。

在这个过程中产生了两次 NESTED LOOPS，这就是嵌套循环，所谓嵌套循环就是两层循环结构，在代码中类似于 for(...) { for(...) {...} }，时间复杂度理论上是： $M \times N$ ，那么嵌套循环是不是一种最慢的执行路径呢？

通过图 6-17 可以看出，SQL 执行过程经历了两轮嵌套循环，第一轮嵌套循环的结果是从 ZZ_XIEYU_TABLE2 表中找到的每一个 ID 值，然后逐个通过索引 PK_ZZ_XIEYU_TABLE1 找到 ZZ_XIEYU_TABLE1 表中对应的数据行，说明内层循环并不是一个真正的 for 循环，而是根据主键索引 PK_ZZ_XIEYU 查找数据。换句话说，即使内层的表有可能很大，只要走了索引，它通常就不会太慢，反推外层循环的结果是通过 name 过滤后的结果，通常外层循环的次数不会太多，这个外层循环的结果集更像是一个小表，这也是为什么我们通常希望 SQL 执行过程用小表驱动大表的原因，如果仅仅用嵌套循环是无法解释，我们的真正目的其实是想让大表走索引的查找方式。

既然提到了两个表，就不得不说一下 JOIN，由于 JOIN 的情况非常多，胖哥就不再一一列举了，并且子查询本身就是一种 JOIN 方式，在上面的嵌套循环例子中，如果将这两个表查询换为 JOIN 方式，并且带上 name 字段的过滤，那么执行计划中的顺序会有所变化，但是总体开销应当是一致的，因为始终会走类似的索引。

接下来看一个 HASH JOIN 的例子，如图 6-18 所示。

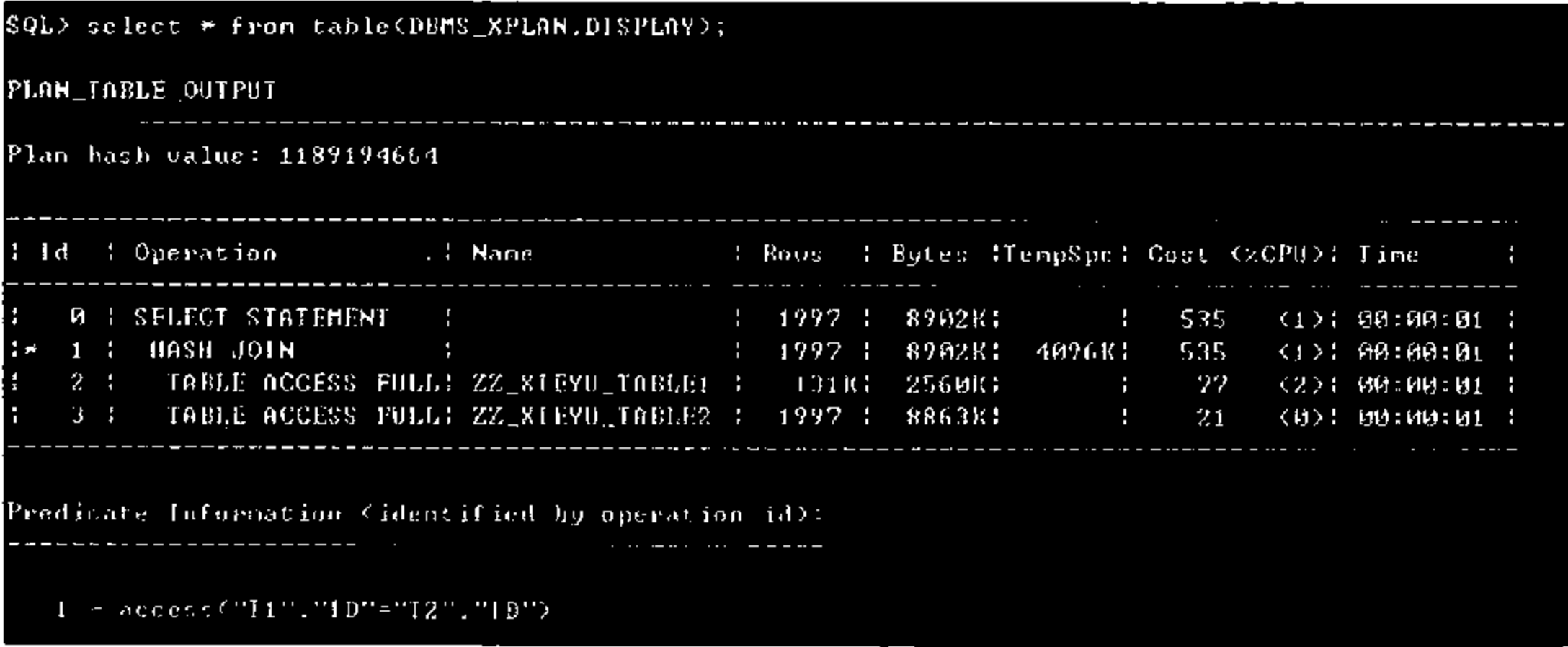


图 6-18 一个简单的 HASH JOIN 例子

HASH JOIN 通常是通过两个表的全表扫描方式来合并的（不过并不代表所有的 HASH JOIN 的来源表都是走全表扫描的），当 Oracle 认为嵌套循环不太合适（例如不论怎么循环差别都不大，这种情况通常发生在多个结果集都不太小但又要做 JOIN 的情况下，或者 Oracle 根本无法判定结果集的大小）的时候，那么这个时候会走 HASH JOIN。

所以 HASH JOIN 自然会提到一些 Hash 算法，而且用在多个结果集都不小的情况下，自然会需要不小的空间来 JOIN，如果我们是程序员会怎么做？自然是从两个表中拖 JOIN 的值放入一个 HASH TABLE 中，如果找到相应的值，则记录对应的 ROWID 或直接将内容记录下来，最终输出结果。

这种 JOIN 方式是 Oracle 所独有的，所以并不是所有的数据库都适用，不过它确实可以帮助用户解决很多常见问题。

回到前面的话题中，字段上有索引并不一定非要走索引，就像前文中提到的 flag 字段只有两个值一样的道理，如果把 flag 字段作为 where 条件，那么它不会走索引，即使是在 JOIN 时也是一样的道理，现在来证明这个结论，怎么证明呢？我们用一个错误的 JOIN 方法来证明，用 ZZ_XIEYU_TABLE1 表的 flag 字段与 ZZ_XIEYU_TABLE2 表的 ID 字段来 JOIN，我们让 ZZ_XIEYU_TABLE2 表走主键索引，也就是说，只会返回很少量的数据，结果请看图 6-19。

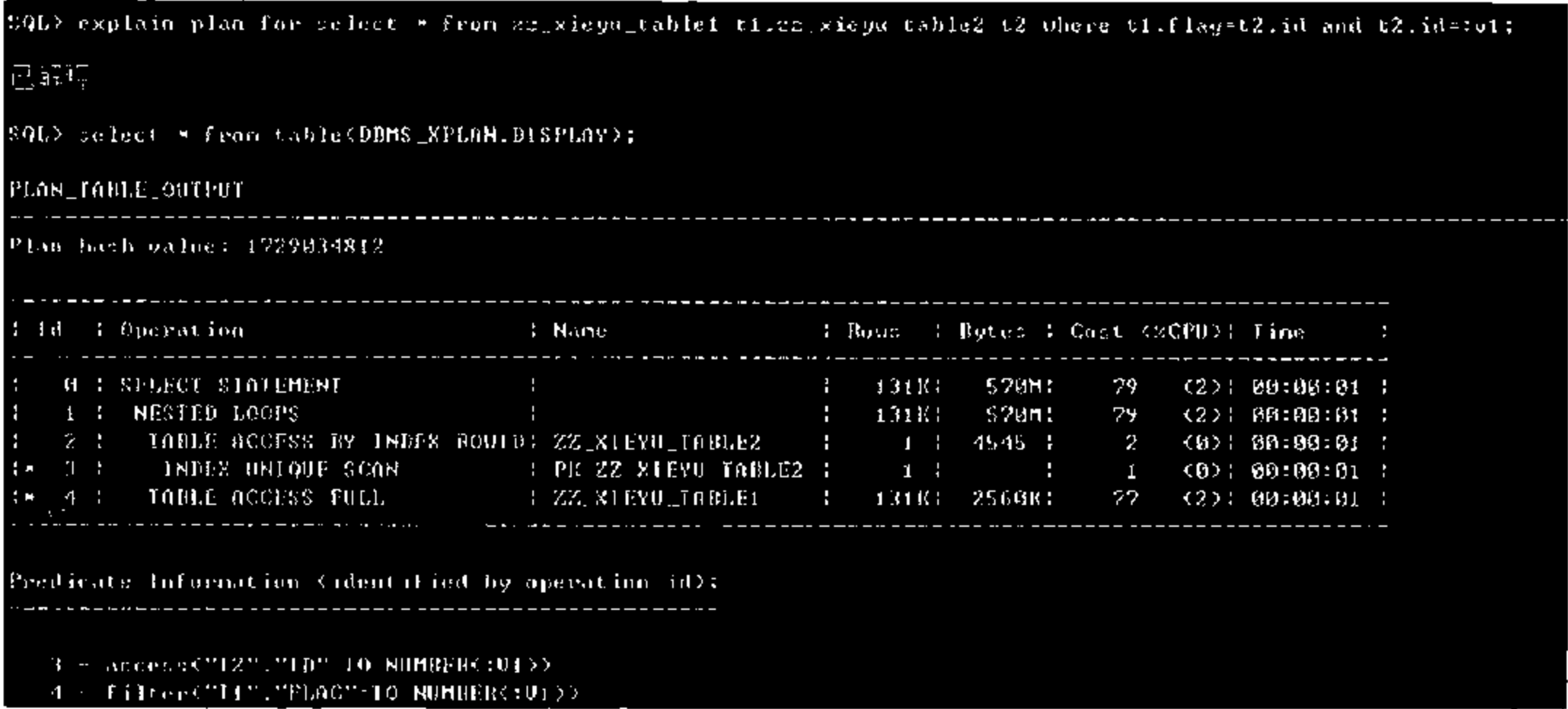


图 6-19 一个表走了唯一性索引，一个表走了全表扫描

这种数据分布导致不走索引的结论在 JOIN 上依然会成立，大家可以依此类推做许多实验，胖哥就不再做一一演示了。另外，在 Oracle 数据库上经常会因为表太大或 JOIN 的关系过于复杂，导致执行计划走错，应该走索引的地方却没有走，因为 Oracle 再智能也会有它做不到的事情（因为执行计划的计算本身也需要开销，随着 SQL 复杂性的增强，数据量变大，它就可能会算错），如果我们知道如何走索引更好，那么可以通过一种 Hint 的方式告诉 Oracle 应该如何走索引。细节请大家自行查阅，本书不再详细讲解。

胖哥一直在说 Oracle，似乎忽略了其他数据库。

当然不是，其实 Oracle 的执行计划是很完善的产品，如果学会了它，其他数据库的查看方式是类似的，下面列举了几个简单的 MySQL 执行计划，如图 6-20 所示。

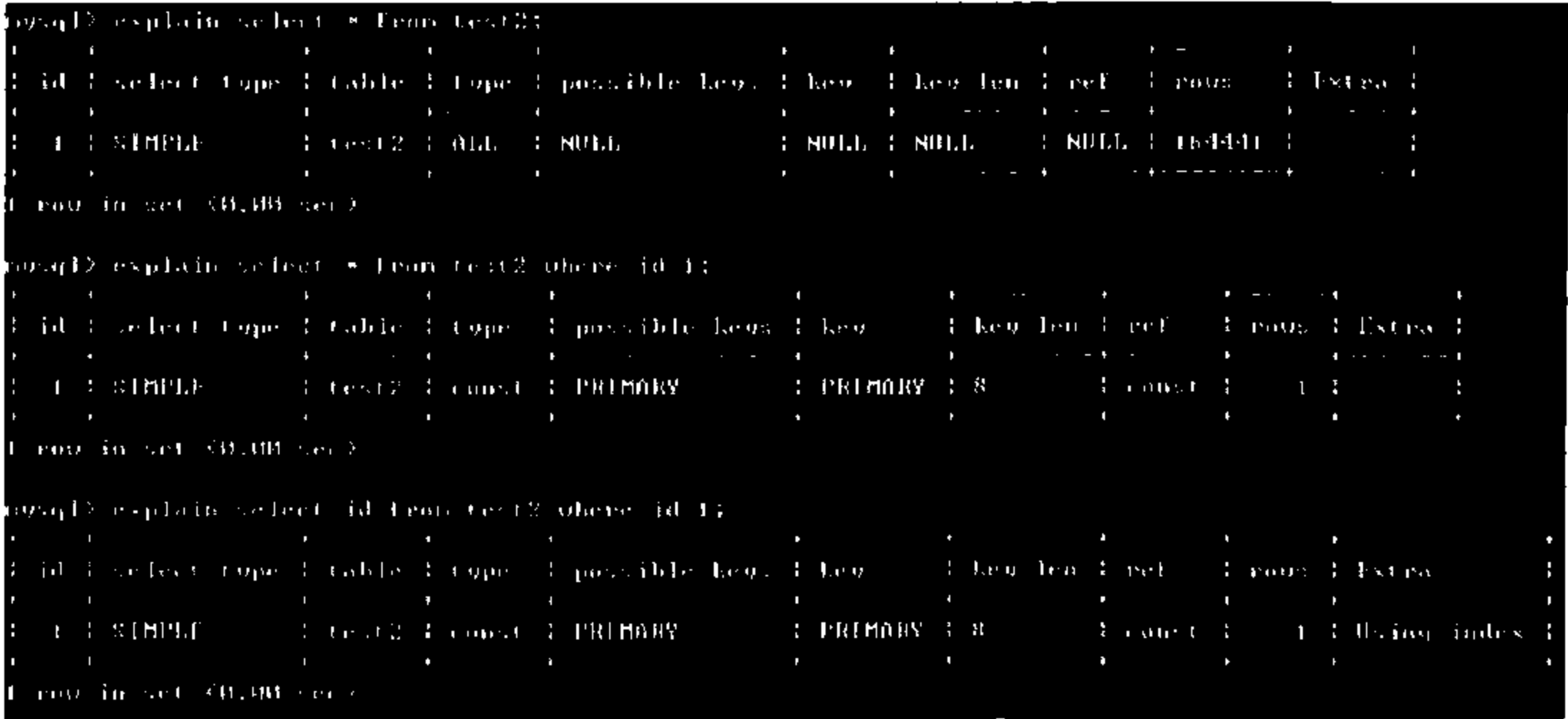


图 6-20 MySQL 的执行计划

MySQL 的执行计划做得相对难看一些，但是也有一定的参考价值，有关更多的语法在实际情况中如何执行，请大家自行测试。大家练习多了，自然而然就会理解索引的道理，

有意识去编写对数据库友好的 SQL 语句，这也是为什么胖哥说一个优秀的程序员应当懂得一些数据库的基本道理。

6.3.3 SQL 逻辑的例子

很多时候依赖于数据库本身做优化不是那么靠谱，而 SQL 的逻辑性是十分重要的，这一点在前文中大家都有所体会。本小节会谈谈到一些和执行计划无关的逻辑，看看这方面知识是否可以让一些问题好解决。

逻辑不仅仅是单纯地对 SQL 的顺序进行修改，它还是一种很神奇的东西，换位思考一下就会发现许多逻辑是可以被合并的，也就是说，这是思维方式的问题。胖哥这样说有点抽象，现在举个例子：有一张学生成绩表，有学生姓名、科目、分数这几个字段，此时要求取出表中所有成绩都大于 80 分的学生名单。



注意 是所有成绩都大于 80 分，不是最高分，也不是平均分。

还有不考虑缺考的同学，如果缺考分数表中也会有记录，那成绩是 0 分，这个在设计层面就可以控制。

此时就有许多思路来解决这个问题，而最直接的思路是找到“所有科目没有小于 80 分的成绩的学生”，按照这个实现思路，SQL 语句应当是先取出小于 80 分成绩的学生名单，然后外部包装一个 SQL 查询所有学生，再使用 not in 或 not exists 的操作达到目的。这种思路最常见，也确实可以解决问题。但是 not 类的操作通常不会走索引，因为这类操作并不是将索引上找到的数据直接进行返回或回表，而是将索引上没有找到的数据索引值回表，这样的方式很多时候不如直接在原表上进行操作。

另一种思路就是按照学生进行 GROUP BY，求出科目的总数作为一个返回值 counter1，然后 sum(case when 成绩 >= 80 then 1 else 0 end) counte2 作为另一个返回值，两个值对比是否一致（可以在 having 中对比出来），如果一致则认为都是大于 80 分的学生名单。此时如果建立合适的索引是可以有效利用的，不过这样的结果集每一行数据都会用来做判定，并分别用于 sum 和 count 运算，我们希望它的开销更小一点。

再拓展一下思路，同样按照学生做 GROUP BY 操作，但是计算每个学生小于 80 分的数量，这样就只需要一个 sum(case when 成绩 < 80 then 1 else 0 end)操作，如果这个数量等于 0，则该学生的所有成绩都是大于 80 分的，将其返回就是正确结果，现在可以将 count 运算去掉了，判定逻辑依然在，能否更加简洁呢？

再拓展一下思路，同样按照学生做 GROUP BY 操作，如果一个学生最低的成绩也大于等于 80 分，那么自然他所有的成绩都会大于等于 80 分，而且这样的方式通常可以更好地利用索引（有可能不需要扫描所有的索引值来做计算，而只需要找到范围的边界即可）。

例子很简单，但是这里说明的是问题有许多解决方法，我们要抽象出来做换位思考，看看有没有可以做逻辑合并的地方。

如果此时不仅仅要求返回学生名单，而且要求返回这些学生的所有成绩信息，在默认情况下也会做一层关联才能得到，这个效率也没有什么问题，如果希望在没有关联的情况下搞定，此时可以利用分析函数来完成。

>> 6.3.4 模型结构设计的优化

一个表结构的设计，不应当是单纯地从对象设计的属性中一一映射而来，有些时候稍微修改一下设计思路，就可以让程序变得简单，而且问题容易解决。下面举个例子来说明一下。

当尝试通过二维表来表达一个树的结构时，通常会用一个 `parent_id` 来代表父亲节点的 ID，这样就能在逻辑上串联起来，表结构如下：

id	name	parent_id
1	xxx	null
2	xxx	1
3	xxx	1
4	xxx	2
5	xxx	2

但是当需要通过某个节点来访问所有子节点或父亲节点时问题就麻烦了，要实现这个功能可能会写很多代码，当然部分数据库提供了递归方法（例如 Oracle 的 `start with connect by prior` 语法），但是这种方法有个毛病，就是在向下递归的过程中，每一层的每一个子节点都将作为单独的条件去检索，在这个时候 Oracle 会采用全表扫描的方式来访问，当树的节点越来越多（例如达到 10 万个节点）时，访问将会变得很慢，数据量继续上升会更慢。向上递归是没有问题的，因为在树结构中父亲节点最多只有一个。

有这样大的一棵树吗？当然有！想想全国地区从省、市、县到区、镇、乡、村、门牌号等是多么大的一棵树。难道这种问题无界吗？当然不是，我们并不一定非要和技术较劲，可以通过设计手段避开一些问题。

我们开始幻想：假如在表中设计一个字段，它能否保存自己所有的父亲节点路径上的 ID 呢？因为这样问题将迎刃而解，于是表结构变成如下状况：

id	name	parent_id	all_parent_id
1	xxx	null	{1}
2	xxx	1	{1}{2}
3	xxx	1	{1}{3}
4	xxx	2	{1}{4}
5	xxx	2	{1}{5}
6	xxx	3	{1}{3}{6}

程序中从创建这一棵树的根节点开始，`all_parent_id` 将作为树节点中的一个属性，根节点的 `all_parent_id` 就是 {id} 本身，其子节点肯定是基于父亲节点创建的，此时创建子节点需要生成的 `all_parent_id`，根据父亲节点的 `all_parent_id`+{自身 id} 便可以得到。在这个问题上，我们可以发现层次不会过多，7、8 层就是十分恐怖的了，所以这个字段宽度也不会太宽，可以建立一个索引。

如果要查询这个节点的所有父亲节点，自然的根据这个字段拆分出 `all_parent_id` 中的数字，根据主键 ID 就能过滤出来，而由于层次不会太多，因此拆分出来的 ID 值也不会太多，所以这里通常都会走索引。

如果要查询这个节点的所有子节点，则将当前节点的 `all_parent_id` 后缀拼接一个 %，使用 `where all_parent_id like ?` 就可以检索出来，很显然是前缀匹配，所以通常会走索引。而我们很少会从根节点去查询下面所有的子节点，而且返回的数据也会有所限制，所以它通常是高效的，而且是十分简单的。

另外，如果按照这个字段排序，它就会按照一种树的层次结构进行展现，而它本身又是在索引上，所以是有序的结构，效率上也不存在太大的问题。

总而言之，有些时候实际场景可能会与我们的这种假设有所区别，此时希望大家加以变通来完成。

编码方式的例子远不止如此，为此我们再讲讲“编码规则”，它与上述案例类似，但是它不需要额外的字段来存储一个 `all_parent_id`，因为它的 ID 上本身就有父亲节点的信息。

例如车牌号、身份证号都会有类似的设计，它们每个部分的数据都标识一个维度（例如身份证号码前 6 位代表地址，中间 8 位代表出生年月日等），如果将这个维度变换为层次，就可以设计出树的结构。假如一个数据分解为 3 个层次，第一个层次最多几十条数据，那么只需要 2 位十进制数字即可表达，后面两层中每一层需要表达几万种可能性，那么每一层需要 5 位数字表达，那么自然的可以用定长 12 个十进制位来表达即可。此时的编码就像这样：

```
02 00001 00131
11 00033 89121
45 13312 13213
...
```



注意 数据之间的空格只是为了方便大家看数据，实际存储进去的数据是没有空格的。

这种数据如果用字符串来存放，那么使用方式就像 `all_parent_id` 一样（注意传入的参数隐式转换导致不走索引），如果使用数字存储，第一层次的 01~09 这类数字在程序中使用时需要稍加处理，否则不足 12 位，这一层也可以考虑从 10 开始。

如果用数字存储，那么使用方式可以做一些转变，假如要查询出以 02 为前缀的所有编码，则使用 `between 20000000000 and 29999999999`，自然的如果要求以 11 00123 为前缀的所有编码，就使用 `between 110012300000 and 110012399999` 即可。

从这些道理上大家是否体会到设计给我们带来的好处，同时还有一种真正玩转数字的享受，在实际的业务场景中通过设计手段解决问题的例子非常多。这些内容可以说与技术没有多大关系，但是确实很重要，与我们的思维习惯有关系，在解决问题的过程中一定不要仅仅局限于一种思维方式。我们要多去看看别人是如何解决问题的，多学习别人的方法和思路，而不仅仅是一种语法和技术，这些思维方式其实很多时候来源于生活场景，需要我们多思考。

总而言之，如果在一个系统中做一些常规的业务操作，需要十分复杂的操作与运算才能达到目的，而且这些复杂的逻辑运算是极度消耗性能的，那么可以考虑，是不是可以改一改设计。

➤➤ 6.3.5 临时表

临时表很少会用到，但是有些时候也挺好用的，如果要用它自然首先要了解什么是临时表。

临时表通常是会话级别的表，每个会话写入临时表的数据都是私有的，其他会话看不到这些数据，这样我们可以利用临时表作为会话的中间存储区域来更好地解决一些问题。

而且临时表的写入通常日志很少或几乎没有日志，比起直接使用物理表来做中间存储区域会更加高效一些。

临时表在不同的数据库中实现方式不同，而且创建的语法也不尽相同，大家只需要知道它的基本概念和使用场景，在实际应用中看看语法便知道真相了。

胖哥曾经看到过一个十分复杂的 SQL 查询，它的子查询有多层，而且子查询内部还有很多表的 JOIN，导致这个 SQL 查询的执行时间接近 100s，在数据库中其实有这么长时间的 SQL，但是这种 SQL 通常是用来做数据转换过滤形成定期报表的，而不是一个普通的 SQL 查询。

这种操作是由于表结构设计过分细化，导致业务逻辑很不好实现，但是当时这个业务系统已经处于长时间的运行状态，无法修改表结构的设计了。那么只有在 SQL 上做文章了，首先将 IN 改成 exists，发现效果不明显，执行计划有一些细微上的变化，快了一些，但是依然是几十秒才能得到结果。

经过业务确认，子查询内部返回的数据虽然是许多表的 JOIN 操作，但是会走索引，而且只返回几十条数据。而当时的数据库的执行计划并不知道这个 SQL 只返回几十条数据，因为内部的 JOIN 太过复杂，即使要在 Oracle 这类数据库中指定 Hint 都是十分麻烦的。此时想到的方法就是利用临时表，分解为两个动作，其中第一个动作是将内部查询的结果放入临时表，第二个动作是利用临时表作为驱动表与外部做嵌套循环连接，这样就达到目的了，优化后的 SQL 语句不到 100ms 便返回了数据。

从这一点我们可以看出，自定义的执行计划很多时候会更加有效（因为我们很了解业务背景，所以很清楚数据的规则，但是数据库是一种软件，它未必完全清楚，尤其是将一些经过多表复杂 JOIN 后的返回结果作为子查询的条件很容易让数据库“犯糊涂”），而执行计划在整个执行效率中显得十分重要。另外，临时表在关键时刻杀出，可以轻松搞定许多问题。大家不用过分担心临时表的性能，因为在很多数据库中，它所占用的空间与表之间 JOIN 时所用的临时空间是类似的地方。

►► 6.3.6 分页知识补充

前文从技术上提到了分页，无论怎么分页，下一次提取数据时都要从新数行数，即使我们清楚下一次应该从哪个位置开始数，数据库也依然不会搭理我们。但如果从逻辑的角度去理解，则可能会想到一些其他的思路和方法。

退一步讲，数据库分页本身是基于行号的，在不断翻页的过程中如果有业务数据写入是很正常的，所以在翻页的过程中可能会看到前一页的数据或漏掉几条数据，总而言之，翻页本身不是一种强数据一致性的操作。为此，我们试想一下，可否在这个思路做一些优化呢？例如尝试将当前结果集的一些关键字记录下来（如 ID），下一次从这个位置开始找后续的 10 条、20 条数据，其实这样的做法并不影响全局，但由于它每次都是根据条件查询 10~20 条数据的，因此在一定程度上解决了向后翻页过程中越来越慢的问题。

另外考虑每当翻页时，不止获取一页的数据，而是获取最近 5 页、10 页的数据并将其放在缓存中保持一个经验时间长度，以页号为单位存放，翻阅最近几页时，不论是向前还是向后翻阅都可以看缓存中是否有该对象。这样的思路有点类似于顺序读/写，每次因为数数变得慢，但是数据数到那个位置开始读取数据，多一点也没多大关系，只是会占用网络带宽，但如果能有效地提升命中率，那么网络带宽其实总体上是减少的。有人会说这样的数据不能完全与数据库保持一致，其实胖哥想说的是分页的数据本身就没有和数据库完全

一致。另外，如果已经读取出来的数据发生修改，则可以通过其他的方法来进行处理，能否达到一致也是由处理方法，以及对一致性的要求决定的。

►► 6.3.7 计算 count 值

在 SQL 语句中，我们都用 count 来计算结果，对于超大的表做全表统计是很恐怖的一件事情（在索引字段上过滤后统计是另一回事），所以对于存储海量数据的表，通常有些时候建议不要做这样的操作，或通过其他的手段来做。

有的小伙伴提出用触发器计数的方式来做，不过这样的方式往往在设计之初就要考虑到，并且效率自然也会下降一点。这也仅仅能统计总量，当 SQL 带有 where 条件时，这些都没有用了。有的小伙伴会用延迟统计的方式，甚至交给另一个系统来做。有的小伙伴认为在很多时候没有必要非要求总数，所以会采用与其他系统结合处理的方式来完成，例如通过数据仓库来做一些事情。

其实我们可以想一想，如果真的要对一个超大的表求总数，需要求出十分精确的值吗？即使你求出了十分精确的值，也可能下一秒的值就发生了很大的变化。很多时候我们要一个超大表的行数，通常是要一个大概，能标志表在行数上的增长情况即可，而不是一个十分精确的值。

如果不是为了求一个精确值，从何下手呢？数据行数是与表的容量大小成正比的，或许我们可以从表的容量变化上来思考，每个表的结构不同，因此无法根据容量直接判定行数。不过，我们可以换一个思路，假如我们知道一个表中 1M、10M 的容量有多少行，那么根据表的总容量便可以得出一个大概行数。例如：有的数据库在这个时候会离散地获取数据库中的几十个块来计算行数，再根据块数进行综合计算，由于块数通常在分配空间时就会记录下来，因此这个数字是真实的。通过这种方式得到的行数其实也是很接近真实值的，但是代价却很低。

胖哥为何要说这个道理呢？因为这是一种设计思路，一种变通的设计思路，很多时候我们未必要 100% 保证精确，因为在需求层面没有那个必要，此时可能换个角度看问题，就可以把问题简单化。

另外，这种方式充分地利用了数据划分为块、区这些概念，或者说如果没有这些概念，数据单纯以行为单位存储在数据库中，那么就很难找到其他下手的地方。从设计的角度来讲，区、块的设计之初可能大家都没想到它有多大的价值，但在使用的过程中可以将它变化万千地使用，因此从程序员的角度来看，一个好的设计可以给未来带来巨大的扩展性，一个好的设计未必是当前价值最高的，但是它一定具有很好的伸缩性。

►► 6.3.8 分布式事务探讨

胖哥认为，到目前为止，分布式事务几乎是一个无解的话题。

有的小伙伴会反问：谁说无解，那些大型系统肯定是有办法解决的呀？JTA 是吃素的啊？还有网上公开的各种算法，以及大名鼎鼎的 Spanner 不都是用来解决这些问题的吗？

对于这些反问，胖哥只能说，它们能在某些情况下尽量做到一致性，但是在真正的分布式绝对一致性上几乎是无解的。例如 JTA 通过 prepare、recover 等机制尝试达到预提交、数据库恢复的能力，但是提交动作毕竟需要有一个顺序，在面对多个数据库提交的过程中，无法 100% 保证绝对没有问题，尤其是在大型分布式系统下，经常会遇到数据库切换、宕机等问题，若某个数据库失败，要将其他 commit 成功的数据库恢复到写入数据之前，此时就

不是 rollback 了，而是 recover 数据库。数据库未必会支持这样的动作，即使支持也通常会基于 SQL 来恢复（因为修改是基于行的，恢复就是生成反向 SQL），如果是 SQL，时间点已经延迟了，也就是外界已经看到的数据又要被修改回去，在这个时间差之间可能还会有其他的事务进来修改这些数据，此时已经算不上分布式一致了，至于 JTA 的性能如何胖哥在这里就不再评论了。同样的，许多公开的算法表面上很通顺，与实际的网络通信、数据处理联系起来就会有许多是行不通的。至于 Spanner，胖哥的理解是它更多的借鉴了数据库的版本号原理（例如 Oracle 的 SCN 号），将它的实现架设在网络上，形成服务体系，并且通过冗余方式来保证更高的可靠性。似乎一个完美的 RDBMS 在网络上实现了，不过仔细想一想，它毕竟不是一台机器，它需要十分良好的网络环境，网络中的局部宕机也是十分正常的，虽然它通过某些冗余方式能保证高可靠性，但依然不能说完全保证，只能说它是一套十分成体系的分布式事务解决方案，胖哥相信未来会有更好的方式。

通常在互联网公司流传一个“最终一致性”的说法，因为很多时候数据缺失没有必要完全一致，或者说最后能达到一致即可，并且通过实践发现，发生问题的概率非常低，但是效率提升明显，因此它们会选择这样的方式。不过，并不是所有的最终一致性的做法都是一样的，需要因地制宜，根据一致性要求来做方案。我们举几个例子来说明。

例子 1：异地数据

某数据库，除数据中心存放外，在异地会有局部数据的备份，程序会根据数据规则访问数据中心或异地备份的数据。数据中心和异地备份肯定存在一定延迟，且受到网络的限制，因此它们任意一边写入，都可能导致不同地区的人访问到的结果是不同的。在某些时候或某种容忍度下，我们可以接受这样的情况，或者在架构层面可以做到即使不一致，也几乎不太可能会影响到业务。

例子 2：经典的金融转账

银行之间转账或三方支付平台参与其中，我们需要解决的问题是扣款和入账需要绝对一致，且扣款不能扣除负数。

如果是同行转账或某三方支付机构用户之间转账，则通常是在当前数据库中操作的，问题会简单点吗？未必，数据量庞大，不同用户的账户信息可能会分布在不同的分库分表下面。这样扣款、入账很可能在两个数据库中完成，自然就面临分布式一致性问题，但这样的问题是必须要解决的。可以换个思路，在扣款方的库内部有一个记录日志的表，记录谁向谁转账多少资金，由于在同一个库中扣款日志写入与扣款动作是可以发生在一个事务内的，而目标账户将根据日志信息进行入账，若入账失败则可以选择几次重试或交给后台的异步系统来重试。通常我们乐观地认为在绝大部分情况下不会发生重试，即使重试也不会重试太多次，除非数据库本身的体系架构方案做得与实际偏差太大。但是这样问题通常不会反过来处理，即收款方先入账再扣款的动作，因为在这个过程中可能会出现收款方入账时付款方余额是足够的，但由于某种系统原因导致扣款延迟，中途发生了其他的交易，导致余额不足，但收款方已经得到了资金。跨行转账也是类似的，只是将目标转换为一个目标收款银行接口而已，双方约定各种资金交互和状态细节，就可以达到最终一致的目的。

例子 3：冻结资金

在转账过程中，扣款方的钱没了，但是对方还没收到，双方都有些着急，有可能需要

等较长的时间，如果经过较长的时间依然没有结果，此时要让客户来找人，则可能会彻底傻眼。因为这个转账过程可能经过许多支付公司、银联、银行系统，当客户咨询这个支付公司时则会推到银联，银联再推到银行系统，客户在这个里面似乎成了定位问题的“总架构师”。在技术上需要双方的配合，技术不可抗拒，所以在体验上可能会有一些新思路，那就是“冻结资金”的方式。

付款方付款后，以冻结资金及明细的方式存在，在没有达到某一个步骤前，用户可以取消操作，这样用户可以看到自己的资金明细，也可以从心里上认为资金是更为安全的，而技术层面需要更为详细的资金到账的确认机制。

冻结资金用户可以看到但不可以使用，即用户在使用前需要通过未被冻结的资金来计算余额是否足够，至于实现方式有很多，胖哥不再多说。

►► 6.3.9 其他

本节就前文中谈到的执行计划再细化说明一点，执行计划并不能代表一切，通过它所能看出的是 SQL 总体的执行路径，也包含了可能发生的各种读/写次数，许多基于动态参数的执行计划只是一种预算（例如预编译的参数是执行时才知道的），在实际的场景中可能会存在数据热点分布的问题，导致某些执行计划走错。

在执行计划中总是希望：

- ◎ 在 JOIN 表以前，需要被 JOIN 的表能先过滤掉大部分数据。
- ◎ 在 JOIN 时，小表对应大表总是希望小表驱动大表，做某种嵌套循环，如果是有序的，则走有序的嵌套循环，效率会更高一些。
- ◎ 在 JOIN 时，如果多个结果集大小类似，但是都不算太小（例如每个结果集都在 1~40 万之间），那么如果多个结果集用嵌套循环连接时，就会匹配几亿到几百亿次，即使内层循环走索引，外层也会循环几万次到几十万次。显然这种方式并不是我们想要的，此时我们希望换一个思路，那就是 HASH JOIN，它是通过 HASH 方式 JOIN 两个表的数据。这是如何实现的呢？从程序员的角度来思考，如果我们自己来写这个逻辑或许是这样的：建立一个 HashMap 结构，Key 为 JOIN 的条件，Value 是 JOIN 的“多个表”要提取的行对象或行的物理标识符，每个表的对象内部可能存放一条或多条数据，先将一个表的 Key 放入 HashMap 中，然后用另一个表的关联条件到 HashMap 中查找，若找到相应的 Key 则标记，若未找到则丢弃，最终未标记的则是双方未关联上的，可以根据连接方式选择是否返回这部分数据。

至于到底哪一种方法更好，这根据具体情况而定。例如将一个 50 万数据的表和一个 1 亿数据的表 JOIN 后求 count，如果通过 HASH JOIN 来连接，内存是放不下的，就会放在磁盘上来做，那么在磁盘上来做这种 HASH JOIN 将会非常慢。此时我们会尝试用嵌套循环的方式来做，逐步通过 50 万数据的表来驱动具有 1 亿数据的表，让其在走索引的情况下提取 count 值，最终将这些 count 值通过 sum 方式合并起来得出自己想要的值，因为内存中的数据不会太多，就算被放在磁盘上，最后一次 sum 也可以被批量加载出来。

抛开这个问题，它也未必代表全部，当我们遇上了函数（尤其是逐行函数）时，SELECT 返回结果中的某个字段是通过子查询获取的，可能它的开销是隐藏的。

例子 1：

```
select fun1(a), fun2(b) ...  
from table;
```


例子 2:

```
select a,b
      (select a from table2 t2 where t2.xx=t1.xx) as t2_a,
      (select a from table3 t3 where t3.xx=t1.xx) as t3_a
from table t1;
```

在这两个例子中，有可能在执行计划中无法完全看清楚所有的开销，至少第 1 个例子通常在执行路径上看不到函数的处理，因为函数转换通常不属于执行计划的范畴。这种执行方式表面上看没有什么问题，但是用得不太好就可能有问题。

当这两个例子的查询用于分页，并且包含了 order by 时，这些语句将作为外部的返回结果，所以必须要计算出结果，也就意味着必须计算出每一行数据相应的结果，这样就增加了许多额外的开销（而且返回结果的速度在一定程度上取决于所使用的函数本身内在代码的处理速度）。其实外部就只需要一页的数据，理想的是对这一页中每一行数据做转换就可以了，因此我们的建议是，将这种结果集数据的转换操作放在 SQL 的最外层来处理，而不是最内层，这样就可以对尽量少的结果集做函数转换。

许多框架提供的自动分页技术（例如 iBatis），一般是“假”分页，它会将你的 SQL 交给数据库执行得到一个 ResultSet 对象，然后通过 ResultSet 的 absolute(int rows) 方法来实现行数的偏移（JDBC 的 API）。这个在数据库端很可能已经由函数计算每一行的结果，因此这种“假”分页会带来许多隐藏的开销，这一点请大家注意。

由于这些内容会不断细化，而且需要更为具体的场景来说明问题，因此需要你投入大量的精力。

由于本书不是讲解数据库的重点，所以不会一一说明数据库各种细节，即使从程序员角度看数据库也还有许多知识。比如分区表、各种函数、物化视图、大字段等可能无时无刻不影响我们的工作。另外，还有一些数据库支持 Hash 索引机制，它与 B+树的结构又有何共通之处呢？在 OLAP 应用中，如果要读取大量的数据，每次 SQL 几乎都是全表扫描，那么全表扫描又有何可优化之处呢？这些问题都是需要我们去思考和解决的。本书后文中还会提到一些相关的问题和知识，但无法一一列举这些问题的解决方案。

数据库的基本思想可以扩展到分布式系统存储的设计思路，扩展到写代码的思路，扩展到编写优秀 SQL 的思路，用在搜索的一些原理理解上等。简单地说，我们不要仅仅为了数据库而去学习数据库，因为我们是开发者，开发者应当换一个角度来理解数据库本身。

6.4 学会最基本的性能诊断

6.4.1 进入云数据库时代

前文中提到了自主化，并提到了云数据库的广泛推广，当一个云数据库交予我们时，我们便拥有了该数据库绝对的所有权，因此许多常规的问题需要我们自己来解决。要解决问题，除了会优化代码，搞懂数据库原理外，还得学会看懂一些数据库的性能指标，因为一个系统开发出来肯定是有优化空间的，这些优化的基础很大成分来自于性能指标的数据。

专业的云数据库通常会为相应的用户提供一些数据库的诊断报告（如果没有，那就只是在卖硬件，而并非云服务，因为云数据库服务应当包含专业 DBA 的经验抽象，帮助用户来解决问题），大家可以每天登录上来看看各项数据库的指标是否正常，如果不正常是哪里

不正常。下面我们就梳理下可以关注的信息。

6.4.2 从程序员角度关注的数据库诊断信息

IOPS: IOPS 在前文中提到过，即磁盘每秒的访问情况，一般云数据库售卖时会有上限告知，是否正常一看便知。

磁盘使用情况: 也就是数据、日志等信息的存放空间，此信息也有上限，所以一看便知是否需要扩容。

CPU: CPU 通常不设立上限，但是作为机器的使用者需要知道 CPU 的使用百分比，若过大肯定是有问题的。

连接数: 也称为会话数 (Session)，这是数据库当前存在的连接个数，云数据库通常也会告知最大值。有些连接是空闲的，有些连接是活跃的，即正在执行 SQL 语句，参考方式是一样要使用所有的连接来对比，因为即使连接是空闲的也会占用连接数。

网络输入/输出: 网络本身的输入/输出使用过大也可能会导致性能变差 (例如大量的数据导出将网络出口占满)，所以在这方面也需要关注数据的趋势。

慢 SQL 语句: 一些云数据库厂商会在后台采集慢 SQL，并将其按照某些时间点汇总，这些慢 SQL 可能是单个执行时间很长的，也可能是单位时间内执行次数非常多导致总时间占比很大的。对于这样的 SQL 语句，有些时候只需要做一些非常小的修改，或许就可以给系统的性能提升带来许多好处，而不至于我们盲目地将大量时间放在优化一些执行次数很少且本身很快的 SQL 上。不过需要注意的是，某些慢 SQL 未必是业务代码引起的，或许仅仅是人为发起的统计操作，处理方式就需要根据实际情况而定了。

QPS/TPS: 即每秒的读/写次数，这个次数在一定程度上可以反映一些问题，也就是数据库每秒可以承载的 SQL 执行量。但它未必能完全说明问题，因为对于数据库的 QPS、TPS，大多数时候的解释为每秒读/写 SQL 执行次数，这样的说法对于不同的系统评判标准是完全不同的，有些系统对于数据库的操作通常是简单的单行操作或非常少的几行操作，某些系统读和写一次或许会操作几百行甚至更多的数据。另外，在同样的系统环境下，它与 SQL 语句的执行效率有密切关系。因此，这个数据的参考价值并不是很大，云数据库也不会承诺你能达到多大的 QPS 或 TPS，这完全取决于你的应用场景。

不过，我们很多时候可以通过这个信息配合 SQL 执行效率来推算连接数的合适配置。因为连接数的配置需要“合适的”数量，并不是越多越好，更不是将云数据库售卖的连接数用满为止，如果是配置，即使大量的连接没有使用，也会造成应用和数据库双方的开销更多，并且许多连接池会定期检查所有的连接是否有效，监测方式通常是使用相应的连接发起一个十分简单的 SQL 语句操作，由于每个连接都要监测，如果配置大量的连接，程序中会定时发起大量的 SQL 语句操作，虽然它们都是简单的 SQL，但是同样会消耗资源。

其他一些信息: 上述信息对于开发人员来讲，只需要稍微深入就可以玩得很顺，而且也可以通过扩容或优化代码来完成。而更为专业的问题就需要 DBA 来完成，例如，在存储引擎级别的各项读/写频率、内存使用情况、内存命中率、Load、IO 等待、锁等待、主备延迟等信息，同时也需要思考如何让许多问题的处理自动化，让使用者更加了解数据库。

6.5 数风流存储，还看今朝

今朝 RDBMS 依然雄霸天下，不过以 NoSQL 为代表的新思想不断冲击 RDBMS 的“盟

主地位”，NoSQL 希望将某些强耦合的地方打开，突破数据库本身体系的界限，解决更庞大的数据存储和并发问题，或者希望将原本需要编写许多代码完成多种存储配合操作才能达到的目的在一个服务体系中完成。

在 2011 年的中国，NoSQL 几乎是一个胚胎，活跃性强，大家争相热捧，大多数还是以解决某些特定问题而存在，维护也比较麻烦，尚未形成体系，不过它确实能够解决许多特定场景的问题，也可以在某些 NoSQL 产品上做二次开发来不断满足自己的需求。相比数据库本身而言，可以说“姜还是老的辣”，它毕竟经历了数十年的洗礼，在许多问题上有成熟的思路来处理。经过近 3 年的发展，NoSQL 终于开始有点要落地的感觉，许多 NoSQL 产品开始逐步向 SQL 走，并且在向事务一致性靠拢，向通用化靠拢，并且它们的架构经过不断演变也越来越灵活，适应越来越多的场景，通常性能也更好，细节上有更多的选择，相信 NoSQL 会逐步拥有不少的市场（尤其是在互联网领域）。

可见，NoSQL 正在飞速的发展中，许多 NoSQL 产品也在开始借鉴数据库已经存在的问题处理方案（例如前文中提到的 Spanner 借鉴的版本号原理），而数据库也在借鉴一些 NoSQL 的思想。在未来的大数据时代中，两者将可能继续共存，也有可能是其中之一来引领潮流、“独霸武林”，未来的市场需求和它们的发展思路将决定未来，让我们拭目以待。

NoSQL 和 RDBMS 都在不断相互学习，说明它们有着许多值得对方学习的地方。因此，我们不要仅仅因为它是最新的，就认为它是最好的，旧的东西就全盘否认。即使认为它是好东西，也要思考它是在什么思想的基础上建立起来的，因为现代的知识不是完全凭空想出来的，它们都是在老技术无法满足或很难满足一些新需求时，人们通过思考得出的一些新方法。只有这样去不断思考，才能将许多思想体系贯通起来，从而在自己解决问题时就会考虑到许多已经存在的方法。换句话说，我们不能老是在遇到一个问题时解决一个问题，可能带来了更多的问题（如果业务背景发生重大变化，出现新问题是正常的）。

胖哥宣布：特种兵第一阶段魔鬼训练营正式结束，开心吧！大家也应当发现这一篇内容占据了本书上册的大部分篇幅，因为胖哥认为这些内容很重要，如果你能坚持看到这里，那么后面的内容将会相对比较轻松。

但是这并不代表结束，就算是基础也并不代表结束了，本书只是抛砖引玉，希望大家可以根据这些内容不断细化自己的知识和思想。

接下来的第二篇，将从训练营拉到一些演练场上去看别人的实战，看实战中的技巧、方法，看实战中的残酷。就是看别人的源码，很多人不知道为什么要看，也许你看得头都大了，那么下一篇的几章内容也许对你有帮助。

第2篇 源码篇

有人“为了读取源码而读取源码”，有人为了“追风”而读取源码，有人为了研究本质而读取源码，有人为了研究问题而读取源码，有人为了兴趣而阅读源码——如果你曾经读取过源码，你属于哪一种呢？

初学者看源码，通常会觉得“费劲、头大，看完一段时间就会丢掉”，尤其是对于很多不了解设计思想的情况，但了解设计思想又是一个漫长的过程，大师们留下的优秀源码其设计思想也是较为抽象的。对于初学者并不知道为什么要这样写代码，自然会越看越费劲，即使看源码记住了一些东西也是死记硬背下来的。

看源码需要经历一个过程，往往是一个迭代的过程，胖哥对某些优秀源码也不知道阅读了多少遍，每一次看完相关的代码都会有新的感受。胖哥的窍门是每一次阅读完相应的源码后，都会体会下源码的一些思路可能会用在哪里，尽自己的努力去理解，逐渐地当接触的场景更多时，阅读源码会变成一种理解、一种快乐。

能读懂源码需要理解其设计思想，接触一些自己没有碰过的技术。有些纠结的同学就会反问：“既然要懂设计思想才能读懂，但设计思想很多又来源于对好的源码的阅读，以及大量的编码设计的总结，这不是一个死结吗？”这就是学习思想上的一个死嵌套，那么初学者如何成长，我们可以先来了解一些方法。

我是小鸟，首先要放平心态，落实于当下，不要怕痛苦，也不用想一次性把所有的代码看懂，但是需要“迈出第一步”，否则永远也走不出这个先与后的逻辑圈子，成长快的人不会将太多的精力放在这个问题上纠结，他们更多的是在默默地付出努力。

在面对阅读困难时要想到“一回生、二回熟”，看多了自然就会养成看很多复杂源码的习惯，就像一个婴儿需要去适应这个世界，一个大学毕业生需要去适应这个复杂的社会，进入一家新公司需要去熟悉新的人文环境和技术，第一次写代码时要突破心理的那道障碍，等等。很多时候是环境逼着自己去改变，改变总是痛苦的，但是只要愿意去改变，就始终能改变自己。

在看源码的过程中，会经历一段地狱式的思想斗争，如果能熬过来，通

常会自然而然地开始学会从技术的优劣势、功能、设计思想等角度来参看源码。对设计思想的理解不仅仅来自于源码,也同时来源于自身工作的许多体会和总结,看源码是一种方式,它们相互依赖存在。

到了一定的阶段,胖哥认为应当逐步强化一种观念:“要去读一段自己知道功能的代码”,或者“要去读一段自己很想知道内部如何处理的代码”,也许这段代码不足以将整个体系串通起来,只是对一个点的深入挖掘,甚至就是一个 if else 的逻辑判定,但是它能帮自己真正理解问题的本质,这种驱动可以让自己很有“欲望”去知道一些事务的内在,逐步地阅读源码将会事半功倍。

有些时候小伙伴会通过测试从某些角度验证一些结论,但是不足以说明事务的本质(就像在第 1 篇中讲解的许多原理那样,它们都与实际场景息息相关),因为“错误+错误也可能等于正确”,所以会经常出现各种场景结果不一样的情况。空闲时去读一读源码,论证结论,这个过程再配合更多的测试用例可能会帮助你“知根知底”。另外,每天空闲时可以去接触一些技术点的源码,这些技术点是自己感兴趣的,有一天你会“自然地有兴趣”来串通这些点,在这个过程中你会积累很多设计思想和方法。

胖哥认为,看源码头痛的原因之一是跟踪代码是一种面向过程的思路,而 Java 是面向对象设计的。一些小伙伴又会想:“是不是必须要学完设计模式再来看这部分呢?”如果你愿意当然可以,但是如果先看看,一个土办法就是大家看到自己感兴趣的类时,将它先记录下来,花 2 分钟左右时间,简单理清楚这个类的大致继承关系,理清楚大致的属性和方法有哪些,这样有了一个总体的脉络就好了,细节上的方法如何实现可以空闲时再梳理。

看源码时,不要想一口气将一段没有见过的源码完全看懂,需要采用以点带面逐步深入的方式,这至少比较适合于一些刚开始看源码的朋友,更加适合于特别想去看一些比较复杂的源码的朋友。

上面提到的或许很抽象,下面胖哥讲一个简单的例子。比如我们要去看 Tomcat 中是如何处理“字符集”的,那么是否需要将 Tomcat 从启动、Socket 创建、线程池管理到各种流的转换等源码通看一次呢?当然不是,当对 Tomcat 本身的源码还不是那么熟悉时,要学会用“逆向思维”,也是“换位思考”的一种体现方式,此时可以直接从 `HttpServletRequest` 类的 `getParameter()` 方法开始去看(有点像暴力破解)。

不过这样看也许还是很费劲,因为在看源码的过程中会涉及很多非常复杂的逻辑处理,导致看源码十分痛苦,所以胖哥提醒大家还需要注意:现在要去看的是字符集部分的源码,虽然中间还会有许多其他的逻辑,但是大多情况下只需要关心字符集处理部分的源码,而不需要过多关注其他的源码,至少不需要关注那些一时半会看不懂的源码,这样看源码又会简单许多。

这样看源码是不是会让自己错过很多知识呢?

答:胖哥认为不会,我们看源码其实并不是要学习它的体系,而是要知道一些内在和思想,用以更好地工作,当很多知识点铺开,就会游刃有余,正所谓“船到桥头自然直”。

从代码调用路径上讲也是类似的, 当一个框架平台很完善时, 会有许多配置项, 每个参数配置项都会对应到相应的代码执行路径上, 自然执行路径就会非常多。因此看源码时需要注意: “并非要跟踪所有的路径”。很多时候只需要关心代码中目前调用的路径即可, 某些中间路径可以忽略。

要找到当前代码的调用路径, 一个简单的方法是使用 `Thread.currentThread().getStackTrace()` 来得到调用路径 (在第5章中已经提到), 也可以直接通过 `new Exception().printStackTrace()` 打印出来。换句话说, 刚开始看源码时可以暴力地将中间步骤省掉, 先通过宏观的方式输出代码调用路径, 这样便知道了大概意思, 接下来开始对自己需要关注的内容进行“踩点”, 如何踩点? 通过上面方式输出的代码调用路径中包含了具体的类和具体的方法, 以及代码的行号, 在对应的方法内部, 读懂大概意思后, 就寻找想要的内容即可。

很多人会提出这样的问题: 读源码有什么用?

- ◎ 可以改变一个技术人员看问题的角度。例如, 当你再去做某些测试时, 不会再根据某些一两次的测试用例而得到一个结论, 因为你知道某些测试用例根本是站不住脚的测试方式。
- ◎ 当需要解决某些问题时, 改掉一个参数解决一个问题, 但是又出现新的问题, 不断反反复复, 此时通过看源码, 也许你可以找到根治的方法。
- ◎ 对于一些开源组件官方的各类参数, 当参数和参数之间有冲突时 (例如一个参数是设置比例, 一个参数是设置数量, 都有默认值, 那么到底默认什么呢?), 源码中到底采用的是哪一种标准, 参数是否会动态变化因而产生某些诡异的处理, 如果看过源码这些都不是秘密。
- ◎ 很多时候大家会经常说: “我听别人说”、“我觉得”、“网上都说”、“我反正是没见过” 某种问题是怎么处理的, 没有自己的见解, 但是当你看过源码中是如何处理的, 你对这方面的认识和了解就足以和别人进行深度探讨和知识挖掘, 后者是引导者, 没有一定的思考和研究不会张口就说, 前者是被引导者, 两者的境界就不在一个段位上。

我们不是神, 不用掌握世界上所有的源码, 但是在你所在的技术领域, 如果需要挖掘到更深, 则需要去阅读这些源码, 以便于更好地在这个技术领域深造。

前文中提到了看源码需要一些兴趣点, 那么兴趣点从何而来呢? 胖哥认为兴趣点就是: “或许可以用它来做点什么”。还有没有高雅一点的理由? 好吧, 它可能来源于“程序员们的自我艺术追求”。

总的看来, 要读懂源码, 很多时候工作的平台是很重要的, 不过没有工作场景是否就没有阅读源码的机会呢?

胖哥认为不是 (除非你已经积累了很多, 需要场景来发挥), 只有使用过很多 API 才会考虑开始看源码。如果你开始考虑看源码, 那么只要勇敢

地迈出这一步，然后坚持一段时间就对了。另外，我们不要顾忌到过多的眼前是否用得上这些东西，很多东西现在用不到，并不代表永远用不到，在真正使用以前，需要的是大量积累，当真正需要时，这些东西会赋予你无限的灵感，正所谓“厚积薄发”，这些就是积累，加上合理的学习计划，通常会有很快的成长。

如果没有场景，该如何去学习呢？胖哥认为学习和场景始终是矛盾而又统一的，没有人天生下来就是有场景的。反过来讲，如果你没有很强的能力，老板为什么要给你具有很高挑战与风险的场景呢？，这个世界的需求是相互制衡的，与其被动去被这个世界拖着走，不如主动去适应它，甚至改变它。当某一天你能做到别人做不到的事情时，那么你就有机会在众人之中脱颖而出，反过来可能就是被人群所淹没。

在没有场景的情况下，学习方法依然有许多，除了上面提到的入门方法外，大家多去关注开源界的框架，以及平时工作中用到的框架的源码。只要写过 Java 代码就肯定用过很多开源的组件或工具，JDK 本身的大部分源码也都是可以找到的，所以你周围有丰富的学习资源，而且你天天都在用它们，天天都有场景，只是很多人不以为然，以“没有场景”作为一种借口而已。在这些开源框架中，有着许多优秀的设计思想，以及代码的设计风格，自己想想这样写代码有何好处，有什么坏处，如果自己来做框架会怎么去思考，应当注意些什么。

许多小伙伴会对这些框架持极度仰望的态度，仅仅将自己当成了“IT 行业软件的手工业制造者”，要成为“老 A 级程序员”，应当跳过这个“坎”。需要知道的是如果想继续走技术之路，终究有一天需要成为设计者、架构师，这不是老板给你一个 Title 你就真的是了，更加不是在程序员的行业“混”多少年就真的是了，而是需要真正经历许多磨练和成长，逐步走到这个程度的。这样不论你走到哪里，都可以被认可。

有人曾经问胖哥：你哪里来的那么多时间去看那么多源码？是不是成天的工作就是干这件事情？

胖哥认为如果仅仅是为了源码而去读源码就会失去阅读源码的价值，除非仅仅是为了去寻求艺术感。胖哥认为自己去阅读源码除了带着一些兴趣外，同时也是为了更好地解决工作中的问题，让自己所学能够有机会落地。

比如第 5 章，讲解了线程池、调度池的源码，顺带引导出非常多的组件，胖哥提到的那些知识其实就是相互关联的。不过早在几年前，胖哥第一次看这些源码时，并不知道有这些东西，也不知道为什么要那样写代码，更加不知道它们是相互关联的，当时只是处于懵懂阶段，但是胖哥记住了它们就是那样写的，至于为什么，胖哥带着一些疑问逐步去取证。在这几年中，胖哥通过许多实践、阅读专业资料、思考，在问题的认识上与几年前已经不可同日而语，至今时不时还会将这些源码拿出来翻看一下，再翻看这些源码时，会有不一样的感受，看到的技术面也将更加宽广。

总的来讲，胖哥不愿意为技术而学技术，但是并不会因为不用就不去学

技术，更加不会认为学习一些与工作不相关的技术是错误的。在学习的道路上只要懂得不断去实践、思考、查阅权威资料，与人进行交流并发表看法，但不要在完全没有理解真谛之前轻易地去下结论，坚持去做这样的事情，胖哥认为在技术道路上就可以走得更远。

归纳起来就是在说：心态、方法论！

也就是在内心上要战胜自己，就能战胜周围的事情。

方法要学习别人的，最终还要找到自己的路子。

做了事情、处理了问题要思考和总结，不要就任务结束而结束了，虽然我们可能很“苦逼”，但我们不要甘心去做“手工业制造者”。

本篇是“源码篇”，希望引导源码阅读的思想和方法。在本篇的内容中，胖哥希望你首先阅读第7章，本章胖哥会简单阐述框架的基本思想，以及阅读源码前的技术储备，以便于你更好地阅读。第8、9章会写一些开源界的源码，大家根据自己的兴趣和喜好选择其一阅读就可以了，重点是要能认识到一些基本的读取源码之道。

第 7 章

源码基础

本章主要内容

- 为何会出现框架

- 阅读框架前的技术储备

在项目的开发过程中经常会提到框架、组件、源码等，这些东西有何用途？这些问题不像“人类起源”问题那么复杂，因为你是专业的，所以需要知道这些，才会更加有想法去阅读框架的源码，也才能考虑它的好与坏。

没有任何技术储备，单纯地想要阅读很多开源代码是痛苦的，除了第 1 篇提到的一些 Java 基础外，还涉及一些内容，胖哥也会在本章中阐述。

7.1 为何会出现框架

为何会出现框架？为了说明这个问题，我们先看看一些争论性的话题。一些学习其他语言的朋友，也看过一些 Java 的东西，可能会觉得 Java 很多框架只是虚有其表，或者讽刺为“带着手铐做事情”，类似的说法十分多。胖哥并不否认某些观点，不过胖哥也有自己的一些理解。

首先我们偷换下概念：“世界的任何地方都存在一些生存规则，例如人类要生存需要靠水和粮食，需要劳动，社会发展到现在有各种商业和管制，在相互依赖和制衡中生存和成长，生物和社会形成一个生态体系”。胖哥认为，从 Java 语言设计的初衷来讲，它的一些灵感来自于生活，考虑问题的方法更多来自于生活中的形形色色。这个社会就是想各种办法来包装一些事情，让成本更加低廉，让许多变化产生规律，让效率提升。例如现代的世界有了自动化高科技的工具，手工业劳动变得越来越少，从计算机的角度来讲，封装过程也是从二进制到汇编再到高级语言层层封装起来的，目的在于让上层的编写者无须关心太多的细节。为何要这样？

原因一：就目前的程序员来讲，没有任何人敢说一旦上手便是一流高手，也没有任何人敢说一旦接触计算机就什么都知道，从高级语言到底层能够一目了然，显然这种境界的人并不多。并且随着 IT 技术的飞速发展，如果什么都要搞得非常清楚的话，那么学习的速度还没有技术刷新的速度快。在现实中不可能等到每个人都成为这样的顶尖级高手才让他写代码（虽然我们希望每个人通过成长后能达到类似的水平），但是又希望尽量让这些工作

时间不久的小伙伴少犯错误，因此我们希望这些小伙伴能不能尽量少关注一些不需要关注的细节。

退一步来讲，即使“老虎也有打盹的时候”，也许一个打盹会犯下大错误，我们希望将这些容易犯错的内容浓缩和抽象，将一些很细的细节尽量屏蔽掉，或者说开发者即使是小菜鸟也能将业务写得出来（虽然可能有一些问题，但是可以逐步进步），因为 Java 设计的初衷是为了给社会带来价值，让开发变得更加简单、顺畅。

IT 领域需要的计算机自动化非常多，社会中需要大量的技术人员来完成这样的事情，并且可以快速满足一些业务需求，或许 Java 还比较适合，也许小伙伴会说在现代的语言中不止 Java 可以办到，胖哥只能说 Java 设计的思想体系就是让它比较适合做这个。

原因二：每一种语言都有它的发展思路，偏底层的语言通常重点在于技术细节，Java 的偏重点不在这里，它希望你把精力放在设计和架构上，在设计时也许只需要思考主体、思考接口、思考测试用例，系统由平台组件+业务逻辑组建而成，开源界提供了许多通用组件，针对业务场景加以扩展和改造，也许就能组装出一台不错的“跑车”，让它跑得越快。

很多人说现在有很多语言也是面向对象的，Java 能实现的功能这些语言都能实现。胖哥也不得不承认这一点，不过就语言本身的层面来讲，比较功能意义不是特别大，因为在竞争对手面前只要知道了有这样的功能，要做出类似的功能是很容易的，而语言的“血脉和基因”会决定它的“性格”，Java 语言之所以这 20 年来一直立于不败之地，其中一个重要因素也是来自于它原本设计的初衷。

胖哥只是不希望大家因为自己是学习 Java 的就成为了 Java 的“信徒”，要客观看待每种语言的价值，尤其是不要拿 Java 有什么功能而别的语言没有来比较，因为这不是优势所在，我们只需要知道 Java 只要朝着良性的方向发展，必然能走得更远。

也许很多人会说 Java 太重，很多时候要操作一些简单功能太麻烦。确实是这样，不过不可否认的是 Java 为设计和架构而生，其最终铺垫的是为系统维护而生。为此，胖哥给大家说一个简单的小例子。

例如，大家在使用 Java 编写完对象的属性后，就开始为对象的每个属性编写 Setter 和 Getter 方法，喜欢思考的同学开始想为什么不直接 public，而要用这样的方式，通常这个时候老师会告诉他们这是一种规范，让大家先用，于是大家便开始用了，甚至有的同学会将这样的过程当成一种享受，因为有了代码量的享受，而且几乎是没有 Bug 的代码量，还不用动脑筋就能写出这么多的代码，或者说赚到收入。

对于这个为什么或许可以换个角度来理解，如果从一个“非严谨”的角度去理解它，则可以说是人学会穿上了衣服不至于“春光乍泄”。其实 Java 并没有在语法、规范、官方等方面强制约束要写 Setter 和 Getter，也没有要求必须是私有的，胖哥在此观点上做一点补充。

- ◎ 有 Setter 不一定非要有 Getter，反之也同样成立。
- ◎ Setter 并不一定要用来做赋值，或者说仅仅做简单的赋值，可以进行指定的某些运算。
- ◎ Getter 也并不一定只是做简单的 return 操作，可以返回一些经过转换后的数据。
- ◎ 类中没有相应的属性，但基于某种程序需要（例如，JSON 输出需要补充某些属性组合后的值），也可以提供一种 Getter 方法来满足宏观上的需求。
- ◎ 对某属性的操作也不仅仅局限于 Setter 和 Getter，可以有很多操作方式，Setter 和

Getter 是一种 Java 规范，可以满足许多框架制作的基础，而我们使用时需要在实际场景中加以变化，让代码更加清晰。

再换个生活中的角度来理解这个问题，某段程序被谁“摸”了一下，有时候想要知道是被谁“摸”过，或者想要控制这段程序允许被谁“摸”和不让谁“摸”，还想要记录每一次被“摸”过所发生的时间，或控制每个外部实体所能“摸”到的程度。如果是开放式的，对象就成了公共用品，当真正想要控制的时候就没有任何办法了。“暴力”一点讲，如果没有这样的机制，摸出问题谁也不承认，外部操作实体可能会看了不该看的或写了不该写的。

从外观的角度来讲，如果想要让外界看到自己“不同方式的美”，public 的做法就需要让外界每个“关注你的人”带上有色眼镜来看你，每次想要改变自己的外观时，就要让每个“关注你的人”再去换一副眼镜才能达到目的。或许这个时候你根本不知道到底有多少个“关注你的人”，你可能也没有经历去让他们每个人换一副眼镜，所以可能无法让每个关注你的人去换一副眼镜。反过来讲，如果自己通过换换着装、头型、皮鞋等方式，就能让别人看到不一样的自己。也许你理解了 this 另类的例子，也许你还是没理解！没关系，没理解的同学可以根据自己所经历过的项目慢慢理解。

下面来谈谈上面说的 Java 的“重”。

不得不承认，Java 在某些时候使用起来是很重的。

- ◎ 使用 Java 编写一个简单程序需要很多基础设施，例如安装 JVM，配置环境变量，有些同学还要安装 IDE 工具，再引入一大堆的 jar 包，写个小程序也得搞这样一堆复杂的配置和依赖。
- ◎ 许多 jar 包之间还有一大堆的依赖性，我们需要剥离冲突。
- ◎ 在一大堆的开源 API 中，某些 API 功能极为相似，要选择就得很熟悉它们，因此需要学习它们。
- ◎ 在不同的框架、容器下又会产生大量的参数配置、原理需要大家去学习，总之要跑起来一样东西确实不那么容易，等等。

现在也有许多基于 JVM 的脚本语言来帮助你解决某些问题，它们提供了许多包装后简单的语法规则（例如 Scala、Groovy），相对 Java 本身很轻。

回过头来说，不能因为 Java 的某些细节不好就否认 Java 的一切，因为它的初衷不是这个（有些时候宁愿为了 10% 的可能性变数去做许多对象化处理，而不愿等到需要维护时再去找哪些地方需要修改），有一种血脉就必然会丧失一些东西，只要它生存于这个世界，被人们所选择那就有价值，作为程序员需要找到的是适合于场景的语言。

也有很多人会说框架就是让人写更少的代码，胖哥也不得不承认在很多时候的确是这样，以前胖哥也会给别人说“低手写代码，一群人写一堆代码搞定一个问题；高手写代码，一个人写几条代码搞定一堆问题”。为何？高手看待问题有项目和产品的全局观念，从系统的角度考虑问题，他会考虑架构的扩展，以及如何利用写好的组件，将公共逻辑抽象出来完成，以达到解决通用问题的目的——“解决一个问题，相关问题都已经得到解决”，同时也减少了大量的测试用例。

在关于框架的一些问题上，胖哥认为不仅仅是代码量的问题，需要补充的是：

自定义框架的缺陷——自己扩展的框架，别人来使用会有“学习成本”，在这种自定义框架上如果出现问题，在网络上搜索答案可能没有任何结果，这会让使用框架的小伙伴前后难做（在本书的第 10 章还会提到）。

要解决这个问题，就框架的应用范围和企业文化来看，例如可以采用分享、培训、开源、文档化等方式。从做一个好的架构来讲，要尽量去做的是如何降低学习成本，降低使用门槛，而不是一味地显摆自己的技术多么牛气。换句话说，如果没人会用或无法给项目带来价值，这些技术就一文不值，技术和知识需要“学以致用”，做框架也是同样的道理，做框架并不意味着牛气，只是自己在这个方面花的时间比别人多，自己所研发的内容是为开发服务的。很多时候，懂得开发痛苦的人或许可以做出更好的框架，换句话说，做任何事情也要本着它存在的目的去完成。

一个框架的优势——前文中提到，框架在某些时候可以节约许多代码，尤其是重复的代码，但胖哥认为这只是短期利益，更加明显的优势应当是“维护代价”。在软件的生命周期中，最长的是维护，因为在系统运行和发展中不可能一成不变，完全按照自己的思路来做，始终会有很多问题和不符合用户需求的地方。在这个过程中需要不断去修改，可能某些修改会使得系统的架构变乱。

系统架构的伸缩性在这个上面的体现十分重要（主要体现在业务层面的架构上），如果系统中出现大量重复的代码，尤其是和业务相关的代码，当某一天此业务发生变化时，那么要修改多少个地方呢？当这样重复的逻辑在新的场景下出现 Bug，要修复这样的 Bug 又要改多少个地方呢？此时又会把“苦逼”的程序员们带入“手工业制造者”的时代，反复地做着枯燥无味而又“感觉充实”的修改工作，在修改过程中如果遗漏了什么你能完全知道吗？如果一个系统仅仅是为了完成业务需求，大家都在上面增加 `else if` 来完成代码，不断地一代新人换旧人，时间长了就没有人敢修改代码了，更加没有人敢重构代码了。

架构的思想或许由此而来，因此很多语言都有相应的架构来解决不同的问题。Java 只是这个开源领域中的佼佼者，它更多地传达架构的思想，让开源的爱好者来提供一些大家都需要的代码，程序员们可以跨国界共享大量的知识圈，在这里是一个知识的社会，有好的知识，也有乌合之众，各种各样的框架都有类似的功能和应用场景。在这个混杂的技术中，一个老 A 级 Java 程序员需要学会做“选择题”。或者说如何将这些从一个鱼龙混杂的地方挑选出来的零部件组装成一台设备，一台你觉得“爽”的设备，让别人用得爽的设备，让用户更好地体验的设备，这些也许就是程序员们所追求的吧。

框架可以在一定伸缩度上让维护变得更加简单、便捷，程序员的性价比得到提升，用户问题的解决速度也得到快速提升，与其说是“带着手铐做事情”，不如说是“穿着自己喜欢的衣服逛街”，或者说是利用“交通工具”在城市间穿梭。

软件本身就是逐层封装出来的，任何人都需要成长，今天的 IT 行业应该不会有人用汇编语言去写一个网站，更加不会有“奇葩”用二进制代码给一个企业做一个复杂业务逻辑的 ERP 系统。Java 语言面对的大多数是客户的业务问题，在语言本身与运行效率上，无须做到“微秒”级甚至“纳秒”级那样快速，因为用系统的是“人”，大多数时候人无法感知。某些时候技术更重要的目的是要为人带来便利和服务，不论是程序员还是客户都需要。这就是胖哥本人所理解的“架构的由来”，不知道你满意吗？

要用智慧创造财富，而不仅仅是通过“体力”创造价值，因为你是老 A（胖哥再次强调，必要的体力活是需要付出的）。

而智慧能替代多少体力，能否替代体力，是一个设计者的水平问题，也是我们在本书下册第 3 篇“设计篇”中会详细阐述的，也许你会发现烂的架构不如没有，好的架构让我们爱不释手，而本篇我们需要先做好技术准备，然后看看别人的框架。

7.2 阅读框架前的技术储备

7.2.1 反射基础知识

说到反射，知道反射的同学可能会联想到 Class、Method、Field 等，没错，它们很多时候会利用到反射。如果以前没接触过反射，胖哥这里也可以为你开个头。为了了解反射，也同样需要知道它存在的一些价值。为了说明这一点，先来了解一下结构化语言的局限性。

举个例子。现在需要从数据库中将一个表的数据加载到程序的对象中，为了将表的属性与程序中类的属性映射起来，在没有任何框架的情况下需要逐个编写每个属性写入方法。这样反反复复的操作是十分烦人的，做的时间长了会发现对于某个单独的业务表来讲，大多数 SQL 语句只是条件不同，而获取的属性基本差别不大或有几种特征。对于这样的现象，抽象出一个“逐行回调方法”的概念，类似的 SQL 只需要用相同的回调对象来处理，即可实现一个简单的属性对象映射。对于 SQL 语句而言，大家又发现 SELECT 部分也可以将公共部分分解出来，这样可以减少许多烦琐的拷贝，以及维护时的改动。

即使这样，代码依然十分烦琐，我们很希望返回结果名称直接与属性对应就可以自动写入，直接调用对应属性的 set 方法。要达到这种目的，就需要用到反射，也只能用反射来解决，利用反射获取到方法对象，通过方法对象来动态调用完成。



注意 这里所讲的方法对象，是反射中 JVM 提供的一种机制，这些对象是用于调用类的对应方法的。

再来举一个 Web 的例子。当从浏览器请求一个 URL 时，这个 URL 请求最终要对应到一个资源上，这个资源如果是 Servlet 资源，Web 容器是如何实现的呢？想一想配置文件和代码编写方法，每个自己编写的 Servlet 都需要在 Web.xml 内部注册，每个 Servlet 都必须实现统一的 HttpServlet 接口，因此容器是基于通用接口来实现这个分发动作的。

但这种编写代码的方式会让大家感觉十分烦琐，因为每个不同的请求都需要编写一个 Servlet，再写一次配置文件，而且 Web.xml 配置文件会变得十分庞大，不便于许多人同时开发。

此时第三方框架在这个基础上做了扩展，这些框架通常会提供一个通用的 Servlet 来处理请求，经过这些框架封装后跳转到 Action 资源或 Service 资源上，对于配置可以按照模块、子模块来不断细化，这样就将这个耦合打开了。

不过，此时 URL 以及传入的参数仅仅是一些字符串，外部请求最终需要访问的是实际的用户所定义的类，并且可能定位到具体的方法，因此就不一定是基于接口来定义的。换句话说，在接口的定义中方法名是固定好的，它无法直接引用到其他的方法，此时要动态执行对应类中对应的方法该怎么做呢？也是通过反射来达到目的。

反射是一种技术实现手段，一个 URL 具体映射到哪一个资源上，完全由框架提供的配置文件或 Annotation 来决定。简单地讲，配置是映射关系的定义，告诉跳转的引擎请求要分发到哪一段代码中，而引擎需要解析和加载配置信息，并根据 URL 找到目标类及方法，然后利用反射技术进行调用。

如果看懂了反射的这些好处，那么就可以来看反射是如何使用的了（其实反射有很多知识，胖哥这里也只是列举其一二而已）。首先来看看方法动态调用的例子。

代码清单 7-1 一个简单的方法动态调用的例子

```
public class MethodInvokeSample {

    public static void main(String []args)
        throws Exception {
        Method method = MethodInvokeSample.class.getDeclaredMethod("test",
            String.class , int.class);
        String result = (String)method.invoke(null, "fuck" , 2);
        System.out.println(result);
    }

    public static String test(String a , int b) {
        return"传入参数 1: " + a + ",传入的参数 2: " + b;
    }
}
```

在这段代码中，获取到“当前类”的 test 方法，程序中所获得的 Method 对象是一个经过反射包装后的对象，全名为：java.lang.reflect.Method，它提供了一个 invoke()方法，调用该方法就会间接调用实际的方法。

代码中获取方法对象时，除了传入方法名外还传入了 String.class、int.class，它们代表入口参数的类型，并按照顺序编写，这样就可以避免重载方法在动态调用时冲突。

这是一个简单的调用例子，在实际的项目应用中会比这个复杂很多倍，但如果你理解了，则可以发现它是由一些简单的元素所组成的。例如，Spring 在填充表单属性时，会采用 java.beans.PropertyDescriptor 类的 getWriteMethod()方法来确定属性的写入方法；不过，在 ibatis 框架中对 JavaBean 的属性填充时，却是 ibatis 自己单独搞了一套方式来获取属性的写入方法。

既然提到了 PropertyDescriptor，这里写一段使用 PropertyDescriptor 的简单的演示代码让大家有个宏观的认识。

代码清单 7-2 PropertyDescriptor 的简单例子

```
public class PropertiesDescriptorSample {

    public static void main(String []args)
        throws IntrospectionException {
        Field []fields = Node.class.getDeclaredFields();
        for(Field field : fields) {
            System.out.println("=====>" + field.getName()
                + "\t" + field.getType());
            PropertyDescriptor propertyDescriptor
                = new PropertyDescriptor(field.getName() , Node.class);
            printDesc(propertyDescriptor);
        }
    }

    private static void printDesc(PropertyDescriptor propertyDescriptor) {
        System.out.println(propertyDescriptor.getDisplayName());
        System.out.println(propertyDescriptor.getShortDescription());
        System.out.println(propertyDescriptor.getPropertyEditorClass());
        System.out.println(propertyDescriptor.getPropertyType());
        System.out.println(propertyDescriptor.getReadMethod());
        System.out.println(propertyDescriptor.getWriteMethod());
    }
}
```

```

    static class Node {

        private String name;

        private String email;

        public String getName() {
            return name;
        }

        public void setName(String name) {
            this.name = name;
        }

        public String getEmail() {
            return email;
        }

        public void setEmail(String email) {
            this.email = email;
        }
    }
}

```

在这段代码中简单定义了一个 Node 类，程序运行时遍历其属性，遍历每个属性并通过 PropertyDescriptor 得到相应的描述信息，这些描述信息中包含了对属性的读、写方法，其余的一些输出信息大家也可以关注一下，总之有一个宏观认识就好。

代码中获取的 read、write 方法名称是可以通过 setReadMethod、setWriteMethod 来指定的，如果没有这样设置，系统默认是 set、get 方法，不过在 boolean 类型的“读”方法上默认会用 is 开头的方法。如果小伙伴对这些代码的实现有兴趣，则可以自己去看看，看过以后应当会发现也就是一些字符串拼接和类型判定。

在代码清单 7-2 中，大家看到了提取属性的方式，类似的反射 API 还有很多，大家可以根据需要自己动手做实验，实验做多了，加上自己的思考，会发现反射会给自己带来非常多的帮助。

如果使用过反射来动态填充属性，则可能遇到过一些属性类型的问题，例如日期值、自定义类型的对象，它们不像基本类型那样简单（例如，从 UI 上提交到服务器端的数据都基本可以被认为是字符串，需要将它从文本转换为对应的类型）。为了解决这样的问题，可以在反射前做很多类型判定及转换处理，不过那样会使得反射代码变得十分庞大，也会导致这部分代码和业务套在一起。

此时希望剥离出类型转换这一层，这就利用到反射中的 java.beans.PropertyEditor。它是一个接口，需要实现很多方法，不过幸运的是，Java 提供了 java.beans.PropertyEditorSupport 对其中大量的方法进行了默认实现，业务中代码只需要继承这个类，做一些小扩展就可以了。在扩展过程中，比较关键的是要扩展 setAsText(String)方法，它始终会传入一个 String（不论是配置文件的内容，还是通过网络输入后解析完的 parameter 参数，大多数都以 String 形式存在），需要通过这个 String 解析为某个自定义的对象，生成对象后，调用 setValue(Object)方法传入，就写到当前 PropertyEditor 对象的 value 属性中，外部可以通过这个 PropertyEditor 对象的 getValue()方法获取到实际的对象。

在 Spring 的表单填充方法中，就使用了类似的方式来扩展，系统中可以在配置文件或 Annotation 中将自己的 Properties 注册到 Spring 中，Spring 在填充属性时就会调用对应的

setAsText 方法来完成字符串到属性实际类型的转换,然后再通过调用其 getValue()方法得到转换后的对象。简单来说,这是一个属性转换器,传入的是 String,把它转换为自己的目标对象, Spring 再获取这个对象作为相应的“属性写入方法”的入参。

很多人认为这个是 Spring 的方法,其实不然, Spring 只是在 Java 提供该功能的基础上做了一层扩展而已,框架进一步包装后将更加实用。在 Spring 中具体的使用方式有很多,通过注解或配置文件都可以达到目的。例如,在某方法前增加注解 @InitBinder,该方法增加类型为 org.springframework.web.bind.ServletRequestDataBinder 的入口参数,调用该参数的 bind()方法,绑定某种数据类型用什么样的 PropertyEditor 来处理,如果没有绑定 Spring 自然会使用一些默认处理方法,如果没有提供相应的类型处理,就可能会报错。

需要补充说明的是, Spring 增加了一种 ConversionService,它支持任意类型之间的转换,它位于 org.springframework.core.convert 包下 (Spring 在支持了 ConversionService 后也继续支持 PropertyEditor)。

所谓的任意类型之间的转换都支持,并不是说我们自定义的类型它也支持,这种说法只是代表来源的类型可以是任意一种, Spring 依旧提供了常见数据类型的相互转换,但自定义的许多数据类型依旧需要自己去编写转换方法。

ConversionService 的由来或许还有另一个需求: PropertyEditor 所产生的对象需要先通过 setAsText()方法将中间生成的对象保存到这个 PropertyEditor 对象中,然后再调用 getValue()方法获取对象。这似乎没有什么问题,但是仔细一想:这个 PropertyEditor 对象包含了运行时的数据信息,此时如果有大量的并发运行,或多个同样的属性需要设置,就要考虑线程安全的问题,最简单的方法就是为每个属性的实体分配多个 PropertyEditor 对象,还有一种方法就是使用一个或有限个 PropertyEditor 对象作为对象池,用锁来调度处理。后者通常会被直接否掉,简单问题搞复杂了,但前者很明显比较浪费不必要的开销。

ConversionService 并不是这样实现的,它不会保存上下文信息,只是作为一个转换器存在,需要编写入口参数和返回值,也就是当前的转换方法直接返回具体的结果,那么就不需要一个属性来保存结果了。

说了这么多,还需要来看看一个实例中 Java 默认的 PropertyEditor 是如何操作的,至于 Spring 是如何实现的,其实思想是类似的。

首先设计一个类 NodeDO,它拥有 3 个属性,分别为 name、email、dateTime,其中 name、email 为 String 类型,而 dateTime 为 Date 类型。

再设计一个类用于包装 NodeDO,换句话说, NodeDO 将作为它的一个属性存在才能模拟一些实际的场景,我们把它命名为 TestDO,内部一个属性的类型是 String,一个属性的类型是 NodeDO。由于这两个类的属性十分简单,胖哥就不将此代码贴出了(比较占用篇幅),大家可以参看本书光盘中 src/chapter07/reflect/editor/目录下的内容。

代码清单 7-3 PropertyEditor 的简单例子片段 1——自定义 Editor 定义部分

```
public class NodeDOEditor extends PropertyEditorSupport {

    public final static SimpleDateFormat DEFAULT_DATE_FORMAT
        = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");

    @Override
    public void setAsText(String text) throws IllegalArgumentException {
        String []tokens = text.split("\\|");
```



```

NodeDO nodeDO = new NodeDO();
nodeDO.setName(tokens[0]);
nodeDO.setEmail(tokens[1]);
try {
    nodeDO.setDateTime(DEFAULT_DATE_FORMAT.parse(tokens[2]));
} catch (ParseException e) {
    throw new IllegalArgumentException(e);
}
setValue(nodeDO);
}
}

```

这里在 `setAsText()` 方法中将传入的字符串按照 “|” 拆分为 3 个字符串，然后分别设置到自定义对象的各个属性中，并通过 `setValue()` 将其保存起来。这样做有点看不懂，程序是如何使用它的呢？不急，接下来我们来写段测试代码，简单模拟反射写入属性的部分。

代码清单 7-4 PropertyEditor 的简单例子片段 2——模拟调用

```

public class PropertiesEditorSample {

    private final static Field[] TEST_FIELDS
        = TestDO.class.getDeclaredFields();

    static { //注册 Editor
        PropertyEditorManager.registerEditor(NodeDO.class,
            NodeDOEditor.class);
    }

    public static void main(String []args)
        throws IllegalAccessException, IllegalArgumentException,
        InvocationTargetException, IntrospectionException {
        Map<String, String> parameter = new HashMap<String, String>() {
            private static final long serialVersionUID = 1L;
            {
                put("nodeName", "小胖测试");
                put("nodeDO", "xieyuooo|pangpang@qq.com|2012-12-21 12:21:12");
            }
        };
        //模拟参数信息，与 request 的 ParameterMap 类似
        TestDO testDO = convertParameterToDO(parameter);
        System.out.println(testDO.getNodeName());
        System.out.println(testDO.getNodeDO());
    }

    private static TestDO convertParameterToDO(Map<String, String> parameter)
        throws IntrospectionException, IllegalAccessException,
        IllegalArgumentException, InvocationTargetException {
        TestDO testDO = new TestDO();
        for(Field field : TEST_FIELDS) {
            PropertyDescriptor propertyDescriptor
                = new PropertyDescriptor(field.getName(), TestDO.class);
            Method method = propertyDescriptor.getWriteMethod();
            if(field.getType() == String.class) {
                method.invoke(testDO, parameter.get(field.getName()));
            } else {
                //查找 Editor
                PropertyEditor propertyEditor
                    = PropertyEditorManager.findEditor(field.getType());
                if(propertyEditor != null) {
                    propertyEditor.setAsText(parameter.get(field.getName()));
                    method.invoke(testDO, propertyEditor.getValue());
                } else {
                    System.out.println("properties : "

```



```

        + field.getName() + ", can not find editor");
    }
}
}
return testDO;
}
}

```

代码中在初始化时，就注册了 `NodeDOEditor` 类与 `NodeDO` 对应起来，当在 `convertParameterToDO()` 方法中解析属性时，通过 `PropertyEditorManager.findEditor()` 来查找对应属性类型的 `Editor`，跟踪源码可以发现，在该方法中如果找到了对应的 `Editor`，则会实例化一个相应的对象返回；而如果没有找到，则返回空。我们的代码中会输出一条日志信息，系统框架会采用类似的方式来实现。

这些代码看似十分复杂，但是它们在项目中通常不需要反复去编写，一旦抽象好以后，代码中即使有再重复的工作（尤其是那些低级的属性赋值，漏掉一个你还蒙在鼓里的事情），也变得不是什么问题了。

通过本节介绍大家应该感觉到反射是大多数框架的基础，没有它很多框架无法谈及实现层面，所以当有一天你需要自己去做框架或扩展某些开源框架时，必然要用到反射。在此，胖哥不得不提到反复说的一句话：本小节内容仅仅是反射的入门知识，更为深入的反射知识，就要靠你自己了。

反射并不是万能的，它依然有十分多的问题和缺陷，很多人会说它的性能不好，确实，在早期的反射中，性能是相对较差的，虽然在 JVM 的发展中对它做了明显的性能改善，但比起直接调用的效率还是稍微差一些。

很多时候不得不用到反射来解决问题时，这点性能几乎可以忽略，只是我们不要因此而滥用反射，或者说是为了用反射技术而强迫自己使用反射，很多时候可以通过接口规约来简单实现的东西，就没有必要用反射，因为那样更加清晰。如果代码中滥用反射，进行大量的动态调用，那么一个 API 动态调用中到底调用的是什么呢根本不清楚。此时只有对代码跟踪并查阅某些配置信息才能知道实际情况，如果你的系统中处处都是反射，而且这样的反射没有任何规范，那么看代码的人会疯掉。

很多时候，反射只是使用在某些通用模型设计中，在受某些因素影响通过常规的设计思路不太方便实现的情况下，就可以考虑用反射技术来帮助解决问题。反射的具体使用场景，胖哥在本节中举了一些例子，更多场景需要大家去思考、去实践、去领悟。

通过对本小节知识的学习，当你再一次看到一种框架功能时，是否会思考它的哪些功能会用到反射，是如何使用反射的，为什么要用反射，或者依然用仰望的态度面对这些技术点，认为这不是自己应该去了解的。

关于反射，最后补充一个话题，也是一个简单的题目——反射在泛型中的应用。大家可以通过一些代码去测试，在测试中会发现反射在泛型中会有一些诡异的行为，例如定义一个 `List<String>list = new ArrayList<String>();`，此时通过反射的方式来调用 `add()` 方法，并传入一个数字，会是什么结果？如果是 `List<Integer>list = new ArrayList<Integer>();`，此时也通过反射的方式来调用 `add()` 方法，但是传入一个字符串，又是什么结果？再用自定义对象是什么结果？它们的结果会一样吗？都会报错吗？还是都不会报错？大家做做就知道结果会很诡异。再通过 `javap` 输出的指令看看有何区别，可能你会发现官方所提到的对泛型的“擦除机制”，并非擦除所有的东西。类似的，大家是否还记得第 1 章中提到的 `Integer`

与 int 之间的自动拆装箱问题，如果通过反射来调用相应的方法，自动拆装箱的结果也很怪异。

反射在通用模型中并非能解决所有的问题，如果要在某些指定的代码前后进行公共的操作，最土的办法就是在每个相关的方法上都写一次这样的动作，显然这样的动作十分的笨重和臃肿，更加重要的是，如果项目组成员有人“搞忘”去增加这些统一的代码，这又会多出一个隐藏的 Bug，隐藏的是百年不遇的 Bug，当我们需要修改这个公共的逻辑时，就又要改很多地方。因为这些问题，AOP 出现了，让我们对这些问题有了新的认识。下面开始讲解 AOP，你准备好了吗？

7.2.2 AOP 基础

AOP 是什么？很多人一提起 AOP 就想到 Spring，这也难怪，因为有太多的程序员用 Spring 来做 AOP。从真正意义上来讲，AOP 只是一个概念，而 Spring 只是通过 Java 的一些技术手段实现了这个功能。

单纯地理解 AOP 这个概念不是那么好理解，有人把它翻译成“面向切面编程”，也有人说它是“非侵入式编程”，当胖哥第一次听到这些名词时，感觉十分的神奇，因为在那个时候胖哥还处于 Web 开发的入门阶段，自然听到这个“可怕的名词”望而生畏。

现在胖哥有一些自己的领悟，更加愿意通过一种技术能解决什么问题来理解这个技术，然后再去看这些“可恶的名词”，这些可恶的名词也是很多程序员的可爱之处吧，翻译者有自己的理解，学习者也有自己的理解，传达者更带有显摆的味道。

想一想：如果需要在业务层包装业务日志，你会发现大部分方法都会做类似的动作，如果某一天写业务日志的动作发生改变，那么需要修改多少代码呢？

我们首先想到的解决方法就是封装成一个方法，只需要在适当的位置调用方法即可，但也有可能某些人“写着写着”就忘记要写这条代码了，为什么容易忘呢？因为这是和业务不相关的代码，也不是语法检测中所必须要求的，自然再考虑问题时不会把它当成重点，偶然出现遗漏是十分正常的事情。

再举个例子，例如数据库的事务管理，如果要完成事务，代码中通常会出现 try、catch、finally 操作，先取出 Connection 对象，并调用 setAutoCommit(false)，然后执行业务代码，正确地结束后会调用 Connection 的 commit() 方法，出现异常会调用 Connection 对象的 rollback() 方法，在 finally 里面会调用相关资源的回收方法。

如果每个数据库操作的方法都这样写，是否会觉得十分的繁杂（注意：不是复杂，而是繁杂），因为一个应用系统做下来少则几十个，多则成百上千个这样的方法，这些方法表面上十分规整，都长一个样子，但是仔细一看会发现是“很土”的样子。为何说土呢？写起来烦琐、古板是一回事，关键是回收资源这一块，很难去保证所有的程序员能将应该回收的资源全部都回收。例如在一个没有中间件支撑的分布式数据库访问中，这些代码会有多么烦琐。

很多时候希望写业务代码的人员专注于业务层面，在系统层面少犯错误，尽量不要因为一些小错误而浪费大量的时间。AOP 希望通过“插入代码”的方式来实现上面提到的功能，更确切地说是插入公共代码的方式，程序员无须关注这部分代码，他们写的代码大多数应该都是不一样的，而不是反复地拷贝，以避免前文中提到的各种问题。

此时理解 AOP 要干什么了吗？它就是要切进去，所以我们说面向切面。此时要切入，

那么在技术上要怎么切入呢？总体上来讲，通常有动态代理技术、字节码增强技术。

提到字节码增强技术，在本书 3.2.4 节中做了大篇幅的讲解，这里就不再说明了，相信知道字节码增强技术后，就不难理解切入是怎么回事了。在 Spring 中使用字节码增强就是其中一种手段：通过创建子类来实现，同名的方法子类完成参数捕获和返回值捕获，通过 super 调用父类中同名的方法来达到目的。

在 Spring 中很多地方使用过动态代理（早期版本的 Spring AOP 都是用动态代理来完成的），它是虚拟机提供的方法，在功能上与字节码增强有一些微小的区别，是什么区别呢？调一调你的胃口，胖哥先使用纯 Java API 来实现一个简单的动态代理功能（即不使用任何第三方的包来完成，代码可以直接运行）。

同样这些代码会比较烦琐，代码相对较长，部分内容将使用文字描述，具体的代码大家可以参阅本书光盘中的 src/chapter07/dynamic/目录。

动态代理都需要接口来实现，所以首先在代码中需要使用一个接口，在接口中我们任意定义几个方法，有传参数的，有不传参数的，有返回值的，有不返回值的，至于命名一切从简即可。这里假设接口名称为 HelloInterface，然后编写实现类 HelloImpl，具体的 body 部分大家可以自己定义。在这里没有必要将这段代码贴出，接下来开始做动态代理部分的关键代码。

在动态代理中比较关键的部分是需要有一个类实现接口 InvocationHandler，类名的全称为 java.lang.reflect.InvocationHandler，可以看出这也是反射中的内容，看到这个类名应该能猜到它有两个功能，也就是将单词拆为“Invocation”和“Handler”，前者意为执行，后者意为句柄（实际对象的间接引用）。

此时再来看演示代码也许会轻松许多，胖哥知道大家还有很多疑问，但是先不要想太多，把这段代码写好，因为它是动态代理的非常重要的组件。

代码清单 7-5 自定义一个 InvocationHandler

```
public class AOPHandler implements InvocationHandler {

    private Object target;

    public AOPHandler(Object target) {
        this.target = target;
    }

    public void println(String str, Object... args) {
        System.out.println(str);
        if (args == null) {
            System.out.println("\t 未传入任何值....");
        } else {
            for (Object obj : args) {
                System.out.println(obj);
            }
        }
    }

    public Object invoke(Object proxyed, Method method, Object[] args)
        throws IllegalArgumentException, IllegalAccessException,
        InvocationTargetException {
        System.out.println("\n\n====>调用方法名: " + method.getName());
        Class<?>[] variables = method.getParameterTypes();
        System.out.println("\n\t 参数类型列表: \n");
        for (Class<?> typevariables : variables) {
```

```

        System.out.println("\t\t\t" + typevariables.getName());
    }
    println("\n\n\t传入参数值为: \n");
    for(Object arg : args) {
        System.out.println("\t\t\t" + arg);
    }

    Object result = method.invoke(target, args);
    println("返回的参数为: ", result);
    println("返回值类型为: ", method.getReturnType());
    return result;
}
}

```

大家需要关注这段代码中的 `invoke()` 方法，通过它可以输出方法相关的信息。另外，代码中通过 `target`（也就是实际的对象）来调用实际的方法。也就是说，动态代理通过将原始对象作为代理对象的一个属性存在，当外部发生方法调用时，会执行这个 `invoke()` 方法，然后由它代理执行相应对象的方法，它可以在执行方法前后做许多事情，甚至在代理层可以决定是否执行。

看到这里，大家可能会迫不及待地想要知道对象的实例化应当如何处理。其实这个问题可以这样来考虑：为什么要从 Spring 中获取对应的对象引用相应的对象，才能达到 AOP 的功能，但自己直接 `new` 出来的同样的对象不行，这是玄机所在，对象的创建来自于一个公共的地方。为此，胖哥写了一段模拟一个假的 `BeanFactory` 的代码，如下所示。

代码清单 7-6 自己写个 `BeanFactory` 玩玩

```

public class BeanFactory {

    public static Object getBean(String className)
        throws InstantiationException, IllegalAccessException,
        ClassNotFoundException {
        Object obj = Class.forName(className).newInstance();
        InvocationHandler handler = new AOPHandler(obj); // 定义过滤器
        return Proxy.newProxyInstance(obj.getClass().getClassLoader(),
            obj.getClass().getInterfaces(), handler);
    }

    @SuppressWarnings("unchecked")
    public static<T> T getBean(String className, Class<T> clazz)
        throws InstantiationException, IllegalAccessException,
        ClassNotFoundException {
        return (T) getBean(className);
    }
}

```

这里提供了两个静态方法，真正核心的是第一个方法，方法内非常关键的是通过 JVM 的 `Proxy.newProxyInstance` 来实例化对象，这好像是给它穿上了一件衣服，这件衣服就是前一段代码中提到的 `AOPHandler` 的代码。AOP 竟然就这个样子，不相信我们就测试看看，以下为一小段测试代码，大家可以根据自己写的接口和实现类对测试代码进行相应的调整。

```

HelloInterface hello = BeanFactory.getBean("chapter7.dynamic.HelloImpl",
    HelloInterface.class);
hello.setInfo("xieyuooo", "xiaopang");

```

运行这段代码，在输出的内容中有些内容是实际方法输出的，有些则是输出代理方法 `invoke()` 中的相应内容。

相信很多好学的小伙伴开始思考 `Proxy.newProxyInstance()` 到底做了什么事情？为何可

以给它穿上一件衣服？是如何穿上去的？我们看了源码后不难发现，还是“字节码增强技术”，关键的源码片段如图 7-1 所示。

```

/*
 * Generate the specified proxy class.
 */
byte[] proxyClassFile = ProxyGenerator.generateProxyClass(
    proxyName, interfaces);
try {
    proxyClass = defineClass0(loader, proxyName,
        proxyClassFile, 0, proxyClassFile.length);
} catch (ClassFormatError e) {
    /*
     * A ClassFormatError here means that (barring bugs in the
     * proxy class generation code) there was some other
     * invalid aspect of the arguments supplied to the proxy
     * class creation (such as virtual machine limitations
     * exceeded).
     */
    throw new IllegalArgumentException(e.toString());
}

```

图 7-1 动态代理也是使用字节码增强技术实现的

在本书 3.2.3 节中提到过，在字节码增强技术中会通过一些技术手段得到一个 Class 经过字节码增强后的 byte[] 数组，然后通过 ClassLoader 来加载就动态装入内存。这这段代码也采用类似的方式，不过区别在于动态代理在生成动态字节码时，并不是通过实现类创建子类的方式，而是通过类所拥有的接口列表来完成的。也就是说，构造出来的类和实际的实现类一点关系都没有，是完全独立的一个类，只有接口的方法名与原始对象的方法名相同，那么和直接使用其他的字节码增强就有了使用上的区别。

◎ 这个类必须基于接口才能用到动态代理，因为这个地方构造出来的类，是基于接口列表中的所有方法来创建的，也就是会实现所有相关接口的方法。

◎ 这样构造出来的类无法用实现类类型的引用来获取，比如在上面的例子中，无法用 HelloImpl 来引用，因为创建的类和这个 HelloImpl 类一点关系都没有。

这个类创建并获得后，它实例化时的动作有点玄机，那就是将自定义的 InvocationHandler 实例化的对象传入进去，如图 7-2 所示。

```

private static Object newInstance(Constructor<?> cons, InvocationHandler h) {
    try {
        return cons.newInstance(new Object[] {h});
    } catch (IllegalAccessException | InstantiationException e) {
        throw new InternalError(e.toString());
    } catch (InvocationTargetException e) {
        Throwable t = e.getCause();
        if (t instanceof RuntimeException) {
            throw (RuntimeException) t;
        } else {
            throw new InternalError(t.toString());
        }
    }
}

```

图 7-2 动态代理中的实例化方法

构造方法对象调用 newInstance() 时需要传入 InvocationHandler 的实例，为何？

自然是因为 InvocationHandler 的实现类 AOPHandler 中包含了实际对象的引用，要操作实际对象就得通过它来操作，才能形成间接代理访问。

如果所有接口的方法没有命名和入参冲突，那么动态创建的类中所有的方法都很清楚自己源自哪个接口，如果多个接口中有同样的方法，那么这里的代理机制会按照 implements 的顺序认为最先存在这个方法的接口是真正的接口，这一点大家可以在 AOPHandler 类的

invoke()方法中,对通过传入的 method 方法调用 method.getDeclaredClass()得到这个方法到底来源于哪里,如果再给实现类 HelloImpl 增加几个接口,测试结论依然一致。

创建的新类与原始的类完全是不相关的,那么即便方法名通用又如何达到相互调用呢?

invoke()方法的 method 参数来自于接口而不是实现类,通过接口的虚方法就可以适配,在反射中也遵循这一点,总的来讲,动态代理实现类与用户自定义实现类的关系如图 7-3 所示。

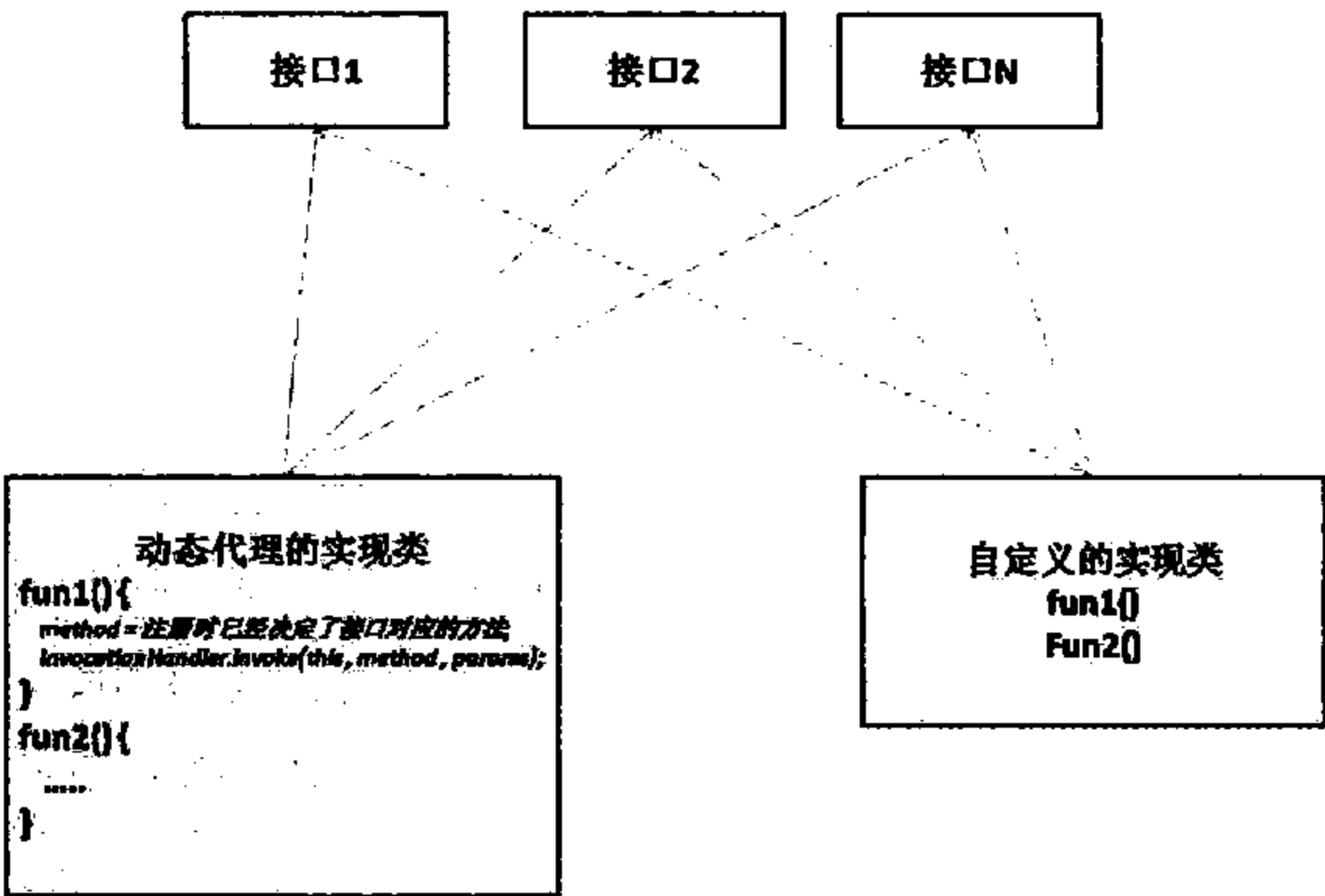


图 7-3 动态代理实现类与自定义实现类的关系

看懂该图的关系后,再结合动态代理中程序是基于接口的方法对象来调用具体实现类方法的,有些小伙伴会疑问 JVM 到底会不会做到这一点?非常好证明,用两条代码即可搞定,代码如下:

```
Method method = HelloInterface.class.getDeclaredMethod("setInfo",
    String.class ,String.class);
method.invoke(new HelloImpl(), "aaa" , "bbb");
```

看到这里,大家对动态代理的理解是否又回到了前面提到的一些基础知识上,所谓的万变不离其宗或许就是这个道理,随着代码的深入,就能看清很多本质所在。

在这里看到了动态代理的一些本质所在,那么还有什么需要总结的吗?

肯定有,前面已经有一些关于接口和类型的总结。看到这里再补充一下:虽然动态代理实现了 AOP,但是只能是外部程序通过接口调用相应的方法才会实现 AOP,如果在其内部方法再调用内部方法,是无法完成 AOP 切入的,为何?

根据图 7-3 解释这个道理,很明显,当内部方法相互调用时,此时的程序已经进入这个自定义类的相应方法中,如果再次调用其他的方法,就是直接调用,无须经过动态代理的实体类从外部绕一圈了。

如果使用字节码增强构建子类或直接修改类的字节码就不会存在这类问题。以子类来讲,由于对象实体最终返回的是子类的实例,只要子类覆盖了父类的方法,那么方法调用过程中就始终会调用子类的方法,即使是对象内部方法之间的调用也是如此。

再回到宏观使用上来,通常在程序运行时所看到的动态代理类的类名是\$Proxy 加上一个数字,以及一些统一的 package 前缀,这是动态代理程序中 JVM 为它们做的统一命名,再次说明这个类并不是自己所定义的类,如果要看自己所创建的对象,该怎么办?请看图 7-4。

variable	value
args	String[0] (id=29)
hello	\$Proxy0 (id=31)
h	AOPHandler (id=36)
target	HelloImpl (id=39)

图 7-4 代理类中的 h 就是 AOPHandler，点开后的 h 才是用户自定义类型对象

如果在断点跟踪的过程中发现有这样的类名，那么该程序必然使用了类似于代理的方式来实现，而不是使用创建子类的方式，类似 CGLib 通过创建子类的方式实现时，类名通常是会在原有类名后增加\$\$EnhancerByCGLIB\$\$字符串，再叠加一些随机数字字符串，这样是为了避免产生重复名称的类。

了解了 AOP 的基本原理后，大家现在再去看看 Spring 各个版本的 AOP 配置以及代码的编写，也许非常轻松，因为它怎么做都没有逃出这些基本的原理。

看到这里是不是很多问题开始清晰，接下来我们再来谈谈 ORM，也许你天天在用，可能你还有自己的理解，那么看看胖哥的理解，也许是一种交流。

7.2.3 ORM 基础

现在来探讨一下什么是 ORM。做过与数据库打交道的程序的小伙伴应该都知道有一些相关的框架，例如 Hibernate、ibatis 等，那么为何会有这些框架？或者说它们的好处在哪里？为何会有多种框架出现？

在早期做应用系统时，可能仅仅是操作数据库的几个表，做一些简单的处理，但是随着社会需求的发展，以及 IT 行业的飞速发展，各类商业需求越来越复杂，大家会在数据库中建立大量的二维表来满足需求，即使是小项目所需要建立的表也越来越多。这些开发人员在经历了越来越多的项目后，大家发现在一个项目中，非常多的时间用在了表单数据到程序的 JavaBean 再到数据库的字段上，开发人员每天大量的时间花在对数据库的每个表的查询、修改、删除、新增等动作上，因而反反复复编写每一个属性的映射关系代码，更加重要的是，项目在维护过程中，尤其是在复杂的商业环境下，多个系统之间可能还会共用一些表，当这些表的字段发生变化时，需要修改多少代码是不可控的，在业务做得比较多的情况下（不一定是复杂的业务），很少有人能讲清楚这里面到底有多少陷阱。

ORM 的出现正是希望将这一层抽象出来，抽象出来后对这些问题就有单独的管理模块和配置，ORM 的终极目标是让数据库操作对象化、简单化，不过就目前的 IT 行业发展来讲，这种设想有些过于理想，很多时候其实只能尽量减少那些反反复复的过程，使得这些烦琐的映射关系只做一次或者只做极少的次数，在这个基础上还可以通过一些变通的手段，让这些映射关系通过某些自动化工具帮助生成出来即可。

疑惑：有自动化工具来生成，那就不需要减少次数了，ORM 也没多大用途了？
答曰：其实大家不要忽略软件的巨大成本是在维护上，在开发阶段的代价相对还不是特别大，最终要面临的是各种复杂的需求对数据模型的挑战，进而导致数据库表结构的巨大变动，自动化工具可以帮助我们解决一些烦琐的问题，但是无法帮助我们抽象共用模块。另外，各个项目结构、数据类型的映射关系、框架体系等有所区别，通用的自动化生成工具无法完全满足所有的需求。

疑惑：为什么说 ORM 的终极目标过于理想，而只能尽量减少映射次数？

答曰：在目前的商业应用中，大多还是以二维表的关系型数据库为主，虽然出现了许多空间数据库，但并非有大家想象当中那么好，NoSQL 的出现也并非以解决商业逻辑为目标。通过二维表来表达各种各样的逻辑复杂关系的数据类型，就会投入人类的思想和设计理念，一种复杂的设计，在代码上自然不会像单纯的映射那样简单。

不过反过来看，一个应用系统中大部分逻辑并不是那么复杂的，绝大部分的业务可能只需要做一次映射即可。换句话说，既然叫 ORM，就必然存在对象到关系型数据库的 Mapping 过程，只是希望有了这一映射基础，相关的操作就不再去关心映射了，那么当数据库发生变更时，相应系统只需要修改一次映射即可，而不需要“到处去找代码”，到处去找 SQL，到处去找 UI，至少这些动作可以减少很多。

怎样？看了胖哥在这里自问自答，你对 ORM 的来源是否有了新的体会？

至于 ORM 的工具很多，它们各自对待问题的态度不一样，所希望解决问题的目标也不一样，所以最终的走向也不一样。虽然它们都可以实现类似的功能，但是注定了它们的血统和基因不同，胖哥认为它们没有好坏之分，更加没有对错之分，我们需要懂的是它们的性情，以便于做出较为合适的选择。

以大家口口称道的 Hibernate 为例，它是一个理想主义者，它希望操作数据库都是对象级别的，它希望表结构是根据对象来建立的，而不是通过表结构设计后再来设计对象，很多时候数据库的二维表也和 Hibernate 的对象有着相应的关系。

Hibernate 发展到现在，从配置到 Annotation 都做了非常多的改变和完善，在大多数不太复杂的业务逻辑场景下，Hibernate 确实是不错的选择，它在开发效率和维护代价上都做到了非常不错的地步。

Hibernate 为了提升性能也做了对各类缓存的处理，而且是分多级别的缓存，某些级别的缓存可以设置到远程的共享位置，而内在是由 Hibernate 自己来完成的。

也不得不承认它的许多弱点，在许多商业逻辑下，很多现实的设计模型并不会简单地与数据库表映射就可以将问题搞定，在层次关联中本身的关系可能会非常深，例如在复杂的嵌套和级联中（例如自嵌套表达树、图结构、循环嵌套等），要删除数据，将会是十分恐怖的事情。另外，在一些复杂的逻辑中做统计、合并等操作，只希望在几十个列中返回其中的少数列，这时我们就不得不用 SQL 去完成（例如，在一些数据库中包含了大量的 CLOB、BLOB 字段，在列表页面通常不会直接显示，否则会占用非常大的带宽。在字段不多且不包含大字段的情况下，影响并不大）。类似的场景还有很多，大家在实际的应用中就会遇到。

胖哥需要补充下自己的看法：社会在发展，世界在进步，今天的缺陷在明天不一定是缺陷，所以这里提到的仅仅是这个时代胖哥自己的见解。学会用客观的态度看待身边技术的发展，也许明天它就能将我们所提到的问题解决掉，异军突起。

再谈谈现在系统中使用非常广泛的 ibatis。ibatis 是十分轻量级的 ORM，它使用 resultMap 的方式或通过查询结果集的别名来达到映射的目的，SQL 编写仍然需要自己来完成，ibatis 可能认为这样非常灵活，可以自己去控制性能及 SQL 调优。

ibatis 也同时考虑到很多语句会有很多内容会重复出现，例如查询一个前台列表、详情、后台列表，可能 SELECT 部分的代码是一样的，区别在 WHERE 条件部分，那么它也提供了一种“include”机制。

ibatis 通过 dynamic 来完成动态条件输入判定，当然它不仅仅可以拼接条件，而且完全

可以把它当成一种动态 SQL 的拼装。动态条件的拼装使得业务代码无须关心这部分逻辑，ibatis 在这些方面都做得非常不错，虽然大家可能还是需要去写字段，但是此时只要设计合理，写字段的次数也会很少（其实 Hibernate 不论是用配置文件还是注解不也要写字段吗），因此大家还是愿意去写的。

总的来讲，它的可控性更强，但由于其松散性比 Hibernate 要强，因此更加要求编写代码的人员考虑到未来的可维护性，否则也可能会出现许多漏掉修改的地方。

最后我们来谈谈 Spring JdbcTemplate。很多人说 JdbcTemplate 也是 ORM，但是胖哥并不这么认为，要严格地说 ORM 的话，ibatis 也只能算半个，或者最多算大半个。JdbcTemplate 只是提供了 JDBC 连接的操作封装，简单的操作，无须程序员去关心获取 Connection、回收 Connection 等相关的操作。它也提供了一些类似于 RowMapper 的回调机制，即使这样所有的内容也都是自己在代码中完成的，包括 SQL 语句编写、动态参数拼接、判定逻辑、结果集回调组装对象都需要我们自己来编写。

JdbcTemplate 相对来讲更加松散，也许在回调部分可以抽象出一系列类来做回调映射关系，这样看起来有点像 ORM 了，但是这种松散是不可控的，也就是它是否会将回调部分的内容抽象出来是完全不可控的，程序员也无法判定自己抽象出来的回调是否会多余的。

如果没有任何抽象，仅仅是简单的回调，那么这种方式与直接字段取值就没有任何分别了。换句话说，ORM 最终要解决的依旧是抽象共享模块，这才算是一个真正的框架模型，而并非简单地进行逐个属性赋值。

也许用过 JdbcTemplate 的小伙伴会说：不做任何回调会映射为 Map 结构。没错，如果是做一种通用设计，那么用 Map 也无可厚非，这样会使得代码变得十分简单，甚至比 Hibernate 还要简单。但是这样的 Map 用法也是语义松散的，也许只有写这段代码的人或者仔细看过这段代码的人才知道这个 Map 里面到底放的是什么，每个属性的数据类型是什么，对它做的任何操作也将是裸奔方式，不能直接通过对象的方式对属性进行封装和修饰。这样说很抽象，我们来举个简单例子。

当提取出数据和写入数据时，需要对数据做许多的逻辑操作和扩展，就不得不又以某种函数模式来扩展，在 Java 中使用类似于 Utils 的方式来给它们贴补丁，但这也算是在某些项目中比较好用的一种方式。

通过 JdbcTemplate 操作的 SQL 可能会被写在代码中或其他某个指定的地方，但是动态条件也需要自己去写代码完成拼接，SQL 本身不具有可控性，程序员是否会写出有可能出现注入风险的 SQL 是很难判定的。此时的代码和 SQL 混在一起，当代码段十分复杂、SQL 十分复杂、拼接的条件十分复杂时，这段代码会越来越难以维护。

当然它也并非一无是处，例如做一些不算复杂的通用模型设计，而且系统中表结构的变化也不是那么频繁，尤其是一两个人做一个项目时，选择它也是十分简单的，至少在开发阶段不用写了代码又写配置文件，写了配置文件又返回来写代码。

另外，JdbcTemplate 框架虽然轻量，但为我们提供了许多基础连接操作的包装思想，很多时候我们只需要稍微变化下就可以在一些特定的场景中灵活地应用起来，我们也可以基于这种基础思想来解决一些特定的问题。

除了本节所提到的几种 ORM 框架外，在该领域中还有许多的开源框架以不同的思路出现，期望做到完美，包括 Java 语言本身也希望能有一天真正可以做到理想中的 ORM。

作为开发人员，面临许多选择，只要本着去解决 O 和 R 之间的耦合关系的基础思想，

以降低使用和维护成本为目的，那么胖哥认为它就可以是 ORM 的解决方案。也就是说，ORM 并不是某种技术，而是一种思想或者说一种导向，至于它是否必须使用某种框架，并不那么重要，而因地制宜的选择才是重要的。

框架并无对错，只有设计初衷和理念，要知道它的秉性与特征，才能做好选择，我们并非必须要选择哪一种，更不要拍脑袋去选择，而是要选择最适合自己的。

自己做框架扩展一样，并非要将自己的框架做得完美无缺、天衣无缝，因为最完美的事务往往是缺陷最多的，框架之所以存在，就是因为它并不是完美的，或许经过自己精心改造后的框架其模块的扩展性已经达到不可挑剔的程度，但可能在几年后发现这些模块根本不需要扩展，当时投入那么大的精力完全是一种浪费，但是编写好相应的框架后，使用框架的程序员还不得不按照这一套复杂的分层结构来完成自己的任务，那么这是完美还是缺陷呢？框架本身就是带有缺陷的，所以才有一代又一代的架构师去改造它，因为社会在发展、世界在不断变化，架构也随之改变。

开源框架也是如此，它也有缺陷，有很多问题它们无法解决，在这个时候如果寄托于官方本身对开源框架进行改造，周期会很长，通常实际问题没法等待，此时就应当是架构师通过智慧去改变现状的时候，这也应当是架构师所需要承担的重要职责之一，更是架构师的艺术行为之一，让程序员跑得更快，让程序员的性价比更高。相反，架构师并非是每天在程序员面前炫耀自己的架构是多么“牛”，然后让程序员在夜以继日地加班的同时还无法给客户带来多少价值的人，程序员可能会有苦说不出，还要“被逼”使用，而自己却认为理所当然。

胖哥有些牢骚，这些牢骚并非胖哥本人的，更多的是胖哥所见所闻的一些牢骚，对于这些事情，有些年轻的程序员也不知如何面对，即使在非常开放的一家公司工作，基层的员工也很难说出自己的这些想法，因为大多数程序员对架构师都有敬畏之心，这也是应该有的。胖哥只能说，如果有一天你成为了架构师，那么是依靠这份敬畏之心来满足自己还是尊重这份敬畏之心为程序员创造更好的条件？换句话说，“架构师”只是一个名号、一个称呼而已，并非一个称呼决定人之高低，应该更加理解对于这个称呼所需要承担的责任和需要付出的努力。

7.2.4 Annotation 与配置文件

Annotation 又名注解，在 JDK 1.5 以后，Java 开始从语法层面支持注解。刚开始时很多人仅仅是将它作为一种注释性质的东西或者用来去掉一些编译时检查，那么 Java 为何要有这种东西呢？它又能为 Java 的世界产生什么新的价值呢？你是否考虑过这些问题，如果不明白或许用了注解也不知道怎么回事。

为了说明这些问题，先来回顾一下早期的 Spring 或 Struts 的配置文件。如果需要编写一个业务逻辑请求，除了正常的页面代码和后台代码外，还需要配置许多配置文件，虽然框架中还会提供一些模式匹配的方式来减少配置，但是也同时降低了配置的可读性。

自从 JDK 1.5 语法支持注解后，许多开源框架开始为之做出改变，例如 SSH 体系的框架几乎都支持 Annotation。JVM 允许在 package、类、方法、属性、代码中进行注解，用得较多的是类、方法、属性。

注解是可以设定生命周期的，例如注解的生命周期设定为 `RetentionPolicy.RUNTIME` 时，代表该注解是运行时可以被程序读取到的。注解依附于类、方法、属性而存在，相当

于附属品，本身没有任何意义，程序中可以通过反射将注解读取出来，通过相应的代码给它们赋予实际的意义，所以首先不要将 Annotation 看得很神奇，神奇之处是外部程序根据注解所实现的功能。

胖哥先给 Annotation 泼点冷水，是希望大家认识到它是什么东西，因为太多人接触 Annotation 都是从一些框架入手的，认为这是一个神奇的东西，太过于高估了，导致无法了解它的真相，更不用说如何玩转 Annotation 甚至扩展自定义的 Annotation。

为了让大家更加形象地了解 Annotation，胖哥做一个试验，就是写一个自定义的 Annotation。假如没有框架，我们希望通过自定义 Annotation，然后通过这个自定义的 Annotation 包装出一些特殊的功能。

需求描述：假如没有 Hibernate，要实现一个单表的数据与对象的映射，这里假设通过 JDBC 已经拿到了类似于 K-V 的 Map 结构，此时要映射到一个 JavaBean 中。也就是说，需要写一个 Map 到 JavaBean 的映射。

问题和方法：数据库字段的命名规范是以下画线为单词分割符的，Java 采用大家公认的驼峰式命名规范，且部分名称并非可以直接拆分下画线转换为驼峰格式（因为许多属性会为了某些区分条件而被修改名字），此时我们希望能够像 Hibernate 那样在属性上编写属性注解来说明该属性对应的数据库表的字段名，否则默认将数据库表的字段名按照下画线拆分后转换为驼峰格式。

扩展：胖哥在 7.2.1 节中已经用反射做过类似的填充处理模拟，这里就不再对“自定义对象类型”进行处理了，即只使用一些简单类型，也不再用 Setter 方法给属性赋值，而是简单粗暴地通过 Filed 反射修改属性值。这也许不符合某些规范，但是大家需要知道这仅仅是为了说明一个注解的道理，而并非讲解注入（这也是一种碎片化学习的过程）。

首先写个自己的 Annotation，用在属性上，里面有个属性叫 name 且没有默认值，这代表某个属性引用到这个 Annotation 时必须填写 name 值，否则语法不能通过。

代码清单 7-7 编写一个自定义的 Annotation

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Column {

    String name();
}
```

大家看到 @interface 这个定义可能觉得有点陌生，它就像 class、interface 等类型标识符，是一个语法糖，是 JDK 1.5 为了专门标识 Annotation 增加的一种语法格式，编译器会识别到这是一种注解的定义。

然后将它用在一个 JavaBean 上，我们定义几个简单属性，然后为这些属性加上注解，并让部分属性没有注解。代码如下：

代码清单 7-8 将自定义的注解写到类的属性中

```
public class UserDO {
```

```

@Column(name = "name")
private String userName;

@Column(name = "title")
private String userTitle;

private int loginTimes;

private String empId;

.....setter and getter methods
}

```

接下来我们需要写一段公共程序，用于将数据库中对应的值填充到对象中。不过，在这里我们不想牵涉数据库连接操作，因此我们假定已经从数据库中获取到一行数据，并得到一个 Map 对象，这个 Map 对象的 Key 就是数据库字段，而 Value 是表中的值。因此这段公共程序的入口参数应当是这个 Map 对象，返回的是 UserDO。代码如下：

代码清单 7-9 将 Map 转换为自己的类

```

@SuppressWarnings("unchecked")
public static<T> T convertMapToBean(Map<String , String>row ,
                                   Class<T>clazz)
    throws InstantiationException, IllegalAccessException {
    Object object = clazz.newInstance();
    List<Field>list = findFields(clazz);
    for(Field field : list) {
        if (!Modifier.isStatic(field.getModifiers())
            && !Modifier.isFinal(field.getModifiers())) {
            if(!field.isAccessible()) field.setAccessible(true);
            Column columnAnnotation = field.getAnnotation(Column.class);
            if(columnAnnotation != null) {
                //通过 Annotation 注解的名称来获取对应的值
                String value = row.get(columnAnnotation.name());
                Class<?>fieldType = field.getType();
                if(fieldType == String.class) {
                    field.set(object, value);
                }else if(fieldType == Integer.class) {
                    field.set(object, getInteger(value));
                }else if(fieldType == int.class) {
                    field.setInt(object, getInt(value));
                }else if(fieldType == Long.class) {
                    field.set(object, getLongWrapper(value));
                }else if(fieldType == long.class) {
                    field.setLong(object , getLong(value));
                }/*
                更多的数据类型大家可以自己补充
            */
            }else {
                /*没有 Annotation 的代码交给大家自己去完成*/
            }
        }
    }
    return (T)object;
}

```

该类中还有许多其他的非关键性方法未一一列举，更多代码的细节请大家参看本书光盘中 src/chapter07/annotation/下的 ConversionService 类。

写好这段代码后，再写段小程序来测试。写一个小 List，先用 Map 存放数据，然后再转换为 UserDO 的模式，最后大家根据需要输出 List<UserDO>的内容。测试代码如下：

代码清单 7-10 测试封装的 Annotation

```
public class TestMain {

    public static void main(String []args)
        throws InstantiationException, IllegalAccessException {
        @SuppressWarnings({ "unchecked", "serial" })
        List<HashMap<String , String>>list = Arrays.asList(
            new HashMap<String , String>() {
                {
                    put("name" , "xieyuooo");
                    put("title" , "小胖");
                }
            },
            new HashMap<String , String>() {
                {
                    put("name" , "ffff");
                    put("title" , "标题 2");
                }
            }
        );
        List <UserDO>users = new ArrayList<UserDO>(list.size());
        for(HashMap<String , String> row : list) {
            users.add(ConversionService.convertMapToBean(row,
                                                            UserDO.class));
        }
        //这里可以将 users 列表输出
    }
}
```

我们在代码运行过程中的结束位置设置断点，可以看到 users 列表最终如图 7-5 所示。

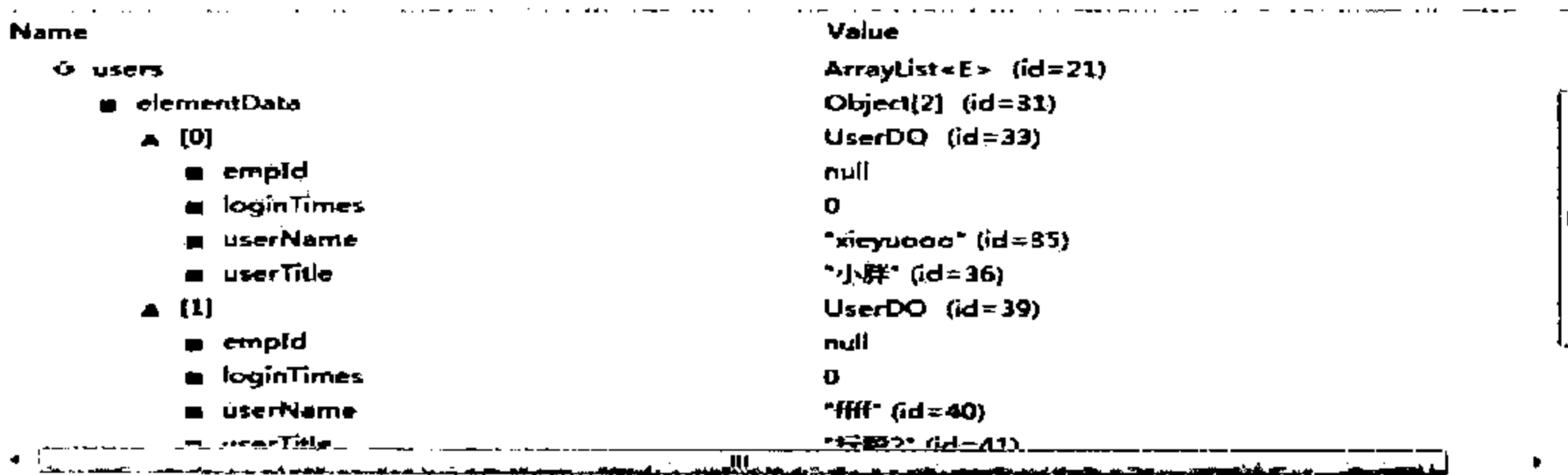


图 7-5 这里说明我们的数据已经被填充到 UserDO 中了

如果不理解 Annotation 是什么，现在胖哥希望你能动手将这段程序做一做，然后闭上眼睛仔细想一想，自定义的 Annotation 在哪里起了作用，是如何发挥作用的。不用在乎这段代码有多么强大的功能，只在乎你是否真的理解 Annotation 是用来做什么的。

如果有一天你理解了 Annotation 只是附带在对象上的附加产品，它本身不具有功能，功能是由外部程序来完成的，那么胖哥认为你真的理解了。

而回过头来我们 Annotation 对比一下配置文件，是否有什么坏处？

有的，配置文件是独立维护一些关系，例如 Struts 的配置文件就有请求路径与处理类和方法的映射关系，以及渲染页面的映射关系，这样只要你知道一个 URL，那么通过配置文件就能很快找到处理程序和渲染页面。

把这些东西转换成 Annotation，可能就不是那么容易了，因为注解是写在代码中的，没有集中式管理，Annotation 带来的是简化处理，不用我们在编程过程中刚写一小段业务

代码，又需要切换思维方式到文件中写配置信息，折腾了半天就是为了实现一个小功能，给需求方的感觉就是写个小功能为什么这么费劲。

在使用 Annotation 的过程中，如果你没有事先做好规划，那么最后的项目管理会比较麻烦，这时通常需要你真的记得代码在哪里，如果不记得了也许只有一个土办法可以帮助你，那就是全文匹配，可以将非 jar 包中的代码全部匹配出来。但在全文匹配中，或许会遇到一些变态的家伙，将名称的重复概率搞得很高，所以这只是减少痛苦，而没有解决痛苦。

如果我们在初期建设上选择用 Annotation，那么就要有一定的规范，包括目录和命名方面，在维护这个系统时，只需要遵循这个规范，就能很容易找到位置，并没有想象中的那么麻烦。换句话说，好不好用其实也要看使用的人。

了解了这些基本技术储备后，学习源码篇或许会轻松不少，但是代码中可能会有许多设计模式让你看得头晕脑胀，因此后面的路依然艰苦。在本书下册中，会提到许多和设计相关的内容，大家也可以参考其他有关设计模式方面的基础书籍来学习。

第 8 章

部分 JDBC 源码讲解

本章主要内容

- JDBC 通用接口规范
- JDBC 注册过程
- 建立 Connection
- 创建 Statement
- 连接池基本原理

本章讲解 JDBC 的部分源码。胖哥希望你学到的是看源码的思路和方法，而不希望大家过分纠结这些源码的功能细节，因为有可能根本不用到，并且本章也并不是对 JDBC 源码的诠释，只是通过阅读部分 JDBC 源码引导一些读取源码的方法。

8.1 JDBC 通用接口规范

要看 JDBC 源码，自然要看看 JDBC 的一些基本规范，在用纯 JDBC 编写代码时，Driver、Connection、Statement、Batch、setFetchSize、commit 与 rollback、setQueryTimeout 与 cancel、MaxRows、DataSource 这些常见的接口，不论用任何数据库驱动程序，它们都无时无刻地存在。

在这里可以反向来理解 Java 接口设计的初衷，曾经创造 Java 的 SUN 公司，希望对数据库的操作是简单的、统一的。为什么呢？由于各个数据库厂商对于数据库的内在设计，以及对外服务协议细节并不一样，这些交互细节还可能会随着数据库的升级而变化，因此 SUN 公司不太可能自己来实现每个数据库的交互程序，但又希望这些操作是通用的，该怎么办呢？

聪明的 SUN 公司定义了一种接口，或者说是一种规范，让厂商自己提供 Java 语言的驱动程序来实现这些接口和规范。从这一点上可以简单地理解为“JDBC 的驱动程序实现了通用接口，并屏蔽了程序底层与数据库服务器端的交互细节”。这一点类似于计算机上的各种插口，以及主板上的许多插槽，有通用的接口规范、尺寸等。如果要与其交互，只要适配接口即可，如果接口与其不一致，则需要采用转换口来完成，这样就可以保证在整体的设计中是清晰而简单的（这些内容将在本书下册的“设计篇”中讲解）。

Java 的基本思想之一就是这样的，基于接口的设计可以让整个设计变得简单，因为在设计接口过程中，重点关注 API 的功能是否能满足需求，而并非关注每一个 API 的实现细节，这样可以让设计者在设计过程中就将思维的重点落实到框架层面。反之，在设计中去思考每个细节的实现是十分痛苦的，会将设计者的思路引导到某些细节的点上，可能会在全局上有所疏忽，找不到主干。

这样可能会很矛盾，在设计中如果不思考细节如何去设计，而本书中还反复提到了要知根知底！

胖哥的回答是：所谓不要陷入太多的细节中是指不要在每一个小细节上过分纠结，关键细节是核心业务的问题，一个设计接口的人必然非常熟悉相关领域，所谓有经验也就是有相应领域中的一些总结，能把控一些核心问题的规律。在这个基础上加之设计思想的配合，通常可以将接口设计得不错。

JDBC 也是这样的，它就服务于程序员，在设计 JDBC 接口时，不考虑每种数据库该如何实现，而是如何让程序员可以简单、完善地操作数据库，这样程序员就不用关注底层的交互协议的细节。本着这样的基础，抽象绝大部分数据库服务的输入/输出的公共特性，设计出 API。

从设计者的角度来讲，首先考虑使用者的方式、方法，将其提炼与抽象，设计者会说：“这样操作是通用的、简单的、清晰的”。至于实现部分，在实际的场景中由应用程序员去扩展。

上面谈了 JDBC 的接口，下面就来看看 JDBC 的部分源码。

8.2 JDBC Driver 注册

在前文提到的第一个接口就是 Driver，全名为“java.sql.Driver”，也就是驱动的意思。首先从 Driver 注册开始说起，看看它是如何与业务代码衔接在一起的。

刚开始写 JDBC 程序时，我们通常都会使用下面两行代码来获取数据库的连接：

```
Class.forName("xxx.xxx.xxxx.xxx");
DriverManager.getConnection(url, username, password);
```

大家都知道这两行关键代码可以用来获取连接信息，那么是否考虑过为何在获取连接前要做一次 Class.forName() 操作呢？为何 DriverManager.getConnection(url, username, password) 就能获取到连接信息呢？再换一种问法：如果一个应用中既使用了 MySQL，又使用了 Oracle，或者还有其他的数据库，各种驱动程序套在一起会不会有问题呢？

胖哥对这些稀奇古怪的事情十分感兴趣，于是胖哥下载了 MySQL 与 Oracle JDBC 驱动的一些源码来参看（注意：胖哥下载的可能是某种版本的驱动，也许与你所下载的代码并不完全相同，但是由于我们是在学习一些源码阅读的方式，以及阅读源码后应当有什么样的思考，因此不用纠结用的到底是什么版本）。

要如何跟踪代码呢？由于 Class.forName() 不好跟踪，所以我们可以反过来下手，先跟踪 DriverManager.getConnection() 方法，跟踪进去会发现内部重载了多个方法，如图 8-1 所示。


```

●s getConnection(String url) : Connection - DriverManager
●s getConnection(String url, Properties info) : Connection - DriverManager
●s getConnection(String url, String user, String password) : Connection - DriverManager

```

图 8-1 获取连接的方法

经过跟踪图 8-1 中的几个方法会发现，它们经过一些处理后，最终都调用了内部的一个私有的 getConnection()方法，此方法的描述如下：

```

private static Connection getConnection(String url,
                                       java.util.Properties info,
                                       ClassLoader callerCL)
    throws SQLException;

```

相对图 8-1 中的几个方法描述，它增加了一个 ClassLoader 类型的入参，这个对象默认是传入当前线程的 ClassLoader。下面来看看这个私有方法里面比较关键的一部分代码，如图 8-2 所示。

```

synchronized (DriverManager.class) {
    // use the readcopy of drivers
    drivers = readDrivers;
}

// Walk through the loaded drivers attempting to make a connection.
// Remember the first exception that gets raised so we can reraise it.
SQLException reason = null;
for (int i = 0; i < drivers.size(); i++) {
    DriverInfo di = (DriverInfo)drivers.elementAt(i);

    // If the caller does not have permission to load the driver then
    // skip it.
    if ( getCallerClass(callerCL, di.driverClassName ) != di.driverClass ) {
        println("    skipping: " + di);
        continue;
    }
    try {
        println("    trying " + di);
        Connection result = di.driver.connect(url, info);
        if (result != null) {
            // Success!
            println("getConnection returning " + di);
            return (result);
        }
    } catch (SQLException ex) {
        if (reason == null) {
            reason = ex;
        }
    }
}
}

```

图 8-2 getConnection()的最终操作关键代码

在这段代码中，首先将 DriverManager.class 锁住，然后开始进行操作。其中局部变量 drivers 来自于 readDrivers 的赋值，那么 readDrivers 又来自哪里呢？通过代码可以发现，这是一个 Vector 类型的集合类对象（在 JDK 1.7 中换成了 CopyOnWriteArrayList）。

readDrivers 本身的值来自于 registerDriver(Driver driver)、deregisterDrive(Driver driver)方法中，这两个方法会对 DriverManager 中的另一个 Vector 对象（writeDrivers）写数据，然后通过 writeDrivers 克隆赋值给 readDrivers。这样做的主要目的自然是为了防止对相关操作的并发，在 JDK 1.7 中换成了 CopyOnWriteArrayList 就没有这些问题了，只有一个集合类。

总地来讲，在 JDK 1.6 及以前的版本中，DriverManager 中的 readDrivers、writeDrivers

理论上包含的值应该是一样的。不相信？下面就来看看 registerDriver(Driver driver)的代码，如图 8-3 所示。

```
public static synchronized void registerDriver(java.sql.Driver driver)
throws SQLException {
    if (!initialized) {
        initialize();
    }

    DriverInfo di = new DriverInfo();

    di.driver = driver;
    di.driverClass = driver.getClass();
    di.driverClassName = di.driverClass.getName();

    // Not Required -- drivers.addElement(di);

    writeDrivers.addElement(di);
    println("registerDriver: " + di);

    /* update the read copy of drivers vector */
    readDrivers = (java.util.Vector) writeDrivers.clone();
}
}
```

图 8-3 这里就是注册 Driver 的地方 (JDK 1.6 及以前版本的代码)

大家应该都在想一个问题：readDrivers 这个集合类初始化时应该是没有任何数据的，或者说驱动列表中没有任何 Driver 信息，DriverManager 本身并不知道驱动程序在哪里（或许它们都实现了 Driver 接口，但是 DriverManager 不会去主动扫描所有 jar 包的所有类是否实现了 Driver 接口）。另外，通过图 8-2 看到，程序中需要通过某个 Driver 的 connect(String, Properties)方法来返回 Connection 对象。此时思路有点混乱，我们将问题分解为如下两个。

- ◎ readDrivers 中的数据是从哪里来的？换句话说，驱动列表是从哪里来的？
- ◎ connect(String, Properties)方法大概做了些什么？

通过图 8-3 发现，要向 readDrivers 中添加驱动的实例，需要通过 registerDriver(Driver driver)方法，它是在哪里调用的呢？此时回想一下代码中的 Class.forName("xxx")，或许与它关系密切，为此我们打开 JDBC 的源码来看看究竟。

首先我们打开 Oracle JDBC 的源码，从哪里开始看呢？用 Oracle 加载驱动通常是这样写的：Class.forName("oracle.jdbc.driver.OracleDriver")，这个类应该就是入口了，打开这段代码后，大概浏览一番后便看到了关键部分，如图 8-4 所示。

```
static
{
    m_driverAccess = new Properties();
    m_driverAccess.put("thin-server", "oracle.jdbc.thinserver.ServerTTC7Protocol");
    m_driverAccess.put("oci-server", "oracle.jdbc.ociserver.ServerOCIDBAccess");
    m_driverAccess.put("thin", "oracle.jdbc.ttc7.TTC7Protocol");
    m_driverAccess.put("oci8", "oracle.jdbc.oci8.OCIDBAccess");
    m_driverAccess.put("oci", "oracle.jdbc.oci8.OCIDBAccess");
    m_driverAccess.put("kprb", "oracle.jdbc.kprb.KprbDBAccess");
    m_defaultDriver = null;
    Timestamp timestamp = Timestamp.valueOf("2000-01-01 00:00:00.0");
    try
    {
        if(m_defaultDriver == null)
        {
            m_defaultDriver = new OracleDriver();
            DriverManager.registerDriver(m_defaultDriver);
        }
    }
    catch(RuntimeException _ex) { }
    catch(SQLException _ex) { }
}
```

图 8-4 在这里我们看到了注册的 OracleDriver

这段代码是较低版本的 Oracle 驱动程序，新版本的 Oracle 驱动程序代码会有所区别，这一点大家在自己看源码时需要注意。

通过图 8-4 可以发现，Oracle JDBC 调用 DriverManager 注册驱动的代码在 OracleDriver 类的 static 块中，所以当程序中运行 Class.forName("oracle.jdbc.driver.OracleDriver")时会直接调用该代码。如果 new 一个相应的 Driver，也会运行这段代码，那么就相当于实例化了两个驱动类，这并没有什么问题，Driver 只是提供了一些方法而已，不过大家要知道是这样一回事。

另外，应该注意到这段代码只需要执行一次，所以一个驱动只需要做一次 Class.forName()操作，不需要每次请求都要去做，当然每次都去做对系统性能也没有太大的影响。

MySQL 的 Driver 也应该是这样的吧？我们带着一丝小希望和一些好奇心看看 MySQL 的 Driver 是怎么样的。

同样的，Class.forName 中所加载的 MySQL 驱动类路径是 com.mysql.jdbc.Driver，打开源码后果然如我们所料，它也是写在 static 块里面的，如图 8-5 所示。

```
static {
    try {
        java.sql.DriverManager.registerDriver(new Driver());
    } catch (SQLException E) {
        throw new RuntimeException("Can't register driver!");
    }
}
```

图 8-5 将 MySQL 的驱动注册到 DriverManager

如何将驱动加载到 DriverManager 不用胖哥多说了吧，为了让大家更好地做实验，胖哥可以给大家两个简单的方法。

- ◎ 使用 DriverManager.setLogWriter(new PrintWriter(System.out))让 DriverManager 的操作可以输出一些日志，例如在注册驱动、获取连接、获取驱动等方法被调用时都会打印出相关的日志。
- ◎ 通过 DriverManager.getDrivers();来获取 Drivers 的迭代器，然后输出这些 Drivers 信息。除此之外，也可以通过某个指定的 Driver 直接获取数据库连接，这样就无须 DriverManager 每次都遍历驱动列表了。

从目前的分析结果来讲，至少可以证明的就是在同一个应用中可以加载多个不同类型的数据库的驱动程序，甚至在不同的 ClassLoader 中可以加载同一种数据库不同版本的驱动程序，这就是通过源码所能看到的一些东西。那么只是学到这一点吗？不是，如果仔细阅读源码，就会发现这段代码中有许多地方用全局锁，需要遍历尝试。虽然从全局锁这个角度已经做了许多粒度优化，但是有不少的开销。难道这个问题有解吗？当然有，因为程序是自己写的，所以有哪些 Driver 自己是很清楚的，如果可以自己处理好驱动列表或枚举，那么就没有必要让 DriverManager 帮助去遍历和加锁了，根据数据库类型直接选择 Driver 即可直接使用。

看了过后突然发现代码有挺多地方可以改改，不用着急，等有了一个相对全面的了解后再去修改代码也不迟。

还有一个问题没有想清楚：如果系统中存在多个数据库的驱动，该如何决定选择哪一个驱动？

在前面已经提到了使用的方法是循环每个 Driver，然后调用其 connect()方法获取 Connection 对象，循环中的每一个 Driver 都会去调用 connect()方法，如果返回值为 null，则会继续循环，很显然能否获取到连接是由每一个 Driver 自己决定的。

此时迫不及待地想看看 Driver 中的 connect()方法到底做了些什么，我们依旧先看看 Oracle 的驱动中提供的 connect()方法（这段代码很长，这里只截取部分代码），如图 8-6 所示。

```
public Connection connect(String jdbcUrl, Properties properties)
    throws SQLException
{
    if(jdbcUrl.regionMatches(0, "jdbc:default:connection", 0, 23))
    {
        String s1 = "jdbc:oracle:kprb";
        int j = jdbcUrl.length();
        if(j > 23)
            jdbcUrl = s1.concat(jdbcUrl.substring(23, jdbcUrl.length()));
        else
            jdbcUrl = s1.concat(":");
        s1 = null;
    }
    int i = oracleAcceptsURL(jdbcUrl);
    if(i == 1)
        return null;
    if(i == 2)
    {
        DBError.throwSQLException(67);
        return null;
    }
    Hashtable hashtable = parseUrl(jdbcUrl);
    if(hashtable == null)
        return null;
}
```

图 8-6 connect()方法部分代码

大家可能会觉得奇怪，这个方法里面不就是在截取字符串吗？说得没错，这段代码就是将 JDBC URL 上的字符串截取下来放在一个 Hashtable 里面，逻辑上十分重要的是 oracleAcceptsURL()方法，如果它的返回值是 1，则会直接返回 null。oracleAcceptsURL()的代码如图 8-7 所示。

```
private int oracleAcceptsURL(String jdbcUrl)
{
    int i = jdbcUrl.indexOf(':');
    if(i == -1)
        return 1;
    int j = jdbcUrl.indexOf(':', i + 1);
    if(j == -1)
        return 1;
    if(!jdbcUrl.regionMatches(true, i + 1, "oracle", 0, j - (i + 1)))
        return 1;
    int k = jdbcUrl.indexOf(':', j + 1);
    if(k == -1)
        return 2;
    String protocol = jdbcUrl.substring(j + 1, k);
    return !protocol.equals("oci8") && !protocol.equals("oci")
        && !protocol.equals("thin") && !protocol.equals("kprb")
        && !protocol.equals("ultra") ? 2 : 3;
}
```

图 8-7 oracleAcceptsURL()的代码

这段代码是解析 JdbcURL 的代码，依旧可以明显地看出如果解析 JDBC URL 出现了一些问题，则会返回 1；如果返回 2，则被认为是 Oracle 的驱动程序所能识别的 URL，但是 JdbcURL 依然存在问题（例如协议或不完整）；如果返回 3，则代表已经正确获取到相关信息。

此时返回到图 8-6，可以发现对该方法的返回值进行了判定，如果结果为 1，则返回 null；如果为 2，则抛出异常。

根据刚才的分析，返回 1 代表 Oracle 驱动认为“这个 JDBC URL 不是自己可以解析的”，

那么就返回 `null`，这样在 `DriverManager` 循环的过程中就知道了需要去找下一个驱动程序。

如果返回 2，则认为这是一个 Oracle 的驱动连接，但是由于格式不完整或者协议无法匹配，会抛出 `java.sql.SQLException` 异常。

现在可以简单总结一下 `DriverManager` 与每一个 `Driver` 的关系：

每个 `Driver` 如果认为这个 `JdbcURL` 自己无法处理，那么就返回 `null` 或抛出异常，所以 `DriverManager` 就可以通过这样的方式来决定是否继续循环尝试下一个驱动程序，如果获得了合适的驱动程序就不会再循环了。

不同的驱动程序，它们的 `JdbcURL` 写法通常有较大的区别，因此这就成了 JDBC 选择驱动的关键因素，如果我们自己要为一种数据库做一个驱动，就需要考虑清楚这个 `JdbcURL` 的规则。如果格式上和此数据库驱动十分相似，而且系统中确实使用了对应关系型数据库的驱动，那么就有可能导致驱动程序被混用。

关于 Oracle JDBC 可以继续跟踪 `parseUrl()` 方法会发现 Oracle JDBC 的用户名、密码可以用多种方式传入，例如可以写成这样：`jdbc:oracle:thin:username/password@ip:port:sid`，驱动程序也会认为是正确的 `JdbcURL`。

当你知道了一种数据库的 `JdbcURL` 格式后，还可以通过 `DriverManager.getDriver(String)` 来查找对应的驱动程序，并且传入的 `JdbcURL` 无须使用 IP、PORT、SID 的具体值，只需要让格式与相应的驱动要求格式一致就能获取到正确的 `Driver`。该方法也是循环使用 `Driver`，不过是调用 `Driver` 的 `acceptURL()` 方法来判定该 `Driver` 是否接受这个 URL，其返回的是一个 `boolean` 值（跟踪源码可以发现 Oracle 的 `acceptURL()` 方法是通过间接调用前文中提到的 `oracleAcceptsURL()` 方法来间接完成的）。

代码之间的调用就是这样的，阅读源码时大家理清一条线后，就要想到相关的东西，虽然这个过程会很繁杂，但是可以搞清楚许多事情。只要搞清楚了，再看代码就不那么麻烦了。

这里没有给出代码调用关系图，是因为胖哥认为，我们不是看代码关系图的结论，而是要学会在杂乱的代码中逐步理清自己的思路，代码调用关系图其实只是从某个角度来阐述，也通常只能阐述某些版本的局部逻辑，更为细节的知识都需要我们自己去领悟。

接下来再来印证 MySQL 是不是有类似的处理方式。

MySQL 这部分方法放在了相应 `Driver` 的父类 `com.mysql.jdbc.NonRegisteringDriver` 中，它的 `connect()` 方法又是怎么写的呢（见图 8-8）？它的 `acceptURL()` 又是怎么回事呢？

在这段代码中先判定了 URL 是否为空，然后进行了 URL “非大小写敏感的前缀判定”，如果成立则返回相应的内容。那么判定前缀是什么呢？它们分别是：

```
private static final String REPLICATION_URL_PREFIX
    = "jdbc:mysql:replication://";
private static final String LOADBALANCE_URL_PREFIX
    = "jdbc:mysql:loadbalance://";
```

也就是说，如果传入的 `JdbcURL` 与这两者之一匹配，那么就会返回相应驱动的连接。奇怪？这两个默认的 `JdbcURL` 前缀都不是日常工作用到的前缀。

不急，这里只是一个最开始判定，后面还有更多的代码呢（其实 MySQL 这部分代码写得很烂），还有 `parseURL()` 方法，如图 8-9 所示。

```
public java.sql.Connection connect(String url, Properties info)
    throws SQLException {
    if (url != null) {
        if (StringUtils.startsWithIgnoreCase(url, LOADBALANCE_URL_PREFIX) {
            return connectLoadBalanced(url, info);
        } else if (StringUtils.startsWithIgnoreCase(url,
            REPLICATION_URL_PREFIX) {
            return connectReplicationConnection(url, info);
        }
    }

    Properties props = null;

    if ((props = parseURL(url, info)) == null) {
        return null;
    }

    if (!"1".equals(props.getProperty(NUM_HOSTS_PROPERTY_KEY)) {
        return connectFailover(url, info);
    }
}
```

图 8-8 MySQL 的 connect()方法的前半部分代码

```
public Properties parseURL(String url, Properties defaults)
    throws java.sql.SQLException {
    Properties urlProps = (defaults != null) ? new Properties(defaults)
        : new Properties();

    if (url == null) {
        return null;
    }

    if (!StringUtils.startsWithIgnoreCase(url, URL_PREFIX)
        && !StringUtils.startsWithIgnoreCase(url, MXJ_URL_PREFIX)
        && !StringUtils.startsWithIgnoreCase(url,
            LOADBALANCE_URL_PREFIX)
        && !StringUtils.startsWithIgnoreCase(url,
            REPLICATION_URL_PREFIX)) { //NON-NLS-15

        return null;
    }
}
```

图 8-9 parseURL()方法的前半部分代码

通过代码截图 8-9 可以看出，对于 JdbcURL 的判定，还使用了常量 URL_PREFIX、MXJ_URL_PREFIX，它们的定义如下：

```
private static final String URL_PREFIX = "jdbc:mysql://";
private static final String MXJ_URL_PREFIX = "jdbc:mysql:mxj://";
```

URL_PREFIX 就是代码中最常用的 JdbcURL 前缀，当这些前缀都不匹配时，MySQL 的 JDBC 驱动认为自己处理不了，返回 null，否则继续向下解析 URL（继续向下解析 URL 的代码十分长，也不是加载驱动的关键，所以这里就不一一列举了）。

小伙伴们可以继续跟踪 acceptURL()方法，会发现它通过调用 parseURL()方法，并通过它的返回值是否为 null 来判定驱动是否接受这个 URL，这样就与 Oracle 的 JDBC 逻辑类似，并验证了 DriverManager 的统一逻辑。

胖哥对本小节做一个简单总结：驱动只需要加载一次，不需要反复加载，也不需要自己 new。程序中通过 DriverManager.getDriver(String url)就能得到相应的驱动，可以将它保存在某个地方，这样可以节约很多遍历和锁的开销。另外，DriverManager 没有并发问题，一个应用中可以同时拥有多个 Driver，不同的 ClassLoader 也可以有不同版本的 Driver。

这是一种方法，也正是前面提到的：不需要看的代码就不看，只看想要看的代码。而且小伙伴们也不用着急，后续关于 URL 解析和创建 Connection 与 Driver 的驱动注册与选择关系不大，等真正提到这一点时，再来说明就会清晰多了。



8.3 创建 Connection

接下来简单说说 JDBC 建立通信的过程。同样以 Oracle JDBC 为例，回到前面提到的 connect()方法，我们来看看 OracleDriver 中的 connect()方法的更多细节。

在图 8-6 中重点标注了一个 parseUrl()方法，前文中没有说明这个方法也是负责解析 JdbcURL 的，在它之前不是有一个方法 oracleAcceptsURL()已经解析了吗？为何还要有一个方法来解析 URL？其实 oracleAcceptsURL()方法仅仅是判定 URL 的格式是否符合 Oracle 的 JDBC 要求，还没有真正地去解析一个 URL 参数细节，而 parseUrl()方法会将 JdbcURL 解析为 K-V 结构，代码如图 8-10 所示。

```
private Hashtable parseUrl(String jdbcUrl)
    throws SQLException
{
    Hashtable hashtable = new Hashtable(5);
    int i = jdbcUrl.indexOf(':', jdbcUrl.indexOf(':') + 1) + 1;
    int j = jdbcUrl.length();
    if(i == j)
        return hashtable;
    int k = jdbcUrl.indexOf(':', i);
    if(k == -1)
        return hashtable;
    hashtable.put("protocol", jdbcUrl.substring(i, k));
    int l = k + 1;
    int i1 = jdbcUrl.indexOf('/', l);
    int j1 = jdbcUrl.indexOf('@', l);
    if(j1 > l && l > i && i1 == -1)
        return null;
    if(j1 == -1)
        j1 = j;
    if(i1 == -1)
        i1 = j1;
    if(i1 < j1)
    {
        hashtable.put("user", jdbcUrl.substring(l, i1));
        hashtable.put("password", jdbcUrl.substring(i1 + 1, j1));
    }
    if(j1 < j)
        hashtable.put("database", jdbcUrl.substring(j1 + 1));
    return hashtable;
}
```

图 8-10 parseUrl()方法将 JdbcURL 解析为 K-V 结构

从这段代码可以看出，在 JdbcURL 中也同样可以设置用户名和密码，但建议从参数中设置，那样方便做单独的加密处理。另外，在这段代码的最后一行，即写入到 hashtable 中的 database 大家也要留意一下，后文中会详细讲解它。

总之，在这段代码中，解析 URL 后会向 hashtable 中写入 user、password、database 几个参数。现在就可以回到调用 parseUrl()的地方了，也就是前文中提到的驱动程序 (Driver) 的 connect()方法中，如图 8-11 所示。

在这段代码中，JdbcURL 的参数与 Properties 中传入的参数进行了合并。代码中的变量 database 包含了 Oracle 数据库的 IP 地址、端口、SID 信息，其格式通常为 IP:PORT:SID，也可能设置为 Oracle TNS 格式的数据库实例描述。

接下来继续看方法中后续的一些源码，会发现这里有许多参数可以进行设置，例如 JDBC 调优当中比较重要的参数 fetchSize，在获取连接时就可以做统一的默认处理，如图 8-12 所示。

```
Hashtable hashtable = parseUrl(jdbcUrl);
if(hashtable == null)
    return null;
String user = properties.getProperty("user");
String password = properties.getProperty("password");
String database = properties.getProperty("database");
if(database == null)
    database = properties.getProperty("server");
if(user == null)
    user = (String)hashtable.get("user");
user = parseLoginOption(user, properties);
if(password == null)
    password = (String)hashtable.get("password");
if(database == null)
    database = (String)hashtable.get("database");
String protocol = (String)hashtable.get("protocol");
properties.put("protocol", protocol);
```

图 8-11 这里合并了参数 Properties 与解析 JdbcURL 后参数

```
String defaultRowFetch = properties.getProperty("prefetch");
if(defaultRowFetch == null)
    defaultRowFetch = properties.getProperty("rowPrefetch");
if(defaultRowFetch == null)
    defaultRowFetch = properties.getProperty("defaultRowPrefetch");
if(defaultRowFetch != null && Integer.parseInt(defaultRowFetch) <= 0)
    defaultRowFetch = null;
```

图 8-12 fetchSize 在创建 Connection 层可以这样设置

通过这段代码能发现这些不同的参数名似乎都在设置同一个内在参数，并且有着优先级的关系，对于外部写程序的人来讲，只需要设置其一即可解决问题，如果混杂编写可能会带来一些奇怪的问题。同样的，如果你遇到了类似的问题，要学会从多个冲突参数来进行排查问题。

在创建 Connection 后，JDBC 会自动根据这个参数设置相关的值，如图 8-13 所示。

```
Connection connection = getConnectionInstance(protocol, jdbcUrl, user, password, database, properties);
if(defaultRowFetch != null)
    ((oracle.jdbc.driver.OracleConnection) connection).setDefaultRowPrefetch(Integer.parseInt(defaultRowFetch));
```

图 8-13 对创建的 Connection 设置一些默认参数

Oracle 也会在创建连接时，通过 OracleConnection 类的 initialize()方法设定一个初始化参数（换句话说，如果不设置，就会采用默认参数），例如 fetchSize 参数的初始化值是 10，相应的关键代码片段为：

```
default_row_prefetch = DEFAULT_ROW_PREFETCH;
```

相应的参数准备好以后，便可以开始创建连接了。上一节中我们提到，在 Oracle 驱动注册的过程中，有一个模式与协议的关系在驱动类初始化时写到一个叫 m_dirverAccess 的 K-V 结构中了，现在要创建实际的连接就需要从这个里面取出相应的处理类，如图 8-14 所示。

```
String protocolServer = null;
if(protocol.equals("thin") && System.getProperty("oracle.jserver.version") != null)
    protocolServer = "thin-server";
else
if((protocol.equals("oci8") || protocol.equals("oci")) && System.getProperty("oracle.jserver.version") != null)
    protocolServer = "oci-server";
else
    protocolServer = protocol;
String protocolClassName = (String)m_dirverAccess.get(protocolServer);
if(protocolClassName == null)
    DBError.throwSQLException(67, "Invalid protocol " + protocol);
DBAccess dbaccess = null;
try
{
    dbaccess = (DBAccess)Class.forName(protocolClassName).newInstance();
}
```

图 8-14 协议选择部分的代码

大多数应用程序中会使用 thin 协议,不过应用系统中通常并未设定 `oracle.jsrver.version` 参数的值,因此该代码通常不会返回“thin-server”,更加不会返回“oci-server”,逻辑上会直接将传入的协议名“thin”作为协议的关键字,通过 `m_driverAccess.get(protocolServer)` 获取协议的处理类,获取到相应的处理类以后,通过 `Class.forName().newInstance()`来实例化一个协议处理对象,赋值给本地变量 `dbaccess`。这个对象后文中会用到,大家注意了。

现在回过头来看看这个 `m_driverAccess` 是在哪里被初始化的,见图 8-15。

```
static
{
    m_driverAccess = new Properties();
    m_driverAccess.put("thin-server", "oracle.jdbc.thinserver.ServerTTC7Protocol");
    m_driverAccess.put("oci-server", "oracle.jdbc.ociserver.ServerOCIDBAccess");
    m_driverAccess.put("thin", "oracle.jdbc.ttc7.TTC7Protocol");
    m_driverAccess.put("oci8", "oracle.jdbc.oci8.OCIDBAccess");
    m_driverAccess.put("oci", "oracle.jdbc.oci8.OCIDBAccess");
    m_driverAccess.put("kprb", "oracle.jdbc.kprb.KprbDBAccess");
    m_defaultDriver = null;
    Timestamp timestamp = Timestamp.valueOf("2000-01-01 00:00:00.0");
    try
    {
        if(m_defaultDriver == null)
        {
            m_defaultDriver = new OracleDriver();
            DriverManager.registerDriver(m_defaultDriver);
        }
    }
}
```

图 8-15 这个列表是驱动注册前被写入的

这样代码似乎与前文关联上了。

在看代码的过程中,虽然胖哥建议先只看自己想要看的代码,不需要关注那些和问题不相关的代码,但是如果发现是“不是特别耽误思维过程”的代码,而且这类代码是配置属性类的,则可以留个简单的印象,为看其他代码埋下伏笔。例如:在前文中看驱动注册代码时,注意到有协议名和类的对应关系,而在这里开始用到它,这就是我们一直反复提到的“灵感”。

了解到驱动程序后,现在回到图 8-13 中,可以发现 `getConnectionInstance()`方法是获取连接的关键,因此继续跟踪 `getConnectionInstance()`方法。在其后续的代码中会实例化一个 `OracleConnection` 类,全名为 `oracle.jdbc.driver.OracleConnection`,实例化时会传入 `dbaccess` (前文中提到这是协议的实例)。

现在追溯到 `OracleConnection` 的构造方法,其内部经过一系列处理后,通过调用传入的 `dbaccess` 对象的 `logon()`方法进行处理(确切地说是将传入的 `dbaccess` 赋值给 `OracleConnection` 的一个属性,再通过这个属性来调用 `logon()`方法)。通过方法名基本猜测出这是登录数据库用的方法,此时就应当追溯到 `TTC7Protocol` 这个协议类当中,因为我们已经知道 `dbaccess` 对象就是该类的一个实例(跟踪到接口不用害怕,因为它一定有一个实体类,理清楚关系就知道是谁实现的)。

目前看到的 `TTC7Protocol` 还只是上层协议,继续跟踪代码会发现它本身也是一层层抽象的,因为驱动程序在与 Oracle 服务器端通信时,除了传递 SQL 外还有许多其他的信息和状态,它们都被封装在协议里面,每一层协议做每一层协议的包装,实现每个层次的模块化、代码的简单化。

在 `TTC7Protocol` 中的 `logon()`方法会再通过 `connect()`方法继续调用 `oracle.net.ns.NSProtocol` 的 `connect()`方法,该方法内部组装好相应的连接参数后,会调用 `establishConnection()`方法来进一步做底层的操作,如图 8-16 所示。

```
private SessionActs establishConnection(String database)
    throws NetException, IOException
{
    sActs.cOption = addrRes.resolveAndExecute(database);
    sActs.ntInputStream = sActs.cOption.nt.getInputStream();
    sActs.ntOutputStream = sActs.cOption.nt.getOutputStream();
    sActs.setTDU(sActs.cOption.tdu);
    sActs.setSDU(sActs.cOption.sdu);
    sActs.nsOutputStream = new NetOutputStream(sActs, 255);
    sActs.nsInputStream = new NetInputStream(sActs);
    return sActs;
}
```

图 8-16 NSProtocol 类中的 establishConnection()方法

在这段代码中，比较关键的是第一个 resolveAndExecute()方法，看名称或许可以猜到它应该是“分析和执行”的意思。可以将代码反过来推导：在 resolveAndExecute()方法执行后，就可以拿到 InputStream 和 OutputStream 对象了，换句话说，是拿到与数据库通信的输入/输出流对象了，那么 resolveAndExecute()方法中必然已经创建了连接（这是看源码的推断方式之一，代码不是你写的，也不清楚每段代码调用的意思，某些时候这样推断也会八九不离十）。

跟踪到 resolveAndExecute(database)方法内部，会发现其内部解析代码有一个核心处理是处理前文中所提到的 JdbcURL 中@符号后面的部分，也就是 database 串（可以传入 IP:PORT:SID 或者 TNS 描述信息），假如此时按照常规的 IP:PORT:SID 传入，则会调用内部的 resolveAddr()方法来进行分解，它最终会将连接地址修改为与 TNS 描述类似的方式，如图 8-17 所示。

```
connoption.addr = "(ADDRESS=(PROTOCOL=tcp) (HOST=" + connoption.host + ") (PORT=" + connoption.port + "))";
cs.addOption(connoption);
```

图 8-17 最终 JDBC 是用这样的地址来描述的

这段代码中的 connoption 变量的类型是 oracle.net.nt.ConnOption，cs 就像一个集合类，可以添加 oracle.net.nt.ConnOption 类型的对象。最后代码中会调用 cs 的 execute()方法，并间接调用 ConnOption 的 connect()方法来完成。跟踪进去看看其方法，如图 8-18 所示。

```
public ConnOption()
{
    conn_data = new StringBuffer();
}

public void connect()
    throws IOException
{
    nt = getNT();
    nt.connect();
}

private NTAdapter getNT()
    throws NetException
{
    try
    {
        nt = new TcpNTAdapter(addr);
    }
    catch(NLEException _ex)
    {
        throw new NetException(501);
    }
    return nt;
}
```

图 8-18 还调用了一层协议，不过看到了 Tcp 关键字

竟然还有一层！还有一层 connect 的调用！

不过，今我们惊喜的是，看到了 Tcp 关键字就知道离真相不远了，看看 TcpNTAdapter 类的 connect()方法便知道真相了，代码如图 8-19 所示。

```
public void connect()
    throws IOException
{
    socket = new Socket(host, port);
}
```

图 8-19 看来它也是创建一个 Socket，带入 host 和 port

在这里终于看到通信中 Socket 连接的代码了，是不是有点亲切啊！

Oracle JDBC 接下来的代码就是对 Socket 套接字进行层层包装，并通过该套接字与 Oracle 数据库进行一些简单的通信，例如发送 package 给 Oracle 服务器端，Oracle 服务器端返回一些数据库状态信息、版本信息、字符集信息等。更多的细节在这里不再详述，有兴趣的小伙伴可以自己跟踪。

创建连接部分的主干代码就这些，如果大家在第一次看这部分代码的过程中过于纠结“字节是如何发送与接收的”、“协议字符串是如何拆分与解析的”等细节，那将会十分痛苦，如果不是在解决问题或者要专门研究细节，则不需要阅读得那么细致。

换句话说，“细节”到底细到什么程度？当关注主干时，许多细节只需要知道是这样处理的即可，当真的需要了解细节时可以快速找到入口。

大家是否感觉到 Oracle JDBC 中的创建连接代码一层套一层这种方式很难看懂？胖哥认为 Oracle JDBC 编写时有一些自己的初衷，例如 Oracle JDBC 本身支持的协议比较多，分层的目的在于每个层次可以横向扩展，这样可以方便底层协议的模块化共享。当然在某些时候，自己觉得没有必要这样分层，用另一种方式来设计，是不是也可以呢？胖哥认为设计在于人本身，只要你发自内心地感受这样的设计在场景中是合理的，那么它自然就是合理的。下面一起来看看 MySQL 的 JDBC 驱动中创建连接的代码吧。

现在来简单跟踪 MySQL 的相应方法。

在前文中跟踪到了 MySQL 的 connect()方法，它在驱动程序 Driver 的父类 NonRegisteringDriver 中，该方法通过 parseURL()解析了 JdbcURL 串，将其解析为 Properties 类型的对象，然后通过这个对象实例化一个 com.mysql.jdbc.Connection，代码如图 8-20 所示。

```
if ((props = parseURL(url, info)) == null) {
    return null;
}

if (!"1".equals(props.getProperty(NUM_HOSTS_PROPERTY_KEY))) {
    return connectFailover(url, info);
}

try {
    Connection newConn = com.mysql.jdbc.ConnectionImpl.getInstance(
        host(props), port(props), props, database(props), url);
    return newConn;
} catch (SQLException sqlEx) {
    // Don't wrap SQLExceptions, throw
    // them un-changed.
    throw sqlEx;
}
```

图 8-20 MySQL JDBC 创建连接对象的主要代码

进入 Connection 的构造方法可以发现，最为关键的代码是 createNewIO()这个方法，它负责返回一个 com.mysql.jdbc.MysqlIO 类型的对象，相信用过 MySQL JDBC 的同学在遇到连接异常时，会在抛出的异常堆栈中看到 MysqlIO 字样，其实就是从这个类中抛出来的。

在 MysqlIO 的构造方法中，对于创建连接主干代码如图 8-21 所示。

```
this.port = port;
this.host = host;

this.socketFactoryClassName = socketFactoryClassName;
this.socketFactory = createSocketFactory();
this.exceptionInterceptor = this.connection.getExceptionInterceptor();

try {
    this.mysqlConnection = this.socketFactory.connect(this.host,
        this.port, props);

    if (socketTimeout != 0) {
        try {
            this.mysqlConnection.setSoTimeout(socketTimeout);
        } catch (Exception ex) {
            /* Ignore if the platform does not support it */
        }
    }
}
```

图 8-21 MySQL 创建连接的主干代码

这个 socketFactory.connect()返回的对象就是 Socket 类型的对象，跟踪代码也最终发现是通过调用 new Socket()来完成的，只是还会对 Socket 设置相应的参数，如图 8-22 所示。

```
private void configureSocket(Socket sock, Properties props) throws SocketException,
    IOException {
    try {
        sock.setTcpNoDelay(Boolean.valueOf(
            props.getProperty(TCP_NO_DELAY_PROPERTY_NAME,
                TCP_NO_DELAY_DEFAULT_VALUE)).booleanValue());

        String keepAlive = props.getProperty(TCP_KEEP_ALIVE_PROPERTY_NAME,
            TCP_KEEP_ALIVE_DEFAULT_VALUE);

        if (keepAlive != null && keepAlive.length() > 0) {
            sock.setKeepAlive(Boolean.valueOf(keepAlive)
                .booleanValue());
        }

        int receiveBufferSize = Integer.parseInt(props.getProperty(
            TCP_RCV_BUF_PROPERTY_NAME, TCP_RCV_BUF_DEFAULT_VALUE));

        if (receiveBufferSize > 0) {
            sock.setReceiveBufferSize(receiveBufferSize);
        }

        int sendBufferSize = Integer.parseInt(props.getProperty(
            TCP_SND_BUF_PROPERTY_NAME, TCP_SND_BUF_DEFAULT_VALUE));

        if (sendBufferSize > 0) {
            sock.setSendBufferSize(sendBufferSize);
        }

        int trafficClass = Integer.parseInt(props.getProperty(
            TCP_TRAFFIC_CLASS_PROPERTY_NAME,
            TCP_TRAFFIC_CLASS_DEFAULT_VALUE));
    }
}
```

图 8-22 对 Socket 进行设置

在这里对 Socket 的基本设置是否有点熟悉，因为在第 2 章中我们就对这些参数做了介绍。这些设置参数与 JDBC 的连接参数相对应，大家也可以通过看源码的方式反向找到对应的参数信息。

在最终通过 Socket 对象调用 connect 方法进行远程连接时，会使用连接参数中的 connectTimeout 来设置建立网络连接的最大超时时间（如果 jdbcURL 上没设置则该超时时间通常与 OS 设置相关，在 Linux 下的默认值一般是 180s），当程序与目标数据库网络不通或网络通但协议不通的情况下，程序通常会阻塞一段时间，connectTimeout 就是程序阻塞最长的超时时间，当超过这个时间后若未建立网络连接，则会抛出异常。

相对来讲, MySQL 的 JDBC 源码比 Oracle 的 JDBC 源码长, Oracle JDBC 的代码分层较多, 不过它们都是基于 Socket 本身来包装协议的。从这里也可以看出, 目前数据库连接的协议通常为基于 TCP 的“长连接”模型, 并且也是 BIO 模型, 主要原因还是由于数据库与业务层的绑定是很强的, 也就是业务端通常要获取到数据库返回的数据后才会进行下一步操作(甚至可能会有事务), 并且还会保留许多中间数据, 甚至还包含其他的网络连接(如分布式缓存以及客户端), 即使将底层改为非阻塞 I/O 或者异步 I/O, 在应用使用上, 也通常应当是同步的; 否则业务代码就会写得十分复杂, 每次数据库访问后, 要么得到 Future, 要么写回调类, 然后在回调类中做下一步操作。

在这一节的源码阅读中, 你是否学到了一些阅读方法? 学到了在 JDBC 内部你以前不知道的一些奥秘? 也许今天看到这些源码你只是有一个初步的印象, 也许过一段时间后再回过头来看这些源码, 你就会发现更多的秘密。

接下来我们开始讲解 JDBC 源码中比较重要的部分, 即 Statement, 前文中所提到的驱动注册、Connection 创建都只是为建立连接通信而服务的, 还没涉及 SQL 语句的执行, 换句话说, 真正和业务打交道的还是 Statement, 许多参数设置在 Statement 里面通常也是最有效的。

8.4 SQL 执行及处理

➤➤ 8.4.1 创建 Statement

提到 Statement, 如果是做测试, 那么程序很多时候是通过 `connection.createStatement()` 来完成的, 但是对于线上应用, 大部分情况下现在的程序都使用 `connection.prepareStatement()` 方法来完成调用。为何? 因为预编译可以带来许多好处, 例如对某些数据库可以做 SQL 软解析, 还可以防止 SQL 注入。

通过 `connection.prepareStatement()` 方法所创建的 Statement 对象对应的接口类型是 `java.sql.PreparedStatement`, 它是接口 `java.sql.Statement` 的子类, 因此它有许多方法直接来自于 Statement, 不过功能相对会更多, 所以我们就从它来了解吧。

同样从 Oracle 开始看起, 由于 `PreparedStatement` 对象都是通过 `Connection` 对象的 `prepareStatement()` 方法来创建的, 因此看源码应当从 `Connection` 的实现类开始。前文中提到了 Oracle 的 `Connection` 实现类为 `oracle.jdbc.driver.OracleConnection`, 现在的源码又要从这里看起(这就是知识上的环环相扣)。

找到这个类的实体后, 再找到这个类的 `prepareStatement()` 方法, 发现其间接调用内部的 `privatePrepareStatement()` 方法, 这个方法中值得我们关注的代码片段如图 8-23 所示。

```
if((statementCache == null || sl == null) && obj == null)
    if(i != -1 || j != -1) obj = new oracle.jdbc.driver.OraclePreparedStatement(this,
        sql, default_batch, default_row_prefetch, i, j);
    else obj = new oracle.jdbc.driver.OraclePreparedStatement(this,
        sql, default_batch, default_row_prefetch);
```

图 8-23 `privatePrepareStatement()` 方法中值得我们关注的代码片段

通过该段代码可见内部创建了一个 `oracle.jdbc.driver.OraclePreparedStatement` 类型的对象, 同时初始化设置的 `default_row_prefetch` 从 `Connection` 交给了 `Statement`。

进一步跟踪 `OraclePreparedStatement` 类的构造方法, 内部会间接调用 `parseSqlKind()` 方法负责解析和转换 SQL, 进一步的调用路径为:

```

OraclePreparedStatement.expandSqlEscapes(sql) ->
connection.nativeSQL(sql) ->
OracleSql.parse(sql)

```

OracleSql.parse(sql)方法主要用于解析 SQL 语句,如果这个 SQL 语句中包含有关键字,则会做一些转换处理,例如预编译 SQL 语句中出现的字符“?”,会被认为是 SQL 中的占位符,或者说需要填充参数的位置,Oracle JDBC 会在发送 SQL 到 Oracle 服务器端前通过调用 OracleSql.appendChar()方法将其转换,如图 8-24 所示。

```

void appendChar(StringBuffer stringbuffer, char c1)
{
    if(c1 == '?')
        stringbuffer.append(nextArgument());
    else
        stringbuffer.append(c1);
}

String nextArgument()
{
    String s = ":" + current_argument;
    current_argument++;
    return s;
}

```

图 8-24 appendChar()方法详情

nextArgument()方法负责数字的叠加,并带上一个冒号,也就是预编译中的“?”在经过 Oracle JDBC 这段代码编译时,Oracle JDBC 会将其编译为:1、:2、:3…这样的字符串,在 Oracle 服务器端执行动态语句也接收这样的字符串(在第 6 章讲解执行计划时提到过)。

在 Oracle 服务器端如果要执行动态 SQL,有一种方法是“execute [动态 SQL] using 参数列表”,在动态 SQL 语句中的参数也是以“:”开头的,即代表 SQL 语句中当前位置需要填充一个具体参数值,这样的 SQL 语句可以在 Oracle 服务器端通过 v\$sqlText 视图看到。

关于 MySQL 的创建预编译的代码,就不再像 Oracle 那样细看代码了,不过 MySQL 的预编译中有几个默认参数需要知道,见图 8-25。

```

public java.sql.PreparedStatement prepareStatement(String sql)
    throws SQLException {
    return prepareStatement(sql, DEFAULT_RESULT_SET_TYPE,
        DEFAULT_RESULT_SET_CONCURRENCY);
}

```

图 8-25 请注意默认的参数

这里采用了 ResultSet.TYPE_FORWARD_ONLY、ResultSet.CONCUR_READ_ONLY 作为入口参数,也就是在 MySQL 的 JDBC 中只要使用默认的 prepareStatement()方法,这几个参数就无须自己来设置了,在后面的章节中也会应用到。

其余的预编译功能大家可以先自行跟踪。在本书下册中还会提到一些和预编译相关的技术。为何要到本书下册中讲解呢?因为那个时候胖哥将给大家分享开源框架的许多坑,这些坑当中包含了 JDBC 的一些坑,这或许是许多小伙伴所感兴趣的吧。

预编译对于 Oracle 来讲,可以将硬解析变为软解析,在并发系统中性能会提高很多,并且防止 SQL 注入,而在 MySQL 上仅仅有防止注入的功能。

通过预编译对象 PreparedStatement 继续跟踪创建 executeQuery 方法便可以跟踪到结果集的处理方式(在 MySQL JDBC 中有几种不同的结果集处理方式),在 8.4.3 节中将会有更多的说明。

有了预编译对象，那么剩下的就是执行 SQL，执行 SQL 时可以设定批处理（Batch），那么批处理在 JDBC 层面是如何处理的呢？请看下面的介绍。

8.4.2 Batch 设置批处理

写代码时为提高批量写入的能力，通常会用 Batch 方式，配合 autoCommit 为 false 一起来完成，伪代码的例子如下：

```
connection.setAutoCommit(false);
for(...) {
    preparedStatement.setString(...);
    ...
    preparedStatement.addBatch();
    if(i % 1000) {
        preparedStatement.executeBatch();
        connection.commit();
    }
}
preparedStatement.executeBatch();
connection.commit();
```

即每条语句不是通过 executeUpdate() 直接执行，而是放入到一个 Batch 中，待达到一定的量时，一起执行并一起提交。

关于这样的代码处理会有很多种理解，有人会认为是一次性发送很多 SQL 给数据库服务器端，也有人认为是 1 个 SQL 语句加批量的参数值发送到数据库服务器端。在 JDBC 层面批处理到底做的是什么事情呢？我们一起来看看源码便知真相。

依旧先来看看 Oracle 的批处理。addBatch() 是通过 PreparedStatement 来操作的，Oracle 的实现类是前文中提到的 OraclePreparedStatement，打开该类的 addBatch() 方法，如图 8-26 所示。

```
public synchronized void addBatch()
    throws SQLException
{
    setJdbcBatchStyle();
    super.binds_in.rowCompleted(-1);
    if(super.binds_in.hasStreamType())
        DBError.throwSQLException(90, "streams type cannot be used in batching");
    checkBindsInAndOut();
    super.m_currentRank++;
}
```

图 8-26 Oracle 中的 addBatch() 方法代码

在这段代码中没有发现特殊的处理，只能看出对 binds_in 的单行结束标志，校验了一些绑定 IN、OUT 参数信息，那么就需要继续来看看 executeBatch() 到底做了什么，跟踪后发现最终会调用到 OraclePreparedStatement 的父类 OracleStatement 中的 executeNonQuery() 方法，它最终做了一个 for 循环进行处理，如图 8-27 所示。

Oracle 的 JDBC 在批处理执行时，是利用一个 for 循环来运行的，那么与直接循环调用 executeUpdate() 又有何区别呢？

每次执行一个 SQL 语句时，会经历编译、检查、对象创建、参数设置等动作，最终才会调用到 dbaccess.parseExecuteFetch() 方法。内部循环并非每次都去做这样的事情，它是抽象层次更低的循环，并且对于同样的 SQL 语句，会一次性发送多个不同的绑定参数信息，这样就大大提高了性能。

```
private void executeNonQuery(boolean flag)
    throws SQLException
{
    int j = m_currentRank;
    DBAccess dbaccess = connection.db_access;
    try
    {
        if(flag && sql_kind == 1 && j > 1)
        {
            for(int i = 0; i < j; i++)
            {
                if(need_to_parse)
                    valid_rows = dbaccess.parseExecuteFetch(dbstmt, sql_kind, array_sql, binds_in, 1, binds_out, 1);
                else
                    valid_rows = dbaccess.executeFetch(dbstmt, sql_kind, binds_in, 1, binds_out, 1);
                binds_in.removeFirstRow();
            }
        }
        else
        {
            if(need_to_parse)
                valid_rows = dbaccess.parseExecuteFetch(dbstmt, sql_kind, array_sql, binds_in, j, binds_out, j);
            else
                valid_rows = dbaccess.executeFetch(dbstmt, sql_kind, binds_in, j, binds_out, j);
        }
    }
    catch(IOException ioexception)
    {
        DBError.throwSQLException(ioexception);
    }
}
```

图 8-27 批处理原来是内部的一个循环

接下来看看 MySQL 的批处理会是什么样的？
它的 addBatch()方法如图 8-28 所示（在 com.mysql.jdbc.PreparedStatement 当中）。

```
public void addBatch() throws SQLException {
    synchronized (checkClosed().getConnectionMutex()) {
        if (this.batchedArgs == null) {
            this.batchedArgs = new ArrayList<Object>();
        }

        for (int i = 0; i < this.parameterValues.length; i++) {
            checkAllParametersSet(this.parameterValues[i],
                this.parameterStreams[i], i);
        }

        this.batchedArgs.add(new BatchParams(this.parameterValues,
            this.parameterStreams, this.isStream, this.streamLengths,
            this.isNull));
    }
}
```

图 8-28 MySQL 的参数原来是放到一个 ArrayList 中的

这段代码看上去比 Oracle 的 addBatch()要好懂一些，就像一个 ArrayList 写入参数列表信息，代码就不用多说了。再来看看 MySQL 的 executeBatch()方法，又是什么样子的？
在这段源码中有 3 个主要的逻辑，如下所示：

```
if (!this.batchHasPlainStatements
    && this.connection.getRewriteBatchedStatements()) {
    if (canRewriteAsMultiValueInsertAtSqlLevel()) {
        return executeBatchedInserts(batchTimeout);
    }
    if (this.connection.versionMeetsMinimum(4, 1, 0)
        && !this.batchHasPlainStatements
        && this.batchedArgs != null
        && this.batchedArgs.size() > 3) {
        return executePreparedBatchAsMultiStatement(batchTimeout);
    }
}
return executeBatchSerially(batchTimeout);
```

在 3 个核心逻辑中，默认的代码逻辑是 executeBatchSerially()方法，这个方法的关键部分如图 8-29 所示。


```
for (batchCommandIndex = 0; batchCommandIndex < nbrCommands; batchCommandIndex++) {
    Object arg = this.batchedArgs.get(batchCommandIndex);

    if (arg instanceof String) {
        updateCounts[batchCommandIndex] = executeUpdate((String) arg);
    } else {
        BatchParams paramArg = (BatchParams) arg;

        try {
            updateCounts[batchCommandIndex] = executeUpdate(
                paramArg.parameterStrings,
                paramArg.parameterStreams, paramArg.isStream,
                paramArg.streamLengths, paramArg.isNull, true);
        } catch (SQLException e) {
            // ...
        }
    }
}
```

图 8-29 MySQL 的批处理最终竟然是循环调用 executeUpdate()来实现的

它与 Oracle 类似的是循环处理，但最大的区别是循环最终会调用多次 executeUpdate() 方法来完成，可见在这样的情况下 MySQL JDBC 的批处理无法提升性能，这一点大家可以通过性能测试来印证，而这样做的目的也只是为了适配 Java 标准的批处理接口。

对于其他两个逻辑，当 getRewriteBatchedStatements()方法成立时就会进入批处理模式，该方法如果返回 true，则需要在 JDBC URL 中设置 rewriteBatchedStatements=true，否则默认返回 false。

当 getRewriteBatchedStatements()方法返回 true 时，执行代码会出现两个不同的分支，当 canRewriteAsMultiValueInsertAtSqlLevel()条件成立时，会走到第 1 个分支中执行 executeBatchedInserts(batchTimeout)方法，canRewriteAsMultiValueInsertAtSqlLevel()方法用于判定 SQL 是否为 insert 语句，而且不是 insert ... select 之类的语句。如果成立，它会尝试将多个 insert 语句合并为一个 insert 语句。

例如语句：

```
Insert into table(a,b) values(1,2);
Insert into table(a,b) values(3,4);
```

合并后的结果是：

```
Insert into table(a,b) values(1,2),(3,4);
```

依此类推。

如果不是 insert 语句，则会判定相应的版本是否在 4.0 以上，以及参数行数是否大于 3，在这个条件成立的情况下也会执行另一种批处理模式，这种批处理模式会将多个 SQL 语句用分号分隔拼接在一起发送给服务器端。其实这种方式在服务器端依然会逐条执行，所以总体性能区别不大，唯一的区别在于减少了网络交互次数，在网络较好的情况下区别并不大。

这个逻辑依然适用于 MySQL 的 StatementImpl 当中的 Batch 机制，由于它执行的多个 SQL 语句应该完全不同，且没有绑定变量的概念，因此它在支持批处理上，如果启用了参数 rewriteBatchedStatements=true，那么也会将多个 SQL 语句用分号拼接发送到 MySQL 服务器端。

但是不论是拼接参数还是拼接 SQL 本身，并不一定一次性提交到服务器端，MySQL JDBC 为了防止一次发送的内容太多，设定了网络字节的大小限制，与 maxAllowedPacket 参数相关，在默认情况下这个参数会在连接初始化时被设置，它会在自定义的值和 MySQL 服务器端的限制值之间提取一个最小值，如果未进行设置则默认为 65536。因此，当 SQL 语句较长时（或者参数的内容较大时），JDBC 客户端与服务器端的交互也可能会比较频繁，批处理带来的性能提升效果会下降。

有同学又会问了：第三方框架中的批处理，例如 ibatis 所提供的 startBatch()、executeBatch()方法，它们又是如何与 JDBC 联系在一起的呢？

胖哥针对这些内容对批处理做一个简单补充，一起来看看 ibatis 中的 startBatch()、update()、executeBatch()等相关方法的内部到底做了什么。

通常用 ibatis 时使用的实现类是 com.ibatis.sqlmap.engine.impl.SqlMapClientImpl，简称为“SqlMapClient”，调用其 startBatch()方法时相继会调用内部的许多层的 startBatch()方法，细节上大家可以自己去跟踪。其最终动作只是将 SessionScope 的一个 boolean 类型属性 inBatch 设置为 true（SessionScope 的全名为：com.ibatis.sqlmap.engine.scope.SessionScope），代码如图 8-30 所示。

```
public void startBatch(SessionScope sessionScope) {
    sessionScope.setInBatch(true);
}
```

图 8-30 ibatis 的 startBatch()方法仅仅是设置一个属性

此时自然联想到：当程序中开始执行 update()、insert()、delete()这些方法时，应该就会使用到这个参数。接下来就来跟踪 update()方法的源码来印证自己的结论。

通过跟踪源码最终会跟踪到 MappedStatement 类(类的全名为：com.ibatis.sqlmap.engine.mapping.statement.MappedStatement)的 sqlExecuteUpdate()方法中，源码如图 8-31 所示。

代码中判定如果 Session 是批处理，则会将参数添加到一个地方，这个 getSession()方法获取的就是前文中提到的 SessionScope，获取的属性正好就是 inBatch，这样两者就对上号了。

此时我们会想：参数放到哪里呢？

```
protected int sqlExecuteUpdate(StatementScope statementScope, Connection conn, String sqlString, Object[] parameters)
{
    if (statementScope.getSession().isInBatch()) {
        getSqlExecutor().addBatch(statementScope, conn, sqlString, parameters);
        return 0;
    } else {
        return getSqlExecutor().executeUpdate(statementScope, conn, sqlString, parameters);
    }
}
```

图 8-31 ibatis 若设置批处理，则会在执行时记录参数

参数最终在 com.ibatis.sqlmap.engine.execution.DefaultSqlExecutor 类的一个内部类 Batch 中，它的结构很简单，一看便懂，如图 8-32 所示。

```
private static class Batch {
    private String currentSql;
    private List statementList = new ArrayList();
    private List batchResultList = new ArrayList();
    private int size;
}
```

图 8-32 ibatis 中的 Batch 参数就是使用 ArrayList 来存放的

在这里 Batch 中可以存放很多 PreparedStatement 对象，根据这个数据结构，是不是每次 addBatch()时都会创建一个 PreparedStatement 呢？不会！ibatis 不会那么傻，那样会导致许多问题。现在带着这个问题来看看 ibatis 的 addBatch()真正写入时是怎么做的，代码如图 8-33 所示。

可以看到这个 currentSql 在 else 语句中被赋值。换句话说，每次传入的 SQL 语句将与 currentSql 进行匹配，若匹配则被认为是相同的 SQL 语句，从 statementList 中取出最后一个 PreparedStatement 对象即可，而无须再创建一个 PreparedStatement 对象。


```

public void addBatch(StatementScope statementScope, Connection conn, String sql, Object[] parameters) throws SQLException {
    PreparedStatement ps = null;
    if (currentSql != null && currentSql.equals(sql)) {
        int last = statementList.size() - 1;
        ps = (PreparedStatement) statementList.get(last);
    } else {
        ps = prepareStatement(statementScope.getSession(), conn, sql);
        setStatementTimeout(statementScope.getStatement(), ps);
        currentSql = sql;
        statementList.add(ps);
        batchResultList.add(new BatchResult(statementScope.getStatement().getId(), sql));
    }
    statementScope.getParameterMap().setParameters(statementScope, ps, parameters);
    ps.addBatch();
    size++;
}

```

图 8-33 ibatis 在 addBatch()时有对 currentSql 的判定

若不匹配是不是也会一定创建一个 PreparedStatement 对象呢？不会！ibatis 还是不会那么傻。prepareStatement()方法的代码如图 8-34 所示。

```

private static PreparedStatement prepareStatement(SessionScope sessionScope, Connection conn, String sql)
    SqlMapExecutorDelegate delegate = ((SqlMapClientImpl) sessionScope.getSqlMapExecutor()).getDelegate();
    if (sessionScope.hasPreparedStatementFor(sql)) {
        return sessionScope.getPreparedStatement((sql));
    } else {
        PreparedStatement ps = conn.prepareStatement(sql);
        sessionScope.putPreparedStatement(delegate, sql, ps);
        return ps;
    }
}

```

图 8-34 prepareStatement()方法里面还有两个逻辑

这两个逻辑大家可以继续去阅读，是十分轻松的。简单来讲，内部会有一个 HashMap 结构，用 SQL 作为 Key，用 PreparedStatement 对象作为 Value 值，如果通过此 SQL 检测到这个会话中有同样的 SQL 的 PreparedStatement 对象，则直接返回；若没找到，则新建一个。换句话说，ibatis 也不会乱创建对象。也就是说，同一个 SQL 语句的多个参数，最终还是由同一个 PreparedStatement 对象来保存。

那么前文中为何要出现一个 currentSql 呢？其实这是一个技巧性做法，ibatis 认为在很多批处理的情况下，连续的 SQL 都是一样的，当前的 SQL 如果匹配命中，那么就无须到 Hash 表去查找；若未命中，则由 Hash 表做后备保证（Hash 表查找肯定会更慢，这一点在第 1 篇的多个地方都提到过）。

最后再来看看 ibatis 的 executeBatch()方法的源码。这部分代码也应当是 ibatis 与 JDBC 最后交接的部分，可能一看它的运行便明白了许多事情，它遍历了所保存的 PreparedStatement 对象列表，逐个执行其 executeBatch()方法，这里调用的就是 JDBC 驱动中具体的 PreparedStatement 实现类对象的 executeBatch()方法，因此它将如何执行就和具体的 JDBC 相关了，ibatis 只是在上面做了一层封装而已，代码如图 8-35 所示。

看到这里是否可以串通一些东西呢？我们经常会被框架所迷惑，因为它提供的方法和底层的方法太相似了，有些时候不知道这是谁提供的，更不知道分工情况是怎么样的，所以当你看到这里时，不仅仅应该学会一些基本的技术，更应该懂得看源码可以让自己更加清晰地认识问题的本质。

```

public int executeBatch() throws SQLException {
    int totalRowCount = 0;
    for (int i = 0, n = statementList.size(); i < n; i++) {
        PreparedStatement ps = (PreparedStatement) statementList.get(i);
        int[] rowCounts = ps.executeBatch();
        for (int j = 0; j < rowCounts.length; j++) {
            if (rowCounts[j] == Statement.SUCCESS_NO_INFO) {
                // do nothing
            } else if (rowCounts[j] == Statement.EXECUTE_FAILED) {
                throw new SQLException("The batched statement at index " + j + " failed to execute.");
            } else {
                totalRowCount += rowCounts[j];
            }
        }
    }
    return totalRowCount;
}

```

图 8-35 ibatis 经过封装后，最终调用具体 JDBC 驱动中的 executeBatch()方法

8.4.3 fetchSize 与 maxRows

在很多编程中都会用到 fetchSize 参数，尤其是在一些调优的过程中，所以胖哥单独将它拿出来讲解。

在前面看 Oracle 源码时，应该已经注意到初始化 Connection 时可以设置 fetchSize 参数（默认值是 10），在 JDBC 源码中会修改属性 row_prefetch 的值。不过这个 Connection 在真正运行 SQL 时可以单独设置，也就是获取到 Statement 对象后可以通过设置 setFetchSize(int) 来实现，也会修改相应的值。在 SQL 运行的过程中，在调用结果集 ResultSet 的 next() 方法时启用，下面来看看 next() 方法源码中比较关键的部分，如图 8-36 所示。

```

statement.current_row++;
statement.total_rows_visited++;
if(statement.max_rows != 0 && statement.total_rows_visited > statement.max_rows)
{
    internal_close();
    return false;
}
if(statement.current_row >= statement.valid_rows)
    try
    {
        if(statement.got_last_batch)
        {
            internal_close();
            return false;
        }
        statement.check_row_prefetch_changed();
        prepare_for_new_row();
        statement.connection.needsLine();
        statement.valid_rows = connection.db_access.fetch(statement.dbstmt, statement.defines, statement.row_prefetch);
        statement.fetchedFirstColumns();
        if(statement.valid_rows == 0)
        {
            internal_close();
            return false;
        } else
        {
            statement.current_row = 0;
            statement.checkValidRowsStatus();
            return true;
        }
    }
}

```

图 8-36 next()方法源码中比较为关键的部分

这段代码首先让 Statement 中的 current_row++，这个操作的意思自然是当前行数增加，但还有一个 total_rows_visited++ 操作，就让小伙伴们迷惑了。

if(statement.max_rows != 0 && statement.total_rows_visited > statement.max_rows) 判定说明如果 max_rows 不为 0，则由 total_rows_visited 和 max_rows 来决定是否返回 false。这个 max_rows 是通过程序中 Statement 对象的 setMaxRows(int) 来设置的，表示最多从结果集中

获取的数据量，自然的这个判定中的 `total_rows_visited` 肯定就是结果集遍历的总行数。那 `current_row` 是什么意思呢？不用着急，还有下文呢。

继续跟踪发现了新的判定逻辑：`if(statement.current_row >= statement.valid_rows)`，这里出现了一个 `valid_rows`，它又是什么呢？现在还不知道，需要继续向下看才行。如果该条件成立，则会调用 `db_access.fetch()` 方法进行一次游标 `fetch` 操作，并传入 `row_prefetch` 参数，也就是这里指定了每次遍历的行数（前文中提到默认值是 10）。但是并不代表每次指定多少行就一定能加载多少行，所以方法会返回实际的行数，也就是 `valid_rows` 的值。接下来的代码中判定 `valid_rows` 不为 0，则会将 `current_row` 参数设置为 0，若返回 0 则代表没有数据，会通过 `close` 来关闭相应的资源（但肯定不是关闭通信连接）。

综合起来思考，几个参数的意思已经很明确了，`current_row` 在每一次从数据库获取数据后重新设置为 0，`valid_rows` 是每次从数据库返回的数据量，循环中逐步会增加 `current_row` 的值，如果达到了 `valid_rows` 的值，则再与数据库通信一次。在某些特定的场景下（例如跨国访问数据库，网络延迟极高），可以适当增加这个值来提升性能。不过这个参数并不是永远生效的，在本书下册中，胖哥将会给大家分享它不生效的场景，但是同样有相应的解决方案。

胖哥自己刚开始看这段代码也看不懂，这个时候胖哥将一些看不懂的变量先放那里，先看一些看得懂的，反复迭代多看几次就看懂了！这也是一种看源码的方法。

代码中还出现 `statement.total_rows_visited`、`statement.max_rows`，它们又是做什么的呢？

它们分别控制的是当前访问行数的总和、最大访问的行数。由于 `current_row` 每次 `fetch` 后都会清零，所以必须有另一个变量来控制访问行数的总和，这个总和将会与最大行数 `max_rows` 对比大小，如果超过这个值，则不会再进行扫描。

`max_rows` 默认为 0，没有任何限制（从上面代码中就可以得出结论），在程序中可以通过 `Statement.setMaxRows(int)` 来完成设置，如图 8-37 所示。

```
public synchronized void setMaxRows(int i)
    throws SQLException
{
    if(i < 0)
        DBError.throwSQLException(68);
    max_rows = i;
}
```

图 8-37 在 Oracle 中 Statement 设置 MaxRows 参数

从这段代码可以发现对于一个 SQL 语句，如果没有加任何 `where` 条件，在 JDBC 层依然可以控制最大返回行数。

在 MySQL 中是如何使用的呢？我们简单来看看在 MySQL JDBC 的 `StatementImpl` 类中 `executeQuery` 方法的一段代码片段，如图 8-38 所示。

在代码中应当发现，如果 SQL 语句中带有 `LIMIT` 关键字，则会进行特殊处理。在这里对于代码细节我们不再详述，有兴趣的同学可以自己跟踪细节，下面我们介绍 MySQL JDBC 的应用细节。

有些时候大家在用 JDBC 读取数据库的大量数据时，会发生宕机。有的小伙伴开始自己想办法，读取一部分数据，处理一部分数据，清理掉一部分数据，这是个不错的方式，不过还是会发生宕机，这到底是个什么状况？

```
if (locallyScopedConn.useMaxRows()) {
    // We need to execute this all together
    // So synchronize on the Connection's mutex (because
    // even queries going through there synchronize
    // on the connection
    if (StringUtils.indexOfIgnoreCase(sql, "LIMIT") != -1) { //NON-NLS-15
        this.results = locallyScopedConn.execSQL(this, sql,
            this.maxRows, null, this.resultSetType,
            this.resultSetConcurrency,
            doStreaming,
            this.currentCatalog, cachedFields);
    } else {
        if (this.maxRows <= 0) {
            executeSimpleNonQuery(locallyScopedConn,
                "SET SQL_SELECT_LIMIT=DEFAULT");
        } else {
            executeSimpleNonQuery(locallyScopedConn,
                "SET SQL_SELECT_LIMIT=" + this.maxRows);
        }
    }

    statementBegins();
}
```

图 8-38 MySQL 中执行 SQL 查询时的关键代码部分

有些时候吃内存并不一定是因为自己的程序，可能来自于三方包。相信用 JDBC 操作过 MySQL 的大表提取大量数据的同学都会遇到过类似的问题（Oracle 中貌似没有出现过这样的现象，因为 Oracle 的 `fetchSize` 默认就是 10），很多时候 MySQL 中的 JDBC 设置 `setFetchSize` 貌似没有什么效果，在读取大数据时依然会导致 JVM 的内存溢出，原因是发现 JDBC 将大量的数据载入到 JVM 中，导致 JVM 最终无法承受。此问题有解吗？MySQL 为什么这样做？请看下面胖哥的理解。

有人说 MySQL 为互联网而生，或许是这样，因为它足够简单，它不希望在一些处理的细节上考虑过多，或者说不希望将功能的细节做得很深，更在乎本身功能的横向扩展，这也是开源界的一个弊病。在许多问题的处理上 MySQL 更像是一个勤恳做事的人，大家把它改得越简单，在某些场景下的效率就自然会越高，那么相应的它就需要使用者遵循更多的规则或开发者本身拥有较强的技术功底。而类似 Oracle 更像是一个智慧型的数据库，它可以解决许多小白用户的低级错误，但它始终是一款软件，无法解决所有的问题，在当今的世界上还不是那么适合大型互联网。

MySQL 默认会将用户请求的数据直接全部返回，这样数据库服务器端就比较简单，因为不需要保存相应的状态和数据，这样对于网络交互设计来讲也更为简单，对于互联网的应用来讲，大部分高并发操作都是小数据的读/写，因此 MySQL 是很合适的。不过这样一来，就会导致所提到的问题：“MySQL JDBC 处理大数据库时会内存溢出”。为了解决这个问题 MySQL JDBC 进行了扩展，在结果集 `ResultSet` 对象内部有一个 `RowData` 属性，这是一个接口，全名为 `com.mysql.jdbc.RowData`，它通常有 3 个实现类，如图 8-39 所示。

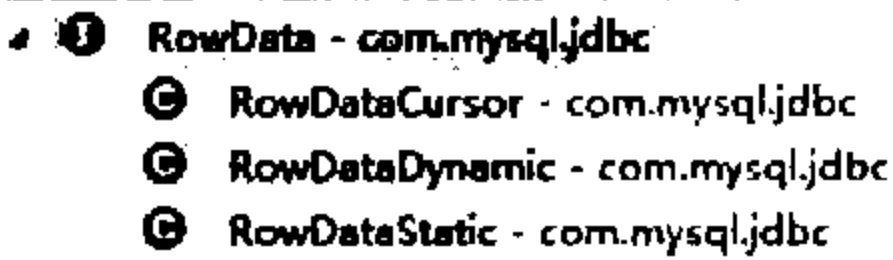


图 8-39 RowData 的 3 个实现类

这些实现类是真正负责数据处理的，`ResultSet` 只是从它们当中提取数据。在默认情况下返回所有数据使用的对象就是 `RowDataStatic` 类的实例。那么另外两个类又是怎么样呢？难道是摆设？当然不是，它们与 JDBC 启动参数相关。

◎ 在 `JdbcURL` 上增加参数：`useCursorFetch=true&defaultFetchSize=1024`，这样在运行

时就不会一次性将数据加载到内存中了。也可以只在 JDBC URL 中设置 `useCursorFetch=true`，而在程序中设置 `setFetchSize` 参数，设置的效果是一致的。在这种 `JdbcURL` 参数下，使用的是 `RowDataCursor` 类的实例。这种获取数据的模式可以叫作“游标模式”，是 MySQL 提取大数据的方式之一，但是并非唯一的方式，也不是效率最高的方式。

- ◎ `Statement.setFetchSize(Integer.MIN_VALUE)`，这个方法本身需要配合设置参数 `setResultSetType(ResultSet.TYPE_FORWARD_ONLY)` 才会生效（在 8.4.1 节中发现这个参数是默认的），这两条语句等价于 `enableStreamingResults()` 方法的调用，源码如图 8-40 所示。

```
public void enableStreamingResults() throws SQLException {
    synchronized (checkClosed().getConnectionMutex()) {
        this.originalResultSetType = this.resultSetType;
        this.originalFetchSize = this.fetchSize;

        setFetchSize(Integer.MIN_VALUE);
        setResultSetType(ResultSet.TYPE_FORWARD_ONLY);
    }
}
```

图 8-40 `enableStreamingResults()` 方法用于设置几个基本属性值

但是 `enableStreamingResults()` 方法不是通用的 `Statement` 接口的 API 来调用的，也就是说，它不是 JDBC 规范中的 API，如果要使用，则需要强制类型转换为 MySQL 的 `Statement` 类型才能调用，自然也需要确保这个 `Statement` 是 MySQL 的。在这种模式下，使用的就是 `RowDataDynamic`，这种模式也是 MySQL JDBC 拖大数据时性能最好的一种，但是并不意味着它一点问题都没有，它的 Bug 非常多，存在许多问题，在本书下册中将会给大家分享一些使用流模式结果集的悲剧。所以在有些时候，依然会采用游标模式来做一些大数据的处理操作。

这一章是介绍源码的，自然不能只是说点结论，还得通过源码来验证胖哥上面的说法。首先通过 `StatementImpl` 的 `executeQuery()` 方法来看看 `streamResult` 的判定逻辑部分，如图 8-41 所示。

```
public java.sql.ResultSet executeQuery(String sql)
    throws SQLException {
    synchronized (checkClosed().getConnectionMutex()) {
        MySQLConnection locallyScopedConn = this.connection;

        this.retrieveGeneratedKeys = false;

        resetCancelledState();

        checkNullOrEmptyQuery(sql);

        boolean doStreaming = createStreamingResultSet();

        protected boolean createStreamingResultSet() {
            try {
                synchronized (checkClosed().getConnectionMutex()) {
                    return ((this.resultSetType == java.sql.ResultSet.TYPE_FORWARD_ONLY)
                        && (this.resultSetConcurrency == java.sql.ResultSet.CONCUR_READ_ONLY) && (this.fetchSize == Integer.MIN_VALUE));
                }
            } catch (SQLException e) {
                // we can't break the interface, having this be no-op in case of error is ok
                return false;
            }
        }
    }
}
```

图 8-41 通过图 8-40 所设置的参数判定是否要启用 `streamResult`

在进一步跟踪代码的过程中发现，通过层层方法调用最终调用到 `MysqlIO` 类中的 `getResultSet()` 方法，这个方法是这样使用 `streamResult` 参数的，见图 8-42。

```
RowData rowData = null;

if (!streamResults) {
    rowData = readSingleRowSet(columnCount, maxRows,
        resultSetConcurrency, isBinaryEncoded,
        (metadataFromCache == null) ? fields : metadataFromCache);
} else {
    rowData = new RowDataDynamic(this, (int) columnCount,
        (metadataFromCache == null) ? fields : metadataFromCache,
        isBinaryEncoded);
    this.streamingData = rowData;
}
```

图 8-42 在这里使用了 streamResult 参数

如果是 streamingData，则会创建一个 RowDataDynamic 将 this 传入，这个 this 就是 MysqlIO 的当前实例，并保存在 streamingData 中。换句话说，数据并未一次性获取完整。

如果不是 streamingData，则会调用 readSingleRowSet()方法，这个方法读完便知道数据是一次性加载的，如图 8-43 所示。

```
private RowData readSingleRowSet(long columnCount, int maxRows,
    int resultSetConcurrency, boolean isBinaryEncoded, Field[] fields)
    throws SQLException {
    RowData rowData;
    ArrayList<ResultSetRow> rows = new ArrayList<ResultSetRow>();

    boolean useBufferRowExplicit = useBufferRowExplicit(fields);

    // Now read the data
    ResultSetRow row = nextRow(fields, (int) columnCount, isBinaryEncoded,
        resultSetConcurrency, false, useBufferRowExplicit, false, null);

    int rowCount = 0;

    if (row != null) {
        rows.add(row);
        rowCount = 1;
    }

    while (row != null) {
        row = nextRow(fields, (int) columnCount, isBinaryEncoded,
            resultSetConcurrency, false, useBufferRowExplicit, false, null);

        if (row != null) {
            if ((maxRows == -1) || (rowCount < maxRows)) {
                rows.add(row);
                rowCount++;
            }
        }
    }

    rowData = new RowDataStatic(rows);

    return rowData;
}
```

图 8-43 RowDataStatic 是一次性加载数据的

除此之外，由于 RowDataStatic 每次都会加载所有的数据，因此在获取完数据后便可以将输入流清理掉。回到调用 getResultSet()方法之前的代码中，见图 8-44。

```
if (!streamResults) {
    clearInputStream();
}

protected void clearInputStream() throws SQLException {

    try {
        int len = this.mysqlInput.available();

        while (len > 0) {
            this.mysqlInput.skip(len);
            len = this.mysqlInput.available();
        }
    } catch (IOException ioEx) {
        throw SQLError.createCommunicationsException(this.connection,
            this.lastPacketSentTimeMs, this.lastPacketReceivedTimeMs, ioEx, getExceptionInterceptor());
    }
}
```

图 8-44 这个动作是将输入流缓冲区的数据读取干净

换句话说，MySQL 的一个连接是一个套接字，输入/输出流自然用的是同一个，在这个过程中需要将输入/输出流的数据读取干净才能让下一个执行语句进来。

下面再来看看游标模式的行走路线。代码自然要切换到 StatementImpl 类的 executeQuery()方法中，在这里有这样一段代码，如图 8-45 所示。

```

    if (useServerFetch()) {
        this.results = createResultSetUsingServerFetch(sql);

        return this.results;
    }
    private boolean useServerFetch() throws SQLException {
        synchronized (checkClosed().getConnectionMutex()) {
            return this.connection.isCursorFetchEnabled() && this.fetchSize > 0
                && this.resultSetConcurrency == ResultSet.CONCUR_READ_ONLY
                && this.resultSetType == ResultSet.TYPE_FORWARD_ONLY;
        }
    }
    public boolean isCursorFetchEnabled() throws SQLException {
        return (versionMeetsMinimum(5, 0, 2) && getUseCursorFetch());
    }

```

图 8-45 JDBC URL 参数的设置在这里生效

在 useServerFetch()方法的判定条件中，最后两个判定条件（resultSetConcurrency == ResultSet.CONCUR_READ_ONLY、resultSetType == ResultSet.TYPE_FORWARD_ONLY）是 MySQL JDBC 中创建 Statement 时的默认值，前面两个参数就是前文中提到的启用游标模式的两个参数，例如 getUseCursorFetch()所对应的参数如图 8-46 所示。

```

private BooleanConnectionProperty useCursorFetch = new BooleanConnectionProperty(
    "useCursorFetch",
    false,
    "If connected to MySQL > 5.0.2, and setFetchSize() > 0 on a statement, should "
    + "that statement use cursor-based fetching to retrieve rows?",
    "5.0.0", PERFORMANCE_CATEGORY, Integer.MAX_VALUE);

```

图 8-46 useCursorFetch 参数的描述及默认值

通过这些源码也可以发现，该参数必须在 MySQL 5.0.2 版本以上才可以使用，并且要求 fetchsize> 0。

不论是流模式还是游标模式，它们都有一个共同的特征，就是数据不是一次性获取到应用程序中的，那么数据库服务器端就必须保存这些数据，这似乎没有什么问题，因为数据库本身就是保存数据的地方，但数据库所需要保存的是 SQL 查询时所对应的数据版本，如果 MySQL 服务器端的某种存储引擎不支持版本号怎么办？例如 MyIsam 存储引擎，这个时候就会导致锁表的情况，如果应用端长时间未将数据提取完，就会导致某些表长期被锁住，以至于这个表无法做更新操作。这一点大家在选择存储引擎和操作方式时要注意了。

对于 MySQL 的逻辑就看到这里，如果大家对内部是如何处理的十分感兴趣，则可以继续深入查看源码，以满足自己的好奇心。在本节中主要阐述了 fetchSize 和 maxRows 的原理，至此，对源码已经基本可以理清思路，接下来继续看 JDBC 中关注较多的其他方面的问题。

➤➤ 8.4.4 setQueryTimeout()与 cancel()

在本章最后，胖哥带大家来看看 Statemnet 类中的 setQueryTimeout(int)与 cancel()的实现。胖哥将它们放到一起来介绍，是为了说明它们的实现方式有包含关系或是类似的。

如果还不知道 JDBC API 中这两个方法的功能, 那么这里胖哥给你补课:

`setQueryTimeout(int)`方法可以设置一个 SQL 语句最长的执行时间, 单位是秒, 若未执行完会自动断开, 服务器端通常会自动释放资源 (某些数据库未必), 客户端会抛出相应的异常。

`cancel()`方法是 `Statement` 对象上的方法, 若某个 SQL 语句正在执行中, 可通过调用该 SQL 语句对应的 `Statement` 对象的 `cancel()`方法, 此时该 SQL 语句的执行通常会被中断, 执行 SQL 的线程中将抛出 `SQLException` 异常。

`setQueryTimeout(int)`方法执行时只是设置了 `Statement` 对象中的一个属性, 在执行 SQL 语句时会利用到这个参数, 我们可以通过查阅 `execute()`相关的方法来了解这个参数到底用于何处。前文中我们看的是 `executeQuery()`方法, 现在还是继续来看这个方法, 为什么? 因为这样我们就能对这段代码的细节越来越熟悉了。

先来看看 Oracle 的 `executeQuery()`是如何使用这个参数的? 经过跟踪源码发现, Oracle 在这个方法的调用中间接调用了 `doExecuteWithTimeout()`方法, 如图 8-47 所示。

```
void doExecuteWithTimeout()
    throws SQLException
{
    if(sql_query == null || sql_query == "")
        DBError.throwSQLException(104);
    dbstmt.rows_processed = 0;
    checkBindsInAndOut();
    if(sql_string_changed)
        parseSqlKind();
    if(sql_kind == 0)
    {
        if(m_queryTimeout != 0)
        {
            OracleCancelThread oraclecancelthread = null;
            try
            {
                oraclecancelthread = new OracleCancelThread(this, m_queryTimeout);
                oraclecancelthread.start();
                doExecuteQuery();
            }
            finally
            {
                oraclecancelthread.interrupt();
                try
                {
                    oraclecancelthread.join();
                }
                catch(InterruptedException _ex) { }
            }
        }
        else
        {
            doExecuteQuery();
        }
    }
}
```

图 8-47 queryTimeout 参数在 Oracle JDBC 中的使用方式

在这段代码中可以十分清晰地看到当设置了 `queryTimeout` 参数后, 在执行 SQL 时将会启动一个名为 `OracleCancelThread` 的线程, 并将当前的 `Statement` 对象作为参数传递进去, 也将超时的时间长度传递进去, 那么跟踪进去你会觉得很熟悉, 而且也会觉得操作十分简单, 见图 8-48。

这个 `OracleCancelThread` 继承于 `Thread` 类, 因此这里相当于启动一个线程进行休眠, 如果达到时间了, 则会执行 `cancel()`方法终止执行。如果外部执行结束后, 这个线程还依然在休眠, 那么外部会调用 `interrupt()`方法将其唤醒。

也就是当设置了 `queryTimeOut` 参数后, 每次执行 SQL 语句时, Oracle JDBC 都会单独创建一个线程来准备操作 `cancel()`的动作, 这个大家要注意了。


```
public class OracleCancelThread extends Thread
{
    public OracleCancelThread(OracleStatement oraclestatement, int i)
    {
        timeout = i * 1000;
        statement = oraclestatement;
    }

    public void run()
    {
        try
        {
            Thread.sleep(timeout);
            statement.cancel();
        }
        catch(SQLException _ex) { }
        catch(InterruptedException _ex) { }
    }
}
```

图 8-48 Oracle 的关闭线程就是 Sleep 一段时间，然后发起 cancel()

跟踪到 cancel()方法里面后，最终会跟踪到先前提到的 NSProtocol 类中的 sendBreak()方法，它会向 Oracle 服务器端发送一个指令，该指令告诉 Oracle 服务器端需要结束这个 SQL 语句的执行（但由于数据库端的复杂处理过程，这个取消未必能够绝对成功）。

跟踪一次代码，便知道了多个 JDBC 方法的意义，貌似小赚一笔。再来看看 MySQL 是不是也是这样做的。在其 executeQuery()方法中发现了这样的代码，见图 8-49。

```
try {
    if (locallyScopedConn.getEnableQueryTimeouts() &&
        this.timeoutInMillis != 0
        && locallyScopedConn.versionMeetsMinimum(5, 0, 0)) {
        timeoutTask = new CancelTask(this);
        locallyScopedConn.getCancelTimer().schedule(timeoutTask,
            this.timeoutInMillis);
    }
}
```

图 8-49 MySQL 中和 queryTimeOut 相关的代码

通过第 5 章的学习，应该清楚这段代码中的 CancelTask 代表执行的任务，很明显是一个清除连接的任务。它通过 getCancelTimer()来调用，它是一个 Connection 级别的 Timer 对象，在早期的版本中，它是一个 static 类型的 Timer 对象，与 Class 实例相关，算是一个单例。我们知道 Timer 为一个单独的线程在运行，也就是老版本的 JDBC 在同一个 ClassLoader 下的 MySQLJDBC 驱动实例，只会有一个 Timer 对象，MySQL 为了让 cancel()效率更高，就在每个任务启动时单独启动一个线程去执行它，如图 8-50 所示。

新版本中的 Timer 为 Connection 对象所拥有，即每个 Connection 对象中都有一个 Timer 对象，但是从源码中依然发现 MySQL 会在任务中启动一个线程来回收连接资源。这样一来，MySQL JDBC 也可能会分配很多的线程来回收资源。

按照常理来讲，一个连接对象中的 Statement 对象资源用一个线程回收已经足矣，但是有些时候往往并非完全如此，在流模式或游标模式下，如果使用了很复杂的 JOIN 操作（可能是一些全表扫描），返回的结果是庞大的结果集，在结果集获取尚未完成时，发起对 Statement 的 cancel()或关闭操作，则可能会延迟很长的时间（这个动作似乎在 MySQL 服务器端需要处理不少的清理工作才会返回）。对于新建一个线程处理完后释放，虽然有开销，不过也可以认为这样做是有必要的（只是 MySQL JDBC 将所有的问题一概而论了）。

```

public void run() {
    Thread cancelThread = new Thread() {
        public void run() {
            Connection cancelConn = null;
            java.sql.Statement cancelStmt = null;

            try {
                synchronized (cancelTimeoutMutex) {
                    cancelConn = connection.duplicate();
                    cancelStmt = cancelConn.createStatement();
                    cancelStmt.execute("KILL QUERY " + connectionId);
                    wasCancelled = true;
                }
            } catch (SQLException sqlEx) {
                caughtWhileCancelling = sqlEx;
            } catch (NullPointerException npe) {
                // 注: 这里...
            } finally {
                if (cancelStmt != null) {
                    try {
                        cancelStmt.close();
                    } catch (SQLException sqlEx) {
                        throw new RuntimeException(sqlEx.toString());
                    }
                }

                if (cancelConn != null) {
                    try {
                        cancelConn.close();
                    } catch (SQLException sqlEx) {
                        throw new RuntimeException(sqlEx.toString());
                    }
                }
            }
        }
    };
    cancelThread.start();
}

```

图 8-50 老版本的 JDBC 中 CancelTask 里面会单独启动一个线程来做

从图 8-50 中也同样看出了 cancel()操作其实是向 MySQL 服务器端发送一个 kill query 指令,指令中携带了一个 connectionId,这个 ID 就是 MySQL 服务器端为了与这个 Connection 保持连接分配的一个线程 ID。这个 ID 可以通过很多种方式来获取,而且不同的 JDBC 版本的 API 是不同的,不要依赖于 API,通过 SQL 语句 SELECT CONNECTION_ID()就可以拿到 MySQL 服务器端的连接 ID,这个 ID 与 MySQL 服务器端 show processlist 输出的 ID 是一样的。

如果大家想要依赖于 API 的版本来获取到 ID 也不是不可以,但是和版本确实有一些关系,例如发展到 JDBC 4,MySQL 相应的类名是 com.mysql.jdbc.JDBC4Connection,这个类是 ConnectionImpl 的子类,ConnectionImpl 中有一个方法 getId()获取到的就是这个 ID 值,但这个方法不是 public 的,所以要用反射来调用,可以这样来获取:

```

Method method
    = connection.getClass().getSuperclass().getDeclaredMethod("getId");
method.setAccessible(true);
System.out.println(method.invoke(connection));

```

早期的 JDBC 版本需要将 getSuperClass()去掉才能获取到。另外,也可以在 MyqllIO 类的 getThreadId()方法中获取到。总之,这个就要看具体源码了。

看了这些源码后你可能会会有更多的疑问去深入,通过本章学到的一些方法去跟踪源码,日积月累,对这方面的知识就会越来越熟悉,越来越深刻!

下一章讲解 Spring 框架的一些奥秘,有兴趣的小伙伴一起来看一看吧!

第9章

部分 Spring 源码讲解

本章主要内容

- Spring MVC
- Spring 事务管理器
- 思考：自己做框架有眉目了吗

本章大部分源码都以 Spring 3.x 为基础进行说明。

在第 7 章中胖哥挑选了许多 Web 层框架的基础知识来单独讲解，如果大家对相关的技术还不是很了解，则可以先看看第 7 章，然后再来看本章的内容，那样会方便很多。也许你看了本章的一些源码后会开始对框架有新的体会和见解。

9.1 Spring MVC

9.1.1 Spring 加载

在 Spring 3.x 后如果希望将 Spring 框架集成到 Web 应用中，通常只需要在 Web 应用的 web.xml 文件中配置 `org.springframework.web.servlet.DispatcherServlet` 即可，而不需要像 Spring 2.x 那样配置监听程序（`org.springframework.web.context.ContextLoaderListener`）。

对于使用过 Spring 的同学，可能会对 Spring 是如何加载类的，以及加载了哪些类比较感兴趣，因为在实际的工作中都在使用。在 Spring 2.x 时代，有配置文件还比较好理解，而在 Spring 3.x 后全是基于 Annotation 的配置（虽然还是支持配置文件，但大家开始接受注解的方式），Spring 又是如何加载的呢？

要了解 Spring 的加载自然要从 `DispatcherServlet` 类开始讲起，首先来了解这个类的相关父类关系，以便于在查找源码时更加轻松，如图 9-1 所示。

因为 `DispatcherServlet` 本身也是 Servlet 程序，因此它满足于 Servlet 的规范，跟踪 Servlet 的启动方法就是跟踪其 `init()` 方法（因为只有这个方法是在 Servlet

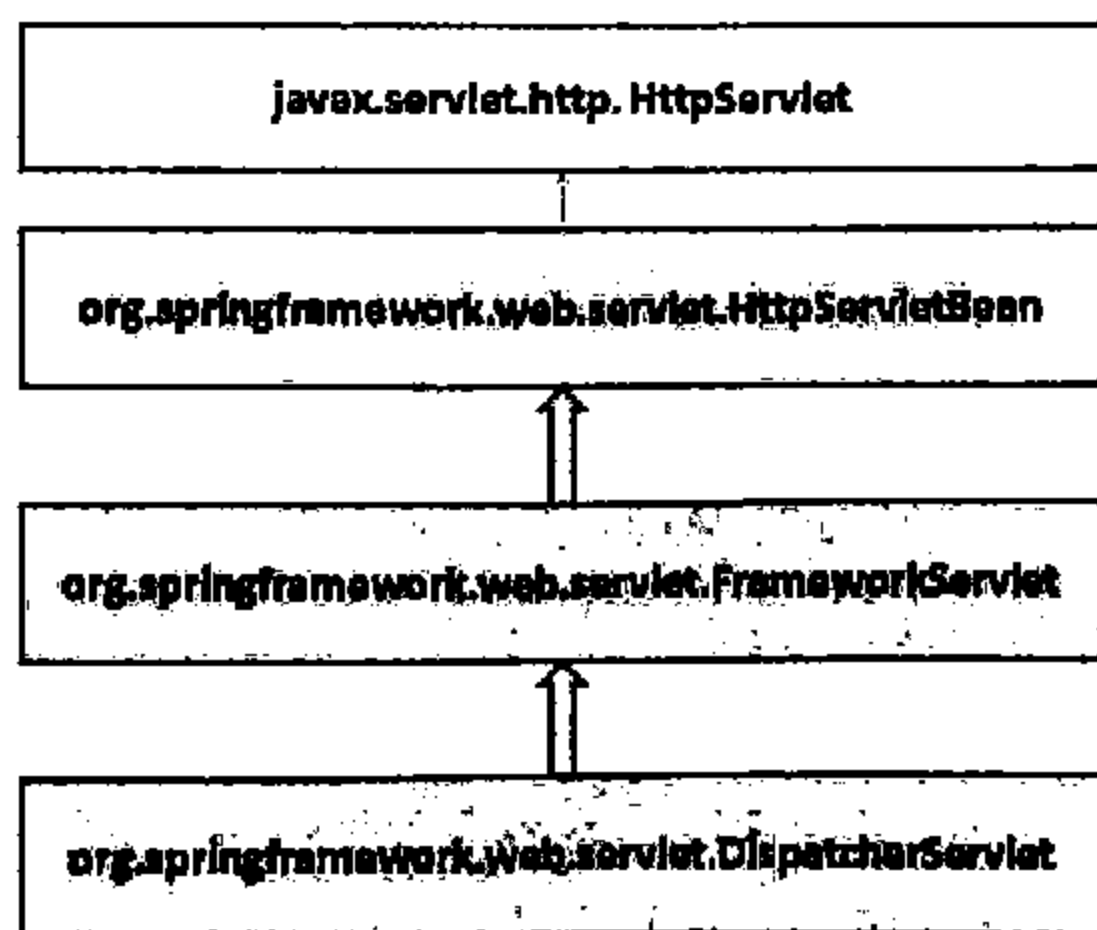


图 9-1 `DispatcherServlet` 上面还有两个父类

被加载时调用的)。这个方法隐藏在图 9-1 所示的父类 `HttpServletBean` 中，如图 9-2 所示。

```
public final void init() throws ServletException {
    if (logger.isDebugEnabled()) {
        logger.debug("Initializing servlet '" + getServletName() + "'");
    }

    // Set bean properties from init parameters.
    try {
        PropertyValues pvs = new ServletConfigPropertyValues(getServletConfig(), this.requiredProperties);
        BeanWrapper bw = PropertyAccessorFactory.forBeanPropertyAccess(this);
        ResourceLoader resourceLoader = new ServletContextResourceLoader(getServletContext());
        bw.registerCustomEditor(Resource.class, new ResourceEditor(resourceLoader, this.environment));
        initBeanWrapper(bw);
        bw.setPropertyValues(pvs, true);
    }
    catch (BeansException ex) {
        logger.error("Failed to set bean properties on servlet '" + getServletName() + "'", ex);
        throw ex;
    }

    // Let subclasses do whatever initialization they like.
    initServletBean();
}
```

图 9-2 在 `init()` 方法中，注意圈出的代码调用

在这段代码中，与 Spring 启动相关的细节需要继续跟踪 `initServletBean()` 方法，直接跟踪 `initServletBean()` 方法时发现其方法体为空，这是怎么回事呢？图 9-1 告诉我们 `HttpServletBean` 类还有子类，子类应该可以重写这个方法，果然不出我们所料，跟踪子类发现在 `FrameworkServlet` 类中重写了这个方法。主要代码如图 9-3 所示。

```
protected final void initServletBean() throws ServletException {
    getServletContext().log("Initializing Spring FrameworkServlet '" + getServletName() + "'");
    if (this.logger.isInfoEnabled()) {
        this.logger.info("FrameworkServlet '" + getServletName() + "': initialization started");
    }
    long startTime = System.currentTimeMillis();

    try {
        this.webApplicationContext = initWebApplicationContext();
        initFrameworkServlet();
    }
    catch (ServletException ex) {
        this.logger.error("Context initialization failed", ex);
        throw ex;
    }
    catch (RuntimeException ex) {
        this.logger.error("Context initialization failed", ex);
        throw ex;
    }
}
```

图 9-3 初始化 `ServletBean` 的信息

在这段代码中通过调用 `initWebApplicationContext()` 方法来创建 `ApplicationContext` 的实例，一般来讲，有 `WebApplicationContext` 对象实例时，就已经有 `Bean` 信息了，因此基本可以确定 `initWebApplicationContext()` 方法内部就已经将 `Bean` 信息组装好了。跟踪进去进一步看关键代码，在这个方法内部首先是查找 `WebApplicationContext`，如果没找到则会创建一个。代码如图 9-4 所示。

初始化时，系统肯定还没有创建 `WebApplicationContext` 对象，自然会创建一个新的 `WebApplicationContext` 对象。既然是初始化，又为何要做这样的判定呢？这段代码之所以这样判定，自然是为了通用化处理，换句话说，是给自己的代码留下出路，以便于更多的地方可以调用这段代码。


```
protected WebApplicationContext initWebApplicationContext() {
    WebApplicationContext wac = findWebApplicationContext();
    if (wac == null) {
        // No fixed context defined for this servlet - create a local one.
        WebApplicationContext parent =
            WebApplicationContextUtils.getWebApplicationContext(getServletContext());
        wac = createWebApplicationContext(parent);
    }
}
```

图 9-4 代码截图来自 Spring 3.0.0，在更高的版本中代码会有些变化

接下来将要关注的重点是在 createWebApplicationContext(parent)方法内部，在该方法的内部有一段比较关键的代码，如图 9-5 所示。

```
wac.setParent(parent);
wac.setServletContext(getServletContext());
wac.setServletConfig(getServletConfig());
wac.setNamespace(getNamespace());
wac.setConfigLocation(getContextConfigLocation());
wac.addApplicationListener(new SourceFilteringListener(wac, this));

postProcessWebApplicationContext(wac);
wac.refresh();
```

图 9-5 createWebApplicationContext(parent)方法的关键代码片段

在这段代码中有一个容易被忽略的 refresh()方法调用是 Spring 启动的核心部分，refresh()方法名虽然有刷新的意思，但是同样有初始化的概念在里面，换句话说，名称之所以定义为 refresh()，有可以反复调用、重新加载的意思在里面。

这个 refresh()方法来自于一个名称为 wac 的对象中，通过源码可以发现，这个对象的类型为 org.springframework.web.context.support.XmlWebApplicationContext，refresh()方法来自于其父类 AbstractApplicationContext，它负责完成初始化工作。

提到 AbstractApplicationContext 类，可以来看看这个父类到底有哪些常见的子类。通过一些 IED 工具一看便知道会有很多子类，如图 9-6 所示为一些比较常用的子类。

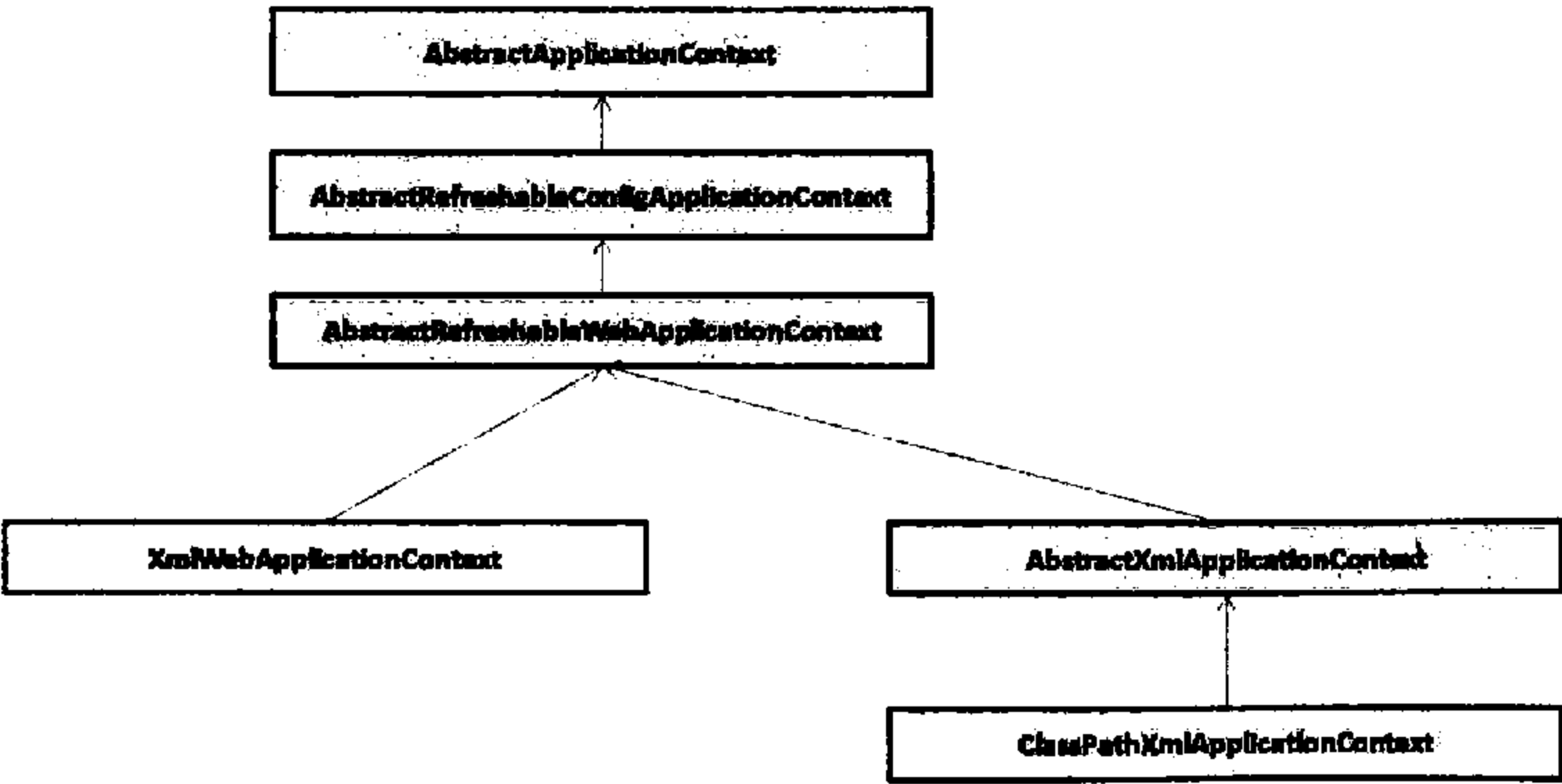


图 9-6 这些类归属于 org.springframework.web.context.support 这个 package 下

从图中可以看出的是，不论程序中使用 ClassPathXmlApplicationContext 还是 XmlWebApplicationContext 都会继承 AbstractApplicationContext 类，因此它们都有 refresh()方法来做类似的加载过程。为了印证这个结论，来看看 ClassPathXmlApplicationContext 中相关的代码，如图 9-7 所示。

```

public ClassPathXmlApplicationContext(String[] configLocations, boolean refresh, ApplicationContext parent)
    throws BeansException {

    super(parent);
    setConfigLocations(configLocations);
    if (refresh) {
        refresh();
    }
}

```

图 9-7 ClassPathXmlApplicationContext 中的 refresh()调用

由此可见,自己编写代码启动 Spring 时,可以使用 new ClassPathXmlApplicationContext() 或 new XmlWebApplicationContext()这两种方式。

有人在这里可能会问:使用 new XmlBeanFactory(new ClassPathResource("beans.xml")) 也可以创建一个 BeanFactory 对象,且同样可以通过 BeanFactory 的 getBean()方法来获取 Spring 所注入的实例。这又是怎么回事呢?请看下面的一些解释。

在图 9-6 所示的类继承关系中还有一个共同的父类 AbstractRefreshableApplicationContext,其内部包含了一个属性定义为:private DefaultListableBeanFactory beanFactory;,当通过 ApplicationContext 的实例调用 getBean()方法时,其实是调用这个属性的 getBean()方法,如图 9-8 所示。

```

public Object getBean(String name) throws BeansException {
    return getBeanFactory().getBean(name);
}

public <T> T getBean(String name, Class<T> requiredType) throws BeansException {
    return getBeanFactory().getBean(name, requiredType);
}

public <T> T getBean(Class<T> requiredType) throws BeansException {
    return getBeanFactory().getBean(requiredType);
}

```

图 9-8 getBean()获取实例其实最终是通过 beanFactory 对象来完成的

上面提到的 XmlBeanFactory 类为 DefaultListableBeanFactory 的子类,它的初始化也是靠 DefaultListableBeanFactory 中的相应方法完成的,如图 9-9 所示。

```

public XmlBeanFactory(Resource resource) throws BeansException {
    this(resource, null);
}

/**
 * Create a new XmlBeanFactory with the given input stream,
 * which must be parsable using DOM.
 * @param resource XML resource to load bean definitions from
 * @param parentBeanFactory parent bean factory
 * @throws BeansException in case of loading or parsing errors
 */
public XmlBeanFactory(Resource resource, BeanFactory parentBeanFactory) throws BeansException {
    super(parentBeanFactory);
    this.reader.loadBeanDefinitions(resource);
}

```

图 9-9 最终调用 DefaultListableBeanFactory 的 loadBeanDefinitions()方法完成加载

综合起来讲,Bean 的初始化重点是 beanFactory,外部的各种 ApplicationContext 只是

对 beanFactory 的一层包装而已，包装的原因当然是为了适应各种不同的入参。

胖哥为何又把话题扯远了？目的在于为了让大家理解有很多方法可以创建 Spring 的全局实例，只是它们都穿着美丽的外衣，但是内在的实现实体都是一样的。

回到图 9-5 所示的 refresh()方法，继续跟踪 AbstractApplicationContext 类的 refresh()方法，其方法内部调用了 obtainFreshBeanFactory()方法构造了 beanFactory 对象，再间接调用了 refreshBeanFactory()方法来完成。

refreshBeanFactory()与 refresh()方法一样，它也是创建+刷新的组合，此方法所在的类就是图 9-6 所示的 AbstractRefreshableApplicationContext，其代码如图 9-10 所示。

```
protected final void refreshBeanFactory() throws BeansException {
    if (hasBeanFactory()) {
        destroyBeans();
        closeBeanFactory();
    }
    try {
        DefaultListableBeanFactory beanFactory = createBeanFactory();
        beanFactory.setSerializationId(getId());
        customizeBeanFactory(beanFactory);
        loadBeanDefinitions(beanFactory);
        synchronized (this.beanFactoryMonitor) {
            this.beanFactory = beanFactory;
        }
    }
    catch (IOException ex) {
        throw new ApplicationContextException("I/O error parsing bean definition source for " + getDisplayName());
    }
}
```

图 9-10 AbstractRefreshableApplicationContext.refreshBeanFactory()方法

这里有一个 createBeanFactory()方法用于创建一个 beanFactory 实例，可是跟踪进去发现代码里面仅仅是 new DefaultListableBeanFactory()的动作，没有看到类的加载动作，难道类加载不是在这里操作的吗？难道到此结束了吗？不是，还得继续看。

图 9-9 中提到 XmlBeanFactory()构造方法，它调用 super.reader.loadBeanDefinitions()方法，在这个方法体里面也看到了一个 loadBeanDefinitions()的动作，虽然它没有调用 reader.loadBeanDefinitions()，但是很有可能包装在方法内部，带着好奇心一起进去看看吧。

经过层层跟踪，跟踪到 XmlWebApplicationContext 类的 loadBeanDefinitions()里面了，看了它的代码，这段代码与 XmlBeanFactory()的构造非常相似，如图 9-11 所示。

```
protected void loadBeanDefinitions(XmlBeanDefinitionReader reader) throws IOException {
    String[] configLocations = getConfigLocations();
    if (configLocations != null) {
        for (String configLocation : configLocations) {
            reader.loadBeanDefinitions(configLocation);
        }
    }
}
```

图 9-11 XmlWebApplicationContext 中加载 Bean 的列表

看到这段代码，我们开始看到了一些希望。接下来还有很多中间处理，胖哥不再做烦琐的列举，将中间部分的主要路径列举出来给大家。

```
AbstractBeanDefinitionReader.loadBeanDefinitions(Resource [] resource) -> 循环调用下面的方法
XmlBeanDefinitionReader.loadBeanDefinitions(Resource resource) -> 具体子类实现资源读取
```

```

XmlBeanDefinitionReader.doLoadBeanDefinitions(Resource resource)           ->间接调用
XmlBeanDefinitionReader.registerBeanDefinitions(Document doc, Resource resource) ->
DefaultBeanDefinitionDocumentReader.registerBeanDefinitions(Document doc
    , XmlReaderContext readerContext)           ->开始准备解析 XML 了
DefaultBeanDefinitionDocumentReader.parseBeanDefinitions(Element root
    , BeanDefinitionParserDelegate delegate) ->解析根节点
DefaultBeanDefinitionDocumentReader.parseCustomElement(Element ele
    , BeanDefinitionContainingBd)           ->解析自定义的节点

```

跟踪到 `parseCustomElement()` 这个方法的源码，现在又到了代码逻辑比较关键的地方，见图 9-12。

```

public BeanDefinition parseCustomElement(Element ele, BeanDefinition containingBd) {
    String namespaceUri = getNamespaceURI(ele);
    NamespaceHandler handler = this.readerContext.getNamespaceHandlerResolver().resolve(namespaceUri);
    if (handler == null) {
        error("Unable to locate Spring NamespaceHandler for XML schema namespace [" + namespaceUri + "]",
            return null;
    }
    return handler.parse(ele, new ParserContext(this.readerContext, this, containingBd));
}

```

图 9-12 `parseCustomElement()` 的源码

代码逻辑是根据节点来获取一个 `namespaceUri`，然后再根据这个 `namespaceUri` 获取一个 `NamespaceHandler` 的实例对象来解析处理。

那么问题出现了，这个 `ele` 对象我们可以简单地认为是 XML 中的一个配置项，这个 `namespaceUri` 到底是什么？获取了一个什么样的 `NamespaceHandler` 来处理？在这里我们需要展开一个话题，这个话题需要回到配置文件上面。

在 Spring 3.x 中，通常会配置以下一段参数：

```
<context:component-scan base-package="com.xxx" />
```

或者配置为：

```
<context:component-scan base-package="*" />
```

这里的 `context` 就成为一个关键字，在方法中它会变成什么呢？它会根据 Spring 的 XML 配置文件中头部的定义来获取命名空间的信息，头部是什么样的呢？见图 9-13。

```

<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:aop="http://www.springframework.org/schema/aop"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:avc="http://www.springframework.org/schema/avc"
    xmlns:tx="http://www.springframework.org/schema/tx"
    xmlns:util="http://www.springframework.org/schema/util" xmlns:p
    xsi:schemaLocation="http://www.springframework.org/schema/beans

```

图 9-13 这就是头部的命名空间的定义

Spring 配置文件中每一个关键字都会对应 XML 头部的一个命名空间，例如关键字 `context` 命名空间 `xmlns:context` 对应到 `http://www.springframework.org/schema/context` 上，这个名称很像 URL，容易“忽悠”我们，有些时候我们认为需要到网上去找资源，其实未必，如果不是这样，通过这个 URL 怎么找到处理类呢？

它来自于每个 Spring 的 jar 包中 META-INF 下面的 `spring.handlers` 文件信息中，而 `context` 关键字所对应的 jar 包自然是 `spring-context-xxx.jar` 包，解开该 jar 包后发现其内容为：


```

http://www.springframework.org/schema/context=org.springframework.context.config.ContextNamespaceHandler
http://www.springframework.org/schema/jee=org.springframework.ejb.config.JeeNamespaceHandler
http://www.springframework.org/schema/lang=org.springframework.scripting.config.LangNamespaceHandler
http://www.springframework.org/schema/task=org.springframework.scheduling.config.TaskNamespaceHandler

```

对于 context，当然对应到 org.springframework.context.config.ContextNamespaceHandler 类，来看看这个类中写了些什么代码，如图 9-14 所示。

```

public void init() {
    registerBeanDefinitionParser("property-placeholder", new PropertyPlaceholderBeanDefinitionParser());
    registerBeanDefinitionParser("property-override", new PropertyOverrideBeanDefinitionParser());
    registerBeanDefinitionParser("annotation-config", new AnnotationConfigBeanDefinitionParser());
    registerBeanDefinitionParser("component-scan", new ComponentScanBeanDefinitionParser());
    registerBeanDefinitionParser("load-time-weaver", new LoadTimeWeaverBeanDefinitionParser());
    registerBeanDefinitionParser("spring-configured", new SpringConfiguredBeanDefinitionParser());
    registerBeanDefinitionParser("mbean-export", new MBeanExportBeanDefinitionParser());
    registerBeanDefinitionParser("mbean-server", new MBeanServerBeanDefinitionParser());
}

```

图 9-14 自然用的是 ComponentScanBeanDefinitionParser 来处理

根据 9-12 所示的 parse() 方法来讲，确定调用 ComponentScanBeanDefinitionParser.parse() 方法，通过名称可以认识到这个方法是专门用于对 Bean 定义的扫描器，在 Spring MVC 的配置中，还有许多节点类型，它们都对应到不同的处理程序，这样就不难理解为何会有这么多层的解析和选择了。

目前我们仅仅拿到一个 Spring 配置文件中的 context:component-scan 节点的内容，离“修成正果”还早（我们需要根据该节点的描述，找到所需要扫描的类），因此需要继续跟踪看这个方法内部做了什么，见图 9-15。

```

public BeanDefinition parse(Element element, ParserContext parserContext) {
    String[] basePackages = StringUtils.tokenizeToStringArray(element.getAttribute(BASE_PACKAGE_ATTRIBUTE),
        ConfigurableApplicationContext.CONFIG_LOCATION_DELIMITERS);

    // Actually scan for bean definitions and register them.
    ClassPathBeanDefinitionScanner scanner = configureScanner(parserContext, element);
    Set<BeanDefinitionHolder> beanDefinitions = scanner.doScan(basePackages);
    registerComponents(parserContext.getReaderContext(), beanDefinitions, element);

    return null;
}

```

图 9-15 parse() 方法的内容

方法内定义了一个 scanner，通过名称可以猜测到就是扫描类的信息列表，这里的 basePackage 就是在配置文件中配置的 base-package 属性的内容，在 Spring 配置文件中可以配置多个 base-package 路径，多个 base-package 路径之间可以由“,”、“;”、空格、“\t”、“\n”这些字符中的任意一个来分隔开。为何胖哥知道这些呢？请参看源码 org.springframework.util.StringUtils.tokenizeToStringArray(String, String)，以及参数 ConfigurableApplicationContext.CONFIG_LOCATION_DELIMITERS 的值，真相便明了。

找到了基本路径，就要开始找类了，因为配置都是基于 Annotation 的，而不是基于 XML 配置文件的，有哪些类需要被加载是未知的，因此小伙伴应当对类的扫描比较感兴趣，那么自然会考虑 scanner.doScan() 到底做了什么，见图 9-16。

```

protected Set<BeanDefinitionHolder> doScan(String... basePackages) {
    Set<BeanDefinitionHolder> beanDefinitions = new LinkedHashSet<BeanDefinitionHolder>();
    for (String basePackage : basePackages) {
        Set<BeanDefinition> candidates = findCandidateComponents(basePackage);
        for (BeanDefinition candidate : candidates) {
            ScopeMetadata scopeMetadata = this.scopeMetadataResolver.resolveScopeMetadata(candidate);
            candidate.setScope(scopeMetadata.getScopeName());
            String beanName = this.beanNameGenerator.generateBeanName(candidate, this.registry);
            if (candidate instanceof AbstractBeanDefinition) {
                postProcessBeanDefinition((AbstractBeanDefinition) candidate, beanName);
            }
            if (candidate instanceof AnnotatedBeanDefinition) {
                AnnotationConfigUtils.processCommonDefinitionAnnotations((AnnotatedBeanDefinition) candidate);
            }
            if (checkCandidate(beanName, candidate)) {
                BeanDefinitionHolder definitionHolder = new BeanDefinitionHolder(candidate, beanName);
                definitionHolder = AnnotationConfigUtils.applyScopedProxyMode(scopeMetadata, definitionHolder, this);
                beanDefinitions.add(definitionHolder);
                registerBeanDefinition(definitionHolder, this.registry);
            }
        }
    }
}

```

图 9-16 doScan()做的事情

这段代码比较长，大家拆分来看，外部的循环可以抛弃掉，为何？因为循环的是 base-package，我们可以认为只有一个 base-package，那么循环只有一次。在循环体中，首先根据 basePackage 找一个 Set 列表，这个列表中存放的就是 Bean 的定义信息，初步判定应该是我们想要的内容。再看看后面的代码，都是处理注解相关的内容，那么再次可以确定它确实是启动过程中需要找到类列表的方法。

findCandidateComponents(String)成为关注的重点，经历重重源码的痛苦跟踪，现在终于快接近真相了，下面继续来看这部分源码，如图 9-17 所示。

```

public Set<BeanDefinition> findCandidateComponents(String basePackage) {
    Set<BeanDefinition> candidates = new LinkedHashSet<BeanDefinition>();
    try {
        String packageSearchPath = ResourcePatternResolver.CLASSPATH_ALL_URL_PREFIX +
            resolveBasePackage(basePackage) + "/" + this.resourcePattern;
        Resource[] resources = this.resourcePatternResolver.getResources(packageSearchPath);
        boolean traceEnabled = logger.isTraceEnabled();
        boolean debugEnabled = logger.isDebugEnabled();
        for (Resource resource : resources) {
            if (traceEnabled) {
                logger.trace("Scanning " + resource);
            }
        }
    }
}

```

图 9-17 findCandidateComponents()的代码

这段代码中对 package 做了一个自定义的组装，它会组装成什么样子呢？

- ◎ 当 base-package 参数为 com.abc 时，组装为：classpath*:com/abc/**/*.class。
- ◎ 当 base-package 参数为*时，组装为：classpath*:/**/*.class。

大家需要注意的是，这是 Spring 自己内部程序的规范，不是通用的规范，不是在什么地方都可以使用的。在这个方法中会调用一个名为 resourcePatternResolver 的对象的 getResources()方法来获取资源列表，为了了解其细节，我们来看看 resourcePatternResolver 的定义：

```

private ResourcePatternResolver resourcePatternResolver
    = new PathMatchingResourcePatternResolver()

```

该类的全名为：org.springframework.core.io.support.PathMatchingResourcePatternResolver，

它的 `getResources()` 方法就可以得到资源列表，为了证明这一点，我们可以单独写一段程序来模拟一下：

```
ResourcePatternResolver resourcePatternResolver
    = new PathMatchingResource PatternResolver();
Resource[] resources
    = resourcePatternResolver.getResources("classpath*:com/ abc/**/*.*.class");
```

运行这两条代码，应该能通过这个方法得到想要的结果，它会列出所有 `classpath` 路径下和 `com/abc/` 相关的信息。于是，胖哥猜测它应该是通过 `getClassLoader().getResources()` 来完成的，接下来我们一起带着猜测和印证来阅读 `getResources(String)` 方法，如图 9-18 所示。

```
public Resource[] getResources(String locationPattern) throws IOException {
    Assert.notNull(locationPattern, "Location pattern must not be null");
    if (locationPattern.startsWith(CLASSPATH_ALL_URL_PREFIX)) {
        // a class path resource (multiple resources for same name possible)
        if (getPathMatcher().isPattern(locationPattern.substring(CLASSPATH_ALL_URL_PREFIX.length()))) {
            // a class path resource pattern
            return findPathMatchingResources(locationPattern);
        }
        else {
            // all class path resources with the given name
            return findAllClassPathResources(locationPattern.substring(CLASSPATH_ALL_URL_PREFIX.length()));
        }
    }
}
```

图 9-18 `getResources(String)` 方法

代码中所匹配的常量：`CLASSPATH_ALL_URL_PREFIX` 的值是“`classpath:*`”，因此必然会进入最外层的 `if` 判定逻辑中，而里面的 `getPathMatcher().isPattern()` 方法，需要跟踪 `AntPathMatcher` 类的 `isPattern()` 方法（类全名为：`org.springframework.util.AntPathMatcher`）。总体逻辑就是将传入的参数，截取掉“`classpath:*`”前缀后的部分后，然后判定其是否满足表达式格式。`AntPathMatcher.isPattern()` 的代码如图 9-19 所示。

```
public boolean isPattern(String path) {
    return (path.indexOf('*') != -1 || path.indexOf('?') != -1);
}
```

图 9-19 它主要判定字符串中是否包含“*”或“?”符号

通过上面的分析，传入的 `package` 路径总会带有 `**/*.class` 这样的信息，因此这个判定条件肯定会成立，自然的也会进入 `findPathMatchingResources(locationPattern)` 方法中，其代码如图 9-20 所示。

这段代码中会获取一个 `rootDirPath`，然后截取一次字符串，再回到图 9-18 中所描述的 `getResources(String)` 方法中，但是这一次进入 `getResources(String)` 方法时会进入它的第 2 个逻辑：`findAllClassPathResources()` 中，为何呢？因为这一次传入的不再是路径参数，而是 `rootDirPath` 变量，它是什么样的格式呢？我们需要跟踪 `determineRootDir()` 方法的细节才能清楚，一起来看看。

在图 9-20 中，`rootDirPath` 的结果来自于 `determineRootDir()` 方法，它主要做截取字符串的操作，首先找到第一个“`:`”出现的位置，然后循环，每次循环中通过 `lastIndexOf("/")` 方法找到字符串的最后一个反斜杠，截取冒号到这个反斜杠中间的字符串，继续根据这个字符串循环，直到这个字符串中没有“*”和“?”为止，或者说通过 `AntPathMatcher.isPattern()` 无法匹配为止，找到最终位置后，截取字符串从下标 0 到该位置，并将截取的子字符串返回，如图 9-21 所示。

```
protected Resource[] findPathMatchingResources(String locationPattern) throws IOException {
    String rootDirPath = determineRootDir(locationPattern);
    String subPattern = locationPattern.substring(rootDirPath.length());
    Resource[] rootDirResources = getResources(rootDirPath);
    Set<Resource> result = new LinkedHashSet<>(16);
    for (Resource rootDirResource : rootDirResources) {
        if (isJarResource(rootDirResource)) {
            result.addAll(doFindPathMatchingJarResources(rootDirResource, subPattern));
        }
        else if (rootDirResource.getURL().getProtocol().startsWith(ResourceUtils.URL_PROTOCOL_VFS)) {
            result.addAll(VfsResourceMatchingDelegate.findMatchingResources(rootDirResource, subPattern, getPathMatcher()));
        }
        else {
            result.addAll(doFindPathMatchingFileResources(rootDirResource, subPattern));
        }
    }
    if (logger.isDebugEnabled()) {
        logger.debug("Resolved location pattern [" + locationPattern + "] to resources " + result);
    }
    return result.toArray(new Resource[result.size()]);
}
```

图 9-20 findPathMatchingResources()方法

```
protected String determineRootDir(String location) {
    int prefixEnd = location.indexOf(":") + 1;
    int rootDirEnd = location.length();
    while (rootDirEnd > prefixEnd && getPathMatcher().isPattern(location.substring(prefixEnd, rootDirEnd))) {
        rootDirEnd = location.lastIndexOf('/', rootDirEnd - 2) + 1;
    }
    if (rootDirEnd == 0) {
        rootDirEnd = prefixEnd;
    }
    return location.substring(0, rootDirEnd);
}
```

图 9-21 截取关键字字符串

如果传入的参数是 classpath*:com/abc/**/*.class，则自然会返回 classpath*:com/abc/；而如果传入参数 classpath*:*/**/*.class，则会返回 classpath*:.，而不是 classpath*:.*。

当图 9-20 中的代码再次调用 getResources(String)时，会传入这些修改后的字符串，该字符串也是以“classpath*.”为前缀的，所以在图 9-18 的代码中，外层的 if 判定语句也是会成立的，但是当继续判定 getPathMatcher().isPattern()的结果时，就肯定不会成功了，因为在截取字符串时，已经将匹配上的字符串全部截取掉了，或者说图 9-21 的循环代码退出的条件就是需要去掉所有能匹配的字符：“*”、“?”，因此必然会走第 2 个逻辑，调用 findAllClassPathResources()方法，其方法内容如图 9-22 所示。

```
protected Resource[] findAllClassPathResources(String location) throws IOException {
    String path = location;
    if (path.startsWith("/")) {
        path = path.substring(1);
    }
    Enumeration resourceUris = getClassLoader().getResources(path);
    Set<Resource> result = new LinkedHashSet<Resource>(16);
    while (resourceUris.hasMoreElements()) {
        URL url = (URL) resourceUris.nextElement();
        result.add(convertClassLoaderURL(url));
    }
    return result.toArray(new Resource[result.size()]);
}
```

图 9-22 检查出所有 class 信息的列表

参数入口截取了 `classpath*` 这个前缀，就只剩下真正的 URL 信息了，也就是最终和我们配置的 `base-package` 基本是一样的，只有一点区别是，如果配置的是 “*”，那么这里传入的是空字符串：“”，或者说是长度为 0 的字符串，而不是 `null` 值，也不是 “*”。

果然不出我们所料，是用 `getClassLoader().getResources()` 方法来实现的，为此大家可以自己写一段程序来测试这样的方法：`XXX.class.getClassLoader().getResources(String)` 来获取相应类的路径列表（这个代码是 Java 本身提供的，无须三方工具的支持），便会有所体会。

例子中使用的是 “`com/abc/`” 或 “”（长度为 0 的字符串）作为参数，前者将会找出所有 `classpath` 目录下与 `com/abc/` 相关的路径，包括 `jar` 包中的路径；而后者会找出 `classpath` 的根路径，但是不会遍历到 `jar` 包里面，也就是它不会把 `jar` 包作为 `classpath` 根目录之一。换句话说，如果你需要加载的 `Bean` 写在了 `jar` 包中，那么在配置文件中配置的 `base-package=“*”` 是扫描不到的，原因就不用多说了。

接下来回到图 9-20 中，代码中会根据得到的 `Resource` 列表循环，如果是不同的文件类型，则采用不同的读取方法，将 `class` 文件递归出来。既然 `class` 文件能递归出来，还有什么问题吗？`class` 信息获取到后，就有办法加载了，加载后，自然就有方法获取 `Annotation` 的信息了（这些方法我们在第 7 章中讲解过），大家有兴趣可以回到先前提到的代码段中查看处理的细节。

大家需要注意：虽然 `base-package` 能扫描到 `jar` 包里面的内容，但是如果 `jar` 包中还包含 `jar` 包是扫描不到的，首先 `getClassLoader().getResources()` 方法不会去递归 `jar` 包来扫描，而只会关注 `classpath` 当前目录下的文件以及 `jar` 包，`jar` 包下面的 `jar` 包只会被当作是普通的文件。它匹配的是以 `.class` 为后缀的文件名，同理也解释了：如果配置中使用了 `base-package=“*”`，扫描器不会扫描 `jar` 包中的 `Class` 文件。

Spring 的启动相对较为复杂，而且胖哥在这里也仅仅介绍了基于 `Annotation` 的扫描方式，在 Spring 中还有其他方式，大家有兴趣可以自己参看。

而本节中胖哥在查看源码时遇到了许多困难，经过层层剥离、猜测、印证、迭代、尝试才得到结论，不知这种方法你学到了吗？其实有些时候确实需要点土办法。

►► 9.1.2 Spring MVC 处理一个简单请求

看了上面的 Spring 的启动加载 `Bean`，结合在第 7 章中提到的 `Annotation` 与反射相关的技术，应该可以猜测到一个外部请求到来后，Spring 会如何选择资源，或者说一个请求到来时，Spring 会选择哪个 `Controller` 的哪个方法来处理。对于源码我们依然“刨根问底”，本节就来看看一个请求分发是如何操作的。

前文中提到，在 Web 应用中，Spring 需要配置一个 `DispatchServlet`，它实现了 `Servlet` 的标准接口，在启动时跟踪了它的 `init()` 方法，请求时就跟踪其 `doGet()` 和 `doPost()` 方法，这两个方法包装在 `FrameworkServlet` 类中，代码如图 9-23 所示。

```
@Override
protected final void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}
@Override
protected final void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}
```

图 9-23 它们都间接调用 `processRequest()` 方法来处理

进入 `processRequest()` 方法，经过设置相应的参数后，继而调用 `doService()` 方法。`processRequest()` 方法里面有一个比较有意思的地方是 `catch` 异常处理时，最后有一个 `catch(Throwable ex)` 的操作，这个 `catch` 是可以捕获到 `Error`、`Exception` 在内的所有信息的，除非 JVM crash 掉，否则从请求线程抛出的错误在这里都能捕获到做相应的处理（大家自己写框架时，就要考虑是否需要用类似的方式来完成）。实际的代码如图 9-24 所示。

```
try {
    doService(request, response);
}
catch (ServletException ex) {
    failureCause = ex;
    throw ex;
}
catch (IOException ex) {
    failureCause = ex;
    throw ex;
}
catch (Throwable ex) {
    failureCause = ex;
    throw new NestedServletException("Request processing failed", ex);
}
```

图 9-24 调用 `doService()` 的异常处理列表

另外，在这个方法的前面设置参数的部分还有这样一段代码，见图 9-25。

```
RequestAttributes previousRequestAttributes = RequestContextHolder.getRequestAttributes(),
ServletRequestAttributes requestAttributes = null;
if (previousRequestAttributes == null || previousRequestAttributes.getClass().equals(ServletRequestAttributes.class)) {
    requestAttributes = new ServletRequestAttributes(request);
    RequestContextHolder.setRequestAttributes(requestAttributes, this.threadContextInheritable);
}
```

图 9-25 将 `request` 放入到一个对象中

这段代码有什么用途呢？其实在某些方法中，如果不方便于传入 `request`，则可通过调用 `((ServletRequestAttributes)RequestContextHolder.getRequestAttributes()).getRequest()` 来获取到，当然，如果不是 Web 应用这样的结果肯定是有问题的，自己加以封装便可以通用化。

在进入 `doService()` 方法中，做了许多参数设置后，继而调用 `doDispatch()` 方法开始正式做分发操作，在这些参数设置中大家可能会觉得很多都没有多大用途，不过胖哥认为有一个参数可能大家会用得上，那就是 Spring 的 `ApplicationContext` 信息，在这里通过：

```
request.setAttribute(WEB_APPLICATION_CONTEXT_ATTRIBUTE,
getWebApplicationContext());
```

代码将 `webApplication` 放入 `request` 的变量中，`WEB_APPLICATION_CONTEXT_ATTRIBUTE` 是一个 `public` 的静态变量，说明通过 `request.getAttribute()` 能轻松获取到 `ApplicationContext` 的引用。当然获取 Spring 的 `ApplicationContext` 信息有很多种方式，这里仅仅是其中一种而已。需要注意的是，在 Spring 3.x 中获取 `ApplicationContext` 的方式与 Spring 2.x 有很大的区别（在 Spring 2.x 中是使用一个叫“`WebApplicationContextUtils`”的工具类来获取的），除非在 Spring 3.x 中也配置了一个 `Listener`，才可以拥有和 Spring 2.x 同样的获取 `ApplicationContext` 的方式。

回到请求处理的正题上来，在 `doDispatch()` 方法中主要做如下几件事。

- ◎ 校验 `request` 是否为“`Multipart`”，如果是则会通过 `CommonsMultipartResolver` 类的 `resolveMultipart()` 方法对 `request` 进行处理，对其参数和文件进行处理。
- ◎ 取出拦截器列表 (`HandlerInterceptor[]`)，循环该列表并调用每个拦截器的 `preHandle()`

方法，如果返回 false，则调用所有拦截器的 afterCompletion()方法后退出（不再调用业务代码）。

- ◎ 通过 HandlerAdapter.handle()调用业务代码，也就是调用自己编写的程序。
- ◎ 设置渲染的 ModelAndView，如果没有则不进行设置操作。
- ◎ 再次循环拦截器列表，调用其 postHandle()方法。
- ◎ 再次循环拦截器列表，调用其 afterCompletion()方法。

有人问：胖哥，你的这些东西是在哪个网站里面抄过来的？
回答：胖哥没有从哪里去抄，源码就是这样写的，源码中的关键部分如图 9-26 所示。

```
protected void doDispatch(HttpServletRequest request, HttpServletResponse response) throws Exception {
    HttpServletRequest processedRequest = request;
    HandlerExecutionChain mappedHandler = null;
    int interceptorIndex = -1;

    try {
        ModelAndView mv;
        boolean errorView = false;

        try {
            processedRequest = checkMultipart(request); 1. 处理Multipart
            //中间代码省略...
            // Apply preHandle methods of registered interceptors. 2. 循环调用Interceptor的preHandle()方法
            HandlerInterceptor[] interceptors = mappedHandler.getInterceptors();
            if (interceptors != null) {
                for (int i = 0, i < interceptors.length; i++) {
                    HandlerInterceptor interceptor = interceptors[i];
                    if (!interceptor.preHandle(processedRequest, response, mappedHandler.getHandler())) {
                        triggerAfterCompletion(mappedHandler, interceptorIndex, processedRequest, response, null);
                        return;
                    }
                    interceptorIndex = i;
                }
            }

            // Determine handler adapter for the current request.
            HandlerAdapter ha = getHandlerAdapter(mappedHandler.getHandler());
            // Actually invoke the handler.
            mv = ha.handle(processedRequest, response, mappedHandler.getHandler()); 3. 获取并调用业务方法，返回ModelAndView
            // Do we need view name translation?
            if (mv != null && !mv.hasView()) {
                mv.setViewName(getDefaultViewName(request)); 4. 设置默认的渲染页面（若没有返回MV，且业务方法没有传入HttpServletResponse）
            }

            // Apply postHandle methods of registered interceptors. 5. 循环调用Interceptor的postHandle()方法
            if (interceptors != null) {
                for (int i = interceptors.length - 1; i >= 0; i--) {
                    HandlerInterceptor interceptor = interceptors[i];
                    interceptor.postHandle(processedRequest, response, mappedHandler.getHandler(), mv);
                }
            }
        } catch (Exception ex) {
            // ...
            6. 无论是否异常都会调用triggerAfterCompletion()方法，该方法会循环调用每一个Interceptor的afterCompletion()方法。
            // Trigger after-completion for successful outcome.
            triggerAfterCompletion(mappedHandler, interceptorIndex, processedRequest, response, null);
        } catch (Exception ex) {
            // Trigger after-completion for thrown exception.
            triggerAfterCompletion(mappedHandler, interceptorIndex, processedRequest, response, ex);
            throw ex;
        } catch (Error err) {
            ServletException ex = new ServletException("Handler processing failed", err);
            // Trigger after-completion for thrown exception.
            triggerAfterCompletion(mappedHandler, interceptorIndex, processedRequest, response, ex);
            throw ex;
        }
    }
}
```

图 9-26 doDispatch()的源码步骤说明

知道了这些，就清楚了拦截器的调用顺序，也自然知道了在什么情况下拦截器该做什么操作，拦截器的什么方法在什么情况下是必然会被调用的，在什么情况下不会被调用。在心中有了这样一个概念后，写拦截器时应当会更加游刃有余。

根据图 9-26 中的步骤 3 可以看出，在 HandlerAdapter 类的 handle()方法中将调用具体的业务方法，于是我们跟踪进去，HandlerAdapter 是一个接口，handle()方法在多个实现类中有实现体，如图 9-27 所示。

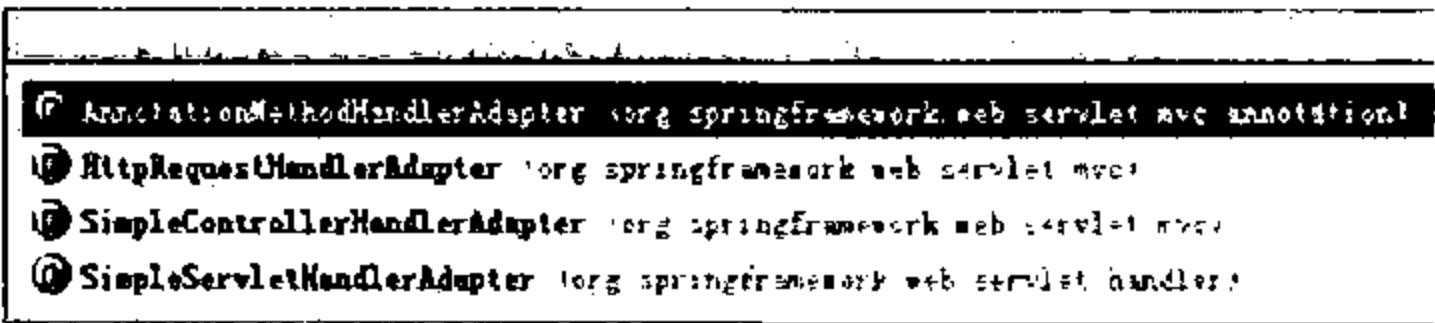


图 9-27 HandlerAdapter 的实现类信息

在 Spring 3.x 后，在大多数情况下以使用 Annotation 为主，跟踪源码时，自然优先选择使用 AnnotationMethodHandlerAdapter 类的对应方法（即使用的不是这个类，在看完源码后，再看其他的实现类也都会发现其中有许多相似的处理）。在 handle()方法中经过一些处理后，继续调用了 invokeHandlerMethod()方法来处理，这个方法的代码如图 9-28 所示。

```
protected ModelAndView invokeHandlerMethod(HttpServletRequest request, HttpServletResponse response, Object handler)
    throws Exception {

    ServletHandlerMethodResolver methodResolver = getMethodResolver(handler);
    Method handlerMethod = methodResolver.resolveHandlerMethod(request);
    ServletHandlerMethodInvoker methodInvoker = new ServletHandlerMethodInvoker(methodResolver);
    ServletWebRequest webRequest = new ServletWebRequest(request, response);
    ExtendedModelMap implicitModel = new BindingAwareModelMap();

    Object result = methodInvoker.invokeHandlerMethod(handlerMethod, handler, webRequest, implicitModel);
    ModelAndView mav =
        methodInvoker.getModelAndView(handlerMethod, handler.getClass(), result, implicitModel, webRequest);
    methodInvoker.updateModelAttributes(handler, (mav != null ? mav.getModel() : null), implicitModel, webRequest);
    return mav;
}
```

图 9-28 invokeHandlerMethod()方法的代码

在这段代码中，通过 methodResolver.resolveHandlerMethod(request)得到需要调用业务方法的对象：handlerMethod，并将其作为 methodInvoker.invokeHandlerMethod()方法的参数传入，最终在该方法内部通过 doInvokeMethod()方法来执行业务方法，如图 9-29 所示。

```
private Object doInvokeMethod(Method method, Object target, Object[] args) throws Exception {
    ReflectionUtils.makeAccessible(method);
    try {
        return method.invoke(target, args);
    }
    catch (InvocationTargetException ex) {
        ReflectionUtils.rethrowException(ex.getTargetException());
    }
    throw new IllegalStateException("Should never get here");
}
```

图 9-29 最终调用的执行方法

在这段代码中使用了 Method 类的 invoke(object, params...)，这种方式就是第 7 章中介绍的反射基本知识，其实 Spring 调用业务代码也是以类似的方式来调用的，那么通过该例子我们可以试想一下，在自己编写框架时什么时候更加适合使用反射。

在这里有些小伙伴可能会比较好奇 Handler 类的实体是从哪里获取的，因为在代码分析中我们仅仅看到使用这个 Handler 的实体。

要了解这个问题，需要回到图 9-26 中，Handler 类的实体对象是通过 `getHandler()` 方法来获取的，在方法体内部循环了一个 `handlerMappings` 列表进行 URL 匹配，这个列表是从哪里来的？Spring 启动时通过扫描器加载的类，自动就可以获取到相应的 `@Controller` 注解，相应的类中的每个 `@RequestMapping` 注解都可以被加载，自然也可以通过反射获取到，根据这些 URL 就可以形成这样的列表。

通过本节介绍应当大致理解了 Spring 在请求过程中 URL 分发的代码逻辑，接下来我们看看 Spring 在事务管理器上的实现逻辑。胖哥之所以选择 Spring 事务管理器来讲源码，是有多种原因的，其一是很多朋友不清楚 Spring 的事务所能支撑的范畴，甚至连事务的基本概念都会搞忘；其二是事务管理器的代码相对较多、复杂，牵涉到基础技术 AOP 的应用，我们将根据“点”来分析“面”，采用逐个突破的方式来完成代码分析。

9.2 Spring 事务管理器

9.2.1 JDBC 事务的基本思想

谈到 JDBC 事务的基本原则，需要回顾一下用纯 JDBC 代码编写事务代码的过程，通常这些代码都有如下一些共同特征：

- ◎ 调用 `Connection` 对象的 `setAutoCommit(false)` 方法，将连接设置为非自动提交模式。
- ◎ 然后执行业务代码。
- ◎ 正常执行完成调用 `commit()` 方法。
- ◎ 出现异常调用 `rollback()` 方法。
- ◎ 结束时会选择是否关闭连接。
- ◎ 代码中将使用 `try`、`catch`、`finally` 来做相应的步骤控制。

疑惑：JDBC 事务必须是同一个 `Connection` 吗？

答曰：胖哥认为这是肯定的，更为确切地说是同一个会话，会话是建立在实际的连接基础上的，在连接基础上包装出来的对象才是 `Connection`。有些 `Connection` 下面不仅仅有一个连接，可能会有多个，例如可以自定义一个类将多个 `Connection` 包装在其中，然后将这个类适配到 `Connection` 的接口。也就是说，拿到的 `Connection` 仅仅是被包装后的对象，它理论上也是支持事务的，但不肯定。

疑惑：分布式事务中存在多个 `Conneciton` 对象，那么它的事务又是如何保证的呢？

答曰：分布式事务是内核对通信的包装，如果要求分布式的绝对一致性，那么就要求状态一致提交，也就是两个数据库服务器要一起 `commit`，但是两个 `commit` 就必然会存在一个成功，一个不成功的情况，虽然这种问题出现的概率极低（`commit` 操作仅仅是向数据库发出相关的指令，类似的 SQL Parser、约束校验、锁处理、容量处理等早已经完成），但也是有可能发生的，例如正好服务器出现宕机、切换、断电等情况。

在分布式事务中，JTA 如同 JDBC 一样是一组接口规范，JTA 希望通过其规约中的 `prepare()` 方法来确定 SQL 语句执行的事务是否成功并准备提交。然后再一起进行多个连接的事务提交，但是在顺序提交的过程中同样可能会存在上文中提到的问题。此时 JTA 希望通过 `recover()` 来解决类似的问题，或者说将已经提交的数据库做一次数据库级别的恢复操作。表面上看似没有任何问题，但是仔细想一想：提交后的内容要恢复其代价要高不少，相当于在数据库端做一些反向 SQL 操作并提交，这个操作也是有可能会失败的，如果是一

些 update 语句，在并发条件下，这很可能会产生其他时间顺序的问题，即使在一切 OK 的情况下，这个开销也是十分巨大的。这种多个库之间相互牵制的做法，只需要一个偶然的网络问题就可能会导致程序变得十分慢。

在分布式层面上，我们更希望通过最终一致性来解决问题，也就是理论上将提交动作分解为多个步骤，一个库的修改或准备修改要记录在一个地方（例如缓存或本地日志），这个动作必须在一个事务内部，以保证“写入即存在”。

在依次向不同的数据库发起 commit 操作时，若某个库提交失败，则可以通过操作日志根据时间点进行回补，回补的过程可以通过重试、自动程序、人工介入等方式来处理，数据之间会存在一定的延迟，但是在理论上可以达到最终一致，或者说在绝大部分情况下可以达到最终一致（在第 6 章所介绍的转账案例中就是采用类似的方式）。在一致性问题，根据不同的处理模式，一致性会分成多种级别，大家可以到互联网上查找相关资料了解细节，通过了解你也会发现，即使是数据库本身也未必完全绝对一致。

在 JDBC 层面上，一个库的事务是在同一个通信连接上的，数据库端也保持了一个对应的连接状态，才会有会话与事务的概念，否则事务的概念无从谈起。

而在众多的 JDBC 操作对象中，例如 ibatis、JdbcTemplate 等，它们在每次发起数据库操作时，都会从 DataSource 中调用 getConnection() 方法获取连接来进行操作，此时获取到的连接如何确保是同一个连接呢？Spring 事务管理器除了切入业务方法还做了什么？程序中用的 Connection，Spring 是在哪里保存和提供的呢？

在接下来的几个小节中胖哥为你一一揭晓答案。

9.2.2 Spring 事务管理器的基本架构

Spring 事务管理器，不用多说，自然是通过 AOP 来实现的，也就是相关运行的程序已经是被 Spring 切入后的代码了（确切地说是被字节码增强后的子类），我们如何跟踪代码呢？

为此，胖哥这里用 Spring 2.x 的事务管理配置文件（因为 Annotation 是相对配置文件换汤不换药的东西）来跟踪类的处理，进而跟踪源码。

同样的，如果要跟踪源码就要从配置入手，一般来说，如果要配置 Spring 事务管理器，就需要配置一个 transactionManager，如下所示：

```
<bean id="transactionManager" class="org.springframework.jdbc.datasource.
DataSourceTransactionManager">
    <property name="dataSource">
        <ref bean="dataSource" />
    </property>
</bean>
```

胖哥给出的是基于 DataSourceTransactionManager 的 transactionManager，这也是用得最多的一种事务管理器。不过事务管理器不仅仅局限于此，例如 HibernateTransactionManager、JpaTransactionManager、JtaTransactionManager、CciLocalTransactionManager 等各种事务管理器的实现类，它们都实现了接口 PlatformTransactionManager，每种事务管理器都是为了对某种特定的三方 API 操作，例如 HibernateTransactionManager 就是间接调用 Hibernate 本身的 SessionFactory 事务管理机制来完成事务管理的，分布式事务也是类似的。总之，Spring 是在设计这种接口后，再编写对应框架的实现类来完成对接工作的。

Spring 为了将它们通用化，需要通过这种基于接口的方式来完成，并且对事务的处理使用的是一个特定的包装对象，而不会在通用代码中出现类似 Connection 这样的关键字，

这一点大家在阅读源码时要知道。

在本节的内容中，我们将使用 `DataSourceTransactionManager` 来进行讲解，也就是通过保存 `Connection` 来达到目的。要阅读这部分源码，我们需要给自己提出一些疑问：Spring 是如何保存 `Connection` 的？Spring 如何让相应代码中获取到的 `Connection` 与它保存的 `Connection` 是同一个？程序中如何证明连接是同一个？其他的事务管理器是否也是类似的呢？带着这些疑问我们阅读后面的内容。

9.2.3 Spring 如何保存 Connection

要说明这个问题，就要回到一个基础的问题上：Spring 事务管理器是通过 AOP 切入的方式来完成的，而最为关键的是，它是怎么切入的？

在 Spring 事务管理器配置好以后，需要将其配置到一个拦截器（Interceptor）上，有多种配置方式，比如在 Spring 2.x 中通常有如下几种方式。

- ◎ 为指定的 DAO 类配置 `TransactionProxyFactoryBean` 代理来达到目的，它的全名叫“`org.springframework.transaction.interceptor.TransactionProxyFactoryBean`”。
- ◎ 单独配置一个 `TransactionProxyFactoryBean` 的实体，在每个 DAO 类的配置中通过 `parent` 属性来引用代理。
- ◎ 配置一个 `TransactionInterceptor`（拦截器），然后使用 `BeanNameAutoProxyCreator` 指定的 DAO 类名称的模式匹配，拦截器便可以动态拦截 DAO 类的方法了。

在现在的 Spring 版本中还增加了使用 `tx:advice` 方法配置，也可以通过注解来达到目的，无论用哪种方式配置，总体思想是不会变化的。

总的来讲，就是将需要拦截的类使用 `TransactionInterceptor` 来做一层代理包装，或者说它是方法的代理访问类，因此这个 `TransactionInterceptor` 是访问处理的核心与关键。该类的全名为“`org.springframework.transaction.interceptor.TransactionInterceptor`”，我们要跟踪这个代理类，应该从哪里下手呢？根据对第 7 章中动态代理的理解，自然是 `invoke()` 方法了（Spring 虽然没有使用 Java 本身提供的 `Interceptor` 来完成操作，但是依然保证了 `invoke()` 方法名称的通用性），其代码如图 9-30 所示。

```
public Object invoke(final MethodInvocation invocation) throws Throwable {
    // Make sure the target class can be made null (void).
    // The transactionAttribute should be passed the target class.
    // as well as the method, which may be taken an interface.
    Class targetClass = (invocation.getThis() != null ? invocation.getThis().getClass() : null);

    // If the transaction attribute is null, the method is non-transactional
    final TransactionAttribute txAttr =
        getTransactionAttributeSource().getTransactionAttribute(invocation.getMethod(), targetClass);
    final PlatformTransactionManager tm = determineTransactionManager(txAttr);
    final String joinpointIdentification = methodIdentification(invocation.getMethod());

    if (txAttr == null || !(tm instanceof CallbackPreferringPlatformTransactionManager)) {
        // Use the standard transaction logic with getTransaction and commit/rollback calls
        TransactionInfo txInfo = createTransactionIfNecessary(tm, txAttr, joinpointIdentification);
        Object retVal = null;
        try {
            // This is an around advice; invoke the next interceptor in the chain.
            // This will normally result in a target object being returned.
            retVal = invocation.proceed();
        }
        catch (Throwable ex) {
            // target object exception
            completeTransactionAfterThrowing(txInfo, ex);
            throw ex;
        }
        finally {
            cleanupTransactionInfo(txInfo);
        }
        commitTransactionAfterReturning(txInfo);
        return retVal;
    }
}
```

图 9-30 `invoke()`方法的关键部分

这部分代码是一个 try catch finally 结构, 在 try 的 body 部分中调用 invocation.proceed() 就是业务代码的调用处理。在 catch、finally 的代码中, 根据其调用方法的名称可以看出是对业务方法处理完成后的清理工作。

在前面的代码中, 有一个关键的点是在 createTranscationIfNecessary()方法调用中, 这个方法内部做了什么呢? 进去看看, 见图 9-31。

```
protected TransactionInfo createTransactionIfNecessary(
    PlatformTransactionManager tm, TransactionAttribute txAttr, final String joinpointIdentification) {

    // If no name specified, apply method identification as transaction name.
    if (txAttr != null && txAttr.getName() == null) {
        txAttr = new DelegatingTransactionAttribute(txAttr) {
            @Override
            public String getName() {
                return joinpointIdentification;
            }
        };
    }

    TransactionStatus status = null;
    if (txAttr != null) {
        if (tm != null) {
            status = tm.getTransaction(txAttr);
        }
        else {
            if (logger.isDebugEnabled()) {
                logger.debug("Skipping transactional joinpoint [" + joinpointIdentification +
                    "] because no transaction manager has been configured");
            }
        }
    }

    return prepareTransactionInfo(tm, txAttr, joinpointIdentification, status);
}
```

图 9-31 获取事务的 Status 对象

这里获取事务对象的内部状态是一个关键部分, 要看清事务存储 Connection 就必须要看这部分代码 (注意: 如果你看的是 Spring 2.x 的代码, 从这里开始, 代码上将会有很大的区别, 总的来讲, Spring 3.x 的这部分代码更加简单、易懂)。

代码中的 tm 变量是 PlatformTransactionManager 类型的对象, 也就是在前文中提到的在配置文件中所配置的不同种类的事务管理器, 以 DataSourceTransactionManager 为例, 调用它的 getTransaction()方法时, 默认会走入这样一个关键逻辑中, 如图 9-32 所示。

```
try {
    boolean newSynchronization = (getTransactionSynchronization() != SYNCHRONIZATION_NEVER);
    DefaultTransactionStatus status = newTransactionStatus(
        definition, transaction, true, newSynchronization, debugEnabled, suspendedResources);
    doBegin(transaction, definition);
    prepareSynchronization(status, definition);
    return status;
}
catch (RuntimeException ex) {
    resume(null, suspendedResources);
    throw ex;
}
catch (Error err) {
    resume(null, suspendedResources);
    throw err;
}
```

图 9-32 这里间接调用了 doBegin()方法

看名称似乎很接近于连接的预处理操作, 跟踪进去没有让我们失望, 这段代码相对较长, 胖哥截取出比较关键的部分展示给大家, 如图 9-33 所示。


```

try {
    if (txObject.getConnectionHolder() == null ||
        txObject.getConnectionHolder().isSynchronizedWithTransaction()) {
        Connection newCon = this.dataSource.getConnection(); //从连接池中获取连接
        if (logger.isDebugEnabled()) {
            logger.debug("Acquired Connection [" + newCon + "] for JDBC transaction");
        }
        txObject.setConnectionHolder(new ConnectionHolder(newCon), true); //创建连接的Holder并放入到txObject当中
    }
    if (con.getAutoCommit()) {
        txObject.setMustRestoreAutoCommit(true); //设置restore参数
        if (logger.isDebugEnabled()) {
            logger.debug("Switching JDBC Connection [" + con + "] to manual commit");
        }
        con.setAutoCommit(false); //给JDBC连接设置为非自动提交模式
    }
    txObject.getConnectionHolder().setTransactionActive(true); //设置状态, 表示事务已经开启
    // Bind the session holder to the thread.
    if (txObject.isNewConnectionHolder()) { //关键部分: 将连接的Holder绑定到线程变量中
        TransactionSynchronizationManager.bindResource(getDataSource(), txObject.getConnectionHolder());
    }
}

```

图 9-33 begin()方法的关键代码

这部分代码中前面的部分大多是获取连接、包装连接设置状态之类的动作，最后有一个 `bindResource(getDataSource(), txObject.getConnectionHolder())`，它到底是做什么的？胖哥已经标注它是关键部分，那么就不得不看看它里面是什么，见图 9-34。

```

public static void bindResource(Object key, Object value) throws IllegalStateException {
    Object actualKey = TransactionSynchronizationUtils.unwrapResourceIfNecessary(key);
    Assert.notNull(value, "Value must not be null");
    Map<Object, Object> map = resources.get();
    // set ThreadLocal Map if none found
    if (map == null) {
        map = new HashMap<Object, Object>();
        resources.set(map);
    }
    if (map.put(actualKey, value) != null) {
        throw new IllegalStateException("Transaction already exists for key");
    }
}

private static final ThreadLocal<Map<Object, Object>> resources =
    new NamedThreadLocal<Map<Object, Object>>("Transactional resources");

```

图 9-34 bindResource()方法的关键代码

通过该代码可以发现，Spring 将 Connection 对象信息放入了一个 ThreadLocal 变量中，而且是以 Map 方式存在的，Map 的 Key 是 DataSource。通过第 5 章中对 ThreadLocal 的详细讲解，我们知道 ThreadLocal 变量的写入本身是线程安全的，但为何要使用 DataSource 作为 Key 放入一个 Map 对象中，然后再将 Map 对象放入 ThreadLocal 中呢？这是因为在一个线程的运行中依然可以访问到多个数据库，也就是使用到多个连接池，Spring 可以通过这一点将不同连接池上所分配的连接区分开。

这样一来，有一个连接就被保存到与线程相关的变量中了。接下来我们来了解第二个问题。

9.2.4 Spring 如何保证程序中多次获取到的连接是同一个

这个问题换一个问法，就是程序中可以使用 `getDataSource().getConnection()`，三方框架 ibatis 在内部也会用类似的方式来获取连接，那么 Spring 又是如何保证从连接池中获取到的

连接就是它保存的连接呢？

同样我们要看一下配置文件，以 ibatis 与 Spring 整合为例，Spring 通常会使用 `org.springframework.orm.ibatis.SqlMapClientFactoryBean` 类来与之整合，如下所示：

```
<bean id="sqlMapClientAone" class="org.springframework.orm.ibatis.
SqlMapClientFactoryBean">
    <property name="configLocation" value="classpath:config-XXX.xml"/>
    <property name="dataSource" ref="dataSourceXXX" />
</bean>
```

这似乎没有什么稀奇的，不过跟踪代码后发现它确实有些稀奇，我们一起来跟踪看看吧。该类实现了接口 `InitializingBean`，因此通过 Spring 加载后会调用 `afterPropertiesSet()` 方法，这个方法中比较关键的代码如图 9-25 所示。

```
this.sqlMapClient = buildSqlMapClient(this.configLocations, this.mappingLocations, this.sqlMapClientProperties);

// Tell the SqlMapClient to use the given DataSource, if any.
if (this.dataSource != null) {
    TransactionConfig transactionConfig = (TransactionConfig) this.transactionConfigClass.newInstance(),
    DataSource dataSourceToUse = this.dataSource;
    if (this.useTransactionAwareDataSource && !(this.dataSource instanceof TransactionAwareDataSourceProxy)) {
        dataSourceToUse = new TransactionAwareDataSourceProxy(this.dataSource);
    }
    transactionConfig.setDataSource(dataSourceToUse);
    transactionConfig.initialize(this.transactionConfigProperties);
    applyTransactionConfig(this.sqlMapClient, transactionConfig);
}
```

图 9-35 afterPropertiesSet()方法的关键部分

该段代码中首先会通过 `buildSqlMapClient()` 方法创建一个 `SqlMapClient` 实例对象，其内在这里不再详述。然后在判定连接池存在的情况下，首先会通过 `transactionConfigClass` 实例化一个 `TransactionConfig` 类型对象，`TransactionConfig` 是一个接口，在 ibatis 中提供的 `ExternalTransactionConfig` 类，用于 Spring 集成 ibatis 中默认选择的 `TransactionConfig` 的实现类，然后判定当前的 `DataSource` 如果需要使用事务，且不是事务包装的连接池，则会使用一个名为“`TransactionAwareDataSourceProxy`”的类对连接池进行包装，我们猜想真正使用的应该是这样的连接池。最后一行代码是调用 `applyTransactionConfig()` 做了一些配置处理，如图 9-36 所示。

```
protected void applyTransactionConfig(SqlMapClient sqlMapClient, TransactionConfig transactionConfig) {
    if (!(sqlMapClient instanceof ExtendedSqlMapClient)) {
        throw new IllegalArgumentException(
            "Cannot set TransactionConfig with DataSource for SqlMapClient if not of type " +
            "ExtendedSqlMapClient: " + sqlMapClient);
    }
    ExtendedSqlMapClient extendedClient = (ExtendedSqlMapClient) sqlMapClient;
    transactionConfig.setMaximumConcurrentTransactions(extendedClient.getDelegate().getMaxTransactions(),
    extendedClient.getDelegate().setTxManager(new TransactionManager(transactionConfig)));
}
```

图 9-36 applyTransactionConfig()方法的源码

从这段代码中可以看出，使用的 `sqlMapClient` 也是经过包装后的 `ExtendedSqlMapClient` 对象，并且会设置一个事务管理器，将 `ExternalTransactionConfig` 设置进去。

现在我们提到的这些还仅仅是配置，要看到怎么使用这些配置，还需要跟踪到代码中，首先跟踪什么代码呢？胖哥认为应该先跟踪 `SqlMapClient` 的 `getDataSource()` 方法代码，因

为它将决定使用什么样的连接，SqlMapClient 的实现类为 SqlMapClientImpl，它内部也会间接调用代理的 `getDataSource()` 方法来获取真正的连接池，如图 9-37 所示。

```
public DataSource getDataSource() {
    DataSource ds = null;
    if (txManager != null) {
        ds = txManager.getConfig().getDataSource();
    }
    return ds;
}
```

图 9-37 获取连接池的方法

聪明的小伙伴们想一想，这里的 `txManager` 是什么？就是图 9-36 中设置的 `TransactionManager`，那么该方法所获取到的 `DataSource` 对象也就是经过包装后的 `TransactionAwareDataSourceProxy` 类型对象（在相应的程序中，可以通过 `getDataSource()` 方法获取连接池对象，并输出其类名即可得到验证），当程序中调用 `getConnection()` 方法时就是 `TransactionAwareDataSourceProxy` 类的 `getConnection()` 了，换句话说，它将决定返回的内容是什么，如图 9-38 所示。

```
public Connection getConnection() throws SQLException {
    DataSource ds = getTargetDataSource();
    Assert.state(ds != null, "targetDataSource' is required");
    return getTransactionAwareConnectionProxy(ds);
}

protected Connection getTransactionAwareConnectionProxy(DataSource targetDataSource) {
    return (Connection) Proxy.newProxyInstance(
        ConnectionProxy.class.getClassLoader(),
        new Class[] {ConnectionProxy.class},
        new TransactionAwareInvocationHandler(targetDataSource));
}
```

图 9-38 这里的连接使用 Java 默认的动态代理机制

这里使用了默认的动态代理对连接 `Connection` 进行二次包装返回，换句话说，在具有事务的方法中我们所拿到的连接也不是 `Connection` 本身，而是被代理出来的连接类，所以两次获取 `Connection`，得到的 `HashCode` 肯定是不同的。但是我们知道代理类对象中必然会存放原始对象，这样才能真正起到代理的作用，因此可以看看它的属性结构，如图 9-39 所示。

```
private class TransactionAwareInvocationHandler implements InvocationHandler {

    private final DataSource targetDataSource;

    private Connection target;

    private boolean closed = false;
```

图 9-39 TransactionAwareInvocationHandler 代理的属性描述

这里的 `target` 属性就是原始的 `Connection` 连接，如果在同一个事务中，那么无论多少次获取出来的连接，其内部的 `target` 属性肯定是同一个。

回到前面的话题，知道了这是一个代理类，还要看看它是如何从连接池中获取连接的，采用同样的方法：动态代理类从 `invoke()` 方法开始着手。大家在跟踪这个 `invoke()` 方法的时候会发现其内部首先会判定调用的方法名称，并根据该方法名称做不同的操作。细节上大家可以自行去细看，在这里仅仅列出其中与连接获取相关的代码，如图 9-40 所示。

```

    Connection actualTarget = this.target;
    if (actualTarget == null) {
        actualTarget = DataSourceUtils.doGetConnection(this.targetDataSource);
    }

```

图 9-40 与获取连接相关的代码

至此，就到了真正获取连接的代码中，这里还真传入了 DataSource 对象。什么？真的没看错，跟踪进去看看，相信各位小伙伴就能获取到真相了，代码如图 9-41 所示。

```

public Connection getConnection() throws SQLException {
    DataSource ds = getTargetDataSource();
    Assert.state(ds != null, "'targetDataSource' is required");
    return getTransactionAwareConnectionProxy(ds);
}

protected Connection getTransactionAwareConnectionProxy(DataSource targetDataSource) {
    return (Connection) Proxy.newProxyInstance(
        ConnectionProxy.class.getClassLoader(),
        new Class[] {ConnectionProxy.class},
        new TransactionAwareInvocationHandler(targetDataSource));
}

public static Object getResource(Object key) {
    Object actualKey = TransactionSynchronizationUtils.unwrapResourceIfNecessary(key);
    Object value = doGetResource(actualKey);

    private static Object doGetResource(Object actualKey) {
        Map<Object, Object> map = resources.get();
        if (map == null) {
            return null;
        }
        Object value = map.get(actualKey);
        // Transparently remove ResourceHolder that was marked as void...
        if (value instanceof ResourceHolder && ((ResourceHolder) value).isVoid()) {
            map.remove(actualKey);
            value = null;
        }
        return value;
    }
}

```

图 9-41 这样保证了获取的连接对象始终是同一个

代码读到这里，大家是否有点清楚 Spring 的事务是怎么回事了？如果还不清楚的话，就多读取几次代码。下面胖哥给出几个题目请大家自己去细化。

- ◎ 图 9-30 中有关于事务的结束清理工作，看懂这部分代码才是一个闭环。
- ◎ 在具有事务控制的方法中，使用 ibatis 自己的 commitTransaction() 会不会自动提交事务呢？答案是：不会，但是为何？大家可以自己去跟踪源码，这个小节中多个地方有提示，胖哥给大家的小提示是充分利用前文中提到的 ExternalTransactionConfig 类的具体实现。
- ◎ 如果是自己通过 getDataSource().getConnection() 来做事务提交，Spring 管得着吗？如果管得着或者管不着，你应该如何写代码呢？

9.3 思考：自己做框架有眉目了吗

前面所提到的框架其实都包含了一些优秀的设计思想，如果你自己来设计一种框架，

首先应该思考什么？是毫无头绪还是随便找个点入手？是先由业务模块来驱动还是先考虑全局？是否会考虑先分析业务逻辑？考虑如何选型框架？考虑需要多少人来完成？考虑需要多长的时间？考虑有哪些技术难题？……

胖哥认为这些也许都是在项目实施过程中需要去摸索的问题，没有对与错的区别，只需要角色的清晰定位，以及项目大小的定位。关于这类问题，胖哥将在本书下册的“实现篇”中与大家详细交流，本节胖哥将就框架本身与大家谈谈一些简单的问题。

如果考虑这个项目在未来需要有长远的发展，可能支撑许多年，也希望这个项目在未来几年内不会发生整体架构上的重构，那么需要你对业务背景十分了解，并具有较好的抽象能力。具体来讲，前者是指非常了解业务背景下的“可变性”，将可变因素进行抽象后就是剥离“可变条件”，当这个部分真的发生改变时就很容易修改了，而不是“到处修改”。

例如“工作流引擎”是大多数企业级应用中都会用到的，大家发现工作流中有太多共同的特征，于是开始抽象它，这样业务代码开始变得简单，而无须关心流程的跳转，而只关心每个环节应该做什么，进而工作流开始分离设计，可以在业务上继续深入扩展，也可以在工作流的使用上继续升级到自定义工作流，这些就是模块化带来的价值。

疑惑：模块化什么粒度比较好？

答曰：能问出这个问题的人，通常是做出思考的人，而并非一味追求细粒度或粗粒度模块化的人，追求细粒度的人在技术上和设计上精益求精，不论是否会用，甚至未来N年都不会用，先抽象了再说；粗粒度的抽象随心所欲，觉得可以抽象就抽象，在写代码时才开始抽象。

前者的细粒度设计，其实也不可能做到任何场景下完全通用，但是越细的设计，将会使得框架本身变得越“重”，如果再考虑灵活性，那么框架的设计将会投入大量的人力，而使用框架的人也将会付出许多的学习成本，甚至是使用成本；而后者过于粗粒度的抽象未来是否能被复用是一个未知数，只是知道目前这段代码不抽象，可读性真的就太低了。

为了说明这个问题，胖哥再来谈谈 SSH。其实在应用开发的过程中没有人会强制约束应用程序就是这样的三层结构，但很多程序员张口闭口就会以 SSH 来定位思路。这在众多程序员心目中形成了一个抹不去、擦不掉的印象，认为不这样写就不好，这样写才算好！那好在哪里呢？

分层的目的在于可扩展、可复用，所谓可扩展一个很重要的因素就是可变性，也就是这一层的可变性是相对独立的，每个层次有自己的职责所在，当职责发生变化以及职责变得多样性，这一层就需要修改或扩展。层次之间的调用是模块化的、可复用的，什么样粒度的模块可以被复用？并不是一个方法就写一条代码，而是职责的基本粒度，例如业务层就是操作一个业务的基本粒度，而数据层就是操作数据库的基本粒度，但这也并非绝对，一个基本的业务逻辑可能还会包含更细的业务模块被共享，到底多大粒度，还是懂业务的人最清楚。

在实际的应用中，框架可能会被设计成2层、3层、4层甚至更多的层次来满足需求，SSH 只是一个基本模板，它没有什么大不了的，如果非要把它想得比天还高，那么可以看看微软体系的平台，其实在可用性上它似乎更好，这也是很多人“吐槽 SSH”的一个重要原因之一。换句话说，其重要的卖点不是 SSH 本身，而是一种分层剥离可变因素的思想，

使得框架更加松散。这样剥离后不仅仅让代码扩展性更好，在必要的情况下可以实现分布式部署以达到更多的目的。

懂业务的人，更加懂得业务发展中的一种可变性，也更加懂得应用规模和用户群体，在选择技术层面知道哪一种更适合自己的，在开发速度和维护代价上有一个权衡，他考虑层次后，便会开始考虑 API 接口和测试驱动，那么自然模块化就完成了。

胖哥这里说的懂业务的人并不是只懂业务的人，如果对 Java 语言本身的思想没有许多理解，那么再怎么抽象也抽象不出什么东西来。

反过来讲，如果真的要使框架做得很好、很灵活，你首先要明白它用来干什么，懂得业务背景是什么样的，考虑到在功能强大的基础上是否还具备灵活性，许多公司级别做得不错的框架都是在多年的吐槽声中成长起来的，它们并不追求完美，而只追求改进，因为并没有完美，追求完美只会让自己的框架走入死路：“无法扩展的死路”。

类似三方开源框架，读了它的代码不仅仅要学会它的逻辑（因为它的逻辑每天都可能会修改），也要学会它的许多设计思想，以便于你有一天成为项目中的核心成员时，不仅可以从根本上解决框架的关键问题，还可以自定义扩展框架，甚至根据个性化业务编写自己的框架。

第 10 章

看源码的一些总结

本章主要内容

- 高手看 API 的能力
- 通过源码能否量化性能与稳定性
- 思考相似方案和技术优缺点
- 明确场景和业务，不做技术控
- 胖哥对框架的浅析
- 学海无涯，心境无限

本章和大家一起再回过头来做一些“论道”，论源码之道，所以大家可以当作“闲谈”，小伙伴们可千万不要带着过于紧张的心情来阅读本章。

10.1 高手看 API 的能力

学习 Java 的同学们可能会经常接触新的框架、新的 API，那么当你接触这些 API 时，是否联想过它内在的基本原理呢？

很多时候正是这些 API 让人感到迷惑，让人搞不清楚某些功能到底是平台提供的还是框架提供的，加上“以讹传讹”，假的变成真的了，许多小伙伴对 API 的能力理解为“万能的”，性能也是“无限的”。胖哥经常遇到这些小伙伴问问题挺可爱的，在回答者告知如何去配置、修改来解决问题时，他们可能认为这些东西是万能的，并且很自信地回答：“我用的是什么框架或工具，我为什么还要配置成这样”。胖哥想对这些小伙伴说的是：如果这个世界上有万能的东西，那么就不应该遇到任何问题，也不需要我们程序员了。

任何东西有利必有弊，任何事只要你去想都有好的一面和坏的一面。在技术层面也是这样的，仔细阅读前文中的例子，小伙伴应该能体会技术很多时候是把“双刃剑”。

然而你我都需要成长，我们不会立即从今天的初级程序员明天就成长为高级程序员，始终需要过程，但绝对不是“混”时间的过程，而是自我摸索的过程，那么从你决定开始走“技术之路”的那一天起，就要开始学会摸索一些内在，开始学会对技术客观看待，学会做出选择。这个过程是痛苦的，但熬过痛苦后的成长是可见的，你的功底会越来越深，看源码也会越来越快，定位问题越来越清晰，选择技术越来越有道理，在这个领域的理解能力也会越来越好，因为你知道内在，而不是“拍脑袋”。

为了更加形象地说明这些问题，我们来举几个简单例子，这些例子会不断反问，这些反问正是多数程序员不愿意去思考的。

- ◎ 当学会了使用 IoC、AOP 这些技术时，是否考虑过与反射、字节码增强的关系？当遇到其他框架中有类似例子时，是否考虑过它们的内在区别与联系？当你需要用到注入等类似手段，而此时第三方框架无法满足需求时，是否考虑到借鉴某些代码和思想来做扩展或都在其基础上做个性化改造？

在某些时候我们还得进一步来思考这些问题：这些技术中利用到了哪些底层机制？这些底层机制是否会导致系统不稳定？它的开销怎么评估？它能给系统带来什么好处、坏处？这种解决问题的方法是什么？它的原理和计算机的基本思想有何共同点？……

- ◎ 当你用程序实例化对象时，是否考虑过它需要多大的内存空间？为什么这么大？配置多大的内存可以让程序承载多大的 QPS？如何看待程序有没有内存使用上的问题？

大家不需要去背诵上面的那些为什么，只需要在做完事情后多给自己提出几个为什么。当你发现自己有许多不知道的时候，就会去想办法知道更多，这个过程是痛苦的，但学会的是方法、坚持，学会在逆境中成长自我。

当你真正知道了许多未曾涉足的领域后，也会发现不知道的东西会更多。为何？因为在这个过程中，技术视野面变得更大，自然看得更远、更高，而并非“坐井观天”。换句话说，只有爬出井外，才能够知道天高地厚。

现在来说说一些有功底的人看 API。

如果有功底的人看一些自己未曾见过的 API，但是通常看了功能就基本能猜出来内部许多技术实现的细节，也许他们都还不知道太多的细节，但是基本已经知道了大概，如果要定位问题，去看源码也会非常快，只需抓关键点。另外，如果要评估性能也基本心里有数，因为他们很清楚要实现这样的功能无非几种方式，可能花几分钟时间看看源码关键部分就能印证结论，再通过一定的关键路径测试便可以得到更切合场景的结论。

再说一个反面例子。

有一天，一位参与过一段时间 Struts 2 开发的同学前来问一个关于请求跳转的问题。这位同学描述到：自己知道一个页面请求后如何渲染的开发过程，但不知道 jQuery 的 Ajax 与 Struts 2 怎么整合。胖哥看到觉得挺好玩，因为胖哥认为这几者之间从某种角度来讲就没有什么关系，但总有这样的同学会将几个不相关的东西放在一起问问题，让回答者不知所措，但有些时候某些回答者还没搞清楚就开始回答得津津有味。

胖哥不想直接告诉他如何去操作，因为这些问题的答案是 HTTP 服务中最基本的输入/输出的道理，这些该死的框架将其包装后让程序员连这些最基本的道理都搞不清楚了（再加上某些教学方式的诱导）。在这样的情况下，即使告诉这位同学应该如何操作，帮他搞定了，他自己也不知道怎么回事，下次遇到同等级的问题甚至类似的问题时，还会继续来问。于是，胖哥开始给那位同学讲解 Response 的一些基本知识，但是那位同学的思维一直集中在 Struts2 与 jQuery 有何关系上。从胖哥的角度来讲，已经无能为力了！胖哥唯一可做的是，如果相信胖哥的话就去看看 Request、Response 的基本原则、网络 IO 的基本道理，搞清楚什么是服务器端、什么是客户端的代码，再回头来看这些内容就会清楚很多了。

在胖哥眼里，框架是包着这些东西的美丽外衣，它最终操作的是这些基础元素，它基于这些基础元素可以展现出各种各样的“美”，很多时候，如果别人写得不错，没有必要自

己花费精力去写。在知道了内在的基础上，要定位问题和查阅资料，就是非常容易的事情，不说一通百通，至少你会逐步感觉游刃有余。

上面的问题是不是觉得挺“土”的？胖哥还遇到过更土的呢。

“问问题的人，总想将自己的思路带着回答问题的人走，自己未曾搞清楚状况，就开始大量谈论自己的看法，也不听回答者的沟通与建议，总想一步到位，总希望将回答者当成答题器，把自己当作演讲者。”其实这样问问题通常要么把答题者带入死坑（可能问题根本不在那里），要么答题者没有心情来了解问题，所以提问也是一门艺术。

再来说一个看功能猜内在的简单例子。

有一天胖哥被人问起 Sitemesh 框架的使用问题，胖哥那个时候因为工作范畴所限，还没有用过这个框架，也没有把握能否搞定这个问题，由于对方提到已经在胡同里面转了几个礼拜没有转出来，胖哥就答应了帮助解决。

开始时这位同学也是一顿现象描述，发表自己的看法，且带着激动的情绪，然后开始发问：“这个该怎么解决啊？快点啊，你到底知道不知道啊？你玩过没有啊？好复杂啊，这个问题怎么解啊？”胖哥不知道怎么解决，因为胖哥不知道具体细节，即使有经验，胖哥没看到现场也不会做判断，最多做可能性猜测，况且这种框架胖哥根本未曾接触过。胖哥需要这位同学将代码以及配置文件发送过来，在发送过程中，胖哥到网上查阅了一下这个框架的基本功能，发现是渲染框架，可用于页面的公共部分渲染（例如一些样式），使用业务代码所编写的页面就无须再关注编写这些信息，从而更为专注业务开发，而样式的修订也可以与业务编写分离。

搞清楚这一点，胖哥猜想这个框架集成在 Web 应用中，需要对输出数据进行修改，需要有一个 Filter 才能做到，这个 Filter 应该会代理 Response 流的输出数据，那么在业务中输出数据时就不会逃过这个 Filter 的“法眼”了。按照这种猜测对这个框架进行理解后，便开始打开相应的代码，并让其在 Web 应用中跑起来，然后与提问者沟通，经过简单沟通后，发现问题竟然是如此简单：有些页面的样式没出来，经过确认发现确实是这样，胖哥根据对过滤器的理解，应当有配置可以指定哪些 URL 需要过滤，找到配置文件后，一眼便定位到有 URL 的设置，根据名称基本猜测出这就是拦截请求 URL 的模式，此时胖哥故意加了一个没有显示样式的页面 URL，效果立即就出来了，渲染框架生效了。

胖哥认为此事已经解决，万事大吉！但当胖哥将答案告诉对方时，这位同学说：“我的 JSP 页面确实是这个路径呀？”胖哥恍然大悟：“原来这位仁兄没有搞清楚系统的渲染过程，以及 JSP 页面是属于什么程序，因为他认为这个 URL 就是 JSP 的路径，但是经过这些该死的框架包装后，让这些程序员们找不到门路了（尤其是直接上手用框架的程序员）”。

简单说来，Struts 2 的跳转在默认情况下都是 forward 的（特殊的配置 redirect），其实更确切地说这不算是跳转，只是一种服务器端的“资源定位”，它定位服务器的相对文件资源在哪里，它始终在服务器端运行，而不会通知浏览器（所以在这种情况下 request 和 response 始终是同一个，中间存储的值不会丢失），或者说浏览器根本不知道服务器端做了什么跳转，最终经过服务器端一系列处理后返回给浏览器的始终是静态资源信息。那么此时回到问题的本质上，当默认使用 forward 时根本不会再一次从浏览器上发起请求，从而经过过滤器（Filter），自然它不会被 Sitemesh 框架捕获到。

那么要捕获到，怎么改？当然不是改为 redirect 到一个 JSP，redirect 是告诉浏览器需要访问另一个地址，浏览器的 URL 会发生改变。因此，如果发生 redirect 跳转，request 携带

的值就会丢失，必须通过其他方式来传递参数。如果这样做的话，则意味着 JSP 的路径是暴露给外部的，外部可以不通过业务框架来访问 JSP，这表面上看没有问题的 URL 如果用户收藏了，下一次访问直接进入时可能在框架中处理的数据就没有办法传递进来。

最终修改的办法自然是要调整 Sitemesh 框架的拦截 URL 配置，并对框架访问的 URL 进行规划，规划在不同的 namespace 下采用不同的样式渲染或者不用样式渲染，这些都是提前需要梳理好的。

胖哥最后来简单谈谈这个 JSP 与这些同学的不解。

胖哥曾遇到许多写 Web 的程序员不知道 JSP 到底是什么程序，因为 JSP 是页面，很多人会认为这是客户端程序，而大家忽略了它是动态的，其实动态页面都是服务器端来处理的，浏览器无法处理，而浏览器得到的是服务器端的渲染结果，或者说浏览器拿到的都是静态的内容。

要模拟这个问题很简单，即使你不懂也可以用浏览器查看源码的方式试出来。胖哥不清楚这位同学为什么在这个问题上纠结了几个礼拜，它只是一些小的问题技术点，而且是非常基础的技术点，这些基本可以认为不是技术问题的问题。但很多时候提问者通常会把自己认为可能出问题的地方告诉答题者，或者是故意把问题中最诡异的地方告诉答题者，而忽略了许多细节，让答题者陷入迷惑，似乎期望于让问题变成一宗“冥案”（变得无解），变成一个“RP”问题。

其实很多时候是细节问题，甚至是很低级的错误，场景中的许多细节可能会导致各种各样的情况发生，就算是高手也未必马上能说出问题所在。例如，胖哥曾经遇到一位同学在一个群中问“读取网络文件用 `http:\\xxx.xxx.xx\\xxx\\xxx.txt`”为什么读取不到啊？”回答者成群结队，大家都发现路径中“\”是转义字符，让他将一个“\”改为“\\”，并编写代码来证明应该这样写，不过他的反馈都是不行。胖哥一直在旁边看，实在看不下去了，就插了一句“网络中的相关 URL 都是以“/”作为分隔符的，没有“\”反斜杠，确切地说，就在 Windows 上见过。”但是他还是说不行，为何？神奇了吧。

其实，他不止在 URL 上有问题，他并没有告知大家他阅读文件的代码是什么，胖哥要求他贴出代码，最后发现他竟然是用 `java.io.File` 来读取的，又是小问题揪住了大帮人（但贴出代码后大部分人的焦点依然还在 URL 的斜杠问题上），这很明显是读取文件系统的类，并且不同的 OS 上都会有不同的实现类来完成，在读取网络资源时，这个类根本就不清楚哪是文件系统的内容，而只是一个可以有输出流的资源，自然是用 `java.net.URL` 方式来读取了。

问题解决了，为何胖哥发现了问题，而别人慢半拍或没发现呢？胖哥能在看代码时一眼就认出来的原因是，至少胖哥知道 `File` 是基于文件系统来实现的，因为胖哥阅读过相关源码，马上联想到用 `java.net.URL` 方式，也使用过 URL 方式来读取过网络资源，这就是知识面与知识点在解决问题过程中给你带来的灵感。

胖哥给提问的同学几句小话：其实有些时候某些高手不回答问题，是因为问题没描述清楚，他不想或根本没空来与你手把手问每一个细节，因为他没有那个义务。话虽然有些狠，但是确实是实话。答题者需要的是场景的过程、变化的过程，然后才是现象，可能还需要一些业务背景。有的同学经常会问这个技术和那个技术怎么结合，其实有些时候它们根本是不搭嘎的东西，但这些问题往往很喜欢被提出来，让回答者都感觉不知所措，或者不知道怎么回答。

本节的故事我们就讲到这里，这几个故事给你带来了什么？你有所体会吗？

10.2 通过源码能否量化性能与稳定性

通过源码我们能否知道性能与稳定性呢？

胖哥的答案是 NO！但是胖哥认为可以从源码的角度理解性能与稳定性，只是它无法完全量化，为何？要回答这个问题，我们不如提出一些疑问来看看自己是如何思考的，胖哥的看法又是什么样的。

- ◎ 当通过 `new` 创建一个对象时，看看属性能否知道需要多少内存开销？一次请求访问会创建多少个这样的对象？这个对象的存活期大概有多久？如果生命周期太久的大对象是否有必要手工释放？

答曰：如果你明白 GC 的基本原理和引用关系，存活期对于 GC 的影响，对象空间的浪费，一个对象的属性、头部、padding 有多大，阅读过相应类的代码，知道它的内部结构，便会清楚它的空间占用，这些自然在你心目中一目了然。

- ◎ 如果遇到方法使用上的困惑，例如在 `String.substring()` 时（JDK 1.6 及以下），你是否能理解该方法运行过程中在空间上的使用情况？

答曰：当你阅读源码后，会发现该方法所返回的字符串使用的是原字符串中的 `char[]` 数组，然后通过 `offset` 和 `count` 来定位数据，此时再仔细想想具体场景便知道它有什么好处或坏处了。如果你又看过 JDK 1.7 中的相应代码，则会发现处理方式发生了改变，那么它在实际运用过程中的好处与坏处也会随之改变。

- ◎ 当使用字节码增强相关技术并用得很“爽”时，是否考虑它可能会导致 FULL GC 甚至 OOM？这些问题又将如何解决？

答曰：当明白 Class 的加载区域、与 `ClassLoader` 的关系、Class 的卸载条件、字节码增强几种原理后，对这些问题就会比较清楚了。

- ◎ 在 Java Web 应用中，对于一个 HTTP 请求，需要经过容器和框架多层跳转后才会分发到业务代码中运行，容器和框架本身也会申请大量的对象，因此我们会感觉到一个简单的请求会有不小的内存开销，但是反过来想一想：运行的速度并没有想象当中那么慢，计算机也没有我们想的那么弱。

从程序设计角度理解，很多程序设计者可能为了节约一小点的空间，将代码的设计搞得乱七八糟很多时候是不值得的。但这一点并不代表写 Java 代码时不用考虑性能，只是我们关注应当在一些十分明显的性能开销问题上做特殊处理（在第 3 章中有详细说明）。

- ◎ 当使用 `BufferedXXXStream` 对 I/O 进行读写时，会不会在适当的时候设置 Buffer 的大小？

答曰：当了解了磁盘 IOPS、I/O 的大致过程、`Buffered` 是 JVM 中的一个数组时，也许这些问题开始变得清晰了。

- ◎ 当使用 MySQL JDBC 默认的预编译和批处理时，发现它的效果并不是那么明显，甚至没有什么效果，那么如何才能有作用呢？为什么没有作用呢？

答曰：在别人提供方法的基础上测试后，需要在源码上得到印证，这才是可靠的。

◎ 当使用某些数据操作的 API 时,例如数据库的 JDBC 驱动,在面对大数据提取和小数据提取时,是否会考虑设置不同的参数?在某种情况下是否会导致内存溢出?

◎ 当你使用 ThreadLocal 很“爽”的时候,是否考虑过相关数据的生命周期?换句话说, set 到 ThreadLocal 的数据何时注销?

答曰:阅读了 ThreadLocal 的源码,理解了 ThreadLocal 如何与 Thread 绑定后,再与实际场景中的线程池结合起来,问题便十分容易解决了。

◎ 当直接或间接使用“锁”的时候(例如 System.out.print 就间接使用了锁),是否会考虑锁的粒度?

答曰:锁的粒度是由人来决定的,不是由技术决定的,我们只能参考技术中某些思想来优化,而不是依赖技术,细节上请大家参看第 5 章的一些知识。

◎ 当使用 CAS 相关的组件时,肯定发现许多 API 内在是一个死循环,只是它并非阻塞,这样做的好处与坏处我们想过吗?

答曰:理解何为乐观、何为可控,便知道答案真相,在第 5 章也有同样的解释。

◎ 通过大批量的循环来测试一些简单代码的性能并得出性能结论,可否考虑这些结果中有许多是 JIT 优化、CPU 优化带来的结果?

答曰:很多深入的问题需要深入了解一些基础技术的原理,加上对许多源码的认知,才能用理论来指导实践的思路,来指导可靠性的一些评估。当然可能在评估过程中,我们会忽略一些未曾深入的领域,导致评估的结论数据是错误的,但是这也可以让我们不断去扩大知识面去了解更多的细节。

例子还有许多,胖哥就不再一一列举了。你可以列举自己所用的框架,只要你愿意去思考,多少可以和性能沾上一点边,胖哥在这里列举了一些简单的基础框架的例子,可能你还需要面对更为复杂、高端的问题场景。不过从某些角度来讲,它们和基础技术是息息相关的。比如 HBase 可能并不是实时写磁盘,会有一个缓冲区,这个缓冲区通常较大,这样就需要考虑 JVM 的设置,也需要考虑 I/O 磁盘的 IOPS 进而选型磁盘,还可能根据日志写与数据库写不同而做不同的选择;再比如 MySQL 的 Buffer 设置太大,突然有一天导致 DDL 很慢,即使删除空表也会导致系统延迟变得很大,这是不是和 MySQL 本身的架构及运算方式有关系呢?

例子我们就说到这里,看了这些例子后你应当开始认识到看源码、原理对理解性能和稳定性的帮助。不过无论如何,胖哥始终认为性能和稳定性“无法完全量化”,为何?请看下面胖哥的理解。

通常阅读源码能够读到自己所接触的下一层,很少有人会阅读到真正的底层,即使阅读到底层,也无法巨细无遗地判断出实际场景的变化及实际系统的具体演进。也就是说,即使坐在家里研究这些,也需要知道实际场景是什么才能开始做判定。

例如研究异步 I/O 时,知道理论上它能在有限的资源内服务更多的请求,也需要知道它更加适合于什么样的 I/O 场景,并非说异步就是绝对的好,阻塞 IO 就是绝对的不好。很多时候可以根据相关的资料查阅,加上对计算机基础的理解和源码的阅读,同时对项目中的代码路径要有总体上的认识,才有可能预估出大致的上下范围。或许这样还不够,我们还得知道许多热点路径的代码是很关键的,进而再做评估。接下来的工作是测试和验证来做进一步的推算,测试那些热点路径,各种方式的请求,不同大小的数据输入和输出,单台机器能抗得住多大的并发(这里的并发测试,绝不仅仅是测试一些简单的小数据请求,

不仅仅是测试已经被 Server 端 Cache 的数据，也不仅仅是单纯使用短连接或长连接进行并发，更不能脱离程序中的关键路径去做测试）。

此时测试的结论也并非完全能说明问题（因为内在的许多因素我们无法一一考证和综合），但是这些方式可以帮助我们缩小范围，可能会测试出在不同机型、JVM 参数、线程池、连接池等情况下承载请求的范围。实际值与理论值对比，如果偏差过大通常是遗漏了一些关键的细节，需要通过监控和更细的测试等来找到原因，直到和预期差别不大。很多时候通过坚持和努力确实可以做到这一点，这个时候从某种角度说理论开始指导实践。

这些就是胖哥提到的量化，它并不是要精确到多少，而是理论指导实践的过程，它的基础就是功底、源码、场景的结合，这就是为什么胖哥说源码并非完全可以量化事务，而是要有实际项目的场景、要有测试、要有微调，在不断磨合的过程中，量化的概念会逐渐加深和形成概念。

阅读到这里，你是否觉得功底、源码对需要成长的你我来讲十分重要，如果你有一天做了资深工程师甚至架构师，这些或许是必备的基础知识，并且伴随你的长久发展。

源码的好处说了很多，也说得很杂，就像一个大杂烩。而接下来谈的是另一个大杂烩，另一个维度的大杂烩，那就是技术的横向扩展：类似方案，胖哥会尽量让大家理解为何要知道类似技术，以及如何知道类似技术。

10.3 思考相似方案和技术优缺点

胖哥认为，如果要成为一个老 A，应通百家所长，哲学上一句名言：“任何事物都有两面性”就是指有好处必有坏处，因此我们要学会深入理解并取长补短。换句话说，需要知道相关领域的技术。

知道了相关的技术，我们就会有意识地去研究它们之间的联系和区别，它们各自的优势与缺陷，以对应到它们最佳的应用场景上来，这就是前面提到的“选择”。研究它们并不是浪费时间，而是让我们对专业领域的技术认识更加客观、更加深刻。同样的，我们来看看一些例子。

面对存储数据时可以选择 RDBMS（如：Oracle、MySQL），也可以选择 NoSQL（如：HBase、MongoDB），还可以选择分布式缓存（如：Memcached、Redis），甚至可以选择分布式文件系统等，面对如此多的条件该如何选择呢？此时我们就需要了解它们之间的区分方式。这些区别细节会非常多，根据方便调用、存储大小、一致性要求、读写比重、灵活性、成本等都有不同的考虑，胖哥在这里就不再多提了。

对于存储技术，某些小伙伴可能会有点晕，那么我们就先抛开这些存储技术，先来谈一谈常见领域的技术对比，希望可以给大家一点点启发。

（1）使用 Web 的前端框架时，你会选择 ExtJS、jQuery、Flex 等吗？如果你选择了，那么选择的依据是什么？——前段框架有胖瘦之分，主要体现在封装程度，但同时扩展性和学习成本会随着封装的度而变大。

（2）选择 MVC 组件时，你会选择 Struts、Spring MVC 还是别的？同样需要讲讲原因，有些时候我们不能仅仅因为使用很熟练来选择。——MVC 框架在乎是否简单、方便、可控性强、是否安全。

（3）使用 DAO 框架时，你会选择 ibatis、jdbcTemplate、Hibernate 等吗？我们选择时也不能仅仅听别人说什么好，自己要有所体会。——从统一管理、扩展性、操作方便性、

代码简易性、SQL 灵活优化等角度考虑，自然知道该如何选择。

(4) 选择磁盘存储时，你会选择 SATA 盘、SAS 盘、SSD 盘等吗？不要因为 SSD 硬盘快就一定要选择它。——在效率提升的同时价格也在提升，有大量顺序读写或对磁盘没有太多要求的地方用 SSD 就是浪费钱财。

(5) 系统交互时，RMI、WebService、HttpClient、Socket、NIO、AIO、JMS 等你选择什么？——通信技术很多，它们的外壳都是在通信上进行协议包装或封装通信的方式，对我们来讲首先是要看这种通信方式是否符合场景；其次是从性能、可用性、稳定性、使用方便、依赖性小、扩展性良好、限制性少等角度去考虑。

(6) 在众多语言之中选择一种语言来做脚本程序、某底层程序、某种高性能的计算程序、某种复杂业务逻辑程序、某种分布式程序，你会选择什么语言（如：C、C++、Java、PHP、Perl、Python、Groovy、Scala 等）？——语言有血脉和基因，将决定传承的性格，适用于某种场景就自有它的用途所在，需要知道的是在什么情况下使用什么语言更加合适。

(7) Web 渲染，选择 JSP、JSF、Velocity，甚至自己写代码来渲染。——或许你会考虑性能、代码可读性、操作便捷性、组件扩展性、内容可控性等因素。

(8) 加密技术选择 Base64、AES、MD5、Blowfish 等，或者使用这些加密算法的组合物，再带上自己的算法。——要想清楚加密主要用来做什么，它的场景是什么，是否有必要搞得很复杂，加密后被破解的难度有多大。

(9) 选择 Web 容器，有 Jetty、Tomcat、JBoss、WebLogic、WebSphere 等，仅仅是根据功能强大来选择吗？——也许功能不是那么强大，但扩展性会更好，更容易懂得它的内在处理模式，可控性也会更强。

(10) 对数据逻辑删除，选择加一个逻辑标志字段，还是做一个历史表？——前者需要在几乎所有与该表相关的 SQL 中加上这个字段的判定，被漏掉将是一个 Bug。另外，若这个表中存在 UK，逻辑删除后，增加同样的值就可能出错，要解决此问题又要想其他方式。后者将面临的是物理删除原表数据时，必须在一个事务内写入历史表，恢复时再拷贝回来。另外，若原表的表结构发生改变，历史表也必须做相应的改变。

无限多的技术点，会产生无限多的选择，胖哥没听过、没见过的有很多，不要局限于上面所提到的对比，学会自己去找身边的技术做对比，学会“快”、“准”、“狠”的学习能力。

做程序员“屌丝”是痛苦的，如果在这个行业就必须面对这些选择与对比。也许你会说自己现在还年轻可以慢慢来，那么胖哥要说的是，技术的积累正是年轻人应该做的，许多 IT 工程师通常在工作几年后便会发现思维的灵活度开始比不过后起之秀，在接受新事物上也会变慢。几年时间一晃而过，我们为自己所留下的应当是一些长足发展的财富，同时也是我们职业技术“屌丝”的一种追寻未知的乐趣。

接下来重点从另一个角度来探讨技术，探讨技术的场景和业务，因为有太多的同学因为技术而技术，这就是我们提到的“技术控”，最终难以自拔。

10.4 明确场景和业务，不做技术控

10.4.1 谈谈技术控的那些事

相信“技术控”是几乎所有技术人员都必然经历的阶段和经历，为什么呢？因为能坚

持做技术的人通常是在某个阶段寻找到了自己做技术的“乐趣”，这种乐趣通常来自于自己对技术的驾驭，对自己能认识到“神器”的一种“显摆”，一种陶冶自我职业情操的乐趣。这的确是我们的乐趣，我们失去这些，就失去了应当拥有的追求。

从零开始学习时并不知道需要去学习什么，因此可能会尝试性学习许多知识，在浩瀚的知识海洋中，痴迷于技术本身的同学，可能经常会因为享受技术带来的乐趣或是理解疑难技术的细节而让自己难以自拔。

就算是世界级的一流程序员所编写的程序，可能都会考虑技术可能会用在什么地方，也许是一种市场驱动的因素，也许是技术人员结合自己的兴趣爱好或主动发现某些问题，我们说他们在做事情前都动了脑筋。某些程序员更多的是为了生活，如果做了多年程序依然这样，那么这个行业并不太适合自己。到达一定程度后，需要认识到：技术如果不能发挥实际场景的价值，那么它就一无是处。

不过，对于这个问题也有许多反面看法，换句话说，有人会利用这一点来“打败你”！

例如，有人可能会这样来问你：你学来的技术在工作中用来做什么？这似乎与前文中所提到的“应用场景”相辅相成，但是胖哥认为就问题本身而言是一种悲剧的问法，或者是中国式的一种问法，为什么呢？

如果这种问法被当作一种导向，换句话说，是告诉大家技术本身必须要有自己的工作场景，而不能是自己所领悟的场景（有些场景其实是通过交流见多识广的），那么大家就只会关注于工作中用到的技术，不会关心别的技术，而关注技术的深度通常也仅仅局限于工作本身。其实就做IT的人来讲，基于工作驱动的技术有场景印证，自然容易深入理解，非工作驱动的技术更多来自于兴趣和体会，来自于对技术的好奇心，做技术的人不能丢失这份好奇心，才真的可以深挖技术。

胖哥认为作为职业的技术人员，学习一些和自己目前工作不太相关的技术并没有什么坏处（当然前提是在工作中的技术本身掌握得不错的情况下，且在工作之余），深入挖掘技术本身也并没有什么坏处，知识都是相通的，思想上也是互补的，今天用不到的并不代表未来用不上，在真正需要的时候，它给予的就是灵感，而不是瞎碰。对于这个问题的问法，胖哥认为这样会更加合适一些：你认为这种技术可以用在什么场景下？它的价值体现在哪里？

胖哥崇尚研究技术，但并非“技术控”，现在我们就来说说什么是“技术控”。

有一位研究技术的同学，在他自己的脑海里大多数时候只想到了技术如何使用，甚至是技术中一些“变幻莫测的使用方法”，并以此为乐。其实研究技术本不为过，而且全面地了解使用方法更是一个专业技术人员所应具备的素质，不过“技术控”们通常会忽略技术本身的价值，或者本身不想去研究价值，因为会使用就已经足以让他们充满成就感。

“技术控”们对于原理的理解通常只是“听说”、“好像是”这样的印象，或者最多看看文档，因为文档已经成为他们思想的一个围栏，没有通过自己的验证和思考。刚使用某种新技术时可以这样，因为我们确实需要研究它如何使用，但是离真正做出一些深入判断还有很大的差距。

可能他们对自己的视野面和某些测试结果还非常自信，甚至已成为一种“信仰”，成为这方面技术的一个“信徒”。在这样的情况下是很难客观看待技术本身的，因为他们眼中只有自己的是最好的，自己的结论是绝对说明真理的。

还有的小伙伴会被自己所用到的技术搞得难以自拔，为啥呢？他们可能会对某种新技

术研究很久，不惜长时间地拖延项目，但思维一直钻在某个坑里无法自拔，又不愿意与人交流和梳理思路，这类人不能算是技术控，但这个过程却是大多数技术人员都会经历的。

有一位武者，十八般武艺样样都打得行云流水，口诀倒背如流，但很少去思考招式是用来做什么的，也更加搞不清楚武术的一些道理，或许这就是传说中的“功力不够”。

有人会说：和平年代，武术是用来养生或表演的，不用考虑实战。或许这有理，但问题也可以反过来看，如果武术是用来养生的，而你理解了每个动作的设计会引导身体的哪些部位，达到相应的运动效果，自己开始领悟动作的细节，加上别人的指导便会事半功倍，动作将更加到位。如果武术是用于表演的，则可以去思考一个动作如何做出来更加潇洒自如，如何显得更加有高手风范等，让观赏者更加赏心悦目。实战更加不用多说，在实战中花式动作一般会去掉绝大部分，直截了当的动作更加切合实际，那么在练习过程中自然会考虑动作的实战用途，在实战中形成更好的自然反应。这也许就是武侠片中提到的所谓“悟性”吧！

胖哥认为自己的工作也是一门艺术，注重一种学术和思想，武术表面上使用了外观形态表现出各种姿势，不过专业的习武者，拥有对武学本身的理解和思考，它甚至会引导人的心态，改变一些价值观，让人变得更加坚强等；而平庸武者通常是好勇斗狠之辈，缺乏这些思考。

人总有一些惰性，对于我们最为惰性的是思考，因为这种痛苦可能大于体力的付出，尤其是对于思考方式的改变。那么“技术控”的心态开始逐渐形成了，因为对技术本身的使用可以给我们带来一种不用过多思考的“进步”，这就是胖哥所理解的技术控的由来。

总的来讲，胖哥认为“技术控”和“我工作中用不上”算是两个极端，不要輕易地走向任何一个极端，要想办法找到平衡点。例如去研究一种新技术时，我们可以全身心地研究它（就技术而技术，无须考虑过多的场景），因为那样可以让我们更加投入地去学习和研究，但学习完后一定要思考，有没有什么地方用到过，它在什么场景下可以使用，然后想一想自己所了解的深度，是否已经足以支撑自己去做一些对比和选择，逐渐的便会开始掌握它的秉性，学会如何去驾驭它，而并非被它搞得难以自拔。

10.4.2 明确业务背景的例子

“明确业务”，这几个字说起来轻松，也许做起来还有一定的难度，因为这个问题从较为粗的问题到很细的细节，都会涉及许多思考方式。在本节，胖哥按照较粗的项目方式来简单说说。

(1) 小得不能再小的项目，也许就是开发几个网页，几个简单的增、删、改、查项目，就像我们在学校里做的课程实验一样，这样的系统通常不用考虑太多业务，甚至可以到互联网上下载一个来修改。胖哥认为只要你能读懂代码并且自己可以修改就 OK 了（当然，若要商用则需要遵循相关开源协议），这类项目基本不需要任何框架的支持，即使需要也可以按照某些“工具书”上的 step by step 逐步配置，很快可以将项目做完。

(2) 业务场景较为单一的项目，这类项目有了流程，有了许多工单处理，工单处理只是简单的“确定”、“退回”、“转派”等这类基本的操作，在个别的流程中会出现业务逻辑或数据处理。简单来看，有流程一般就会有业务，但是这类模板化的业务是可以被抽象的，

并且这种抽象的难度通常并不是那么大，模板经过抽象后，我们只需要配合模板编写一些代码就可以轻松完成业务的开发工作，例如常见的业务开发模板组件： workflow、自定义表单、自定义报表等都曾是业务开发过程中不断摸索和抽象出来的。

(3) 行业软件，这类软件针对一个行业，在多年的业务积累中，形成了一系列经验，这些经验可以被传承和迭代，在传承和迭代的过程中就形成了一种行业标准，这类软件的业务积累过程会变得复杂，由于在可传承的项目中对于业务领域需要非常熟悉，所以经验显得相对重要些，逐渐的行业中的业务会变成一种“解决方案”，系统会随着业务的复杂性而增强，如果系统没有基于业务背景的良好框架基础，那么在业务变化的过程中很快会变得臃肿不堪，维护代价增加很多。

这样的软件架构，通常希望有一位非常懂得业务发展方向，软件抽象能力也很不错的人来做，为何？在行业软件抽象的过程中，这些深知业务背景的人会发现同行业的软件有许多共同的特征，他们希望通过“配置”的方式来完成可变元素的抽象，达到这样的目的后，这款软件在为不同客户服务的过程中，几乎只需要修改配置就可以满足大部分需求，大量地降低了开发、部署等各方面的成本，这样的软件就逐步会形成产品，形成行业的产品。

有人会说，这样的动作可以交给技术和业务这两个不同角色的人来完成。确实是这样，但是不能过于隔行，隔行如隔山，尤其是过分注重于技术的人可能在考虑业务的全面性方面会有问题，与业务人员交流会有代沟和遗漏，一点不懂技术的业务人员也自然只会从自己的角度出发，这是一个十分矛盾的事实，两者要共同向中间靠近才能将事情做好，也就是技术人员多少要站在业务的角度去思考一些问题，而业务人员也多少要懂得一些抽象模型，这样双方交流才会有较强的心灵沟通，否则当大家都认为自己是权威时，被害的其实是客户方。

(4) 摸着石头过河的开发，也就是在业界还没有良好方案的产品，没有太多的思路，但是大家知道很有可能会用来解决未来的问题或已经面临的问题，由于业界没有思路或很好的思路，大家都在摸着石头过河，甚至提出需求的人也不知道到底要做什么样子。

这类软件通常没有所谓的规范基础，无法把控发展方向，可能会每天一个新想法，一些抽象模型可能很快被颠覆和打破。那么软件的抽象是否就无用了呢？

这类软件要么是老板没经验，要么是一种创新项目。前者就不多提了，在创新项目中一些基本技术我们还是可以沿用的，在发展过程中发现是一个类似于“测试汽车马力”的过程，可能会不断地去更换汽车的零部件，在这个过程中不断去改造，最终可能会发现这辆汽车被换得不伦不类。但是我们开始有点思路了，按照现在的思路开始重建汽车吧，新车开始变得比较漂亮，它开始适应许多变化，但是紧接着的就是巨大的变化，这类创新项目永远离不开的就是变化，所以几乎很快又需要去组装另一辆新车。反过来会发现汽车确实一代胜过一代，不在乎它必须有多好，但是它在引领一种潮流和思想。另外，在更新换代的过程中，同样会发现其中有许多经验和方法可以被传承，软件的抽象模型也会逐渐形成，只是它相对行业软件会变化得更加快速，可能快得让很多人难以接受。

(5) 调度平台，调度系统通常是基于时间点的一层抽象，它会提供达到时间点后需要回调的类，理论上这样的思路已经做得很清晰了，但是你会发现业务需求中还会存在许多复杂性：希望某些任务之间存在某些串行或并行关系、多个任务之间通信、任务优先级、任务调度失败重试、忽略、分布式调度之间的协调。许多个性化的调度模型以及和业务结

合十分密切的场景，是开源框架通常不会去做的事情，也许需要去做进一步的抽象。

(6) 中间件抽象，其实这是个比较明显的例子，中间件有很多，它们通常是自己直接写或基于开源框架来做扩展，有一个十分明显的例子就是 JDBC，若某个系统需要进行分布式数据库访问而又不希望业务系统关注分布式细节，这就需要抽象出中间件了。

传统的 JDBC 是针对指定的一个实例的，此时外层封装会基于原始的 JDBC 抽象出分布式逻辑计算，然后将相关 API 也设计成 JDBC 的规范接口，那么外部应用中 ibatis 这类框架就可以照常使用了。

反过来讲，如果不去做这一层抽象，每个应用都可能会创建大量的 DataSource，然后自己去编写许多算法来完成路由，也就是许多应用系统在重复制造轮子，虽然可控性更强，但是不利于整体发展和维护。

(7) 统计或监控，或许某些时候我们看到监控报表和数据是最多的，它们也通常是华丽的外表展示，但是我们不要被外表所迷惑，要做出合理的统计或监控才具有价值，而不仅仅是做出一个外表。

何为合理呢？就是我们做出来的统计信息或监控数据是用户看到后觉得有价值的（当然不是说对所有的用户都要有价值），也是很容易找到的。例如，要统计一个平台的访问情况，如果我们仅仅给一个注册用户的趋势图，则会存在许多问题，因为有可能用户访问过平台一两次就没有再上来，那么需要的内容到底是什么？用户的行为习惯。例如，用户多久会登录一次，有多少用户只来过网站一次就没有再来，来网站的用户会打开多少个页面，用户喜欢访问什么样的内容，访问量按天、按周、按月、按季度等方式同比增长多少，这样可以帮助我们了解到许多用户的趋势，为未来在产品、运营等方向提供数据参考。再拿监控来讲，如果仅仅是一些趋势图之类的，则可能在某些时候有用，不过很多时候是事后才知道，尤其是一些服务器监控，如果能有所“预测”或许可以避免很多问题。

或许你开始理解，许多系统不像增、删、改、查那么简单了，或者说不是那么无聊了，在业务越来越复杂的同时，需要我们付出更多的思考，利用技术和方法来解决问题。换句话说，业务变得越来越复杂是对能力的一种考验，所以不要烦躁，更不要“扔电脑”、“殴打老板”，当自己能判定的需求是“费力不讨好的”时，也可以去沟通。

10.5 胖哥对框架的浅析

➤➤ 10.5.1 框架由来的一个补充

“源码篇”导言中就提到了框架的由来，在这里我们再从不同层次的框架解决什么样的问题来说明框架的由来。

抛开“框架本身”这个词汇，将它理解为一种封装，高级语言可以简单地认为是对底层的封装，而高级语言的封装也只是语言级别的，通常语言所提供的功能还比较基础，而进一步的封装就需要中间件、框架来完成。

例如 Web 容器，通常会提供对用户的连接 Socket 通信、访问线程池、字符集处理、HTTP 协议、Cookie、文件上传等内容处理的封装，对于用户发起的 HTTP 服务请求，我们无须关心这些细节或者很少关心这些细节，即使使用纯 Servlet 去编写程序，也不用考虑线程是如何分配的，不用考虑 Socket 通信是如何进行的。

Web 容器可能对它的包装仅仅局限于基本的请求对象化，使得许多较为复杂的公共逻

辑变得简单，但是随着访问人数的增加需要一个系统提供大量用户的访问能力，随着业务复杂性的增强每个请求都会有许多业务逻辑处理，每种业务逻辑处理都有许多数据库访问、缓存访问、RMI 调用等，如果仅仅是用 Servlet 或 JSP 级别，那么这些业务、逻辑、页面渲染都会套在一起很难维护，模块很难共享，也更难以做到分工明确的编程。

于是产生了一些框架，这些框架将中间的许多跳转细节、注入细节、AOP 细节、连接池细节、事务管理细节等进行了包装，业务逻辑代码单纯地只有业务逻辑，而框架本身的代码也只处理公共的部分，这也许就是说的“干净”吧。

第三方框架包装成这样是否就够了，我们可以直接拿来用，还是包装没有底线，可以无穷无尽地包装下去？相信每个人都有自己的看法与见解，接下来胖哥将从不同的角度来阐述自己对这些问题的看法。

10.5.2 开源框架与扩展

1. 参考业界框架

前文中提到业务场景决定了系统框架的多种因素，并面临一些选择，而框架能否较为完善地支持系统开发也是一个话题，这里我们来了解一下框架封装的“度”的问题。

开源框架通常解决的是业界通用的一些模型，然后根据许多实际需求中无法满足的功能做一些升级版本（当然要定位这种功能是否需要该框架来提供）。因此，不可能让一个通用模型的框架来解决所有的业务场景的开发问题，很多有一定能力的公司开始自己研发框架，开发自己的组件，我们通常叫它“二方包”，而开源的 jar 包就叫“三方包”。

在企业级通用扩展的目的在于形成企业内部或行业内通用模型，它们相对开源软件来讲更适合于这个行业领域，而又可以降低维护成本。

不过，一个行业的通用模型也是无法满足所有个性化需求的，当它无法满足时，就需要本地扩展，本地扩展可能在框架级别，可能在业务代码级别。框架级别解决的依然是一些通用模型，虽然本地化，但依然是通用的，只是它更加偏重于业务层面。例如某项目在具体发布时，某些地区的客户要求有自己的策略配置方式、管理模式、流程等，此时可能不得不去做一些个性化，但是这些个性化可能也是可以再做一层抽象的，以至于不会编写硬代码来完成。

对于一个业务成熟的行业中的软件产品，如果产品做到了足够好，那么当它面对绝大部分常规需求的变化时是无须做太多开发的，甚至不需要开发工作。这些来源于大家对这个行业的深刻理解，也建立在行业中的业务有了稳定的发展趋势上。

反过来讲，一个框架的制作并非“闭门造车”式地去完成，而且还不能扩展，那么现场通常会有各种诡异的代码出现，例如可能会出现 if（某个地区）else if（另一个地区）这样的代码，这种代码是硬生生地将业务套在代码中，没有做任何逻辑模型的抽象，长期这样下去，这样的代码会有无限多的判定逻辑，且这些判定逻辑中还可能会多层嵌套，包含业务、计算等，时间久了就会导致没有任何人敢去修改这些代码，因为逻辑结构没有模型化，所以没人读得懂这个逻辑到底是做什么的，改了不清楚会有什么样的后果，逻辑和业务强制嵌套在一起，没有模型的概念自然不敢修改，更加不敢重构。

也就是说，逻辑模型的抽象应当使得业务变得简单清晰，将可变元素抽象为常见的模型（可能是数据、属性、算法、逻辑等），逻辑模型应当有类似的 API，它们有对外的统一接口，有规范的输入和输出。内在建立各种层次的抽象模型，最通用的自然在最上层，个

个性化的内容由继承者去扩展，哪一个抽象层次需要修改就修改哪一层。

2. 是否能扩展——设计框架的弊病

拿到一个框架后，无论是前端框架还是后端框架，始终会面对一些框架不好解决的问题，由于我们是在一个框架下做事情，因此考虑它如何扩展成为一个问题，这样的问题也分二方包和三方包。

三方包通常设计得较为灵活、轻便，因此容易扩展。也曾出现重量级框架，相信在项目中需要极高灵活性的都会放弃重量级开源框架。

二方包是企业在做过很多软件后发现许多业内的特征，提取出其中许多共享的元素，便于直接使用。不过有些企业级二方包抽象得实在是让人难以接受，也许表面上华丽多彩，实际使用上对个性化的业务需求扩展性非常差。也有一些企业级二方包说是高手设计的，但是代码是低手写的，那么效果可想而知。

此时此刻，可能你会十分好奇，什么样的框架让我们十分难以扩展呢？甚至就连写某些业务代码都不好写，胖哥来举几个例子给大家看一看。

(1) 某家公司自己扩展的前端 UI 框架，表面上十分牛气，什么控件都有，并且相应空间都提供了相应的“标签”来渲染处理，这样貌似没有问题。有一天大家发现一个控件是右侧点击一下出来一棵树用来选择，这棵树在加载页面时会加载所有的数据，当数据量较大时，点击右侧的图标时会“卡”很长时间才会弹出树，由于是标签直接将其渲染出来的，想要改为异步模式，也没有找到方法。悲剧的事情并没有结束，更多的问题来了，某界面上需要在客户端点击某按钮时，生成多个这样的控件，这是服务器端的标签，最后用尽各种手段得到的静态内容竟然发现生成 DOM 的 id 是一样的，此时大家傻眼了，还得自己去替换 id，否则问题就更大。

最后吐槽：“还不如自己写，简单、可控性还强。”更加痛苦的是可能老板要求强制使用这样的框架，这是最悲剧的事情。再反思一下这个框架：既然什么控件都提供，为什么相应的参数不稍微全面提供一些呢？既然都生成 JS 和页面元素，为何不提供相应的 JS 操作的 API，让系统更加灵活呢？

(2) 一个报表工具，也是一家公司写的，它也是拥有华丽的外表，拖拽几个控件系统就出来了，大家无须关心 SQL，没错，要的就是这样的效果。但并非所有的情况都不关心，当面对复杂的表结构设计，以及较大的数据量时就必须关注 SQL，关注执行计划，否则一个 SQL 就能将系统搞宕机。

果然问题发生了，某相关的系统先是变慢，然后是宕机，最后是天天宕机，要解决这个问题就必须需要扩展，最少要能够从某个角度写入 SQL 来处理这个问题，但是框架不提供，大家很无奈。那么要源码吧，看看是怎么处理的，“这是机密，因此源码也不提供”。也是同样的悲剧，老板可能会让你必须采用这个框架来完成项目开发。

(3) 一个框架中在做 Dispatch 时，要求每个不同的 URL 必须对应一个单独的 Java 类来处理，这样的方式其实对于分工明确、变化不多或业务逻辑并不是那么繁杂的情况是很不错的（例如，在网站中各个子系统拆分后，通常一个系统不会提供大量的请求路径），因为 Java 文件与资源 URI 是对应起来的，通过 URI 就可以十分简单地找到 Java 代码，系统无须什么配置成本，名称对应即可。

但它也并非适合所有的系统，某些系统也许有许多请求逻辑，注重的并不是类似于网站的高性能，而是繁杂的逻辑处理，两者有着重大的区别。前者可能会将系统水平垂直拆

分成多个应用，每个应用接受较为单一的业务服务，便于做性能测试和扩展，开发人员可以专心去处理某部分的业务逻辑，将它做得很深、很精通，技术上自然也可以投入大量的精力；而后者无论是什么样的业务可能都在一个系统中完成，它需要接受各种路径访问，此时开发人员会反反复复去创建这些类，使用大量的类来处理不同的 URL 请求，也许一个十分简单的增、删、改、查就要写很多个类来处理，更不用说一些大模块的各类 Ajax 请求、各个板块的数据提取、页面跳转等。

或许这个时候，你比谁都清楚这样的框架并不适合自的系统，但是如果你不这样用可能会被判“死刑”。

(4) 一个 DAO 框架，它有自己的数据库操作逻辑，因为它想将分布式一致性以及传统的数据库都在这个里面解决掉，它要求程序员在业务层写上 try catch finally 这样的代码，并要求捕获自定义异常后做处理，捕获 Exception 也做一样的处理，胖哥发现不论什么异常完全是一模一样的处理。

好奇心极强的胖哥就问：如果所有的 Exception 都做一样的处理，为何要设置多个捕获，还要设立自定义异常呢？难道非要像军队一样站得整整齐齐才叫规范吗？胖哥认为这比较“土”，不是啥规范，但是这种“土”可以拿去忽悠“某些客户”，因为这简直太整齐了。

框架应当让程序设计变得简单、便捷，让程序员不容易犯错误，程序员编写的尽量都是不一样的业务，而并非一模一样的外观（这种事情机器可以做），这是一种通过人为约束性的东西来让程序员去做一些和业务不相关的事情，即使程序员不犯任何错误，这个过程也会令人烦躁不堪，浪费大量的时间，程序员便养成习惯：拷贝、粘贴、改改！那么这样的程序员自然觉得自己的工作没有任何价值所在，任何人都可以做这样的事情，发现自己在用体力谋求生存。

至此，胖哥认为无论是设计框架还是实现框架的人都比较重要，不是在显摆什么，而是在解决程序员的各种问题，让他们从痛苦中解脱出来，也就是说，客户就是程序员。

自然的，在设计和实现框架的过程中，不仅仅要考虑外观多么完美，可以给客户一顿“忽悠”，而且要多结合实际场景，多与现场保持对话，多收集现场反馈的问题，通过不断抽象来完成框架的设计工作。若框架交于他人完成，也一定要让实现者明白这些道理，这样设计出来的内容才会真正具有含量。

3. 学习代价及投入产出比

说到框架时不断提到框架解决的人力性价比问题，其实也关乎整个软件的投入成本。框架也无时无刻不与成本挂钩，大部分做技术的人也无时无刻不都在解决和成本相关的问题。程序员编写的软件是为了提升客户人均性价比，而框架设计者自然是为了程序员。

框架设计人员也同样有投入，而且设计人员通常由一些公司的员工多年逐步成长起来或从外部引进高手。因此，通常设计人员的人力成本非普通人员可比，那么他们创造的财富到底有多少？财富和投入进行对比，就知道是否值得投入了。

一个框架设计出来后，需要开发人员去学习，而且提供的框架越重，学习的东西就会越多，遵循的规则也越多。程序员需要花时间去学习这个框架，虽然可以通过培训的方式来解决，但也同样需要相同的培训成本投入。

一些自定义的框架拿到实际的项目中，可能会有很多不适应。由于这些问题是框架产生的，自然在网上很难找到答案。此时我们需要编写大量的文档，可能又会有许多其他的成本，而如果不编写文档，大家自然会去咨询编写框架的人，当一个公司存在非常多的项

目时, 这些编写框架的人可能逐步会变成“客服人员”, 难道再招聘一些水平稍微偏低的人来专门做这样的事情吗?

如果一个公司在相同的成本下使用了更多的人来做事情, 自然需要降低单位人力的价格, 这样的公司通常不太尊重人才发展, 自然也不会过于注重人才的成长, 靠一种生产力的模式形成一种社会风气, 表面上是解决就业, 但也是一种行业悲剧。

做很多事情都会考虑成本问题, 成本并不是在公司省几张纸、几度电省出来的 (因为那是利用成本的幌子在精打细算, 可能捡到芝麻丢了西瓜), 而是靠大家一起将很明显的浪费成本的“毒瘤”拔出来逐步做到的。作为框架设计者, 在设计时是否会考虑一些学习和使用成本的问题, 培训是一种方式, FAQ 也可以解决许多问题, 但这个中间只是为了消除某些瓶颈, 而并非解放开发者。

不过话说回来, 我们不能因为成本问题而过于限制自己的脚步, 不能因为成本问题导致该做的事情不去做。在不同规模的公司中有不同数量的项目, 也有不同种类的项目, 在项目较少的公司对框架还处于摸索阶段, 不如将更多的自主权交给项目本身, 而抽象出来的东西应当是可扩展和轻量级的, 这些模式应当被程序员所熟悉、可传承。对于不同种类的项目, 确切地说是业务背景完全不同的项目, 应当有自己的扩展 (例如企业级管理系统与网站, 在框架应用上是有很多区别的, 不可生搬硬套)。

4. 胖哥对框架设计的理解

不要想将框架做得过于完美, 也就是不要想要解决所有的问题, 但是要做得够简单、够灵活。面对场景需要认知一些重点性的问题根源, 尽量用简单、统一的方法来控制以便于扩展, 过于重的包装往往让学习的内容更加多, 出现问题也不容易定位。

一个设计框架的人, 如果考虑到这个框架要用 10 年甚至时间更长, 那有点过分自信, 技术革命就是两年左右的事情, 所以不要想自己设计的框架会长盛不衰, 它和业务软件一样, 不可能永久, 那样的东西很快会跟不上潮流。

那么, 什么样的才叫合适呢? 胖哥认为话题又要回到业务场景上来, 如果你对业务领域熟悉, 自然会把握一些业务在未来很长时间内不会变化太多, 那么这个地方有必要抽象你应当很清楚。

框架本身就像软件一样, 它的开发模型并不是瀑布模型, 并非一触而就, 它也需要一个成熟的过程, 在考虑投入产出比时, 如果考虑到这样的一个抽象模型未来很多年都基本用不上 (注意: 并不是指现在用不上, 而是发现未来相对较长的时间都用不上), 且要抽象这个模型还十分麻烦, 那么自然不如不抽象。当真正条件成熟时, 抽象方式自然明了, 且更加切合实际。反过来一个极端是, 简单的一些代码可以让程序员从烦躁中解脱出来, 让程序员的每一条代码具有真正的业务价值, 那么这样的投入必然是值得的。

抽象方式有很多, 我们可以借鉴许多开源的容器或框架, 例如 Tomcat 在设计 JNDI 数据源时, 可以在文件中配置相关的数据源, 大家只需要按照模板化的要求配置好即可, 也可以不用 JNDI 数据源而使用自己的方式访问数据库, 选择有很多, 而且都很简单。另外, 如果遇到一个项目要求对密码进行加密, 并且需要提供一种自定义的加密算法, 那么只需要编写一个类来实现接口 `javax.naming.spi.ObjectFactory`, 然后将它配置在 `Resource` 的 `factory` 参数上, 系统在创建 JNDI 数据源时会使用这个类来完成。因为 Tomcat 在设计时考虑到了有这样的可变性存在, 不会一板一眼地要求必须怎么样去写配置和写代码。

一个老 A 级程序员，掌握框架设计的思想对于理解技术的原理是非常有帮助的，可以达到事半功倍的效果。你可能会在适当的时机提出对框架改善的建议，在这个过程中站在框架、业务等角度理解系统，这样也许你会成长得更加快速。

关于成本更细的问题，胖哥将在本书下册与大家进行更多的探讨。

►► 10.5.3 框架与解决问题

在实际的项目中选定了框架，就注定了你需要解决框架本身所带来的问题，在实际的项目中我们解决的问题通常可以分为技术型问题和业务型问题。

解决技术问题：如果是自己的业务代码，相信比较容易跟踪和定位，但是若出现在二方包、三方包中问题就会相对比较麻烦，因为很多种可能性都会出现，代码由别人来控制，可能框架本身有问题，可能在代码调用上有一些注意事项没有让程序员清楚地知道而导致问题，这时对源码的熟悉程度至关重要。也许我们可以通过一些工具来判定一些大概性的东西，或根据异常信息找到入口，但是熟悉源码者或善于阅读源码的人可以快速定位到问题的根本，想办法避免问题的再次发生，阅读源码者在实际项目中通常解决问题会更加彻底。

解决业务问题：如果框架无法支撑某些个性化业务，使得业务代码的编写有大量的可变性元素或通用性元素存在，此时适当地做二次扩展是有必要的，自己身处于这个项目中，应当很清楚它的业务动向，所以扩展自然不是什么难事。或许一个简单的扩展，不需要修改业务代码或修改少量的业务代码就可以搞定非常多的通用性问题，这就是为什么说高手写代码，几条代码就可以搞定一堆问题的道理，他们是在理解和思考的基础上得出代码修改的最佳位置，从框架的角度解决问题。

抽象的开发模型更像是先去考虑如何设计模型和组件，再去设计业务代码。在设计组件时，通常也会考虑这些东西应当是通用的、可被复用的、可被简单引用的。

框架也并非万能的，它不能解决所有的问题，至少对业务的细节它没法处理，所以不要想将所有的问题都抛给框架来处理。

框架封装了通用元素，当通用元素出现问题时，可能会引起大面积的问题，虽然一对全对，但也可能会一错全错。

“一对全对”将会减少大量的测试成本，在某些封装好共享逻辑的代码中，若一个较为全面的模块的测试用例通过，那么几乎相关的模块都能通过，因为它们的内部代码的路径是一样的，如果部分模块在某个地方出现问题，只能说测试用例的设计还存在一些遗漏和疏忽。比如我们设计一个数据交换中心，不需要对每个库、每个表去做测试，那样会做得没完没了，而只是针对不同字符集、不同数据类型、不同的权限划分、不同的分布式规则、不同的数据提取和写入方法去做一些测试。

“一错全错”正好看到一个反面，可能这个逻辑已经开始正式使用，但是由于测试用例上的疏忽，内在执行路径上的一个潜在 Bug 未在上线前被发现，那么在系统实际运行中，用户输入的某种参数，正好导致程序走入这个逻辑，就会全面出错，可能会算错数据，可能会走错流程，可能会死循环，可能会分配大量的内存，可能会把系统表锁住等，此时就真是一失足成千古恨了。业务代码的编写者不可能去阅读所有的源码，那么框架的编写者就承担更大的责任，千万不要认为系统出现问题和自己一点关系都没有。

本节我们说了很多的内容，总的来说，从软件本身的设计角度需要把握好可变性、通用性，这是抽象框架的基本要素，具体细节和抽象层次与业务场景相关。在本书下册的“设计篇”中将做更多的阐述，在“设计篇”中对模式不仅仅是“纸上谈兵”，而是用一些场景来说明一些问题，让我们切身体会代码设计带来的感受。

10.6 学海无涯，心境无限

在这一节中，简单谈谈心态，谈谈学习广度的心态，因为很多时候心态决定了我们的技术之路到底能走多远。

前文中提到：你知道的越多，就会发现自己不知道的越多”，因为只有当你知道的越多时，看到的视野面才会越宽，那么自然发现未知的东西就越多，只有坐在井底的青蛙才会觉得对这个世界已经什么都知道了。

要知道知识是无界的，时代也在不断变化，今天的理解可能在明天变成落后，许多世界级的大师也会经常说自己需要进一步来学习，何况是我们呢？要让自己学得更多，自己要有“菜鸟之心”，同时也要有“海纳百川之心”。只有菜鸟之心才能令自己不断成长，看清自我，认知别人，只有海纳百川之心才能让自己不能为小小的成就而自我满足。

反过来也不用悲伤于自己所知甚少，因为世界级的大师都在不断地学习和思考，我们还有什么好“悲催”的呢？沉下心去学习就会有自己的成长。

有人问胖哥：这么多的技术，应该从哪里下手呢？胖哥只能用通用的“浮躁”来解释，浮躁往往会令人失去自我。他们通常不是没有技术，而是不知道怎么深挖，或者想要一步深挖到底，找不到方向而迷茫，为何会找不到方向呢？因为心太大、太着急。

心太大是什么意思？它和“海纳百川”的心态是否有区别呢？当然有，它注重于什么都想学，为什么会出现这种情况呢？当一个自信的技术人员看到别人会的东西自己不会，发现自己如果不学就落后了，突然发现自己什么也不是，这种心态值得理解，也是上进心的表现。但是是否思考过别人的领域可能和你的不同，或者说别人对这方面的技术已经研究了很长时间，如果是很有功底的人，或许花不了多长时间就可以达到一个不错的水平。不过，这种情况通常发生在功底很薄的人身上，因为自身对技术的落脚点在使用的面上，所以看问题通常也是看到一个结果的华丽，从而容易迷失自我的发展方向。

“太着急”就很好解释了，自然是一种“想一夜成就自我的心态”，需要知道想要深挖一些东西，达到境界上的提升并非朝夕之功。虽然每个人的悟性不同，天分不同，但是没有人在一夜之间就可以从一个境界提升到另一个境界，除非他已经在这个“坎”上思考了很久，就差一点点，在不经意间的一个小灵感中找到了思路，或者在高人的点拨中茅塞顿开。

着急的同学通常在问技术问题都希望一步到位，同时又想知道原理和细节。要知道，我们不是学文学的，只要不是极为简单的问题，要原理和细节，必然会有一个繁杂的过程，也就是了解各种侧面和思考的过程。“画龙点睛”是在知道了这些细节后对外的一个简单说法，可以让不懂技术的人听懂一些，可以让没有接触过这种技术的人知道大概，但是绝非内在的机制和细节。换句话说，如果你认为自己是专业的，那么了解这些还远远不够。

学习也首先要放平心态，从当下的工作驱动做起，同时也要有计划地学习，看到太多

不知道的东西，也要有自己的学习计划。此时可以先碰一碰相关技术的软硬程度，学习计划自然就有了一些眉目，然后会逐步清晰，走上路了成长就会越来越快。按照计划去坚持学习和总结，每天你能学到一些小东西，一周你会有一个小成长，一月有个大成长，几个月也许会有所感悟和体会，一年半载也许就可能发生从量变到质变。

本书上册到此结束。在上册中，胖哥着重提到基础、源码以及一些学习方法；在下册中，我们将会探讨设计思路、项目实施中的各种“阴差阳错”、诡异的坑、分布式的基础知识，以及胖哥对于程序员之路的价值理解。